

Lemma Supplement: A Lean-Verified 51/50 Cutoff for Four-Arc Minimizers

Proof built, Lean-verified, and semantically accepted by
Agentic Lean Math Assistant · github.com/RosaRojacr/agentic-lean-math-assistant

Technical supplement and declaration guide

The supplement records the formal inventory behind the conventional proof. Non-generated declarations are listed exactly once; geometric definitions, analytic envelope results, carrier bridges, cap replacement, candidate exclusion, and the published root are classified as main, while lower-level measure, topology, interval-checker, and repetitive certificate infrastructure is classified as supplement. Each entry distinguishes its mathematical contract from source metadata. The two generated families are summarized separately because their leaves repeat one certificate schema.

The notation follows the main proof: λ is the exterior density, h_4 and h_3 are type-(iv) and type-(iii) curvatures, and `WeightedArea` and `WeightedPerimeter` are the coordinate integrals from `CMVGeometry.lean`. Generated compact cells provide rational slabs and signs for the scalar suffix; generated middle-face cells provide the slanted four-dimensional transition bricks. Their representatives identify records in the common schemas, not substitutes for the aggregate composition theorems. Compilation checks proof terms; semantic interpretation remains the model-and-source analysis in the main proof.

Conceptual declaration ledger

AdmissibleCompetitor

Classification: main

Source: `CMVGeometry.lean`:3952-3960

Declaration SHA-256: 52c3c4597213ca5cc7d4e12b21ad852ff148bcb8179b3f52a9c03b7d3665311b

Mathematical statement. The competitor class: empty, general integrable carrier, four-arc assembly, or replacement assembly.

A type-(iii) realization enters through the general constructor, so comparisons are not restricted to four-arc shapes.

```
inductive AdmissibleCompetitor (lam : ℝ) where
| empty
| general (region : FinitePerimeterRegion lam)
| fourArc (a : FourArcAssembly)
| replacement (a : ReplacementAssembly)

namespace AdmissibleCompetitor

/-- Coordinate carrier of an admissible regular competitor. -/
```

AdmissibleCompetitor.WeightedArea

Classification: main

Source: `CMVGeometry.lean`:3968-3975

Declaration SHA-256: d58b51a28690771d1b404e9048905bb6eae43f08f6798b27166b1234f3251f5e

Mathematical statement. Weighted area evaluated according to the competitor constructor.

The next carrier-identity theorem identifies every case with $A_\lambda(E)$.

```

def WeightedArea {lam : ℝ} : AdmissibleCompetitor lam → ℝ
| .empty => 0
| .general region => region.weightedArea
| .fourArc a => a.weightedArea lam
| .replacement a => a.weightedArea lam

/-- Weighted perimeter is canonically determined by the complete topological
frontier of the competitor's coordinate carrier. -/

```

AdmissibleCompetitor.WeightedPerimeter

Classification: main

Source: CMVGeometry.lean:3976-3980

Declaration SHA-256: 17b5c5914d38bc0479d502c48cde9715c303aa7070585ade87ba56d307321437

Mathematical statement. Canonical weighted complete-frontier perimeter of the competitor's carrier.

No constructor can choose its own cheaper boundary measure.

```

def WeightedPerimeter {lam : ℝ} (r : AdmissibleCompetitor lam) : ℝ :=
  _root_.WeightedPerimeter lam (FrontierMeasure r.carrier)

/-- The constructor-level weighted area is always the canonical density
integral over the constructor's coordinate carrier. -/

```

AdmissibleCompetitor.carrier

Classification: main

Source: CMVGeometry.lean:3961-3967

Declaration SHA-256: b57d4c4c33776a05ffbbbc22f1b363a7f356aa7cd089d7c2b3335ead1bb65644

Mathematical statement. The planar set carried by each competitor constructor.

The empty case gives $\emptyset$; every other case gives the corresponding geometric carrier.

```

def carrier {lam : ℝ} : AdmissibleCompetitor lam → Set PlanePoint
| .empty => ∅
| .general region => region.carrier
| .fourArc a => a.carrier
| .replacement a => a.carrier

/-- Weighted area on the regular constructor model. -/

```

AdmissibleCompetitor.weightedArea_eq_carrier

Classification: main

Source: CMVGeometry.lean:3981-3990

Declaration SHA-256: bf49ad5d7ddb4e090057dafec5dba684f6e36be017290e55451b4a9c6cfabce1

Mathematical statement. Constructor weighted area equals the density integral over its carrier.

This aligns the minimization equation with actual geometric equal area.

```

theorem weightedArea_eq_carrier {lam : ℝ} (r : AdmissibleCompetitor lam) :
  r.WeightedArea = _root_.WeightedArea lam r.carrier := by
  cases r with
  | empty =>
    simp [AdmissibleCompetitor.WeightedArea,
      AdmissibleCompetitor.carrier, _root_.WeightedArea]
  | general region => rfl

```

```
| fourArc a => rfl
| replacement a => rfl
```

CMVPublishedCutoff51_50.candidate_not_isWeightedPerimeterMinimizer_from_51_50

Classification: main

Source: CMVPublishedCutoff51_50.lean:22-39

Declaration SHA-256: fe8c3d4297822263e0764fc31d2cba27c4c202e21a8868a614e1c8b94789b37f

Mathematical statement. Every modeled four-arc candidate satisfying the formal CMV hypotheses is nonminimal for λ at least $51/50$.

For $\lambda \geq 51/50$, $0 < h \leq 1$, $0 < \alpha < \pi/2$, and $\lambda \cos \alpha = h$ with $\lambda > 1$, the proof supplies an equal-area strict improvement by the three density cases in the main text.

```
theorem candidate_not_isWeightedPerimeterMinimizer_from_51_50
  {lam : ℝ}
  (hlower : (51 / 50 : ℝ) ≤ lam)
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  by_cases hface : lam ≤ (33 / 32 : ℝ)
  · exact MiddleFaceCells.candidate_not_isWeightedPerimeterMinimizer_from_51_50
    hlower hface candidate hcandidate
  · by_cases hcompact : lam ≤ (9 / 7 : ℝ)
    · exact
      CompactCells1003To1023.bboxes0To1023_candidate_not_isWeightedPerimeterMinimizer
        (not_le.mp hface).le hcompact candidate hcandidate
    · have hsplice : (1.2581840884 : ℝ) < 9 / 7 := by norm_num
      exact cmv_type_four_not_minimizing
        (le_trans hsplice.le (not_le.mp hcompact).le) candidate hcandidate

end CMVPublishedCutoff51_50
```

CMVSuffixModel.FourArcCandidate.endpoint_weightedArea_eq_typeFourArea

Classification: main

Source: CMVSuffixModel.lean:159-166

Declaration SHA-256: 1d3c440482f713868366732790a87ef12add8058ecc71af2e6fb07df04bb3b83

Mathematical statement. The area identification also holds when $h=1$.

This is an endpoint formula, not an application of an interior differentiability theorem.

```
theorem endpoint_weightedArea_eq_typeFourArea
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  (_hendpoint : candidate.h = 1) :
  candidate.WeightedArea =
    LeanSuffixAnalytic.typeFourArea lam candidate.h :=
  weightedArea_eq_typeFourArea_all candidate hcandidate

/-- Exact type-(iv) perimeter identification at `h₄ = 1`. -/
```

CMVSuffixModel.FourArcCandidate.endpoint_weightedPerimeter_eq_typeFourPerimeter

Classification: main

Source: CMVSuffixModel.lean:167-176

Declaration SHA-256: 9a9144c9e0869ee21b16e09b6aad1d31570ea82a7d5d876263c80c006e5e5661

Mathematical statement. The perimeter identification holds at $h=1$.

The finite arc-length expression is valid even though some scalar derivatives are singular there.

```
theorem endpoint_weightedPerimeter_eq_typeFourPerimeter
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  (_hendpoint : candidate.h = 1) :
  candidate.WeightedPerimeter =
    LeanSuffixAnalytic.typeFourPerimeter lam candidate.h :=
  weightedPerimeter_eq_typeFourPerimeter_all candidate hcandidate

/-- Branch-complete formula identification. The proof deliberately splits
the regular curvature from `h₄ = 1`, so later arguments cannot silently apply
a regular fold hypothesis at the endpoint. -/
```

CMVSuffixModel.FourArcCandidate.not_isWeightedPerimeterMinimizer_of_analytic_typeThree

Classification: main

Source: CMVSuffixModel.lean:201-224

Declaration SHA-256: 99c923b45cbc09e9ba012e383efbf5a498fe0a194180526cbe54c89d50a0f151

Mathematical statement. A scalar equal-area type-(iii) strict improvement with $0 < s < 1$ disproves candidate minimality.

Realize the scalar by a measurable integrable carrier, identify both geometric objectives, and contradict the universal minimizing inequality.

```
theorem not_isWeightedPerimeterMinimizer_of_analytic_typeThree
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  {h₃ : ℝ} (hh₃ : 0 < h₃) (hh₃_one : h₃ < 1)
  (equalArea : LeanSuffixAnalytic.typeThreeArea lam h₃ =
    LeanSuffixAnalytic.typeFourArea lam candidate.h)
  (strictPerimeter : LeanSuffixAnalytic.typeThreePerimeter lam h₃ <
    LeanSuffixAnalytic.typeFourPerimeter lam candidate.h) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  let a : _root_.TypeThreeAssembly lam :=
    { h := h₃
      density_jump := hcandidate.density_jump
      h_pos := hh₃
      h_lt_one := hh₃_one }
  rcases a.hasModeledCompetitor with ⟨realization⟩
  apply candidate.not_isWeightedPerimeterMinimizer_of_typeThree a realization
  · rw [TypeThreeAssembly.scalarWeightedArea_eq_typeThreeArea a,
    weightedArea_eq_typeFourArea candidate hcandidate]
    exact equalArea
  · rw [TypeThreeAssembly.scalarWeightedPerimeter_eq_typeThreePerimeter a,
    weightedPerimeter_eq_typeFourPerimeter candidate hcandidate]
    exact strictPerimeter

/-- A single certified stationary descending pair excludes every modeled
type-(iv) candidate at the same density, including the closure endpoint. -/
```

CMVSuffixModel.FourArcCandidate.not_isWeightedPerimeterMinimizer_of_stationaryEqualAreaPair

Classification: main

Source: CMVSuffixModel.lean:225-248

Declaration SHA-256: d6670f69ae0dd1a579233b44583ff3b5ab90f13ac210ff1ddaabbb1088c5b5e2

Mathematical statement. One stationary pair with negative type-(iii) fold and negative perimeter gap excludes every candidate at that density.

The descending equal-area envelope supplies a strict improvement for each $0 < h \leq 1$; the scalar-to-carrier bridge then supplies an admissible competitor.


```

theorem not_isWeightedPerimeterMinimizer_of_stationaryEqualAreaPair
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  (pair : LeanSuffixAnalytic.StationaryEqualAreaPair lam)
  (hfoldThree :
    LeanSuffixAnalytic.typeThreeFold lam pair.h3 < 0)
  (hperimeter :
    LeanSuffixAnalytic.typeThreePerimeter lam pair.h3 <
    LeanSuffixAnalytic.typeFourPerimeter lam pair.h4) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  rcases pair.allCurvature_typeThreeImprovement
    with hcandidate.density_jump hfoldThree hperimeter
    with candidate.h_pos candidate.h_le_one with
    (h3, hh3, hh3_one, hequal, hstrict)
  exact not_isWeightedPerimeterMinimizer_of_analytic_typeThree
    candidate hcandidate hh3 hh3_one hequal hstrict

end FourArcCandidate

/-- Honest candidate-level consumer for the retained suffix. The two analytic
inputs are the explicit root-envelope existence theorem and the genuine
stationary fold-gap theorem; neither is an alias for candidate exclusion.
Given those separately proved inputs, the conclusion compares the candidate
with the canonical admissible type-(iii) region. -/

```

CMVSuffixModel.FourArcCandidate.regular_weightedArea_eq_typeFourArea

Classification: main

Source: CMVSuffixModel.lean:142-149

Declaration SHA-256: 48c010f5756d8d71b74c4c11f50c789619d79d3e32a79f62433bedf4290ea164

Mathematical statement. At $h < 1$ the candidate's actual weighted area is $A_4(h)$.

The CMV incidence equation and exact carrier area formula identify the scalar profile on the interior curvature domain.

```

theorem regular_weightedArea_eq_typeFourArea
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  (_hregular : candidate.h < 1) :
  candidate.WeightedArea =
    LeanSuffixAnalytic.typeFourArea lam candidate.h :=
  weightedArea_eq_typeFourArea_all candidate hcandidate

/-- Exact type-(iv) perimeter identification on the regular source domain. -/

```

CMVSuffixModel.FourArcCandidate.regular_weightedPerimeter_eq_typeFourPerimeter

Classification: main

Source: CMVSuffixModel.lean:150-158

Declaration SHA-256: 7c7abd9d4131a70966d7ee06757ae9000eb6c0370ad6826014e6939680589328

Mathematical statement. At $h < 1$ the candidate's actual perimeter is $P_4(h)$.

The four arc lengths and principal exterior angle give $4B_4(h)/h$.

```

theorem regular_weightedPerimeter_eq_typeFourPerimeter
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  (_hregular : candidate.h < 1) :
  candidate.WeightedPerimeter =
    LeanSuffixAnalytic.typeFourPerimeter lam candidate.h :=
  weightedPerimeter_eq_typeFourPerimeter_all candidate hcandidate

/-- Exact type-(iv) area identification at the source endpoint `h4 = 1`.
This is separate from the regular theorem and invokes no fold expression. -/

```

CMVSuffixModel.FourArcCandidate.weightedArea_eq_typeFourArea

Classification: main

Source: CMVSuffixModel.lean:177-186

Declaration SHA-256: e8473928799276d8ba026ded04a5483fef2171ee9679b0cfa48b5117289324b3

Mathematical statement. Every candidate satisfying the CMV hypotheses has weighted area $A_4(h)$.

The proof explicitly combines $h < 1$ and $h = 1$.

```
theorem weightedArea_eq_typeFourArea
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  candidate.WeightedArea =
    LeanSuffixAnalytic.typeFourArea lam candidate.h := by
  rcases lt_or_eq_of_le candidate.h_le_one with hregular | hendpoint
  · exact regular_weightedArea_eq_typeFourArea candidate hcandidate hregular
  · exact endpoint_weightedArea_eq_typeFourArea candidate hcandidate hendpoint

/-- Branch-complete perimeter identification, with the same explicit regular
versus endpoint split as the area theorem. -/
```

CMVSuffixModel.FourArcCandidate.weightedPerimeter_eq_typeFourPerimeter

Classification: main

Source: CMVSuffixModel.lean:187-200

Declaration SHA-256: c47649486a6e39478b2697659276a1423a248517e94449da31be1eb9b63be5ce

Mathematical statement. Every such candidate has canonical weighted perimeter $P_4(h)$.

The formula is branch-complete over $0 < h \leq 1$.

```
theorem weightedPerimeter_eq_typeFourPerimeter
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  candidate.WeightedPerimeter =
    LeanSuffixAnalytic.typeFourPerimeter lam candidate.h := by
  rcases lt_or_eq_of_le candidate.h_le_one with hregular | hendpoint
  · exact regular_weightedPerimeter_eq_typeFourPerimeter candidate
    hcandidate hregular
  · exact endpoint_weightedPerimeter_eq_typeFourPerimeter candidate
    hcandidate hendpoint

/-- A scalar type-(iii) equal-area and strict-perimeter witness becomes a
counterexample to minimality through the canonical finite-perimeter
realization of its coordinate carrier. The scalar `h_3` is required to lie in
the honest regular source domain. -/
```

CMVSuffixModel.TypeThreeAssembly.angleTerm_eq_typeThreeAngle

Classification: main

Source: CMVSuffixModel.lean:42-52

Declaration SHA-256: fe9ab761a6d7819237c18cc5b6d5b714cc699cb312b84007d27510fce813ca50

Mathematical statement. The geometric type-(iii) angle term equals the analytic angle term plus $\pi/2$.

$\lambda \arccos(q/\lambda) + \pi - \arccos q = B_3(h)$ by the principal inverse-trigonometric identity.

```
theorem angleTerm_eq_typeThreeAngle :
  a.angleTerm = LeanSuffixAnalytic.typeThreeAngle lam a.h +  $\pi / 2$  := by
  rw [_root_.TypeThreeAssembly.angleTerm,
    _root_.TypeThreeAssembly.outerAngle,
    _root_.TypeThreeAssembly.outerArgument,
    _root_.TypeThreeAssembly.innerAngle,
    LeanSuffixAnalytic.typeThreeAngle_add_halfPi]
```

```
rfl
```

```
-- The coordinate carrier's horizontal sine displacement is exactly the  
radical displacement in the retained analytic type-(iii) formula. -/
```

CMVSuffixModel.TypeThreeAssembly.scalarWeightedArea_eq_typeThreeArea

Classification: main

Source: CMVSuffixModel.lean:78-86

Declaration SHA-256: 09eff0e6af5f2cf4916a82b585443109aad4f3e8bb020859b5a5f6526f9c2f0d

Mathematical statement. The assembly scalar area equals the analytic type-(iii) area.

Substitute the angle and displacement identities into $(B_3 + (2h + 1)D_3)/h^2$.

```
theorem scalarWeightedArea_eq_typeThreeArea :  
  a.scalarWeightedArea = LeanSuffixAnalytic.typeThreeArea lam a.h := by  
  rw [_root_.TypeThreeAssembly.scalarWeightedArea,  
    LeanSuffixAnalytic.typeThreeArea,  
    angleTerm_eq_typeThreeAngle a, sineGap_eq_typeThreeDelta a]  
  rfl  
  
-- Exact identification of CMV equation (26)'s perimeter with the analytic  
quantity used by the suffix theorem, on the full regular type-(iii) domain. -/
```

CMVSuffixModel.TypeThreeAssembly.scalarWeightedPerimeter_eq_typeThreePerimeter

Classification: main

Source: CMVSuffixModel.lean:87-99

Declaration SHA-256: 1cbaa2764b23e5bf11163fb498e9653b454e16f1f8f6b84d752362e7a842718b

Mathematical statement. The assembly scalar perimeter equals the analytic type-(iii) perimeter.

The expression becomes $2(B_3 + D_3)/h$.

```
theorem scalarWeightedPerimeter_eq_typeThreePerimeter :  
  a.scalarWeightedPerimeter =  
    LeanSuffixAnalytic.typeThreePerimeter lam a.h := by  
  rw [_root_.TypeThreeAssembly.scalarWeightedPerimeter,  
    LeanSuffixAnalytic.typeThreePerimeter,  
    angleTerm_eq_typeThreeAngle a, sineGap_eq_typeThreeDelta a]  
  
end TypeThreeAssembly  
  
namespace FourArcCandidate  
  
variable {lam : ℝ} (candidate : _root_.FourArcCandidate lam)
```

CMVSuffixModel.TypeThreeAssembly.sineGap_eq_typeThreeDelta

Classification: main

Source: CMVSuffixModel.lean:53-77

Declaration SHA-256: 0b54dd8dd926b7670949326ff7f7fb7d21250d76068f1da2eb207447216f1aac

Mathematical statement. The geometric sine displacement equals D_3 .

Principal arccosines have nonnegative sine; their radical expressions give $D_3(h)$.

```
theorem sineGap_eq_typeThreeDelta :  
  a.sineGap = LeanSuffixAnalytic.typeThreeDelta lam a.h := by  
  have hlam_pos : 0 < lam :=  
    lt_trans (by norm_num : (0 : ℝ) < 1) a.density_jump
```

```

have hshape := a.shapeParameter_mem
have hminus : 0 < lam - a.shapeParameter := by
  linarith [hshape.2, a.density_jump]
have hplus : 0 < lam + a.shapeParameter := by
  linarith [hshape.1, a.density_jump]
have hrad : 0 ≤ lam ^ 2 - a.shapeParameter ^ 2 := by
  nlinarith [mul_pos hminus hplus]
rw [_root_.TypeThreeAssembly.sineGap,
    _root_.TypeThreeAssembly.outerAngle,
    _root_.TypeThreeAssembly.innerAngle, Real.sin_arccos,
    Real.sin_arccos, _root_.TypeThreeAssembly.outerArgument,
    LeanSuffixAnalytic.typeThreeDelta]
change sqrt (1 - (a.shapeParameter / lam) ^ 2) -
  sqrt (1 - a.shapeParameter ^ 2) =
  sqrt (lam ^ 2 - a.shapeParameter ^ 2) / lam -
  sqrt (1 - a.shapeParameter ^ 2)
rw [sqrt_one_sub_div_sq hlam_pos hrad]

/-- Exact identification of CMV equation (26)'s area with the analytic
quantity used by the suffix theorem. Its domain is the full regular
`0 < h₃ < 1` type-(iii) domain, across all three arc branches. -/

```

CompactCellAssembly.Conditions

Classification: main

Source: CompactCellAssembly.lean:25-62

Declaration SHA-256: 5c3b2aa26c0b8b3d8801158e0abfe33b06aca94b7f9fdd276d46c23ec0373f44

Mathematical statement. Primitive branch, fold-face, equal-area-face, and perimeter-sign obligations for a compact cell.

For each density in the cell, the record brackets a regular type-(iv) fold, a descending type-(iii) root above 1/2, and a strict perimeter improvement.

```

structure Conditions (lambdaInterval h3Interval h4Interval : QInterval) : Prop where
  lambda_one_lt : ∀ {lam : ℝ},
    lambdaInterval.RealContains lam → 1 < lam
  h3_slab_regular : ∀ {h : ℝ},
    h3Interval.RealContains h → h ∈ Ioo (0 : ℝ) 1
  h3_slab_above_half : ∀ {h : ℝ},
    h3Interval.RealContains h → 1 / 2 < h
  h4_slab_regular : ∀ {h : ℝ},
    h4Interval.RealContains h → h ∈ Ioo (0 : ℝ) 1
  typeFourFold_lowerFace_neg : ∀ {lam : ℝ},
    lambdaInterval.RealContains lam →
      LeanSuffixAnalytic.typeFourFold lam (h4Interval.lo : ℝ) < 0
  typeFourFold_upperFace_pos : ∀ {lam : ℝ},
    lambdaInterval.RealContains lam →
      0 < LeanSuffixAnalytic.typeFourFold lam (h4Interval.hi : ℝ)
  typeThreeFold_neg_on_slab : ∀ {lam h : ℝ},
    lambdaInterval.RealContains lam → h3Interval.RealContains h →
      LeanSuffixAnalytic.typeThreeFold lam h < 0
  typeThreeArea_lowerFace_gap_pos : ∀ {lam h₄ : ℝ},
    lambdaInterval.RealContains lam → h4Interval.RealContains h₄ →
      LeanSuffixAnalytic.typeFourFold lam h₄ = 0 →
      0 < LeanSuffixAnalytic.typeThreeArea lam (h3Interval.lo : ℝ) -
        LeanSuffixAnalytic.typeFourArea lam h₄
  typeThreeArea_upperFace_gap_neg : ∀ {lam h₄ : ℝ},
    lambdaInterval.RealContains lam → h4Interval.RealContains h₄ →
      LeanSuffixAnalytic.typeFourFold lam h₄ = 0 →
      LeanSuffixAnalytic.typeThreeArea lam (h3Interval.hi : ℝ) -
        LeanSuffixAnalytic.typeFourArea lam h₄ < 0
  typeThreePerimeter_gap_neg : ∀ {lam h₃ h₄ : ℝ},
    lambdaInterval.RealContains lam → h3Interval.RealContains h₃ →
      h4Interval.RealContains h₄ →
      LeanSuffixAnalytic.typeThreePerimeter lam h₃ -
        LeanSuffixAnalytic.typeFourPerimeter lam h₄ < 0

```

namespace Conditions

```
variable {lambdaInterval h3Interval h4Interval : QInterval}
```

CompactCellAssembly.Conditions.candidate_not_isWeightedPerimeterMinimizer

Classification: main

Source: CompactCellAssembly.lean:312-338

Declaration SHA-256: 6281e20f4e6b992e95255e50221808300ba4ec7b1c545ab00d1868e256be0776

Mathematical statement. A valid compact cell excludes every modeled candidate at each enclosed density.

The stationary witness is promoted through the all-curvature envelope and geometric realization.

```
theorem candidate_not_isWeightedPerimeterMinimizer
  (conditions : Conditions lambdaInterval h3Interval h4Interval)
  {lam : ℝ} (hlam : lambdaInterval.RealContains lam)
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  rcases conditions.stationaryEqualArea_improvement_exists hlam with
    ⟨h₄, h₃, hh₄, hstationary, hh₃, hequal, hperimeter⟩
  have hh₃reg := conditions.h3_slab_regular hh₃
  have hh₄reg := conditions.h4_slab_regular hh₄
  let pair : LeanSuffixAnalytic.StationaryEqualAreaPair lam :=
    { h₃ := h₃
      h₄ := h₄
      h₃_gt_half := conditions.h3_slab_above_half hh₃
      h₃_lt_one := hh₃reg.2
      h₄_pos := hh₄reg.1
      h₄_lt_one := hh₄reg.2
      equalArea := hequal
      stationary := hstationary }
  apply
    CMVSuffixModel.FourArcCandidate.not_isWeightedPerimeterMinimizer_of_stationaryEqualAreaPair
      candidate hcandidate pair
  · exact conditions.typeThreeFold_neg_on_slab hlam hh₃
  · exact hperimeter

end Conditions
end CompactCellAssembly
```

CompactCellAssembly.Conditions.stationaryEqualArea_improvement_exists

Classification: main

Source: CompactCellAssembly.lean:244-264

Declaration SHA-256: e63a75b5ca3c02a2700397ede0d62a135c22960b730b6191885ca6432b43a9fe

Mathematical statement. Compact-cell conditions produce a stationary equal-area strict improvement.

The resulting regular pair has $K_3 < 0$ and $P_3 < P_4$, exactly the additional envelope hypotheses.

```
theorem stationaryEqualArea_improvement_exists
  (conditions : Conditions lambdaInterval h3Interval h4Interval)
  {lam : ℝ} (hlam : lambdaInterval.RealContains lam) :
  ∃ h₄ h₃ : ℝ,
    h4Interval.RealContains h₄ ∧
    LeanSuffixAnalytic.typeFourFold lam h₄ = 0 ∧
    h3Interval.RealContains h₃ ∧
    LeanSuffixAnalytic.typeThreeArea lam h₃ =
      LeanSuffixAnalytic.typeFourArea lam h₄ ∧
    LeanSuffixAnalytic.typeThreePerimeter lam h₃ <
      LeanSuffixAnalytic.typeFourPerimeter lam h₄ := by
  rcases conditions.typeFourFold_existsUnique_in_slab hlam with
    ⟨h₄, ⟨hh₄, hfold⟩, _⟩
  rcases conditions.typeThreeEqualArea_existsUnique_in_slab
```

```

    hlam hh4 hfold with ⟨h3, ⟨hh3, hequal⟩, _⟩
  refine ⟨h4, h3, hh4, hfold, hh3, hequal, ?_⟩
  linarith [conditions.typeThreePerimeter_gap_neg hlam hh3 hh4]

/-- Global type-(iv) fold uniqueness identifies any regular stationary root
with the root isolated by this cell; slab membership need not be assumed for
the arbitrary root. -/

```

CompactCellAssembly.Conditions.typeFourFold_existsUnique_in_slab

Classification: main

Source: CompactCellAssembly.lean:76-106

Declaration SHA-256: c2359e743fc358937d19c1ac3887ef019ddb92c97b2b12e00651546f5b742979

Mathematical statement. A compact certificate brackets a unique stationary type-(iv) curvature.

Continuity and opposite face signs give existence; strict fold monotonicity gives uniqueness.

```

theorem typeFourFold_existsUnique_in_slab
  (conditions : Conditions lambdaInterval h3Interval h4Interval)
  {lam : ℝ} (hlam : lambdaInterval.RealContains lam) :
  ∃! h4 : ℝ, h4Interval.RealContains h4 ∧
    LeanSuffixAnalytic.typeFourFold lam h4 = 0 := by
  let a : ℝ := h4Interval.lo
  let b : ℝ := h4Interval.hi
  have hab : a ≤ b := by
    simpa [a, b] using (show (h4Interval.lo : ℝ) ≤ h4Interval.hi by
      exact_mod_cast h4Interval.ordered)
  have hcont : ContinuousOn (LeanSuffixAnalytic.typeFourFold lam) (Icc a b) := by
    intro h hh
    have hhSlab : h4Interval.RealContains h := by
      simpa [a, b, LeanSuffixReflective.QInterval.RealContains, Set.mem_Icc] using hh
    have hhReg := conditions.h4_slab_regular hhSlab
    exact (LeanSuffixAnalytic.hasDerivAt_typeFourFold
      (conditions.lambda_one_lt hlam) hhReg.1 hhReg.2).continuousAt.continuousWithinAt
  rcases ScalarSuffixCertificate.oppositeFace_zero hab hcont
    (conditions.typeFourFold_lowerFace_neg hlam).le
    (conditions.typeFourFold_upperFace_pos hlam).le with ⟨h4, hh4, hzero⟩
  have hhSlab : h4Interval.RealContains h4 := by
    simpa [a, b, LeanSuffixReflective.QInterval.RealContains, Set.mem_Icc] using hh4
  refine ⟨h4, ⟨hhSlab, hzero⟩, ?_⟩
  intro y hy
  exact LeanSuffixAnalytic.typeFourFold_zero_unique
    (conditions.lambda_one_lt hlam)
    (conditions.h4_slab_regular hy.1)
    (conditions.h4_slab_regular hhSlab) hy.2 hzero

/-- A stationary type-(iv) root in the cell has exactly one equal-area
height on the descending type-(iii) slab. -/

```

CompactCellAssembly.Conditions.typeThreeEqualArea_existsUnique_in_slab

Classification: main

Source: CompactCellAssembly.lean:107-177

Declaration SHA-256: 79c92a7e4df192f65dc043cccd8bea4a018e8826ba49c09591fd046f371a56f7

Mathematical statement. At the stationary type-(iv) area, the type-(iii) slab contains a unique equal-area root.

The two area-face signs give existence and the negative fold throughout the slab gives strict decrease and uniqueness.

```

theorem typeThreeEqualArea_existsUnique_in_slab
  (conditions : Conditions lambdaInterval h3Interval h4Interval)
  {lam h4 : ℝ}
  (hlam : lambdaInterval.RealContains lam)
  (hh4 : h4Interval.RealContains h4)

```

```

(hstationary : LeanSuffixAnalytic.typeFourFold lam h4 = 0) :
  ∃! h3 : ℝ, h3Interval.RealContains h3 ∧
    LeanSuffixAnalytic.typeThreeArea lam h3 =
      LeanSuffixAnalytic.typeFourArea lam h4 := by
let a : ℝ := h3Interval.lo
let b : ℝ := h3Interval.hi
let f : ℝ → ℝ := fun y =>
  LeanSuffixAnalytic.typeFourArea lam h4 -
    LeanSuffixAnalytic.typeThreeArea lam y
have hab : a ≤ b := by
  simpa [a, b] using (show (h3Interval.lo : ℝ) ≤ h3Interval.hi by
    exact_mod_cast h3Interval.ordered)
have hcont : ContinuousOn f (Icc a b) := by
  intro y hy
  have hySlab : h3Interval.RealContains y := by
    simpa [a, b, LeanSuffixReflective.QInterval.RealContains, Set.mem_Icc]
    using hy
  have hyReg := conditions.h3_slab_regular hySlab
  exact (continuousAt_const.sub
    (LeanSuffixAnalytic.hasDerivAt_typeThreeArea
      (conditions.lambda_one_lt hlam) hyReg.1 hyReg.2).continuousWithinAt)
have hfa : f a ≤ 0 := by
  dsimp [f, a]
  linarith [conditions.typeThreeArea_lowerFace_gap_pos
    hlam hh4 hstationary]
have hfb : 0 ≤ f b := by
  dsimp [f, b]
  linarith [conditions.typeThreeArea_upperFace_gap_neg
    hlam hh4 hstationary]
rcases ScalarSuffixCertificate.oppositeFace_zero hab hcont hfa hfb with
  ⟨h3, hh3, hzero⟩
have hhSlab : h3Interval.RealContains h3 := by
  simpa [a, b, LeanSuffixReflective.QInterval.RealContains, Set.mem_Icc]
  using hh3
have hequal :
  LeanSuffixAnalytic.typeThreeArea lam h3 =
    LeanSuffixAnalytic.typeFourArea lam h4 := by
  dsimp [f] at hzero
  linarith
have hderiv : ∀ y ∈ Ioo a b,
  deriv (LeanSuffixAnalytic.typeThreeArea lam) y < 0 := by
  intro y hy
  have hySlab : h3Interval.RealContains y := by
    simpa [a, b, LeanSuffixReflective.QInterval.RealContains, Set.mem_Ioo]
    using ⟨hy.1.le, hy.2.le⟩
  have hyReg := conditions.h3_slab_regular hySlab
  rw [(LeanSuffixAnalytic.hasDerivAt_typeThreeArea
    (conditions.lambda_one_lt hlam) hyReg.1 hyReg.2).deriv]
  exact div_neg_of_neg_of_pos
    (conditions.typeThreeFold_neg_on_slab hlam hySlab)
    (pow_pos hyReg.1 3)
have hanti := LeanSuffixAnalytic.typeThreeArea_strictAntiOn_Icc
  (conditions.lambda_one_lt hlam)
  (conditions.h3_slab_regular (lower_mem h3Interval)).1
  (conditions.h3_slab_regular (upper_mem h3Interval)).2 hderiv
refine ⟨h3, ⟨hhSlab, hequal⟩, ?_⟩
intro y hy
apply hanti.injOn
· simpa [a, b, LeanSuffixReflective.QInterval.RealContains, Set.mem_Icc]
  using hy.1
· exact hh3
· exact hy.2.trans hequal.symm

```

-- The slab-isolated negative-fold root is the global descending root of its equal-area level. Every root in $0 < h \leq 1$ lies at or above it, and there is at most one distinct upper root. -/

CompactFirstCell.compactCandidate_not_isWeightedPerimeterMinimizer

Classification: main

Source: CompactFirstCell.lean:1749-1757

Declaration SHA-256: 0cf4dd7a93e6150799254c4879293eeac9af163fdbd193bd279b77559f43d237

Mathematical statement. Every CMV-model candidate is nonminimal for densities in compact box zero.

The box certificates instantiate the generic compact assembly and then the all-curvature competitor theorem.

```
theorem compactCandidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ}
  (hlam : lambdaInterval.RealContains lam)
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer :=
  checkedConditions.candidate_not_isWeightedPerimeterMinimizer
    hlam candidate hcandidate
end CompactFirstCell
```

CompactSecondCell.compactCandidate_not_isWeightedPerimeterMinimizer

Classification: main

Source: CompactSecondCell.lean:672-681

Declaration SHA-256: bf9eb3ed65e181f0e0de3991de3f82fd1169d1b5acecfb53b060a331f19ad1ae

Mathematical statement. Candidates at densities in compact box one are nonminimal.

The checked conditions produce an actual equal-area strict competitor through the generic assembly.

```
theorem compactCandidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ} (hlam : lambdaInterval.RealContains lam)
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer :=
  checkedConditions.candidate_not_isWeightedPerimeterMinimizer
    hlam candidate hcandidate

/-- Candidate-facing exclusion on the closed union of zero-based boxes 0 and 1
across their exact seam. -/
```

CompactSecondCell.firstTwoBoxes_candidate_not_isWeightedPerimeterMinimizer

Classification: main

Source: CompactSecondCell.lean:682-702

Declaration SHA-256: 1f54334812872c86ebe3794a932ab059b0b9f0ae3518130a266bed3d7f94e321

Mathematical statement. The first two compact boxes exclude candidates on $[33/32, 118329/114688]$.

A case split at 236601/229376 proves exact closed-interval coverage.

```
theorem firstTwoBoxes_candidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ}
  (hlower : (33 / 32 : ℝ) ≤ lam) (hupper : lam ≤ (118329 / 114688 : ℝ))
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  by_cases hseam : lam ≤ (236601/229376 : ℝ)
  · have hbox : CompactFirstCell.lambdaInterval.RealContains lam := by
      norm_num [CompactFirstCell.lambdaInterval,
        LeanSuffixReflective.QInterval.RealContains] at hlower hseam ⊢
      exact ⟨hlower, hseam⟩
    exact CompactFirstCell.compactCandidate_not_isWeightedPerimeterMinimizer
```



```

    hbox candidate hcandidate
  · have hbox : lambdaInterval.RealContains lam := by
    norm_num [lambdaInterval, LeanSuffixReflective.QInterval.RealContains]
    at hseam hupper ⊢
    exact ⟨hseam.le, hupper⟩
  exact compactCandidate_not_isWeightedPerimeterMinimizer
    hbox candidate hcandidate

end CompactSecondCell

```

EuclideanPlane

Classification: main

Source: CMVGeometry.lean:34-37

Declaration SHA-256: dfa2db26da624a2b5333b886c ff8c9cd3151218ea8aa3e3cb39c78359b32470e

Mathematical statement. The coordinate product with its Euclidean L2 metric.

Hausdorff length is formed using $\| (x, y) \|_2$, not the product sup metric.

```

abbrev EuclideanPlane := WithLp 2 (ℝ × ℝ)

/-- The coordinate-preserving homeomorphism from the existing coordinate plane
to its Euclidean L2 realization. -/

```

F

Classification: main

Source: Minimiser.lean:43-45

Declaration SHA-256: 5b2bb966fff4872f29347307b668a0e2d316ac47616a7d86bd1b9925aa189c42

Mathematical statement. The scalar equation determining the minimum of the cap-saving function.

$$F(\theta) = 3\theta - 3\sin\theta\cos\theta - \pi.$$

```

noncomputable def F (θ : ℝ) : ℝ := 3 * θ - 3 * (sin θ * cos θ) - π

/-- `F' θ = 6 sin²θ`, using `3 - 3(cos²θ - sin²θ) = 6 sin²θ`. -/

```

F_strictMonoOn

Classification: main

Source: Minimiser.lean:62-71

Declaration SHA-256: 159fcb80f1fa4e926941eda01dd71a676dd66fef614b9045e53e04469f3d4efe

Mathematical statement. The function F is strictly increasing on the full closed interval $[0, \pi]$.

For $0 \leq a < b \leq \pi$, $F(a) < F(b)$. The function is continuous on $[0, \pi]$ and has derivative $6\sin^2\theta > 0$ throughout $(0, \pi)$. Thus the monotonicity domain is the full closed interval, not merely $[0, \pi/2]$ used to bracket the selected root.

```

lemma F_strictMonoOn : StrictMonoOn F (Icc 0 π) := by
  apply strictMonoOn_of_hasDerivWithinAt_pos (convex_Icc 0 π)
  · exact F_continuous.continuousOn
  · intro x _
    exact (hasDerivAt_F x).hasDerivWithinAt
  · intro x hx
    rw [interior_Icc] at hx
    have := sin_pos_of_pos_of_lt_pi hx.1 hx.2
    positivity

```

FinitePerimeterRegion

Classification: main

Source: CMVGeometry.lean:74-82

Declaration SHA-256: 2efb7176d17b946809f4cd07b78237ddd03cd4197c405c38d08e97342106a1a0

Mathematical statement. A measurable carrier with integrable weighted area and weighted complete-frontier measure.

The record requires both integrability statements. It does not identify this class with all measure-theoretic finite-perimeter sets.

```
structure FinitePerimeterRegion (lam : ℝ) where
  carrier : Set PlanePoint
  measurable_carrier : MeasurableSet carrier
  finite_weighted_area : IntegrableOn (StripDensity lam) carrier
  finite_weighted_perimeter :
    Integrable (StripDensity lam) (FrontierMeasure carrier)

namespace FinitePerimeterRegion
```

FinitePerimeterRegion.weightedArea

Classification: main

Source: CMVGeometry.lean:83-85

Declaration SHA-256: 3892310b68bd795c4ddff8f1d276f58be0efcb96c1e8c4fad96ec7d73deea20a

Mathematical statement. Weighted area of the record's carrier.

The value is $A_\lambda(E)$, not independent area data.

```
def weightedArea {lam : ℝ} (r : FinitePerimeterRegion lam) : ℝ :=
  WeightedArea lam r.carrier
```

FourArcAssembly

Classification: main

Source: CMVGeometry.lean:2491-2498

Declaration SHA-256: 1ac9185bddaf1e6a0074fb39d1e29417262cc431ccce861d3c65ec2e5216fdb1

Mathematical statement. A strip core with congruent reflected exterior caps of angle in $(0, \pi/2)$.

The caps share the core's horizontal chord and are based at the two strip interfaces.

```
structure FourArcAssembly where
  core : StripCore
  outerAngle : ℝ
  outerAngle_pos : 0 < outerAngle
  outerAngle_lt_pi_div_two : outerAngle < π / 2

namespace FourArcAssembly
```

FourArcAssembly.boundaryTrace

Classification: main

Source: CMVGeometry.lean:2625-2630

Declaration SHA-256: d8944f364e38eaa5c7b77ce182d417bec91ac425efc229739d886be00ad658c6

Mathematical statement. Union of the two side traces and the two exterior cap traces.

This is the proposed complete frontier, with only finite junction overlaps.

```
def boundaryTrace (a : FourArcAssembly) : Set PlanePoint :=
  StripCore.leftArcTrace a.core ∪
  (StripCore.rightArcTrace a.core ∪
   (OneSidedCircularCap.arcTrace a.upperCap ∪
    OneSidedCircularCap.arcTrace a.lowerCap))
```

FourArcAssembly.carrier

Classification: main

Source: CMVGeometry.lean:2519-2524

Declaration SHA-256: a55616796db22264c4fe11d48e20ab173ecb661c3bdced26752617ea90d1f816

Mathematical statement. Union of the core and the two exterior caps.

This set, rather than its boundary parametrizations, determines admissible perimeter.

```
def carrier (a : FourArcAssembly) : Set PlanePoint :=
  a.core.carrier ∪ a.upperCap.carrier ∪ a.lowerCap.carrier

/-- Explicit arc-length pushforward measure of the complete regular boundary.
The attachment chords are omitted because they are internal; endpoint atoms
are null for every summand. -/
```

FourArcAssembly.frontier_carrier

Classification: main

Source: CMVGeometry.lean:3252-3259

Declaration SHA-256: 7da383d241cb3cc30fc9da784ef7200d9bb87d82e25006745acf6f7f7d86a78f

Mathematical statement. The complete frontier is exactly the four-arc trace union.

The two inclusions rule out both omitted boundary and spurious internal chords.

```
theorem frontier_carrier (a : FourArcAssembly) :
  frontier a.carrier = a.boundaryTrace :=
  Set.Subset.antisymm a.frontier_carrier_subset_boundaryTrace
    a.boundaryTrace_subset_frontier_carrier

/-- All four boundary arcs have the same radius when the exterior chord has the
source incidence length  $2 * \sin \alpha / h$ . -/
```

FourArcAssembly.lowerCap

Classification: main

Source: CMVGeometry.lean:2509-2518

Declaration SHA-256: a60a9393b363872f08f12dbab41380b51afefa746b9504c5841f28d389091bfe

Mathematical statement. The reflected lower exterior cap.

It is congruent to the upper cap and based at $y = -1$.

```
def lowerCap (a : FourArcAssembly) : OneSidedCircularCap where
  chord := a.core.chord
  theta := a.outerAngle
  midpointX := 0
  baseY := -1
  side := .lower
  chord_pos := a.core.chord_pos
```

```
theta_pos := a.outerAngle_pos
theta_lt_pi := lt_trans a.outerAngle_lt_pi_div_two (by linarith [Real.pi_pos])
```

FourArcAssembly.upperCap

Classification: main

Source: CMVGeometry.lean:2499-2508

Declaration SHA-256: 3f084f0dbec6bee8992e556fe2ca304aae5eb8981815636bf15454a3b8e613f9

Mathematical statement. The upper exterior cap of a four-arc assembly.

It is based at $y = 1$ with the core chord and the assembly's exterior half-angle.

```
def upperCap (a : FourArcAssembly) : OneSidedCircularCap where
  chord := a.core.chord
  theta := a.outerAngle
  midpointX := 0
  baseY := 1
  side := .upper
  chord_pos := a.core.chord_pos
  theta_pos := a.outerAngle_pos
  theta_lt_pi := lt_trans a.outerAngle_lt_pi_div_two (by linarith [Real.pi_pos])
```

FourArcAssembly.weightedArea

Classification: main

Source: CMVGeometry.lean:2541-2545

Declaration SHA-256: ad43970e4ac79b935baed41701092f4d901053e35bb88583dcbd5a9003db13a3

Mathematical statement. Weighted area of the four-arc carrier.

It is the density integral over the assembled set.

```
def weightedArea (lam : ℝ) (a : FourArcAssembly) : ℝ :=
  WeightedArea lam a.carrier

/-- Weighted perimeter is the density integral against the explicit regular
boundary measure. -/
```

FourArcAssembly.weightedArea_formula

Classification: main

Source: CMVGeometry.lean:2554-2602

Declaration SHA-256: 839e3c1d7a83b5801ac118ef77feb57b415f9e5a71dfa154b8987ba3b91118a0

Mathematical statement. Area equals core area plus lambda times the two cap areas.

Shared horizontal chords are area-null, so the three integrals add.

```
theorem weightedArea_formula (lam : ℝ) (a : FourArcAssembly) :
  a.weightedArea lam =
    a.core.euclideanArea + lam *
      (a.upperCap.euclideanArea + a.lowerCap.euclideanArea) := by
  have hcoreUpper : AEDisjoint volume a.core.carrier a.upperCap.carrier := by
    refine measure_mono_null ?_ (volume_horizontalline 1)
  rintro p ⟨hcore, hupper⟩
  have hyCore := a.core.carrier_y_bounds hcore
  have hyUpper : (1 : ℝ) ≤ p.2 := by
    simp [upperCap] using hupper.2
  exact le_antisymm (le_trans (le_abs_self p.2) hyCore) hyUpper
  have hfirstLower : AEDisjoint volume
    (a.core.carrier ∪ a.upperCap.carrier) a.lowerCap.carrier := by
```

```

refine measure_mono_null ?_ (volume_horizontalline (-1))
rintro p ⟨hfirst, hlower⟩
rcases hfirst with hcore | hupper
· have hyCore := a.core.carrier_y_bounds hcore
  have hyLower : p.2 ≤ (-1 : ℝ) := by
    simpa [lowerCap] using hlower.2
  have hneg : -p.2 ≤ 1 := le_trans (neg_le_abs p.2) hyCore
  exact le_antisymm hyLower (by linarith)
· have hyUpper : (1 : ℝ) ≤ p.2 := by
    simpa [upperCap] using hupper.2
  have hyLower : p.2 ≤ (-1 : ℝ) := by
    simpa [lowerCap] using hlower.2
  linarith
have hcore := core_integrableOn lam a.core
have hupper := cap_integrableOn lam a.upperCap
have hlower := cap_integrableOn lam a.lowerCap
rw [weightedArea, WeightedArea, carrier,
  setIntegral_union₀ hfirstLower
  a.lowerCap.measurableSet_carrier.nullMeasurableSet
  (hcore.union hupper) hlower,
  setIntegral_union₀ hcoreUpper
  a.upperCap.measurableSet_carrier.nullMeasurableSet hcore hupper]
rw [show (∫ p in a.core.carrier, StripDensity lam p) =
  a.core.euclideanArea by
  simpa [WeightedArea] using a.core.weightedArea_formula lam,
  show (∫ p in a.upperCap.carrier, StripDensity lam p) =
  lam * a.upperCap.euclideanArea by
  simpa [WeightedArea] using
    cap_upper_weightedArea lam a.upperCap rfl rfl,
  show (∫ p in a.lowerCap.carrier, StripDensity lam p) =
  lam * a.lowerCap.euclideanArea by
  simpa [WeightedArea] using
    cap_lower_weightedArea lam a.lowerCap rfl rfl]
ring

/-- The boundary integral equals the four-arc component formula. -/

```

FourArcAssembly.weightedPerimeter_formula

Classification: main

Source: CMVGeometry.lean:2603-2624

Declaration SHA-256: 43ff4cdbd6774032399c0e7d7f5767a18500986a522e0ace1e470e2035290acd

Mathematical statement. The component perimeter is core arc length plus lambda times both exterior arc lengths.

No horizontal chord is present in this four-arc boundary sum.

```

theorem weightedPerimeter_formula (lam : ℝ) (a : FourArcAssembly) :
  a.weightedPerimeter lam =
    a.core.boundaryArcLength + lam *
      (a.upperCap.arcLength + a.lowerCap.arcLength) := by
rw [weightedPerimeter, WeightedPerimeter, boundaryMeasure]
rw [integral_add_measure
  (((leftCoreBoundaryMeasure_integrable lam a.core).add_measure
    (rightCoreBoundaryMeasure_integrable lam a.core)).add_measure
    (capBoundaryMeasure_integrable lam a.upperCap))
  (capBoundaryMeasure_integrable lam a.lowerCap)]
rw [integral_add_measure
  ((leftCoreBoundaryMeasure_integrable lam a.core).add_measure
    (rightCoreBoundaryMeasure_integrable lam a.core))
  (capBoundaryMeasure_integrable lam a.upperCap)]
rw [integral_add_measure
  (leftCoreBoundaryMeasure_integrable lam a.core)
  (rightCoreBoundaryMeasure_integrable lam a.core)]
rw [leftCore_weightedIntegral, rightCore_weightedIntegral,
  cap_upper_weightedIntegral lam a.upperCap rfl rfl,
  cap_lower_weightedIntegral lam a.lowerCap rfl rfl,

```

```
StripCore.boundaryArcLength]
ring
```

FourArcCandidate

Classification: main

Source: FourArcCandidate.lean:23-35

Declaration SHA-256: 24dc0ff22e0918bdb2c7f16fab0da52bc08326d88e5cb8e1d2e8a8f7ec2bee62

Mathematical statement. A modeled four-arc candidate with $0 < h \leq 1$ and $0 < \alpha < \pi/2$.

The real parameters determine the actual carrier; the incidence equation is an additional hypothesis, not a constructor field.

```
structure FourArcCandidate (lam : ℝ) where
  h : ℝ
  alpha : ℝ
  h_pos : 0 < h
  h_le_one : h ≤ 1
  alpha_pos : 0 < alpha
  alpha_lt_pi_div_two : alpha < π / 2

namespace FourArcCandidate

variable {lam : ℝ} (candidate : FourArcCandidate lam)

/-- Radius shared by the four source arcs. -/
```

FourArcCandidate.CapReplacement

Classification: main

Source: CapReplacement.lean:51-58

Declaration SHA-256: e1ad18311ba23ee8f10b78f8ffe05ca7f5160069b4c67ed9620f490f9f00dfa9

Mathematical statement. The concrete retained-core replacement assembly.

It uses the candidate's core and the full-branch angle $\theta(2X)$.

```
def CapReplacement : ReplacementAssembly where
  core := candidate.stripCore
  replacementAngle := candidate.replacementAngle
  replacementAngle_pos := candidate.replacementAngle_pos
  replacementAngle_lt_pi := candidate.replacementAngle_lt_pi

/-- The inserted upper cap is definitionally the reusable positive-area cap
constructor at normalized area `2x`. -/
```

FourArcCandidate.IsWeightedPerimeterMinimizer

Classification: main

Source: FourArcCandidate.lean:90-94

Declaration SHA-256: f04b5c158c1a510cd5a8af38df8f99424aeed5e376f986e1d437f95e6a4cbe17

Mathematical statement. No admissible equal-area competitor has smaller perimeter.

For every competitor E , $A_\lambda(E) = A_\lambda(C)$ implies $P_\lambda(C) \leq P_\lambda(E)$.

```
def IsWeightedPerimeterMinimizer : Prop :=
  ∀ competitor : AdmissibleCompetitor lam,
    competitor.WeightedArea = candidate.WeightedArea →
      candidate.WeightedPerimeter ≤ competitor.WeightedPerimeter
```

FourArcCandidate.SatisfiesCMVTypeIVHypotheses

Classification: main

Source: FourArcCandidate.lean:84-89

Declaration SHA-256: c09b9c7180b6131040333c0b3c2623f3c89af7f54fe8ab0b9f6a41be873ca0dc

Mathematical statement. Density jump and Snell incidence for a modeled candidate.

The fields are exactly $1 < \lambda$ and $\lambda \cos \alpha = h$.

```
structure SatisfiesCMVTypeIVHypotheses : Prop where
  density_jump : 1 < lam
  snell_incidence : lam * cos candidate.alpha = candidate.h

/-- General minimization semantics: comparison is against every admissible
regular competitor, not merely another four-arc candidate. -/
```

FourArcCandidate.WeightedArea

Classification: main

Source: FourArcCandidate.lean:76-78

Declaration SHA-256: 8256dd516e32d853aaed6a85d09fdb0b62411bd0bf67f1df7ffd123e47baed99

Mathematical statement. The weighted area of the candidate region.

It is inherited from the admissible carrier, not defined by the scalar comparison formula.

```
def WeightedArea : ℝ := candidate.region.WeightedArea

/-- Candidate weighted perimeter, computed from its four boundary arcs. -/
```

FourArcCandidate.WeightedPerimeter

Classification: main

Source: FourArcCandidate.lean:79-83

Declaration SHA-256: d649676f2fd717389a6e9a17a385fe854bc399e17ecfdad24bd8a7bcc9a36521

Mathematical statement. The candidate region's canonical weighted frontier perimeter.

The four-arc formula is proved from this geometric definition.

```
def WeightedPerimeter : ℝ := candidate.region.WeightedPerimeter

/-- Source-specific hypotheses not already enforced by the geometric
constructor. The equation is CMV (27), equivalently
`alpha = arccos (h / lam)` on the recorded angle domain. -/
```

FourArcCandidate.alpha_eq_arccos

Classification: main

Source: FourArcCandidate.lean:276-292

Declaration SHA-256: 46219b8ad0fc6b679968f8972242a6ad511174ce969ce84539a4054206de6595

Mathematical statement. CMV incidence identifies alpha with $\arccos(h/\lambda)$.

The principal angle inequalities eliminate the ambiguity in solving $\cos \alpha = h/\lambda$.

```
theorem alpha_eq_arccos (h : candidate.SatisfiesCMVTypeIVHypotheses) :
  candidate.alpha = arccos (candidate.h / lam) := by
```

```

have hlam_pos : 0 < lam := lt_trans zero_lt_one h.density_jump
have hratio_nonneg : 0 ≤ candidate.h / lam :=
  (div_pos candidate.h_pos hlam_pos).le
have hratio_le_one : candidate.h / lam ≤ 1 := by
  apply (div_le_one hlam_pos).2
  exact le_trans candidate.h_le_one h.density_jump.le
symm
apply Real.arccos_eq_of_eq_cos candidate.alpha_pos.le
  (le_trans candidate.alpha_lt_pi_div_two.le
    (by linarith [Real.pi_pos]))
exact (div_eq_iff (ne_of_gt hlam_pos)).2 (by
  simp [mul_comm] using h.snell_incidence.symm)

/-- The literal closed-curvature candidate at `h = 1` and principal exterior
angle `arccos (1 / lam)`. -/

```

FourArcCandidate.assembly

Classification: main

Source: FourArcCandidate.lean:66-72

Declaration SHA-256: ca67fddb2d167afab63544438387902bb645349ebfde9d83c7a2719a2277c941

Mathematical statement. The four-arc assembly determined by the candidate parameters.

Attach upper and lower caps of half-angle α to the candidate's core.

```

def assembly : FourArcAssembly where
  core := candidate.stripCore
  outerAngle := candidate.alpha
  outerAngle_pos := candidate.alpha_pos
  outerAngle_lt_pi_div_two := candidate.alpha_lt_pi_div_two

/-- The candidate as an admissible regular coordinate region. -/

```

FourArcCandidate.candidate_area_formula

Classification: main

Source: FourArcCandidate.lean:410-430

Declaration SHA-256: b6d944ecc5d4bab77cadf263520f194142a9d6855088a9b98e5873be144ab8cc

Mathematical statement. The candidate satisfies the source type-(iv) area formula.

The rectangle and cap areas simplify using $\lambda \cos \alpha = h$ to the expression displayed after source equation (27), hence to $A_4(h)$.

```

theorem candidate_area_formula
  (h : candidate.SatisfiesCMVTypeIVHypotheses) :
  candidate.WeightedArea =
    4 * sin candidate.alpha / candidate.h +
    2 * (lam * candidate.alpha -
      sin candidate.alpha * cos candidate.beta) / candidate.h ^ 2 +
    2 * (arcsin candidate.h -
      candidate.h * √(1 - candidate.h ^ 2)) / candidate.h ^ 2 := by
  rw [candidate.weightedArea_components, StripCore.euclideanArea,
    candidate.side_caps_area_formula]
  simp only [stripCore, capChord]
  have hcosBeta : cos candidate.beta = candidate.h := by
    rw [beta, Real.cos_pi_div_two_sub]
    exact Real.sin_arcsin (by linarith [candidate.h_pos]) candidate.h_le_one
  rw [hcosBeta, capArea]
  field_simp [ne_of_gt candidate.h_pos]
  have hsnell_mul := congrArg (fun z : ℝ => sin candidate.alpha * z)
    h.snell_incidence
  nlinarith [hsnell_mul]

```



```
end FourArcCandidate
```

FourArcCandidate.candidate_perimeter_cap_components

Classification: main

Source: GeometricBridge.lean:98-105

Declaration SHA-256: 5aef3e70645e6bc7cb387847d94c3bc5e794e60aba832cf7143171cce0f52718

Mathematical statement. Candidate perimeter equals core arc length plus 2 lambda c L(X).

Normalize each congruent exterior cap by its common chord.

```
theorem candidate_perimeter_cap_components :
  candidate.WeightedPerimeter =
    candidate.stripCore.boundaryArcLength +
    2 * lam * candidate.capChord * arc candidate.capX := by
  rw [candidate.weightedPerimeter_components,
    candidate.original_arc_eq_ell]

/-- Replacement perimeter from its three retained/new boundary components. -/
```

FourArcCandidate.candidate_perimeter_formula

Classification: main

Source: FourArcCandidate.lean:391-409

Declaration SHA-256: b4949bdc7019f429151f2faf734db8466461943a33be006f4c104caebbb493fe0

Mathematical statement. The candidate satisfies the type-(iv) source perimeter formula.

$P_\lambda(C) = 4\lambda\alpha/h + 4\arcsinh/h$ under the recorded CMV hypotheses.

```
theorem candidate_perimeter_formula
  (_h : candidate.SatisfiesCMVTypeIVHypotheses) :
  candidate.WeightedPerimeter =
    4 * lam * candidate.alpha / candidate.h +
    4 * arcsin candidate.h / candidate.h := by
  rw [candidate.weightedPerimeter_components,
    StripCore.boundaryArcLength]
  change 4 * (arcsin candidate.h / sin (arcsin candidate.h)) +
    2 * lam * (2 * sin candidate.alpha / candidate.h) *
    (candidate.alpha / sin candidate.alpha) =
  4 * lam * candidate.alpha / candidate.h +
  4 * arcsin candidate.h / candidate.h
  rw [Real.sin_arcsin (by linarith [candidate.h_pos])
    candidate.h_le_one]
  field_simp [ne_of_gt candidate.h_pos, ne_of_gt candidate.sin_alpha_pos]
  ring

/-- CMV's area display following equation (27), derived from the rectangle and
three cap pieces. -/
```

FourArcCandidate.capArea

Classification: main

Source: FourArcCandidate.lean:45-49

Declaration SHA-256: eb6c25ab8d5e1e4d2ce589b98fc3af2d04e58bd682ad273d1b07834426ee23d0

Mathematical statement. Euclidean area of either original exterior cap.

It is $(\alpha - \sin\alpha\cos\alpha)/h^2$.

```
def capArea : ℝ :=
  (candidate.alpha - sin candidate.alpha * cos candidate.alpha) /
  candidate.h ^ 2

/-- Scale-free cap area passed to Morgan's arc function. -/
```

FourArcCandidate.capArea_eq_chord_sq_mul_area

Classification: main

Source: FourArcCandidate.lean:104-109

Declaration SHA-256: 0e9545ae1662d69d55500dc9264e42c4c5b03b78571f6473a85822aca53c6073

Mathematical statement. Candidate cap area is chord squared times normalized cap area.

$\text{capArea} = c^2 a(\alpha)$ by the circular-segment formula.

```
theorem capArea_eq_chord_sq_mul_area :
  candidate.capArea = candidate.capChord ^ 2 * area candidate.alpha := by
  rw [capArea, capChord, area]
  field_simp [ne_of_gt candidate.h_pos, ne_of_gt candidate.sin_alpha_pos]
  ring
```

FourArcCandidate.capChord

Classification: main

Source: FourArcCandidate.lean:42-44

Declaration SHA-256: c09aa141e8bbabbe7279bce2473dd47aa931f875442fe1fc79d759685536fb1e

Mathematical statement. The common horizontal chord of the exterior caps.

$c = 2\sin\alpha/h > 0$.

```
def capChord : ℝ := 2 * sin candidate.alpha / candidate.h

/-- Euclidean area of either original exterior cap. -/
```

FourArcCandidate.capReplacementRegion

Classification: main

Source: CapReplacement.lean:69-78

Declaration SHA-256: c2d6906ef5b69db6b314e7ac4d82d54432092c90cdf06b81f1a87bed1432f7c3

Mathematical statement. The cap replacement as an admissible competitor.

Its perimeter is evaluated from the assembly's full frontier, including the exposed lower chord.

```
def capReplacementRegion : AdmissibleCompetitor lam :=
  .replacement candidate.CapReplacement

@[simp] theorem capReplacement_core :
  candidate.CapReplacement.core = candidate.stripCore := rfl

@[simp] theorem capReplacement_angle :
  candidate.CapReplacement.replacementAngle = candidate.replacementAngle := rfl

/-- The inserted coordinate cap has Euclidean area exactly `2A`. -/
```

FourArcCandidate.capX

Classification: main

Source: FourArcCandidate.lean:50-52

Declaration SHA-256: a20fa39233b648b55a696be0bec2aefc6c5d3bf3467f9a8163b812924a00e64

Mathematical statement. Scale-free area of an original cap.

$X = \text{capArea}/c^2 = a(\alpha)$ is the input to the unit-chord arc function.

```
def capX : ℝ := candidate.capArea / candidate.capChord ^ 2

/-- The retained central strip region. -/
```

FourArcCandidate.capX_eq_area

Classification: main

Source: FourArcCandidate.lean:116-119

Declaration SHA-256: ef35b1cb5934a4532e1255f444a69fb5e100682afde4a7e8de56f9cb84bae537

Mathematical statement. The normalized candidate cap area is $a(\alpha)$.

Cancel the nonzero chord square in the area scaling identity.

```
theorem capX_eq_area : candidate.capX = area candidate.alpha := by
  rw [capX, candidate.capArea_eq_chord_sq_mul_area]
  field_simp [candidate.capChord_ne]
```

FourArcCandidate.cap_replacement_perimeter_difference

Classification: main

Source: GeometricBridge.lean:124-133

Declaration SHA-256: cef0cb1c446892a6b4545fed077efd78b2014155506ae9d18d92702507d05712

Mathematical statement. For every modeled candidate and arbitrary real density parameter, replacement perimeter minus candidate perimeter equals $c(1-\lambda g(X))$.

For a candidate with its constructor bounds $0 < h \leq 1$ and $0 < \alpha < \pi/2$, let c be its positive cap chord and X its normalized cap area. The declaration proves the division-free identity $P_{\lambda}(C) - P(C) = c \bigl((1 - \lambda g(X)) \bigr)$. No density-jump or incidence hypothesis is needed, and λ may be any real number. The retained core cancels; the exposed lower chord contributes c . If $\lambda \neq 0$, negating and factoring gives the saving formula $P_{\lambda}(C) - P(C) = \lambda c (g(X) - 1/\lambda)$ used in the main proof, where $\lambda > 1$.

```
theorem cap_replacement_perimeter_difference :
  candidate.capReplacementRegion.WeightedPerimeter -
  candidate.WeightedPerimeter =
  candidate.capChord * (1 - lam * g candidate.capX) := by
  rw [candidate.replacement_perimeter_cap_components,
    candidate.candidate_perimeter_cap_components]
  rw [g]
  ring

/-- Equivalent opposite signed factorization, only after `lam ≠ 0`. -/
```

FourArcCandidate.cap_replacement_preserves_area

Classification: main

Source: GeometricBridge.lean:90-97

Declaration SHA-256: 40dbfacc980c0caa8be7bb02e41318ad70f3b454256d042f4cd60d664125314b

Mathematical statement. The cap replacement has exactly the candidate's weighted area.

Both regions contain the same core and total exterior cap area at the same density.

```
theorem cap_replacement_preserves_area :
  candidate.capReplacementRegion.WeightedArea =
    candidate.WeightedArea := by
  rw [candidate.replacement_weightedArea_components,
    candidate.weightedArea_components]
  ring

/-- Candidate perimeter rewritten in the Morgan cap normalization. -/
```

FourArcCandidate.cap_replacement_strictly_improves

Classification: main

Source: GeometricBridge.lean:157-176

Declaration SHA-256: c953f49d8456f94f9248f042331d7c54782ee3366628a4a57c9db7cc82ecdd4a

Mathematical statement. Cap replacement strictly lowers perimeter at the certified high-density threshold.

Combine $g(X) > 1/\lambda$ with $c > 0$ and $\lambda > 0$ in the exact difference identity.

```
theorem cap_replacement_strictly_improves
  (hlam : (1.2581840884 : ℝ) ≤ lam) :
  candidate.capReplacementRegion.WeightedPerimeter <
    candidate.WeightedPerimeter := by
  have hlam_pos : 0 < lam := density_pos_of_campaign_threshold hlam
  have hcomparison : 1 / lam < g candidate.capX :=
    cmv_comparison_bound_optimized hlam candidate.capX_pos
  have hscaled := mul_lt_mul_of_pos_left hcomparison hlam_pos
  have hunit : lam * (1 / lam) = 1 := by
    field_simp [ne_of_gt hlam_pos]
  rw [hunit] at hscaled
  have hfactor : 1 - lam * g candidate.capX < 0 := sub_neg.mpr hscaled
  have hdifference :
    candidate.capReplacementRegion.WeightedPerimeter -
      candidate.WeightedPerimeter < 0 := by
    rw [candidate.cap_replacement_perimeter_difference]
    exact mul_neg_of_pos_of_neg candidate.capChord_pos hfactor
  linarith

/-- Explicit receipt for the included non-strict threshold boundary. -/
```

FourArcCandidate.exposed_lower_chord_length

Classification: main

Source: CapReplacement.lean:95-99

Declaration SHA-256: f04f252743e50cee8e8705384217c84f1c347cf4e87d10a166f74cf59b5b7fd6

Mathematical statement. The exposed lower chord has the candidate's chord length c .

This is the unweighted positive term added to replacement perimeter.

```
theorem exposed_lower_chord_length :
  candidate.CapReplacement.exposedLowerChord.euclideanLength =
    candidate.capChord := rfl
```

```
-- The lower chord lies on the density-one branch, including its endpoints. -/
```

FourArcCandidate.not_isWeightedPerimeterMinimizer_of_typeThree

Classification: main

Source: TypeThreeAssembly.lean:3101-3118

Declaration SHA-256: 6807359794d5b2629c85ed715f93a0b06f1e0f5dc3b504c99a54401226562f55

Mathematical statement. A realized equal-area type-(iii) competitor with smaller perimeter disproves candidate minimality.

Apply the candidate's universal minimizing inequality to the realized competitor and contradict the strict perimeter comparison.

```
theorem not_isWeightedPerimeterMinimizer_of_typeThree
  (a : TypeThreeAssembly lam)
  (realization : TypeThreeAssembly.ModeledCompetitorRealization a)
  (equalArea :
    a.scalarWeightedArea = candidate.WeightedArea)
  (strictPerimeter :
    a.scalarWeightedPerimeter < candidate.WeightedPerimeter) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  intro hmin
  have harea :
    realization.asCompetitor.WeightedArea =
      candidate.WeightedArea := by
    rw [realization.asCompetitor_weightedArea, equalArea]
  have hle := hmin realization.asCompetitor harea
  rw [realization.asCompetitor_weightedPerimeter] at hle
  exact (not_le_of_gt strictPerimeter) hle

end FourArcCandidate
```

FourArcCandidate.original_arc_eq_ell

Classification: main

Source: GeometricBridge.lean:27-33

Declaration SHA-256: b932d1e6553b24d46c1b0161df0040ebd64f34413f6c447a3da4b2b0f95c234a

Mathematical statement. The normalized arc function at the candidate cap area equals $\ell(\alpha)$.

$L(X) = \ell(\alpha)$ since $X = a(\alpha)$.

```
theorem original_arc_eq_ell :
  arc candidate.capX = ell candidate.alpha := by
  rw [candidate.capX_eq_area, arc_eq,
    00f_area ⟨candidate.alpha_pos,
      lt_trans candidate.alpha_lt_pi_div_two (by linarith [Real.pi_pos])⟩]

/-- Old exterior boundary cost, derived from the two cap arcs. -/
```

FourArcCandidate.region

Classification: main

Source: FourArcCandidate.lean:73-75

Declaration SHA-256: 8ab9f84c197a1a6dbe90236469e9ff5cc2696f04e42133db44cdf34546558b75

Mathematical statement. The candidate assembly as an admissible four-arc competitor.

This connects parameter data to the minimization domain.

```
def region : AdmissibleCompetitor lam := .fourArc candidate.assembly

/-- Candidate weighted area, computed from its region constructor. -/
```

FourArcCandidate.replacementAngle

Classification: main

Source: CapReplacement.lean:27-28

Declaration SHA-256: e82e3e1d3191a88d1c944b28e4f02eaa8d736356029ed2fa74ba08f57ef344d7

Mathematical statement. The cap angle whose normalized area is twice that of one candidate cap.

It is $\theta(2X)$, where $X = \text{capArea}/c^2 > 0$.

```
def replacementAngle : ℝ := θ0f (2 * candidate.capX)
```

FourArcCandidate.replacementAngle_area

Classification: main

Source: CapReplacement.lean:39-43

Declaration SHA-256: c942d60f253b362e97e724222a6abdc2af0e9c796571a40aeeb1a4c26cb3d0d7

Mathematical statement. The normalized replacement-cap area is $2X$.

The inverse identity $a(\theta(2X)) = 2X$ proves exact area doubling.

```
theorem replacementAngle_area :
  area candidate.replacementAngle = 2 * candidate.capX := by
  exact area_θ0f (mul_pos (by norm_num) candidate.capX_pos)

/-- Uniqueness on the accepted positive-area branch. -/
```

FourArcCandidate.replacement_cap_arc_length

Classification: main

Source: CapReplacement.lean:89-94

Declaration SHA-256: 15d3b8bc212201f87fa01ba9f302a81f8ee2f8ead5c18096e4f073110885dea8

Mathematical statement. The replacement upper arc has length $cL(2X)$.

The unit-chord arc function scales linearly with chord length.

```
theorem replacement_cap_arc_length :
  candidate.CapReplacement.upperCap.arcLength =
  candidate.capChord * arc (2 * candidate.capX) := by
  rfl

/-- The exposed lower interface segment has Euclidean and weighted cost `L`. -/
```

FourArcCandidate.replacement_cap_area

Classification: main

Source: CapReplacement.lean:79-88

Declaration SHA-256: 5517f888e5fc2a04dba3edea18ff03637b20f17abc45bd337950afb694709ec8

Mathematical statement. The replacement upper cap has twice the area of an original exterior cap.

Multiplying $a(\theta(2X)) = 2X$ by the common chord square proves the geometric identity.

```

theorem replacement_cap_area :
  candidate.CapReplacement.upperCap.euclideanArea =
    2 * candidate.capArea := by
  rw [OneSidedCircularCap.euclideanArea]
  change candidate.capChord ^ 2 * area candidate.replacementAngle =
    2 * candidate.capArea
  rw [candidate.replacementAngle_area, capX]
  field_simp [candidate.capChord_ne]

/-- Its arc length is the exact Morgan scaling `L * arc (2x)`. -/

```

FourArcCandidate.replacement_perimeter_cap_components

Classification: main

Source: GeometricBridge.lean:106-123

Declaration SHA-256: ad5283b2c2158e01ae2eb9db43ef15a8cb068a20feefa5923311c31d1fd30f36

Mathematical statement. Replacement perimeter equals core arc length plus $\lambda c L(2X)$ plus c .

The final c is the exposed lower chord at density one.

```

theorem replacement_perimeter_cap_components :
  candidate.capReplacementRegion.WeightedPerimeter =
    candidate.stripCore.boundaryArcLength +
    lam * candidate.capChord * arc (2 * candidate.capX) +
    candidate.capChord := by
  change _root_.WeightedPerimeter lam
  (FrontierMeasure candidate.CapReplacement.carrier) =
    candidate.stripCore.boundaryArcLength +
    lam * candidate.capChord * arc (2 * candidate.capX) +
    candidate.capChord
  rw [← replacement_frontier_weightedPerimeter_eq,
    ReplacementAssembly.weightedPerimeter_formula, capReplacement_core,
    candidate.replacement_cap_arc_length,
    candidate.exposed_lower_chord_length]
  ring

/-- Exact signed perimeter-difference identity after cancellation of the
retained side arcs. -/

```

FourArcCandidate.replacement_weightedArea_components

Classification: main

Source: GeometricBridge.lean:79-89

Declaration SHA-256: f60c2f169a3606748cb65977bfe0e42bbb44dc76480dad9bc42ac52ed335dcbb

Mathematical statement. Replacement weighted area is core area plus the doubled weighted cap area.

The upper cap realizes exactly twice the original cap's Euclidean area.

```

theorem replacement_weightedArea_components :
  candidate.capReplacementRegion.WeightedArea =
    candidate.stripCore.euclideanArea +
    lam * (2 * candidate.capArea) := by
  change candidate.CapReplacement.weightedArea lam =
    candidate.stripCore.euclideanArea + lam * (2 * candidate.capArea)
  rw [ReplacementAssembly.weightedArea_formula, capReplacement_core,
    candidate.replacement_cap_area]

/-- Exact weighted-area preservation. The exposed chord contributes no planar
area; both sides retain the same central piece. -/

```

FourArcCandidate.side_caps_area_formula

Classification: main

Source: FourArcCandidate.lean:351-363

Declaration SHA-256: a639bd8c9eef40f2c4f87fdb62b671cf7db0ce41013e7d923721f6f472bc8cf1

Mathematical statement. The two core side segments have the standard arcsine area expression.

$$8a(\arcsinh) = 2(\arcsinh - h\sqrt{1-h^2})/h^2.$$

```
theorem side_caps_area_formula :
  8 * area candidate.stripCore.sideAngle =
    2 * (arcsin candidate.h -
      candidate.h * √(1 - candidate.h ^ 2)) / candidate.h ^ 2 := by
change 8 * area (arcsin candidate.h) =
  2 * (arcsin candidate.h -
    candidate.h * √(1 - candidate.h ^ 2)) / candidate.h ^ 2
rw [area, Real.sin_arcsin (by linarith [candidate.h_pos])
  candidate.h_le_one, Real.cos_arcsin]
field_simp [ne_of_gt candidate.h_pos]
ring

/-- Candidate area as the sum of the actual core and cap constructors. -/
```

FourArcCandidate.stripCore

Classification: main

Source: FourArcCandidate.lean:53-65

Declaration SHA-256: 66aff9e1eb934bbbeeb8dd09a1a68a4c23aef3ee09735d3ba9bc12eadb4664ee

Mathematical statement. The candidate's central strip core.

Its chord is c and its curvature is h .

```
def stripCore : StripCore where
  chord := candidate.capChord
  curvature := candidate.h
  chord_pos := by
    exact div_pos (mul_pos (by norm_num)
      (sin_pos_of_pos_of_lt_pi candidate.alpha_pos
        (lt_trans candidate.alpha_lt_pi_div_two
          (by linarith [Real.pi_pos])))) candidate.h_pos
  curvature_pos := candidate.h_pos
  curvature_le_one := candidate.h_le_one

/-- Candidate assembled from the retained strip core and reflected upper/lower
caps. -/
```

FourArcCandidate.weightedArea_components

Classification: main

Source: FourArcCandidate.lean:364-373

Declaration SHA-256: 654554fba4e2f4847eae4f853550061afa29cf158da02de52109cffffacefd702e

Mathematical statement. Candidate area is core area plus twice lambda times one cap area.

The two exterior caps are congruent and their interfaces are area-null.

```
theorem weightedArea_components :
  candidate.WeightedArea = candidate.stripCore.euclideanArea +
    2 * lam * candidate.capArea := by
change candidate.assembly.weightedArea lam =
  candidate.stripCore.euclideanArea + 2 * lam * candidate.capArea
```



```

rw [FourArcAssembly.weightedArea_formula, assembly_core,
  candidate.upperCap_euclideanArea, candidate.lowerCap_euclideanArea]
ring

/-- Candidate perimeter as the sum of its four actual arc constructors. -/

```

FourArcCandidate.weightedPerimeter_components

Classification: main

Source: FourArcCandidate.lean:374-390

Declaration SHA-256: d02953cec6a51d470b1b940fa2318bbd5c86354d0e420d179322c86356c9434a

Mathematical statement. Candidate perimeter is core arc length plus $2\lambda c_{\ell}(\alpha)$.

The full-frontier identity justifies summing exactly the four exposed arcs.

```

theorem weightedPerimeter_components :
  candidate.WeightedPerimeter = candidate.stripCore.boundaryArcLength +
    2 * lam * candidate.capChord * ell candidate.alpha := by
rw [FourArcCandidate.WeightedPerimeter, region,
  AdmissibleCompetitor.WeightedPerimeter]
change _root_.WeightedPerimeter lam
  (FrontierMeasure candidate.assembly.carrier) =
  candidate.stripCore.boundaryArcLength +
    2 * lam * candidate.capChord * ell candidate.alpha
rw [← fourArc_frontier_weightedPerimeter_eq,
  FourArcAssembly.weightedPerimeter_formula]
simp only [StripCore.boundaryArcLength,
  OneSidedCircularCap.arcLength, FourArcAssembly.upperCap,
  FourArcAssembly.lowerCap, assembly, stripCore]
ring

/-- CMV equation (27), derived from the boundary pieces. -/

```

FrontierMeasure

Classification: main

Source: CMVGeometry.lean:61-66

Declaration SHA-256: 7323bb80c0a565daf410d27d199e2143ee33a0ecca0460c52504f2e7e50b48e2

Mathematical statement. Euclidean Hausdorff measure restricted to the complete frontier, transported to coordinates.

This canonically fixes perimeter data from E ; it is not a freely supplied boundary measure.

```

def FrontierMeasure (carrier : Set PlanePoint) : Measure PlanePoint :=
  Measure.map planeEuclideanHomeomorph.symm
    ((μH[1] : Measure EuclideanPlane).restrict
      (frontier (planeEuclideanHomeomorph '' carrier)))

/-- Weighted perimeter is the density integral against a boundary measure. -/

```

LeanSuffixAnalytic.DescendingEqualAreaEnvelope

Classification: main

Source: EqualAreaEnvelope.lean:279-300

Declaration SHA-256: ee2db888cf8e5e37251e350d2425fb86808e5c1f7d808208572e20c0e3ea40a9

Mathematical statement. A stationary type-(iv) fold and a regular negative-fold equal-area type-(iii) root for every modeled curvature.

The record requires $\lambda > 1$, $0 < t_* < 1$, $K_4(t_*) = 0$, $0 < r(t) < 1$, equal area, and $K_3(r(t)) < 0$ for $0 < t \leq 1$.

```

structure DescendingEqualAreaEnvelope (lam : ℝ) where
  /-- The unique type-(iv) fold selected by the retained fold-gap statement. -/
  fold : ℝ
  /-- The descending type-(iii) equal-area root for every modeled curvature. -/
  root : ℝ → ℝ
  density_jump : 1 < lam
  fold_pos : 0 < fold
  fold_lt_one : fold < 1
  fold_stationary : typeFourFold lam fold = 0
  root_pos : ∀ {h₄ : ℝ}, 0 < h₄ → h₄ ≤ 1 → 0 < root h₄
  root_lt_one : ∀ {h₄ : ℝ}, 0 < h₄ → h₄ ≤ 1 → root h₄ < 1
  equalArea : ∀ {h₄ : ℝ}, 0 < h₄ → h₄ ≤ 1 →
    typeThreeArea lam (root h₄) = typeFourArea lam h₄
  /-- Every selected type-(iii) root lies on the strictly descending branch.
  This identifies it globally as the least equal-area root and implies the
  simple-root condition used by the inverse-function argument. -/
  root_fold_neg : ∀ {h₄ : ℝ}, 0 < h₄ → h₄ ≤ 1 →
    typeThreeFold lam (root h₄) < 0

  /-- Every selected descending root is bounded above by the root at the
  stationary type-(iv) fold. This fixed regular anchor supplies the compactness
  needed for branch continuity. -/

```

LeanSuffixAnalytic.FoldGapNegative

Classification: main

Source: EqualAreaEnvelope.lean:577-581

Declaration SHA-256: e0a51358d0910c8fb2dd4ffa9d95125b8210328ea93cd0771ca9e9c1138b2ced

Mathematical statement. Strict negativity of the genuine perimeter gap at the selected stationary fold.

It asserts only $\Delta(t_*) < 0$, not the all-curvature conclusion.

```

def FoldGapNegative {lam : ℝ} (env : DescendingEqualAreaEnvelope lam) : Prop :=
  equalAreaPerimeterGap lam env.root env.fold < 0

  /-- L12, including the closure endpoint: the stationary fold is the strict
  maximum of the genuine perimeter gap along the selected equal-area branch. -/

```

LeanSuffixAnalytic.InRetainedSuffix

Classification: main

Source: LeanSuffixAnalytic.lean:1159-1161

Declaration SHA-256: 18acd275369860c6ce3970cef1307119f694180bde02eaaf9f16412e8cd68698

Mathematical statement. The closed certificate density interval $[51/50, 9/7]$.

It means $51/50 \leq \lambda \leq 9/7$ and therefore implies $\lambda > 1$.

```

def InRetainedSuffix (lam : ℝ) : Prop :=
  (51 / 50 : ℝ) ≤ lam ∧ lam ≤ 9 / 7

```

LeanSuffixAnalytic.StationaryEqualAreaPair

Classification: main

Source: LeanSuffixAnalytic.lean:1177-1188

Declaration SHA-256: 865710967f267b9ec6707c5d925cd799ff4862bc66ec64cf0a22afce00862eef

Mathematical statement. A type-(iii) curvature above one half and a regular stationary type-(iv) curvature with equal area.

The fields are $1/2 < s < 1$, $0 < t < 1$, $A_3(s) = A_4(t)$, and $K_4(t) = 0$. Negative K_3 and strict perimeter improvement are separate assumptions.

```
structure StationaryEqualAreaPair (lam : ℝ) where
  h₃ : ℝ
  h₄ : ℝ
  h₃_gt_half : 1 / 2 < h₃
  h₃_lt_one : h₃ < 1
  h₄_pos : 0 < h₄
  h₄_lt_one : h₄ < 1
  equalArea : typeThreeArea lam h₃ = typeFourArea lam h₄
  stationary : typeFourFold lam h₄ = 0

/-- Concrete universal counter-obligation left by the direct reduction. It
contains the exact cutoff, root equations, branch domains, and radical gap. -/
```

LeanSuffixAnalytic.allCurvature_typeThreeImprovement_of_envelope

Classification: main

Source: EqualAreaEnvelope.lean:645-658

Declaration SHA-256: c4f7d83c3e3a4e728eb9be863cb4e20b5d04c133005f9a44bb6548808441f1ae

Mathematical statement. A descending envelope with negative fold gap improves every type-(iv) scalar for $0 < t \leq 1$.

Take $s = r(t)$; equal area is a record field and $P_3(s) - P_4(t) \leq \Delta(t_*) < 0$.

```
theorem allCurvature_typeThreeImprovement_of_envelope {lam h₄ : ℝ}
  (env : DescendingEqualAreaEnvelope lam) (hfoldGap : FoldGapNegative env)
  (hh₄ : 0 < h₄) (hh₄_one : h₄ ≤ 1) :
  ∃ h₃ : ℝ, 0 < h₃ ∧ h₃ < 1 ∧
    typeThreeArea lam h₃ = typeFourArea lam h₄ ∧
    typeThreePerimeter lam h₃ < typeFourPerimeter lam h₄ := by
  refine ⟨env.root h₄, env.root_pos hh₄ hh₄_one,
    env.root_lt_one hh₄ hh₄_one, env.equalArea hh₄ hh₄_one, ?_⟩
  rw [← sub_lt_zero]
  exact lt_of_le_of_lt (equalAreaPerimeterGap_le_fold env hh₄ hh₄_one) hfoldGap

/-- One certified stationary pair with negative type-(iii) fold and strict
perimeter improvement supplies an improving equal-area type-(iii) scalar at
every regular or endpoint type-(iv) curvature. -/
```

LeanSuffixAnalytic.centered_meanValue_exists

Classification: main

Source: LeanSuffixAnalytic.lean:93-129

Declaration SHA-256: 9e16c64246998ba49ad719b4074913f0337c8612efed42e98f32935fcd55a3e5

Mathematical statement. The centered mean-value identity with a single intermediate point.

Under differentiability on the intervening interval, $f(h) = f(c) + f'(\xi)(h - c)$ for some ξ between c and h ; repeated occurrences in $f'(\xi)$ stay correlated.

```
theorem centered_meanValue_exists
  (f f' : ℝ → ℝ) {c h : ℝ}
  (hderiv : ∀ x ∈ Set.uIcc c h, HasDerivAt f (f' x) x) :
  ∃ ξ ∈ Set.uIcc c h, f h = f c + f' ξ * (h - c) := by
  rcases lt_trichotomy c h with hlt | heq | hgt
  · have hcont : ContinuousOn f (Set.Icc c h) := by
    intro x hx
    exact (hderiv x (Set.mem_uIcc.mpr (Or.inl hx))).continuousAt.continuousWithinAt
  have hdiff : DifferentiableOn ℝ f (Set.Ioo c h) := by
    intro x hx
    exact (hderiv x (Set.mem_uIcc.mpr
```

```

    (Or.inl ⟨hx.1.le, hx.2.le⟩)).differentiableAt.differentiableWithinAt
rcases exists_deriv_eq_slope f hlt hcont hdiff with ⟨ξ, hξ, hslope⟩
have hξu : ξ ∈ Set.uIcc c h :=
  Set.mem_uIcc.mpr (Or.inl ⟨hξ.1.le, hξ.2.le⟩)
refine ⟨ξ, hξu, ?_⟩
rw [(hderiv ξ hξu).deriv] at hslope
field_simp [ne_of_gt (sub_pos.mpr hlt)] at hslope
linarith
· subst h
  exact ⟨c, by simp, by ring⟩
· have hcont : ContinuousOn f (Set.Icc h c) := by
  intro x hx
  exact (hderiv x (Set.mem_uIcc.mpr (Or.inr hx))).continuousAt.continuousWithinAt
have hdiff : DifferentiableOn ℝ f (Set.Ioo h c) := by
  intro x hx
  exact (hderiv x (Set.mem_uIcc.mpr
    (Or.inr ⟨hx.1.le, hx.2.le⟩)).differentiableAt.differentiableWithinAt
rcases exists_deriv_eq_slope f hgt hcont hdiff with ⟨ξ, hξ, hslope⟩
have hξu : ξ ∈ Set.uIcc c h :=
  Set.mem_uIcc.mpr (Or.inr ⟨hξ.1.le, hξ.2.le⟩)
refine ⟨ξ, hξu, ?_⟩
rw [(hderiv ξ hξu).deriv] at hslope
field_simp [ne_of_gt (sub_pos.mpr hgt)] at hslope
linarith
/-- The type-(iii) angle term is exactly the checkers' `B3` expression after
the substitution `q = cos w`. -/

```

LeanSuffixAnalytic.equalAreaPerimeterGap

Classification: main

Source: EqualAreaEnvelope.lean:34-39

Declaration SHA-256: e8643a1549a791c31f1d0a91f7cb9b302d6832ce9c0d1d8e354d9c3e22351fcb

Mathematical statement. The perimeter difference along a selected equal-area root function.

$\Delta(t) = P_3(r(t)) - P_4(t)$ for the supplied root map r .

```

def equalAreaPerimeterGap (lam : ℝ) (root : ℝ → ℝ) (h₄ : ℝ) : ℝ :=
  typeThreePerimeter lam (root h₄) - typeFourPerimeter lam h₄

/-- A function selected as the unique zero in a compact branch is continuous.
Compactness is used only for the branch: the zero graph is closed, and its
projection along the compact factor is closed. -/

```

LeanSuffixAnalytic.equalAreaPerimeterGap_hasDerivAt

Classification: main

Source: EqualAreaEnvelope.lean:184-225

Declaration SHA-256: 42f42f4f683c9b8ff6e876958c9ae3b87d6bfd1ebf63072c7e612401d7b5ee93

Mathematical statement. The equal-area perimeter gap has derivative $(r(t)-t)A_4'(t)$.

Differentiate $A_3(r(t)) = A_4(t)$ and use $P_i' = hA_i'$.

```

theorem equalAreaPerimeterGap_hasDerivAt
  {lam h : ℝ} {root : ℝ → ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1)
  (hroot_pos : 0 < root h) (hroot_lt_one : root h < 1)
  (hroot_differentiable : DifferentiableAt ℝ root h)
  (hequalArea : ∀ {x : ℝ}, 0 < x → x ≤ 1 →
    typeThreeArea lam (root x) = typeFourArea lam x) :
  HasDerivAt (equalAreaPerimeterGap lam root)
    ((root h - h) * deriv (typeFourArea lam) h) h := by
rcases typeThree_variational_hasDerivAt hlam hroot_pos hroot_lt_one with
  ⟨hareaThree, hperimeterThree⟩

```

```

rcases typeFour_variational_hasDerivAt hlam hh hh1 with
  (hareaFour, hperimeterFour)
have hroot := hroot_differentiable.hasDerivAt
have harea_comp := hareaThree.comp h hroot
have hequalArea_eventually :
  (typeThreeArea lam ° root) =f[nhds h] typeFourArea lam := by
  filter_upwards [Ioo_mem_nhds hh hh1] with x hx
  exact hequalArea hx.1 hx.2.le
have harea_comp_from_equalArea :
  HasDerivAt (typeThreeArea lam ° root)
    (deriv (typeFourArea lam) h) h :=
  hareaFour.congr_of_eventuallyEq hequalArea_eventually
have hchain :
  deriv (typeThreeArea lam) (root h) * deriv root h =
  deriv (typeFourArea lam) h :=
  harea_comp.unique harea_comp_from_equalArea
have hgap :=
  (hperimeterThree.comp h hroot).sub hperimeterFour
have hgap' : HasDerivAt
  (typeThreePerimeter lam ° root - typeFourPerimeter lam)
  ((root h - h) * deriv (typeFourArea lam) h) h :=
  hgap.congr_deriv (by
    rw [mul_assoc, hchain]
    ring)
apply hgap'.congr_of_eventuallyEq
filter_upwards
intro x
rfl

/-- The canonical descending equal-area root selected from one certified
stationary pair. Values outside  $0 < h_4 \leq 1$  are deliberately irrelevant. -/

```

LeanSuffixAnalytic.equalAreaPerimeterGap_le_fold

Classification: main

Source: EqualAreaEnvelope.lean:632-644

Declaration SHA-256: 0bde810fe503972e30290f54b6517ae65288b7e84131b161bebac58ed0796e33

Mathematical statement. Every modeled curvature has gap at most the stationary gap.

The equality case $t = t_*$ is combined with the strict off-fold maximum theorem.

```

theorem equalAreaPerimeterGap_le_fold {lam : ℝ}
  (env : DescendingEqualAreaEnvelope lam) {h₄ : ℝ}
  (hh₄ : 0 < h₄) (hh₄_one : h₄ ≤ 1) :
  equalAreaPerimeterGap lam env.root h₄ ≤
  equalAreaPerimeterGap lam env.root env.fold := by
  by_cases h : h₄ = env.fold
  · subst h₄
    exact le_rfl
  · exact (equalAreaPerimeterGap_lt_fold env hh₄ hh₄_one h).le

/-- L14 at the modeled scalar level. A negative genuine fold gap yields an
improving equal-area type-(iii) scalar for every  $0 < h_4 \leq 1$ , including the
modeled endpoint  $h_4 = 1$ . -/

```

LeanSuffixAnalytic.equalAreaPerimeterGap_lt_fold

Classification: main

Source: EqualAreaEnvelope.lean:582-631

Declaration SHA-256: 1a0b26ad0e4af7fdd933118930486f792fc517471c75bd1495b190e7f97f3420

Mathematical statement. The stationary fold is the strict maximum of the equal-area perimeter gap.

For $0 < t \leq 1$ with $t \neq t_*$, root order and area-derivative signs give $\Delta(t) < \Delta(t_*)$, including the continuous endpoint.

```

theorem equalAreaPerimeterGap_lt_fold {lam : ℝ}
  (env : DescendingEqualAreaEnvelope lam) {h₄ : ℝ}
  (hh₄ : 0 < h₄) (hh₄_one : h₄ ≤ 1) (hne : h₄ ≠ env.fold) :
  equalAreaPerimeterGap lam env.root h₄ <
    equalAreaPerimeterGap lam env.root env.fold := by
  have hfold_mem_right : env.fold ∈ Icc env.fold (1 : ℝ) :=
    ⟨le_rfl, env.fold_lt_one.le⟩
  by_cases hside : h₄ < env.fold
  · have hleft : StrictMonoOn (equalAreaPerimeterGap lam env.root)
      (Icc h₄ env.fold) := by
    apply strictMonoOn_of_deriv_pos (convex_Icc h₄ env.fold)
  · intro h hh
    exact (env.gap_hasDerivAt (hh₄.trans_le hh.1)
      (hh.2.trans_lt env.fold_lt_one)).continuousAt.continuousWithinAt
  · intro h hh
    rw [interior_Icc] at hh
    rw [(env.gap_hasDerivAt (hh₄.trans hh.1)
      (hh.2.trans env.fold_lt_one)).deriv]
    exact mul_pos_of_neg_of_neg
      (sub_neg.mpr (env.root_lt_curvature (hh₄.trans hh.1)
        (hh.2.trans env.fold_lt_one).le))
      (typeFourArea_deriv_neg_before_zero env.density_jump
        (env.fold_pos, env.fold_lt_one) env.fold_stationary
        (hh₄.trans hh.1, hh.2.trans env.fold_lt_one) hh.2)
    exact hleft ⟨le_rfl, hside.le⟩ ⟨hside.le, le_rfl⟩ hside
  · have hfold_lt : env.fold < h₄ := lt_of_le_of_ne
      (le_of_not_gt hside) (Ne.symm hne)
  have hright : StrictAntiOn (equalAreaPerimeterGap lam env.root)
      (Icc env.fold (1 : ℝ)) := by
    apply strictAntiOn_of_deriv_neg (convex_Icc env.fold 1)
  · intro h hh
    by_cases hh_one : h = 1
    · simp [hh_one] using env.gap_continuousWithinAt_one.mono
      (fun x hx => ⟨env.fold_pos.trans_le hx.1, hx.2⟩)
    · exact (env.gap_hasDerivAt
      (env.fold_pos.trans_le hh.1) (lt_of_le_of_ne hh.2 hh_one)
      ).continuousAt.continuousWithinAt
  · intro h hh
    rw [interior_Icc] at hh
    rw [(env.gap_hasDerivAt (env.fold_pos.trans hh.1) hh.2).deriv]
    exact mul_neg_of_neg_of_pos
      (sub_neg.mpr (env.root_lt_curvature (env.fold_pos.trans hh.1)
        hh.2.le))
      (typeFourArea_deriv_pos_after_zero env.density_jump
        (env.fold_pos, env.fold_lt_one) env.fold_stationary
        (env.fold_pos.trans hh.1, hh.2) hh.1)
    exact hright hfold_mem_right ⟨hfold_lt.le, hh₄_one⟩ hfold_lt

/-- Non-strict form of the global fold maximum, convenient when only fold-gap
negativity is needed downstream. -/

```

LeanSuffixAnalytic.equalAreaRoot_continuousOn_of_compact_unique

Classification: main

Source: EqualAreaEnvelope.lean:71-107

Declaration SHA-256: 58b73f54ea82a27e264112e64332402373c869c542bc7277160adca9dcc99fdd

Mathematical statement. Fix $\lambda > 1$, a set S contained in $(0,1]$, a compact set K contained in $(0,1)$, and a function r from the reals to the reals. If, for every t in S , $r(t)$ belongs to K and is the unique y in K satisfying $A_3(y) = A_4(t)$, then r is continuous on S . Neither S nor K is required to be an interval.

Let $\lambda > 1$, $S \subseteq (0,1]$, let $K \subseteq (0,1)$ be compact, and let $r: \mathbb{R} \rightarrow \mathbb{R}$. Assume that for every $t \in S$, $r(t) \in K$ and $A_3(r(t)) = A_4(t)$, and that every $y \in K$ satisfying $A_3(y) = A_4(t)$ equals $r(t)$. Then $r|_S$ is continuous in the relative topology. Continuity of the defining areas gives a closed graph $G = \{(t,y) \in S \times K: A_3(y) = A_4(t)\}$. For each closed $C \subseteq K$, compactness of K makes the projection of $G \cap (S \times C)$ closed in S ; uniqueness identifies this projection with $\{t \in S: r(t) \in C\}$. Thus the selected root is

continuous as a map into K , and hence into $\mathbb{R}$. No interval or connectedness hypothesis is imposed on K or S , and uniqueness is required only within K , not among all regular type-(iii) parameters. The source set S may include $t = 1$; only the selected-root set K must stay inside $(0, 1)$.

```

theorem equalAreaRoot_continuousOn_of_compact_unique
  {lam : ℝ} {s K : Set ℝ} {root : ℝ → ℝ}
  (hlam : 1 < lam)
  (hK : IsCompact K)
  (hs_domain : s ⊆ Ioc 0 1)
  (hK_regular : K ⊆ Ioo 0 1)
  (hroot_mem : MapsTo root s K)
  (hequalArea : ∀ h ∈ s,
    typeThreeArea lam (root h) = typeFourArea lam h)
  (hunique : ∀ h ∈ s, ∀ y ∈ K,
    typeThreeArea lam y = typeFourArea lam h → y = root h) :
  ContinuousOn root s := by
let _ : CompactSpace K := isCompact_iff_compactSpace.mp hK
let r : s → K := hroot_mem.restrict root s K
have hA3 : ContinuousOn (typeThreeArea lam) K := by
  intro y hy
  exact (typeThree_variational_hasDerivAt hlam
    (hK_regular hy).1 (hK_regular hy).2).1.continuousAt.continuousWithinAt
have hA4 : ContinuousOn (typeFourArea lam) s := by
  intro h hh
  have hh0 : h ≠ 0 := ne_of_gt (hs_domain hh).1
  unfold typeFourArea typeFourAngle typeFourDelta
  fun_prop (disch := simp_all)
have hr : Continuous r := continuous_of_compact_unique_eq
  (F := fun h : s => fun y : K =>
    typeThreeArea lam y - typeFourArea lam h)
  (z := 0)
  ((hA3.domRestrict.comp continuous_snd).sub
    (hA4.domRestrict.comp continuous_fst))
  (fun h => sub_eq_zero.mpr (hequalArea h h.property))
  (fun h y hy => Subtype.ext
    (hunique h h.property y y.property (sub_eq_zero.mp hy)))
rw [continuousOn_iff_continuous_domRestrict]
exact continuous_subtype_val.comp hr

/-- The type-(iii) area is strictly decreasing on a compact regular interval
when its derivative is negative throughout the interval's interior. -/

```

LeanSuffixAnalytic.equalAreaRoot_hasDerivAt

Classification: main

Source: EqualAreaEnvelope.lean:165-183

Declaration SHA-256: 4240cf112f1c537233001e48d7956d1d7de811e741deeb8b94332f375fdbb1f0

Mathematical statement. A continuous simple equal-area root is differentiable at an interior curvature, without requiring it to be a descending root.

Assume $\lambda > 1$, $0 < t < 1$, $0 < r(t) < 1$, continuity of $r: \mathbb{R} \rightarrow \mathbb{R}$ at t , and $A_3'(r(t)) \neq 0$. Suppose also that $A_3(r(x)) = A_4(x)$ for every $0 < x \leq 1$. Then

$$r'(t) = \frac{A_4'(t)}{A_3'(r(t))}.$$

Local inversion of the nonvanishing type-(iii) area derivative gives the result. The theorem assumes neither a negative derivative nor global root uniqueness; descending-branch negativity is the application's way to discharge the nonzero-derivative hypothesis.

```

theorem equalAreaRoot_hasDerivAt
  {lam h : ℝ} {root : ℝ → ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1)

```

```

(hroot_pos : 0 < root h) (hroot_lt_one : root h < 1)
(hroot_continuous : ContinuousAt root h)
(hareaThree_ne : deriv (typeThreeArea lam) (root h) ≠ 0)
(hequalArea : ∀ {x : ℝ}, 0 < x → x ≤ 1 →
  typeThreeArea lam (root x) = typeFourArea lam x) :
HasDerivAt root
  (deriv (typeFourArea lam) h /
    deriv (typeThreeArea lam) (root h)) h := by
have hareaThree :=
  (typeThree_variational_hasDerivAt hlam hroot_pos hroot_lt_one).1
have hareaFour := (typeFour_variational_hasDerivAt hlam hh hh1).1
apply hareaThree.of_comp_left hroot_continuous hareaFour hareaThree_ne
filter_upwards [Ioo_mem_nhds hh hh1] with x hx
exact hequalArea hx.1 hx.2.le
/-- Differentiating the equal-area constraint and applying the exact
type-(iii) and type-(iv) variational identities gives the envelope identity. -/

```

LeanSuffixAnalytic.equalArea_gap_identity

Classification: main

Source: LeanSuffixAnalytic.lean:1208-1220

Declaration SHA-256: 7a449a0d73201ece195743b096dbde5032c8598ab1748985129132f301a8ba9d

Mathematical statement. Exact perimeter-gap elimination under equal area and nonzero curvatures.

$$P_3(s) - P_4(t) = 4[(s - t)/t^2 B_4(t) + (s/t)D_4(t) - D_3(s)].$$

```

theorem equalArea_gap_identity {lam h3 h4 : ℝ}
  (hh3 : h3 ≠ 0) (hh4 : h4 ≠ 0)
  (harea : typeThreeArea lam h3 = typeFourArea lam h4) :
  typeThreePerimeter lam h3 - typeFourPerimeter lam h4 =
    4 * ((h3 - h4) / h4 ^ 2) * typeFourAngle lam h4 +
    h3 / h4 * typeFourDelta lam h4 - typeThreeDelta lam h3) := by
  apply equalArea_gap_algebra hh3 hh4
  unfold typeThreeArea typeFourArea typeThreeShape at harea
  convert harea using 1
  all_goals ring

/-- At a stationary type-(iv) fold, the exact perimeter gap is four times the
single reduced radical expression. -/

```

LeanSuffixAnalytic.hasDerivAt_typeFourArea

Classification: main

Source: LeanSuffixAnalytic.lean:496-553

Declaration SHA-256: 406477265a826e873557d3950f02d636becf1dd68c773947e69154fec9631e9e

Mathematical statement. The type-(iv) area derivative is K^4/h cubed.

This exact identity links stationary folds and area monotonicity for $\lambda > 1, 0 < h < 1$.

```

theorem hasDerivAt_typeFourArea {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeFourArea lam) (typeFourFold lam h / h ^ 3) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hh_ne : h ≠ 0 := ne_of_gt hh
  have hu_arg : 0 < 1 - h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - h ^ 2 := by nlinarith
  have hu_ne : sqrt (1 - h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hu_arg)
  have hd_ne : sqrt (lam ^ 2 - h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  have hangle := hasDerivAt_typeFourAngle hlam hh hh1
  have hdelta := hasDerivAt_typeFourDelta hlam hh hh1

```



```

have hdelta_key :
  h * (h / sqrt (1 - h ^ 2) -
    h / (lam * sqrt (lam ^ 2 - h ^ 2))) -
    typeFourDelta lam h =
  1 / sqrt (1 - h ^ 2) -
    lam / sqrt (lam ^ 2 - h ^ 2) := by
  unfold typeFourDelta
  calc
    h * (h / sqrt (1 - h ^ 2) -
      h / (lam * sqrt (lam ^ 2 - h ^ 2))) -
      (sqrt (lam ^ 2 - h ^ 2) / lam - sqrt (1 - h ^ 2)) =
      (h ^ 2 + sqrt (1 - h ^ 2) ^ 2) / sqrt (1 - h ^ 2) -
        (h ^ 2 + sqrt (lam ^ 2 - h ^ 2) ^ 2) /
          (lam * sqrt (lam ^ 2 - h ^ 2)) := by
      field_simp [hlam_ne, hu_ne, hd_ne]
      ring
    _ = 1 / sqrt (1 - h ^ 2) -
      lam / sqrt (lam ^ 2 - h ^ 2) := by
      rw [Real.sq_sqrt hu_arg.le, Real.sq_sqrt hd_arg.le]
      field_simp [hlam_ne, hu_ne, hd_ne]
      ring
have hquotient_identity (B D p dp : ℝ) (hk : h * dp - D = p) :
  (2 * (p + (D + h * dp)) * h ^ 2 -
    2 * (B + h * D) * (2 * h)) / (h ^ 2) ^ 2 =
    4 * (h * p - B) / h ^ 3 := by
  rw [← hk]
  field_simp [hh_ne]
  ring
have hnumerator :=
  (hangle.add ((hasDerivAt_id h).mul hdelta)).const_mul 2
have hraw := hnumerator.div ((hasDerivAt_id h).pow 2)
  (pow_ne_zero 2 hh_ne)
simp only [typeFourArea, typeFourFold, id_eq] using! hraw.congr_deriv (by
  norm_num [id_eq, Pi.add_apply, Pi.mul_apply, Pi.pow_apply]
  exact hquotient_identity
    (typeFourAngle lam h) (typeFourDelta lam h)
    ((sqrt (1 - h ^ 2))⁻¹ -
      lam / sqrt (lam ^ 2 - h ^ 2))
    (h / sqrt (1 - h ^ 2) -
      h / (lam * sqrt (lam ^ 2 - h ^ 2)))
    (by simp only [one_div] using hdelta_key))

/-- The regular type-(iii) area and perimeter are differentiable, and their
derivatives satisfy the exact variational identity `P₃' = h A₃'`. -/

```

LeanSuffixAnalytic.hasDerivAt_typeFourFold

Classification: main

Source: LeanSuffixAnalytic.lean:457-495

Declaration SHA-256: df765bf2b71951f20ce86d7897c71befd43183478a79e7706b6061a82ade97f0

Mathematical statement. K_4 has derivative four h squared times the reciprocal-cube radical difference.

$$K_4'(h) = 4h^2[(1 - h^2)^{-3/2} - \lambda(\lambda^2 - h^2)^{-3/2}] > 0.$$

```

theorem hasDerivAt_typeFourFold {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeFourFold lam)
    (4 * h ^ 2 * (1 / sqrt (1 - h ^ 2) ^ 3 -
      lam / sqrt (lam ^ 2 - h ^ 2) ^ 3)) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hu_arg : 0 < 1 - h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - h ^ 2 := by nlinarith
  have hu_ne : sqrt (1 - h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hu_arg)
  have hd_ne : sqrt (lam ^ 2 - h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  have hunit :
    HasDerivAt (fun x : ℝ => 1 / sqrt (1 - x ^ 2))

```

```

      (h / sqrt (1 - h ^ 2) ^ 3) h := by
have hraw := (((hasDerivAt_const h 1).sub
  ((hasDerivAt_id h).pow 2)).sqrt (ne_of_gt hu_arg)).inv hu_ne)
simp only [one_div] using! hraw.congr_deriv (by
  simp only [id_eq, Pi.sub_apply, Pi.pow_apply]
  field_simp [hu_ne]
  ring)
have hscaled :
  HasDerivAt (fun x : ℝ => lam / sqrt (lam ^ 2 - x ^ 2))
    (lam * h / sqrt (lam ^ 2 - h ^ 2) ^ 3) h := by
have hraw := (((hasDerivAt_const h (lam ^ 2)).sub
  ((hasDerivAt_id h).pow 2)).sqrt (ne_of_gt hd_arg)).inv hd_ne).const_mul lam)
simp only [div_eq_mul_inv] using! hraw.congr_deriv (by
  simp only [id_eq, Pi.sub_apply, Pi.pow_apply]
  field_simp [hd_ne]
  ring)
have hangle := hasDerivAt_typeFourAngle hlam hh hh1
have hraw :=
  (((hasDerivAt_id h).mul (hunit.sub hscaled)).sub hangle).const_mul 4
simp only [typeFourFold, id_eq] using! hraw.congr_deriv (by
  simp only [id_eq, Pi.sub_apply]
  ring)

/-- Exact derivative of the regular type-(iv) weighted area on its natural
open domain. Its numerator is precisely the stationary-fold function. -/

```

LeanSuffixAnalytic.hasDerivAt_typeFourPerimeter

Classification: main

Source: LeanSuffixAnalytic.lean:761-767

Declaration SHA-256: fe6311de3122c760c199f2555194ef9855f7218b10bcc5afe2c6f598e18071ca

Mathematical statement. The type-(iv) perimeter derivative is K_4/h squared.

This identity supplies both centered perimeter enclosures and the equal-area envelope derivative.

```

theorem hasDerivAt_typeFourPerimeter {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeFourPerimeter lam) (typeFourFold lam h / h ^ 2) h := by
refine (typeFour_variational_hasDerivAt hlam hh hh1).2.congr_deriv ?_
rw [(hasDerivAt_typeFourArea hlam hh hh1).deriv]
field_simp [ne_of_gt hh]

```

LeanSuffixAnalytic.hasDerivAt_typeThreeArea

Classification: main

Source: LeanSuffixAnalytic.lean:700-724

Declaration SHA-256: 7e0c0da90901c546a8edb152b4ed9a40769e6bd51ef8318f50390135bcace455

Mathematical statement. The type-(iii) area derivative is K_3/h cubed.

For positive curvature the derivative has exactly the sign of K_3 .

```

theorem hasDerivAt_typeThreeArea {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeArea lam) (typeThreeFold lam h / h ^ 3) h := by
refine (typeThree_variational_hasDerivAt hlam hh hh1).1.congr_deriv ?_
rw [typeThreeArea_deriv_eq_explicit hlam hh hh1]
have hlam_ne : lam ≠ 0 := ne_of_gt (lt_trans (by norm_num) hlam)
have hq_arg : 0 < 1 - typeThreeShape h ^ 2 := by
  unfold typeThreeShape
  nlinarith
have hd_arg : 0 < lam ^ 2 - typeThreeShape h ^ 2 := by
  have hlam_sq : 1 < lam ^ 2 := by nlinarith
  have hq_sq : typeThreeShape h ^ 2 < 1 := by

```

```

    unfold typeThreeShape
    nlinarith
  linarith
have hu_ne : sqrt (1 - typeThreeShape h ^ 2) ≠ 0 :=
  ne_of_gt (Real.sqrt_pos.2 hq_arg)
have hd_ne : sqrt (lam ^ 2 - typeThreeShape h ^ 2) ≠ 0 :=
  ne_of_gt (Real.sqrt_pos.2 hd_arg)
unfold typeThreeFold
field_simp [hlam_ne, hu_ne, hd_ne]
ring

/-- The same type-(iii) fold numerator controls the perimeter derivative,
with one fewer power of the shared height. -/

```

LeanSuffixAnalytic.hasDerivAt_typeThreeFold

Classification: main

Source: LeanSuffixAnalytic.lean:375-456

Declaration SHA-256: c8c79d60a2ba5a7f202a48aa5b5851e2b132f448f609c9f5925a2a8edab62ee5

Mathematical statement. K3 has derivative 16h squared times the reciprocal-cube radical difference at q.

$K_3'(h) = 16h^2[(1 - q^2)^{-3/2} - \lambda(\lambda^2 - q^2)^{-3/2}]$ on $0 < h < 1$.

```

theorem hasDerivAt_typeThreeFold {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeFold lam)
    (16 * h ^ 2 * (1 / sqrt (1 - typeThreeShape h ^ 2) ^ 3 -
      lam / sqrt (lam ^ 2 - typeThreeShape h ^ 2) ^ 3)) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hq_low : -1 < typeThreeShape h := by
    unfold typeThreeShape
    linarith
  have hq_high : typeThreeShape h < 1 := by
    unfold typeThreeShape
    linarith
  have hu_arg : 0 < 1 - typeThreeShape h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - typeThreeShape h ^ 2 := by nlinarith
  have hu_ne : sqrt (1 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hu_arg)
  have hd_ne : sqrt (lam ^ 2 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  let U := fun x : ℝ => 1 / sqrt (1 - typeThreeShape x ^ 2)
  let V := fun x : ℝ => (1 / sqrt (lam ^ 2 - typeThreeShape x ^ 2)) / lam
  have hshape := hasDerivAt_typeThreeShape h
  have hunitBase :
    HasDerivAt (fun x : ℝ => 1 / sqrt (1 - x ^ 2))
      (typeThreeShape h / sqrt (1 - typeThreeShape h ^ 2) ^ 3)
      (typeThreeShape h) := by
  have hraw := (((hasDerivAt_const (typeThreeShape h) 1).sub
    ((hasDerivAt_id (typeThreeShape h)).pow 2)).sqrt
    (ne_of_gt hu_arg)).inv hu_ne)
  simpa only [one_div] using! hraw.congr_deriv (by
    simpa only [id_eq, Pi.sub_apply, Pi.pow_apply]
    field_simp [hu_ne]
    ring)
  have houterBase :
    HasDerivAt (fun x : ℝ => 1 / sqrt (lam ^ 2 - x ^ 2))
      (typeThreeShape h / sqrt (lam ^ 2 - typeThreeShape h ^ 2) ^ 3)
      (typeThreeShape h) := by
  have hraw := (((hasDerivAt_const (typeThreeShape h) (lam ^ 2)).sub
    ((hasDerivAt_id (typeThreeShape h)).pow 2)).sqrt
    (ne_of_gt hd_arg)).inv hd_ne)
  simpa only [one_div] using! hraw.congr_deriv (by
    simpa only [id_eq, Pi.sub_apply, Pi.pow_apply]
    field_simp [hd_ne]
    ring)
  have hunit :

```

```

HasDerivAt U
(2 * typeThreeShape h / sqrt (1 - typeThreeShape h ^ 2) ^ 3) h := by
  simpa only [U, Function.comp_apply] using!
  (hunitBase.comp h hshape).congr_deriv (by ring)
have hscaled :
  HasDerivAt V
  (2 * typeThreeShape h /
   (lam * sqrt (lam ^ 2 - typeThreeShape h ^ 2) ^ 3)) h := by
  simpa only [V, Function.comp_apply] using!
  ((houterBase.comp h hshape).div_const lam).congr_deriv (by
    field_simp [hlam_ne])
have hA :=
  (((hasDerivAt_const h 1).add hshape).mul hunit).sub
  (((hasDerivAt_const h (lam ^ 2)).add hshape).mul hscaled))
have hangle := hasDerivAt_typeThreeAngle hlam hh hh1
have hdelta := hasDerivAt_typeThreeDelta hlam hh hh1
have hraw :=
  (((hasDerivAt_id h).mul hA).const_mul 4).sub
  (((hangle.add_const (π / 2)).add hdelta).const_mul 2)
have hderiv : HasDerivAt _
  (16 * h ^ 2 * (1 / sqrt (1 - typeThreeShape h ^ 2) ^ 3 -
   lam / sqrt (lam ^ 2 - typeThreeShape h ^ 2) ^ 3)) h :=
  hraw.congr_deriv (by
    simp only [Pi.add_apply, Pi.sub_apply, Pi.mul_apply, zero_add]
    dsimp only [U, V]
    simp only [id_eq]
    field_simp [hlam_ne, hu_ne, hd_ne]
    rw [Real.sq_sqrt hu_arg.le, Real.sq_sqrt hd_arg.le]
    unfold typeThreeShape
    ring)
apply hderiv.congr_of_eventuallyEq
filter_upwards [] with x
simp only [typeThreeFold, U, V, id_eq, Pi.add_apply, Pi.sub_apply,
  Pi.mul_apply]
field_simp [hlam_ne]
/-- Exact derivative of the regular type-(iv) stationary-fold function on its
natural open domain. -/

```

LeanSuffixAnalytic.hasDerivAt_typeThreePerimeter

Classification: main

Source: LeanSuffixAnalytic.lean:725-732

Declaration SHA-256: bd5364506b45967c0ea0097c8997e2b79343b02b6ddca131541b965a63dfb88f

Mathematical statement. The type-(iii) perimeter derivative is K_3/h squared.

Together with $A_3 = K_3/h^3$, this is the exact variational relation.

```

theorem hasDerivAt_typeThreePerimeter {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreePerimeter lam) (typeThreeFold lam h / h ^ 2) h := by
  refine (typeThree_variational_hasDerivAt hlam hh hh1).2.congr_deriv ?_
  rw [(hasDerivAt_typeThreeArea hlam hh hh1).deriv]
  field_simp [ne_of_gt hh]
/-- The regular type-(iv) area and perimeter are differentiable, and their
derivatives satisfy the exact variational identity `P₄' = h A₄'`. -/

```

LeanSuffixAnalytic.radicalCubeGap_pos

Classification: main

Source: LeanSuffixAnalytic.lean:349-374

Declaration SHA-256: ed04fbfe0d2a5c8c68ae2429d8da8593a273f2471a8fcc329e85c97a6780176d

Mathematical statement. The reciprocal-cube radical difference is strictly positive for $\lambda > 1$ and $-1 < x < 1$, including negative x and $x = 0$.

For $\lambda > 1$ and $-1 < x < 1$, both radicals are positive and $(1 - x^2)^{-3/2} - \lambda(\lambda^2 - x^2)^{-3/2} > 0$. The expression depends only on x^2 , so the negative-input case follows from the positive-input case by replacing x with $|x|$. At $x = 0$ it is $1 - \lambda^{-2} > 0$. This full shape domain is needed for type-(iii) fold monotonicity, where $x = 2h - 1$ ranges over $(-1, 1)$ as $0 < h < 1$.

```
theorem radicalCubeGap_pos {lam x : ℝ}
  (hlam : 1 < lam) (hx : -1 < x) (hx1 : x < 1) :
  0 < 1 / sqrt (1 - x ^ 2) ^ 3 -
  lam / sqrt (lam ^ 2 - x ^ 2) ^ 3 := by
  rcases lt_trichotomy x 0 with hxneg | rfl | hxpos
  · have habs_pos : 0 < |x| := abs_pos.mpr (ne_of_lt hxneg)
    have habs_lt_one : |x| < 1 := (abs_lt).2 ⟨hx, hx1⟩
    simpa only [sq_abs] using
      radicalCubeGap_pos_of_pos hlam habs_pos habs_lt_one
  · have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
    have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
    have hlam_sq : 1 < lam ^ 2 := by nlinarith
    have hsqrt : sqrt (lam ^ 2) = lam := by
      rw [Real.sqrt_sq_eq_abs, abs_of_pos hlam_pos]
    simp only [zero_pow (show (2 : ℕ) ≠ 0 by norm_num), sub_zero,
      Real.sqrt_one, one_pow]
    rw [hsqrt]
    have hquot : lam / lam ^ 3 = 1 / lam ^ 2 := by
      field_simp [hlam_ne]
    rw [hquot, sub_pos]
    simpa only [div_one] using
      (div_lt_one (sq_pos_of_pos hlam_pos)).2 hlam_sq
  · exact radicalCubeGap_pos_of_pos hlam hxpos hx1

/-- Exact derivative of the regular type-(iii) stationary-fold function on its
natural open domain. -/
```

LeanSuffixAnalytic.reducedFoldGap

Classification: main

Source: LeanSuffixAnalytic.lean:83-92

Declaration SHA-256: 90c9f564cc40a2e05814e95c52b8ea113ecc45335e828ff2a54c8e9f2caf08b3

Mathematical statement. The radical expression obtained after equal-area and fold elimination.

For curvatures s, t , it is $(s - t)U(t)/t + sD_4(t)/t - D_3(s)$, where $U(t) = 1/\sqrt{1 - t^2} - \lambda/\sqrt{\lambda^2 - t^2}$.

```
def reducedFoldGap (lam h3 h4 : ℝ) : ℝ :=
  (h3 - h4) / h4 *
  (1 / sqrt (1 - h4 ^ 2) - lam / sqrt (lam ^ 2 - h4 ^ 2)) +
  h3 / h4 * typeFourDelta lam h4 - typeThreeDelta lam h3

/-- Correlation-preserving centered mean-value identity. Unlike a bound
assumed as part of a certificate schema, this produces the intermediate point
from the mean value theorem. Every occurrence in `f' ξ` therefore shares the
same `ξ`, which is the analytic justification for evaluating a derivative
expression once on the whole height interval. -/
```

LeanSuffixAnalytic.stationaryEqualArea_gap_identity

Classification: main

Source: LeanSuffixAnalytic.lean:1221-1238

Declaration SHA-256: 7ca190ef8d3e84eb702882d4666162a0576db626556bb2591b0ba437f522132f

Mathematical statement. At a stationary equal-area pair the perimeter gap is four times the reduced radical gap.

Use $B_4(t) = tU(t)$ from $K_4(t) = 0$ in the equal-area elimination formula.

```

theorem stationaryEqualArea_gap_identity {lam h3 h4 : ℝ}
  (hh3 : h3 ≠ 0) (hh4 : h4 ≠ 0)
  (harea : typeThreeArea lam h3 = typeFourArea lam h4)
  (hfold : typeFourFold lam h4 = 0) :
  typeThreePerimeter lam h3 - typeFourPerimeter lam h4 =
    4 * reducedFoldGap lam h3 h4 := by
  rw [equalArea_gap_identity hh3 hh4 harea]
  unfold typeFourFold at hfold
  unfold reducedFoldGap
  have hangle : typeFourAngle lam h4 =
    h4 * (1 / sqrt (1 - h4 ^ 2) -
      lam / sqrt (lam ^ 2 - h4 ^ 2)) := by
    linarith
  rw [hangle]
  field_simp [hh4]

/-- Precise remaining scalar obligation for a direct suffix proof: negativity
of the reduced fold gap is equivalent to strict type-(iii) improvement. -/

```

LeanSuffixAnalytic.stationaryEqualArea_typeThree_improves_iff

Classification: main

Source: LeanSuffixAnalytic.lean:1239-1252

Declaration SHA-256: d313a9a9b483a140c805abe01de0a03576005db4ae69b2c417e95c5e91300b63

Mathematical statement. At a nonzero stationary equal-area pair, strict improvement is equivalent to negative reduced gap.

The exact gap identity has the positive factor four, so it preserves strict sign.

```

theorem stationaryEqualArea_typeThree_improves_iff {lam h3 h4 : ℝ}
  (hh3 : h3 ≠ 0) (hh4 : h4 ≠ 0)
  (harea : typeThreeArea lam h3 = typeFourArea lam h4)
  (hfold : typeFourFold lam h4 = 0) :
  typeThreePerimeter lam h3 < typeFourPerimeter lam h4 ↔
    reducedFoldGap lam h3 h4 < 0 := by
  rw [← sub_lt_zero]
  rw [stationaryEqualArea_gap_identity hh3 hh4 harea hfold]
  constructor <,> intro h <,> linarith

/-- Both strict radical inequalities used by the retained fold- and
equal-area-root uniqueness arguments follow uniformly from the suffix domain;
they do not require interval cells. -/

```

LeanSuffixAnalytic.typeFourAngle

Classification: main

Source: LeanSuffixAnalytic.lean:53-56

Declaration SHA-256: 94e6a9f5b491cb7ca4802d89fba1fa7ccb864bbe777d6e41d89b1d6bf892bdd0

Mathematical statement. The type-(iv) angle contribution B₄.

$$B_4(h) = \lambda \arccos(h/\lambda) + \arcsin h.$$

```

def typeFourAngle (lam h : ℝ) : ℝ :=
  lam * arccos (h / lam) + arcsin h

/-- The radial displacement in the exact regular type-(iv) formulas. -/

```

LeanSuffixAnalytic.typeFourArea

Classification: main

Source: LeanSuffixAnalytic.lean:61-64

Declaration SHA-256: 775f7921b66716563dd5e0e4c35562d880f5cef2a71b40cef616cd10eb9488d2

Mathematical statement. The exact scalar type-(iv) weighted area.

$A_4(h) = 2(B_4(h) + hD_4(h))/h^2$, with a finite value at $h = 1$.

```
def typeFourArea (lam h : ℝ) : ℝ :=
  2 * (typeFourAngle lam h + h * typeFourDelta lam h) / h ^ 2

/-- Exact regular type-(iv) weighted perimeter used by the suffix comparison. -/
```

LeanSuffixAnalytic.typeFourArea_deriv_neg_before_zero

Classification: main

Source: LeanSuffixAnalytic.lean:1071-1082

Declaration SHA-256: e504f043a30eb739ae5f3ba9174d0ca2cd6aa51665b8ee6815ad3b1b3ea9932b

Mathematical statement. Type-(iv) area decreases before a regular stationary fold.

For $h < t_*$, strict fold increase gives $K_4(h) < K_4(t_*) = 0$.

```
theorem typeFourArea_deriv_neg_before_zero {lam fold h : ℝ}
  (hlam : 1 < lam) (hfold : fold ∈ Set.Ioo (0 : ℝ) 1)
  (hzero : typeFourFold lam fold = 0)
  (hh : h ∈ Set.Ioo (0 : ℝ) 1) (hlt : h < fold) :
  deriv (typeFourArea lam) h < 0 := by
  rw [(hasDerivAt_typeFourArea hlam hh.1 hh.2).deriv]
  exact div_neg_of_neg_of_pos
    (by simpa [hzero] using typeFourFold_strictMonoOn hlam hh hfold hlt)
    (pow_pos hh.1 3)

/-- After a regular zero of the fold, the type-(iv) weighted area has
strictly positive derivative. -/
```

LeanSuffixAnalytic.typeFourArea_deriv_pos_after_zero

Classification: main

Source: LeanSuffixAnalytic.lean:1083-1094

Declaration SHA-256: 574cf814295ee9aa4cb9406582955f50506dd66296979a4f903f27b9ae8e2667

Mathematical statement. Type-(iv) area increases after a regular stationary fold.

For $t_* < h < 1$, one has $K_4(h) > 0$, hence $A_4(h) > 0$.

```
theorem typeFourArea_deriv_pos_after_zero {lam fold h : ℝ}
  (hlam : 1 < lam) (hfold : fold ∈ Set.Ioo (0 : ℝ) 1)
  (hzero : typeFourFold lam fold = 0)
  (hh : h ∈ Set.Ioo (0 : ℝ) 1) (hlt : fold < h) :
  0 < deriv (typeFourArea lam) h := by
  rw [(hasDerivAt_typeFourArea hlam hh.1 hh.2).deriv]
  exact div_pos
    (by simpa [hzero] using typeFourFold_strictMonoOn hlam hfold hh hlt)
    (pow_pos hh.1 3)

/-- A regular zero of the type-(iv) fold is a global minimum of the
type-(iv) area on  $0 < h \leq 1$ , including the closure endpoint. -/
```

LeanSuffixAnalytic.typeFourArea_fold_le

Classification: main

Source: LeanSuffixAnalytic.lean:1095-1129

Declaration SHA-256: 00808b809e3e67ecf06ff39be8e712a367908c043a9c526b98df7d97ff33901d

Mathematical statement. The stationary type-(iv) fold minimizes A4 on (0,1].

Interior derivative signs prove the minimum; continuity includes $h = 1$.

```
theorem typeFourArea_fold_le {lam fold h : ℝ} (hlam : 1 < lam)
  (hfold : fold ∈ Set.Ioo (0 : ℝ) 1)
  (hzero : typeFourFold lam fold = 0)
  (hh : h ∈ Set.Ioc (0 : ℝ) 1) :
  typeFourArea lam fold ≤ typeFourArea lam h := by
rcases lt_trichotomy h fold with hlt | rfl | hgt
· have hanti : StrictAntiOn (typeFourArea lam) (Set.Icc h fold) := by
  apply strictAntiOn_of_deriv_neg (convex_Icc h fold)
  · intro x hx
    exact (hasDerivAt_typeFourArea hlam
      (hh.1.trans_le hx.1) (hx.2.trans_lt hfold.2)
    ).continuousAt.continuousWithinAt
  · intro x hx
    rw [interior_Icc] at hx
    exact typeFourArea_deriv_neg_before_zero
      hlam hfold hzero
      (hh.1.trans hx.1, hx.2.trans hfold.2) hx.2
  exact (hanti ⟨le_rfl, hlt.le⟩ ⟨hlt.le, le_rfl⟩ hlt).le
· exact le_rfl
· have hmono : StrictMonoOn (typeFourArea lam) (Set.Icc fold h) := by
  apply strictMonoOn_of_deriv_pos (convex_Icc fold h)
  · intro x hx
    have hx0 : x ≠ 0 := ne_of_gt (hfold.1.trans_le hx.1)
    unfold typeFourArea typeFourAngle typeFourDelta
    fun_prop (disch := simp_all)
  · intro x hx
    rw [interior_Icc] at hx
    exact typeFourArea_deriv_pos_after_zero
      hlam hfold hzero
      (hfold.1.trans hx.1, hx.2.trans_le hh.2) hx.1
  exact (hmono ⟨le_rfl, hgt.le⟩ ⟨hgt.le, le_rfl⟩ hgt).le

/-- A stationary equal-area pair with negative type-(iii) fold constructs the
unique descending equal-area root for every regular or endpoint type-(iv)
curvature. -/
```

LeanSuffixAnalytic.typeFourDelta

Classification: main

Source: LeanSuffixAnalytic.lean:57-60

Declaration SHA-256: a4c8b30971f59f8f04626dbddf49af92c98dbfa9126bf3766c0be8b6472cbfb3

Mathematical statement. The type-(iv) radical displacement D4.

$$D_4(h) = \sqrt{\lambda^2 - h^2} / \lambda - \sqrt{1 - h^2}.$$

```
def typeFourDelta (lam h : ℝ) : ℝ :=
  sqrt (lam ^ 2 - h ^ 2) / lam - sqrt (1 - h ^ 2)

/-- Exact regular type-(iv) weighted area used by the suffix comparison. -/
```


LeanSuffixAnalytic.typeFourFold

Classification: main

Source: LeanSuffixAnalytic.lean:78-82

Declaration SHA-256: e492724315b735482cab607f768694fcd028864cb336800b7469d841099293ee

Mathematical statement. The type-(iv) stationary-fold numerator K_4 .

$$K_4 = 4[h(1/\sqrt{1-h^2} - \lambda/\sqrt{\lambda^2-h^2}) - B_4].$$

```
def typeFourFold (lam h : ℝ) : ℝ :=
  4 * (h * (1 / sqrt (1 - h ^ 2) -
    lam / sqrt (lam ^ 2 - h ^ 2)) - typeFourAngle lam h)

/-- The radical expression left after equal-area and fold elimination. -/
```

LeanSuffixAnalytic.typeFourFold_strictMonoOn

Classification: main

Source: LeanSuffixAnalytic.lean:1056-1070

Declaration SHA-256: 16af12c38f3bb1c629d7204f781995879e693ae4c65a0b5d7c3475b2043d0222

Mathematical statement. The type-(iv) fold is strictly increasing on $(0,1)$.

Its derivative is positive for every regular curvature at density $\lambda > 1$.

```
theorem typeFourFold_strictMonoOn {lam : ℝ} (hlam : 1 < lam) :
  StrictMonoOn (typeFourFold lam) (Set.Ioo 0 1) := by
  apply strictMonoOn_of_hasDerivWithinAt_pos (convex_Ioo (0 : ℝ) 1)
  · intro h hh
    exact (hasDerivAt_typeFourFold hlam hh.1 hh.2).continuousAt.continuousWithinAt
  · intro h hh
    rw [interior_Ioo] at hh
    exact (hasDerivAt_typeFourFold hlam hh.1 hh.2).hasDerivWithinAt
  · intro h hh
    rw [interior_Ioo] at hh
    exact mul_pos (mul_pos (by norm_num) (sq_pos_of_pos hh.1))
      (radicalCubeGap_pos hlam (by linarith [hh.1]) hh.2)

/-- Before a regular zero of the fold, the type-(iv) weighted area has
strictly negative derivative. -/
```

LeanSuffixAnalytic.typeFourFold_zero_unique

Classification: main

Source: LeanSuffixAnalytic.lean:1150-1158

Declaration SHA-256: bc70631d3da63af952fb4321a7d0a57845b40dd8854182d958b30d4e8971577c

Mathematical statement. A regular type-(iv) stationary fold is unique.

Strict increase of K_4 prevents two zeros in $(0,1)$.

```
theorem typeFourFold_zero_unique {lam h1 h2 : ℝ}
  (hlam : 1 < lam) (hh1 : h1 ∈ Set.Ioo (0 : ℝ) 1)
  (hh2 : h2 ∈ Set.Ioo (0 : ℝ) 1)
  (hz1 : typeFourFold lam h1 = 0) (hz2 : typeFourFold lam h2 = 0) :
  h1 = h2 := by
  apply (typeFourFold_strictMonoOn hlam).injOn hh1 hh2
  exact hz1.trans hz2.symm

/-- Closed lambda interval covered by the retained scalar suffix evidence. -/
```

LeanSuffixAnalytic.typeFourPerimeter

Classification: main

Source: LeanSuffixAnalytic.lean:65-70

Declaration SHA-256: ad0bebbd79c17351c0f12ce6a7fdae3e930dba33de0077415a6116df1d1e6e25

Mathematical statement. The exact scalar type-(iv) weighted perimeter.

$P_4(h) = 4B_4(h)/h$ for modeled curvatures $0 < h \leq 1$.

```
def typeFourPerimeter (lam h : ℝ) : ℝ :=
  4 * typeFourAngle lam h / h

/-- The checker numerator for the derivative of the regular type-(iii) area.
It is kept as one expression in the shared height argument so interval
evaluation does not decorrelate its repeated occurrences. -/
```

LeanSuffixAnalytic.typeFour_variational_hasDerivAt

Classification: main

Source: LeanSuffixAnalytic.lean:733-753

Declaration SHA-256: be216789e3804b1c530f202d3470319a386e5c80dc03952cfaf3ebc685f27297

Mathematical statement. The type-(iv) profiles are differentiable and satisfy $P_4' = h A_4'$ on $(0,1)$.

The theorem is deliberately restricted away from the closure endpoint $h = 1$.

```
theorem typeFour_variational_hasDerivAt {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeFourArea lam) (deriv (typeFourArea lam) h) h ^
  HasDerivAt (typeFourPerimeter lam)
    (h * deriv (typeFourArea lam) h) h := by
  have hh_ne : h ≠ 0 := ne_of_gt hh
  have hangle := hasDerivAt_typeFourAngle hlam hh hh1
  have hraw := (hangle.const_mul (4 : ℝ)).div (hasDerivAt_id h) hh_ne
  have hperimeter : HasDerivAt (typeFourPerimeter lam)
    ((4 * (1 / sqrt (1 - h ^ 2)) - lam / sqrt (lam ^ 2 - h ^ 2)) * h -
     4 * typeFourAngle lam h) / h ^ 2 h := by
  simp only [typeFourPerimeter, id_eq] using! hraw.congr_deriv (by
    simp only [id_eq]
    ring)
  have harea := hasDerivAt_typeFourArea hlam hh hh1
  refine {harea.congr_deriv harea.deriv.symm, hperimeter.congr_deriv ?_}
  rw [harea.deriv]
  unfold typeFourFold
  field_simp [hh_ne]

/-- Exact type-(iv) variational identity on the regular domain. -/
```

LeanSuffixAnalytic.typeThreeAngle

Classification: main

Source: LeanSuffixAnalytic.lean:35-38

Declaration SHA-256: fb7617b8b745792c6f019f3cbcf3c26d5e54f1fc7dd47bb69cec919c712d110a

Mathematical statement. The analytic type-(iii) angle term before adding $\pi/2$.

It is $\lambda \arccos(q/\lambda) + \arcsin q$; B_3 in the main proof adds $\pi/2$.

```
def typeThreeAngle (lam h : ℝ) : ℝ :=
  lam * arccos (typeThreeShape h / lam) + arcsin (typeThreeShape h)
```

```
-- The radial displacement in the exact type-(iii) formulas. -/
```

LeanSuffixAnalytic.typeThreeArea

Classification: main

Source: LeanSuffixAnalytic.lean:44-48

Declaration SHA-256: 63d13d5879252e16ac6f53a2d4449b4f511e353d6016e98158793484dcfaa8f7

Mathematical statement. The exact scalar type-(iii) weighted area.

$A_3(h) = (B_3(h) + (2h + 1)D_3(h))/h^2$ on the regular domain.

```
def typeThreeArea (lam h : ℝ) : ℝ :=
  (typeThreeAngle lam h + π / 2 +
   (typeThreeShape h + 2) * typeThreeDelta lam h) / h ^ 2

/-- Exact regular type-(iii) weighted perimeter used by the suffix comparison. -/
```

LeanSuffixAnalytic.typeThreeArea_descending_root_existsUnique

Classification: main

Source: LeanSuffixAnalytic.lean:1013-1039

Declaration SHA-256: 15660b04d9615f11056408ab4ae99cb03c7fde3d8d26f5056c89e742987b7edb

Mathematical statement. An area at least a negative-fold anchor area has a unique negative-fold regular root.

Strict decrease to the anchor and divergence at zero give existence; fold monotonicity and Rolle's theorem give global descending-root uniqueness.

```
theorem typeThreeArea_descending_root_existsUnique
  {lam anchor target : ℝ}
  (hlam : 1 < lam) (hanchor : anchor ∈ Set.Ioo (0 : ℝ) 1)
  (hfold : typeThreeFold lam anchor < 0)
  (htarget : typeThreeArea lam anchor ≤ target) :
  ∃! root : ℝ, root ∈ Set.Ioo (0 : ℝ) 1 ∧
    typeThreeArea lam root = target ∧ typeThreeFold lam root < 0 := by
  rcases typeThreeArea_descending_root_exists_below
    hlam hanchor hfold htarget with
    ⟨root, hroot_mem, hroot_area, hroot_fold⟩
  have hroot_regular : root ∈ Set.Ioo (0 : ℝ) 1 :=
    ⟨hroot_mem.1, hroot_mem.2.trans_lt hanchor.2⟩
  refine ⟨root, ⟨hroot_regular, hroot_area, hroot_fold⟩, ?_⟩
  intro y hy
  apply le_antisymm
  · exact typeThreeArea_root_le_of_fold_neg
    (root := y) (y := root) hlam hy.1
    ⟨hroot_regular.1, hroot_regular.2.le⟩ hy.2.2
    (hy.2.1.trans hroot_area.symm)
  · exact typeThreeArea_root_le_of_fold_neg
    (root := root) (y := y) hlam hroot_regular
    ⟨hy.1.1, hy.1.2.le⟩ hroot_fold
    (hroot_area.trans hy.2.1.symm)

/-- Above a fixed equal-area root there is at most one further root in
`0 < h ≤ 1`. Combined with `typeThreeArea_root_le_of_fold_neg`, this classifies
all roots as the certified descending root and at most one upper root. -/
```

LeanSuffixAnalytic.typeThreeArea_descending_root_existsUnique_of_stationary_pair

Classification: main

Source: LeanSuffixAnalytic.lean:1130-1149

Declaration SHA-256: f368bc947db4d0b119fefc4c63c0de6d0d14cb14db7e5a28faa26c535eb880b8

Mathematical statement. A negative-fold stationary equal-area anchor supplies a unique descending root at every modeled type-(iv) area.

The fold minimum ensures $A_4(h) \geq A_3(s^*)$; apply the descending-root existence and uniqueness theorem.

```
theorem typeThreeArea_descending_root_existsUnique_of_stationary_pair
  {lam h3fold h4fold h4 : ℝ}
  (hlam : 1 < lam)
  (hh3fold : h3fold ∈ Set.Ioo (0 : ℝ) 1)
  (hh4fold : h4fold ∈ Set.Ioo (0 : ℝ) 1)
  (hh4 : h4 ∈ Set.Ioc (0 : ℝ) 1)
  (hfoldThree : typeThreeFold lam h3fold < 0)
  (hfoldFour : typeFourFold lam h4fold = 0)
  (hequalFold :
    typeThreeArea lam h3fold = typeFourArea lam h4fold) :
  ∃! root : ℝ, root ∈ Set.Ioo (0 : ℝ) 1 ∧
    typeThreeArea lam root = typeFourArea lam h4 ∧
    typeThreeFold lam root < 0 := by
  apply typeThreeArea_descending_root_existsUnique
  hlam hh3fold hfoldThree
  rw [hequalFold]
  exact typeFourArea_fold_le hlam hh4fold hfoldFour hh4

/-- The regular type-(iv) fold has at most one zero in its admissible height
interval. -/
```

LeanSuffixAnalytic.typeThreeArea_descending_root_lt

Classification: main

Source: SameCurvatureArea.lean:586-622

Declaration SHA-256: f0218743e3c66d2b3ba1e5bdde4d44d6f7c58ed79dc9416fb8fb18d852a14824

Mathematical statement. A regular negative-fold type-(iii) root at a modeled type-(iv) area lies below that type-(iv) curvature.

Under equal area, $K_3(s) < 0$, $0 < s < 1$, and $0 < t \leq 1$, the result is $s < t$; it is the strict sign needed in $\Delta' = (s - t)A_4$.

```
theorem typeThreeArea_descending_root_lt
  {lam root h : ℝ}
  (hlam : 1 < lam)
  (hroot : root ∈ Set.Ioo (0 : ℝ) 1)
  (hh : 0 < h) (hh1 : h ≤ 1)
  (hfold : typeThreeFold lam root < 0)
  (harea : typeThreeArea lam root = typeFourArea lam h) :
  root < h := by
  have hsame := typeThreeArea_lt_typeFourArea hlam hh hh1
  apply lt_of_not_ge
  intro hle
  have hne : h ≠ root := by
    intro heq
    subst root
    linarith
  have hlt : h < root := lt_of_le_of_ne hle hne
  have hanti : StrictAntiOn (typeThreeArea lam) (Set.Icc h root) := by
    apply strictAntiOn_of_deriv_neg (convex_Icc h root)
    · intro y hy
      exact (hasDerivAt_typeThreeArea hlam
        (hh.trans_le hy.1) (hy.2.trans_lt hroot.2))
```

```

    ).continuousAt.continuousWithinAt
  · intro y hy
    rw [interior_Icc] at hy
    have hyreg : y ∈ Set.Ioo (0 : ℝ) 1 :=
      ⟨hh.trans hy.1, hy.2.trans hroot.2⟩
    rw [(hasDerivAt_typeThreeArea hlam hyreg.1 hyreg.2).deriv]
    exact div_neg_of_neg_of_pos
      (lt_trans (typeThreeFold_strictMonoOn hlam hyreg hroot hy.2) hfold)
      (pow_pos hyreg.1 3)
  have hc := hanti
    (show h ∈ Set.Icc h root from ⟨le_rfl, hle⟩)
    (show root ∈ Set.Icc h root from ⟨hle, le_rfl⟩) hlt
  rw [harea] at hc
  linarith
end LeanSuffixAnalytic

```

LeanSuffixAnalytic.typeThreeArea_lt_typeFourArea

Classification: main

Source: SameCurvatureArea.lean:225-265

Declaration SHA-256: 292616e61180882b74b7405ba97fd168e8ed5d3653115a72ad0fa3e78e338f65

Mathematical statement. At the same curvature, type-(iii) scalar area is strictly less than type-(iv) scalar area.

For $\lambda > 1$ and $0 < h \leq 1$, $A_3(h) < A_4(h)$; the full type-(iii) formula is used across $h = 1/2$.

```

theorem typeThreeArea_lt_typeFourArea {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h ≤ 1) :
  typeThreeArea lam h < typeFourArea lam h := by
  rw [(area_normal_forms lam h).1, (area_normal_forms lam h).2]
  apply (div_lt_div_iff_of_pos_right (sq_pos_of_pos hh)).2
  let q := 2 * h - 1
  have hmem : h ∈ Set.Icc (0 : ℝ) 1 := ⟨hh.le, hh1⟩
  have hqb : -1 < q ∧ q ≤ h := by
    dsimp [q]
    constructor <=> linarith
  rcases le_total 0 q with hq | hq
  · have hqmem : q ∈ Set.Icc (0 : ℝ) 1 :=
      ⟨hq, hqb.2.trans hh1⟩
    have hF : angleGap lam q ≤ angleGap lam h :=
      (angleGap_strictMonoOn hlam).monotoneOn hqmem hqb.2
    have hd : typeFourDelta lam q ≤ typeFourDelta lam h :=
      (typeFourDelta_strictMonoOn hlam).monotoneOn hqmem hqb.2
    have hd0 := typeFourDelta_nonneg hlam hmem
    have hk := angleGap_gt_mul_delta hlam hmem
    change π + angleGap lam q + (2 * h + 1) * typeFourDelta lam q <
      π + 2 * angleGap lam h + 2 * h * typeFourDelta lam h
    nlinarith
  · have hxmem : -q ∈ Set.Icc (0 : ℝ) 1 := by
      constructor <=> dsimp [q] at hqb hq ⊢ <=> linarith
    have hz : (0 : ℝ) ∈ Set.Icc (0 : ℝ) 1 := by norm_num
    have hF : angleGap lam 0 < angleGap lam h :=
      angleGap_strictMonoOn hlam hz hmem hh
    have hd0 := typeFourDelta_nonneg hlam hmem
    have hk := angleGap_gt_mul_delta hlam hxmem
    have hsF := angleGap_neg lam (-q)
    have hsd := typeFourDelta_neg lam (-q)
    have hF0 := angleGap_neg lam 0
    simp only [neg_neg] at hsF hsd
    simp only [neg_zero] at hF0
    change π + angleGap lam q + (2 * h + 1) * typeFourDelta lam q <
      π + 2 * angleGap lam h + 2 * h * typeFourDelta lam h
    dsimp [q] at hsF hsd hk ⊢
    nlinarith
/-- The support term whose concavity compares the two regular topologies at
the same curvature. -/

```

LeanSuffixAnalytic.typeThreeArea_root_le_of_fold_neg

Classification: main

Source: LeanSuffixAnalytic.lean:954-973

Declaration SHA-256: a9a71e3bf8960b5c96a8fc59bf3fe2a372f70aa365bd91817e7ca128cb8520b9

Mathematical statement. For $\lambda > 1$, a root r in $(0,1)$ with $K_3(r) < 0$ is no larger than any y in $(0,1]$ with $A_3(y) = A_3(r)$.

Assume $\lambda > 1$, $0 < r < 1$, $0 < y \leq 1$, $K_3(r) < 0$, and $A_3(r) = A_3(y)$. If $y < r$, Rolle's theorem yields $c \in (y, r)$ with $K_3(c) = 0$, contradicting strict increase of K_3 and its negative value at r . Therefore $r \leq y$. The competitor root $y = 1$ is allowed; no negative-fold premise is imposed on y .

```
theorem typeThreeArea_root_le_of_fold_neg {lam root y : ℝ}
  (hlam : 1 < lam) (hroot : root ∈ Set.Ioo (0 : ℝ) 1)
  (hy : y ∈ Set.Ioc (0 : ℝ) 1) (hfold : typeThreeFold lam root < 0)
  (harea : typeThreeArea lam root = typeThreeArea lam y) :
  root ≤ y := by
  apply le_of_not_gt
  intro hyroot
  have hyreg : y ∈ Set.Ioo (0 : ℝ) 1 :=
    ⟨hy.1, hyroot.trans hroot.2⟩
  rcases typeThreeFold_zero_between_of_area_eq
    hlam hyreg ⟨hroot.1, hroot.2.le⟩ hyroot harea.symm with
    ⟨c, hc, hfoldC⟩
  have hcreg : c ∈ Set.Ioo (0 : ℝ) 1 :=
    ⟨hyreg.1.trans hc.1, hc.2.trans hroot.2⟩
  have hmono := typeThreeFold_strictMonoOn hlam hcreg hroot hc.2
  rw [hfoldC] at hmono
  linarith

/-- If one regular point on the descending type-(iii) branch lies below a
target area, the target is attained before that point. -/
```

LeanSuffixAnalytic.typeThreeArea_strictAntiOn_Icc

Classification: main

Source: EqualAreaEnvelope.lean:108-123

Declaration SHA-256: fd210f8a463759cd45da345eae2826fb3a7bdf59d29239656d68c8efc64fb7fc

Mathematical statement. For $\lambda > 1$, $a > 0$ and $b < 1$, negativity of A_3 derivative on (a,b) implies strict decrease of A_3 on $[a,b]$.

Assume $\lambda > 1$, $0 < a, b < 1$, and $A_3'(y) < 0$ for every $y \in (a, b)$. Then for $x, y \in [a, b]$ with $x < y$, $A_3(y) < A_3(x)$. Continuity on the closed interval and interior derivative negativity prove this by the mean value theorem. No derivative or fold sign is assumed at the endpoints, and no separate $a \leq b$ premise is needed: the empty and singleton interval cases are vacuous.

```
theorem typeThreeArea_strictAntiOn_Icc
  {lam a b : ℝ}
  (hlam : 1 < lam) (ha : 0 < a) (hb : b < 1)
  (hderiv : ∀ y ∈ Ioo a b,
    deriv (typeThreeArea lam) y < 0) :
  StrictAntiOn (typeThreeArea lam) (Icc a b) := by
  apply strictAntiOn_of_deriv_neg (convex_Icc a b)
  · intro y hy
    exact (typeThree_variational_hasDerivAt hlam
      (ha.trans_le hy.1) (hy.2.trans_lt hb)).1.continuousAt.continuousWithinAt
  · intro y hy
    rw [interior_Icc] at hy
    exact hderiv y hy

/-- A negative fold at the right endpoint forces strict decrease of the
type-(iii) area on the entire preceding regular interval. -/
```

LeanSuffixAnalytic.typeThreeArea_strictAntiOn_Icc_of_fold_neg_right

Classification: main

Source: EqualAreaEnvelope.lean:124-138

Declaration SHA-256: 1b1d7870499bdaf74cbcd036814fc1cce139b4cf2463cce4ba39b71ae55024b7

Mathematical statement. A negative fold at a slab's right endpoint suffices for strict area decrease throughout it.

Strict fold increase makes every earlier fold value negative.

```
theorem typeThreeArea_strictAntiOn_Icc_of_fold_neg_right
  {lam a b : ℝ} (hlam : 1 < lam) (ha : 0 < a) (hb : b < 1)
  (hb_pos : 0 < b) (hfoldb : typeThreeFold lam b < 0) :
  StrictAntiOn (typeThreeArea lam) (Icc a b) := by
  apply typeThreeArea_strictAntiOn_Icc hlam ha hb
  intro y hy
  rw [(hasDerivAt_typeThreeArea hlam
    (ha.trans hy.1) (hy.2.trans hb)).deriv]
  apply div_neg_of_neg_of_pos _ (pow_pos (ha.trans hy.1) 3)
  exact (typeThreeFold_strictMonoOn hlam
    (ha.trans hy.1, hy.2.trans hb) (hb_pos, hb) hy.2).trans hfoldb

/-- Certificate-facing specialization for a root contained in a compact
curvature interval. Uniform derivative negativity supplies both uniqueness
and continuity of the selected equal-area branch. -/
```

LeanSuffixAnalytic.typeThreeArea_tendsto_atRight_zero

Classification: main

Source: LeanSuffixAnalytic.lean:828-871

Declaration SHA-256: e82ffa97f750ea7c64d3d22eebba58e03fe2fc0f731297b9787ae711ded5a49a

Mathematical statement. Type-(iii) area tends to positive infinity as curvature decreases to zero.

For each $\lambda > 1$, the exact positive asymptotic numerator forces $A_3(h) \rightarrow +\infty$ as $h \downarrow 0$.

```
theorem typeThreeArea_tendsto_atRight_zero {lam : ℝ} (hlam : 1 < lam) :
  Tendsto (typeThreeArea lam) (ℳ[>] 0) atTop := by
  let numerator : ℝ → ℝ := fun h =>
    typeThreeAngle lam h + π / 2 +
    (typeThreeShape h + 2) * typeThreeDelta lam h
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hrad : 0 < lam ^ 2 - 1 := by nlinarith
  have hnum_cont : ContinuousAt numerator 0 := by
    dsimp [numerator, typeThreeAngle, typeThreeDelta, typeThreeShape]
    fun_prop
  have hnum_zero : numerator 0 =
    lam * arccos (-1 / lam) + sqrt (lam ^ 2 - 1) / lam := by
    simp [numerator, typeThreeAngle, typeThreeDelta, typeThreeShape]
    ring
  have hC : 0 < numerator 0 := by
    rw [hnum_zero]
    exact add_pos_of_nonneg_of_pos
      (mul_nonneg hlam_pos.le (Real.arccos_nonneg _))
      (div_pos (Real.sqrt_pos.2 hrad) hlam_pos)
  have hnum_tendsto : Tendsto numerator (ℳ[>] (0 : ℝ)) (ℳ (numerator 0)) :=
    hnum_cont.tendsto.mono_left inf_le_left
  have hnum_lb : ∀f h in ℳ[>] (0 : ℝ), numerator 0 / 2 < numerator h :=
    hnum_tendsto.eventually_const_lt (by linarith)
  have hsmall : ∀f h in ℳ[>] (0 : ℝ), h < 1 :=
    (eventually_lt_nhds (show (0 : ℝ) < 1 by norm_num)).filter_mono inf_le_left
  have hlower : Tendsto (fun h : ℝ => numerator 0 / 2 * h-1)
    (ℳ[>] 0) atTop :=
    tendsto_inv_nhdsGT_zero.const_mul_atTop (by linarith)
  apply tendsto_atTop_mono' (ℳ[>] (0 : ℝ)) _ hlower
  filter_upwards [hnum_lb, self_mem_nhdsWithin, hsmall] with h hnum hh hlt
```

```

have hhpos : 0 < h := hh
have hsq_pos : 0 < h ^ 2 := sq_pos_of_pos hhpos
have hsquare_le : h ^ 2 ≤ h := by nlinarith
calc
  numerator 0 / 2 * h-1 = numerator 0 / 2 / h := by
    simp only [div_eq_mul_inv]
  _ ≤ numerator 0 / 2 / h ^ 2 :=
    div_le_div_of_nonneg_left (by linarith) hsq_pos hsquare_le
  _ ≤ numerator h / h ^ 2 :=
    div_le_div_of (by linarith) hnum.le hsq_pos le_rfl
  _ = typeThreeArea lam h := by rfl

/-- For `lam > 1`, the regular type-(iii) fold is strictly increasing
throughout the admissible height interval. -/

```

LeanSuffixAnalytic.typeThreeDelta

Classification: main

Source: LeanSuffixAnalytic.lean:39-43

Declaration SHA-256: f96722ed089f4e5911c6013a0aa601eb4080090036491702cd0fa78e3e31113f

Mathematical statement. The type-(iii) radical displacement D_3 .

$$D_3 = \sqrt{\lambda^2 - q^2} / \lambda - \sqrt{1 - q^2}.$$

```

def typeThreeDelta (lam h : ℝ) : ℝ :=
  sqrt (lam ^ 2 - typeThreeShape h ^ 2) / lam -
  sqrt (1 - typeThreeShape h ^ 2)

/-- Exact regular type-(iii) weighted area used by the suffix comparison. -/

```

LeanSuffixAnalytic.typeThreeFold

Classification: main

Source: LeanSuffixAnalytic.lean:71-77

Declaration SHA-256: de466d8508a0def3484419e5a98402f15940017ff9d116432fb67c21c1d22c86

Mathematical statement. The explicit numerator K_3 controlling the type-(iii) area derivative.

$$K_3 = 4h((1 + q)/\sqrt{1 - q^2} - (\lambda^2 + q)/(\lambda\sqrt{\lambda^2 - q^2})) - 2(B_3 + D_3).$$

```

def typeThreeFold (lam h : ℝ) : ℝ :=
  4 * h * ((1 + typeThreeShape h) / sqrt (1 - typeThreeShape h ^ 2) -
    (lam ^ 2 + typeThreeShape h) /
    (lam * sqrt (lam ^ 2 - typeThreeShape h ^ 2))) -
  2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h)

/-- The stationary-fold equation for the regular type-(iv) profile. -/

```

LeanSuffixAnalytic.typeThreeFold_strictMonoOn

Classification: main

Source: LeanSuffixAnalytic.lean:872-893

Declaration SHA-256: 548db5145f9bcf2011e54d3648e735633b97c51f2514a21b4368c3304bb06a32

Mathematical statement. The type-(iii) fold is strictly increasing on $(0,1)$.

The derivative formula and positive reciprocal-cube radical gap give strict monotonicity, including across $h = 1/2$.

```

theorem typeThreeFold_strictMonoOn {lam : ℝ} (hlam : 1 < lam) :
  StrictMonoOn (typeThreeFold lam) (Set.Ioo 0 1) := by

```



```

apply strictMonoOn_of_hasDerivWithinAt_pos (convex_Ioo (0 : ℝ) 1)
· intro h hh
  exact
    (hasDerivAt_typeThreeFold hlam hh.1 hh.2).continuousAt.continuousWithinAt
· intro h hh
  rw [interior_Ioo] at hh
  exact (hasDerivAt_typeThreeFold hlam hh.1 hh.2).hasDerivWithinAt
· intro h hh
  rw [interior_Ioo] at hh
  have hq_low : -1 < typeThreeShape h := by
    unfold typeThreeShape
    linarith [hh.1]
  have hq_high : typeThreeShape h < 1 := by
    unfold typeThreeShape
    linarith [hh.2]
  exact mul_pos (mul_pos (by norm_num) (sq_pos_of_pos hh.1))
    (radicalCubeGap_pos hlam hq_low hq_high)

/-- The regular type-(iii) fold has at most one zero in its admissible height
interval. -/

```

LeanSuffixAnalytic.typeThreeFold_zero_between_of_area_eq

Classification: main

Source: LeanSuffixAnalytic.lean:905-925

Declaration SHA-256: 314dbeb5db22bb018c3bb2b871eb2f76a4711df985ead7c117e1cc5036bed173

Mathematical statement. For $\lambda > 1$, $0 < a < 1$, $0 < b \leq 1$ and $a < b$, equality $A_3(a) = A_3(b)$ implies a zero of K_3 strictly between a and b .

Assume the displayed density and curvature bounds and $A_3(a) = A_3(b)$. The area profile is continuous on $[a, b]$, including the possible endpoint $b = 1$. Rolle's theorem supplies $c \in (a, b)$ with $A_3'(c) = 0$. Since $0 < c < 1$ and $A_3'(c) = K_3(c)/c^3$, one has $K_3(c) = 0$. The theorem does not require the upper comparison curvature to be strictly below one.

```

theorem typeThreeFold_zero_between_of_area_eq {lam a b : ℝ}
  (h1 : 1 < lam) (ha : a ∈ Set.Ioo (0 : ℝ) 1)
  (hb : b ∈ Set.Ioc (0 : ℝ) 1) (hab : a < b)
  (harea : typeThreeArea lam a = typeThreeArea lam b) :
  ∃ c ∈ Set.Ioo a b, typeThreeFold lam c = 0 := by
  have hcont : ContinuousOn (typeThreeArea lam) (Set.Icc a b) := by
    intro x hx
    have hx0 : x ≠ 0 := ne_of_gt (ha.1.trans_le hx.1)
    unfold typeThreeArea typeThreeAngle typeThreeDelta typeThreeShape
    fun_prop (disch := simp_all)
  rcases exists_deriv_eq_zero hab hcont harea with ⟨c, hc, hzero⟩
  have hcreg : c ∈ Set.Ioo (0 : ℝ) 1 :=
    ⟨ha.1.trans hc.1, hc.2.trans_le hb.2⟩
  rw [(hasDerivAt_typeThreeArea hlam hcreg.1 hcreg.2).deriv] at hzero
  refine ⟨c, hc, ?_⟩
  exact (div_eq_zero_iff.mp hzero).resolve_right
    (pow_ne_zero 3 (ne_of_gt hcreg.1))

/-- A horizontal level meets the type-(iii) area profile at no more than two
heights in `0 < h ≤ 1`. This includes a possible third height at the closure
endpoint. -/

```

LeanSuffixAnalytic.typeThreePerimeter

Classification: main

Source: LeanSuffixAnalytic.lean:49-52

Declaration SHA-256: 22e35e36f9c0930c9f9233f0004c686c48f6f282298a578def5a6e6bba11f34e

Mathematical statement. The exact scalar type-(iii) weighted perimeter.

$P_3(h) = 2(B_3(h) + D_3(h))/h$ includes the bottom-segment contribution.

```
def typeThreePerimeter (lam h : ℝ) : ℝ :=
  2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h) / h

/-- The angle contribution in the exact regular type-(iv) formulas. -/
```

LeanSuffixAnalytic.typeThreePerimeter_deriv_eq_height_mul_area_deriv

Classification: main

Source: LeanSuffixAnalytic.lean:640-648

Declaration SHA-256: 7607f8124dad06702d520d2b8e20b8eab5117f85a2007d935f98d72479aad4e2

Mathematical statement. The derivative equality $P_3' = h A_3'$.

This is the derivative-value form used to cancel the implicit-root denominator in the gap calculation.

```
theorem typeThreePerimeter_deriv_eq_height_mul_area_deriv {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  deriv (typeThreePerimeter lam) h =
    h * deriv (typeThreeArea lam) h :=
  (typeThree_variational_hasDerivAt hlam hh hh1).2.deriv

/-- Explicit numerator for the type-(iii) area derivative. This form is
designed for interval certificates: after replacing the two radicals by their
angle parametrizations, the numerator is exactly the checker quantity `K3`. -/
```

LeanSuffixAnalytic.typeThreeShape

Classification: main

Source: LeanSuffixAnalytic.lean:32-34

Declaration SHA-256: 9ef91461acc972465b98decfe31d601788f1354eb0ffbb8d094be6f5c46ca593

Mathematical statement. The type-(iii) shape parameter $q = 2h - 1$.

For $0 < h < 1$, one has $-1 < q < 1$, including both cap branches around $h = 1/2$.

```
def typeThreeShape (h : ℝ) : ℝ := 2 * h - 1

/-- The angle contribution in the exact type-(iii) formulas. -/
```

LeanSuffixAnalytic.typeThree_variational_hasDerivAt

Classification: main

Source: LeanSuffixAnalytic.lean:554-639

Declaration SHA-256: 727f5f60431aed129e76617edde1e3ccdf8d18a1ae71fbf80ca33147d09b9cfa

Mathematical statement. The type-(iii) area and perimeter are differentiable and satisfy the variational identity.

$P_3'(h) = h A_3'(h)$ on $0 < h < 1$; all branch and denominator hypotheses are explicit.

```
theorem typeThree_variational_hasDerivAt {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeArea lam) (deriv (typeThreeArea lam) h) h ∧
  HasDerivAt (typeThreePerimeter lam)
    (h * deriv (typeThreeArea lam) h) h := by
  let q := typeThreeShape h
  let D := typeThreeDelta lam h
  let p := 2 * (1 / sqrt (1 - q ^ 2) -
    lam / sqrt (lam ^ 2 - q ^ 2))
  let dp := 2 * q * (1 / sqrt (1 - q ^ 2) -
```

```

1 / (lam * sqrt (lam ^ 2 - q ^ 2)))
have hh_ne : h ≠ 0 := ne_of_gt hh
have hshape := hasDerivAt_typeThreeShape h
have hangle : HasDerivAt (typeThreeAngle lam) p h := by
  simpa only [p, q] using hasDerivAt_typeThreeAngle hlam hh hh1
have hdelta : HasDerivAt (typeThreeDelta lam) dp h := by
  simpa only [dp, q] using hasDerivAt_typeThreeDelta hlam hh hh1
have hq_arg : 0 < 1 - q ^ 2 := by
  dsimp only [q, typeThreeShape]
  nlinarith
have hd_arg : 0 < lam ^ 2 - q ^ 2 := by
  have hlam_sq : 1 < lam ^ 2 := by nlinarith
  have hq_sq : q ^ 2 < 1 := by
    dsimp only [q, typeThreeShape]
    nlinarith
  linarith
have hlam_ne : lam ≠ 0 := ne_of_gt (lt_trans (by norm_num) hlam)
have hq_ne : sqrt (1 - q ^ 2) ≠ 0 :=
  ne_of_gt (Real.sqrt_pos.2 hq_arg)
have hd_ne : sqrt (lam ^ 2 - q ^ 2) ≠ 0 :=
  ne_of_gt (Real.sqrt_pos.2 hd_arg)
have hkey : p - q * dp + 2 * D = 0 := by
  dsimp only [p, dp, D, typeThreeDelta]
  change
    2 * (1 / sqrt (1 - q ^ 2) - lam / sqrt (lam ^ 2 - q ^ 2)) -
    q * (2 * q * (1 / sqrt (1 - q ^ 2) -
      1 / (lam * sqrt (lam ^ 2 - q ^ 2)))) +
    2 * (sqrt (lam ^ 2 - q ^ 2) / lam - sqrt (1 - q ^ 2)) = 0
  field_simp [hlam_ne, hq_ne, hd_ne]
  ring_nf
  rw [Real.sq_sqrt hq_arg.le, Real.sq_sqrt hd_arg.le]
  ring
have hB := hangle.add_const (π / 2)
have hareaRaw :=
  (hB.add ((hshape.add_const 2).mul hdelta)).div
  ((hasDerivAt_id h).pow 2) (pow_ne_zero 2 hh_ne)
have harea : HasDerivAt (typeThreeArea lam)
  (((p + (2 * D + (q + 2) * dp)) * h ^ 2 -
    (typeThreeAngle lam h + π / 2 + (q + 2) * D) * (2 * h)) /
    (h ^ 2) ^ 2) h := by
  simpa only [typeThreeArea, q, D, id_eq] using!
  hareaRaw.congr_deriv (by
    norm_num [id_eq, Pi.add_apply, Pi.mul_apply, Pi.pow_apply])
have hperimeterRaw :=
  ((hB.add hdelta).const_mul (2 : ℝ)).div (hasDerivAt_id h) hh_ne
have hperimeter : HasDerivAt (typeThreePerimeter lam)
  (((2 * (p + dp)) * h -
    2 * (typeThreeAngle lam h + π / 2 + D)) / h ^ 2) h := by
  simpa only [typeThreePerimeter, D, id_eq] using!
  hperimeterRaw.congr_deriv (by
    norm_num [id_eq, Pi.add_apply, Pi.mul_apply, Pi.pow_apply])
refine ⟨harea.congr_deriv harea.deriv.symm, hperimeter.congr_deriv ?_⟩
rw [harea.deriv]
have halgebra : ∀ (q D p dp B h : ℝ), h ≠ 0 → q = 2 * h - 1 →
  p - q * dp + 2 * D = 0 →
  ((2 * (p + dp)) * h - 2 * (B + D)) / h ^ 2 =
  h * (((p + (2 * D + (q + 2) * dp)) * h ^ 2 -
    (B + (q + 2) * D) * (2 * h)) / (h ^ 2) ^ 2) := by
  intro q D p dp B h hh_ne hq hkey
  calc
    ((2 * (p + dp)) * h - 2 * (B + D)) / h ^ 2 =
    h * (((p + (2 * D + (q + 2) * dp)) * h ^ 2 -
      (B + (q + 2) * D) * (2 * h)) / (h ^ 2) ^ 2) +
    (p - q * dp + 2 * D) / h := by
      rw [hq]
      field_simp [hh_ne]
      ring
    _ = h * (((p + (2 * D + (q + 2) * dp)) * h ^ 2 -
      (B + (q + 2) * D) * (2 * h)) / (h ^ 2) ^ 2) := by
      rw [hkey]
      ring
  exact halgebra

```

```

q D p dp (typeThreeAngle lam h +  $\pi$  / 2) h hh_ne (by
  simp only [q, typeThreeShape]) hkey

/-- Exact type-(iii) variational identity on the regular domain. -/

```

LeanSuffixReflective.QInterval.RealContains

Classification: main

Source: LeanSuffixReflective.lean:30-33

Declaration SHA-256: 82c7a510d68058d7816af0b7c59da61b39569db9b49cfeb125a24aaafc395c508

Mathematical statement. Real containment in the rational interval after endpoint casting.

It means $(lo: \mathbb{R}) \leq x \leq (hi: \mathbb{R})$; all certificate soundness uses this semantics.

```

def RealContains (i : QInterval) (x :  $\mathbb{R}$ ) : Prop :=
  (i.lo :  $\mathbb{R}$ )  $\leq$  x  $\wedge$  x  $\leq$  (i.hi :  $\mathbb{R}$ )

/-- Point intervals are the leaves of exact certificate evaluation. -/

```

MiddleFaceAssembly.Brick

Classification: main

Source: MiddleFaceAssembly.lean:27-37

Declaration SHA-256: 0941d7d9eb0eaebceb5359d2c2683a5354c44a25e853225ca2ccb9f95f4460178

Mathematical statement. Exact rational density endpoints, affine angle predictors, coupling slopes, and positive error radii.

The record describes a closed family of four-variable root boxes over a nonempty density interval.

```

structure Brick where
  lamLo :  $\mathbb{Q}$ 
  lamHi :  $\mathbb{Q}$ 
  centers : Fin 4  $\rightarrow$   $\mathbb{Q}$ 
  slopes : Fin 4  $\rightarrow$   $\mathbb{Q}$ 
  yXiSlope :  $\mathbb{Q}$ 
  vRhoSlope :  $\mathbb{Q}$ 
  radii : Fin 4  $\rightarrow$   $\mathbb{Q}$ 
  lamOrdered : lamLo < lamHi
  radiiPos :  $\forall j, 0 < \text{radii } j$ 

```

MiddleFaceAssembly.CellCertificate

Classification: main

Source: MiddleFaceAssembly.lean:1444-1484

Declaration SHA-256: c5e1550765346522815e726fec39295f689896e3151daad7810df51dd4df337f

Mathematical statement. Primitive strict face signs, branch bounds, density jump, negative K_3 , and negative G for a brick.

The certificate asserts F positive on the low xi face and negative on the high face, while C_4, E, C_3 have negative low and positive high faces; $K_3, G < 0$ hold throughout.

```

structure CellCertificate (b : Brick) : Prop where
  lambda_one_lt :  $\forall t, (tInterval b).RealContains t \rightarrow$ 
    1 < lam b (pointOf t 0)
  branches :  $\forall t z, (tInterval b).RealContains t \rightarrow$ 
    InSymmetricBox (radiiReal b) z  $\rightarrow$ 
    let p := pointOf t z

```

```

0 < x b p ^ x b p < π / 4 ^
0 < y b p ^ y b p < π / 4 ^
0 < w b p ^ w b p < π / 4 ^
0 < v b p ^ v b p < π / 4
F_xi_low_pos : ∀ t z, (tInterval b).RealContains t →
  InSymmetricBox (radiiReal b) z →
  z 0 = -radiiReal b 0 → 0 < F b (pointOf t z)
F_xi_high_neg : ∀ t z, (tInterval b).RealContains t →
  InSymmetricBox (radiiReal b) z →
  z 0 = radiiReal b 0 → F b (pointOf t z) < 0
C4_eta_low_neg : ∀ t z, (tInterval b).RealContains t →
  InSymmetricBox (radiiReal b) z →
  z 1 = -radiiReal b 1 → C4 b (pointOf t z) < 0
C4_eta_high_pos : ∀ t z, (tInterval b).RealContains t →
  InSymmetricBox (radiiReal b) z →
  z 1 = radiiReal b 1 → 0 < C4 b (pointOf t z)
E_rho_low_neg : ∀ t z, (tInterval b).RealContains t →
  InSymmetricBox (radiiReal b) z →
  z 2 = -radiiReal b 2 → E b (pointOf t z) < 0
E_rho_high_pos : ∀ t z, (tInterval b).RealContains t →
  InSymmetricBox (radiiReal b) z →
  z 2 = radiiReal b 2 → 0 < E b (pointOf t z)
C3_sigma_low_neg : ∀ t z, (tInterval b).RealContains t →
  InSymmetricBox (radiiReal b) z →
  z 3 = -radiiReal b 3 → C3 b (pointOf t z) < 0
C3_sigma_high_pos : ∀ t z, (tInterval b).RealContains t →
  InSymmetricBox (radiiReal b) z →
  z 3 = radiiReal b 3 → 0 < C3 b (pointOf t z)
K3_neg : ∀ t z, (tInterval b).RealContains t →
  InSymmetricBox (radiiReal b) z → K3 b (pointOf t z) < 0
G_neg : ∀ t z, (tInterval b).RealContains t →
  InSymmetricBox (radiiReal b) z → G b (pointOf t z) < 0

/-- Numerical obligations discharged by an exact-rational middle-face checker.
The fields state primitive checker facts, not a candidate-facing conclusion. -/

```

MiddleFaceAssembly.CellCertificate.sound

Classification: main

Source: MiddleFaceAssembly.lean:1510-1536

Declaration SHA-256: 3fff65f4ffa94f40ec377aeb7e5b713831d3e0a0f69889bb8c84ffd5cf5fd19a

Mathematical statement. The primitive certificate implies the correctly oriented root conditions.

Negating F reverses its face signs; all other coordinate and strict-gap conditions are retained.

```

theorem sound {b : Brick} (certificate : CellCertificate b) : Conditions b where
  lambda_one_lt := certificate.lambda_one_lt
  branches := certificate.branches
  lowFaces := by
    intro t z ht hz j hj
    fin_cases j
    · simp [rootMap] using
      (neg_lt_zero.mpr (certificate.F_xi_low_pos t z ht hz hj))
    · simp [rootMap] using certificate.C4_eta_low_neg t z ht hz hj
    · simp [rootMap] using certificate.E_rho_low_neg t z ht hz hj
    · simp [rootMap] using certificate.C3_sigma_low_neg t z ht hz hj
  highFaces := by
    intro t z ht hz j hj
    fin_cases j
    · simp [rootMap] using
      (neg_pos.mpr (certificate.F_xi_high_neg t z ht hz hj))
    · simp [rootMap] using certificate.C4_eta_high_pos t z ht hz hj
    · simp [rootMap] using certificate.E_rho_high_pos t z ht hz hj
    · simp [rootMap] using certificate.C3_sigma_high_pos t z ht hz hj
  K3_neg := certificate.K3_neg
  G_neg := certificate.G_neg

end CellCertificate

```

```

/-- The exact analytic interpretation of the checker coordinates on the
recorded angle branch. Its equation and sign consequences remain separate
from the primitive numerical conditions. -/

```

MiddleFaceAssembly.Conditions

Classification: main

Source: MiddleFaceAssembly.lean:1485-1509

Declaration SHA-256: 5c36f9057f1d62e8c0bcb35e1c262b414fda26bcb9bf4a32565a3c1e70dc5568

Mathematical statement. Root-map face conditions and strict branch/comparison conditions for a brick.

Each component of $(-F, C_4, E, C_3)$ has the same negative-low, positive-high orientation on its corresponding box faces.

```

structure Conditions (b : Brick) : Prop where
  lambda_one_lt : ∀ t, (tInterval b).RealContains t → 1 < lam b (pointOf t 0)
  branches : ∀ t z, (tInterval b).RealContains t →
    InSymmetricBox (radiiReal b) z →
      let p := pointOf t z
      0 < x b p ^ x b p < π / 4 ^
      0 < y b p ^ y b p < π / 4 ^
      0 < w b p ^ w b p < π / 4 ^
      0 < v b p ^ v b p < π / 4
  lowFaces : ∀ t z, (tInterval b).RealContains t →
    InSymmetricBox (radiiReal b) z →
      ∀ j, z j = -radiiReal b j → rootMap b t z j < 0
  highFaces : ∀ t z, (tInterval b).RealContains t →
    InSymmetricBox (radiiReal b) z →
      ∀ j, z j = radiiReal b j → 0 < rootMap b t z j
  K3_neg : ∀ t z, (tInterval b).RealContains t →
    InSymmetricBox (radiiReal b) z → K3 b (pointOf t z) < 0
  G_neg : ∀ t z, (tInterval b).RealContains t →
    InSymmetricBox (radiiReal b) z → G b (pointOf t z) < 0

namespace CellCertificate

/-- The checker decisions have exactly the orientation required by the
Poincaré--Miranda root assembly. In particular, the first component of
`rootMap` is `-F`, so its two face decisions reverse orientation. -/

```

MiddleFaceAssembly.Conditions.RootProjection

Classification: main

Source: MiddleFaceAssembly.lean:1993-2008

Declaration SHA-256: 2f76fbe7f8dbd852f7dc66c768c04d395e099b30fab88a0cc5ace08e7978fcf4

Mathematical statement. A checker zero together with its branch, box, and strict-sign information.

The projection retains the data required to interpret a root as a stationary comparison, not merely an abstract zero.

```

structure RootProjection (b : Brick) (t : ℝ) (z : Vec4) : Prop where
  mem_box : InSymmetricBox (radiiReal b) z
  equations : ∀ j, rootMap b t z j = 0
  branches :
    let p := pointOf t z
    0 < x b p ^ x b p < π / 4 ^
    0 < y b p ^ y b p < π / 4 ^
    0 < w b p ^ w b p < π / 4 ^
    0 < v b p ^ v b p < π / 4
  K3_neg : K3 b (pointOf t z) < 0
  G_neg : G b (pointOf t z) < 0
  typeThreeHeight_regular :
    (1 + cos (w b (pointOf t z))) / 2 ∈ Ioo (0 : ℝ) 1
  typeFourHeight_regular : cos (x b (pointOf t z)) ∈ Ioo (0 : ℝ) 1

```

```
/-- Generic root/projection assembly for a checked middle-face brick. -/
```

MiddleFaceAssembly.Conditions.candidate_not_isWeightedPerimeterMinimizer

Classification: main

Source: MiddleFaceAssembly.lean:2424-2438

Declaration SHA-256: ceb5515954483428fb5ae9be3255392f99e773ed979053baf3c18aa42dc8d6cd

Mathematical statement. A checked middle-face brick excludes all modeled candidates at each density in its interval.

The stationary pair is promoted through the descending envelope and realized as an admissible equal-area competitor.

```
theorem candidate_not_isWeightedPerimeterMinimizer
  (conditions : Conditions b) {t lam0 : ℝ}
  (ht : (tInterval b).RealContains t)
  (hlam : (lamMid b : ℝ) + t = lam0)
  (candidate : _root_.FourArcCandidate lam0)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  subst lam0
  rcases stationaryEqualAreaPair_exists conditions ht with
  ⟨pair, hfoldThree, hperimeter⟩
  exact
  CMVSuffixModel.FourArcCandidate.not_isWeightedPerimeterMinimizer_of_stationaryEqualAreaPair
  candidate hcandidate pair hfoldThree hperimeter

/-- Generic candidate-facing conclusion for the complete projected slice. -/
```

MiddleFaceAssembly.Conditions.rootProjection

Classification: main

Source: MiddleFaceAssembly.lean:2009-2035

Declaration SHA-256: b0b149b1edc9d2dd81f962164ee26472b0cb945a7e8f0c2a208f3fb617b6fe84

Mathematical statement. A valid brick supplies the complete root projection.

Choose the opposite-face zero and carry the uniform branch, negative-fold, and gap bounds to that point.

```
theorem rootProjection (conditions : Conditions b) (t : ℝ)
  (ht : (tInterval b).RealContains t) : ∃ z, RootProjection b t z := by
  rcases conditions.root_exists t ht with ⟨z, hz, hroot⟩
  have hb := conditions.branches t z ht hz
  have hK3 := conditions.K3_neg t z ht hz
  have hG := conditions.G_neg t z ht hz
  rcases hb with ⟨hx0, hxq, hy0, hyq, hw0, hwq, hv0, hvq⟩
  have hxpi : x b (pointOf t z) < π := hxq.trans (by linarith [Real.pi_pos])
  have hwpi : w b (pointOf t z) < π := hwq.trans (by linarith [Real.pi_pos])
  have hxcosPos : 0 < cos (x b (pointOf t z)) :=
    Real.cos_pos_of_mem_Ioo ⟨by linarith [Real.pi_pos],
    hxq.trans (by linarith [Real.pi_pos])⟩
  have hwcosp : 0 < cos (w b (pointOf t z)) :=
    Real.cos_pos_of_mem_Ioo ⟨by linarith [Real.pi_pos],
    hwq.trans (by linarith [Real.pi_pos])⟩
  have hxcosLt : cos (x b (pointOf t z)) < 1 := by
    simpa using Real.strictAntiOn_cos
    (show (0 : ℝ) ∈ Icc 0 π from ⟨le_rfl, Real.pi_pos.le⟩)
    ⟨hx0.le, hxpi.le⟩ hx0
  have hwcsl : cos (w b (pointOf t z)) < 1 := by
    simpa using Real.strictAntiOn_cos
    (show (0 : ℝ) ∈ Icc 0 π from ⟨le_rfl, Real.pi_pos.le⟩)
    ⟨hw0.le, hwpi.le⟩ hw0
  refine ⟨z, hz, hroot, ?_, hK3, hG, ?_, ⟨hxcosPos, hxcosLt⟩⟩
  · exact ⟨hx0, hxq, hy0, hyq, hw0, hwq, hv0, hvq⟩
  · constructor <=> linarith
```

MiddleFaceAssembly.Conditions.root_exists

Classification: main

Source: MiddleFaceAssembly.lean:1914-1992

Declaration SHA-256: beb4b56e77f5562892d76e328ff8aac16683506f9b4a5b897368a7309f80cb5d

Mathematical statement. The continuous, oppositely signed checker map has a zero in each closed error box.

For every centered density in the brick's recorded closed interval, the certificate conditions give a vector in the closed error box solving $F = C_4 = E = C_3 = 0$. The proof uses this model's triangular dependence: principal-angle projections solve the two incidences continuously, then one-dimensional intermediate value arguments in the first and third error coordinates solve the stationary and equal-area equations. This is a construction for the specified checker map, not an invocation of an unspecified multidimensional root solver.

```
theorem root_exists (conditions : Conditions b) (t : ℝ)
  (ht : (tInterval b).RealContains t) :
  ∃ z, InSymmetricBox (radiiReal b) z ^ ∀ j, rootMap b t z j = 0 := by
  let zF : ℝ → Vec4 := fun xi => ![xi, etaRoot b t xi, 0, 0]
  have hzF_cont : Continuous zF := by
    apply continuous_pi
    intro j
    fin_cases j
    · change Continuous (fun xi : ℝ => xi); fun_prop
    · exact etaRoot_continuous b t
    · change Continuous (fun _ : ℝ => (0 : ℝ)); fun_prop
    · change Continuous (fun _ : ℝ => (0 : ℝ)); fun_prop
  have hxiLow : -radiiReal b 0 ∈ Icc (-radiiReal b 0) (radiiReal b 0) :=
    ⟨le_rfl, neg_le_self (radiiReal_pos 0).le⟩
  have hxiHigh : radiiReal b 0 ∈ Icc (-radiiReal b 0) (radiiReal b 0) :=
    ⟨neg_le_self (radiiReal_pos 0).le, le_rfl⟩
  have hetaLow := (etaRoot_spec conditions ht hxiLow).1
  have hetaHigh := (etaRoot_spec conditions ht hxiHigh).1
  have hzFlow : InSymmetricBox (radiiReal b) (zF (-radiiReal b 0)) :=
    vector_mem_box hxiLow hetaLow (zero_mem_coordinate 2) (zero_mem_coordinate 3)
  have hzFHigh : InSymmetricBox (radiiReal b) (zF (radiiReal b 0)) :=
    vector_mem_box hxiHigh hetaHigh (zero_mem_coordinate 2) (zero_mem_coordinate 3)
  have hloF : rootMap b t (zF (-radiiReal b 0)) 0 ≤ 0 :=
    (conditions.lowFaces t _ ht hzFlow 0 (by simp [zF])).le
  have hhiF : 0 ≤ rootMap b t (zF (radiiReal b 0)) 0 :=
    (conditions.highFaces t _ ht hzFHigh 0 (by simp [zF])).le
  have hroot_continuous : Continuous (rootMap b t) := rootMap_continuous b t
  have hcontF : Continuous (fun xi => rootMap b t (zF xi) 0) :=
    (continuous_apply (0 : Fin 4)).comp (hroot_continuous.comp hzF_cont)
  rcases oppositeFace_zero (neg_le_self (radiiReal_pos 0).le)
    hcontF.continuousOn hloF hhiF with ⟨xi, hxi, hF⟩
  let eta := etaRoot b t xi
  have hetaSpec := etaRoot_spec conditions ht hxi
  have heta : eta ∈ Icc (-radiiReal b 1) (radiiReal b 1) := hetaSpec.1
  have hC4 : C4 b (pointOf t ![xi, eta, 0, 0]) = 0 := by
    simpa [eta] using hetaSpec.2
  let zE : ℝ → Vec4 := fun rho => ![xi, eta, rho, sigmaRoot b t rho]
  have hzE_cont : Continuous zE := by
    apply continuous_pi
    intro j
    fin_cases j
    · change Continuous (fun _ : ℝ => xi); fun_prop
    · change Continuous (fun _ : ℝ => eta); fun_prop
    · change Continuous (fun rho : ℝ => rho); fun_prop
    · exact sigmaRoot_continuous b t
  have hrhoLow : -radiiReal b 2 ∈ Icc (-radiiReal b 2) (radiiReal b 2) :=
    ⟨le_rfl, neg_le_self (radiiReal_pos 2).le⟩
  have hrhoHigh : radiiReal b 2 ∈ Icc (-radiiReal b 2) (radiiReal b 2) :=
    ⟨neg_le_self (radiiReal_pos 2).le, le_rfl⟩
  have hsigmaLow := (sigmaRoot_spec conditions ht hrhoLow).1
  have hsigmaHigh := (sigmaRoot_spec conditions ht hrhoHigh).1
  have hzELow : InSymmetricBox (radiiReal b) (zE (-radiiReal b 2)) :=
    vector_mem_box hxi heta hrhoLow hsigmaLow
  have hzEHigh : InSymmetricBox (radiiReal b) (zE (radiiReal b 2)) :=
```



```

vector_mem_box hxi heta hrhoHigh hsigmaHigh
have hloE : rootMap b t (zE (-radiiReal b 2)) 2 ≤ 0 :=
  (conditions.lowFaces t _ ht hzELow 2 (by simp [zE, Matrix.cons_val_two])).le
have hhiE : 0 ≤ rootMap b t (zE (radiiReal b 2)) 2 :=
  (conditions.highFaces t _ ht hzEHigh 2 (by simp [zE, Matrix.cons_val_two])).le
have hcontE : Continuous (fun rho => rootMap b t (zE rho) 2) :=
  (continuous_apply (2 : Fin 4)).comp (hroot_continuous.comp hzE_cont)
rcases oppositeFace_zero (neg_le_self (radiiReal_pos 2)).le)
  hcontE.continuousOn hloE hhiE with ⟨rho, hrho, hE⟩
let sigma := sigmaRoot b t rho
have hsigmaSpec := sigmaRoot_spec conditions ht hrho
have hsigma : sigma ∈ Icc (-radiiReal b 3) (radiiReal b 3) := hsigmaSpec.1
have hC3 : C3 b (pointOf t ![0, 0, rho, sigma]) = 0 := by
  simpa [sigma] using hsigmaSpec.2
let z : Vec4 := ![xi, eta, rho, sigma]
refine ⟨z, vector_mem_box hxi heta hrho hsigma, ?_⟩
intro j
fin_cases j
· simpa [zF, z, eta, rootMap, Model.F, B4, D4, x, y, lam, pointOf] using hF
· simpa [z, rootMap, C4, x, y, lam, pointOf] using hC4
· simpa [zE, z, sigma, rootMap, Matrix.cons_val_two] using hE
· simpa [z, rootMap, C3, w, v, lam, pointOf, Matrix.cons_val_three] using hC3

/-- Root and curvature-projection facts assembled solely from checked numerical
semantics. This remains below the candidate layer. -/

```

MiddleFaceAssembly.Conditions.stationaryEqualAreaPair_exists

Classification: main

Source: MiddleFaceAssembly.lean:2349-2423

Declaration SHA-256: c9edc4aa9422891e0d25fbd6875c005c42dad37c4605fbf65a3d8fdd6fa0a683

Mathematical statement. Each checked density slice supplies a stationary equal-area pair with negative K_3 and strict perimeter improvement.

The root equations and checker semantics give the pair; uniform $K_3 < 0$ and $G < 0$ give the two extra envelope obligations.

```

theorem stationaryEqualAreaPair_exists
  (conditions : Conditions b) {t : ℝ}
  (ht : (tInterval b).RealContains t) :
  ∃ pair :
    LeanSuffixAnalytic.StationaryEqualAreaPair ((lamMid b : ℝ) + t),
    LeanSuffixAnalytic.typeThreeFold ((lamMid b : ℝ) + t) pair.h₃ < 0 ∧
    LeanSuffixAnalytic.typeThreePerimeter
      ((lamMid b : ℝ) + t) pair.h₃ <
    LeanSuffixAnalytic.typeFourPerimeter
      ((lamMid b : ℝ) + t) pair.h₄ := by
  rcases conditions.rootProjection t ht with ⟨z, projection⟩
  let p : Point := pointOf t z
  have hbranches :
    0 < x b p ^ x b p < π / 4 ∧
    0 < y b p ^ y b p < π / 4 ∧
    0 < w b p ^ w b p < π / 4 ∧
    0 < v b p ^ v b p < π / 4 := by
    simpa [p] using projection.branches
  rcases hbranches with
    ⟨hx0, hxq, hy0, hyq, hw0, hwq, hv0, hvq⟩
  have hFNeg : -F b p = 0 := by
    simpa [rootMap, p] using projection.equations 0
  have hF : F b p = 0 := neg_eq_zero.mp hFNeg
  have hC4 : C4 b p = 0 := by
    simpa [rootMap, p] using projection.equations 1
  have hE : E b p = 0 := by
    simpa [rootMap, p] using projection.equations 2
  have hC3 : C3 b p = 0 := by
    simpa [rootMap, p] using projection.equations 3
  have hlam : 1 < lam b p := by
    simpa [p, lam, pointOf] using conditions.lambda_one_lt t ht

```

```

have hsem := pointEquationSemantics b p hlam
  ⟨hx0, hxq, hy0, hyq, hw0, hwq, hv0, hvq⟩ hC4 hC3
have harea := hsem.equalArea_of_E_zero hE
have hstationary := hsem.stationary_of_F_zero hF
have hperimeter :=
  hsem.perimeter_lt_of_G_neg (by simpa [p] using projection.G_neg)
have hh3Pos : 0 < (1 + cos (w b p)) / 2 :=
  lt_trans (by norm_num) hsem.typeThreeHeight_gt_half
have hderiv :
  deriv (LeanSuffixAnalytic.typeThreeArea (lam b p))
    ((1 + cos (w b p)) / 2) < 0 := by
  rw [MiddleFaceAssembly.typeThreeArea_deriv_eq_K3 b p hlam
    ⟨hw0, hwq, hv0, hvq⟩ hC3]
  exact div_neg_of_neg_of_pos
    (by simpa [p] using projection.K3_neg)
    (pow_pos hh3Pos 3)
have hfoldThree :
  LeanSuffixAnalytic.typeThreeFold (lam b p)
    ((1 + cos (w b p)) / 2) < 0 := by
  rw [(LeanSuffixAnalytic.hasDerivAt_typeThreeArea hlam hh3Pos
    hsem.typeThreeHeight_lt_one).deriv] at hderiv
  rcases div_neg_iff.mp hderiv with hbad | hgood
  · linarith [pow_pos hh3Pos 3]
  · exact hgood.1
have hlamEq : lam b p = (lamMid b : ℝ) + t := by
  simp [p, lam, pointOf]
rw [hlamEq] at harea hstationary hperimeter hfoldThree
let pair :
  LeanSuffixAnalytic.StationaryEqualAreaPair ((lamMid b : ℝ) + t) :=
{ h3 := (1 + cos (w b p)) / 2
  h4 := cos (x b p)
  h3_gt_half := hsem.typeThreeHeight_gt_half
  h3_lt_one := hsem.typeThreeHeight_lt_one
  h4_pos := hsem.typeFourHeight_pos
  h4_lt_one := hsem.typeFourHeight_lt_one
  equalArea := harea
  stationary := hstationary }
refine ⟨pair, ?_, ?_⟩
· simpa [pair] using hfoldThree
· simpa [pair] using hperimeter

/-- A single stationary pair from a checked brick promotes through the global
descending equal-area envelope and excludes every modeled type-(iv) candidate
at that density, including the `h4 = 1` endpoint. -/

```

MiddleFaceAssembly.Model.B3

Classification: main

Source: MiddleFaceAssembly.lean:71-71

Declaration SHA-256: 89bf9f374fc276401ac90ed32d58d6789681d13bdd40586c5140a98d2d570b68

Mathematical statement. Angular checker expression for the complete type-(iii) angle term.

$$B_3 = \lambda v + \pi - w.$$

```
def B3 (p : Point) : ℝ := lam b p * v b p + π - w b p
```

MiddleFaceAssembly.Model.B4

Classification: main

Source: MiddleFaceAssembly.lean:68-68

Declaration SHA-256: 4571608d53f0160a6715950590485e849c7587509687cead6580411b7968f1d6

Mathematical statement. Angular checker expression for the type-(iv) angle term.

$$B_4 = \lambda y + \pi/2 - x.$$

```
def B4 (p : Point) : ℝ := lam b p * y b p + π / 2 - x b p
```

MiddleFaceAssembly.Model.C3

Classification: main

Source: MiddleFaceAssembly.lean:78-78

Declaration SHA-256: 2033f5e817a7ce4ac881490411dee667ce9a476eadf4a060349976ec81a234e9

Mathematical statement. The type-(iii) incidence residual.

$$C_3 = \cos w - \lambda \cos v.$$

```
def C3 (p : Point) : ℝ := cos (w b p) - lam b p * cos (v b p)
```

MiddleFaceAssembly.Model.C4

Classification: main

Source: MiddleFaceAssembly.lean:75-75

Declaration SHA-256: 60b08b58076f301aaff633a26cf94d71e808af4d3ade0beea2f7a2c0dba972c5

Mathematical statement. The type-(iv) incidence residual.

$$C_4 = \cos x - \lambda \cos y.$$

```
def C4 (p : Point) : ℝ := cos (x b p) - lam b p * cos (y b p)
```

MiddleFaceAssembly.Model.D3

Classification: main

Source: MiddleFaceAssembly.lean:72-72

Declaration SHA-256: 68fc1a2e05e1a65f1772f92a4660582f4e3c321b287cc3594448b912c6e7c624

Mathematical statement. Angular checker expression for type-(iii) displacement.

$$D_3 = \sin v - \sin w.$$

```
def D3 (p : Point) : ℝ := sin (v b p) - sin (w b p)
```

MiddleFaceAssembly.Model.D4

Classification: main

Source: MiddleFaceAssembly.lean:69-69

Declaration SHA-256: b20a244421f72357884724349d2944bd92ce6795e69082a93fe34e5d606f3a73

Mathematical statement. Angular checker expression for type-(iv) displacement.

$$D_4 = \sin y - \sin x.$$

```
def D4 (p : Point) : ℝ := sin (y b p) - sin (x b p)
```

MiddleFaceAssembly.Model.E

Classification: main

Source: MiddleFaceAssembly.lean:79-80

Declaration SHA-256: 7b79ccc631128297b34feba1be2bb747e1b7da7d54a858df51a0cb1a25f13437

Mathematical statement. The denominator-cleared equal-area residual.

$E = 2\cos^2 x N_3 - (1 + \cos w)^2 N_4$; at incidence, its zero gives $A_3((1 + \cos w)/2) = A_4(\cos x)$.

```
def E (p : Point) : ℝ :=
  2 * cos (x b p) ^ 2 * N3 b p - (1 + cos (w b p)) ^ 2 * N4 b p
```

MiddleFaceAssembly.Model.F

Classification: main

Source: MiddleFaceAssembly.lean:76-77

Declaration SHA-256: 6d400bd716c439d9d2d74f6bd49be9dc19b55b45847d35d694074dc09fbf9777

Mathematical statement. The type-(iv) stationary-fold residual in angular coordinates.

$F = \cos x D_4 - \sin x \sin y B_4$; on the branch and incidence equation, $F = 0$ means $K_4(\cos x) = 0$.

```
def F (p : Point) : ℝ :=
  cos (x b p) * D4 b p - sin (x b p) * sin (y b p) * B4 b p
```

MiddleFaceAssembly.Model.G

Classification: main

Source: MiddleFaceAssembly.lean:81-83

Declaration SHA-256: cd83bee785b1a55d445030d234df204752ee92464928a982e6215564d1ecad3b

Mathematical statement. The positively scaled perimeter-gap residual.

$G = \cos x (B_3 + D_3) - (1 + \cos w) B_4$; positive denominators make $G < 0$ equivalent to strict improvement.

```
def G (p : Point) : ℝ :=
  cos (x b p) * (B3 b p + D3 b p) - (1 + cos (w b p)) * B4 b p
```

MiddleFaceAssembly.Model.K3

Classification: main

Source: MiddleFaceAssembly.lean:84-89

Declaration SHA-256: ced932efb916edd340f60f22ba0266b2b91a924f0bdc75b35e1b476fed409a5e

Mathematical statement. The type-(iii) fold numerator in checker angles.

Substitute $s = (1 + \cos w)/2$ and the incidence relation into the analytic K_3 formula, using positive sines for radicals.

```
def K3 (p : Point) : ℝ :=
  let h3 := (1 + cos (w b p)) / 2
  4 * h3 * ((1 + cos (w b p)) / sin (w b p) -
    (lam b p ^ 2 + cos (w b p)) / (lam b p * (lam b p * sin (v b p)))) -
  2 * (B3 b p + D3 b p)
```

MiddleFaceAssembly.Model.N3

Classification: main

Source: MiddleFaceAssembly.lean:73-74

Declaration SHA-256: 558a63c2734e558c548a2f1dae67adb206ef229c2adc0edc698dc7c3f00e2c9

Mathematical statement. The type-(iii) area numerator.

$$N_3 = B_3 + (\cos w + 2)D_3.$$

```
def N3 (p : Point) : ℝ := B3 b p + (cos (w b p) + 2) * D3 b p
```

MiddleFaceAssembly.Model.N4

Classification: main

Source: MiddleFaceAssembly.lean:70-70

Declaration SHA-256: 471118e366d83932c15b82b181a3387515c42d96f12d08eec3d20f86b701eec8

Mathematical statement. The type-(iv) area numerator before its factor two.

$$N_4 = B_4 + \cos x D_4.$$

```
def N4 (p : Point) : ℝ := B4 b p + cos (x b p) * D4 b p
```

MiddleFaceAssembly.Model.rootMap

Classification: main

Source: MiddleFaceAssembly.lean:94-97

Declaration SHA-256: bbd8ee6a24e6b32afa58618b3b175c83e9f669c90a7f1ad7d90339051f50892f

Mathematical statement. The four-dimensional map whose zero solves the comparison equations.

It is $(-F, C_4, E, C_3)$; the minus sign on F aligns all opposite-face orientations.

```
def rootMap (t : ℝ) (z : Vec4) : Vec4 :=
  let p := pointOf t z
  ![-F b p, C4 b p, E b p, C3 b p]
```

MiddleFaceAssembly.Model.rootMap_continuous

Classification: main

Source: MiddleFaceAssembly.lean:114-134

Declaration SHA-256: 6092edc775a5fa26e9773ae9ff53a2555f68e09f5126315e8d8440fbed6260b3

Mathematical statement. The exact checker root map is continuous for each fixed centered density.

Its coordinates are finite algebraic combinations of affine functions, sine, and cosine.

```
theorem rootMap_continuous (t : ℝ) : Continuous (rootMap b t) := by
  apply continuous_pi
  intro j
  fin_cases j <|>
  simp [rootMap, Model.F, C4, E, C3, N3, N4, B3, B4, D3, D4, pointOf,
    lam, x, y, w, v] <|> fun_prop

end Model

/-! ### Equal-area residual derivative and opposite-`rho` face lane -/
```

```

/- Brick-parametric analytic machinery for the equal-area residual.

Individual bricks retain only their numerical trigonometric enclosures,
derivative-bound certificate, and origin certificate. -/
namespace EResidual

variable (b : Brick)

/-- Scalar form of the equal-area residual, retaining all five affine
variables and hence all correlations used by the face estimate. -/

```

MiddleFaceAssembly.PointEquationSemantics

Classification: main

Source: MiddleFaceAssembly.lean:1537-1551

Declaration SHA-256: 478344dbc781da1c2b9f6b07701b41f4488a93fff689eda8b3059bdb2ad9f499

Mathematical statement. The analytic meaning of checker equations on the recorded angle branch.

It supplies regular curvature domains and translates $E = 0$, $F = 0$, and $G < 0$ into equal area, stationary fold, and strict perimeter improvement.

```

structure PointEquationSemantics (b : Brick) (p : Point) : Prop where
  typeThreeHeight_gt_half : 1 / 2 < (1 + cos (w b p)) / 2
  typeThreeHeight_lt_one : (1 + cos (w b p)) / 2 < 1
  typeFourHeight_pos : 0 < cos (x b p)
  typeFourHeight_lt_one : cos (x b p) < 1
  equalArea_of_E_zero : E b p = 0 →
    LeanSuffixAnalytic.typeThreeArea (lam b p) ((1 + cos (w b p)) / 2) =
      LeanSuffixAnalytic.typeFourArea (lam b p) (cos (x b p))
  stationary_of_F_zero : F b p = 0 →
    LeanSuffixAnalytic.typeFourFold (lam b p) (cos (x b p)) = 0
  perimeter_lt_of_G_neg : G b p < 0 →
    LeanSuffixAnalytic.typeThreePerimeter (lam b p) ((1 + cos (w b p)) / 2) <
      LeanSuffixAnalytic.typeFourPerimeter (lam b p) (cos (x b p))

/-- Honest analytic semantics of the reusable checker model. -/

```

MiddleFaceAssembly.pointEquationSemantics

Classification: main

Source: MiddleFaceAssembly.lean:1552-1700

Declaration SHA-256: c757c9ef8ed58ec827195f2ddcc9f974e6ab47521674ab7d055018fa79f178cf

Mathematical statement. The checker semantics follow from the branch bounds and the two incidence equations.

Principal inverse-trigonometric identities identify $s = (1 + \cos w)/2$ and $t = \cos x$ with the analytic scalar formulas.

```

theorem pointEquationSemantics (b : Brick) (p : Point) (hlam : 1 < lam b p)
  (hbranches :
    0 < x b p ^ x b p < π / 4 ^
    0 < y b p ^ y b p < π / 4 ^
    0 < w b p ^ w b p < π / 4 ^
    0 < v b p ^ v b p < π / 4)
  (hC4 : C4 b p = 0) (hC3 : C3 b p = 0) :
  PointEquationSemantics b p := by
  rcases hbranches with ⟨hx0, hxq, hy0, hyq, hw0, hwq, hv0, hvq⟩
  have hpi4_lt_pi : π / 4 < π := by linarith [Real.pi_pos]
  have hpi4_lt_pi_div_two : π / 4 < π / 2 := by linarith [Real.pi_pos]
  have hxpi : x b p < π := hxq.trans hpi4_lt_pi
  have hypi : y b p < π := hyq.trans hpi4_lt_pi
  have hwpi : w b p < π := hwq.trans hpi4_lt_pi
  have hvpi : v b p < π := hvq.trans hpi4_lt_pi
  have hlam_pos : 0 < lam b p := lt_trans (by norm_num) hlam
  have hxcos_pos : 0 < cos (x b p) :=

```

```

Real.cos_pos_of_mem_Ioo <by linarith [Real.pi_pos],
  hxq.trans hpi4_lt_pi_div_two>
have hwcpos_pos :  $0 < \cos (w \ b \ p) :=$ 
  Real.cos_pos_of_mem_Ioo <by linarith [Real.pi_pos],
    hwq.trans hpi4_lt_pi_div_two>
have hzero_mem :  $(0 : \mathbb{R}) \in \text{Icc } 0 \ \pi := \langle \text{le\_rfl}, \text{Real.pi\_pos.le} \rangle$ 
have hx_mem :  $x \ b \ p \in \text{Icc } 0 \ \pi := \langle \text{hx0.le}, \text{hxpi.le} \rangle$ 
have hw_mem :  $w \ b \ p \in \text{Icc } 0 \ \pi := \langle \text{hw0.le}, \text{hwpi.le} \rangle$ 
have hxcos_lt_one :  $\cos (x \ b \ p) < 1 :=$  by
  simp using Real.strictAntiOn_cos hzero_mem hx_mem hx0
have hwcpos_lt_one :  $\cos (w \ b \ p) < 1 :=$  by
  simp using Real.strictAntiOn_cos hzero_mem hw_mem hw0
have hxsin_pos :  $0 < \sin (x \ b \ p) :=$ 
  Real.sin_pos_of_pos_of_lt_pi hx0 hxpi
have hysin_pos :  $0 < \sin (y \ b \ p) :=$ 
  Real.sin_pos_of_pos_of_lt_pi hy0 hypi
have hwsin_pos :  $0 < \sin (w \ b \ p) :=$ 
  Real.sin_pos_of_pos_of_lt_pi hw0 hwpi
have hvsin_pos :  $0 < \sin (v \ b \ p) :=$ 
  Real.sin_pos_of_pos_of_lt_pi hv0 hvpi
have hcos_xy :  $\cos (x \ b \ p) = \text{lam } b \ p * \cos (y \ b \ p) :=$  by
  unfold C4 at hC4
  linarith
have hcos_wv :  $\cos (w \ b \ p) = \text{lam } b \ p * \cos (v \ b \ p) :=$  by
  unfold C3 at hC3
  linarith
have hshape :
  LeanSuffixAnalytic.typeThreeShape ((1 +  $\cos (w \ b \ p)$ ) / 2) =
     $\cos (w \ b \ p) :=$  by
  unfold LeanSuffixAnalytic.typeThreeShape
  ring
have hcos_xy_div :  $\cos (x \ b \ p) / \text{lam } b \ p = \cos (y \ b \ p) :=$  by
  rw [hcos_xy]
  field_simp [ne_of_gt hlam_pos]
have hcos_wv_div :  $\cos (w \ b \ p) / \text{lam } b \ p = \cos (v \ b \ p) :=$  by
  rw [hcos_wv]
  field_simp [ne_of_gt hlam_pos]
have hB4 :
  LeanSuffixAnalytic.typeFourAngle (lam b p) ( $\cos (x \ b \ p)$ ) = B4 b p := by
  rw [LeanSuffixAnalytic.typeFourAngle_eq_checker, hcos_xy_div,
    Real.arccos_cos hy0.le hypi.le, Real.arccos_cos hx0.le hxpi.le]
  rfl
have hB3 :
  LeanSuffixAnalytic.typeThreeAngle (lam b p)
    ((1 +  $\cos (w \ b \ p)$ ) / 2) +  $\pi / 2 =$  B3 b p := by
  rw [LeanSuffixAnalytic.typeThreeAngle_add_halfPi, hshape, hcos_wv_div,
    Real.arccos_cos hv0.le hvpi.le, Real.arccos_cos hw0.le hwpi.le]
  rfl
have hsqrt_x :  $\sqrt{1 - \cos (x \ b \ p)^2} = \sin (x \ b \ p) :=$  by
  have htrig := Real.sin_sq_add_cos_sq (x b p)
  rw [show  $1 - \cos (x \ b \ p)^2 = \sin (x \ b \ p)^2$  by linarith,
    Real.sqrt_sq_eq_abs, abs_of_pos hxsin_pos]
have hsqrt_w :  $\sqrt{1 - \cos (w \ b \ p)^2} = \sin (w \ b \ p) :=$  by
  have htrig := Real.sin_sq_add_cos_sq (w b p)
  rw [show  $1 - \cos (w \ b \ p)^2 = \sin (w \ b \ p)^2$  by linarith,
    Real.sqrt_sq_eq_abs, abs_of_pos hwsin_pos]
have hsqrt_lam_x :
   $\sqrt{(\text{lam } b \ p)^2 - \cos (x \ b \ p)^2} = \text{lam } b \ p * \sin (y \ b \ p) :=$  by
  have htrig := Real.sin_sq_add_cos_sq (y b p)
  rw [show  $(\text{lam } b \ p)^2 - \cos (x \ b \ p)^2 =$ 
     $(\text{lam } b \ p * \sin (y \ b \ p))^2$  by
    rw [hcos_xy]
    linear_combination -((lam b p) ^ 2 * htrig,
      Real.sqrt_sq_eq_abs, abs_of_pos (mul_pos hlam_pos hysin_pos))]
have hsqrt_lam_w :
   $\sqrt{(\text{lam } b \ p)^2 - \cos (w \ b \ p)^2} = \text{lam } b \ p * \sin (v \ b \ p) :=$  by
  have htrig := Real.sin_sq_add_cos_sq (v b p)
  rw [show  $(\text{lam } b \ p)^2 - \cos (w \ b \ p)^2 =$ 
     $(\text{lam } b \ p * \sin (v \ b \ p))^2$  by
    rw [hcos_wv]
    linear_combination -((lam b p) ^ 2 * htrig,
      Real.sqrt_sq_eq_abs, abs_of_pos (mul_pos hlam_pos hvsin_pos))]

```

```

have hD4 :
  LeanSuffixAnalytic.typeFourDelta (lam b p) (cos (x b p)) = D4 b p := by
  rw [LeanSuffixAnalytic.typeFourDelta, hsqrt_lam_x, hsqrt_x]
  field_simp [ne_of_gt hlam_pos]
  rfl
have hD3 :
  LeanSuffixAnalytic.typeThreeDelta (lam b p)
    ((1 + cos (w b p)) / 2) = D3 b p := by
  rw [LeanSuffixAnalytic.typeThreeDelta, hshape, hsqrt_lam_w, hsqrt_w]
  field_simp [ne_of_gt hlam_pos]
  rfl
have hh3_pos : 0 < (1 + cos (w b p)) / 2 := by linarith
have hh4_pos : 0 < cos (x b p) := hxcos_pos
have equalArea_of_E_zero (hE : E b p = 0) :
  LeanSuffixAnalytic.typeThreeArea (lam b p) ((1 + cos (w b p)) / 2) =
    LeanSuffixAnalytic.typeFourArea (lam b p) (cos (x b p)) := by
  rw [LeanSuffixAnalytic.typeThreeArea, LeanSuffixAnalytic.typeFourArea,
    hshape, hB3, hD3, hB4, hD4]
  apply (div_eq_div_iff (pow_ne_zero 2 (ne_of_gt hh3_pos))
    (pow_ne_zero 2 (ne_of_gt hh4_pos))).2
  unfold E N3 N4 at hE
  linear_combination (1 / 2 : ℝ) * hE
have stationary_of_F_zero (hF : F b p = 0) :
  LeanSuffixAnalytic.typeFourFold (lam b p) (cos (x b p)) = 0 := by
  have hden_ne : sin (x b p) * sin (y b p) ≠ 0 :=
    mul_ne_zero (ne_of_gt hxsin_pos) (ne_of_gt hysin_pos)
  have hnumerator :
    cos (x b p) * (sin (y b p) - sin (x b p)) =
      sin (x b p) * sin (y b p) * B4 b p := by
    unfold Model.F D4 at hF
    linarith
  have hcore :
    cos (x b p) * (1 / sin (x b p) - 1 / sin (y b p)) = B4 b p := by
    calc
      cos (x b p) * (1 / sin (x b p) - 1 / sin (y b p)) =
        (cos (x b p) * (sin (y b p) - sin (x b p))) /
          (sin (x b p) * sin (y b p)) := by
          field_simp [ne_of_gt hxsin_pos, ne_of_gt hysin_pos]
      _ = B4 b p := (div_eq_iff hden_ne).2 (by
        simpa [mul_assoc, mul_comm, mul_left_comm] using hnumerator)
  have hcancel : lam b p / (lam b p * sin (y b p)) =
    1 / sin (y b p) := by
    field_simp [ne_of_gt hlam_pos, ne_of_gt hysin_pos]
  rw [LeanSuffixAnalytic.typeFourFold, hsqrt_x, hsqrt_lam_x, hB4,
    hcancel, hcore]
  ring
have perimeter_lt_of_G_neg (hG : G b p < 0) :
  LeanSuffixAnalytic.typeThreePerimeter (lam b p)
    ((1 + cos (w b p)) / 2) <
    LeanSuffixAnalytic.typeFourPerimeter (lam b p) (cos (x b p)) := by
  rw [LeanSuffixAnalytic.typeThreePerimeter,
    LeanSuffixAnalytic.typeFourPerimeter, hB3, hD3, hB4]
  apply (div_lt_div_iff₀ hh3_pos hh4_pos).2
  unfold G at hG
  nlinarith
exact ⟨by linarith, by linarith, hxcos_pos, hxcos_lt_one,
  equalArea_of_E_zero, stationary_of_F_zero, perimeter_lt_of_G_neg⟩

/-- The checker `K3` is the numerator of the actual type-(iii) area
derivative on the certified branch. -/

```

MiddleFaceAssembly.typeThreeArea_deriv_eq_K3

Classification: main

Source: MiddleFaceAssembly.lean:1701-1774

Declaration SHA-256: a51562aaac4e05b4f1f95d421868e79ec4eae70d59c9a31981d5897af5fec042

Mathematical statement. The checker K3 divided by the positive curvature cube is the type-(iii) area derivative.

On the incidence and branch hypotheses, the sine radicals agree with the analytic derivative expression.

```

theorem typeThreeArea_deriv_eq_K3 (b : Brick) (p : Point)
  (h1am : 1 < lam b p)
  (hbranches :
     $0 < w b p \wedge w b p < \pi / 4 \wedge 0 < v b p \wedge v b p < \pi / 4$ )
  (hC3 : C3 b p = 0) :
  deriv (LeanSuffixAnalytic.typeThreeArea (lam b p))
    ((1 + cos (w b p)) / 2) =
    K3 b p / ((1 + cos (w b p)) / 2) ^ 3 := by
  rcases hbranches with ⟨hw0, hwq, hv0, hvq⟩
  have hpi4_lt_pi :  $\pi / 4 < \pi$  := by linarith [Real.pi_pos]
  have hpi4_lt_pi_div_two :  $\pi / 4 < \pi / 2$  := by
    linarith [Real.pi_pos]
  have hwpi :  $w b p < \pi$  := hwq.trans hpi4_lt_pi
  have hvpi :  $v b p < \pi$  := hvq.trans hpi4_lt_pi
  have hlam_pos :  $0 < lam b p$  := lt_trans (by norm_num) h1am
  have hwcosp :  $0 < \cos (w b p)$  :=
    Real.cos_pos_of_mem_Ioo (by linarith [Real.pi_pos],
      hwq.trans hpi4_lt_pi_div_two)
  have hzero_mem :  $(0 : \mathbb{R}) \in Icc\ 0\ \pi$  := ⟨le_refl, Real.pi_pos.le⟩
  have hw_mem :  $w b p \in Icc\ 0\ \pi$  := ⟨hw0.le, hwpi.le⟩
  have hwcosp_one :  $\cos (w b p) < 1$  := by
    simp using Real.strictAntiOn_cos hzero_mem hw_mem hw0
  have hwsin_pos :  $0 < \sin (w b p)$  :=
    Real.sin_pos_of_pos_of_lt_pi hw0 hwpi
  have hvsin_pos :  $0 < \sin (v b p)$  :=
    Real.sin_pos_of_pos_of_lt_pi hv0 hvpi
  have hcos_wv :  $\cos (w b p) = lam b p * \cos (v b p)$  := by
    unfold C3 at hC3
    linarith
  have hshape :
    LeanSuffixAnalytic.typeThreeShape ((1 + cos (w b p)) / 2) =
      cos (w b p) := by
    unfold LeanSuffixAnalytic.typeThreeShape
    ring
  have hcos_wv_div :  $\cos (w b p) / lam b p = \cos (v b p)$  := by
    rw [hcos_wv]
    field_simp [ne_of_gt hlam_pos]
  have hB3 :
    LeanSuffixAnalytic.typeThreeAngle (lam b p)
      ((1 + cos (w b p)) / 2) +  $\pi / 2$  = B3 b p := by
    rw [LeanSuffixAnalytic.typeThreeAngle_add_halfPi, hshape, hcos_wv_div,
      Real.arccos_cos hv0.le hvpi.le, Real.arccos_cos hw0.le hwpi.le]
    rfl
  have hsqrt_w :  $\sqrt{1 - \cos (w b p)^2} = \sin (w b p)$  := by
    have htrig := Real.sin_sq_add_cos_sq (w b p)
    rw [show  $1 - \cos (w b p)^2 = \sin (w b p)^2$  by linarith,
      Real.sqrt_sq_eq_abs, abs_of_pos hwsin_pos]
  have hsqrt_lam_w :
     $\sqrt{lam b p^2 - \cos (w b p)^2} = lam b p * \sin (v b p)$  := by
    have htrig := Real.sin_sq_add_cos_sq (v b p)
    rw [show  $lam b p^2 - \cos (w b p)^2 =$ 
       $(lam b p * \sin (v b p))^2$  by
      rw [hcos_wv]
      linear_combination -(lam b p)^2 * htrig,
      Real.sqrt_sq_eq_abs, abs_of_pos (mul_pos hlam_pos hvsin_pos)]
  have hD3 :
    LeanSuffixAnalytic.typeThreeDelta (lam b p)
      ((1 + cos (w b p)) / 2) = D3 b p := by
    rw [LeanSuffixAnalytic.typeThreeDelta, hshape, hsqrt_lam_w, hsqrt_w]
    field_simp [ne_of_gt hlam_pos]
    rfl
  have hh3_pos :  $0 < (1 + \cos (w b p)) / 2$  := by linarith
  have hh3_one :  $(1 + \cos (w b p)) / 2 < 1$  := by linarith
  rw [LeanSuffixAnalytic.typeThreeArea_deriv_eq_explicit
    hlam hh3_pos hh3_one, hshape, hsqrt_w, hsqrt_lam_w, hB3, hD3]
  unfold K3
  dsimp only
  field_simp [ne_of_gt hlam_pos, ne_of_gt hwsin_pos, ne_of_gt hvsin_pos]
  ring

```

```
namespace Conditions

variable {b : Brick}
```

MiddleFaceCells.candidate_not_isWeightedPerimeterMinimizer

Classification: main

Source: MiddleFaceCells.lean:29-74

Declaration SHA-256: 65047cf47373d68e2f54c04db5fee6610caf78e9eb55ac3e2777e49e93efefff1

Mathematical statement. The generated middle-face cells exclude candidates from 102001/100000 to 25781/25000.

The nine batch coverage theorems join the 113 exact closed cell intervals without gaps.

```
theorem candidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ}
  (hlower : (102001 / 100000 : ℝ) ≤ lam)
  (hupper : lam ≤ (25781 / 25000 : ℝ))
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  by_cases hbatch₀ : lam ≤ (102201 / 100000 : ℝ)
  · exact
    MiddleFaceCells0To19.candidate_not_isWeightedPerimeterMinimizer
      hlower hbatch₀ candidate hcandidate
  · by_cases hbatch₁ : lam ≤ (102321 / 100000 : ℝ)
    · exact
      MiddleFaceCells20To31.candidate_not_isWeightedPerimeterMinimizer
        (not_le.mp hbatch₀).le hbatch₁ candidate hcandidate
    · by_cases hbatch₂ : lam ≤ (102441 / 100000 : ℝ)
      · exact
        MiddleFaceCells32To43.candidate_not_isWeightedPerimeterMinimizer
          (not_le.mp hbatch₁).le hbatch₂ candidate hcandidate
      · by_cases hbatch₃ : lam ≤ (102561 / 100000 : ℝ)
        · exact
          MiddleFaceCells44To55.candidate_not_isWeightedPerimeterMinimizer
            (not_le.mp hbatch₂).le hbatch₃ candidate hcandidate
        · by_cases hbatch₄ : lam ≤ (102681 / 100000 : ℝ)
          · exact
            MiddleFaceCells56To67.candidate_not_isWeightedPerimeterMinimizer
              (not_le.mp hbatch₃).le hbatch₄ candidate hcandidate
          · by_cases hbatch₅ : lam ≤ (102801 / 100000 : ℝ)
            · exact
              MiddleFaceCells68To79.candidate_not_isWeightedPerimeterMinimizer
                (not_le.mp hbatch₄).le hbatch₅ candidate hcandidate
            · by_cases hbatch₆ : lam ≤ (102921 / 100000 : ℝ)
              · exact
                MiddleFaceCells80To91.candidate_not_isWeightedPerimeterMinimizer
                  (not_le.mp hbatch₅).le hbatch₆ candidate hcandidate
              · by_cases hbatch₇ : lam ≤ (103041 / 100000 : ℝ)
                · exact
                  MiddleFaceCells92To103.candidate_not_isWeightedPerimeterMinimizer
                    (not_le.mp hbatch₆).le hbatch₇ candidate hcandidate
                · exact
                  MiddleFaceCells104To112.candidate_not_isWeightedPerimeterMinimizer
                    (not_le.mp hbatch₇).le hupper candidate hcandidate

/-- Every modeled type-(iv) candidate in both retained prototypes and all 113
middle-face bricks fails weighted-perimeter minimality. The closed interval
includes both exact prototype seams and reaches `33 / 32`. -/
```

MiddleFaceCells.candidate_not_isWeightedPerimeterMinimizer_from_51_50

Classification: main

Source: MiddleFaceCells.lean:75-102

Declaration SHA-256: 4e139d2067a97ded95a1f47d09a5d5b68e8fd03114a4a32d5431bb047f26e9f1

Mathematical statement. The two prototypes and middle-face cells exclude candidates on $[51/50, 33/32]$.

The first prototype, middle family, and right prototype meet at $102001/100000$ and $25781/25000$.

```
theorem candidate_not_isWeightedPerimeterMinimizer_from_51_50
  {lam : ℝ}
  (hlower : (51 / 50 : ℝ) ≤ lam)
  (hupper : lam ≤ (33 / 32 : ℝ))
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  by_cases hp : lam ≤ (102001 / 100000 : ℝ)
  · have hcell :
      ScalarSuffixCertificate.FirstPrototypeCell.lamInterval.RealContains lam := by
      norm_num [ScalarSuffixCertificate.FirstPrototypeCell.lamInterval,
        ScalarSuffixCertificate.FirstPrototypeCell.brick,
        LeanSuffixReflective.QInterval.RealContains] at hlower hp ⊢
      exact ⟨hlower, hp⟩
    exact
      ScalarSuffixCertificate.FirstPrototypeCell.candidate_not_isWeightedPerimeterMinimizer
        hcell candidate hcandidate
  · by_cases hm : lam ≤ (25781 / 25000 : ℝ)
    · exact candidate_not_isWeightedPerimeterMinimizer (not_le.mp hp).le hm
      candidate hcandidate
    · have hcell : RightPrototypeCell.lamInterval.RealContains lam := by
        norm_num [RightPrototypeCell.lamInterval, RightPrototypeCell.brick,
          LeanSuffixReflective.QInterval.RealContains] at hm hupper ⊢
        exact ⟨hm.le, hupper⟩
      exact RightPrototypeCell.candidate_not_isWeightedPerimeterMinimizer
        hcell candidate hcandidate

end MiddleFaceCells
```

OneSidedCircularCap

Classification: main

Source: CMVGeometry.lean:100-112

Declaration SHA-256: a9dd2ba7ee653be367b6a2072de1384c3362b0214117e3aecceae763bc6f8403

Mathematical statement. A circular segment specified by a positive chord, angle in $(0, \pi)$, placement, and side.

The disk-half-plane carrier includes minor, semicircular, and major caps; no minor-cap restriction is implicit.

```
structure OneSidedCircularCap where
  chord : ℝ
  theta : ℝ
  midpointX : ℝ
  baseY : ℝ
  side : CapSide
  chord_pos : 0 < chord
  theta_pos : 0 < theta
  theta_lt_pi : theta < π

namespace OneSidedCircularCap

/-- Radius determined by chord length and half-central-angle. -/
```

OneSidedCircularCap.arcLength

Classification: main

Source: CMVGeometry.lean:167-170

Declaration SHA-256: 60daa6fa20eb20410aecf4ceebd5a41e50c48a0eb958e6b55f1bcb20cd85ff25

Mathematical statement. The circular arc length expression.

It is $c\ell(\theta) = 2R\theta$, excluding the chord.

```
def arcLength (c : OneSidedCircularCap) : ℝ := c.chord * ell c.theta

/-- The branch-correct cap with prescribed positive normalized area. This is
the reusable geometric constructor behind the candidate-specific replacement. -/
```

OneSidedCircularCap.carrier

Classification: main

Source: CMVGeometry.lean:146-152

Declaration SHA-256: e0644dd0b5f330958b79efbb3810eb1cc2f4356e5db251a95736b532b75bfcf1

Mathematical statement. The closed disk intersected with the selected chord half-plane.

The definition is branch-correct for every $0 < \theta < \pi$.

```
def carrier (c : OneSidedCircularCap) : Set PlanePoint :=
  {p | c.radiusSquaredAt p ≤ c.radius ^ 2 ∧
    match c.side with
    | .upper => c.baseY ≤ p.2
    | .lower => p.2 ≤ c.baseY}

/-- The open cap interior. Endpoints and the chord are deliberately absent. -/
```

OneSidedCircularCap.euclideanArea

Classification: main

Source: CMVGeometry.lean:164-166

Declaration SHA-256: 5c476b88f35c9e9f6afc55ed7ed9ef6be1395a7b5a3003452ff8691443c49da

Mathematical statement. The circular-segment area expression.

The expression is $c^2a(\theta) = R^2(\theta - \sin\theta\cos\theta)$; equality with volume is proved separately.

```
def euclideanArea (c : OneSidedCircularCap) : ℝ := c.chord ^ 2 * area c.theta

/-- Euclidean length of the selected circular arc. -/
```

OneSidedCircularCap.frontier_carrier

Classification: main

Source: CMVGeometry.lean:697-761

Declaration SHA-256: d9cb2c749721b51457faae17a95033cf908527d3fdc652ebedacfb2292d579d

Mathematical statement. The cap frontier is exactly its arc trace union its chord.

Both pieces are exposed for an isolated cap; glued assemblies require a separate frontier calculation.

```
theorem frontier_carrier (c : OneSidedCircularCap) :
  frontier c.carrier = arcTrace c ∪ c.chordCarrier := by
```

```

apply Subset.antisymm
· intro p hp
  have hpc : p ∈ c.carrier :=
    c.isClosed_carrier.frontier_subset hp
  by_cases heq : c.radiusSquaredAt p = c.radius ^ 2
  · exact Or.inl (heq, hpc.2)
  have hlt : c.radiusSquaredAt p < c.radius ^ 2 :=
    lt_of_le_of_ne hpc.1 heq
  cases hside : c.side with
  | upper =>
    have hsidep : c.baseY ≤ p.2 := by
      simpa [OneSidedCircularCap.carrier, hside] using hpc.2
    by_cases hy : p.2 = c.baseY
    · exact Or.inr
      (c.mem_chordCarrier_of_mem_carrier_of_eq_base hpc hy)
    · exfalso
      have hi : p ∈ c.interiorCarrier := by
        simpa [OneSidedCircularCap.interiorCarrier, hside] using
          (hlt, lt_of_le_of_ne hsidep (Ne.symm hy))
      exact (mem_frontier_iff_notMem_interior hpc).mp hp
        (interiorCarrier_subset_interior c hi)
  | lower =>
    have hsidep : p.2 ≤ c.baseY := by
      simpa [OneSidedCircularCap.carrier, hside] using hpc.2
    by_cases hy : p.2 = c.baseY
    · exact Or.inr
      (c.mem_chordCarrier_of_mem_carrier_of_eq_base hpc hy)
    · exfalso
      have hi : p ∈ c.interiorCarrier := by
        simpa [OneSidedCircularCap.interiorCarrier, hside] using
          (hlt, lt_of_le_of_ne hsidep hy)
      exact (mem_frontier_iff_notMem_interior hpc).mp hp
        (interiorCarrier_subset_interior c hi)
· rintro p (hp | hp)
· have hpc : p ∈ c.carrier := (hp.1.le, hp.2)
  rw [mem_frontier_iff_notMem_interior hpc]
  intro hi
  apply circle_eq_not_mem_interior_disk
    c.center.1 c.center.2 c.radius c.radius_pos p
    (by simpa [OneSidedCircularCap.radiusSquaredAt] using hp.1)
  apply interior_mono _ hi
  intro q hq
  exact hq.1
· have hpc := c.chordCarrier_subset_carrier hp
  rw [mem_frontier_iff_notMem_interior hpc]
  intro hi
  cases hside : c.side with
  | upper =>
    apply second_eq_not_mem_interior_ge c.baseY p hp.1
    apply interior_mono _ hi
    intro q hq
    simpa [OneSidedCircularCap.carrier, hside] using hq.2
  | lower =>
    apply second_eq_not_mem_interior_le c.baseY p hp.1
    apply interior_mono _ hi
    intro q hq
    simpa [OneSidedCircularCap.carrier, hside] using hq.2

end OneSidedCircularCap

namespace StripCore

/-- The exact left circular trace of a strip core. -/

```

OneSidedCircularCap.radius

Classification: main

Source: CMVGeometry.lean:113-116

Declaration SHA-256: e898cc7066a75a606fdc3b1d1e563939ad569bdb0f66fd289569e58c7f4c44b8

Mathematical statement. Radius determined by the chord and half-angle.

$R = c/(2\sin\theta)$ for $c > 0$ and $0 < \theta < \pi$.

```
def radius (c : OneSidedCircularCap) : ℝ := c.chord / (2 * sin c.theta)

/-- The center lies below an upper chord and above a lower chord. For a major
cap `cos θ < 0`, so the center crosses to the cap side, as it should. -/
```

OneSidedCircularCap.volume_carrier

Classification: main

Source: CMVGeometry.lean:1996-2028

Declaration SHA-256: 0d51f9d06b45af9eac50b2062d616aee51a260200ad5a2e96d2ecbe8e98be85c

Mathematical statement. The cap carrier's Lebesgue volume equals its Euclidean area expression.

This proves that $c^2a(\theta)$ measures the actual disk-half-plane carrier.

```
theorem volume_carrier (c : OneSidedCircularCap) :
  volume c.carrier = ENNReal.ofReal c.euclideanArea := by
  have hreal :
    c.radius ^ 2 * (c.theta - sin c.theta * cos c.theta) =
      c.euclideanArea := by
    rw [euclideanArea, area, radius]
    field_simp [ne_of_gt c.sin_theta_pos]
    ring
  cases hside : c.side with
  | upper =>
    calc
      volume c.carrier =
        ENNReal.ofReal
          (c.radius ^ 2 *
            (c.theta - sin c.theta * cos c.theta)) := by
      simpa [carrier, radiusSquaredAt, center, hside] using
        volume_upper_circular_segment c.midpointX
          (c.baseY - c.radius * cos c.theta) c.radius c.theta
          c.radius_pos c.theta_pos.le c.theta_lt_pi.le
      _ = ENNReal.ofReal c.euclideanArea := congrArg ENNReal.ofReal hreal
  | lower =>
    calc
      volume c.carrier =
        ENNReal.ofReal
          (c.radius ^ 2 *
            (c.theta - sin c.theta * cos c.theta)) := by
      simpa [carrier, radiusSquaredAt, center, hside] using
        volume_lower_circular_segment c.midpointX
          (c.baseY + c.radius * cos c.theta) c.radius c.theta
          c.radius_pos c.theta_pos.le c.theta_lt_pi.le
      _ = ENNReal.ofReal c.euclideanArea := congrArg ENNReal.ofReal hreal

end OneSidedCircularCap
```

PlanePoint

Classification: main

Source: CMVGeometry.lean:30-33

Declaration SHA-256: b6c33ab7d9e6652b4833188977c0694ee665f31ebd07e572ea7a67ee7a8a0183

Mathematical statement. The coordinate plane is the product of two real lines.

A point is $(x, y) \in \mathbb{R}^2$; carriers are subsets of this product.

```
abbrev PlanePoint := ℝ × ℝ

/-- The Euclidean L2 realization used to form geometric Hausdorff measure.
`PlanePoint` keeps its coordinate topology and all existing carrier definitions. -/
```

ReplacementAssembly

Classification: main

Source: CMVGeometry.lean:3281-3288

Declaration SHA-256: e0b73790946058b82b8d6a4bdde47ad7560092e26acf3d58cb0c91dc0cfa445f

Mathematical statement. A retained strip core with one upper cap and an exposed lower chord.

The upper cap angle may lie anywhere in $(0, \pi)$, allowing area doubling beyond a minor cap.

```
structure ReplacementAssembly where
  core : StripCore
  replacementAngle : ℝ
  replacementAngle_pos : 0 < replacementAngle
  replacementAngle_lt_pi : replacementAngle < π

namespace ReplacementAssembly
```

ReplacementAssembly.boundaryTrace

Classification: main

Source: CMVGeometry.lean:3487-3492

Declaration SHA-256: c14f256441519af3907ade461995fd9aa134a8b3a94b950059f337d7edd4be97

Mathematical statement. The replacement's two side arcs, upper arc, and lower chord.

This trace union includes every proposed exposed boundary piece.

```
def boundaryTrace (a : ReplacementAssembly) : Set PlanePoint :=
  StripCore.leftArcTrace a.core ∪
  (StripCore.rightArcTrace a.core ∪
   (OneSidedCircularCap.arcTrace a.upperCap ∪
    a.exposedLowerChord.carrier))
```

ReplacementAssembly.carrier

Classification: main

Source: CMVGeometry.lean:3305-3309

Declaration SHA-256: 36b80491dc0fecbf358d87c6bfe7194faf4bcad4ac91372874e8af60a30fd23e

Mathematical statement. Union of the retained core and the replacement upper cap.

There is no exterior lower cap in this carrier.

```
def carrier (a : ReplacementAssembly) : Set PlanePoint :=
  a.core.carrier ∪ a.upperCap.carrier

/-- Explicit pushforward measure of the two retained side arcs, inserted upper
arc, and genuinely exposed lower chord. -/
```

ReplacementAssembly.exposedLowerChord

Classification: main

Source: CMVGeometry.lean:3299-3304

Declaration SHA-256: f5b2acb16292464686c2dc7d4da985f73189845b89d73512b7e850cb9f2f8b26

Mathematical statement. The lower horizontal segment left exposed after removing the lower cap.

It lies on $y = -1$, has length c , and carries density one.

```
def exposedLowerChord (a : ReplacementAssembly) : HorizontalSegment where
  chord := a.core.chord
  midpointX := 0
  baseY := -1
  chord_pos := a.core.chord_pos
```

ReplacementAssembly.exposedLowerChord_subset_frontier

Classification: main

Source: CMVGeometry.lean:3460-3481

Declaration SHA-256: f8eb7dd82f960c22028fc1d5bb6797612d50aa3f62628a630157cdc32d61c499

Mathematical statement. The entire exposed lower chord lies on the complete frontier.

Carrier membership and arbitrarily close exterior points establish its non-optional perimeter contribution.

```
theorem exposedLowerChord_subset_frontier (a : ReplacementAssembly) :
  a.exposedLowerChord.carrier ⊆ frontier a.carrier := by
  intro p hp
  rw [mem_frontier_iff_notMem_interior
    (a.exposedLowerChord_subset_carrier hp)]
  intro hinterior
  have hnhds : interior a.carrier ∈ nhds p :=
    isOpen_interior.mem_nhds hinterior
  rcases Metric.mem_nhds_iff.mp hnhds with
    ⟨epsilon, hepsilon, hball⟩
  have hhalf : 0 < epsilon / 2 := div_pos hepsilon (by norm_num)
  have hnear :
    (p.1, p.2 - epsilon / 2) ∈ Metric.ball p epsilon := by
    rw [Metric.mem_ball, Prod.dist_eq, dist_self, Real.dist_eq,
      show p.2 - epsilon / 2 - p.2 = -(epsilon / 2) by ring,
      abs_neg, abs_of_pos hhalf, max_eq_right hhalf.le]
    linarith
  have hmember :
    (p.1, p.2 - epsilon / 2) ∈ a.carrier :=
    interior_subset (hball hnear)
  exact a.below_exposedLowerChord_not_mem hp hhalf hmember
/-- Every interior point of the inserted upper cap lies in `y > 1`. -/
```

ReplacementAssembly.frontier_carrier

Classification: main

Source: CMVGeometry.lean:3941-3951

Declaration SHA-256: 5eb2c9bf9d9bf270c9c126108b477ade628695b91787f859535adefef6f41836a

Mathematical statement. The replacement frontier equals its complete trace union.

This exact identity underlies the perimeter difference used in the high-density argument.

```
theorem frontier_carrier (a : ReplacementAssembly) :
  frontier a.carrier = a.boundaryTrace :=
  Set.Subset.antisymm a.frontier_carrier_subset_boundaryTrace
    a.boundaryTrace_subset_frontier_carrier

end ReplacementAssembly

/-- Admissible competitors include arbitrary finite-perimeter regions, as well
as the two piecewise-circular constructors whose exact formulas are proved in
the bridge. The replacement constructor is not restricted by stationarity,
Snell, or equal-curvature conditions. -/
```

ReplacementAssembly.upperCap

Classification: main

Source: CMVGeometry.lean:3289-3298

Declaration SHA-256: b30d86506333a29d0fea0a0c213d5ab5e404eb8902dd90c6019d57dad3b8f56f

Mathematical statement. The replacement's upper cap on the core chord.

It is based at $y = 1$ on the chosen full angle branch.

```
def upperCap (a : ReplacementAssembly) : OneSidedCircularCap where
  chord := a.core.chord
  theta := a.replacementAngle
  midpointX := 0
  baseY := 1
  side := .upper
  chord_pos := a.core.chord_pos
  theta_pos := a.replacementAngle_pos
  theta_lt_pi := a.replacementAngle_lt_pi
```

ReplacementAssembly.weightedArea

Classification: main

Source: CMVGeometry.lean:3326-3329

Declaration SHA-256: 30517c93164485b2476e45ea547b19187ad53db44a33af2bf70144ca4c93c662

Mathematical statement. Density integral over the replacement carrier.

It is determined by the union of core and upper cap.

```
def weightedArea (lam : ℝ) (a : ReplacementAssembly) : ℝ :=
  WeightedArea lam a.carrier

/-- Density integral against the complete explicit replacement boundary. -/
```

ReplacementAssembly.weightedArea_formula

Classification: main

Source: CMVGeometry.lean:3338-3362

Declaration SHA-256: f1ba4256fd2f5c3e87624e59042924594647c2f49dcba6ed06cd14dc8d287789

Mathematical statement. Replacement area is core area plus lambda times upper-cap area.

The common upper chord has zero planar area.

```
theorem weightedArea_formula (lam : ℝ) (a : ReplacementAssembly) :
  a.weightedArea lam =
```

```

    a.core.euclideanArea + lam * a.upperCap.euclideanArea := by
have hcoreUpper : AEDisjoint volume a.core.carrier a.upperCap.carrier := by
  refine measure_mono_null ?_ (volume_horizontalline 1)
  rintro p ⟨hcore, hupper⟩
  have hyCore := a.core.carrier_y_bounds hcore
  have hyUpper : (1 : ℝ) ≤ p.2 := by
    simpa [upperCap] using hupper.2
  exact le_antisymm (le_trans (le_abs_self p.2) hyCore) hyUpper
have hcore := core_integrableOn lam a.core
have hupper := cap_integrableOn lam a.upperCap
rw [weightedArea, WeightedArea, carrier,
  setIntegral_union₀ hcoreUpper
  a.upperCap.measurableSet_carrier.nullMeasurableSet hcore hupper]
rw [show (f p in a.core.carrier, StripDensity lam p) =
  a.core.euclideanArea by
  simpa [WeightedArea] using a.core.weightedArea_formula lam,
show (f p in a.upperCap.carrier, StripDensity lam p) =
  lam * a.upperCap.euclideanArea by
  simpa [WeightedArea] using
  cap_upper_weightedArea lam a.upperCap rfl rfl]

/-- The boundary integral equals the two side arcs, inserted cap arc, and
exposed lower chord formula. -/

```

ReplacementAssembly.weightedPerimeter_formula

Classification: main

Source: CMVGeometry.lean:3363-3387

Declaration SHA-256: 0ec330da63b1183620a69c25b9b52f69e166e1fbc55093558ecffbfdc73ed619

Mathematical statement. Replacement perimeter includes side arcs, weighted upper arc, and lower chord.

The exposed lower chord contributes its full length at density one.

```

theorem weightedPerimeter_formula (lam : ℝ) (a : ReplacementAssembly) :
  a.weightedPerimeter lam =
    a.core.boundaryArcLength + lam * a.upperCap.arcLength +
    a.exposedLowerChord.euclideanLength := by
rw [weightedPerimeter, WeightedPerimeter, boundaryMeasure]
rw [integral_add_measure
  (((leftCoreBoundaryMeasure_integrable lam a.core).add_measure
    (rightCoreBoundaryMeasure_integrable lam a.core)).add_measure
    (capBoundaryMeasure_integrable lam a.upperCap))
  (segmentBoundaryMeasure_integrable lam a.exposedLowerChord)]
rw [integral_add_measure
  ((leftCoreBoundaryMeasure_integrable lam a.core).add_measure
    (rightCoreBoundaryMeasure_integrable lam a.core))
  (capBoundaryMeasure_integrable lam a.upperCap)]
rw [integral_add_measure
  (leftCoreBoundaryMeasure_integrable lam a.core)
  (rightCoreBoundaryMeasure_integrable lam a.core)]
rw [leftCore_weightedIntegral, rightCore_weightedIntegral,
  cap_upper_weightedIntegral lam a.upperCap rfl rfl,
  segment_weightedIntegral lam a.exposedLowerChord (by
    simp [exposedLowerChord]),
  StripCore.boundaryArcLength]
ring

/-- The cap closure and retained strip core can meet only on `y = 1`. -/

```

RightPrototypeCell.candidate_not_isWeightedPerimeterMinimizer

Classification: main

Source: RightPrototypeCell.lean:2222-2240

Declaration SHA-256: 5c6a266d071e9cf7ce0214b6d5870976fd096dbfbf585b5acb5f6cd350590871

Mathematical statement. Every modeled candidate is nonminimal on [25781/25000,33/32].

The right brick supplies a stationary pair and the all-curvature envelope supplies the actual competitor.

```
theorem candidate_not_isWeightedPerimeterMinimizer
  {lam0 : ℝ} (hlam : lamInterval.RealContains lam0)
  (candidate : _root_.FourArcCandidate lam0)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  have ht0 : tInterval.RealContains (lam0 - (lamMid : ℝ)) :=
    lambda_iff_centered.mp hlam
  have ht : (MiddleFaceAssembly.tInterval brick).RealContains
    (lam0 - (MiddleFaceAssembly.lamMid brick : ℝ)) := by
    change tInterval.RealContains (lam0 - (lamMid : ℝ))
    exact ht0
  have hcenter :
    (MiddleFaceAssembly.lamMid brick : ℝ) +
    (lam0 - (MiddleFaceAssembly.lamMid brick : ℝ)) = lam0 := by
    ring
  exact
    MiddleFaceAssembly.Conditions.candidate_not_isWeightedPerimeterMinimizer
      checkedConditions ht hcenter candidate hcandidate
end RightPrototypeCell
```

ScalarSuffixCertificate.CertificateExpr.sound

Classification: main

Source: ScalarSuffixCertificate.lean:191-200

Declaration SHA-256: 7903a28236dd3b57076df8e33c74ea38e4b9e0dfa4dd8e31c52c47cb44f7b8cc

Mathematical statement. Every certificate expression's real value belongs to its computed enclosure.

Structural induction combines the primitive operation-containment theorems; arithmetic certificates therefore imply real statements.

```
theorem sound (e : CertificateExpr) : e.enclosure.RealContains e.value := by
  induction e with
  | rational q => exact
    (LeanSuffixReflective.QInterval.realContains_point q (q : ℝ)).2 rfl
  | atom x i h => exact h
  | add a b ha hb => exact LeanSuffixReflective.QInterval.realContains_add ha hb
  | neg a ha => exact LeanSuffixReflective.QInterval.realContains_neg ha
  | mul a b ha hb => exact QInterval.realContains_mul ha hb

/-- Checked negativity proves a proposition about the represented real value. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.AnalyticFaceBounds

Classification: main

Source: ScalarSuffixCertificate.lean:3244-3249

Declaration SHA-256: 55d57a38a3bc2fde4dddd82f77d73cfb2e9ed05adcef3d85713a2b783119493f

Mathematical statement. The first prototype's analytic opposite-face sign interface.

It packages the low and high root-map signs used by the explicit root construction.

```
structure AnalyticFaceBounds : Prop where
  lowFaces : ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    ∀ j, z j = -radiiReal j → rootMap t z j ≤ 0
  highFaces : ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    ∀ j, z j = radiiReal j → 0 ≤ rootMap t z j
```

ScalarSuffixCertificate.FirstPrototypeCell.Brick

Classification: main

Source: ScalarSuffixCertificate.lean:425-436

Declaration SHA-256: fe061415d65cea6f77fb8c663f249825b26067221bd27165829e3e0950822743

Mathematical statement. The first prototype's rational density and affine-predictor data record.

It stores exact density endpoints, angular centers and slopes, couplings, and positive error radii.

```
structure Brick where
  lamLo : ℚ
  lamHi : ℚ
  centers : Fin 4 → ℚ
  slopes : Fin 4 → ℚ
  yXiSlope : ℚ
  vRhoSlope : ℚ
  radii : Fin 4 → ℚ
  lamOrdered : lamLo < lamHi
  radiiPos : ∀ j, 0 < radii j

/-- Representative `[51/50, 102001/100000]` exact-data brick. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.C3

Classification: main

Source: ScalarSuffixCertificate.lean:512-512

Declaration SHA-256: bb334eb31664c80305d156013a66d4b3847b5f60bc740d68b3db43dd6612439b

Mathematical statement. The first prototype's type-(iii) incidence residual.

$C_3 = \cos w - \lambda \cos v$.

```
def C3 (p : Point) : ℝ := cos (w p) - lam p * cos (v p)
```

ScalarSuffixCertificate.FirstPrototypeCell.C4

Classification: main

Source: ScalarSuffixCertificate.lean:510-510

Declaration SHA-256: 0c76dd8701b0777d1f4e908247b8a477e6ed59455006924fb2e56c23786a4bd0

Mathematical statement. The first prototype's type-(iv) incidence equation residual.

$C_4 = \cos x - \lambda \cos y$ vanishes when the principal angles satisfy incidence.

```
def C4 (p : Point) : ℝ := cos (x p) - lam p * cos (y p)
```

ScalarSuffixCertificate.FirstPrototypeCell.E

Classification: main

Source: ScalarSuffixCertificate.lean:513-514

Declaration SHA-256: 473eec5a0b00f868ac4f7ad090edf435fe47f07ca5bd2b59758512636541fbfa

Mathematical statement. The first prototype's denominator-cleared equal-area residual.

$$E = 2\cos^2 x N_3 - (1 + \cos w)^2 N_4.$$

```
def E (p : Point) : ℝ :=  
  2 * cos (x p) ^ 2 * N3 p - (1 + cos (w p)) ^ 2 * N4 p
```

ScalarSuffixCertificate.FirstPrototypeCell.F

Classification: main

Source: ScalarSuffixCertificate.lean:511-511

Declaration SHA-256: ff764cf036982bdc0d990c94070c7ae41de884ec397b4b250e1096fedd89ff34

Mathematical statement. The first prototype's stationary-fold residual.

$$F = \cos x D_4 - \sin x \sin y B_4.$$

```
def F (p : Point) : ℝ := cos (x p) * D4 p - sin (x p) * sin (y p) * B4 p
```

ScalarSuffixCertificate.FirstPrototypeCell.G

Classification: main

Source: ScalarSuffixCertificate.lean:515-521

Declaration SHA-256: 818854a9d2ac34b27a374a124d11eeffc488199875161c552aa19dcc6709e1ea

Mathematical statement. The first prototype's scaled perimeter gap.

$G = \cos x (B_3 + D_3) - (1 + \cos w) B_4$ has the strict-improvement sign on the checked branch.

```
def G (p : Point) : ℝ :=  
  cos (x p) * (B3 p + D3 p) - (1 + cos (w p)) * B4 p  
  
/-- The shared analytic interpretation of the checker coordinates at a point  
on the recorded angle branch. The equation and sign fields deliberately take  
their checker hypotheses as arguments, so the expensive trigonometric  
translation is proved only once. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.K3

Classification: main

Source: ScalarSuffixCertificate.lean:720-727

Declaration SHA-256: dded0a92a5475f561e4ec09b53d31fd672246c1e3a18679f1d82548595f59954

Mathematical statement. The first prototype's angular type-(iii) fold expression.

It is the exact numerator determining the type-(iii) area derivative at the checker curvature.

```
def K3 (p : Point) : ℝ :=  
  let h3 := (1 + cos (w p)) / 2  
  4 * h3 * ((1 + cos (w p)) / sin (w p) -  
    (lam p ^ 2 + cos (w p)) / (lam p * (lam p * sin (v p)))) -  
  2 * (B3 p + D3 p)
```

```

/-- On the certified angle branch, the checker quantity `K3` is exactly the
numerator of the type-(iii) area derivative. -/

```

ScalarSuffixCertificate.FirstPrototypeCell.PointEquationSemantics

Classification: main

Source: ScalarSuffixCertificate.lean:522-537

Declaration SHA-256: 57c3cf6c23b8d0095e640676ca4732afaae3f1e4c4e649a7f2e21c72d97bbd64

Mathematical statement. Analytic interpretation of the first prototype's equations and gap sign.

It records the regular curvature ranges and the implications from equal-area, stationary, and strict-gap residuals.

```

structure PointEquationSemantics (p : Point) : Prop where
  typeThreeHeight_gt_half : 1 / 2 < (1 + cos (w p)) / 2
  typeThreeHeight_lt_one : (1 + cos (w p)) / 2 < 1
  typeFourHeight_pos : 0 < cos (x p)
  typeFourHeight_lt_one : cos (x p) < 1
  equalArea_of_E_zero : E p = 0 →
    LeanSuffixAnalytic.typeThreeArea (lam p) ((1 + cos (w p)) / 2) =
      LeanSuffixAnalytic.typeFourArea (lam p) (cos (x p))
  stationary_of_F_zero : F p = 0 →
    LeanSuffixAnalytic.typeFourFold (lam p) (cos (x p)) = 0
  perimeter_lt_of_G_neg : G p < 0 →
    LeanSuffixAnalytic.typeThreePerimeter (lam p) ((1 + cos (w p)) / 2) <
      LeanSuffixAnalytic.typeFourPerimeter (lam p) (cos (x p))

/-- The checker coordinates have their intended analytic semantics on the
recorded angle branch. -/

```

ScalarSuffixCertificate.FirstPrototypeCell.candidate_not_isWeightedPerimeterMinimizer

Classification: main

Source: ScalarSuffixCertificate.lean:4190-4203

Declaration SHA-256: addf63fb8e32b74a68ab723aad485618435e5da3c6625d624ab901eb3e183553

Mathematical statement. Every modeled candidate is nonminimal on [51/50,102001/100000].

The first stationary certificate is promoted to every curvature, including $h = 1$, and then realized geometrically.

```

theorem candidate_not_isWeightedPerimeterMinimizer
  {lam0 : ℝ} (hlam : lamInterval.RealContains lam0)
  (candidate : _root_.FourArcCandidate lam0)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  rcases stationaryEqualAreaPair_exists hlam with
  ⟨pair, hfoldThree, hperimeter⟩
  exact
    CMVSuffixModel.FourArcCandidate.not_isWeightedPerimeterMinimizer_of_stationaryEqualAreaPair
      candidate hcandidate pair hfoldThree hperimeter

/-- Candidate-facing end of the pilot-cell vertical slice for every regular
stationary type-(iv) root. No stationarity is inferred from minimality: the
fold equation remains an explicit hypothesis, and `h₄ = 1` remains separate. -/

```

ScalarSuffixCertificate.FirstPrototypeCell.pointEquationSemantics

Classification: main

Source: ScalarSuffixCertificate.lean:538-687

Declaration SHA-256: 55d2eea611fa72c52ef509b0c8b4a1aa98765b6f9627ee1108103f1668f29bc9

Mathematical statement. The first prototype's checker equations have the claimed scalar meaning.

The angle branch and incidence equations identify the analytic type-(iii) and type-(iv) formulas exactly.

```
theorem pointEquationSemantics
  (p : Point) (hlam : 1 < lam p)
  (hbranches :
    0 < x p ^ x p < π / 4 ^
    0 < y p ^ y p < π / 4 ^
    0 < w p ^ w p < π / 4 ^
    0 < v p ^ v p < π / 4)
  (hC4 : C4 p = 0) (hC3 : C3 p = 0) :
  PointEquationSemantics p := by
  rcases hbranches with ⟨hx0, hxq, hy0, hyq, hw0, hwq, hv0, hvq⟩
  have hpi4_lt_pi : π / 4 < π := by linarith [Real.pi_pos]
  have hpi4_lt_pi_div_two : π / 4 < π / 2 := by linarith [Real.pi_pos]
  have hxpi : x p < π := hxq.trans hpi4_lt_pi
  have hypi : y p < π := hyq.trans hpi4_lt_pi
  have hwpi : w p < π := hwq.trans hpi4_lt_pi
  have hvpi : v p < π := hvq.trans hpi4_lt_pi
  have hlam_pos : 0 < lam p := lt_trans (by norm_num) hlam
  have hxcos_pos : 0 < cos (x p) :=
    Real.cos_pos_of_mem_Ioo ⟨by linarith [Real.pi_pos],
    hxq.trans hpi4_lt_pi_div_two⟩
  have hwcpos_pos : 0 < cos (w p) :=
    Real.cos_pos_of_mem_Ioo ⟨by linarith [Real.pi_pos],
    hwq.trans hpi4_lt_pi_div_two⟩
  have hzero_mem : (0 : ℝ) ∈ Icc 0 π := ⟨le_refl, Real.pi_pos.le⟩
  have hx_mem : x p ∈ Icc 0 π := ⟨hx0.le, hxpi.le⟩
  have hw_mem : w p ∈ Icc 0 π := ⟨hw0.le, hwpi.le⟩
  have hxcos_lt_one : cos (x p) < 1 := by
    have h := Real.strictAntiOn_cos hzero_mem hx_mem hx0
    simpa using h
  have hwcpos_lt_one : cos (w p) < 1 := by
    have h := Real.strictAntiOn_cos hzero_mem hw_mem hw0
    simpa using h
  have hxsin_pos : 0 < sin (x p) :=
    Real.sin_pos_of_pos_of_lt_pi hx0 hxpi
  have hysin_pos : 0 < sin (y p) :=
    Real.sin_pos_of_pos_of_lt_pi hy0 hypi
  have hwsin_pos : 0 < sin (w p) :=
    Real.sin_pos_of_pos_of_lt_pi hw0 hwpi
  have hvsin_pos : 0 < sin (v p) :=
    Real.sin_pos_of_pos_of_lt_pi hv0 hvpi
  have hcos_xy : cos (x p) = lam p * cos (y p) := by
    unfold C4 at hC4
    linarith
  have hcos_wv : cos (w p) = lam p * cos (v p) := by
    unfold C3 at hC3
    linarith
  have hshape :
    LeanSuffixAnalytic.typeThreeShape ((1 + cos (w p)) / 2) = cos (w p) := by
    unfold LeanSuffixAnalytic.typeThreeShape
    ring
  have hcos_xy_div : cos (x p) / lam p = cos (y p) := by
    rw [hcos_xy]
    field_simp [ne_of_gt hlam_pos]
  have hcos_wv_div : cos (w p) / lam p = cos (v p) := by
    rw [hcos_wv]
    field_simp [ne_of_gt hlam_pos]
  have hB4 :
    LeanSuffixAnalytic.typeFourAngle (lam p) (cos (x p)) = B4 p := by
    rw [LeanSuffixAnalytic.typeFourAngle_eq_checker, hcos_xy_div,
    Real.arccos_cos hy0.le hypi.le, Real.arccos_cos hx0.le hxpi.le]
```

```

rfl
have hB3 :
  LeanSuffixAnalytic.typeThreeAngle (lam p)
    ((1 + cos (w p)) / 2) +  $\pi$  / 2 = B3 p := by
  rw [LeanSuffixAnalytic.typeThreeAngle_add_halfPi, hshape, hcos_wv_div,
    Real.arccos_cos hv0.le hvpi.le, Real.arccos_cos hw0.le hwpi.le]
rfl
have hsqrt_x : sqrt (1 - cos (x p) ^ 2) = sin (x p) := by
  have htrig := Real.sin_sq_add_cos_sq (x p)
  rw [show 1 - cos (x p) ^ 2 = sin (x p) ^ 2 by linarith,
    Real.sqrt_sq_eq_abs, abs_of_pos hxsin_pos]
have hsqrt_w : sqrt (1 - cos (w p) ^ 2) = sin (w p) := by
  have htrig := Real.sin_sq_add_cos_sq (w p)
  rw [show 1 - cos (w p) ^ 2 = sin (w p) ^ 2 by linarith,
    Real.sqrt_sq_eq_abs, abs_of_pos hwsin_pos]
have hsqrt_lam_x :
  sqrt (lam p ^ 2 - cos (x p) ^ 2) = lam p * sin (y p) := by
  have htrig := Real.sin_sq_add_cos_sq (y p)
  rw [show lam p ^ 2 - cos (x p) ^ 2 = (lam p * sin (y p)) ^ 2 by
    rw [hcos_xy]
    linear_combination -(lam p) ^ 2 * htrig,
    Real.sqrt_sq_eq_abs, abs_of_pos (mul_pos hlam_pos hysin_pos)]
have hsqrt_lam_w :
  sqrt (lam p ^ 2 - cos (w p) ^ 2) = lam p * sin (v p) := by
  have htrig := Real.sin_sq_add_cos_sq (v p)
  rw [show lam p ^ 2 - cos (w p) ^ 2 = (lam p * sin (v p)) ^ 2 by
    rw [hcos_wv]
    linear_combination -(lam p) ^ 2 * htrig,
    Real.sqrt_sq_eq_abs, abs_of_pos (mul_pos hlam_pos hvsin_pos)]
have hD4 :
  LeanSuffixAnalytic.typeFourDelta (lam p) (cos (x p)) = D4 p := by
  rw [LeanSuffixAnalytic.typeFourDelta, hsqrt_lam_x, hsqrt_x]
  field_simp [ne_of_gt hlam_pos]
rfl
have hD3 :
  LeanSuffixAnalytic.typeThreeDelta (lam p)
    ((1 + cos (w p)) / 2) = D3 p := by
  rw [LeanSuffixAnalytic.typeThreeDelta, hshape, hsqrt_lam_w, hsqrt_w]
  field_simp [ne_of_gt hlam_pos]
rfl
have hh3_pos : 0 < (1 + cos (w p)) / 2 := by linarith
have hh4_pos : 0 < cos (x p) := hxcos_pos
have equalArea_of_E_zero (hE : E p = 0) :
  LeanSuffixAnalytic.typeThreeArea (lam p) ((1 + cos (w p)) / 2) =
    LeanSuffixAnalytic.typeFourArea (lam p) (cos (x p)) := by
  rw [LeanSuffixAnalytic.typeThreeArea, LeanSuffixAnalytic.typeFourArea,
    hshape, hB3, hD3, hB4, hD4]
  apply (div_eq_div_iff (pow_ne_zero 2 (ne_of_gt hh3_pos))
    (pow_ne_zero 2 (ne_of_gt hh4_pos))).2
  unfold E N3 N4 at hE
  linear_combination (1 / 2 :  $\mathbb{R}$ ) * hE
have stationary_of_F_zero (hF : F p = 0) :
  LeanSuffixAnalytic.typeFourFold (lam p) (cos (x p)) = 0 := by
  have hden_ne : sin (x p) * sin (y p)  $\neq$  0 :=
    mul_ne_zero (ne_of_gt hxsin_pos) (ne_of_gt hysin_pos)
  have hnumerator :
    cos (x p) * (sin (y p) - sin (x p)) =
      sin (x p) * sin (y p) * B4 p := by
    unfold F D4 at hF
    linarith
  have hcore :
    cos (x p) * (1 / sin (x p) - 1 / sin (y p)) = B4 p := by
    calc
      cos (x p) * (1 / sin (x p) - 1 / sin (y p)) =
        (cos (x p) * (sin (y p) - sin (x p))) /
          (sin (x p) * sin (y p)) := by
          field_simp [ne_of_gt hxsin_pos, ne_of_gt hysin_pos]
        _ = B4 p := (div_eq_iff hden_ne).2 (by
          simp [mul_assoc, mul_comm, mul_left_comm] using hnumerator)
  have hcancel : lam p / (lam p * sin (y p)) = 1 / sin (y p) := by
    field_simp [ne_of_gt hlam_pos, ne_of_gt hysin_pos]
  rw [LeanSuffixAnalytic.typeFourFold, hsqrt_x, hsqrt_lam_x, hB4,

```



```

    hcancel, hcore]
  ring
  have perimeter_lt_of_G_neg (hG : G p < 0) :
    LeanSuffixAnalytic.typeThreePerimeter (lam p)
      ((1 + cos (w p)) / 2) <
      LeanSuffixAnalytic.typeFourPerimeter (lam p) (cos (x p)) := by
  rw [LeanSuffixAnalytic.typeThreePerimeter,
    LeanSuffixAnalytic.typeFourPerimeter, hB3, hD3, hB4]
  apply (div_lt_div_iff₀ hh3_pos hh4_pos).2
  unfold G at hG
  nlinarith
  exact ⟨by linarith, by linarith, hxcos_pos, hxcos_lt_one,
    equalArea_of_E_zero, stationary_of_F_zero, perimeter_lt_of_G_neg⟩

/-- The primitive checker equations have their intended analytic meaning on
the recorded angle branch. In particular, `E = 0` is equal weighted area,
`F = 0` is the type-(iv) stationary fold, and `G < 0` is strict type-(iii)
perimeter improvement. -/

```

ScalarSuffixCertificate.FirstPrototypeCell.point_equations_to_scalar_improvement

Classification: main

Source: ScalarSuffixCertificate.lean:688-719

Declaration SHA-256: db5cd193da271e760511f0742066e9fa2712d960f6dbb5c7963df325913da93

Mathematical statement. A first-prototype equation solution with negative G yields a stationary scalar strict improvement.

Set $s = (1 + \cos w)/2$ and $t = \cos x$; the residual identities give equal area, stationarity, and $P_3(s) < P_4(t)$.

```

theorem point_equations_to_scalar_improvement
  (p : Point) (hlam : 1 < lam p)
  (hbranches :
    0 < x p ^ x p < π / 4 ^
    0 < y p ^ y p < π / 4 ^
    0 < w p ^ w p < π / 4 ^
    0 < v p ^ v p < π / 4)
  (hC4 : C4 p = 0) (hF : F p = 0)
  (hC3 : C3 p = 0) (hE : E p = 0) (hG : G p < 0) :
  let h3 := (1 + cos (w p)) / 2
  let h4 := cos (x p)
  1 / 2 < h3 ^ h3 < 1 ^
  0 < h4 ^ h4 < 1 ^
  LeanSuffixAnalytic.typeThreeArea (lam p) h3 =
    LeanSuffixAnalytic.typeFourArea (lam p) h4 ^
  LeanSuffixAnalytic.typeFourFold (lam p) h4 = 0 ^
  LeanSuffixAnalytic.typeThreePerimeter (lam p) h3 <
    LeanSuffixAnalytic.typeFourPerimeter (lam p) h4 ^
  LeanSuffixAnalytic.reducedFoldGap (lam p) h3 h4 < 0 := by
  dsimp only
  have hs := pointEquationSemantics p hlam hbranches hC4 hC3
  have harea := hs.equalArea_of_E_zero hE
  have hfold := hs.stationary_of_F_zero hF
  have hperimeter := hs.perimeter_lt_of_G_neg hG
  have hgap :=
    (LeanSuffixAnalytic.stationaryEqualArea_typeThree_improves_iff
      (ne_of_gt (lt_trans (by norm_num) hs.typeThreeHeight_gt_half))
      (ne_of_gt hs.typeFourHeight_pos) harea hfold).mp hperimeter
  exact ⟨hs.typeThreeHeight_gt_half, hs.typeThreeHeight_lt_one,
    hs.typeFourHeight_pos, hs.typeFourHeight_lt_one, harea, hfold,
    hperimeter, hgap⟩

```

ScalarSuffixCertificate.FirstPrototypeCell.realConclusion

Classification: main

Source: ScalarSuffixCertificate.lean:3950-3975

Declaration SHA-256: dc3fbce7b832643e59d10760d43a70cfab194d45c7293f0a25257849be954285

Mathematical statement. The first prototype provides a real equation solution with its recorded strict branch and sign bounds.

The root remains inside the certified box, so uniform $K_3 < 0$ and $G < 0$ are available at the solution.

```
theorem realConclusion
  {lam0 : ℝ} (hlam : lamInterval.RealContains lam0) :
  ∃ p : Point, lam p = lam0 ∧ InSymmetricBox radiiReal (vars p) ∧
  C4 p = 0 ∧ F p = 0 ∧ C3 p = 0 ∧ E p = 0 ∧ G p < 0 := by
  let t : ℝ := lam0 - (lamMid : ℝ)
  have ht : tInterval.RealContains t := lambda_iff_centered.mp hlam
  rcases triangular_root t ht with ⟨z, hz, hzero⟩
  let p := pointOf t z
  have h0 := hzero 0
  have h1 := hzero 1
  have h2 := hzero 2
  have h3 := hzero 3
  refine ⟨p, ?_, ?_, ?_, ?_, ?_, ?_, lt_trans (gap_upper t z ht hz) (by norm_num)⟩
  · simp [p, pointOf, lam, t]
  · intro j
    have hj := hz j
    fin_cases j <|> simp [p, vars, pointOf] using hj
  · simp [rootMap, p] using h1
  · have hn : -F p = 0 := by simp [rootMap, p] using h0
    exact neg_eq_zero.mp hn
  · simp [rootMap, p] using h3
  · simp [rootMap, p] using h2

/-- The prototype-cell root is a genuine stationary equal-area scalar
comparison with a certified simple descending type-(iii) root, rather than an
uninterpreted zero of the checker coordinates. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.realConclusion_scalarImprovement

Classification: main

Source: ScalarSuffixCertificate.lean:3976-4031

Declaration SHA-256: bb2d746cbd9e64bb1870debe196a905e1574b27a24542c427057bd40fc2db5ef

Mathematical statement. The first prototype's real solution yields a genuine scalar equal-area perimeter improvement.

The equation semantics convert the angular witness to regular curvatures and exact analytic objectives.

```
theorem realConclusion_scalarImprovement
  {lam0 : ℝ} (hlam : lamInterval.RealContains lam0) :
  ∃ p : Point, lam p = lam0 ∧ InSymmetricBox radiiReal (vars p) ∧
  let h3 := (1 + cos (w p)) / 2
  let h4 := cos (x p)
  1 / 2 < h3 ∧ h3 < 1 ∧
  0 < h4 ∧ h4 < 1 ∧
  LeanSuffixAnalytic.typeThreeArea (lam p) h3 =
  LeanSuffixAnalytic.typeFourArea (lam p) h4 ∧
  LeanSuffixAnalytic.typeFourFold (lam p) h4 = 0 ∧
  LeanSuffixAnalytic.typeThreePerimeter (lam p) h3 <
  LeanSuffixAnalytic.typeFourPerimeter (lam p) h4 ∧
  LeanSuffixAnalytic.reducedFoldGap (lam p) h3 h4 < 0 ∧
  deriv (LeanSuffixAnalytic.typeThreeArea (lam p)) h3 < 0 := by
  rcases realConclusion hlam with
  ⟨p, hpLam, hpbox, hC4, hF, hC3, hE, hG⟩
  refine ⟨p, hpLam, hpbox, ?_⟩
  have ht0 : tInterval.RealContains (lam0 - (lamMid : ℝ)) :=
```

```

lambda_iff_centered.mp hlam
have hpt : p.t = lam0 - (lamMid : ℝ) := by
  unfold lam at hpLam
  linarith
have ht : tInterval.RealContains p.t := by
  rw [hpt]
  exact ht0
have hbranches0 := affine_branches p.t (vars p) ht hpbox
have hbranches :
  0 < x p ^ x p < π / 4 ^
  0 < y p ^ y p < π / 4 ^
  0 < w p ^ w p < π / 4 ^
  0 < v p ^ v p < π / 4 := by
  simpa [pointOf, vars] using hbranches0
have hlamPoint : 1 < lam p := by
  rw [hpLam]
  exact (lambda_in_retainedSuffix hlam).one_lt
rcases hbranches with
  ⟨hx0, hxq, hy0, hyq, hw0, hwq, hv0, hvq⟩
have hscalar := point_equations_to_scalar_improvement p hlamPoint
  ⟨hx0, hxq, hy0, hyq, hw0, hwq, hv0, hvq⟩
  hC4 hF hC3 hE hG
dsimp only at hscalar ⊢
rcases hscalar with
  ⟨hh3_half, hh3_one, hh4_pos, hh4_one, harea, hfold,
    hperimeter, hgap⟩
refine ⟨hh3_half, hh3_one, hh4_pos, hh4_one, harea, hfold,
  hperimeter, hgap, ?_⟩
rw [typeThreeArea_deriv_eq_K3 p hlamPoint
  ⟨hw0, hwq, hv0, hvq⟩ hC3]
exact div_neg_of_neg_of_pos
  (by simpa [pointOf, vars] using descendingK3 p.t (vars p) ht hpbox)
  (pow_pos (lt_trans (by norm_num) hh3_half) 3)

/-- At the certified stationary type-(iv) fold, the equal-area type-(iii)
curvature is the unique root in the prototype cell's compact projection.
Unlike pointwise derivative negativity, this is a branch-level consequence of
the uniform cell certificate. -/

```

ScalarSuffixCertificate.FirstPrototypeCell.regularStationaryFold_existsUnique

Classification: main

Source: ScalarSuffixCertificate.lean:4082-4103

Declaration SHA-256: 0524e42b6946a1eef76cb933934bd9181ca74751bcf4f0c445f7b35510a5a81c

Mathematical statement. There is a unique regular stationary type-(iv) fold at every first-prototype density.

The checker root gives existence and analytic strict fold increase gives uniqueness.

```

theorem regularStationaryFold_existsUnique
  {lam0 : ℝ} (hlam : lamInterval.RealContains lam0) :
  ∃! h4 : ℝ, h4 ∈ Ioo (0 : ℝ) 1 ^
    LeanSuffixAnalytic.typeFourFold lam0 h4 = 0 := by
  rcases realConclusion_scalarImprovement hlam with
    ⟨p, hpLam, _hpbox, hscalar⟩
  dsimp only at hscalar
  rcases hscalar with
    ⟨_hh3_half, _hh3_one, hh4_pos, hh4_one, _harea, hfold,
      _hperimeter, _hgap, _hderiv⟩
  have hfold0 :
    LeanSuffixAnalytic.typeFourFold lam0 (cos (x p)) = 0 := by
    simpa [hpLam] using hfold
  refine ⟨cos (x p), ⟨(hh4_pos, hh4_one), hfold0⟩, ?_⟩
  intro h4 hh4
  exact LeanSuffixAnalytic.typeFourFold_zero_unique
    (lambda_in_retainedSuffix hlam).one_lt
    hh4.1 ⟨hh4_pos, hh4_one⟩ hh4.2 hfold0

/-- The pilot certificate supplies a stationary fold with the exact

```

before/after derivative signs needed by the equal-area envelope. The signs are consequences of the analytic fold identity, not certificate hypotheses. -/

ScalarSuffixCertificate.FirstPrototypeCell.regularStationaryFold_scalarImprovement

Classification: main

Source: ScalarSuffixCertificate.lean:4124-4157

Declaration SHA-256: 7fefcab04ab1793e87d3e44914f4cea3510747aca02a4b68cef1c2b92278828d

Mathematical statement. The first prototype's stationary fold admits a regular equal-area type-(iii) strict improvement.

Uniqueness identifies the stationary curvature produced by the certificate with the regular fold under comparison.

```
theorem regularStationaryFold_scalarImprovement
  {lam0 h4 : ℝ} (hlam : lamInterval.RealContains lam0)
  (hh4 : h4 ∈ Ioo (0 : ℝ) 1)
  (hstationary : LeanSuffixAnalytic.typeFourFold lam0 h4 = 0) :
  ∃ h3 : ℝ, 1 / 2 < h3 ∧ h3 < 1 ∧
    LeanSuffixAnalytic.typeThreeArea lam0 h3 =
      LeanSuffixAnalytic.typeFourArea lam0 h4 ∧
      LeanSuffixAnalytic.typeThreePerimeter lam0 h3 <
        LeanSuffixAnalytic.typeFourPerimeter lam0 h4 ∧
      LeanSuffixAnalytic.reducedFoldGap lam0 h3 h4 < 0 ∧
      deriv (LeanSuffixAnalytic.typeThreeArea lam0) h3 < 0 := by
  rcases realConclusion_scalarImprovement hlam with
    ⟨p, hpLam, _hpbox, hscalar⟩
  dsimp only at hscalar
  rcases hscalar with
    ⟨hh3_half, hh3_one, hh4_pos, hh4_one, harea, hfold,
      hperimeter, hgap, hderiv⟩
  have hfold0 :
    LeanSuffixAnalytic.typeFourFold lam0 (cos (x p)) = 0 := by
    simpa [hpLam] using hfold
  have hh4_eq : cos (x p) = h4 :=
    LeanSuffixAnalytic.typeFourFold_zero_unique
      (lambda_in_retainedSuffix hlam).one_lt
      ⟨hh4_pos, hh4_one⟩ hh4 hfold0 hstationary
  refine ⟨(1 + cos (w p)) / 2, hh3_half, hh3_one, ?_, ?_, ?_, ?_⟩
  · simpa [hpLam, hh4_eq] using harea
  · simpa [hpLam, hh4_eq] using hperimeter
  · simpa [hpLam, hh4_eq] using hgap
  · simpa [hpLam] using hderiv

/-- The pilot certificate supplies one stationary equal-area pair on the
descending type-(iii) branch. This pair promotes the local certificate to the
global equal-area envelope without restricting the eventual candidate
curvature to the pilot projection. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.rootMap

Classification: main

Source: ScalarSuffixCertificate.lean:803-806

Declaration SHA-256: 9341d8d5e2fed305c16e751079c6bca9d9e015e2672d8c238a0e48e3da9b8c83

Mathematical statement. The first prototype's map (-F,C4,E,C3).

A zero simultaneously satisfies both incidence equations, stationarity, and equal area.

```
def rootMap (t : ℝ) (z : Vec4) : Vec4 :=
  let p := pointOf t z
  ![-F p, C4 p, E p, C3 p]
```

ScalarSuffixCertificate.FirstPrototypeCell.rootMap_continuous

Classification: main

Source: ScalarSuffixCertificate.lean:807-815

Declaration SHA-256: 2df03b7ca4b2fad1bc9d6c65609dfe1a30f18068ea4f5844ffe3335f9f65c08b

Mathematical statement. The first prototype's exact root map is continuous.

It is assembled from affine coordinates and trigonometric polynomial expressions.

```
theorem rootMap_continuous (t : ℝ) : Continuous (rootMap t) := by
  apply continuous_pi
  intro j
  fin_cases j <;>
  simp [rootMap, F, C4, E, C3, N3, N4, B3, B4, D3, D4, pointOf,
    lam, x, y, w, v] <;> fun_prop

/-- Scalar form of `C4`, exposing the three correlated variables used by the
face certificate. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.stationaryEqualAreaPair_exists

Classification: main

Source: ScalarSuffixCertificate.lean:4158-4189

Declaration SHA-256: 48939925c6e8499402fbc852be5c65cf7f2c3959981d591edde1ffff923f7115e

Mathematical statement. Each first-prototype density has a stationary equal-area pair with negative K3 and strict improvement.

This exposes exactly the input needed by the all-curvature envelope bridge.

```
theorem stationaryEqualAreaPair_exists
  {lam0 : ℝ} (hlam : lamInterval.RealContains lam0) :
  ∃ pair : LeanSuffixAnalytic.StationaryEqualAreaPair lam0,
    LeanSuffixAnalytic.typeThreeFold lam0 pair.h₃ < 0 ∧
    LeanSuffixAnalytic.typeThreePerimeter lam0 pair.h₃ <
      LeanSuffixAnalytic.typeFourPerimeter lam0 pair.h₄ := by
  rcases regularStationaryFold_existsUnique hlam with
    (h₄, (hh₄, hstationary), _hunique)
  rcases regularStationaryFold_scalarImprovement hlam hh₄ hstationary with
    (h₃, hh₃_half, hh₃_one, harea, hperimeter, _hgap, hderiv)
  have hh₃_pos : 0 < h₃ := lt_trans (by norm_num) hh₃_half
  have hfoldThree : LeanSuffixAnalytic.typeThreeFold lam0 h₃ < 0 := by
    rw [(LeanSuffixAnalytic.hasDerivAt_typeThreeArea
      (lambda_in_retainedSuffix hlam).one_lt hh₃_pos hh₃_one).deriv] at hderiv
  rcases div_neg_iff.mp hderiv with hbad | hgood
  · linarith [pow_pos hh₃_pos 3]
  · exact hgood.1
  let pair : LeanSuffixAnalytic.StationaryEqualAreaPair lam0 :=
    { h₃ := h₃
      h₄ := h₄
      h₃_gt_half := hh₃_half
      h₃_lt_one := hh₃_one
      h₄_pos := hh₄.1
      h₄_lt_one := hh₄.2
      equalArea := harea
      stationary := hstationary }
  refine (pair, ?_, ?_)
  · simpa [pair] using hfoldThree
  · simpa [pair] using hperimeter

/-- A single stationary pair from the pilot certificate excludes every modeled
type-(iv) candidate at that density, including the `h₄ = 1` endpoint. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.triangular_root

Classification: main

Source: ScalarSuffixCertificate.lean:3856-3942

Declaration SHA-256: 84268649affbe380ad5f3767cb23b92f1867804e119cb1a35ce8cf1fd4dbb9a5

Mathematical statement. Every first-prototype density slice has a simultaneous checker-equation root in its box.

The explicit triangular construction uses opposite-face zeros with branch control to solve both incidences, stationarity, and equal area.

```
theorem triangular_root (t : ℝ) (ht : tInterval.RealContains t) :
  ∃ z, InSymmetricBox radiiReal z ^ ∀ j, rootMap t z j = 0 := by
  let zF : ℝ → Vec4 := fun xi => ![xi, etaRoot t xi, 0, 0]
  have hzF_cont : Continuous zF := by
    apply continuous_pi
    intro j
    fin_cases j
    · change Continuous (fun xi : ℝ => xi)
      fun_prop
    · change Continuous (fun xi : ℝ => etaRoot t xi)
      exact etaRoot_continuous t
    · change Continuous (fun _ : ℝ => (0 : ℝ))
      fun_prop
    · change Continuous (fun _ : ℝ => (0 : ℝ))
      fun_prop
  have hxiLow : -radiiReal 0 ∈ Icc (-radiiReal 0) (radiiReal 0) :=
    ⟨le_rfl, neg_le_self (radiiReal_pos 0).le⟩
  have hxiHigh : radiiReal 0 ∈ Icc (-radiiReal 0) (radiiReal 0) :=
    ⟨neg_le_self (radiiReal_pos 0).le, le_rfl⟩
  have hetaLow := (etaRoot_spec ht hxiLow).1
  have hetaHigh := (etaRoot_spec ht hxiHigh).1
  have hzFlow : InSymmetricBox radiiReal (zF (-radiiReal 0)) :=
    vector_mem_box hxiLow hetaLow (zero_mem_coordinate 2) (zero_mem_coordinate 3)
  have hzFHigh : InSymmetricBox radiiReal (zF (radiiReal 0)) :=
    vector_mem_box hxiHigh hetaHigh (zero_mem_coordinate 2) (zero_mem_coordinate 3)
  have hloF : rootMap t (zF (-radiiReal 0)) 0 ≤ 0 :=
    analyticFaceBounds.lowFaces t _ ht hzFlow 0 (by simp [zF])
  have hhiF : 0 ≤ rootMap t (zF (radiiReal 0)) 0 :=
    analyticFaceBounds.highFaces t _ ht hzFHigh 0 (by simp [zF])
  have hcontF : Continuous (fun xi => rootMap t (zF xi) 0) :=
    (continuous_apply (0 : Fin 4)).comp ((rootMap_continuous t).comp hzF_cont)
  rcases oppositeFace_zero
    (show -radiiReal 0 ≤ radiiReal 0 from neg_le_self (radiiReal_pos 0).le)
    hcontF.continuousOn hloF hhiF with ⟨xi, hxi, hF⟩
  let eta := etaRoot t xi
  have hetaSpec := etaRoot_spec ht hxi
  have heta : eta ∈ Icc (-radiiReal 1) (radiiReal 1) := hetaSpec.1
  have hC4 : C4 (pointOf t ![xi, eta, 0, 0]) = 0 := by
    simpa [eta] using hetaSpec.2
  let zE : ℝ → Vec4 := fun rho => ![xi, eta, rho, sigmaRoot t rho]
  have hzE_cont : Continuous zE := by
    apply continuous_pi
    intro j
    fin_cases j
    · change Continuous (fun _ : ℝ => xi)
      fun_prop
    · change Continuous (fun _ : ℝ => eta)
      fun_prop
    · change Continuous (fun rho : ℝ => rho)
      fun_prop
    · change Continuous (fun rho : ℝ => sigmaRoot t rho)
      exact sigmaRoot_continuous t
  have hrhoLow : -radiiReal 2 ∈ Icc (-radiiReal 2) (radiiReal 2) :=
    ⟨le_rfl, neg_le_self (radiiReal_pos 2).le⟩
  have hrhoHigh : radiiReal 2 ∈ Icc (-radiiReal 2) (radiiReal 2) :=
    ⟨neg_le_self (radiiReal_pos 2).le, le_rfl⟩
  have hsigmaLow := (sigmaRoot_spec ht hrhoLow).1
  have hsigmaHigh := (sigmaRoot_spec ht hrhoHigh).1
```

```

have hzELow : InSymmetricBox radiiReal (zE (-radiiReal 2)) :=
  vector_mem_box hxi heta hrhoLow hsigmaLow
have hzEHigh : InSymmetricBox radiiReal (zE (radiiReal 2)) :=
  vector_mem_box hxi heta hrhoHigh hsigmaHigh
have hloE : rootMap t (zE (-radiiReal 2)) 2 ≤ 0 :=
  analyticFaceBounds.lowFaces t _ ht hzELow 2
  (by simp [zE, Matrix.cons_val_two])
have hhiE : 0 ≤ rootMap t (zE (radiiReal 2)) 2 :=
  analyticFaceBounds.highFaces t _ ht hzEHigh 2
  (by simp [zE, Matrix.cons_val_two])
have hcontE : Continuous (fun rho => rootMap t (zE rho) 2) :=
  (continuous_apply (2 : Fin 4)).comp ((rootMap_continuous t).comp hzE_cont)
rcases oppositeFace_zero
  (show -radiiReal 2 ≤ radiiReal 2 from neg_le_self (radiiReal_pos 2).le)
  hcontE.continuousOn hloE hhiE with ⟨rho, hrho, hE⟩
let sigma := sigmaRoot t rho
have hsigmaSpec := sigmaRoot_spec ht hrho
have hsigma : sigma ∈ Icc (-radiiReal 3) (radiiReal 3) := hsigmaSpec.1
have hC3 : C3 (pointOf t ![0, 0, rho, sigma]) = 0 := by
  simpa [sigma] using hsigmaSpec.2
let z : Vec4 := ![xi, eta, rho, sigma]
refine ⟨z, vector_mem_box hxi heta hrho hsigma, ?_⟩
intro j
fin_cases j
· simpa [zF, z, eta, rootMap, F, B4, D4, x, y, lam, pointOf] using hF
· simpa [z, rootMap, C4, x, y, lam, pointOf] using hC4
· simpa [zE, z, sigma, rootMap, Matrix.cons_val_two] using hE
· simpa [z, rootMap, C3, w, v, lam, pointOf, Matrix.cons_val_three] using hC3

```

ScalarSuffixCertificate.FirstPrototypeCell.typeThreeArea_deriv_eq_K3

Classification: main

Source: ScalarSuffixCertificate.lean:728-798

Declaration SHA-256: 8a9cde70bef0dfad1eb697f78cb39f5a363eb9e41679d7d14dd8c12231f3ec28

Mathematical statement. The first prototype's K3 divided by the curvature cube equals the analytic area derivative.

Principal sine/radical identities and incidence justify transferring its strict negative sign to the descending branch.

```

theorem typeThreeArea_deriv_eq_K3
  (p : Point) (hlam : 1 < lam p)
  (hbranches :
    0 < w p ^ w p < π / 4 ^ 0 < v p ^ v p < π / 4)
  (hC3 : C3 p = 0) :
  deriv (LeanSuffixAnalytic.typeThreeArea (lam p))
    ((1 + cos (w p)) / 2) =
    K3 p / ((1 + cos (w p)) / 2) ^ 3 := by
rcases hbranches with ⟨hw0, hwq, hv0, hvq⟩
have hpi4_lt_pi : π / 4 < π := by linarith [Real.pi_pos]
have hpi4_lt_pi_div_two : π / 4 < π / 2 := by
  linarith [Real.pi_pos]
have hwpi : w p < π := hwq.trans hpi4_lt_pi
have hvpi : v p < π := hvq.trans hpi4_lt_pi
have hlam_pos : 0 < lam p := lt_trans (by norm_num) hlam
have hwcpos_pos : 0 < cos (w p) :=
  Real.cos_pos_of_mem_Ioo ⟨by linarith [Real.pi_pos],
    hwq.trans hpi4_lt_pi_div_two⟩
have hzero_mem : (0 : ℝ) ∈ Icc 0 π := ⟨le_refl, Real.pi_pos.le⟩
have hw_mem : w p ∈ Icc 0 π := ⟨hw0.le, hwpi.le⟩
have hwcpos_lt_one : cos (w p) < 1 := by
  have h := Real.strictAntiOn_cos hzero_mem hw_mem hw0
  simpa using h
have hwsin_pos : 0 < sin (w p) :=
  Real.sin_pos_of_pos_of_lt_pi hw0 hwpi
have hvsin_pos : 0 < sin (v p) :=
  Real.sin_pos_of_pos_of_lt_pi hv0 hvpi
have hcos_wv : cos (w p) = lam p * cos (v p) := by
  unfold C3 at hC3
  linarith

```

```

have hshape :
  LeanSuffixAnalytic.typeThreeShape ((1 + cos (w p)) / 2) =
    cos (w p) := by
  unfold LeanSuffixAnalytic.typeThreeShape
  ring
have hcos_wv_div : cos (w p) / lam p = cos (v p) := by
  rw [hcos_wv]
  field_simp [ne_of_gt hlam_pos]
have hB3 :
  LeanSuffixAnalytic.typeThreeAngle (lam p)
    ((1 + cos (w p)) / 2) + π / 2 = B3 p := by
  rw [LeanSuffixAnalytic.typeThreeAngle_add_halfPi, hshape, hcos_wv_div,
    Real.arccos_cos hv0.le hvpi.le, Real.arccos_cos hw0.le hwpi.le]
  rfl
have hsqrt_w : sqrt (1 - cos (w p) ^ 2) = sin (w p) := by
  have htrig := Real.sin_sq_add_cos_sq (w p)
  rw [show 1 - cos (w p) ^ 2 = sin (w p) ^ 2 by linarith,
    Real.sqrt_sq_eq_abs, abs_of_pos hwsin_pos]
have hsqrt_lam_w :
  sqrt (lam p ^ 2 - cos (w p) ^ 2) = lam p * sin (v p) := by
  have htrig := Real.sin_sq_add_cos_sq (v p)
  rw [show lam p ^ 2 - cos (w p) ^ 2 =
    (lam p * sin (v p)) ^ 2 by
    rw [hcos_wv]
    linear_combination -(lam p) ^ 2 * htrig,
    Real.sqrt_sq_eq_abs, abs_of_pos (mul_pos hlam_pos hvsin_pos)]
have hD3 :
  LeanSuffixAnalytic.typeThreeDelta (lam p)
    ((1 + cos (w p)) / 2) = D3 p := by
  rw [LeanSuffixAnalytic.typeThreeDelta, hshape, hsqrt_lam_w, hsqrt_w]
  field_simp [ne_of_gt hlam_pos]
  rfl
have hh3_pos : 0 < (1 + cos (w p)) / 2 := by linarith
have hh3_one : (1 + cos (w p)) / 2 < 1 := by linarith
rw [LeanSuffixAnalytic.typeThreeArea_deriv_eq_explicit
  hlam hh3_pos hh3_one, hshape, hsqrt_w, hsqrt_lam_w, hB3, hD3]
unfold K3
dsimp only
field_simp [ne_of_gt hlam_pos, ne_of_gt hwsin_pos, ne_of_gt hvsin_pos]
ring

```

ScalarSuffixCertificate.cosTaylor26Interval_sound

Classification: main

Source: ScalarSuffixCertificate.lean:311-325

Declaration SHA-256: 527e561394d27648791691ec45e120c58633f8289f9131a5419b3b4150dbd9b7

Mathematical statement. For every rational x , the degree-26 Taylor cosine interval contains $\cos(x)$, with no size or sign restriction on x .

Let $C_{26}(x) = \sum_{k=0}^{13} (-1)^k x^{2k} / (2k)!$. For every $x \in \mathbb{Q}$, the interval $[C_{26}(x) - |x|^{27}/27!, C_{26}(x) + |x|^{27}/27!]$ contains $\cos(x)$ after embedding endpoints into $\mathbb{R}$. The bounded trigonometric derivative gives this global Lagrange remainder estimate. No principal-branch or small-argument bound is a premise here.

```

theorem cosTaylor26Interval_sound (x : ℚ) :
  (cosTaylor26Interval x).RealContains (cos (x : ℝ)) := by
  have h := abs_le.1 (cos_bound26 (x : ℝ))
  simpa [cosTaylor26Interval, cosTaylor26,
    LeanSuffixReflective.QInterval.RealContains] using
  (show
    ((∑ k ∈ Finset.range 14, (-1 : ℝ) ^ k * (x : ℝ) ^ (2 * k) /
      (2 * k).factorial) - |(x : ℝ)| ^ 27 / Nat.factorial 27 ≤
      cos (x : ℝ)) ∧
    (cos (x : ℝ) ≤
      (∑ k ∈ Finset.range 14, (-1 : ℝ) ^ k * (x : ℝ) ^ (2 * k) /
        (2 * k).factorial) + |(x : ℝ)| ^ 27 / Nat.factorial 27) by
    constructor <=> linarith [h.1, h.2])

```



```
-- Sine endpoint certificates extend to a principal-branch interval. -/
```

ScalarSuffixCertificate.oppositeFace_zero

Classification: main

Source: ScalarSuffixCertificate.lean:411-417

Declaration SHA-256: 77378bc472ea0851732650cda76cddb51de35a2f8573fa992dd84ae00b26c15

Mathematical statement. If $a \leq b$ and f is continuous on $[a, b]$ with $f(a) \leq 0 \leq f(b)$, then f has a zero in the closed interval $[a, b]$; endpoint zeros and $a = b$ are allowed.

Let $a, b \in \mathbb{R}$ with $a \leq b$, and let $f: \mathbb{R} \rightarrow \mathbb{R}$ be continuous on $[a, b]$. If $f(a) \leq 0$ and $0 \leq f(b)$, the intermediate value theorem gives $x \in [a, b]$ with $f(x) = 0$. The zero may be an endpoint. If $a = b$, the sign hypotheses force $f(a) = 0$, and $x = a$ is valid. Neither $a < b$ nor strict endpoint signs are assumed. This closed-interval existence statement is used repeatedly in the triangular face-root construction.

```
theorem oppositeFace_zero {a b : ℝ} (hab : a ≤ b) {f : ℝ → ℝ}
  (hf : ContinuousOn f (Icc a b)) (ha : f a ≤ 0) (hb : 0 ≤ f b) :
  ∃ x ∈ Icc a b, f x = 0 := by
  rcases intermediate_value_Icc hab hf
  (show (0 : ℝ) ∈ Icc (f a) (f b) from ⟨ha, hb⟩) with ⟨x, hx, hzero⟩
  exact ⟨x, hx, hzero⟩
```

ScalarSuffixCertificate.realContains_arcsin_of_sin_bounds

Classification: main

Source: ScalarSuffixCertificate.lean:393-402

Declaration SHA-256: 1ba394185e599a9b82bbb3c3e941ce7390ce6cfcc17342f097ef698f1fe9212b

Mathematical statement. Principal-branch sine bounds certify an arcsine enclosure.

The explicit branch conditions prevent using periodic sine inequalities to infer an invalid inverse-angle bound.

```
theorem realContains_arcsin_of_sin_bounds {i : QInterval} {x : ℝ}
  (hx : x ∈ Icc (-1 : ℝ) 1)
  (hbranch : (i.lo : ℝ) ∈ Icc (-(π / 2)) (π / 2) ∧
    (i.hi : ℝ) ∈ Icc (-(π / 2)) (π / 2))
  (hlo : sin (i.lo : ℝ) ≤ x) (hhi : x ≤ sin (i.hi : ℝ)) :
  i.RealContains (arcsin x) := by
  exact ⟨(Real.le_arcsin_iff_sin_le hbranch.1 hx).2 hlo,
    (Real.arcsin_le_iff_le_sin hx hbranch.2).2 hhi⟩

/-- `arccos x = π/2 - arcsin x` transports primitive enclosures. -/
```

ScalarSuffixCertificate.sinTaylor27Interval_sound

Classification: main

Source: ScalarSuffixCertificate.lean:296-310

Declaration SHA-256: cccc0e43a547af36c7ebd29335f5601a5c6d0913d8691b65f34e008faa2bf0e0

Mathematical statement. For every rational x , the degree-27 Taylor sine interval contains $\sin(x)$, with no size or sign restriction on x .

Let $S_{27}(x) = \sum_{k=0}^{13} (-1)^k x^{2k+1} / (2k+1)!$. For every $x \in \mathbb{Q}$, the interval $[S_{27}(x) - |x|^{28}/28!, S_{27}(x) + |x|^{28}/28!]$ contains $\sin(x)$ after embedding endpoints into $\mathbb{R}$. The Lagrange remainder estimate is valid for all real arguments and hence all rationals; the branch restrictions used later for monotonicity and inversion are not hypotheses of this pointwise soundness theorem.

```

theorem sinTaylor27Interval_sound (x : ℚ) :
  (sinTaylor27Interval x).RealContains (sin (x : ℝ)) := by
  have h := abs_le.1 (sin_bound27 (x : ℝ))
  simp [sinTaylor27Interval, sinTaylor27,
    LeanSuffixReflective.QInterval.RealContains] using
  (show
    ((∑ k ∈ Finset.range 14, (-1 : ℝ) ^ k * (x : ℝ) ^ (2 * k + 1) /
      (2 * k + 1).factorial) - |(x : ℝ)| ^ 28 / Nat.factorial 28 ≤
      sin (x : ℝ)) ^
    (sin (x : ℝ) ≤
      (∑ k ∈ Finset.range 14, (-1 : ℝ) ^ k * (x : ℝ) ^ (2 * k + 1) /
        (2 * k + 1).factorial) + |(x : ℝ)| ^ 28 / Nat.factorial 28) by
    constructor <=> linarith [h.1, h.2])

/-- Kernel proof of the exact degree-26 rational cosine enclosure. -/

```

StripCore

Classification: main

Source: CMVGeometry.lean:416-425

Declaration SHA-256: a32ba501747725a0a1307cf7bd2abbffd387c8bc9d810f24cf348c7c04513aad

Mathematical statement. A strip core specified by positive chord and curvature in (0,1].

Its carrier is a central rectangle with left and right circular segments, all within $|y| \leq 1$.

```

structure StripCore where
  chord : ℝ
  curvature : ℝ
  chord_pos : 0 < chord
  curvature_pos : 0 < curvature
  curvature_le_one : curvature ≤ 1

namespace StripCore

/-- Common radius of both strip arcs. -/

```

StripCore.boundaryArcLength

Classification: main

Source: CMVGeometry.lean:457-459

Declaration SHA-256: 74e634700740d7651d4ed773b1cb3cb71b11c7eedacd0c71b6a84a0e6ec43665

Mathematical statement. Combined length of the two curved core sides.

It is $4\arcsin(h)/h$; upper and lower horizontal chords are not included.

```

def boundaryArcLength (c : StripCore) : ℝ :=
  4 * ell c.sideAngle

```

StripCore.carrier

Classification: main

Source: CMVGeometry.lean:449-452

Declaration SHA-256: 05f714bedd5ca3f09fd1dac168964dc190e4830b8099d994e590458d8167e259

Mathematical statement. Union of the rectangle and the two side segments.

This is the actual central carrier retained in cap replacement.

```
def carrier (c : StripCore) : Set PlanePoint :=
  c.rectangleCarrier ∪ c.leftCapCarrier ∪ c.rightCapCarrier

/-- Rectangle plus two side circular segments. -/
```

StripCore.euclideanArea

Classification: main

Source: CMVGeometry.lean:453-456

Declaration SHA-256: d05ac0be663f49bfb5425917ab3d2dcdd3367e8ef25ec467ccc3d1ef4ccf07a7

Mathematical statement. Area expression for the strip core.

The rectangle contributes $2c$ and the side segments contribute $8a(\arcsinh)$.

```
def euclideanArea (c : StripCore) : ℝ :=
  2 * c.chord + 8 * area c.sideAngle

/-- The two side arcs, each of chord length two. -/
```

StripCore.frontier_carrier

Classification: main

Source: CMVGeometry.lean:1088-1199

Declaration SHA-256: 34bdf246812f32ba3d6167bf3b3ff5fea490b0816170740d97a7d77e703585bf

Mathematical statement. The core frontier consists of two side arcs and two horizontal chords.

This exact decomposition determines which chords disappear after exterior caps are glued.

```
theorem frontier_carrier (c : StripCore) :
  frontier c.carrier =
    leftArcTrace c ∪
    (rightArcTrace c ∪ (upperChordTrace c ∪ lowerChordTrace c)) := by
  apply Subset.antisymm
  · intro p hp
    have hpc : p ∈ c.carrier := (isClosed_carrier c).frontier_subset hp
    have hnint : p ∉ interior c.carrier :=
      (mem_frontier_iff_notMem_interior hpc).mp hp
    rcases hpc with (hrect | hleft) | hright
    · change -c.chord / 2 ≤ p.1 ∧ p.1 ≤ c.chord / 2 ∧
      |p.2| ≤ 1 at hrect
      by_cases htop : p.2 = 1
      · exact Or.inr (Or.inr (Or.inl
        (htop, abs_le.mpr
          (by linarith [hrect.1], hrect.2.1))))
      by_cases hbottom : p.2 = -1
      · exact Or.inr (Or.inr (Or.inr
        (hbottom, abs_le.mpr
          (by linarith [hrect.1], hrect.2.1))))
      have hybounds := abs_le.mp hrect.2.2
      have hylt : |p.2| < 1 := abs_lt.mpr
        (lt_of_le_of_ne hybounds.1 (Ne.symm hbottom),
          lt_of_le_of_ne hybounds.2 htop)
      by_cases hx1 : p.1 = -c.chord / 2
      · exfalso
        exact hnint (leftInterface_subset_interior c (hx1, hylt))
      by_cases hxr : p.1 = c.chord / 2
      · exfalso
        exact hnint (rightInterface_subset_interior c (hxr, hylt))
      · exfalso
        apply hnint
        exact strictRectangle_subset_interior c
          (lt_of_le_of_ne hrect.1 (Ne.symm hx1),
            lt_of_le_of_ne hrect.2.1 hxr, hylt)
```

```

· change
  ((p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2) ^
  p.1 ≤ -c.chord / 2 ^ |p.2| ≤ 1 at hleft
by_cases heq :
  (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2
· exact Or.inl (heq, hleft.2)
have hdisk :
  (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 < c.radius ^ 2 :=
  lt_of_le_of_ne hleft.1 heq
have hylt : |p.2| < 1 := by
  apply lt_of_le_of_ne hleft.2.2
  intro hone
  exact heq (leftCap_circle_eq_of_abs_eq_one c hleft hone)
by_cases hx : p.1 = -c.chord / 2
· exfalso
  exact hnint (leftInterface_subset_interior c (hx, hylt))
· exfalso
  exact hnint (strictLeftCap_subset_interior c
    (hdisk, lt_of_le_of_ne hleft.2.1 hx, hylt))
· change
  ((p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2) ^
  c.chord / 2 ≤ p.1 ^ |p.2| ≤ 1 at hright
by_cases heq :
  (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2
· exact Or.inr (Or.inl (heq, hright.2))
have hdisk :
  (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 < c.radius ^ 2 :=
  lt_of_le_of_ne hright.1 heq
have hylt : |p.2| < 1 := by
  apply lt_of_le_of_ne hright.2.2
  intro hone
  exact heq (rightCap_circle_eq_of_abs_eq_one c hright hone)
by_cases hx : p.1 = c.chord / 2
· exfalso
  exact hnint (rightInterface_subset_interior c (hx, hylt))
· exfalso
  exact hnint (strictRightCap_subset_interior c
    (hdisk, lt_of_le_of_ne hright.2.1 (Ne.symm hx), hylt))
· rintro p (hleft | hright | hupper | hlower)
· have hpc : p ∈ c.carrier :=
  Or.inl (Or.inr (hleft.1.le, hleft.2))
  exact (mem_frontier_iff_notMem_interior hpc).2
  (leftArc_not_mem_interior c hleft)
· have hpc : p ∈ c.carrier :=
  Or.inr (hright.1.le, hright.2)
  exact (mem_frontier_iff_notMem_interior hpc).2
  (rightArc_not_mem_interior c hright)
· have hpc : p ∈ c.carrier := by
  apply Or.inl
  apply Or.inl
  have hxbounds := abs_le.mp hupper.2
  exact (by linarith [hxbounds.1], hxbounds.2,
    by simp [hupper.1])
rw [mem_frontier_iff_notMem_interior hpc]
intro hi
apply second_eq_not_mem_interior_le 1 p hupper.1
exact interior_mono (by
  intro q hq
  exact le_trans (le_abs_self q.2) (c.carrier_y_bounds hq)) hi
· have hpc : p ∈ c.carrier := by
  apply Or.inl
  apply Or.inl
  have hxbounds := abs_le.mp hlower.2
  exact (by linarith [hxbounds.1], hxbounds.2,
    by simp [hlower.1])
rw [mem_frontier_iff_notMem_interior hpc]
intro hi
apply second_eq_not_mem_interior_ge (-1) p hlower.1
exact interior_mono (by
  intro q hq
  have hneg : -q.2 ≤ 1 :=
  le_trans (neg_le_abs q.2) (c.carrier_y_bounds hq))

```

```

      show (-1 : ℝ) ≤ q.2
      linarith) hi

end StripCore
/-- Push one-dimensional Lebesgue measure on a closed interval through a curve. -/

```

StripCore.sideAngle

Classification: main

Source: CMVGeometry.lean:429-430

Declaration SHA-256: 4f6b6599037149db7ce42f881c45d0eb7f35fbc43f124df7507d6a83a6d6e15b

Mathematical statement. The core side-arc half-angle.

It is $\arcsin h \in (0, \pi/2]$.

```

def sideAngle (c : StripCore) : ℝ := arcsin c.curvature

```

StripCore.volume_carrier

Classification: main

Source: CMVGeometry.lean:2202-2242

Declaration SHA-256: a426f0908aec7b73455f4a0093170232e841cf46ab8f580ee2fe442ff66a7997

Mathematical statement. The core carrier has its stated Euclidean area.

The rectangle and two side segments add modulo area-null shared edges.

```

lemma volume_carrier (c : StripCore) :
  volume c.carrier = ENNReal.ofReal c.euclideanArea := by
  have hrectLeft : AEDisjoint volume c.rectangleCarrier c.leftCapCarrier := by
    refine measure_mono_null ?_ (volume_verticalline (-c.chord / 2))
    rintro p ⟨hrect, hleft⟩
    exact le_antisymm hleft.2.1 hrect.1
  have hfirstRight : AEDisjoint volume
    (c.rectangleCarrier ∪ c.leftCapCarrier) c.rightCapCarrier := by
    refine measure_mono_null ?_ (volume_verticalline (c.chord / 2))
    rintro p ⟨hfirst, hright⟩
    rcases hfirst with hrect | hleft
    · exact le_antisymm hrect.2.1 hright.2.1
    · exfalso
      nlinarith [hleft.2.1, hright.2.1, c.chord_pos]
  have hleftMeas : MeasurableSet c.leftCapCarrier := by
    rw [leftCapCarrier]
    exact ((isClosed_le
      (((continuous_fst.sub continuous_const).pow 2).add
        (continuous_snd.pow 2)) continuous_const).inter
      ((isClosed_le continuous_fst continuous_const).inter
        (isClosed_le continuous_snd.abs continuous_const)))
      .measurableSet
  have hrightMeas : MeasurableSet c.rightCapCarrier := by
    rw [rightCapCarrier]
    exact ((isClosed_le
      (((continuous_fst.sub continuous_const).pow 2).add
        (continuous_snd.pow 2)) continuous_const).inter
      ((isClosed_le continuous_const continuous_fst).inter
        (isClosed_le continuous_snd.abs continuous_const)))
      .measurableSet
  rw [carrier, measure_union₀, hrightMeas.nullMeasurableSet hfirstRight,
    measure_union₀, hleftMeas.nullMeasurableSet hrectLeft,
    volume_rectangleCarrier, volume_leftCapCarrier, volume_rightCapCarrier]
  have hq : 0 ≤ 2 * c.chord := mul_nonneg (by norm_num) c.chord_pos.le
  have ha : 0 ≤ 4 * area c.sideAngle := by
    exact mul_nonneg (by norm_num) (area_pos c.sideAngle_pos c.sideAngle_lt_pi).le
  rw [← ENNReal.ofReal_add hq ha, ← ENNReal.ofReal_add (add_nonneg hq ha) ha]
  congr 1
  rw [euclideanArea]

```

```

ring
end StripCore

```

StripCore.weightedArea_formula

Classification: main

Source: CMVGeometry.lean:2315-2333

Declaration SHA-256: 22039501f3884648956188fd011ab629e4d948f733db9c19194b8b310134e20a

Mathematical statement. Weighted core area equals its Euclidean area.

The core lies entirely in $|y| \leq 1$.

```

lemma StripCore.weightedArea_formula (lam : ℝ) (c : StripCore) :
  WeightedArea lam c.carrier = c.euclideanArea := by
  rw [WeightedArea]
  calc
    (∫ p in c.carrier, StripDensity lam p) =
      ∫ _p in c.carrier, (1 : ℝ) := by
        apply setIntegral_congr_fun c.measurableSet_carrier
      intro p hp
      rw [StripDensity, if_pos (c.carrier_y_bounds hp)]
    _ = c.euclideanArea := by
      have ha : 0 ≤ c.euclideanArea := by
        rw [StripCore.euclideanArea]
        exact add_nonneg (mul_nonneg (by norm_num) c.chord_pos.le)
          (mul_nonneg (by norm_num)
            (area_pos c.sideAngle_pos c.sideAngle_lt_pi).le)
      rw [integral_const, measureReal_restrict_apply_univ,
        Measure.real, c.volume_carrier, ENNReal.toReal_ofReal ha]
      simp

```

StripDensity

Classification: main

Source: CMVGeometry.lean:43-54

Declaration SHA-256: 51e629a8115629e4652f417b9dcfc46beea6a584d8ebcb351e746df181f97ab1

Mathematical statement. Density one on the closed strip and lambda outside.

$f_\lambda(x, y) = 1$ for $|y| \leq 1$ and $f_\lambda(x, y) = \lambda$ otherwise; both interfaces have density one.

```

def StripDensity (lam : ℝ) (p : PlanePoint) : ℝ :=
  if |p.2| ≤ 1 then 1 else lam

@[simp] theorem stripDensity_upper_interface (lam x : ℝ) :
  StripDensity lam (x, 1) = 1 := by
  simp [StripDensity]

@[simp] theorem stripDensity_lower_interface (lam x : ℝ) :
  StripDensity lam (x, -1) = 1 := by
  simp [StripDensity]

/-- Lebesgue weighted area for a general measurable planar carrier. -/

```

TypeThreeAssembly

Classification: main

Source: TypeThreeAssembly.lean:29-35

Declaration SHA-256: ba80fb031079587a301c08eeb69ef793b83ab9e2bcde30a061c829264b7e1959

Mathematical statement. A regular type-(iii) geometric assembly at density $\lambda > 1$ and curvature $0 < h < 1$.

The constructor accepts the entire curvature interval, including the major exterior-cap and semicircular transition branches.

```
structure TypeThreeAssembly (lam : ℝ) where
  h : ℝ
  density_jump : 1 < lam
  h_pos : 0 < h
  h_lt_one : h < 1

/-- The three geometric branches of the source type-(iii) upper arc. -/
```

TypeThreeAssembly.HasModeledCompetitor

Classification: main

Source: TypeThreeAssembly.lean:3076-3080

Declaration SHA-256: 6f62c793e9d706d4618e398ebe39871915cbb898f90f26305f1f8dad0a496a66

Mathematical statement. Existence of a modeled admissible realization of the type-(iii) assembly.

It asserts that the realization record is inhabited, retaining its exact area and perimeter identities.

```
def HasModeledCompetitor : Prop :=
  Nonempty (ModeledCompetitorRealization a)

/-- Every regular type-(iii) coordinate carrier is a genuine admissible
finite-perimeter region; no external realization hypothesis is needed. -/
```

TypeThreeAssembly.ModeledCompetitorRealization

Classification: main

Source: TypeThreeAssembly.lean:3043-3052

Declaration SHA-256: 7461e543281be1bed3a7575ad4267ecf1122dd3b3edd989aadf4754226aea410

Mathematical statement. An admissible competitor together with equality of its carrier to the specified type-(iii) carrier.

For a type-(iii) assembly a with $\lambda > 1$ and $0 < h < 1$, a realization records an admissible competitor E and $E = a.\text{carrier}$ as an equality of carrier sets. It does not store independent area or perimeter values, nor require a particular competitor constructor. The canonical objectives and the assembly formulas derive both scalar identities from carrier equality.

```
structure ModeledCompetitorRealization where
  competitor : AdmissibleCompetitor lam
  carrier_eq : competitor.carrier = a.carrier

namespace ModeledCompetitorRealization

variable {a : TypeThreeAssembly lam}
  (realization : ModeledCompetitorRealization a)

/-- The genuine admissible competitor selected by a geometric realization. -/
```

TypeThreeAssembly.ModeledCompetitorRealization.asCompetitor

Classification: main

Source: TypeThreeAssembly.lean:3053-3054

Declaration SHA-256: 6c62dc13fbd8bcc95bc9815a8c9dace231a72bb0614d3ccc231b26790b97baab

Mathematical statement. The admissible competitor already stored in a type-(iii) realization.

Given a realization of the assembly a , this definition returns its existing competitor field. It does not construct a new region or prove integrability. The realization's carrier equality and the assembly's geometric formulas imply the objective identities proved in the following declarations. The explicit general-competitor construction with measurability and both integrability proofs belongs to .

Lean references

TypeThreeAssembly.leanTypeThreeAssembly.hasModeledCompetitor

```
def asCompetitor : AdmissibleCompetitor lam := realization.competitor
```

TypeThreeAssembly.ModeledCompetitorRealization.asCompetitor_weightedArea

Classification: main

Source: TypeThreeAssembly.lean:3059-3064

Declaration SHA-256: ac19f7d26c088671b4dd0552d598cd1124637e2815ea1238d0d5daf917f4ca9b

Mathematical statement. The realized admissible competitor has the type-(iii) scalar area.

This is the equality used to satisfy the minimizer's equal-area premise.

```
theorem asCompetitor_weightedArea :  
  realization.asCompetitor.WeightedArea = a.scalarWeightedArea := by  
  rw [asCompetitor, realization.competitor.weightedArea_eq_carrier,  
      realization.carrier_eq]  
  exact a.weightedArea_formula
```

TypeThreeAssembly.ModeledCompetitorRealization.asCompetitor_weightedPerimeter

Classification: main

Source: TypeThreeAssembly.lean:3065-3075

Declaration SHA-256: c73f289c45ecd5870b19789b66c484f95d8c8aad15bb42c8e172d87d107523d8

Mathematical statement. The realized admissible competitor has the type-(iii) scalar perimeter.

The canonical frontier measure, not arbitrary perimeter data, realizes the scalar objective.

```
theorem asCompetitor_weightedPerimeter :  
  realization.asCompetitor.WeightedPerimeter =  
    a.scalarWeightedPerimeter := by  
  change _root_.WeightedPerimeter lam  
  (FrontierMeasure realization.competitor.carrier) =  
    a.scalarWeightedPerimeter  
  rw [realization.carrier_eq]  
  exact a.weightedPerimeter_formula  
end ModeledCompetitorRealization
```


TypeThreeAssembly.WeightedArea

Classification: main

Source: TypeThreeAssembly.lean:265-268

Declaration SHA-256: 5d054747c494e793da416a19e2d895420eb9e121366a5645e79b7ab53a0c3c7f

Mathematical statement. The density integral over the actual type-(iii) carrier.

The scalar area formula is proved equal to this integral.

```
def WeightedArea : ℝ := _root_.WeightedArea lam a.carrier

/-- Density integral against the complete topological frontier of the
coordinate carrier. -/
```

TypeThreeAssembly.WeightedPerimeter

Classification: main

Source: TypeThreeAssembly.lean:269-272

Declaration SHA-256: ac2df4f2daf804e358c901d1a6b496eaadc35076745e389cd399733fc5dead10

Mathematical statement. Canonical complete-frontier weighted perimeter of the type-(iii) carrier.

It charges all exposed side arcs, upper cap, and bottom segment.

```
def WeightedPerimeter : ℝ :=
  _root_.WeightedPerimeter lam (FrontierMeasure a.carrier)

/-- The horizontal expansion between the two translated side half-caps. -/
```

TypeThreeAssembly.angleTerm

Classification: main

Source: TypeThreeAssembly.lean:289-291

Declaration SHA-256: 8089feb5fd033f48e23e1ab4579cce22afbbbfde128400123f3b8b2b2a245332

Mathematical statement. The source angle term λa_3 plus π minus b_3 .

It agrees with $B_3(h)$ in the analytic comparison.

```
def angleTerm : ℝ := lam * a.outerAngle + π - a.innerAngle

/-- The horizontal sine displacement between the upper and side arcs. -/
```

TypeThreeAssembly.bottomChord

Classification: main

Source: TypeThreeAssembly.lean:136-139

Declaration SHA-256: 08cc63d3aedc396b5efa9b27fd1bac931beb8b7d14cbce558e5fde942be277c2

Mathematical statement. The bottom-segment length.

It is $2d = 2R(\sin a_3 - \sin b_3)$.

```
def bottomChord : ℝ := 2 * a.sideHalfWidth

/-- Finite branch selection; it does not assert membership in a later
area-fold branch. -/
```

TypeThreeAssembly.bottomChord_eq_zero_iff

Classification: main

Source: TypeThreeAssembly.lean:355-358

Declaration SHA-256: 1b82dffbe772075bcb5095c9dc075644068e9232dc927ae0cd23919b863062dd

Mathematical statement. The bottom chord has zero length exactly at $h=1/2$.

The complete frontier retains the singleton segment, whose Hausdorff length is zero.

```
theorem bottomChord_eq_zero_iff : a.bottomChord = 0 ↔ a.h = 1 / 2 := by
  rw [bottomChord, mul_eq_zero]
  norm_num
  exact a.sideHalfWidth_eq_zero_iff
```

TypeThreeAssembly.bottomSegmentCarrier

Classification: main

Source: TypeThreeAssembly.lean:132-135

Declaration SHA-256: 18e70891a1386b33220d27cc96af45d0ffdcc88b970fd467c64035c04da589f5

Mathematical statement. The possibly degenerate exposed segment on $y=-1$.

It is $\{(x, -1) : |x| \leq d\}$ and must be retained even at the zero-length transition.

```
def bottomSegmentCarrier : Set PlanePoint :=
  {p | p.2 = -1} ∩ {p | |p.1| ≤ a.sideHalfWidth}

/-- Bottom-segment length, including the zero-length semicircular transition. -/
```

TypeThreeAssembly.bottomSegmentCarrier_subset_frontier

Classification: main

Source: TypeThreeAssembly.lean:1058-1079

Declaration SHA-256: 82d466e4484f4553cdf0fe29df6958b297f06ff314377fa09237775f676763e1

Mathematical statement. The entire bottom segment lies on the type-(iii) carrier frontier.

It belongs to the carrier, while arbitrarily close points below $y = -1$ lie outside; the degenerate singleton case is included.

```
theorem bottomSegmentCarrier_subset_frontier :
  a.bottomSegmentCarrier ⊆ frontier a.carrier := by
  intro p hp
  rw [mem_frontier_iff_notMem_interior
    (a.bottomSegmentCarrier_subset_carrier hp)]
  intro hinterior
  have hnhds : interior a.carrier ∈ nhds p :=
    isOpen_interior.mem_nhds hinterior
  rcases Metric.mem_nhds_iff.mp hnhds with
    (epsilon, hepsilon, hball)
  have hhalf : 0 < epsilon / 2 := div_pos hepsilon (by norm_num)
  have hnear :
    (p.1, p.2 - epsilon / 2) ∈ Metric.ball p epsilon := by
    rw [Metric.mem_ball, Prod.dist_eq, dist_self, Real.dist_eq,
      show p.2 - epsilon / 2 - p.2 = -(epsilon / 2) by ring,
      abs_neg, abs_of_pos hhalf, max_eq_right hhalf.le]
    linarith
  have hmember :
    (p.1, p.2 - epsilon / 2) ∈ a.carrier :=
    interior_subset (hball hnear)
  exact a.below_bottomSegmentCarrier_not_mem hp hhalf hmember
```

TypeThreeAssembly.boundaryTrace

Classification: main

Source: TypeThreeAssembly.lean:1595-1599

Declaration SHA-256: 77ded3af26dffe034b843162e20202cd8e868c69330a0cb95819f9e3b2a0bbf5

Mathematical statement. Union of the two side arcs, upper exterior arc, and bottom segment.

It is the proposed complete type-(iii) frontier, including the zero-length segment branch.

```
def boundaryTrace : Set PlanePoint :=
  a.leftArcTrace ∪
  (a.rightArcTrace ∪
   (OneSidedCircularCap.arcTrace a.outerCap ∪ a.bottomSegmentCarrier))
```

TypeThreeAssembly.carrier

Classification: main

Source: TypeThreeAssembly.lean:262-264

Declaration SHA-256: 0849c97e398a59f917657f18205fab9ffd6aa8e524ee751befd78f1c4735a4ff

Mathematical statement. Union of the type-(iii) strip core and upper exterior cap.

This is the competitor carrier used by the scalar-to-geometric bridge.

```
def carrier : Set PlanePoint := a.coreCarrier ∪ a.outerCap.carrier

/-- Ambient weighted-area integral of the coordinate carrier. -/
```

TypeThreeAssembly.coreCarrier

Classification: main

Source: TypeThreeAssembly.lean:99-103

Declaration SHA-256: 5e82c9d38bf8a70ad9c2ae143c293bbf0238ee48148c3dd2d922c95471160878

Mathematical statement. The type-(iii) strip core is the union of the rectangle and two disk segments.

This union gives an actual coordinate carrier on all regular branches.

```
def coreCarrier : Set PlanePoint :=
  a.rectangleCarrier ∪ a.leftSegmentCarrier ∪ a.rightSegmentCarrier

/-- The exact left circular trace of the type-(iii) strip-side boundary. -/
```

TypeThreeAssembly.core_weightedArea_formula

Classification: main

Source: TypeThreeAssembly.lean:1530-1550

Declaration SHA-256: 5a80fac760bbad4e64452fa4d575b54c924b85ffbad6442cb8801547dabcbb7

Mathematical statement. The type-(iii) core weighted area equals its component Euclidean area sum.

The whole core lies in the density-one strip.

```
theorem core_weightedArea_formula :
  _root_.WeightedArea lam a.coreCarrier =
  a.rectangleArea + a.sideCap.euclideanArea := by
  rw [_root_.WeightedArea]
```

```

calc
  (f p in a.coreCarrier, StripDensity lam p) =
    f _p in a.coreCarrier, (1 : ℝ) := by
      apply setIntegral_congr_fun a.isClosed_coreCarrier.measurableSet
      intro p hp
      rw [StripDensity, if_pos (a.coreCarrier_y_bounds hp)]
    _ = a.rectangleArea + a.sideCap.euclideanArea := by
      have hnonneg :
        0 ≤ a.rectangleArea + a.sideCap.euclideanArea :=
          add_nonneg
            (mul_nonneg (by norm_num) a.sideHalfWidth_nonneg)
            a.sideCap.euclideanArea_pos.le
      rw [integral_const, measureReal_restrict_apply_univ,
        Measure.real, a.volume_coreCarrier,
        ENNReal.toReal_ofReal hnonneg]
      simp

```

TypeThreeAssembly.euclidean_frontier

Classification: main

Source: TypeThreeAssembly.lean:2937-2949

Declaration SHA-256: 5e03c2ded7f93ebce1b77460290c9d90f4e5be41990287351d94b69c624d60ea

Mathematical statement. The Euclidean type-(iii) frontier is the union of its four realized boundary pieces.

The coordinate frontier identity is transported by the Euclidean homeomorphism.

```

theorem euclidean_frontier :
  frontier (planeEuclideanHomeomorph '' a.carrier) =
    (planeEuclideanHomeomorph '' a.leftArcTrace) ∪
    ((planeEuclideanHomeomorph '' a.rightArcTrace) ∪
      ((planeEuclideanHomeomorph ''
        OneSidedCircularCap.arcTrace a.outerCap) ∪
        (planeEuclideanHomeomorph '' a.bottomSegmentCarrier))) := by
  rw [← planeEuclideanHomeomorph.image_frontier, a.frontier_carrier]
  simp only [boundaryTrace, image_union]

/-- The component perimeter is the canonical weighted `H¹` integral over the
complete topological frontier. This includes the exposed bottom segment and
is valid at its zero-length semicircular transition. -/

```

TypeThreeAssembly.exact_euclidean_bottomSegmentCarrier_hausdorffMeasure

Classification: main

Source: TypeThreeAssembly.lean:1101-1129

Declaration SHA-256: 56949577898a4e9f1638a1a8bab2c43a8263ea9e6cf0976fce47fbff56b3fe00

Mathematical statement. The Euclidean bottom segment has Hausdorff measure equal to bottomChord.

The theorem includes length zero at $h = 1/2$, so the perimeter formula is branch-complete.

```

theorem exact_euclidean_bottomSegmentCarrier_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' a.bottomSegmentCarrier) =
    ENNReal.ofReal a.bottomChord := by
  rcases eq_or_ne a.h (1 / 2) with hh | hh
  · have hw : a.sideHalfWidth = 0 :=
      a.sideHalfWidth_eq_zero_iff.mpr hh
    have hc : a.bottomChord = 0 :=
      a.bottomChord_eq_zero_iff.mpr hh
    have hcarrier : a.bottomSegmentCarrier = {(0, -1)} := by
      ext p
      simp only [bottomSegmentCarrier, mem_inter_iff, mem_ofPred_eq,
        hw, abs_nonpos_iff, mem_singleton_iff]
    constructor

```

```

· rintro (hy, hx)
  exact Prod.ext hx hy
· intro hp
  rw [hp]
  exact (rfl, rfl)
rw [hcarrier, image_singleton, hc, ENNReal.ofReal_zero]
let _ : NullSingletonClass (μH[1] : Measure EuclideanPlane) :=
  Measure.nullSingletonClass_hausdorff EuclideanPlane (by norm_num)
exact measure_singleton _
· rw [← a.bottomHorizontalSegment_carrier hh,
  exact_euclidean_segmentCarrier_hausdorffMeasure]
  rfl

/-- The four primitive boundary joins. The two bottom points coincide at the
semicircular transition. -/

```

TypeThreeAssembly.exact_euclidean_leftArcTrace_hausdorffMeasure

Classification: main

Source: TypeThreeAssembly.lean:817-824

Declaration SHA-256: 9b484f3710815971495ad59b2a292d470a94ac2f3cad3edb96b8870979733b3

Mathematical statement. The actual Euclidean left side trace has length $R \gamma$.

The exact image identity converts the parameter length into a boundary-set measure.

```

theorem exact_euclidean_leftArcTrace_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' a.leftArcTrace) =
  ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  rw [← a.realizedLeftArcParam_image_Icc]
  exact a.exact_euclidean_leftArcParam_hausdorffMeasure

/-- Exact Euclidean  $H^1$  mass of the right strip-side trace. -/

```

TypeThreeAssembly.exact_euclidean_rightArcTrace_hausdorffMeasure

Classification: main

Source: TypeThreeAssembly.lean:825-832

Declaration SHA-256: 82deac992e3b3149a73b5a9867d35c880f6af8171bd8434cc5a883788eaac184

Mathematical statement. The actual Euclidean right side trace has length $R \gamma$.

The two traces together have the reference side-cap arc length $2R\gamma$.

```

theorem exact_euclidean_rightArcTrace_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' a.rightArcTrace) =
  ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  rw [← a.realizedRightArcParam_image_Icc]
  exact a.exact_euclidean_rightArcParam_hausdorffMeasure

/-- The two translated strip-side half-caps have the area of `sideCap`. -/

```

TypeThreeAssembly.frontier_carrier

Classification: main

Source: TypeThreeAssembly.lean:2848-2854

Declaration SHA-256: 0ce2b73ef2aba31ce763d170404d962fb774d3c494bf5afe10757388adbfbd30

Mathematical statement. The complete type-(iii) frontier equals the listed trace union.

Both inclusions are necessary to identify the canonical perimeter with the scalar formula.

```
theorem frontier_carrier :
  frontier a.carrier = a.boundaryTrace :=
  Set.Subset.antisymm a.frontier_carrier_subset_boundaryTrace
  a.boundaryTrace_subset_frontier_carrier

/-- The Euclidean realization of the bottom segment is measurable, including
the singleton occurring at the semicircular transition. -/
```

TypeThreeAssembly.hasModeledCompetitor

Classification: main

Source: TypeThreeAssembly.lean:3081-3100

Declaration SHA-256: 6010bc1b92d07ee7af37566e925a969f227d520e7144371bb334d772b94b8386

Mathematical statement. Every regular type-(iii) assembly has the required admissible realization.

The explicit carrier, full-frontier formulas, and integrability theorems construct the record on every allowed branch.

```
theorem hasModeledCompetitor : a.HasModeledCompetitor := by
  refine {
    competitor := .general {
      carrier := a.carrier
      measurable_carrier := a.measurableSet_carrier
      finite_weighted_area := a.integrableOn_carrier
      finite_weighted_perimeter := a.integrable_frontierMeasure }
    carrier_eq := rfl }

end TypeThreeAssembly

namespace FourArcCandidate

variable {lam : ℝ} (candidate : FourArcCandidate lam)

/-- Observable modeled-geometry bridge. Once the coordinate type-(iii)
carrier is realized as an admissible competitor, the scalar certificate needs
only equal weighted area and strict scalar perimeter improvement. Minimality
then fails against that genuine competitor. -/
```

TypeThreeAssembly.innerAngle

Classification: main

Source: TypeThreeAssembly.lean:53-55

Declaration SHA-256: bb9acabc2bc9f5427d7041305c401b4e2793720759dd063e3153448db0793dd9

Mathematical statement. The source inner angle b_3 .

$$b_3 = \arccos(2h - 1) \in (0, \pi).$$

```
def innerAngle : ℝ := arccos a.shapeParameter

/-- Snell-adjusted arccosine argument for the exterior angle `a₃`. -/
```

TypeThreeAssembly.integrableOn_carrier

Classification: main

Source: TypeThreeAssembly.lean:3000-3011

Declaration SHA-256: f50bce5009ef96b4081db29960a7e72186fb76edbfedf9fa6fd538bc59ab8c22

Mathematical statement. The density is integrable over the type-(iii) carrier.

Finite component areas and bounded density establish admissible weighted-area finiteness.

```
theorem integrableOn_carrier :
  IntegrableOn (StripDensity lam) a.carrier := by
  rw [carrier]
  have hcore : IntegrableOn (StripDensity lam) a.coreCarrier := by
    apply stripDensity_integrableOn_of_volume_ne_top
    rw [a.volume_coreCarrier]
    exact ENNReal.ofReal_ne_top
  exact hcore.union (cap_integrableOn lam a.outerCap)

/-- The density is integrable against the canonical frontier measure. This is
the finite-perimeter obligation for treating the coordinate carrier as a
general admissible competitor. -/
```

TypeThreeAssembly.integrable_frontierMeasure

Classification: main

Source: TypeThreeAssembly.lean:3012-3042

Declaration SHA-256: 242285e9e93302048576853114a4bc7f6ca16cdc917f22c33a2fe025def880a8

Mathematical statement. The density is integrable against the complete type-(iii) frontier measure.

The frontier is a finite union of finite-length arcs and a finite segment, so the canonical weighted perimeter is finite.

```
theorem integrable_frontierMeasure :
  Integrable (StripDensity lam) (FrontierMeasure a.carrier) := by
  have hfrontier :
    (μH[1] : Measure EuclideanPlane)
      (frontier (planeEuclideanHomeomorph '' a.carrier)) * T := by
  rw [a.euclidean_frontier]
  apply measure_union_ne_top
  · rw [a.exact_euclidean_leftArcTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top
  apply measure_union_ne_top
  · rw [a.exact_euclidean_rightArcTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top
  apply measure_union_ne_top
  · rw [a.exact_euclidean_capTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top
  · rw [a.exact_euclidean_bottomSegmentCarrier_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top
  unfold FrontierMeasure
  rw [integrable_map_measure
    (measurable_stripDensity lam).astronglyMeasurable
    planeEuclideanHomeomorph.symm.continuous.measurable.aemeasurable]
  change IntegrableOn (euclideanStripDensity lam)
    (frontier (planeEuclideanHomeomorph '' a.carrier))
    (μH[1] : Measure EuclideanPlane)
  exact euclideanStripDensity_integrableOn_of_measure_ne_top lam hfrontier

/-- Geometric realization contract for the coordinate carrier. Both weighted
measure formulas follow from `carrier_eq`; neither is independently selectable
data. -/
```

TypeThreeAssembly.modeledWeightedArea

Classification: main

Source: TypeThreeAssembly.lean:278-284

Declaration SHA-256: 35499fdca0a3bbbed11909d6dec0bbb1c83123d6ea5cb0599717ee289f567e87e

Mathematical statement. The type-(iii) component area sum.

It is rectangle area plus reference side-cap area plus λ times outer-cap area.

```
def modeledWeightedArea : ℝ :=
  a.rectangleArea + a.sideCap.euclideanArea +
    lam * a.outerCap.euclideanArea

/-- Component perimeter of the modeled type-(iii) profile. The side-cap arc
is the sum of the two translated side arcs; `bottomChord` also covers the
zero-length transition at `h = 1 / 2`. -/
```

TypeThreeAssembly.modeledWeightedArea_formula

Classification: main

Source: TypeThreeAssembly.lean:885-895

Declaration SHA-256: b0e70e8aeaefad1e18dbaff1d3d6364f544c6042f48ce97bebd3fa5c6cc8e74e

Mathematical statement. The geometric component area sum equals the source scalar type-(iii) area formula.

Substitute the rectangle and segment areas and simplify using the incidence and complementary-angle identities.

```
theorem modeledWeightedArea_formula :
  a.modeledWeightedArea = a.scalarWeightedArea := by
  rw [modeledWeightedArea, rectangleArea, a.sideCap.euclideanArea,
    a.outerCap.euclideanArea, scalarWeightedArea, angleTerm, sineGap,
    sideHalfWidth, a.sin_sideArcAngle, a.cos_sideArcAngle, a.cos_outerAngle,
    outerArgument, sideArcAngle, radius, shapeParameter]
  field_simp [ne_of_gt a.h_pos,
    ne_of_gt (lt_trans (by norm_num : (0 : ℝ) < 1) a.density_jump)]
  ring

/-- CMV equation (26), including the degenerate bottom-chord branch. -/
```

TypeThreeAssembly.modeledWeightedPerimeter

Classification: main

Source: TypeThreeAssembly.lean:285-288

Declaration SHA-256: 76198be6eb5b8790a4af34449bf29c4b963314e1e4bd268b4d3c9cdb2e69ea0d

Mathematical statement. The type-(iii) component perimeter sum including the bottom chord.

It is side-cap arc length plus λ times upper-cap arc length plus $2d$.

```
def modeledWeightedPerimeter : ℝ :=
  a.sideCap.arclength + lam * a.outerCap.arclength + a.bottomChord

/-- The angle term `lambda * a_3 + pi - b_3` in CMV equation (26). -/
```

TypeThreeAssembly.modeledWeightedPerimeter_formula

Classification: main

Source: TypeThreeAssembly.lean:896-903

Declaration SHA-256: bb4c05a2ca2fd9d5b63be1b6ad16da716cc43497abcad228054765e6eaa2e39e

Mathematical statement. The complete component perimeter sum equals the source scalar type-(iii) perimeter formula.

The sum $2Ry + 2\lambda Ra_3 + 2R(\sin a_3 - \sin b_3)$ becomes $P_3(h)$.

```
theorem modeledWeightedPerimeter_formula :
  a.modeledWeightedPerimeter = a.scalarWeightedPerimeter := by
  rw [modeledWeightedPerimeter, a.sideCap_arclength,
    a.outerCap_arclength, scalarWeightedPerimeter, angleTerm, sineGap,
```



```

bottomChord, sideHalfWidth, sideArcAngle, radius]
field_simp [ne_of_gt a.h_pos]
ring

```

TypeThreeAssembly.outerAngle

Classification: main

Source: TypeThreeAssembly.lean:59-63

Declaration SHA-256: 0e2ff2070394fdda5b4ec09824f47ed543dabdfef101c076ad378e0c92be2ff47

Mathematical statement. The exterior upper-cap half-angle a_3 .

$a_3 = \arccos((2h - 1)/\lambda)$, allowing $a_3 > \pi/2$ when $h < 1/2$.

```

def outerAngle : ℝ := arccos a.outerArgument

/-- Half-central-angle of either strip-side arc. This is the complementary
branch to `innerAngle`, so it remains valid on both sides of `h = 1 / 2`. -/

```

TypeThreeAssembly.outerCap

Classification: main

Source: TypeThreeAssembly.lean:251-261

Declaration SHA-256: 5482055613f2ec0454ddd9c71d722bea04494543bc659e447732870c662eb86f

Mathematical statement. The actual upper exterior type-(iii) cap.

It has radius R , chord $2R\sin a_3$, base $y = 1$, and half-angle $a_3 \in (0, \pi)$.

```

def outerCap : OneSidedCircularCap where
  chord := 2 * a.radius * sin a.outerAngle
  theta := a.outerAngle
  midpointX := 0
  baseY := 1
  side := .upper
  chord_pos := mul_pos (mul_pos (by norm_num) a.radius_pos) a.outerSin_pos
  theta_pos := a.outerAngle_pos
  theta_lt_pi := a.outerAngle_lt_pi

/-- Genuine closed coordinate carrier `E_3 = K_3 ∪ upper_cap`. -/

```

TypeThreeAssembly.outerCap_arcLength

Classification: main

Source: TypeThreeAssembly.lean:875-884

Declaration SHA-256: e4c6a09e492a4c9395262e1c277f2cd0b185e72ae34042a7f14138b12c6b4c19

Mathematical statement. The upper exterior arc length is $2R a_3$.

The chord normalization cancels its positive sine factor.

```

theorem outerCap_arcLength :
  a.outerCap.arcLength = 2 * a.radius * a.outerAngle := by
  rw [OneSidedCircularCap.arcLength, ell]
  change
    (2 * a.radius * sin a.outerAngle) *
      (a.outerAngle / sin a.outerAngle) =
    2 * a.radius * a.outerAngle
  field_simp [ne_of_gt a.outerSin_pos]

```

```
-- CMV equation (26), derived from the branch-complete component geometry. -/
```

TypeThreeAssembly.outerCap_euclideanArea

Classification: main

Source: TypeThreeAssembly.lean:851-864

Declaration SHA-256: 3c7709918bd2e08d7c7ea431b339d946f2eb6c3c9d1138f4304b6afe64a153c2

Mathematical statement. The upper exterior cap has the standard full-branch circular-segment area.

Its area is $R^2(a_3 - \sin a_3 \cos a_3)$, valid for minor and major caps.

```
theorem outerCap_euclideanArea :
  a.outerCap.euclideanArea =
    a.radius ^ 2 *
      (a.outerAngle - sin a.outerAngle * cos a.outerAngle) := by
  rw [OneSidedCircularCap.euclideanArea, area]
  change
    (2 * a.radius * sin a.outerAngle) ^ 2 *
      ((a.outerAngle - sin a.outerAngle * cos a.outerAngle) /
        (4 * sin a.outerAngle ^ 2)) =
    a.radius ^ 2 *
      (a.outerAngle - sin a.outerAngle * cos a.outerAngle)
  field_simp [ne_of_gt a.outerSin_pos]
  ring
```

TypeThreeAssembly.pairwiseOverlap_h1_null

Classification: main

Source: TypeThreeAssembly.lean:1258-1273

Declaration SHA-256: 545b8d3e4922237126e7364ee9bb07b444cf8cada72b02e8afca89501d2f79b3

Mathematical statement. The type-(iii) boundary overlaps have zero Euclidean Hausdorff length.

Finite junctions are $\mathcal{H}^1$ -null, so component perimeter integrals add without double-counting.

```
theorem pairwiseOverlap_h1_null :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' pairwiseOverlap
    a.leftArcTrace a.rightArcTrace
    (OneSidedCircularCap.arcTrace a.outerCap)
    a.bottomSegmentCarrier) = 0 := by
  let _ := Measure.nullSingletonClass_hausdorff EuclideanPlane
  (by norm_num : (0 : ℝ) < 1)
  have hjoin :
    (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' a.boundaryJoinSet) = 0 :=
    (a.boundaryJoinSet_finite.image planeEuclideanHomeomorph).measure_zero
  μH[1]
  exact measure_mono_null
    (image_mono a.pairwiseOverlap_subset_boundaryJoinSet) hjoin
```

TypeThreeAssembly.radius

Classification: main

Source: TypeThreeAssembly.lean:47-49

Declaration SHA-256: 44bdb144ee33a0fe634670e914d151b3f9a50f2b2f7db8da7c3fad23bdfbf70e

Mathematical statement. The common type-(iii) circle radius.

$R = 1/h > 1$.

```
def radius : ℝ := 1 / a.h

/-- Source scalar `2h - 1`, equal to `cos b₃`. -/
```

TypeThreeAssembly.scalarWeightedArea

Classification: main

Source: TypeThreeAssembly.lean:295-299

Declaration SHA-256: 195fdb382725116907cc721bab689c5c6f6f34b356a442f900ba649d89b62b9c

Mathematical statement. The source scalar area expression attached to the assembly.

It is $(\text{angleTerm} + (q + 2)\text{sineGap})/h^2$.

```
def scalarWeightedArea : ℝ :=
  (a.angleTerm + (a.shapeParameter + 2) * a.sineGap) / a.h ^ 2

/-- Scalar type-(iii) weighted-perimeter formula used by the suffix
certificate. -/
```

TypeThreeAssembly.scalarWeightedPerimeter

Classification: main

Source: TypeThreeAssembly.lean:300-302

Declaration SHA-256: fcbac6602a2b133a8e714c37e1ef51bfb698ab0ae754dcd7eca0cb6e9c49204d

Mathematical statement. The source scalar perimeter expression attached to the assembly.

It is $2(\text{angleTerm} + \text{sineGap})/h$.

```
def scalarWeightedPerimeter : ℝ :=
  2 * (a.angleTerm + a.sineGap) / a.h
```

TypeThreeAssembly.shapeParameter

Classification: main

Source: TypeThreeAssembly.lean:50-52

Declaration SHA-256: 29190b0ec0676db3ee8da889fc7b1e078bce75188f6b4271945e3fee045f2920

Mathematical statement. The source cosine parameter $q=2h-1$.

It lies in $(-1, 1)$ and changes sign at the semicircular transition.

```
def shapeParameter : ℝ := 2 * a.h - 1

/-- The source angle `b₃`, represented by its principal arccosine. -/
```

TypeThreeAssembly.sideArcAngle

Classification: main

Source: TypeThreeAssembly.lean:64-65

Declaration SHA-256: c68849312591ff361810874588f817eab8327b61ad9b815244ad1ab61a55597a

Mathematical statement. The complementary angle controlling the combined side arcs.

$\gamma = \pi - b_3$; each side trace uses half of the corresponding full side-cap arc.

```
def sideArcAngle : ℝ := π - a.innerAngle
/-- Half the bottom-segment length. -/
```

TypeThreeAssembly.sideCap

Classification: main

Source: TypeThreeAssembly.lean:238-250

Declaration SHA-256: 62a5fb2c6003ebc71ae1cea156d819929dd2061bdfd35d03ac33c3bf896e8fd0

Mathematical statement. The reference circular cap whose two translated halves form the side regions.

Its radius is R , chord is $2R\sin b_3$, and half-angle is $\gamma = \pi - b_3$.

```
def sideCap : OneSidedCircularCap where
  chord := 2 * a.radius * sin a.innerAngle
  theta := a.sideArcAngle
  midpointX := 0
  baseY := 1
  side := .lower
  chord_pos := mul_pos (mul_pos (by norm_num) a.radius_pos) a.innerSin_pos
  theta_pos := a.sideArcAngle_pos
  theta_lt_pi := a.sideArcAngle_lt_pi

/-- The upper one-sided cap. `OneSidedCircularCap` deliberately permits a
half-central-angle above `π / 2`, so this definition covers the major branch.
-/
```

TypeThreeAssembly.sideCap_arcLength

Classification: main

Source: TypeThreeAssembly.lean:865-874

Declaration SHA-256: 0735084a84605f457abd29f126bfc3b41f13fe2e361355b34befa71a89121ae9

Mathematical statement. The reference side-cap arc length is $2R\gamma$.

This is the sum of the two actual side-trace lengths.

```
theorem sideCap_arcLength :
  a.sideCap.arcLength = 2 * a.radius * a.sideArcAngle := by
  rw [OneSidedCircularCap.arcLength, ell]
  change
    (2 * a.radius * sin a.innerAngle) *
    (a.sideArcAngle / sin a.sideArcAngle) =
    2 * a.radius * a.sideArcAngle
  rw [a.sin_sideArcAngle]
  field_simp [ne_of_gt a.innerSin_pos]
```

TypeThreeAssembly.sideCap_euclideanArea

Classification: main

Source: TypeThreeAssembly.lean:833-850

Declaration SHA-256: b6b94a03ab653ee46e011e7fa757125fdbdfe26f1473cd6911d9100fe92583e4

Mathematical statement. The reference side cap has area $R^2(\gamma - \sin \gamma \cos \gamma)$.

This cap area equals the combined area of the two translated side half-segments.

```
theorem sideCap_euclideanArea :
  a.sideCap.euclideanArea =
    a.radius ^ 2 *
    (a.sideArcAngle -
```

```

      sin a.sideArcAngle * cos a.sideArcAngle) := by
rw [OneSidedCircularCap.euclideanArea, area]
change
  (2 * a.radius * sin a.innerAngle) ^ 2 *
    ((a.sideArcAngle -
      sin a.sideArcAngle * cos a.sideArcAngle) /
      (4 * sin a.sideArcAngle ^ 2)) =
a.radius ^ 2 *
  (a.sideArcAngle -
    sin a.sideArcAngle * cos a.sideArcAngle)
rw [a.sin_sideArcAngle]
field_simp [ne_of_gt a.innerSin_pos]
ring

```

TypeThreeAssembly.sideHalfWidth

Classification: main

Source: TypeThreeAssembly.lean:66-69

Declaration SHA-256: c1a6374b950f6fdf60b58062fd5753e6cafd953a8fbac46cb6164191b66af1b3

Mathematical statement. Half the exposed bottom segment's length.

$d = R(\sin a_3 - \sin b_3) \geq 0$, with equality exactly at $h = 1/2$.

```

def sideHalfWidth : ℝ :=
  a.radius * (sin a.outerAngle - sin a.innerAngle)

/-- Source circle center for the left strip arc. -/

```

TypeThreeAssembly.sideHalfWidth_eq_zero_iff

Classification: main

Source: TypeThreeAssembly.lean:344-351

Declaration SHA-256: aa31042682af8c9fc65e3a19a4a9ce7dc71ed7e2ad3e807732dcfcc2da35644f

Mathematical statement. The half-width vanishes exactly at $h=1/2$.

This characterizes the degenerate rectangle/bottom-segment branch rather than excluding it.

```

theorem sideHalfWidth_eq_zero_iff : a.sideHalfWidth = 0 ↔ a.h = 1 / 2 := by
  constructor
  · intro hzero
    by_contra hh
    linarith [a.sideHalfWidth_pos hh]
  · intro hh
    simp [sideHalfWidth, innerAngle, outerAngle, outerArgument, shapeParameter, hh]

```

TypeThreeAssembly.sineGap

Classification: main

Source: TypeThreeAssembly.lean:292-294

Declaration SHA-256: e9d7a7074e27fe28a23aa0db9c4cf9b0ef4e5f2181f1cfab6098f7cc2e240ecb

Mathematical statement. The exterior-minus-inner sine displacement.

It is $\sin a_3 - \sin b_3 = D_3(h)$.

```

def sineGap : ℝ := sin a.outerAngle - sin a.innerAngle

/-- Scalar type-(iii) weighted-area formula used by the suffix certificate. -/

```

TypeThreeAssembly.volume_coreCarrier

Classification: main

Source: TypeThreeAssembly.lean:1481-1529

Declaration SHA-256: e422839d8a82e578aa8619bf8a6d240c5dc840fd4d4e1880ecc09a0307558380

Mathematical statement. The type-(iii) core volume equals rectangle area plus reference side-cap area.

The two translated half-segments add to the reference cap modulo area-null shared edges.

```
theorem volume_coreCarrier :
  volume a.coreCarrier =
    ENNReal.ofReal (a.rectangleArea + a.sideCap.euclideanArea) := by
  have hrectLeft :
    AEDisjoint volume a.rectangleCarrier a.leftSegmentCarrier := by
    refine measure_mono_null ?_ (volume_verticalline (-a.sideHalfWidth))
    rintro p ⟨hrect, hleft⟩
    exact le_antisymm hleft.2.1 hrect.1.1
  have hfirstRight :
    AEDisjoint volume
      (a.rectangleCarrier ∪ a.leftSegmentCarrier)
      a.rightSegmentCarrier := by
    refine measure_mono_null ?_ (volume_verticalline a.sideHalfWidth)
    rintro p ⟨hfirst, hright⟩
    rcases hfirst with hrect | hleft
    · exact le_antisymm hrect.1.2 hright.2.1
    · have hx1 : p.1 ≤ -a.sideHalfWidth := hleft.2.1
      have hxr : a.sideHalfWidth ≤ p.1 := hright.2.1
      exact le_antisymm (by linarith [a.sideHalfWidth_nonneg]) hxr
  have hrectangle : 0 ≤ a.rectangleArea := by
    exact mul_nonneg (by norm_num) a.sideHalfWidth_nonneg
  calc
    volume a.coreCarrier =
      volume a.rectangleCarrier + volume a.leftSegmentCarrier +
      volume a.rightSegmentCarrier := by
    rw [coreCarrier,
      measure_union,
      a.isClosed_rightSegmentCarrier.measurableSet.nullMeasurableSet
      hfirstRight,
      measure_union,
      a.isClosed_leftSegmentCarrier.measurableSet.nullMeasurableSet
      hrectLeft]
  _ = ENNReal.ofReal a.rectangleArea +
    (volume a.sideCapLeftHalf + volume a.sideCapRightHalf) := by
    rw [a.volume_rectangleCarrier, a.volume_leftSegmentCarrier,
      a.volume_rightSegmentCarrier, add_assoc]
  _ = ENNReal.ofReal a.rectangleArea + volume a.sideCap.carrier := by
    rw [a.volume_sideCap_partition]
  _ = ENNReal.ofReal a.rectangleArea +
    ENNReal.ofReal a.sideCap.euclideanArea := by
    rw [a.sideCap.volume_carrier]
  _ = ENNReal.ofReal
    (a.rectangleArea + a.sideCap.euclideanArea) := by
    exact
      (ENNReal.ofReal_add hrectangle
        a.sideCap.euclideanArea_pos.le).symm

/-- The actual density integral over the strip core is its Euclidean component
area. -/
```

TypeThreeAssembly.weightedArea_eq_modeledWeightedArea

Classification: main

Source: TypeThreeAssembly.lean:1562-1583

Declaration SHA-256: 5f7913f0171c133683a735330062b186a060d0e518987c5488549de7bfd06347

Mathematical statement. The actual carrier area equals the type-(iii) component area expression.

The core and exterior cap meet only on an area-null interface, where the density convention does not change the area integral.

```
theorem weightedArea_eq_modeledWeightedArea :
  a.WeightedArea = a.modeledWeightedArea := by
  have hcore : IntegrableOn (StripDensity lam) a.coreCarrier := by
    apply stripDensity_integrableOn_of_volume_ne_top
    rw [a.volume_coreCarrier]
    exact ENNReal.ofReal_ne_top
  have hcap := cap_integrableOn lam a.outerCap
  unfold TypeThreeAssembly.WeightedArea _root_.WeightedArea
  rw [carrier,
    setIntegral_union, a.carrier_core_outer_aedisjoint
    a.outerCap.measurableSet_carrier.nullMeasurableSet hcore hcap]
  rw [show (∫ p in a.coreCarrier, StripDensity lam p) =
    a.rectangleArea + a.sideCap.euclideanArea by
    simp [a._root_.WeightedArea] using a.core_weightedArea_formula,
    show (∫ p in a.outerCap.carrier, StripDensity lam p) =
    lam * a.outerCap.euclideanArea by
    simp [a._root_.WeightedArea] using
    cap_upper_weightedArea lam a.outerCap rfl rfl,
    modeledWeightedArea]

/-- CMV equation (26) is the exact canonical weighted area of the coordinate
carrier, not only a sum assigned to its modeled components. -/
```

TypeThreeAssembly.weightedArea_formula

Classification: main

Source: TypeThreeAssembly.lean:1584-1588

Declaration SHA-256: 53616c2c25615a95257aa4773c87d4c69464dc905650e6ed5b53696dc5f08570

Mathematical statement. The actual type-(iii) weighted area equals its scalar source formula.

Combine the carrier-to-component area identity with the algebraic component formula.

```
theorem weightedArea_formula :
  a.WeightedArea = a.scalarWeightedArea := by
  rw [a.weightedArea_eq_modeledWeightedArea,
    a.modeledWeightedArea_formula]
```

TypeThreeAssembly.weightedPerimeter_eq_modeledWeightedPerimeter

Classification: main

Source: TypeThreeAssembly.lean:2950-2993

Declaration SHA-256: 3dd334c8f3cfff9146f23a5d90e6da2520151c567d49c8604446b90aa7a9f4d2b

Mathematical statement. The actual complete-frontier perimeter equals the full type-(iii) component perimeter sum.

Exact trace lengths, density values, measurable pieces, and null junction overlaps establish the identity.

```
theorem weightedPerimeter_eq_modeledWeightedPerimeter :
  a.WeightedPerimeter = a.modeledWeightedPerimeter := by
  rcases a.boundaryTrace_pairwise with
    ⟨hAB, hAC, hAD, hBC, hBD, hCD⟩
  have hAint := euclideanStripDensity_integrableOn_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph '' a.leftArcTrace) (by
      rw [a.exact_euclidean_leftArcTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
  have hBint := euclideanStripDensity_integrableOn_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph '' a.rightArcTrace) (by
      rw [a.exact_euclidean_rightArcTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
  have hCint := euclideanStripDensity_integrableOn_of_measure_ne_top
```

```

    lam (s := planeEuclideanHomeomorph ''
      OneSidedCircularCap.arcTrace a.outerCap) (by
        rw [exact_euclidean_capTrace_hausdorffMeasure]
        exact ENNReal.ofReal_ne_top)
  have hDint := euclideanStripDensity_integrableOn_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph '' a.bottomSegmentCarrier) (by
      rw [a.exact_euclidean_bottomSegmentCarrier_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
  unfold TypeThreeAssembly.WeightedPerimeter _root_.WeightedPerimeter
    FrontierMeasure
  rw [integral_map
    planeEuclideanHomeomorph.symm.continuous.measurable.aemeasurable
    (measurable_stripDensity lam).aestronglyMeasurable]
  change (∫ z in frontier (planeEuclideanHomeomorph '' a.carrier),
    euclideanStripDensity lam z
      ∂(μH[1] : Measure EuclideanPlane)) =
    a.modeledWeightedPerimeter
  rw [a.euclidean_frontier]
  rw [integral_four_union hAB hAC hAD hBC hBD hCD
    a.measurableSet_euclidean_rightArcTrace
    (measurableSet_euclidean_capTrace a.outerCap)
    a.measurableSet_euclidean_bottomSegmentCarrier
    hAint hBint hCint hDint]
  rw [a.leftArc_integral, a.rightArc_integral,
    upper_cap_integral lam a.outerCap rfl rfl,
    a.bottomSegment_integral, modeledWeightedPerimeter,
    a.sideCap_arcLength]
  ring

/-- CMV equation (26) is the exact canonical weighted perimeter of the
coordinate carrier, not only a sum assigned to its modeled components. -/

```

TypeThreeAssembly.weightedPerimeter_formula

Classification: main

Source: TypeThreeAssembly.lean:2994-2999

Declaration SHA-256: c6f51109b06f9f61cbbcb63e9d6e1d950e5a79c8decdcf76b0e2fc3f366f8eca

Mathematical statement. The canonical type-(iii) weighted perimeter equals its scalar source formula.

Combine the complete-frontier component calculation with the algebraic expression $2(B_3 + D_3)/h$.

```

theorem weightedPerimeter_formula :
  a.WeightedPerimeter = a.scalarWeightedPerimeter := by
  rw [a.weightedPerimeter_eq_modeledWeightedPerimeter,
    a.modeledWeightedPerimeter_formula]

/-- The density is integrable over the complete coordinate carrier. -/

```

WeightedArea

Classification: main

Source: CMVGeometry.lean:55-60

Declaration SHA-256: 959b5ddd7363861d4d0d79c211781b3b19e09df50087472690ae0581725b8c6d

Mathematical statement. Lebesgue integral of the density over a carrier.

$A_\lambda(E) = \int_E f_\lambda d\mathcal{L}^2$. Integrability is verified for every competitor used.

```

def WeightedArea (lam : ℝ) (carrier : Set PlanePoint) : ℝ :=
  ∫ p in carrier, StripDensity lam p

/-- Canonical Euclidean one-dimensional Hausdorff measure on the complete
topological frontier of a planar carrier, mapped back to the coordinate plane.
A general competitor cannot choose perimeter data independently of its region. -/

```


WeightedPerimeter

Classification: main

Source: CMVGeometry.lean:67-73

Declaration SHA-256: 8d1f1a60a6d4c5ce03ccda31c1bc34ec4f837b6ce55275e4b0c06fe04e566dba

Mathematical statement. The density integral against a given boundary measure.

For admissible carriers the measure is $\mathcal{H}^1 \upharpoonright \partial E$ in Euclidean coordinates.

```
def WeightedPerimeter (lam : ℝ)
  (boundaryMeasure : Measure PlanePoint) : ℝ :=
  ∫ p, StripDensity lam p ∂boundaryMeasure

/-- General finite-perimeter input to minimization. Its perimeter measure is
canonically the one-dimensional Hausdorff measure restricted to the complete
frontier; finiteness is an invariant, not independently supplied data. -/
```

arc

Classification: main

Source: MorgansArcFunction.lean:267-268

Declaration SHA-256: d1b33dc05b11585801ca5ae18eed6d15a605f3d852a3120f8561f8ec87bdc3a5

Mathematical statement. The unit-chord arc length as a function of enclosed area.

$L(X) = \ell(\theta(X))$.

```
noncomputable def arc (x : ℝ) : ℝ := ell (θof x)
```

area

Classification: main

Source: MorgansArcFunction.lean:92-93

Declaration SHA-256: 8c44e8dff416b2208afe91aad54bf1678ad6f2299c75592fe1fed4f6cbf55dc2

Mathematical statement. Area of a unit-chord circular segment.

$a(\theta) = (\theta - \sin\theta\cos\theta)/(4\sin^2\theta)$.

```
noncomputable def area (θ : ℝ) : ℝ := (θ - sin θ * cos θ) / (4 * sin θ ^ 2)
```

area_deriv_pos

Classification: main

Source: MorgansArcFunction.lean:125-130

Declaration SHA-256: 9b40e28d2efa8873c77b49eab27397ba54c7d20988e9930377fea092b5b1fc66

Mathematical statement. The normalized cap-area derivative is positive.

Its numerator and denominator are both positive on the full cap-angle domain.

```
lemma area_deriv_pos {θ : ℝ} (h0 : 0 < θ) (hπ : θ < π) :
  0 < (sin θ - θ * cos θ) / (2 * sin θ ^ 3) := by
  have hs : (0:ℝ) < sin θ := sin_pos_of_pos_of_lt_pi h0 hπ
  have := sin_sub_mul_cos_pos h0 hπ
  positivity
```

area_pi_sub_eq_two_mul

Classification: main

Source: Minimiser.lean:141-153

Declaration SHA-256: a8e24b12489f7a578d21b419b549e47a8c62b05b8eaac09a11261e5958d93945

Mathematical statement. At the selected root, the supplementary angle has twice the normalized area.

The equation $3\theta^* - 3\sin\theta^*\cos\theta^* = \pi$ rearranges to $a(\pi - \theta^*) = 2a(\theta^*)$.

```
lemma area_pi_sub_eq_two_mul {θ : ℝ} (hθ : θ ∈ Ioo 0 π) (hF : F θ = 0) :
  area (π - θ) = 2 * area θ := by
  have hs : 0 < sin θ := sin_pos_of_pos_of_lt_pi hθ.1 hθ.2
  have hne : (4 : ℝ) * sin θ ^ 2 ≠ 0 := by positivity
  have hpi : π = 3 * θ - 3 * (sin θ * cos θ) := by
    have := hF; rw [F] at this; linarith
  rw [area_pi_sub hθ, area, hpi]
  field_simp
  ring

/-! ## The critical point of `g` -/

/-- `x* = area θ*`, the area of the smaller cap at the critical point. -/
```

area_strictMonoOn

Classification: main

Source: MorgansArcFunction.lean:131-147

Declaration SHA-256: 5637c9200e5eb02be872785f063a999d67c4275f1b2e3906ef15f278ad2dd469

Mathematical statement. Normalized cap area is strictly increasing on $(0, \pi)$.

The positive derivative makes the area-to-angle inverse unique.

```
lemma area_strictMonoOn : StrictMonoOn area (Ioo 0 π) := by
  apply strictMonoOn_of_hasDerivWithinAt_pos (convex_Ioo 0 π)
  · exact fun x hx => ((hasDerivAt_area hx.1 hx.2).continuousAt).continuousWithinAt
  · intro x hx
    rw [isOpen_Ioo.interior_eq] at hx
    exact (hasDerivAt_area hx.1 hx.2).hasDerivWithinAt
  · intro x hx
    rw [isOpen_Ioo.interior_eq] at hx
    exact area_deriv_pos hx.1 hx.2

/-! ## Endpoint behaviour

With `area_strictMonoOn`, these give that `area` is a strictly increasing
bijection from `(0, π)` onto `(0, ∞)`.
-/

/-- `area` is positive on `(0, π)`. -/
```

area_surjOn

Classification: main

Source: MorgansArcFunction.lean:217-246

Declaration SHA-256: fd1b58a2c230b6612a3da5e4c5ae32d43aeae3c716f031eb3ab7a09dd77ecd3b

Mathematical statement. Every positive normalized area is realized by a cap angle in $(0, \pi)$.

Continuity and the endpoint limits give surjectivity onto $(0, \infty)$; strict monotonicity gives uniqueness.

```

lemma area_surjOn : SurjOn area (Ioo 0 π) (Ioi 0) := by
  intro y hy
  simp only [mem_Ioi] at hy
  obtain ⟨a, haI, ha⟩ : ∃ a, a ∈ Ioo (0:ℝ) π ∧ area a < y := by
    have h1 : ∀ θ : ℝ in ℳ[>] 0, area θ < y :=
      area_tendsto_zero.eventually (eventually_lt_nhds hy)
    have h2 : ∀ θ : ℝ in ℳ[>] 0, θ ∈ Ioo (0:ℝ) π := by
      filter_upwards [eventually_mem_nhdsWithin,
        (eventually_lt_nhds pi_pos).filter_mono nhdsWithin_le_nhds] with θ ha hb
      exact ⟨ha, hb⟩
    exact (h2.and h1).exists
  obtain ⟨b, hbI, hb⟩ : ∃ b, b ∈ Ioo (0:ℝ) π ∧ y < area b := by
    have h1 : ∀ θ : ℝ in ℳ[<] π, y < area θ :=
      area_tendsto_atTop.eventually_gt_atTop y
    have h2 : ∀ θ : ℝ in ℳ[<] π, θ ∈ Ioo (0:ℝ) π := by
      filter_upwards [eventually_mem_nhdsWithin,
        (eventually_gt_nhds pi_pos).filter_mono nhdsWithin_le_nhds] with θ ha hb
      exact ⟨hb, ha⟩
    exact (h2.and h1).exists
  have hsub : uIcc a b ⊆ Ioo (0:ℝ) π := ordConnected_Ioo.uIcc_subset haI hbI
  have hcont : ContinuousOn area (uIcc a b) := area_continuousOn.mono hsub
  have hmem : y ∈ uIcc (area a) (area b) := by
    rw [Set.mem_uIcc]; left; exact ⟨le_of_lt ha, le_of_lt hb⟩
  obtain ⟨c, hc, hcy⟩ := intermediate_value_uIcc hcont hmem
  exact ⟨c, hsub hc, hcy⟩

/-! ## The inverse `θ0f`, and `arc` -/

/-- `θ0f x` is the unique `θ ∈ (0, π)` with `area θ = x`, for `x > 0`.
The junk value `π / 2` for `x ≤ 0` is never used. -/

```

area_tendsto_atTop

Classification: main

Source: MorgansArcFunction.lean:188-211

Declaration SHA-256: f69fc5f4e4ff813e6ef3d3de0124d162050bb67630aeab29d98accb32509b368

Mathematical statement. Normalized cap area tends to infinity as the angle increases to π .

The denominator tends to zero through positive values while the numerator tends to π .

```

lemma area_tendsto_atTop : Tendsto area (ℳ[<] π) atTop := by
  have hnum : Tendsto (fun θ : ℝ => θ - sin θ * cos θ) (ℳ[<] π) (ℳ π) := by
    refine Tendsto.mono_left ?_ nhdsWithin_le_nhds
    exact (continuous_id.sub (continuous_sin.mul continuous_cos)).tendsto' π π (by simp)
  have h0 : ∀ θ : ℝ in ℳ[<] π, 0 < θ :=
    (eventually_gt_nhds pi_pos).filter_mono nhdsWithin_le_nhds
  have hlt : ∀ θ : ℝ in ℳ[<] π, θ < π := eventually_mem_nhdsWithin
  have hden : Tendsto (fun θ : ℝ => 4 * sin θ ^ 2) (ℳ[<] π) (ℳ[>] 0) := by
    rw [tendsto_nhdsWithin_iff]
    refine ⟨?, ?⟩
    · refine Tendsto.mono_left ?_ nhdsWithin_le_nhds
      exact (continuous_const.mul (continuous_sin.pow 2)).tendsto' π 0 (by simp)
    · filter_upwards [h0, hlt] with θ ha hb
      have := sin_pos_of_pos_of_lt_pi ha hb
      simp only [mem_Ioi]
      positivity
  have key := Filter.Tendsto.pos_mul_atTop pi_pos hnum hden.inv_tendsto_nhdsGT_zero
  simp only [Pi.inv_apply] at key
  have harea : area = fun θ : ℝ => (θ - sin θ * cos θ) * (4 * sin θ ^ 2)-1 := by
    funext θ; rw [area, div_eq_mul_inv]
  rw [harea]
  exact key

/-- `area` is continuous on `(0, π)` – needed for the intermediate value step. -/

```

area_tendsto_zero

Classification: main

Source: MorgansArcFunction.lean:173-187

Declaration SHA-256: 6f401985b37aaf23b22e61608890d9398b8c4c885ef124fa9b1095661462feec

Mathematical statement. Normalized cap area tends to zero as the angle decreases to zero.

The small-angle bounds squeeze $a(\theta)$ between zero and a quantity tending to zero.

```
lemma area_tendsto_zero : Tendsto area ( $\mathcal{N}[>] 0$ ) ( $\mathcal{N} 0$ ) := by
  have hmem :  $\forall^\text{f} \theta : \mathbb{R} \text{ in } \mathcal{N}[>] 0, 0 < \theta \wedge \theta \leq 1$  := by
    filter_upwards [eventually_mem_nhdsWithin,
      (eventually_le_nhds (by norm_num :  $(0:\mathbb{R}) < 1$ )).filter_mono nhdsWithin_le_nhds]
      with  $\theta$  ha hb
    exact  $\langle \text{ha}, \text{hb} \rangle$ 
  apply squeeze_zero'
  · filter_upwards [hmem] with  $\theta$  h
    exact le_of_lt (area_pos h.1 (by linarith [pi_gt_three]))
  · filter_upwards [hmem] with  $\theta$  h
    exact le_of_lt (area_lt_self h.1 h.2)
  · exact tendsto_id.mono_left nhdsWithin_le_nhds

/-- `area  $\theta \rightarrow \infty$  as ` $\theta \rightarrow \pi^-$ `: the numerator tends to ` $\pi > 0$ ` while ` $4 \sin^2 \theta$ `
tends to ` $0$ ` from above. -/
```

area_00f

Classification: main

Source: MorgansArcFunction.lean:254-258

Declaration SHA-256: 18eb44e54860ffad838e65be5e17f4d170f11530a4f7b414a16d834abbb9e2c1

Mathematical statement. Normalized area of the selected inverse angle equals the positive input.

$a(\theta(X)) = X$.

```
lemma area_00f {x :  $\mathbb{R}$ } (hx :  $0 < x$ ) : area (00f x) = x := by
  rw [00f, dif_pos hx]
  exact (area_surjOn (mem_Ioi.mpr hx)).choose_spec.2

/-- Uniqueness: `00f` really is the inverse on ` $(0, \pi)$ `. -/
```

cmv_comparison_bound_optimized

Classification: main

Source: Certificate.lean:746-753

Declaration SHA-256: 507439948a37eb806f48ea9934d720941448689e7e9b24960cacb11d308a3d9b

Mathematical statement. For lambda at least 1.2581840884 and $X > 0$, $1/\text{lambda}$ is strictly less than $g(X)$.

The global minimum bound for g and an exact rational trigonometric certificate give a strict, uniform comparison.

```
theorem cmv_comparison_bound_optimized {lam x :  $\mathbb{R}$ }
  (hlam :  $(1.2581840884 : \mathbb{R}) \leq \text{lam}$ )
  (hx :  $0 < x$ ) :
   $1 / \text{lam} < g\ x$  := by
  have hinv :  $1 / \text{lam} \leq 1 / (1.2581840884 : \mathbb{R})$  :=
    one_div_le_one_div_of_le (by norm_num) hlam
  exact lt_of_le_of_lt hinv
    (lt_trans opt_inv_candidate_lt_three_cos_witness (opt_min_g_gt_witness hx))
```

cmv_type_four_not_minimizing

Classification: main

Source: CandidateExclusion.lean:18-37

Declaration SHA-256: 6d0ea3949f7dbdcb13da7efae9522f57a17102179e82571502380037275e9220

Mathematical statement. Candidates satisfying the formal CMV hypotheses are nonminimal for λ at least 1.2581840884.

The exact rational threshold makes the cap-replacement perimeter difference strictly negative while preserving area:
 $\mathcal{P}_{\text{rep}}(\mathcal{P}_{\text{candidate}}) - c \cdot \text{bigl}(1 - \lambda \cdot g(X) \cdot \text{bigr}) < 0$.

```
theorem cmv_type_four_not_minimizing {lam : ℝ}
  (hlam : (1.2581840884 : ℝ) ≤ lam)
  (candidate : FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses)
  (hmin : candidate.IsWeightedPerimeterMinimizer) :
  False := by
  have hsourceDomain : 1 < lam := hcandidate.density_jump
  have harea : candidate.capReplacementRegion.WeightedArea =
    candidate.WeightedArea :=
    candidate.cap_replacement_preserves_area
  have hminimal : candidate.WeightedPerimeter ≤
    candidate.capReplacementRegion.WeightedPerimeter :=
    hmin candidate.capReplacementRegion harea
  have himproves : candidate.capReplacementRegion.WeightedPerimeter <
    candidate.WeightedPerimeter :=
    candidate.cap_replacement_strictly_improves hlam
  exact (not_lt_of_ge hminimal) himproves

/-- A modeled CMV type-(iv) weighted-perimeter minimizer can occur only below
the certified improved cutoff. -/
```

ell

Classification: main

Source: MorgansArcFunction.lean:89-91

Declaration SHA-256: 9adb32c6bf9324acec1e791c0ec46f0e17df835ffee7b3549cab07bdc3a760c9

Mathematical statement. Arc length of a unit-chord circular segment as a function of half-angle.

$\ell(\theta) = \theta / \sin \theta$ on $0 < \theta < \pi$.

```
noncomputable def ell (θ : ℝ) : ℝ := θ / sin θ

/-- Area enclosed by a unit-chord circular segment with half-central-angle `θ`. -/
```

exact_euclidean_capParam_hausdorffMeasure

Classification: main

Source: FrontierPerimeter.lean:457-483

Declaration SHA-256: ce3a2c8ff8b380f7da5859563d117de2848052560e457ef5f88e25aa88808f43

Mathematical statement. The Euclidean image of the full cap parameter interval has the cap's stated arc length.

The cap has radius $R > 0$ and half-angle $0 < \theta < \pi$. Its realized parametrization is an isometric placement of u_R on $[-R\theta, R\theta]$, so the radius-scaled arc theorem gives length $2R\theta$, equal to the cap's stated arc length.

```
theorem exact_euclidean_capParam_hausdorffMeasure (c : OneSidedCircularCap) :
  (μH[1] : Measure EuclideanPlane)
  (realizedCapParam c '' Icc (capStart c) (capEnd c)) =
```

```

    ENNReal.ofReal c.arcLength := by
have hfun : realizedCapParam c = capComplexIsometryMap c ∘ unitCircleArc c.radius := by
  funext s
  exact realizedCapParam_eq c s
calc
  (μH[1] : Measure EuclideanPlane)
  (realizedCapParam c '' Icc (capStart c) (capEnd c)) =
  (μH[1] : Measure EuclideanPlane)
  (capComplexIsometryMap c ''
    (unitCircleArc c.radius '' Icc (capStart c) (capEnd c))) := by
    rw [hfun]
    congr 1
    simp only [Function.comp_apply] using (Set.image_image _ _ _).symm
_ = (μH[1] : Measure ℂ)
  (unitCircleArc c.radius '' Icc (capStart c) (capEnd c)) :=
(isometry_capComplexIsometryMap c).hausdorffMeasure_image
  (Or.inl (by norm_num)) _
_ = ENNReal.ofReal (2 * c.radius * c.theta) := by
  simp [capStart, capEnd] using
    hausdorffMeasure_unitCircleArc_image_eq c.radius_pos
    ⟨c.theta_pos, c.theta_lt_pi⟩
_ = ENNReal.ofReal c.arcLength := by rw [cap_arcLength_eq_two_radius_theta]

```

exact_euclidean_capTrace_hausdorffMeasure

Classification: main

Source: FrontierPerimeter.lean:1081-1088

Declaration SHA-256: 8a0602f46d010a0d38a019d5e1052cfe4c5d69fc3db5988eaac8c5dbcc5c5e1c

Mathematical statement. The actual Euclidean cap trace has length $2R\theta$.

This is the geometric version of the normalized arc-length formula.

```

theorem exact_euclidean_capTrace_hausdorffMeasure
  (c : OneSidedCircularCap) :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' c.arcTrace) =
  ENNReal.ofReal c.arcLength := by
  rw [← realizedCapParam_image_Icc]
  exact exact_euclidean_capParam_hausdorffMeasure c

```

exact_euclidean_leftArcTrace_hausdorffMeasure

Classification: main

Source: FrontierPerimeter.lean:821-827

Declaration SHA-256: f6b756b038d2dd53ff9e214034534494d6216a6823376bb73c389a754a1ad0d9

Mathematical statement. The actual Euclidean left core trace has length $2R \arcsin(h)$.

The parameter-image equality and exact image measure establish the geometric length.

```

theorem exact_euclidean_leftArcTrace_hausdorffMeasure (c : StripCore) :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' c.leftArcTrace) =
  ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  rw [← realizedLeftCoreParam_image_Icc]
  exact exact_euclidean_leftCoreParam_hausdorffMeasure c

```

exact_euclidean_rightArcTrace_hausdorffMeasure

Classification: main

Source: FrontierPerimeter.lean:828-834

Declaration SHA-256: 47e43c0cab0cb015904a2fe672eeec0720a25537d058b25dd2b767b05bcf1d39

Mathematical statement. The actual Euclidean right core trace has length $2R \arcsin(h)$.

The two side lengths sum to $4\arcsin(h)/h$.

```
theorem exact_euclidean_rightArcTrace_hausdorffMeasure (c : StripCore) :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' c.rightArcTrace) =
  ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  rw [← realizedRightCoreParam_image_Icc]
  exact exact_euclidean_rightCoreParam_hausdorffMeasure c
```

exact_euclidean_segmentCarrier_hausdorffMeasure

Classification: main

Source: FrontierPerimeter.lean:937-946

Declaration SHA-256: f2e6ec79942c7df8c258fab2922138cb782506dbaa0184c81a94b12e6e9b9896

Mathematical statement. The Euclidean segment carrier has Hausdorff length equal to its length parameter.

This supplies the exact cost of an exposed horizontal chord.

```
theorem exact_euclidean_segmentCarrier_hausdorffMeasure
  (seg : HorizontalSegment) :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' seg.carrier) =
  ENNReal.ofReal seg.euclideanLength := by
  rw [← realizedSegmentParam_image_carrier,
    realizedSegmentParam_image_affineSegment,
    hausdorffMeasure_affineSegment,
    segmentEndpoint_edist]
```

exists_F_root

Classification: main

Source: Minimiser.lean:82-91

Declaration SHA-256: 95a62157307a9e9688c52df86b638986f0064730d81ed190bb31889a2f44b580

Mathematical statement. F has a root between zero and $\pi/2$.

Continuity and the opposite endpoint signs give existence; strict increase gives uniqueness.

```
lemma exists_F_root : ∃ θ ∈ Icc 0 (π / 2), F θ = 0 := by
  have hab : (0 : ℝ) ≤ π / 2 := by linarith [pi_pos]
  have hcont : ContinuousOn F (Icc 0 (π / 2)) := F_continuous.continuousOn
  have hmem : (0 : ℝ) ∈ Icc (F 0) (F (π / 2)) := by
    rw [F_zero, F_pi_div_two]
    exact ⟨by linarith [pi_pos], by linarith [pi_pos]⟩
  obtain ⟨θ, hθ, hval⟩ := intermediate_value_Icc hab hcont hmem
  exact ⟨θ, hθ, hval⟩

/-- The root of `F` in `(0, π / 2)`. -/
```

fourArc_euclidean_frontier

Classification: main

Source: FrontierPerimeter.lean:1338-1348

Declaration SHA-256: 748ff0ec3e12377aa952eabaeafb59718df5bc069c8651bdcd45a9c687a1d41d

Mathematical statement. The Euclidean four-arc carrier frontier is the union of the four Euclidean traces.

Homeomorphisms preserve frontiers, transporting the coordinate frontier identity.

```
theorem fourArc_euclidean_frontier (a : FourArcAssembly) :
  frontier (planeEuclideanHomeomorph '' a.carrier) =
    (planeEuclideanHomeomorph '' StripCore.leftArcTrace a.core) ∪
    ((planeEuclideanHomeomorph '' StripCore.rightArcTrace a.core) ∪
     (planeEuclideanHomeomorph ''
       OneSidedCircularCap.arcTrace a.upperCap) ∪
     (planeEuclideanHomeomorph ''
       OneSidedCircularCap.arcTrace a.lowerCap))) := by
  rw [← planeEuclideanHomeomorph.image_frontier, a.frontier_carrier]
  simp only [FourArcAssembly.boundaryTrace, image_union]
```

fourArc_frontier_weightedPerimeter_eq

Classification: main

Source: FrontierPerimeter.lean:1794-1844

Declaration SHA-256: e95ee1864c7bc7d9bc3a3ec2659625a510c6cefccdafe0cd2efab0e2d93e1067

Mathematical statement. The component four-arc perimeter equals the canonical complete-frontier perimeter.

Exact frontier decomposition, Euclidean trace lengths, density values, and null junction overlaps prove the identification.

```
theorem fourArc_frontier_weightedPerimeter_eq
  (lam : ℝ) (assembly : FourArcAssembly) :
  assembly.weightedPerimeter lam =
    WeightedPerimeter lam (FrontierMeasure assembly.carrier) := by
  rcases fourArc_pairwise assembly with
    ⟨hAB, hAC, hAD, hBC, hBD, hCD⟩
  have hAint := euclideanStripDensity_integrable0n_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph ''
      StripCore.leftArcTrace assembly.core) (by
      rw [exact_euclidean_leftArcTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
  have hBint := euclideanStripDensity_integrable0n_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph ''
      StripCore.rightArcTrace assembly.core) (by
      rw [exact_euclidean_rightArcTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
  have hCint := euclideanStripDensity_integrable0n_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph ''
      OneSidedCircularCap.arcTrace assembly.upperCap) (by
      rw [exact_euclidean_capTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
  have hDint := euclideanStripDensity_integrable0n_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph ''
      OneSidedCircularCap.arcTrace assembly.lowerCap) (by
      rw [exact_euclidean_capTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
  rw [FourArcAssembly.weightedPerimeter_formula]
  symm
  unfold WeightedPerimeter FrontierMeasure
  rw [integral_map
    planeEuclideanHomeomorph.symm.continuous.measurable.aemeasurable
    (measurable_stripDensity lam).astronglyMeasurable]
  change (∫ z in frontier (planeEuclideanHomeomorph '' assembly.carrier),
    euclideanStripDensity lam z
    ∂(μH[1] : Measure EuclideanPlane)) =
```



```

    assembly.core.boundaryArcLength +
      lam * (assembly.upperCap.arcLength + assembly.lowerCap.arcLength)
rw [fourArc_euclidean_frontier]
rw [integral_four_union hAB hAC hAD hBC hBD hCD
    (measurableSet_euclidean_rightArcTrace assembly.core)
    (measurableSet_euclidean_capTrace assembly.upperCap)
    (measurableSet_euclidean_capTrace assembly.lowerCap)
    hAint hBint hCint hDint]
rw [left_integral, right_integral,
    upper_cap_integral lam assembly.upperCap rfl rfl,
    lower_cap_integral lam assembly.lowerCap rfl rfl,
    core_param_length, StripCore.boundaryArcLength]
ring

/-- The explicit replacement boundary cost equals the canonical weighted  $H^1$ 
integral on the complete topological frontier of its coordinate carrier. -/

```

fourArc_pairwiseOverlap_h1_null

Classification: main

Source: FrontierPerimeter.lean:1269-1279

Declaration SHA-256: 01eb58d468da4075ed5a4b3c4808bf8deddca98be0f05ef720c74c5a64fd9f4f

Mathematical statement. Four-arc boundary component overlaps have zero Hausdorff length.

Thus adding the four component integrals does not double-count positive perimeter.

```

theorem fourArc_pairwiseOverlap_h1_null (a : FourArcAssembly) :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' pairwiseOverlap
      (StripCore.leftArcTrace a.core)
      (StripCore.rightArcTrace a.core)
      (OneSidedCircularCap.arcTrace a.upperCap)
      (OneSidedCircularCap.arcTrace a.lowerCap)) = 0 := by
exact measure_mono_null
  (image_mono (fourArc_pairwiseOverlap_subset_joinSet a))
  (hausdorffMeasure_one_euclideanJoinSet a.core.chord)

```

g

Classification: main

Source: GFunction.lean:115-118

Declaration SHA-256: 84e86ef24e01fc8948ba482190efcf999cb2f81449ac1865f6de67d08ad6760b

Mathematical statement. The normalized arc saving from combining two equal-area caps.

$g(X) = 2L(X) - L(2X)$ for positive normalized cap area X .

```

noncomputable def g (x : ℝ) : ℝ := 2 * arc x - arc (2 * x)

/-- The chain rule on `arc (2 * x)` contributes a factor of `2`, so both terms
carry a `4`. -/

```

g_deriv_neg

Classification: main

Source: GFunction.lean:156-166

Declaration SHA-256: 89dcd5f4ccfab31e9736dc2f8394ba49f8c59134f8b039132ae14f9404c9bd9

Mathematical statement. The derivative of g is negative in the small-area branch.

When both inverse angles lie below the semicircle threshold, their sine order makes $g'(X) < 0$.

```

theorem g_deriv_neg {x : ℝ} (hx : 0 < x) (h : x ≤ π / 16) :
  4 * sin (00f x) - 4 * sin (00f (2 * x)) < 0 := by
  have h2x : (0:ℝ) < 2 * x := by linarith
  have h1e : 2 * x ≤ π / 8 := by linarith
  have h1t : 00f x < 00f (2 * x) := 00f_lt_00f_two_mul hx
  have hup : 00f (2 * x) ≤ π / 2 := 00f_le_pi_div_two h2x h1e
  have hlow : 0 ≤ 00f x := le_of_lt (00f_mem hx).1
  have := sin_lt_sin_of_le_pi_div_two hlow h1t hup
  linarith

/-- On  $[\pi / 8, \infty)$  the derivative of `g` is positive. -/

```

g_deriv_pos

Classification: main

Source: GFunction.lean:167-193

Declaration SHA-256: 4d597413573bb96cb2e13a079780c61d7a55d50fcebe5b68b68cbf58a0cbfa39

Mathematical statement. The derivative of g is positive in the large-area branch.

When the inverse angles lie on the descending sine branch, the larger angle has smaller sine.

```

theorem g_deriv_pos {x : ℝ} (h : π / 8 ≤ x) :
  0 < 4 * sin (00f x) - 4 * sin (00f (2 * x)) := by
  have hx : (0:ℝ) < x := lt_of_lt_of_le pi_div_eight_pos h
  have h2x : (0:ℝ) < 2 * x := by linarith
  have h1t : 00f x < 00f (2 * x) := 00f_lt_00f_two_mul hx
  have hlow : π / 2 ≤ 00f x := pi_div_two_le_00f h
  have hup : 00f (2 * x) ≤ π := le_of_lt (00f_mem h2x).2
  have := sin_lt_sin_of_pi_div_two_le hlow h1t hup
  linarith

/-! ## What remains

The minimiser `x*` lies in  $(\pi / 16, \pi / 8)$ . There  $00f\ x^* < \pi / 2 < 00f\ (2x^*)$ , so
`sin (00f x)` is increasing in `x` while `sin (00f (2x))` is decreasing, making
`g` strictly increasing on that interval – hence a unique zero, and a unique
minimum. At that point  $\sin (00f\ x^*) = \sin (00f\ (2x^*))$  with the two angles
distinct, forcing

 $00f\ x^* + 00f\ (2x^*) = \pi$ ,

which combined with  $\text{area} (\pi - \theta) = 2 * \text{area } \theta$  reduces to the single equation

 $3 (\theta^* - \sin \theta^* \cos \theta^*) = \pi$ ,

and then  $k = g\ x^* = 3 \cos \theta^*$ . Numerically  $\theta^* = 1.30266\dots$ ,  $k = 0.79479\dots$ ,
 $1/k = 1.25818\dots$ , against CMV's published  $4/\pi = 1.27324\dots$ .
- /

```

g_deriv_xstar

Classification: main

Source: Minimiser.lean:173-182

Declaration SHA-256: 1eebac5648246e00a782c35d06dac92c6d57fafb232dc20b208113c4fcc2af00

Mathematical statement. The derivative of g vanishes at Xstar.

The two inverse angles are supplementary and therefore have equal sine in the derivative formula.

```

theorem g_deriv_xstar : 4 * sin (00f xstar) - 4 * sin (00f (2 * xstar)) = 0 := by
  rw [00f_xstar, 00f_two_mul_xstar, sin_pi_sub]
  ring

/-! ## The value of `g` at the critical point -/

```

```

/-- **The closed form.** `g x* = 3 cos θ*`.

`g x* = 2 ell θ* - ell (π - θ*) = (2θ* - (π - θ*)) / sin θ* = (3θ* - π) / sin θ*`,
and `F θ* = 0` gives `3θ* - π = 3 sin θ* cos θ*`. -/

```

g_strictAntiOn

Classification: main

Source: MinimumValue.lean:110-119

Declaration SHA-256: 9a778ff550b85fbf2a285e9d241dc6696a5778402499a6962e2ebab67dde9834

Mathematical statement. The cap-saving function strictly decreases up to X_{star} .

The negative derivative on the positive interval before X_* gives strict antitonicity.

```

lemma g_strictAntiOn : StrictAntiOn g (Ioc 0 xstar) := by
  apply strictAntiOn_of_hasDerivWithinAt_neg (convex_Ioc 0 xstar)
  · exact fun x hx => (hasDerivAt_g' hx.1).continuousAt.continuousWithinAt
  · intro x hx
    rw [interior_Ioc] at hx
    exact (hasDerivAt_g' hx.1).hasDerivWithinAt
  · intro x hx
    rw [interior_Ioc] at hx
    exact gd_neg_of_lt_xstar hx.1 hx.2

```

g_strictMonoOn_Ici

Classification: main

Source: MinimumValue.lean:120-131

Declaration SHA-256: bbc00bca790c91cdaec1fe6d2fa43a3abe6e68cd5e0920d9ebd378f169cf5602

Mathematical statement. The cap-saving function strictly increases from X_{star} onward.

The derivative is positive to the right of the stationary point.

```

lemma g_strictMonoOn_Ici : StrictMonoOn g (Ici xstar) := by
  apply strictMonoOn_of_hasDerivWithinAt_pos (convex_Ici xstar)
  · intro x hx
    exact (hasDerivAt_g' (lt_of_lt_of_le xstar_pos hx)).continuousAt.continuousWithinAt
  · intro x hx
    rw [interior_Ici] at hx
    exact (hasDerivAt_g' (lt_trans xstar_pos hx)).hasDerivWithinAt
  · intro x hx
    rw [interior_Ici] at hx
    exact gd_pos_of_xstar_lt hx

/-- **`g` attains its minimum over `(0, ∞)` at `x*`.** -/

```

g_xstar_le

Classification: main

Source: MinimumValue.lean:132-147

Declaration SHA-256: b1c43512db9300ddb51ff2adbde05838f6b0dd21a49572dbdd5454f8a1f394a2

Mathematical statement. X_{star} is a global minimizer of g on positive normalized areas.

Decrease before and increase after X_* imply $g(X_*) \leq g(X)$ for every $X > 0$.

```

theorem g_xstar_le {x : ℝ} (hx : 0 < x) : g xstar ≤ g x := by
  rcases le_or_gt x xstar with hle | hgt
  · rcases eq_or_lt_of_le hle with heq | hlt

```

```

· rw [heq]
· exact le_of_lt (g_strictAntiOn ⟨hx, hle⟩ ⟨xstar_pos, le_refl _⟩ hlt)
· exact le_of_lt (g_strictMonoOn_Ici (mem_Ici.mpr (le_refl xstar))
  (mem_Ici.mpr (le_of_lt hgt)) hgt)

/-- **The constant of CMV Remark 3.17, in closed form.**

For every `x > 0`, `2 arc x - arc (2 x) ≥ 3 cos θ*`, where `θ*` is the unique
root of `3 θ - 3 sin θ cos θ = π`. The bound is attained at `x* = area θ*`,
so it is sharp.

Numerically `θ* = 1.30266283730...` and `3 cos θ* = 0.79479625381...`, against
CMV's published lower bound `π / 4 = 0.78539816340...`. -/

```

g_θstar

Classification: main

Source: Minimiser.lean:183-206

Declaration SHA-256: b7fb48fb7e90dd4fccd7f3be6e957eefae58ef3ae08e86762f2e1b503b38585a

Mathematical statement. The saving at Xstar equals three times cosine of the root angle.

Substitute the supplementary-angle identities into $2L(X^*) - L(2X^*)$ and use the root equation.

```

theorem g_θstar : g xstar = 3 * cos θstar := by
  have hs : sin θstar ≠ 0 := ne_of_gt sin_θstar_pos
  have harc1 : arc xstar = θstar / sin θstar := by
    rw [arc_eq, θ0f_xstar, ell]
  have harc2 : arc (2 * xstar) = (π - θstar) / sin θstar := by
    rw [arc_eq, θ0f_two_mul_xstar, ell, sin_pi_sub]
  have hpi : 3 * θstar - π = 3 * (sin θstar * cos θstar) := by
    have := θstar_spec; rw [F] at this; linarith
  rw [g, harc1, harc2]
  field_simp
  linarith [hpi]

/-! ## What remains

To conclude that `k := g x*` is the *minimum* of `g`, and hence that CMV's
cap-substitution argument succeeds for all `lam > 1 / k`, one still needs:

* `g` strictly increasing on `(π / 16, π / 8)` – because `sin (θ0f x)` rises
  while `sin (θ0f (2 * x))` falls there – giving uniqueness of the critical
  point, and with `GFunction`'s sign results, that it is the global minimum;
* a rational certificate: `F` is strictly increasing, so exhibiting `t₀` with
  `F t₀ > 0` gives `θ* < t₀` and hence `k = 3 cos θ* > 3 cos t₀`. With
  `t₀ = 1.3027` this yields `k > 0.7946` and the threshold `lam ≥ 1.2584`.
-/

```

gd_neg_of_lt_xstar

Classification: main

Source: MinimumValue.lean:90-98

Declaration SHA-256: f2df0e64632660e264e46b7db88395768f82d48e3e7f2345a8a2f9c75cc667c9

Mathematical statement. The derivative of g is negative before Xstar.

Combine the small-area branch sign with mixed-interval monotonicity and $g_d(X^*) = 0$.

```

lemma gd_neg_of_lt_xstar {x : ℝ} (hx : 0 < x) (h : x < xstar) : gd x < 0 := by
  rcases le_or_gt x (π / 16) with hle | hgt
  · have := g_deriv_neg hx hle
    rwa [- gd] at this
  · have hmem : x ∈ Ioo (π / 16) (π / 8) :=
    ⟨hgt, lt_trans h xstar_lt_pi_div_eight⟩

```

```
have := gd_strictMonoOn hmem xstar_mem_Ioo h
rwa [gd_xstar] at this
```

gd_pos_of_xstar_lt

Classification: main

Source: MinimumValue.lean:99-109

Declaration SHA-256: aa9cc07829ae7f90ff1904fd8a3bd95114f930be277c8de2503c494dfa26a0dc

Mathematical statement. The derivative of g is positive after X_{star} .

Mixed-interval monotonicity and the large-area branch sign cover every $X > X_*$.

```
lemma gd_pos_of_xstar_lt {x : ℝ} (h : xstar < x) : 0 < gd x := by
  rcases le_or_gt (π / 8) x with hle | hgt
  · have := g_deriv_pos hle
    rwa [- gd] at this
  · have hmem : x ∈ Ioo (π / 16) (π / 8) :=
    (lt_trans pi_div_sixteen_lt_xstar h, hgt)
    have := gd_strictMonoOn xstar_mem_Ioo hmem h
    rwa [gd_xstar] at this

/-! ## Step 4: `g` has its global minimum at `x*` -/
```

gd_strictMonoOn

Classification: main

Source: MinimumValue.lean:69-89

Declaration SHA-256: 882d961320b58e7e7b5a68f1c9b6da8b8cd9f145ae2152f7eeda7c84659a8f5c

Mathematical statement. The derivative function gd is strictly increasing across the mixed minor/major interval.

The first sine increases while the doubled-angle sine decreases, so their difference strictly increases.

```
lemma gd_strictMonoOn : StrictMonoOn gd (Ioo (π / 16) (π / 8)) := by
  intro a ha b hb hab
  have ha0 : 0 < a := lt_trans (by linarith [pi_pos]) ha.1
  have hb0 : 0 < b := lt_trans (by linarith [pi_pos]) hb.1
  have ha2 : (0:ℝ) < 2 * a := by linarith
  have hb2 : (0:ℝ) < 2 * b := by linarith
  -- the small angles rise
  have h1 : sin (θ0f a) < sin (θ0f b) := by
    refine sin_lt_sin_of_le_pi_div_two (le_of_lt (θ0f_mem ha0).1) ?_ ?_
    · exact θ0f_strictMonoOn (mem_Ioi.mpr ha0) (mem_Ioi.mpr hb0) hab
    · exact le_of_lt (θ0f_lt_pi_div_two_of_mem hb)
  -- the large angles fall
  have h2 : sin (θ0f (2 * b)) < sin (θ0f (2 * a)) := by
    refine sin_lt_sin_of_pi_div_two_le (pi_div_two_le_θ0f_two_mul ha) ?_ ?_
    · exact θ0f_strictMonoOn (mem_Ioi.mpr ha2) (mem_Ioi.mpr hb2) (by linarith)
    · exact le_of_lt (θ0f_mem hb2).2
  rw [gd, gd]
  linarith

/-! ## Step 3: the sign of `gd` on either side of `x*` -/
```

hasDerivAt_F

Classification: main

Source: Minimiser.lean:46-56

Declaration SHA-256: dcdd81a18d4f4a1ae529be33b11aab6304bb984338d093081cc21b62e19958fa

Mathematical statement. For every real θ , F has derivative $6 \sin(\theta)^2$.

For all $\theta \in \mathbb{R}$, $F'(\theta) = 6\sin^2\theta$. The derivative is strictly positive on $(0, \pi)$ and vanishes at 0 and π ; interior positivity is sufficient for strict increase on the closed interval.

```
lemma hasDerivAt_F (θ : ℝ) : HasDerivAt F (6 * sin θ ^ 2) θ := by
  have h1 : HasDerivAt (fun t : ℝ => 3 * t) 3 θ := by
    simp using (hasDerivAt_id θ).const_mul (3 : ℝ)
  have h2 : HasDerivAt (fun t : ℝ => sin t * cos t)
    (cos θ * cos θ + sin θ * -sin θ) θ :=
    (Real.hasDerivAt_sin θ).mul (Real.hasDerivAt_cos θ)
  have h3 := h1.sub (h2.const_mul (3 : ℝ))
  have h4 := h3.sub_const π
  refine h4.congr_deriv ?_
  linear_combination (-3 : ℝ) * sin_sq_add_cos_sq θ
```

hasDerivAt_arc

Classification: main

Source: MorgansArcFunction.lean:312-333

Declaration SHA-256: eb72e29cac89c3bb9db09a43c638398f57ad6c7fbce7c36f45c192d4285ebfc2

Mathematical statement. The arc-area function has derivative twice sine of the inverse angle.

$L'(X) = \ell'(\theta(X))/a'(\theta(X)) = 2\sin\theta(X)$ for $X > 0$.

```
theorem hasDerivAt_arc {x : ℝ} (hx : 0 < x) :
  HasDerivAt arc (2 * sin (θof x)) x := by
  have hθ : θof x ∈ Ioo 0 π := θof_mem hx
  have hs : (0:ℝ) < sin (θof x) := sin_pos_of_pos_of_lt_pi hθ.1 hθ.2
  have hk : 0 < sin (θof x) - θof x * cos (θof x) := sin_sub_mul_cos_pos hθ.1 hθ.2
  set A := (sin (θof x) - θof x * cos (θof x)) / (2 * sin (θof x) ^ 3) with hAdef
  have hAne : A ≠ 0 := by rw [hAdef]; positivity
  have hfwd : HasDerivAt area A (θof x) := hasDerivAt_area hθ.1 hθ.2
  have hloc : ∀ y in ℳ x, area (θof y) = y := by
    filter_upwards [Ioi_mem_nhds hx] with y hy
    exact area_θof hy
  have hθderiv : HasDerivAt θof A⁻¹ x :=
    HasDerivAt.of_local_left_inverse (f := area) (θof_continuousAt hx) hfwd hAne hloc
  have hchain := (hasDerivAt_ell hθ.1 hθ.2).comp x hθderiv
  have hc : (sin (θof x) - θof x * cos (θof x)) / sin (θof x) ^ 2 * A⁻¹
    = 2 * sin (θof x) := by
    rw [hAdef]
  have h1 : sin (θof x) ≠ 0 := ne_of_gt hs
  have h2 : sin (θof x) - θof x * cos (θof x) ≠ 0 := ne_of_gt hk
  field_simp
  rw [hc] at hchain
  exact hchain
```

hasDerivAt_area

Classification: main

Source: MorgansArcFunction.lean:104-124

Declaration SHA-256: 77cb2c9579c9e12eb25a7bd63dc9a791629029d7c8ad37f065713a7afa870b98

Mathematical statement. Derivative of normalized cap area.

$a'(\theta) = (\sin\theta - \theta\cos\theta)/(2\sin^3\theta)$ on $(0, \pi)$.

```
lemma hasDerivAt_area {θ : ℝ} (hθ : 0 < θ) (hπ : θ < π) :
  HasDerivAt area ((sin θ - θ * cos θ) / (2 * sin θ ^ 3)) θ := by
  have hs : (0:ℝ) < sin θ := sin_pos_of_pos_of_lt_pi hθ hπ
  have hs' : sin θ ≠ 0 := ne_of_gt hs
  have hd : (4 : ℝ) * sin θ ^ 2 ≠ 0 := by positivity
  have hnum : HasDerivAt (fun t => t - sin t * cos t)
    (1 - (cos θ * cos θ + sin θ * -sin θ)) θ :=
```

```

    (hasDerivAt_id θ).sub ((Real.hasDerivAt_sin θ).mul (Real.hasDerivAt_cos θ))
  have hden : HasDerivAt (fun t => 4 * sin t ^ 2) (4 * (2 * sin θ * cos θ)) θ := by
    have := ((Real.hasDerivAt_sin θ).pow 2)
    simp using this.const_mul 4
  have h := hnum.div hden hd
  have harea : area = ((fun t => t - sin t * cos t) / fun t : ℝ => 4 * sin t ^ 2) := rfl
  rw [harea]
  refine h.congr_deriv ?_
  have hcos : cos θ ^ 2 = 1 - sin θ ^ 2 := by nlinarith [sin_sq_add_cos_sq θ]
  field_simp
  ring_nf
  rw [hcos]
  ring

```

hasDerivAt_g

Classification: main

Source: GFunction.lean:130-150

Declaration SHA-256: decc0673d8052a837dfd48ad4a92fc973c064c1bdb19f47382d7d73801df7cf

Mathematical statement. The derivative of the normalized cap-saving function.

$$g'(X) = 4\sin\theta(X) - 4\sin\theta(2X).$$

```

theorem hasDerivAt_g {x : ℝ} (hx : 0 < x) :
  HasDerivAt g (4 * sin (θ0f x) - 4 * sin (θ0f (2 * x))) x := by
  have h1 : HasDerivAt (fun t => 2 * arc t) (2 * (2 * sin (θ0f x))) x :=
    (hasDerivAt_arc hx).const_mul (2:ℝ)
  have h2 := hasDerivAt_arc_two_mul hx
  refine (h1.sub h2).congr_deriv ?_
  ring
/-! ## The sign of `g` away from `(π / 16, π / 8)`

Since `x < 2x`, monotonicity of `θ0f` gives `θ0f x < θ0f (2x)` always. The sign
of `g` is then decided by where those two angles sit relative to `π / 2`.

* If `2x ≤ π / 8` both angles are at most `π / 2`, sine is increasing there, so
`sin (θ0f x) < sin (θ0f (2x))` and `g' < 0`.
* If `π / 8 ≤ x` both angles are at least `π / 2`, sine is decreasing there, so
`sin (θ0f (2x)) < sin (θ0f x)` and `g' > 0`.

Between `π / 16` and `π / 8` the two angles straddle `π / 2` and the comparison is
genuinely delicate – that is where the minimum lies.
-/

```

hausdorffMeasure_unitCircleArc_image_eq

Classification: main

Source: FrontierPerimeter.lean:294-311

Declaration SHA-256: f4746d8c9b2c6ef1416bccca1f814590cdef08c393e10b7189ff167eb0402ab3

Mathematical statement. For $r > 0$ and $0 < \theta < \pi$, the image $u_r([-r\theta, r\theta])$ has Hausdorff length $2r\theta$.

The exact statement is

$$\mathcal{H}^1(u_r([-r\theta, r\theta])) = \text{ofReal}(2r\theta).$$

The 1-Lipschitz upper bound is the signed-arclength span $2r\theta$, and the inscribed-polygon lower bounds converge to the same value. The angular span is 2θ ; the length includes the radius factor r .

```

theorem hausdorffMeasure_unitCircleArc_image_eq {r theta : ℝ} (hr : 0 < r)
  (htheta : theta ∈ Ioo 0 Real.pi) :
  (μH[1] : Measure ℂ)
    (unitCircleArc r '' Icc (-r * theta) (r * theta)) =

```

```

    ENNReal.ofReal (2 * r * theta) := by
  apply le_antisymm
  · calc
    (μH[1] : Measure ℂ)
      (unitCircleArc r '' Icc (-r * theta) (r * theta)) ≤
      ENNReal.ofReal (r * theta - (-r * theta)) :=
      hausdorffMeasure_unitCircleArc_image_le hr
    _ = ENNReal.ofReal (2 * r * theta) := by congr 1 <|> ring
  · apply le_of_tendsto'
    (ENNReal.tendsto_ofReal (tendsto_cap_polygon_length (r := r) htheta.1))
  intro n
  exact cap_partition_polygon_lower hr htheta (n + 1) (Nat.zero_lt_succ n)

/-- Rotation of the standard complex circle by the signed-arclength phase `m`. -/

```

hausdorffMeasure_unitCircleArc_image_eq_of_span_lt_two_pi

Classification: main

Source: FrontierPerimeter.lean:350-389

Declaration SHA-256: a5a68294ffad2a36064599c08079161b28ea03171c66437e00e8f24eba632c26

Mathematical statement. For $r > 0$, $u < v$ and $v - u < 2\pi r$, the circle image $u_r([u, v])$ has Hausdorff length $v - u$.

For signed-arclength endpoints satisfying $0 < v - u < 2\pi r$,

$$\mathcal{H}^1(u_r([u, v])) = \text{ofReal}(v - u).$$

Set $\theta = (v - u)/(2r) \in (0, \pi)$ and $m = (u + v)/2$. Rotation by m/r takes the symmetric interval image to this arc and preserves Hausdorff measure. The no-full-turn condition is $v - u < 2\pi r$, not $v - u < 2\pi$ for an arbitrary radius.

```

theorem hausdorffMeasure_unitCircleArc_image_eq_of_span_lt_two_pi
  {r u v : ℝ} (hr : 0 < r) (huv : u < v)
  (hspan : v - u < 2 * Real.pi * r) :
  (μH[1] : Measure ℂ) (unitCircleArc r '' Icc u v) =
    ENNReal.ofReal (v - u) := by
  let theta : ℝ := (v - u) / (2 * r)
  let m : ℝ := (u + v) / 2
  have htheta : theta ∈ Ioo 0 Real.pi := by
    dsimp [theta]
    constructor
  · exact div_pos (sub_pos.mpr huv) (mul_pos (by norm_num) hr)
  · apply (div_lt_iff₀ (mul_pos (by norm_num) hr)).2
    nlinarith [Real.pi_pos]
  have hleft : -r * theta + m = u := by
    dsimp [theta, m]
    field_simp [hr.ne']
    ring
  have hright : r * theta + m = v := by
    dsimp [theta, m]
    field_simp [hr.ne']
    ring
  calc
    (μH[1] : Measure ℂ) (unitCircleArc r '' Icc u v) =
      (μH[1] : Measure ℂ)
        (unitCircleArcRotation r m ''
          (unitCircleArc r '' Icc (-r * theta) (r * theta))) := by
        rw [unitCircleArcRotation_image_Icc hr.ne', hleft, hright]
    _ = (μH[1] : Measure ℂ)
        (unitCircleArc r '' Icc (-r * theta) (r * theta)) :=
        (isometry_unitCircleArcRotation r m).hausdorffMeasure_image
        (Or.inl (by norm_num)) _
    _ = ENNReal.ofReal (2 * r * theta) :=
        hausdorffMeasure_unitCircleArc_image_eq hr htheta
    _ = ENNReal.ofReal (v - u) := by
      congr 1
      dsimp [theta]
      field_simp [hr.ne']

```



```
-- Translation and possible reflection carrying the standard complex circle to a cap. -/
```

min_g_eq_three_mul_cos_θstar

Classification: main

Source: MinimumValue.lean:148-153

Declaration SHA-256: 76e47f07f4c6982bfe46b74b225bb1e09621a21e59136f8d33d5ed15a7efc3b0

Mathematical statement. Three times cosine of the root angle is a global lower bound for g .

For every $X > 0$, $3\cos\theta_* \leq g(X)$; the separate value identity shows equality at X_* .

```
theorem min_g_eq_three_mul_cos_θstar {x : ℝ} (hx : 0 < x) :
  3 * cos θstar ≤ g x := by
  rw [← g_θstar]
  exact g_xstar_le hx

/-- The bound is attained, so no larger constant works. -/
```

planeEuclideanHomeomorph

Classification: main

Source: CMVGeometry.lean:38-42

Declaration SHA-256: 0cae97f48ab4c061d7f08a907381bf52e9560477df33ffb993c23c2cbbf11049

Mathematical statement. The coordinate-preserving homeomorphism to the Euclidean realization.

It transports carriers and their frontiers without changing their coordinates.

```
noncomputable def planeEuclideanHomeomorph : PlanePoint ≈t EuclideanPlane :=
  (WithLp.homeomorphProd 2 ℝ ℝ).symm

/-- CMV's strip density. The closed interfaces belong to the density-one
branch. -/
```

replacement_euclidean_frontier

Classification: main

Source: FrontierPerimeter.lean:1349-1358

Declaration SHA-256: e7e5168c04a70cc7a8c8b17e4b2ad4201ee6c71706067d29f52984410e3fd5d4

Mathematical statement. The Euclidean replacement frontier is the two side arcs, upper cap, and bottom segment.

This transports the complete coordinate frontier, not only a subset of it.

```
theorem replacement_euclidean_frontier (a : ReplacementAssembly) :
  frontier (planeEuclideanHomeomorph '' a.carrier) =
    (planeEuclideanHomeomorph '' StripCore.leftArcTrace a.core) ∪
    ((planeEuclideanHomeomorph '' StripCore.rightArcTrace a.core) ∪
     (planeEuclideanHomeomorph ''
       OneSidedCircularCap.arcTrace a.upperCap) ∪
     (planeEuclideanHomeomorph '' a.exposedLowerChord.carrier))) := by
  rw [← planeEuclideanHomeomorph.image_frontier, a.frontier_carrier]
  simp only [ReplacementAssembly.boundaryTrace, image_union]
```

replacement_frontier_weightedPerimeter_eq

Classification: main

Source: FrontierPerimeter.lean:1845-1898

Declaration SHA-256: 4689c717c4c51ee163362ad12dd8fdb9855d6e5f3a329efb232be3060d7d4cf

Mathematical statement. The replacement's component perimeter equals its canonical frontier perimeter.

The equality includes the full density-one lower chord and the full upper-cap branch.

```
theorem replacement_frontier_weightedPerimeter_eq
  (lam : ℝ) (assembly : ReplacementAssembly) :
  assembly.weightedPerimeter lam =
    WeightedPerimeter lam (FrontierMeasure assembly.carrier) := by
  rcases replacement_pairwise assembly with
    ⟨hAB, hAC, hAD, hBC, hBD, hCD⟩
  have hAint := euclideanStripDensity_integrable0n_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph '')
    StripCore.leftArcTrace assembly.core) (by
    rw [exact_euclidean_leftArcTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top)
  have hBint := euclideanStripDensity_integrable0n_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph '')
    StripCore.rightArcTrace assembly.core) (by
    rw [exact_euclidean_rightArcTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top)
  have hCint := euclideanStripDensity_integrable0n_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph '')
    OneSidedCircularCap.arcTrace assembly.upperCap) (by
    rw [exact_euclidean_capTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top)
  have hDint := euclideanStripDensity_integrable0n_of_measure_ne_top
    lam (s := planeEuclideanHomeomorph '')
    assembly.exposedLowerChord.carrier) (by
    rw [exact_euclidean_segmentCarrier_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top)
  rw [ReplacementAssembly.weightedPerimeter_formula]
  symm
  unfold WeightedPerimeter FrontierMeasure
  rw [integral_map
    planeEuclideanHomeomorph.symm.continuous.measurable.aemeasurable
    (measurable_stripDensity lam).aestronglyMeasurable]
  change (∫ z in frontier (planeEuclideanHomeomorph '' assembly.carrier),
    euclideanStripDensity lam z
    ∂(μH[1] : Measure EuclideanPlane)) =
    assembly.core.boundaryArcLength + lam * assembly.upperCap.arcLength +
    assembly.exposedLowerChord.euclideanLength
  rw [replacement_euclidean_frontier]
  rw [integral_four_union hAB hAC hAD hBC hBD hCD
    (measurableSet_euclidean_rightArcTrace assembly.core)
    (measurableSet_euclidean_capTrace assembly.upperCap)
    (measurableSet_euclidean_segmentCarrier assembly.exposedLowerChord)
    hAint hBint hCint hDint]
  rw [left_integral, right_integral,
    upper_cap_integral lam assembly.upperCap rfl rfl,
    segment_integral lam assembly.exposedLowerChord (by
      simp [ReplacementAssembly.exposedLowerChord]),
    core_param_length, StripCore.boundaryArcLength]
  ring

/-! ## Canonical finite-perimeter realizations -/

/-- A finite Euclidean `H¹` mass for the complete frontier is enough to make
the bounded strip density integrable against its canonical frontier measure. -/
```

replacement_pairwiseOverlap_h1_null

Classification: main

Source: FrontierPerimeter.lean:1280-1290

Declaration SHA-256: 757396ecdaece07d8adc70b78cb910b95f948774e1e20f4dcb7710270834cc6c

Mathematical statement. Replacement boundary overlaps have zero Hausdorff length.

The lower segment can be included in the perimeter sum without positive-measure overlap.

```
theorem replacement_pairwiseOverlap_h1_null (a : ReplacementAssembly) :  
  (μH[1] : Measure EuclideanPlane)  
  (planeEuclideanHomeomorph '' pairwiseOverlap  
    (StripCore.leftArcTrace a.core)  
    (StripCore.rightArcTrace a.core)  
    (OneSidedCircularCap.arcTrace a.upperCap)  
    a.exposedLowerChord.carrier) = 0 := by  
  exact measure_mono_null  
    (image_mono (replacement_pairwiseOverlap_subset_joinSet a))  
    (hausdorffMeasure_one_euclideanJoinSet a.core.chord)
```

xstar

Classification: main

Source: Minimiser.lean:154-155

Declaration SHA-256: 4590a64da97090d3af07ee66b4ba81cd8d175d4ce5739ecba3f1415fc4cba3d8

Mathematical statement. The normalized cap area at the selected root angle.

$X_* = a(\theta_*)$.

```
noncomputable def xstar : ℝ := area θstar
```

θ0f

Classification: main

Source: MorgansArcFunction.lean:247-249

Declaration SHA-256: ae683fbf0fd357d5b8a59395fa6c08fffe184192942daf192789541eb610f89e3

Mathematical statement. A total real function equal to the inverse normalized-area angle for positive input and to $\pi/2$ for nonpositive input.

For $X > 0$, $\theta(X)$ is the selected, necessarily unique $u \in (0, \pi)$ with $a(u) = X$. For $X \leq 0$, the definition sets $\theta(X) = \pi/2$. This nonpositive-input convention makes the function total; it is not an inverse-area identity there and is unused in the positive-cap-area comparison.

```
noncomputable def θ0f (x : ℝ) : ℝ :=  
  if h : 0 < x then (area_surjOn (mem_Ioi.mpr h)).choose else π / 2
```

θ0f_area

Classification: main

Source: MorgansArcFunction.lean:259-266

Declaration SHA-256: f8dabe04d85804b4ae48d760060d4f14fcddf7d4245871a8f753f3695333c49b

Mathematical statement. Applying the inverse to the area of a valid angle recovers that angle.

$\theta(a(u)) = u$ for $0 < u < \pi$, by strict area monotonicity.

```

lemma  $\theta$ 0f_area { $\theta$  :  $\mathbb{R}$ } (h $\theta$  :  $\theta \in \text{Ioo } 0 \ \pi$ ) :  $\theta$ 0f (area  $\theta$ ) =  $\theta$  := by
  have hpos :  $0 < \text{area } \theta := \text{area\_pos } h\theta.1 \ h\theta.2$ 
  have h1 :  $\theta$ 0f (area  $\theta$ )  $\in \text{Ioo } 0 \ \pi := \theta$ 0f_mem hpos
  have h2 : area ( $\theta$ 0f (area  $\theta$ )) = area  $\theta := \text{area\_}\theta$ 0f hpos
  exact area_strictMonoOn.injOn h1 h $\theta$  h2

/-- **Morgan's arc function**: the length of a unit-chord circular arc
enclosing area `x`. -/

```

θ 0f_continuousAt

Classification: main

Source: MorgansArcFunction.lean:283-291

Declaration SHA-256: efcd76351169996ba87c9b4ec14eb080fd4c43d9ee67eed507a1d6cc3182dbda

Mathematical statement. The inverse cap-area angle is continuous at each positive input.

Continuity follows from the monotone continuous area bijection on its open interval.

```

lemma  $\theta$ 0f_continuousAt {x :  $\mathbb{R}$ } (hx :  $0 < x$ ) : ContinuousAt  $\theta$ 0f x := by
  refine  $\theta$ 0f_strictMonoOn.continuousAt_of_image_mem_nhds ?_ ?_
  · exact Ioi_mem_nhds hx
  · exact mem_of_superset (isOpen_Ioo.mem_nhds ( $\theta$ 0f_mem hx))  $\theta$ 0f_image

/-! ## The derivative identity -/

/-- The cancellation: the common factor `sin  $\theta$  -  $\theta$  cos  $\theta$ ` in `ell`` and `area``
divides out, leaving `2 sin  $\theta$ `. -/

```

θ 0f_strictMonoOn

Classification: main

Source: MorgansArcFunction.lean:272-278

Declaration SHA-256: 27129f3095ed7521a354aa523bb8840a0881e0fcc6c3fb3346c9443b50265957

Mathematical statement. The inverse cap-area angle is strictly increasing on positive inputs.

It is the inverse of a strictly increasing bijection between $(0, \pi)$ and $(0, \infty)$.

```

lemma  $\theta$ 0f_strictMonoOn : StrictMonoOn  $\theta$ 0f (Ioi 0) := by
  intro a ha b hb hab
  simp only [mem_Ioi] at ha hb
  rw [← area_strictMonoOn.lt_iff_lt ( $\theta$ 0f_mem ha) ( $\theta$ 0f_mem hb)] at *
  rwa [area_0f ha, area_0f hb]

/-- The image of `(0,  $\infty$ )` under ` $\theta$ 0f` is all of `(0,  $\pi$ )`. -/

```

θ 0f_two_mul_xstar

Classification: main

Source: Minimiser.lean:165-172

Declaration SHA-256: 9e384c7c73738ac8b54bf413e27f2a7a3ad3afa8254d6f30bca551f7d7a2b751

Mathematical statement. The inverse angle at twice Xstar is pi minus the root angle.

Apply inverse-area uniqueness to $a(\pi - \theta_*) = 2X_*$.

```

lemma  $\theta$ 0f_two_mul_xstar :  $\theta$ 0f (2 * xstar) =  $\pi - \theta$ star := by
  have h : area ( $\pi - \theta$ star) = 2 * xstar :=
    area_pi_sub_eq_two_mul ( $\theta$ star_pos,  $\theta$ star_lt_pi)  $\theta$ star_spec
  have :=  $\theta$ 0f_area pi_sub_0star_mem

```

```

    rwa [h] at this

/-- `x*` is a critical point of `g`: the two sines agree because the angles are
supplementary. -/

```

θstar

Classification: main

Source: Minimiser.lean:92-93

Declaration SHA-256: 10465817c5977e25bde5deaa090eec0b969594d7d01505755dd8387cec5029e1

Mathematical statement. The selected root of F in the closed half- π interval.

Subsequent endpoint-sign lemmas place it strictly in $(0, \pi/2)$.

```

noncomputable def θstar : ℝ := exists_F_root.choose

```

θstar_mem

Classification: main

Source: Minimiser.lean:99-114

Declaration SHA-256: c88631345f244802cc83acea97a1e4d6db39816cf66b6157c7b1a365c1b58e68

Mathematical statement. The selected root lies strictly between zero and $\pi/2$.

Neither endpoint solves $F = 0$, so the closed root interval sharpens to the open one.

```

lemma θstar_mem : θstar ∈ Ioo 0 (π / 2) := by
  obtain ⟨hlo, hhi⟩ := θstar_mem_Icc
  constructor
  · rcases eq_or_lt_of_le hlo with heq | hlt
    · exfalso
      have := θstar_spec
      rw [← heq, F_zero] at this
      linarith [pi_pos]
    · exact hlt
  · rcases eq_or_lt_of_le hhi with heq | hlt
    · exfalso
      have := θstar_spec
      rw [heq, F_pi_div_two] at this
      linarith [pi_pos]
    · exact hlt

```

θstar_spec

Classification: main

Source: Minimiser.lean:94-95

Declaration SHA-256: a1f729b0df782b1b7a5288e2aba2fab9def7f4caa3e0a74817fc2fd6b33f74fd

Mathematical statement. The selected angle θ_{star} satisfies $F(\theta_{\text{star}}) = 0$.

This declaration asserts only the equation $F(\theta_*) = 0$, equivalently $3\theta_* - 3\sin\theta_*\cos\theta_* = \pi$. Membership in $[0, \pi/2]$ and then in $(0, \pi/2)$ is supplied separately by `θstar_mem` and `θstar_spec`, not as additional conclusions of this declaration.

Lean references

Minimiser.lean:θstar_mem_Icc; θstar_mem

```

lemma θstar_spec : F θstar = 0 := exists_F_root.choose_spec.2

```

CapSide

Classification: supplement

Source: CMVGeometry.lean:92-99

Declaration SHA-256: 6863ffc025769eb8169e9582a3d673fe180025902f766c2557257f6937cc7a7e

Mathematical statement. Choice of upper or lower side of a horizontal chord.

The two cases determine the cap's half-plane and reflected parametrization.

```
inductive CapSide where
  | upper
  | lower
  deriving DecidableEq

/-- A branch-correct circular segment on a horizontal chord. Its carrier is
`disk ∩ half-plane`; this single definition includes minor, semicircular, and
major caps. -/
```

CompactCellAssembly.QInterval

Classification: supplement

Source: CompactCellAssembly.lean:20-24

Declaration SHA-256: bf869b1f520d1cedce20479248192ec784229f327d43ecc88c3fd90b71146250

Mathematical statement. The shared rational interval type in compact-cell assembly.

This abbreviation uses the existing ordered-endpoint interval representation.

```
abbrev QInterval := ScalarSuffixCertificate.QInterval

/-- Sound real consequences required from one compact-cell interval
certificate. Numerical cell modules prove these fields from exact rational
interval arithmetic; the shared assembly below contains no numerical oracle. -/
```

CompactCellCertificate.QInterval

Classification: supplement

Source: CompactCellCertificate.lean:19-23

Declaration SHA-256: baf33f14dafc66f34a92b548a53e5c251fc54f718db15c2e52f89ec02947b54d

Mathematical statement. The shared exact rational interval type used by compact certificates.

No separate interval semantics are introduced.

```
abbrev QInterval := ScalarSuffixCertificate.QInterval

namespace QInterval

/-- Exact-rational wrapper around the degree-27 Taylor inverse-sine rule. -/
```

CompactCellCertificate.QInterval.realContains_arcsin_of_taylor

Classification: supplement

Source: CompactCellCertificate.lean:24-44

Declaration SHA-256: 4923f85862130adb8d9efbeee24dfca326699114fa32f80b790af2693e065f1b

Mathematical statement. Taylor sine bounds certify an arcsine enclosure on its principal branch.

Monotonicity of sine converts certified endpoint inequalities into bounds for the inverse.

```

theorem realContains_arcsin_of_taylor
  (a : QInterval) (x : ℚ)
  (hx : (-1 : ℚ) ≤ x ∧ x ≤ 1)
  (hbranch : (a.lo : ℝ) ∈ Icc (-(π / 2)) (π / 2) ∧
    (a.hi : ℝ) ∈ Icc (-(π / 2)) (π / 2))
  (hlo : (ScalarSuffixCertificate.sinTaylor27Interval a.lo).hi ≤ x)
  (hhi : x ≤ (ScalarSuffixCertificate.sinTaylor27Interval a.hi).lo) :
  a.RealContains (arcsin (x : ℝ)) := by
  apply ScalarSuffixCertificate.realContains_arcsin_of_sin_bounds
  · simpa [Set.mem_Icc] using (show ((-1 : ℝ) ≤ (x : ℝ) ∧ (x : ℝ) ≤ 1) by
    exact_mod_cast hx)
  · exact hbranch
  · exact (ScalarSuffixCertificate.sinTaylor27Interval_sound a.lo).2.trans
    (show ((ScalarSuffixCertificate.sinTaylor27Interval a.lo).hi : ℝ) ≤
      (x : ℝ) by exact_mod_cast hlo)
  · exact (show (x : ℝ) ≤
    ((ScalarSuffixCertificate.sinTaylor27Interval a.hi).lo : ℝ) by
    exact_mod_cast hhi).trans
    (ScalarSuffixCertificate.sinTaylor27Interval_sound a.hi).1

/-- Interval-input version of the inverse-sine Taylor wrapper. -/

```

CompactCellCertificate.QInterval.realContains_arcsin_of_taylor_interval

Classification: supplement

Source: CompactCellCertificate.lean:45-68

Declaration SHA-256: 72d1f4184baa0c96534f19a83e2653ab69774d487022002946ed89c5dcb4c89d

Mathematical statement. The same inverse-sine enclosure for an interval of inputs.

Endpoint bounds enclose every real input in the source interval.

```

theorem realContains_arcsin_of_taylor_interval
  (a xInterval : QInterval) (x : ℝ)
  (hx : xInterval.RealContains x)
  (hxUnit : (-1 : ℝ) ≤ x ∧ x ≤ 1)
  (hbranch : (a.lo : ℝ) ∈ Icc (-(π / 2)) (π / 2) ∧
    (a.hi : ℝ) ∈ Icc (-(π / 2)) (π / 2))
  (hlo : (ScalarSuffixCertificate.sinTaylor27Interval a.lo).hi ≤ xInterval.lo)
  (hhi : xInterval.hi ≤
    (ScalarSuffixCertificate.sinTaylor27Interval a.hi).lo) :
  a.RealContains (arcsin x) := by
  apply ScalarSuffixCertificate.realContains_arcsin_of_sin_bounds hxUnit hbranch
  · exact (ScalarSuffixCertificate.sinTaylor27Interval_sound a.lo).2.trans
    (show ((ScalarSuffixCertificate.sinTaylor27Interval a.lo).hi : ℝ) ≤ x by
    exact (show ((ScalarSuffixCertificate.sinTaylor27Interval a.lo).hi : ℝ) ≤
      (xInterval.lo : ℝ) by exact_mod_cast hlo) |>.trans hx.1)
  · exact hx.2.trans
    ((show (xInterval.hi : ℝ) ≤
      ((ScalarSuffixCertificate.sinTaylor27Interval a.hi).lo : ℝ) by
      exact_mod_cast hhi).trans
      (ScalarSuffixCertificate.sinTaylor27Interval_sound a.hi).1)

end QInterval

/-- Convexity of an enclosing rational interval, stated for an unordered pair. -/

```

CompactCellCertificate.halfPiBox

Classification: supplement

Source: CompactCellCertificate.lean:193-201

Declaration SHA-256: b24b5a3000649c97c552eff6f175f59f2e5aa6602a533417871ec37dfc3c128c

Mathematical statement. A rational interval enclosing $\pi/2$.

It supplies a certified inverse-trigonometric branch endpoint.

```
def halfPiBox : QInterval :=
  ScalarSuffixCertificate.piInterval.nsmul (1 / 2) (by norm_num)

@[simp] theorem halfPiBox_lo : halfPiBox.lo = 392699 / 250000 := by
  norm_num [halfPiBox, ScalarSuffixCertificate.piInterval,
    LeanSuffixReflective.QInterval.nsmul]

/-- The common rational half-pi box encloses the exact principal-branch
endpoint. -/
```

CompactCellCertificate.halfPiBox_sound

Classification: supplement

Source: CompactCellCertificate.lean:202-210

Declaration SHA-256: 32569ba98b96e5c623e6b4690e061422a54fc3e99bc88a52ad664b5adadcdbf6

Mathematical statement. $\pi/2$ belongs to the chosen rational interval.

The claim is an exact enclosure, not a decimal approximation.

```
theorem halfPiBox_sound : halfPiBox.RealContains ( $\pi$  / 2) := by
  unfold halfPiBox
  convert ScalarSuffixCertificate.QInterval.realContains_nsmul
    (q := (1 / 2 :  $\mathbb{Q}$ )) (by norm_num)
  ScalarSuffixCertificate.piInterval_sound using 1 <;> norm_num <;> ring

/-- Exact rational branch check for inverse-sine certificates. Unlike the old
`3 / 2` cutoff, this accepts every interval below the certified lower endpoint
of the common half-pi box. -/
```

CompactCellCertificate.principalAsinBranch_of_nonneg_of_hi_le_halfPiBox

Classification: supplement

Source: CompactCellCertificate.lean:211-223

Declaration SHA-256: 5d213510a122acf90e70e2f5809f269336785657fb7f1298c72b17c1dfca9aca

Mathematical statement. Nonnegative angle bounds below the certified half-pi limit lie on the principal sine branch.

This validates inversion of the sine endpoint estimates.

```
theorem principalAsinBranch_of_nonneg_of_hi_le_halfPiBox
  (a : QInterval) (h : 0 ≤ a.lo ∧ a.hi ≤ halfPiBox.lo) :
  (a.lo :  $\mathbb{R}$ ) ∈ Icc (-( $\pi$  / 2)) ( $\pi$  / 2) ∧
  (a.hi :  $\mathbb{R}$ ) ∈ Icc (-( $\pi$  / 2)) ( $\pi$  / 2) := by
  have hlo : (0 :  $\mathbb{R}$ ) ≤ a.lo := by exact_mod_cast h.1
  have hord : (a.lo :  $\mathbb{R}$ ) ≤ a.hi := by exact_mod_cast a.ordered
  have hhi : (a.hi :  $\mathbb{R}$ ) ≤  $\pi$  / 2 :=
    (show (a.hi :  $\mathbb{R}$ ) ≤ (halfPiBox.lo :  $\mathbb{R}$ ) by exact_mod_cast h.2) |>.trans
      halfPiBox_sound.1
  exact ⟨(by linarith [Real.pi_pos], hord.trans hhi),
    (by linarith [Real.pi_pos], hhi)⟩

/-- Exact atom production from rational square and degree-27 Taylor bounds. -/
```


CompactCellCertificate.realContains_of_mem_uIcc

Classification: supplement

Source: CompactCellCertificate.lean:69-76

Declaration SHA-256: ebce25efd40e9df8676596e53d18f779f851450dc32118fcfdf8ff8a4a80eceb

Mathematical statement. An intermediate point between two enclosed reals remains enclosed.

Closed intervals are convex; this transfers mean-value witnesses into a certificate slab.

```
theorem realContains_of_mem_uIcc {i : QInterval} {c h x : ℝ}
  (hc : i.RealContains c) (hh : i.RealContains h)
  (hx : x ∈ uIcc c h) : i.RealContains x := by
  rcases Set.mem_uIcc.mp hx with hx | hx
  · exact ⟨hc.1.trans hx.1, hx.2.trans hh.2⟩
  · exact ⟨hh.1.trans hx.1, hx.2.trans hc.2⟩

/-- Box-independent centered type-(iv) area enclosure. -/
```

CompactCellCertificate.typeFourAngleBox

Classification: supplement

Source: CompactCellCertificate.lean:582-584

Declaration SHA-256: 0938a788ac4433df3e54045185685f79b05098a7ac19313f95d43a2d5b337139

Mathematical statement. Interval expression for B4.

It combines $\lambda \arccos(h/\lambda)$ and $\arcsinh$.

```
def typeFourAngleBox (lambdaInterval asinHI asinRatioI : QInterval) : QInterval :=
  (lambdaInterval.mul (halfPiBox.sub asinRatioI)).add asinHI
```

CompactCellCertificate.typeFourAreaBox

Classification: supplement

Source: CompactCellCertificate.lean:589-596

Declaration SHA-256: d6bce36771e12b0ae40beb52da88876842d09c71cb25452547718f2a63131dfa

Mathematical statement. Interval expression for A4.

It evaluates $2(B_4 + hD_4)/h^2$ using positive denominator bounds.

```
def typeFourAreaBox (lambdaInterval hI uI dI asinHI asinRatioI : QInterval)
  (hlambda : 0 < lambdaInterval.lo) (hhSq : 0 < (hI.mul hI).lo) : QInterval :=
  ScalarSuffixCertificate.QInterval.divPos
  (LeanSuffixReflective.QInterval.nsmul 2 (by norm_num)
    ((typeFourAngleBox lambdaInterval asinHI asinRatioI).add
      (hI.mul (typeFourDeltaBox lambdaInterval uI dI hlambda))))
  (hI.mul hI) hhSq
```

CompactCellCertificate.typeFourArea_centered_enclosure

Classification: supplement

Source: CompactCellCertificate.lean:77-105

Declaration SHA-256: 5c97a83974bdea22e535e6c42e2259805e5f45e9f57a2175a11ade991b773562

Mathematical statement. A centered value and derivative enclosure bound the type-(iv) area.

Use $A_4(h) = A_4(c) + (h - c)K_4(\xi)/\xi^3$ with one shared ξ between c and h .

```

theorem typeFourArea_centered_enclosure
  (lambdaInterval hInterval : QInterval) (center : Q)
  {lam h : ℝ}
  (_hlam : lambdaInterval.RealContains lam)
  (hh : hInterval.RealContains h)
  (hcenter : hInterval.RealContains (center : ℝ))
  (hlambda : 1 < lam)
  (hregular : ∀ {x : ℝ}, hInterval.RealContains x → x ∈ Ioo (0 : ℝ) 1)
  (base slope displacement : QInterval)
  (hbase : base.RealContains (LeanSuffixAnalytic.typeFourArea lam (center : ℝ)))
  (hslope : ∀ x, hInterval.RealContains x →
    slope.RealContains (LeanSuffixAnalytic.typeFourFold lam x / x ^ 3))
  (hdisplacement : displacement.RealContains (h - (center : ℝ))) :
  (base.add (slope.mul displacement)).RealContains
    (LeanSuffixAnalytic.typeFourArea lam h) := by
  apply ScalarSuffixCertificate.QInterval.realContains_centered_meanValue
    (LeanSuffixAnalytic.typeFourArea lam)
    (fun x => LeanSuffixAnalytic.typeFourFold lam x / x ^ 3)
  · intro x hx
    have hxSlab := realContains_of_mem_uIcc hcenter hh hx
    have hxReg := hregular hxSlab
    exact LeanSuffixAnalytic.hasDerivAt_typeFourArea
      hlamba hxReg.1 hxReg.2
  · exact hbase
  · intro x hx
    exact hslope x (realContains_of_mem_uIcc hcenter hh hx)
  · exact hdisplacement

/-- Box-independent centered type-(iv) perimeter enclosure. -/

```

CompactCellCertificate.typeFourDeltaBox

Classification: supplement

Source: CompactCellCertificate.lean:585-588

Declaration SHA-256: ab7d789a19d88d75ee1647ab012fc735ddecd9dc900020900794197c4a8a7372

Mathematical statement. Interval expression for D4.

It encloses the normalized exterior radical minus the unit-circle radical.

```

def typeFourDeltaBox (lambdaInterval uI dI : QInterval)
  (hlamba : 0 < lambdaInterval.lo) : QInterval :=
  (dI.divPos lambdaInterval hlamba).sub uI

```

CompactCellCertificate.typeFourFoldBox

Classification: supplement

Source: CompactCellCertificate.lean:602-607

Declaration SHA-256: c03c54c4b04b12af79ff4aba9a1baf5c6d0801c18971d3619c3c5a83e175fdfd

Mathematical statement. Interval expression for K4.

Its opposite endpoint signs bracket the unique stationary type-(iv) curvature.

```

def typeFourFoldBox (lambdaInterval hI uI dI asinHI asinRatioI : QInterval)
  (_hlamba : 0 < lambdaInterval.lo) (hu : 0 < uI.lo) (hd : 0 < dI.lo) : QInterval :=
  ((hI.mul ((uI.recipPos hu).sub (lambdaInterval.divPos dI hd))).sub
    (typeFourAngleBox lambdaInterval asinHI asinRatioI)).nsmul 4 (by norm_num)

/-- Formula enclosures assembled from type-(iv) atoms. -/

```

CompactCellCertificate.typeFourPerimeterBox

Classification: supplement

Source: CompactCellCertificate.lean:597-601

Declaration SHA-256: 9acb08313f7038b05bcb4ead730aaeb55230fdc38d1e200f1e4b75f2584d5063

Mathematical statement. Interval expression for P4.

It evaluates $4B_4/h$ on the regular curvature slab.

```
def typeFourPerimeterBox (lambdaInterval hI asinHI asinRatioI : QInterval)
  (hh : 0 < hI.lo) : QInterval :=
  (ScalarSuffixCertificate.QInterval.divPos
    ((typeFourAngleBox lambdaInterval asinHI asinRatioI).nsmul 4 (by norm_num)) hI hh)
```

CompactCellCertificate.typeFourPerimeter_centered_enclosure

Classification: supplement

Source: CompactCellCertificate.lean:106-134

Declaration SHA-256: b6de403709bd1fd67e6a59e747fa0828b9b2000aa26dc332ce4736414ad4a315

Mathematical statement. Centered enclosure for type-(iv) perimeter.

The derivative term is $K_4(\xi)/\xi^2$, retaining correlations in the fold numerator.

```
theorem typeFourPerimeter_centered_enclosure
  (lambdaInterval hInterval : QInterval) (center : Q)
  {lam h : ℝ}
  (_hlam : lambdaInterval.RealContains lam)
  (hh : hInterval.RealContains h)
  (hcenter : hInterval.RealContains (center : ℝ))
  (hlambda : 1 < lam)
  (hregular : ∀ {x : ℝ}, hInterval.RealContains x → x ∈ Ioo (0 : ℝ) 1)
  (base slope displacement : QInterval)
  (hbase : base.RealContains (LeanSuffixAnalytic.typeFourPerimeter lam (center : ℝ)))
  (hslope : ∀ x, hInterval.RealContains x →
    slope.RealContains (LeanSuffixAnalytic.typeFourFold lam x / x ^ 2))
  (hdisplacement : displacement.RealContains (h - (center : ℝ))) :
  (base.add (slope.mul displacement)).RealContains
    (LeanSuffixAnalytic.typeFourPerimeter lam h) := by
  apply ScalarSuffixCertificate.QInterval.realContains_centered_meanValue
    (LeanSuffixAnalytic.typeFourPerimeter lam)
    (fun x => LeanSuffixAnalytic.typeFourFold lam x / x ^ 2)
  · intro x hx
    have hxSlab := realContains_of_mem_uIcc hcenter hh hx
    have hxReg := hregular hxSlab
    exact LeanSuffixAnalytic.hasDerivAt_typeFourPerimeter
      hlambda hxReg.1 hxReg.2
  · exact hbase
  · intro x hx
    exact hslope x (realContains_of_mem_uIcc hcenter hh hx)
  · exact hdisplacement

/-- Box-independent centered type-(iii) area enclosure. -/
```

CompactCellCertificate.typeFourShiftedInterval

Classification: supplement

Source: CompactCellCertificate.lean:361-368

Declaration SHA-256: c59ab8bb2b04642fd5ba4e5bffd2f3c42cf4105e98a359d7aa614a9c22b9d322

Mathematical statement. The exact image of a rational curvature interval under h maps to $(h+1)/2$, used to reuse the type-(iii) atom checker for type-(iv) atoms.

For a rational interval $I = [a, b]$ with $a \leq b$, the definition gives $[(a + 1)/2, (b + 1)/2]$ by adding the point interval $[1, 1]$ and scaling by the nonnegative rational $1/2$. Thus an enclosed type-(iv) curvature h yields an enclosed parameter $s = (h + 1)/2$. The type-(iii) shape map $s \mapsto 2s - 1$ then returns h , since $2((h + 1)/2) - 1 = h$. This affine curvature reparametrization lets the type-(iv) atom theorem reuse the type-(iii) radical and inverse-trigonometric bounds; it is not an angle complement.

```
def typeFourShiftedInterval (hI : QInterval) : QInterval :=
  (hI.add (LeanSuffixReflective.QInterval.point 1)).nsmul
    (1 / 2) (by norm_num)

/-- Exact type-(iv) atom production from rational square and degree-27 Taylor
bounds. This is the box-independent counterpart of
`typeThree_atoms_of_exact_bounds`; concrete generated cells discharge the
single rational conjunction with `norm_num`. -/
```

CompactCellCertificate.typeFour_atom_enclosures

Classification: supplement

Source: CompactCellCertificate.lean:608-665

Declaration SHA-256: 6e810d9ecc043611b95444c9f9834f7abe166e7b764027bba215f104fde2bca9

Mathematical statement. Soundness of the type-(iv) objective and fold boxes.

Certified atom bounds imply containment of the exact scalar expressions.

```
theorem typeFour_atom_enclosures
  (lambdaInterval hI uI dI asinHI asinRatioI : QInterval)
  {lam h : ℝ} (hlam : lambdaInterval.RealContains lam)
  (hh : hI.RealContains h)
  (hatoms : uI.RealContains (sqrt (1 - h ^ 2)) ^
    dI.RealContains (sqrt (lam ^ 2 - h ^ 2)) ^
    asinHI.RealContains (arcsin h) ^
    asinRatioI.RealContains (arcsin (h / lam)))
  (hlambda : 0 < lambdaInterval.lo) (hhPos : 0 < hI.lo)
  (hhSq : 0 < (hI.mul hI).lo) (huPos : 0 < uI.lo) (hdPos : 0 < dI.lo) :
  (typeFourAreaBox lambdaInterval hI uI dI asinHI asinRatioI hlambda hhSq).RealContains
    (LeanSuffixAnalytic.typeFourArea lam h) ^
  (typeFourPerimeterBox lambdaInterval hI asinHI asinRatioI hhPos).RealContains
    (LeanSuffixAnalytic.typeFourPerimeter lam h) ^
  (typeFourFoldBox lambdaInterval hI uI dI asinHI asinRatioI
    hlambda huPos hdPos).RealContains (LeanSuffixAnalytic.typeFourFold lam h) := by
  rcases hatoms with ⟨hu, hd, hasinH, hasinRatio⟩
  have hhalfPi : halfPiBox.RealContains (π / 2) := halfPiBox_sound
  have hacos : (halfPiBox.sub asinRatioI).RealContains (arccos (h / lam)) :=
    ScalarSuffixCertificate.realContains_arccos_of_arcsin
  have hangle : (typeFourAngleBox lambdaInterval asinHI asinRatioI).RealContains
    (LeanSuffixAnalytic.typeFourAngle lam h) := by
    unfold typeFourAngleBox LeanSuffixAnalytic.typeFourAngle
    exact LeanSuffixReflective.QInterval.realContains_add
      (ScalarSuffixCertificate.QInterval.realContains_mul hlam hacos) hasinH
  have hdelta : (typeFourDeltaBox lambdaInterval uI dI hlambda).RealContains
    (LeanSuffixAnalytic.typeFourDelta lam h) := by
    unfold typeFourDeltaBox LeanSuffixAnalytic.typeFourDelta
    exact LeanSuffixReflective.QInterval.realContains_sub
      (ScalarSuffixCertificate.QInterval.realContains_divPos hlamba hd hlam) hu
  have hareaNum := ScalarSuffixCertificate.QInterval.realContains_nsmul
    (q := (2 : ℚ)) (by norm_num)
  (LeanSuffixReflective.QInterval.realContains_add hangle
    (ScalarSuffixCertificate.QInterval.realContains_mul hh hdelta))
  have hhSquare := ScalarSuffixCertificate.QInterval.realContains_mul hh hh
  have harea := ScalarSuffixCertificate.QInterval.realContains_divPos
    hhSq hareaNum hhSquare
  have hperimeterNum := ScalarSuffixCertificate.QInterval.realContains_nsmul
    (q := (4 : ℚ)) (by norm_num) hangle
  have hperimeter := ScalarSuffixCertificate.QInterval.realContains_divPos
    hhPos hperimeterNum hh
```

```

have hinuU := ScalarSuffixCertificate.QInterval.realContains_recipPos huPos hu
have hlamDivD := ScalarSuffixCertificate.QInterval.realContains_divPos hdPos hlam hd
have hfold := ScalarSuffixCertificate.QInterval.realContains_nsmul
  (q := (4 : ℚ)) (by norm_num)
  (LeanSuffixReflective.QInterval.realContains_sub
    (ScalarSuffixCertificate.QInterval.realContains_mul hh
      (LeanSuffixReflective.QInterval.realContains_sub hinuU hlamDivD))) hangle)
refine {?, ?, ?}
· convert harea using 1 <;>
  simp only [typeFourAreaBox, LeanSuffixAnalytic.typeFourArea] <;> ring
· convert hperimeter using 1 <;>
  simp only [typeFourPerimeterBox, LeanSuffixAnalytic.typeFourPerimeter] <;> ring
· convert hfold using 1 <;>
  simp only [typeFourFoldBox, LeanSuffixAnalytic.typeFourFold] <;> ring

end CompactCellCertificate

```

CompactCellCertificate.typeFour_atoms_of_exact_bounds

Classification: supplement

Source: CompactCellCertificate.lean:369-432

Declaration SHA-256: 782e78df1d670aee0c342670496e6eb38d375401e162a64061d13d6eb4739dd2

Mathematical statement. Exact bounds enclose the type-(iv) elementary atoms.

The soundness statement handles radicals and principal inverse-trigonometric arguments on the regular slab.

```

theorem typeFour_atoms_of_exact_bounds
  (lambdaInterval hI uI dI asinHI asinRI : QInterval) {lam h : ℝ}
  (hlam : lambdaInterval.RealContains lam) (hh : hI.RealContains h)
  (hlambdaLoPos : 0 < lambdaInterval.lo)
  (hlambdaOne : 1 < lam)
  (hexact :
    0 ≤ (((typeFourShiftedInterval hI).nsmul 2 (by norm_num)).sub
      (LeanSuffixReflective.QInterval.point 1)).lo ^ 2
    (((typeFourShiftedInterval hI).nsmul 2 (by norm_num)).sub
      (LeanSuffixReflective.QInterval.point 1)).hi ≤ 1 ^ 2
    0 ≤ uI.lo ^ 2
    uI.lo ^ 2 ≤ 1 - (((typeFourShiftedInterval hI).nsmul 2 (by norm_num)).sub
      (LeanSuffixReflective.QInterval.point 1)).hi ^ 2
    1 - (((typeFourShiftedInterval hI).nsmul 2 (by norm_num)).sub
      (LeanSuffixReflective.QInterval.point 1)).lo ^ 2 ≤ uI.hi ^ 2
    0 ≤ dI.lo ^ 2
    dI.lo ^ 2 ≤ lambdaInterval.lo ^ 2 -
      (((typeFourShiftedInterval hI).nsmul 2 (by norm_num)).sub
        (LeanSuffixReflective.QInterval.point 1)).hi ^ 2
    lambdaInterval.hi ^ 2 -
      (((typeFourShiftedInterval hI).nsmul 2 (by norm_num)).sub
        (LeanSuffixReflective.QInterval.point 1)).lo ^ 2 ≤ dI.hi ^ 2
    (0 ≤ asinHI.lo ^ 2 ^ asinHI.hi ≤ halfPiBox.lo) ^
    (ScalarSuffixCertificate.sinTaylor27Interval asinHI.lo).hi ≤
      (((typeFourShiftedInterval hI).nsmul 2 (by norm_num)).sub
        (LeanSuffixReflective.QInterval.point 1)).lo ^ 2
    (((typeFourShiftedInterval hI).nsmul 2 (by norm_num)).sub
      (LeanSuffixReflective.QInterval.point 1)).hi ≤
      (ScalarSuffixCertificate.sinTaylor27Interval asinHI.hi).lo ^ 2
    (0 ≤ asinRI.lo ^ 2 ^ asinRI.hi ≤ halfPiBox.lo) ^
    (ScalarSuffixCertificate.sinTaylor27Interval asinRI.lo).hi ≤
      (ScalarSuffixCertificate.QInterval.divPos
        (((typeFourShiftedInterval hI).nsmul 2 (by norm_num)).sub
          (LeanSuffixReflective.QInterval.point 1)) lambdaInterval
        hlambdaLoPos).lo ^ 2
    (ScalarSuffixCertificate.QInterval.divPos
      (((typeFourShiftedInterval hI).nsmul 2 (by norm_num)).sub
        (LeanSuffixReflective.QInterval.point 1)) lambdaInterval
      hlambdaLoPos).hi ≤
      (ScalarSuffixCertificate.sinTaylor27Interval asinRI.hi).lo) :
  uI.RealContains (sqrt (1 - h ^ 2)) ^
  dI.RealContains (sqrt (lam ^ 2 - h ^ 2)) ^

```

```

    asinHI.RealContains (arcsin h) ^
    asinRI.RealContains (arcsin (h / lam)) := by
let shifted : QInterval :=
  (hI.add (LeanSuffixReflective.QInterval.point 1)).nsmul
  (1 / 2) (by norm_num)
have ht : shifted.RealContains ((h + 1) / 2) := by
  have ha := LeanSuffixReflective.QInterval.realContains_add hh
    (LeanSuffixReflective.QInterval.realContains_point 1 (1 : ℝ) |>.2 (by norm_num))
  convert ScalarSuffixCertificate.QInterval.realContains_nsmul
    (q := (1 / 2 : ℚ)) (by norm_num) ha using 1 <=> ring
rcases hexact with
  ⟨hq0, hq1, hu0, hul, huh, hd0, hd1, hdh, haB, hal, hah, hrB, hrl, hrh⟩
have ha := typeThree_atoms_of_exact_bounds
  lambdaInterval shifted uI dI asinHI asinRI hlam ht
  hlambdaLoPos hlambdaOne hq0 hq1 hu0 hul huh hd0 hd1 hdh
  haB hal hah hrB hrl hrh
have hs : LeanSuffixAnalytic.typeThreeShape ((h + 1) / 2) = h := by
  unfold LeanSuffixAnalytic.typeThreeShape
  ring
simp [hs] using ha

```

CompactCellCertificate.typeThreeAngleBox

Classification: supplement

Source: CompactCellCertificate.lean:436-438

Declaration SHA-256: 07cf7ee9a93efff6da0c0dde5b35b60c96d4a49a9bfb27da2b96e97cee62677e

Mathematical statement. Interval expression for the type-(iii) angle contribution.

It combines the enclosed inverse-trigonometric atoms with the density interval.

```

def typeThreeAngleBox (lambdaInterval asinQI asinRatioI : QInterval) : QInterval :=
  (lambdaInterval.mul (halfPiBox.sub asinRatioI)).add asinQI

```

CompactCellCertificate.typeThreeAreaBox

Classification: supplement

Source: CompactCellCertificate.lean:443-450

Declaration SHA-256: ef992989a954766b4ace02b65812003aa8c6c32c075d2783aff3bdaa521effd7

Mathematical statement. Interval expression for A3.

It follows the exact numerator and divides by the positive curvature square.

```

def typeThreeAreaBox (lambdaInterval hI uI dI asinQI asinRatioI : QInterval)
  (hlambda : 0 < lambdaInterval.lo) (hhSq : 0 < (hI.mul hI).lo) : QInterval :=
  ScalarSuffixCertificate.QInterval.divPos
    ((typeThreeAngleBox lambdaInterval asinQI asinRatioI).add halfPiBox |>.add
      (ScalarSuffixCertificate.QInterval.mul
        ((typeThreeQBox hI).add (LeanSuffixReflective.QInterval.point 2))
        (typeThreeDeltaBox lambdaInterval uI dI hlambda))) (hI.mul hI) hhSq

```

CompactCellCertificate.typeThreeDeltaBox

Classification: supplement

Source: CompactCellCertificate.lean:439-442

Declaration SHA-256: 1e88f30488cceb14eb20e3a119a4f1d4e5d4e9a2623b803c1f4033ff8844eadb

Mathematical statement. Interval expression for D3.

It encloses the difference of the two normalized radical terms.

```
def typeThreeDeltaBox (lambdaInterval uI dI : QInterval)
  (hlambd : 0 < lambdaInterval.lo) : QInterval :=
  (dI.divPos lambdaInterval hlambd).sub uI
```

CompactCellCertificate.typeThreeFoldBox

Classification: supplement

Source: CompactCellCertificate.lean:458-472

Declaration SHA-256: 134abfc2384eaab1be42cb464447703ca4b8c8946a097fc0f4846639ef17f9e4

Mathematical statement. Interval expression for K_3 .

Its sign controls the type-(iii) area derivative on a positive-curvature slab.

```
def typeThreeFoldBox (lambdaInterval hI uI dI asinQI asinRatioI : QInterval)
  (hlambd : 0 < lambdaInterval.lo) (hu : 0 < uI.lo)
  (hlambdD : 0 < (lambdaInterval.mul dI).lo) : QInterval :=
  let qI := typeThreeQBox hI
  let angleI := typeThreeAngleBox lambdaInterval asinQI asinRatioI
  let deltaI := typeThreeDeltaBox lambdaInterval uI dI hlambd
  let first := ScalarSuffixCertificate.QInterval.divPos
    ((LeanSuffixReflective.QInterval.point 1).add qI) uI hu
  let second := ScalarSuffixCertificate.QInterval.divPos
    ((lambdaInterval.mul lambdaInterval).add qI)
    (lambdaInterval.mul dI) hlambdD
  ((hI.mul (first.sub second)).nsmul 4 (by norm_num)).sub
    (((angleI.add halfPiBox).add deltaI).nsmul 2 (by norm_num))

/-- Formula enclosure assembled from type-(iii) atoms for the fold numerator. -/
```

CompactCellCertificate.typeThreePerimeterBox

Classification: supplement

Source: CompactCellCertificate.lean:451-457

Declaration SHA-256: 5a67f5bcbe9b9ff1470b4fe9640a7fcc5189a1c047837c4c7b6352a695bd3349

Mathematical statement. Interval expression for P_3 .

The enclosure evaluates $2(B_3 + D_3)/h$.

```
def typeThreePerimeterBox (lambdaInterval hI uI dI asinQI asinRatioI : QInterval)
  (hlambd : 0 < lambdaInterval.lo) (hh : 0 < hI.lo) : QInterval :=
  (ScalarSuffixCertificate.QInterval.divPos
    ((typeThreeAngleBox lambdaInterval asinQI asinRatioI).add halfPiBox |>.add
      (typeThreeDeltaBox lambdaInterval uI dI hlambd)) hI hh).nsmul 2 (by norm_num)

/-- Exact interval expression for the type-(iii) fold numerator. -/
```

CompactCellCertificate.typeThreePerimeter_centered_enclosure

Classification: supplement

Source: CompactCellCertificate.lean:164-192

Declaration SHA-256: 716e11bc278bf7afe71f82cb84a51dd8766c11970a07ccc39643388de278cc03

Mathematical statement. Centered enclosure for type-(iii) perimeter.

The mean-value derivative is $K_3(\xi)/\xi^2$ on the regular slab.

```
theorem typeThreePerimeter_centered_enclosure
  (lambdaInterval hInterval : QInterval) (center : Q)
  {lam h : ℝ}
```



```

(_hlam : lambdaInterval.RealContains lam)
(hh : hInterval.RealContains h)
(hcenter : hInterval.RealContains (center : ℝ))
(hlambda : 1 < lam)
(hregular : ∀ {x : ℝ}, hInterval.RealContains x → x ∈ Ioo (0 : ℝ) 1)
(base slope displacement : QInterval)
(hbase : base.RealContains (LeanSuffixAnalytic.typeThreePerimeter lam (center : ℝ)))
(hslope : ∀ x, hInterval.RealContains x →
  slope.RealContains (LeanSuffixAnalytic.typeThreeFold lam x / x ^ 2))
(hdisplacement : displacement.RealContains (h - (center : ℝ))) :
(base.add (slope.mul displacement)).RealContains
  (LeanSuffixAnalytic.typeThreePerimeter lam h) := by
apply ScalarSuffixCertificate.QInterval.realContains_centered_meanValue
  (LeanSuffixAnalytic.typeThreePerimeter lam)
  (fun x => LeanSuffixAnalytic.typeThreeFold lam x / x ^ 2)
· intro x hx
  have hxSlab := realContains_of_mem_uIcc hcenter hh hx
  have hxReg := hregular hxSlab
  exact LeanSuffixAnalytic.hasDerivAt_typeThreePerimeter
    hlambda hxReg.1 hxReg.2
· exact hbase
· intro x hx
  exact hslope x (realContains_of_mem_uIcc hcenter hh hx)
· exact hdisplacement

/-- Common half-pi enclosure. -/

```

CompactCellCertificate.typeThreeQBox

Classification: supplement

Source: CompactCellCertificate.lean:433-435

Declaration SHA-256: 9902aca80d47ee5ab1219050853cf20b2a80b983bed70658848ab626cc339f9f

Mathematical statement. Interval expression for the type-(iii) shape parameter.

Apply $q = 2h - 1$ to the curvature interval.

```

def typeThreeQBox (hI : QInterval) : QInterval :=
  (hI.nsmul 2 (by norm_num)).sub (LeanSuffixReflective.QInterval.point 1)

```

CompactCellCertificate.typeThree_area_perimeter_atom_enclosure

Classification: supplement

Source: CompactCellCertificate.lean:529-581

Declaration SHA-256: b6ef0ef761987f79259cc0d419f8a32fbed4691f7270b539e416653bb41f839b

Mathematical statement. Soundness of the type-(iii) area and perimeter boxes.

The theorem turns elementary atom containment into simultaneous objective containment.

```

theorem typeThree_area_perimeter_atom_enclosure
  (lambdaInterval hI uI dI asinQI asinRatioI : QInterval)
  {lam h : ℝ} (hlam : lambdaInterval.RealContains lam)
  (hh : hI.RealContains h)
  (hatoms : uI.RealContains (sqrt (1 - LeanSuffixAnalytic.typeThreeShape h ^ 2)) ^
    dI.RealContains (sqrt (lam ^ 2 - LeanSuffixAnalytic.typeThreeShape h ^ 2)) ^
    asinQI.RealContains (arcsin (LeanSuffixAnalytic.typeThreeShape h)) ^
    asinRatioI.RealContains (arcsin (LeanSuffixAnalytic.typeThreeShape h / lam)))
  (hlambda : 0 < lambdaInterval.lo) (hhPos : 0 < hI.lo)
  (hhSq : 0 < (hI.mul hI).lo) :
  (typeThreeAreaBox lambdaInterval hI uI dI asinQI asinRatioI hlambda hhSq).RealContains
    (LeanSuffixAnalytic.typeThreeArea lam h) ^
  (typeThreePerimeterBox lambdaInterval hI uI dI asinQI asinRatioI hlambda hhPos).RealContains
    (LeanSuffixAnalytic.typeThreePerimeter lam h) := by
rcases hatoms with ⟨hu, hd, hasinQ, hasinRatio⟩

```



```

have hhalfPi : halfPiBox.RealContains ( $\pi / 2$ ) := halfPiBox_sound
have hq : (typeThreeQBox hI).RealContains (LeanSuffixAnalytic.typeThreeShape h) := by
  unfold typeThreeQBox LeanSuffixAnalytic.typeThreeShape
  exact LeanSuffixReflective.QInterval.realContains_sub
    (ScalarSuffixCertificate.QInterval.realContains_nsmul (by norm_num) hh)
    (LeanSuffixReflective.QInterval.realContains_point 1 (1 :  $\mathbb{R}$ ) |>.2 (by norm_num))
have hacos : (halfPiBox.sub asinRatioI).RealContains
  (arccos (LeanSuffixAnalytic.typeThreeShape h / lam)) :=
  ScalarSuffixCertificate.realContains_arccos_of_arcsin
    ScalarSuffixCertificate.piInterval_sound hasinRatio
have hangle : (typeThreeAngleBox lambdaInterval asinQI asinRatioI).RealContains
  (LeanSuffixAnalytic.typeThreeAngle lam h) := by
  unfold typeThreeAngleBox LeanSuffixAnalytic.typeThreeAngle
  exact LeanSuffixReflective.QInterval.realContains_add
    (ScalarSuffixCertificate.QInterval.realContains_mul hlam hacos) hasinQ
have hdelta : (typeThreeDeltaBox lambdaInterval uI dI hlambda).RealContains
  (LeanSuffixAnalytic.typeThreeDelta lam h) := by
  unfold typeThreeDeltaBox LeanSuffixAnalytic.typeThreeDelta
  exact LeanSuffixReflective.QInterval.realContains_sub
    (ScalarSuffixCertificate.QInterval.realContains_divPos hlambda hd hlam) hu
have hqPlusTwo := LeanSuffixReflective.QInterval.realContains_add hq
  (LeanSuffixReflective.QInterval.realContains_point 2 (2 :  $\mathbb{R}$ ) |>.2 (by norm_num))
have hnumArea := LeanSuffixReflective.QInterval.realContains_add
  (LeanSuffixReflective.QInterval.realContains_add hangle hhalfPi)
  (ScalarSuffixCertificate.QInterval.realContains_mul hqPlusTwo hdelta)
have hhSquare := ScalarSuffixCertificate.QInterval.realContains_mul hh hh
have harea := ScalarSuffixCertificate.QInterval.realContains_divPos hhSq hnumArea hhSquare
have hnumPerimeter := LeanSuffixReflective.QInterval.realContains_add
  (LeanSuffixReflective.QInterval.realContains_add hangle hhalfPi) hdelta
have hperimeterDiv := ScalarSuffixCertificate.QInterval.realContains_divPos
  hhPos hnumPerimeter hh
constructor
· convert harea using 1 <;>
  simp only [typeThreeAreaBox, LeanSuffixAnalytic.typeThreeArea] <;> ring
· convert ScalarSuffixCertificate.QInterval.realContains_nsmul
  (q := (2 :  $\mathbb{Q}$ )) (by norm_num) hperimeterDiv using 1 <;>
  simp only [typeThreePerimeterBox, LeanSuffixAnalytic.typeThreePerimeter] <;> ring

```

CompactCellCertificate.typeThree_atoms_of_exact_bounds

Classification: supplement

Source: CompactCellCertificate.lean:224-360

Declaration SHA-256: c51a322c3566880879bc58cb2483ec639ff97dddc9cd3e9ac9a18afac5600b84

Mathematical statement. Exact elementary bounds enclose the type-(iii) angular and radical atoms.

The theorem checks the branch and denominator conditions before combining atom intervals.

```

theorem typeThree_atoms_of_exact_bounds
  (lambdaInterval hI uI dI asinQI asinRatioI : QInterval)
  {lam h :  $\mathbb{R}$ }
  (hlam : lambdaInterval.RealContains lam)
  (hh : hI.RealContains h)
  (hlambdaLoPos :  $0 < \text{lambdaInterval.lo}$ )
  (hlambdaOne :  $1 < \text{lam}$ )
  (hqNonneg :  $0 \leq ((hI.\text{nsmul } 2 \text{ (by norm\_num)})).\text{sub}$ 
    (LeanSuffixReflective.QInterval.point 1)).lo)
  (hqLeOne :  $((hI.\text{nsmul } 2 \text{ (by norm\_num)})).\text{sub}$ 
    (LeanSuffixReflective.QInterval.point 1)).hi  $\leq 1$ )
  (huNonneg :  $0 \leq uI.\text{lo}$ )
  (huLower :  $uI.\text{lo}^2 \leq 1 - ((hI.\text{nsmul } 2 \text{ (by norm\_num)})).\text{sub}$ 
    (LeanSuffixReflective.QInterval.point 1)).hi^2)
  (huUpper :  $1 - ((hI.\text{nsmul } 2 \text{ (by norm\_num)})).\text{sub}$ 
    (LeanSuffixReflective.QInterval.point 1)).lo^2  $\leq uI.\text{hi}^2$ )
  (hdNonneg :  $0 \leq dI.\text{lo}$ )
  (hdLower :  $dI.\text{lo}^2 \leq \text{lambdaInterval.lo}^2 -$ 
     $((hI.\text{nsmul } 2 \text{ (by norm\_num)})).\text{sub}$ 
    (LeanSuffixReflective.QInterval.point 1)).hi^2)
  (hdUpper :  $\text{lambdaInterval.hi}^2 -$ 

```

```

    ((hI.nsmul 2 (by norm_num)).sub
      (LeanSuffixReflective.QInterval.point 1)).lo ^ 2 ≤ dI.hi ^ 2)
(hasinQBranch : 0 ≤ asinQI.lo ^ asinQI.hi ≤ halfPiBox.lo)
(hasinQLower : (ScalarSuffixCertificate.sinTaylor27Interval asinQI.lo).hi ≤
  ((hI.nsmul 2 (by norm_num)).sub
    (LeanSuffixReflective.QInterval.point 1)).lo)
(hasinQUpper : ((hI.nsmul 2 (by norm_num)).sub
  (LeanSuffixReflective.QInterval.point 1)).hi ≤
  (ScalarSuffixCertificate.sinTaylor27Interval asinQI.hi).lo)
(hasinRatioBranch : 0 ≤ asinRatioI.lo ^ asinRatioI.hi ≤ halfPiBox.lo)
(hasinRatioLower :
  (ScalarSuffixCertificate.sinTaylor27Interval asinRatioI.lo).hi ≤
  (ScalarSuffixCertificate.QInterval.divPos
    ((hI.nsmul 2 (by norm_num)).sub
      (LeanSuffixReflective.QInterval.point 1))
    lambdaInterval hlambdaLoPos).lo)
(hasinRatioUpper :
  (ScalarSuffixCertificate.QInterval.divPos
    ((hI.nsmul 2 (by norm_num)).sub
      (LeanSuffixReflective.QInterval.point 1))
    lambdaInterval hlambdaLoPos).hi ≤
  (ScalarSuffixCertificate.sinTaylor27Interval asinRatioI.hi).lo) :
uI.RealContains (sqrt (1 - LeanSuffixAnalytic.typeThreeShape h ^ 2)) ^
dI.RealContains (sqrt (lam ^ 2 - LeanSuffixAnalytic.typeThreeShape h ^ 2)) ^
asinQI.RealContains (arcsin (LeanSuffixAnalytic.typeThreeShape h)) ^
asinRatioI.RealContains
  (arcsin (LeanSuffixAnalytic.typeThreeShape h / lam)) := by
let qI : QInterval := (hI.nsmul 2 (by norm_num)).sub
  (LeanSuffixReflective.QInterval.point 1)
have hq : qI.RealContains (LeanSuffixAnalytic.typeThreeShape h) := by
  unfold LeanSuffixAnalytic.typeThreeShape
  exact LeanSuffixReflective.QInterval.realContains_sub
    (ScalarSuffixCertificate.QInterval.realContains_nsmul (by norm_num) hh)
    (LeanSuffixReflective.QInterval.realContains_point 1 (1 : ℝ) |>.2 (by norm_num))
have hqNonnegR : (0 : ℝ) ≤ qI.lo := by exact_mod_cast hqNonneg
have hqLeOneR : (qI.hi : ℝ) ≤ 1 := by exact_mod_cast hqLeOne
have hu : uI.RealContains
  (sqrt (1 - LeanSuffixAnalytic.typeThreeShape h ^ 2)) := by
  apply ScalarSuffixCertificate.QInterval.realContains_sqrt huNonneg
· have huLowerR : ((uI.lo : ℚ) : ℝ) ^ 2 ≤
  1 - ((qI.hi : ℚ) : ℝ) ^ 2 := by exact_mod_cast huLower
have hqhiNonneg : (0 : ℝ) ≤ qI.hi :=
  hqNonnegR.trans (by exact_mod_cast qI.ordered)
have hsq : LeanSuffixAnalytic.typeThreeShape h ^ 2 ≤
  (qI.hi : ℝ) ^ 2 := by
  nlinarith [mul_nonneg (sub_nonneg.mpr hq.2)
    (add_nonneg (hqNonnegR.trans hq.1) hqhiNonneg)]
  linarith
· have huUpperR : 1 - ((qI.lo : ℚ) : ℝ) ^ 2 ≤
  ((uI.hi : ℚ) : ℝ) ^ 2 := by exact_mod_cast huUpper
have hsq : (qI.lo : ℝ) ^ 2 ≤
  LeanSuffixAnalytic.typeThreeShape h ^ 2 := by
  nlinarith [mul_nonneg (sub_nonneg.mpr hq.1)
    (add_nonneg hqNonnegR (hqNonnegR.trans hq.1))]
  linarith
have hd : dI.RealContains
  (sqrt (lam ^ 2 - LeanSuffixAnalytic.typeThreeShape h ^ 2)) := by
  apply ScalarSuffixCertificate.QInterval.realContains_sqrt hdNonneg
· have hdLowerR : ((dI.lo : ℚ) : ℝ) ^ 2 ≤
  (lambdaInterval.lo : ℝ) ^ 2 - (qI.hi : ℝ) ^ 2 := by
  exact_mod_cast hdLower
have hlamLoNonneg : (0 : ℝ) ≤ lambdaInterval.lo := by
  exact_mod_cast hlambdaLoPos.le
have hlamSq : (lambdaInterval.lo : ℝ) ^ 2 ≤ lam ^ 2 := by
  nlinarith [mul_nonneg (sub_nonneg.mpr hlam.1)
    (add_nonneg hlamLoNonneg (hlamLoNonneg.trans hlam.1))]
have hqhiNonneg : (0 : ℝ) ≤ qI.hi :=
  hqNonnegR.trans (by exact_mod_cast qI.ordered)
have hqSq : LeanSuffixAnalytic.typeThreeShape h ^ 2 ≤
  (qI.hi : ℝ) ^ 2 := by
  nlinarith [mul_nonneg (sub_nonneg.mpr hq.2)
    (add_nonneg (hqNonnegR.trans hq.1) hqhiNonneg)]

```

```

linarith
· have hdUpperR : (lambdaInterval.hi : ℝ) ^ 2 -
  (qI.lo : ℝ) ^ 2 ≤ ((dI.hi : ℚ) : ℝ) ^ 2 := by
  exact_mod_cast hdUpper
have hlamLoNonneg : (0 : ℝ) ≤ lambdaInterval.lo := by
  exact_mod_cast hlambdaLoPos.le
have hlamNonneg : (0 : ℝ) ≤ lam := hlamLoNonneg.trans hlam.1
have hlamHiNonneg : (0 : ℝ) ≤ lambdaInterval.hi := by
  exact_mod_cast hlambdaLoPos.le.trans lambdaInterval.ordered
have hlamSq : lam ^ 2 ≤ (lambdaInterval.hi : ℝ) ^ 2 := by
  nlinarith [mul_nonneg (sub_nonneg.mpr hlam.2)
    (add_nonneg hlamNonneg hlamHiNonneg)]
have hqSq : (qI.lo : ℝ) ^ 2 ≤
  LeanSuffixAnalytic.typeThreeShape h ^ 2 := by
  nlinarith [mul_nonneg (sub_nonneg.mpr hq.1)
    (add_nonneg hqNonnegR (hqNonnegR.trans hq.1))]
linarith
have hasinQ : asinQI.RealContains
  (arcsin (LeanSuffixAnalytic.typeThreeShape h)) := by
  apply QInterval.realContains_arcsin_of_taylor_interval asinQI qI _ hq
· exact ⟨by linarith [hqNonnegR, hq.1], by linarith [hqLeOneR, hq.2]⟩
· exact principalAsinBranch_of_nonneg_of_hi_le_halfPiBox asinQI hasinQBranch
· simpa [qI] using hasinQLower
· simpa [qI] using hasinQUpper
have hratio : (ScalarSuffixCertificate.QInterval.divPos qI lambdaInterval
  hlambdaLoPos).RealContains (LeanSuffixAnalytic.typeThreeShape h / lam) :=
  ScalarSuffixCertificate.QInterval.realContains_divPos hlambdaLoPos hq hlam
have hasinRatio : asinRatioI.RealContains
  (arcsin (LeanSuffixAnalytic.typeThreeShape h / lam)) := by
  apply QInterval.realContains_arcsin_of_taylor_interval asinRatioI
  (ScalarSuffixCertificate.QInterval.divPos qI lambdaInterval hlambdaLoPos) _ hratio
· have hlamPos : 0 < lam := lt_trans (by norm_num) hlambdaOne
  have hq0 : 0 ≤ LeanSuffixAnalytic.typeThreeShape h := hqNonnegR.trans hq.1
  have hq1 : LeanSuffixAnalytic.typeThreeShape h ≤ 1 := hq.2.trans hqLeOneR
  exact ⟨(show (-1 : ℝ) ≤ 0 by norm_num).trans (div_nonneg hq0 hlamPos.le),
    (div_le_one hlamPos).2 (hq1.trans hlambdaOne.le)⟩
· exact principalAsinBranch_of_nonneg_of_hi_le_halfPiBox
  asinRatioI hasinRatioBranch
· simpa [qI] using hasinRatioLower
· simpa [qI] using hasinRatioUpper
exact ⟨hu, hd, hasinQ, hasinRatio⟩

/-- Reparametrization used to reuse the type-(iii) atom checker for type-(iv)
atoms: `typeThreeShape ((h + 1) / 2) = h`. -/

```

CompactCellCertificate.typeThree_fold_atom_enclosure

Classification: supplement

Source: CompactCellCertificate.lean:473-528

Declaration SHA-256: e0e739ea56f950a43598c09273d8fde5874e105be082082bb67c5da1055a66a6

Mathematical statement. Soundness of the type-(iii) fold interval assembled from atom bounds.

Every enclosed density and curvature produces a real K_3 value in the resulting box.

```

theorem typeThree_fold_atom_enclosure
  (lambdaInterval hI uI dI asinQI asinRatioI : QInterval)
  {lam h : ℝ} (hlam : lambdaInterval.RealContains lam)
  (hh : hI.RealContains h)
  (hatoms : uI.RealContains (sqrt (1 - LeanSuffixAnalytic.typeThreeShape h ^ 2)) ^
    dI.RealContains (sqrt (lam ^ 2 - LeanSuffixAnalytic.typeThreeShape h ^ 2)) ^
    asinQI.RealContains (arcsin (LeanSuffixAnalytic.typeThreeShape h)) ^
    asinRatioI.RealContains (arcsin (LeanSuffixAnalytic.typeThreeShape h / lam)))
  (hlambda : 0 < lambdaInterval.lo) (huPos : 0 < uI.lo)
  (hlambdaD : 0 < (lambdaInterval.mul dI).lo) :
  (typeThreeFoldBox lambdaInterval hI uI dI asinQI asinRatioI
    hlambda huPos hlambdaD).RealContains
    (LeanSuffixAnalytic.typeThreeFold lam h) := by
  rcases hatoms with ⟨hu, hd, hasinQ, hasinRatio⟩

```

```

have hq : (typeThreeQBox hI).RealContains
  (LeanSuffixAnalytic.typeThreeShape h) := by
  unfold typeThreeQBox LeanSuffixAnalytic.typeThreeShape
  exact LeanSuffixReflective.QInterval.realContains_sub
    (ScalarSuffixCertificate.QInterval.realContains_nsmul (by norm_num) hh)
    (LeanSuffixReflective.QInterval.realContains_point 1 (1 : ℝ) |>.2 (by norm_num))
have hhalfPi : halfPiBox.RealContains ( $\pi / 2$ ) := halfPiBox_sound
have hacos : (halfPiBox.sub asinRatioI).RealContains
  (arccos (LeanSuffixAnalytic.typeThreeShape h / lam)) :=
  ScalarSuffixCertificate.realContains_arccos_of_arcsin
  ScalarSuffixCertificate.piInterval_sound hasinRatio
have hangle : (typeThreeAngleBox lambdaInterval asinQI asinRatioI).RealContains
  (LeanSuffixAnalytic.typeThreeAngle lam h) := by
  unfold typeThreeAngleBox LeanSuffixAnalytic.typeThreeAngle
  exact LeanSuffixReflective.QInterval.realContains_add
    (ScalarSuffixCertificate.QInterval.realContains_mul hlam hacos) hasinQ
have hdelta : (typeThreeDeltaBox lambdaInterval uI dI hlambda).RealContains
  (LeanSuffixAnalytic.typeThreeDelta lam h) := by
  unfold typeThreeDeltaBox LeanSuffixAnalytic.typeThreeDelta
  exact LeanSuffixReflective.QInterval.realContains_sub
    (ScalarSuffixCertificate.QInterval.realContains_divPos hlambda hd hlam) hu
have hnumOne := LeanSuffixReflective.QInterval.realContains_add
  (LeanSuffixReflective.QInterval.realContains_point 1 (1 : ℝ) |>.2 (by norm_num)) hq
have hdivOne := ScalarSuffixCertificate.QInterval.realContains_divPos
  huPos hnumOne hu
have hlamSq := ScalarSuffixCertificate.QInterval.realContains_mul hlam hlam
have hnumTwo := LeanSuffixReflective.QInterval.realContains_add hlamSq hq
have hlamD := ScalarSuffixCertificate.QInterval.realContains_mul hlam hd
have hdivTwo := ScalarSuffixCertificate.QInterval.realContains_divPos
  hlambdaD hnumTwo hlamD
have hfirst := ScalarSuffixCertificate.QInterval.realContains_nsmul
  (q := (4 : ℚ)) (by norm_num)
  (ScalarSuffixCertificate.QInterval.realContains_mul hh
    (LeanSuffixReflective.QInterval.realContains_sub hdivOne hdivTwo))
have hsecond := ScalarSuffixCertificate.QInterval.realContains_nsmul
  (q := (2 : ℚ)) (by norm_num)
  (LeanSuffixReflective.QInterval.realContains_add
    (LeanSuffixReflective.QInterval.realContains_add hangle hhalfPi) hdelta)
convert LeanSuffixReflective.QInterval.realContains_sub hfirst hsecond using 1 <>
  simp only [typeThreeFoldBox, LeanSuffixAnalytic.typeThreeFold] <> ring

/-- Formula enclosure assembled from type-(iii) atoms. -/

```

CompactFirstCell.QInterval

Classification: supplement

Source: CompactFirstCell.lean:19-23

Declaration SHA-256: c522fb7a23b8535f69b64bd9659ad4aed8fc8dc5b89426617a6ca4e04ee665a3

Mathematical statement. Shared rational interval type in the first compact cell.

The first cell uses the same real-containment semantics as the generated cells.

```

abbrev QInterval := ScalarSuffixCertificate.QInterval

namespace QInterval

/-- A convenient exact-rational wrapper around the Taylor inverse-sine rule. -/

```

CompactFirstCell.QInterval.realContains_arcsin_of_taylor

Classification: supplement

Source: CompactFirstCell.lean:24-38

Declaration SHA-256: 89a8b20d50a5cd5fff8193c5fcee59fcf1459ec24caa265aff4f074622f22d58

Mathematical statement. A first-cell inverse-sine enclosure justified by Taylor bounds.

The principal branch and endpoint sine inequalities certify the inverse value.

```
theorem realContains_arcsin_of_taylor
  (a : QInterval) (x : ℚ)
  (hx : (-1 : ℚ) ≤ x ∧ x ≤ 1)
  (hbranch : (a.lo : ℝ) ∈ Icc (-(π / 2)) (π / 2) ∧
    (a.hi : ℝ) ∈ Icc (-(π / 2)) (π / 2))
  (hlo : (ScalarSuffixCertificate.sinTaylor27Interval a.lo).hi ≤ x)
  (hhi : x ≤ (ScalarSuffixCertificate.sinTaylor27Interval a.hi).lo) :
  a.RealContains (arcsin (x : ℝ)) :=
  CompactCellCertificate.QInterval.realContains_arcsin_of_taylor
  a x hx hbranch hlo hhi

end QInterval

/-- Interval-input version of the inverse-sine Taylor wrapper. This is useful
when the inverse-sine argument is itself obtained by exact interval division. -/
```

CompactFirstCell.h3Center

Classification: supplement

Source: CompactFirstCell.lean:70-71

Declaration SHA-256: 56aacc195857950b6ae7b136dff6ad4341c307ca690bae0f998846b98084779a

Mathematical statement. Exact midpoint of the first type-(iii) slab.

Centered enclosures expand around this rational point.

```
def h3Center : ℚ := (h3Interval.lo + h3Interval.hi) / 2
```

CompactFirstCell.h3Interval

Classification: supplement

Source: CompactFirstCell.lean:62-66

Declaration SHA-256: cfb76a49841952dce830375124f9a01e8a81c790ef5f2c6d14a5bb739e437d32

Mathematical statement. Rational type-(iii) equal-area curvature slab for box zero.

It is $[132572791017099/140737488355328, 133157808072187/140737488355328]$.

```
def h3Interval : QInterval :=
  (132572791017099 / 140737488355328,
   133157808072187 / 140737488355328, by norm_num)

/-- Exact midpoint used by the centered type-(iv) checker evaluation. -/
```

CompactFirstCell.h4Center

Classification: supplement

Source: CompactFirstCell.lean:67-69

Declaration SHA-256: fcc3fc93d1f6d9a14b3c9f8cd836a4da459d6e2e9d5879a858b078d451152165

Mathematical statement. Exact midpoint of the first type-(iv) slab.

The center is the average of its rational endpoints.

```
def h4Center : ℚ := (h4Interval.lo + h4Interval.hi) / 2

/-- Exact midpoint used by the centered type-(iii) checker evaluation. -/
```

CompactFirstCell.h4Interval

Classification: supplement

Source: CompactFirstCell.lean:57-61

Declaration SHA-256: 21261a7072ad9b4e7e79f4c2420b3ea2c0b04219a9a6039f38c82d6959dee705

Mathematical statement. Rational stationary type-(iv) curvature slab for box zero.

It is $[34445267662685/35184372088832, 275599301332823/281474976710656]$.

```
def h4Interval : QInterval :=
  (34445267662685 / 35184372088832,
   275599301332823 / 281474976710656, by norm_num)

/-- Type-(iii) equal-area root slab of box 0. -/
```

CompactFirstCell.lambdaInterval

Classification: supplement

Source: CompactFirstCell.lean:53-56

Declaration SHA-256: 33288724289e17e6b58f251168fcac1ba8bb868b76aed0ce4b2d7a7825c0065c

Mathematical statement. Density interval of compact box zero.

It is $[33/32, 236601/229376]$.

```
def lambdaInterval : QInterval :=
  (33 / 32, 236601 / 229376, by norm_num)

/-- Type-(iv) root slab of box 0. -/
```

CompactFirstCell.realContains_arcsin_of_taylor_interval

Classification: supplement

Source: CompactFirstCell.lean:39-52

Declaration SHA-256: 04625f4bdbd8d2aa4776f4b5c665d47c58bb244e2be98c92835ce4982a969a1c

Mathematical statement. The first-cell arcsine enclosure extends to every enclosed input.

Monotonicity propagates certified bounds across the entire source interval.

```
theorem realContains_arcsin_of_taylor_interval
  (a xInterval : QInterval) (x : ℝ)
  (hx : xInterval.RealContains x)
  (hxUnit : (-1 : ℝ) ≤ x ∧ x ≤ 1)
  (hbranch : (a.lo : ℝ) ∈ Icc (-(π / 2)) (π / 2) ∧
    (a.hi : ℝ) ∈ Icc (-(π / 2)) (π / 2))
  (hlo : (ScalarSuffixCertificate.sinTaylor27Interval a.lo).hi ≤ xInterval.lo)
  (hhi : xInterval.hi ≤
    (ScalarSuffixCertificate.sinTaylor27Interval a.hi).lo) :
  a.RealContains (arcsin x) :=
  CompactCellCertificate.QInterval.realContains_arcsin_of_taylor_interval
  a xInterval x hx hxUnit hbranch hlo hhi

/-- Lambda interval of box 0. -/
```

CompactFirstCell.typeFourArea_centered_enclosure

Classification: supplement

Source: CompactFirstCell.lean:422-439

Declaration SHA-256: 23fd914cbcc4badfdce7ba2492c48ade47c8348ae2e3255286932df9b5b0b656

Mathematical statement. Centered type-(iv) area enclosure used by box zero.

A center value and a shared-point fold derivative enclose A_4 over the slab.

```
theorem typeFourArea_centered_enclosure {lam h : ℝ}
  (hlam : lambdaInterval.RealContains lam)
  (hh : h4Interval.RealContains h)
  (base slope displacement : QInterval)
  (hbase : base.RealContains
    (LeanSuffixAnalytic.typeFourArea lam (h4Center : ℝ)))
  (hslope : ∀ x, h4Interval.RealContains x →
    slope.RealContains
      (LeanSuffixAnalytic.typeFourFold lam x / x ^ 3))
  (hdisplacement : displacement.RealContains (h - (h4Center : ℝ))) :
  (base.add (slope.mul displacement)).RealContains
    (LeanSuffixAnalytic.typeFourArea lam h) :=
  CompactCellCertificate.typeFourArea_centered_enclosure
    lambdaInterval h4Interval h4Center hlam hh h4Center_mem
    (lambda_one_lt hlam) (fun hx => h4_slab_regular hx)
    base slope displacement hbase hslope hdisplacement

/-- Box-0 specialization of the centered type-(iv) perimeter enclosure. -/
```

CompactFirstCell.typeFourFold_lowerFace_neg

Classification: supplement

Source: CompactFirstCell.lean:182-300

Declaration SHA-256: dcdb660b276bba4f208c5ded09cca5851f937ad4f80478890332c45cce47b8a0

Mathematical statement. K_4 is negative on the first box's lower curvature face.

The strict sign holds uniformly throughout the closed density interval.

```
theorem typeFourFold_lowerFace_neg {lam : ℝ}
  (hlam : lambdaInterval.RealContains lam) :
  LeanSuffixAnalytic.typeFourFold lam (h4Interval.lo : ℝ) < 0 := by
  let h : ℚ := h4Interval.lo
  have hu : u4LowInterval.RealContains (sqrt (1 - (h : ℝ) ^ 2)) := by
    apply ScalarSuffixCertificate.QInterval.realContains_sqrt
    · norm_num [u4LowInterval]
    · norm_num [u4LowInterval, h, h4Interval]
    · norm_num [u4LowInterval, h, h4Interval]
  have hd : d4LowInterval.RealContains (sqrt (lam ^ 2 - (h : ℝ) ^ 2)) := by
    apply ScalarSuffixCertificate.QInterval.realContains_sqrt
    · norm_num [d4LowInterval]
    · norm_num [lambdaInterval, LeanSuffixReflective.QInterval.RealContains] at hlam
      norm_num [d4LowInterval, h, h4Interval]
      nlinarith
    · norm_num [lambdaInterval, LeanSuffixReflective.QInterval.RealContains] at hlam
      norm_num [d4LowInterval, h, h4Interval]
      nlinarith
  have hasinH : asinH4LowInterval.RealContains (arcsin (h : ℝ)) := by
    apply QInterval.realContains_arcsin_of_taylor
    · norm_num [h, h4Interval]
    · constructor <=> constructor <=>
      norm_num [asinH4LowInterval] <=>
      have hp := Real.pi_gt_three <=> nlinarith
    · norm_num [ScalarSuffixCertificate.sinTaylor27Interval,
      ScalarSuffixCertificate.sinTaylor27, asinH4LowInterval, h, h4Interval,
      Nat.factorial]
```



```

· norm_num [ScalarSuffixCertificate.sinTaylor27Interval,
  ScalarSuffixCertificate.sinTaylor27, asinH4LowInterval, h, h4Interval,
  Nat.factorial]
have hlamLoPos : 0 < lambdaInterval.lo := by norm_num [lambdaInterval]
have hratio : (ScalarSuffixCertificate.QInterval.divPos
  (LeanSuffixReflective.QInterval.point h) lambdaInterval hlamLoPos).RealContains
  ((h : ℝ) / lam) := by
  exact ScalarSuffixCertificate.QInterval.realContains_divPos hlamLoPos
  (LeanSuffixReflective.QInterval.realContains_point h (h : ℝ) |>.2 rfl) hlam
have hasinRatio : asinRatioLowInterval.RealContains (arcsin ((h : ℝ) / lam)) := by
  apply ScalarSuffixCertificate.realContains_arcsin_of_sin_bounds
· constructor
  · norm_num [h, h4Interval, lambdaInterval,
    ScalarSuffixCertificate.QInterval.divPos,
    ScalarSuffixCertificate.QInterval.recipPos,
    ScalarSuffixCertificate.QInterval.mul,
    LeanSuffixReflective.QInterval.point,
    LeanSuffixReflective.QInterval.RealContains] at hratio ⊢
    linarith [hratio.1]
  · norm_num [h, h4Interval, lambdaInterval,
    ScalarSuffixCertificate.QInterval.divPos,
    ScalarSuffixCertificate.QInterval.recipPos,
    ScalarSuffixCertificate.QInterval.mul,
    LeanSuffixReflective.QInterval.point,
    LeanSuffixReflective.QInterval.RealContains] at hratio ⊢
    linarith [hratio.2]
· constructor <=> constructor <=>
  norm_num [asinRatioLowInterval] <=>
  have hp := Real.pi_gt_three <=> nlinarith
· refine (ScalarSuffixCertificate.sinTaylor27Interval_sound
  asinRatioLowInterval.lo).2.trans ?_
norm_num [ScalarSuffixCertificate.sinTaylor27Interval,
  ScalarSuffixCertificate.sinTaylor27, asinRatioLowInterval,
  lambdaInterval, h, h4Interval,
  ScalarSuffixCertificate.QInterval.divPos,
  ScalarSuffixCertificate.QInterval.recipPos,
  ScalarSuffixCertificate.QInterval.mul,
  LeanSuffixReflective.QInterval.point,
  LeanSuffixReflective.QInterval.RealContains, Nat.factorial] at hratio ⊢
  linarith [hratio.1]
· have hupper : ((h : ℝ) / lam) ≤
  (ScalarSuffixCertificate.sinTaylor27Interval asinRatioLowInterval.hi).lo := by
  norm_num [ScalarSuffixCertificate.sinTaylor27Interval,
    ScalarSuffixCertificate.sinTaylor27, asinRatioLowInterval,
    lambdaInterval, h, h4Interval,
    ScalarSuffixCertificate.QInterval.divPos,
    ScalarSuffixCertificate.QInterval.recipPos,
    ScalarSuffixCertificate.QInterval.mul,
    LeanSuffixReflective.QInterval.point,
    LeanSuffixReflective.QInterval.RealContains, Nat.factorial] at hratio ⊢
    linarith [hratio.2]
  exact hupper.trans
  (ScalarSuffixCertificate.sinTaylor27Interval_sound
    asinRatioLowInterval.hi).1
have hacos : acosRatioLowInterval.RealContains (arccos ((h : ℝ) / lam)) := by
  exact ScalarSuffixCertificate.realContains_arccos_of_arcsin
  ScalarSuffixCertificate.piInterval_sound hasinRatio
have hinvU := ScalarSuffixCertificate.QInterval.realContains_recipPos
  (i := u4LowInterval) (by norm_num [u4LowInterval]) hu
have hlamDivD := ScalarSuffixCertificate.QInterval.realContains_divPos
  (i := lambdaInterval) (j := d4LowInterval) (by norm_num [d4LowInterval]) hlam hd
let e : ScalarSuffixCertificate.CertificateExpr :=
  .mul (.rational 4)
  (.add
    (.mul (.rational h)
      (.add (.atom (1 / sqrt (1 - (h : ℝ) ^ 2))
        (u4LowInterval.recipPos (by norm_num [u4LowInterval])) hinvU)
      (.neg (.atom (lam / sqrt (lam ^ 2 - (h : ℝ) ^ 2))
        (lambdaInterval.divPos d4LowInterval (by norm_num [d4LowInterval]))
        hlamDivD))))))
  (.neg (.add
    (.mul (.atom lam lambdaInterval hlam)

```



```

      (.atom (arccos ((h : ℝ) / lam)) acosRatioLowInterval hacos))
      (.atom (arcsin (h : ℝ)) asinH4LowInterval hasinH)))
have he : e.value < 0 := by
  apply ScalarSuffixCertificate.CertificateExpr.value_neg_of_hi_neg
  norm_num [e, ScalarSuffixCertificate.CertificateExpr.enclosure,
    ScalarSuffixCertificate.QInterval.mul,
    LeanSuffixReflective.QInterval.add, LeanSuffixReflective.QInterval.neg,
    LeanSuffixReflective.QInterval.point,
    LeanSuffixReflective.QInterval.sub, LeanSuffixReflective.QInterval.nsmul,
    ScalarSuffixCertificate.QInterval.recipPos,
    ScalarSuffixCertificate.QInterval.divPos, lambdaInterval, u4LowInterval,
    d4LowInterval, asinH4LowInterval, asinRatioLowInterval,
    acosRatioLowInterval, ScalarSuffixCertificate.piInterval, h, h4Interval]
convert he using 1 <=>
  simp only [e, ScalarSuffixCertificate.CertificateExpr.value,
    LeanSuffixAnalytic.typeFourFold, LeanSuffixAnalytic.typeFourAngle, h] <=>
  ring

/-- The upper face of the exact box-0 type-(iv) slab has positive fold value,
uniformly over the whole exact lambda interval. -/

```

CompactFirstCell.typeFourFold_upperFace_pos

Classification: supplement

Source: CompactFirstCell.lean:301-421

Declaration SHA-256: 2a5ed0fdf81ba4e8918a25fc36a2303fc5b7fea92508d672753fd14471fc1b6f

Mathematical statement. K4 is positive on the first box's upper curvature face.

Together with the lower sign, this brackets the stationary curvature.

```

theorem typeFourFold_upperFace_pos {lam : ℝ}
  (hlam : lambdaInterval.RealContains lam) :
  0 < LeanSuffixAnalytic.typeFourFold lam (h4Interval.hi : ℝ) := by
let h : ℚ := h4Interval.hi
have hu : u4HighInterval.RealContains (sqrt (1 - (h : ℝ) ^ 2)) := by
  apply ScalarSuffixCertificate.QInterval.realContains_sqrt
  · norm_num [u4HighInterval]
  · norm_num [u4HighInterval, h, h4Interval]
  · norm_num [u4HighInterval, h, h4Interval]
have hd : d4HighInterval.RealContains (sqrt (lam ^ 2 - (h : ℝ) ^ 2)) := by
  apply ScalarSuffixCertificate.QInterval.realContains_sqrt
  · norm_num [d4HighInterval]
  · norm_num [lambdaInterval, LeanSuffixReflective.QInterval.RealContains] at hlam
    norm_num [d4HighInterval, h, h4Interval]
    nlinarith
  · norm_num [lambdaInterval, LeanSuffixReflective.QInterval.RealContains] at hlam
    norm_num [d4HighInterval, h, h4Interval]
    nlinarith
have hasinH : asinH4HighInterval.RealContains (arcsin (h : ℝ)) := by
  apply QInterval.realContains_arcsin_of_taylor
  · norm_num [h, h4Interval]
  · constructor <=> constructor <=>
      norm_num [asinH4HighInterval] <=>
        have hp := Real.pi_gt_three <=> nlinarith
  · norm_num [ScalarSuffixCertificate.sinTaylor27Interval,
    ScalarSuffixCertificate.sinTaylor27, asinH4HighInterval, h, h4Interval,
    Nat.factorial]
  · norm_num [ScalarSuffixCertificate.sinTaylor27Interval,
    ScalarSuffixCertificate.sinTaylor27, asinH4HighInterval, h, h4Interval,
    Nat.factorial]
have hlamLoPos : 0 < lambdaInterval.lo := by norm_num [lambdaInterval]
have hratio : (ScalarSuffixCertificate.QInterval.divPos
  (LeanSuffixReflective.QInterval.point h) lambdaInterval hlamLoPos).RealContains
  ((h : ℝ) / lam) := by
  exact ScalarSuffixCertificate.QInterval.realContains_divPos hlamLoPos
  (LeanSuffixReflective.QInterval.realContains_point h (h : ℝ) |>.2 rfl) hlam
have hasinRatio : asinRatioHighInterval.RealContains (arcsin ((h : ℝ) / lam)) := by
  apply ScalarSuffixCertificate.realContains_arcsin_of_sin_bounds

```

```

· constructor
· norm_num [h, h4Interval, lambdaInterval,
  ScalarSuffixCertificate.QInterval.divPos,
  ScalarSuffixCertificate.QInterval.recipPos,
  ScalarSuffixCertificate.QInterval.mul,
  LeanSuffixReflective.QInterval.point,
  LeanSuffixReflective.QInterval.RealContains] at hratio ⊢
linarith [hratio.1]
· norm_num [h, h4Interval, lambdaInterval,
  ScalarSuffixCertificate.QInterval.divPos,
  ScalarSuffixCertificate.QInterval.recipPos,
  ScalarSuffixCertificate.QInterval.mul,
  LeanSuffixReflective.QInterval.point,
  LeanSuffixReflective.QInterval.RealContains] at hratio ⊢
linarith [hratio.2]
· constructor <=> constructor <=>
  norm_num [asinRatioHighInterval] <=>
  have hp := Real.pi_gt_three <=> nlinarith
· refine (ScalarSuffixCertificate.sinTaylor27Interval_sound
  asinRatioHighInterval.lo).2.trans ?_
norm_num [ScalarSuffixCertificate.sinTaylor27Interval,
  ScalarSuffixCertificate.sinTaylor27, asinRatioHighInterval,
  lambdaInterval, h, h4Interval,
  ScalarSuffixCertificate.QInterval.divPos,
  ScalarSuffixCertificate.QInterval.recipPos,
  ScalarSuffixCertificate.QInterval.mul,
  LeanSuffixReflective.QInterval.point,
  LeanSuffixReflective.QInterval.RealContains, Nat.factorial] at hratio ⊢
linarith [hratio.1]
· have hupper : ((h : ℝ) / lam) ≤
  (ScalarSuffixCertificate.sinTaylor27Interval asinRatioHighInterval.hi).lo := by
norm_num [ScalarSuffixCertificate.sinTaylor27Interval,
  ScalarSuffixCertificate.sinTaylor27, asinRatioHighInterval,
  lambdaInterval, h, h4Interval,
  ScalarSuffixCertificate.QInterval.divPos,
  ScalarSuffixCertificate.QInterval.recipPos,
  ScalarSuffixCertificate.QInterval.mul,
  LeanSuffixReflective.QInterval.point,
  LeanSuffixReflective.QInterval.RealContains, Nat.factorial] at hratio ⊢
linarith [hratio.2]
exact hupper.trans
  (ScalarSuffixCertificate.sinTaylor27Interval_sound
  asinRatioHighInterval.hi).1
have hacos : acosRatioHighInterval.RealContains (arccos ((h : ℝ) / lam)) := by
exact ScalarSuffixCertificate.realContains_arccos_of_arcsin
  ScalarSuffixCertificate.piInterval_sound hasinRatio
have hinvU := ScalarSuffixCertificate.QInterval.realContains_recipPos
  (i := u4HighInterval) (by norm_num [u4HighInterval]) hu
have hlamDivD := ScalarSuffixCertificate.QInterval.realContains_divPos
  (i := lambdaInterval) (j := d4HighInterval) (by norm_num [d4HighInterval]) hlam hd
let e : ScalarSuffixCertificate.CertificateExpr :=
  .mul (.rational 4)
  (.add
    (.mul (.rational h)
      (.add (.atom (1 / sqrt (1 - (h : ℝ) ^ 2))
        (u4HighInterval.recipPos (by norm_num [u4HighInterval])) hinvU)
      (.neg (.atom (lam / sqrt (lam ^ 2 - (h : ℝ) ^ 2))
        (lambdaInterval.divPos d4HighInterval (by norm_num [d4HighInterval]))
        hlamDivD))))))
    (.neg (.add
      (.mul (.atom lam lambdaInterval hlam)
        (.atom (arccos ((h : ℝ) / lam)) acosRatioHighInterval hacos))
      (.atom (arcsin (h : ℝ)) asinH4HighInterval hasinH))))))
have he : 0 < e.value := by
apply ScalarSuffixCertificate.QInterval.positive_of_realContains e.sound
norm_num [e, ScalarSuffixCertificate.CertificateExpr.enclosure,
  ScalarSuffixCertificate.QInterval.mul,
  LeanSuffixReflective.QInterval.add, LeanSuffixReflective.QInterval.neg,
  LeanSuffixReflective.QInterval.point,
  LeanSuffixReflective.QInterval.sub, LeanSuffixReflective.QInterval.nsmul,
  ScalarSuffixCertificate.QInterval.recipPos,
  ScalarSuffixCertificate.QInterval.divPos, lambdaInterval, u4HighInterval,

```

```

    d4HighInterval, asinH4HighInterval, asinRatioHighInterval,
    acosRatioHighInterval, ScalarSuffixCertificate.piInterval, h, h4Interval]
convert he using 1 <=>
simp only [e, ScalarSuffixCertificate.CertificateExpr.value,
  LeanSuffixAnalytic.typeFourFold, LeanSuffixAnalytic.typeFourAngle, h] <=>
ring

/-- Box-0 specialization of the centered type-(iv) area enclosure used by
`centered_type4`. The endpoint interval is a conclusion of the generic
mean-value soundness theorem; only the center and derivative leaf enclosures
remain to be checked by exact arithmetic. -/

```

CompactFirstCell.typeFourPerimeter_centered_enclosure

Classification: supplement

Source: CompactFirstCell.lean:440-458

Declaration SHA-256: af4df9fa49b636ddab2f85af4f4c2bbf86e908c96cd4cc740bea472cf83b8977

Mathematical statement. Centered type-(iv) perimeter enclosure for box zero.

The derivative is K_4/h^2 , evaluated without breaking repeated-variable correlation.

```

theorem typeFourPerimeter_centered_enclosure {lam h : ℝ}
  (hlam : lambdaInterval.RealContains lam)
  (hh : h4Interval.RealContains h)
  (base slope displacement : QInterval)
  (hbase : base.RealContains
    (LeanSuffixAnalytic.typeFourPerimeter lam (h4Center : ℝ)))
  (hslope : ∀ x, h4Interval.RealContains x →
    slope.RealContains
      (LeanSuffixAnalytic.typeFourFold lam x / x ^ 2))
  (hdisplacement : displacement.RealContains (h - (h4Center : ℝ))) :
  (base.add (slope.mul displacement)).RealContains
    (LeanSuffixAnalytic.typeFourPerimeter lam h) :=
CompactCellCertificate.typeFourPerimeter_centered_enclosure
  lambdaInterval h4Interval h4Center hlam hh h4Center_mem
  (lambda_one_lt hlam) (fun hx => h4_slab_regular hx)
  base slope displacement hbase hslope hdisplacement

/-- Box-0 specialization of the centered type-(iii) area enclosure used by
`centered_type3`. -/

```

CompactFirstCell.typeThreeArea_lowerFace_minus_stationaryTypeFour_pos

Classification: supplement

Source: CompactFirstCell.lean:1527-1550

Declaration SHA-256: 38de632d8808d27f117549af1dc1f71c9d50996e4ee9a767cd56ea761fcaac99

Mathematical statement. At the lower type-(iii) face, A3 exceeds stationary type-(iv) area.

This is the positive side of the equal-area root bracket.

```

theorem typeThreeArea_lowerFace_minus_stationaryTypeFour_pos
  {lam h4 : ℝ}
  (hlam : lambdaInterval.RealContains lam)
  (hh4 : h4Interval.RealContains h4)
  (_hstationary : LeanSuffixAnalytic.typeFourFold lam h4 = 0) :
  0 < LeanSuffixAnalytic.typeThreeArea lam (h3Interval.lo : ℝ) -
    LeanSuffixAnalytic.typeFourArea lam h4 := by
have hthree := typeThreeArea_lower_enclosure hlam
have hfour := (typeFour_centered_enclosures hlam hh4).1
norm_num [typeThreeLowerAreaInterval, typeFourAreaCenteredInterval,
  typeFourCenterAreaInterval, typeFourAreaSlopeInterval,
  typeFourFoldCellInterval, h4CubeInterval, h4SquareInterval,
  h4DisplacementInterval, h4Interval, h4Center,

```

```

    ScalarSuffixCertificate.QInterval.mul,
    ScalarSuffixCertificate.QInterval.divPos,
    ScalarSuffixCertificate.QInterval.recipPos,
    LeanSuffixReflective.QInterval.RealContains,
    LeanSuffixReflective.QInterval.add, LeanSuffixReflective.QInterval.neg,
    LeanSuffixReflective.QInterval.sub, LeanSuffixReflective.QInterval.point]
  at hthree hfour ⊢
linarith

/-- The upper type-(iii) face lies strictly below the stationary type-(iv)
area, uniformly on the exact lambda box. -/

```

CompactFirstCell.typeThreeArea_upperFace_minus_stationaryTypeFour_neg

Classification: supplement

Source: CompactFirstCell.lean:1551-1574

Declaration SHA-256: 7ae9709ef08dafd6acf18487ff86ef2f78b6f5204c9d53ab45605c720440adb8

Mathematical statement. At the upper type-(iii) face, A_3 is below stationary type-(iv) area.

Strict decrease and this negative side locate the equal-area root inside the slab.

```

theorem typeThreeArea_upperFace_minus_stationaryTypeFour_neg
  {lam h4 : ℝ}
  (hlam : lambdaInterval.RealContains lam)
  (hh4 : h4Interval.RealContains h4)
  (_hstationary : LeanSuffixAnalytic.typeFourFold lam h4 = 0) :
  LeanSuffixAnalytic.typeThreeArea lam (h3Interval.hi : ℝ) -
    LeanSuffixAnalytic.typeFourArea lam h4 < 0 := by
have hthree := typeThreeArea_upper_enclosure hlam
have hfour := (typeFour_centered_enclosures hlam hh4).1
norm_num [typeThreeUpperAreaInterval, typeFourAreaCenteredInterval,
  typeFourCenterAreaInterval, typeFourAreaSlopeInterval,
  typeFourFoldCellInterval, h4CubeInterval, h4SquareInterval,
  h4DisplacementInterval, h4Interval, h4Center,
  ScalarSuffixCertificate.QInterval.mul,
  ScalarSuffixCertificate.QInterval.divPos,
  ScalarSuffixCertificate.QInterval.recipPos,
  LeanSuffixReflective.QInterval.RealContains,
  LeanSuffixReflective.QInterval.add, LeanSuffixReflective.QInterval.neg,
  LeanSuffixReflective.QInterval.sub, LeanSuffixReflective.QInterval.point]
  at hthree hfour ⊢
linarith

/-- The centered perimeter boxes retain a strict gap everywhere in the two
recorded root slabs. -/

```

CompactFirstCell.typeThreeFold_cell_enclosure

Classification: supplement

Source: CompactFirstCell.lean:1018-1122

Declaration SHA-256: c9da53af261130db3061bf11c0e74ee8196fae038a2277ad282922335d64dc0a

Mathematical statement. Exact enclosure of K_3 over the first compact box.

Its upper endpoint is used to certify the descending branch.

```

theorem typeThreeFold_cell_enclosure {lam h : ℝ}
  (hlam : lambdaInterval.RealContains lam)
  (hh : h3Interval.RealContains h) :
  typeThreeFoldCellInterval.RealContains
    (LeanSuffixAnalytic.typeThreeFold lam h) := by
rcases typeThreeCell_atoms hlam hh with ⟨hu, hd, hasinQ, hasinRatio⟩
let qI : QInterval :=
  (h3Interval.nsmul 2 (by norm_num)).sub

```

```

(LeanSuffixReflective.QInterval.point 1)
have hq : qI.RealContains (LeanSuffixAnalytic.typeThreeShape h) := by
  unfold LeanSuffixAnalytic.typeThreeShape
  exact LeanSuffixReflective.QInterval.realContains_sub
    (ScalarSuffixCertificate.QInterval.realContains_nsmul (by norm_num) hh)
    (LeanSuffixReflective.QInterval.realContains_point 1 (1 : ℝ) |>.2 (by norm_num))
let halfPi : QInterval :=
  ScalarSuffixCertificate.piInterval.nsmul (1 / 2) (by norm_num)
have hhalfPi : halfPi.RealContains ( $\pi / 2$ ) := by
  convert ScalarSuffixCertificate.QInterval.realContains_nsmul
    (q := (1 / 2 : ℚ)) (by norm_num)
  ScalarSuffixCertificate.piInterval_sound using 1 <=> ring
let acosI : QInterval := halfPi.sub asinRatio3CellInterval
have hacos : acosI.RealContains
  (arccos (LeanSuffixAnalytic.typeThreeShape h / lam)) := by
  exact ScalarSuffixCertificate.realContains_arccos_of_arcsin
  ScalarSuffixCertificate.piInterval_sound hasinRatio
have hangle : (lambdaInterval.mul acosI).add asinQ3CellInterval |>.RealContains
  (LeanSuffixAnalytic.typeThreeAngle lam h) := by
  unfold LeanSuffixAnalytic.typeThreeAngle
  exact LeanSuffixReflective.QInterval.realContains_add
    (ScalarSuffixCertificate.QInterval.realContains_mul hlam hacos) hasinQ
have hlamPos : 0 < lambdaInterval.lo := by norm_num [lambdaInterval]
have huPos : 0 < u3CellInterval.lo := by norm_num [u3CellInterval]
have hdPos : 0 < d3CellInterval.lo := by norm_num [d3CellInterval]
have hdelta :
  ((d3CellInterval.divPos lambdaInterval hlamPos).sub u3CellInterval).RealContains
    (LeanSuffixAnalytic.typeThreeDelta lam h) := by
  unfold LeanSuffixAnalytic.typeThreeDelta
  exact LeanSuffixReflective.QInterval.realContains_sub
    (ScalarSuffixCertificate.QInterval.realContains_divPos hlamPos hd hlam) hu
have hnumOne :
  ((LeanSuffixReflective.QInterval.point 1).add qI).RealContains
    (1 + LeanSuffixAnalytic.typeThreeShape h) :=
  LeanSuffixReflective.QInterval.realContains_add
    (LeanSuffixReflective.QInterval.realContains_point 1 (1 : ℝ) |>.2 (by norm_num))
  hq
have hdivOne :=
  ScalarSuffixCertificate.QInterval.realContains_divPos huPos hnumOne hu
have hlamSq := ScalarSuffixCertificate.QInterval.realContains_mul hlam hlam
have hnumTwo :
  ((lambdaInterval.mul lambdaInterval).add qI).RealContains
    (lam ^ 2 + LeanSuffixAnalytic.typeThreeShape h) := by
  convert LeanSuffixReflective.QInterval.realContains_add hlamSq hq using 1 <=> ring
have hlamD := ScalarSuffixCertificate.QInterval.realContains_mul hlam hd
have hlamDPos : 0 < (lambdaInterval.mul d3CellInterval).lo := by
  norm_num [lambdaInterval, d3CellInterval,
    ScalarSuffixCertificate.QInterval.mul]
have hdivTwo :=
  ScalarSuffixCertificate.QInterval.realContains_divPos hlamDPos hnumTwo hlamD
let e : ScalarSuffixCertificate.CertificateExpr :=
  .add
    (.mul (.rational 4)
      (.mul (.atom h h3Interval hh)
        (.add
          (.atom
            ((1 + LeanSuffixAnalytic.typeThreeShape h) /
              sqrt (1 - LeanSuffixAnalytic.typeThreeShape h ^ 2))
            (ScalarSuffixCertificate.QInterval.divPos
              ((LeanSuffixReflective.QInterval.point 1).add qI)
              u3CellInterval huPos) hdivOne)
          (.neg (.atom
            ((lam ^ 2 + LeanSuffixAnalytic.typeThreeShape h) /
              (lam * sqrt (lam ^ 2 - LeanSuffixAnalytic.typeThreeShape h ^ 2)))
            (ScalarSuffixCertificate.QInterval.divPos
              ((lambdaInterval.mul lambdaInterval).add qI)
              (lambdaInterval.mul d3CellInterval) hlamDPos) hdivTwo))))))
    (.neg (.mul (.rational 2)
      (.add
        (.add
          (.atom (LeanSuffixAnalytic.typeThreeAngle lam h)
            ((lambdaInterval.mul acosI).add asinQ3CellInterval) hangle)

```

```

      (.atom (π / 2) halfPi hhalfPi))
      (.atom (LeanSuffixAnalytic.typeThreeDelta lam h)
      ((d3CellInterval.divPos lambdaInterval hlamPos).sub u3CellInterval)
      hdelta))))
have hv : e.value = LeanSuffixAnalytic.typeThreeFold lam h := by
  simp only [e, ScalarSuffixCertificate.CertificateExpr.value,
  LeanSuffixAnalytic.typeThreeFold]
  ring
rw [← hv]
have he := e.sound
norm_num [typeThreeFoldCellInterval, e,
  ScalarSuffixCertificate.CertificateExpr.enclosure,
  ScalarSuffixCertificate.QInterval.mul,
  ScalarSuffixCertificate.QInterval.divPos,
  ScalarSuffixCertificate.QInterval.recipPos,
  LeanSuffixReflective.QInterval.add, LeanSuffixReflective.QInterval.neg,
  LeanSuffixReflective.QInterval.sub, LeanSuffixReflective.QInterval.point,
  LeanSuffixReflective.QInterval.nsmul, qI, halfPi, acosI, h3Interval,
  lambdaInterval, u3CellInterval, d3CellInterval, asinQ3CellInterval,
  asinRatio3CellInterval, ScalarSuffixCertificate.piInterval,
  LeanSuffixReflective.QInterval.RealContains] at he ⊢
exact ⟨by linarith [he.1], by linarith [he.2]⟩

/-- The type-(iii) branch is genuinely descending on the whole exact box-0
height slab. -/

```

CompactFirstCell.typeThreeFold_neg_on_slab

Classification: supplement

Source: CompactFirstCell.lean:1123-1130

Declaration SHA-256: 708577dd88ded1a5fe8468072b9f381b014a4ba74d97e65eac8e88ea179dff0e

Mathematical statement. K_3 is strictly negative throughout the first type-(iii) slab.

Thus A_3 is strictly decreasing there for every density in box zero.

```

theorem typeThreeFold_neg_on_slab {lam h : ℝ}
  (hlam : lambdaInterval.RealContains lam)
  (hh : h3Interval.RealContains h) :
  LeanSuffixAnalytic.typeThreeFold lam h < 0 :=
  ScalarSuffixCertificate.QInterval.negative_of_realContains
  (typeThreeFold_cell_enclosure hlam hh)
  (by norm_num [typeThreeFoldCellInterval])

```

CompactFirstCell.typeThreePerimeter_centered_enclosure

Classification: supplement

Source: CompactFirstCell.lean:477-495

Declaration SHA-256: 0acb33ec361f95f1dd6ecbaa5f29b078d12d8e006095b8006b41890c9e85efd5

Mathematical statement. Centered type-(iii) perimeter enclosure for box zero.

The mean-value identity uses K_3/h^2 on a regular slab.

```

theorem typeThreePerimeter_centered_enclosure {lam h : ℝ}
  (hlam : lambdaInterval.RealContains lam)
  (hh : h3Interval.RealContains h)
  (base slope displacement : QInterval)
  (hbase : base.RealContains
  (LeanSuffixAnalytic.typeThreePerimeter lam (h3Center : ℝ)))
  (hslope : ∀ x, h3Interval.RealContains x →
  slope.RealContains
  (LeanSuffixAnalytic.typeThreeFold lam x / x ^ 2))
  (hdisplacement : displacement.RealContains (h - (h3Center : ℝ))) :
  (base.add (slope.mul displacement)).RealContains

```

```

(LeanSuffixAnalytic.typeThreePerimeter lam h) :=
CompactCellCertificate.typeThreePerimeter_centered_enclosure
  lambdaInterval h3Interval h3Center hlam hh h3Center_mem
  (lambda_one_lt hlam) (fun hx => h3_slab_regular hx)
  base slope displacement hbase hslope hdisplacement

```

```

/-- Any two type-(iv) fold roots lying in the exact box-0 root slab coincide. -/

```

CompactFirstCell.typeThreePerimeter_minus_typeFour_neg

Classification: supplement

Source: CompactFirstCell.lean:1575-1598

Declaration SHA-256: 131bce34e1b590459900c2653104e4d2af2313f086bde253fa5a38a3822148fb

Mathematical statement. Throughout compact box zero, every pair of enclosed type-(iii) and type-(iv) curvatures has strictly negative perimeter difference.

Let λ belong to compact box zero's density interval, s to its closed type-(iii) slab, and t to its closed type-(iv) slab. Then $P_3(s) - P_4(t) < 0$. Centered perimeter enclosures establish the inequality on the entire Cartesian product of these three intervals. Neither $K_4(t) = 0$ nor $A_3(s) = A_4(t)$ is an assumption; those equations are used separately to select the eventual comparison pair.

```

theorem typeThreePerimeter_minus_typeFour_neg
  {lam h3 h4 : ℝ}
  (hlam : lambdaInterval.RealContains lam)
  (hh3 : h3Interval.RealContains h3)
  (hh4 : h4Interval.RealContains h4) :
  LeanSuffixAnalytic.typeThreePerimeter lam h3 -
  LeanSuffixAnalytic.typeFourPerimeter lam h4 < 0 := by
have hthree := typeThreePerimeter_centered_cell_enclosure hlam hh3
have hfour := (typeFour_centered_enclosures hlam hh4).2
norm_num [typeThreePerimeterCenteredInterval,
  typeThreeCenterPerimeterInterval, typeThreePerimeterSlopeInterval,
  typeThreeFoldCellInterval, h3SquareInterval, h3DisplacementInterval,
  h3Interval, h3Center, typeFourPerimeterCenteredInterval,
  typeFourCenterPerimeterInterval, typeFourPerimeterSlopeInterval,
  typeFourFoldCellInterval, h4SquareInterval, h4DisplacementInterval,
  h4Interval, h4Center, ScalarSuffixCertificate.QInterval.mul,
  ScalarSuffixCertificate.QInterval.divPos,
  ScalarSuffixCertificate.QInterval.recipPos,
  LeanSuffixReflective.QInterval.RealContains,
  LeanSuffixReflective.QInterval.add, LeanSuffixReflective.QInterval.neg,
  LeanSuffixReflective.QInterval.sub, LeanSuffixReflective.QInterval.point]
at hthree hfour ⊢
linarith

```

CompactSecondCell.QInterval

Classification: supplement

Source: CompactSecondCell.lean:19-21

Declaration SHA-256: 1a69616e446623c478186f47d8f25ae8cb966c1989db56c2e34bb70bf61b9a3c

Mathematical statement. Shared rational interval type in compact box one.

It uses the same ordered rational endpoints as the first box.

```

abbrev QInterval := ScalarSuffixCertificate.QInterval

/-- Lambda interval of zero-based box 1. -/

```


CompactSecondCell.checkedConditions

Classification: supplement

Source: CompactSecondCell.lean:639-659

Declaration SHA-256: c9b3f9f7a265a22f128e05df51ad2c73f46eedace1983e930b75a5ccecf6ea3ba

Mathematical statement. All generic compact-cell conditions hold for box one.

Exact rational calculations discharge branch, fold, equal-area-face, and perimeter inequalities.

```
theorem checkedConditions :
  CompactCellAssembly.Conditions lambdaInterval h3Interval h4Interval where
  lambda_one_lt := fun hlam => lambda_one_lt hlam
  h3_slab_regular := fun hh => h3_slab_regular hh
  h3_slab_above_half := by
    intro h hh
    have hlo : (h3Interval.lo : ℝ) ≤ h := hh.1
    norm_num [h3Interval] at hlo ⊢
    linarith
  h4_slab_regular := fun hh => h4_slab_regular hh
  typeFourFold_lowerFace_neg := fun hlam => typeFourFold_lowerFace_neg hlam
  typeFourFold_upperFace_pos := fun hlam => typeFourFold_upperFace_pos hlam
  typeThreeFold_neg_on_slab := fun hlam hh => typeThreeFold_neg_on_slab hlam hh
  typeThreeArea_lowerFace_gap_pos := fun hlam hh hz =>
    typeThreeArea_lowerFace_gap_pos hlam hh hz
  typeThreeArea_upperFace_gap_neg := fun hlam hh hz =>
    typeThreeArea_upperFace_gap_neg hlam hh hz
  typeThreePerimeter_gap_neg := fun hlam hh3 hh4 =>
    typeThreePerimeter_gap_neg hlam hh3 hh4

/-- Every regular stationary candidate in zero-based compact box 1 is excluded. -/
```

CompactSecondCell.h3Center

Classification: supplement

Source: CompactSecondCell.lean:39-40

Declaration SHA-256: 571f803f2356f6d9d1aef243b2c46e837820a4eb7a7aa3d5322e7d084660f89b

Mathematical statement. Exact type-(iii) center for compact box one.

It is $8301881813483/8796093022208$.

```
def h3Center : ℚ := 8301881813483 / 8796093022208
```

CompactSecondCell.h3Interval

Classification: supplement

Source: CompactSecondCell.lean:31-35

Declaration SHA-256: bee35652f5b82397fd6b36e73591556e7291b9c519843c28a1442d7cd4f919a5

Mathematical statement. Type-(iii) equal-area slab for compact box one.

Its endpoints are $265075342400937/281474976710656$ and $266245093661975/281474976710656$.

```
def h3Interval : QInterval :=
  ⟨265075342400937 / 281474976710656,
    266245093661975 / 281474976710656, by norm_num⟩

/-- Certificate center for the type-(iv) slab. -/
```


CompactSecondCell.h4Center

Classification: supplement

Source: CompactSecondCell.lean:36-38

Declaration SHA-256: 666f145a9e19df37e5fddc56423955a5afdef7d5d8965955b503afe70df65af5

Mathematical statement. Exact type-(iv) center for compact box one.

It is $275560807394631/281474976710656$.

```
def h4Center : ℚ := 275560807394631 / 281474976710656

/-- Certificate center for the type-(iii) slab. -/
```

CompactSecondCell.h4Interval

Classification: supplement

Source: CompactSecondCell.lean:26-30

Declaration SHA-256: af804f12d01723b2d7a86305f48af3d27bbb99fca40d12171d80e480fb2a6639

Mathematical statement. Stationary type-(iv) slab for compact box one.

Its endpoints are $275542293618597/281474976710656$ and $275579321170665/281474976710656$.

```
def h4Interval : QInterval :=
  ⟨275542293618597 / 281474976710656,
    275579321170665 / 281474976710656, by norm_num⟩

/-- Type-(iii) equal-area-root slab of box 1. -/
```

CompactSecondCell.lambdaInterval

Classification: supplement

Source: CompactSecondCell.lean:22-25

Declaration SHA-256: 69be0395e97b77b018ac7b8ae43d4c795e095c167e0e517d474ad3537fb6b8ec

Mathematical statement. Density interval of compact box one.

It is $[236601/229376, 118329/114688]$, meeting box zero exactly.

```
def lambdaInterval : QInterval :=
  ⟨236601 / 229376, 118329 / 114688, by norm_num⟩

/-- Type-(iv) stationary-root slab of box 1. -/
```

F_continuous

Classification: supplement

Source: Minimiser.lean:57-61

Declaration SHA-256: 944215df53ab5a1f8d80d30f68581f0d2605e8a8c87867772cbaa9697833f26f

Mathematical statement. F is continuous.

It is built from real arithmetic, sine, and cosine.

```
lemma F_continuous : Continuous F := by
  unfold F
  fun_prop
```

```
/-- `F` is strictly increasing on `[0, π]` since `F' = 6 sin²θ > 0` inside. -/
```

F_pi_div_two

Classification: supplement

Source: Minimiser.lean:76-81

Declaration SHA-256: 1bd957238f9de023b62b37f505e2ceebe0cf17d7fa9db27be00533881b095aac

Mathematical statement. F at pi/2 equals pi/2.

$$F(\pi/2) = \pi/2 > 0.$$

```
lemma F_pi_div_two : F (π / 2) = π / 2 := by
  rw [F, sin_pi_div_two, cos_pi_div_two]
  ring

/-! ## Existence and uniqueness of the root -/
```

F_zero

Classification: supplement

Source: Minimiser.lean:72-75

Declaration SHA-256: 0054c59a03a184000fc9f7673875bd1fa77b8d5a1989b4d2de26bcf5f101a9b0

Mathematical statement. F at zero equals minus pi.

$$F(0) = -\pi < 0.$$

```
lemma F_zero : F 0 = -π := by
  rw [F, sin_zero, cos_zero]
  ring
```

FourArcAssembly.boundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:2525-2529

Declaration SHA-256: a07e8d5b98aab97a737c50d9ee9eae871142e1adb2b58a3132be8c4b6a093cce

Mathematical statement. Sum of the four parametrized arc measures.

Its weighted integral is later identified with the canonical full-frontier integral.

```
def boundaryMeasure (a : FourArcAssembly) : Measure PlanePoint :=
  leftCoreBoundaryMeasure a.core + rightCoreBoundaryMeasure a.core +
  capBoundaryMeasure a.upperCap + capBoundaryMeasure a.lowerCap

/-- The explicit four-arc boundary measure gives every endpoint zero mass. -/
```

FourArcAssembly.boundaryTrace_subset_frontier_carrier

Classification: supplement

Source: CMVGeometry.lean:3244-3251

Declaration SHA-256: 160385ee99b171abc63b7cc7cb5ade687af8d5249c1ab26a5150be49b8c4654b

Mathematical statement. Every point on the four arc traces is exposed boundary.

Local exterior points establish frontier membership, including junctions.

```

theorem boundaryTrace_subset_frontier_carrier (a : FourArcAssembly) :
  a.boundaryTrace  $\subseteq$  frontier a.carrier := by
  rintro p (hleft | hright | hupper | hlower)
  · exact leftArc_mem_frontier a hleft
  · exact rightArc_mem_frontier a hright
  · exact upperArc_mem_frontier a hupper
  · exact lowerArc_mem_frontier a hlower

```

FourArcAssembly.frontier_carrier_subset_boundaryTrace

Classification: supplement

Source: CMVGeometry.lean:2795-2819

Declaration SHA-256: 0d72e7b21e3528564863313aa2366a2e4806e01c00a4772872911d424abf3cc5

Mathematical statement. Every frontier point lies on one of the four arcs.

The glued chord interiors lie in the assembly interior, not on its frontier.

```

theorem frontier_carrier_subset_boundaryTrace (a : FourArcAssembly) :
  frontier a.carrier  $\subseteq$  a.boundaryTrace := by
  intro p hp
  have htop := frontier_union_subset
    (a.core.carrier  $\cup$  a.upperCap.carrier) a.lowerCap.carrier hp
  rcases htop with hfirst | hlower
  · have hmiddle := frontier_union_subset a.core.carrier a.upperCap.carrier hfirst.1
    rcases hmiddle with hcore | hupper
    · rw [a.core.frontier_carrier] at hcore
      rcases hcore.1 with hleft | hright | hupperChord | hlowerChord
      · exact Or.inl hleft
      · exact Or.inr (Or.inl hright)
      · exact chord_to_boundaryTrace a (Or.inl hupperChord.1) hupperChord.2 hp
      · exact chord_to_boundaryTrace a (Or.inr hlowerChord.1) hlowerChord.2 hp
    · rw [a.upperCap.frontier_carrier] at hupper
      rcases hupper.2 with harc | hchord
      · exact Or.inr (Or.inr (Or.inl harc))
      · exact chord_to_boundaryTrace a (Or.inl hchord.1)
        (by simpa [upperCap] using hchord.2) hp
  · rw [a.lowerCap.frontier_carrier] at hlower
    rcases hlower.2 with harc | hchord
    · exact Or.inr (Or.inr (Or.inr harc))
    · exact chord_to_boundaryTrace a (Or.inr hchord.1)
      (by simpa [lowerCap] using hchord.2) hp

```

FourArcAssembly.weightedPerimeter

Classification: supplement

Source: CMVGeometry.lean:2546-2548

Declaration SHA-256: df8ef218db7076c4681d4506265ae824cfa4b1a2ec872c320aeb4f283b25493b

Mathematical statement. Weighted integral against the component arc measure.

The frontier theorem is needed to identify this component expression with canonical perimeter.

```

def weightedPerimeter (lam :  $\mathbb{R}$ ) (a : FourArcAssembly) :  $\mathbb{R}$  :=
  WeightedPerimeter lam a.boundaryMeasure

```

FourArcCandidate.beta

Classification: supplement

Source: FourArcCandidate.lean:39-41

Declaration SHA-256: f3520601e601026585d5024a59c3bee3be514d265897b65e66919a9d3ea7fd1e

Mathematical statement. The complementary source angle beta.

$$\beta = \pi/2 - \arcsinh.$$

```
def beta : ℝ := π / 2 - arcsin candidate.h

/-- Common horizontal chord length of the two exterior caps. -/
```

FourArcCandidate.capChord_ne

Classification: supplement

Source: FourArcCandidate.lean:102-103

Declaration SHA-256: 58b855530ffa4fa2baf00c3fc6982a6cafa75c224fae7347ad43ffd8cbb2b782

Mathematical statement. The candidate cap chord is nonzero.

Positivity permits division by its square in normalized-area calculations.

```
theorem capChord_ne : candidate.capChord ≠ 0 := ne_of_gt candidate.capChord_pos
```

FourArcCandidate.capChord_pos

Classification: supplement

Source: FourArcCandidate.lean:99-101

Declaration SHA-256: f52b3f7ad66a860b3cbec5075902f43e4ec10a9d92c573767d420b9a7f6a16c6

Mathematical statement. The candidate cap chord is positive.

$2\sin\alpha/h > 0$ from the constructor inequalities.

```
theorem capChord_pos : 0 < candidate.capChord := by
  exact div_pos (mul_pos (by norm_num) candidate.sin_alpha_pos) candidate.h_pos
```

FourArcCandidate.capX_pos

Classification: supplement

Source: FourArcCandidate.lean:120-126

Declaration SHA-256: be506fe40b7a525a5e22df3e7a8d9312d3e56af70a1f9488dad065e7eeb4fa1f

Mathematical statement. The normalized candidate cap area is positive.

This validates use of $\theta(X)$ and the comparison theorem for $g(X)$.

```
theorem capX_pos : 0 < candidate.capX := by
  rw [candidate.capX_eq_area]
  exact area_pos candidate.alpha_pos
    (lt_trans candidate.alpha_lt_pi_div_two (by linarith [Real.pi_pos]))

/-- The normalized area of each original minor cap lies in CMV's strict
geometric interval `(0, π/8)`. -/
```

FourArcCandidate.density_pos_of_campaign_threshold

Classification: supplement

Source: GeometricBridge.lean:150-156

Declaration SHA-256: cc9e8590326c21e2d3058fbcef78da468bb9c4210d5a293f7f28898b6488b124

Mathematical statement. The high-density rational threshold implies positive density.

$\lambda \geq 1.2581840884 > 0$ permits division and positive multiplication.

```
theorem density_pos_of_campaign_threshold
  (hlam : (1.2581840884 : ℝ) ≤ lam) : 0 < lam := by
  norm_num at hlam ⊢
  linarith

/-- This is the sole geometric use of the accepted optimized analytic theorem.
Strictness remains valid when `hlam` is equality. -/
```

FourArcCandidate.lowerCap_euclideanArea

Classification: supplement

Source: FourArcCandidate.lean:159-166

Declaration SHA-256: 14a860cf99daaf759c3fa7838aed86edfb2879de1a5a3f6f7d84f9996c7da155

Mathematical statement. The constructed lower cap has the same cap-area expression.

The lower cap is congruent by reflection.

```
theorem lowerCap_euclideanArea :
  candidate.assembly.lowerCap.euclideanArea = candidate.capArea := by
  rw [OneSidedCircularCap.euclideanArea,
    candidate.capArea_eq_chord_sq_mul_area]
  rfl

/-- The endpoint-incidence definition forces the exterior and strip radii to
agree. -/
```

FourArcCandidate.replacementAngle_lt_pi

Classification: supplement

Source: CapReplacement.lean:35-38

Declaration SHA-256: 5c7cddce688eec8cbb1b3197a2fe6a78bfdc4de573e3f9b6cf8479b677800d6f

Mathematical statement. The replacement angle is strictly below pi.

This ensures a finite, nondegenerate circle radius on the prescribed chord.

```
theorem replacementAngle_lt_pi : candidate.replacementAngle < π :=
  candidate.replacementAngle_mem.2

/-- The replacement angle realizes exactly twice the normalized old cap area. -/
```

FourArcCandidate.replacementAngle_mem

Classification: supplement

Source: CapReplacement.lean:29-31

Declaration SHA-256: 01b8487a31a244058e7174a351228b05206d647acdd690d9902ffea504fec4b6

Mathematical statement. The replacement angle lies in (0,pi).

The inverse normalized-area function returns this full cap branch for positive input.

```
theorem replacementAngle_mem : candidate.replacementAngle ∈ Ioo 0 π := by
  exact 00f_mem (mul_pos (by norm_num) candidate.capX_pos)
```

FourArcCandidate.replacementAngle_pos

Classification: supplement

Source: CapReplacement.lean:32-34

Declaration SHA-256: 9919f7c2d00e4667c39a44e5eadcdd729471effc6713284062fbdbad13333572

Mathematical statement. The replacement angle is positive.

It is the lower inequality of its certified angle domain.

```
theorem replacementAngle_pos : 0 < candidate.replacementAngle :=
  candidate.replacementAngle_mem.1
```

FourArcCandidate.sin_alpha_pos

Classification: supplement

Source: FourArcCandidate.lean:95-98

Declaration SHA-256: b92e14d3743fcaa1d6c447d96c6334ba3f5f129152aa26ff47b97d6fd8de2e28

Mathematical statement. The candidate's exterior angle has positive sine.

It lies in $(0, \pi/2)$.

```
theorem sin_alpha_pos : 0 < sin candidate.alpha :=
  sin_pos_of_pos_of_lt_pi candidate.alpha_pos
  (lt_trans candidate.alpha_lt_pi_div_two (by linarith [Real.pi_pos]))
```

FourArcCandidate.upperCap_euclideanArea

Classification: supplement

Source: FourArcCandidate.lean:153-158

Declaration SHA-256: 63eb34459de3e09e8c932883edd9446c50efc2cc1a148ef60c5c77cbd56d7496

Mathematical statement. The constructed upper cap has the candidate cap-area expression.

Its chord and angle agree with the candidate parameters.

```
theorem upperCap_euclideanArea :
  candidate.assembly.upperCap.euclideanArea = candidate.capArea := by
  rw [OneSidedCircularCap.euclideanArea,
    candidate.capArea_eq_chord_sq_mul_area]
  rfl
```

HorizontalSegment

Classification: supplement

Source: CMVGeometry.lean:392-399

Declaration SHA-256: e7bebca91d2e392d80b21e5e8b5487591a5477fe10be16627e5b6180dd120506

Mathematical statement. A horizontal segment with positive length and specified midpoint and height.

The record supplies the placement and positivity needed by segment length calculations.

```

structure HorizontalSegment where
  chord : ℝ
  midpointX : ℝ
  baseY : ℝ
  chord_pos : 0 < chord

namespace HorizontalSegment

```

HorizontalSegment.carrier

Classification: supplement

Source: CMVGeometry.lean:406-408

Declaration SHA-256: 094b96b3006204081e6d336250f56e63f6b337cc71255210b9bae8ae3b894041

Mathematical statement. The closed set between the horizontal endpoints.

It is a fixed-height closed coordinate interval.

```

def carrier (s : HorizontalSegment) : Set PlanePoint :=
  {p | p.2 = s.baseY ∧ |p.1 - s.midpointX| ≤ s.chord / 2}

```

HorizontalSegment.euclideanLength

Classification: supplement

Source: CMVGeometry.lean:409-415

Declaration SHA-256: c965c7e2ff7089171e9318c88e3733eb9629da0051af6a9aad082bf1ad5a5aef

Mathematical statement. The segment's Euclidean length parameter.

The later Hausdorff calculation identifies this parameter with actual length.

```

def euclideanLength (s : HorizontalSegment) : ℝ := s.chord

end HorizontalSegment

/-- The unchanged central part of a type-(iv) candidate. It is the rectangle
of height two and width `chord`, together with the two radius-`1/h` side
segments. -/

```

HorizontalSegment.leftEndpoint

Classification: supplement

Source: CMVGeometry.lean:400-402

Declaration SHA-256: d64f9e0ea48cfbbc1b06bed874068659f0011957eca2b8226c738430113a0615

Mathematical statement. The segment's left endpoint.

Subtract half its length from the midpoint's horizontal coordinate.

```

def leftEndpoint (s : HorizontalSegment) : PlanePoint :=
  (s.midpointX - s.chord / 2, s.baseY)

```

HorizontalSegment.rightEndpoint

Classification: supplement

Source: CMVGeometry.lean:403-405

Declaration SHA-256: a14791633639531ab0cb8801c8e0e16383dd7712b7d5730aa7f0d9178c83e5da

Mathematical statement. The segment's right endpoint.

Add half its length to the midpoint's horizontal coordinate.

```
def rightEndpoint (s : HorizontalSegment) : PlanePoint :=
  (s.midpointX + s.chord / 2, s.baseY)
```

LeanSuffixAnalytic.hasDerivAt_typeFourAngle

Classification: supplement

Source: LeanSuffixAnalytic.lean:241-278

Declaration SHA-256: ed3b0f3804a218b0cd68be6333d68c33832835819c8ca12dd83eb68bae07439d

Mathematical statement. Derivative of B4 on the interior curvature domain.

$$B_4'(h) = 1/\sqrt{1-h^2} - \lambda/\sqrt{\lambda^2-h^2}.$$

```
theorem hasDerivAt_typeFourAngle {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeFourAngle lam)
    (1 / sqrt (1 - h ^ 2) - lam / sqrt (lam ^ 2 - h ^ 2)) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hu_arg : 0 < 1 - h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - h ^ 2 := by nlinarith
  have hd_ne : sqrt (lam ^ 2 - h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  have hratio_pos : 0 < h / lam := div_pos hh hlam_pos
  have hratio_lt_one : h / lam < 1 :=
    (div_lt_one hlam_pos).2 (lt_trans hh1 hlam)
  have hratio_ne_neg_one : h / lam ≠ -1 := by linarith
  have hratio_ne_one : h / lam ≠ 1 := ne_of_lt hratio_lt_one
  have hh_ne_neg_one : h ≠ -1 := by linarith
  have hh_ne_one : h ≠ 1 := ne_of_lt hh1
  have hsqrt_scale :
    sqrt (1 - (h / lam) ^ 2) = sqrt (lam ^ 2 - h ^ 2) / lam := by
    have hratio_identity :
      1 - (h / lam) ^ 2 = (lam ^ 2 - h ^ 2) / lam ^ 2 := by
        field_simp [hlam_ne]
    rw [hratio_identity, Real.sqrt_div hd_arg.le, Real.sqrt_sq_eq_abs,
      abs_of_pos hlam_pos]
  unfold typeFourAngle
  have harccos :=
    (Real.hasDerivAt_arccos (x := h / lam) hratio_ne_neg_one
      hratio_ne_one).comp h ((hasDerivAt_id h).div_const lam)
  have harsin :=
    Real.hasDerivAt_arcsin (x := h) hh_ne_neg_one hh_ne_one
  have hraw := (harccos.const_mul lam).add harsin
  simp only [Function.comp_apply, id_eq] using! hraw.congr_deriv (by
    rw [hsqrt_scale]
    field_simp [hlam_ne, hd_ne]
    ring)

/-- Exact derivative of the regular type-(iv) radial displacement on its
natural open domain. -/
```

LeanSuffixAnalytic.hasDerivAt_typeFourDelta

Classification: supplement

Source: LeanSuffixAnalytic.lean:279-305

Declaration SHA-256: 71475be52cec7e059c20046cff749d550fce2a4275677924c383af205dd192c8

Mathematical statement. Derivative of D4 on the interior curvature domain.

$$D_4'(h) = h(1/\sqrt{1-h^2} - 1/(\lambda\sqrt{\lambda^2-h^2})).$$


```

theorem hasDerivAt_typeFourDelta {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeFourDelta lam)
    (h / sqrt (1 - h ^ 2) -
     h / (lam * sqrt (lam ^ 2 - h ^ 2))) h := by
have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
have hu_arg : 0 < 1 - h ^ 2 := by nlinarith
have hd_arg : 0 < lam ^ 2 - h ^ 2 := by nlinarith
have hu_ne : sqrt (1 - h ^ 2) ≠ 0 :=
  ne_of_gt (Real.sqrt_pos.2 hu_arg)
have hd_ne : sqrt (lam ^ 2 - h ^ 2) ≠ 0 :=
  ne_of_gt (Real.sqrt_pos.2 hd_arg)
unfold typeFourDelta
have houter := (((hasDerivAt_const h (lam ^ 2)).sub
  ((hasDerivAt_id h).pow 2)).sqrt (ne_of_gt hd_arg)).div_const lam)
have hinner := (((hasDerivAt_const h 1).sub
  ((hasDerivAt_id h).pow 2)).sqrt (ne_of_gt hu_arg))
have hraw := houter.sub hinner
simp only [id_eq] using! hraw.congr_deriv (by
  norm_num [id_eq, Pi.sub_apply, Pi.pow_apply]
  field_simp [hlam_ne, hu_ne, hd_ne]
  ring)

/-- The positive radical-cube gap on the positive half of the regular domain.
Its square witness is the exact factorization
` (lam^2-x^2) - lam^2(1-x^2) = x^2(lam^2-1)`. -/

```

LeanSuffixAnalytic.hasDerivAt_typeThreeAngle

Classification: supplement

Source: LeanSuffixAnalytic.lean:154-205

Declaration SHA-256: 6ec918819525fbc766cbbf40969985e480000f0bcbcf23d536b033e9e80b6380

Mathematical statement. Derivative of the type-(iii) analytic angle contribution.

For $q = 2h - 1$, it is $2/\sqrt{1-q^2} - 2\lambda/\sqrt{\lambda^2-q^2}$.

```

theorem hasDerivAt_typeThreeAngle {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeAngle lam)
    (2 * (1 / sqrt (1 - typeThreeShape h ^ 2) -
     lam / sqrt (lam ^ 2 - typeThreeShape h ^ 2))) h := by
have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
have hshape := hasDerivAt_typeThreeShape h
have hq_low : -1 < typeThreeShape h := by
  unfold typeThreeShape
  linarith
have hq_high : typeThreeShape h < 1 := by
  unfold typeThreeShape
  linarith
have hu_arg : 0 < 1 - typeThreeShape h ^ 2 := by nlinarith
have hd_arg : 0 < lam ^ 2 - typeThreeShape h ^ 2 := by nlinarith
have hd_ne : sqrt (lam ^ 2 - typeThreeShape h ^ 2) ≠ 0 :=
  ne_of_gt (Real.sqrt_pos.2 hd_arg)
have hratio_low : -1 < typeThreeShape h / lam :=
  (lt_div_iff₀ hlam_pos).2 (by linarith)
have hratio_high : typeThreeShape h / lam < 1 :=
  (div_lt_one hlam_pos).2 (by linarith)
have hratio_ne_neg_one : typeThreeShape h / lam ≠ -1 :=
  ne_of_gt hratio_low
have hratio_ne_one : typeThreeShape h / lam ≠ 1 :=
  ne_of_lt hratio_high
have hq_ne_neg_one : typeThreeShape h ≠ -1 := ne_of_gt hq_low
have hq_ne_one : typeThreeShape h ≠ 1 := ne_of_lt hq_high
have hsqrt_scale :
  sqrt (1 - (typeThreeShape h / lam) ^ 2) =

```

```

    sqrt (lam ^ 2 - typeThreeShape h ^ 2) / lam := by
  have hratio_identity :
    1 - (typeThreeShape h / lam) ^ 2 =
      (lam ^ 2 - typeThreeShape h ^ 2) / lam ^ 2 := by
    field_simp [hlam_ne]
  rw [hratio_identity, Real.sqrt_div hd_arg.le, Real.sqrt_sq_eq_abs,
    abs_of_pos hlam_pos]
  unfold typeThreeAngle
  have harccos :=
    (Real.hasDerivAt_arccos (x := typeThreeShape h / lam)
      hratio_ne_neg_one hratio_ne_one).comp h (hshape.div_const lam)
  have harsin :=
    (Real.hasDerivAt_arcsin (x := typeThreeShape h)
      hq_ne_neg_one hq_ne_one).comp h hshape
  have hraw := (harccos.const_mul lam).add harsin
  simpa only [Function.comp_apply, id_eq] using! hraw.congr_deriv (by
    rw [hsqrt_scale]
    field_simp [hlam_ne, hd_ne]
    ring)

/-- Exact derivative of the regular type-(iii) radial displacement on its
natural open domain. -/

```

LeanSuffixAnalytic.hasDerivAt_typeThreeDelta

Classification: supplement

Source: LeanSuffixAnalytic.lean:206-240

Declaration SHA-256: 6a259e36542b424c2657b96b59d66f9d38d054281073fe5c590c3110a33522c8

Mathematical statement. Derivative of the type-(iii) radical displacement.

It is $2q(1/\sqrt{1-q^2} - 1/(\lambda\sqrt{\lambda^2-q^2}))$ on $0 < h < 1$.

```

theorem hasDerivAt_typeThreeDelta {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeDelta lam)
    (2 * typeThreeShape h *
      (1 / sqrt (1 - typeThreeShape h ^ 2) -
        1 / (lam * sqrt (lam ^ 2 - typeThreeShape h ^ 2)))) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hshape := hasDerivAt_typeThreeShape h
  have hq_low : -1 < typeThreeShape h := by
    unfold typeThreeShape
    linarith
  have hq_high : typeThreeShape h < 1 := by
    unfold typeThreeShape
    linarith
  have hu_arg : 0 < 1 - typeThreeShape h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - typeThreeShape h ^ 2 := by nlinarith
  have hu_ne : sqrt (1 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hu_arg)
  have hd_ne : sqrt (lam ^ 2 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  unfold typeThreeDelta
  have houter :=
    (((hasDerivAt_const h (lam ^ 2)).sub (hshape.pow 2)).sqrt
      (ne_of_gt hd_arg)).div_const lam)
  have hinner :=
    (((hasDerivAt_const h 1).sub (hshape.pow 2)).sqrt (ne_of_gt hu_arg))
  have hraw := houter.sub hinner
  simpa only [id_eq] using! hraw.congr_deriv (by
    norm_num [id_eq, Pi.sub_apply, Pi.pow_apply]
    field_simp [hlam_ne, hu_ne, hd_ne]
    ring)

/-- Exact derivative of the regular type-(iv) angle contribution on its
natural open domain. -/

```

LeanSuffixAnalytic.hasDerivAt_typeThreeShape

Classification: supplement

Source: LeanSuffixAnalytic.lean:146-153

Declaration SHA-256: fc5b59d714869fde00b69f1950c83bd7a32c47524e01ad8d04455ff346c39d58

Mathematical statement. The derivative of $q=2h-1$ is two.

This affine derivative supplies the chain-rule factor in type-(iii) formulas.

```
theorem hasDerivAt_typeThreeShape (h : ℝ) :
  HasDerivAt typeThreeShape 2 h := by
  unfold typeThreeShape
  simp only [id_eq, mul_one] using
    (((hasDerivAt_id h).const_mul (2 : ℝ)).sub_const 1)

/-- Exact derivative of the regular type-(iii) angle contribution on its
natural open domain. -/
```

LeanSuffixAnalytic.typeFourAngle_eq_checker

Classification: supplement

Source: LeanSuffixAnalytic.lean:139-145

Declaration SHA-256: d6ea741d7af136e9d0263d84561c8171caafa7b1249aceaa9a355bce3a415957

Mathematical statement. The analytic type-(iv) angle term agrees with the checker form.

The complementary principal angle converts $\arcsin h$ to the checker's half-pi-minus-arccos expression.

```
theorem typeFourAngle_eq_checker (lam h : ℝ) :
  typeFourAngle lam h =
    lam * arccos (h / lam) + π / 2 - arccos h := by
  rw [typeFourAngle, Real.arcsin_eq_pi_div_two_sub_arccos]
  ring

/-- Exact derivative of the affine type-(iii) shape parameter. -/
```

LeanSuffixAnalytic.typeThreeAngle_add_halfPi

Classification: supplement

Source: LeanSuffixAnalytic.lean:130-138

Declaration SHA-256: 309f9c361ad038b7eae75200b2b081130c71a155a43f2f07c9ad2ae282ce68ff

Mathematical statement. The analytic type-(iii) angle term plus $\pi/2$ equals the source arccosine expression.

Use $\arcsin q + \pi/2 = \pi - \arccos q$.

```
theorem typeThreeAngle_add_halfPi (lam h : ℝ) :
  typeThreeAngle lam h + π / 2 =
    lam * arccos (typeThreeShape h / lam) + π -
    arccos (typeThreeShape h) := by
  rw [typeThreeAngle, Real.arcsin_eq_pi_div_two_sub_arccos]
  ring

/-- The type-(iv) angle term is exactly the checkers' `B4` expression after
the substitution `h = cos x`. -/
```

LeanSuffixAnalytic.typeThreeArea_deriv_eq_explicit

Classification: supplement

Source: LeanSuffixAnalytic.lean:649-699

Declaration SHA-256: ec96d74eb7c231919df47f4c5a7bcce391a0bf68733b2ba3385beb5c882b9ca6

Mathematical statement. Explicit algebraic formula for the type-(iii) area derivative.

The quotient and chain rules simplify to the numerator subsequently identified as K_3 .

```
theorem typeThreeArea_deriv_eq_explicit {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  deriv (typeThreeArea lam) h =
    ((2 * (2 * (1 / sqrt (1 - typeThreeShape h ^ 2)) -
      lam / sqrt (lam ^ 2 - typeThreeShape h ^ 2)) +
      2 * typeThreeShape h *
        (1 / sqrt (1 - typeThreeShape h ^ 2)) -
        1 / (lam * sqrt (lam ^ 2 - typeThreeShape h ^ 2)))) * h -
      2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h)) /
      h ^ 3) := by
  let q := typeThreeShape h
  let p := 2 * (1 / sqrt (1 - q ^ 2)) -
    lam / sqrt (lam ^ 2 - q ^ 2)
  let dp := 2 * q * (1 / sqrt (1 - q ^ 2)) -
    1 / (lam * sqrt (lam ^ 2 - q ^ 2))
  have hh_ne : h ≠ 0 := ne_of_gt hh
  have hangle : HasDerivAt (typeThreeAngle lam) p h := by
    simpa only [p, q] using hasDerivAt_typeThreeAngle hlam hh hh1
  have hdelta : HasDerivAt (typeThreeDelta lam) dp h := by
    simpa only [dp, q] using hasDerivAt_typeThreeDelta hlam hh hh1
  have hperimeterRaw :=
    ((hangle.add_const (π / 2)).add hdelta).const_mul (2 : ℝ)).div
    (hasDerivAt_id h) hh_ne
  have hperimeter : HasDerivAt (typeThreePerimeter lam)
    ((2 * (p + dp)) * h -
      2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h)) /
      h ^ 2) h := by
    simpa only [typeThreePerimeter, id_eq] using!
    hperimeterRaw.congr_deriv (by
      norm_num [id_eq, Pi.add_apply, Pi.mul_apply, Pi.pow_apply])
  have hvariational :=
    typeThreePerimeter_deriv_eq_height_mul_area_deriv hlam hh hh1
  rw [hperimeter.deriv] at hvariational
  change deriv (typeThreeArea lam) h =
    (((2 * (p + dp)) * h -
      2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h)) /
      h ^ 3)
  apply (eq_div_iff (pow_ne_zero 3 hh_ne)).2
  calc
    deriv (typeThreeArea lam) h * h ^ 3 =
      (h * deriv (typeThreeArea lam) h) * h ^ 2 := by ring
    _ = (((2 * (p + dp)) * h -
      2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h)) /
      h ^ 2) * h ^ 2 := by rw [hvariational]
    _ = (2 * (p + dp)) * h -
      2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h) := by
      field_simp [hh_ne]

/-- The checker quantity `typeThreeFold` is exactly the numerator of the
type-(iii) area derivative. -/
```

LeanSuffixReflective.QInterval

Classification: supplement

Source: LeanSuffixReflective.lean:19-26

Declaration SHA-256: aa9fdae3c31a981fd9f3e3c84090b4a600570661da326e67bfa152cc3da18001

Mathematical statement. An ordered pair of rational interval endpoints.

The record requires $lo \leq hi$ and represents a closed interval.

```
structure QInterval where
  lo : Q
  hi : Q
  ordered : lo ≤ hi

namespace QInterval

/-- Membership interpreted in exact rational arithmetic. -/
```

LeanSuffixReflective.QInterval.add

Classification: supplement

Source: LeanSuffixReflective.lean:37-40

Declaration SHA-256: bdc18247a80ffa4a9b050a8d17f0cc87f5a29f49311d73038324983c96603c22

Mathematical statement. Endpoint interval addition.

$[a, b] + [c, d] = [a + c, b + d]$.

```
def add (i j : QInterval) : QInterval :=
  ⟨i.lo + j.lo, i.hi + j.hi, add_le_add i.ordered j.ordered⟩

/-- Exact interval negation. -/
```

LeanSuffixReflective.QInterval.neg

Classification: supplement

Source: LeanSuffixReflective.lean:41-44

Declaration SHA-256: 81fd6f63aa66c1f67c60e066b58e12cde99767a33f0cf5e8a79a120837332892

Mathematical statement. Interval negation with reversed endpoints.

$-[a, b] = [-b, -a]$.

```
def neg (i : QInterval) : QInterval :=
  ⟨-i.hi, -i.lo, neg_le_neg i.ordered⟩

/-- Exact interval subtraction. -/
```

LeanSuffixReflective.QInterval.nsmul

Classification: supplement

Source: LeanSuffixReflective.lean:48-59

Declaration SHA-256: aee3f878b71376e5cd4779ff5df12d1e44522ca38190dd8feca337d507ede104

Mathematical statement. Exact scaling of a rational interval by any nonnegative rational number, including zero and $1/2$.

For $I = [a, b]$ with $a, b \in \mathbb{Q}$ and $a \leq b$, and for $q \in \mathbb{Q}$ with $0 \leq q$, the result is $[qa, qb]$. Multiplication by a nonnegative number preserves endpoint order, so no sign condition on a or b is needed. In particular, $q = 0$ gives $[0, 0]$ and $q = 1/2$ gives $[a/2, b/2]$. Despite the declaration name, the scalar is rational, not restricted to a natural number.

```
def nsmul (q : ℚ) (hq : 0 ≤ q) (i : QInterval) : QInterval :=
  ⟨q * i.lo, q * i.hi, mul_le_mul_of_nonneg_left i.ordered hq⟩

@[simp] theorem realContains_point (x : ℚ) (y : ℝ) :
  (point x).RealContains y ↔ y = (x : ℝ) := by
  constructor
  · intro h
    exact le_antisymm h.2 h.1
  · rintro rfl
    exact ⟨le_rfl, le_rfl⟩

/-- Rational membership transports without loss to real membership. -/
```

LeanSuffixReflective.QInterval.point

Classification: supplement

Source: LeanSuffixReflective.lean:34-36

Declaration SHA-256: 35a71df9981ec16d7188d1e717bf2773425dff09af794e6cabfae2678adb7fb9

Mathematical statement. The singleton interval at a rational number.

Both endpoints equal the rational input.

```
def point (x : ℚ) : QInterval := ⟨x, x, le_rfl⟩

/-- Exact interval addition. -/
```

LeanSuffixReflective.QInterval.realContains_add

Classification: supplement

Source: LeanSuffixReflective.lean:65-71

Declaration SHA-256: 29bd8dc11237ddddd13e05a341310e25a85723026439683a6ff469efc9539f38

Mathematical statement. Interval addition encloses the sum of enclosed reals.

Add the respective lower and upper inequalities.

```
theorem realContains_add {i j : QInterval} {x y : ℝ}
  (hx : i.RealContains x) (hy : j.RealContains y) :
  (i.add j).RealContains (x + y) := by
  exact ⟨by simpa [RealContains, add] using add_le_add hx.1 hy.1,
    by simpa [RealContains, add] using add_le_add hx.2 hy.2⟩

/-- Soundness of exact interval negation. -/
```

LeanSuffixReflective.QInterval.realContains_neg

Classification: supplement

Source: LeanSuffixReflective.lean:72-77

Declaration SHA-256: c819be23ad835c80289fa7a60e6d94935c325c424a832a1d3dc0780c7f34f5fd

Mathematical statement. Interval negation encloses the negative of every enclosed real.

Negation reverses both endpoint inequalities.

```

theorem realContains_neg {i : QInterval} {x : ℝ}
  (hx : i.RealContains x) : i.neg.RealContains (-x) := by
  exact ⟨by simp [RealContains, neg] using neg_le_neg hx.2,
    by simp [RealContains, neg] using neg_le_neg hx.1⟩

/-- Soundness of exact interval subtraction. -/

```

LeanSuffixReflective.QInterval.realContains_sub

Classification: supplement

Source: LeanSuffixReflective.lean:78-85

Declaration SHA-256: 8bd28ee4b73b928fde4081285e5ca72b48a1703fbb61327a22c0b7e93e0fc99c

Mathematical statement. Interval subtraction encloses differences of enclosed reals.

Combine sound addition and negation.

```

theorem realContains_sub {i j : QInterval} {x y : ℝ}
  (hx : i.RealContains x) (hy : j.RealContains y) :
  (i.sub j).RealContains (x - y) := by
  exact realContains_add hx (realContains_neg hy)

end QInterval

/-- The exact suffix covered by the retained certificate families. -/

```

LeanSuffixReflective.QInterval.sub

Classification: supplement

Source: LeanSuffixReflective.lean:45-47

Declaration SHA-256: edd84d3089fe87cd8f669b75895897b3124604659a9914bc80d5d6266ee85fd2

Mathematical statement. Interval subtraction as addition of the negated interval.

$[a, b] - [c, d] = [a - d, b - c]$.

```

def sub (i j : QInterval) : QInterval := i.add j.neg

/-- Exact nonnegative scaling, sufficient for Taylor error radii and seams. -/

```

MiddleFaceAssembly.C3Residual.derivRho

Classification: supplement

Source: MiddleFaceAssembly.lean:1190-1197

Declaration SHA-256: c1e483238adef1a69bda282637a3ed4325a085039bf7cd8b957f9f3fdb1c5e6e

Mathematical statement. Explicit rho derivative of the type-(iii) incidence residual.

The w variation and its coupling into v are kept together.

```

def derivRho (t rho sigma : ℝ) : ℝ :=
  let ww := (b.centers 2 : ℝ) + b.slopes 2 * t + rho
  let vv := (b.centers 3 : ℝ) + b.slopes 3 * t +
    b.vRhoSlope * rho + sigma
  let ll := (lamMid b : ℝ) + t
  (-sin ww + (b.vRhoSlope : ℝ) * (ll * sin vv))

/-- Derivative of the correlated `C3` residual in the `sigma` direction. -/

```

MiddleFaceAssembly.C3Residual.derivSigma

Classification: supplement

Source: MiddleFaceAssembly.lean:1198-1202

Declaration SHA-256: 36bccfa8df42ee44f7ce2efdeb4df70c3f66a17d14d767e920177bbf1afa9876

Mathematical statement. Explicit sigma derivative of the type-(iii) incidence residual.

It is the exterior-angle sine contribution from differentiating the cosine.

```
def derivSigma (t rho sigma : ℝ) : ℝ :=
  let vv := (b.centers 3 : ℝ) + b.slopes 3 * t +
    b.vRhoSlope * rho + sigma
  ((lamMid b : ℝ) + t) * sin vv
```

MiddleFaceAssembly.C3Residual.derivT

Classification: supplement

Source: MiddleFaceAssembly.lean:1181-1189

Declaration SHA-256: c4e519ef7409cb708317db418241213f452ef4b4aa73c4ae26a3279d8dcb518d

Mathematical statement. Explicit t derivative of the type-(iii) incidence residual.

The derivative includes the direct density factor and both angular slopes.

```
def derivT (t rho sigma : ℝ) : ℝ :=
  let ww := (b.centers 2 : ℝ) + b.slopes 2 * t + rho
  let vv := (b.centers 3 : ℝ) + b.slopes 3 * t +
    b.vRhoSlope * rho + sigma
  let ll := (lamMid b : ℝ) + t
  (-(b.slopes 2 : ℝ) * sin ww - cos vv +
    (b.slopes 3 : ℝ) * (ll * sin vv))

/-- Derivative of the correlated `C3` residual in the `rho` direction. -/
```

MiddleFaceAssembly.C3Residual.reduced

Classification: supplement

Source: MiddleFaceAssembly.lean:1170-1175

Declaration SHA-256: 4b58f212985cabd9417414d5466594d9a780e38a8e0c1797d384042e1674c68b

Mathematical statement. The type-(iii) incidence residual in centered coordinates.

It is $\cos w - \lambda \cos v$ after the coupled affine substitution.

```
def reduced (t rho sigma : ℝ) : ℝ :=
  cos ((b.centers 2 : ℝ) + b.slopes 2 * t + rho) -
  ((lamMid b : ℝ) + t) *
  cos ((b.centers 3 : ℝ) + b.slopes 3 * t +
    b.vRhoSlope * rho + sigma)
```

MiddleFaceAssembly.C3Residual.reduced_hasDerivAt_rho

Classification: supplement

Source: MiddleFaceAssembly.lean:1229-1250

Declaration SHA-256: 78960ca166459039fcd13d2b254e90bc61da69bb828a41a82ae24260c24dc81e

Mathematical statement. The recorded rho derivative of C3 is exact.

It preserves the specified angular coupling.


```

theorem reduced_hasDerivAt_rho (t rho sigma : ℝ) :
  HasDerivAt (fun u => reduced b t u sigma)
    (derivRho b t rho sigma) rho := by
  have hw : HasDerivAt
    (fun u : ℝ => (b.centers 2 : ℝ) + b.slopes 2 * t + u) 1 rho := by
  convert (hasDerivAt_const rho
    ((b.centers 2 : ℝ) + b.slopes 2 * t)).add (hasDerivAt_id rho) using 1 <;>
  first | rfl | ring_nf
  have hv : HasDerivAt
    (fun u : ℝ => (b.centers 3 : ℝ) + b.slopes 3 * t +
      b.vRhoSlope * u + sigma) (b.vRhoSlope : ℝ) rho := by
  convert ((hasDerivAt_const rho
    ((b.centers 3 : ℝ) + b.slopes 3 * t)).add
    ((hasDerivAt_const rho (b.vRhoSlope : ℝ)).mul
    (hasDerivAt_id rho))).add (hasDerivAt_const rho sigma)) using 1 <;>
  first | rfl | ring_nf
  convert ((Real.hasDerivAt_cos _).comp rho hw).sub
    ((hasDerivAt_const rho ((lamMid b : ℝ) + t)).mul
    ((Real.hasDerivAt_cos _).comp rho hv)) using 1 <;> try rfl
  simp [derivRho, Function.comp_apply]
  ring

```

MiddleFaceAssembly.C3Residual.reduced_hasDerivAt_sigma

Classification: supplement

Source: MiddleFaceAssembly.lean:1251-1270

Declaration SHA-256: 4b0f795e660ad8612256b86b0826502532784fc2e8dabb3224b9d16a883aa33e

Mathematical statement. The recorded sigma derivative of C3 is exact.

This supports the opposite sigma-face signs.

```

theorem reduced_hasDerivAt_sigma (t rho sigma : ℝ) :
  HasDerivAt (fun u => reduced b t rho u)
    (derivSigma b t rho sigma) sigma := by
  have hv : HasDerivAt
    (fun u : ℝ => (b.centers 3 : ℝ) + b.slopes 3 * t +
      b.vRhoSlope * rho + u) 1 sigma := by
  convert (hasDerivAt_const sigma
    ((b.centers 3 : ℝ) + b.slopes 3 * t +
      b.vRhoSlope * rho)).add (hasDerivAt_id sigma) using 1 <;>
  first | rfl | ring_nf
  convert (hasDerivAt_const sigma
    (cos ((b.centers 2 : ℝ) + b.slopes 2 * t + rho))).sub
    ((hasDerivAt_const sigma ((lamMid b : ℝ) + t)).mul
    ((Real.hasDerivAt_cos _).comp sigma hv)) using 1 <;> try rfl
  simp [derivSigma, Function.comp_apply]

/-- Uniform numerical derivative contract proved by each brick's exact
rational enclosure certificate on a lambda-centered interval of radius `R`.
The `t` and `rho` derivatives are needed only on the `rho = sigma = 0` and
`sigma = 0` legs, respectively. -/

```

MiddleFaceAssembly.C3Residual.reduced_hasDerivAt_t

Classification: supplement

Source: MiddleFaceAssembly.lean:1203-1228

Declaration SHA-256: 7dbe2b5150934c034dcc9733f2ecaa267ad4409fadfc1f5c1ad22c9ee3309dc2

Mathematical statement. The recorded t derivative of C3 is exact.

The chain rule validates density-dependent incidence enclosures.

```

theorem reduced_hasDerivAt_t (t rho sigma : ℝ) :
  HasDerivAt (fun u => reduced b u rho sigma)

```

```

      (derivT b t rho sigma) t := by
have hw : HasDerivAt
  (fun u : ℝ => (b.centers 2 : ℝ) + b.slopes 2 * u + rho)
  (b.slopes 2 : ℝ) t := by
convert ((hasDerivAt_const t (b.centers 2 : ℝ)).add
  ((hasDerivAt_const t (b.slopes 2 : ℝ)).mul (hasDerivAt_id t))).add
  (hasDerivAt_const t rho) using 1 <;>
first | rfl | ring_nf
have hv : HasDerivAt
  (fun u : ℝ => (b.centers 3 : ℝ) + b.slopes 3 * u +
    b.vRhoSlope * rho + sigma) (b.slopes 3 : ℝ) t := by
convert (((hasDerivAt_const t (b.centers 3 : ℝ)).add
  ((hasDerivAt_const t (b.slopes 3 : ℝ)).mul (hasDerivAt_id t))).add
  (hasDerivAt_const t ((b.vRhoSlope : ℝ) * rho))).add
  (hasDerivAt_const t sigma) using 1 <;>
first | rfl | ring_nf
have hl : HasDerivAt (fun u : ℝ => (lamMid b : ℝ) + u) 1 t := by
  convert (hasDerivAt_const t (lamMid b : ℝ)).add (hasDerivAt_id t) using 1 <;>
  first | rfl | ring_nf
convert ((Real.hasDerivAt_cos _).comp t hw).sub
  (hl.mul ((Real.hasDerivAt_cos _).comp t hv)) using 1 <;> try rfl
simp [derivT, Function.comp_apply]
ring

```

MiddleFaceAssembly.C4Residual.derivEta

Classification: supplement

Source: MiddleFaceAssembly.lean:707-710

Declaration SHA-256: 1a437771f514f75dbb66dda75e523e87384fcd4e7f50961082bd2649fc01f800

Mathematical statement. Explicit eta derivative of the type-(iv) incidence residual.

Differentiating the exterior cosine gives the positive branch sine factor.

```

def derivEta (t xi eta : ℝ) : ℝ :=
  ((lamMid b : ℝ) + t) *
    sin ((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta)

```

MiddleFaceAssembly.C4Residual.derivT

Classification: supplement

Source: MiddleFaceAssembly.lean:693-700

Declaration SHA-256: 4215795dac7a2deb7f79984f7d7c608299b079841931e2bdd1d8a1d5e697b0802

Mathematical statement. Explicit t derivative of the type-(iv) incidence residual.

The density factor and both angle predictors contribute.

```

def derivT (t xi eta : ℝ) : ℝ :=
  -(b.slopes 0 : ℝ) *
    sin ((b.centers 0 : ℝ) + b.slopes 0 * t + xi) -
    cos ((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta) +
    (b.slopes 1 : ℝ) * ((lamMid b : ℝ) + t) *
      sin ((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta))

/-- Derivative of the correlated `C4` residual in `xi`. -/

```

MiddleFaceAssembly.C4Residual.derivXi

Classification: supplement

Source: MiddleFaceAssembly.lean:701-706

Declaration SHA-256: ed53afb5e433a1f19562ef174d9cbdc3d2d47f108aac2160d30aaf8712b1aa82

Mathematical statement. Explicit xi derivative of the type-(iv) incidence residual.

It includes the xi coupling in the exterior angle.

```
def derivXi (t xi eta : ℝ) : ℝ :=
  -sin ((b.centers 0 : ℝ) + b.slopes 0 * t + xi) +
    (b.yXiSlope : ℝ) * ((lamMid b : ℝ) + t) *
      sin ((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta))

/-- Derivative of the correlated `C4` residual in `eta`. -/
```

MiddleFaceAssembly.C4Residual.reduced

Classification: supplement

Source: MiddleFaceAssembly.lean:683-687

Declaration SHA-256: 6ad85092e6bdf9c53b20e583c6a631e4f162d95899defb77f3673f60e1415681

Mathematical statement. Type-(iv) incidence residual in centered density and angle errors.

It is $\cos x - \lambda \cos y$ after affine substitution.

```
def reduced (t xi eta : ℝ) : ℝ :=
  cos ((b.centers 0 : ℝ) + b.slopes 0 * t + xi) -
    ((lamMid b : ℝ) + t) *
      cos ((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta)
```

MiddleFaceAssembly.C4Residual.reduced_hasDerivAt_eta

Classification: supplement

Source: MiddleFaceAssembly.lean:758-775

Declaration SHA-256: 3ee50bbc4e028a15db6fda4f71c882933ff623e2cf7891f6aa4ee3930451e70b

Mathematical statement. The eta derivative formula for C4 is correct.

It supports opposite eta-face sign estimates.

```
theorem reduced_hasDerivAt_eta (t xi eta : ℝ) :
  HasDerivAt (fun u => reduced b t xi u) (derivEta b t xi eta) eta := by
  have hy : HasDerivAt
    (fun u : ℝ => (b.centers 1 : ℝ) + b.slopes 1 * t +
      b.yXiSlope * xi + u) 1 eta := by
    convert (hasDerivAt_const eta
      ((b.centers 1 : ℝ) + b.slopes 1 * t +
        b.yXiSlope * xi)).add (hasDerivAt_id eta) using 1 <=>
      first | rfl | ring_nf
    convert (hasDerivAt_const eta
      (cos ((b.centers 0 : ℝ) + b.slopes 0 * t + xi))).sub
      ((hasDerivAt_const eta ((lamMid b : ℝ) + t)).mul
        ((Real.hasDerivAt_cos _).comp eta hy)) using 1 <=> try rfl
  simp only [Function.comp_apply, derivEta]
  ring

/-- Uniform numerical derivative contract proved by each brick's exact
rational enclosure certificate on a lambda-centered interval of radius `R`. -/
```

MiddleFaceAssembly.C4Residual.reduced_hasDerivAt_t

Classification: supplement

Source: MiddleFaceAssembly.lean:711-736

Declaration SHA-256: 57a105b0af38480d894e98b27a5a27170b796d726656e5e9828ac63318bb34f4

Mathematical statement. The t derivative of the reduced C4 residual equals its explicit formula.

This supplies a proved derivative, rather than an assumed numerical bound.

```
theorem reduced_hasDerivAt_t (t xi eta : ℝ) :
  HasDerivAt (fun u => reduced b u xi eta) (derivT b t xi eta) t := by
  have hx : HasDerivAt
    (fun u : ℝ => (b.centers 0 : ℝ) + b.slopes 0 * u + xi)
    (b.slopes 0 : ℝ) t := by
    convert ((hasDerivAt_const t (b.centers 0 : ℝ)).add
      ((hasDerivAt_const t (b.slopes 0 : ℝ)).mul
        (hasDerivAt_id t))).add (hasDerivAt_const t xi) using 1 <=>
    first | rfl | ring_nf
  have hy : HasDerivAt
    (fun u : ℝ => (b.centers 1 : ℝ) + b.slopes 1 * u +
      b.yXiSlope * xi + eta) (b.slopes 1 : ℝ) t := by
    convert (((hasDerivAt_const t (b.centers 1 : ℝ)).add
      ((hasDerivAt_const t (b.slopes 1 : ℝ)).mul
        (hasDerivAt_id t))).add
      (hasDerivAt_const t ((b.yXiSlope : ℝ) * xi))).add
      (hasDerivAt_const t eta) using 1 <=>
    first | rfl | ring_nf
  have hlam : HasDerivAt (fun u : ℝ => (lamMid b : ℝ) + u) 1 t := by
    convert (hasDerivAt_const t (lamMid b : ℝ)).add (hasDerivAt_id t) using 1 <=>
    first | rfl | ring_nf
  convert ((Real.hasDerivAt_cos _).comp t hx).sub
    (hlam.mul ((Real.hasDerivAt_cos _).comp t hy)) using 1 <=> try rfl
  simp only [Function.comp_apply, derivT]
  ring
```

MiddleFaceAssembly.C4Residual.reduced_hasDerivAt_xi

Classification: supplement

Source: MiddleFaceAssembly.lean:737-757

Declaration SHA-256: 5c5aed8fcfa67c76068e264e5cfd03a674d4b1ee2a4b7f25289608330e62d6c

Mathematical statement. The ξ derivative formula for C4 is correct.

The coupled affine chain rule is checked exactly.

```
theorem reduced_hasDerivAt_xi (t xi eta : ℝ) :
  HasDerivAt (fun u => reduced b t u eta) (derivXi b t xi eta) xi := by
  have hx : HasDerivAt
    (fun u : ℝ => (b.centers 0 : ℝ) + b.slopes 0 * t + u) 1 xi := by
    convert (hasDerivAt_const xi
      ((b.centers 0 : ℝ) + b.slopes 0 * t)).add (hasDerivAt_id xi) using 1 <=>
    first | rfl | ring_nf
  have hy : HasDerivAt
    (fun u : ℝ => (b.centers 1 : ℝ) + b.slopes 1 * t +
      b.yXiSlope * u + eta) (b.yXiSlope : ℝ) xi := by
    convert ((hasDerivAt_const xi
      ((b.centers 1 : ℝ) + b.slopes 1 * t)).add
      ((hasDerivAt_const xi (b.yXiSlope : ℝ)).mul
        (hasDerivAt_id xi))).add (hasDerivAt_const xi eta) using 1 <=>
    first | rfl | ring_nf
  convert ((Real.hasDerivAt_cos _).comp xi hx).sub
    ((hasDerivAt_const xi ((lamMid b : ℝ) + t)).mul
      ((Real.hasDerivAt_cos _).comp xi hy)) using 1 <=> try rfl
  simp only [Function.comp_apply, derivXi]
  ring
```

MiddleFaceAssembly.EResidual.abs_image_sub_le_of_hasDerivAt

Classification: supplement

Source: MiddleFaceAssembly.lean:435-447

Declaration SHA-256: b7f0e88d7d03b44d4326633ce2286459e2f413f783621bf0b54b9742012c3366

Mathematical statement. Let $R \geq 0$ and u belong to $[-R, R]$. If f has derivative $D(q)$ at every q in $[-R, R]$ and $|D(q)| \leq M$ throughout that interval, then $|f(u) - f(0)| \leq M|u|$.

Let $f, D: \mathbb{R} \rightarrow \mathbb{R}$ and $R, M, u \in \mathbb{R}$, with $R \geq 0$ and $-R \leq u \leq R$. Assume that f has ordinary derivative $D(q)$ at every $q \in [-R, R]$, including the endpoints, and that $|D(q)| \leq M$ for every such q . Then $|f(u) - f(0)| \leq M|u|$. Both 0 and u lie in the convex interval $[-R, R]$, so the mean-value derivative bound applied between these two points gives the estimate. If $u = 0$, the conclusion is immediate; in particular the degenerate case $R = 0$ is allowed. No continuity of D is assumed, and $M \geq 0$ follows from the bound at $q = 0$ rather than being a separate hypothesis. This declaration is the center-anchored symmetric-interval estimate, not a statement quantified over arbitrary endpoints a, b .

```
theorem abs_image_sub_le_of_hasDerivAt {f f' : ℝ → ℝ} {R M u : ℝ}
  (hR : 0 ≤ R) (hu : u ∈ Icc (-R) R)
  (hderiv : ∀ q ∈ Icc (-R) R, HasDerivAt f (f' q) q)
  (hbound : ∀ q ∈ Icc (-R) R, |f' q| ≤ M) :
  |f u - f 0| ≤ M * |u| := by
  have hzero : (0 : ℝ) ∈ Icc (-R) R := by constructor <|> linarith
  have h := (convex_Icc (-R) R).norm_image_sub_le_of_norm_hasFDerivWithin_le
    (C := M)
  (fun q hq => (hderiv q hq).hasFDerivAt.hasFDerivWithinAt)
  (fun q hq => by simp [Real.norm_eq_abs] using hbound q hq)
  hzero hu
  simp [Real.norm_eq_abs] using h
```

MiddleFaceAssembly.EResidual.derivEta

Classification: supplement

Source: MiddleFaceAssembly.lean:239-246

Declaration SHA-256: 3f40afdbac974dd2d7b928190653d4044cf2a42449bc6353b0884369f5086079

Mathematical statement. The explicit eta derivative of the equal-area residual.

Only the exterior type-(iv) angle varies directly with this error coordinate.

```
def derivEta (t xi eta rho sigma : ℝ) : ℝ :=
  formulaDeriv ((lamMid b : ℝ) + t)
  ((b.centers 0 : ℝ) + b.slopes 0 * t + xi)
  ((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta)
  ((b.centers 2 : ℝ) + b.slopes 2 * t + rho)
  ((b.centers 3 : ℝ) + b.slopes 3 * t + b.vRhoSlope * rho + sigma)
  0 0 1 0 0
```

MiddleFaceAssembly.EResidual.derivRho

Classification: supplement

Source: MiddleFaceAssembly.lean:247-254

Declaration SHA-256: 3de77d285680f5b628dc186984128f3143ab98fcad154680680ec2be723c01a7

Mathematical statement. The explicit rho derivative of the equal-area residual.

It differentiates w and the coupled contribution to v .

```
def derivRho (t xi eta rho sigma : ℝ) : ℝ :=
  formulaDeriv ((lamMid b : ℝ) + t)
  ((b.centers 0 : ℝ) + b.slopes 0 * t + xi)
```

```
((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta)
((b.centers 2 : ℝ) + b.slopes 2 * t + rho)
((b.centers 3 : ℝ) + b.slopes 3 * t + b.vRhoSlope * rho + sigma)
0 0 0 1 b.vRhoSlope
```

MiddleFaceAssembly.EResidual.derivSigma

Classification: supplement

Source: MiddleFaceAssembly.lean:255-262

Declaration SHA-256: c526b5c951f77329daf2d0b5ffbf1fbd6d8a07576e3b33918a14d59b930c31ad

Mathematical statement. The explicit sigma derivative of the equal-area residual.

This is the variation in the exterior type-(iii) error angle.

```
def derivSigma (t xi eta rho sigma : ℝ) : ℝ :=
  formulaDeriv ((lamMid b : ℝ) + t)
    ((b.centers 0 : ℝ) + b.slopes 0 * t + xi)
    ((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta)
    ((b.centers 2 : ℝ) + b.slopes 2 * t + rho)
    ((b.centers 3 : ℝ) + b.slopes 3 * t + b.vRhoSlope * rho + sigma)
  0 0 0 0 1
```

MiddleFaceAssembly.EResidual.derivT

Classification: supplement

Source: MiddleFaceAssembly.lean:223-230

Declaration SHA-256: 63d0ee648f437cabdbbba64daa954d7e00b41424d9d35808ce7891e2d6f80a5

Mathematical statement. The equal-area residual's explicit density-coordinate derivative.

It includes all predictor slopes and the direct density derivative.

```
def derivT (t xi eta rho sigma : ℝ) : ℝ :=
  formulaDeriv ((lamMid b : ℝ) + t)
    ((b.centers 0 : ℝ) + b.slopes 0 * t + xi)
    ((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta)
    ((b.centers 2 : ℝ) + b.slopes 2 * t + rho)
    ((b.centers 3 : ℝ) + b.slopes 3 * t + b.vRhoSlope * rho + sigma)
  1 (b.slopes 0) (b.slopes 1) (b.slopes 2) (b.slopes 3)
```

MiddleFaceAssembly.EResidual.derivXi

Classification: supplement

Source: MiddleFaceAssembly.lean:231-238

Declaration SHA-256: d1a73b3463380014d7f1b81b1d3b02ebd95fa6a7e6e2d4551638737c0c79430f

Mathematical statement. The explicit xi derivative of the equal-area residual.

It differentiates x and the coupled contribution to y .

```
def derivXi (t xi eta rho sigma : ℝ) : ℝ :=
  formulaDeriv ((lamMid b : ℝ) + t)
    ((b.centers 0 : ℝ) + b.slopes 0 * t + xi)
    ((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta)
    ((b.centers 2 : ℝ) + b.slopes 2 * t + rho)
    ((b.centers 3 : ℝ) + b.slopes 3 * t + b.vRhoSlope * rho + sigma)
  0 1 b.yXiSlope 0 0
```

MiddleFaceAssembly.EResidual.formulaDeriv

Classification: supplement

Source: MiddleFaceAssembly.lean:151-164

Declaration SHA-256: 7bb93e09ba7a6620ea0d2d9c467898bc5f608acecb169dcb1185ff20303362eb

Mathematical statement. Explicit directional derivative expression for the equal-area residual.

The formula differentiates all angular and density terms with their affine rates.

```
def formulaDeriv (ll xx yy ww vv dl dx dy dw dv : ℝ) : ℝ :=
  let nn4 := ll * yy + π / 2 - xx + cos xx * (sin yy - sin xx)
  let nn3 := ll * vv + π - ww + (cos ww + 2) * (sin vv - sin ww)
  let dn4 := dl * yy + ll * dy - dx +
    (-sin xx * dx) * (sin yy - sin xx) +
    cos xx * (cos yy * dy - cos xx * dx)
  let dn3 := dl * vv + ll * dv - dw +
    (-sin ww * dw) * (sin vv - sin ww) +
    (cos ww + 2) * (cos vv * dv - cos ww * dw)
  (-4 : ℝ) * cos xx * sin xx * dx * nn3 + 2 * cos xx ^ 2 * dn3 +
  2 * (1 + cos ww) * sin ww * dw * nn4 - (1 + cos ww) ^ 2 * dn4

/-- Certificate expression matching `formulaDeriv`. Brick-local interval
certificates supply its atom enclosures. -/
```

MiddleFaceAssembly.EResidual.formulaDerivExpr

Classification: supplement

Source: MiddleFaceAssembly.lean:165-222

Declaration SHA-256: c2d0ab98cf3dfe4dcf08e2029ce80163a7ccf657f84dbd95007f146980c85f86

Mathematical statement. Certificate-expression representation of the equal-area derivative formula.

The expression tree supports sound interval evaluation of the derivative terms.

```
def formulaDerivExpr
  (ell ex ey ew ev esx ecx esy ecy esw ecw esv ecv ep :
    CertificateExpr) (dl dx dy dw dv : ℚ) : CertificateExpr :=
  let nn4 :=
    .add
      (.add
        (.add (.mul ell ey) (.mul (.rational (1 / 2)) ep))
        (.neg ex))
        (.mul ecx (.add esy (.neg esx))))
  let nn3 :=
    .add
      (.add (.add (.mul ell ev) ep) (.neg ew))
      (.mul (.add ecw (.rational 2)) (.add esv (.neg esw)))
  let dn4 :=
    .add
      (.add
        (.add
          (.add (.mul (.rational dl) ey) (.mul ell (.rational dy)))
          (.neg (.rational dx)))
          (.mul
            (.mul (.neg esx) (.rational dx))
            (.add esy (.neg esx))))
        (.mul ecx
          (.add
            (.mul ecy (.rational dy))
            (.neg (.mul ecx (.rational dx))))))
  let dn3 :=
    .add
      (.add
        (.add
          (.add (.mul (.rational dl) ev) (.mul ell (.rational dv)))
```

```

      (.neg (.rational dw)))
    (.mul
      (.mul (.neg esw) (.rational dw))
      (.add esv (.neg esw))))
    (.mul (.add ecw (.rational 2))
      (.add
        (.mul ecv (.rational dv))
        (.neg (.mul ecw (.rational dw))))))
  .add
    (.add
      (.mul
        (.mul
          (.mul (.mul (.rational (-4)) ecx) esx)
          (.rational dx))
          nn3)
        (.mul (.mul (.rational 2) (.mul ecx ecx)) dn3))
      (.add
        (.mul
          (.mul
            (.mul (.mul (.rational 2) (.add (.rational 1) ecw)) esw)
            (.rational dw))
            nn4)
          (.neg
            (.mul
              (.mul (.add (.rational 1) ecw) (.add (.rational 1) ecw))
              dn4))))

```

MiddleFaceAssembly.EResidual.reduced

Classification: supplement

Source: MiddleFaceAssembly.lean:135-144

Declaration SHA-256: ccf5bbc3e7c269d91f270f90b97e975e0915b24935827f4200f4750d122e6b7c

Mathematical statement. The equal-area residual expressed in the five centered affine variables.

It retains all shared-variable correlations needed for face estimates.

```

def reduced (t xi eta rho sigma : ℝ) : ℝ :=
  let ll := (lamMid b : ℝ) + t
  let xx := (b.centers 0 : ℝ) + b.slopes 0 * t + xi
  let yy := (b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta
  let ww := (b.centers 2 : ℝ) + b.slopes 2 * t + rho
  let vv := (b.centers 3 : ℝ) + b.slopes 3 * t + b.vRhoSlope * rho + sigma
  let nn4 := ll * yy + π / 2 - xx + cos xx * (sin yy - sin xx)
  let nn3 := ll * vv + π - ww + (cos ww + 2) * (sin vv - sin ww)
  2 * cos xx ^ 2 * nn3 - (1 + cos ww) ^ 2 * nn4

```

MiddleFaceAssembly.EResidual.reduced_hasDerivAt_eta

Classification: supplement

Source: MiddleFaceAssembly.lean:358-375

Declaration SHA-256: 6360f8f0c5e7362adbbd3906be241fc3112c0fa4aed9c65f6b00b6c0506d2a0e

Mathematical statement. The recorded eta derivative is exact.

Differentiation of the trigonometric residual yields the certificate expression.

```

theorem reduced_hasDerivAt_eta (t xi eta rho sigma : ℝ) :
  HasDerivAt (fun u => reduced b t xi u rho sigma)
    (derivEta b t xi eta rho sigma) eta := by
  have h1 := hasDerivAt_const eta ((lamMid b : ℝ) + t)
  have hx := hasDerivAt_const eta
    ((b.centers 0 : ℝ) + b.slopes 0 * t + xi)
  have hy : HasDerivAt
    (fun u : ℝ => (b.centers 1 : ℝ) + b.slopes 1 * t +

```



```

    b.yXiSlope * xi + u) 1 eta := by
  convert hasDerivAt_affine
  ((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi) 1 0 eta using 1 <=>
    first | rfl | ring_nf
  have hw := hasDerivAt_const eta
    ((b.centers 2 : ℝ) + b.slopes 2 * t + rho)
  have hv := hasDerivAt_const eta
    ((b.centers 3 : ℝ) + b.slopes 3 * t + b.vRhoSlope * rho + sigma)
  exact formula_hasDerivAt h1 hx hy hw hv rfl rfl rfl rfl rfl

```

MiddleFaceAssembly.EResidual.reduced_hasDerivAt_rho

Classification: supplement

Source: MiddleFaceAssembly.lean:376-397

Declaration SHA-256: 5cd6c45099afc8ddab65369fcdbbfab3f07c85f8b5c029f56ce626072c6d9e4f

Mathematical statement. The recorded rho derivative is exact.

Both coupled type-(iii) angles are differentiated with their specified rates.

```

theorem reduced_hasDerivAt_rho (t xi eta rho sigma : ℝ) :
  HasDerivAt (fun u => reduced b t xi eta u sigma)
    (derivRho b t xi eta rho sigma) rho := by
  have h1 := hasDerivAt_const rho ((lamMid b : ℝ) + t)
  have hx := hasDerivAt_const rho
    ((b.centers 0 : ℝ) + b.slopes 0 * t + xi)
  have hy := hasDerivAt_const rho
    ((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta)
  have hw : HasDerivAt
    (fun u : ℝ => (b.centers 2 : ℝ) + b.slopes 2 * t + u) 1 rho := by
    convert hasDerivAt_affine
    ((b.centers 2 : ℝ) + b.slopes 2 * t) 1 0 rho using 1 <=>
      first | rfl | ring_nf
  have hv : HasDerivAt
    (fun u : ℝ => (b.centers 3 : ℝ) + b.slopes 3 * t +
      b.vRhoSlope * u + sigma) (b.vRhoSlope : ℝ) rho := by
    convert hasDerivAt_affine
    ((b.centers 3 : ℝ) + b.slopes 3 * t)
    (b.vRhoSlope : ℝ) sigma rho using 1 <=>
      first | rfl | ring_nf
  exact formula_hasDerivAt h1 hx hy hw hv rfl rfl rfl rfl rfl

```

MiddleFaceAssembly.EResidual.reduced_hasDerivAt_sigma

Classification: supplement

Source: MiddleFaceAssembly.lean:398-418

Declaration SHA-256: bd919b6d5eb11263919e208acc2f1dc4dcb2acf50a28cc2e3a87fcc770f90e3b

Mathematical statement. The recorded sigma derivative is exact.

This validates the last coordinate's contribution to residual variation bounds.

```

theorem reduced_hasDerivAt_sigma (t xi eta rho sigma : ℝ) :
  HasDerivAt (fun u => reduced b t xi eta rho u)
    (derivSigma b t xi eta rho sigma) sigma := by
  have h1 := hasDerivAt_const sigma ((lamMid b : ℝ) + t)
  have hx := hasDerivAt_const sigma
    ((b.centers 0 : ℝ) + b.slopes 0 * t + xi)
  have hy := hasDerivAt_const sigma
    ((b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta)
  have hw := hasDerivAt_const sigma
    ((b.centers 2 : ℝ) + b.slopes 2 * t + rho)
  have hv : HasDerivAt
    (fun u : ℝ => (b.centers 3 : ℝ) + b.slopes 3 * t +
      b.vRhoSlope * rho + u) 1 sigma := by

```

```

    convert hasDerivAt_affine
      ((b.centers 3 : ℝ) + b.slopes 3 * t + b.vRhoSlope * rho)
      1 0 sigma using 1 <=> first | rfl | ring_nf
exact formula_hasDerivAt hl hx hy hw hv rfl rfl rfl rfl rfl

/-- The uniform numerical derivative contract proved separately by each
brick's exact-rational interval certificate on a lambda-centered interval of
radius `R`. -/

```

MiddleFaceAssembly.EResidual.reduced_hasDerivAt_t

Classification: supplement

Source: MiddleFaceAssembly.lean:311-336

Declaration SHA-256: 14eb220aacc6bd2935a6edcbfbb7db3a21ce8b05aba5a15bc8f61479beb9b277

Mathematical statement. The recorded t derivative is the genuine derivative of the reduced equal-area residual.

Chain and product rules justify the density-coordinate enclosure formula.

```

theorem reduced_hasDerivAt_t (t xi eta rho sigma : ℝ) :
  HasDerivAt (fun u => reduced b u xi eta rho sigma)
    (derivT b t xi eta rho sigma) t := by
  have hl : HasDerivAt (fun u : ℝ => (lamMid b : ℝ) + u) 1 t := by
    convert hasDerivAt_affine (lamMid b : ℝ) 1 0 t using 1 <=>
      first | rfl | ring_nf
  have hx : HasDerivAt
    (fun u : ℝ => (b.centers 0 : ℝ) + b.slopes 0 * u + xi)
    (b.slopes 0 : ℝ) t := hasDerivAt_affine _ _ _ _
  have hy : HasDerivAt
    (fun u : ℝ => (b.centers 1 : ℝ) + b.slopes 1 * u +
      b.yXiSlope * xi + eta) (b.slopes 1 : ℝ) t := by
    convert hasDerivAt_affine (b.centers 1 : ℝ) (b.slopes 1 : ℝ)
      ((b.yXiSlope : ℝ) * xi + eta) t using 1 <=>
      first | rfl | ring_nf
  have hw : HasDerivAt
    (fun u : ℝ => (b.centers 2 : ℝ) + b.slopes 2 * u + rho)
    (b.slopes 2 : ℝ) t := hasDerivAt_affine _ _ _ _
  have hv : HasDerivAt
    (fun u : ℝ => (b.centers 3 : ℝ) + b.slopes 3 * u +
      b.vRhoSlope * rho + sigma) (b.slopes 3 : ℝ) t := by
    convert hasDerivAt_affine (b.centers 3 : ℝ) (b.slopes 3 : ℝ)
      ((b.vRhoSlope : ℝ) * rho + sigma) t using 1 <=>
      first | rfl | ring_nf
exact formula_hasDerivAt hl hx hy hw hv rfl rfl rfl rfl rfl

```

MiddleFaceAssembly.EResidual.reduced_hasDerivAt_xi

Classification: supplement

Source: MiddleFaceAssembly.lean:337-357

Declaration SHA-256: 9be284052453dd5776bd813cbdef160106e0042f33d5c122d353506cebfd8680

Mathematical statement. The recorded xi derivative is exact.

The proof preserves the affine coupling between the type-(iv) angles.

```

theorem reduced_hasDerivAt_xi (t xi eta rho sigma : ℝ) :
  HasDerivAt (fun u => reduced b t u eta rho sigma)
    (derivXi b t xi eta rho sigma) xi := by
  have hl := hasDerivAt_const xi ((lamMid b : ℝ) + t)
  have hx : HasDerivAt
    (fun u : ℝ => (b.centers 0 : ℝ) + b.slopes 0 * t + u) 1 xi := by
    convert hasDerivAt_affine
      ((b.centers 0 : ℝ) + b.slopes 0 * t) 1 0 xi using 1 <=>
      first | rfl | ring_nf
  have hy : HasDerivAt

```

```

    (fun u : ℝ => (b.centers 1 : ℝ) + b.slopes 1 * t +
      b.yXiSlope * u + eta) (b.yXiSlope : ℝ) xi := by
  convert hasDerivAt_affine
  ((b.centers 1 : ℝ) + b.slopes 1 * t) (b.yXiSlope : ℝ)
  eta xi using 1 <|> first | rfl | ring_nf
have hw := hasDerivAt_const xi
  ((b.centers 2 : ℝ) + b.slopes 2 * t + rho)
have hv := hasDerivAt_const xi
  ((b.centers 3 : ℝ) + b.slopes 3 * t + b.vRhoSlope * rho + sigma)
exact formula_hasDerivAt hl hx hy hw hv rfl rfl rfl rfl rfl

```

MiddleFaceAssembly.FResidual.derivEta

Classification: supplement

Source: MiddleFaceAssembly.lean:971-977

Declaration SHA-256: 074c2a2bce7a0d615a389c52439e114479e4ad4b8be810f92b43ae51021af99c

Mathematical statement. Explicit eta derivative of the reduced stationary residual.

It measures sensitivity to the exterior type-(iv) angular error.

```

def derivEta (t xi eta : ℝ) : ℝ :=
  let xx := (b.centers 0 : ℝ) + b.slopes 0 * t + xi
  let yy := (b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta
  let ll := (lamMid b : ℝ) + t
  let bb := ll * yy + π / 2 - xx
  cos xx * cos yy - sin xx * cos yy * bb - sin xx * sin yy * ll

```

MiddleFaceAssembly.FResidual.derivT

Classification: supplement

Source: MiddleFaceAssembly.lean:958-970

Declaration SHA-256: 88426adb6d025457273a686218437b5b2fa20ed7c46488942a2e65430d12f11b

Mathematical statement. Explicit density-coordinate derivative of the reduced stationary residual.

All density and predictor dependence is differentiated together.

```

def derivT (t xi eta : ℝ) : ℝ :=
  let xx := (b.centers 0 : ℝ) + b.slopes 0 * t + xi
  let yy := (b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta
  let ll := (lamMid b : ℝ) + t
  let bb := ll * yy + π / 2 - xx
  let dx := (b.slopes 0 : ℝ)
  let dy := (b.slopes 1 : ℝ)
  (-dx) * sin xx * (sin yy - sin xx) +
    cos xx * (dy * cos yy - dx * cos xx) -
    (dx * cos xx * sin yy * bb + sin xx * dy * cos yy * bb +
      sin xx * sin yy * (yy + ll * dy - dx))

/-- Derivative of the correlated `F` residual in the thin `eta` direction. -/

```

MiddleFaceAssembly.FResidual.reduced

Classification: supplement

Source: MiddleFaceAssembly.lean:946-952

Declaration SHA-256: 0e2d8d4b1eb446b26168d3d1be2a5c6a59a042bb296a5fe1b9a5da048c481863

Mathematical statement. The stationary-fold residual after centered affine substitution.

The reduced form is used to estimate variation around its certified anchor.

```

def reduced (t xi eta : ℝ) : ℝ :=
  let xx := (b.centers 0 : ℝ) + b.slopes 0 * t + xi
  let yy := (b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta
  let ll := (lamMid b : ℝ) + t
  cos xx * (sin yy - sin xx) -
    sin xx * sin yy * (ll * yy + π / 2 - xx)

```

MiddleFaceAssembly.FResidual.reduced_hasDerivAt_eta

Classification: supplement

Source: MiddleFaceAssembly.lean:1009-1036

Declaration SHA-256: 3fc54562c683d5fdcf05f0c5a400e333c95946017d61238476c9d5e2fb027ad3

Mathematical statement. The reduced stationary residual has the recorded eta derivative.

The trigonometric product rule proves the sensitivity formula.

```

theorem reduced_hasDerivAt_eta (t xi eta : ℝ) :
  HasDerivAt (fun u => reduced b t xi u) (derivEta b t xi eta) eta := by
  let xx := (b.centers 0 : ℝ) + b.slopes 0 * t + xi
  let yy := (b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta
  have hy : HasDerivAt
    (fun u : ℝ => (b.centers 1 : ℝ) + b.slopes 1 * t +
      b.yXiSlope * xi + u) 1 eta := by
  convert (hasDerivAt_const eta
    ((b.centers 1 : ℝ) + b.slopes 1 * t +
      b.yXiSlope * xi)).add (hasDerivAt_id eta) using 1 <|>
  first | rfl | ring_nf
  have hys : HasDerivAt
    (fun u : ℝ => sin ((b.centers 1 : ℝ) + b.slopes 1 * t +
      b.yXiSlope * xi + u)) (cos yy) eta := by
  convert (Real.hasDerivAt_sin yy).comp eta hy using 1 <|> try rfl
  ring
  have hb := (((hasDerivAt_const eta ((lamMid b : ℝ) + t)).mul hy).add
    (hasDerivAt_const eta (π / 2))).sub (hasDerivAt_const eta xx)
  convert ((hasDerivAt_const eta (cos xx)).mul
    (hys.sub (hasDerivAt_const eta (sin xx)))).sub
    (((hasDerivAt_const eta (sin xx)).mul hys).mul hb) using 1 <|>
  try rfl
  simp [derivEta, xx, yy]
  ring

/-- Uniform numerical derivative contract proved by each brick's exact
rational enclosure certificate on a lambda-centered interval of radius `R`.
The `t` derivative is needed only on the `eta = 0` leg of the path. -/

```

MiddleFaceAssembly.FResidual.reduced_hasDerivAt_t

Classification: supplement

Source: MiddleFaceAssembly.lean:978-1008

Declaration SHA-256: e3ee85a81e52b7d5809d62785536115b9fa172b7ee957d2ee76c2badb09aafc0

Mathematical statement. The reduced stationary residual has the recorded t derivative.

This justifies the density variation bound used in face certification.

```

theorem reduced_hasDerivAt_t (t xi eta : ℝ) :
  HasDerivAt (fun u => reduced b u xi eta) (derivT b t xi eta) t := by
  let xx := (b.centers 0 : ℝ) + b.slopes 0 * t + xi
  let yy := (b.centers 1 : ℝ) + b.slopes 1 * t + b.yXiSlope * xi + eta
  have hx : HasDerivAt
    (fun u : ℝ => (b.centers 0 : ℝ) + b.slopes 0 * u + xi)
    (b.slopes 0 : ℝ) t := by
  convert ((hasDerivAt_const t (b.centers 0 : ℝ)).add
    ((hasDerivAt_const t (b.slopes 0 : ℝ)).mul (hasDerivAt_id t))).add

```

```

      (hasDerivAt_const t xi) using 1 <=>
    first | rfl | ring_nf
  have hy : HasDerivAt
    (fun u : ℝ => (b.centers 1 : ℝ) + b.slopes 1 * u +
      b.yXiSlope * xi + eta) (b.slopes 1 : ℝ) t := by
    convert (((hasDerivAt_const t (b.centers 1 : ℝ)).add
      ((hasDerivAt_const t (b.slopes 1 : ℝ)).mul (hasDerivAt_id t))).add
      (hasDerivAt_const t ((b.yXiSlope : ℝ) * xi))).add
      (hasDerivAt_const t eta) using 1 <=>
    first | rfl | ring_nf
  have hl : HasDerivAt (fun u : ℝ => (lamMid b : ℝ) + u) 1 t := by
    convert (hasDerivAt_const t (lamMid b : ℝ)).add (hasDerivAt_id t) using 1 <=>
    first | rfl | ring_nf
  have hxxs := (Real.hasDerivAt_sin xx).comp t hx
  have hxxc := (Real.hasDerivAt_cos xx).comp t hx
  have hyys := (Real.hasDerivAt_sin yy).comp t hy
  have hb := (((hl.mul hy).add (hasDerivAt_const t (π / 2))).sub hx)
  convert (hxxc.mul (hyys.sub hxxs)).sub ((hxxs.mul hyys).mul hb) using 1 <=>
  try rfl
  simp [Function.comp_apply, xx, yy, derivT]
  ring

```

MiddleFaceAssembly.InSymmetricBox

Classification: supplement

Source: MiddleFaceAssembly.lean:24-26

Declaration SHA-256: 7113db679549bacbee758495cf87555b49cdbe440060902d1105828bd48ba194

Mathematical statement. Coordinatewise membership in a symmetric four-dimensional box.

Each coordinate satisfies $-r_j \leq z_j \leq r_j$.

```

abbrev InSymmetricBox := ScalarSuffixCertificate.InSymmetricBox

/-- Exact rational data defining one centered middle-face brick. -/

```

MiddleFaceAssembly.Model.lam

Classification: supplement

Source: MiddleFaceAssembly.lean:62-62

Declaration SHA-256: 09c8d628c82b18ad9565a08f3c76eeb7195571504223cb2dac220bc1b4b12c5a

Mathematical statement. Density reconstructed from a centered brick point.

$\$ \lambda = \lambda_{\text{mid}} + t$.

```

def lam (p : Point) : ℝ := (lamMid b : ℝ) + p.t

```

MiddleFaceAssembly.Model.pointOf

Classification: supplement

Source: MiddleFaceAssembly.lean:92-93

Declaration SHA-256: 123eaac2727784b857a8a1d07bab6c6e08557c023e8c73b6b961970da83630664

Mathematical statement. A centered point built from density t and a four-vector of errors.

The vector entries become ξ, η, ρ, σ .

```

def pointOf (t : ℝ) (z : Vec4) : Point := {t, z 0, z 1, z 2, z 3}

```

MiddleFaceAssembly.Model.radiiReal

Classification: supplement

Source: MiddleFaceAssembly.lean:91-91

Declaration SHA-256: 1a51bc7fbcd7528bb90b612010e93b671df7d5284f9acc9f65a81a654617ee53

Mathematical statement. The brick's rational error radii cast to reals.

They define the real box used by the root theorem.

```
def radiiReal : Vec4 := fun j => (b.radii j : ℝ)
```

MiddleFaceAssembly.Model.v

Classification: supplement

Source: MiddleFaceAssembly.lean:66-67

Declaration SHA-256: 384327e74651ca26a49098121b5edc0d2aec2203e7adb9b9c2f01e9786a1b96b

Mathematical statement. The coupled type-(iii) exterior-angle predictor.

$$v = c_3 + m_3 t + k_v \rho + \sigma.$$

```
def v (p : Point) : ℝ := b.centers 3 + b.slopes 3 * p.t + b.vRhoSlope * p.rho + p.sigma
```

MiddleFaceAssembly.Model.w

Classification: supplement

Source: MiddleFaceAssembly.lean:65-65

Declaration SHA-256: e0c3c0b0f31f78a2f6c2d6cd4c7b667830ebe97b8269a630415448ba726152aa

Mathematical statement. The affine type-(iii) inner-angle predictor with rho error.

$$w = c_2 + m_2 t + \rho.$$

```
def w (p : Point) : ℝ := b.centers 2 + b.slopes 2 * p.t + p.rho
```

MiddleFaceAssembly.Model.x

Classification: supplement

Source: MiddleFaceAssembly.lean:63-63

Declaration SHA-256: 4080f6cd78cee5e77729c1d21f119ce966fec918b9c6006a3798cb8dc3bc93f

Mathematical statement. The affine type-(iv) inner-angle predictor with xi error.

$$x = c_0 + m_0 t + \xi.$$

```
def x (p : Point) : ℝ := b.centers 0 + b.slopes 0 * p.t + p.xi
```

MiddleFaceAssembly.Model.y

Classification: supplement

Source: MiddleFaceAssembly.lean:64-64

Declaration SHA-256: e9aa09104ad8189cc4fa9f32855b42af7f901b88654816cd3974a6e8c5d751d4

Mathematical statement. The coupled type-(iv) exterior-angle predictor.

$$y = c_1 + m_1 t + k_y \xi + \eta.$$

```
def y (p : Point) : ℝ := b.centers 1 + b.slopes 1 * p.t + b.yXiSlope * p.xi + p.eta
```

MiddleFaceAssembly.Point

Classification: supplement

Source: MiddleFaceAssembly.lean:50-61

Declaration SHA-256: f6b6070cff3d67fea5f3a1bb07944579342b7bc314fc32fbf625a57ff72fc6

Mathematical statement. A centered density parameter and four real error coordinates.

The coordinates are $(t, \xi, \eta, \rho, \sigma)$.

```
structure Point where
  t : ℝ
  xi : ℝ
  eta : ℝ
  rho : ℝ
  sigma : ℝ

/- Exact analytic model attached to a rational brick. -/
namespace Model

variable (b : Brick)
```

MiddleFaceAssembly.QInterval

Classification: supplement

Source: MiddleFaceAssembly.lean:22-22

Declaration SHA-256: ab557efab6ff5d67c61f860d37b3645b971e70e7a32bfb18825874961bbbc432

Mathematical statement. The shared rational interval type in middle-face assembly.

The abbreviation preserves the same exact real-containment semantics.

```
abbrev QInterval := ScalarSuffixCertificate.QInterval
```

MiddleFaceAssembly.Vec4

Classification: supplement

Source: MiddleFaceAssembly.lean:23-23

Declaration SHA-256: 26b6384e4234f00c2040b7fed071a12c3e6f967500dfcaee15eb69b43a2cf5e3

Mathematical statement. A real vector indexed by four coordinates.

It holds the four error variables of the face root problem.

```
abbrev Vec4 := ScalarSuffixCertificate.Vec4
```

MiddleFaceAssembly.lamMid

Classification: supplement

Source: MiddleFaceAssembly.lean:38-38

Declaration SHA-256: 2d4db33efad481c4328c1c7dfb050b5a976c6575fe3896f4791861bbf4743632

Mathematical statement. The rational midpoint of a brick's density interval.

It is $(\lambda_{\text{lo}} + \lambda_{\text{hi}})/2$.

```
abbrev lamMid (b : Brick) : ℚ := (b.lamLo + b.lamHi) / 2
```

MiddleFaceAssembly.tInterval

Classification: supplement

Source: MiddleFaceAssembly.lean:41-45

Declaration SHA-256: f0133991a8583c976d09c8410dc69827234287519ce36ced5debf237d26a4d63

Mathematical statement. The centered density-parameter interval.

It is $[-t_{\text{radius}}, t_{\text{radius}}]$.

```
def tInterval (b : Brick) : QInterval :=
  ⟨-tRadius b, tRadius b, by
    dsimp [tRadius]
    linarith [b.lamOrdered]⟩
```

MiddleFaceAssembly.tRadius

Classification: supplement

Source: MiddleFaceAssembly.lean:39-40

Declaration SHA-256: 18f0ba106ace07af0a9a6397628ed7aba1c4508d8c6f08edfb081b20e3385643

Mathematical statement. Half the width of the brick's density interval.

It is $(\lambda_{\text{hi}} - \lambda_{\text{lo}})/2 > 0$.

```
abbrev tRadius (b : Brick) : ℚ := (b.lamHi - b.lamLo) / 2
```

MiddleFaceCells0To19.candidate_not_isWeightedPerimeterMinimizer

Classification: supplement

Source: MiddleFaceCells0To19.lean:40-265

Declaration SHA-256: b28637e44e18b6358362a61f82cb49563ab505b8a06a5dc1be41f5b00b1477fb

Mathematical statement. Candidate exclusion over the union of middle-face cells zero through nineteen.

Exact rational seam comparisons combine the cell boundary theorems into one closed-interval result.

```
theorem candidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ}
  (hlower : (102001 / 100000 : ℝ) ≤ lam)
  (hupper : lam ≤ (102201 / 100000 : ℝ))
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  by_cases hsplit10 : lam ≤ (102101 / 100000 : ℝ)
  · by_cases hsplit5 : lam ≤ (102051 / 100000 : ℝ)
    · by_cases hsplit2 : lam ≤ (102021 / 100000 : ℝ)
      · by_cases hsplit1 : lam ≤ (102011 / 100000 : ℝ)
        · have hcell : MiddleFaceCell0.lamInterval.RealContains lam := by
            norm_num [MiddleFaceCell0.lamInterval, MiddleFaceCell0.brick,
              LeanSuffixReflective.QInterval.RealContains]
            at hlower hsplit1 ⊢
            exact ⟨hlower, hsplit1⟩
        exact
          MiddleFaceCell0.candidate_not_isWeightedPerimeterMinimizer
            hcell candidate hcandidate
      · have hlower1 : (102011 / 100000 : ℝ) ≤ lam :=
```



```

(not_le.mp hsplit1).le
have hcell : MiddleFaceCell1.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell1.lamInterval, MiddleFaceCell1.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower1 hsplit2 ⊢
  exact ⟨hlower1, hsplit2⟩
exact
MiddleFaceCell1.candidate_not_isWeightedPerimeterMinimizer
hcell candidate hcandidate
· have hlower2 : (102021 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit2).le
by_cases hsplit3 : lam ≤ (102031 / 100000 : ℝ)
· have hcell : MiddleFaceCell2.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell2.lamInterval, MiddleFaceCell2.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower2 hsplit3 ⊢
  exact ⟨hlower2, hsplit3⟩
exact
MiddleFaceCell2.candidate_not_isWeightedPerimeterMinimizer
hcell candidate hcandidate
· have hlower3 : (102031 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit3).le
by_cases hsplit4 : lam ≤ (102041 / 100000 : ℝ)
· have hcell : MiddleFaceCell3.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell3.lamInterval, MiddleFaceCell3.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower3 hsplit4 ⊢
  exact ⟨hlower3, hsplit4⟩
exact
MiddleFaceCell3.candidate_not_isWeightedPerimeterMinimizer
hcell candidate hcandidate
· have hlower4 : (102041 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit4).le
have hcell : MiddleFaceCell4.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell4.lamInterval, MiddleFaceCell4.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower4 hsplit5 ⊢
  exact ⟨hlower4, hsplit5⟩
exact
MiddleFaceCell4.candidate_not_isWeightedPerimeterMinimizer
hcell candidate hcandidate
· have hlower5 : (102051 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit5).le
by_cases hsplit7 : lam ≤ (102071 / 100000 : ℝ)
· by_cases hsplit6 : lam ≤ (102061 / 100000 : ℝ)
· have hcell : MiddleFaceCell5.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell5.lamInterval, MiddleFaceCell5.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower5 hsplit6 ⊢
  exact ⟨hlower5, hsplit6⟩
exact
MiddleFaceCell5.candidate_not_isWeightedPerimeterMinimizer
hcell candidate hcandidate
· have hlower6 : (102061 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit6).le
have hcell : MiddleFaceCell6.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell6.lamInterval, MiddleFaceCell6.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower6 hsplit7 ⊢
  exact ⟨hlower6, hsplit7⟩
exact
MiddleFaceCell6.candidate_not_isWeightedPerimeterMinimizer
hcell candidate hcandidate
· have hlower7 : (102071 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit7).le
by_cases hsplit8 : lam ≤ (102081 / 100000 : ℝ)
· have hcell : MiddleFaceCell7.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell7.lamInterval, MiddleFaceCell7.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower7 hsplit8 ⊢
  exact ⟨hlower7, hsplit8⟩
exact

```

```

MiddleFaceCell7.candidate_not_isWeightedPerimeterMinimizer
hcell candidate hcandidate
· have hlower8 : (102081 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit8).le
by_cases hsplit9 : lam ≤ (102091 / 100000 : ℝ)
· have hcell : MiddleFaceCell8.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell8.lamInterval, MiddleFaceCell8.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower8 hsplit9 ⊢
  exact ⟨hlower8, hsplit9⟩
exact
  MiddleFaceCell8.candidate_not_isWeightedPerimeterMinimizer
  hcell candidate hcandidate
· have hlower9 : (102091 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit9).le
have hcell : MiddleFaceCell9.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell9.lamInterval, MiddleFaceCell9.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower9 hsplit10 ⊢
  exact ⟨hlower9, hsplit10⟩
exact
  MiddleFaceCell9.candidate_not_isWeightedPerimeterMinimizer
  hcell candidate hcandidate
· have hlower10 : (102101 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit10).le
by_cases hsplit15 : lam ≤ (102151 / 100000 : ℝ)
· by_cases hsplit12 : lam ≤ (102121 / 100000 : ℝ)
  · by_cases hsplit11 : lam ≤ (102111 / 100000 : ℝ)
    · have hcell : MiddleFaceCell10.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell10.lamInterval, MiddleFaceCell10.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower10 hsplit11 ⊢
      exact ⟨hlower10, hsplit11⟩
    exact
      MiddleFaceCell10.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower11 : (102111 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit11).le
  have hcell : MiddleFaceCell11.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell11.lamInterval, MiddleFaceCell11.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower11 hsplit12 ⊢
    exact ⟨hlower11, hsplit12⟩
  exact
    MiddleFaceCell11.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower12 : (102121 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit12).le
by_cases hsplit13 : lam ≤ (102131 / 100000 : ℝ)
· have hcell : MiddleFaceCell12.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell12.lamInterval, MiddleFaceCell12.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower12 hsplit13 ⊢
  exact ⟨hlower12, hsplit13⟩
exact
  MiddleFaceCell12.candidate_not_isWeightedPerimeterMinimizer
  hcell candidate hcandidate
· have hlower13 : (102131 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit13).le
by_cases hsplit14 : lam ≤ (102141 / 100000 : ℝ)
· have hcell : MiddleFaceCell13.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell13.lamInterval, MiddleFaceCell13.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower13 hsplit14 ⊢
  exact ⟨hlower13, hsplit14⟩
exact
  MiddleFaceCell13.candidate_not_isWeightedPerimeterMinimizer
  hcell candidate hcandidate
· have hlower14 : (102141 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit14).le
have hcell : MiddleFaceCell14.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell14.lamInterval, MiddleFaceCell14.brick,

```

```

        LeanSuffixReflective.QInterval.RealContains]
      at hlower14 hsplit15 ⊢
    exact ⟨hlower14, hsplit15⟩
  exact
    MiddleFaceCell14.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
· have hlower15 : (102151 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit15).le
by_cases hsplit17 : lam ≤ (102171 / 100000 : ℝ)
· by_cases hsplit16 : lam ≤ (102161 / 100000 : ℝ)
  · have hcell : MiddleFaceCell15.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell15.lamInterval, MiddleFaceCell15.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower15 hsplit16 ⊢
    exact ⟨hlower15, hsplit16⟩
  exact
    MiddleFaceCell15.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
· have hlower16 : (102161 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit16).le
have hcell : MiddleFaceCell16.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell16.lamInterval, MiddleFaceCell16.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower16 hsplit17 ⊢
  exact ⟨hlower16, hsplit17⟩
exact
  MiddleFaceCell16.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower17 : (102171 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit17).le
by_cases hsplit18 : lam ≤ (102181 / 100000 : ℝ)
· have hcell : MiddleFaceCell17.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell17.lamInterval, MiddleFaceCell17.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower17 hsplit18 ⊢
  exact ⟨hlower17, hsplit18⟩
exact
  MiddleFaceCell17.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower18 : (102181 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit18).le
by_cases hsplit19 : lam ≤ (102191 / 100000 : ℝ)
· have hcell : MiddleFaceCell18.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell18.lamInterval, MiddleFaceCell18.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower18 hsplit19 ⊢
  exact ⟨hlower18, hsplit19⟩
exact
  MiddleFaceCell18.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower19 : (102191 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit19).le
have hcell : MiddleFaceCell19.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell19.lamInterval, MiddleFaceCell19.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower19 hupper ⊢
  exact ⟨hlower19, hupper⟩
exact
  MiddleFaceCell19.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate

```

end MiddleFaceCells0To19

MiddleFaceCells104To112.candidate_not_isWeightedPerimeterMinimizer

Classification: supplement

Source: MiddleFaceCells104To112.lean:29-133

Declaration SHA-256: 4570e7b8cef789aedf57b10eef603ac2c0b313f81a77a3d704cb2879ca1fd52d

Mathematical statement. Candidate exclusion over middle-face cells 104 through 112.

This final batch reaches the right-prototype seam 25781/25000.

```
theorem candidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ}
  (hlower : (103041 / 100000 : ℝ) ≤ lam)
  (hupper : lam ≤ (25781 / 25000 : ℝ))
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
by_cases hsplit108 : lam ≤ (103081 / 100000 : ℝ)
· by_cases hsplit106 : lam ≤ (103061 / 100000 : ℝ)
  · by_cases hsplit105 : lam ≤ (103051 / 100000 : ℝ)
    · have hcell : MiddleFaceCell104.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell104.lamInterval, MiddleFaceCell104.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower hsplit105 ⊢
      exact ⟨hlower, hsplit105⟩
    exact
      MiddleFaceCell104.candidate_not_isWeightedPerimeterMinimizer
        hcell candidate hcandidate
  · have hlower105 : (103051 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit105).le
    have hcell : MiddleFaceCell105.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell105.lamInterval, MiddleFaceCell105.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower105 hsplit106 ⊢
      exact ⟨hlower105, hsplit106⟩
    exact
      MiddleFaceCell105.candidate_not_isWeightedPerimeterMinimizer
        hcell candidate hcandidate
  · have hlower106 : (103061 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit106).le
    by_cases hsplit107 : lam ≤ (103071 / 100000 : ℝ)
    · have hcell : MiddleFaceCell106.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell106.lamInterval, MiddleFaceCell106.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower106 hsplit107 ⊢
      exact ⟨hlower106, hsplit107⟩
    exact
      MiddleFaceCell106.candidate_not_isWeightedPerimeterMinimizer
        hcell candidate hcandidate
  · have hlower107 : (103071 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit107).le
    have hcell : MiddleFaceCell107.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell107.lamInterval, MiddleFaceCell107.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower107 hsplit108 ⊢
      exact ⟨hlower107, hsplit108⟩
    exact
      MiddleFaceCell107.candidate_not_isWeightedPerimeterMinimizer
        hcell candidate hcandidate
  · have hlower108 : (103081 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit108).le
    by_cases hsplit110 : lam ≤ (103101 / 100000 : ℝ)
    · by_cases hsplit109 : lam ≤ (103091 / 100000 : ℝ)
      · have hcell : MiddleFaceCell108.lamInterval.RealContains lam := by
        norm_num [MiddleFaceCell108.lamInterval, MiddleFaceCell108.brick,
          LeanSuffixReflective.QInterval.RealContains]
        at hlower108 hsplit109 ⊢
        exact ⟨hlower108, hsplit109⟩
      exact
        MiddleFaceCell108.candidate_not_isWeightedPerimeterMinimizer
```

```

      hcell candidate hcandidate
    · have hlower109 : (103091 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit109).le
    have hcell : MiddleFaceCell109.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell109.lamInterval, MiddleFaceCell109.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower109 hsplit110 ⊢
      exact ⟨hlower109, hsplit110⟩
    exact
      MiddleFaceCell109.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
    · have hlower110 : (103101 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit110).le
    by_cases hsplit111 : lam ≤ (103111 / 100000 : ℝ)
    · have hcell : MiddleFaceCell110.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell110.lamInterval, MiddleFaceCell110.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower110 hsplit111 ⊢
      exact ⟨hlower110, hsplit111⟩
    exact
      MiddleFaceCell110.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
    · have hlower111 : (103111 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit111).le
    by_cases hsplit112 : lam ≤ (103121 / 100000 : ℝ)
    · have hcell : MiddleFaceCell111.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell111.lamInterval, MiddleFaceCell111.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower111 hsplit112 ⊢
      exact ⟨hlower111, hsplit112⟩
    exact
      MiddleFaceCell111.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
    · have hlower112 : (103121 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit112).le
    have hcell : MiddleFaceCell112.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell112.lamInterval, MiddleFaceCell112.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower112 hupper ⊢
      exact ⟨hlower112, hupper⟩
    exact
      MiddleFaceCell112.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate

```

end MiddleFaceCells104To112

MiddleFaceCells20To31.candidate_not_isWeightedPerimeterMinimizer

Classification: supplement

Source: MiddleFaceCells20To31.lean:32-169

Declaration SHA-256: a93747927463ab3765198fcfbb919eb2715b5ee65fc623d802fbd018e733d4eb

Mathematical statement. Candidate exclusion over middle-face cells twenty through thirty-one.

The proof composes the exact adjacent closed cell intervals.

```

theorem candidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ}
  (hlower : (102201 / 100000 : ℝ) ≤ lam)
  (hupper : lam ≤ (102321 / 100000 : ℝ))
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
by_cases hsplit26 : lam ≤ (102261 / 100000 : ℝ)
· by_cases hsplit23 : lam ≤ (102231 / 100000 : ℝ)
· by_cases hsplit21 : lam ≤ (102211 / 100000 : ℝ)
  · have hcell : MiddleFaceCell20.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell20.lamInterval, MiddleFaceCell20.brick,
      LeanSuffixReflective.QInterval.RealContains]

```

```

    at hlower hsplit21 ⊢
    exact ⟨hlower, hsplit21⟩
  exact
    MiddleFaceCell20.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
  · have hlower21 : (102211 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit21).le
  by_cases hsplit22 : lam ≤ (102221 / 100000 : ℝ)
  · have hcell : MiddleFaceCell21.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell21.lamInterval, MiddleFaceCell21.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower21 hsplit22 ⊢
    exact ⟨hlower21, hsplit22⟩
  exact
    MiddleFaceCell21.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
  · have hlower22 : (102221 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit22).le
  have hcell : MiddleFaceCell22.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell22.lamInterval, MiddleFaceCell22.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower22 hsplit23 ⊢
    exact ⟨hlower22, hsplit23⟩
  exact
    MiddleFaceCell22.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
  · have hlower23 : (102231 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit23).le
  by_cases hsplit24 : lam ≤ (102241 / 100000 : ℝ)
  · have hcell : MiddleFaceCell23.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell23.lamInterval, MiddleFaceCell23.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower23 hsplit24 ⊢
    exact ⟨hlower23, hsplit24⟩
  exact
    MiddleFaceCell23.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
  · have hlower24 : (102241 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit24).le
  by_cases hsplit25 : lam ≤ (102251 / 100000 : ℝ)
  · have hcell : MiddleFaceCell24.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell24.lamInterval, MiddleFaceCell24.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower24 hsplit25 ⊢
    exact ⟨hlower24, hsplit25⟩
  exact
    MiddleFaceCell24.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
  · have hlower25 : (102251 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit25).le
  have hcell : MiddleFaceCell25.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell25.lamInterval, MiddleFaceCell25.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower25 hsplit26 ⊢
    exact ⟨hlower25, hsplit26⟩
  exact
    MiddleFaceCell25.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
  · have hlower26 : (102261 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit26).le
  by_cases hsplit29 : lam ≤ (102291 / 100000 : ℝ)
  · by_cases hsplit27 : lam ≤ (102271 / 100000 : ℝ)
  · have hcell : MiddleFaceCell26.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell26.lamInterval, MiddleFaceCell26.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower26 hsplit27 ⊢
    exact ⟨hlower26, hsplit27⟩
  exact
    MiddleFaceCell26.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
  · have hlower27 : (102271 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit27).le

```

```

by_cases hsplit28 : lam ≤ (102281 / 100000 : ℝ)
· have hcell : MiddleFaceCell27.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell27.lamInterval, MiddleFaceCell27.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower27 hsplit28 ⊢
  exact ⟨hlower27, hsplit28⟩
exact
  MiddleFaceCell27.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower28 : (102281 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit28).le
have hcell : MiddleFaceCell28.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell28.lamInterval, MiddleFaceCell28.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower28 hsplit29 ⊢
  exact ⟨hlower28, hsplit29⟩
exact
  MiddleFaceCell28.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower29 : (102291 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit29).le
by_cases hsplit30 : lam ≤ (102301 / 100000 : ℝ)
· have hcell : MiddleFaceCell29.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell29.lamInterval, MiddleFaceCell29.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower29 hsplit30 ⊢
  exact ⟨hlower29, hsplit30⟩
exact
  MiddleFaceCell29.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower30 : (102301 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit30).le
by_cases hsplit31 : lam ≤ (102311 / 100000 : ℝ)
· have hcell : MiddleFaceCell30.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell30.lamInterval, MiddleFaceCell30.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower30 hsplit31 ⊢
  exact ⟨hlower30, hsplit31⟩
exact
  MiddleFaceCell30.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower31 : (102311 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit31).le
have hcell : MiddleFaceCell31.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell31.lamInterval, MiddleFaceCell31.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower31 hupper ⊢
  exact ⟨hlower31, hupper⟩
exact
  MiddleFaceCell31.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate

end MiddleFaceCells20To31

```

MiddleFaceCells32To43.candidate_not_isWeightedPerimeterMinimizer

Classification: supplement

Source: MiddleFaceCells32To43.lean:32-169

Declaration SHA-256: 2e75386747807f06bb44919c15005abd3135f1cb638447100a8d72debc73bacf

Mathematical statement. Candidate exclusion over middle-face cells thirty-two through forty-three.

The batch theorem is finite interval-coverage bookkeeping.

```

theorem candidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ}
  (hlower : (102321 / 100000 : ℝ) ≤ lam)
  (hupper : lam ≤ (102441 / 100000 : ℝ))
  (candidate : _root_.FourArcCandidate lam)

```

```

(hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
by_cases hsplit38 : lam ≤ (102381 / 100000 : ℝ)
· by_cases hsplit35 : lam ≤ (102351 / 100000 : ℝ)
· by_cases hsplit33 : lam ≤ (102331 / 100000 : ℝ)
  · have hcell : MiddleFaceCell32.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell32.lamInterval, MiddleFaceCell32.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower hsplit33 ⊢
    exact ⟨hlower, hsplit33⟩
  exact
    MiddleFaceCell32.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower33 : (102331 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit33).le
  by_cases hsplit34 : lam ≤ (102341 / 100000 : ℝ)
  · have hcell : MiddleFaceCell33.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell33.lamInterval, MiddleFaceCell33.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower33 hsplit34 ⊢
    exact ⟨hlower33, hsplit34⟩
  exact
    MiddleFaceCell33.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower34 : (102341 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit34).le
  have hcell : MiddleFaceCell34.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell34.lamInterval, MiddleFaceCell34.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower34 hsplit35 ⊢
    exact ⟨hlower34, hsplit35⟩
  exact
    MiddleFaceCell34.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower35 : (102351 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit35).le
  by_cases hsplit36 : lam ≤ (102361 / 100000 : ℝ)
  · have hcell : MiddleFaceCell35.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell35.lamInterval, MiddleFaceCell35.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower35 hsplit36 ⊢
    exact ⟨hlower35, hsplit36⟩
  exact
    MiddleFaceCell35.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower36 : (102361 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit36).le
  by_cases hsplit37 : lam ≤ (102371 / 100000 : ℝ)
  · have hcell : MiddleFaceCell36.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell36.lamInterval, MiddleFaceCell36.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower36 hsplit37 ⊢
    exact ⟨hlower36, hsplit37⟩
  exact
    MiddleFaceCell36.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower37 : (102371 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit37).le
  have hcell : MiddleFaceCell37.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell37.lamInterval, MiddleFaceCell37.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower37 hsplit38 ⊢
    exact ⟨hlower37, hsplit38⟩
  exact
    MiddleFaceCell37.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower38 : (102381 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit38).le
  by_cases hsplit41 : lam ≤ (102411 / 100000 : ℝ)
  · by_cases hsplit39 : lam ≤ (102391 / 100000 : ℝ)
    · have hcell : MiddleFaceCell38.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell38.lamInterval, MiddleFaceCell38.brick,

```



```

        LeanSuffixReflective.QInterval.RealContains]
      at hlower38 hsplit39 ⊢
    exact ⟨hlower38, hsplit39⟩
  exact
    MiddleFaceCell38.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower39 : (102391 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit39).le
  by_cases hsplit40 : lam ≤ (102401 / 100000 : ℝ)
  · have hcell : MiddleFaceCell39.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell39.lamInterval, MiddleFaceCell39.brick,
      LeanSuffixReflective.QInterval.RealContains]
      at hlower39 hsplit40 ⊢
    exact ⟨hlower39, hsplit40⟩
  exact
    MiddleFaceCell39.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower40 : (102401 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit40).le
  have hcell : MiddleFaceCell40.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell40.lamInterval, MiddleFaceCell40.brick,
      LeanSuffixReflective.QInterval.RealContains]
      at hlower40 hsplit41 ⊢
    exact ⟨hlower40, hsplit41⟩
  exact
    MiddleFaceCell40.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower41 : (102411 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit41).le
  by_cases hsplit42 : lam ≤ (102421 / 100000 : ℝ)
  · have hcell : MiddleFaceCell41.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell41.lamInterval, MiddleFaceCell41.brick,
      LeanSuffixReflective.QInterval.RealContains]
      at hlower41 hsplit42 ⊢
    exact ⟨hlower41, hsplit42⟩
  exact
    MiddleFaceCell41.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower42 : (102421 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit42).le
  by_cases hsplit43 : lam ≤ (102431 / 100000 : ℝ)
  · have hcell : MiddleFaceCell42.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell42.lamInterval, MiddleFaceCell42.brick,
      LeanSuffixReflective.QInterval.RealContains]
      at hlower42 hsplit43 ⊢
    exact ⟨hlower42, hsplit43⟩
  exact
    MiddleFaceCell42.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower43 : (102431 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit43).le
  have hcell : MiddleFaceCell43.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell43.lamInterval, MiddleFaceCell43.brick,
      LeanSuffixReflective.QInterval.RealContains]
      at hlower43 hupper ⊢
    exact ⟨hlower43, hupper⟩
  exact
    MiddleFaceCell43.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
end MiddleFaceCells32To43

```

MiddleFaceCells44To55.candidate_not_isWeightedPerimeterMinimizer

Classification: supplement

Source: MiddleFaceCells44To55.lean: 32-169

Declaration SHA-256: 3d231e37509eafaa66d654d02538cb90db625899f8572fbb742c157a20888d4a

Mathematical statement. Candidate exclusion over middle-face cells forty-four through fifty-five.

All densities in the batch interval belong to a certified cell.

```

theorem candidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ}
  (hlower : (102441 / 100000 : ℝ) ≤ lam)
  (hupper : lam ≤ (102561 / 100000 : ℝ))
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
by_cases hsplit50 : lam ≤ (102501 / 100000 : ℝ)
· by_cases hsplit47 : lam ≤ (102471 / 100000 : ℝ)
  · by_cases hsplit45 : lam ≤ (102451 / 100000 : ℝ)
    · have hcell : MiddleFaceCell144.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell144.lamInterval, MiddleFaceCell144.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower hsplit45 ⊢
      exact ⟨hlower, hsplit45⟩
    exact
      MiddleFaceCell144.candidate_not_isWeightedPerimeterMinimizer
        hcell candidate hcandidate
  · have hlower45 : (102451 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit45).le
    by_cases hsplit46 : lam ≤ (102461 / 100000 : ℝ)
      · have hcell : MiddleFaceCell145.lamInterval.RealContains lam := by
        norm_num [MiddleFaceCell145.lamInterval, MiddleFaceCell145.brick,
          LeanSuffixReflective.QInterval.RealContains]
        at hlower45 hsplit46 ⊢
        exact ⟨hlower45, hsplit46⟩
      exact
        MiddleFaceCell145.candidate_not_isWeightedPerimeterMinimizer
          hcell candidate hcandidate
    · have hlower46 : (102461 / 100000 : ℝ) ≤ lam :=
        (not_le.mp hsplit46).le
      have hcell : MiddleFaceCell146.lamInterval.RealContains lam := by
        norm_num [MiddleFaceCell146.lamInterval, MiddleFaceCell146.brick,
          LeanSuffixReflective.QInterval.RealContains]
        at hlower46 hsplit47 ⊢
        exact ⟨hlower46, hsplit47⟩
      exact
        MiddleFaceCell146.candidate_not_isWeightedPerimeterMinimizer
          hcell candidate hcandidate
  · have hlower47 : (102471 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit47).le
    by_cases hsplit48 : lam ≤ (102481 / 100000 : ℝ)
      · have hcell : MiddleFaceCell147.lamInterval.RealContains lam := by
        norm_num [MiddleFaceCell147.lamInterval, MiddleFaceCell147.brick,
          LeanSuffixReflective.QInterval.RealContains]
        at hlower47 hsplit48 ⊢
        exact ⟨hlower47, hsplit48⟩
      exact
        MiddleFaceCell147.candidate_not_isWeightedPerimeterMinimizer
          hcell candidate hcandidate
    · have hlower48 : (102481 / 100000 : ℝ) ≤ lam :=
        (not_le.mp hsplit48).le
      by_cases hsplit49 : lam ≤ (102491 / 100000 : ℝ)
        · have hcell : MiddleFaceCell148.lamInterval.RealContains lam := by
          norm_num [MiddleFaceCell148.lamInterval, MiddleFaceCell148.brick,
            LeanSuffixReflective.QInterval.RealContains]
          at hlower48 hsplit49 ⊢
          exact ⟨hlower48, hsplit49⟩
        exact
          MiddleFaceCell148.candidate_not_isWeightedPerimeterMinimizer
            hcell candidate hcandidate
        · have hlower49 : (102491 / 100000 : ℝ) ≤ lam :=
            (not_le.mp hsplit49).le
          have hcell : MiddleFaceCell149.lamInterval.RealContains lam := by
            norm_num [MiddleFaceCell149.lamInterval, MiddleFaceCell149.brick,
              LeanSuffixReflective.QInterval.RealContains]
            at hlower49 hsplit50 ⊢
            exact ⟨hlower49, hsplit50⟩
          exact
            MiddleFaceCell149.candidate_not_isWeightedPerimeterMinimizer
              hcell candidate hcandidate

```

```

    MiddleFaceCell49.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower50 : (102501 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hspl50).le
  by_cases hspl53 : lam ≤ (102531 / 100000 : ℝ)
  · by_cases hspl51 : lam ≤ (102511 / 100000 : ℝ)
    · have hcell : MiddleFaceCell50.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell50.lamInterval, MiddleFaceCell50.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower50 hspl51 ⊢
      exact ⟨hlower50, hspl51⟩
    exact
      MiddleFaceCell50.candidate_not_isWeightedPerimeterMinimizer
        hcell candidate hcandidate
  · have hlower51 : (102511 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hspl51).le
  by_cases hspl52 : lam ≤ (102521 / 100000 : ℝ)
  · have hcell : MiddleFaceCell51.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell51.lamInterval, MiddleFaceCell51.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower51 hspl52 ⊢
    exact ⟨hlower51, hspl52⟩
  exact
    MiddleFaceCell51.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower52 : (102521 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hspl52).le
  have hcell : MiddleFaceCell52.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell52.lamInterval, MiddleFaceCell52.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower52 hspl53 ⊢
    exact ⟨hlower52, hspl53⟩
  exact
    MiddleFaceCell52.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower53 : (102531 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hspl53).le
  by_cases hspl54 : lam ≤ (102541 / 100000 : ℝ)
  · have hcell : MiddleFaceCell53.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell53.lamInterval, MiddleFaceCell53.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower53 hspl54 ⊢
    exact ⟨hlower53, hspl54⟩
  exact
    MiddleFaceCell53.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower54 : (102541 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hspl54).le
  by_cases hspl55 : lam ≤ (102551 / 100000 : ℝ)
  · have hcell : MiddleFaceCell54.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell54.lamInterval, MiddleFaceCell54.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower54 hspl55 ⊢
    exact ⟨hlower54, hspl55⟩
  exact
    MiddleFaceCell54.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower55 : (102551 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hspl55).le
  have hcell : MiddleFaceCell55.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell55.lamInterval, MiddleFaceCell55.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower55 hupper ⊢
    exact ⟨hlower55, hupper⟩
  exact
    MiddleFaceCell55.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
end MiddleFaceCells44To55

```

MiddleFaceCells56To67.candidate_not_isWeightedPerimeterMinimizer

Classification: supplement

Source: MiddleFaceCells56To67.lean:32-169

Declaration SHA-256: b26f20f880618fce0adba5633b5b3fe601d543d544685cd5c5e6e541816f591e

Mathematical statement. Candidate exclusion over middle-face cells fifty-six through sixty-seven.

Closed rational endpoints ensure that the batch introduces no seam gap.

```
theorem candidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ}
  (hlower : (102561 / 100000 : ℝ) ≤ lam)
  (hupper : lam ≤ (102681 / 100000 : ℝ))
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
by_cases hsplit62 : lam ≤ (102621 / 100000 : ℝ)
· by_cases hsplit59 : lam ≤ (102591 / 100000 : ℝ)
· by_cases hsplit57 : lam ≤ (102571 / 100000 : ℝ)
  · have hcell : MiddleFaceCell56.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell56.lamInterval, MiddleFaceCell56.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower hsplit57 ⊢
      exact ⟨hlower, hsplit57⟩
  exact
    MiddleFaceCell56.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower57 : (102571 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit57).le
  by_cases hsplit58 : lam ≤ (102581 / 100000 : ℝ)
  · have hcell : MiddleFaceCell57.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell57.lamInterval, MiddleFaceCell57.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower57 hsplit58 ⊢
      exact ⟨hlower57, hsplit58⟩
  exact
    MiddleFaceCell57.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower58 : (102581 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit58).le
  have hcell : MiddleFaceCell58.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell58.lamInterval, MiddleFaceCell58.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower58 hsplit59 ⊢
      exact ⟨hlower58, hsplit59⟩
  exact
    MiddleFaceCell58.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower59 : (102591 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit59).le
  by_cases hsplit60 : lam ≤ (102601 / 100000 : ℝ)
  · have hcell : MiddleFaceCell59.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell59.lamInterval, MiddleFaceCell59.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower59 hsplit60 ⊢
      exact ⟨hlower59, hsplit60⟩
  exact
    MiddleFaceCell59.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower60 : (102601 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit60).le
  by_cases hsplit61 : lam ≤ (102611 / 100000 : ℝ)
  · have hcell : MiddleFaceCell60.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell60.lamInterval, MiddleFaceCell60.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower60 hsplit61 ⊢
      exact ⟨hlower60, hsplit61⟩
  exact
    MiddleFaceCell60.candidate_not_isWeightedPerimeterMinimizer
```

```

      hcell candidate hcandidate
    · have hlower61 : (102611 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit61).le
      have hcell : MiddleFaceCell61.lamInterval.RealContains lam := by
        norm_num [MiddleFaceCell61.lamInterval, MiddleFaceCell61.brick,
          LeanSuffixReflective.QInterval.RealContains]
        at hlower61 hsplit62 ⊢
        exact ⟨hlower61, hsplit62⟩
      exact
        MiddleFaceCell61.candidate_not_isWeightedPerimeterMinimizer
          hcell candidate hcandidate
    · have hlower62 : (102621 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit62).le
      by_cases hsplit65 : lam ≤ (102651 / 100000 : ℝ)
    · by_cases hsplit63 : lam ≤ (102631 / 100000 : ℝ)
      · have hcell : MiddleFaceCell62.lamInterval.RealContains lam := by
        norm_num [MiddleFaceCell62.lamInterval, MiddleFaceCell62.brick,
          LeanSuffixReflective.QInterval.RealContains]
        at hlower62 hsplit63 ⊢
        exact ⟨hlower62, hsplit63⟩
      exact
        MiddleFaceCell62.candidate_not_isWeightedPerimeterMinimizer
          hcell candidate hcandidate
    · have hlower63 : (102631 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit63).le
      by_cases hsplit64 : lam ≤ (102641 / 100000 : ℝ)
      · have hcell : MiddleFaceCell63.lamInterval.RealContains lam := by
        norm_num [MiddleFaceCell63.lamInterval, MiddleFaceCell63.brick,
          LeanSuffixReflective.QInterval.RealContains]
        at hlower63 hsplit64 ⊢
        exact ⟨hlower63, hsplit64⟩
      exact
        MiddleFaceCell63.candidate_not_isWeightedPerimeterMinimizer
          hcell candidate hcandidate
    · have hlower64 : (102641 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit64).le
      have hcell : MiddleFaceCell64.lamInterval.RealContains lam := by
        norm_num [MiddleFaceCell64.lamInterval, MiddleFaceCell64.brick,
          LeanSuffixReflective.QInterval.RealContains]
        at hlower64 hsplit65 ⊢
        exact ⟨hlower64, hsplit65⟩
      exact
        MiddleFaceCell64.candidate_not_isWeightedPerimeterMinimizer
          hcell candidate hcandidate
    · have hlower65 : (102651 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit65).le
      by_cases hsplit66 : lam ≤ (102661 / 100000 : ℝ)
      · have hcell : MiddleFaceCell65.lamInterval.RealContains lam := by
        norm_num [MiddleFaceCell65.lamInterval, MiddleFaceCell65.brick,
          LeanSuffixReflective.QInterval.RealContains]
        at hlower65 hsplit66 ⊢
        exact ⟨hlower65, hsplit66⟩
      exact
        MiddleFaceCell65.candidate_not_isWeightedPerimeterMinimizer
          hcell candidate hcandidate
    · have hlower66 : (102661 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit66).le
      by_cases hsplit67 : lam ≤ (102671 / 100000 : ℝ)
      · have hcell : MiddleFaceCell66.lamInterval.RealContains lam := by
        norm_num [MiddleFaceCell66.lamInterval, MiddleFaceCell66.brick,
          LeanSuffixReflective.QInterval.RealContains]
        at hlower66 hsplit67 ⊢
        exact ⟨hlower66, hsplit67⟩
      exact
        MiddleFaceCell66.candidate_not_isWeightedPerimeterMinimizer
          hcell candidate hcandidate
    · have hlower67 : (102671 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit67).le
      have hcell : MiddleFaceCell67.lamInterval.RealContains lam := by
        norm_num [MiddleFaceCell67.lamInterval, MiddleFaceCell67.brick,
          LeanSuffixReflective.QInterval.RealContains]
        at hlower67 hupper ⊢

```

```

    exact ⟨hlower67, hupper⟩
  exact
    MiddleFaceCell67.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
end MiddleFaceCells56To67

```

MiddleFaceCells68To79.candidate_not_isWeightedPerimeterMinimizer

Classification: supplement

Source: MiddleFaceCells68To79.lean:32-169

Declaration SHA-256: 64c898dd26ac2e9b8857bb1839001b7ae6426e285d6390e32038f347657f972f

Mathematical statement. Candidate exclusion over middle-face cells sixty-eight through seventy-nine.

It combines the corresponding generated candidate-facing boundary results.

```

theorem candidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ}
  (hlower : (102681 / 100000 : ℝ) ≤ lam)
  (hupper : lam ≤ (102801 / 100000 : ℝ))
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
by_cases hsplit74 : lam ≤ (102741 / 100000 : ℝ)
· by_cases hsplit71 : lam ≤ (102711 / 100000 : ℝ)
· by_cases hsplit69 : lam ≤ (102691 / 100000 : ℝ)
  · have hcell : MiddleFaceCell68.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell68.lamInterval, MiddleFaceCell68.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower hsplit69 ⊢
      exact ⟨hlower, hsplit69⟩
  exact
    MiddleFaceCell68.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
· have hlower69 : (102691 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit69).le
by_cases hsplit70 : lam ≤ (102701 / 100000 : ℝ)
· have hcell : MiddleFaceCell69.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell69.lamInterval, MiddleFaceCell69.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower69 hsplit70 ⊢
  exact ⟨hlower69, hsplit70⟩
exact
  MiddleFaceCell69.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower70 : (102701 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit70).le
have hcell : MiddleFaceCell70.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell70.lamInterval, MiddleFaceCell70.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower70 hsplit71 ⊢
  exact ⟨hlower70, hsplit71⟩
exact
  MiddleFaceCell70.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower71 : (102711 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit71).le
by_cases hsplit72 : lam ≤ (102721 / 100000 : ℝ)
· have hcell : MiddleFaceCell71.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell71.lamInterval, MiddleFaceCell71.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower71 hsplit72 ⊢
  exact ⟨hlower71, hsplit72⟩
exact
  MiddleFaceCell71.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower72 : (102721 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit72).le

```

```

by_cases hsplit73 : lam ≤ (102731 / 100000 : ℝ)
· have hcell : MiddleFaceCell172.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell172.lamInterval, MiddleFaceCell172.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower72 hsplit73 ⊢
  exact ⟨hlower72, hsplit73⟩
exact
  MiddleFaceCell172.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower73 : (102731 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit73).le
  have hcell : MiddleFaceCell173.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell173.lamInterval, MiddleFaceCell173.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower73 hsplit74 ⊢
    exact ⟨hlower73, hsplit74⟩
  exact
    MiddleFaceCell173.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
· have hlower74 : (102741 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit74).le
by_cases hsplit77 : lam ≤ (102771 / 100000 : ℝ)
· by_cases hsplit75 : lam ≤ (102751 / 100000 : ℝ)
  · have hcell : MiddleFaceCell174.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell174.lamInterval, MiddleFaceCell174.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower74 hsplit75 ⊢
    exact ⟨hlower74, hsplit75⟩
  exact
    MiddleFaceCell174.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower75 : (102751 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit75).le
    by_cases hsplit76 : lam ≤ (102761 / 100000 : ℝ)
    · have hcell : MiddleFaceCell175.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell175.lamInterval, MiddleFaceCell175.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower75 hsplit76 ⊢
      exact ⟨hlower75, hsplit76⟩
    exact
      MiddleFaceCell175.candidate_not_isWeightedPerimeterMinimizer
        hcell candidate hcandidate
    · have hlower76 : (102761 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hsplit76).le
      have hcell : MiddleFaceCell176.lamInterval.RealContains lam := by
        norm_num [MiddleFaceCell176.lamInterval, MiddleFaceCell176.brick,
          LeanSuffixReflective.QInterval.RealContains]
        at hlower76 hsplit77 ⊢
        exact ⟨hlower76, hsplit77⟩
      exact
        MiddleFaceCell176.candidate_not_isWeightedPerimeterMinimizer
          hcell candidate hcandidate
  · have hlower77 : (102771 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit77).le
  by_cases hsplit78 : lam ≤ (102781 / 100000 : ℝ)
  · have hcell : MiddleFaceCell177.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell177.lamInterval, MiddleFaceCell177.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower77 hsplit78 ⊢
    exact ⟨hlower77, hsplit78⟩
  exact
    MiddleFaceCell177.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower78 : (102781 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit78).le
  by_cases hsplit79 : lam ≤ (102791 / 100000 : ℝ)
  · have hcell : MiddleFaceCell178.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell178.lamInterval, MiddleFaceCell178.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower78 hsplit79 ⊢
    exact ⟨hlower78, hsplit79⟩
  exact

```

```

      MiddleFaceCell178.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
    · have hlower79 : (102791 / 100000 : ℝ) ≤ lam :=
      (not_le.mp hspl179).le
    have hcell : MiddleFaceCell179.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell179.lamInterval, MiddleFaceCell179.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower79 hupper ⊢
      exact ⟨hlower79, hupper⟩
    exact
      MiddleFaceCell179.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate

end MiddleFaceCells68To79

```

MiddleFaceCells80To91.candidate_not_isWeightedPerimeterMinimizer

Classification: supplement

Source: MiddleFaceCells80To91.lean:32-169

Declaration SHA-256: 506c0e3dda4c848a77bd896e79f1ba0ef204016d55e8b8f1dccd1e1a06f429f4

Mathematical statement. Candidate exclusion over middle-face cells eighty through ninety-one.

A finite sequence of exact density comparisons selects an applicable cell.

```

theorem candidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ}
  (hlower : (102801 / 100000 : ℝ) ≤ lam)
  (hupper : lam ≤ (102921 / 100000 : ℝ))
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
by_cases hspl186 : lam ≤ (102861 / 100000 : ℝ)
· by_cases hspl183 : lam ≤ (102831 / 100000 : ℝ)
· by_cases hspl181 : lam ≤ (102811 / 100000 : ℝ)
  · have hcell : MiddleFaceCell180.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell180.lamInterval, MiddleFaceCell180.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower hspl181 ⊢
      exact ⟨hlower, hspl181⟩
    exact
      MiddleFaceCell180.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower81 : (102811 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hspl181).le
  by_cases hspl182 : lam ≤ (102821 / 100000 : ℝ)
  · have hcell : MiddleFaceCell181.lamInterval.RealContains lam := by
      norm_num [MiddleFaceCell181.lamInterval, MiddleFaceCell181.brick,
        LeanSuffixReflective.QInterval.RealContains]
      at hlower81 hspl182 ⊢
      exact ⟨hlower81, hspl182⟩
    exact
      MiddleFaceCell181.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower82 : (102821 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hspl182).le
  have hcell : MiddleFaceCell182.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell182.lamInterval, MiddleFaceCell182.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower82 hspl183 ⊢
    exact ⟨hlower82, hspl183⟩
  exact
    MiddleFaceCell182.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower83 : (102831 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hspl183).le
by_cases hspl184 : lam ≤ (102841 / 100000 : ℝ)
· have hcell : MiddleFaceCell183.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell183.lamInterval, MiddleFaceCell183.brick,

```



```

LeanSuffixReflective.QInterval.RealContains]
  at hlower83 hsplit84 ⊢
exact ⟨hlower83, hsplit84⟩
exact
  MiddleFaceCell83.candidate_not_isWeightedPerimeterMinimizer
  hcell candidate hcandidate
· have hlower84 : (102841 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit84).le
by_cases hsplit85 : lam ≤ (102851 / 100000 : ℝ)
· have hcell : MiddleFaceCell84.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell84.lamInterval, MiddleFaceCell84.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower84 hsplit85 ⊢
  exact ⟨hlower84, hsplit85⟩
exact
  MiddleFaceCell84.candidate_not_isWeightedPerimeterMinimizer
  hcell candidate hcandidate
· have hlower85 : (102851 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit85).le
have hcell : MiddleFaceCell85.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell85.lamInterval, MiddleFaceCell85.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower85 hsplit86 ⊢
  exact ⟨hlower85, hsplit86⟩
exact
  MiddleFaceCell85.candidate_not_isWeightedPerimeterMinimizer
  hcell candidate hcandidate
· have hlower86 : (102861 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit86).le
by_cases hsplit89 : lam ≤ (102891 / 100000 : ℝ)
· by_cases hsplit87 : lam ≤ (102871 / 100000 : ℝ)
  · have hcell : MiddleFaceCell86.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell86.lamInterval, MiddleFaceCell86.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower86 hsplit87 ⊢
    exact ⟨hlower86, hsplit87⟩
  exact
    MiddleFaceCell86.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
  · have hlower87 : (102871 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit87).le
  by_cases hsplit88 : lam ≤ (102881 / 100000 : ℝ)
  · have hcell : MiddleFaceCell87.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell87.lamInterval, MiddleFaceCell87.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower87 hsplit88 ⊢
    exact ⟨hlower87, hsplit88⟩
  exact
    MiddleFaceCell87.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
  · have hlower88 : (102881 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit88).le
  have hcell : MiddleFaceCell88.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell88.lamInterval, MiddleFaceCell88.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower88 hsplit89 ⊢
    exact ⟨hlower88, hsplit89⟩
  exact
    MiddleFaceCell88.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower89 : (102891 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit89).le
by_cases hsplit90 : lam ≤ (102901 / 100000 : ℝ)
· have hcell : MiddleFaceCell89.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell89.lamInterval, MiddleFaceCell89.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower89 hsplit90 ⊢
  exact ⟨hlower89, hsplit90⟩
exact
  MiddleFaceCell89.candidate_not_isWeightedPerimeterMinimizer
  hcell candidate hcandidate
· have hlower90 : (102901 / 100000 : ℝ) ≤ lam :=

```

```

(not_le.mp hsplit90).le
by_cases hsplit91 : lam ≤ (102911 / 100000 : ℝ)
· have hcell : MiddleFaceCell190.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell190.lamInterval, MiddleFaceCell190.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower90 hsplit91 ⊢
  exact ⟨hlower90, hsplit91⟩
exact
  MiddleFaceCell190.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower91 : (102911 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit91).le
have hcell : MiddleFaceCell191.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell191.lamInterval, MiddleFaceCell191.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower91 hupper ⊢
  exact ⟨hlower91, hupper⟩
exact
  MiddleFaceCell191.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate

end MiddleFaceCells80To91

```

MiddleFaceCells92To103.candidate_not_isWeightedPerimeterMinimizer

Classification: supplement

Source: MiddleFaceCells92To103.lean:32-169

Declaration SHA-256: eb8d5eea5007ec9d93249eb58e5ea168ebe71c7c90ef757bae9ec70ea6beb8ed

Mathematical statement. Candidate exclusion over middle-face cells ninety-two through 103.

This is the penultimate batch in the middle-family coverage.

```

theorem candidate_not_isWeightedPerimeterMinimizer
{lam : ℝ}
(hlower : (102921 / 100000 : ℝ) ≤ lam)
(hupper : lam ≤ (103041 / 100000 : ℝ))
(candidate : _root_.FourArcCandidate lam)
(hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
¬ candidate.IsWeightedPerimeterMinimizer := by
by_cases hsplit98 : lam ≤ (102981 / 100000 : ℝ)
· by_cases hsplit95 : lam ≤ (102951 / 100000 : ℝ)
· by_cases hsplit93 : lam ≤ (102931 / 100000 : ℝ)
  · have hcell : MiddleFaceCell192.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell192.lamInterval, MiddleFaceCell192.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower hsplit93 ⊢
    exact ⟨hlower, hsplit93⟩
  exact
    MiddleFaceCell192.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
· have hlower93 : (102931 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit93).le
by_cases hsplit94 : lam ≤ (102941 / 100000 : ℝ)
· have hcell : MiddleFaceCell193.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell193.lamInterval, MiddleFaceCell193.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower93 hsplit94 ⊢
  exact ⟨hlower93, hsplit94⟩
exact
  MiddleFaceCell193.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower94 : (102941 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit94).le
have hcell : MiddleFaceCell194.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell194.lamInterval, MiddleFaceCell194.brick,
    LeanSuffixReflective.QInterval.RealContains]
  at hlower94 hsplit95 ⊢
  exact ⟨hlower94, hsplit95⟩

```

```

    exact
      MiddleFaceCell194.candidate_not_isWeightedPerimeterMinimizer
        hcell candidate hcandidate
  · have hlower95 : (102951 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit95).le
  by_cases hsplit96 : lam ≤ (102961 / 100000 : ℝ)
  · have hcell : MiddleFaceCell195.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell195.lamInterval, MiddleFaceCell195.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower95 hsplit96 ⊢
    exact ⟨hlower95, hsplit96⟩
  exact
    MiddleFaceCell195.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower96 : (102961 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit96).le
  by_cases hsplit97 : lam ≤ (102971 / 100000 : ℝ)
  · have hcell : MiddleFaceCell196.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell196.lamInterval, MiddleFaceCell196.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower96 hsplit97 ⊢
    exact ⟨hlower96, hsplit97⟩
  exact
    MiddleFaceCell196.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower97 : (102971 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit97).le
  have hcell : MiddleFaceCell197.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell197.lamInterval, MiddleFaceCell197.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower97 hsplit98 ⊢
    exact ⟨hlower97, hsplit98⟩
  exact
    MiddleFaceCell197.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower98 : (102981 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit98).le
  by_cases hsplit101 : lam ≤ (103011 / 100000 : ℝ)
  · by_cases hsplit99 : lam ≤ (102991 / 100000 : ℝ)
  · have hcell : MiddleFaceCell198.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell198.lamInterval, MiddleFaceCell198.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower98 hsplit99 ⊢
    exact ⟨hlower98, hsplit99⟩
  exact
    MiddleFaceCell198.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower99 : (102991 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit99).le
  by_cases hsplit100 : lam ≤ (103001 / 100000 : ℝ)
  · have hcell : MiddleFaceCell199.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell199.lamInterval, MiddleFaceCell199.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower99 hsplit100 ⊢
    exact ⟨hlower99, hsplit100⟩
  exact
    MiddleFaceCell199.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower100 : (103001 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit100).le
  have hcell : MiddleFaceCell1100.lamInterval.RealContains lam := by
    norm_num [MiddleFaceCell1100.lamInterval, MiddleFaceCell1100.brick,
      LeanSuffixReflective.QInterval.RealContains]
    at hlower100 hsplit101 ⊢
    exact ⟨hlower100, hsplit101⟩
  exact
    MiddleFaceCell1100.candidate_not_isWeightedPerimeterMinimizer
      hcell candidate hcandidate
  · have hlower101 : (103011 / 100000 : ℝ) ≤ lam :=
    (not_le.mp hsplit101).le
  by_cases hsplit102 : lam ≤ (103021 / 100000 : ℝ)
  · have hcell : MiddleFaceCell1101.lamInterval.RealContains lam := by

```

```

norm_num [MiddleFaceCell101.lamInterval, MiddleFaceCell101.brick,
  LeanSuffixReflective.QInterval.RealContains]
  at hlower101 hsplit102 ⊢
exact ⟨hlower101, hsplit102⟩
exact
  MiddleFaceCell101.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower102 : (103021 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit102).le
by_cases hsplit103 : lam ≤ (103031 / 100000 : ℝ)
· have hcell : MiddleFaceCell102.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell102.lamInterval, MiddleFaceCell102.brick,
    LeanSuffixReflective.QInterval.RealContains]
    at hlower102 hsplit103 ⊢
  exact ⟨hlower102, hsplit103⟩
exact
  MiddleFaceCell102.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate
· have hlower103 : (103031 / 100000 : ℝ) ≤ lam :=
  (not_le.mp hsplit103).le
have hcell : MiddleFaceCell103.lamInterval.RealContains lam := by
  norm_num [MiddleFaceCell103.lamInterval, MiddleFaceCell103.brick,
    LeanSuffixReflective.QInterval.RealContains]
    at hlower103 hupper ⊢
  exact ⟨hlower103, hupper⟩
exact
  MiddleFaceCell103.candidate_not_isWeightedPerimeterMinimizer
    hcell candidate hcandidate

end MiddleFaceCells92To103

```

OneSidedCircularCap.arcLength_pos

Classification: supplement

Source: CMVGeometry.lean:220-222

Declaration SHA-256: 615464c0ac73967d872be72b9e5d931fb0644d2db70b6e9168ad1dd7c81aa184

Mathematical statement. The cap arc has positive length.

Its length is the product of positive chord and normalized arc length.

```

theorem arcLength_pos (c : OneSidedCircularCap) : 0 < c.arcLength :=
  mul_pos c.chord_pos c.ell_theta_pos

```

OneSidedCircularCap.arcPoint

Classification: supplement

Source: CMVGeometry.lean:124-133

Declaration SHA-256: 2f12b44987266410d1b21333c27225fca467817287b4028245b465856f5540c3

Mathematical statement. Angular parametrization of the cap arc.

The circle of radius R is placed and reflected according to the cap's side.

```

def arcPoint (c : OneSidedCircularCap) (t : ℝ) : PlanePoint :=
  match c.side with
  | .upper =>
    (c.midpointX + c.radius * sin t,
     c.baseY + c.radius * (cos t - cos c.theta))
  | .lower =>
    (c.midpointX + c.radius * sin t,
     c.baseY - c.radius * (cos t - cos c.theta))

/-- Left endpoint of the oriented chord. -/

```

OneSidedCircularCap.arcPoint_in_closed_side

Classification: supplement

Source: CMVGeometry.lean:240-252

Declaration SHA-256: 4692c99be4beede3a58f1e164acd1cfe21db7a9160da63340de9490acd46c184

Mathematical statement. The parametrized arc stays in the chosen closed half-plane.

The assertion includes the two base-height endpoints.

```
theorem arcPoint_in_closed_side (c : OneSidedCircularCap) {t : ℝ}
  (ht : |t| ≤ c.theta) :
  match c.side with
  | .upper => c.baseY ≤ (c.arcPoint t).2
  | .lower => (c.arcPoint t).2 ≤ c.baseY := by
  have hc : cos c.theta ≤ cos t := by
    rw [- Real.cos_abs t]
    exact Real.cos_le_cos_of_nonneg_of_le_pi (abs_nonneg t)
      (le_of_lt c.theta_lt_pi) ht
  have hr := c.radius_pos
  cases h : c.side <|> simp [arcPoint, h] <|> nlinarith

/-- A non-endpoint arc point is strictly on the cap side of the chord. -/
```

OneSidedCircularCap.arcPoint_in_open_side

Classification: supplement

Source: CMVGeometry.lean:253-266

Declaration SHA-256: 3dbf6a865d1d6331f9c3406c05af249acf00d7c487349d982b588394a814d697

Mathematical statement. Interior arc parameters lie strictly on the cap side of the chord.

Only the endpoint parameters can meet the base line.

```
theorem arcPoint_in_open_side (c : OneSidedCircularCap) {t : ℝ}
  (ht : |t| < c.theta) :
  match c.side with
  | .upper => c.baseY < (c.arcPoint t).2
  | .lower => (c.arcPoint t).2 < c.baseY := by
  have hc : cos c.theta < cos t := by
    rw [- Real.cos_abs t]
    exact Real.cos_lt_cos_of_nonneg_of_le_pi (abs_nonneg t)
      (le_of_lt c.theta_lt_pi) ht
  have hr := c.radius_pos
  cases h : c.side <|> simp [arcPoint, h] <|> nlinarith

/-- The selected circular arc has no self-intersections, including on the
major branch. -/
```

OneSidedCircularCap.arcPoint_left

Classification: supplement

Source: CMVGeometry.lean:228-232

Declaration SHA-256: 4e22ae8901c6f895322d9f1591e370b613671c823882adf707a5fac1ae750287

Mathematical statement. The initial angular endpoint equals the left chord endpoint.

Substitution into the angular parametrization gives $(m - c/2, b)$.

```
theorem arcPoint_left (c : OneSidedCircularCap) :
  c.arcPoint (-c.theta) = c.leftEndpoint := by
```

```
cases h : c.side <=>
  simp [arcPoint, leftEndpoint, h, sin_neg, c.radius_mul_sin] <=> ring
```

OneSidedCircularCap.arcPoint_right

Classification: supplement

Source: CMVGeometry.lean:233-239

Declaration SHA-256: ba639553cc75debbcac47cb7f164b0cdee0e14e9586cdc6d1cf857f7fa27f21b

Mathematical statement. The final angular endpoint equals the right chord endpoint.

Substitution gives $(m + c/2, b)$.

```
theorem arcPoint_right (c : OneSidedCircularCap) :
  c.arcPoint c.theta = c.rightEndpoint := by
  cases h : c.side <=>
  simp [arcPoint, rightEndpoint, h, c.radius_mul_sin]

/-- Every point of the selected arc lies in the declared closed half-plane,
including on the major branch. -/
```

OneSidedCircularCap.arcTrace

Classification: supplement

Source: CMVGeometry.lean:656-661

Declaration SHA-256: 5e3edb81f4f16ef8dbff7c15d5756a176b1f99c3bd5fb86ce703134a0a727a89

Mathematical statement. The circular part of the cap boundary.

It is the circle restricted to the selected closed side of the chord.

```
def arcTrace (c : OneSidedCircularCap) : Set PlanePoint :=
  {p | c.radiusSquaredAt p = c.radius ^ 2 ^
    match c.side with
    | .upper => c.baseY ≤ p.2
    | .lower => p.2 ≤ c.baseY}
```

OneSidedCircularCap.center

Classification: supplement

Source: CMVGeometry.lean:117-123

Declaration SHA-256: 58a018a3af71f1eb6c5aebb7ee395b60aab79f7b55a9d384d43694baace0d346

Mathematical statement. The circle center for the chosen cap placement.

Its horizontal coordinate is the chord midpoint; its signed vertical offset is $R\cos\theta$.

```
def center (c : OneSidedCircularCap) : PlanePoint :=
  match c.side with
  | .upper => (c.midpointX, c.baseY - c.radius * cos c.theta)
  | .lower => (c.midpointX, c.baseY + c.radius * cos c.theta)

/-- Coordinate realization of the selected arc, with parameter
`-theta ≤ t ≤ theta`. -/
```

OneSidedCircularCap.chordCarrier

Classification: supplement

Source: CMVGeometry.lean:160-163

Declaration SHA-256: 0705736836c9203b120221cb0770c365bada15ab749e72d0791d55f58eba4c1e

Mathematical statement. The closed horizontal chord segment.

It consists of base-height points with horizontal distance at most $c/2$ from the midpoint.

```
def chordCarrier (c : OneSidedCircularCap) : Set PlanePoint :=
  {p | p.2 = c.baseY ^ |p.1 - c.midpointX| ≤ c.chord / 2}

/-- Euclidean area computed by the circular-segment constructor. -/
```

OneSidedCircularCap.chordCarrier_subset_carrier

Classification: supplement

Source: CMVGeometry.lean:311-334

Declaration SHA-256: a35dcecd8fd218da748131deaec9b02f5328c1034952ad269b6ba1f5b686d947

Mathematical statement. The entire chord lies in the cap carrier.

Chord points satisfy both the disk and closed half-plane conditions.

```
theorem chordCarrier_subset_carrier (c : OneSidedCircularCap) :
  c.chordCarrier ⊆ c.carrier := by
  intro p hp
  have hxabs : |p.1 - c.midpointX| ≤ c.chord / 2 := hp.2
  have hhalf : 0 < c.chord / 2 := div_pos c.chord_pos (by norm_num)
  have hxsq : (p.1 - c.midpointX) ^ 2 ≤ (c.chord / 2) ^ 2 :=
    sq_le_sq.mpr (by simpa [abs_of_pos hhalf] using hxabs)
  rw [← c.radius_mul_sin] at hxsq
  have htrig := Real.sin_sq_add_cos_sq c.theta
  have htrigScaled := congrArg
    (fun z : ℝ => c.radius ^ 2 * z) htrig
  cases hside : c.side with
  | upper =>
    constructor
    · simp only [radiusSquaredAt, center, hside, hp.1]
      nlinarith
    · simp [hside, hp.1]
  | lower =>
    constructor
    · simp only [radiusSquaredAt, center, hside, hp.1]
      nlinarith
    · simp [hside, hp.1]

/-- The coordinate cap is measurable. -/
```

OneSidedCircularCap.ell_theta_pos

Classification: supplement

Source: CMVGeometry.lean:216-219

Declaration SHA-256: 88c4230a8ce8dbe3ed5e64385c18de0588039e6c242ead1e2df6299d087f0ad8

Mathematical statement. Normalized arc length is positive at the cap angle.

$\ell(\theta) = \theta/\sin\theta > 0$.

```
theorem ell_theta_pos (c : OneSidedCircularCap) : 0 < ell c.theta := by
  rw [ell]
```

```
exact div_pos c.theta_pos c.sin_theta_pos
```

OneSidedCircularCap.euclideanArea_pos

Classification: supplement

Source: CMVGeometry.lean:213-215

Declaration SHA-256: 3902ccdc9eb3694014c5a47aad3179ebe09a3fd3cddb066b9c4c1ca933463218

Mathematical statement. The cap area expression is positive.

$c^2 > 0$ and $a(\theta) > 0$ on the recorded angle domain.

```
theorem euclideanArea_pos (c : OneSidedCircularCap) : 0 < c.euclideanArea := by
  exact mul_pos (sq_pos_of_pos c.chord_pos) (area_pos c.theta_pos c.theta_lt_pi)
```

OneSidedCircularCap.interiorCarrier

Classification: supplement

Source: CMVGeometry.lean:153-159

Declaration SHA-256: d9986ab2dd520dfd41d6ccb99ca9878df085185d6c10c136685b43549b4c4dc1

Mathematical statement. The corresponding strict disk and strict half-plane intersection.

Strict inequalities give an explicit open subset of the carrier's interior.

```
def interiorCarrier (c : OneSidedCircularCap) : Set PlanePoint :=
  {p | c.radiusSquaredAt p < c.radius ^ 2 ^
    match c.side with
    | .upper => c.baseY < p.2
    | .lower => p.2 < c.baseY}

/-- The horizontal chord, including both endpoints. -/
```

OneSidedCircularCap.interiorCarrier_subset_interior

Classification: supplement

Source: CMVGeometry.lean:687-696

Declaration SHA-256: e055c7441e3e2230b6bd3c3989fc3043eb631862e9cdc4e7fe3a7963b0242ddc

Mathematical statement. The strict cap carrier lies in the topological interior.

It is an open subset of the closed carrier.

```
theorem interiorCarrier_subset_interior (c : OneSidedCircularCap) :
  c.interiorCarrier ⊆ interior c.carrier := by
  apply interior_maximal
  intro p hp
  exact ⟨hp.1.le, by
    cases hside : c.side <|>
    simp only [OneSidedCircularCap.interiorCarrier, hside] at hp <|>
    simp only [OneSidedCircularCap.carrier, hside] <|> linarith [hp.2]⟩
  exact isOpen_interiorCarrier c
```


OneSidedCircularCap.isClosed_carrier

Classification: supplement

Source: CMVGeometry.lean:364-391

Declaration SHA-256: d757ff8cbf988a97323a1f1c27dc667eab9b6475f2eb7d1c16beffa82e3009a4

Mathematical statement. The cap carrier is closed.

It is an intersection of a closed disk and a closed half-plane.

```
theorem isClosed_carrier (c : OneSidedCircularCap) :
  IsClosed c.carrier := by
  have hdist : Continuous (fun p : PlanePoint => c.radiusSquaredAt p) := by
    simp only [radiusSquaredAt]
    fun_prop
  have hdisk : IsClosed {p : PlanePoint |
    c.radiusSquaredAt p ≤ c.radius ^ 2} :=
    isClosed_le hdist continuous_const
  cases hside : c.side with
  | upper =>
    rw [carrier]
    simp only [hside]
    change IsClosed
      ({p : PlanePoint | c.radiusSquaredAt p ≤ c.radius ^ 2} ∩
       {p : PlanePoint | c.baseY ≤ p.2})
    exact hdisk.inter (isClosed_le continuous_const continuous_snd)
  | lower =>
    rw [carrier]
    simp only [hside]
    change IsClosed
      ({p : PlanePoint | c.radiusSquaredAt p ≤ c.radius ^ 2} ∩
       {p : PlanePoint | p.2 ≤ c.baseY})
    exact hdisk.inter (isClosed_le continuous_snd continuous_const)

end OneSidedCircularCap

/-- A horizontal segment. Its metric length is its positive chord parameter,
not an independently asserted scalar. -/
```

OneSidedCircularCap.isOpen_interiorCarrier

Classification: supplement

Source: CMVGeometry.lean:662-686

Declaration SHA-256: ec8c11c26c05148d770698c167cdec56869de41dc90864a365cf2514de87bc1

Mathematical statement. The strict cap carrier is open.

Strict disk and half-plane inequalities define open sets.

```
theorem isOpen_interiorCarrier (c : OneSidedCircularCap) :
  IsOpen c.interiorCarrier := by
  have hdisk : IsOpen {p : PlanePoint |
    c.radiusSquaredAt p < c.radius ^ 2} := by
    apply isOpen_lt
    · change Continuous (fun p : PlanePoint =>
      (p.1 - c.center.1) ^ 2 + (p.2 - c.center.2) ^ 2)
      fun_prop
    · fun_prop
  cases hside : c.side with
  | upper =>
    rw [OneSidedCircularCap.interiorCarrier]
    simp only [hside]
    change IsOpen
      ({p : PlanePoint | c.radiusSquaredAt p < c.radius ^ 2} ∩
       {p : PlanePoint | c.baseY < p.2})
    exact hdisk.inter (isOpen_lt continuous_const continuous_snd)
```

```

| lower =>
  rw [OneSidedCircularCap.interiorCarrier]
  simp only [hside]
  change IsOpen
    ({p : PlanePoint | c.radiusSquaredAt p < c.radius ^ 2} ∩
     {p : PlanePoint | p.2 < c.baseY})
  exact hdisk.inter (isOpen_lt continuous_snd continuous_const)

```

OneSidedCircularCap.leftEndpoint

Classification: supplement

Source: CMVGeometry.lean:134-137

Declaration SHA-256: a193f337e9021d7e081f4666c9f784092558b2268ec0e9060f11b3d18dfc3c8d

Mathematical statement. Left endpoint of the horizontal chord.

For midpoint m and base b , the point is $(m - c/2, b)$.

```

def leftEndpoint (c : OneSidedCircularCap) : PlanePoint :=
  (c.midpointX - c.chord / 2, c.baseY)

/-- Right endpoint of the oriented chord. -/

```

OneSidedCircularCap.measurableSet_carrier

Classification: supplement

Source: CMVGeometry.lean:335-363

Declaration SHA-256: e0f551f525fbf0409babb736205ec484e0a0c2ff7039ce55de1e7a7fc5595a27

Mathematical statement. The cap carrier is measurable.

Its defining inequalities involve continuous coordinate polynomials.

```

theorem measurableSet_carrier (c : OneSidedCircularCap) :
  MeasurableSet c.carrier := by
  have hdist : Continuous (fun p : PlanePoint => c.radiusSquaredAt p) := by
    simp only [radiusSquaredAt]
  fun_prop
  have hdisk : MeasurableSet {p : PlanePoint |
    c.radiusSquaredAt p ≤ c.radius ^ 2} :=
    (isClosed_le hdist continuous_const).measurableSet
  cases h : c.side with
  | upper =>
    have hhalf : MeasurableSet {p : PlanePoint | c.baseY ≤ p.2} :=
      (isClosed_le continuous_const continuous_snd).measurableSet
    rw [carrier]
    simp only [h]
    change MeasurableSet
      ({p : PlanePoint | c.radiusSquaredAt p ≤ c.radius ^ 2} ∩
       {p : PlanePoint | c.baseY ≤ p.2})
    exact hdisk.inter hhalf
  | lower =>
    have hhalf : MeasurableSet {p : PlanePoint | p.2 ≤ c.baseY} :=
      (isClosed_le continuous_snd continuous_const).measurableSet
    rw [carrier]
    simp only [h]
    change MeasurableSet
      ({p : PlanePoint | c.radiusSquaredAt p ≤ c.radius ^ 2} ∩
       {p : PlanePoint | p.2 ≤ c.baseY})
    exact hdisk.inter hhalf

/-- The disk/closed-half-plane cap carrier is closed. -/

```

OneSidedCircularCap.mem_chordCarrier_of_mem_carrier_of_eq_base

Classification: supplement

Source: CMVGeometry.lean:291-310

Declaration SHA-256: 424261710f758c71ebe7e57dd13ebd11ffc59cb51b840ff8fc86319890ddae19

Mathematical statement. A cap point at base height belongs to its chord.

The disk inequality at the base reduces to $|x - m| \leq c/2$.

```
theorem mem_chordCarrier_of_mem_carrier_of_eq_base
  (c : OneSidedCircularCap) {p : PlanePoint}
  (hp : p ∈ c.carrier) (hy : p.2 = c.baseY) :
  p ∈ c.chordCarrier := by
  refine ⟨hy, ?_⟩
  have hdisk := hp.1
  cases hside : c.side <|>
  simp only [radiusSquaredAt, center, hside, hy] at hdisk
  all_goals
  have htrig := Real.sin_sq_add_cos_sq c.theta
  have htrigScaled := congrArg
    (fun z : ℝ => c.radius ^ 2 * z) htrig
  have hsq : (p.1 - c.midpointX) ^ 2 ≤
    (c.radius * sin c.theta) ^ 2 := by
    nlinarith
  rw [c.radius_mul_sin] at hsq
  exact abs_le_of_sq_le_sq hsq
    (div_nonneg c.chord_pos.le (by norm_num))

/-- Every point of the finite chord belongs to the closed cap. -/
```

OneSidedCircularCap.radiusSquaredAt

Classification: supplement

Source: CMVGeometry.lean:142-145

Declaration SHA-256: a1c0f970b426c7f0e6a013d8d8bbb613fbba3b5c253445dacecdb9b47cb6785a

Mathematical statement. Squared distance from a point to the cap circle center.

This polynomial expresses disk membership without square-root inequalities.

```
def radiusSquaredAt (c : OneSidedCircularCap) (p : PlanePoint) : ℝ :=
  (p.1 - c.center.1) ^ 2 + (p.2 - c.center.2) ^ 2

/-- The genuine closed coordinate carrier of a one-sided cap. -/
```

OneSidedCircularCap.radius_mul_sin

Classification: supplement

Source: CMVGeometry.lean:223-227

Declaration SHA-256: 4b800b6287f8bc78c34b40f39fff02ab7d25631ad2fd22f8819095ceda60143e

Mathematical statement. Radius times sine of half-angle is half the chord.

$R \sin \theta = c/2$.

```
theorem radius_mul_sin (c : OneSidedCircularCap) :
  c.radius * sin c.theta = c.chord / 2 := by
  rw [radius]
  field_simp [ne_of_gt c.sin_theta_pos]
```

OneSidedCircularCap.radius_pos

Classification: supplement

Source: CMVGeometry.lean:210-212

Declaration SHA-256: 7bcdcf3411946f45cb3c854e76728bf4e1477fc16c0bd8673b5d0f6f904d6adf

Mathematical statement. The cap radius is positive.

Both c and $2\sin\theta$ are positive.

```
theorem radius_pos (c : OneSidedCircularCap) : 0 < c.radius := by
  exact div_pos c.chord_pos (mul_pos (by norm_num) c.sin_theta_pos)
```

OneSidedCircularCap.rightEndpoint

Classification: supplement

Source: CMVGeometry.lean:138-141

Declaration SHA-256: e83d4942af5a95dd72bd914bc15f416e046e239fc23caff7a851807f832f26a3

Mathematical statement. Right endpoint of the horizontal chord.

For midpoint m and base b , the point is $(m + c/2, b)$.

```
def rightEndpoint (c : OneSidedCircularCap) : PlanePoint :=
  (c.midpointX + c.chord / 2, c.baseY)

/-- Squared distance from the cap's circle center. -/
```

OneSidedCircularCap.sin_theta_pos

Classification: supplement

Source: CMVGeometry.lean:207-209

Declaration SHA-256: 8c22795350624813eb7b9aec2fdcd7bfc438d144ebb38b607db3cac77688583

Mathematical statement. The cap angle has positive sine.

$0 < \theta < \pi$ implies $\sin\theta > 0$.

```
theorem sin_theta_pos (c : OneSidedCircularCap) : 0 < sin c.theta :=
  sin_pos_of_pos_of_lt_pi c.theta_pos c.theta_lt_pi
```

ReplacementAssembly.below_exposedLowerChord_not_mem

Classification: supplement

Source: CMVGeometry.lean:3442-3459

Declaration SHA-256: 3316f6365842786c2f5614825bd4efdba288774b79cdd05c569f08a4717428cd

Mathematical statement. Points immediately below the lower chord are outside the replacement.

Neither the retained core nor the upper cap extends below $y = -1$.

```
theorem below_exposedLowerChord_not_mem (a : ReplacementAssembly)
  {p : PlanePoint} (hp : p ∈ a.exposedLowerChord.carrier)
  {epsilon : ℝ} (hepsilon : 0 < epsilon) :
  (p.1, p.2 - epsilon) ∉ a.carrier := by
  intro hbelow
  rcases hbelow with hcore | hcap
  · have hy := a.core.carrier_y_bounds hcore
    have hpY : p.2 = -1 := hp.1
```

```

have hneg : p.2 - epsilon < 0 := by linarith
rw [abs_of_neg hneg, hpY] at hy
linarith
· have hbase : (1 : ℝ) ≤ p.2 - epsilon := by
  simp [upperCap] using hcap.2
have hpY : p.2 = -1 := hp.1
linarith

/-- The exposed lower chord is an actual boundary subset of the coordinate
replacement carrier. -/

```

ReplacementAssembly.boundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:3310-3314

Declaration SHA-256: 75dfabe9f5e9f8aa75724a2e5f410dd1e358abaf0454833ef91226757c6630cb

Mathematical statement. Sum of side-arc, upper-cap, and lower-segment parameter measures.

The lower chord is explicitly included in the component perimeter.

```

def boundaryMeasure (a : ReplacementAssembly) : Measure PlanePoint :=
  leftCoreBoundaryMeasure a.core + rightCoreBoundaryMeasure a.core +
  capBoundaryMeasure a.upperCap + segmentBoundaryMeasure a.exposedLowerChord

/-- The explicit replacement boundary measure gives every endpoint zero mass. -/

```

ReplacementAssembly.boundaryTrace_subset_frontier_carrier

Classification: supplement

Source: CMVGeometry.lean:3933-3940

Declaration SHA-256: 6c897595e7667c418f2a95e4221cabe00a508a6ab4dac16eff0c51ccb2b4d8a9

Mathematical statement. Every listed replacement trace is exposed boundary.

The assertion includes the bottom chord and arc junctions.

```

theorem boundaryTrace_subset_frontier_carrier (a : ReplacementAssembly) :
  a.boundaryTrace ⊆ frontier a.carrier := by
  rintro p (hleft | hright | hupper | hlower)
  · exact leftArc_mem_frontier a hleft
  · exact rightArc_mem_frontier a hright
  · exact upperArc_mem_frontier a hupper
  · exact a.exposedLowerChord_subset_frontier hlower

```

ReplacementAssembly.exposedLowerChord_subset_carrier

Classification: supplement

Source: CMVGeometry.lean:3429-3441

Declaration SHA-256: 4d73b08bb97103964040bbdf24b1dbc10e728893bf174f22b32f933c62dcef7f

Mathematical statement. The exposed lower chord belongs to the replacement carrier.

It is contained in the retained core rectangle.

```

theorem exposedLowerChord_subset_carrier (a : ReplacementAssembly) :
  a.exposedLowerChord.carrier ⊆ a.carrier := by
  intro p hp
  have hp' :
    p ∈ {q : PlanePoint | q.2 = (-1 : ℝ) ∧
      |q.1| ≤ a.core.chord / 2} := by
    simp [exposedLowerChord, HorizontalSegment.carrier] using hp

```

```

exact Or.inl
  (a.core.horizontalChord_subset_carrier (by norm_num) hp')

/-- Every positive downward displacement from the lower chord leaves the
replacement. This is the local exposure fact used to identify that segment as
boundary rather than disconnected scalar data. -/

```

ReplacementAssembly.frontier_carrier_subset_boundaryTrace

Classification: supplement

Source: CMVGeometry.lean:3595-3613

Declaration SHA-256: 690416c38a1e91ad4ec069f59dc7a6046805e39bbae1ba5d6d8c4028b6ea7f3e

Mathematical statement. No replacement frontier point lies outside the listed traces.

The upper glued chord interior is eliminated from the frontier.

```

theorem frontier_carrier_subset_boundaryTrace (a : ReplacementAssembly) :
  frontier a.carrier ⊆ a.boundaryTrace := by
  intro p hp
  have htop := frontier_union_subset a.core.carrier a.upperCap.carrier hp
  rcases htop with hcore | hupper
  · rw [a.core.frontier_carrier] at hcore
    rcases hcore.1 with hleft | hright | hupperChord | hlowerChord
    · exact Or.inl hleft
    · exact Or.inr (Or.inl hright)
    · exact chord_to_boundaryTrace a hupperChord.1 hupperChord.2 hp
    · exact Or.inr (Or.inr (Or.inr (by
      simpa [StripCore.lowerChordTrace, exposedLowerChord,
        HorizontalSegment.carrier] using hlowerChord)))
  · rw [a.upperCap.frontier_carrier] at hupper
    rcases hupper.2 with harc | hchord
    · exact Or.inr (Or.inr (Or.inl harc))
    · exact chord_to_boundaryTrace a hchord.1
      (by simpa [upperCap] using hchord.2) hp

```

ReplacementAssembly.isClosed_carrier

Classification: supplement

Source: CMVGeometry.lean:3579-3580

Declaration SHA-256: 7d390bd6162828599bf858ab60165e80e65e5c04a4677e586ff23aba97d10394

Mathematical statement. The replacement carrier is closed.

It is a finite union of closed core and cap sets.

```

theorem isClosed_carrier (a : ReplacementAssembly) : IsClosed a.carrier :=
  a.core.isClosed_carrier.union a.upperCap.isClosed_carrier

```

ReplacementAssembly.weightedPerimeter

Classification: supplement

Source: CMVGeometry.lean:3330-3332

Declaration SHA-256: ff8986acb5666758cd2b4555262842f122d66f79d9aaa241c37692769a5c2fbc

Mathematical statement. Weighted component-boundary integral for the replacement.

Its equality with canonical frontier perimeter is a separate geometric theorem.

```

def weightedPerimeter (lam : ℝ) (a : ReplacementAssembly) : ℝ :=
  WeightedPerimeter lam a.boundaryMeasure

```

RightPrototypeCell.B3

Classification: supplement

Source: RightPrototypeCell.lean:122-122

Declaration SHA-256: fbc773e40d79a1d6d31822cdbd56d9a9607c6a7f8abc1ba0f47027df7b631eae

Mathematical statement. The right brick's complete type-(iii) angular term.

$$B_3 = \lambda v + \pi - w.$$

```
def B3 (p : Point) : ℝ := lam p * v p + π - w p
```

RightPrototypeCell.B4

Classification: supplement

Source: RightPrototypeCell.lean:119-119

Declaration SHA-256: 30e079ec002735e04de6acf4de75c1ef255bd47fc8db540544074fad80b3a6bc

Mathematical statement. The right brick's type-(iv) angular term.

$$B_4 = \lambda y + \pi/2 - x.$$

```
def B4 (p : Point) : ℝ := lam p * y p + π / 2 - x p
```

RightPrototypeCell.Brick

Classification: supplement

Source: RightPrototypeCell.lean:20-22

Declaration SHA-256: c2ed826f9f797c84fdf65a35205275ba3596085d3c93941f0721f070655d9db0

Mathematical statement. The shared middle-face brick type used by the right prototype.

The prototype instantiates the existing rational predictor-and-radius record.

```
abbrev Brick := MiddleFaceAssembly.Brick

/-- Exact retained upper prototype brick, spanning `[25781/25000, 33/32]`. -/
```

RightPrototypeCell.C3

Classification: supplement

Source: RightPrototypeCell.lean:128-128

Declaration SHA-256: bb334eb31664c80305d156013a66d4b3847b5f60bc740d68b3db43dd6612439b

Mathematical statement. The right prototype's type-(iii) incidence residual.

$$C_3 = \cos w - \lambda \cos v.$$

```
def C3 (p : Point) : ℝ := cos (w p) - lam p * cos (v p)
```

RightPrototypeCell.C3_sigma_high_face_margin

Classification: supplement

Source: RightPrototypeCell.lean:1818-1844

Declaration SHA-256: 5b3bf1ce51b69bccdd8bfe9c8c7de37ad57b8cb9ed698111103e6ab5d416bdc b

Mathematical statement. C3 is strictly positive on the high sigma face.

This completes the type-(iii) incidence face pair.

```

theorem C3_sigma_high_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 3 = radiiReal 3 →
    (1 / 1000000000 : ℝ) < C3 (pointOf t z) := by
intro t z ht hz hsigma
have ht' : -(1 / 200000 : ℝ) ≤ t ∧ t ≤ 1 / 200000 := by
  norm_num [tInterval, tRadius, brick,
    LeanSuffixReflective.QInterval.RealContains] at ht
  exact ht
have hrho := hz 2
norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at hrho
have hsigma' : z 3 = (1 / 1000000000 : ℝ) := by
  norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three] at hsigma
  exact hsigma
have hzero := c3Reduced_sigma_zero_abs_lt ht' hrho
have hgrowth := c3Reduced_sigma_growth ht' hrho
  (sigma1 := 0) (sigma2 := (1 / 1000000000 : ℝ))
  (by constructor <=> norm_num) (by constructor <=> norm_num) (by norm_num)
change (1 / 1000000000 : ℝ) < c3Reduced t (z 2) (z 3)
rw [hsigma']
have hzeroLo := (abs_lt.mp hzero).1
norm_num at hgrowth hzeroLo ⊢
linarith

set_option maxHeartbeats 1000000 in
-- Six exact degree-27/26 point enclosures require extra normalization time.

```

RightPrototypeCell.C3_sigma_low_face_margin

Classification: supplement

Source: RightPrototypeCell.lean:1791-1817

Declaration SHA-256: 6fb3362f5b47edfd5c864f48fb4f2f4eb79f2e4e22847db0995b5bffe22e621e

Mathematical statement. C3 is strictly negative on the low sigma face.

The exact margin holds throughout the other error-coordinate ranges.

```

theorem C3_sigma_low_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 3 = -radiiReal 3 →
    C3 (pointOf t z) < -(1 / 1000000000 : ℝ) := by
intro t z ht hz hsigma
have ht' : -(1 / 200000 : ℝ) ≤ t ∧ t ≤ 1 / 200000 := by
  norm_num [tInterval, tRadius, brick,
    LeanSuffixReflective.QInterval.RealContains] at ht
  exact ht
have hrho := hz 2
norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at hrho
have hsigma' : z 3 = -(1 / 1000000000 : ℝ) := by
  norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three] at hsigma
  exact hsigma
have hzero := c3Reduced_sigma_zero_abs_lt ht' hrho
have hgrowth := c3Reduced_sigma_growth ht' hrho
  (sigma1 := -(1 / 1000000000 : ℝ)) (sigma2 := 0)
  (by constructor <=> norm_num) (by constructor <=> norm_num) (by norm_num)
change c3Reduced t (z 2) (z 3) < -(1 / 1000000000 : ℝ)
rw [hsigma']
have hzeroHi := (abs_lt.mp hzero).2
norm_num at hgrowth hzeroHi ⊢
linarith

/-- The upper `sigma` face has the required `C3` sign with the strict exact
rational margin `10-9`. -/

```


RightPrototypeCell.C4

Classification: supplement

Source: RightPrototypeCell.lean:126-126

Declaration SHA-256: 0c76dd8701b0777d1f4e908247b8a477e6ed59455006924fb2e56c23786a4bd0

Mathematical statement. The right prototype's type-(iv) incidence residual.

$$C_4 = \cos x - \lambda \cos y.$$

```
def C4 (p : Point) : ℝ := cos (x p) - lam p * cos (y p)
```

RightPrototypeCell.C4_eta_high_face_margin

Classification: supplement

Source: RightPrototypeCell.lean:1215-1244

Declaration SHA-256: 0c746a428e5a281f55bd8109c26b40469bd8fb48b26367a9a7fa6a1522dfd978

Mathematical statement. C4 is strictly positive on the high eta face.

Together with the low-face estimate, it supplies the incidence coordinate's root orientation.

```
theorem C4_eta_high_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 1 = radiiReal 1 →
    (11 / 10000000000 : ℝ) < C4 (pointOf t z) := by
  intro t z ht hz heta
  have ht' : -(1 / 200000 : ℝ) ≤ t ∧ t ≤ 1 / 200000 := by
    norm_num [tInterval, tRadius, brick,
      LeanSuffixReflective.QInterval.RealContains] at ht
  exact ht
  have hxi := hz 0
  norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
    Matrix.cons_val_three] at hxi
  have heta' : z 1 = (1 / 1000000000 : ℝ) := by
    norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three] at heta
  exact heta
  have hzero := c4Reduced_eta_zero_abs_lt ht' hxi
  have hgrowth := c4Reduced_eta_growth ht' hxi
    (eta₁ := 0) (eta₂ := (1 / 1000000000 : ℝ))
    (by constructor <=> norm_num) (by constructor <=> norm_num) (by norm_num)
  change (11 / 10000000000 : ℝ) < c4Reduced t (z 0) (z 1)
  rw [heta']
  have hzeroLo := (abs_lt.mp hzero).1
  norm_num at hgrowth hzeroLo ⊢
  linarith

/- Exact point certificate at the lower `xi` face, before the small `t` and
`eta` variations are introduced. -/
set_option maxHeartbeats 1000000 in
-- Exact degree-27/26 endpoint normalization exceeds the default heartbeat budget.
```

RightPrototypeCell.C4_eta_low_face_margin

Classification: supplement

Source: RightPrototypeCell.lean:1188-1214

Declaration SHA-256: b33b3e4037ad423afeac36d4ddac306b90781f3f172e1644c7e6268fd9b3e55b

Mathematical statement. The right prototype's C4 residual is strictly negative on the low eta face.

The proved rational margin is uniform over all other allowed coordinates.

```

theorem C4_eta_low_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 1 = -radiiReal 1 →
    C4 (pointOf t z) < -(11 / 1000000000 : ℝ) := by
intro t z ht hz heta
have ht' : -(1 / 200000 : ℝ) ≤ t ∧ t ≤ 1 / 200000 := by
  norm_num [tInterval, tRadius, brick,
    LeanSuffixReflective.QInterval.RealContains] at ht
exact ht
have hxi := hz 0
norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at hxi
have heta' : z 1 = -(1 / 100000000 : ℝ) := by
  norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three] at heta
exact heta
have hzero := c4Reduced_eta_zero_abs_lt ht' hxi
have hgrowth := c4Reduced_eta_growth ht' hxi
(eta1 := -(1 / 100000000 : ℝ)) (eta2 := 0)
  (by constructor <=> norm_num) (by constructor <=> norm_num) (by norm_num)
change c4Reduced t (z 0) (z 1) < -(11 / 1000000000 : ℝ)
rw [heta']
have hzeroHi := (abs_lt.mp hzero).2
norm_num at hgrowth hzeroHi ⊢
linarith

/-- The upper `eta` face has the required `C4` sign with the strict exact
rational margin `11/10 · 10-9`. -/

```

RightPrototypeCell.D3

Classification: supplement

Source: RightPrototypeCell.lean:123-123

Declaration SHA-256: 4496291242cc1a371596577f5bdbec61dd292b63c92e9b87cf551b140d32a368

Mathematical statement. The right brick's type-(iii) sine displacement.

$$D_3 = \sin v - \sin w.$$

```
def D3 (p : Point) : ℝ := sin (v p) - sin (w p)
```

RightPrototypeCell.D4

Classification: supplement

Source: RightPrototypeCell.lean:120-120

Declaration SHA-256: 458c17f197940395eb065d606c56a29538dee5cfc382d507f31979fa9b14e3b0

Mathematical statement. The right brick's type-(iv) sine displacement.

$$D_4 = \sin y - \sin x.$$

```
def D4 (p : Point) : ℝ := sin (y p) - sin (x p)
```

RightPrototypeCell.E

Classification: supplement

Source: RightPrototypeCell.lean:129-130

Declaration SHA-256: 473eec5a0b00f868ac4f7ad090edf435fe47f07ca5bd2b59758512636541fbfa

Mathematical statement. The right prototype's cleared equal-area residual.

$$E = 2\cos^2 x N_3 - (1 + \cos w)^2 N_4.$$

```
def E (p : Point) : ℝ :=
  2 * cos (x p) ^ 2 * N3 p - (1 + cos (w p)) ^ 2 * N4 p
```

RightPrototypeCell.E_rho_high_face_margin

Classification: supplement

Source: RightPrototypeCell.lean:2056-2088

Declaration SHA-256: eb3eec8dea60a8df64d88b6b4837361d3d879b92fece643a52712269468a17a0

Mathematical statement. E is strictly positive on the high rho face.

This is the upper equal-area face needed for root existence.

```
theorem E_rho_high_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 2 = radiiReal 2 →
    (14 / 1000000000 : ℝ) < E (pointOf t z) := by
  intro t z ht hz hrhoFace
  have hxi := hz 0
  have heta := hz 1
  have hsigma := hz 3
  norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
    Matrix.cons_val_three] at hxi heta hsigma
  have ht' : -(1 / 200000 : ℝ) ≤ t ∧ t ≤ 1 / 200000 := by
    norm_num [tInterval, tRadius, brick,
      LeanSuffixReflective.QInterval.RealContains] at ht
  exact ht
  have hrho : z 2 = (1 / 100000000 : ℝ) := by
    norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three]
    at hrhoFace
  exact hrhoFace
  have hzero := eReduced_rho_zero_abs_lt ht' hxi heta hsigma
  have hgrowth := eReduced_rho_growth ht' hxi heta hsigma
  (rho₁ := 0) (rho₂ := (1 / 100000000 : ℝ))
  (by constructor <=> norm_num) (by constructor <=> norm_num) (by norm_num)
  rw [show E (pointOf t z) = eReduced t (z 0) (z 1) (z 2) (z 3) by rfl,
    hrho]
  have hzeroLo := (abs_lt.mp hzero).1
  norm_num at hgrowth hzeroLo ⊢
  linarith

/-- Proof-producing checker instance for the retained upper prototype brick.

Every field is one strict semantic decision from the retained checker contract;
the rational margins are proved by the interval lemmas above rather than
trusted as serialized data. -/
```

RightPrototypeCell.E_rho_low_face_margin

Classification: supplement

Source: RightPrototypeCell.lean:2028-2055

Declaration SHA-256: 6272cbb2e67c611a239abd88e60ef57c905b209d0a231e982ffd7c0dc0e89972

Mathematical statement. E is strictly negative on the low rho face.

The cleared equal-area equation has the required lower-face orientation.

```
theorem E_rho_low_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 2 = -radiiReal 2 →
    E (pointOf t z) < -(14 / 1000000000 : ℝ) := by
  intro t z ht hz hrhoFace
  have hxi := hz 0
  have heta := hz 1
  have hsigma := hz 3
```

```

norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at hxi heta hsigma
have ht' : -(1 / 200000 : ℝ) ≤ t ∧ t ≤ 1 / 200000 := by
  norm_num [tInterval, tRadius, brick,
    LeanSuffixReflective.QInterval.RealContains] at ht
exact ht
have hrho : z 2 = -(1 / 10000000 : ℝ) := by
  norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three]
  at hrhoFace
exact hrhoFace
have hzero := eReduced_rho_zero_abs_lt ht' hxi heta hsigma
have hgrowth := eReduced_rho_growth ht' hxi heta hsigma
  (rho₁ := -(1 / 10000000 : ℝ)) (rho₂ := 0)
  (by constructor <;> norm_num) (by constructor <;> norm_num) (by norm_num)
rw [show E (pointOf t z) = eReduced t (z 0) (z 1) (z 2) (z 3) by rfl,
  hrho]
have hzeroHi := (abs_lt.mp hzero).2
norm_num at hgrowth hzeroHi ⊢
linarith

```

RightPrototypeCell.F

Classification: supplement

Source: RightPrototypeCell.lean:127-127

Declaration SHA-256: ff764cf036982bdc0d990c94070c7ae41de884ec397b4b250e1096fedd89ff34

Mathematical statement. The right prototype's stationary type-(iv) residual.

$$F = \cos x D_4 - \sin x \sin y B_4.$$

```

def F (p : Point) : ℝ := cos (x p) * D4 p - sin (x p) * sin (y p) * B4 p

```

RightPrototypeCell.F_xi_high_face_margin

Classification: supplement

Source: RightPrototypeCell.lean:1572-1597

Declaration SHA-256: ffe50f040a2dc348cba7032d1ad7f3678a00d83ce29c2e6aded2308288cab74e

Mathematical statement. F is strictly negative on the high xi face.

Negating it gives the positive high-face sign required by the root map.

```

theorem F_xi_high_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 0 = radiiReal 0 →
    F (pointOf t z) < -(86 / 1000000000 : ℝ) := by
intro t z ht hz hxiFace
have ht' : -(1 / 200000 : ℝ) ≤ t ∧ t ≤ 1 / 200000 := by
  norm_num [tInterval, tRadius, brick,
    LeanSuffixReflective.QInterval.RealContains] at ht
exact ht
have heta := hz 1
norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at heta
have hxi : z 0 = (1 / 10000000 : ℝ) := by
  norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three]
  at hxiFace
exact hxiFace
have hdev := fReduced_face_deviation ht'
  (show -(1 / 10000000 : ℝ) ≤ 1 / 10000000 ∧
    (1 / 10000000 : ℝ) ≤ 1 / 10000000 by norm_num) heta
change fReduced t (z 0) (z 1) < -(86 / 1000000000 : ℝ)
rw [hxi]
rw [abs_le] at hdev
linarith [fReduced_high_anchor]

```

```
/-- Exact origin certificate for the correlated `C3` residual. -/
```

RightPrototypeCell.F_xi_low_face_margin

Classification: supplement

Source: RightPrototypeCell.lean:1546-1571

Declaration SHA-256: 09956ba2c2677ff4925e529c6578299ab10ba3f2d08cee3e462aa50640f59f58

Mathematical statement. F is strictly positive on the right prototype's low ξ face.

The root-map component is $-F$, so its low-face sign is negative.

```
theorem F_xi_low_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 0 = -radiiReal 0 →
    (88 / 1000000000 : ℝ) < F (pointOf t z) := by
  intro t z ht hz hxiFace
  have ht' : -(1 / 200000 : ℝ) ≤ t ∧ t ≤ 1 / 200000 := by
    norm_num [tInterval, tRadius, brick,
      LeanSuffixReflective.QInterval.RealContains] at ht
  exact ht
  have heta := hz 1
  norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
    Matrix.cons_val_three] at heta
  have hxi : z 0 = -(1 / 10000000 : ℝ) := by
    norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three]
  at hxiFace
  exact hxiFace
  have hdev := fReduced_face_deviation ht'
  (show -(1 / 10000000 : ℝ) ≤ -(1 / 10000000) ∧
    -(1 / 10000000 : ℝ) ≤ 1 / 10000000 by norm_num) heta
  change (88 / 1000000000 : ℝ) < fReduced t (z 0) (z 1)
  rw [hxi]
  rw [abs_le] at hdev
  linarith [fReduced_low_anchor]

/-- On the upper `xi` face, `F` is uniformly negative with exact strict
margin `86 · 10-9` over the complete retained upper prototype brick. -/
```

RightPrototypeCell.G

Classification: supplement

Source: RightPrototypeCell.lean:131-137

Declaration SHA-256: 818854a9d2ac34b27a374a124d11eeffc488199875161c552aa19dcc6709e1ea

Mathematical statement. The right prototype's scaled perimeter-gap residual.

$$G = \cos x(B_3 + D_3) - (1 + \cos w)B_4.$$

```
def G (p : Point) : ℝ :=
  cos (x p) * (B3 p + D3 p) - (1 + cos (w p)) * B4 p

/-- The shared analytic interpretation of the checker coordinates at a point
on the recorded angle branch. The equation and sign fields deliberately take
their checker hypotheses as arguments, so the expensive trigonometric
translation is proved only once. -/
```

RightPrototypeCell.K3

Classification: supplement

Source: RightPrototypeCell.lean:238-245

Declaration SHA-256: dded0a92a5475f561e4ec09b53d31fd672246c1e3a18679f1d82548595f59954

Mathematical statement. The right prototype's type-(iii) fold expression.

It is the angular form of K_3 at $s = (1 + \cos w)/2$.

```
def K3 (p : Point) : ℝ :=
  let h3 := (1 + cos (w p)) / 2
  4 * h3 * ((1 + cos (w p)) / sin (w p) -
    (lam p ^ 2 + cos (w p)) / (lam p * (lam p * sin (v p)))) -
    2 * (B3 p + D3 p)

/-- On the certified angle branch, the checker quantity `K3` is exactly the
numerator of the type-(iii) area derivative. -/
```

RightPrototypeCell.N3

Classification: supplement

Source: RightPrototypeCell.lean:124-125

Declaration SHA-256: 79fd22b0e88fadf424caa5c32cd0621ba146426141946780c13923b4adb4108c

Mathematical statement. The right brick's type-(iii) area numerator.

$N_3 = B_3 + (\cos w + 2)D_3$.

```
def N3 (p : Point) : ℝ := B3 p + (cos (w p) + 2) * D3 p
```

RightPrototypeCell.N4

Classification: supplement

Source: RightPrototypeCell.lean:121-121

Declaration SHA-256: 8317292c5d87208707548fa6a07d20553105db710dba94daee68266201ce16ef

Mathematical statement. The right brick's type-(iv) area numerator.

$N_4 = B_4 + \cos x D_4$.

```
def N4 (p : Point) : ℝ := B4 p + cos (x p) * D4 p
```

RightPrototypeCell.Point

Classification: supplement

Source: RightPrototypeCell.lean:109-110

Declaration SHA-256: cff2e93e4339fbf01155e35df8506905eb069e6d43c06718e6b5df7c41ed32d5

Mathematical statement. The centered five-coordinate point type for the right prototype.

It carries density displacement and four angular error variables.

```
abbrev Point := MiddleFaceAssembly.Point
```

RightPrototypeCell.affine_branches

Classification: supplement

Source: RightPrototypeCell.lean:707-741

Declaration SHA-256: 9a77fd11c6c2c0e0a916f823adc4aea4c608cadccacf0dbe2100c6ac6d1bce78

Mathematical statement. All four right-prototype angle predictors stay on the checked principal branch.

For every point in the certified density/error box, $0 < x, y, w, v < \pi/4$.

```
theorem affine_branches :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    let p := pointOf t z
    0 < x p ^ x p < π / 4 ∧ 0 < y p ^ y p < π / 4 ∧
    0 < w p ^ w p < π / 4 ∧ 0 < v p ^ v p < π / 4 := by
intro t z ht hz
have hz0 := hz 0
have hz1 := hz 1
have hz2 := hz 2
have hz3 := hz 3
norm_num [tInterval, tRadius, brick,
  LeanSuffixReflective.QInterval.RealContains] at ht
norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at hz0 hz1 hz2 hz3
dsimp only
norm_num [pointOf, x, y, w, v, brick, Matrix.cons_val_two,
  Matrix.cons_val_three]
constructor
· linarith
constructor
· linarith [Real.pi_gt_d6]
constructor
· linarith
constructor
· linarith [Real.pi_gt_d6]
constructor
· linarith
constructor
· linarith [Real.pi_gt_d6]
constructor
· linarith
constructor
· linarith [Real.pi_gt_d6]
constructor
· linarith
· linarith [Real.pi_gt_d6]

/-- Exact affine box arithmetic separates the type-(iv) and type-(iii) angle
pairs throughout the retained upper prototype brick. -/
```

RightPrototypeCell.brick

Classification: supplement

Source: RightPrototypeCell.lean:23-39

Declaration SHA-256: 46529d69eb028da84cb1e1cd48a1286b5a20bbf7043954e20190c659fde37b2d

Mathematical statement. The exact right-prototype rational brick.

Its density interval is $[25781/25000, 33/32]$; the frozen source records all predictor coefficients and radii.

```
def brick : Brick where
lamLo := 25781 / 25000
lamHi := 33 / 32
centers := ![2048263294902610889 / 10000000000000000000,
  3193765780936307979 / 10000000000000000000,
  4769698287901120439 / 10000000000000000000,
  2663161348942592843 / 5000000000000000000]
slopes := ![140927032410729353 / 1000000000000000000,
  954393643324404653 / 2500000000000000000,
  4401546318656929969 / 10000000000000000000,
```

```

5503812760533025527 / 1000000000000000000]
yXiSlope := 13 / 20
vRhoSlope := 219 / 250
radii := ![1 / 10000000, 1 / 100000000, 1 / 10000000, 1 / 100000000]
lamOrdered := by norm_num
radiiPos := by intro j; fin_cases j <=> norm_num

```

RightPrototypeCell.cellCertificate

Classification: supplement

Source: RightPrototypeCell.lean:2089-2179

Declaration SHA-256: 37a6b153f1c09f5b0bd0af4e9abeec7bf284e6e510faf353cac76a90f727cac4

Mathematical statement. The right prototype satisfies every primitive middle-face certificate field.

It assembles the checked branch, density, face, descending-fold, and strict-gap estimates.

```

theorem cellCertificate : MiddleFaceAssembly.CellCertificate brick where
  lambda_one_lt := by
    intro t ht
    change tInterval.RealContains t at ht
    change 1 < lam (pointOf t 0)
    have hlamCell : lamInterval.RealContains ((lamMid : ℝ) + t) := by
      apply lambda_iff_centered.mpr
      convert ht using 1 <=> first | ring | rfl
    simp [lam, pointOf] using (lambda_in_retainedSuffix hlamCell).one_lt
  branches := by
    intro t z ht hz
    change tInterval.RealContains t at ht
    change InSymmetricBox radiiReal z at hz
    change
      let p := pointOf t z
      0 < x p ^ x p < π / 4 ^
      0 < y p ^ y p < π / 4 ^
      0 < w p ^ w p < π / 4 ^
      0 < v p ^ v p < π / 4
    exact affine_branches t z ht hz
  F_xi_low_pos := by
    intro t z ht hz hface
    change tInterval.RealContains t at ht
    change InSymmetricBox radiiReal z at hz
    change z 0 = -radiiReal 0 at hface
    change 0 < F (pointOf t z)
    linarith [F_xi_low_face_margin t z ht hz hface]
  F_xi_high_neg := by
    intro t z ht hz hface
    change tInterval.RealContains t at ht
    change InSymmetricBox radiiReal z at hz
    change z 0 = radiiReal 0 at hface
    change F (pointOf t z) < 0
    linarith [F_xi_high_face_margin t z ht hz hface]
  C4_eta_low_neg := by
    intro t z ht hz hface
    change tInterval.RealContains t at ht
    change InSymmetricBox radiiReal z at hz
    change z 1 = -radiiReal 1 at hface
    change C4 (pointOf t z) < 0
    linarith [C4_eta_low_face_margin t z ht hz hface]
  C4_eta_high_pos := by
    intro t z ht hz hface
    change tInterval.RealContains t at ht
    change InSymmetricBox radiiReal z at hz
    change z 1 = radiiReal 1 at hface
    change 0 < C4 (pointOf t z)
    linarith [C4_eta_high_face_margin t z ht hz hface]
  E_rho_low_neg := by
    intro t z ht hz hface
    change tInterval.RealContains t at ht
    change InSymmetricBox radiiReal z at hz

```



```

change z 2 = -radiiReal 2 at hface
change E (pointOf t z) < 0
linarith [E_rho_low_face_margin t z ht hz hface]
E_rho_high_pos := by
intro t z ht hz hface
change tInterval.RealContains t at ht
change InSymmetricBox radiiReal z at hz
change z 2 = radiiReal 2 at hface
change 0 < E (pointOf t z)
linarith [E_rho_high_face_margin t z ht hz hface]
C3_sigma_low_neg := by
intro t z ht hz hface
change tInterval.RealContains t at ht
change InSymmetricBox radiiReal z at hz
change z 3 = -radiiReal 3 at hface
change C3 (pointOf t z) < 0
linarith [C3_sigma_low_face_margin t z ht hz hface]
C3_sigma_high_pos := by
intro t z ht hz hface
change tInterval.RealContains t at ht
change InSymmetricBox radiiReal z at hz
change z 3 = radiiReal 3 at hface
change 0 < C3 (pointOf t z)
linarith [C3_sigma_high_face_margin t z ht hz hface]
K3_neg := by
intro t z ht hz
change tInterval.RealContains t at ht
change InSymmetricBox radiiReal z at hz
change K3 (pointOf t z) < 0
exact descendingK3 t z ht hz
G_neg := by
intro t z ht hz
change tInterval.RealContains t at ht
change InSymmetricBox radiiReal z at hz
change G (pointOf t z) < 0
exact (gap_upper t z ht hz).trans (by norm_num)

/-- Sound projection of the retained upper prototype checker instance into the generic
root/projection assembly contract. -/

```

RightPrototypeCell.checkedConditions

Classification: supplement

Source: RightPrototypeCell.lean:2180-2184

Declaration SHA-256: 907b696193f7401fd5d37dde529193bc1f7dd81a2b600bbb7b76d63e5238086e

Mathematical statement. The right prototype satisfies the reusable oriented root conditions.

Apply generic certificate soundness to its primitive sign record.

```

theorem checkedConditions : MiddleFaceAssembly.Conditions brick :=
  MiddleFaceAssembly.CellCertificate.sound cellCertificate

/-- Every projected type-(iv) height in the retained upper prototype has an equal-area regular
type-(iii) competitor with strictly smaller weighted perimeter. -/

```

RightPrototypeCell.descendingK3

Classification: supplement

Source: RightPrototypeCell.lean:766-934

Declaration SHA-256: e9afb66c8c2aa5658ce6fd84c67f02d2de09068f00ce68b48aceab8eeae79c86

Mathematical statement. The right prototype has strictly negative K3 throughout its box.

The rational enclosure certifies the descending type-(iii) branch uniformly.

```

theorem descendingK3 :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    K3 (pointOf t z) < 0 := by
intro t z ht hz
let p := pointOf t z
have hz0 := hz 0
have hz2 := hz 2
have hz3 := hz 3
norm_num [tInterval, tRadius, brick,
  LeanSuffixReflective.QInterval.RealContains] at ht
norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at hz0 hz2 hz3
have hlamLo : (1 : ℝ) < lam p := by
  dsimp [p, pointOf, lam]
  norm_num [lamMid, brick]
  linarith
have hlamHi : lam p < (26 / 25 : ℝ) := by
  dsimp [p, pointOf, lam]
  norm_num [lamMid, brick]
  linarith
have hwLo : (2 / 5 : ℝ) ≤ w p := by
  dsimp [p, pointOf, w]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hwHi : w p ≤ (239 / 500 : ℝ) := by
  dsimp [p, pointOf, w]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hvLo : (9 / 20 : ℝ) ≤ v p := by
  dsimp [p, pointOf, v]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hvHi : v p ≤ (533 / 1000 : ℝ) := by
  dsimp [p, pointOf, v]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hwI :
  ((2 / 5, 239 / 500, by norm_num) : QInterval).RealContains (w p) := by
  constructor <=> norm_num <=> assumption
have hvI :
  ((9 / 20, 533 / 1000, by norm_num) : QInterval).RealContains (v p) := by
  constructor <=> norm_num <=> assumption
have hsw :
  ((389 / 1000, 47 / 100, by norm_num) : QInterval).RealContains
    (sin (w p)) := by
  apply realContains_sin_interval hwI
  · constructor <=> linarith [Real.pi_gt_d6]
  · have h := (sinTaylor27Interval_sound (2 / 5)).1
    norm_num [sinTaylor27Interval, sinTaylor27, Finset.sum_range_succ,
      Nat.factorial] at h
    norm_num
    linarith
  · have h := (sinTaylor27Interval_sound (239 / 500)).2
    norm_num [sinTaylor27Interval, sinTaylor27, Finset.sum_range_succ,
      Nat.factorial] at h
    norm_num
    linarith
have hsv :
  ((2 / 5, 51 / 100, by norm_num) : QInterval).RealContains
    (sin (v p)) := by
  apply realContains_sin_interval hvI
  · constructor <=> linarith [Real.pi_gt_d6]
  · have h := (sinTaylor27Interval_sound (9 / 20)).1
    norm_num [sinTaylor27Interval, sinTaylor27, Finset.sum_range_succ,
      Nat.factorial] at h
    norm_num
    linarith
  · have h := (sinTaylor27Interval_sound (533 / 1000)).2
    norm_num [sinTaylor27Interval, sinTaylor27, Finset.sum_range_succ,
      Nat.factorial] at h
    norm_num

```

```

linarith
have hsvHi : sin (v p) ≤ (51 / 100 : ℝ) := by
  norm_num
  nlinarith [hsv.2]
have hcw :
  ((22 / 25, 461 / 500, by norm_num) : QInterval).RealContains
    (cos (w p)) := by
  apply realContains_cos_interval hwI
  · constructor <=> linarith [Real.pi_gt_d6]
  · have h := (cosTaylor26Interval_sound (239 / 500)).1
    norm_num [cosTaylor26Interval, cosTaylor26, Finset.sum_range_succ,
      Nat.factorial] at h
    norm_num
    linarith
  · have h := (cosTaylor26Interval_sound (2 / 5)).2
    norm_num [cosTaylor26Interval, cosTaylor26, Finset.sum_range_succ,
      Nat.factorial] at h
    norm_num
    linarith
have hswPos : 0 < sin (w p) := lt_of_lt_of_le (by norm_num) hsw.1
have hsvPos : 0 < sin (v p) := lt_of_lt_of_le (by norm_num) hsv.1
have hlamPos : 0 < lam p := lt_trans (by norm_num) hlamLo
have hlamSqPos : 0 < lam p ^ 2 := sq_pos_of_pos hlamPos
have hA : (1 + cos (w p)) / sin (w p) < (99 / 20 : ℝ) := by
  apply (div_lt_iff₀ hswPos).2
  nlinarith [hcw.2, hsw.1]
have hlamSqHi : lam p ^ 2 < (676 / 625 : ℝ) := by
  nlinarith
have hdenHi :
  lam p * (lam p * sin (v p)) < (69 / 125 : ℝ) := by
  rw [show lam p * (lam p * sin (v p)) =
    lam p ^ 2 * sin (v p) by ring]
  have h₁ :
    lam p ^ 2 * sin (v p) ≤ lam p ^ 2 * (51 / 100 : ℝ) :=
    mul_le_mul_of_nonneg_left hsvHi hlamSqPos.le
  have h₂ :
    lam p ^ 2 * (51 / 100 : ℝ) <
      (676 / 625 : ℝ) * (51 / 100 : ℝ) :=
    mul_lt_mul_of_pos_right hlamSqHi (by norm_num)
  norm_num at h₂ ⊢
  exact h₁.trans_lt (h₂.trans (by norm_num))
have hdenPos : 0 < lam p * (lam p * sin (v p)) := by positivity
have hnumLo : (47 / 25 : ℝ) < lam p ^ 2 + cos (w p) := by
  nlinarith [hcw.1]
have hB : (17 / 5 : ℝ) <
  (lam p ^ 2 + cos (w p)) /
  (lam p * (lam p * sin (v p))) := by
  apply (lt_div_iff₀ hdenPos).2
  nlinarith [hnumLo, hdenHi]
have hdiff :
  (1 + cos (w p)) / sin (w p) -
  (lam p ^ 2 + cos (w p)) /
  (lam p * (lam p * sin (v p))) <
  (31 / 20 : ℝ) := by
  linarith
have hfactorPos : 0 < 2 * (1 + cos (w p)) := by
  nlinarith [hcw.1]
have hfactorHi : 2 * (1 + cos (w p)) ≤ (961 / 250 : ℝ) := by
  nlinarith [hcw.2]
have hproduct :
  2 * (1 + cos (w p)) *
  ((1 + cos (w p)) / sin (w p) -
  (lam p ^ 2 + cos (w p)) /
  (lam p * (lam p * sin (v p)))) <
  (149 / 25 : ℝ) := by
  by_cases hd :
    0 ≤ (1 + cos (w p)) / sin (w p) -
      (lam p ^ 2 + cos (w p)) /
      (lam p * (lam p * sin (v p)))
  · nlinarith [mul_lt_mul_of_pos_left hdiff hfactorPos,
    mul_le_mul_of_nonneg_right hfactorHi hd]
  · have hd' := lt_of_not_ge hd

```

```

have hnegative := mul_neg_of_pos_of_neg hfactorPos hd'
norm_num at hnegative ⊢
linarith
have hlamv : (9 / 20 : ℝ) < lam p * v p := by
  have hvPos : 0 < v p := lt_of_lt_of_le (by norm_num) hvLo
  nlinarith [mul_lt_mul_of_pos_right hlamLo hvPos]
have hbase : (3 : ℝ) < B3 p + D3 p := by
  unfold B3 D3
  nlinarith [hlamv, Real.pi_gt_d6, hwHi, hsv.1, hsw.2]
change K3 p < 0
unfold K3
dsimp only
rw [show
  4 * ((1 + cos (w p)) / 2) *
    ((1 + cos (w p)) / sin (w p) -
      (lam p ^ 2 + cos (w p)) /
        (lam p * (lam p * sin (v p)))) =
  2 * (1 + cos (w p)) *
    ((1 + cos (w p)) / sin (w p) -
      (lam p ^ 2 + cos (w p)) /
        (lam p * (lam p * sin (v p)))) by ring]
linarith

/-- A direct rational interval decomposition proves the scalar perimeter gap
throughout the retained upper prototype brick. The explicit negative margin keeps this
leaf independent of the root-existence argument. -/

```

RightPrototypeCell.gap_upper

Classification: supplement

Source: RightPrototypeCell.lean:935-1049

Declaration SHA-256: bfd6569809511ed8dd358788886ef4c9bc9af352f129411e514e6363ef806197

Mathematical statement. The right prototype's G residual has a strictly negative upper bound.

Hence every root in the box has strict perimeter improvement.

```

theorem gap_upper :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    G (pointOf t z) < -(437 / 1000000 : ℝ) := by
  intro t z ht hz
  let p := pointOf t z
  have hz0 := hz 0
  have hz1 := hz 1
  have hz2 := hz 2
  have hz3 := hz 3
  norm_num [tInterval, tRadius, brick,
    LeanSuffixReflective.QInterval.RealContains] at ht
  norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
    Matrix.cons_val_three] at hz0 hz1 hz2 hz3
  have hlamLo : (25781 / 25000 : ℝ) ≤ lam p := by
    dsimp [p, pointOf, lam]
    norm_num [lamMid, brick]
    linarith
  have hlamHi : lam p ≤ (33 / 32 : ℝ) := by
    dsimp [p, pointOf, lam]
    norm_num [lamMid, brick]
    linarith
  have hxLo : (204818 / 1000000 : ℝ) ≤ x p := by
    dsimp [p, pointOf, x]
    norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
    linarith
  have hxHi : x p ≤ (204834 / 1000000 : ℝ) := by
    dsimp [p, pointOf, x]
    norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
    linarith
  have hyLo : (319357 / 1000000 : ℝ) ≤ y p := by
    dsimp [p, pointOf, y]
    norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]

```

```

linarith
have hwLo : (476947 / 1000000 : ℝ) ≤ w p := by
  dsimp [p, pointOf, w]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hwHi : w p ≤ (476993 / 1000000 : ℝ) := by
  dsimp [p, pointOf, w]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hvLo : (532604 / 1000000 : ℝ) ≤ v p := by
  dsimp [p, pointOf, v]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hvHi : v p ≤ (532661 / 1000000 : ℝ) := by
  dsimp [p, pointOf, v]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hcXHi : cos (x p) ≤ (9790981 / 10000000 : ℝ) := by
  have hmono : cos (x p) ≤ cos (204818 / 1000000 : ℝ) :=
    Real.cos_le_cos_of_nonneg_of_le_pi (by norm_num)
    (by linarith [Real.pi_gt_d6]) hxLo
  have h := (cosTaylor26Interval_sound (204818 / 1000000)).2
  norm_num [cosTaylor26Interval, cosTaylor26, Finset.sum_range_succ,
    Nat.factorial] at h
  linarith
have hcwLo : (8883794 / 10000000 : ℝ) ≤ cos (w p) := by
  have hmono : cos (476993 / 1000000 : ℝ) ≤ cos (w p) :=
    Real.cos_le_cos_of_nonneg_of_le_pi (by linarith)
    (by linarith [Real.pi_gt_d6]) hwHi
  have h := (cosTaylor26Interval_sound (476993 / 1000000)).1
  norm_num [cosTaylor26Interval, cosTaylor26, Finset.sum_range_succ,
    Nat.factorial] at h
  linarith
have hsvHi : sin (v p) ≤ (5078275 / 10000000 : ℝ) := by
  have hmono : sin (v p) ≤ sin (532661 / 1000000 : ℝ) :=
    Real.sin_le_sin_of_le_of_le_pi_div_two (by linarith [Real.pi_pos])
    (by linarith [Real.pi_gt_d6]) hvHi
  have h := (sinTaylor27Interval_sound (532661 / 1000000)).2
  norm_num [sinTaylor27Interval, sinTaylor27, Finset.sum_range_succ,
    Nat.factorial] at h
  linarith
have hswLo : (4590690 / 10000000 : ℝ) ≤ sin (w p) := by
  have hmono : sin (476947 / 1000000 : ℝ) ≤ sin (w p) :=
    Real.sin_le_sin_of_le_of_le_pi_div_two (by linarith [Real.pi_pos])
    (by linarith [Real.pi_gt_d6]) hwLo
  have h := (sinTaylor27Interval_sound (476947 / 1000000)).1
  norm_num [sinTaylor27Interval, sinTaylor27, Finset.sum_range_succ,
    Nat.factorial] at h
  linarith
have hlamvHi : lam p * v p ≤
  (33 / 32 : ℝ) * (532661 / 1000000 : ℝ) := by
  exact mul_le_mul hlamHi hvHi (by linarith) (by norm_num)
have hlamyLo : (25781 / 25000 : ℝ) * (319357 / 1000000 : ℝ) ≤
  lam p * y p := by
  exact mul_le_mul hlamLo hyLo (by norm_num) (by linarith)
have hAHi : B3 p + D3 p < (16315 / 5000 : ℝ) := by
  unfold B3 D3
  linarith [Real.pi_lt_d6]
have hALo : (0 : ℝ) < B3 p + D3 p := by
  unfold B3 D3
  have hsinv : -(1 : ℝ) ≤ sin (v p) := neg_one_le_sin _
  have hsinw : sin (w p) ≤ 1 := sin_le_one _
  nlinarith [Real.pi_gt_d6]
have hBLo : (16951 / 10000 : ℝ) < B4 p := by
  unfold B4
  linarith [Real.pi_gt_d6]
have hfirst : cos (x p) * (B3 p + D3 p) <
  (9790981 / 10000000 : ℝ) * (16315 / 5000 : ℝ) := by
  have hcXPos : 0 < cos (x p) :=
    Real.cos_pos_of_mem_Ioo (by linarith [Real.pi_pos],
      by linarith [Real.pi_gt_d6])
  have hmul := mul_lt_mul hAHi hcXHi hcXPos (by norm_num)

```

```

nlinarith
have hsecond : (188837 / 100000 : ℝ) * (16951 / 10000 : ℝ) <
  (1 + cos (w p)) * B4 p := by
  have hf : (188837 / 100000 : ℝ) < 1 + cos (w p) := by linarith
  exact mul_lt_mul hf hBLo.le (by norm_num) (by linarith)
unfold G
nlinarith [hfirst, hsecond]

/-- Exact Taylor enclosure of the correlated `C4` residual at the brick
origin. -/

```

RightPrototypeCell.lam

Classification: supplement

Source: RightPrototypeCell.lean:111-111

Declaration SHA-256: 80d4ea0b007cc38c9cef09318804f92bb1d2031dffde45077e78cad062961da3

Mathematical statement. The right prototype's reconstructed density.

$\lambda = \lambda_{\text{mid}} + t$.

```
def lam (p : Point) : ℝ := (lamMid : ℝ) + p.t
```

RightPrototypeCell.lamInterval

Classification: supplement

Source: RightPrototypeCell.lean:79-80

Declaration SHA-256: f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6

Mathematical statement. The right prototype's closed density interval.

It is $[25781/25000, 33/32]$.

```
def lamInterval : QInterval := ⟨brick.lamLo, brick.lamHi, brick.lamOrdered.le⟩
```

RightPrototypeCell.lamMid

Classification: supplement

Source: RightPrototypeCell.lean:75-75

Declaration SHA-256: 6872199d838a1fa3e543ff329eb7ed7979df53e9ab7df478a48be79baedaf0cf

Mathematical statement. The right brick's density midpoint.

It is the exact average of its rational density endpoints.

```
abbrev lamMid : ℚ := (brick.lamLo + brick.lamHi) / 2
```

RightPrototypeCell.lambda_iff_centered

Classification: supplement

Source: RightPrototypeCell.lean:81-93

Declaration SHA-256: 8f9d4d2567e466a05274e0772e055d5bcc7985aa0cebd52f3a01300950766dca

Mathematical statement. Density-interval membership is equivalent to the centered parameter bound.

Subtract the exact midpoint from λ to obtain t .

```

theorem lambda_iff_centered {lam : ℝ} :
  lamInterval.RealContains lam ↔ tInterval.RealContains (lam - (lamMid : ℝ)) := by
  constructor
  · rintro ⟨h1, h2⟩
    constructor <=>
      norm_num [lamInterval, tInterval, lamMid, tRadius, brick,
        LeanSuffixReflective.QInterval.RealContains] at h1 h2 ⊢ <=> linarith
  · rintro ⟨h1, h2⟩
    constructor <=>
      norm_num [lamInterval, tInterval, lamMid, tRadius, brick,
        LeanSuffixReflective.QInterval.RealContains] at h1 h2 ⊢ <=> linarith

/-- Unconditional real-range conclusion for the retained upper prototype. -/

```

RightPrototypeCell.lambda_in_retainedSuffix

Classification: supplement

Source: RightPrototypeCell.lean:94-100

Declaration SHA-256: d64b377e3281e7e0b8cb140bc4c8494ca73e2578a3420d0547192d8dcd31aa70

Mathematical statement. Every right-prototype density belongs to the retained certificate interval.

Its endpoints lie between 51/50 and 9/7.

```

theorem lambda_in_retainedSuffix {lam : ℝ} (h : lamInterval.RealContains lam) :
  LeanSuffixAnalytic.InRetainedSuffix lam := by
  unfold LeanSuffixAnalytic.InRetainedSuffix
  norm_num [lamInterval, LeanSuffixReflective.QInterval.RealContains, brick] at h ⊢
  exact ⟨by linarith [h.1], h.2.trans (by norm_num)⟩

/-- The whole exact-data cell lies in the retained real suffix. -/

```

RightPrototypeCell.pointOf

Classification: supplement

Source: RightPrototypeCell.lean:268-269

Declaration SHA-256: 123eaac2727784b857a8a1d07bab6c6e08557c023e8c73b6b961970da83630664

Mathematical statement. Build a right-prototype point from centered density and error vector.

The vector entries are assigned to the four angular error coordinates.

```

def pointOf (t : ℝ) (z : Vec4) : Point := ⟨t, z 0, z 1, z 2, z 3⟩

```

RightPrototypeCell.radiiReal

Classification: supplement

Source: RightPrototypeCell.lean:267-267

Declaration SHA-256: ffbda00bb8ae49e94f67d9ad5dd16012ff60af58d9c1760a6abfb99b1bb96909

Mathematical statement. The right prototype's error radii as a real four-vector.

These cast rational radii define its closed symmetric error box.

```

def radiiReal : Vec4 := fun j => (brick.radii j : ℝ)

```

RightPrototypeCell.tInterval

Classification: supplement

Source: RightPrototypeCell.lean:78-78

Declaration SHA-256: 4051c1639ae05eb9e136816b4912782037642807485ecc541b8a4729aa41a839

Mathematical statement. The right prototype's symmetric centered density interval.

The interval is $[-t_{\text{radius}}, t_{\text{radius}}]$.

```
def tInterval : QInterval := ⟨-tRadius, tRadius, by norm_num [tRadius, brick]⟩
```

RightPrototypeCell.tRadius

Classification: supplement

Source: RightPrototypeCell.lean:76-77

Declaration SHA-256: 7ec2e049acded7f92cfecfef90c8523058c254046f34ddfc61af74c906f331c0

Mathematical statement. The right brick's centered density radius.

It is half the exact interval width.

```
abbrev tRadius : ℚ := (brick.lamHi - brick.lamLo) / 2
```

RightPrototypeCell.v

Classification: supplement

Source: RightPrototypeCell.lean:116-118

Declaration SHA-256: 7b17effff67b8a9b22200a9061ba58fb57a901753e3d49bff62bf8ea7f6d2a8ea

Mathematical statement. Its coupled type-(iii) exterior-angle predictor.

The formula includes the rho coupling and sigma error.

```
def v (p : Point) : ℝ := brick.centers 3 + brick.slopes 3 * p.t +  
  brick.vRhoSlope * p.rho + p.sigma
```

RightPrototypeCell.w

Classification: supplement

Source: RightPrototypeCell.lean:115-115

Declaration SHA-256: 52d839481355b169feb0cd07c321cec9f11b8bdc2b69f116735e48df02da7c3d

Mathematical statement. Its type-(iii) inner-angle predictor.

The affine density prediction is corrected by rho.

```
def w (p : Point) : ℝ := brick.centers 2 + brick.slopes 2 * p.t + p.rho
```

RightPrototypeCell.x

Classification: supplement

Source: RightPrototypeCell.lean:112-112

Declaration SHA-256: 3f1552bfdd08e7586bffec5b6dc6b1a410691f11d8196031e1e7ca9cbf58889a

Mathematical statement. Its type-(iv) inner-angle affine predictor.

The predictor is the recorded center plus density slope and xi error.

```
def x (p : Point) : ℝ := brick.centers 0 + brick.slopes 0 * p.t + p.xi
```

RightPrototypeCell.y

Classification: supplement

Source: RightPrototypeCell.lean:113-114

Declaration SHA-256: 9e7642719b786de830ac8e4bdac8c25ca9f746737432d174122c9b0d847d92b9

Mathematical statement. Its coupled type-(iv) exterior-angle predictor.

The formula includes density, xi coupling, and eta error.

```
def y (p : Point) : ℝ := brick.centers 1 + brick.slopes 1 * p.t +  
  brick.yXiSlope * p.xi + p.eta
```

ScalarSuffixCertificate.CertificateExpr

Classification: supplement

Source: ScalarSuffixCertificate.lean:167-175

Declaration SHA-256: b7255be61ccd2c3030071696969cf641da72d2598375adad83bc3d9088d769f2

Mathematical statement. An arithmetic certificate expression with real-value and interval interpretations.

The expression syntax allows compositional enclosure proofs rather than trusting numerical evaluation.

```
inductive CertificateExpr where  
  | rational (q : ℚ)  
  | atom (x : ℝ) (i : QInterval) (sound : i.RealContains x)  
  | add (a b : CertificateExpr)  
  | neg (a : CertificateExpr)  
  | mul (a b : CertificateExpr)  
  
namespace CertificateExpr
```

ScalarSuffixCertificate.CertificateExpr.enclosure

Classification: supplement

Source: ScalarSuffixCertificate.lean:183-190

Declaration SHA-256: 93def124f5286c57b1f3cae76f5c9b5b01ebe4c4e710fe712c5ca0db00b71550

Mathematical statement. The rational interval interpretation of a certificate expression.

Each arithmetic node is evaluated with its sound interval operation.

```
def enclosure : CertificateExpr → QInterval  
  | rational q => LeanSuffixReflective.QInterval.point q  
  | atom _ i _ => i  
  | add a b => a.enclosure.add b.enclosure  
  | neg a => a.enclosure.neg  
  | mul a b => a.enclosure.mul b.enclosure  
  
/-- Generic kernel soundness for every certificate expression. -/
```

ScalarSuffixCertificate.CertificateExpr.value

Classification: supplement

Source: ScalarSuffixCertificate.lean:176-182

Declaration SHA-256: a0149812d66bc28eb03b0052f9694933e7c6247aec19b155763c96f72897c502

Mathematical statement. The real-valued interpretation of a certificate expression.

Arithmetic nodes are evaluated as the corresponding real operations.

```
def value : CertificateExpr → ℝ
| rational q => q
| atom x _ => x
| add a b => a.value + b.value
| neg a => -a.value
| mul a b => a.value * b.value
```

ScalarSuffixCertificate.CertificateExpr.value_neg_of_hi_neg

Classification: supplement

Source: ScalarSuffixCertificate.lean:201-205

Declaration SHA-256: db3beb0c3287dfb56bc535077265688e9bbbd719c9e044e825c88f873d5dd3ed

Mathematical statement. A negative computed upper endpoint certifies a negative real expression value.

Apply expression soundness and the interval sign-transfer theorem.

```
theorem value_neg_of_hi_neg (e : CertificateExpr) (h : e.enclosure.hi < 0) :
  e.value < 0 := QInterval.negative_of_realContains e.sound h

end CertificateExpr
```

ScalarSuffixCertificate.FirstPrototypeCell.B3

Classification: supplement

Source: ScalarSuffixCertificate.lean:506-506

Declaration SHA-256: fbc773e40d79a1d6d31822cdbc56d9a9607c6a7f8abc1ba0f47027df7b631eae

Mathematical statement. First-prototype complete type-(iii) angle expression.

$$B_3 = \lambda v + \pi - w.$$

```
def B3 (p : Point) : ℝ := lam p * v p + π - w p
```

ScalarSuffixCertificate.FirstPrototypeCell.B4

Classification: supplement

Source: ScalarSuffixCertificate.lean:503-503

Declaration SHA-256: 30e079ec002735e04de6acf4de75c1ef255bd47fc8db540544074fad80b3a6bc

Mathematical statement. First-prototype type-(iv) angle expression.

$$B_4 = \lambda y + \pi/2 - x.$$

```
def B4 (p : Point) : ℝ := lam p * y p + π / 2 - x p
```

ScalarSuffixCertificate.FirstPrototypeCell.C3_sigma_high_face_margin

Classification: supplement

Source: ScalarSuffixCertificate.lean:2949-2970

Declaration SHA-256: d14003647889eccbbfb10aaee4768200f899b458693a4ef6f074530360518a08

Mathematical statement. C3 is strictly positive on the high sigma face.

This completes the opposite incidence-face pair.

```
theorem C3_sigma_high_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 3 = radiiReal 3 →
    (1 / 1000000000 : ℝ) < C3 (pointOf t z) := by
intro t z ht hz hsigma
have hrho := hz 2
norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at hrho
have hsigma' : z 3 = (1 / 1000000000 : ℝ) := by
  norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three] at hsigma
  exact hsigma
have hzero := c3Reduced_sigma_zero_abs_lt ht hrho
have hgrowth := c3Reduced_sigma_growth ht hrho
  (sigma1 := 0) (sigma2 := (1 / 1000000000 : ℝ))
  (by constructor <=> norm_num) (by constructor <=> norm_num) (by norm_num)
rw [C3_pointOf_eq_c3Reduced, hsigma']
have hzeroLo := (abs_lt.mp hzero).1
norm_num at hgrowth hzeroLo ⊢
linarith

set_option maxHeartbeats 1000000 in
-- Six exact degree-27/26 point enclosures require extra normalization time.
```

ScalarSuffixCertificate.FirstPrototypeCell.C3_sigma_low_face_margin

Classification: supplement

Source: ScalarSuffixCertificate.lean:2927-2948

Declaration SHA-256: 33c17a00a6d5f6981f6c2016b520d75f7bdfba5da4ccd3f18d97796ac3d45fac

Mathematical statement. C3 is strictly negative on the low sigma face.

This is the lower type-(iii) incidence bound.

```
theorem C3_sigma_low_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 3 = -radiiReal 3 →
    C3 (pointOf t z) < -(1 / 1000000000 : ℝ) := by
intro t z ht hz hsigma
have hrho := hz 2
norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at hrho
have hsigma' : z 3 = -(1 / 1000000000 : ℝ) := by
  norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three] at hsigma
  exact hsigma
have hzero := c3Reduced_sigma_zero_abs_lt ht hrho
have hgrowth := c3Reduced_sigma_growth ht hrho
  (sigma1 := -(1 / 1000000000 : ℝ)) (sigma2 := 0)
  (by constructor <=> norm_num) (by constructor <=> norm_num) (by norm_num)
rw [C3_pointOf_eq_c3Reduced, hsigma']
have hzeroHi := (abs_lt.mp hzero).2
norm_num at hgrowth hzeroHi ⊢
linarith

/-- The upper `sigma` face has the required `C3` sign with the strict exact
rational margin `10-9`. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.C4_eta_high_face_margin

Classification: supplement

Source: ScalarSuffixCertificate.lean:2212-2232

Declaration SHA-256: 4dda50fdf98d96d0c6d5298b4b08eca53dc73079c6357bc4d6c0fad14a84b1e3

Mathematical statement. C4 is strictly positive on the high eta face.

The two eta faces bracket the type-(iv) incidence zero.

```
theorem C4_eta_high_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 1 = radiiReal 1 →
      (11 / 1000000000 : ℝ) < C4 (pointOf t z) := by
intro t z ht hz heta
have hxi := hz 0
norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at hxi
have heta' : z 1 = (1 / 100000000 : ℝ) := by
  norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three] at heta
  exact heta
have hzero := c4Reduced_eta_zero_abs_lt ht hxi
have hgrowth := c4Reduced_eta_growth ht hxi
  (eta1 := 0) (eta2 := (1 / 100000000 : ℝ))
  (by constructor <=> norm_num) (by constructor <=> norm_num) (by norm_num)
rw [C4_pointOf_eq_c4Reduced, heta']
have hzeroLo := (abs_lt.mp hzero).1
norm_num at hgrowth hzeroLo ⊢
linarith

/-- Derivative of the correlated `F` residual in the brick parameter. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.C4_eta_low_face_margin

Classification: supplement

Source: ScalarSuffixCertificate.lean:2190-2211

Declaration SHA-256: 9d228ca47508753791f14fd62b25ef0aa8eb7814e1715715afaf3fe27d1a441b

Mathematical statement. C4 is strictly negative on the first prototype's low eta face.

The rational margin is uniform in the remaining allowed variables.

```
theorem C4_eta_low_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 1 = -radiiReal 1 →
      C4 (pointOf t z) < -(11 / 1000000000 : ℝ) := by
intro t z ht hz heta
have hxi := hz 0
norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at hxi
have heta' : z 1 = -(1 / 100000000 : ℝ) := by
  norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three] at heta
  exact heta
have hzero := c4Reduced_eta_zero_abs_lt ht hxi
have hgrowth := c4Reduced_eta_growth ht hxi
  (eta1 := -(1 / 100000000 : ℝ)) (eta2 := 0)
  (by constructor <=> norm_num) (by constructor <=> norm_num) (by norm_num)
rw [C4_pointOf_eq_c4Reduced, heta']
have hzeroHi := (abs_lt.mp hzero).2
norm_num at hgrowth hzeroHi ⊢
linarith

/-- The upper `eta` face has the required `C4` sign with the strict exact
rational margin `11/10 · 10-9`. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.D3

Classification: supplement

Source: ScalarSuffixCertificate.lean:507-507

Declaration SHA-256: 4496291242cc1a371596577f5bdbec61dd292b63c92e9b87cf551b140d32a368

Mathematical statement. First-prototype type-(iii) sine displacement.

$$D_3 = \sin v - \sin w.$$

```
def D3 (p : Point) : ℝ := sin (v p) - sin (w p)
```

ScalarSuffixCertificate.FirstPrototypeCell.D4

Classification: supplement

Source: ScalarSuffixCertificate.lean:504-504

Declaration SHA-256: 458c17f197940395eb065d606c56a29538dee5cfc382d507f31979fa9b14e3b0

Mathematical statement. First-prototype type-(iv) sine displacement.

$$D_4 = \sin y - \sin x.$$

```
def D4 (p : Point) : ℝ := sin (y p) - sin (x p)
```

ScalarSuffixCertificate.FirstPrototypeCell.E_rho_high_face_margin

Classification: supplement

Source: ScalarSuffixCertificate.lean:3219-3243

Declaration SHA-256: a325916020527c8ce3fd863cee68df30e9b7de45125ec6369905d4e21da5a392

Mathematical statement. E is strictly positive on the high rho face.

Together the rho-face signs bracket the equal-area solution.

```
theorem E_rho_high_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 2 = radiiReal 2 →
    (14 / 1000000000 : ℝ) < E (pointOf t z) := by
  intro t z ht hz hrhoFace
  have hxi := hz 0
  have heta := hz 1
  have hsigma := hz 3
  norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
    Matrix.cons_val_three] at hxi heta hsigma
  have hrho : z 2 = (1 / 100000000 : ℝ) := by
    norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three]
    at hrhoFace
    exact hrhoFace
  have hzero := eReduced_rho_zero_abs_lt ht hxi heta hsigma
  have hgrowth := eReduced_rho_growth ht hxi heta hsigma
    (rho₁ := 0) (rho₂ := (1 / 100000000 : ℝ))
    (by constructor <=> norm_num) (by constructor <=> norm_num) (by norm_num)
  rw [E_pointOf_eq_eReduced, hrho]
  have hzeroLo := (abs_lt.mp hzero).1
  norm_num at hgrowth hzeroLo ⊢
  linarith

/-- Kernel-checked bundle of all eight opposite-face signs used by the
root-existence argument. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.E_rho_low_face_margin

Classification: supplement

Source: ScalarSuffixCertificate.lean:3196-3218

Declaration SHA-256: fbca51390968494bba3f952ecf1e30c03e3575007068f814db8a31c612e35640

Mathematical statement. E is strictly negative on the low rho face.

The bound is for the cleared equal-area residual over all other allowed coordinates.

```
theorem E_rho_low_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 2 = -radiiReal 2 →
      E (pointOf t z) < -(14 / 1000000000 : ℝ) := by
intro t z ht hz hrhoFace
have hxi := hz 0
have heta := hz 1
have hsigma := hz 3
norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at hxi heta hsigma
have hrho : z 2 = -(1 / 100000000 : ℝ) := by
  norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three]
  at hrhoFace
exact hrhoFace
have hzero := eReduced_rho_zero_abs_lt ht hxi heta hsigma
have hgrowth := eReduced_rho_growth ht hxi heta hsigma
  (rho₁ := -(1 / 100000000 : ℝ)) (rho₂ := 0)
  (by constructor <=> norm_num) (by constructor <=> norm_num) (by norm_num)
rw [E_pointOf_eq_eReduced, hrho]
have hzeroHi := (abs_lt.mp hzero).2
norm_num at hgrowth hzeroHi ⊢
linarith
```

ScalarSuffixCertificate.FirstPrototypeCell.F_xi_high_face_margin

Classification: supplement

Source: ScalarSuffixCertificate.lean:2639-2663

Declaration SHA-256: ab944f08473f17ea4d7ed3fe1083b1fc50813a95437c8cb991f719dd9c66f87b

Mathematical statement. F is strictly negative on the high xi face.

The root-map stationary component has positive high-face sign.

```
theorem F_xi_high_face_margin :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    z 0 = radiiReal 0 →
      F (pointOf t z) < -(86 / 1000000000 : ℝ) := by
intro t z ht hz hxiFace
have heta := hz 1
norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
  Matrix.cons_val_three] at heta
have hxi : z 0 = (1 / 100000000 : ℝ) := by
  norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three]
  at hxiFace
exact hxiFace
have hdev := fReduced_face_deviation ht
  (xi := (1 / 100000000 : ℝ)) (eta := z 1)
  (by constructor <=> norm_num) heta
have hdevHi := (abs_le.mp hdev).2
rw [F_pointOf_eq_fReduced, hxi]
have hanchor := fReduced_high_anchor
norm_num at hdevHi hanchor ⊢
linarith
```

```
-- At `sigma = 0`, the correlated `C3` residual stays within
```

```
`17/5 · 10-9`. The shared `t` and `rho` dependence of both cosine arguments  
is retained through one-dimensional mean-value estimates. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.F_xi_low_face_margin

Classification: supplement

Source: ScalarSuffixCertificate.lean:2616-2638

Declaration SHA-256: 3a89e7a208c35818420fe325762c3418f78b64fba4a4d38be2bc20f08e1bb47d

Mathematical statement. F is strictly positive on the low ξ face.

The corresponding component $-F$ therefore has negative lower-face sign.

```
theorem F_xi_low_face_margin :  
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →  
    z 0 = -radiiReal 0 →  
    (88 / 100000000 : ℝ) < F (pointOf t z) := by  
  intro t z ht hz hxiFace  
  have heta := hz 1  
  norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,  
    Matrix.cons_val_three] at heta  
  have hxi : z 0 = -(1 / 10000000 : ℝ) := by  
    norm_num [radiiReal, brick, Matrix.cons_val_two, Matrix.cons_val_three]  
    at hxiFace  
  exact hxiFace  
  have hdev := fReduced_face_deviation ht  
    (xi := -(1 / 10000000 : ℝ)) (eta := z 1)  
    (by constructor <=> norm_num) heta  
  have hdevLo := (abs_le.mp hdev).1  
  rw [F_pointOf_eq_fReduced, hxi]  
  have hanchor := fReduced_low_anchor  
  norm_num at hdevLo hanchor ⊢  
  linarith  
  
/-- On the upper `xi` face, `F` is uniformly negative with exact strict  
margin `86 · 10-9` over the complete first brick. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.N3

Classification: supplement

Source: ScalarSuffixCertificate.lean:508-509

Declaration SHA-256: 79fd22b0e88fadf424caa5c32cd0621ba146426141946780c13923b4adb4108c

Mathematical statement. First-prototype type-(iii) area numerator.

$$N_3 = B_3 + (\cos w + 2)D_3.$$

```
def N3 (p : Point) : ℝ := B3 p + (cos (w p) + 2) * D3 p
```

ScalarSuffixCertificate.FirstPrototypeCell.N4

Classification: supplement

Source: ScalarSuffixCertificate.lean:505-505

Declaration SHA-256: 8317292c5d87208707548fa6a07d20553105db710dba94daee68266201ce16ef

Mathematical statement. First-prototype type-(iv) area numerator.

$$N_4 = B_4 + \cos x D_4.$$

```
def N4 (p : Point) : ℝ := B4 p + cos (x p) * D4 p
```

ScalarSuffixCertificate.FirstPrototypeCell.Point

Classification: supplement

Source: ScalarSuffixCertificate.lean:488-494

Declaration SHA-256: faab379d3c93e3fdbbeaacb3e4e673d5518c5a8dbdfc4b2177da8137b5ec11ed2

Mathematical statement. The first prototype's density displacement and four angular error coordinates.

A point is $(t, \xi, \eta, \rho, \sigma)$ in the recorded real coordinate space.

```
structure Point where
  t : ℝ
  xi : ℝ
  eta : ℝ
  rho : ℝ
  sigma : ℝ
```

ScalarSuffixCertificate.FirstPrototypeCell.affine_branches

Classification: supplement

Source: ScalarSuffixCertificate.lean:1590-1624

Declaration SHA-256: e4d28d57906a58d6905767ae59f9a6cca32fef94fecf64598fa3b381471d528f

Mathematical statement. The first prototype's four angles stay in $(0, \pi/4)$ over its entire box.

Exact affine bounds validate all inverse-trigonometric branch interpretations.

```
theorem affine_branches :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    let p := pointOf t z
    0 < x p ^ x p < π / 4 ∧ 0 < y p ^ y p < π / 4 ∧
    0 < w p ^ w p < π / 4 ∧ 0 < v p ^ v p < π / 4 := by
  intro t z ht hz
  have hz0 := hz 0
  have hz1 := hz 1
  have hz2 := hz 2
  have hz3 := hz 3
  norm_num [tInterval, tRadius, brick,
    LeanSuffixReflective.QInterval.RealContains] at ht
  norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
    Matrix.cons_val_three] at hz0 hz1 hz2 hz3
  dsimp only
  norm_num [pointOf, x, y, w, v, brick, Matrix.cons_val_two,
    Matrix.cons_val_three]
  constructor
  · linarith
  constructor
  · linarith [Real.pi_gt_d6]
  constructor
  · linarith
  constructor
  · linarith [Real.pi_gt_d6]
  constructor
  · linarith
  constructor
  · linarith [Real.pi_gt_d6]
  constructor
  · linarith
  constructor
  · linarith [Real.pi_gt_d6]
```

```
/-- Exact affine box arithmetic separates the type-(iv) and type-(iii) angle
pairs throughout the first prototype brick. -/
```


ScalarSuffixCertificate.FirstPrototypeCell.analyticFaceBounds

Classification: supplement

Source: ScalarSuffixCertificate.lean:3250-3280

Declaration SHA-256: f2c7f2a169ba9c55d2865d26a4898b9b5bc1410e5acd882f9326aa435e600409

Mathematical statement. The first prototype satisfies its analytic face-bound interface.

The eight exact face-margin theorems are assembled with the correct stationary-component sign reversal.

```
theorem analyticFaceBounds : AnalyticFaceBounds := by
  constructor
  · intro t z ht hz j hj
    fin_cases j
    · have h := F_xi_low_face_margin t z ht hz hj
      norm_num [rootMap] at h
      linarith
    · have h := C4_eta_low_face_margin t z ht hz hj
      norm_num [rootMap] at h
      linarith
    · have h := E_rho_low_face_margin t z ht hz hj
      norm_num [rootMap] at h
      linarith
    · have h := C3_sigma_low_face_margin t z ht hz hj
      norm_num [rootMap] at h
      linarith
  · intro t z ht hz j hj
    fin_cases j
    · have h := F_xi_high_face_margin t z ht hz hj
      norm_num [rootMap] at h
      linarith
    · have h := C4_eta_high_face_margin t z ht hz hj
      norm_num [rootMap] at h
      linarith
    · have h := E_rho_high_face_margin t z ht hz hj
      norm_num [rootMap] at h
      linarith
    · have h := C3_sigma_high_face_margin t z ht hz hj
      norm_num [rootMap] at h
      linarith
```

ScalarSuffixCertificate.FirstPrototypeCell.brick

Classification: supplement

Source: ScalarSuffixCertificate.lean:437-453

Declaration SHA-256: 688841e317d3df26432b03f8a47b552ed700304ce27b849b8c43cf708ddb321b

Mathematical statement. The first prototype's fixed exact rational brick.

Its density interval begins at 51/50 and ends at 102001/100000.

```
def brick : Brick where
  lamLo := 51 / 50
  lamHi := 102001 / 100000
  centers := ![925629104183851933 / 5000000000000000000,
    2705068892345424047 / 10000000000000000000,
    1046818865074353649 / 25000000000000000000,
    4608185511215091997 / 10000000000000000000]
  slopes := ![439944824343995293 / 2000000000000000000,
    2510476247114540243 / 5000000000000000000,
    6186129361874951931 / 10000000000000000000,
    3760062975752449919 / 5000000000000000000]
  yXiSlope := 67 / 100
  vRhoSlope := 112 / 125
  radii := ![1 / 10000000, 1 / 100000000, 1 / 10000000, 1 / 100000000]
```

```

lamOrdered := by norm_num
radiiPos := by intro j; fin_cases j <=> norm_num

```

ScalarSuffixCertificate.FirstPrototypeCell.descendingK3

Classification: supplement

Source: ScalarSuffixCertificate.lean:1646-1814

Declaration SHA-256: 77707dff8f9a285fbbcbce0fee4560bb9495981e335a6a037b5c4be7daa5c9da3

Mathematical statement. The first prototype's K3 is uniformly strictly negative.

Its exact upper enclosure lies below zero on the closed density/error box.

```

theorem descendingK3 :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    K3 (pointOf t z) < 0 := by
  intro t z ht hz
  let p := pointOf t z
  have hz0 := hz 0
  have hz2 := hz 2
  have hz3 := hz 3
  norm_num [tInterval, tRadius, brick,
    LeanSuffixReflective.QInterval.RealContains] at ht
  norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
    Matrix.cons_val_three] at hz0 hz2 hz3
  have hlamLo : (1 : ℝ) < lam p := by
    dsimp [p, pointOf, lam]
    norm_num [lamMid, brick]
    linarith
  have hlamHi : lam p < (103 / 100 : ℝ) := by
    dsimp [p, pointOf, lam]
    norm_num [lamMid, brick]
    linarith
  have hwLo : (2 / 5 : ℝ) ≤ w p := by
    dsimp [p, pointOf, w]
    norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
    linarith
  have hwHi : w p ≤ (21 / 50 : ℝ) := by
    dsimp [p, pointOf, w]
    norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
    linarith
  have hvLo : (9 / 20 : ℝ) ≤ v p := by
    dsimp [p, pointOf, v]
    norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
    linarith
  have hvHi : v p ≤ (47 / 100 : ℝ) := by
    dsimp [p, pointOf, v]
    norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
    linarith
  have hwI :
    ((2 / 5, 21 / 50, by norm_num) : QInterval).RealContains (w p) := by
    constructor <=> norm_num <=> assumption
  have hvI :
    ((9 / 20, 47 / 100, by norm_num) : QInterval).RealContains (v p) := by
    constructor <=> norm_num <=> assumption
  have hsw :
    ((19 / 50, 21 / 50, by norm_num) : QInterval).RealContains
      (sin (w p)) := by
    apply realContains_sin_interval hwI
    · constructor <=> linarith [Real.pi_gt_d6]
    · have h := (sinTaylor27Interval_sound (2 / 5)).1
      norm_num [sinTaylor27Interval, sinTaylor27, Finset.sum_range_succ,
        Nat.factorial] at h
      norm_num
      linarith
    · have h := (sinTaylor27Interval_sound (21 / 50)).2
      norm_num [sinTaylor27Interval, sinTaylor27, Finset.sum_range_succ,
        Nat.factorial] at h
      norm_num

```

```

linarith
have hsv :
  ((2 / 5, 47 / 100, by norm_num) : QInterval).RealContains
    (sin (v p)) := by
  apply realContains_sin_interval hvI
  · constructor <=> linarith [Real.pi_gt_d6]
  · have h := (sinTaylor27Interval_sound (9 / 20)).1
    norm_num [sinTaylor27Interval, sinTaylor27, Finset.sum_range_succ,
      Nat.factorial] at h
    norm_num
    linarith
  · have h := (sinTaylor27Interval_sound (47 / 100)).2
    norm_num [sinTaylor27Interval, sinTaylor27, Finset.sum_range_succ,
      Nat.factorial] at h
    norm_num
    linarith
have hsvHi : sin (v p) ≤ (47 / 100 : ℝ) := by
  norm_num
  nlinarith [hsv.2]
have hcw :
  ((9 / 10, 93 / 100, by norm_num) : QInterval).RealContains
    (cos (w p)) := by
  apply realContains_cos_interval hwI
  · constructor <=> linarith [Real.pi_gt_d6]
  · have h := (cosTaylor26Interval_sound (21 / 50)).1
    norm_num [cosTaylor26Interval, cosTaylor26, Finset.sum_range_succ,
      Nat.factorial] at h
    norm_num
    linarith
  · have h := (cosTaylor26Interval_sound (2 / 5)).2
    norm_num [cosTaylor26Interval, cosTaylor26, Finset.sum_range_succ,
      Nat.factorial] at h
    norm_num
    linarith
have hswPos : 0 < sin (w p) := lt_of_lt_of_le (by norm_num) hsw.1
have hsvPos : 0 < sin (v p) := lt_of_lt_of_le (by norm_num) hsv.1
have hlamPos : 0 < lam p := lt_trans (by norm_num) hlamLo
have hlamSqPos : 0 < lam p ^ 2 := sq_pos_of_pos hlamPos
have hA : (1 + cos (w p)) / sin (w p) < (51 / 10 : ℝ) := by
  apply (div_lt_iff₀ hswPos).2
  nlinarith [hcw.2, hsw.1]
have hlamSqHi : lam p ^ 2 < (10609 / 10000 : ℝ) := by
  nlinarith
have hdenHi :
  lam p * (lam p * sin (v p)) < (498623 / 1000000 : ℝ) := by
  rw [show lam p * (lam p * sin (v p)) =
    lam p ^ 2 * sin (v p) by ring]
  have h₁ :
    lam p ^ 2 * sin (v p) ≤ lam p ^ 2 * (47 / 100 : ℝ) :=
    mul_le_mul_of_nonneg_left hsvHi hlamSqPos.le
  have h₂ :
    lam p ^ 2 * (47 / 100 : ℝ) <
      (10609 / 10000 : ℝ) * (47 / 100 : ℝ) :=
    mul_lt_mul_of_pos_right hlamSqHi (by norm_num)
  norm_num at h₂ ⊢
  exact h₁.trans_lt h₂
have hdenPos : 0 < lam p * (lam p * sin (v p)) := by positivity
have hnumLo : (19 / 10 : ℝ) < lam p ^ 2 + cos (w p) := by
  nlinarith [hcw.1]
have hB : (19 / 5 : ℝ) <
  (lam p ^ 2 + cos (w p)) /
  (lam p * (lam p * sin (v p))) := by
  apply (lt_div_iff₀ hdenPos).2
  nlinarith [hnumLo, hdenHi]
have hdiff :
  (1 + cos (w p)) / sin (w p) -
  (lam p ^ 2 + cos (w p)) /
  (lam p * (lam p * sin (v p))) <
  (13 / 10 : ℝ) := by
  linarith
have hfactorPos : 0 < 2 * (1 + cos (w p)) := by
  nlinarith [hcw.1]

```

```

have hfactorHi : 2 * (1 + cos (w p)) ≤ (193 / 50 : ℝ) := by
  nlinarith [hcv.2]
have hproduct :
  2 * (1 + cos (w p)) *
    ((1 + cos (w p)) / sin (w p) -
      (lam p ^ 2 + cos (w p)) /
        (lam p * (lam p * sin (v p)))) <
    (5018 / 1000 : ℝ) := by
  by_cases hd :
    0 ≤ (1 + cos (w p)) / sin (w p) -
      (lam p ^ 2 + cos (w p)) /
        (lam p * (lam p * sin (v p)))
  · nlinarith [mul_lt_mul_of_pos_left hdiff hfactorPos,
    mul_le_mul_of_nonneg_right hfactorHi hd]
  · have hd' := lt_of_not_ge hd
    have hnegative := mul_neg_of_pos_of_neg hfactorPos hd'
    norm_num at hnegative ⊢
    linarith
have hlamv : (9 / 20 : ℝ) < lam p * v p := by
  have hvPos : 0 < v p := lt_of_lt_of_le (by norm_num) hvLo
  nlinarith [mul_lt_mul_of_pos_right hlamLo hvPos]
have hbase : (31 / 10 : ℝ) < B3 p + D3 p := by
  unfold B3 D3
  nlinarith [hlamv, Real.pi_gt_d6, hwHi, hsv.1, hsw.2]
change K3 p < 0
unfold K3
dsimp only
rw [show
  4 * ((1 + cos (w p)) / 2) *
    ((1 + cos (w p)) / sin (w p) -
      (lam p ^ 2 + cos (w p)) /
        (lam p * (lam p * sin (v p)))) =
  2 * (1 + cos (w p)) *
    ((1 + cos (w p)) / sin (w p) -
      (lam p ^ 2 + cos (w p)) /
        (lam p * (lam p * sin (v p)))) by ring]
linarith

/-- A direct rational interval decomposition proves the scalar perimeter gap
throughout the first prototype brick. The explicit negative margin keeps this
leaf independent of the root-existence argument. -/

```

ScalarSuffixCertificate.FirstPrototypeCell.gap_upper

Classification: supplement

Source: ScalarSuffixCertificate.lean:1815-1930

Declaration SHA-256: bfd0e938dc561a18e65fa20cc38b1be820b9d6c633f0485b08fc114b41463547

Mathematical statement. The first prototype's perimeter residual G has a negative upper bound.

The strict sign survives selection of any equation root inside the certified box.

```

theorem gap_upper :
  ∀ t z, tInterval.RealContains t → InSymmetricBox radiiReal z →
    G (pointOf t z) < -(437 / 1000000 : ℝ) := by
  intro t z ht hz
  let p := pointOf t z
  have hz0 := hz 0
  have hz1 := hz 1
  have hz2 := hz 2
  have hz3 := hz 3
  norm_num [tInterval, tRadius, brick,
    LeanSuffixReflective.QInterval.RealContains] at ht
  norm_num [InSymmetricBox, radiiReal, brick, Matrix.cons_val_two,
    Matrix.cons_val_three] at hz0 hz1 hz2 hz3
  have hlamLo : (51 / 50 : ℝ) ≤ lam p := by
    dsimp [p, pointOf, lam]
    norm_num [lamMid, brick]
    linarith

```

```

have hlamHi : lam p ≤ (102001 / 100000 : ℝ) := by
  dsimp [p, pointOf, lam]
  norm_num [lamMid, brick]
  linarith
have hxLo : (1851 / 10000 : ℝ) ≤ x p := by
  dsimp [p, pointOf, x]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hxHi : x p ≤ (1852 / 10000 : ℝ) := by
  dsimp [p, pointOf, x]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hyLo : (2704 / 10000 : ℝ) ≤ y p := by
  dsimp [p, pointOf, y]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hwLo : (4186 / 10000 : ℝ) ≤ w p := by
  dsimp [p, pointOf, w]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hwHi : w p ≤ (4188 / 10000 : ℝ) := by
  dsimp [p, pointOf, w]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hvLo : (4607 / 10000 : ℝ) ≤ v p := by
  dsimp [p, pointOf, v]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hvHi : v p ≤ (4609 / 10000 : ℝ) := by
  dsimp [p, pointOf, v]
  norm_num [brick, Matrix.cons_val_two, Matrix.cons_val_three]
  linarith
have hcxHi : cos (x p) ≤ (983 / 1000 : ℝ) := by
  have hmono : cos (x p) ≤ cos (1851 / 10000 : ℝ) :=
    Real.cos_le_cos_of_nonneg_of_le_pi (by norm_num)
    (by linarith [Real.pi_gt_d6]) hxLo
  have h := (cosTaylor26Interval_sound (1851 / 10000)).2
  norm_num [cosTaylor26Interval, cosTaylor26, Finset.sum_range_succ,
    Nat.factorial] at h
  linarith
have hcwLo : (9135 / 10000 : ℝ) ≤ cos (w p) := by
  have hmono : cos (4188 / 10000 : ℝ) ≤ cos (w p) :=
    Real.cos_le_cos_of_nonneg_of_le_pi (by linarith)
    (by linarith [Real.pi_gt_d6]) hwHi
  have h := (cosTaylor26Interval_sound (4188 / 10000)).1
  norm_num [cosTaylor26Interval, cosTaylor26, Finset.sum_range_succ,
    Nat.factorial] at h
  linarith
have hsvHi : sin (v p) ≤ (4449 / 10000 : ℝ) := by
  have hmono : sin (v p) ≤ sin (4609 / 10000 : ℝ) :=
    Real.sin_le_sin_of_le_of_le_pi_div_two (by linarith [Real.pi_pos])
    (by linarith [Real.pi_gt_d6]) hvHi
  have h := (sinTaylor27Interval_sound (4609 / 10000)).2
  norm_num [sinTaylor27Interval, sinTaylor27, Finset.sum_range_succ,
    Nat.factorial] at h
  linarith
have hswLo : (4064 / 10000 : ℝ) ≤ sin (w p) := by
  have hmono : sin (4186 / 10000 : ℝ) ≤ sin (w p) :=
    Real.sin_le_sin_of_le_of_le_pi_div_two (by linarith [Real.pi_pos])
    (by linarith [Real.pi_gt_d6]) hwLo
  have h := (sinTaylor27Interval_sound (4186 / 10000)).1
  norm_num [sinTaylor27Interval, sinTaylor27, Finset.sum_range_succ,
    Nat.factorial] at h
  linarith
have hlamvHi : lam p * v p ≤
  (102001 / 100000 : ℝ) * (4609 / 10000 : ℝ) := by
  exact mul_le_mul hlamHi hvHi (by linarith) (by norm_num)
have hlamyLo : (51 / 50 : ℝ) * (2704 / 10000 : ℝ) ≤
  lam p * y p := by
  exact mul_le_mul hlamLo hyLo (by norm_num) (by linarith)
have hAHi : B3 p + D3 p < (3232 / 1000 : ℝ) := by
  unfold B3 D3

```

```

linarith [Real.pi_lt_d6]
have hALo : (0 : ℝ) < B3 p + D3 p := by
  unfold B3 D3
  have hsinv : -(1 : ℝ) ≤ sin (v p) := neg_one_le_sin _
  have hsinw : sin (w p) ≤ 1 := sin_le_one _
  nlinarith [Real.pi_gt_d6]
have hBLo : (1661 / 1000 : ℝ) < B4 p := by
  unfold B4
  linarith [Real.pi_gt_d6]
have hfirst : cos (x p) * (B3 p + D3 p) <
  (983 / 1000 : ℝ) * (3232 / 1000 : ℝ) := by
  have hcXPos : 0 < cos (x p) :=
    Real.cos_pos_of_mem_Ioo ⟨by linarith [Real.pi_pos],
      by linarith [Real.pi_gt_d6]⟩
  have hmul := mul_lt_mul hAHi hcXHi hcXPos (by norm_num)
  nlinarith
have hsecond : (1913 / 1000 : ℝ) * (1661 / 1000 : ℝ) <
  (1 + cos (w p)) * B4 p := by
  have hf : (1913 / 1000 : ℝ) < 1 + cos (w p) := by linarith
  exact mul_lt_mul hf hBLo.le (by norm_num) (by linarith)
unfold G
nlinarith [hfirst, hsecond]

```

/-- At `eta = 0`, the correlated `C4` residual stays within ` $31/20 \cdot 10^{-9}$ `.
 The `t` and `xi` variations are bounded by mean value estimates rather than
 decorrelating the two cosine arguments. -/

ScalarSuffixCertificate.FirstPrototypeCell.lam

Classification: supplement

Source: ScalarSuffixCertificate.lean:495-495

Declaration SHA-256: 80d4ea0b007cc38c9cef09318804f92bb1d2031dffde45077e78cad062961da3

Mathematical statement. The reconstructed first-prototype density.

$\lambda = \lambda_{\text{mid}} + t$.

```
def lam (p : Point) : ℝ := (lamMid : ℝ) + p.t
```

ScalarSuffixCertificate.FirstPrototypeCell.lamInterval

Classification: supplement

Source: ScalarSuffixCertificate.lean:458-459

Declaration SHA-256: f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6

Mathematical statement. The first prototype's closed density interval.

It is $[51/50, 102001/100000]$.

```
def lamInterval : QInterval := ⟨brick.lamLo, brick.lamHi, brick.lamOrdered.le⟩
```

ScalarSuffixCertificate.FirstPrototypeCell.lamMid

Classification: supplement

Source: ScalarSuffixCertificate.lean:454-454

Declaration SHA-256: 6872199d838a1fa3e543ff329eb7ed7979df53e9ab7df478a48be79baedaf0cf

Mathematical statement. The first prototype's exact density midpoint.

It is the average of the two rational endpoints.

```
abbrev lamMid : ℝ := (brick.lamLo + brick.lamHi) / 2
```

ScalarSuffixCertificate.FirstPrototypeCell.lambda_iff_centered

Classification: supplement

Source: ScalarSuffixCertificate.lean:460-472

Declaration SHA-256: 80585faf74c765a727b4d57aad40e1258e6fc487898dae74ceaba8d89c1df2a3

Mathematical statement. The first prototype's density bounds are equivalent to its centered parameter bounds.

The change of variables is $t = \lambda - \lambda_{\text{mid}}$.

```
theorem lambda_iff_centered {lam : ℝ} :
  lamInterval.RealContains lam ↔ tInterval.RealContains (lam - (lamMid : ℝ)) := by
  constructor
  · rintro ⟨h₁, h₂⟩
    constructor <=>
      norm_num [lamInterval, tInterval, lamMid, tRadius, brick,
        LeanSuffixReflective.QInterval.RealContains] at h₁ h₂ <=> linarith
  · rintro ⟨h₁, h₂⟩
    constructor <=>
      norm_num [lamInterval, tInterval, lamMid, tRadius, brick,
        LeanSuffixReflective.QInterval.RealContains] at h₁ h₂ <=> linarith

/-- Unconditional real-range conclusion for the representative cell. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.lambda_in_retainedSuffix

Classification: supplement

Source: ScalarSuffixCertificate.lean:473-479

Declaration SHA-256: 7d841cce60699559819b45f6ee022939aef986b15a3732746d752fc33d94e2fa

Mathematical statement. Every first-prototype density lies in the retained suffix interval.

The exact endpoints are contained in $[51/50, 9/7]$.

```
theorem lambda_in_retainedSuffix {lam : ℝ} (h : lamInterval.RealContains lam) :
  LeanSuffixAnalytic.InRetainedSuffix lam := by
  unfold LeanSuffixAnalytic.InRetainedSuffix
  norm_num [lamInterval, LeanSuffixReflective.QInterval.RealContains, brick] at h <=>
  exact ⟨h.1, h.2.trans (by norm_num)⟩

/-- The whole exact-data cell lies in the retained real suffix. -/
```

ScalarSuffixCertificate.FirstPrototypeCell.pointOf

Classification: supplement

Source: ScalarSuffixCertificate.lean:801-802

Declaration SHA-256: 123eaac2727784b857a8a1d07bab6c6e08557c023e8c73b6b961970da83630664

Mathematical statement. Build a first-prototype point from centered density and an error vector.

It reverses the extraction of the four error coordinates at fixed density.

```
def pointOf (t : ℝ) (z : Vec4) : Point := ⟨t, z 0, z 1, z 2, z 3⟩
```

ScalarSuffixCertificate.FirstPrototypeCell.radiiReal

Classification: supplement

Source: ScalarSuffixCertificate.lean:800-800

Declaration SHA-256: ffbda00bb8ae49e94f67d9ad5dd16012ff60af58d9c1760a6abfb99b1bb96909

Mathematical statement. The first prototype's rational error radii cast to reals.

They define its symmetric root box.

```
def radiiReal : Vec4 := fun j => (brick.radii j : ℝ)
```

ScalarSuffixCertificate.FirstPrototypeCell.tInterval

Classification: supplement

Source: ScalarSuffixCertificate.lean:457-457

Declaration SHA-256: 4051c1639ae05eb9e136816b4912782037642807485ecc541b8a4729aa41a839

Mathematical statement. The first prototype's centered density interval.

The endpoints are the negative and positive density radius.

```
def tInterval : QInterval := ⟨-tRadius, tRadius, by norm_num [tRadius, brick]⟩
```

ScalarSuffixCertificate.FirstPrototypeCell.tRadius

Classification: supplement

Source: ScalarSuffixCertificate.lean:455-456

Declaration SHA-256: 7ec2e049acded7f92cfecfef90c8523058c254046f34ddfc61af74c906f331c0

Mathematical statement. Half the first prototype's density width.

It determines the symmetric centered density interval.

```
abbrev tRadius : ℚ := (brick.lamHi - brick.lamLo) / 2
```

ScalarSuffixCertificate.FirstPrototypeCell.v

Classification: supplement

Source: ScalarSuffixCertificate.lean:500-502

Declaration SHA-256: 7b17efff67b8a9b22200a9061ba58fb57a901753e3d49bff62bf8ea7f6d2a8ea

Mathematical statement. The first type-(iii) exterior-angle predictor with rho coupling.

Its remaining independent error is sigma.

```
def v (p : Point) : ℝ := brick.centers 3 + brick.slopes 3 * p.t +  
  brick.vRhoSlope * p.rho + p.sigma
```

ScalarSuffixCertificate.FirstPrototypeCell.vars

Classification: supplement

Source: ScalarSuffixCertificate.lean:799-799

Declaration SHA-256: d389f5673106602bb401f76b3d594d64ec6f00b7fcff209bdfc571267bc953d5

Mathematical statement. Extract the four angular errors from a first-prototype point.

The result is the vector $(\xi, \eta, \rho, \sigma)$.

```
def vars (p : Point) : Vec4 := ![p.xi, p.eta, p.rho, p.sigma]
```

ScalarSuffixCertificate.FirstPrototypeCell.w

Classification: supplement

Source: ScalarSuffixCertificate.lean:499-499

Declaration SHA-256: 52d839481355b169feb0cd07c321cec9f11b8bdc2b69f116735e48df02da7c3d

Mathematical statement. The first type-(iii) inner-angle predictor.

The affine density approximation is corrected by rho.

```
def w (p : Point) : ℝ := brick.centers 2 + brick.slopes 2 * p.t + p.rho
```

ScalarSuffixCertificate.FirstPrototypeCell.x

Classification: supplement

Source: ScalarSuffixCertificate.lean:496-496

Declaration SHA-256: 3f1552bfdd08e7586bffec5b6dc6b1a410691f11d8196031e1e7ca9cbf58889a

Mathematical statement. The first type-(iv) inner-angle predictor.

The affine density approximation is corrected by xi.

```
def x (p : Point) : ℝ := brick.centers 0 + brick.slopes 0 * p.t + p.xi
```

ScalarSuffixCertificate.FirstPrototypeCell.y

Classification: supplement

Source: ScalarSuffixCertificate.lean:497-498

Declaration SHA-256: 9e7642719b786de830ac8e4bdac8c25ca9f746737432d174122c9b0d847d92b9

Mathematical statement. The first type-(iv) exterior-angle predictor with xi coupling.

Its remaining independent error is eta.

```
def y (p : Point) : ℝ := brick.centers 1 + brick.slopes 1 * p.t +  
  brick.yXiSlope * p.xi + p.eta
```

ScalarSuffixCertificate.InSymmetricBox

Classification: supplement

Source: ScalarSuffixCertificate.lean:420-424

Declaration SHA-256: 5499419d9ab0a5d1357ae99f00c375e3edfdb9ed0f97f8657ce5edf639f1f5d0

Mathematical statement. Membership in the closed symmetric error box.

Each coordinate satisfies $-r_j \leq z_j \leq r_j$.

```
def InSymmetricBox (r z : Vec4) : Prop := ∀ j, -r j ≤ z j ∧ z j ≤ r j  
  
namespace FirstPrototypeCell
```

ScalarSuffixCertificate.QInterval

Classification: supplement

Source: ScalarSuffixCertificate.lean:24-29

Declaration SHA-256: 9fe7a736ba4c430250485562ba236783155c010cef8e4943fa7819a28f9aa406

Mathematical statement. The shared rational interval type used by scalar certificates.

It reuses the ordered rational endpoints and their real containment relation.

```
abbrev QInterval := LeanSuffixReflective.QInterval

namespace QInterval

/-- The tight interval product: its endpoints are the extrema of the four
corner products. -/
```

ScalarSuffixCertificate.QInterval.divPos

Classification: supplement

Source: ScalarSuffixCertificate.lean:50-53

Declaration SHA-256: 2642acecc562d12111d17e260928f80af5d34794a1e8e1d3abc20a33a085768a

Mathematical statement. Interval division by a strictly positive denominator interval.

Multiply the numerator interval by the positive reciprocal enclosure.

```
def divPos (i j : QInterval) (hj : 0 < j.lo) : QInterval :=
  i.mul (j.recipPos hj)

/-- Real soundness of tight exact-rational multiplication. -/
```

ScalarSuffixCertificate.QInterval.mul

Classification: supplement

Source: ScalarSuffixCertificate.lean:30-44

Declaration SHA-256: 0d7a9188e958ff3c0238d78f3d3a0728ab82af07abbb6a56e4a749efa852062a

Mathematical statement. Interval product using the minimum and maximum of four endpoint products.

This handles all endpoint signs without assuming nonnegativity.

```
def mul (i j : QInterval) : QInterval where
  lo := min (min (i.lo * j.lo) (i.lo * j.hi))
        (min (i.hi * j.lo) (i.hi * j.hi))
  hi := max (max (i.lo * j.lo) (i.lo * j.hi))
        (max (i.hi * j.lo) (i.hi * j.hi))
  ordered := by
    calc
      min (min (i.lo * j.lo) (i.lo * j.hi))
        (min (i.hi * j.lo) (i.hi * j.hi))
        ≤ i.lo * j.lo := (min_le_left _ _).trans (min_le_left _ _)
      _ ≤ max (max (i.lo * j.lo) (i.lo * j.hi))
        (max (i.hi * j.lo) (i.hi * j.hi)) :=
        (le_max_left _ _).trans (le_max_left _ _)

/-- Positive reciprocal; positivity is checked in exact rational arithmetic. -/
```

ScalarSuffixCertificate.QInterval.negative_of_realContains

Classification: supplement

Source: ScalarSuffixCertificate.lean:153-158

Declaration SHA-256: dd92d9ea0b74bda9075f7d8e3782bf5b435ea264bc0d9f911316c7d07b50aa26

Mathematical statement. A negative interval upper endpoint forces every enclosed value to be negative.

From $x \leq b < 0$, infer $x < 0$.

```
theorem negative_of_realContains {i : QInterval} {x : ℝ}
  (hx : i.RealContains x) (hi : i.hi < 0) : x < 0 := by
  have hiR : (i.hi : ℝ) < 0 := by exact_mod_cast hi
  exact hx.2.trans_lt hiR

/-- Strict sign transfer from an exact rational lower endpoint. -/
```

ScalarSuffixCertificate.QInterval.positive_of_realContains

Classification: supplement

Source: ScalarSuffixCertificate.lean:159-166

Declaration SHA-256: 4196ef90c8a23949f2eeda7be784e128a7fc085f82c49c1113a0dbdd35e4d445

Mathematical statement. A positive interval lower endpoint forces every enclosed value to be positive.

From $0 < a \leq x$, infer $0 < x$.

```
theorem positive_of_realContains {i : QInterval} {x : ℝ}
  (hx : i.RealContains x) (hi : 0 < i.lo) : 0 < x := by
  have hiR : 0 < (i.lo : ℝ) := by exact_mod_cast hi
  exact hiR.trans_le hx.1

end QInterval

/-- Expression language whose analytic atoms carry proved real enclosures. -/
```

ScalarSuffixCertificate.QInterval.realContains_centered_meanValue

Classification: supplement

Source: ScalarSuffixCertificate.lean:131-145

Declaration SHA-256: 0ed29af5eec0bee2a0714dc6eef9fd9127d30190d983af8320b3b10560c11496

Mathematical statement. Center and derivative intervals yield a sound enclosure by the mean value theorem.

The exact intermediate derivative value is enclosed once, preserving repeated-variable correlation.

```
theorem realContains_centered_meanValue
  (f f' : ℝ → ℝ) {c h : ℝ}
  (base slope displacement : QInterval)
  (hderiv : ∀ x ∈ Set.uIcc c h, HasDerivAt f (f' x) x)
  (hbase : base.RealContains (f c))
  (hslope : ∀ x ∈ Set.uIcc c h, slope.RealContains (f' x))
  (hdisplacement : displacement.RealContains (h - c)) :
  (base.add (slope.mul displacement)).RealContains (f h) := by
  rcases LeanSuffixAnalytic.centered_meanValue_exists f f' hderiv with
    (ξ, hξ, hmean)
  rw [hmean]
  exact LeanSuffixReflective.QInterval.realContains_add hbase
    (realContains_mul (hslope ξ hξ) hdisplacement)

/-- Rational square bounds imply a real square-root enclosure. -/
```

ScalarSuffixCertificate.QInterval.realContains_divPos

Classification: supplement

Source: ScalarSuffixCertificate.lean:120-130

Declaration SHA-256: f39233850c94c2277651049ffd4664a24cd1dbc0d60498abc5472f08a163d72f

Mathematical statement. Positive-denominator interval division is sound.

Combine sound reciprocal and multiplication enclosures.

```
theorem realContains_divPos {i j : QInterval} {x y : ℝ} (hj : 0 < j.lo)
  (hx : i.RealContains x) (hy : j.RealContains y) :
  (i.divPos j hj).RealContains (x / y) := by
  unfold divPos
  rw [div_eq_mul_one_div]
  exact realContains_mul hx (realContains_recipPos hj hy)

/-- Soundness kernel for centered mean-value interval evaluation. The output
enclosure is derived, not assumed: the mean value theorem supplies one shared
intermediate point, `slope` encloses the derivative expression at that point,
and ordinary interval addition and multiplication finish the check. -/
```

ScalarSuffixCertificate.QInterval.realContains_mul

Classification: supplement

Source: ScalarSuffixCertificate.lean:54-97

Declaration SHA-256: bcb27cfff58a492c7470bc52a00dd315717a6bfbcb49a464e152ee6b024b200dfd

Mathematical statement. The interval product encloses products of enclosed real values.

The four endpoint products bound the bilinear product over a rectangle.

```
theorem realContains_mul {i j : QInterval} {x y : ℝ}
  (hx : i.RealContains x) (hy : j.RealContains y) :
  (i.mul j).RealContains (x * y) := by
  have corner_lower (q : ℚ) :
    min ((q : ℝ) * (j.lo : ℝ)) ((q : ℝ) * (j.hi : ℝ)) ≤ (q : ℝ) * y := by
    by_cases hq : 0 ≤ q
    · have hqR : (0 : ℝ) ≤ (q : ℝ) := by exact_mod_cast hq
      exact (min_le_left _ _).trans (mul_le_mul_of_nonneg_left hy.1 hqR)
    · have hq' : (q : ℝ) ≤ 0 := by exact_mod_cast (le_of_not_ge hq)
      exact (min_le_right _ _).trans (mul_le_mul_of_nonpos_left hy.2 hq')
  have corner_upper (q : ℚ) :
    (q : ℝ) * y ≤ max ((q : ℝ) * (j.lo : ℝ)) ((q : ℝ) * (j.hi : ℝ)) := by
    by_cases hq : 0 ≤ q
    · have hqR : (0 : ℝ) ≤ (q : ℝ) := by exact_mod_cast hq
      exact (mul_le_mul_of_nonneg_left hy.2 hqR).trans (le_max_right _ _)
    · have hq' : (q : ℝ) ≤ 0 := by exact_mod_cast (le_of_not_ge hq)
      exact (mul_le_mul_of_nonpos_left hy.1 hq').trans (le_max_left _ _)
  have hlo :
    min
      (min ((i.lo : ℝ) * (j.lo : ℝ)) ((i.lo : ℝ) * (j.hi : ℝ)))
      (min ((i.hi : ℝ) * (j.lo : ℝ)) ((i.hi : ℝ) * (j.hi : ℝ)))
    ≤ x * y := by
    by_cases hy0 : 0 ≤ y
    · exact ((min_le_left _ _).trans (corner_lower i.lo)).trans
      (mul_le_mul_of_nonneg_right hx.1 hy0)
    · have hy0' : y ≤ 0 := le_of_not_ge hy0
      exact ((min_le_right _ _).trans (corner_lower i.hi)).trans
      (mul_le_mul_of_nonpos_right hx.2 hy0')
  have hhi :
    x * y ≤
      max
        (max ((i.lo : ℝ) * (j.lo : ℝ)) ((i.lo : ℝ) * (j.hi : ℝ)))
        (max ((i.hi : ℝ) * (j.lo : ℝ)) ((i.hi : ℝ) * (j.hi : ℝ))) := by
    by_cases hy0 : 0 ≤ y
```

```

· exact (mul_le_mul_of_nonneg_right hx.2 hy0).trans
  ((corner_upper i.hi).trans (le_max_right _ _))
· have hy0' : y ≤ 0 := le_of_not_ge hy0
  exact (mul_le_mul_of_nonpos_right hx.1 hy0').trans
    ((corner_upper i.lo).trans (le_max_left _ _))
constructor
· simpa [mul, LeanSuffixReflective.QInterval.RealContains] using hlo
· simpa [mul, LeanSuffixReflective.QInterval.RealContains] using hhi

/-- Real soundness of nonnegative rational scaling. -/

```

ScalarSuffixCertificate.QInterval.realContains_nsmul

Classification: supplement

Source: ScalarSuffixCertificate.lean:98-108

Declaration SHA-256: 230f7166aba334e53ba446c86429b3ff8b9f0576ec86363de14a61caa8a2624d

Mathematical statement. Scaling an enclosing rational interval by a nonnegative rational q encloses the real product qx , including $q = 0$ and $q = 1/2$.

Let $I = [a, b]$ have rational endpoints, let $x \in \mathbb{R}$ satisfy $(a : \mathbb{R}) \leq x \leq (b : \mathbb{R})$, and let $q \in \mathbb{Q}$ satisfy $0 \leq q$. Multiplying both inequalities by $(q : \mathbb{R}) \geq 0$ gives $(qa : \mathbb{R}) \leq (q : \mathbb{R})x \leq (qb : \mathbb{R})$. Hence the interval produced by rational scaling contains the exact real product. The theorem permits every nonnegative rational scalar, not only natural numbers; $q = 1/2$ certifies halving, and $q = 0$ certifies the zero product.

```

theorem realContains_nsmul {i : QInterval} {x : ℝ} {q : ℚ} (hq : 0 ≤ q)
  (hx : i.RealContains x) : (i.nsmul q hq).RealContains ((q : ℝ) * x) := by
  constructor
· simpa [LeanSuffixReflective.QInterval.nsmul,
  LeanSuffixReflective.QInterval.RealContains] using
  mul_le_mul_of_nonneg_left hx.1 (by exact_mod_cast hq)
· simpa [LeanSuffixReflective.QInterval.nsmul,
  LeanSuffixReflective.QInterval.RealContains] using
  mul_le_mul_of_nonneg_left hx.2 (by exact_mod_cast hq)

/-- Real soundness of a positive reciprocal certificate. -/

```

ScalarSuffixCertificate.QInterval.realContains_recipPos

Classification: supplement

Source: ScalarSuffixCertificate.lean:109-119

Declaration SHA-256: 6bebb0eb9a5daccfb0e032373488c963884df078d4bcae42d816f15559377e3f

Mathematical statement. Positive reciprocal intervals enclose reciprocals of enclosed reals.

Reciprocation reverses order on $(0, \infty)$.

```

theorem realContains_recipPos {i : QInterval} {x : ℝ} (hi : 0 < i.lo)
  (hx : i.RealContains x) : (i.recipPos hi).RealContains (1 / x) := by
  have hiR : 0 < (i.lo : ℝ) := by exact_mod_cast hi
  have hxpos : 0 < x := hiR.trans_le hx.1
  constructor
· simpa [recipPos, LeanSuffixReflective.QInterval.RealContains] using
  one_div_le_one_div_of_le hxpos hx.2
· simpa [recipPos, LeanSuffixReflective.QInterval.RealContains] using
  one_div_le_one_div_of_le hiR hx.1

/-- Real soundness of division by a positive interval. -/

```

ScalarSuffixCertificate.QInterval.realContains_sqrt

Classification: supplement

Source: ScalarSuffixCertificate.lean:146-152

Declaration SHA-256: 78d08ac2e456c76c8cc84e056f4a63de293d6015fbff06b8f125a80a09ec3dd5

Mathematical statement. Certified nonnegative square bounds enclose a real square root.

Monotonicity of squaring on nonnegative reals transfers endpoint square inequalities to the radical.

```
theorem realContains_sqrt {i : QInterval} {x : ℝ}
  (hlo : 0 ≤ i.lo) (hloSq : (i.lo : ℝ) ^ 2 ≤ x)
  (hhiSq : x ≤ (i.hi : ℝ) ^ 2) : i.RealContains (sqrt x) := by
  have hhi : 0 ≤ (i.hi : ℝ) := by exact_mod_cast (hlo.trans i.ordered)
  exact ⟨Real.le_sqrt_of_sq_le hloSq, Real.sqrt_le_iff.2 ⟨hhi, hhiSq⟩⟩

/-- Strict sign transfer from an exact rational upper endpoint. -/
```

ScalarSuffixCertificate.QInterval.recipPos

Classification: supplement

Source: ScalarSuffixCertificate.lean:45-49

Declaration SHA-256: 90ef275967dd2ae91683ae88e032a660973bb176bd69a1f9f83086d63e1c2bed

Mathematical statement. Reciprocal interval for a strictly positive interval.

For $0 < a \leq b$, the result is $[1/b, 1/a]$.

```
def recipPos (i : QInterval) (hi : 0 < i.lo) : QInterval where
  lo := 1 / i.hi
  hi := 1 / i.lo
  ordered := one_div_le_one_div_of_le hi i.ordered
```

ScalarSuffixCertificate.Vec4

Classification: supplement

Source: ScalarSuffixCertificate.lean:418-419

Declaration SHA-256: 5221a72a407ada42251ae04520dcef5209887e8aa5988ed22eb9dbd06e5ba073

Mathematical statement. Four real error coordinates as an indexed vector.

The indices correspond to xi, eta, rho, and sigma.

```
abbrev Vec4 := Fin 4 → ℝ
```

ScalarSuffixCertificate.cosTaylor26

Classification: supplement

Source: ScalarSuffixCertificate.lean:219-223

Declaration SHA-256: 46d8e337f6347f1173e5244107f4620ec9e73c77dbe96ffed5fdc f32a93ba782

Mathematical statement. The degree-26 Taylor polynomial for cosine.

It is the even alternating sum through the x^{26} term.

```
def cosTaylor26 (x : ℚ) : ℚ :=
  ∑ k ∈ Finset.range 14, (-1 : ℚ) ^ k * x ^ (2 * k) /
    (2 * k).factorial
```

```
-- Degree-27 sine enclosure with the exact Lagrange radius  $|x|^{28}/28!$ . -/
```

ScalarSuffixCertificate.cosTaylor26Interval

Classification: supplement

Source: ScalarSuffixCertificate.lean:230-234

Declaration SHA-256: 97a38531dd63947f237378c23320eb2947ffd81d0b5929b322bafd6a94ec891e

Mathematical statement. For any rational x , the interval centered at the degree-26 cosine polynomial with error radius $|x|^{27}/27!$.

The interval is $[C_{26}(x) - |x|^{27}/27!, C_{26}(x) + |x|^{27}/27!]$, where C_{26} is the even Taylor polynomial through degree 26. The definition is valid for every rational input; the following soundness theorem proves containment without an argument-domain restriction.

```
def cosTaylor26Interval (x : ℚ) : QInterval :=
  let p := cosTaylor26 x
  let r := |x| ^ 27 / (Nat.factorial 27 : ℚ)
  ⟨p - r, p + r, by linarith [show 0 ≤ r by positivity]⟩
```

ScalarSuffixCertificate.piInterval

Classification: supplement

Source: ScalarSuffixCertificate.lean:206-208

Declaration SHA-256: da867cd8e471ad99a2b0bc5bbf7e433df88cee3a9989f797f883fa79eab1b795

Mathematical statement. A fixed rational enclosure of π .

Its exact endpoints are used in all dependent trigonometric certificate expressions.

```
def piInterval : QInterval :=
  ⟨3.141592, 3.141593, by norm_num⟩
```

ScalarSuffixCertificate.piInterval_sound

Classification: supplement

Source: ScalarSuffixCertificate.lean:209-213

Declaration SHA-256: 0e2096e4d4fa24f53f2ff61998135a39f6cbe4314d1809b974cfbb5eccd9fe5b

Mathematical statement. π belongs to the fixed rational enclosure.

This supplies the real interpretation of the certificate's π bounds.

```
theorem piInterval_sound : piInterval.RealContains  $\pi$  := by
  simp [piInterval, LeanSuffixReflective.QInterval.RealContains] using
    And.intro Real.pi_gt_d6.le Real.pi_lt_d6.le

-- Degree-27 sine polynomial used by the face-certificate checker. -/
```

ScalarSuffixCertificate.realContains_arccos_of_arcsin

Classification: supplement

Source: ScalarSuffixCertificate.lean:403-410

Declaration SHA-256: 6b1989559f420605398bd419fe5c29784f5a3ffc9b8acebbd0bee1ba892adc65

Mathematical statement. An arcsine enclosure and π bounds give an arccosine enclosure.

Use $\arccos x = \pi/2 - \arcsin x$ with sound interval subtraction.

```
theorem realContains_arccos_of_arcsin {p a : QInterval} {x : ℝ}
  (hp : p.RealContains π) (ha : a.RealContains (arcsin x)) :
  ((p.nsmul (1 / 2) (by norm_num)).sub a).RealContains (arccos x) := by
  have hp2 := QInterval.realContains_nsmul (q := (1 / 2 : ℚ)) (by norm_num) hp
  have h := LeanSuffixReflective.QInterval.realContains_sub hp2 ha
  simp [Real.arccos, div_eq_mul_inv, mul_comm] using h

/-- One-dimensional opposite-face lemma for scalar IVT seams. -/
```

ScalarSuffixCertificate.realContains_cos_interval

Classification: supplement

Source: ScalarSuffixCertificate.lean:339-352

Declaration SHA-256: eedd5e166794798d683fb5483e2b9b97111a653338c01d010cd7592bb93d98f1

Mathematical statement. Cosine endpoint enclosures extend to the recorded interval of arguments.

The decreasing principal-branch order is reflected in the chosen endpoints.

```
theorem realContains_cos_interval {input output : QInterval} {x : ℝ}
  (hx : input.RealContains x)
  (hbranch : 0 ≤ (input.lo : ℝ) ∧ (input.hi : ℝ) ≤ π)
  (hlo : (output.lo : ℝ) ≤ cos (input.hi : ℝ))
  (hhi : cos (input.lo : ℝ) ≤ (output.hi : ℝ)) :
  output.RealContains (cos x) := by
  constructor
  · exact hlo.trans (Real.cos_le_cos_of_nonneg_of_le_pi
    (hbranch.1.trans hx.1) hbranch.2 hx.2)
  · exact (Real.cos_le_cos_of_nonneg_of_le_pi hbranch.1
    (hx.2.trans hbranch.2) hx.1).trans hhi

/-- Lift the degree-27 point certificates over a monotone sine interval.
`hordered` is an exact-rational check on the resulting endpoints. -/
```

ScalarSuffixCertificate.realContains_sin_interval

Classification: supplement

Source: ScalarSuffixCertificate.lean:326-338

Declaration SHA-256: 21959c159381deb1e01f48cd4776b6a292e590b4e5bd4cd351519ff872a0aa25

Mathematical statement. Sine endpoint enclosures extend to an interval of arguments on the checked monotone branch.

Monotonicity and certified endpoint values enclose every allowed sine value.

```
theorem realContains_sin_interval {input output : QInterval} {x : ℝ}
  (hx : input.RealContains x)
  (hbranch : -(π / 2) ≤ (input.lo : ℝ) ∧ (input.hi : ℝ) ≤ π / 2)
  (hlo : (output.lo : ℝ) ≤ sin (input.lo : ℝ))
  (hhi : sin (input.hi : ℝ) ≤ (output.hi : ℝ)) :
  output.RealContains (sin x) := by
  constructor
  · exact hlo.trans (Real.sin_le_sin_of_le_of_le_pi_div_two hbranch.1
    (hx.2.trans hbranch.2) hx.1)
  · exact (Real.sin_le_sin_of_le_of_le_pi_div_two
    (hbranch.1.trans hx.1) hbranch.2 hx.2).trans hhi

/-- Cosine endpoint certificates extend to a principal-branch interval. -/
```


ScalarSuffixCertificate.sinTaylor27

Classification: supplement

Source: ScalarSuffixCertificate.lean:214-218

Declaration SHA-256: cad29fad0ca5f0f5a021df9a703aaac6f17eda67757816193f0541445c680252

Mathematical statement. The degree-27 Taylor polynomial for sine.

It is the odd alternating sum through the x^{27} term.

```
def sinTaylor27 (x : ℚ) : ℚ :=
  ∑ k ∈ Finset.range 14, (-1 : ℚ) ^ k * x ^ (2 * k + 1) /
    (2 * k + 1).factorial

/-- Degree-26 cosine polynomial used by the face-certificate checker. -/
```

ScalarSuffixCertificate.sinTaylor27Interval

Classification: supplement

Source: ScalarSuffixCertificate.lean:224-229

Declaration SHA-256: 9588051f8a2ff67152b4c9b894d6f120c8732ea5b8195eebd297f8f7e92a5437

Mathematical statement. A rational sine enclosure built from the degree-27 polynomial and a remainder bound.

Both endpoints account for truncation error; the polynomial alone is not treated as exact sine.

```
def sinTaylor27Interval (x : ℚ) : QInterval :=
  let p := sinTaylor27 x
  let r := |x| ^ 28 / (Nat.factorial 28 : ℚ)
  ⟨p - r, p + r, by linarith [show 0 ≤ r by positivity]⟩

/-- Degree-26 cosine enclosure with the exact Lagrange radius `|x|27 / 27!`. -/
```

StripCore.carrier_y_bounds

Classification: supplement

Source: CMVGeometry.lean:482-490

Declaration SHA-256: dfd3510aaac02f0e3230984bb1d2516c15f954b7d416fa7e30c71ad44efa3ab8

Mathematical statement. Every core point lies between heights minus one and one.

Thus its density is one, including its horizontal boundary.

```
theorem carrier_y_bounds (c : StripCore) {p : PlanePoint}
  (hp : p ∈ c.carrier) : |p.2| ≤ 1 := by
  rcases hp with (hp | hp) | hp
  · exact hp.2.2
  · exact hp.2.2
  · exact hp.2.2

/-- Every horizontal segment across the core rectangle is contained in the
retained region. -/
```

StripCore.cos_sideAngle

Classification: supplement

Source: CMVGeometry.lean:478-481

Declaration SHA-256: 5c26f39a6dccf790fede705362af68051955e479d2108518ff90a5d9985148ed

Mathematical statement. The cosine of the core side angle is the nonnegative radical.

$$\cos(\arcsin h) = \sqrt{1 - h^2}.$$

```
theorem cos_sideAngle (c : StripCore) :
  cos c.sideAngle =  $\sqrt{1 - c.curvature ^ 2}$  := by
  simp [sideAngle, Real.cos_arcsin]
```

StripCore.horizontalChord_subset_carrier

Classification: supplement

Source: CMVGeometry.lean:491-499

Declaration SHA-256: 334fc66a21b600efd108f9065db91fc12f64adadeb2b25ecfa3e64aa16686847

Mathematical statement. The horizontal chord segments lie in the core.

The rectangle contains the chords at the strip interfaces.

```
theorem horizontalChord_subset_carrier (c : StripCore) {y : ℝ}
  (hy : |y| ≤ 1) :
  {p : PlanePoint | p.2 = y ∧ |p.1| ≤ c.chord / 2} ⊆ c.carrier := by
  intro p hp
  have hx := abs_le.mp hp.2
  apply Or.inl
  apply Or.inl
  exact ⟨by linarith, by linarith, by simp [hp.1] using hy⟩
```

StripCore.isClosed_carrier

Classification: supplement

Source: CMVGeometry.lean:805-843

Declaration SHA-256: d28fca8c993d5d2b9aa0b2ebe7f3f0372ebdd951a5cda90d26adf748bface0e4

Mathematical statement. The core carrier is closed.

Its three constituent closed pieces form a finite union.

```
theorem isClosed_carrier (c : StripCore) : IsClosed c.carrier := by
  have hx : Continuous (fun p : PlanePoint => p.1) := continuous_fst
  have hy : Continuous (fun p : PlanePoint => p.2) := continuous_snd
  have habsy : Continuous (fun p : PlanePoint => |p.2|) := hy.abs
  have hrect : IsClosed c.rectangleCarrier := by
    rw [StripCore.rectangleCarrier]
    change IsClosed
    ({p : PlanePoint | -c.chord / 2 ≤ p.1} ∩
     {p : PlanePoint | p.1 ≤ c.chord / 2} ∩
     {p : PlanePoint | |p.2| ≤ 1})
  exact (isClosed_le continuous_const hx).inter
    ((isClosed_le hx continuous_const).inter
     (isClosed_le habsy continuous_const))
  have hleft : IsClosed c.leftCapCarrier := by
    rw [StripCore.leftCapCarrier]
    change IsClosed
    ({p : PlanePoint |
      (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} ∩
     {p : PlanePoint | p.1 ≤ -c.chord / 2} ∩
     {p : PlanePoint | |p.2| ≤ 1})
  exact (isClosed_le
    (((hx.sub continuous_const).pow 2).add (hy.pow 2))
    continuous_const).inter
    ((isClosed_le hx continuous_const).inter
     (isClosed_le habsy continuous_const))
  have hright : IsClosed c.rightCapCarrier := by
    rw [StripCore.rightCapCarrier]
    change IsClosed
    ({p : PlanePoint |
```

```

      (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} ∩
      ({p : PlanePoint | c.chord / 2 ≤ p.1} ∩
       {p : PlanePoint | |p.2| ≤ 1}))
exact (isClosed_le
      (((hx.sub continuous_const).pow 2).add (hy.pow 2))
       continuous_const).inter
      ((isClosed_le continuous_const hx).inter
       (isClosed_le habsy continuous_const))
exact (hrect.union hleft).union hright

```

StripCore.leftArcTrace

Classification: supplement

Source: CMVGeometry.lean:762-766

Declaration SHA-256: 702872b40533619867ab566579d52d3f6a44e812910796d96106d1c50acbbfc2

Mathematical statement. The circular trace of the left core side.

It is the exposed circle portion outside the left rectangle edge.

```

def leftArcTrace (c : StripCore) : Set PlanePoint :=
  {p | (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2 ∧
       p.1 ≤ -c.chord / 2 ∧ |p.2| ≤ 1}

/-- The exact right circular trace of a strip core. -/

```

StripCore.leftCapCarrier

Classification: supplement

Source: CMVGeometry.lean:440-443

Declaration SHA-256: 2ca301cf56be587995c60fad814a6021c666b20d0f45e1fa3f2964b23f678cbc

Mathematical statement. The left circular segment attached to the core rectangle.

It is the portion of the left disk outside the rectangle's left vertical side.

```

def leftCapCarrier (c : StripCore) : Set PlanePoint :=
  {p | (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2 ∧
       p.1 ≤ -c.chord / 2 ∧ |p.2| ≤ 1}

```

StripCore.leftCenterX

Classification: supplement

Source: CMVGeometry.lean:431-433

Declaration SHA-256: 01fd3b14ca7e8a9d341e71e2fb86b773a9bea051016f7f6499489c3b65bab474

Mathematical statement. Horizontal center coordinate for the left core circle.

It places the circle through the two left chord endpoints at heights ± 1 .

```

def leftCenterX (c : StripCore) : ℝ :=
  -c.chord / 2 + c.radius * cos c.sideAngle

```

StripCore.lowerChordTrace

Classification: supplement

Source: CMVGeometry.lean:774-776

Declaration SHA-256: faa702d9afdf2eed044fad4474692be94c38bb3a9868779993853eb74445596

Mathematical statement. The core's lower horizontal chord trace.

It lies at $y = -1$ and has length c .

```
def lowerChordTrace (c : StripCore) : Set PlanePoint :=
  {p | p.2 = -1 ∧ |p.1| ≤ c.chord / 2}
```

StripCore.measurableSet_carrier

Classification: supplement

Source: CMVGeometry.lean:500-551

Declaration SHA-256: c4a110b00efc328d9afc36bfb523f064a5cb69170025121ec86c0022fc798171

Mathematical statement. The core carrier is measurable.

It is a finite union of measurable rectangle and circular-segment sets.

```
theorem measurableSet_carrier (c : StripCore) : MeasurableSet c.carrier := by
  have hx : Continuous (fun p : PlanePoint => p.1) := continuous_fst
  have hy : Continuous (fun p : PlanePoint => p.2) := continuous_snd
  have hrect : MeasurableSet c.rectangleCarrier := by
    have hxmin : IsClosed {p : PlanePoint | -c.chord / 2 ≤ p.1} :=
      isClosed_le continuous_const hx
    have hxmax : IsClosed {p : PlanePoint | p.1 ≤ c.chord / 2} :=
      isClosed_le hx continuous_const
    have hybound : IsClosed {p : PlanePoint | |p.2| ≤ 1} :=
      isClosed_le hy.abs continuous_const
    rw [rectangleCarrier]
    change MeasurableSet
    ({p : PlanePoint | -c.chord / 2 ≤ p.1} ∩
     {p : PlanePoint | p.1 ≤ c.chord / 2} ∩
     {p : PlanePoint | |p.2| ≤ 1})
  exact (hxmin.inter (hxmax.inter hybound)).measurableSet
  have hleft : MeasurableSet c.leftCapCarrier := by
    have hdisk : IsClosed {p : PlanePoint |
      (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} :=
      isClosed_le (((hx.sub continuous_const).pow 2).add (hy.pow 2))
      continuous_const
    have hxbound : IsClosed {p : PlanePoint | p.1 ≤ -c.chord / 2} :=
      isClosed_le hx continuous_const
    have hybound : IsClosed {p : PlanePoint | |p.2| ≤ 1} :=
      isClosed_le hy.abs continuous_const
    rw [leftCapCarrier]
    change MeasurableSet
    ({p : PlanePoint |
      (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} ∩
     {p : PlanePoint | p.1 ≤ -c.chord / 2} ∩
     {p : PlanePoint | |p.2| ≤ 1})
  exact (hdisk.inter (hxbound.inter hybound)).measurableSet
  have hright : MeasurableSet c.rightCapCarrier := by
    have hdisk : IsClosed {p : PlanePoint |
      (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} :=
      isClosed_le (((hx.sub continuous_const).pow 2).add (hy.pow 2))
      continuous_const
    have hxbound : IsClosed {p : PlanePoint | c.chord / 2 ≤ p.1} :=
      isClosed_le continuous_const hx
    have hybound : IsClosed {p : PlanePoint | |p.2| ≤ 1} :=
      isClosed_le hy.abs continuous_const
    rw [rightCapCarrier]
    change MeasurableSet
```

```

    ({p : PlanePoint |
      (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} ∩
      ({p : PlanePoint | c.chord / 2 ≤ p.1} ∩
        {p : PlanePoint | |p.2| ≤ 1}))
    exact (hdisk.inter (hxbound.inter hybound)).measurableSet
    exact (hrect.union hleft).union hright
  end StripCore

```

StripCore.radius

Classification: supplement

Source: CMVGeometry.lean:426-428

Declaration SHA-256: f23d45618cc8f0a402155f6b0b9c9bed86df54007d8b97c4f4027753ebfcd373

Mathematical statement. The core circle radius is reciprocal curvature.

$R = 1/h$, with $0 < h \leq 1$.

```

def radius (c : StripCore) : ℝ := 1 / c.curvature

/-- Half-central-angle of either strip arc. -/

```

StripCore.radius_pos

Classification: supplement

Source: CMVGeometry.lean:460-462

Declaration SHA-256: 754f25da7343b067aca1ccf07fea956e09bfd461a16a072435b6b938ea98f462

Mathematical statement. The core radius is positive.

Reciprocation preserves positivity for $h > 0$.

```

theorem radius_pos (c : StripCore) : 0 < c.radius :=
  one_div_pos.mpr c.curvature_pos

```

StripCore.rectangleCarrier

Classification: supplement

Source: CMVGeometry.lean:437-439

Declaration SHA-256: 5204ba429abcba2c9a7bfd26b44272d1068d5b57b8a97a88331831661e9408f9

Mathematical statement. The central rectangle of width c and height two.

Its carrier is $[-c/2, c/2] \times [-1, 1]$.

```

def rectangleCarrier (c : StripCore) : Set PlanePoint :=
  {p | -c.chord / 2 ≤ p.1 ∧ p.1 ≤ c.chord / 2 ∧ |p.2| ≤ 1}

```

StripCore.rightArcTrace

Classification: supplement

Source: CMVGeometry.lean:767-770

Declaration SHA-256: 059982ba84229cdd58242cd7f98720be73ad274e207529f914e522d73ba5cbeb

Mathematical statement. The circular trace of the right core side.

It is the reflected exposed side arc.

```
def rightArcTrace (c : StripCore) : Set PlanePoint :=
  {p | (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2 ∧
    c.chord / 2 ≤ p.1 ∧ |p.2| ≤ 1}
```

StripCore.rightCapCarrier

Classification: supplement

Source: CMVGeometry.lean:444-448

Declaration SHA-256: 0f7231f5e97656f30e8e5131e2afc098206a3ff9953e2cd02ae8207ac4d54b12

Mathematical statement. The reflected right circular segment.

It is the portion of the right disk beyond the rectangle's right side.

```
def rightCapCarrier (c : StripCore) : Set PlanePoint :=
  {p | (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2 ∧
    c.chord / 2 ≤ p.1 ∧ |p.2| ≤ 1}

/-- Genuine coordinate carrier of the retained central region. -/
```

StripCore.rightCenterX

Classification: supplement

Source: CMVGeometry.lean:434-436

Declaration SHA-256: ff1acc9e9b3ef170758d23460b0f614f6eb1c034ebd024b460f189e9f303f923

Mathematical statement. Horizontal center coordinate for the right core circle.

It is the reflected placement of the left core circle.

```
def rightCenterX (c : StripCore) : ℝ :=
  c.chord / 2 - c.radius * cos c.sideAngle
```

StripCore.sideAngle_le_pi_div_two

Classification: supplement

Source: CMVGeometry.lean:466-469

Declaration SHA-256: 0c0857ab96c23693b4448efce2e70a6069dda6f460cf0118b7e365f0f1074987

Mathematical statement. The core side angle is at most $\pi/2$.

This includes the curvature endpoint $h = 1$.

```
theorem sideAngle_le_pi_div_two (c : StripCore) :
  c.sideAngle ≤ π / 2 := by
  exact Real.arcsin_le_pi_div_two c.curvature
```

StripCore.sideAngle_lt_pi

Classification: supplement

Source: CMVGeometry.lean:470-473

Declaration SHA-256: 4d165924e20daf34cd0cd3a93af12e6637e356347b723990beeb9d1a85af59a9e

Mathematical statement. The core side angle is strictly below π .

It follows from the stronger upper bound $\pi/2$.

```
theorem sideAngle_lt_pi (c : StripCore) : c.sideAngle <  $\pi$  := by
  have hp := Real.pi_pos
  linarith [c.sideAngle_le_pi_div_two]
```

StripCore.sideAngle_pos

Classification: supplement

Source: CMVGeometry.lean:463-465

Declaration SHA-256: 20886da5ba412dbc0666239218b14d32a1bad68cf7a0f4837b3f6216a46aefbd

Mathematical statement. The core side angle is positive.

$h > 0$ implies $\arcsinh h > 0$.

```
theorem sideAngle_pos (c : StripCore) : 0 < c.sideAngle := by
  simpa [sideAngle] using (Real.arcsin_pos.mpr c.curvature_pos)
```

StripCore.sin_sideAngle

Classification: supplement

Source: CMVGeometry.lean:474-477

Declaration SHA-256: e85f9cb3c0c2b22c1ce74fdc60660033205e7a0821e76cfdd7248571352ea184

Mathematical statement. The sine of the core side angle is h .

The principal arcsine identity applies because $0 < h \leq 1$.

```
theorem sin_sideAngle (c : StripCore) : sin c.sideAngle = c.curvature := by
  rw [sideAngle]
  exact Real.sin_arcsin (by linarith [c.curvature_pos]) c.curvature_le_one
```

StripCore.upperChordTrace

Classification: supplement

Source: CMVGeometry.lean:771-773

Declaration SHA-256: 71db3f396f38ac0c4ed32b75273f08b0092359d4c0fdd7bc77976000e7791fb0

Mathematical statement. The core's upper horizontal chord trace.

It lies at $y = 1$ and has length c .

```
def upperChordTrace (c : StripCore) : Set PlanePoint :=
  {p | p.2 = 1  $\wedge$  |p.1|  $\leq$  c.chord / 2}
```

StripCore.volume_leftCapCarrier

Classification: supplement

Source: CMVGeometry.lean:2076-2138

Declaration SHA-256: ee7dbe308fa5b9feabca0b7d1f31d2c91cdf7e530a43dcc64a52dce7563fb7b8

Mathematical statement. Area of the left core circular segment.

The segment has chord two and area $4a(\arcsinh h)$.

```
lemma volume_leftCapCarrier (c : StripCore) :
  volume c.leftCapCarrier = ENNReal.ofReal (4 * area c.sideAngle) := by
  let U : Set PlanePoint :=
    {p | (p.1 - 0) ^ 2 + (p.2 - c.leftCenterX) ^ 2  $\leq$  c.radius ^ 2  $\wedge$ 
```

```

    p.2 ≤ c.leftCenterX - c.radius * cos c.sideAngle}
have hcos : 0 ≤ cos c.sideAngle := by
  rw [c.cos_sideAngle]
  positivity
have hpre : Prod.swap ⁻¹' U = c.leftCapCarrier := by
  ext p
  change ((p.2 - 0) ^ 2 + (p.1 - c.leftCenterX) ^ 2 ≤ c.radius ^ 2 ∧
    p.1 ≤ c.leftCenterX - c.radius * cos c.sideAngle) ↔
    ((p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2 ∧
    p.1 ≤ -c.chord / 2 ∧ |p.2| ≤ 1)
have hbase : c.leftCenterX - c.radius * cos c.sideAngle = -c.chord / 2 := by
  rw [leftCenterX]
  ring
rw [hbase]
constructor
· rintro ⟨hdisk, hx⟩
  have hdx : p.1 - c.leftCenterX ≤ -(c.radius * cos c.sideAngle) := by
    rw [leftCenterX]
    linarith
  have hdx0 : p.1 - c.leftCenterX ≤ 0 := by
    nlinarith [c.radius_pos]
  have hsquare : (c.radius * cos c.sideAngle) ^ 2 ≤
    (p.1 - c.leftCenterX) ^ 2 := by
    apply sq_le_sq.mpr
    rw [abs_of_nonneg (mul_nonneg c.radius_pos.le hcos), abs_of_nonpos hdx0]
    linarith
  have hy2 : p.2 ^ 2 ≤ 1 := by
    nlinarith [radius_sq_cos_sq' c]
  have hyabs : |p.2| ≤ |(1 : ℝ)| := sq_le_sq.mp (by simpa using hy2)
  exact ⟨by nlinarith, hx, by simpa using hyabs⟩
· rintro ⟨hdisk, hx, _hy⟩
  exact ⟨by nlinarith, hx⟩
have hU : MeasurableSet U := by
  change MeasurableSet
  ({p : PlanePoint |
    (p.1 - 0) ^ 2 + (p.2 - c.leftCenterX) ^ 2 ≤ c.radius ^ 2} ∩
  {p : PlanePoint |
    p.2 ≤ c.leftCenterX - c.radius * cos c.sideAngle})
  exact (isClosed_le (by fun_prop) continuous_const).measurableSet.inter
    (isClosed_le continuous_snd continuous_const).measurableSet
have hmeasure : volume c.leftCapCarrier = volume U := by
  rw [← hpre]
  exact Measure.measurePreserving_swap.measure_preimage hU.nullMeasurableSet
have hreal : c.radius ^ 2 *
  (c.sideAngle - sin c.sideAngle * cos c.sideAngle) =
  4 * area c.sideAngle := by
  rw [area]
  have hs : sin c.sideAngle = c.curvature := c.sin_sideAngle
  rw [hs]
  rw [radius]
  field_simp [ne_of_gt c.curvature_pos]
calc
  volume c.leftCapCarrier = volume U := hmeasure
  _ = ENNReal.ofReal (c.radius ^ 2 *
    (c.sideAngle - sin c.sideAngle * cos c.sideAngle)) := by
    simpa [U] using volume_lower_circular_segment 0 c.leftCenterX
      c.radius c.sideAngle c.radius_pos c.sideAngle_pos.le c.sideAngle_lt_pi.le
  _ = ENNReal.ofReal (4 * area c.sideAngle) := congrArg ENNReal.ofReal hreal

```

StripCore.volume_rectangleCarrier

Classification: supplement

Source: CMVGeometry.lean:2060-2075

Declaration SHA-256: b2c1d8716422a2e0169b6aaca4e89790c14b0594bc75f1e77b6bbec1349d5110

Mathematical statement. The rectangle has area twice the chord.

Width c times height two gives $2c$.


```

lemma volume_rectangleCarrier (c : StripCore) :
  volume c.rectangleCarrier = ENNReal.ofReal (2 * c.chord) := by
  have hset : c.rectangleCarrier =
    Icc (-c.chord / 2) (c.chord / 2) ×ˢ Icc (-1 : ℝ) 1 := by
  ext p
  simp only [rectangleCarrier, mem_setOf_eq, mem_prod, mem_Icc]
  constructor
  · rintro ⟨hx1, hx2, hy⟩
    exact ⟨⟨hx1, hx2⟩, abs_le.mp hy⟩
  · rintro ⟨⟨hx1, hx2⟩, hy1, hy2⟩
    exact ⟨hx1, hx2, abs_le.mpr ⟨hy1, hy2⟩⟩
  rw [hset, Measure.volume_eq_prod, Measure.prod_prod,
    Real.volume_Icc, Real.volume_Icc,
    show c.chord / 2 - -c.chord / 2 = c.chord by ring]
  norm_num [ENNReal.ofReal_mul, c.chord_pos.le, mul_comm]

```

StripCore.volume_rightCapCarrier

Classification: supplement

Source: CMVGeometry.lean:2139-2201

Declaration SHA-256: 7bc214af5d8d445665e416853a1fab80112ca5fe1116500de0e9b2b6285dfdf7

Mathematical statement. Area of the right core circular segment.

Reflection gives the same value $4a(\operatorname{arcsinh})$.

```

lemma volume_rightCapCarrier (c : StripCore) :
  volume c.rightCapCarrier = ENNReal.ofReal (4 * area c.sideAngle) := by
  let U : Set PlanePoint :=
    {p | (p.1 - 0) ^ 2 + (p.2 - c.rightCenterX) ^ 2 ≤ c.radius ^ 2 ∧
      c.rightCenterX + c.radius * cos c.sideAngle ≤ p.2}
  have hcos : 0 ≤ cos c.sideAngle := by
    rw [c.cos_sideAngle]
    positivity
  have hpre : Prod.swap ⁻¹' U = c.rightCapCarrier := by
    ext p
    change ((p.2 - 0) ^ 2 + (p.1 - c.rightCenterX) ^ 2 ≤ c.radius ^ 2 ∧
      c.rightCenterX + c.radius * cos c.sideAngle ≤ p.1) ↔
      ((p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2 ∧
        c.chord / 2 ≤ p.1 ∧ |p.2| ≤ 1)
  have hbase : c.rightCenterX + c.radius * cos c.sideAngle = c.chord / 2 := by
    rw [rightCenterX]
    ring
  rw [hbase]
  constructor
  · rintro ⟨hdisk, hx⟩
    have hdx : c.radius * cos c.sideAngle ≤ p.1 - c.rightCenterX := by
      rw [rightCenterX]
      linarith
    have hdx0 : 0 ≤ p.1 - c.rightCenterX := by
      nlinarith [c.radius_pos]
    have hsquare : (c.radius * cos c.sideAngle) ^ 2 ≤
      (p.1 - c.rightCenterX) ^ 2 := by
      apply sq_le_sq.mpr
      rw [abs_of_nonneg (mul_nonneg c.radius_pos.le hcos), abs_of_nonneg hdx0]
      exact hdx
    have hy2 : p.2 ^ 2 ≤ 1 := by
      nlinarith [radius_sq_cos_sq' c]
    have hyabs : |p.2| ≤ |(1 : ℝ)| := sq_le_sq.mp (by simpa using hy2)
    exact ⟨by nlinarith, hx, by simpa using hyabs⟩
  · rintro ⟨hdisk, hx, _hy⟩
    exact ⟨by nlinarith, hx⟩
  have hU : MeasurableSet U := by
    change MeasurableSet
    ({p : PlanePoint |
      (p.1 - 0) ^ 2 + (p.2 - c.rightCenterX) ^ 2 ≤ c.radius ^ 2} ∩
      {p : PlanePoint |
        c.rightCenterX + c.radius * cos c.sideAngle ≤ p.2})

```

```

    exact (isClosed_le (by fun_prop) continuous_const).measurableSet.inter
      (isClosed_le continuous_const continuous_snd).measurableSet
  have hmeasure : volume c.rightCapCarrier = volume U := by
    rw [← hpre]
    exact Measure.measurePreserving_swap.measure_preimage hU.nullMeasurableSet
  have hreal : c.radius ^ 2 *
    (c.sideAngle - sin c.sideAngle * cos c.sideAngle) =
    4 * area c.sideAngle := by
    rw [area]
    have hs : sin c.sideAngle = c.curvature := c.sin_sideAngle
    rw [hs]
    rw [radius]
    field_simp [ne_of_gt c.curvature_pos]
  calc
    volume c.rightCapCarrier = volume U := hmeasure
    _ = ENNReal.ofReal (c.radius ^ 2 *
      (c.sideAngle - sin c.sideAngle * cos c.sideAngle)) := by
      simp [U] using volume_upper_circular_segment 0 c.rightCenterX
        c.radius c.sideAngle c.radius_pos c.sideAngle_pos.le c.sideAngle_lt_pi.le
    _ = ENNReal.ofReal (4 * area c.sideAngle) := congrArg ENNReal.ofReal hreal

```

TypeThreeAssembly.bottomChord_nonneg

Classification: supplement

Source: TypeThreeAssembly.lean:352-354

Declaration SHA-256: bbe5a0b68c27b1eab70889a99796728aff04d58c47eda0333c16191c78a8f12a

Mathematical statement. The bottom chord length is nonnegative.

It equals twice the nonnegative half-width.

```

theorem bottomChord_nonneg : 0 ≤ a.bottomChord := by
  exact mul_nonneg (by norm_num) a.sideHalfWidth_nonneg

```

TypeThreeAssembly.bottomChord_pos

Classification: supplement

Source: TypeThreeAssembly.lean:359-362

Declaration SHA-256: 5deb1da8ce95e59329d432d4f1484bad3b915566f05bd8a9e6b37a69ad4de177

Mathematical statement. The bottom chord is positive when h differs from one half.

It is twice the strictly positive half-width on that branch.

```

theorem bottomChord_pos (hh : a.h ≠ 1 / 2) : 0 < a.bottomChord := by
  exact mul_pos (by norm_num) (a.sideHalfWidth_pos hh)

```

TypeThreeAssembly.boundaryJoinSet

Classification: supplement

Source: TypeThreeAssembly.lean:1130-1135

Declaration SHA-256: 4d107bd731d106706191e0cc66c1eb6e52b0ad83df797a9379d71e46e9ea68c7

Mathematical statement. The finite junction set of the type-(iii) boundary pieces.

It contains upper arc junctions and bottom segment endpoints, allowing coincidences at the transition.

```

def boundaryJoinSet : Set PlanePoint :=
  {(-a.radius * sin a.outerAngle, 1),
   (a.radius * sin a.outerAngle, 1),

```

```
(-a.sideHalfWidth, -1),  
(a.sideHalfWidth, -1)}
```

TypeThreeAssembly.boundaryJoinSet_finite

Classification: supplement

Source: TypeThreeAssembly.lean:1136-1141

Declaration SHA-256: fb8b2d3307f8905b1616f7f8934468dae8dc2d1df098e8fa72f92f5a3759db5d

Mathematical statement. The type-(iii) junction set is finite.

It is explicitly assembled from finitely many endpoint coordinates.

```
theorem boundaryJoinSet_finite : a.boundaryJoinSet.Finite := by  
  simp [boundaryJoinSet]  
  
/-- Every pairwise overlap of the four primitive frontier traces is a boundary  
join. In particular, when `h = 1 / 2`, the two side arcs meet only at the  
shared bottom point. -/
```

TypeThreeAssembly.boundaryTrace_subset_frontier_carrier

Classification: supplement

Source: TypeThreeAssembly.lean:2840-2847

Declaration SHA-256: daeda01e39ff76764527644f12850660eec76e1d4031dd8579c24a3b55dda6ba

Mathematical statement. Every listed type-(iii) trace belongs to the carrier frontier.

Local exterior points prove exposure of the side arcs, upper cap, and bottom segment.

```
theorem boundaryTrace_subset_frontier_carrier :  
  a.boundaryTrace ⊆ frontier a.carrier := by  
  rintro p (hleft | hright | hupper | hbottom)  
  · exact a.leftArc_mem_frontier hleft  
  · exact a.rightArc_mem_frontier hright  
  · exact a.upperArc_mem_frontier hupper  
  · exact a.bottomSegmentCarrier_subset_frontier hbottom
```

TypeThreeAssembly.continuous_realizedLeftArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:690-699

Declaration SHA-256: 5aa13a8e0a54ece98427e269f77eeefa81d89485ad9d1f32dde5c0e6f344ef9b4

Mathematical statement. The realized type-(iii) left parameter is continuous.

Its trigonometric coordinates depend continuously on arclength.

```
lemma continuous_realizedLeftArcParam :  
  Continuous a.realizedLeftArcParam := by  
  rw [show a.realizedLeftArcParam =  
    sideArcComplexIsometryMap a.leftCenter ∘  
    unitCircleArc a.radius by  
    funext s  
    exact a.realizedLeftArcParam_eq s]  
  exact (isometry_sideArcComplexIsometryMap a.leftCenter).continuous.comp  
    (continuous_unitCircleArc a.radius)
```

TypeThreeAssembly.continuous_realizedRightArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:700-709

Declaration SHA-256: 3d9c0740b38c1fa009799b8c35785a0cb767211b94d4ca0bc37637a119a58afd

Mathematical statement. The realized type-(iii) right parameter is continuous.

Continuity gives compactness of its closed interval image.

```
lemma continuous_realizedRightArcParam :
  Continuous a.realizedRightArcParam := by
  rw [show a.realizedRightArcParam =
    sideArcComplexIsometryMap a.rightCenter ∘
    unitCircleArc a.radius by
  funext s
  exact a.realizedRightArcParam_eq s]
exact (isometry_sideArcComplexIsometryMap a.rightCenter).continuous.comp
  (continuous_unitCircleArc a.radius)
```

TypeThreeAssembly.coreCarrier_y_bounds

Classification: supplement

Source: TypeThreeAssembly.lean:1447-1455

Declaration SHA-256: a0a8c181bd45290d862033f8524f84e76d590a21a370fbbc0214aab783d9df85

Mathematical statement. Every point of the type-(iii) core lies in the closed strip.

Thus the core's area density is exactly one.

```
theorem coreCarrier_y_bounds {p : PlanePoint}
  (hp : p ∈ a.coreCarrier) : |p.2| ≤ 1 := by
  rcases hp with (hrect | hleft) | hright
  · exact abs_le.mpr ⟨hrect.2.1, hrect.2.2⟩
  · exact abs_le.mpr hleft.2.2
  · exact abs_le.mpr hright.2.2

/-- Above the source strip, the genuine type-(iii) carrier is exactly its
actual exterior cap; none of the three core pieces can occur there. -/
```

TypeThreeAssembly.cos_innerAngle

Classification: supplement

Source: TypeThreeAssembly.lean:178-183

Declaration SHA-256: e774d0a2578a71402d098d5ac027f2d6d1dbb90425ca111832168cfb9f013412

Mathematical statement. The cosine of the inner angle is $2h-1$.

Apply the principal arccosine inverse identity on $(-1, 1)$.

```
theorem cos_innerAngle : cos a.innerAngle = a.shapeParameter := by
  rw [innerAngle, Real.cos_arccos a.shapeParameter_mem.1.le
    a.shapeParameter_mem.2.le]

/-- The principal-angle definitions recover the Snell-adjusted exterior
argument exactly. -/
```

TypeThreeAssembly.cos_outerAngle

Classification: supplement

Source: TypeThreeAssembly.lean:184-188

Declaration SHA-256: 3eb066bbadef1d5d0722d2c322b7cfe8bce8e1eb62204f2872b1f8f1bbb41a24

Mathematical statement. The cosine of the exterior angle is $(2h-1)/\lambda$.

This is the built-in incidence relation for the type-(iii) competitor.

```
theorem cos_outerAngle : cos a.outerAngle = a.outerArgument := by
  rw [outerAngle, Real.cos_arccos a.outerArgument_mem.1.le
    a.outerArgument_mem.2.le]

/-- CMV's type-(iii) incidence equation, derived rather than assumed. -/
```

TypeThreeAssembly.cos_sideArcAngle

Classification: supplement

Source: TypeThreeAssembly.lean:231-237

Declaration SHA-256: a8f9a8c72dcee4cc4dc59b4da10f2ae6d2741d7865a4b2faca10168fa17cc76a

Mathematical statement. The side-arc angle has cosine minus the shape parameter.

$$\cos(\pi - b_3) = -\cos b_3 = -(2h - 1).$$

```
theorem cos_sideArcAngle :
  cos a.sideArcAngle = -a.shapeParameter := by
  rw [sideArcAngle, Real.cos_pi_sub, a.cos_innerAngle]

/-- A full lower cap with the same radius and angle as the two translated
half-caps forming the strip-side pieces. Its area and arc length are exactly
the sums of those two congruent halves. -/
```

TypeThreeAssembly.exact_euclidean_leftArcParam_hausdorffMeasure

Classification: supplement

Source: TypeThreeAssembly.lean:742-778

Declaration SHA-256: c6dc496bb93e612df391bdcd13229ada6ddb03a05f7a4a85318498972631a8fc

Mathematical statement. The left type-(iii) parameter image has exact length R times the side angle.

Its angular span is γ , so its Euclidean Hausdorff length is $R\gamma$.

```
theorem exact_euclidean_leftArcParam_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
    (a.realizedLeftArcParam ''
      Icc (-a.radius * a.sideArcAngle) 0) =
  ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  have hfun : a.realizedLeftArcParam =
    sideArcComplexIsometryMap a.leftCenter ◦
    unitCircleArc a.radius := by
    funext s
    exact a.realizedLeftArcParam_eq s
  have huv : -a.radius * a.sideArcAngle < 0 := by
    nlinarith [a.radius_pos, a.sideArcAngle_pos]
  have hspan :
    0 - (-a.radius * a.sideArcAngle) < 2 * π * a.radius := by
    have hangle : a.sideArcAngle < 2 * π := by
      linarith [a.sideArcAngle_lt_pi, Real.pi_pos]
    have hmul :=
      mul_lt_mul_of_pos_left hangle a.radius_pos
    nlinarith
```

```

calc
  _ = (μH[1] : Measure EuclideanPlane)
    (sideArcComplexIsometryMap a.leftCenter ''
      (unitCircleArc a.radius ''
        Icc (-a.radius * a.sideArcAngle) 0)) := by
  rw [hfun]
  congr 1
  simp only [Function.comp_apply] using (Set.image_image _ _ _).symm
  _ = (μH[1] : Measure ℂ)
    (unitCircleArc a.radius ''
      Icc (-a.radius * a.sideArcAngle) 0) :=
    (isometry_sideArcComplexIsometryMap a.leftCenter).hausdorffMeasure_image
      (Or.inl (by norm_num)) _
  _ = ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  simp using
    (hausdorffMeasure_unitCircleArc_image_eq_of_span_lt_two_pi
      a.radius_pos huv hspan)

```

TypeThreeAssembly.exact_euclidean_rightArcParam_hausdorffMeasure

Classification: supplement

Source: TypeThreeAssembly.lean:779-816

Declaration SHA-256: f1f573c6c931308699d4611129bc237b3c89fe21d3a82fbb640a2bcaf452f054

Mathematical statement. The right type-(iii) parameter image has length $R \gamma$.

The radius- R signed-arclength circle theorem applies to its parameter interval of span $R\gamma$; the placement isometry preserves this length.

```

theorem exact_euclidean_rightArcParam_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
    (a.realizedRightArcParam ''
      Icc 0 (a.radius * a.sideArcAngle)) =
  ENNReal.ofReal (a.radius * a.sideArcAngle) := by
have hfun : a.realizedRightArcParam =
  sideArcComplexIsometryMap a.rightCenter ◦
    unitCircleArc a.radius := by
  funext s
  exact a.realizedRightArcParam_eq s
have huv : 0 < a.radius * a.sideArcAngle :=
  mul_pos a.radius_pos a.sideArcAngle_pos
have hspan :
  a.radius * a.sideArcAngle - 0 < 2 * π * a.radius := by
  have hangle : a.sideArcAngle < 2 * π := by
    linarith [a.sideArcAngle_lt_pi, Real.pi_pos]
  have hmul :=
    mul_lt_mul_of_pos_left hangle a.radius_pos
  nlinarith
calc
  _ = (μH[1] : Measure EuclideanPlane)
    (sideArcComplexIsometryMap a.rightCenter ''
      (unitCircleArc a.radius ''
        Icc 0 (a.radius * a.sideArcAngle))) := by
  rw [hfun]
  congr 1
  simp only [Function.comp_apply] using (Set.image_image _ _ _).symm
  _ = (μH[1] : Measure ℂ)
    (unitCircleArc a.radius ''
      Icc 0 (a.radius * a.sideArcAngle)) :=
    (isometry_sideArcComplexIsometryMap a.rightCenter).hausdorffMeasure_image
      (Or.inl (by norm_num)) _
  _ = ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  simp using
    (hausdorffMeasure_unitCircleArc_image_eq_of_span_lt_two_pi
      a.radius_pos huv hspan)

/-- Exact Euclidean  $H^1$  mass of the left strip-side trace. -/

```

TypeThreeAssembly.frontier_carrier_subset_boundaryTrace

Classification: supplement

Source: TypeThreeAssembly.lean:2443-2458

Declaration SHA-256: e4a9f2900fa53b9204f0a28ca553872a35240978918b3cf32712160073f759cb

Mathematical statement. The type-(iii) carrier has no frontier outside the listed traces.

Internal glued interfaces are excluded by local interior arguments.

```
theorem frontier_carrier_subset_boundaryTrace :
  frontier a.carrier  $\subseteq$  a.boundaryTrace := by
  intro p hp
  have htop := frontier_union_subset a.coreCarrier a.outerCap.carrier hp
  rcases htop with hcore | hcap
  · rcases a.frontier_coreCarrier_subset hcore.1 with
    hleft | hright | hchord | hbottom
    · exact 0r.inl hleft
    · exact 0r.inr (0r.inl hright)
    · exact a.outerChord_to_boundaryTrace hchord hp
    · exact 0r.inr (0r.inr (0r.inr hbottom))
  · rw [a.outerCap.frontier_carrier] at hcap
    rcases hcap.2 with harc | hchord
    · exact 0r.inr (0r.inr (0r.inl harc))
    · exact a.outerChord_to_boundaryTrace hchord hp
```

TypeThreeAssembly.innerAngle_lt_pi

Classification: supplement

Source: TypeThreeAssembly.lean:168-170

Declaration SHA-256: 31385b2dc48409bb1698d05b2b8cedcea6151f9ce4bedc38534228b77f838b5c

Mathematical statement. The inner type-(iii) angle is below pi.

Its arccosine argument is strictly above minus one.

```
theorem innerAngle_lt_pi : a.innerAngle <  $\pi$  :=
  Real.arccos_lt_pi.mpr a.shapeParameter_mem.1
```

TypeThreeAssembly.innerAngle_pos

Classification: supplement

Source: TypeThreeAssembly.lean:165-167

Declaration SHA-256: e78599fffc8793999ca239c142df39a815df5b9220a19bd2bd882d169b85658b5

Mathematical statement. The inner type-(iii) angle is positive.

Its arccosine argument is strictly below one.

```
theorem innerAngle_pos : 0 < a.innerAngle :=
  Real.arccos_pos.mpr a.shapeParameter_mem.2
```

TypeThreeAssembly.innerAngle_semicircular

Classification: supplement

Source: TypeThreeAssembly.lean:370-373

Declaration SHA-256: 136b95db3cfbe6ffba49a9d377637e6fd44e92cfd37653b368ff453e7d5af642

Mathematical statement. At $h=1/2$ the inner angle is $\pi/2$.

The shape parameter is zero and $\arccos 0 = \pi/2$.

```
theorem innerAngle_semicircular (hh : a.h = 1 / 2) :  
  a.innerAngle =  $\pi / 2$  := by  
  simp [innerAngle, shapeParameter, hh]
```

TypeThreeAssembly.innerSin_pos

Classification: supplement

Source: TypeThreeAssembly.lean:216-218

Declaration SHA-256: 88694b992a38f1dff3c67f32d082f4eb742c98f01874e62941887bfa3c13f04d

Mathematical statement. The inner angle has positive sine.

The same open principal-angle range gives positivity.

```
theorem innerSin_pos : 0 < sin a.innerAngle :=  
  sin_pos_of_pos_of_lt_pi a.innerAngle_pos a.innerAngle_lt_pi
```

TypeThreeAssembly.isClosed_bottomSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1589-1594

Declaration SHA-256: 4502cfa8cbb9cfeeb9921da9cbccfe0b4a2a7c969d9473aede4beb6ee9ada554

Mathematical statement. The possibly degenerate bottom segment is closed.

It is a fixed-height closed horizontal interval.

```
theorem isClosed_bottomSegmentCarrier : IsClosed a.bottomSegmentCarrier := by  
  unfold bottomSegmentCarrier  
  exact (isClosed_eq continuous_snd continuous_const).inter  
    (isClosed_le continuous_fst.abs continuous_const)  
  
/-- The exact complete trace of the type-(iii) boundary. -/
```

TypeThreeAssembly.isClosed_carrier

Classification: supplement

Source: TypeThreeAssembly.lean:1307-1309

Declaration SHA-256: 86d49c14fed38ff9d123ec7002dc853d7ab13d112817308f4d1ef5ac7d463980

Mathematical statement. The full type-(iii) carrier is closed.

The core and upper cap are closed.

```
theorem isClosed_carrier : IsClosed a.carrier := by  
  exact a.isClosed_coreCarrier.union a.outerCap.isClosed_carrier
```

TypeThreeAssembly.isClosed_coreCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1303-1306

Declaration SHA-256: c084bdc57b0172b2f03efa14200c62e0c7ef9b7e0265bac3ea60cec5c75fc390

Mathematical statement. The type-(iii) core is closed.

It is a finite union of the closed rectangle and side segments.


```
theorem isClosed_coreCarrier : IsClosed a.coreCarrier := by
  exact (a.isClosed_rectangleCarrier.union a.isClosed_leftSegmentCarrier).union
    a.isClosed_rightSegmentCarrier
```

TypeThreeAssembly.isClosed_leftSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1289-1295

Declaration SHA-256: 9a06aa24438ad23f7c7f01c4c87ea6fc01e48b3ecec4d358799e726bea34abb1

Mathematical statement. The left type-(iii) disk segment is closed.

Its disk, side, and strip inequalities are all closed conditions.

```
theorem isClosed_leftSegmentCarrier : IsClosed a.leftSegmentCarrier := by
  unfold leftSegmentCarrier
  exact (isClosed_disk a.leftCenter a.radius).inter
    ((isClosed_le continuous_fst continuous_const).inter
      ((isClosed_le continuous_const continuous_snd).inter
        (isClosed_le continuous_snd continuous_const)))
```

TypeThreeAssembly.isClosed_rectangleCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1282-1288

Declaration SHA-256: a040fbd7853bc5d56529ed0d45def53a17db2084e9e6f891db5067b23f32a560

Mathematical statement. The type-(iii) central rectangle is closed.

It is a finite intersection of closed coordinate half-planes.

```
theorem isClosed_rectangleCarrier : IsClosed a.rectangleCarrier := by
  unfold rectangleCarrier
  exact ((isClosed_le continuous_const continuous_fst).inter
    (isClosed_le continuous_fst continuous_const)).inter
    ((isClosed_le continuous_const continuous_snd).inter
      (isClosed_le continuous_snd continuous_const))
```

TypeThreeAssembly.isClosed_rightSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1296-1302

Declaration SHA-256: 97ec89af687cf23451a62fdd3fb4816f90150533b2e6c5aca3fc7cb1fe8e300f

Mathematical statement. The right type-(iii) disk segment is closed.

The reflected defining inequalities again specify closed sets.

```
theorem isClosed_rightSegmentCarrier : IsClosed a.rightSegmentCarrier := by
  unfold rightSegmentCarrier
  exact (isClosed_disk a.rightCenter a.radius).inter
    ((isClosed_le continuous_const continuous_fst).inter
      ((isClosed_le continuous_const continuous_snd).inter
        (isClosed_le continuous_snd continuous_const)))
```

TypeThreeAssembly.isometry_sideArcComplexIsometryMap

Classification: supplement

Source: TypeThreeAssembly.lean:653-664

Declaration SHA-256: 5f9f98564eebaed09c3cd958295e743ceabd92f2b665793225e0fb64c4812ba4

Mathematical statement. The side-arc placement map is an isometry.

The chosen translation and orientation preserve Euclidean distances.

```
theorem isometry_sideArcComplexIsometryMap (center : PlanePoint) :
  Isometry (sideArcComplexIsometryMap center) := by
  apply Isometry.of_dist_eq
  intro z w
  rw [WithLp.prod_dist_eq_add (by norm_num)]
  simp only [sideArcComplexIsometryMap]
  norm_num [Real.dist_eq, sq_abs]
  rw [← Real.sqrt_eq_rpow, Complex.dist_eq_re_im]
  congr 1
  ring

/-- Euclidean realization of the left signed-arclength parameter. -/
```

TypeThreeAssembly.leftArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:120-126

Declaration SHA-256: 4f29c060ae66edf060cbefb1375e17d2584dc8c159343c7bce4a1ff009d7632c

Mathematical statement. Signed-arclength parametrization of the left side arc.

For $s \in [-Ry, 0]$, use $(-d + R\sin(s/R), R - 1 - R\cos(s/R))$.

```
def leftArcParam (s : ℝ) : PlanePoint :=
  (a.leftCenter.1 + a.radius * sin (s / a.radius),
   a.leftCenter.2 - a.radius * cos (s / a.radius))

/-- Signed-arclength parameter for the right strip-side half-circle. The
parameter runs from the bottom point at zero to the upper junction at
`radius * sideArcAngle`. -/
```

TypeThreeAssembly.leftArcParam_image_Icc

Classification: supplement

Source: TypeThreeAssembly.lean:446-546

Declaration SHA-256: a53d3bd58a5cf0ca117ca4045079faea921dc67e709030e1bc93b3bc04963479

Mathematical statement. The left side parameter interval traces exactly the left side boundary.

The full image identity is proved on $[-Ry, 0]$.

```
theorem leftArcParam_image_Icc :
  a.leftArcParam ''
    Icc (-a.radius * a.sideArcAngle) 0 = a.leftArcTrace := by
  ext p
  constructor
  · rintro (s, hs, rfl)
    have hu :
      s / a.radius ∈ Icc (-a.sideArcAngle) 0 := by
      constructor
      · apply (le_div_iff 0 a.radius_pos).2
        nlinarith [hs.1, a.radius_pos]
      · exact div_nonpos_of_nonpos_of_nonneg hs.2 a.radius_pos.le
```

```

have hsin_nonpos : sin (s / a.radius) ≤ 0 :=
  Real.sin_nonpos_of_nonpos_of_neg_pi_le hu.2
  (by linarith [hu.1, a.sideArcAngle_lt_pi])
have habs : |s / a.radius| ≤ a.sideArcAngle := by
  rw [abs_le]
  exact ⟨hu.1, hu.2.trans a.sideArcAngle_pos.le⟩
have hcos :
  cos a.sideArcAngle ≤ cos (s / a.radius) := by
  simp only [Real.cos_abs] using
    (Real.cos_le_cos_of_nonneg_of_le_pi
      (abs_nonneg (s / a.radius)) a.sideArcAngle_lt_pi.le habs)
have hcircle := Real.sin_sq_add_cos_sq (s / a.radius)
change
  ((a.leftCenter.1 + a.radius * sin (s / a.radius)) -
    a.leftCenter.1) ^ 2 +
    ((a.leftCenter.2 - a.radius * cos (s / a.radius)) -
      a.leftCenter.2) ^ 2 = a.radius ^ 2 ∧
  a.leftCenter.1 + a.radius * sin (s / a.radius) ≤
    a.leftCenter.1 ∧
  (-1 : ℝ) ≤
    a.leftCenter.2 - a.radius * cos (s / a.radius) ∧
  a.leftCenter.2 - a.radius * cos (s / a.radius) ≤ 1
refine (by nlinarith [sq_nonneg a.radius], ?_, ?_, ?_)
· nlinarith [mul_nonpos_of_nonneg_of_nonpos a.radius_pos.le hsin_nonpos]
· change
  (-1 : ℝ) ≤
    (a.radius - 1) - a.radius * cos (s / a.radius)
  nlinarith [mul_le_mul_of_nonneg_left
    (Real.cos_le_one (s / a.radius)) a.radius_pos.le]
· change
  (a.radius - 1) - a.radius * cos (s / a.radius) ≤ 1
  nlinarith [mul_le_mul_of_nonneg_left hc cos a.radius_pos.le,
    a.radius_mul_cos_sideArcAngle]
· intro hp
  change
    (p.1 - a.leftCenter.1) ^ 2 +
      (p.2 - a.leftCenter.2) ^ 2 = a.radius ^ 2 ∧
    p.1 ≤ a.leftCenter.1 ∧
    (-1 : ℝ) ≤ p.2 ∧ p.2 ≤ 1 at hp
let z : ℂ :=
  ⟨a.leftCenter.2 - p.2, p.1 - a.leftCenter.1⟩
have hnorm_sq : ||z|| ^ 2 = a.radius ^ 2 := by
  rw [Complex.norm_sq, Complex.normSq_apply]
  simp only [z]
  nlinarith [hp.1]
have hnorm : ||z|| = a.radius := by
  nlinarith [norm_nonneg z, a.radius_pos]
have hsin :
  a.radius * sin z.arg = p.1 - a.leftCenter.1 := by
  simp [hnorm, z] using Complex.norm_mul_sin_arg z
have hcos :
  a.radius * cos z.arg = a.leftCenter.2 - p.2 := by
  simp [hnorm, z] using Complex.norm_mul_cos_arg z
have hcos_le : cos a.sideArcAngle ≤ cos z.arg := by
  have hmul :
    a.radius * cos a.sideArcAngle ≤ a.radius * cos z.arg := by
    rw [a.radius_mul_cos_sideArcAngle, hcos]
    change a.radius - 2 ≤ (a.radius - 1) - p.2
    linarith [hp.2.2.2]
  nlinarith [hmul, a.radius_pos]
have harg : |z.arg| ≤ a.sideArcAngle := by
  by_contra h
  have hlt : a.sideArcAngle < |z.arg| := lt_of_not_ge h
  have hc := Real.cos_lt_cos_of_nonneg_of_le_pi
    a.sideArcAngle_pos.le (Complex.abs_arg_le_pi z) hlt
  rw [Real.cos_abs] at hc
  linarith
have harg_nonpos : z.arg ≤ 0 := by
  by_contra h
  have harg_pos : 0 < z.arg := lt_of_not_ge h
  have harg_lt_pi : z.arg < π :=
    lt_of_le_of_lt (le_trans (le_abs_self z.arg) harg)

```

```

    a.sideArcAngle_lt_pi
    have hsin_pos := Real.sin_pos_of_pos_of_lt_pi harg_pos harg_lt_pi
    nlinarith [a.radius_pos, hp.2.1]
    refine (a.radius * z.arg, ?_, ?_)
  · constructor
  · nlinarith [a.radius_pos, (abs_le.mp harg).1]
  · exact mul_nonpos_of_nonneg_of_nonpos a.radius_pos.le harg_nonpos
  · have hangle : a.radius * z.arg / a.radius = z.arg := by
      field_simp [a.radius_pos.ne']
    apply Prod.ext
  · simp only [leftArcParam, hangle]
    linarith
  · simp only [leftArcParam, hangle]
    linarith

/-- The right signed-arclength parameter fills exactly the right strip-side
trace, on every geometric branch (including the semicircular transition). -/

```

TypeThreeAssembly.leftArcTrace

Classification: supplement

Source: TypeThreeAssembly.lean:104-110

Declaration SHA-256: cfb74d8a54830ee4e695b37318a8d8c8a086063cc569f3e8d4fa167da3d5d0b9

Mathematical statement. The exposed left circle trace restricted to the strip.

It satisfies the left-circle equality, $x \leq -d$, and $-1 \leq y \leq 1$.

```

def leftArcTrace : Set PlanePoint :=
  {p | (p.1 - a.leftCenter.1) ^ 2 + (p.2 - a.leftCenter.2) ^ 2 =
    a.radius ^ 2 ^
    p.1 ≤ a.leftCenter.1 ^
    (-1 : ℝ) ≤ p.2 ^ p.2 ≤ 1}

/-- The exact right circular trace of the type-(iii) strip-side boundary. -/

```

TypeThreeAssembly.leftCenter

Classification: supplement

Source: TypeThreeAssembly.lean:70-72

Declaration SHA-256: ae634544d50a625f2da0ab83dddef536c2004aef8a002e19ec3ff792534bdf2

Mathematical statement. Center of the left type-(iii) side circle.

The point is $(-d, R - 1)$.

```

def leftCenter : PlanePoint := (-a.sideHalfWidth, a.radius - 1)

/-- Source circle center for the right strip arc. -/

```

TypeThreeAssembly.leftSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:85-91

Declaration SHA-256: 7a6b1fb5a536fc1614b926138e279c79369045c2b292e00c145e1a0e699dbced

Mathematical statement. The left disk segment inside the strip.

Intersect the radius- R disk centered at $(-d, R - 1)$ with $x \leq -d$ and $-1 \leq y \leq 1$.

```

def leftSegmentCarrier : Set PlanePoint :=
  {p | (p.1 - a.leftCenter.1) ^ 2 + (p.2 - a.leftCenter.2) ^ 2 ≤

```

```

    a.radius ^ 2} ∩
    ({p | p.1 ≤ a.leftCenter.1} ∩
    ({p | (-1 : ℝ) ≤ p.2} ∩ {p | p.2 ≤ 1}))

/-- Closed right disk segment of `K₃`. -/

```

TypeThreeAssembly.leftUpperJunction

Classification: supplement

Source: TypeThreeAssembly.lean:946-957

Declaration SHA-256: a660a79d351317f739116ce079a065c0c8bced856fae9b73442ad8d1339dd7ec

Mathematical statement. The left side arc meets the upper cap at the left upper junction.

At the initial side parameter, the coordinate formula gives $(-R\sin a_3, 1)$.

```

theorem leftUpperJunction :
  (a.leftCenter.1 - a.radius * sin a.innerAngle, 1) =
    a.outerCap.leftEndpoint := by
  rw [a.outerCap_leftEndpoint]
  change
    (-a.sideHalfWidth - a.radius * sin a.innerAngle, 1) =
      (-a.radius * sin a.outerAngle, 1)
  rw [sideHalfWidth]
  ring_nf

/-- The right strip arc and exterior cap use the same upper junction
coordinate. -/

```

TypeThreeAssembly.measurableSet_carrier

Classification: supplement

Source: TypeThreeAssembly.lean:1310-1316

Declaration SHA-256: 25696aa660c5c71e8ce3e6a179a069d25633bc99ff664ad8e93ea6e319cb501f

Mathematical statement. The full type-(iii) carrier is measurable.

Closedness in the coordinate plane gives Borel measurability.

```

theorem measurableSet_carrier : MeasurableSet a.carrier :=
  a.isClosed_carrier.measurableSet

/-! ## Carrier-derived weighted area -/

/-- The left half of the standard side cap. Translating it horizontally gives
the actual left strip-side segment. -/

```

TypeThreeAssembly.measurableSet_euclidean_bottomSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:2855-2860

Declaration SHA-256: ac873340c9031353c9d26e2404b771c8a0ff1b638ea75b1a24ed627ed5bc802d

Mathematical statement. The Euclidean bottom-segment carrier is measurable.

It is compact, including when it is a singleton.

```

theorem measurableSet_euclidean_bottomSegmentCarrier :
  MeasurableSet
    (planeEuclideanHomeomorph '' a.bottomSegmentCarrier) :=

```

```
(planeEuclideanHomeomorph.isClosedMap _  
  a.isClosed_bottomSegmentCarrier).measurableSet
```

TypeThreeAssembly.measurableSet_euclidean_leftArcTrace

Classification: supplement

Source: TypeThreeAssembly.lean:732-736

Declaration SHA-256: 791dc1e2fe6e2d0cb10c51a3bd4f4018c5ee53a25eeef24b22bc0d4e6f0f1192

Mathematical statement. The Euclidean type-(iii) left trace is measurable.

It is a compact continuous arc image.

```
theorem measurableSet_euclidean_leftArcTrace :  
  MeasurableSet (planeEuclideanHomeomorph '' a.leftArcTrace) := by  
  rw [← a.realizedLeftArcParam_image_Icc]  
  exact (isCompact_Icc.image a.continuous_realizedLeftArcParam).measurableSet
```

TypeThreeAssembly.measurableSet_euclidean_rightArcTrace

Classification: supplement

Source: TypeThreeAssembly.lean:737-741

Declaration SHA-256: 64197e0a340a1643ad8bc1525fe961f5b8792a0e7cb9321824fa2436e9d02bb4

Mathematical statement. The Euclidean type-(iii) right trace is measurable.

The compact parameter image is Borel.

```
theorem measurableSet_euclidean_rightArcTrace :  
  MeasurableSet (planeEuclideanHomeomorph '' a.rightArcTrace) := by  
  rw [← a.realizedRightArcParam_image_Icc]  
  exact (isCompact_Icc.image a.continuous_realizedRightArcParam).measurableSet
```

TypeThreeAssembly.one_lt_radius

Classification: supplement

Source: TypeThreeAssembly.lean:148-152

Declaration SHA-256: 4e892eef63bc8252d5a243e2b5495e748b55fa956d516b3510c9e5289ec264d9

Mathematical statement. The type-(iii) radius exceeds one.

The regular curvature bound is $h < 1$.

```
theorem one_lt_radius : 1 < a.radius := by  
  rw [radius]  
  apply (lt_div_iff₀ a.h_pos).2  
  nlinarith [a.h_lt_one]
```

TypeThreeAssembly.outerAngle_lt_pi

Classification: supplement

Source: TypeThreeAssembly.lean:174-177

Declaration SHA-256: ce135b6a3dc191fb33efd487aab9da31365b8ec038647a624b62a436ba911d34

Mathematical statement. The exterior type-(iii) angle is below pi.

This verifies the full circular-cap constructor domain.

```
theorem outerAngle_lt_pi : a.outerAngle <  $\pi$  :=
  Real.arccos_lt_pi.mpr a.outerArgument_mem.1

/-- The principal-angle definitions recover the source strip angle exactly. -/
```

TypeThreeAssembly.outerAngle_pos

Classification: supplement

Source: TypeThreeAssembly.lean:171-173

Declaration SHA-256: 760f9b90fd549c278aa760178ec027d7c643f8b5bd6d4bf942c86f6a65bb0e28

Mathematical statement. The exterior type-(iii) angle is positive.

The normalized incidence argument lies strictly below one.

```
theorem outerAngle_pos : 0 < a.outerAngle :=
  Real.arccos_pos.mpr a.outerArgument_mem.2
```

TypeThreeAssembly.outerAngle_semicircular

Classification: supplement

Source: TypeThreeAssembly.lean:374-377

Declaration SHA-256: 9ff26a1092e7089058ecf98f24fecce78972031b475870c0e4d30009e831f8ee

Mathematical statement. At $h=1/2$ the exterior angle is also $\pi/2$.

The incidence-adjusted argument is again zero for every $\lambda > 1$.

```
theorem outerAngle_semicircular (hh : a.h = 1 / 2) :
  a.outerAngle =  $\pi$  / 2 := by
  simp [outerAngle, outerArgument, shapeParameter, hh]
```

TypeThreeAssembly.outerArgument

Classification: supplement

Source: TypeThreeAssembly.lean:56-58

Declaration SHA-256: d6cfc025aa1368c77ccefa6b050dbf8bfa113c35569b4434625ec2d735b91f41

Mathematical statement. The incidence-adjusted exterior arccosine argument.

It is $(2h - 1)/\lambda$.

```
def outerArgument :  $\mathbb{R}$  := a.shapeParameter / lam

/-- The exterior half-central-angle `a_3`. -/
```

TypeThreeAssembly.outerArgument_mem

Classification: supplement

Source: TypeThreeAssembly.lean:156-164

Declaration SHA-256: fc2a8b1032448c01ece41eff0bcd5d748f70ee556d5721867bb36b59d56ce63a

Mathematical statement. The exterior arccosine argument lies in $(-1,1)$.

Dividing $q \in (-1, 1)$ by $\lambda > 1$ keeps it strictly inside that interval.

```
theorem outerArgument_mem : -1 < a.outerArgument ^ a.outerArgument < 1 := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) a.density_jump
```

```

have hx := a.shapeParameter_mem
constructor
· apply (lt_div_iff₀ hlam_pos).2
  nlinarith [a.density_jump, hx.1]
· apply (div_lt_iff₀ hlam_pos).2
  nlinarith [a.density_jump, hx.2]

```

TypeThreeAssembly.outerCap_center

Classification: supplement

Source: TypeThreeAssembly.lean:929-933

Declaration SHA-256: 1057a7a30ad6e7558c1c0e586fc0147d20f1fc01e6c3e508acda901e55191c4c

Mathematical statement. The constructed upper cap has the prescribed upper-circle center.

The cap placement formula gives $(0, 1 - R\cos\alpha_3)$.

```

theorem outerCap_center : a.outerCap.center = a.upperCenter := by
  change (0, 1 - a.outerCap.radius * cos a.outerAngle) =
    (0, 1 - a.radius * cos a.outerAngle)
  rw [a.outerCap_radius]

```

TypeThreeAssembly.outerCap_leftEndpoint

Classification: supplement

Source: TypeThreeAssembly.lean:934-938

Declaration SHA-256: 369e532ae6bbe742ee13471a1a8cc638a185e8c74c4f29cdde432417763b0fdb

Mathematical statement. The upper cap's left endpoint is its intended upper junction.

It is $(-R\sin\alpha_3, 1)$.

```

theorem outerCap_leftEndpoint :
  a.outerCap.leftEndpoint = (-a.radius * sin a.outerAngle, 1) := by
  simp [OneSidedCircularCap.leftEndpoint, outerCap]
  ring

```

TypeThreeAssembly.outerCap_radius

Classification: supplement

Source: TypeThreeAssembly.lean:419-422

Declaration SHA-256: f107696210fe89f5a746d54584303c70da482f05587bc229f2d37aaa9494b57f

Mathematical statement. The constructed exterior cap has radius R .

Its chord $2R\sin\alpha_3$ cancels the positive sine denominator in the cap-radius definition.

```

theorem outerCap_radius : a.outerCap.radius = a.radius := by
  change (2 * a.radius * sin a.outerAngle) / (2 * sin a.outerAngle) = a.radius
  field_simp [ne_of_gt a.outerSin_pos]

```

TypeThreeAssembly.outerCap_rightEndpoint

Classification: supplement

Source: TypeThreeAssembly.lean:939-945

Declaration SHA-256: 0bf0b00b58f0396d9b9b8bc9e08617bcd76cf3f0b76937e23b52dec1f9b5fe12

Mathematical statement. The upper cap's right endpoint is its intended upper junction.

It is $(Rsina_3, 1)$.

```
theorem outerCap_rightEndpoint :
  a.outerCap.rightEndpoint = (a.radius * sin a.outerAngle, 1) := by
  simp [OneSidedCircularCap.rightEndpoint, outerCap]
  ring

/-- The left strip arc and exterior cap use the same upper junction
coordinate. -/
```

TypeThreeAssembly.outerSin_pos

Classification: supplement

Source: TypeThreeAssembly.lean:213-215

Declaration SHA-256: feedd827e6203dedcb21f4f474b9fea36ae3ea708b33b644b944093c5f958959

Mathematical statement. The exterior angle has positive sine.

It lies in $(0, \pi)$ on every regular branch.

```
theorem outerSin_pos : 0 < sin a.outerAngle :=
  sin_pos_of_pos_of_lt_pi a.outerAngle_pos a.outerAngle_lt_pi
```

TypeThreeAssembly.pairwiseOverlap_subset_boundaryJoinSet

Classification: supplement

Source: TypeThreeAssembly.lean:1142-1257

Declaration SHA-256: f8736f4dd5a67fc4149e15b1ed3e3f21030a49e12643cd517b41158566d4664f

Mathematical statement. Intersections of distinct type-(iii) boundary pieces occur only at junctions.

There is no shared subarc of positive length, even when the bottom segment degenerates.

```
theorem pairwiseOverlap_subset_boundaryJoinSet :
  pairwiseOverlap a.leftArcTrace a.rightArcTrace
    (OneSidedCircularCap.arcTrace a.outerCap)
  a.bottomSegmentCarrier ⊆ a.boundaryJoinSet := by
  intro p hp
  simp only [pairwiseOverlap, mem_union, mem_inter_iff] at hp
  rcases hp with hLR | hLU | hLB | hRU | hRB | hUB
  · by_cases hw : a.sideHalfWidth = 0
    · have hx : p.1 = 0 := by
        have hl := hLR.1.2.1
        have hr := hLR.2.2.1
        simp only [leftCenter, rightCenter, hw, neg_zero] at hl hr
        linarith
      have hcircle := hLR.1.1
      rw [hx] at hcircle
      simp only [leftCenter, hw, neg_zero, sub_zero] at hcircle
      have hfactor :
        (p.2 + 1) * (p.2 - (2 * a.radius - 1)) = 0 := by
          nlinarith
      rcases mul_eq_zero.mp hfactor with hy | hy
      · have hy' : p.2 = -1 := by linarith
        have hpEq : p = (0, -1) := Prod.ext hx hy'
        simp [boundaryJoinSet, hpEq, hw]
      · have hy' : p.2 = 2 * a.radius - 1 := by linarith
        have hyUpper := hLR.1.2.2.2
        exfalse
        nlinarith [a.one_lt_radius]
    · have hwpos : 0 < a.sideHalfWidth :=
        lt_of_le_of_ne a.sideHalfWidth_nonneg (Ne.symm hw)
      have hl := hLR.1.2.1
      have hr := hLR.2.2.1
```

```

    simp only [leftCenter, rightCenter] at h1 hr
  exfalse
  linarith
· have hyCore : p.2 ≤ 1 := hLU.1.2.2.2
  have hyCap : (1 : ℝ) ≤ p.2 := by
    simp [outerCap] using hLU.2.2
  have hy : p.2 = 1 := le_antisymm hyCore hyCap
  have hdx : p.1 - a.leftCenter.1 ≤ 0 := by
    linarith [hLU.1.2.1]
  have hsq :
    (p.1 - a.leftCenter.1) ^ 2 =
      (a.radius * sin a.innerAngle) ^ 2 := by
    have hpCircle := hLU.1.1
    rw [hy] at hpCircle
    simp only [leftCenter] at hpCircle
    rw [leftCenter]
    nlinarith [a.side_circle_at_top]
  have habs := (sq_eq_sq_iff_abs_eq_abs
    (p.1 - a.leftCenter.1)
    (a.radius * sin a.innerAngle)).mp hsq
  rw [abs_of_nonpos hdx,
    abs_of_pos (mul_pos a.radius_pos a.innerSin_pos)] at habs
  have hdxEq :
    p.1 - a.leftCenter.1 =
      -(a.radius * sin a.innerAngle) := by linarith
  have hx : p.1 = -(a.radius * sin a.outerAngle) := by
    simp only [leftCenter] at hdxEq
    rw [sideHalfWidth] at hdxEq
    linarith
  have hpEq : p = (-a.radius * sin a.outerAngle, 1) := by
    apply Prod.ext
    · simp using hx
    · simp using hy
  simp [boundaryJoinSet, hpEq]
· have hy : p.2 = -1 := hLB.2.1
  have hbound : |p.1| ≤ a.sideHalfWidth := hLB.2.2
  have hxLower := (abs_le.mp hbound).1
  have hxUpper := hLB.1.2.1
  simp only [leftCenter] at hxUpper
  have hx : p.1 = -a.sideHalfWidth := by linarith
  have hpEq : p = (-a.sideHalfWidth, -1) := Prod.ext hx hy
  simp [boundaryJoinSet, hpEq]
· have hyCore : p.2 ≤ 1 := hRU.1.2.2.2
  have hyCap : (1 : ℝ) ≤ p.2 := by
    simp [outerCap] using hRU.2.2
  have hy : p.2 = 1 := le_antisymm hyCore hyCap
  have hdx : 0 ≤ p.1 - a.rightCenter.1 := by
    linarith [hRU.1.2.1]
  have hsq :
    (p.1 - a.rightCenter.1) ^ 2 =
      (a.radius * sin a.innerAngle) ^ 2 := by
    have hpCircle := hRU.1.1
    rw [hy] at hpCircle
    simp only [rightCenter] at hpCircle
    rw [rightCenter]
    nlinarith [a.side_circle_at_top]
  have habs := (sq_eq_sq_iff_abs_eq_abs
    (p.1 - a.rightCenter.1)
    (a.radius * sin a.innerAngle)).mp hsq
  rw [abs_of_nonneg hdx,
    abs_of_pos (mul_pos a.radius_pos a.innerSin_pos)] at habs
  have hdxEq :
    p.1 - a.rightCenter.1 =
      a.radius * sin a.innerAngle := habs
  have hx : p.1 = a.radius * sin a.outerAngle := by
    simp only [rightCenter] at hdxEq
    rw [sideHalfWidth] at hdxEq
    linarith
  have hpEq : p = (a.radius * sin a.outerAngle, 1) := Prod.ext hx hy
  simp [boundaryJoinSet, hpEq]
· have hy : p.2 = -1 := hRB.2.1
  have hbound : |p.1| ≤ a.sideHalfWidth := hRB.2.2

```

```

have hxUpper := (abs_le.mp hbound).2
have hxLower := hRB.1.2.1
simp only [rightCenter] at hxLower
have hx : p.1 = a.sideHalfWidth := by linarith
have hpEq : p = (a.sideHalfWidth, -1) := Prod.ext hx hy
simp [boundaryJoinSet, hpEq]
· have hyCap : (1 : ℝ) ≤ p.2 := by
  simp [outerCap] using hUB.1.2
have hyBottom : p.2 = -1 := hUB.2.1
exfalse
linarith

```

```

/-- Pairwise primitive overlaps carry no Euclidean `H¹` mass. -/

```

TypeThreeAssembly.radius_mul_h

Classification: supplement

Source: TypeThreeAssembly.lean:197-200

Declaration SHA-256: e90d4095fef5e1e089527a179a53736b5dbf462b6149abecb5375e96b7380078

Mathematical statement. Radius times curvature equals one.

$Rh = (1/h)h = 1$ since $h > 0$.

```

theorem radius_mul_h : a.radius * a.h = 1 := by
  rw [radius]
  field_simp [ne_of_gt a.h_pos]

```

TypeThreeAssembly.radius_pos

Classification: supplement

Source: TypeThreeAssembly.lean:145-147

Declaration SHA-256: a610fc617d9a92f1154e4b9627bb16a0d909464945f9e22ccb900c975acb2c7d

Mathematical statement. The type-(iii) radius is positive.

$h > 0$ gives $R = 1/h > 0$.

```

theorem radius_pos : 0 < a.radius := by
  exact one_div_pos.mpr a.h_pos

```

TypeThreeAssembly.realizedLeftArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:665-668

Declaration SHA-256: 51fbf9517aa10415de53dc106db635c5f70b9c39860ba75778e696cdebfd2d6c3

Mathematical statement. The type-(iii) left arc parameter in Euclidean L2 coordinates.

It is the coordinate parameter followed by the Euclidean realization.

```

noncomputable def realizedLeftArcParam (s : ℝ) : EuclideanPlane :=
  planeEuclideanHomeomorph (a.leftArcParam s)

/-- Euclidean realization of the right signed-arclength parameter. -/

```

TypeThreeAssembly.realizedLeftArcParam_eq

Classification: supplement

Source: TypeThreeAssembly.lean:672-680

Declaration SHA-256: 538a7925bd9a6977d5e74933bd5f951885abb82dc534e3d102db01bf6aa01a1e

Mathematical statement. The realized left parameter equals its placed scaled complex-circle form.

The identity transfers the radius-scaled, signed-arclength circle measure theorem to this side arc.

```
lemma realizedLeftArcParam_eq (s : ℝ) :
  a.realizedLeftArcParam s =
    sideArcComplexIsometryMap a.leftCenter
      (unitCircleArc a.radius s) := by
  rw [show a.realizedLeftArcParam s =
    WithLp.toLp 2 (a.leftArcParam s) by rfl]
  simp [leftArcParam, sideArcComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im]
```

TypeThreeAssembly.realizedLeftArcParam_image_Icc

Classification: supplement

Source: TypeThreeAssembly.lean:710-720

Declaration SHA-256: 06e6662a39cd836487840418c7224db9c1b8779c37aa295aab4e1109503a76c7

Mathematical statement. The realized left parameter image equals the Euclidean left trace.

Transport the exact coordinate trace identity through the homeomorphism.

```
theorem realizedLeftArcParam_image_Icc :
  a.realizedLeftArcParam ''
    Icc (-a.radius * a.sideArcAngle) 0 =
    planeEuclideanHomeomorph '' a.leftArcTrace := by
  calc
  _ = planeEuclideanHomeomorph ''
    (a.leftArcParam '' Icc (-a.radius * a.sideArcAngle) 0) := by
  rw [Set.image_image]
  rfl
  _ = _ := by rw [a.leftArcParam_image_Icc]
```

TypeThreeAssembly.realizedRightArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:669-671

Declaration SHA-256: 8a963395af5bf096a9c5748471f90364728d8fe511c4e2cc0591bd2068d9e016

Mathematical statement. The type-(iii) right arc parameter in Euclidean L2 coordinates.

This is the parameter image used for actual Hausdorff length.

```
noncomputable def realizedRightArcParam (s : ℝ) : EuclideanPlane :=
  planeEuclideanHomeomorph (a.rightArcParam s)
```

TypeThreeAssembly.realizedRightArcParam_eq

Classification: supplement

Source: TypeThreeAssembly.lean:681-689

Declaration SHA-256: 3339fa9296e8c62289cd021c3334658bb86b73401d06cffbaa790d7423ffd93e

Mathematical statement. The realized right parameter equals its complex-circle form.

The right-side placement has the same scale and angular span.

```
lemma realizedRightArcParam_eq (s : ℝ) :
  a.realizedRightArcParam s =
    sideArcComplexIsometryMap a.rightCenter
      (unitCircleArc a.radius s) := by
  rw [show a.realizedRightArcParam s =
    WithLp.toLp 2 (a.rightArcParam s) by rfl]
  simp [rightArcParam, sideArcComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im]
```

TypeThreeAssembly.realizedRightArcParam_image_Icc

Classification: supplement

Source: TypeThreeAssembly.lean:721-731

Declaration SHA-256: 9469dc298ecad88323016b9907111e322a864f0831a991b637ca11824881af16

Mathematical statement. The realized right parameter image equals the Euclidean right trace.

This identifies the measured image with the actual boundary piece.

```
theorem realizedRightArcParam_image_Icc :
  a.realizedRightArcParam ''
    Icc 0 (a.radius * a.sideArcAngle) =
    planeEuclideanHomeomorph '' a.rightArcTrace := by
  calc
    _ = planeEuclideanHomeomorph ''
      (a.rightArcParam '' Icc 0 (a.radius * a.sideArcAngle)) := by
      rw [Set.image_image]
      rfl
    _ = _ := by rw [a.rightArcParam_image_Icc]
```

TypeThreeAssembly.rectangleArea

Classification: supplement

Source: TypeThreeAssembly.lean:273-277

Declaration SHA-256: 3314187b7b48ab958a7d8415bcf10afdb2dd85317d354dfd049f5344c2569e90

Mathematical statement. The type-(iii) rectangle's area expression.

Width $2d$ and height two give $4d$.

```
def rectangleArea : ℝ := 4 * a.sideHalfWidth

/-- Component area of the modeled type-(iii) profile. The side-cap term is
the sum of the two congruent translated half-caps; the outer cap has density
`lam`. -/
```

TypeThreeAssembly.rectangleCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:80-84

Declaration SHA-256: d451775e4018b5bae444c6a8ed6911c5adcc23db4a0efc68ab71c655f2933477

Mathematical statement. The closed type-(iii) central rectangle.

It is $[-d, d] \times [-1, 1]$, possibly of zero width.

```
def rectangleCarrier : Set PlanePoint :=
  ({p | -a.sideHalfWidth ≤ p.1} ∩ {p | p.1 ≤ a.sideHalfWidth}) ∩
  ({p | (-1 : ℝ) ≤ p.2} ∩ {p | p.2 ≤ 1})
```

```
/-- Closed left disk segment of `K3`. -/
```

TypeThreeAssembly.rightArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:127-131

Declaration SHA-256: 74515bd4fc474dfa2c52a14662b8cc34aa0f40ef9d8185819e05d1de94ccf588

Mathematical statement. Signed-arclength parametrization of the right side arc.

For $s \in [0, R\gamma]$, use $(d + R\sin(s/R), R - 1 - R\cos(s/R))$.

```
def rightArcParam (s : ℝ) : PlanePoint :=
  (a.rightCenter.1 + a.radius * sin (s / a.radius),
   a.rightCenter.2 - a.radius * cos (s / a.radius))

/-- The possibly degenerate bottom segment. -/
```

TypeThreeAssembly.rightArcParam_image_Icc

Classification: supplement

Source: TypeThreeAssembly.lean:547-648

Declaration SHA-256: d47f88fb5eadc69e4f4bc7ce3bc201dc34d60ad570f34d3d29c4d0184ef0f8f1

Mathematical statement. The right side parameter interval traces exactly the right side boundary.

The interval is $[0, R\gamma]$ and includes both junction endpoints.

```
theorem rightArcParam_image_Icc :
  a.rightArcParam ''
    Icc 0 (a.radius * a.sideArcAngle) = a.rightArcTrace := by
  ext p
  constructor
  · rintro (s, hs, rfl)
    have hu :
      s / a.radius ∈ Icc 0 a.sideArcAngle := by
      constructor
      · exact div_nonneg hs.1 a.radius_pos.le
      · apply (div_le_iff a.radius_pos).2
        nlinarith [hs.2, a.radius_pos]
    have hsin_nonneg : 0 ≤ sin (s / a.radius) :=
      Real.sin_nonneg_of_nonneg_of_le_pi hu.1
      (hu.2.trans a.sideArcAngle_lt_pi.le)
    have habs : |s / a.radius| ≤ a.sideArcAngle := by
      rw [abs_le]
      exact (by linarith [hu.1, a.sideArcAngle_pos], hu.2)
    have hcos :
      cos a.sideArcAngle ≤ cos (s / a.radius) := by
      simp only [Real.cos_abs] using
        (Real.cos_le_cos_of_nonneg_of_le_pi
         (abs_nonneg (s / a.radius)) a.sideArcAngle_lt_pi.le habs)
    have hcircle := Real.sin_sq_add_cos_sq (s / a.radius)
    change
      ((a.rightCenter.1 + a.radius * sin (s / a.radius)) -
       a.rightCenter.1) ^ 2 +
      ((a.rightCenter.2 - a.radius * cos (s / a.radius)) -
       a.rightCenter.2) ^ 2 = a.radius ^ 2 ^
      a.rightCenter.1 ≤
      a.rightCenter.1 + a.radius * sin (s / a.radius) ^
      (-1 : ℝ) ≤
      a.rightCenter.2 - a.radius * cos (s / a.radius) ^
      a.rightCenter.2 - a.radius * cos (s / a.radius) ≤ 1
    refine (by nlinarith [sq_nonneg a.radius], ?, ?, ?)
    · nlinarith [mul_nonneg a.radius_pos.le hsin_nonneg]
```

```

· change
  (-1 : ℝ) ≤
    (a.radius - 1) - a.radius * cos (s / a.radius)
nlinarith [mul_le_mul_of_nonneg_left
  (Real.cos_le_one (s / a.radius)) a.radius_pos.le]
· change
  (a.radius - 1) - a.radius * cos (s / a.radius) ≤ 1
nlinarith [mul_le_mul_of_nonneg_left hcos a.radius_pos.le,
  a.radius_mul_cos_sideArcAngle]
· intro hp
change
  (p.1 - a.rightCenter.1) ^ 2 +
    (p.2 - a.rightCenter.2) ^ 2 = a.radius ^ 2 ^
    a.rightCenter.1 ≤ p.1 ^
    (-1 : ℝ) ≤ p.2 ^ p.2 ≤ 1 at hp
let z : ℂ :=
  (a.rightCenter.2 - p.2, p.1 - a.rightCenter.1)
have hnorm_sq : ||z|| ^ 2 = a.radius ^ 2 := by
  rw [Complex.sq_norm, Complex.normSq_apply]
simp only [z]
nlinarith [hp.1]
have hnorm : ||z|| = a.radius := by
  nlinarith [norm_nonneg z, a.radius_pos]
have hsin :
  a.radius * sin z.arg = p.1 - a.rightCenter.1 := by
  simpa [hnorm, z] using Complex.norm_mul_sin_arg z
have hcos :
  a.radius * cos z.arg = a.rightCenter.2 - p.2 := by
  simpa [hnorm, z] using Complex.norm_mul_cos_arg z
have hcos_le : cos a.sideArcAngle ≤ cos z.arg := by
  have hmul :
    a.radius * cos a.sideArcAngle ≤ a.radius * cos z.arg := by
    rw [a.radius_mul_cos_sideArcAngle, hcos]
    change a.radius - 2 ≤ (a.radius - 1) - p.2
    linarith [hp.2.2.2]
  nlinarith [hmul, a.radius_pos]
have harg : |z.arg| ≤ a.sideArcAngle := by
  by_contra h
  have hlt : a.sideArcAngle < |z.arg| := lt_of_not_ge h
  have hc := Real.cos_lt_cos_of_nonneg_of_le_pi
    a.sideArcAngle_pos.le (Complex.abs_arg_le_pi z) hlt
  rw [Real.cos_abs] at hc
  linarith
have harg_nonneg : 0 ≤ z.arg := by
  by_contra h
  have harg_neg : z.arg < 0 := lt_of_not_ge h
  have harg_gt_neg_pi : -π < z.arg :=
    lt_of_lt_of_le (neg_lt_neg a.sideArcAngle_lt_pi)
    (abs_le.mp harg).1
  have hsin_neg :=
    Real.sin_neg_of_neg_of_neg_pi_lt harg_neg harg_gt_neg_pi
  nlinarith [a.radius_pos, hp.2.1]
refine (a.radius * z.arg, ?_, ?_)
· constructor
· exact mul_nonneg a.radius_pos.le harg_nonneg
· nlinarith [a.radius_pos, (abs_le.mp harg).2]
· have hangle : a.radius * z.arg / a.radius = z.arg := by
  field_simp [a.radius_pos.ne']
  apply Prod.ext
· simp only [rightArcParam, hangle]
  linarith
· simp only [rightArcParam, hangle]
  linarith

```

```

/-- Euclidean isometry that places the standard signed-arclength circle at a
strip-side center, with the orientation used by both side parameters. -/

```

TypeThreeAssembly.rightArcTrace

Classification: supplement

Source: TypeThreeAssembly.lean:111-119

Declaration SHA-256: 828cebab0c56e67a2a3cfaae9e912a8a76f497287dcea6b589e1a22a40edd616

Mathematical statement. The exposed right circle trace restricted to the strip.

It is the reflected trace with $x \geq d$.

```
def rightArcTrace : Set PlanePoint :=
  {p | (p.1 - a.rightCenter.1) ^ 2 + (p.2 - a.rightCenter.2) ^ 2 =
    a.radius ^ 2 ^
    a.rightCenter.1 ≤ p.1 ^
    (-1 : ℝ) ≤ p.2 ^ p.2 ≤ 1}

/-- Signed-arclength parameter for the left strip-side half-circle. The
parameter runs from the upper junction at `-radius * sideArcAngle` to the
bottom point at zero. -/
```

TypeThreeAssembly.rightCenter

Classification: supplement

Source: TypeThreeAssembly.lean:73-75

Declaration SHA-256: c1d50740a1dfb76a9c424f19f6e78589f8a8ab930ab1727da5282aec42c3f25

Mathematical statement. Center of the right type-(iii) side circle.

The point is $(d, R - 1)$.

```
def rightCenter : PlanePoint := (a.sideHalfWidth, a.radius - 1)

/-- Source circle center for the exterior upper arc. -/
```

TypeThreeAssembly.rightSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:92-98

Declaration SHA-256: 889596cedc4d7d957005581540478e3a458f9f3d1f37c9af6b1a713eee05b078

Mathematical statement. The right disk segment inside the strip.

Intersect the reflected disk with $x \geq d$ and the closed strip.

```
def rightSegmentCarrier : Set PlanePoint :=
  {p | (p.1 - a.rightCenter.1) ^ 2 + (p.2 - a.rightCenter.2) ^ 2 ≤
    a.radius ^ 2} ∩
  ({p | a.rightCenter.1 ≤ p.1} ∩
   ({p | (-1 : ℝ) ≤ p.2} ∩ {p | p.2 ≤ 1}))

/-- The part of the type-(iii) carrier inside the closed strip. -/
```

TypeThreeAssembly.rightUpperJunction

Classification: supplement

Source: TypeThreeAssembly.lean:958-968

Declaration SHA-256: ae6c6a14cc5b76c0f3aa60155a1fb16c629e4d67ae173c7593aa69c135bbff39

Mathematical statement. The right side arc meets the upper cap at the right upper junction.

At the final right parameter, the point is $(R\sin a_3, 1)$.

```
theorem rightUpperJunction :
  (a.rightCenter.1 + a.radius * sin a.innerAngle, 1) =
    a.outerCap.rightEndpoint := by
  rw [a.outerCap.rightEndpoint]
  change
    (a.sideHalfWidth + a.radius * sin a.innerAngle, 1) =
      (a.radius * sin a.outerAngle, 1)
  rw [sideHalfWidth]
  ring_nf

/-- The core and upper cap overlap exactly along the upper cap's chord. -/
```

TypeThreeAssembly.shapeParameter_mem

Classification: supplement

Source: TypeThreeAssembly.lean:153-155

Declaration SHA-256: 018fe2d07fb99b9420ec19a75327dc616f0119968b1c22fb83265de47b1261df

Mathematical statement. The type-(iii) shape parameter lies in $(-1,1)$.

Apply the affine map $q = 2h - 1$ to $0 < h < 1$.

```
theorem shapeParameter_mem : -1 < a.shapeParameter ∧ a.shapeParameter < 1 := by
  constructor <.> simp only [shapeParameter] <.> linarith [a.h_pos, a.h_lt_one]
```

TypeThreeAssembly.sideArcAngle_lt_pi

Classification: supplement

Source: TypeThreeAssembly.lean:223-226

Declaration SHA-256: 27c4b01d9c8710ab675792367c9a16f3b7952720c452f471c97b46d487e08c08

Mathematical statement. The side-arc angle is below π .

The inner angle is strictly positive.

```
theorem sideArcAngle_lt_pi : a.sideArcAngle < π := by
  rw [sideArcAngle]
  linarith [a.innerAngle_pos]
```

TypeThreeAssembly.sideArcAngle_pos

Classification: supplement

Source: TypeThreeAssembly.lean:219-222

Declaration SHA-256: fb3e4cdd6ff568be2285a238c6293705a5d18ec17ca0000cf9fc5fd3be6f53e2

Mathematical statement. The complementary side-arc angle is positive.

$\gamma = \pi - b_3 > 0$ because $b_3 < \pi$.

```
theorem sideArcAngle_pos : 0 < a.sideArcAngle := by
  rw [sideArcAngle]
  linarith [a.innerAngle_lt_pi]
```

TypeThreeAssembly.sideArcComplexIsometryMap

Classification: supplement

Source: TypeThreeAssembly.lean:649-652

Declaration SHA-256: 2d2b0ae6442b1aeb7779e0e2c761b53071a3b1948418645b546ba443dcc3b322

Mathematical statement. The Euclidean placement map for the complex type-(iii) side arc.

It handles the rigid motion separately from radius and angular scaling.

```
noncomputable def sideArcComplexIsometryMap
  (center : PlanePoint) (z : ℂ) : EuclideanPlane :=
  WithLp.toLp 2 (center.1 + z.im, center.2 - z.re)
```

TypeThreeAssembly.sideCap_center

Classification: supplement

Source: TypeThreeAssembly.lean:430-438

Declaration SHA-256: 5c32cc16475c4c5293c8fcbce19421c65fe608e9916172122a7cb5ff16d6b389

Mathematical statement. The reference side-cap center agrees with the intended source placement.

The center calculation uses $R(2h - 1) = 2 - R$ and the complementary angle.

```
theorem sideCap_center :
  a.sideCap.center = (0, a.radius - 1) := by
  change
    (0, 1 + a.sideCap.radius * cos a.sideArcAngle) =
    (0, a.radius - 1)
  rw [a.sideCap_radius, a.cos_sideArcAngle, radius, shapeParameter]
  field_simp [ne_of_gt a.h_pos]
  ring
```

TypeThreeAssembly.sideCap_radius

Classification: supplement

Source: TypeThreeAssembly.lean:423-429

Declaration SHA-256: 860b391d38da56536b3934963761766a28238619240605477033823fc900d1af

Mathematical statement. The reference side cap has radius R .

Use its chord and $\sin y = \sin b_3$.

```
theorem sideCap_radius : a.sideCap.radius = a.radius := by
  change
    (2 * a.radius * sin a.innerAngle) /
    (2 * sin a.sideArcAngle) = a.radius
  rw [a.sin_sideArcAngle]
  field_simp [ne_of_gt a.innerSin_pos]
```

TypeThreeAssembly.sideHalfWidth_nonneg

Classification: supplement

Source: TypeThreeAssembly.lean:338-340

Declaration SHA-256: be70cb71b25ed2f43754461915097af9183cc7aac3504655dda98d8a67b605ec

Mathematical statement. The type-(iii) half-width is nonnegative.

Multiply the nonnegative sine gap by positive R .

```
theorem sideHalfWidth_nonneg : 0 ≤ a.sideHalfWidth := by
  exact mul_nonneg a.radius_pos.le (sub_nonneg.mpr a.sin_inner_le_sin_outer)
```

TypeThreeAssembly.sideHalfWidth_pos

Classification: supplement

Source: TypeThreeAssembly.lean:341-343

Declaration SHA-256: a769284707d6bf25198d747cd5846bae22f2f394cc9d09e601cfcab854579d7a

Mathematical statement. The half-width is positive away from $h=1/2$.

The sine gap is strict exactly off the semicircular transition.

```
theorem sideHalfWidth_pos (hh : a.h ≠ 1 / 2) : 0 < a.sideHalfWidth := by
  exact mul_pos a.radius_pos (sub_pos.mpr (a.sin_inner_lt_sin_outer hh))
```

TypeThreeAssembly.sin_inner_le_sin_outer

Classification: supplement

Source: TypeThreeAssembly.lean:303-311

Declaration SHA-256: def468198882d033570072298a83ae1d41fc003f0ecb9d0b4ee3a6b94e6a1d10

Mathematical statement. The inner-angle sine is no greater than the exterior-angle sine.

$|q/\lambda| \leq |q|$ implies $\sqrt{1-q^2} \leq \sqrt{1-(q/\lambda)^2}$.

```
theorem sin_inner_le_sin_outer : sin a.innerAngle ≤ sin a.outerAngle := by
  rw [innerAngle, outerAngle, Real.sin_arccos, Real.sin_arccos]
  apply Real.sqrt_le_sqrt
  have hlam_pos : 0 < lam := lt_trans (by norm_num) a.density_jump
  have habs : |a.outerArgument| ≤ |a.shapeParameter| := by
    rw [outerArgument, abs_div, abs_of_pos hlam_pos]
    exact div_le_self (abs_nonneg _) (le_of_lt a.density_jump)
  exact sub_le_sub_left (sq_le_sq.mpr habs) 1
```

TypeThreeAssembly.sin_inner_lt_sin_outer

Classification: supplement

Source: TypeThreeAssembly.lean:312-337

Declaration SHA-256: b3a6348544f17baf0cf9a8d51663f5b8b6187359294117163fc5d2afba78dd58

Mathematical statement. The sine inequality is strict unless $h=1/2$.

For $q \neq 0$ and $\lambda > 1$, dividing by λ strictly decreases the squared argument.

```
theorem sin_inner_lt_sin_outer (hh : a.h ≠ 1 / 2) :
  sin a.innerAngle < sin a.outerAngle := by
  rw [innerAngle, outerAngle, Real.sin_arccos, Real.sin_arccos]
  have hlam_pos : 0 < lam := lt_trans (by norm_num) a.density_jump
  have hx : a.shapeParameter ≠ 0 := by
    intro hzero
    apply hh
    simp only [shapeParameter] at hzero
    linarith
  have hx_sq : 0 < a.shapeParameter ^ 2 := sq_pos_of_ne_zero hx
  have hlam_sq : 1 < lam ^ 2 := by nlinarith [a.density_jump]
  have harg_sq : a.outerArgument ^ 2 < a.shapeParameter ^ 2 := by
    rw [outerArgument, div_pow]
    apply (div_lt_iff₀ (sq_pos_of_pos hlam_pos)).2
    nlinarith
  have habs : |a.shapeParameter| < 1 := abs_lt.mpr a.shapeParameter_mem
```

```

have hshape_sq : a.shapeParameter ^ 2 ≤ 1 := by
  have habs_le : |a.shapeParameter| ≤ |(1 : ℝ)| := by
    simpa using le_of_lt habs
  have hsq : a.shapeParameter ^ 2 ≤ (1 : ℝ) ^ 2 :=
    sq_le_sq.mpr habs_le
  simpa using hsq
have hleft : 0 ≤ 1 - a.shapeParameter ^ 2 := by
  linarith
exact Real.sqrt_lt_sqrt hleft (by nlinarith)

```

TypeThreeAssembly.sin_sideArcAngle

Classification: supplement

Source: TypeThreeAssembly.lean:227-230

Declaration SHA-256: 41165bd4a88985079154632e1304282b8868bb98cc6b1d97bb3ea19faaebe0dd

Mathematical statement. The side-arc angle and inner angle have equal sine.

$$\sin(\pi - b_3) = \sin b_3.$$

```

theorem sin_sideArcAngle :
  sin a.sideArcAngle = sin a.innerAngle := by
  rw [sideArcAngle, Real.sin_pi_sub]

```

TypeThreeAssembly.upperCenter

Classification: supplement

Source: TypeThreeAssembly.lean:76-79

Declaration SHA-256: 6dabb1e2d2dfee1af758b047dfb6728a97191f4c64742b75a35f102c81c1c3c1

Mathematical statement. Center of the upper exterior circle.

The point is $(0, 1 - R\cos a_3)$.

```

def upperCenter : PlanePoint :=
  (0, 1 - a.radius * cos a.outerAngle)

/-- Closed central rectangle of `K₃`. -/

```

arc_eq

Classification: supplement

Source: MorgansArcFunction.lean:269-271

Declaration SHA-256: 3b036fd44214b60f3d2baaf90be272e48c10ba5e310883f73c7eb20af846505a

Mathematical statement. The arc function equals ell evaluated at the inverse-area angle.

This exposes the defining composition for later substitutions.

```

lemma arc_eq (x : ℝ) : arc x = ell (θ0f x) := rfl

/-- `θ0f` inherits strict monotonicity from `area`. -/

```

area_continuousOn

Classification: supplement

Source: MorgansArcFunction.lean:212-216

Declaration SHA-256: 7d3a7659000f0acd2289cd80ef6ad9d2f860c6244018818d478eae78e42ab6d

Mathematical statement. Normalized cap area is continuous on $(0, \pi)$.

The sine denominator is nonzero throughout the domain.

```
lemma area_continuousOn : ContinuousOn area (Ioo 0 π) :=
  fun _θ hθ => (hasDerivAt_area hθ.1 hθ.2).continuousAt.continuousWithinAt

/-- `area` maps `(0, π)` onto `(0, ∞)`: the two limits supply points on either
side of any target, and the intermediate value theorem does the rest. -/
```

area_lt_self

Classification: supplement

Source: MorgansArcFunction.lean:158-172

Declaration SHA-256: 505f4cfdbccff59a35c0181afbb77947295aa04ad4c13ed1bce28a912b3e1042

Mathematical statement. For the recorded small-angle domain, normalized cap area is less than the angle.

The elementary sine bounds provide a local squeeze used at the zero endpoint.

```
lemma area_lt_self {θ : ℝ} (h0 : 0 < θ) (h1 : θ ≤ 1) : area θ < θ := by
  have hc : θ^3 ≤ θ := by nlinarith [sq_nonneg θ, mul_pos h0 h0]
  have hs : 5*θ/6 < sin θ := by
    have := sin_gt_sub_cube h0
    linarith
  have hsq : 25*θ^2/36 < sin θ^2 := by nlinarith
  have hnum : θ - sin θ * cos θ < 2*θ^3/3 := by
    have h2 : sin (2*θ) > 2*θ - (2*θ)^3/6 := sin_gt_sub_cube (by linarith)
    rw [sin_two_mul] at h2
    nlinarith
  have hden : 0 < 4 * sin θ^2 := by nlinarith
  rw [area, div_lt_iff, hden]
  nlinarith

/-- `area θ → 0` as `θ → 0+`, by squeezing between `0` and `θ`. -/
```

area_pi_div_two

Classification: supplement

Source: GFunction.lean:66-69

Declaration SHA-256: 038ce6e93459592481f7a2c059cf51404c3244d96fa3828e4004b771a35871c4

Mathematical statement. A unit-chord semicircle has normalized area $\pi/8$.

Substitute $\theta = \pi/2$ in $a(\theta)$.

```
lemma area_pi_div_two : area (π / 2) = π / 8 := by
  rw [area, sin_pi_div_two, cos_pi_div_two]
  ring
```

area_pi_sub

Classification: supplement

Source: Minimiser.lean:134-140

Declaration SHA-256: 6ddb90a2a474720c53dc6e87015838175f9560adcbb74d22d3aa99ec3da8673f

Mathematical statement. The normalized area formula at the supplementary cap angle.

Use $\sin(\pi - \theta) = \sin \theta$ and $\cos(\pi - \theta) = -\cos \theta$ in a .

```
lemma area_pi_sub {θ : ℝ} (hθ : θ ∈ Ioo 0 π) :  
  area (π - θ) = (π - θ + sin θ * cos θ) / (4 * sin θ ^ 2) := by  
  rw [area, sin_pi_sub, cos_pi_sub]  
  ring  
  
/-- **The key substitution.** If `θ = 0` then the supplementary cap has  
exactly twice the area. -/
```

area_pos

Classification: supplement

Source: MorgansArcFunction.lean:148-157

Declaration SHA-256: 0af41b50988d0bace92fc2e638571302802d8538245a2c254bedb9f164463ede

Mathematical statement. Normalized cap area is positive for every valid cap angle.

The circular-segment numerator and denominator are positive.

```
lemma area_pos {θ : ℝ} (h0 : 0 < θ) (hπ : θ < π) : 0 < area θ := by  
  have hs : 0 < sin θ := sin_pos_of_pos_of_lt_pi h0 hπ  
  have hnum : 0 < θ - sin θ * cos θ := by  
    nlinarith [sin_lt h0, neg_one_le_cos θ, cos_le_one θ, sq_nonneg (sin θ)]  
  rw [area]  
  positivity  
  
/-- On `(0, 1]` the area is bounded above by `θ`. The constants come from  
`sin θ > θ - θ³/6`: the numerator is `< 2θ³/3` and `< 4 sin²θ > 25θ²/9`, so  
`area θ < 6θ/25`. -/
```

capBoundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:1249-1251

Declaration SHA-256: 5dbd12f9bc57860ad2ce92a638514e0e8d15006685aca7388e2fa2e1689504cd

Mathematical statement. The cap arc's interval-pushforward measure.

It is used for component integrals before identification with Euclidean frontier measure.

```
def capBoundaryMeasure (c : OneSidedCircularCap) : Measure PlanePoint :=  
  intervalPushforward (capParam c) (capStart c) (capEnd c)
```

capBoundaryMeasure_integrable

Classification: supplement

Source: CMVGeometry.lean:2260-2267

Declaration SHA-256: 0065afd47de261467dd963dc3f68bd9fc5e9eab2c424cfff1d00f2e7c2919a7

Mathematical statement. The density is integrable against a cap's parametrized boundary measure.

The cap parameter interval has finite length.

```
lemma capBoundaryMeasure_integrable (lam : ℝ) (c : OneSidedCircularCap) :  
  Integrable (StripDensity lam) (capBoundaryMeasure c) := by  
  rw [capBoundaryMeasure]  
  exact stripDensity_integrable_intervalPushforward lam (by  
    apply Continuous.measurable  
    unfold capParam OneSidedCircularCap.arcPoint  
    split <=> fun_prop)
```

capComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:390-397

Declaration SHA-256: 3ed78deaf1597a91c6218ca8fe10519ba693355e7329e0301f17fcce226f5a43

Mathematical statement. Isometric placement and reflection of a complex cap arc in Euclidean coordinates.

The map handles translation and cap-side orientation separately from radius scaling.

```
noncomputable def capComplexIsometryMap (c : OneSidedCircularCap) (z : ℂ) :  
  EuclideanPlane :=  
  match c.side with  
  | .upper => WithLp.toLp 2  
    (c.midpointX + z.im, c.baseY - c.radius * Real.cos c.theta + z.re)  
  | .lower => WithLp.toLp 2  
    (c.midpointX + z.im, c.baseY + c.radius * Real.cos c.theta - z.re)
```

capEnd

Classification: supplement

Source: CMVGeometry.lean:1246-1248

Declaration SHA-256: a140ca726638df619aef507f8019eb9f7ca9503d3231f8bfa8f0a64feb377161

Mathematical statement. Final arclength parameter for the cap arc.

The parameter span is $2R\theta$.

```
def capEnd (c : OneSidedCircularCap) : ℝ := c.radius * c.theta  
  
/-- Explicit regular boundary measure of a one-sided circular arc. -/
```

capParam

Classification: supplement

Source: CMVGeometry.lean:1241-1243

Declaration SHA-256: d80d6cd5283176fe8b29770f039d1135a8892539144575a4bc1d9962ad427168

Mathematical statement. Arclength-scaled parametrization of a circular cap.

Angular displacement is the real parameter divided by the cap radius.

```
def capParam (c : OneSidedCircularCap) (s : ℝ) : PlanePoint :=  
  c.arcPoint (s / c.radius)
```

capParam_image_Icc_eq_arcTrace

Classification: supplement

Source: FrontierPerimeter.lean:947-1070

Declaration SHA-256: f412e9294ab946fde4ef494447fa75b0b57b0600c2bd61007639df1bb35c0e9e

Mathematical statement. The full cap parameter interval traces exactly the cap arc.

The equality handles the full allowed angle range, including major caps.

```
theorem capParam_image_Icc_eq_arcTrace (c : OneSidedCircularCap) :
  capParam c '' Icc (capStart c) (capEnd c) = c.arcTrace := by
  apply Set.Subset.antisymm
  · rintro p ⟨s, hs, rfl⟩
    have ht : s / c.radius ∈ Icc (-c.theta) c.theta := by
      have hr := c.radius_pos
      constructor
      · apply (le_div_iff₀ hr).2
        simpa [capStart, mul_comm] using hs.1
      · apply (div_le_iff₀ hr).2
        simpa [capEnd, mul_comm] using hs.2
    have habs : |s / c.radius| ≤ c.theta := (abs_le).2 ht
    constructor
    · cases hside : c.side <|>
      · simp only [capParam, OneSidedCircularCap.radiusSquaredAt,
        OneSidedCircularCap.center, OneSidedCircularCap.arcPoint, hside]
      all_goals
        nlinarith [Real.sin_sq_add_cos_sq (s / c.radius)]
    · exact c.arcPoint_in_closed_side habs
  · intro p hp
    cases hside : c.side with
    | upper =>
      let z : ℂ := ⟨p.2 - (c.baseY - c.radius * Real.cos c.theta),
        p.1 - c.midpointX⟩
      have hcircle :
        (p.1 - c.midpointX) ^ 2 +
        (p.2 - (c.baseY - c.radius * Real.cos c.theta)) ^ 2 =
        c.radius ^ 2 := by
        simpa [OneSidedCircularCap.arcTrace,
          OneSidedCircularCap.radiusSquaredAt,
          OneSidedCircularCap.center, hside] using hp.1
      have hnorm_sq : ||z|| ^ 2 = c.radius ^ 2 := by
        rw [Complex.sq_norm, Complex.normSq_apply]
        simp only [z]
        nlinarith
      have hnorm : ||z|| = c.radius := by
        nlinarith [norm_nonneg z, c.radius_pos]
      have hsin :
        c.radius * Real.sin z.arg = p.1 - c.midpointX := by
        simpa [hnorm, z] using Complex.norm_mul_sin_arg z
      have hcos :
        c.radius * Real.cos z.arg =
        p.2 - (c.baseY - c.radius * Real.cos c.theta) := by
        simpa [hnorm, z] using Complex.norm_mul_cos_arg z
      have hsidep : c.baseY ≤ p.2 := by
        simpa [OneSidedCircularCap.arcTrace, hside] using hp.2
      have hcos_le : Real.cos c.theta ≤ Real.cos z.arg := by
        nlinarith [c.radius_pos]
      have harg : |z.arg| ≤ c.theta := by
        by_contra h
        have hlt : c.theta < |z.arg| := lt_of_not_ge h
        have hc := Real.cos_lt_cos_of_nonneg_of_le_pi
          (le_of_lt c.theta_pos) (Complex.abs_arg_le_pi z) hlt
        rw [Real.cos_abs] at hc
        linarith
      let s := c.radius * z.arg
      have hs : s ∈ Icc (capStart c) (capEnd c) := by
        rcases (abs_le.mp harg) with ⟨hl, hu⟩
        dsimp [s]
        constructor <|> simp only [capStart, capEnd] <|>
```



```

      nlinarith [c.radius_pos]
    refine (s, hs, ?_)
    have hangle : s / c.radius = z.arg := by
      dsimp [s]
      field_simp [ne_of_gt c.radius_pos]
    apply Prod.ext
    · simp only [capParam, OneSidedCircularCap.arcPoint, hside,
      hangle]
      nlinarith
    · simp only [capParam, OneSidedCircularCap.arcPoint, hside,
      hangle]
      nlinarith
| lower =>
  let z : ℂ := ((c.baseY + c.radius * Real.cos c.theta) - p.2,
    p.1 - c.midpointX)
  have hcircle :
    (p.1 - c.midpointX) ^ 2 +
    (p.2 - (c.baseY + c.radius * Real.cos c.theta)) ^ 2 =
    c.radius ^ 2 := by
    simpa [OneSidedCircularCap.arcTrace,
      OneSidedCircularCap.radiusSquaredAt,
      OneSidedCircularCap.center, hside] using hp.1
  have hnorm_sq : ||z|| ^ 2 = c.radius ^ 2 := by
    rw [Complex.sq_norm, Complex.normSq_apply]
    simp only [z]
    nlinarith
  have hnorm : ||z|| = c.radius := by
    nlinarith [norm_nonneg z, c.radius_pos]
  have hsin :
    c.radius * Real.sin z.arg = p.1 - c.midpointX := by
    simpa [hnorm, z] using Complex.norm_mul_sin_arg z
  have hcos :
    c.radius * Real.cos z.arg =
    (c.baseY + c.radius * Real.cos c.theta) - p.2 := by
    simpa [hnorm, z] using Complex.norm_mul_cos_arg z
  have hsidep : p.2 ≤ c.baseY := by
    simpa [OneSidedCircularCap.arcTrace, hside] using hp.2
  have hcos_le : Real.cos c.theta ≤ Real.cos z.arg := by
    nlinarith [c.radius_pos]
  have harg : |z.arg| ≤ c.theta := by
    by_contra h
    have hlt : c.theta < |z.arg| := lt_of_not_ge h
    have hc := Real.cos_lt_cos_of_nonneg_of_le_pi
      (le_of_lt c.theta_pos) (Complex.abs_arg_le_pi z) hlt
    rw [Real.cos_abs] at hc
    nlinarith
  let s := c.radius * z.arg
  have hs : s ∈ Icc (capStart c) (capEnd c) := by
    rcases (abs_le.mp harg) with (hl, hu)
    dsimp [s]
    constructor <=> simp only [capStart, capEnd] <=>
      nlinarith [c.radius_pos]
  refine (s, hs, ?_)
  have hangle : s / c.radius = z.arg := by
    dsimp [s]
    field_simp [ne_of_gt c.radius_pos]
  apply Prod.ext
  · simp only [capParam, OneSidedCircularCap.arcPoint, hside,
    hangle]
    nlinarith
  · simp only [capParam, OneSidedCircularCap.arcPoint, hside,
    hangle]
    nlinarith

```

capPartitionPoint

Classification: supplement

Source: FrontierPerimeter.lean:154-156

Declaration SHA-256: 49ac6ecbe67b9040843eeee672d7b0f08055016781814c3ce0fa4e7d31ff805a

Mathematical statement. The affine signed-arclength partition formula $-r\theta + i(2r\theta/N)$, for real r, θ and natural N, i .

The definition is

$$p_i = -r\theta + i\frac{2r\theta}{N}, \quad N, i \in \mathbb{N}.$$

For $r > 0, \theta > 0, N > 0$, and $0 \leq i \leq N$, these are equally spaced signed-arclength coordinates in $[-r\theta, r\theta]$, not angles. The associated angular coordinates are $p_i/r = -\theta + 2i\theta/N$. The definition itself imposes no index or positivity bounds; with $N = 0$, formal division by zero makes every $p_i = -r\theta$.

```
def capPartitionPoint (r theta : ℝ) (N i : ℕ) : ℝ :=
  -r * theta + (i : ℝ) * (2 * r * theta / (N : ℝ))
```

capPartitionPoint_dist

Classification: supplement

Source: FrontierPerimeter.lean:185-207

Declaration SHA-256: 6b90645e53e16e3855618b966b04af3ee0f15439043926ba1133c583e65b5a00

Mathematical statement. For $r > 0, 0 < \theta < \pi$, positive natural N and any natural i , consecutive partition images have distance $2r \sin(\theta/N)$.

The parameter increment is $p_{i+1} - p_i = 2r\theta/N$. Substitution into the chord formula yields

$$\text{dist}(u_r(p_i), u_r(p_{i+1})) = 2r \sin(\theta/N).$$

Since $0 < \theta/N < \pi$, the sine is positive. This algebraic distance identity holds for every natural i , without the restriction $i < N$ used when forming the inscribed polygon.

```
lemma capPartitionPoint_dist {r theta : ℝ} (hr : 0 < r)
  (htheta : theta ∈ Ioo 0 Real.pi) {N i : ℕ} (hN : 0 < N) :
  dist (unitCircleArc r (capPartitionPoint r theta N i))
    (unitCircleArc r (capPartitionPoint r theta N (i + 1))) =
    2 * r * Real.sin (theta / (N : ℝ)) := by
  rw [unitCircleArc_dist hr]
  have hNreal : 0 < (N : ℝ) := by positivity
  have harg :
    ((capPartitionPoint r theta N i - capPartitionPoint r theta N (i + 1)) / r) / 2 =
      -(theta / (N : ℝ)) := by
    unfold capPartitionPoint
    push_cast
    field_simp [hr.ne', hNreal.ne']
    ring
  have hangle_pos : 0 < theta / (N : ℝ) := div_pos htheta.1 hNreal
  have hN_one : (1 : ℝ) ≤ (N : ℝ) := by exact_mod_cast hN
  have hangle_lt : theta / (N : ℝ) < Real.pi := by
    apply (div_lt_iff₀ hNreal).2
    nlinarith [htheta.2, Real.pi_pos]
  rw [harg, Real.sin_neg, abs_mul, abs_of_pos (by norm_num : (0 : ℝ) < 2),
    abs_neg, abs_of_pos (Real.sin_pos_of_pos_of_lt_pi hangle_pos hangle_lt)]
  ring
```

capPartitionPoint_end

Classification: supplement

Source: FrontierPerimeter.lean:161-166

Declaration SHA-256: b5d0e986af4c921461bd13ce89490a41fdc02991eaadda7d26f70db465413519

Mathematical statement. For real r, θ and a positive natural subdivision count N , the N th partition point equals $r \theta$.

For $N > 0$, $p_N = -r\theta + N(2r\theta/N) = r\theta$. Only the nonzero subdivision denominator is needed; the identity imposes no sign condition on r or θ .

```
lemma capPartitionPoint_end {r theta : ℝ} {N : ℕ} (hN : 0 < N) :
  capPartitionPoint r theta N N = r * theta := by
  unfold capPartitionPoint
  field_simp [Nat.ne_of_gt hN]
  ring
```

capPartitionPoint_mem

Classification: supplement

Source: FrontierPerimeter.lean:175-184

Declaration SHA-256: 8f26d09621c071ac8a6a1935a21253dfb29cb269e0ec21dc71e59a40f0529e9f

Mathematical statement. For $r > 0$, $\theta > 0$, $N > 0$ and natural $i \leq N$, the i th partition point belongs to $[-r \theta, r \theta]$.

Monotonicity and the endpoint identities give $-r\theta = p_0 \leq p_i \leq p_N = r\theta$. The interval is in signed-arclength coordinates. No upper bound $\theta < \pi$ is needed for this membership assertion.

```
lemma capPartitionPoint_mem {r theta : ℝ} (hr : 0 < r) (htheta : 0 < theta)
  {N i : ℕ} (hN : 0 < N) (hi : i ≤ N) :
  capPartitionPoint r theta N i ∈ Icc (-r * theta) (r * theta) := by
  have hm := monotone_capPartitionPoint hr htheta hN
  constructor
  · rw [← capPartitionPoint_zero r theta N]
    exact hm (Nat.zero_le i)
  · rw [← capPartitionPoint_end hN]
    exact hm hi
```

capPartitionPoint_zero

Classification: supplement

Source: FrontierPerimeter.lean:157-160

Declaration SHA-256: 1714782661300a21ee2d4243c7e4fbe79e1e97ce545ea187699c13d709db2db9

Mathematical statement. For all real r, θ and natural N , the zeroth partition point equals $-r \theta$.

Substituting $i = 0$ into $p_i = -r\theta + i(2r\theta/N)$ gives $p_0 = -r\theta$. This identity requires no positivity assumptions and also holds at $N = 0$.

```
lemma capPartitionPoint_zero (r theta : ℝ) (N : ℕ) :
  capPartitionPoint r theta N 0 = -r * theta := by
  simp [capPartitionPoint]
```

capStart

Classification: supplement

Source: CMVGeometry.lean:1244-1245

Declaration SHA-256: 594eb6e2bbf3cbc7a2397b6af7a93506027168fa75ecf7f70b7012b2fc6e208a

Mathematical statement. Initial arclength parameter for the cap arc.

It corresponds to the first endpoint of the cap's angular interval.

```
def capStart (c : OneSidedCircularCap) : ℝ := -c.radius * c.theta
```

cap_arclength_eq_two_radius_theta

Classification: supplement

Source: FrontierPerimeter.lean:451-456

Declaration SHA-256: edad6e24948a1499ce1a66a5efdc84d9c8f7899f5c52d6551716312e95163fb3

Mathematical statement. The cap's algebraic arc length equals $2R\theta$.

Use $c = 2R\sin\theta$ in $c\ell(\theta)$.

```
lemma cap_arclength_eq_two_radius_theta (c : OneSidedCircularCap) :  
  c.arclength = 2 * c.radius * c.theta := by  
  rw [OneSidedCircularCap.arclength, OneSidedCircularCap.radius, ell]  
  field_simp [ne_of_gt c.sin_theta_pos]  
  
/-- Exact Euclidean  $H^1$  mass for every existing `OneSidedCircularCap`, including major arcs. -/
```

cap_integrableOn

Classification: supplement

Source: CMVGeometry.lean:2303-2308

Declaration SHA-256: bbc4a1a9af0d45bee6aa1c1152d956e0e945a869f3550ca289ef7022556292ef

Mathematical statement. The density is integrable over a cap carrier.

The cap has finite planar volume.

```
lemma cap_integrableOn (lam : ℝ) (c : OneSidedCircularCap) :  
  IntegrableOn (StripDensity lam) c.carrier := by  
  apply stripDensity_integrableOn_of_volume_ne_top  
  rw [c.volume_carrier]  
  exact ENNReal.ofReal_ne_top
```

cap_lower_external_density

Classification: supplement

Source: CMVGeometry.lean:1341-1352

Declaration SHA-256: baaf83c56acf8edc18268269454b4946af9c7098831da004f1da856c3f80d9b2

Mathematical statement. The lower exterior cap has density λ along its open arc.

The base is $y = -1$; the two endpoints do not affect the integral.

```
theorem cap_lower_external_density (lam : ℝ) (c : OneSidedCircularCap)  
  (hside : c.side = .lower) (hbase : c.baseY = -1)  
  {s : ℝ} (hs : s ∈ Ioo (capStart c) (capEnd c)) :  
  StripDensity lam (capParam c s) = lam := by
```

```

have hopen := c.arcPoint_in_open_side (cap_angle_mem_open hs)
have hy : (capParam c s).2 < -1 := by
  simpa [capParam, hside, hbase] using hopen
rw [StripDensity, if_neg]
apply not_le.mpr
exact (show 1 < -(capParam c s).2 by linarith).trans_le (neg_le_abs _)

/-- Upper external cap: weighted integral equals `lam` times arc length. -/

```

cap_lower_weightedArea

Classification: supplement

Source: CMVGeometry.lean:2360-2389

Declaration SHA-256: af4bcab3c0cdacd24a9ba22f365ac164c96fd2be59faf0a944ca162de8e4ada2

Mathematical statement. An exterior lower cap has weighted area λ times Euclidean area.

The density-one base is again area-null.

```

lemma cap_lower_weightedArea (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .lower) (hbase : c.baseY = -1) :
  WeightedArea lam c.carrier = lam * c.euclideanArea := by
  have hne : ∀ m p : PlanePoint ∂volume, p.2 ≠ -1 := by
    rw [ae_iff]
    simpa only [not_not] using volume_horizontalline (-1)
  rw [WeightedArea]
  calc
    (∫ p in c.carrier, StripDensity lam p) =
      ∫ _p in c.carrier, lam := by
        apply integral_congr_ae
        filter_upwards [ae_restrict_mem c.measurableSet_carrier,
          ae_restrict_of_ae hne] with p hp hpne
        have hple : p.2 ≤ (-1 : ℝ) := by
          simpa [OneSidedCircularCap.carrier, hside, hbase] using hp.2
        have hplt : p.2 < (-1 : ℝ) := lt_of_le_of_ne hple hpne
        rw [StripDensity, if_neg]
        apply not_le.mpr
        exact (show 1 < -p.2 by linarith).trans_le (neg_le_abs p.2)
    _ = lam * c.euclideanArea := by
      rw [integral_const, measureReal_restrict_apply_univ]
      have hvolume : volume.real c.carrier = c.euclideanArea := by
        rw [Measure.real, c.volume_carrier,
          ENNReal.toReal_ofReal c.euclideanArea_pos.le]
      rw [hvolume]
      ring

/-- The coordinate four-arc assembly: two side arcs from the strip core and two
congruent reflected exterior caps. -/

```

cap_lower_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1373-1394

Declaration SHA-256: 5575ee03ce1e095004c86962659c2bcf1dde6c0b1ad90b439220173f4e11e7fc

Mathematical statement. The lower cap boundary integral is λ times arc length.

Reflection gives the same weighted length computation.

```

theorem cap_lower_weightedIntegral (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .lower) (hbase : c.baseY = -1) :
  ∫ p, StripDensity lam p ∂capBoundaryMeasure c = lam * c.arcLength := by
  rw [capBoundaryMeasure]
  rw [integral_intervalPushforward (capParam_measurable c) (measurable_stripDensity lam)]

```

```

rw [integral_Icc_eq_integral_Ioo]
calc
  (f s in Ioo (capStart c) (capEnd c), StripDensity lam (capParam c s)) =
    f _s in Ioo (capStart c) (capEnd c), lam := by
      apply integral_congr_ae
      filter_upwards [ae_restrict_mem measurableSet_Ioo] with s hs
      exact cap_lower_external_density lam c hside hbase hs
  _ = lam * (capEnd c - capStart c) := by
    have hlen : 0 ≤ capEnd c - capStart c := by
      dsimp [capEnd, capStart]
      nlinarith [c.radius_pos, c.theta_pos]
    simp [max_eq_left hlen, mul_comm]
  _ = lam * c.arclength := by rw [cap_arclength_eq c]

/-! ## Horizontal segments -/

/-- Unit-speed left-to-right segment parametrization. -/

```

cap_partition_polygon_lower

Classification: supplement

Source: FrontierPerimeter.lean:208-268

Declaration SHA-256: 4dc7fcd936b6c56a4643f00c8d67c38ce24d6351f625a8873960fc070d61a446

Mathematical statement. For $r > 0$, $0 < \theta < \pi$ and positive natural N , the N -edge inscribed polygon gives the Hausdorff lower bound $N(2r \sin(\theta/N))$.

Under these hypotheses,

$$\text{ofReal}(N 2r \sin(\theta/N)) \leq \mathcal{H}^1(u_r([-r\theta, r\theta])).$$

Add the endpoint-distance lower bounds over the N disjoint half-open subarc images. Injectivity on the full interval prevents positive-length overlap, and deleting endpoints does not change $\mathcal{H}^1$.

```

lemma cap_partition_polygon_lower {r theta : ℝ} (hr : 0 < r)
  (htheta : theta ∈ Ioo 0 Real.pi) (N : ℕ) (hN : 0 < N) :
  ENNReal.ofReal ((N : ℝ) * (2 * r * Real.sin (theta / (N : ℝ)))) ≤
    (μH[1] : Measure C)
    (unitCircleArc r '' Icc (-r * theta) (r * theta)) := by
let p : ℕ → ℝ := capPartitionPoint r theta N
let A : ℕ → Set ℝ := fun i => Ico (p i) (p (i + 1))
let S : Set ℝ := Icc (-r * theta) (r * theta)
let f : ℝ → C := unitCircleArc r
let μ : Measure C := μH[1]
have hpmono : Monotone p := monotone_capPartitionPoint hr htheta.1 hN
have hinj : Set.InjOn f S := unitCircleArc_injOn hr htheta
have hpiece_sub (i : ℕ) (hi : i < N) : Icc (p i) (p (i + 1)) ⊆ S := by
  intro x hx
  exact ⟨(capPartitionPoint_mem hr htheta.1 hN hi.le).1.trans hx.1,
    hx.2.trans (capPartitionPoint_mem hr htheta.1 hN (Nat.succ_le_iff.mpr hi)).2⟩
have hpair : Set.Pairwise (↑(Finset.range N))
  (Function.onFun Disjoint fun i => f '' A i) := by
  intro i hi j hj hij
  change Disjoint (f '' A i) (f '' A j)
  have hsrc : Disjoint (A i) (A j) := hpmono.pairwise_disjoint_on_Ico_succ hij
  rw [Set.disjoint_left]
  rintro y ⟨x, hxi, rfl⟩ ⟨z, hzj, hzx⟩
  have hiN : i < N := Finset.mem_range.mp hi
  have hjN : j < N := Finset.mem_range.mp hj
  have hxS : x ∈ S := hpiece_sub i hiN ⟨hxi.1, hxi.2.le⟩
  have hzS : z ∈ S := hpiece_sub j hjN ⟨hzj.1, hzj.2.le⟩
  have hxz : x = z := hinj hxS hzS hzx.symm
  exact (Set.disjoint_left.mp hsrc) hxi (hxz ▸ hzj)
have hmeas (i : ℕ) (hi : i ∈ Finset.range N) : MeasurableSet (f '' A i) := by
  have hiN : i < N := Finset.mem_range.mp hi
  rw [show f '' A i = f '' Icc (p i) (p (i + 1)) \ {f (p (i + 1))} by
    exact image_Ico_eq_image_Icc_diff_endpoint hinj (hpmono (Nat.le_succ i))

```

```

    (hpiece_sub i hiN)]
  exact (isCompact_Icc.image (continuous_unitCircleArc r)).measurableSet.diff
    (measurableSet_singleton _)
  have hunion_sub : (U i ∈ Finset.range N, f '' A i) ⊆ f '' S := by
    intro y hy
    rcases mem_iUnion.mp hy with ⟨i, hy⟩
    rcases mem_iUnion.mp hy with ⟨hi, hy⟩
    rcases hy with ⟨x, hxA, rfl⟩
    exact ⟨x, hpiece_sub i (Finset.mem_range.mp hi) ⟨hxA.1, hxA.2.le⟩, rfl⟩
  calc
    ENNReal.ofReal ((N : ℝ) * (2 * r * Real.sin (theta / (N : ℝ)))) =
      ∑ i ∈ Finset.range N,
        ENNReal.ofReal (2 * r * Real.sin (theta / (N : ℝ))) := by
          rw [ENNReal.ofReal_mul (by positivity), ENNReal.ofReal_natCast]
          simp [Finset.sum_const, nsmul_eq_mul]
    _ ≤ ∑ i ∈ Finset.range N, μ (f '' A i) := by
      apply Finset.sum_le_sum
      intro i hi
      have hiN : i < N := Finset.mem_range.mp hi
      rw [hausdorffMeasure_image_Ico_eq_Icc hinj (hpmmono (Nat.le_succ i))
        (hpiece_sub i hiN)]
      simpa [f, p, μ, capPartitionPoint_dist hr htheta hN] using
        ofReal_dist_le_hausdorffMeasure_image_Icc (continuous_unitCircleArc r)
          (hpmmono (Nat.le_succ i))
    _ = μ (U i ∈ Finset.range N, f '' A i) :=
      (measure_biUnion_finset hpair hmeas).symm
    _ ≤ μ (f '' S) := measure_mono hunion_sub

```

cap_upper_external_density

Classification: supplement

Source: CMVGeometry.lean:1329-1340

Declaration SHA-256: 0d4d779bde08a636e8536f594b78dfa9705a31925f9d2e9248d0d62fdf5aa11a

Mathematical statement. The upper exterior cap has density lambda along its open arc.

The base is $y = 1$; endpoint exceptions have zero arclength measure.

```

theorem cap_upper_external_density (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .upper) (hbase : c.baseY = 1)
  {s : ℝ} (hs : s ∈ Ioo (capStart c) (capEnd c)) :
  StripDensity lam (capParam c s) = lam := by
  have hopen := c.arcPoint_in_open_side (cap_angle_mem_open hs)
  have hy : 1 < (capParam c s).2 := by
    simpa [capParam, hside, hbase] using hopen
  rw [StripDensity, if_neg]
  exact not_le.mpr (hy.trans_le (le_abs_self _))

/-- The non-endpoint part of a lower cap based on `y = -1` is in the external
(density `lam`) branch. -/

```

cap_upper_weightedArea

Classification: supplement

Source: CMVGeometry.lean:2334-2359

Declaration SHA-256: 2f2165b544886739ec40b854c29666e67f06c7a423bd046c04eb224a5c6ee0ce

Mathematical statement. An exterior upper cap has weighted area lambda times Euclidean area.

Only its base lies on the interface, an area-null set.

```

lemma cap_upper_weightedArea (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .upper) (hbase : c.baseY = 1) :
  WeightedArea lam c.carrier = lam * c.euclideanArea := by
  have hne : ∀ᵐ p : PlanePoint ∂volume, p.2 ≠ 1 := by

```

```

rw [ae_iff]
simp only [not_not] using volume_horizontalLine 1
rw [WeightedArea]
calc
  (f p in c.carrier, StripDensity lam p) =
    f _p in c.carrier, lam := by
      apply integral_congr_ae
      filter_upwards [ae_restrict_mem c.measurableSet_carrier,
        ae_restrict_of_ae hne] with p hp hpne
      have hpge : (1 : ℝ) ≤ p.2 := by
        simp [OneSidedCircularCap.carrier, hside, hbase] using hp.2
      have hpgt : (1 : ℝ) < p.2 := lt_of_le_of_ne hpge (Ne.symm hpne)
      rw [StripDensity, if_neg]
      exact not_le.mpr (hpgt.trans_le (le_abs_self p.2))
_ = lam * c.euclideanArea := by
  rw [integral_const, measureReal_restrict_apply_univ]
  have hvolume : volume.real c.carrier = c.euclideanArea := by
    rw [Measure.real, c.volume_carrier,
      ENNReal.toReal_ofReal c.euclideanArea_pos.le]
  rw [hvolume]
ring

```

cap_upper_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1353-1372

Declaration SHA-256: 45644ff66dcb2be9e3cb6c626527848bf909f532df9be34ffa1977e24d25b8f4

Mathematical statement. The upper cap boundary integral is λ times arc length.

Density is λ almost everywhere on the parametrized exterior arc.

```

theorem cap_upper_weightedIntegral (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .upper) (hbase : c.baseY = 1) :
  f p, StripDensity lam p ∂capBoundaryMeasure c = lam * c.arcLength := by
  rw [capBoundaryMeasure]
  rw [integral_intervalPushforward (capParam_measurable c) (measurable_stripDensity lam)]
  rw [integral_Icc_eq_integral_Ioo]
  calc
    (f s in Ioo (capStart c) (capEnd c), StripDensity lam (capParam c s)) =
      f _s in Ioo (capStart c) (capEnd c), lam := by
        apply integral_congr_ae
        filter_upwards [ae_restrict_mem measurableSet_Ioo] with s hs
        exact cap_upper_external_density lam c hside hbase hs
    _ = lam * (capEnd c - capStart c) := by
      have hlen : 0 ≤ capEnd c - capStart c := by
        dsimp [capEnd, capStart]
        nlinarith [c.radius_pos, c.theta_pos]
      simp [max_eq_left hlen, mul_comm]
    _ = lam * c.arcLength := by rw [cap_arclength_eq c]

/-- Lower external cap: weighted integral equals `lam` times arc length. -/

```

continuous_realizedCapParam

Classification: supplement

Source: FrontierPerimeter.lean:442-450

Declaration SHA-256: 439602ca8a442d23dec8f858626a27ca71e9723ce90b04b56ee6e94cd89590f1

Mathematical statement. The Euclidean cap parametrization is continuous.

Its coordinate trigonometric functions and realization map are continuous.

```

lemma continuous_realizedCapParam (c : OneSidedCircularCap) :
  Continuous (realizedCapParam c) := by

```



```
rw [show realizedCapParam c = capComplexIsometryMap c ◦
  unitCircleArc c.radius by
  funext s
  exact realizedCapParam_eq c s]
exact (isometry_capComplexIsometryMap c).continuous.comp
  (continuous_unitCircleArc c.radius)
```

continuous_realizedLeftCoreParam

Classification: supplement

Source: FrontierPerimeter.lean:727-735

Declaration SHA-256: 4dc1fbf3a43e8117f791c26e28bd5a2a2466716abc2324d51a2522841ccc46bf

Mathematical statement. The realized left core parameter is continuous.

This supports compact-image measurability and arc-length arguments.

```
lemma continuous_realizedLeftCoreParam (c : StripCore) :
  Continuous (realizedLeftCoreParam c) := by
  rw [show realizedLeftCoreParam c = leftCoreComplexIsometryMap c ◦
    unitCircleArc c.radius by
    funext s
    exact realizedLeftCoreParam_eq c s]
  exact (isometry_leftCoreComplexIsometryMap c).continuous.comp
    (continuous_unitCircleArc c.radius)
```

continuous_realizedRightCoreParam

Classification: supplement

Source: FrontierPerimeter.lean:736-744

Declaration SHA-256: db06971d2e0a6e54b3ce1b2cb1ffe296cb5ed7ac07495118f10c8f5f82bfe132

Mathematical statement. The realized right core parameter is continuous.

The trigonometric coordinate functions are continuous.

```
lemma continuous_realizedRightCoreParam (c : StripCore) :
  Continuous (realizedRightCoreParam c) := by
  rw [show realizedRightCoreParam c = rightCoreComplexIsometryMap c ◦
    unitCircleArc c.radius by
    funext s
    exact realizedRightCoreParam_eq c s]
  exact (isometry_rightCoreComplexIsometryMap c).continuous.comp
    (continuous_unitCircleArc c.radius)
```

continuous_realizedSegmentParam

Classification: supplement

Source: FrontierPerimeter.lean:868-872

Declaration SHA-256: 5ba4505531fe6afcb59f3956aaf9fe6e352c8d17f3326d5e955e4afc3e17c5df

Mathematical statement. The realized segment parametrization is continuous.

It is an affine map followed by a homeomorphism.

```
lemma continuous_realizedSegmentParam (seg : HorizontalSegment) :
  Continuous (realizedSegmentParam seg) := by
  unfold realizedSegmentParam segmentParam
  fun_prop
```

continuous_unitCircleArc

Classification: supplement

Source: FrontierPerimeter.lean:14-17

Declaration SHA-256: 343bc4472a64687ca829ebbf897c060cb6c7c3cad3716259e96fb84cf091470c

Mathematical statement. For every real r , the map $s \mapsto r \exp(i s/r)$ is continuous on the real line.

Fix $r \in \mathbb{R}$. The function $s \mapsto s/r$ is continuous, as are the complex exponential and multiplication by r . This proves continuity of u_r , without a positivity or nonzero-radius premise; at $r = 0$ the function is identically zero.

```
lemma continuous_unitCircleArc (r : ℝ) : Continuous (unitCircleArc r) := by
  unfold unitCircleArc
  fun_prop
```

coreArcEnd

Classification: supplement

Source: CMVGeometry.lean:1474-1476

Declaration SHA-256: 7ee06e3535f2fd49529132649f006dc8125adff906e2bb1c8e325a0ca4b835a4

Mathematical statement. Final arclength parameter for a core side arc.

The interval span is $2R \operatorname{arcsinh} h$.

```
def coreArcEnd (c : StripCore) : ℝ := c.radius * c.sideAngle

/-- Unit-speed parametrization of the left core arc. -/
```

coreArcStart

Classification: supplement

Source: CMVGeometry.lean:1472-1473

Declaration SHA-256: 4c395d23f09d7608737696a9d4043f61933ddaffbb62953e4470c1085d950b4e

Mathematical statement. Initial arclength parameter for a core side arc.

It is the negative endpoint of the symmetric side-arc parameter interval.

```
def coreArcStart (c : StripCore) : ℝ := -c.radius * c.sideAngle
```

core_integrableOn

Classification: supplement

Source: CMVGeometry.lean:2309-2314

Declaration SHA-256: ebecbc727d05931a9afed5e77aa787703ada65169aa70e3f372bec6e04bddcac

Mathematical statement. The density is integrable over a strip core.

Its area is finite and its density is one.

```
lemma core_integrableOn (lam : ℝ) (c : StripCore) :
  IntegrableOn (StripDensity lam) c.carrier := by
  apply stripDensity_integrableOn_of_volume_ne_top
  rw [c.volume_carrier]
  exact ENNReal.ofReal_ne_top
```

cos_two_mul_sin_sq

Classification: supplement

Source: Certificate.lean:117-122

Declaration SHA-256: 457c64175a22d2778423d351b2d6d75e23c9d3a388b2439675e854dd25957517

Mathematical statement. The double-angle cosine identity in sine-square form.

$\cos(2u) = 1 - 2\sin^2 u$ is used in rational trigonometric estimates.

```
lemma cos_two_mul_sin_sq (u : ℝ) : cos (2 * u) = 1 - 2 * sin u ^ 2 := by
  rw [cos_two_mul]
  linear_combination (2:ℝ) * sin_sq_add_cos_sq u

/-! ## Level 4: `s = y / 16 ∈ (0.0335166, 0.0335166625)` -/
```

euclideanJoinSet

Classification: supplement

Source: FrontierPerimeter.lean:1257-1259

Declaration SHA-256: 96d51d86f6c32a7f1c31ecb0290be17bec48e5d48a1aa38fdfbae4457025d389

Mathematical statement. The junction set transported to Euclidean coordinates.

It is the homeomorphic image of the coordinate endpoint set.

```
def euclideanJoinSet (q : ℝ) : Set EuclideanPlane :=
  planeEuclideanHomeomorph '' joinSet q
```

euclideanJoinSet_finite

Classification: supplement

Source: FrontierPerimeter.lean:1260-1262

Declaration SHA-256: 9539ffd4336468eab518f7e951c9dc4db7ee2d6bd08f0fa98bbda9dcf0817f38

Mathematical statement. The Euclidean junction set is finite.

Images of finite sets are finite.

```
theorem euclideanJoinSet_finite (q : ℝ) : (euclideanJoinSet q).Finite :=
  (joinSet_finite q).image planeEuclideanHomeomorph
```

euclideanPairwiseOverlap

Classification: supplement

Source: FrontierPerimeter.lean:1591-1597

Declaration SHA-256: c1559aa1737fb404d99a3376b70fbd4f0ff514e71a947a7d30b01a1ce4c1177e

Mathematical statement. For four subsets of any type, the union of their six pairwise intersections.

For arbitrary subsets A, B, C, D of a type α , the definition is $(A \cap B) \cup (A \cap C) \cup (A \cap D) \cup (B \cap C) \cup (B \cap D) \cup (C \cap D)$. It has no metric, Euclidean, or measurability hypothesis. Its application to Euclidean boundary traces packages the possible overlaps in a four-piece perimeter sum.

```
def euclideanPairwiseOverlap {α : Type*}
  (A B C D : Set α) : Set α :=
  (A ∩ B) ∪ ((A ∩ C) ∪ ((A ∩ D) ∪
    ((B ∩ C) ∪ ((B ∩ D) ∪ (C ∩ D)))))
```

```
/-- A null union of all pairwise overlaps makes the four sets pairwise
almost disjoint. -/
```

euclideanStripDensity

Classification: supplement

Source: FrontierPerimeter.lean:1389-1391

Declaration SHA-256: 3d074997fd7c2f4c47fc25c7e8a9a39259d6fd4628b103b9b3e4da77e1679848

Mathematical statement. The strip density expressed on the Euclidean realization.

Compose the coordinate density with the inverse homeomorphism.

```
def euclideanStripDensity (lam : ℝ) (z : EuclideanPlane) : ℝ :=
  StripDensity lam (planeEuclideanHomeomorph.symm z)
```

euclideanStripDensity_integrableOn_of_measure_ne_top

Classification: supplement

Source: FrontierPerimeter.lean:1404-1411

Declaration SHA-256: 509b7e5c9b45ab61b5e958fce825f92bda813640616ddf5c7591ac420818c5d2

Mathematical statement. For every real lambda and every set of finite Euclidean Hausdorff one-measure, the transported strip density is integrable on that set.

Let $S \subseteq \mathbb{R}^2$ in the Euclidean realization and suppose $\mathcal{H}^1(S) < \infty$. The globally measurable transported density has norm at most $1 + |\lambda|$, so it is integrable for $\mathcal{H}^1 \upharpoonright S$. No measurability hypothesis on S itself is required by this theorem.

```
lemma euclideanStripDensity_integrableOn_of_measure_ne_top
  (lam : ℝ) {s : Set EuclideanPlane}
  (hs : (μH[1] : Measure EuclideanPlane) s ≠ ∞) :
  IntegrableOn (euclideanStripDensity lam) s (μH[1] : Measure EuclideanPlane) := by
  apply Measure.integrableOn_of_bounded (M := 1 + |lam|) hs
  · exact (measurable_euclideanStripDensity lam).aestronglyMeasurable
  · exact Filter.Eventually.of_forall (euclideanStripDensity_bound lam)
```

exact_euclidean_leftCoreParam_hausdorffMeasure

Classification: supplement

Source: FrontierPerimeter.lean:745-772

Declaration SHA-256: e0211c9e8b0941bc0bb3bd977afe28d78688025726aac98a4d738bf7f180e9

Mathematical statement. The left core parameter image has length $2R \arcsin(h)$.

Its angular span is $2\arcsin h$, scaled by radius R .

```
theorem exact_euclidean_leftCoreParam_hausdorffMeasure (c : StripCore) :
  (μH[1] : Measure EuclideanPlane)
  (realizedLeftCoreParam c ' Icc (coreArcStart c) (coreArcEnd c)) =
  ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  have hfun : realizedLeftCoreParam c =
    leftCoreComplexIsometryMap c ∘ unitCircleArc c.radius := by
    funext s
    exact realizedLeftCoreParam_eq c s
  calc
    _ = (μH[1] : Measure EuclideanPlane)
      (leftCoreComplexIsometryMap c '
        (unitCircleArc c.radius ' Icc (coreArcStart c) (coreArcEnd c))) := by
      rw [hfun]
      congr 1
      simp only [Function.comp_apply] using (Set.image_image _ _).symm
```

```

_ = (μH[1] : Measure ℂ)
  (unitCircleArc c.radius '' Icc (coreArcStart c) (coreArcEnd c)) :=
(isometry_leftCoreComplexIsometryMap c).hausdorffMeasure_image
  (Or.inl (by norm_num)) _
_ = ENNReal.ofReal (2 * c.radius * c.sideAngle) := by
  simpa [coreArcStart, coreArcEnd] using
    hausdorffMeasure_unitCircleArc_image_eq c.radius_pos
    ⟨c.sideAngle_pos, c.sideAngle_lt_pi⟩
_ = ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  congr 1
  simp only [coreArcStart, coreArcEnd]
  ring

```

exact_euclidean_rightCoreParam_hausdorffMeasure

Classification: supplement

Source: FrontierPerimeter.lean:773-800

Declaration SHA-256: 5b9d17b9e5921b7fba027be4ebfa3845e68c3c43a92a67ce9d4854bf1b244db0

Mathematical statement. The right core parameter image has the same exact arc length.

The placement isometry preserves the measure of the radius- R circle image with signed-arclength span $2R\text{Arcsinh}$.

```

theorem exact_euclidean_rightCoreParam_hausdorffMeasure (c : StripCore) :
  (μH[1] : Measure EuclideanPlane)
    (realizedRightCoreParam c '' Icc (coreArcStart c) (coreArcEnd c)) =
  ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  have hfun : realizedRightCoreParam c =
    rightCoreComplexIsometryMap c ∘ unitCircleArc c.radius := by
    funext s
    exact realizedRightCoreParam_eq c s
  calc
  _ = (μH[1] : Measure EuclideanPlane)
    (rightCoreComplexIsometryMap c ''
      (unitCircleArc c.radius '' Icc (coreArcStart c) (coreArcEnd c))) := by
    rw [hfun]
    congr 1
    simpa only [Function.comp_apply] using (Set.image_image _ _ _).symm
  _ = (μH[1] : Measure ℂ)
    (unitCircleArc c.radius '' Icc (coreArcStart c) (coreArcEnd c)) :=
(isometry_rightCoreComplexIsometryMap c).hausdorffMeasure_image
  (Or.inl (by norm_num)) _
_ = ENNReal.ofReal (2 * c.radius * c.sideAngle) := by
  simpa [coreArcStart, coreArcEnd] using
    hausdorffMeasure_unitCircleArc_image_eq c.radius_pos
    ⟨c.sideAngle_pos, c.sideAngle_lt_pi⟩
_ = ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  congr 1
  simp only [coreArcStart, coreArcEnd]
  ring

```

fourArc_pairwiseOverlap_subset_joinSet

Classification: supplement

Source: FrontierPerimeter.lean:1142-1195

Declaration SHA-256: eea91f9cc72f6aabac972107df3b4a4acd7547fe5fa392689f59302f202356a4

Mathematical statement. Four-arc boundary overlaps are confined to the junction set.

Distinct component traces have no shared subarc of positive length.

```

theorem fourArc_pairwiseOverlap_subset_joinSet (a : FourArcAssembly) :
  pairwiseOverlap
    (StripCore.leftArcTrace a.core)
    (StripCore.rightArcTrace a.core)

```

```

    (OneSidedCircularCap.arcTrace a.upperCap)
    (OneSidedCircularCap.arcTrace a.lowerCap) ⊆
    joinSet a.core.chord := by
intro p hp
simp only [pairwiseOverlap, mem_union, mem_inter_iff] at hp
rcases hp with hLR | hLU | hLL | hRU | hRL | hUL
· exact False.elim (by
  have hl := hLR.1.2.1
  have hr := hLR.2.2.1
  linarith [a.core.chord_pos])
· have hyCore : p.2 ≤ 1 := (le_abs_self p.2).trans hLU.1.2.2
  have hyCap : (1 : ℝ) ≤ p.2 := by
    simp [FourArcAssembly.upperCap] using hLU.2.2
  have hy : p.2 = 1 := le_antisymm hyCore hyCap
  have hxBound := four_upper_arc_x_bound a hLU.2 hy
  have hx := eq_left_of_bounds hLU.1.2.1 hxBound
  have hpEq : p = (-a.core.chord / 2, 1) := Prod.ext hx hy
  simp [joinSet, hpEq]
· have hyCore : -1 ≤ p.2 := by
  linarith [(neg_le_abs p.2).trans hLL.1.2.2]
  have hyCap : p.2 ≤ (-1 : ℝ) := by
    simp [FourArcAssembly.lowerCap] using hLL.2.2
  have hy : p.2 = -1 := le_antisymm hyCap hyCore
  have hxBound := four_lower_arc_x_bound a hLL.2 hy
  have hx := eq_left_of_bounds hLL.1.2.1 hxBound
  have hpEq : p = (-a.core.chord / 2, -1) := Prod.ext hx hy
  simp [joinSet, hpEq]
· have hyCore : p.2 ≤ 1 := (le_abs_self p.2).trans hRU.1.2.2
  have hyCap : (1 : ℝ) ≤ p.2 := by
    simp [FourArcAssembly.upperCap] using hRU.2.2
  have hy : p.2 = 1 := le_antisymm hyCore hyCap
  have hxBound := four_upper_arc_x_bound a hRU.2 hy
  have hx := eq_right_of_bounds hRU.1.2.1 hxBound
  have hpEq : p = (a.core.chord / 2, 1) := Prod.ext hx hy
  simp [joinSet, hpEq]
· have hyCore : -1 ≤ p.2 := by
  linarith [(neg_le_abs p.2).trans hRL.1.2.2]
  have hyCap : p.2 ≤ (-1 : ℝ) := by
    simp [FourArcAssembly.lowerCap] using hRL.2.2
  have hy : p.2 = -1 := le_antisymm hyCap hyCore
  have hxBound := four_lower_arc_x_bound a hRL.2 hy
  have hx := eq_right_of_bounds hRL.1.2.1 hxBound
  have hpEq : p = (a.core.chord / 2, -1) := Prod.ext hx hy
  simp [joinSet, hpEq]
· have hyUpper : (1 : ℝ) ≤ p.2 := by
  simp [FourArcAssembly.upperCap] using hUL.1.2
  have hyLower : p.2 ≤ (-1 : ℝ) := by
    simp [FourArcAssembly.lowerCap] using hUL.2.2
  linarith

```

gd

Classification: supplement

Source: MinimumValue.lean:33-34

Declaration SHA-256: b105248ee05b475037dd209f9b23e6a5119f0585d6ae1093bea44c7ceded4050

Mathematical statement. The explicit derivative function for g.

$$g_d(X) = 4\sin\theta(X) - 4\sin\theta(2X).$$

```
noncomputable def gd (x : ℝ) : ℝ := 4 * sin (θ0f x) - 4 * sin (θ0f (2 * x))
```

gd_xstar

Classification: supplement

Source: MinimumValue.lean:40-41

Declaration SHA-256: 36e1f834bfffffc0c34e5ca4b702e9f19b5aa6de3e168944b257119d1ee48362

Mathematical statement. The explicit derivative function vanishes at Xstar.

It restates stationarity in the g_d notation.

```
lemma gd_xstar : gd xstar = 0 := g_deriv_xstar
```

hasDerivAt_arc_two_mul

Classification: supplement

Source: GFunction.lean:119-129

Declaration SHA-256: 68db7ce1cccf8d83346e37f43b7e16bbcad82e0649185eda0b8b1c7290d5df1

Mathematical statement. Derivative of the doubled-area arc function.

The chain rule gives $\frac{d}{dX}L(2X) = 4\sin\theta(2X)$ for $X > 0$.

```
lemma hasDerivAt_arc_two_mul {x : ℝ} (hx : 0 < x) :
  HasDerivAt (fun t => arc (2 * t)) (4 * sin (θ0f (2 * x))) x := by
  have h2x : (0:ℝ) < 2 * x := by linarith
  have hlin : HasDerivAt (fun t : ℝ => 2 * t) 2 x := by
    simpa using (hasDerivAt_id x).const_mul (2:ℝ)
  have h := (hasDerivAt_arc h2x).comp x hlin
  have heq : (arc ∘ fun t : ℝ => 2 * t) = fun t => arc (2 * t) := rfl
  rw [heq] at h
  refine h.congr_deriv ?_
  ring
```

hasDerivAt_ell

Classification: supplement

Source: MorgansArcFunction.lean:94-103

Declaration SHA-256: cae8d71b9bdc0c10b243d112a9ada6dbc5d44d1c5f710020d8f577d13d56023c

Mathematical statement. Derivative of normalized arc length.

$$\ell'(\theta) = (\sin\theta - \theta\cos\theta)/\sin^2\theta.$$

```
lemma hasDerivAt_ell {θ : ℝ} (h0 : 0 < θ) (hπ : θ < π) :
  HasDerivAt ell ((sin θ - θ * cos θ) / sin θ ^ 2) θ := by
  have hs : sin θ ≠ 0 := ne_of_gt (sin_pos_of_pos_of_lt_pi h0 hπ)
  have h := HasDerivAt.div (hasDerivAt_id θ) (Real.hasDerivAt_sin θ) hs
  simp only [id_eq, one_mul] at h
  exact h

/-- The quotient rule gives a numerator `8 sin^4 θ - 8 sin θ cos θ (θ - sin θ cos θ)`,
which collapses to `sin θ - θ cos θ` after applying `sin^2 θ + cos^2 θ = 1`. Note the
same numerator as `ell` - this is the cancellation exploited later. -/
```

hasDerivAt_g'

Classification: supplement

Source: MinimumValue.lean:35-39

Declaration SHA-256: 8f0141bd9b2ca2ed0af5052050d85c875e33a71481490dd5e2250b9a2886c5ba

Mathematical statement. The function gd is the derivative of g at every positive x .

This packages the previously established derivative formula under the derivative-function name.

```
lemma hasDerivAt_g' {x : ℝ} (hx : 0 < x) : HasDerivAt g (gd x) x :=
  hasDerivAt_g hx

/-! ## Step 1: locating `x*` -/
```

hausdorffMeasure_image_Ico_eq_Icc

Classification: supplement

Source: FrontierPerimeter.lean:142-153

Declaration SHA-256: 1c850c9b7ca6e169808ebfb87cac4902c44102258f4c3ba0c9b6433987f2a5de

Mathematical statement. Removing one arc endpoint does not change one-dimensional Hausdorff measure.

A singleton is $\mathcal{H}^1$ -null.

```
lemma hausdorffMeasure_image_Ico_eq_Icc
  {E : Type*} [EMetricSpace E] [MeasurableSpace E] [BorelSpace E]
  {f : ℝ → E} {S : Set ℝ} (hf : Set.InjOn f S)
  {u v : ℝ} (huv : u ≤ v) (hsub : Icc u v ⊆ S) :
  (μH[1] : Measure E) (f '' Ico u v) =
    (μH[1] : Measure E) (f '' Icc u v) := by
  rw [image_Ico_eq_image_Icc_diff_endpoint hf huv hsub]
  letI : NullSingletonClass (μH[1] : Measure E) :=
    Measure.nullSingletonClass_hausdorff E (by norm_num)
  exact measure_sdiff_null (measure_singleton (f v))

/-- Equally spaced partition points on the signed-arclength interval. -/
```

hausdorffMeasure_one_euclideanJoinSet

Classification: supplement

Source: FrontierPerimeter.lean:1263-1268

Declaration SHA-256: 7ebe8fae03ad531268ff769afa248cd1067ee674eba1a7f8bc77f36d3ab49ebc

Mathematical statement. The Euclidean junction set has zero one-dimensional Hausdorff measure.

Every finite subset of the Euclidean plane is $\mathcal{H}^1$ -null.

```
theorem hausdorffMeasure_one_euclideanJoinSet (q : ℝ) :
  (μH[1] : Measure EuclideanPlane) (euclideanJoinSet q) = 0 := by
  let _ := Measure.nullSingletonClass_hausdorff EuclideanPlane
  (by norm_num : (0 : ℝ) < 1)
  exact (euclideanJoinSet_finite q).measure_zero μH[1]
```


hausdorffMeasure_unitCircleArc_image_le

Classification: supplement

Source: FrontierPerimeter.lean:93-104

Declaration SHA-256: 07ee598762aa4a7964c4db549c22833ebf32ee2d325dcc190dd9a72212015319

Mathematical statement. For $r > 0$ and arbitrary real a, b , the Hausdorff length of $u_r([a, b])$ is at most $\max(b-a, 0)$.

The 1-Lipschitz image inequality gives

$$\mathcal{H}^1(u_r([a, b])) \leq \text{ofReal}(b - a).$$

Here $\text{ofReal}(x)$ denotes $\max(x, 0)$ viewed as an extended nonnegative real. No ordering of a, b and no no-full-turn hypothesis is required; if $b < a$, the interval is empty. For $a \leq b$, the bound is the signed-arclength span $b - a$.

```
lemma hausdorffMeasure_unitCircleArc_image_le {r a b : ℝ} (hr : 0 < r) :
  (μH[1] : Measure ℂ) (unitCircleArc r '' Icc a b) ≤
    ENNReal.ofReal (b - a) := by
  calc
    (μH[1] : Measure ℂ) (unitCircleArc r '' Icc a b)
      ≤ (μH[1] : Measure ℝ) (Icc a b) := by
        simp using (lipschitzWith_unitCircleArc hr).hausdorffMeasure_image_le
          (d := (1 : ℝ)) (by norm_num) (Icc a b)
    _ = ENNReal.ofReal (b - a) := by
      rw [hausdorffMeasure_real, Real.volume_Icc]

/-- Every continuous curve image has H¹ mass at least the distance between its endpoints. -/
```

image_Ico_eq_image_Icc_diff_endpoint

Classification: supplement

Source: FrontierPerimeter.lean:124-141

Declaration SHA-256: c4fb0e64c0405d8f9fa3cb2429d9a9bc01ac95be8aba1e08560d1aeae3f939e9

Mathematical statement. For an injective map, the half-open interval image removes exactly the final endpoint.

This identity makes adjacent arc pieces disjoint without changing length.

```
lemma image_Ico_eq_image_Icc_diff_endpoint
  {α : Type*} {f : ℝ → α} {S : Set ℝ} (hf : Set.InjOn f S)
  {u v : ℝ} (huv : u ≤ v) (hsub : Icc u v ⊆ S) :
  f '' Ico u v = f '' Icc u v \ {f v} := by
  ext y
  constructor
  · rintro ⟨x, hx, rfl⟩
    refine ⟨⟨x, ⟨hx.1, hx.2.le⟩, rfl⟩, ?_⟩
    simp only [mem_singleton_iff]
    intro hfv
    have hxv := hf (hsub ⟨hx.1, hx.2.le⟩) (hsub ⟨huv, le_rfl⟩) hfv
    exact hx.2.ne hxv
  · rintro ⟨⟨x, hx, rfl⟩, hne⟩
    refine ⟨x, ⟨hx.1, lt_of_le_of_ne hx.2 ?_⟩, rfl⟩
    intro hxv
    subst x
    exact hne (mem_singleton (f v))
```

integral_four_union

Classification: supplement

Source: FrontierPerimeter.lean:1562-1590

Declaration SHA-256: 8b3fbc02eae74d84636263a1b0bedcfff91c58bcc3410ac92807b51d54cc9ea4

Mathematical statement. A real function integrable on each of four pairwise Hausdorff-almost-disjoint Euclidean sets has additive integral over their union, provided the last three sets are measurable.

Let $\mu = \mathcal{H}^1$ on the Euclidean plane and $f: \mathbb{R}^2 \rightarrow \mathbb{R}$. Suppose f is integrable on each of A, B, C, D , all six pairwise intersections are μ -null, and B, C, D are measurable. Then

$$\int_{A \cup (B \cup (C \cup D))} f d\mu = \int_A f d\mu + \int_B f d\mu + \int_C f d\mu + \int_D f d\mu.$$

Successive almost-disjoint union identities prove the equality. The formal statement imposes no separate measurability premise on A .

```
theorem integral_four_union
  {f : EuclideanPlane → ℝ} {A B C D : Set EuclideanPlane}
  (hAB : AEDisjoint (μH[1] : Measure EuclideanPlane) A B)
  (hAC : AEDisjoint (μH[1] : Measure EuclideanPlane) A C)
  (hAD : AEDisjoint (μH[1] : Measure EuclideanPlane) A D)
  (hBC : AEDisjoint (μH[1] : Measure EuclideanPlane) B C)
  (hBD : AEDisjoint (μH[1] : Measure EuclideanPlane) B D)
  (hCD : AEDisjoint (μH[1] : Measure EuclideanPlane) C D)
  (hB : MeasurableSet B) (hC : MeasurableSet C) (hD : MeasurableSet D)
  (hfA : IntegrableOn f A (μH[1] : Measure EuclideanPlane))
  (hfB : IntegrableOn f B (μH[1] : Measure EuclideanPlane))
  (hfC : IntegrableOn f C (μH[1] : Measure EuclideanPlane))
  (hfD : IntegrableOn f D (μH[1] : Measure EuclideanPlane)) :
  (∫ z in A ∪ (B ∪ (C ∪ D)), f z ∂(μH[1] : Measure EuclideanPlane)) =
    (∫ z in A, f z ∂(μH[1] : Measure EuclideanPlane)) +
    ((∫ z in B, f z ∂(μH[1] : Measure EuclideanPlane)) +
     ((∫ z in C, f z ∂(μH[1] : Measure EuclideanPlane)) +
      ∫ z in D, f z ∂(μH[1] : Measure EuclideanPlane))) := by
  rw [setIntegral_union₀
    (hAB.union_right (hAC.union_right hAD))
    (hB.union (hC.union hD)).nullMeasurableSet
    hfA (hfB.union (hfC.union hfD)),
    setIntegral_union₀
    (hBC.union_right hBD)
    (hC.union hD).nullMeasurableSet
    hfB (hfC.union hfD),
    setIntegral_union₀ hCD hD.nullMeasurableSet hfC hfD]

/-- The union of all six pairwise overlaps among four Euclidean sets. -/
```

integral_semicircle_scaled

Classification: supplement

Source: CMVGeometry.lean:1759-1791

Declaration SHA-256: fe66bbeed922271b7cf0e4043f58e6b4f6651310c0d02aa776383ac42b18240d

Mathematical statement. The corresponding integral for a circle of arbitrary positive radius.

Substitution scales the unit-circle integral by the square of the radius.

```
lemma integral_semicircle_scaled {r θ : ℝ} (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  (∫ y in r * cos θ..r, 2 * √(r ^ 2 - y ^ 2)) =
    r ^ 2 * (θ - sin θ * cos θ) := by
  have hcomp := intervalIntegral.integral_comp_mul_deriv
  (a := cos θ) (b := 1) (f := fun x : ℝ ↦ r * x) (f' := fun _ ↦ r)
```

```

(g := fun y : ℝ ↦ 2 * √(r ^ 2 - y ^ 2))
(fun x _ ↦ by simp using (hasDerivAt_id x).const_mul r)
continuousOn_const (by fun_prop)
simp only [Function.comp_apply, mul_one] at hcomp
rw [← hcomp]
calc
  (∫ x in cos θ..1, 2 * √(r ^ 2 - (r * x) ^ 2) * r) =
    ∫ x in cos θ..1, (2 * r ^ 2) * √(1 - x ^ 2) := by
      apply intervalIntegral.integral_congr
      intro x hx
      dsimp
      have hx' : x ∈ Icc (-1 : ℝ) 1 := by
        have hcos : -1 ≤ cos θ := neg_one_le_cos θ
        have hcos' : cos θ ≤ 1 := cos_le_one θ
        have : x ∈ Icc (cos θ) 1 := by
          simp [uIcc, hcos'] using hx
        exact ⟨hcos.trans this.1, this.2⟩
      have hnonneg : 0 ≤ 1 - x ^ 2 := by nlinarith [hx'.1, hx'.2]
      rw [show r ^ 2 - (r * x) ^ 2 = r ^ 2 * (1 - x ^ 2) by ring,
        Real.sqrt_mul (sq_nonneg r), Real.sqrt_sq hr.le]
      ring
  _ = (2 * r ^ 2) * (∫ x in cos θ..1, √(1 - x ^ 2)) := by
    rw [intervalIntegral.integral_const_mul]
  _ = r ^ 2 * (θ - sin θ * cos θ) := by
    rw [integral_sqrt_one_sub_sq_from_cos hθ0 hθπ]
    ring

```

integral_semicircle_translated

Classification: supplement

Source: CMVGeometry.lean:1792-1800

Declaration SHA-256: 06721e5dc911d180205234ff69b820e369a8d82b66f142e17c0a1377b9b27f8d

Mathematical statement. The circular cross-section integral after translation.

Translation moves the integration endpoints and circle center without changing area.

```

lemma integral_semicircle_translated {a r θ : ℝ} (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  (∫ y in a + r * cos θ..a + r, 2 * √(r ^ 2 - (y - a) ^ 2)) =
    r ^ 2 * (θ - sin θ * cos θ) := by
  rw [← intervalIntegral.integral_comp_add_left
    (f := fun y : ℝ ↦ 2 * √(r ^ 2 - (y - a) ^ 2)) a]
  simp only [add_sub_cancel_left]
  exact integral_semicircle_scaled hr hθ0 hθπ

```

integral_sqrt_one_sub_sq_from_cos

Classification: supplement

Source: CMVGeometry.lean:1729-1758

Declaration SHA-256: e4a969c6c1b67c6f267db8fb9582c7cd96174fa24489265c840c646c3085a52a

Mathematical statement. Evaluation of the unit-circle cross-section integral from a cosine endpoint.

Integrating $\sqrt{1-u^2}$ yields the circular-segment area in angle coordinates.

```

lemma integral_sqrt_one_sub_sq_from_cos {θ : ℝ} (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  (∫ x in cos θ..1, √(1 - x ^ 2)) =
    (θ - sin θ * cos θ) / 2 := by
  have hcomp := intervalIntegral.integral_comp_mul_deriv
  (a := θ) (b := 0) (f := cos) (f' := fun x ↦ -sin x)
  (g := fun x : ℝ ↦ √(1 - x ^ 2))
  (fun x _ ↦ Real.hasDerivAt_cos x)
  (continuous_sin.neg.continuousOn) (by fun_prop)
  simp only [Function.comp_apply, cos_zero] at hcomp

```

```

rw [← hcomp]
calc
  (f x in 0..0, √(1 - cos x ^ 2) * -sin x) =
    f x in 0..0, -(sin x ^ 2) := by
      apply intervalIntegral.integral_congr
      intro x hx
      dsimp
      have hx' : x ∈ Icc 0 0 := by
        simpa [uIcc, h00] using hx
      rw [← Real.sin_eq_sqrt_one_sub_cos_sq]
      · ring
      · exact hx'.1
      · exact hx'.2.trans h0π
_ = f x in 0..0, sin x ^ 2 := by
  rw [intervalIntegral.integral_symm]
  simp
_ = (0 - sin 0 * cos 0) / 2 := by
  rw [integral_sin_sq]
  simp
  ring

```

intervalPushforward

Classification: supplement

Source: CMVGeometry.lean:1200-1202

Declaration SHA-256: 366d70f2307ddf912f8fe8519c486284fdacdab2c9bb76f371f9a65dd8ee807d

Mathematical statement. Pushforward of interval-restricted Lebesgue measure by a parametrization.

It expresses arclength integrals through real interval integrals.

```

def intervalPushforward (curve : ℝ → PlanePoint) (a b : ℝ) : Measure PlanePoint :=
  Measure.map curve (volume.restrict (Icc a b))

```

isometry_capComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:398-408

Declaration SHA-256: 5d4136606591a26f6abf24f8ba286ddba7154e77e9f96c81673196c1854009b0

Mathematical statement. The cap placement map preserves Euclidean distance.

Translations and the selected reflection are isometries.

```

lemma isometry_capComplexIsometryMap (c : OneSidedCircularCap) :
  Isometry (capComplexIsometryMap c) := by
  apply Isometry.of_dist_eq
  intro z w
  rw [WithLp.prod_dist_eq_add (by norm_num)]
  simp only [capComplexIsometryMap]
  split <,>
  norm_num [Real.dist_eq, sq_abs] <,>
  rw [← Real.sqrt_eq_rpow, Complex.dist_eq_re_im] <,>
  congr 1 <,> ring

```

isometry_leftCoreComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:685-695

Declaration SHA-256: a61c4ab4c222e88c43376bf46386ccf7697c506b7a23f81458ff102288a9e14a

Mathematical statement. The left core placement map is an isometry.

Only Euclidean rigid motions occur in this map.

```
lemma isometry_leftCoreComplexIsometryMap (c : StripCore) :  
  Isometry (leftCoreComplexIsometryMap c) := by  
  apply Isometry.of_dist_eq  
  intro z w  
  rw [WithLp.prod_dist_eq_add (by norm_num)]  
  simp only [leftCoreComplexIsometryMap]  
  norm_num [Real.dist_eq, sq_abs]  
  rw [← Real.sqrt_eq_rpow, Complex.dist_eq_re_im]  
  congr 1  
  ring
```

isometry_rightCoreComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:696-704

Declaration SHA-256: 2857926ac4225e60a51ede217224fdFDA63879d202e0be14411ea5c13b2fc6bf

Mathematical statement. The right core placement map is an isometry.

Distances are preserved under its translation and reflection.

```
lemma isometry_rightCoreComplexIsometryMap (c : StripCore) :  
  Isometry (rightCoreComplexIsometryMap c) := by  
  apply Isometry.of_dist_eq  
  intro z w  
  rw [WithLp.prod_dist_eq_add (by norm_num)]  
  simp only [rightCoreComplexIsometryMap]  
  norm_num [Real.dist_eq, sq_abs]  
  rw [← Real.sqrt_eq_rpow, Complex.dist_eq_re_im]
```

isometry_unitCircleArcRotation

Classification: supplement

Source: FrontierPerimeter.lean:315-322

Declaration SHA-256: 0390b164e94132c53e908a7960b31f68dc786e7272a9a9f6482fa79cade88362

Mathematical statement. For every real r, m , the map z maps to $z \exp(i m/r)$ is an isometry of the complex plane.

The multiplier $e^{im/r}$ has absolute value one, so $|R_{r,m}(z) - R_{r,m}(w)| = |z - w|$ for every $z, w \in \mathbb{C}$. Neither $r > 0$ nor $r \neq 0$ is a hypothesis of this isometry theorem.

```
lemma isometry_unitCircleArcRotation (r m : ℝ) :  
  Isometry (unitCircleArcRotation r m) := by  
  apply Isometry.of_dist_eq  
  intro z w  
  rw [Complex.dist_eq, Complex.dist_eq]  
  simp only [unitCircleArcRotation, ← sub_mul, norm_mul, Complex.norm_exp]  
  norm_num
```

joinSet

Classification: supplement

Source: FrontierPerimeter.lean:1115-1117

Declaration SHA-256: 613dcfda543c5255d80175237d7db38b3386ce55bc3215682252569ce26c42d5

Mathematical statement. The finite set of geometric boundary junctions.

Adjacent arcs or chords can overlap only at these joining points.

```
def joinSet (q : ℝ) : Set PlanePoint :=
  {(-q / 2, 1), (q / 2, 1), (-q / 2, -1), (q / 2, -1)}
```

joinSet_finite

Classification: supplement

Source: FrontierPerimeter.lean:1253-1256

Declaration SHA-256: c95df1795c640672a88890bc658373f829154b7fa3a2361c967c14e17f22b551

Mathematical statement. The coordinate junction set is finite.

It is formed from finitely many explicitly specified endpoints.

```
theorem joinSet_finite (q : ℝ) : (joinSet q).Finite := by
  simp [joinSet]

/-- The join set realized in the metric where Euclidean  $H^1$  is formed. -/
```

leftCoreBoundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:1487-1490

Declaration SHA-256: e243bfd8c440c79be3e701f3e1ba7f0b4af716d3ad6d1c3de2eb88023b1a30c8

Mathematical statement. Interval-pushforward measure on the left core arc.

It allows direct integration along that boundary component.

```
def leftCoreBoundaryMeasure (c : StripCore) : Measure PlanePoint :=
  intervalPushforward (leftCoreParam c) (coreArcStart c) (coreArcEnd c)

/-- Explicit regular boundary measure of the right core arc. -/
```

leftCoreBoundaryMeasure_integrable

Classification: supplement

Source: CMVGeometry.lean:2275-2281

Declaration SHA-256: efdd79d96b8f43a7c478608fb7242070e102fd586af27d2826a566f07542e0a9

Mathematical statement. The left core boundary component has integrable density.

Its parameter interval is finite and density is bounded.

```
lemma leftCoreBoundaryMeasure_integrable (lam : ℝ) (c : StripCore) :
  Integrable (StripDensity lam) (leftCoreBoundaryMeasure c) := by
  rw [leftCoreBoundaryMeasure]
  exact stripDensity_integrable_intervalPushforward lam (by
    unfold leftCoreParam
    fun_prop)
```

leftCoreComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:677-680

Declaration SHA-256: c0385e604fc62ea1f5b752f17a4538ba2edeedf20e52cbfb6848b3b966b2ab57

Mathematical statement. Complex-to-Euclidean placement map for the left core arc.

It translates and orients a scaled circle at the left core center.

```
noncomputable def leftCoreComplexIsometryMap (c : StripCore) (z : ℂ) :
  EuclideanPlane :=
  WithLp.toLp 2 (c.leftCenterX - z.re, z.im)
```

leftCoreParam

Classification: supplement

Source: CMVGeometry.lean:1477-1481

Declaration SHA-256: 80f940fe52c510e69a283605275f10b49a7ca369836a80a40ece896dbbf606db

Mathematical statement. Unit-speed parametrization of the left core arc.

A scaled circle is translated to the left core center.

```
def leftCoreParam (c : StripCore) (s : ℝ) : PlanePoint :=
  (c.leftCenterX - c.radius * cos (s / c.radius),
   c.radius * sin (s / c.radius))

/-- Unit-speed parametrization of the right core arc. -/
```

leftCoreParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:517-596

Declaration SHA-256: 01c2af95d70e04d0bac530bc681fda34196e33e841f34e26466b4a3a9c6ecd18

Mathematical statement. The left core parameter interval traces exactly the left core arc.

Both image inclusions are established, not merely containment in a circle.

```
theorem leftCoreParam_image_Icc (c : StripCore) :
  leftCoreParam c '' Icc (coreArcStart c) (coreArcEnd c) =
  c.leftArcTrace := by
  ext p
  constructor
  · rintro (s, hs, rfl)
    have hu := core_angle_mem' hs
    have htrig := Real.sin_sq_add_cos_sq (s / c.radius)
    have hcircle :
      ((leftCoreParam c s).1 - c.leftCenterX) ^ 2 +
      (leftCoreParam c s).2 ^ 2 = c.radius ^ 2 := by
      simp only [leftCoreParam, Prod.fst, Prod.snd]
      nlinarith [sq_nonneg c.radius]
    have habs : |s / c.radius| ≤ c.sideAngle := abs_le.mpr hu
    have hcos : Real.cos c.sideAngle ≤ Real.cos (s / c.radius) := by
      simp only [Real.cos_abs] using
      (Real.cos_le_cos_of_nonneg_of_le_pi (abs_nonneg (s / c.radius))
       c.sideAngle_lt_pi.le habs)
    have hx : (leftCoreParam c s).1 ≤ -c.chord / 2 := by
      simp only [leftCoreParam, Prod.fst]
      unfold StripCore.leftCenterX
      nlinarith [mul_le_mul_of_nonneg_left hcos c.radius_pos.le]
    exact ⟨hcircle, hx, coreParam_y_bound' c hs⟩
  · intro hp
    change (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2 ∧
      p.1 ≤ -c.chord / 2 ∧ |p.2| ≤ 1 at hp
    let u : ℝ := Real.arcsin (p.2 / c.radius)
    have hp2 := abs_le.mp hp.2.2
    have hlo : -c.curvature ≤ p.2 / c.radius := by
      apply (le_div_iff c.radius_pos).2
      nlinarith [core_radius_mul_curvature' c]
    have hhi : p.2 / c.radius ≤ c.curvature := by
```

```

    apply (div_le_iff c.radius_pos).2
    nlinarith [core_radius_mul_curvature' c]
have hu : u ∈ Icc (-c.sideAngle) c.sideAngle := by
  constructor
  · simp [u, StripCore.sideAngle] using Real.arcsin_le_arcsin hlo
  · simp [u, StripCore.sideAngle] using Real.arcsin_le_arcsin hhi
have hratio : p.2 / c.radius ∈ Icc (-1 : ℝ) 1 :=
  ⟨(by linarith [c.curvature_le_one] : (-1 : ℝ) ≤ -c.curvature) |>.trans hlo,
    hhi.trans c.curvature_le_one)
have hsinu : Real.sin u = p.2 / c.radius :=
  Real.sin_arcsin hratio.1 hratio.2
have hy : c.radius * Real.sin u = p.2 := by
  rw [hsinu]
  field_simp [ne_of_gt c.radius_pos]
have hcostheta : 0 ≤ Real.cos c.sideAngle :=
  Real.cos_nonneg_of_mem_Icc
  ⟨by linarith [c.sideAngle_pos, Real.pi_pos],
    c.sideAngle_le_pi_div_two)
have hxu : p.1 - c.leftCenterX ≤ 0 := by
  unfold StripCore.leftCenterX
  nlinarith [mul_nonneg c.radius_pos.le hcostheta]
have hcosu : 0 ≤ Real.cos u :=
  Real.cos_nonneg_of_mem_Icc
  ⟨(neg_le_neg c.sideAngle_le_pi_div_two).trans hu.1,
    hu.2.trans c.sideAngle_le_pi_div_two)
have htrig := Real.sin_sq_add_cos_sq u
have hsquares :
  (p.1 - c.leftCenterX) ^ 2 = (c.radius * Real.cos u) ^ 2 := by
  calc
    _ = c.radius ^ 2 - p.2 ^ 2 := by nlinarith [hp.1]
    _ = (c.radius * Real.cos u) ^ 2 := by rw [- hy]; nlinarith
have hcosprod : 0 ≤ c.radius * Real.cos u :=
  mul_nonneg c.radius_pos.le hcosu
have hx : p.1 = c.leftCenterX - c.radius * Real.cos u := by
  rcases sq_eq_sq_iff_eq_or_eq_neg.mp hsquares with hpos | hneg
  · have hz : p.1 - c.leftCenterX = 0 := by nlinarith
    nlinarith
  · linarith
refine (c.radius * u, ?_, ?_)
· change -c.radius * c.sideAngle ≤ c.radius * u ∧
  c.radius * u ≤ c.radius * c.sideAngle
  constructor <|> nlinarith [c.radius_pos, hu.1, hu.2]
· have hdiv : c.radius * u / c.radius = u := by
  field_simp [ne_of_gt c.radius_pos]
  apply Prod.ext
  · simp [leftCoreParam, hdiv, hx]
  · simp [leftCoreParam, hdiv, hy]

```

leftCore_strip_density

Classification: supplement

Source: CMVGeometry.lean:1662-1667

Declaration SHA-256: 3222d64a655ce3af15a638606821b923e89c31ae75558031c08763aeb76b72b6

Mathematical statement. Density is one along the left core arc.

The arc stays in the closed strip.

```

theorem leftCore_strip_density (lam : ℝ) (c : StripCore) {s : ℝ}
  (hs : s ∈ Icc (coreArcStart c) (coreArcEnd c)) :
  StripDensity lam (leftCoreParam c s) = 1 := by
  rw [StripDensity, if_pos]
  simp [leftCoreParam] using coreParam_y_bound c hs

```


leftCore_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1675-1694

Declaration SHA-256: e333f475432d30c40456ab56a9f5187cebdb61288c7e07d5eec293804ac77f49

Mathematical statement. Weighted length of the left core arc is its parameter span.

It contributes $2R\text{arcsinh}$ at density one.

```
theorem leftCore_weightedIntegral (lam : ℝ) (c : StripCore) :
  ∫ p, StripDensity lam p ∂leftCoreBoundaryMeasure c = 2 * ell c.sideAngle := by
  rw [leftCoreBoundaryMeasure]
  rw [integral_intervalPushforward (leftCoreParam_measurable c)
    (measurable_stripDensity lam)]
  calc
    (∫ s in Icc (coreArcStart c) (coreArcEnd c),
      StripDensity lam (leftCoreParam c s)) =
      ∫ _s in Icc (coreArcStart c) (coreArcEnd c), 1 := by
        apply integral_congr_ae
        filter_upwards [ae_restrict_mem measurableSet_Icc] with s hs
        exact leftCore_strip_density lam c hs
    _ = coreArcEnd c - coreArcStart c := by
      have horder : coreArcStart c ≤ coreArcEnd c := by
        dsimp [coreArcStart, coreArcEnd]
        nlinarith [c.radius_pos, c.sideAngle_pos]
      simp [horder]
    _ = 2 * ell c.sideAngle := core_single_arc_length c

/-- Weighted integral of the right core arc equals its Euclidean length. -/
```

lintegral_semicircle_translated

Classification: supplement

Source: CMVGeometry.lean:1801-1819

Declaration SHA-256: 3c8e90cd635d133e832b64fe2731d46db4f68d0a1a341d292df3904cf66e67e8

Mathematical statement. Nonnegative-integral form of the translated circular cross-section formula.

This is the form used by the planar volume computation.

```
lemma lintegral_semicircle_translated {a r θ : ℝ} (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  (∫ y in Icc (a + r * cos θ) (a + r),
    ENNReal.ofReal (2 * √(r ^ 2 - (y - a) ^ 2))) =
    ENNReal.ofReal (r ^ 2 * (θ - sin θ * cos θ)) := by
  have hcos : cos θ ≤ 1 := cos_le_one θ
  have hab : a + r * cos θ ≤ a + r := by nlinarith
  have hcont : Continuous (fun y : ℝ ↦ 2 * √(r ^ 2 - (y - a) ^ 2)) := by
    fun_prop
  have hint :
    IntegrableOn (fun y : ℝ ↦ 2 * √(r ^ 2 - (y - a) ^ 2))
      (Icc (a + r * cos θ) (a + r)) :=
    hcont.integrableOn_Icc
  rw [← MeasureTheory.ofReal_integral_eq_lintegral_ofReal hint
    (Filter.Eventually.of_forall fun _ ↦ by positivity)]
  congr 1
  rw [integral_Icc_eq_integral_Ioc, ← intervalIntegral.integral_of_le hab]
  exact integral_semicircle_translated hr hθ0 hθπ
```

lipschitzWith_unitCircleArc

Classification: supplement

Source: FrontierPerimeter.lean:33-54

Declaration SHA-256: 6ddc026dd017d7b20d15e385a48bc34f5bcbfcad3796885aedaeb1fd06730313

Mathematical statement. For every $r > 0$, the signed-arclength parametrization u_r is 1-Lipschitz on the real line.

For $r > 0$ and all $s, t \in \mathbb{R}$, $|u_r(s) - u_r(t)| \leq |s - t|$. The estimate $|e^{i\delta} - 1| \leq |\delta|$ with $\delta = (s - t)/r$ cancels the outer factor r . The bound uses signed-arclength distance, not angular distance.

```
lemma lipschitzWith_unitCircleArc {r : ℝ} (hr : 0 < r) :
  LipschitzWith 1 (unitCircleArc r) := by
  rw [lipschitzWith_iff_dist_le_mul]
  intro s t
  simp only [NNReal.coe_one, one_mul, Complex.dist_eq]
  rw [unitCircleArc_sub_factor hr.ne', norm_mul, norm_mul]
  have hexp : ‖Complex.exp (Complex.I * (t / r : ℝ))‖ = 1 := by
    rw [Complex.norm_exp]
    simp
  rw [hexp, mul_one]
  have hnormr : ‖(r : ℂ)‖ = r := by simp [abs_of_pos hr]
  rw [hnormr]
  calc
    r * ‖Complex.exp (Complex.I * ((s - t) / r : ℝ)) - 1‖
      ≤ r * ‖(s - t) / r‖ := by
        gcongr
        exact Real.norm_exp_I_mul_ofReal_sub_one_le
    _ = ‖s - t‖ := by
      rw [norm_div, Real.norm_eq_abs, Real.norm_eq_abs, abs_of_pos hr]
      field_simp
    _ = dist s t := by simp only [Real.norm_eq_abs, Real.dist_eq]
```

measurableSet_euclidean_capTrace

Classification: supplement

Source: FrontierPerimeter.lean:1104-1109

Declaration SHA-256: 345ff49d99187e89c9b6a7558ffa41e164108dfd32b3dc525b699de014a17b5a

Mathematical statement. The Euclidean cap trace is measurable.

The continuous arc image is compact.

```
theorem measurableSet_euclidean_capTrace (c : OneSidedCircularCap) :
  MeasurableSet (planeEuclideanHomeomorph '' c.arcTrace) := by
  rw [← realizedCapParam_image_Icc]
  exact (isCompact_Icc.image (continuous_realizedCapParam c)).measurableSet

/-- Union of all six pairwise intersections of four primitive traces. -/
```

measurableSet_euclidean_leftArcTrace

Classification: supplement

Source: FrontierPerimeter.lean:1089-1093

Declaration SHA-256: a31ed55013e95ae690ffcc5bab6587c2933302f4970b8b673a167e6c53e1b7f8

Mathematical statement. The Euclidean left core trace is measurable.

It is a compact continuous image of a closed bounded interval.

```
theorem measurableSet_euclidean_leftArcTrace (c : StripCore) :
  MeasurableSet (planeEuclideanHomeomorph '' c.leftArcTrace) := by
```

```
rw [← realizedLeftCoreParam_image_Icc]
exact (isCompact_Icc.image (continuous_realizedLeftCoreParam c)).measurableSet
```

measurableSet_euclidean_rightArcTrace

Classification: supplement

Source: FrontierPerimeter.lean:1094-1098

Declaration SHA-256: 771198ca90f5054f065ccd16a46e41b5fce0886173f7b3c654e6b61824be845e

Mathematical statement. The Euclidean right core trace is measurable.

Compactness of the parameter image gives measurability.

```
theorem measurableSet_euclidean_rightArcTrace (c : StripCore) :
  MeasurableSet (planeEuclideanHomeomorph '' c.rightArcTrace) := by
  rw [← realizedRightCoreParam_image_Icc]
  exact (isCompact_Icc.image (continuous_realizedRightCoreParam c)).measurableSet
```

measurableSet_euclidean_segmentCarrier

Classification: supplement

Source: FrontierPerimeter.lean:1099-1103

Declaration SHA-256: 82897ecf0c0191705cb160c8a089d85dbe55e0d4633da94f20d433c8b2862b71

Mathematical statement. The Euclidean segment carrier is measurable.

A closed finite segment is compact and Borel.

```
theorem measurableSet_euclidean_segmentCarrier (seg : HorizontalSegment) :
  MeasurableSet (planeEuclideanHomeomorph '' seg.carrier) := by
  rw [← realizedSegmentParam_image_carrier]
  exact (isCompact_Icc.image (continuous_realizedSegmentParam seg)).measurableSet
```

measurable_stripDensity

Classification: supplement

Source: CMVGeometry.lean:1234-1240

Declaration SHA-256: b6fc908d8bba478440a5c35f7c75a94366dc691bdf7d2125848e2a8f512284af

Mathematical statement. The strip density is measurable.

It is constant on each of two measurable regions.

```
theorem measurable_stripDensity (lam : ℝ) : Measurable (StripDensity lam) := by
  unfold StripDensity
  exact Measurable.ite (by measurability) measurable_const measurable_const

/-! ## One-sided circular caps -/

/-- The cap arc, parametrized by signed arclength. -/
```

monotone_capPartitionPoint

Classification: supplement

Source: FrontierPerimeter.lean:167-174

Declaration SHA-256: 62e77199a2eb7f24e5f93307536675c49f368eab2d84be96574f544f2152a857

Mathematical statement. For $r > 0$, $\theta > 0$ and a positive natural N , the partition formula is monotone in the natural index.

The step $2r\theta/N$ is positive under these hypotheses. Thus $i \leq j$ implies $p_i \leq p_j$ for all natural indices; the monotonicity statement does not restrict the index to $i \leq N$.

```
lemma monotone_capPartitionPoint {r theta : ℝ} (hr : 0 < r) (htheta : 0 < theta)
  {N : ℕ} (hN : 0 < N) : Monotone (capPartitionPoint r theta N) := by
  intro i j hij
  unfold capPartitionPoint
  have hden : 0 < (N : ℝ) := by positivity
  have hstep : 0 ≤ 2 * r * theta / (N : ℝ) := (div_pos (by positivity) hden).le
  gcongr
```

ofReal_dist_le_hausdorffMeasure_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:105-123

Declaration SHA-256: 272ec4c298bc0f3582f8dd04d91ee8113b10ec4ec1c37538e792ec5391cc0936

Mathematical statement. A continuous path's Hausdorff length bounds the distance between its endpoints.

Applied to subarcs, this gives lower bounds by polygonal chord lengths.

```
lemma ofReal_dist_le_hausdorffMeasure_image_Icc
  {E : Type*} [MetricSpace E] [MeasurableSpace E] [BorelSpace E]
  {f : ℝ → E} (hf : Continuous f) {u v : ℝ} (huv : u ≤ v) :
  ENNReal.ofReal (dist (f u) (f v)) ≤
    (μH[1] : Measure E) (f '' Icc u v) := by
  let g : E → ℝ := fun z => dist (f u) z
  have hg : LipschitzWith 1 g := LipschitzWith.dist_right (f u)
  have hI : Icc 0 (dist (f u) (f v)) ⊆ g '' (f '' Icc u v) := by
    have hIV := intermediate_value_Icc huv (hg.continuous.comp hf).continuousOn
    simp only [g, dist_self, Set.image_image, Function.comp_apply] using hIV
  calc
    ENNReal.ofReal (dist (f u) (f v)) =
      (μH[1] : Measure ℝ) (Icc 0 (dist (f u) (f v))) := by
        rw [hausdorffMeasure_real, Real.volume_Icc, sub_zero]
    _ ≤ (μH[1] : Measure ℝ) (g '' (f '' Icc u v)) := measure_mono hI
    _ ≤ (μH[1] : Measure E) (f '' Icc u v) := by
      simpa using hg.hausdorffMeasure_image_le (d := (1 : ℝ)) (by norm_num)
      (f '' Icc u v)
```

pairwiseOverlap

Classification: supplement

Source: FrontierPerimeter.lean:1110-1114

Declaration SHA-256: f2051dcd764bb55c1f4c980a58c56ba3f51d928d6141df3262dbdd919f1b5fbd

Mathematical statement. The union of pairwise intersections among four boundary pieces.

It packages all possible double-counting locations in a four-piece frontier sum.

```
def pairwiseOverlap (A B C D : Set PlanePoint) : Set PlanePoint :=
  (A ∩ B) ∪ ((A ∩ C) ∪ ((A ∩ D) ∪
    ((B ∩ C) ∪ ((B ∩ D) ∪ (C ∩ D)))))

/-- The four possible primitive joins in either assembly. -/
```

pairwise_aedisjoint_of_overlap_null

Classification: supplement

Source: FrontierPerimeter.lean:1598-1616

Declaration SHA-256: 4b46ea8bfc070cb38c17e38f03c2486a1eb119281a4706f7a55c5da843e21d49

Mathematical statement. Null total overlap implies pairwise almost-everywhere disjointness.

Each pairwise intersection is a subset of the null overlap union.

```
theorem pairwise_aedisjoint_of_overlap_null
  {A B C D : Set EuclideanPlane}
  (hnull : (μH[1] : Measure EuclideanPlane)
    (euclideanPairwiseOverlap A B C D) = 0) :
  AEDisjoint (μH[1] : Measure EuclideanPlane) A B ^
  AEDisjoint (μH[1] : Measure EuclideanPlane) A C ^
  AEDisjoint (μH[1] : Measure EuclideanPlane) A D ^
  AEDisjoint (μH[1] : Measure EuclideanPlane) B C ^
  AEDisjoint (μH[1] : Measure EuclideanPlane) B D ^
  AEDisjoint (μH[1] : Measure EuclideanPlane) C D := by
  refine ⟨measure_mono_null (fun _ hz => Or.inl hz) hnull,
    measure_mono_null (fun _ hz => Or.inr (Or.inl hz)) hnull,
    measure_mono_null (fun _ hz => Or.inr (Or.inr (Or.inl hz))) hnull,
    measure_mono_null (fun _ hz => Or.inr (Or.inr (Or.inr (Or.inl hz)))) hnull,
    measure_mono_null
      (fun _ hz => Or.inr (Or.inr (Or.inr (Or.inr (Or.inl hz)))) hnull,
    measure_mono_null
      (fun _ hz => Or.inr (Or.inr (Or.inr (Or.inr (Or.inr hz)))) hnull⟩
```

pi_div_eight_pos

Classification: supplement

Source: GFunction.lean:64-65

Declaration SHA-256: 69407ed5917d0d3fc3922aeeb807cf1b44f3f653f3ebf198c99ff7ac4ce4eba5

Mathematical statement. $\pi/8$ is positive.

It is the normalized area of a unit-chord semicircle.

```
lemma pi_div_eight_pos : (0:ℝ) < π / 8 := by linarith [pi_pos]
```

pi_div_sixteen_lt_xstar

Classification: supplement

Source: MinimumValue.lean:42-48

Declaration SHA-256: 57b6eb878ba5fce5bf94c8fc02b54dbda28d42d21c498f918d5255530a944bbc

Mathematical statement. Xstar is greater than $\pi/16$.

The stationary point lies after the threshold where the doubled cap first becomes semicircular.

```
lemma pi_div_sixteen_lt_xstar : π / 16 < xstar := by
  by_contra h
  push Not at h
  have := g_deriv_neg xstar_pos h
  rw [← gd, gd_xstar] at this
  linarith
```

pi_div_two_le_00f

Classification: supplement

Source: GFunction.lean:103-114

Declaration SHA-256: 00f928404b32c84bb29cfcfab41ea9d1b3d396a41398ba66120f2ba22a120cbe

Mathematical statement. Normalized area at least $\pi/8$ has inverse angle at least $\pi/2$.

This locates the semicircular and major-cap branch.

```
lemma pi_div_two_le_00f {x : ℝ} (h : π / 8 ≤ x) : π / 2 ≤ 00f x := by
  rcases eq_or_lt_of_le h with heq | hlt
  · rw [← heq, 00f_pi_div_eight]
  · have := 00f_strictMonoOn (mem_Ioi.mpr pi_div_eight_pos)
    (mem_Ioi.mpr (lt_trans pi_div_eight_pos hlt)) hlt
    rw [00f_pi_div_eight] at this
    exact le_of_lt this

/-! ## The function `g` and its derivative -/

/-- `g x = 2 * arc x - arc (2 * x)`. The cap-substitution argument of CMV
Theorem 3.16 succeeds exactly when `g x > 1 / lam`. -/
```

pi_div_two_le_00f_two_mul

Classification: supplement

Source: MinimumValue.lean:65-68

Declaration SHA-256: 84550d9c9294f46b93923eb76ddfecdc68dbdc45510cbd269c2f4b8188b9c71e4

Mathematical statement. In the mixed-area interval, the doubled inverse angle is at least $\pi/2$.

Doubling an area at least $\pi/16$ reaches the semicircle threshold.

```
lemma pi_div_two_le_00f_two_mul {x : ℝ} (hx : x ∈ Ioo (π / 16) (π / 8)) :
  π / 2 ≤ 00f (2 * x) :=
  pi_div_two_le_00f (by linarith [hx.1])
```

pi_sub_0star_mem

Classification: supplement

Source: Minimiser.lean:161-164

Declaration SHA-256: 2bfa7c68cdb66f6586eaf8c23f7f915842c521244d8070de93397cda7a77e466

Mathematical statement. The supplementary root angle lies in the valid cap domain.

$0 < \pi - \theta_* < \pi$.

```
lemma pi_sub_0star_mem : π - 0star ∈ Ioo 0 π :=
  ⟨by linarith [0star_lt_pi], by linarith [0star_pos]⟩

/-- The doubled area corresponds to the supplementary angle. -/
```

realizedCapParam

Classification: supplement

Source: FrontierPerimeter.lean:430-433

Declaration SHA-256: 95b1661a30d7b5d3f17854c6eabbe1a4da9d944d9ad542e596607c6cb4fd12b0

Mathematical statement. The cap parametrization realized in Euclidean L2 coordinates.

It is the coordinate parametrization followed by the Euclidean homeomorphism.

```
noncomputable def realizedCapParam (c : OneSidedCircularCap) (s : ℝ) :  
  EuclideanPlane :=  
  planeEuclideanHomeomorph (capParam c s)
```

realizedCapParam_eq

Classification: supplement

Source: FrontierPerimeter.lean:434-441

Declaration SHA-256: 9c421cce21d93c49e9d419ecc30c8d31de995d391da25f034d29cf3f0bd85599

Mathematical statement. The realized cap parametrization agrees with the scaled and placed complex arc.

This identity transfers complex-circle length calculations to the geometric cap.

```
lemma realizedCapParam_eq (c : OneSidedCircularCap) (s : ℝ) :  
  realizedCapParam c s = capComplexIsometryMap c (unitCircleArc c.radius s) := by  
  rw [show realizedCapParam c s = WithLp.toLp 2 (capParam c s) by rfl]  
  cases hside : c.side <=>  
  simp [capParam, OneSidedCircularCap.arcPoint, capComplexIsometryMap,  
    unitCircleArc_re, unitCircleArc_im, hside] <=>  
  ring
```

realizedCapParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:1071-1080

Declaration SHA-256: a37b40496e1e5c2ae0b778fe01e7fee7a91557ce9473bf6e0b7755afebbd941b

Mathematical statement. The realized cap image equals the Euclidean cap trace.

Transport the coordinate image identity to the Euclidean realization.

```
theorem realizedCapParam_image_Icc (c : OneSidedCircularCap) :  
  realizedCapParam c '' Icc (capStart c) (capEnd c) =  
  planeEuclideanHomeomorph '' c.arcTrace := by  
  calc  
  _ = planeEuclideanHomeomorph ''  
    (capParam c '' Icc (capStart c) (capEnd c)) := by  
    rw [Set.image_image]  
    rfl  
  _ = _ := by rw [capParam_image_Icc_eq_arcTrace]
```

realizedLeftCoreParam

Classification: supplement

Source: FrontierPerimeter.lean:705-708

Declaration SHA-256: 854e6d1ea17e2b2d6576f273610fd43051ac61c0bb7e75c185b3d3aa821f7da4

Mathematical statement. Left core arc parametrization in Euclidean L2 coordinates.

This is the surface on which Hausdorff measure is calculated.

```
noncomputable def realizedLeftCoreParam (c : StripCore) (s : ℝ) :  
  EuclideanPlane :=  
  planeEuclideanHomeomorph (leftCoreParam c s)
```

realizedLeftCoreParam_eq

Classification: supplement

Source: FrontierPerimeter.lean:713-719

Declaration SHA-256: e2c0026193d6fa0dd58c252b75c6df1a108616049c60130059ddb93311c0b2a0

Mathematical statement. The Euclidean left core parameter equals its scaled complex-circle representation.

The equality transfers the known radius-scaled, signed-arclength circle measure to the left core arc.

```
lemma realizedLeftCoreParam_eq (c : StripCore) (s : ℝ) :
  realizedLeftCoreParam c s =
    leftCoreComplexIsometryMap c (unitCircleArc c.radius s) := by
  rw [show realizedLeftCoreParam c s = WithLp.toLp 2 (leftCoreParam c s) by rfl]
  simp [leftCoreParam, leftCoreComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im]
```

realizedLeftCoreParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:801-810

Declaration SHA-256: 43b7188db3ec73adee2747ff0dc65e5368084e5e17965fa7fdaa8f3c9301ccd7

Mathematical statement. The realized left parameter image equals the Euclidean left trace.

Transport the coordinate image identity through the homeomorphism.

```
theorem realizedLeftCoreParam_image_Icc (c : StripCore) :
  realizedLeftCoreParam c '' Icc (coreArcStart c) (coreArcEnd c) =
    planeEuclideanHomeomorph '' c.leftArcTrace := by
  calc
    _ = planeEuclideanHomeomorph ''
      (leftCoreParam c '' Icc (coreArcStart c) (coreArcEnd c)) := by
      rw [Set.image_image]
      rfl
    _ = _ := by rw [leftCoreParam_image_Icc]
```

realizedRightCoreParam

Classification: supplement

Source: FrontierPerimeter.lean:709-712

Declaration SHA-256: 4f8de565538cdbe799e2faf0a2d6452b4030fcfacd5b43a7347015b8e46924a9

Mathematical statement. Right core arc parametrization in Euclidean L2 coordinates.

It is obtained by transporting the coordinate parametrization.

```
noncomputable def realizedRightCoreParam (c : StripCore) (s : ℝ) :
  EuclideanPlane :=
  planeEuclideanHomeomorph (rightCoreParam c s)
```

realizedRightCoreParam_eq

Classification: supplement

Source: FrontierPerimeter.lean:720-726

Declaration SHA-256: 4aab5f14bc5465771d4021be2ff0ba59e74b69eeaba0b9a9c834bd0bf72ed625

Mathematical statement. The Euclidean right core parameter equals its complex-circle representation.

It is the right-side counterpart of the parameter identification.


```
lemma realizedRightCoreParam_eq (c : StripCore) (s : ℝ) :
  realizedRightCoreParam c s =
    rightCoreComplexIsometryMap c (unitCircleArc c.radius s) := by
  rw [show realizedRightCoreParam c s = WithLp.toLp 2 (rightCoreParam c s) by rfl]
  simp [rightCoreParam, rightCoreComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im]
```

realizedRightCoreParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:811-820

Declaration SHA-256: e28432904f0d4f1c849ed6ed56e30bdae00185a8e05f01ccceab8ef89d16f29b

Mathematical statement. The realized right parameter image equals the Euclidean right trace.

This identifies the measured image with the actual boundary set.

```
theorem realizedRightCoreParam_image_Icc (c : StripCore) :
  realizedRightCoreParam c '' Icc (coreArcStart c) (coreArcEnd c) =
    planeEuclideanHomeomorph '' c.rightArcTrace := by
  calc
  _ = planeEuclideanHomeomorph ''
    (rightCoreParam c '' Icc (coreArcStart c) (coreArcEnd c)) := by
  rw [Set.image_image]
  rfl
  _ = _ := by rw [rightCoreParam_image_Icc]
```

realizedSegmentParam

Classification: supplement

Source: FrontierPerimeter.lean:864-867

Declaration SHA-256: 4c1728c18c2df9a1da871e357fce4f6439c6307fe654c71c273fe06b8caaa0fd

Mathematical statement. The segment parametrization in Euclidean L2 coordinates.

It transports the horizontal coordinate segment to the length-measure space.

```
noncomputable def realizedSegmentParam (seg : HorizontalSegment) (s : ℝ) :
  EuclideanPlane :=
  planeEuclideanHomeomorph (segmentParam seg s)
```

realizedSegmentParam_image_affineSegment

Classification: supplement

Source: FrontierPerimeter.lean:873-912

Declaration SHA-256: c3be9a5bc2d447d5c5665690a46411881b4f0933eb5c3cf50deaa41a88946c72

Mathematical statement. The realized image is the Euclidean affine segment between the endpoints.

This allows use of the standard Hausdorff measure of a line segment.

```
theorem realizedSegmentParam_image_affineSegment (seg : HorizontalSegment) :
  realizedSegmentParam seg '' Icc (segmentStart seg) (segmentEnd seg) =
    affineSegment ℝ
      (planeEuclideanHomeomorph seg.leftEndpoint)
      (planeEuclideanHomeomorph seg.rightEndpoint) := by
  rw [affineSegment]
  ext p
  constructor
  · rintro ⟨s, hs, rfl⟩
    have hs' : -seg.chord / 2 ≤ s ∧ s ≤ seg.chord / 2 := by
```

```

    simp only [segmentStart, segmentEnd, mem_Icc] using hs
  let t : ℝ := (s + seg.chord / 2) / seg.chord
  refine (t, ?_, ?_)
  · constructor
    · exact div_nonneg (by linarith) seg.chord_pos.le
    · exact (div_le_one seg.chord_pos).2 (by linarith)
  · rw [AffineMap.lineMap_apply_module, WithLp.ext_iff]
    apply Prod.ext
    · simp [realizedSegmentParam, planeEuclideanHomeomorph_apply, segmentParam,
      HorizontalSegment.leftEndpoint, HorizontalSegment.rightEndpoint, t]
      field_simp [ne_of_gt seg.chord_pos]
      ring
    · simp [realizedSegmentParam, planeEuclideanHomeomorph_apply, segmentParam,
      HorizontalSegment.leftEndpoint, HorizontalSegment.rightEndpoint]
      ring
  · rintro (t, ht, rfl)
    let s : ℝ := -seg.chord / 2 + t * seg.chord
    refine (s, ?_, ?_)
    · change -seg.chord / 2 ≤ s ∧ s ≤ seg.chord / 2
      dsimp [s]
      constructor <=> nlinarith [seg.chord_pos, ht.1, ht.2]
  · rw [AffineMap.lineMap_apply_module, WithLp.ext_iff]
    apply Prod.ext
    · simp [realizedSegmentParam, planeEuclideanHomeomorph_apply, segmentParam,
      HorizontalSegment.leftEndpoint, HorizontalSegment.rightEndpoint, s]
      ring
    · simp [realizedSegmentParam, planeEuclideanHomeomorph_apply, segmentParam,
      HorizontalSegment.leftEndpoint, HorizontalSegment.rightEndpoint]
      ring

```

realizedSegmentParam_image_carrier

Classification: supplement

Source: FrontierPerimeter.lean:913-922

Declaration SHA-256: fc21a65690cb2d00a27cb6aebde20046f1dffbc15b7f30d3caf58ac1aefd1725

Mathematical statement. The realized image equals the Euclidean image of the coordinate carrier.

The affine-segment and coordinate-carrier descriptions agree.

```

theorem realizedSegmentParam_image_carrier (seg : HorizontalSegment) :
  realizedSegmentParam seg '' Icc (segmentStart seg) (segmentEnd seg) =
    planeEuclideanHomeomorph '' seg.carrier := by
  calc
    _ = planeEuclideanHomeomorph ''
      (segmentParam seg '' Icc (segmentStart seg) (segmentEnd seg)) := by
      rw [Set.image_image]
      rfl
    _ = _ := by rw [segmentParam_image_Icc]

```

replacement_pairwiseOverlap_subset_joinSet

Classification: supplement

Source: FrontierPerimeter.lean:1204-1252

Declaration SHA-256: aade619b0e98a3d318437558f408789f4a791fd841078c5489c3fcd97b06ed64

Mathematical statement. Replacement boundary overlaps are confined to the junctions.

The assertion includes intersections involving the exposed lower chord.

```

theorem replacement_pairwiseOverlap_subset_joinSet (a : ReplacementAssembly) :
  pairwiseOverlap
    (StripCore.leftArcTrace a.core)
    (StripCore.rightArcTrace a.core)
    (OneSidedCircularCap.arcTrace a.upperCap)

```

```

    a.exposedLowerChord.carrier  $\subseteq$  joinSet a.core.chord := by
intro p hp
simp only [pairwiseOverlap, mem_union, mem_inter_iff] at hp
rcases hp with hLR | hLU | hLS | hRU | hRS | hUS
· exact False.elim (by
  have h1 := hLR.1.2.1
  have hr := hLR.2.2.1
  linarith [a.core.chord_pos])
· have hyCore :  $p.2 \leq 1$  := (le_abs_self p.2).trans hLU.1.2.2
  have hyCap :  $(1 : \mathbb{R}) \leq p.2$  := by
    simp [ReplacementAssembly.upperCap] using hLU.2.2
  have hy :  $p.2 = 1$  := le_antisymm hyCore hyCap
  have hxBound := replacement_upper_arc_x_bound a hLU.2 hy
  have hx := eq_left_of_bounds hLU.1.2.1 hxBound
  have hpEq :  $p = (-a.core.chord / 2, 1)$  := Prod.ext hx hy
  simp [joinSet, hpEq]
· have hy :  $p.2 = -1$  := by
  simp [ReplacementAssembly.exposedLowerChord] using hLS.2.1
  have hxBound :  $|p.1| \leq a.core.chord / 2$  := by
    simp [ReplacementAssembly.exposedLowerChord] using hLS.2.2
  have hx := eq_left_of_bounds hLS.1.2.1 hxBound
  have hpEq :  $p = (-a.core.chord / 2, -1)$  := Prod.ext hx hy
  simp [joinSet, hpEq]
· have hyCore :  $p.2 \leq 1$  := (le_abs_self p.2).trans hRU.1.2.2
  have hyCap :  $(1 : \mathbb{R}) \leq p.2$  := by
    simp [ReplacementAssembly.upperCap] using hRU.2.2
  have hy :  $p.2 = 1$  := le_antisymm hyCore hyCap
  have hxBound := replacement_upper_arc_x_bound a hRU.2 hy
  have hx := eq_right_of_bounds hRU.1.2.1 hxBound
  have hpEq :  $p = (a.core.chord / 2, 1)$  := Prod.ext hx hy
  simp [joinSet, hpEq]
· have hy :  $p.2 = -1$  := by
  simp [ReplacementAssembly.exposedLowerChord] using hRS.2.1
  have hxBound :  $|p.1| \leq a.core.chord / 2$  := by
    simp [ReplacementAssembly.exposedLowerChord] using hRS.2.2
  have hx := eq_right_of_bounds hRS.1.2.1 hxBound
  have hpEq :  $p = (a.core.chord / 2, -1)$  := Prod.ext hx hy
  simp [joinSet, hpEq]
· have hyUpper :  $(1 : \mathbb{R}) \leq p.2$  := by
  simp [ReplacementAssembly.upperCap] using hUS.1.2
  have hySegment :  $p.2 = -1$  := by
    simp [ReplacementAssembly.exposedLowerChord] using hUS.2.1
  linarith

```

rightCoreBoundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:1491-1493

Declaration SHA-256: 33cbc13fa6fcd71207ac7e5dfb0579966744c8887bb1c9af1b95e6c688f55ea8

Mathematical statement. Interval-pushforward measure on the right core arc.

It is the right-side component measure in assembly perimeter sums.

```

def rightCoreBoundaryMeasure (c : StripCore) : Measure PlanePoint :=
  intervalPushforward (rightCoreParam c) (coreArcStart c) (coreArcEnd c)

```

rightCoreBoundaryMeasure_integrable

Classification: supplement

Source: CMVGeometry.lean:2282-2288

Declaration SHA-256: 63cd5ea28e72c7cf39aef8bfd2e698b16940d7ae74acc20f074d60c55994913c

Mathematical statement. The right core boundary component has integrable density.

The same finite-measure argument applies to the reflected arc.

```
lemma rightCoreBoundaryMeasure_integrable (lam : ℝ) (c : StripCore) :
  Integrable (StripDensity lam) (rightCoreBoundaryMeasure c) := by
  rw [rightCoreBoundaryMeasure]
  exact stripDensity_integrable_intervalPushforward lam (by
    unfold rightCoreParam
    fun_prop)
```

rightCoreComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:681-684

Declaration SHA-256: 24206b125ec8e6de86a054126e34b32568c5cb9c365728de42d394ff9cc0ede6

Mathematical statement. Placement map for the right core arc.

It uses the corresponding right-side orientation and center.

```
noncomputable def rightCoreComplexIsometryMap (c : StripCore) (z : ℂ) :
  EuclideanPlane :=
  WithLp.toLp 2 (c.rightCenterX + z.re, z.im)
```

rightCoreParam

Classification: supplement

Source: CMVGeometry.lean:1482-1486

Declaration SHA-256: 38ea0d845e704b5d320adc3936a1b6a5fba52e17ce5fb320da7c20200afa9e4d

Mathematical statement. Unit-speed parametrization of the right core arc.

It is the corresponding reflected circle parametrization.

```
def rightCoreParam (c : StripCore) (s : ℝ) : PlanePoint :=
  (c.rightCenterX + c.radius * cos (s / c.radius),
   c.radius * sin (s / c.radius))

/-- Explicit regular boundary measure of the left core arc. -/
```

rightCoreParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:597-676

Declaration SHA-256: 78bb53ade5c270538e82ae914ea314e16857498de0ee3dfc73dde7468e65c773

Mathematical statement. The right core parameter interval traces exactly the right core arc.

The reflected coordinate parametrization has the prescribed trace image.

```
theorem rightCoreParam_image_Icc (c : StripCore) :
  rightCoreParam c '' Icc (coreArcStart c) (coreArcEnd c) =
  c.rightArcTrace := by
  ext p
  constructor
  · rintro ⟨s, hs, rfl⟩
    have hu := core_angle_mem' hs
    have htrig := Real.sin_sq_add_cos_sq (s / c.radius)
    have hcircle :
      ((rightCoreParam c s).1 - c.rightCenterX) ^ 2 +
      (rightCoreParam c s).2 ^ 2 = c.radius ^ 2 := by
      simp only [rightCoreParam, Prod.fst, Prod.snd]
      nlinarith [sq_nonneg c.radius]
    have habs : |s / c.radius| ≤ c.sideAngle := abs_le.mpr hu
```

```

have hcos : Real.cos c.sideAngle ≤ Real.cos (s / c.radius) := by
  simpa only [Real.cos_abs] using
    (Real.cos_le_cos_of_nonneg_of_le_pi (abs_nonneg (s / c.radius))
      c.sideAngle_lt_pi.le habs)
have hx : c.chord / 2 ≤ (rightCoreParam c s).1 := by
  simpa only [rightCoreParam, Prod.fst]
  unfold StripCore.rightCenterX
  nlinarith [mul_le_mul_of_nonneg_left hcos c.radius_pos.le]
exact ⟨hcircle, hx, coreParam_y_bound' c hs⟩
· intro hp
change (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2 ^
  c.chord / 2 ≤ p.1 ^ |p.2| ≤ 1 at hp
let u : ℝ := Real.arcsin (p.2 / c.radius)
have hp2 := abs_le.mp hp.2.2
have hlo : -c.curvature ≤ p.2 / c.radius := by
  apply (le_div_iff₀ c.radius_pos).2
  nlinarith [core_radius_mul_curvature' c]
have hhi : p.2 / c.radius ≤ c.curvature := by
  apply (div_le_iff₀ c.radius_pos).2
  nlinarith [core_radius_mul_curvature' c]
have hu : u ∈ Icc (-c.sideAngle) c.sideAngle := by
  constructor
  · simpa [u, StripCore.sideAngle] using Real.arcsin_le_arcsin hlo
  · simpa [u, StripCore.sideAngle] using Real.arcsin_le_arcsin hhi
have hratio : p.2 / c.radius ∈ Icc (-1 : ℝ) 1 :=
  ⟨(by linarith [c.curvature_le_one] : (-1 : ℝ) ≤ -c.curvature) |>.trans hlo,
    hhi.trans c.curvature_le_one)
have hsinu : Real.sin u = p.2 / c.radius :=
  Real.sin_arcsin hratio.1 hratio.2
have hy : c.radius * Real.sin u = p.2 := by
  rw [hsinu]
  field_simp [ne_of_gt c.radius_pos]
have hcostheta : 0 ≤ Real.cos c.sideAngle :=
  Real.cos_nonneg_of_mem_Icc
  ⟨by linarith [c.sideAngle_pos, Real.pi_pos],
    c.sideAngle_le_pi_div_two)
have hxu : 0 ≤ p.1 - c.rightCenterX := by
  unfold StripCore.rightCenterX
  nlinarith [mul_nonneg c.radius_pos.le hcostheta]
have hcosu : 0 ≤ Real.cos u :=
  Real.cos_nonneg_of_mem_Icc
  ⟨(neg_le_neg c.sideAngle_le_pi_div_two).trans hu.1,
    hu.2.trans c.sideAngle_le_pi_div_two)
have htrig := Real.sin_sq_add_cos_sq u
have hsquares :
  (p.1 - c.rightCenterX) ^ 2 = (c.radius * Real.cos u) ^ 2 := by
  calc
    _ = c.radius ^ 2 - p.2 ^ 2 := by nlinarith [hp.1]
    _ = (c.radius * Real.cos u) ^ 2 := by rw [← hy]; nlinarith
have hcosprod : 0 ≤ c.radius * Real.cos u :=
  mul_nonneg c.radius_pos.le hcosu
have hx : p.1 = c.rightCenterX + c.radius * Real.cos u := by
  rcases sq_eq_sq_iff_eq_or_eq_neg.mp hsquares with hpos | hneg
  · linarith
  · have hz : p.1 - c.rightCenterX = 0 := by nlinarith
    nlinarith
refine (c.radius * u, ?_, ?_)
· change -c.radius * c.sideAngle ≤ c.radius * u ^
  c.radius * u ≤ c.radius * c.sideAngle
  constructor <|> nlinarith [c.radius_pos, hu.1, hu.2]
· have hdiv : c.radius * u / c.radius = u := by
  field_simp [ne_of_gt c.radius_pos]
  apply Prod.ext
  · simp [rightCoreParam, hdiv, hx]
  · simp [rightCoreParam, hdiv, hy]

```

rightCore_strip_density

Classification: supplement

Source: CMVGeometry.lean:1668-1674

Declaration SHA-256: d879e18591ac7ff00f0a228fad33d85d1e290fb3c77820aaf80c672ec9411662

Mathematical statement. Density is one along the right core arc.

The reflected arc also stays in the closed strip.

```
theorem rightCore_strip_density (lam : ℝ) (c : StripCore) {s : ℝ}
  (hs : s ∈ Icc (coreArcStart c) (coreArcEnd c)) :
  StripDensity lam (rightCoreParam c s) = 1 := by
  rw [StripDensity, if_pos]
  simpa [rightCoreParam] using coreParam_y_bound c hs

/-- Weighted integral of the left core arc equals its Euclidean length. -/
```

rightCore_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1695-1714

Declaration SHA-256: 12c65212e5417b38acb8560f789f61c31ede351c3de6efe2b847438f7d6cf718

Mathematical statement. Weighted length of the right core arc is its parameter span.

Its contribution is also $2R\text{Arcsinh}$.

```
theorem rightCore_weightedIntegral (lam : ℝ) (c : StripCore) :
  ∫ p, StripDensity lam p ∂rightCoreBoundaryMeasure c = 2 * ell c.sideAngle := by
  rw [rightCoreBoundaryMeasure]
  rw [integral_intervalPushforward (rightCoreParam_measurable c)
    (measurable_stripDensity lam)]
  calc
    (∫ s in Icc (coreArcStart c) (coreArcEnd c),
      StripDensity lam (rightCoreParam c s)) =
      ∫ _s in Icc (coreArcStart c) (coreArcEnd c), 1 := by
      apply integral_congr_ae
      filter_upwards [ae_restrict_mem measurableSet_Icc] with s hs
      exact rightCore_strip_density lam c hs
  _ = coreArcEnd c - coreArcStart c := by
  have horder : coreArcStart c ≤ coreArcEnd c := by
    dsimp [coreArcStart, coreArcEnd]
    nlinarith [c.radius_pos, c.sideAngle_pos]
  simp [horder]
  _ = 2 * ell c.sideAngle := core_single_arc_length c

/-- The two arc masses add to the declared strip-core boundary-arc length. -/
```

segmentBoundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:1403-1405

Declaration SHA-256: f71d719edefabe163afbd5014b1ba6aa8df0398ddf3cc380ddcc9ee628163feb

Mathematical statement. Interval-pushforward measure of the segment parametrization.

It supplies the component measure for an exposed horizontal chord.

```
def segmentBoundaryMeasure (seg : HorizontalSegment) : Measure PlanePoint :=
  intervalPushforward (segmentParam seg) (segmentStart seg) (segmentEnd seg)
```

segmentBoundaryMeasure_integrable

Classification: supplement

Source: CMVGeometry.lean:2268-2274

Declaration SHA-256: ea2176915a90a62781ded65b3f02aa8993ae5482d22cc82a781ae3b26e8f8120

Mathematical statement. The density is integrable against a segment boundary measure.

This justifies its finite weighted integral.

```
lemma segmentBoundaryMeasure_integrable (lam : ℝ) (seg : HorizontalSegment) :  
  Integrable (StripDensity lam) (segmentBoundaryMeasure seg) := by  
  rw [segmentBoundaryMeasure]  
  exact stripDensity_integrable_intervalPushforward lam (by  
    unfold segmentParam  
    fun_prop)
```

segmentEnd

Classification: supplement

Source: CMVGeometry.lean:1400-1402

Declaration SHA-256: 161abc8655ce59212c511d028e178416823adf64c523c73dae6b889ae03ed1f4

Mathematical statement. Final parameter of the horizontal segment.

The parameter span equals the segment length.

```
def segmentEnd (seg : HorizontalSegment) : ℝ := seg.chord / 2  
  
/-- Explicit regular boundary measure of a horizontal segment. -/
```

segmentEndpoint_edist

Classification: supplement

Source: FrontierPerimeter.lean:923-936

Declaration SHA-256: 667e4fe55a19b902ccd7cc29782bd67f737d862a1bddf625b46565ed0a83a1cd

Mathematical statement. The extended Euclidean distance between segment endpoints is its length.

The endpoints differ only in the horizontal coordinate by the positive length parameter.

```
lemma segmentEndpoint_edist (seg : HorizontalSegment) :  
  edist (planeEuclideanHomeomorph seg.leftEndpoint)  
    (planeEuclideanHomeomorph seg.rightEndpoint) =  
    ENNReal.ofReal seg.euclideanLength := by  
  rw [edist_dist, WithLp.prod_dist_eq_add (by norm_num)]  
  simp only [planeEuclideanHomeomorph_apply]  
  norm_num [Real.dist_eq, HorizontalSegment.leftEndpoint,  
    HorizontalSegment.rightEndpoint, HorizontalSegment.euclideanLength]  
  congr 1  
  have h : seg.midpointX -  
    (seg.midpointX + seg.chord / 2 + seg.chord / 2) = -seg.chord := by ring  
  rw [h, neg_sq, ← Real.sqrt_eq_rpow, Real.sqrt_sq_eq_abs,  
    abs_of_pos seg.chord_pos]
```

segmentParam

Classification: supplement

Source: CMVGeometry.lean:1395-1397

Declaration SHA-256: ba8ed6bbc5cd19b9c405036bea7c221ae24e5c76ea0f6ef5cc2d62c327ae1dc5

Mathematical statement. Unit-speed horizontal parametrization of a segment.

The vertical coordinate stays fixed and the horizontal coordinate varies affinely.

```
def segmentParam (seg : HorizontalSegment) (s : ℝ) : PlanePoint :=
  (seg.midpointX + s, seg.baseY)
```

segmentParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:835-863

Declaration SHA-256: 78cfdac8328c8d0308b83fec45a7d72c3a0a830edb6a3f41f9bd2da54db0f76

Mathematical statement. The segment parameter interval has exactly the segment carrier as its image.

The affine horizontal map covers the entire closed segment.

```
theorem segmentParam_image_Icc (seg : HorizontalSegment) :
  segmentParam seg '' Icc (segmentStart seg) (segmentEnd seg) = seg.carrier := by
  ext p
  constructor
  · rintro ⟨s, hs, rfl⟩
    have hs' : -seg.chord / 2 ≤ s ∧ s ≤ seg.chord / 2 := by
      simpa only [segmentStart, segmentEnd, mem_Icc] using hs
    have habs : |s| ≤ seg.chord / 2 := by
      rw [abs_le]
      constructor <=> linarith [hs'.1]
    change seg.baseY = seg.baseY ∧
      |seg.midpointX + s - seg.midpointX| ≤ seg.chord / 2
    rw [add_sub_cancel_left]
    exact ⟨rfl, habs⟩
  · intro hp
    change p.2 = seg.baseY ∧ |p.1 - seg.midpointX| ≤ seg.chord / 2 at hp
    refine ⟨p.1 - seg.midpointX, ?_, ?_⟩
    · have hb := abs_le.mp hp.2
      simpa only [segmentStart, segmentEnd, mem_Icc]
        using (show -seg.chord / 2 ≤ p.1 - seg.midpointX ∧
          p.1 - seg.midpointX ≤ seg.chord / 2 by
            constructor <=> linarith [hb.1])
    · apply Prod.ext
      · simp [segmentParam]
      · simpa [segmentParam] using hp.1.symm

@[simp] lemma planeEuclideanHomeomorph_apply (p : PlanePoint) :
  planeEuclideanHomeomorph p = WithLp.toLp 2 p := rfl
```

segmentStart

Classification: supplement

Source: CMVGeometry.lean:1398-1399

Declaration SHA-256: e90adf884523aab9d2e3af449adbc9d655d375051a612c0b4970a212a5dca61f

Mathematical statement. Initial parameter of the horizontal segment.

It corresponds to the left endpoint.


```
def segmentStart (seg : HorizontalSegment) : ℝ := -seg.chord / 2
```

segment_strip_density

Classification: supplement

Source: CMVGeometry.lean:1451-1456

Declaration SHA-256: 085d6e1c928eaa3445cf1a074627690d3998f7b411d2733a06ce5de0f2e168c0

Mathematical statement. A segment at a strip height has density one.

The hypothesis places its constant height in $[-1, 1]$.

```
theorem segment_strip_density (lam : ℝ) (seg : HorizontalSegment)
  (hbase : |seg.baseY| ≤ 1) (s : ℝ) :
  StripDensity lam (segmentParam seg s) = 1 := by
  simp [StripDensity, segmentParam, hbase]

/-- Weighted integral of a horizontal segment in the closed strip. -/
```

segment_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1457-1471

Declaration SHA-256: c5e9908e4c946fc4cb2e86548a94c923ba1b55cd20dcedb1e28930ac0fd78876

Mathematical statement. A segment inside the closed strip contributes its Euclidean length.

Integrating the constant density one over its arclength parameter interval gives its length.

```
theorem segment_weightedIntegral (lam : ℝ) (seg : HorizontalSegment)
  (hbase : |seg.baseY| ≤ 1) :
  ∫ p, StripDensity lam p ∂segmentBoundaryMeasure seg = seg.euclideanLength := by
  rw [segmentBoundaryMeasure]
  rw [integral_intervalPushforward (segmentParam_measurable seg)
    (measurable_stripDensity lam)]
  simp_rw [segment_strip_density lam seg hbase]
  unfold segmentStart segmentEnd HorizontalSegment.euclideanLength
  have horder : -seg.chord / 2 ≤ seg.chord / 2 := by
    nlinarith [seg.chord_pos]
  simp [horder]
  ring

/-! ## Left and right arcs of `StripCore` -/
```

sin_lt_sin_of_le_pi_div_two

Classification: supplement

Source: GFunction.lean:44-48

Declaration SHA-256: 8163503cd18ad5473edae576484d993037d91228ee5ab16856af467dd4e699d7

Mathematical statement. Sine is strictly increasing on the relevant lower half of the principal angle interval.

This compares inverse-area angles before $\pi/2$.

```
lemma sin_lt_sin_of_le_pi_div_two {a b : ℝ} (ha : 0 ≤ a) (hab : a < b)
  (hb : b ≤ π / 2) : sin a < sin b := by
  refine Real.strictMonoOn_sin ⟨by linarith [pi_pos], by linarith⟩
  ⟨by linarith [pi_pos], hb⟩ hab
```

sin_lt_sin_of_pi_div_two_le

Classification: supplement

Source: GFunction.lean:49-63

Declaration SHA-256: 7a9054c5f615d11a8d495c714ceecbf95cfd8c6004ddbd3d2a23ef7f883cd55e

Mathematical statement. Sine is strictly decreasing on the relevant interval above $\pi/2$.

The reversed sine order determines the sign of g' for large cap areas.

```
lemma sin_lt_sin_of_pi_div_two_le {a b : ℝ} (ha :  $\pi / 2 \leq a$ ) (hab :  $a < b$ )
  (hb :  $b \leq \pi$ ) :  $\sin b < \sin a$  := by
  have h1 :  $\sin a = \sin (\pi - a)$  := (sin_pi_sub a).symm
  have h2 :  $\sin b = \sin (\pi - b)$  := (sin_pi_sub b).symm
  rw [h1, h2]
  refine Real.strictMonoOn_sin ⟨by linarith [pi_pos], by linarith⟩
    ⟨by linarith [pi_pos], by linarith⟩ (by linarith)

/-! ## The semicircle:  $\theta$  of  $(\pi / 8) = \pi / 2$ 

At  $\theta = \pi / 2$  the segment is a half-disc of radius  $1/2$ , with area  $\pi / 8$  and
arc length  $\pi / 2$ . This pins the value of  $\theta$  at  $\pi / 8$  and recovers Morgan's
two published constants.
-/
```

sin_sub_mul_cos_pos

Classification: supplement

Source: MorgansArcFunction.lean:65-88

Declaration SHA-256: 0b2cec2b87fd3b3f54034a8d9f968aaacb6a3fa0df4915dc18cf123aab5092e4

Mathematical statement. The quantity $\sin(\theta) - \theta \cos(\theta)$ is positive on $(0, \pi)$.

Its derivative is $\theta \sin \theta > 0$ and its limiting value at zero is zero.

```
lemma sin_sub_mul_cos_pos {θ : ℝ} (hθ :  $0 < \theta$ ) (hπ :  $\theta < \pi$ ) :
   $0 < \sin \theta - \theta * \cos \theta$  := by
  have hs :  $0 < \sin \theta$  := sin_pos_of_pos_of_lt_pi hθ hπ
  rcases le_or_gt θ ( $\pi / 2$ ) with h | h
  · rcases eq_or_lt_of_le h with heq | hlt
    · rw [heq]; simp
    · have hc :  $0 < \cos \theta$  := cos_pos_of_mem_Ioo ⟨by linarith [pi_pos], hlt⟩
      have ht :  $\theta < \tan \theta$  := lt_tan hθ hlt
      rw [tan_eq_sin_div_cos] at ht
      have := (lt_div_iff hc).mp ht
      linarith
  · have hπ' :  $\theta < \pi + \pi / 2$  := by linarith [pi_pos]
    have hc :  $\cos \theta < 0$  := cos_neg_of_pi_div_two_lt_of_lt h hπ'
    nlinarith [mul_pos hθ (neg_pos.mpr hc)]

/-! ## The parametrisation

A circular segment on a chord of length  $1$  with half-central-angle  $\theta \in (0, \pi)$ 
has radius  $1 / (2 \sin \theta)$ . Its arc length and enclosed area are  $\ell$  and
 $\text{area } \theta$  below. At  $\theta = \pi / 2$  the segment is a half-disc of radius  $1/2$ , giving
 $\ell = \pi / 2$  and  $\text{area} = \pi / 8$ .

-/

/-- Arc length of a unit-chord circular segment with half-central-angle  $\theta$ . -/
```

sin_0star_pos

Classification: supplement

Source: Minimiser.lean:119-122

Declaration SHA-256: 4e8cec70a1403c2eeb99e9733f21156d53cc03eb293ae991df42fda32054bfd6

Mathematical statement. The selected root angle has positive sine.

The root lies in $(0, \pi)$.

```
lemma sin_0star_pos : 0 < sin 0star :=
  sin_pos_of_pos_of_lt_pi 0star_pos 0star_lt_pi

/-- Uniqueness, from strict monotonicity of `F` on `[0, π]`. -/
```

stripDensity_integrable0n_of_volume_ne_top

Classification: supplement

Source: CMVGeometry.lean:2289-2302

Declaration SHA-256: d26957a576fdcc692079b92ac5054cbbeda2a914af15d77e6d70ad1d418d20fd

Mathematical statement. Finite-volume carriers have integrable strip density.

For fixed real λ , the density is bounded by $\max(1, |\lambda|)$.

```
lemma stripDensity_integrable0n_of_volume_ne_top (lam : ℝ)
  {s : Set PlanePoint} (hs : volume s ≠ T) :
  IntegrableOn (StripDensity lam) s := by
  apply Measure.integrable0n_of_bounded (M := 1 + |lam|) hs
  · exact (measurable_stripDensity lam).aestronglyMeasurable
  · apply Filter.Eventually.of_forall
    intro p
    change |StripDensity lam p| ≤ 1 + |lam|
    unfold StripDensity
    split_ifs
    · simp only [abs_one]
      nlinarith [abs_nonneg lam]
    · nlinarith [abs_nonneg lam]
```

stripDensity_integrable_intervalPushforward

Classification: supplement

Source: CMVGeometry.lean:2243-2259

Declaration SHA-256: 6315c9471ea2031d23e0cb4552b23f2812fe90938358c371a246f04d7f2d8095

Mathematical statement. The bounded strip density is integrable against a finite interval pushforward.

Measurability and finite total parameter measure give integrability.

```
lemma stripDensity_integrable_intervalPushforward (lam : ℝ)
  {curve : ℝ → PlanePoint} {a b : ℝ} (hcurve : Measurable curve) :
  Integrable (StripDensity lam) (intervalPushforward curve a b) := by
  rw [intervalPushforward, integrable_map_measure
    (measurable_stripDensity lam).aestronglyMeasurable hcurve.aemeasurable]
  apply Measure.integrable0n_of_bounded (M := 1 + |lam|)
    measure_Icc_lt_top.ne
  · exact ((measurable_stripDensity lam).comp hcurve).aestronglyMeasurable
  · apply Filter.Eventually.of_forall
    intro x
    change |StripDensity lam (curve x)| ≤ 1 + |lam|
    unfold StripDensity
    split_ifs
    · simp only [abs_one]
```

```

nlinarith [abs_nonneg lam]
· nlinarith [abs_nonneg lam]

```

tendsto_cap_polygon_length

Classification: supplement

Source: FrontierPerimeter.lean:269-293

Declaration SHA-256: 32d7803e3d290677cdac6cd7bd331241d0a1e99888d3a8665e326de2ca241a11

Mathematical statement. For arbitrary real r and $\theta > 0$, the sequence $(n+1) 2r \sin(\theta/(n+1))$ converges to $2r\theta$.

For $r \in \mathbb{R}$ and $\theta > 0$,

$$\lim_{n \rightarrow \infty} (n+1) 2r \sin\left(\frac{\theta}{n+1}\right) = 2r\theta.$$

Rewrite the sequence as $2r\theta \operatorname{sinc}(\theta/(n+1))$ and use $\operatorname{sinc}(u) \rightarrow 1$ as $u \rightarrow 0$. The theorem assumes neither $r > 0$ nor $\theta < \pi$.

In its geometric application, $r > 0$ and $0 < \theta < \pi$ make these terms inscribed polygon lengths; for negative r this is an algebraic signed limit, not a polygonal length.

```

lemma tendsto_cap_polygon_length {r theta : ℝ} (htheta : 0 < theta) :
  Tendsto
    (fun n : ℕ => ((n + 1 : ℕ) : ℝ) *
      (2 * r * Real.sin (theta / ((n + 1 : ℕ) : ℝ))))
    atTop (λ 2 * r * theta) := by
  have hden : Tendsto (fun n : ℕ => ((n + 1 : ℕ) : ℝ)) atTop atTop :=
    (tendsto_natCast_atTop_iff (R := ℝ)).2 (tendsto_add_atTop_nat 1)
  have hquot : Tendsto (fun n : ℕ => theta / ((n + 1 : ℕ) : ℝ)) atTop (λ 0) :=
    hden.const_div_atTop theta
  have hsinc : Tendsto (fun n : ℕ => Real.sinc (theta / ((n + 1 : ℕ) : ℝ)))
    atTop (λ 1) := by
    convert (Real.continuous_sinc.tendsto 0).comp hquot using 1 < >
    simp [Function.comp_def]
  have hscaled : Tendsto
    (fun n : ℕ => (2 * r * theta) * Real.sinc (theta / ((n + 1 : ℕ) : ℝ)))
    atTop (λ 2 * r * theta) := by
    simpa using hsinc.const_mul (2 * r * theta)
  apply hscaled.congr'
  filter_upwards [] with n
  have hN : (((n + 1 : ℕ) : ℝ)) ≠ 0 := by positivity
  have hq : theta / (((n + 1 : ℕ) : ℝ)) ≠ 0 := div_ne_zero htheta.ne' hN
  rw [Real.sinc_of_ne_zero hq]
  field_simp

/-- Exact H¹ mass of every injective circular arc with half-angle in `(0, π)`. -/

```

three_cos_lt_of_θstar_lt

Classification: supplement

Source: Certificate.lean:100-106

Declaration SHA-256: c9da56286228487769dd40657ed749a3741fda3b807926be4434af7975f1b3b8

Mathematical statement. An upper angle bound yields a lower bound on three times the root cosine.

Cosine reverses strict order on the relevant principal angle interval.

```

lemma three_cos_lt_of_θstar_lt {t : ℝ} (ht : t ≤ π) (h : θstar < t) :
  3 * cos t < 3 * cos θstar := by
  have := Real.cos_lt_cos_of_nonneg_of_le_pi (le_of_lt θstar_pos) ht h
  linarith

/-! ## `π` to six decimals -/

```

unitCircleArc

Classification: supplement

Source: FrontierPerimeter.lean:11-13

Declaration SHA-256: 6865b0b68c07403ee39252efa35feb9b50850a4644c2bf915fb506723c7a07c7

Mathematical statement. The complex circle parametrization $u_r(s) = r \exp(i s/r)$, with signed-arclength parameter s when $r > 0$.

For arbitrary $r, s \in \mathbb{R}$, define $u_r(s) = r \exp(is/r)$. For $r > 0$ the circle has radius r , the angular coordinate is s/r , and s is signed arclength. The formula is defined also at $r = 0$: with the formal convention $s/0 = 0$, it gives $u_0(s) = 0$. Neither the radius factor nor division of the parameter by the radius may be omitted.

```
def unitCircleArc (r s : ℝ) : ℂ :=
  (r : ℂ) * Complex.exp (Complex.I * (s / r : ℝ))
```

unitCircleArcRotation

Classification: supplement

Source: FrontierPerimeter.lean:312-314

Declaration SHA-256: cfdc45400ed0d08a69b42a215854d1f81999a500771f48a7f18476aafe48ef

Mathematical statement. The complex rotation z maps to $z \exp(i m/r)$, defined for all real r, m and complex z .

Define $R_{r,m}(z) = z e^{im/r}$. For $r \neq 0$, m is a signed-arclength phase and the rotation angle is m/r . The definition is total: at $r = 0$, formal division by zero gives $R_{0,m}(z) = z$. Its complex multiplier has norm one for every r, m .

```
noncomputable def unitCircleArcRotation (r m : ℝ) (z : ℂ) : ℂ :=
  z * Complex.exp (Complex.I * (m / r : ℝ))
```

unitCircleArcRotation_apply

Classification: supplement

Source: FrontierPerimeter.lean:323-331

Declaration SHA-256: dcedc37c9428c0a722c5bf130023e2389a83c7faa4036432847247c3401f6b9b

Mathematical statement. For r nonzero and arbitrary real m, s , rotation by angle m/r sends $u_r(s)$ to $u_r(s+m)$.

The identity is $R_{r,m}(u_r(s)) = u_r(s + m)$ for $r \neq 0$. Indeed, $re^{is/r}e^{im/r} = re^{i(s+m)/r}$. The increment m shifts the signed-arclength coordinate, not the angle itself.

```
lemma unitCircleArcRotation_apply {r : ℝ} (hr : r ≠ 0) (m s : ℝ) :
  unitCircleArcRotation r m (unitCircleArc r s) =
  unitCircleArc r (s + m) := by
  unfold unitCircleArcRotation unitCircleArc
  rw [mul_assoc, ← Complex.exp_add]
  congr 2
  push_cast
  field_simp
```

unitCircleArcRotation_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:332-349

Declaration SHA-256: 6bbfe5bd6af2f20e278e2e30ff5adc9fe7c0761359113d169ea5baae1f20d6a2

Mathematical statement. For r nonzero and arbitrary real m, u, v , rotation by angle m/r translates the circle-image parameter interval by m .

The exact set identity is

$$R_{r,m}(u_r([u, v])) = u_r([u + m, v + m]).$$

It follows by translating each real parameter and using the rotation identity. The theorem assumes only $r \neq 0$; neither endpoint ordering nor a span bound is required.

```
lemma unitCircleArcRotation_image_Icc {r : ℝ} (hr : r ≠ 0)
  (m u v : ℝ) :
  unitCircleArcRotation r m '' (unitCircleArc r '' Icc u v) =
    unitCircleArc r '' Icc (u + m) (v + m) := by
  ext z
  constructor
  · rintro ⟨_, ⟨s, hs, rfl⟩, rfl⟩
    refine ⟨s + m, ⟨by linarith [hs.1], by linarith [hs.2]⟩, ?_⟩
    exact (unitCircleArcRotation_apply hr m s).symm
  · rintro ⟨t, ht, rfl⟩
    refine ⟨unitCircleArc r (t - m), ?_, ?_⟩
    · exact ⟨t - m, ⟨by linarith [ht.1], by linarith [ht.2]⟩, rfl⟩
    · rw [unitCircleArcRotation_apply hr]
      congr 1
      ring

/-- Exact H¹ mass of an arbitrary injective signed-arclength interval on a
circle. The strict span bound rules out wrapping through a full revolution. -/
```

unitCircleArc_dist

Classification: supplement

Source: FrontierPerimeter.lean:55-66

Declaration SHA-256: 1c583965a8663dcd3d0483716b4801199d867b2ff4cbe96f04d4cbeb57ff9c08

Mathematical statement. For $r > 0$, the chord distance between $u_r(s)$ and $u_r(t)$ is $2r$ times the absolute sine of $(s - t)/(2r)$.

For $r > 0$ and arbitrary $s, t \in \mathbb{R}$,

$$\text{dist}(u_r(s), u_r(t)) = r \left| 2 \sin \left(\frac{s - t}{2r} \right) \right| = 2r \left| \sin \left(\frac{s - t}{2r} \right) \right|.$$

The angular difference is $(s - t)/r$ because s, t are signed-arclength coordinates.

```
lemma unitCircleArc_dist {r s t : ℝ} (hr : 0 < r) :
  dist (unitCircleArc r s) (unitCircleArc r t) =
    r * |2 * Real.sin (((s - t) / r) / 2)| := by
  rw [Complex.dist_eq, unitCircleArc_sub_factor hr.ne', norm_mul, norm_mul]
  have hexp : ||Complex.exp (Complex.I * (t / r : ℝ))|| = 1 := by
    rw [Complex.norm_exp]
    simp
  rw [hexp, mul_one]
  have hnormr : ||(r : ℂ)|| = r := by simp [abs_of_pos hr]
  rw [hnormr, Complex.norm_exp_I_mul_ofReal_sub_one]
  simp only [Real.norm_eq_abs]
```

unitCircleArc_eq

Classification: supplement

Source: FrontierPerimeter.lean:409-415

Declaration SHA-256: 9342d6a8ed8b7dcd1fbce36251ddf6b271aeea6ec0298f5d467e178c8fbf9cec

Mathematical statement. For all real r, s , the complex circle point is $r(\cos(s/r) + i \sin(s/r))$.

Euler's identity gives

$$u_r(s) = r(\cos(s/r) + i \sin(s/r)).$$

The equality holds for every real r, s , including the totalized value at $r = 0$; no radius hypothesis is imposed.

```
lemma unitCircleArc_eq (r s : ℝ) :
  unitCircleArc r s =
    (r : ℂ) * ((Real.cos (s / r) : ℂ) + (Real.sin (s / r) : ℂ) * Complex.I) := by
  unfold unitCircleArc
  rw [show Complex.I * (s / r : ℝ) = ((s / r : ℝ) : ℂ) * Complex.I by ring]
  rw [Complex.exp_ofReal_mul_I]
```

unitCircleArc_im

Classification: supplement

Source: FrontierPerimeter.lean:423-429

Declaration SHA-256: 0666e8d860656cd1bc24ade42af99acdafd9034497a0ba0f27acf44cddcd77e5

Mathematical statement. For all real r, s , the imaginary coordinate of $u_r(s)$ is $r \sin(s/r)$.

Taking imaginary parts gives $\Im u_r(s) = r \sin(s/r)$ for all real r, s . In particular the geometric angular argument is s/r , not s , when the radius is not one.

```
lemma unitCircleArc_im (r s : ℝ) :
  (unitCircleArc r s).im = r * Real.sin (s / r) := by
  rw [unitCircleArc_eq]
  simp only [Complex.mul_im, Complex.add_im, Complex.ofReal_re, Complex.ofReal_im,
    Complex.I_re, Complex.I_im]
  ring
```

unitCircleArc_injOn

Classification: supplement

Source: FrontierPerimeter.lean:67-92

Declaration SHA-256: ceabbb1bd00ed7b3d5a248b0d9d8b4d41f8f1fc0af8194bd56a8092deed637e9

Mathematical statement. If $r > 0$ and $0 < \theta < \pi$, u_r is injective on the closed signed-arclength interval $[-r\theta, r\theta]$.

On $[-r\theta, r\theta]$, division by $r > 0$ puts the angles in $[-\theta, \theta]$. Their total span $2\theta < 2\pi$ precludes distinct parameters differing by a full revolution. Hence $u_r(s) = u_r(t)$ for two parameters in this interval forces $s = t$.

```
lemma unitCircleArc_injOn {r theta : ℝ} (hr : 0 < r)
  (htheta : theta ∈ Ioo 0 Real.pi) :
  Set.InjOn (unitCircleArc r) (Icc (-r * theta) (r * theta)) := by
  intro s hs t ht hst
  have hsdiv : s / r ∈ Icc (-theta) theta := by
    constructor
    · apply (le_div_iff0 hr).2
      nlinarith [hs.1]
    · apply (div_le_iff0 hr).2
      nlinarith [hs.2]
```

```

have htdiv : t / r ∈ Icc (-theta) theta := by
  constructor
  · apply (le_div_iff₀ hr).2
    nlinarith [ht.1]
  · apply (div_le_iff₀ hr).2
    nlinarith [ht.2]
rcases hsdv with ⟨hsl, hsr⟩
rcases htdiv with ⟨htl, htr⟩
have hdist : |s / r - t / r| < 2 * Real.pi := by
  rw [abs_lt]
  constructor <|> nlinarith [htheta.2]
have hcircle : circleMap 0 r (s / r) = circleMap 0 r (t / r) := by
  simp [unitCircleArc, circleMap_zero, mul_comm] using hst
have hang := eq_of_circleMap_eq hr.ne' hdist hcircle
exact (div_left_inj' hr.ne').mp hang

```

unitCircleArc_re

Classification: supplement

Source: FrontierPerimeter.lean:416-422

Declaration SHA-256: 238aa74976048276575d89eb69760ceb968a224b5358a1775e61818ec60df916

Mathematical statement. For all real r, s , the real coordinate of $u_r(s)$ is $r \cos(s/r)$.

Taking real parts in $u_r(s) = r(\cos(s/r) + i\sin(s/r))$ gives $\Re u_r(s) = r\cos(s/r)$. This is valid for all real r, s and retains both the radius factor and the signed-arclength normalization.

```

lemma unitCircleArc_re (r s : ℝ) :
  (unitCircleArc r s).re = r * Real.cos (s / r) := by
  rw [unitCircleArc_eq]
  simp only [Complex.mul_re, Complex.add_re, Complex.ofReal_re, Complex.ofReal_im,
    Complex.I_re, Complex.I_im]
  ring

```

unitCircleArc_sub_factor

Classification: supplement

Source: FrontierPerimeter.lean:18-32

Declaration SHA-256: d6c59e7f498e1352e21969a1c8084fa6600038a792db1172f4e5bd3b2836eece

Mathematical statement. For r nonzero and arbitrary real s, t , the difference of radius-scaled circle points factors into a rotated exponential difference.

For $r \neq 0$ and $s, t \in \mathbb{R}$,

$$u_r(s) - u_r(t) = r e^{it/r} (e^{i(s-t)/r} - 1).$$

The exponential addition law separates the phase at t from the signed-arclength displacement $s - t$, retaining both radius and parameter normalization.

```

lemma unitCircleArc_sub_factor {r s t : ℝ} (_hr : r ≠ 0) :
  unitCircleArc r s - unitCircleArc r t =
    (r : ℂ) * Complex.exp (Complex.I * (t / r : ℝ)) *
    (Complex.exp (Complex.I * ((s - t) / r : ℝ)) - 1) := by
  have he : Complex.exp (Complex.I * (s / r : ℝ)) =
    Complex.exp (Complex.I * (t / r : ℝ)) *
    Complex.exp (Complex.I * ((s - t) / r : ℝ)) := by
  rw [← Complex.exp_add]
  congr 2
  push_cast
  field_simp
  ring

```



```
rw [unitCircleArc, unitCircleArc, he]
ring
```

upper_cap_integral

Classification: supplement

Source: FrontierPerimeter.lean:1545-1552

Declaration SHA-256: 447beffafa9faf9c0047e982aa709b3b243852770c1fde70cc3c6676215458bf

Mathematical statement. The upper exterior cap's Euclidean Hausdorff density integral is lambda times its arc length.

The two interface endpoints are length-null; the open arc has density λ .

```
theorem upper_cap_integral (lam : ℝ)
  (c : OneSidedCircularCap) (hside : c.side = .upper)
  (hbase : c.baseY = 1) :
  (∫ z in planeEuclideanHomeomorph '' c.arcTrace,
    euclideanStripDensity lam z ∂(μH[1] : Measure EuclideanPlane)) =
    lam * c.arcLength :=
  cap_integral lam c (cap_upper_external_density lam c hside hbase)
```

volume_horizontalline

Classification: supplement

Source: CMVGeometry.lean:2029-2036

Declaration SHA-256: ea745db5e4b12e1d05fe0934ca3c81eb24b84ea87dd522e3aa6824c86d1a2dce

Mathematical statement. A horizontal line has zero planar Lebesgue measure.

Interface pieces do not contribute to weighted area.

```
lemma volume_horizontalline (b : ℝ) :
  volume {p : PlanePoint | p.2 = b} = 0 := by
  have hset : {p : PlanePoint | p.2 = b} = Set.univ ×ˢ ({b} : Set ℝ) := by
    ext p
    simp
  rw [hset, Measure.volume_eq_prod, Measure.prod_prod]
  simp
```

volume_lower_circular_segment

Classification: supplement

Source: CMVGeometry.lean:1945-1995

Declaration SHA-256: 2301e25d7ced8d1d9e2c23a9b63e02ba7c6e8f9ea615112d4c41d15aeb777045

Mathematical statement. Lebesgue volume of the reflected lower circular segment.

Reflection preserves area and gives the same segment formula.

```
lemma volume_lower_circular_segment (mx cy r θ : ℝ) (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  volume {p : ℝ × ℝ |
    (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
    p.2 ≤ cy - r * cos θ} =
    ENNReal.ofReal (r ^ 2 * (θ - sin θ * cos θ)) := by
  let U : Set (ℝ × ℝ) :=
    {p | (p.1 - mx) ^ 2 + (p.2 - (-cy)) ^ 2 ≤ r ^ 2 ∧
      -cy + r * cos θ ≤ p.2}
  have hU : MeasurableSet U := by
    have hcircle : Measurable (fun p : ℝ × ℝ ↦
      (p.1 - mx) ^ 2 + (p.2 - (-cy)) ^ 2) := by fun_prop
    have hside : Measurable (fun p : ℝ × ℝ ↦ p.2) := measurable_snd
```

```

    exact (measurableSet_le hcircle measurable_const).inter
      (measurableSet_le measurable_const hside)
  let F :  $\mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R} \times \mathbb{R}$  := Prod.map id (fun y :  $\mathbb{R} \mapsto -y$ )
  have hF :
    MeasurePreserving F (volume : Measure ( $\mathbb{R} \times \mathbb{R}$ )) volume := by
    change MeasurePreserving (Prod.map id (fun y :  $\mathbb{R} \mapsto -y$ ))
      ((volume : Measure  $\mathbb{R}$ ).prod volume) ((volume : Measure  $\mathbb{R}$ ).prod volume)
    exact (MeasurePreserving.id (volume : Measure  $\mathbb{R}$ )).prod
      (Measure.measurePreserving_neg (volume : Measure  $\mathbb{R}$ ))
  have hpre :
    F  $^{-1}$ ' U =
      {p :  $\mathbb{R} \times \mathbb{R}$  |
        (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2  $\leq$  r ^ 2  $\wedge$ 
        p.2  $\leq$  cy - r * cos  $\theta$ } := by
  ext p
  change ((p.1 - mx) ^ 2 + (-p.2 - -cy) ^ 2  $\leq$  r ^ 2  $\wedge$ 
    -cy + r * cos  $\theta$   $\leq$  -p.2)  $\leftrightarrow$ 
    ((p.1 - mx) ^ 2 + (p.2 - cy) ^ 2  $\leq$  r ^ 2  $\wedge$ 
    p.2  $\leq$  cy - r * cos  $\theta$ )
  constructor <=> rintro (hcircle, hside)
  · constructor
  · nlinarith
  · linarith
  · constructor
  · nlinarith
  · linarith
calc
  volume {p :  $\mathbb{R} \times \mathbb{R}$  |
    (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2  $\leq$  r ^ 2  $\wedge$ 
    p.2  $\leq$  cy - r * cos  $\theta$ } =
    volume (F  $^{-1}$ ' U) := by rw [hpre]
_ = volume U := hF.measure_preimage hU.nullMeasurableSet
_ = ENNReal.ofReal (r ^ 2 * ( $\theta$  - sin  $\theta$  * cos  $\theta$ )) :=
  volume_upper_circular_segment mx (-cy) r  $\theta$  hr h $\theta$ 0 h $\theta$  $\pi$ 

```

namespace OneSidedCircularCap

volume_upper_circular_segment

Classification: supplement

Source: CMVGeometry.lean:1820-1944

Declaration SHA-256: d23f6efcc432aaa58959c64bfa3f6f589362c33546093657b1717ec5dbfb1b34

Mathematical statement. Lebesgue volume of an upper circular segment.

Fubini and the cross-section integral give $R^2(\theta - \sin\theta\cos\theta)$.

```

lemma volume_upper_circular_segment (mx cy r  $\theta$  :  $\mathbb{R}$ ) (hr : 0 < r)
  (h $\theta$ 0 : 0  $\leq$   $\theta$ ) (h $\theta$  $\pi$  :  $\theta \leq \pi$ ) :
  volume {p :  $\mathbb{R} \times \mathbb{R}$  |
    (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2  $\leq$  r ^ 2  $\wedge$ 
    cy + r * cos  $\theta \leq$  p.2} =
    ENNReal.ofReal (r ^ 2 * ( $\theta$  - sin  $\theta$  * cos  $\theta$ )) := by
  let S : Set ( $\mathbb{R} \times \mathbb{R}$ ) :=
    {p | p.1  $\in$  Icc (cy + r * cos  $\theta$ ) (cy + r)  $\wedge$ 
      p.2  $\in$  Icc
        (mx -  $\sqrt{r^2 - (p.1 - cy)^2}$ )
        (mx +  $\sqrt{r^2 - (p.1 - cy)^2}$ )}
  have hS : MeasurableSet S := by
    exact measurableSet_region_between_cc
      (f := fun y :  $\mathbb{R} \mapsto$  mx -  $\sqrt{r^2 - (y - cy)^2}$ )
      (g := fun y :  $\mathbb{R} \mapsto$  mx +  $\sqrt{r^2 - (y - cy)^2}$ )
      (s := Icc (cy + r * cos  $\theta$ ) (cy + r))
      (by fun_prop) (by fun_prop) measurableSet_Icc
  have hpre :
    Prod.swap  $^{-1}$ ' S =
      {p :  $\mathbb{R} \times \mathbb{R}$  |
        (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2  $\leq$  r ^ 2  $\wedge$ 

```

```

      cy + r * cos θ ≤ p.2} := by
ext p
constructor
· intro hp
  rcases hp with ⟨hy, hx⟩
  change p.2 ∈ Icc (cy + r * cos θ) (cy + r) at hy
  change p.1 ∈ Icc
    (mx - √(r ^ 2 - (p.2 - cy) ^ 2))
    (mx + √(r ^ 2 - (p.2 - cy) ^ 2)) at hx
  have hcosmul : -r ≤ r * cos θ := by
    simp using mul_le_mul_of_nonneg_left (neg_one_le_cos θ) hr.le
  have hymin : cy - r ≤ p.2 := by
    linarith [hy.1]
  have hprod :
    0 ≤ (r - (p.2 - cy)) * (r + (p.2 - cy)) :=
    mul_nonneg (by linarith [hy.2]) (by linarith)
  have hD : 0 ≤ r ^ 2 - (p.2 - cy) ^ 2 := by
    nlinarith
  have hsqrt :
    (√(r ^ 2 - (p.2 - cy) ^ 2)) ^ 2 =
    r ^ 2 - (p.2 - cy) ^ 2 :=
    Real.sq_sqrt hD
  have hx1 :
    -√(r ^ 2 - (p.2 - cy) ^ 2) ≤ p.1 - mx := by
    linarith [hx.1]
  have hxu :
    p.1 - mx ≤ √(r ^ 2 - (p.2 - cy) ^ 2) := by
    linarith [hx.2]
  have hxprod :
    0 ≤ (√(r ^ 2 - (p.2 - cy) ^ 2) - (p.1 - mx)) *
    (√(r ^ 2 - (p.2 - cy) ^ 2) + (p.1 - mx)) :=
    mul_nonneg (by linarith [hxu]) (by linarith [hx1])
constructor
· nlinarith
· exact hy.1
· rintro ⟨hcircle, hbase⟩
  have hyUpper : p.2 ≤ cy + r := by
    nlinarith [sq_nonneg (p.1 - mx), sq_nonneg (p.2 - cy + r)]
  have hD : 0 ≤ r ^ 2 - (p.2 - cy) ^ 2 := by
    nlinarith [sq_nonneg (p.1 - mx)]
  have hsqrt :
    (√(r ^ 2 - (p.2 - cy) ^ 2)) ^ 2 =
    r ^ 2 - (p.2 - cy) ^ 2 :=
    Real.sq_sqrt hD
  have hsqrt0 : 0 ≤ √(r ^ 2 - (p.2 - cy) ^ 2) :=
    Real.sqrt_nonneg _
  have hdist :
    (p.1 - mx) ^ 2 ≤ (√(r ^ 2 - (p.2 - cy) ^ 2)) ^ 2 := by
    nlinarith
  change p.2 ∈ Icc (cy + r * cos θ) (cy + r) ^
  p.1 ∈ Icc
    (mx - √(r ^ 2 - (p.2 - cy) ^ 2))
    (mx + √(r ^ 2 - (p.2 - cy) ^ 2))
constructor
· exact ⟨hbase, hyUpper⟩
· constructor <=> nlinarith
have hsec (y : ℝ) :
  volume (Prod.mk y ^ -1' S) =
    (Icc (cy + r * cos θ) (cy + r)).indicator
    (fun y ↦ ENNReal.ofReal
      (2 * √(r ^ 2 - (y - cy) ^ 2))) y := by
by_cases hy : y ∈ Icc (cy + r * cos θ) (cy + r)
· simp only [Set.indicator_of_mem hy]
  have heq :
    Prod.mk y ^ -1' S =
      Icc (mx - √(r ^ 2 - (y - cy) ^ 2))
      (mx + √(r ^ 2 - (y - cy) ^ 2)) := by
  ext x
  change (y ∈ Icc (cy + r * cos θ) (cy + r) ^
    x ∈ Icc (mx - √(r ^ 2 - (y - cy) ^ 2))
    (mx + √(r ^ 2 - (y - cy) ^ 2))) ↔
    x ∈ Icc (mx - √(r ^ 2 - (y - cy) ^ 2))

```

```

      (mx + √(r ^ 2 - (y - cy) ^ 2))
    simp only [hy, true_and]
  rw [heq, Real.volume_Icc]
  congr 1
  ring
  · simp only [Set.indicator_of_notMem hy]
  have heq : Prod.mk y ^-1' S = ∅ := by
    ext x
    change (y ∈ Icc (cy + r * cos θ) (cy + r) ^
      x ∈ Icc (mx - √(r ^ 2 - (y - cy) ^ 2))
      (mx + √(r ^ 2 - (y - cy) ^ 2))) ↔ False
  constructor
  · intro h
    exact (hy h.1).elim
  · intro h
    exact h.elim
  rw [heq, measure_empty]
calc
volume {p : ℝ × ℝ |
  (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ^
  cy + r * cos θ ≤ p.2} =
  volume (Prod.swap ^-1' S) := by rw [hpre]
_ = volume S := by
  exact Measure.measurePreserving_swap.measure_preimage hS.nullMeasurableSet
_ = ∫ y in Icc (cy + r * cos θ) (cy + r),
  ENNReal.ofReal (2 * √(r ^ 2 - (y - cy) ^ 2)) := by
  rw [Measure.volume_eq_prod, Measure.prod_apply hS]
  simp_rw [hsec]
  rw [lintegral_indicator measurableSet_Icc]
_ = ENNReal.ofReal (r ^ 2 * (θ - sin θ * cos θ)) :=
  lintegral_semicircle_translated hr h00 h0π

```

volume_verticalLine

Classification: supplement

Source: CMVGeometry.lean:2037-2046

Declaration SHA-256: b32e9d96eca7ded1e8b5354b6d117e61bcda7b34c1fe056587cd14e1871da61b

Mathematical statement. A vertical line has zero planar Lebesgue measure.

Shared rectangle/segment edges do not produce duplicate area.

```

lemma volume_verticalLine (a : ℝ) :
  volume {p : PlanePoint | p.1 = a} = 0 := by
  have hset : {p : PlanePoint | p.1 = a} = ({a} : Set ℝ) ×ˢ Set.univ := by
    ext p
    simp
  rw [hset, Measure.volume_eq_prod, Measure.prod_prod]
  simp

namespace StripCore

```

xstar_lt_pi_div_eight

Classification: supplement

Source: MinimumValue.lean:49-55

Declaration SHA-256: cf3af1a7b133df1071b52ee9fcdff29abd10638feaf9c104776dea7ea70a191e

Mathematical statement. Xstar is less than pi/8.

The original cap at the minimum remains on the minor-cap branch.

```

lemma xstar_lt_pi_div_eight : xstar < π / 8 := by
  by_contra h
  push Not at h

```

```
have := g_deriv_pos h
rw [← gd, gd_xstar] at this
linarith
```

xstar_mem_Ioo

Classification: supplement

Source: MinimumValue.lean:56-60

Declaration SHA-256: 720519699b0830892e8a5d92169d8b53b8ea57339507bc01453bc6fff96e87b0

Mathematical statement. Xstar lies in $(\pi/16, \pi/8)$.

This locates it in the mixed minor/major inverse-angle interval.

```
lemma xstar_mem_Ioo : xstar ∈ Ioo (π / 16) (π / 8) :=
  ⟨pi_div_sixteen_lt_xstar, xstar_lt_pi_div_eight⟩

/-! ## Step 2: `gd` is strictly increasing on `(π / 16, π / 8)` -/
```

xstar_pos

Classification: supplement

Source: Minimiser.lean:156-157

Declaration SHA-256: 285916d9a661ffdef997baeb58111dd2f99ed6b3837830effc46ed8c96b2914d

Mathematical statement. The selected normalized cap area is positive.

α is positive on $(0, \pi)$.

```
lemma xstar_pos : 0 < xstar := area_pos θstar_pos θstar_lt_pi
```

θ0f_image

Classification: supplement

Source: MorgansArcFunction.lean:279-282

Declaration SHA-256: d81b765da2f8cf7cb10a3e3b057fc5bc93f2a0c62b4ee7c5334eaa9f98f1bd323

Mathematical statement. Every angle in $(0, \pi)$ is the inverse-area angle of some positive normalized area.

The exact conclusion is the inclusion

$$(0, \pi) \subseteq \{\theta(X) : X > 0\}.$$

For $u \in (0, \pi)$, take $X = \alpha(u) > 0$ and use $\theta(\alpha(u)) = u$. The reverse inclusion follows separately from ; equality of these sets is a consequence of both facts, not the conclusion asserted by this declaration alone.

Lean references

MorgansArcFunction.leanθ0f_mem

```
lemma θ0f_image : Ioo 0 π ⊆ θ0f '' (Ioi 0) := by
  intro θ hθ
  exact ⟨area θ, mem_Ioi.mpr (area_pos hθ.1 hθ.2), θ0f_area hθ⟩
```

$\theta \leq \pi/2$

Classification: supplement

Source: GFunction.lean:98-102

Declaration SHA-256: e9cfe5c6a9cd5cf538683dfa1919d265b38f9b8ab0aa4ba7f8c7d2c013301928

Mathematical statement. Normalized area at most $\pi/8$ gives an inverse angle at most $\pi/2$.

This is the closed-threshold version of the principal minor-cap branch bound.

```
lemma  $\theta \leq \pi/2$  {x : ℝ} (hx : 0 < x) (h : x ≤ π / 8) :  $\theta$  x ≤ π / 2 := by
  rcases eq_or_lt_of_le h with heq | hlt
  · rw [heq,  $\theta$ _pi_div_eight]
  · exact le_of_lt ( $\theta$ _lt_pi_div_two hx hlt)
```

$\theta < \pi/2$

Classification: supplement

Source: GFunction.lean:94-97

Declaration SHA-256: 5baeed1a6d4a81b4780dc3116c27179937e00e0901298d63c17f2f452433e21c

Mathematical statement. Positive normalized areas below $\pi/8$ have inverse angle below $\pi/2$.

Strict increase of the inverse transfers the semicircle threshold.

```
lemma  $\theta < \pi/2$  {x : ℝ} (hx : 0 < x) (h : x < π / 8) :  $\theta$  x < π / 2 := by
  have :=  $\theta$ _strictMonoOn (mem_Ioi.mpr hx) (mem_Ioi.mpr pi_div_eight_pos) h
  rwa [ $\theta$ _pi_div_eight] at this
```

$\theta < \pi/2$ of mem

Classification: supplement

Source: MinimumValue.lean:61-64

Declaration SHA-256: 80218f0a09a02137efcf79005b6fbcec3641c36ec9134e0575885f8ad1c23afb

Mathematical statement. In the mixed-area interval, the original inverse angle is below $\pi/2$.

The original normalized area is below the semicircle value $\pi/8$.

```
lemma  $\theta < \pi/2$  of mem {x : ℝ} (hx : x ∈ Ioo (π / 16) (π / 8)) :
   $\theta$  x < π / 2 :=
   $\theta$ _lt_pi_div_two (lt_trans (by linarith [pi_pos]) hx.1) hx.2
```

$\theta < 2\theta$

Classification: supplement

Source: GFunction.lean:151-155

Declaration SHA-256: 095143100fd32388bdf4e1d180bcb0168b99a36a86a64f445ac9b5b1dadd6573

Mathematical statement. Doubling a positive cap area strictly increases its inverse angle.

$X < 2X$ and strict monotonicity of θ give the result.

```
lemma  $\theta < 2\theta$  {x : ℝ} (hx : 0 < x) :  $\theta$  x <  $\theta$  (2 * x) := by
  have h2x : (0:ℝ) < 2 * x := by linarith
  exact  $\theta$ _strictMonoOn (mem_Ioi.mpr hx) (mem_Ioi.mpr h2x) (by linarith)

/-- On `(0, π / 16]` the derivative of `g` is negative. -/
```

θ0f_mem

Classification: supplement

Source: MorgansArcFunction.lean:250-253

Declaration SHA-256: cffddd09f082d4b6674b40f5793e1c255fa99c0cb678be3643150d9630636943

Mathematical statement. The selected inverse angle lies in $(0, \pi)$ for positive input.

This keeps all subsequent inverse and derivative calculations on the valid branch.

```
lemma θ0f_mem {x : ℝ} (hx : 0 < x) : θ0f x ∈ Ioo 0 π := by
  rw [θ0f, dif_pos hx]
  exact (area_surjOn (mem_Ioi.mpr hx)).choose_spec.1
```

θ0f_pi_div_eight

Classification: supplement

Source: GFunction.lean:73-78

Declaration SHA-256: 787022501b978041cc5dda7d507ec1898466d85828bc2384ff3ced42461fa420

Mathematical statement. The inverse area angle at $\pi/8$ is $\pi/2$.

Use the semicircle area identity and injectivity of a .

```
lemma θ0f_pi_div_eight : θ0f (π / 8) = π / 2 := by
  have hmem : π / 2 ∈ Ioo 0 π := ⟨by linarith [pi_pos], by linarith [pi_pos]⟩
  have := θ0f_area hmem
  rwa [area_pi_div_two] at this

/-- Morgan, GMT 4th ed. §15.5: `arc (π / 8) = π / 2`. -/
```

θ0f_xstar

Classification: supplement

Source: Minimiser.lean:158-160

Declaration SHA-256: 5f96bbbf1ba25c54aa03e1adcc89a5374c79ae31eab08da194eece7875964d2e

Mathematical statement. The inverse angle at X_{star} is the selected root angle.

$$\theta(a(\theta_*)) = \theta_*.$$

```
lemma θ0f_xstar : θ0f xstar = θstar :=
  θ0f_area ⟨θstar_pos, θstar_lt_pi⟩
```

θstar_lt_of_F_pos

Classification: supplement

Source: Certificate.lean:87-99

Declaration SHA-256: 8aed32b6dfdba14ba36157e57562cf1fa218ff121a66877337d2141e16cb6a9b

Mathematical statement. A positive value of F on the admissible angle branch bounds its root from above.

Strict increase of F and $F(\theta_*) = 0$ give $\theta_* < \theta$ when $F(\theta) > 0$.

```
lemma θstar_lt_of_F_pos {t : ℝ} (ht : t ∈ Icc 0 π) (h : 0 < F t) : θstar < t := by
  by_contra hcon
  push Not at hcon
  have hs : θstar ∈ Icc 0 π := ⟨le_of_lt θstar_pos, le_of_lt θstar_lt_pi⟩
  rcases eq_or_lt_of_le hcon with heq | hlt
```

```

· rw [heq, θstar_spec] at h
  linarith
· have := F_strictMonoOn ht hs hlt
  rw [θstar_spec] at this
  linarith

```

```

/-- `cos` is strictly decreasing on `[0, π]`, so an upper bound on `θ*` becomes
a lower bound on `k = 3 cos θ*`. -/

```

θstar_lt_pi

Classification: supplement

Source: Minimiser.lean:117-118

Declaration SHA-256: 00d6146ca546d963629c29e057f27680f814201f70ca0ff861a828ca133ab65c

Mathematical statement. The selected root angle is below π .

This places it in the normalized cap-area domain.

```

lemma θstar_lt_pi : θstar < π := lt_trans θstar_mem.2 (by linarith [pi_pos])

```

θstar_mem_Icc

Classification: supplement

Source: Minimiser.lean:96-98

Declaration SHA-256: 60a56c312e094733952f7446cf434fefca8186b21010cd17ab60b2b44864dfffb

Mathematical statement. The selected angle belongs to $[0, \pi/2]$.

This is the closed-domain component of its root specification.

```

lemma θstar_mem_Icc : θstar ∈ Icc 0 (π / 2) := exists_F_root.choose_spec.1

/-- The root is interior: `F 0 = -π < 0` and `F (π / 2) = π / 2 > 0`. -/

```

θstar_pos

Classification: supplement

Source: Minimiser.lean:115-116

Declaration SHA-256: 1d24d5264a96edc189b17b575a738900ce2888e0222ccbf2e0c850b185522fca

Mathematical statement. The selected root angle is positive.

It is the lower inequality in $\theta^* \in (0, \pi/2)$.

```

lemma θstar_pos : 0 < θstar := θstar_mem.1

```


Generated certificate families

CompactCells*.lean

Confirmed as the configured compact-cell family. Its exported inventory declarations include rational density intervals, local candidate-exclusion boundaries, and batch coverage theorems. Boxes 2 through 1023 join the explicit first two boxes to cover $[33/32, 9/7]$. Each local proof instantiates the compact-cell branch, stationary-fold, descending equal-area-root, and strict-perimeter obligations; the final coverage result is used in the main proof. The repetitive exact arithmetic stays in Lean. The family is not a collection of floating-point samples.

This family contains **3131** declarations. Every body is retained verbatim under `supporting-materials/lean/`; the table below is exhaustive.

Declaration	Exact source	SHA-256
CompactCells1003To1023.Box1003.QInterval	CompactCells1003To1023.lean:20-22	0aef3ddb1349e43cd56d63408011934a740f7e602452e59270f0bca0f41f74
CompactCells1003To1023.Box1003.lambaInterval	CompactCells1003To1023.lean:23-25	7954f593f751ecc31594321a2fefcd18ceb37ff87af3e760efe036665448a30c
CompactCells1003To1023.Box1003.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:672-683	9266c46b5eaf9a61ee35268398061782200f1890d01f8abf70f7ead86b3f2f99
CompactCells1003To1023.Box1004.QInterval	CompactCells1003To1023.lean:684-686	1a43a246eefb6c50a81ef9733e4893157523d94377df538d1fc2dab7847673e7
CompactCells1003To1023.Box1004.lambaInterval	CompactCells1003To1023.lean:687-689	3c0fcca63260140d2d2cad8e8094d5414589bc30188dad2544feed361432aef
CompactCells1003To1023.Box1004.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:1336-1347	ccafaed6dc01e22df9c10d9dbee848d5ac50f656e30706ad79e37be76d4367b6
CompactCells1003To1023.Box1005.QInterval	CompactCells1003To1023.lean:1348-1350	28f18eb223623f4896f731377948f5570a42a7d33b0b7f32c10ae233383fcc18
CompactCells1003To1023.Box1005.lambaInterval	CompactCells1003To1023.lean:1351-1353	8a1dc2a70ad8a50a185714446fd6705ce11e757f2127ff5da46c4b5df950f6d7
CompactCells1003To1023.Box1005.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:2000-2011	28dc15d7ee00d56557ba18ba2b60b89a7553da5770a61afc20cfbd4b11ce133e
CompactCells1003To1023.Box1006.QInterval	CompactCells1003To1023.lean:2012-2014	a5f119ffec426f5ab5a050693081c58f27cc9424aa9e120872579e9614f0e28a
CompactCells1003To1023.Box1006.lambaInterval	CompactCells1003To1023.lean:2015-2017	cde8901753aee32cbe93e6f042e0b3493e997c8d1b981da5be266b87a90d0e35
CompactCells1003To1023.Box1006.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:2664-2675	2a1a1947a524adc6c1d18c96ef2381e04750e391e58893084248a550082a84f7
CompactCells1003To1023.Box1007.QInterval	CompactCells1003To1023.lean:2676-2678	1182d581448cab66f2e6410ccf404309985f257821d192bfdae177eb6d0e3f9b
CompactCells1003To1023.Box1007.lambaInterval	CompactCells1003To1023.lean:2679-2681	a33c38139e8d46e1461f881822aac58f90046f33177e34402f9a1ff97cf5ff5a
CompactCells1003To1023.Box1007.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:3328-3339	ad937a4c9b26ca0e0956de7fac92be794d8d67fa715d82baab5db857b8c2a74b
CompactCells1003To1023.Box1008.QInterval	CompactCells1003To1023.lean:3340-3342	9d125dc28535496c0f8cc16349e12786fa7e73626641d6fd1599ff592b5808ae
CompactCells1003To1023.Box1008.lambaInterval	CompactCells1003To1023.lean:3343-3345	2535d335a00e480a4431c68f670d33f2c19e4361f267a21d59605cb72bc0c1ec

Declaration	Exact source	SHA-256
CompactCells1003To1023.Box1008.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:3992-4003	e6214ad116ae5df988976937e4fbae694ddb4f26896227aa690c77eb2ad6bcf
CompactCells1003To1023.Box1009.QInterval	CompactCells1003To1023.lean:4004-4006	95d65cfb4207522b49e995dc27a369c5c418f8c1faaf9d8f083340a957974ec4
CompactCells1003To1023.Box1009.lambaInterval	CompactCells1003To1023.lean:4007-4009	d8bcd30ee7afb5b62a0776cfc4c813b0b2a6003aa68013fcff326bf7bb5f2ea7
CompactCells1003To1023.Box1009.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:4656-4667	e92c04dc3bdea8599e90fbde215572b51bfe82d22bcb55502b84b76420efa83c
CompactCells1003To1023.Box1010.QInterval	CompactCells1003To1023.lean:4668-4670	8bd7d96ab0c00f196605ab439c8f85d7c8567c4173c7d648cfe9e2841c5334d8
CompactCells1003To1023.Box1010.lambaInterval	CompactCells1003To1023.lean:4671-4673	0ea3fb52d192d71ae711915519aa674b5faee224a7f0ae1c3f664c6ba4eee82a
CompactCells1003To1023.Box1010.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:5320-5331	83b37f94d9672bda8fabad74e73ea5e6ddd96b225d6265fb8a2267288040b300
CompactCells1003To1023.Box1011.QInterval	CompactCells1003To1023.lean:5332-5334	31d29a9f7cc8f9f491331bfedfa25102ab9921a068e2920a67dea55a3872ff35
CompactCells1003To1023.Box1011.lambaInterval	CompactCells1003To1023.lean:5335-5337	f13fa3e49f4ea2862ef61e5739d5561f6927d729a4d0d3f93fa2e45bfa763d98
CompactCells1003To1023.Box1011.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:5984-5995	e6c94787f7584891c53fb0cbcc50900bd40b8de2b16c5a0a13a6baa01ff897f6
CompactCells1003To1023.Box1012.QInterval	CompactCells1003To1023.lean:5996-5998	5bd7901405f16301c1091d0c85ccb409eead3817f51b900421c2a285bbe60ebd
CompactCells1003To1023.Box1012.lambaInterval	CompactCells1003To1023.lean:5999-6001	4e9e546e5be03a9226056cbafe05558741fcae849ef67c1e1f1ccc00d80f0b99
CompactCells1003To1023.Box1012.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:6648-6659	c79527c69c02ce67a1128e442e52ca9ae20da0912e3823e4b4630307e12ff46a
CompactCells1003To1023.Box1013.QInterval	CompactCells1003To1023.lean:6660-6662	4063f1ead2817760bc617b85317f31e90b7a59c40d9c824d5901c4a15ac8a670
CompactCells1003To1023.Box1013.lambaInterval	CompactCells1003To1023.lean:6663-6665	741eed6722855869eeaa45f4353ca1c4d25607115974f8eb7780800e17447fa88
CompactCells1003To1023.Box1013.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:7312-7323	7d0873dfd3ce4273369f1149a4f3c5da05534202483e7d22c0eceb2409847380
CompactCells1003To1023.Box1014.QInterval	CompactCells1003To1023.lean:7324-7326	ad142bffc1586fabacd10979731e608943777f20ed176797895b53592dd5a163
CompactCells1003To1023.Box1014.lambaInterval	CompactCells1003To1023.lean:7327-7329	d0f3b0b115bfde671e58f6f8c357e8a45006bf2051602064080eaf7b25b9616f
CompactCells1003To1023.Box1014.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:7976-7987	294708d1bae36afb9ffb93cf123e85a1defaf7bf2ec2b2534c647ba3a3314de3
CompactCells1003To1023.Box1015.QInterval	CompactCells1003To1023.lean:7988-7990	8b44b31c87f08aef997f4424f65a7a822ced66045861d2b52d86c0d21e45007c
CompactCells1003To1023.Box1015.lambaInterval	CompactCells1003To1023.lean:7991-7993	867e6813f99d38a785dfdd7fe121ada05edfb9e8922089f830c24804ee0338e9
CompactCells1003To1023.Box1015.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:8640-8651	659d47039b09be2d0a1270bfb78e195672ba710b284a1b659d8b9fbbe979441a

Declaration	Exact source	SHA-256
CompactCells1003To1023.Box1016.QInterval	CompactCells1003To1023.lean:8652-8654	63c272a9c2c0b950e401ca707d1478510ac05380832d9d12a5120cec171d2aec
CompactCells1003To1023.Box1016.lambaInterval	CompactCells1003To1023.lean:8655-8657	b0997fbb9c290a01c9cd97ebe58d27c8be978415ff1722a85ac418faa242a17e
CompactCells1003To1023.Box1016.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:9304-9315	e3ea4685ec24cc960d213a2a451590fd3a02bd61488ac6a981c61c2280ac1473
CompactCells1003To1023.Box1017.QInterval	CompactCells1003To1023.lean:9316-9318	00eac8527cf5985ebc4760a7a98ebf672e3b36950bd46b5ad0f04b383211233e
CompactCells1003To1023.Box1017.lambaInterval	CompactCells1003To1023.lean:9319-9321	44b7020558eb8ff294dbe26a29630b983312e56ed8a9803e85aad347de2d4a43
CompactCells1003To1023.Box1017.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:9968-9979	31b8f0f05bcea5f1222b889fb7bbe3974e02a6c596d5b87a011aff14d40cfaf4
CompactCells1003To1023.Box1018.QInterval	CompactCells1003To1023.lean:9980-9982	4518d7ac6fd604b046e73b0f5c281663adc4327f70e0d6922c4954ccbb26701e
CompactCells1003To1023.Box1018.lambaInterval	CompactCells1003To1023.lean:9983-9985	2478df3775575e7c681d307e0b8e59c4f8038f80b17a481ccfddfaecc6ea12f
CompactCells1003To1023.Box1018.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:10632-10643	c6f689c4446a97f22b3433ac67082410fbb15387a8d32688c18c0e6c60f4eb27
CompactCells1003To1023.Box1019.QInterval	CompactCells1003To1023.lean:10644-10646	b3f49473524decab07888e467370c44ae9092e08d5727e9def03d84b277b20ad
CompactCells1003To1023.Box1019.lambaInterval	CompactCells1003To1023.lean:10647-10649	d685b2279c31a2d937fbe24f348a9527e5f315fb5d6cec7a3304fe9e5e0d7f6e
CompactCells1003To1023.Box1019.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:11296-11307	44859e81753d2655f85543492bcf3fd1091cb1de41fb8d7903c28bc9c1c2ee4d
CompactCells1003To1023.Box1020.QInterval	CompactCells1003To1023.lean:11308-11310	7e9c4b0378b605932a5284215019fe28d85ca5563d4f3c36143d710cfd0e1c7a
CompactCells1003To1023.Box1020.lambaInterval	CompactCells1003To1023.lean:11311-11313	89a28ccc0a4771132b224330899df1d03913328356f7d1ae854103ae4cae6e25
CompactCells1003To1023.Box1020.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:11960-11971	315a2ec244f07531469fcd2e69aa1638c1f927a83772350e31838899625eadb
CompactCells1003To1023.Box1021.QInterval	CompactCells1003To1023.lean:11972-11974	877f2b0a740007b4a4c81257054a3f46884409000faf1d7b0c53eafa2e6d556e
CompactCells1003To1023.Box1021.lambaInterval	CompactCells1003To1023.lean:11975-11977	e6daec92bfe1fe96d7626e9692216bbac722129207e05f7f7666a90f85f6fb7e
CompactCells1003To1023.Box1021.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:12624-12635	da071604bf01fb6cac878ffdc25881c15cbca32d98197713c0727a6272509420
CompactCells1003To1023.Box1022.QInterval	CompactCells1003To1023.lean:12636-12638	d1ea96bb28afdaddc07e3e0f7a388dd90d52c0f2060f313379c79517a635dba4
CompactCells1003To1023.Box1022.lambaInterval	CompactCells1003To1023.lean:12639-12641	3824f8af50dfd32c681a36f24c59a2aa1517cf11996c4f73a5fe08f1c9b67d5a
CompactCells1003To1023.Box1022.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:13288-13299	23cdc580d5edbbb1be74d223b2fc8eeee557aa1f51d3eea78ccd5cc2263eb57d
CompactCells1003To1023.Box1023.QInterval	CompactCells1003To1023.lean:13300-13302	f91b3cb3d3bc591728f8b29f921a230fffb675a60da347e9a9a81590063648f71
CompactCells1003To1023.Box1023.lambaInterval	CompactCells1003To1023.lean:13303-13305	0bae30c198eab1d06be6d8c202438c3ceb56ac62eec7bf2af16380dd627567b8

Declaration	Exact source	SHA-256
CompactCells1003To1023.Box1023.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:13952-13963	4d50f38ea945f7e34f4d467709359e74bf299766fdd152d61f0f523dacb8b78c
CompactCells1003To1023.bboxes1003To1023_candidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:13964-14117	68581cbf6c2cd53b0b398a9c517f4e9be204c435157aa95bd03c1c34523a8049
CompactCells1003To1023.bboxes0To1023_candidate_not_isWeightedPerimeterMinimizer	CompactCells1003To1023.lean:14118-14131	a84497877b925fe28bd82df50e6461419f03e32cc7770a6672102746fe9c6b10
CompactCells107To138.Box107.QInterval	CompactCells107To138.lean:20-22	e667e63a3eee216c3392ab1cf8779ac1d44631d32fc820e30e1b9ceb464d5016
CompactCells107To138.Box107.lambdaInterval	CompactCells107To138.lean:23-25	45e99273f001945e452396917862956752774a252317c428eb1635882e65b66f
CompactCells107To138.Box107.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:672-683	0ca0ea6090e6cb065dce2f787a4e0e91dcaf9c82b3e8fb66460c96f0f1995844
CompactCells107To138.Box108.QInterval	CompactCells107To138.lean:684-686	4755fff945cab80eaf9ad77aa677d0cfa0f10cec1e9ac23c91303a8a3dea3503
CompactCells107To138.Box108.lambdaInterval	CompactCells107To138.lean:687-689	9838b082918b9c77a81e48965754e34bf60bbb7db492975617c5fb519b5fbcd2
CompactCells107To138.Box108.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:1336-1347	19961a78bd33aea336173701b025bdcf860cae1be3c87a7ae60f47aa8c130b93
CompactCells107To138.Box109.QInterval	CompactCells107To138.lean:1348-1350	f62177849be0c8c2e3d0631acc5de30b0bb53c8d8759083ae593531424a3b642
CompactCells107To138.Box109.lambdaInterval	CompactCells107To138.lean:1351-1353	57264cf9e9c7a21afc133ad01a7c4abb3d51218f8c0b8d1495230976114e656c
CompactCells107To138.Box109.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:2000-2011	5620710b85e323945f84ead91a451d8ab2c3f2e3acc0c0c2b3656c0b22bc2944
CompactCells107To138.Box110.QInterval	CompactCells107To138.lean:2012-2014	a9a119ef1024e526062283615281f064afd8de6e00c24683e750e2f7d7765a44
CompactCells107To138.Box110.lambdaInterval	CompactCells107To138.lean:2015-2017	a90707b248fdb52a3f651de6423ffcbf1a3ea55890f217283b0bfcd279668785
CompactCells107To138.Box110.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:2664-2675	63d0e11c528523d49f0a88d2293a8204f6e68c419289d3593b781b570398a9f6
CompactCells107To138.Box111.QInterval	CompactCells107To138.lean:2676-2678	81705fa5e0544cb077800ef741ec3faecffbbf1083ca73318a81657f2a333b4c
CompactCells107To138.Box111.lambdaInterval	CompactCells107To138.lean:2679-2681	1f4464e597a1061a6b77b8cadac099ff7a2407e0d40374784df846440c8af74c
CompactCells107To138.Box111.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:3328-3339	61c8f849e2b3bf48024fbf2000198c47b4ba2ae7a7978acfe19ff165ec191f44
CompactCells107To138.Box112.QInterval	CompactCells107To138.lean:3340-3342	ed34d111c029b5366ad458004b7cb330e2de9ad843b3957444d0eb147bc3111f
CompactCells107To138.Box112.lambdaInterval	CompactCells107To138.lean:3343-3345	cbd41b5a778e5c7bc7299b3170926afecaa cd0173aa50f5f9d9974af03945e30
CompactCells107To138.Box112.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:3992-4003	3a9cd6789a0b668b9e4ef7228ce3067ea7245f71f00f09f5f1283511b8c38108
CompactCells107To138.Box113.QInterval	CompactCells107To138.lean:4004-4006	748a622a97ddcf0fd24f3713cbdd4d8f1c1e9ad735d7f94431c138aa24ed595a

Declaration	Exact source	SHA-256
CompactCells107To138.Box113.lambdaInterval	CompactCells107To138.lean:4007-4009	31d125d981c5781883548cf18715012b9d58c7f9d9972b11c832d11aa5af2137
CompactCells107To138.Box113.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:4656-4667	6cf3753a45ed8aef6d1788c64e1d218afa635feb01e73e4059fe25ca2719de01
CompactCells107To138.Box114.QInterval	CompactCells107To138.lean:4668-4670	d5d96a42dac6417edef434f1606c1c8789144cad93034c4d129c5fd8c0545863
CompactCells107To138.Box114.lambdaInterval	CompactCells107To138.lean:4671-4673	1900f057587833c3a2a62d323cace993939f7c3167f4741a9b95a1086d80915a
CompactCells107To138.Box114.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:5320-5331	79ae99a0c222c41ff1a9249a60ead613f64488a9fef106d22fb8c7e3f4422919
CompactCells107To138.Box115.QInterval	CompactCells107To138.lean:5332-5334	82acc9e99eeee70d8f66c5d0636388f616f02dd9813982a56d04d09e4144eab8
CompactCells107To138.Box115.lambdaInterval	CompactCells107To138.lean:5335-5337	f4263df21fba15fce3d5794aefbbcb7a90168fd44e2e756be1132ef1b55929646
CompactCells107To138.Box115.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:5984-5995	aec94a0efe0bb2956fce2631c039759143a5251f312c60120b5d55b1ae1c92e1
CompactCells107To138.Box116.QInterval	CompactCells107To138.lean:5996-5998	c3281681d3850f6674e15287efabd0a405f98a90f3c958e592c508deb0164012
CompactCells107To138.Box116.lambdaInterval	CompactCells107To138.lean:5999-6001	5f75f8baddeee85d6ba7db854dc399db7626fd7fec254a30f0fe9fe09f18e0c5
CompactCells107To138.Box116.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:6648-6659	195466ad701f5e3615da8fe573e8ea598bc988c1d0ecc522779e416c980abccf
CompactCells107To138.Box117.QInterval	CompactCells107To138.lean:6660-6662	3934f74b5682ea3b2599994dac31687f97f6d282ccb7d45a86736047dc1a4d7b
CompactCells107To138.Box117.lambdaInterval	CompactCells107To138.lean:6663-6665	25e045f7d62cf36fb735d85f381759c265b514dd7f18f0f5769cb957972722b0
CompactCells107To138.Box117.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:7312-7323	d5be3ca025f09c632b40d2996bfb92a26d1f54fb9482f65627fe1f5abe288161
CompactCells107To138.Box118.QInterval	CompactCells107To138.lean:7324-7326	1305db4b75a492be05780d2bef19023193cba5fea1155543abb825a5dda09dde
CompactCells107To138.Box118.lambdaInterval	CompactCells107To138.lean:7327-7329	2bcf787fe6fcdf31fcebdb95de48f7b367fa06c16ecadc0c7330bea0046041f7
CompactCells107To138.Box118.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:7976-7987	8c100461773542c25e3195721dd7d95d68dac52f50a43d5e5e652976b5c12fc3
CompactCells107To138.Box119.QInterval	CompactCells107To138.lean:7988-7990	d2d2cc3b6434d470830cb519e902fd9b96e9dbcbc3500c8c28c472e9049a82b2
CompactCells107To138.Box119.lambdaInterval	CompactCells107To138.lean:7991-7993	2fe4c7dab54dbe7504aa3943fb2d47249d6227118898f8ecde6a0b329d31cd40
CompactCells107To138.Box119.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:8640-8651	058fb3171e312f24b4749feeedf41790798cbecda7dfc69c6b2ffdf8bb32da7a6
CompactCells107To138.Box120.QInterval	CompactCells107To138.lean:8652-8654	103b533a81b2e646db03b4ad427c0b8cf1b2223b8e3a6c09a5566d938246b665
CompactCells107To138.Box120.lambdaInterval	CompactCells107To138.lean:8655-8657	d61a4c01ac3f11004d3efe7c820ada78be6b57a1214f4891d73b2b7586fe0ce3

Declaration	Exact source	SHA-256
CompactCells107To138.Box120.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:9304-9315	bea186bc0e425adcd6913c65f9d1c99a9099cdb00d49e9634140939deee7e71c
CompactCells107To138.Box121.QInterval	CompactCells107To138.lean:9316-9318	eceeb48aa4efa2d80f41189f1bbef7aba7d9f3718d9bcaeb733e8672391d27
CompactCells107To138.Box121.lambdaInterval	CompactCells107To138.lean:9319-9321	63e2b656cc82abe4633b048393401388694a26d0097c0d373f25296f30b965a4
CompactCells107To138.Box121.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:9968-9979	996882e693ffd0be49534227f061dde3e7f18a3d7d34fde94f97a6094a1e9040
CompactCells107To138.Box122.QInterval	CompactCells107To138.lean:9980-9982	35e3792a60b80dacfc16ff79c8f795d658236be89e93d0077333e7e487d3fb29
CompactCells107To138.Box122.lambdaInterval	CompactCells107To138.lean:9983-9985	1c69c53a94321ccfacb37b3a03ef50e6c83da61fa2f6557ff558ac4f1f28576d
CompactCells107To138.Box122.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:10632-10643	ca80a2a08de62d36ddee09def39689d83e1ab8217bd9e2bc63375df5985ac3ba
CompactCells107To138.Box123.QInterval	CompactCells107To138.lean:10644-10646	19c72373c73a5898bdaafdc315e9ef16a823e86a8fb06f66d61400eaab658a6e
CompactCells107To138.Box123.lambdaInterval	CompactCells107To138.lean:10647-10649	5c4791ca74b4df1472d86458b31a4869339f95629b2856a4cf96702faa2b1ccc
CompactCells107To138.Box123.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:11296-11307	29da32bc22f308b61f0582c08c56fcdcc16bb268bc70499cba7428a3277293c9
CompactCells107To138.Box124.QInterval	CompactCells107To138.lean:11308-11310	87baf1dc00778cb04474c082c17f4c3712cd68ce367875707f16a1884c68928f
CompactCells107To138.Box124.lambdaInterval	CompactCells107To138.lean:11311-11313	4f9b3897f26da44eeb9f2801d7045b4bd78ac8caa6563474d5f5fc925d957a8c
CompactCells107To138.Box124.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:11960-11971	b580f6ea51497ea00b1b0cf0a005793bb7527df4d5f01ca0b50e0b91658db2e11
CompactCells107To138.Box125.QInterval	CompactCells107To138.lean:11972-11974	3bb0688ac6c09819422eeb5b03074eaeff1b1d2815a4f14710acfff66b67178d
CompactCells107To138.Box125.lambdaInterval	CompactCells107To138.lean:11975-11977	d75724a7fd42668a349affa0dbf9e46a93f0ae8fe0c925e8c16c0f820b643c90
CompactCells107To138.Box125.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:12624-12635	0bb1c93a2685c852cdd1c3864e0000737ac109603618884fc2cf0029317c7ae1
CompactCells107To138.Box126.QInterval	CompactCells107To138.lean:12636-12638	65dee7bf0f0c3314ba53210981e947c77d80c6ca4e235f0fd007da792b3e675e
CompactCells107To138.Box126.lambdaInterval	CompactCells107To138.lean:12639-12641	37572a7a1fa32e92ad27a7448d903b36826df3d430dcecff860cacb9927cb1d5
CompactCells107To138.Box126.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:13288-13299	8972d2b5e263f16574992fbba6dcf4570cf2e5312aecf37ab044a74d4c11a3ac
CompactCells107To138.Box127.QInterval	CompactCells107To138.lean:13300-13302	6f2417566dcff20547c9d5102c027c4655f0365c69254bbfea9c43897f815af5
CompactCells107To138.Box127.lambdaInterval	CompactCells107To138.lean:13303-13305	b51d044230b38c6f466650541d8eeeff6c33bfd80c44b1d95cb9c6115bb41fca
CompactCells107To138.Box127.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:13952-13963	9ec5488204315d2fcd4369ea6bfd5f0ac8a80c49ecd284f55c2908298ff17a61

Declaration	Exact source	SHA-256
CompactCells107To138.Box128.QInterval	CompactCells107To138.lean:13964-13966	8cf07bbafa97ae35b24e0fc110e0e95be4432d46dd20e84ebdb84d5b73c5f0c
CompactCells107To138.Box128.lambdaInterval	CompactCells107To138.lean:13967-13969	fb3d49e365c561884234e70998a748ace0129d8cb91de3476b0eb522d886dce8
CompactCells107To138.Box128.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:14616-14627	234686df430073286237b44cee801e495b720e190df844d75f0765206fa210ee
CompactCells107To138.Box129.QInterval	CompactCells107To138.lean:14628-14630	c1bf979b33461da9e80649051f552352b338c4a51cfc303dba16e27c3d099961
CompactCells107To138.Box129.lambdaInterval	CompactCells107To138.lean:14631-14633	28324986036051daef5fc64c44bbf48e1d0c7cbffcbd91da47e1ff62e99c3905
CompactCells107To138.Box129.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:15280-15291	f7d81a771e9c79f128aeb371c7b57a95fe5824462a8af09a32e7f337697015b
CompactCells107To138.Box130.QInterval	CompactCells107To138.lean:15292-15294	faaf66d32121c4867fd8587b54234bdf7f8f85a36304216a117dc28a37d5076e
CompactCells107To138.Box130.lambdaInterval	CompactCells107To138.lean:15295-15297	b8d94c45a41c91330f6ddeec159fe7ee2925ef3f283168f4fff88435f7159241
CompactCells107To138.Box130.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:15944-15955	6ac942981ae41b4e13758a82e908c4780ac40187ec10c3929796a8bc98603d0e
CompactCells107To138.Box131.QInterval	CompactCells107To138.lean:15956-15958	570caf3ff4493d7e8dd9c9891e513683c92c1da854f58de6ee819c57640e58b1
CompactCells107To138.Box131.lambdaInterval	CompactCells107To138.lean:15959-15961	e2801375490500dea91ccb37dbaf01ab1ecf2d17e34af1f7a383da10b2dcd65c
CompactCells107To138.Box131.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:16608-16619	98e5d302e1985d3d9f75eea4ed0ebeabee6a4e9c9318c497a774b9dc0f8f77fc
CompactCells107To138.Box132.QInterval	CompactCells107To138.lean:16620-16622	ebfea2b76f6f4417a6215c8b721d5c03417271d3f627b198adb4e9173cc8ea67
CompactCells107To138.Box132.lambdaInterval	CompactCells107To138.lean:16623-16625	aaf982e20f6b7796b9b3722729ad678bccd64f77e7a872260313c1abccc1f2b9
CompactCells107To138.Box132.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:17272-17283	1a7ff74d7fc44e335f8d1a8f1fccb0c8f0b9b92306f012e8696ad50155e5aed4
CompactCells107To138.Box133.QInterval	CompactCells107To138.lean:17284-17286	32fb2d89a006e5e60d72410b0ec5d50a926c094c69117b8ff976e5cf18d6703d
CompactCells107To138.Box133.lambdaInterval	CompactCells107To138.lean:17287-17289	645314099b27c2288c39c4d0cb392b463cc3349381d6aac28a5946f2d19b4e9
CompactCells107To138.Box133.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:17936-17947	abbf978d1e632a7a8a5e2c59ecf2d95d8d91477a5b8711f87bc590f2caed1774
CompactCells107To138.Box134.QInterval	CompactCells107To138.lean:17948-17950	26565a0bd2a432463bbd9ac8fd3cd52509ec23b25f31297f10b8ceca9fe8e4fe
CompactCells107To138.Box134.lambdaInterval	CompactCells107To138.lean:17951-17953	dc27deac50d93b90835e114f7ab6be048e142370a35be5c88e445d7457e89269
CompactCells107To138.Box134.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:18600-18611	274e4bd501cbbdd4ce60205454386c56f9cb42a56f31897b1f347dc36fd0cf99
CompactCells107To138.Box135.QInterval	CompactCells107To138.lean:18612-18614	a2028aa99b5739d5e1f5da386a8a3122d85508341952765c1c270676c7c73e8e
CompactCells107To138.Box135.lambdaInterval	CompactCells107To138.lean:18615-18617	4adba7b09f31eb35eafd8c10e47dd7ef966c661a0edf55ed6276c9294caf061c

Declaration	Exact source	SHA-256
CompactCells107To138.Box135.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:19264-19275	7be5920f912c23898ee1ff7008ec1599c7d254730400af6c9184fae2f0cae11c
CompactCells107To138.Box136.QInterval	CompactCells107To138.lean:19276-19278	fa0f031c6bf7e80abc0e7d6d85ad4661427f8f9bd4349e592343c5f45a16953b
CompactCells107To138.Box136.lambdaInterval	CompactCells107To138.lean:19279-19281	850e68baf3057d94dcfece81ec726425e3fec1ec1da127b18fb276f2fd0e434f
CompactCells107To138.Box136.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:19928-19939	b590a5469c02d52f082c91aec19ce6eec8b17d510b293062f1f82eb6157eb856
CompactCells107To138.Box137.QInterval	CompactCells107To138.lean:19940-19942	ac375d4f35e7d48fb0d5f05cdd05ed25fe73c6c92dae7c1edbd8df18fe349a80
CompactCells107To138.Box137.lambdaInterval	CompactCells107To138.lean:19943-19945	6c7ef894dd9b440266ce4854cd6f2762306d57b453861d1e5f685f6a9a5c45e0
CompactCells107To138.Box137.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:20592-20603	cd37b0388a2fa3d88f69ad1a783cfd5d79af2fd7fc3002d94c1a788ee3aae4e5
CompactCells107To138.Box138.QInterval	CompactCells107To138.lean:20604-20606	93279139f29ace3d5c509f8081cf08bf3818bd7b24ab161c210be0b7187e8571
CompactCells107To138.Box138.lambdaInterval	CompactCells107To138.lean:20607-20609	7434ed749fbc1611f4808238533569f1d23c07d4153856855d87385bec39ea62
CompactCells107To138.Box138.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:21256-21267	1a8e9b21f7fb48f02c6f6710362b345b29fbf1bf53f6bbdc7957ddb3704ce95e
CompactCells107To138.bboxes107To138_candidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:21268-21498	ff0f8614b90de0bf68ac6a45a4b51826c0d9819aebbb752569923f30cb3d197d
CompactCells107To138.bboxes0To138_candidate_not_isWeightedPerimeterMinimizer	CompactCells107To138.lean:21499-21512	6be709e753a98660172ec022123c80f08dc86c5c6dbaaa2e116440ee8e221a25
CompactCells11To42.Box11.QInterval	CompactCells11To42.lean:20-22	5dadf750260546df002a0918e752ce4f8c1000017a18593d0fe9112eb9365954
CompactCells11To42.Box11.lambdaInterval	CompactCells11To42.lean:23-25	35ed6c0fef6f230d169239aca3e04fc09c00308405cd703142032adc28c44ba76
CompactCells11To42.Box11.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:672-683	e25ed488330a5af7dfe27ae3e94cdb50b49ae35f3ef0a4a11d315bc4e2b43693
CompactCells11To42.Box12.QInterval	CompactCells11To42.lean:684-686	6b5cb01da16d94ba8578d164b9d138021aa84004533177fd1aed33b426ffcba9
CompactCells11To42.Box12.lambdaInterval	CompactCells11To42.lean:687-689	fa2ece58f06362739b1482c3f942480e2e4dc4722efb7d590cafbbe42b82e3d5
CompactCells11To42.Box12.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:1336-1347	d2687ea347f469495c814ce5582189853cb5921076e09a9146c3b10f64f65a26
CompactCells11To42.Box13.QInterval	CompactCells11To42.lean:1348-1350	6c899b3819fdeb1f9aa07349283eb22e7b0e151a6cc9196e2c4d1da4a2198f6a
CompactCells11To42.Box13.lambdaInterval	CompactCells11To42.lean:1351-1353	0607af4ba6a5fa8832b97a7f513cc12d266733dd7ef34533b974841135cbeec3
CompactCells11To42.Box13.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:2000-2011	c0dbe2d801c0fd15851a4a3b35021c4751cc44fe9bb3ab0e7210f115bdbbf94
CompactCells11To42.Box14.QInterval	CompactCells11To42.lean:2012-2014	c9ec122c69cffedebe9936c2d91f7f1c14b19b24528db3ebe5ed93d18963359d

Declaration	Exact source	SHA-256
CompactCells11To42.Box14.lambdaiInterval	CompactCells11To42.lean:2015-2017	c1d5aa6d3b5b6a86a032b59712e5f8375242032dc6ce91827856370bc86e3044
CompactCells11To42.Box14.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:2664-2675	a3d548248bc69d9116bcc089592b0ca2ca80d55257768cc508af62ddba5ea920
CompactCells11To42.Box15.QInterval	CompactCells11To42.lean:2676-2678	28fd1c3c8854c403d3966bf52d5c3405a027ca08afad17fe8d3ab2e144e88672
CompactCells11To42.Box15.lambdaiInterval	CompactCells11To42.lean:2679-2681	9a9e1f2e3c3f5d830f510a19cc063a108cef158c5a4a0053bbefe4bc8e601f67
CompactCells11To42.Box15.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:3328-3339	26b45d6f69c5132412e85e23c7e25fa2c0c84e2ff4f2a78db700e62df8795740
CompactCells11To42.Box16.QInterval	CompactCells11To42.lean:3340-3342	bfbea0f46f4e56081432f35964870229724878407ca8329c357eee6fe7afbc27
CompactCells11To42.Box16.lambdaiInterval	CompactCells11To42.lean:3343-3345	7c142e04fd01aa37a68b37c851e84e1cd50192b24e51b10857285bf123f33125
CompactCells11To42.Box16.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:3992-4003	853796148a1f370c1343d2da62354af455085efa7bb86d3f54f1adffefe5f3c8
CompactCells11To42.Box17.QInterval	CompactCells11To42.lean:4004-4006	a706028b21c48f0a047e789de82ab0e189552846a35320f6ac83e6fb98b2afb2
CompactCells11To42.Box17.lambdaiInterval	CompactCells11To42.lean:4007-4009	f8f9fd00f2203e3795e23de641dbd2a44b0b46a5ce50a2db12f43ddf0f821176
CompactCells11To42.Box17.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:4656-4667	3f5988125a896f23e484b8f98d6d958d45969bc1a2a02a82965d22bc3019b721
CompactCells11To42.Box18.QInterval	CompactCells11To42.lean:4668-4670	421939a3e0dade57935a14330b9dc80f68414b42fb63b21b1d489750c5c5e710
CompactCells11To42.Box18.lambdaiInterval	CompactCells11To42.lean:4671-4673	393de9eb26f4a13c47cb6abc31d73cdc1afe0c8773b1179b6b8333ec2911a92
CompactCells11To42.Box18.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:5320-5331	8de32dc221efa0f365dfbb56cb3ae8f7cbfe628aba560bb697be1f00dc4d6358
CompactCells11To42.Box19.QInterval	CompactCells11To42.lean:5332-5334	72252640172414a82d8565298063464f9db5fcab1a67ee3aecc9d3070367f74e
CompactCells11To42.Box19.lambdaiInterval	CompactCells11To42.lean:5335-5337	95bcd6716496c3af1683ace25bbcae84def2a1a82475f91382dd2bd6f450ec40
CompactCells11To42.Box19.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:5984-5995	494fd37b380b5741b2fea7c2fc7a398e0e2de5b9cff332decebebbba062971f68
CompactCells11To42.Box20.QInterval	CompactCells11To42.lean:5996-5998	5cd563653114f2859636ebf98a9b97dd18c4e19fb145a032e9aa4caa88b62c3c
CompactCells11To42.Box20.lambdaiInterval	CompactCells11To42.lean:5999-6001	af736e1cb590a5f2ae1205aca4a1955fe25d0d5c849c09d10ce875f867a8999a
CompactCells11To42.Box20.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:6648-6659	2cd462bc5d6c1462c9d3fe5fc44f3e51fe4afb467fc39c1276a47a92e42b5aa6
CompactCells11To42.Box21.QInterval	CompactCells11To42.lean:6660-6662	8fa3d95272438ee8a6d7a4d1277c977d44d15fa7172a869f0fab78ca3b8ec6cb
CompactCells11To42.Box21.lambdaiInterval	CompactCells11To42.lean:6663-6665	d3ea57470f81dc170630c50ff54a13283c352da3435e3f8cbff5426fe815ff7d

Declaration	Exact source	SHA-256
CompactCells11To42.Box21.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:7312-7323	c5adde95b1ac6d16b334604f3f667603c8286809d12d25c354fdc9e96ec19f65
CompactCells11To42.Box22.QInterval	CompactCells11To42.lean:7324-7326	275894b802bf1e39f33583fb5c03182ad08036c18feee34733a85510b9087aae
CompactCells11To42.Box22.lambdaInterval	CompactCells11To42.lean:7327-7329	6f49fb55096f4c54947e65a27514664ce05e255b22efcdb30680b15a59ceef5f
CompactCells11To42.Box22.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:7976-7987	4194c5da3e9170b039ceb10ba2934731ac7d6c32a862e9f5e2cb4ce461500c37
CompactCells11To42.Box23.QInterval	CompactCells11To42.lean:7988-7990	a300e02dca2aea8e6e9827285162da1cf623e509ccde6bac256b08da727d9021
CompactCells11To42.Box23.lambdaInterval	CompactCells11To42.lean:7991-7993	61d864af315c910404f363ec07754596cfbbd0e130a37cc2ab7ba3b914321e40
CompactCells11To42.Box23.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:8640-8651	2b2ea6bc327b386b73f1d9a29ece89b4a0920be8c1ed1018b9060575c7f776e1
CompactCells11To42.Box24.QInterval	CompactCells11To42.lean:8652-8654	3ce85c9d3b1423764457217a9271ff0c32ed7a623e287024043e0b4af11925a4
CompactCells11To42.Box24.lambdaInterval	CompactCells11To42.lean:8655-8657	53a2313e578ddc6d222950836b3de59d5398f0da6f98ee2619abdda9c9c1157e
CompactCells11To42.Box24.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:9304-9315	f3c337be480a73960fd98a4520aed8c79df9571c25c5d4ffc1cdd67fe0d066a5
CompactCells11To42.Box25.QInterval	CompactCells11To42.lean:9316-9318	171b410c63d124de25758b7a9f2347576e90dc75f16378caba82094285face18
CompactCells11To42.Box25.lambdaInterval	CompactCells11To42.lean:9319-9321	28c680502c80440791ceed14bc4ddbc649c394986c8a82c3a312c56b4eb7cd2d
CompactCells11To42.Box25.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:9968-9979	800b063236ec4bbf34e7dcc65f28b04046e8e93e7e58e96d610da318177777e1
CompactCells11To42.Box26.QInterval	CompactCells11To42.lean:9980-9982	6b726f333664ba5e415c25d4e75af36f8f1b67e74d3ce12ee4479db0b8353d5f
CompactCells11To42.Box26.lambdaInterval	CompactCells11To42.lean:9983-9985	e046af7b919fbb2b5c4b42b11f03be39c237f858dc88b66ba0ad053aca45d931
CompactCells11To42.Box26.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:10632-10643	36103e4a1fdc17c3539886190d961e240c5c6dbbdfa3bc5f84dfe97d7480af04
CompactCells11To42.Box27.QInterval	CompactCells11To42.lean:10644-10646	802bfda20aed2d8b5f8a5480029299eb99c2ceec4d6de9fc733ba17beefa7e24
CompactCells11To42.Box27.lambdaInterval	CompactCells11To42.lean:10647-10649	d92092aabe34d1bb13e25cff39f4812e2d56e1d5807fe80a5b95d3bd896a86bc
CompactCells11To42.Box27.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:11296-11307	70bd6d8a894646d61b390eb3a5e19a933622cace6ead5c9e179fb1bd98d24ed6
CompactCells11To42.Box28.QInterval	CompactCells11To42.lean:11308-11310	02fc6391d60f5f7f7700a22f41a5ce06c41b39e67846c35015881b5642a03ad1
CompactCells11To42.Box28.lambdaInterval	CompactCells11To42.lean:11311-11313	e429588ce38d887e1b92b96b096085976369647573ee1fcae7dd6139a6dc8750
CompactCells11To42.Box28.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:11960-11971	96a38b510e97976752ced1cad8fc92612973e544a444581d711dd6c0214958dc

Declaration	Exact source	SHA-256
CompactCells11To42.Box29.QInterval	CompactCells11To42.lean:11972-11974	0ac331b20d112ba40707de26cdc849de1f24c461eac5ba433506b5c5898f163d
CompactCells11To42.Box29.lambdaiInterval	CompactCells11To42.lean:11975-11977	57e18f665520f6df6c4b31f0f4a72b18bf6e0e04d9e868ac83064d94b0be1c88
CompactCells11To42.Box29.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:12624-12635	57f6bb2f9c56cae0cd5289b3c726da9698dfae81dc6a15435718670c060dd2ee
CompactCells11To42.Box30.QInterval	CompactCells11To42.lean:12636-12638	2c5cd86da62c9e9dd2d366d35a92834bcb8d884f831c058cc69a0f3d16df980d
CompactCells11To42.Box30.lambdaiInterval	CompactCells11To42.lean:12639-12641	3070b83a0add6e0dda42ba58b00dcd3c7db563a57cd7a7956ae6bd074b1cd3bd
CompactCells11To42.Box30.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:13288-13299	0c8285084099205dabdbdf10bc9b65ef26cb67ca3579abb01a62fcecbe2e0931a4
CompactCells11To42.Box31.QInterval	CompactCells11To42.lean:13300-13302	edd7a64e3a8968e2daa7888d695da7eb5c6abe8609692b49ad55754ee7f98cfe
CompactCells11To42.Box31.lambdaiInterval	CompactCells11To42.lean:13303-13305	7373155b15a6ff6c4a671d4bb1e9e181f996da8ab2e5de50a87dafa8721faa35
CompactCells11To42.Box31.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:13952-13963	aaca620af0b65b715b7cebb4045eea6c6588af48a92c2d198b9d7491cf4fe464
CompactCells11To42.Box32.QInterval	CompactCells11To42.lean:13964-13966	992f208ba2c47e2eea338d0bd5dca59eabf7d4efb062a93618227f1812b20663
CompactCells11To42.Box32.lambdaiInterval	CompactCells11To42.lean:13967-13969	a99d2e5f0a44f58d3e54b030483795d9a369431039029e42b17116f2ef4eedb3
CompactCells11To42.Box32.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:14616-14627	9aadfb095a6f5f8ea8894f9a2ce6701be657036c8148a3d488b87b33bd6b8518
CompactCells11To42.Box33.QInterval	CompactCells11To42.lean:14628-14630	29cb5a3a238c0021a974e2ac0c9c45405ad39e71bbf71ee295080814740db7a7
CompactCells11To42.Box33.lambdaiInterval	CompactCells11To42.lean:14631-14633	fe3c59207b513cca74326544fb7993cb37040ac55ede8a93fb7d49865d538844
CompactCells11To42.Box33.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:15280-15291	ba354e595fbec3d31ea420648d6e2a15276e1fecdc8182035139c406b75e18b5
CompactCells11To42.Box34.QInterval	CompactCells11To42.lean:15292-15294	e91e68addcd010ca7c18cf528d434d1c9a62cfbae5989807cbfcfd805c62f4ad
CompactCells11To42.Box34.lambdaiInterval	CompactCells11To42.lean:15295-15297	a31c325d88d7e6420026520f8cf4803a02e09756f2da6dfe522ef0db9cedcfec
CompactCells11To42.Box34.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:15944-15955	5a998dd16fb8ea1b54a2a3a0ce2ccc5aad36971ef785f10b9e7d8c42735f316d
CompactCells11To42.Box35.QInterval	CompactCells11To42.lean:15956-15958	67d421fff5b0f7d23625bcfe706945aeb347acb0c1cea5cf4e3871dd74fa581a
CompactCells11To42.Box35.lambdaiInterval	CompactCells11To42.lean:15959-15961	2be8f367fb1f6de59d489c19dcc084d8abe24c2ee2b02f7a7a105231ddf66251
CompactCells11To42.Box35.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:16608-16619	e472b8b45f7a87a43a42500b93702a016df024a7dbfe4f108615e29eb2ec147c
CompactCells11To42.Box36.QInterval	CompactCells11To42.lean:16620-16622	703b8c1b92e07f6185b60b8f29c34c09d491094a1f243e5c708e03dd164be7a2
CompactCells11To42.Box36.lambdaiInterval	CompactCells11To42.lean:16623-16625	c43bedfdd6219641b741f341fa67517dc080c6f62fcdfae7e894bc94f2417770

Declaration	Exact source	SHA-256
CompactCells11To42.Box36.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:17272-17283	bd31e85eb354f834b0785141f4ece915272818929427fa06cb621c830d6384e6
CompactCells11To42.Box37.QInterval	CompactCells11To42.lean:17284-17286	1d1860ad9f15ffa72c1b9a46b962004426831802a35a0380682af33151aa2601
CompactCells11To42.Box37.lambdaInterval	CompactCells11To42.lean:17287-17289	b41c9694e7b50b0a52f549ff178cd10628887aafa299526a3a3b7f7dcb6143df
CompactCells11To42.Box37.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:17936-17947	bdf39f7c66d9fce9f2656ac21bfff9546c4f0f9cfa0575d393b12aafeb828d95a
CompactCells11To42.Box38.QInterval	CompactCells11To42.lean:17948-17950	259350e6febd8af14c376b67fe99dd162667feda06519bd2ecee1c6b432f7d45
CompactCells11To42.Box38.lambdaInterval	CompactCells11To42.lean:17951-17953	ca19fcb23c5101a837477321e27457ac658b40aef82c8ea7da82d75d5c1e63db
CompactCells11To42.Box38.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:18600-18611	76f774c500c659737d38bc4ca7f4d7ceef2ef4e85c0b3d04a1895d3b84ec0df
CompactCells11To42.Box39.QInterval	CompactCells11To42.lean:18612-18614	fd76225421a3059a0e53bebe8b123dc84dafd68e33b8e46022106dbce6e9b921
CompactCells11To42.Box39.lambdaInterval	CompactCells11To42.lean:18615-18617	df83554a8d6953f3e48e40278e29011ba10aee297efff9eda50bf6f50e7d358f9
CompactCells11To42.Box39.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:19264-19275	499d458b3a3224f0a18221c08bdb8bc01fae88be2eb59c864fec3e78b5e75d9e
CompactCells11To42.Box40.QInterval	CompactCells11To42.lean:19276-19278	112f63a36097d132d1ec31a58c60face95208a28da5fed16567c49515211c19a
CompactCells11To42.Box40.lambdaInterval	CompactCells11To42.lean:19279-19281	52dd9987434b230240fb2e9720069e8e76cca4b27a5b3c34f88ae9b5b7bca142
CompactCells11To42.Box40.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:19928-19939	f5398866846c93fb5701e62e7dcba8680a9acebf47da299afcef802c003b73cc
CompactCells11To42.Box41.QInterval	CompactCells11To42.lean:19940-19942	8ebe75de31823f535dd39b562794e1e4c9eafadc30aacf46e8797030eb99f874
CompactCells11To42.Box41.lambdaInterval	CompactCells11To42.lean:19943-19945	fe35a12b986e353f59b9fcbaffa15a6952620a3f6de90e1c9d785601422d2a15
CompactCells11To42.Box41.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:20592-20603	3fcc77976b61018da6d95a9fd96b8dbec48ecf1000fab4323d96dd0884692522
CompactCells11To42.Box42.QInterval	CompactCells11To42.lean:20604-20606	7bb6e46eed87a28c3c04468260bc9dd51fc9992e94ec24ef01454c93c3cbdaa9
CompactCells11To42.Box42.lambdaInterval	CompactCells11To42.lean:20607-20609	6995331f0ff2bb4bde2a9d33e219a52e79b70146cb518c3d3fbf64c3e2c02f53
CompactCells11To42.Box42.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:21256-21267	4c0cea44e7d218c0242ee5603e92680468f3b73e3466b4c45e5939a9b03b340b
CompactCells11To42.bboxes11To42_candidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:21268-21498	34866d24b391b999bc0cc35b5a2dc9f2cd750ba2fb0cd34020cc9d50bdc54709
CompactCells11To42.bboxes0To42_candidate_not_isWeightedPerimeterMinimizer	CompactCells11To42.lean:21499-21512	87b0dd8c2e42a30635934bd4d240b4f0fe3dfce987f1fc99b5dd1e03c56305df
CompactCells139To170.Box139.QInterval	CompactCells139To170.lean:20-22	96fbfd325d197d6ac810ce3d1ba477d88ad46e03acc7f4a2779aca04473ea51

Declaration	Exact source	SHA-256
CompactCells139To170.Box139.lambdaiInterval	CompactCells139To170.lean:23-25	7eab737548930554c94700a6bd3f19b48954f39c55693c40a36e763a8353c589
CompactCells139To170.Box139.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:672-683	4f5e7898571d4646013f0bb623106dde8df0b482a338c6f60586847e7ed336f6
CompactCells139To170.Box140.QInterval	CompactCells139To170.lean:684-686	b6b8d85ac67ffa5032c2e60b9ab88530165267185734d20bd1a61f57afe6997c
CompactCells139To170.Box140.lambdaiInterval	CompactCells139To170.lean:687-689	bf502e34c46af08c2482b3a01056f55e3dfd8769d00f4ec6a846f040ace9770b
CompactCells139To170.Box140.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:1336-1347	e71b7add99315d28a5c049bb17d3088bb2cd90e286c17148f459167d9af682d5
CompactCells139To170.Box141.QInterval	CompactCells139To170.lean:1348-1350	c5aa8ad0981d3ef46cec79273fd099fcbd4a03e9a773d9edf8330cab642c35b7
CompactCells139To170.Box141.lambdaiInterval	CompactCells139To170.lean:1351-1353	ebd8ed0ef46e0036bdca3713f42250161f2d86f5350cdcb466baaa37e2276e30
CompactCells139To170.Box141.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:2000-2011	b31722ff31480f7fc901c7da37eb4f9683a279bae3035f22f682a6db6a3d0c76
CompactCells139To170.Box142.QInterval	CompactCells139To170.lean:2012-2014	d00d668e1df1ae7297272fe1d0e8fd86bed14c05228ab0dac25041167db0b070
CompactCells139To170.Box142.lambdaiInterval	CompactCells139To170.lean:2015-2017	d68bcc1322a6fc800486747b125a739ee4212111cf0d7ae485f6d573309d7837
CompactCells139To170.Box142.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:2664-2675	29b76b05a3229ced675fee51bdcdf45e7d204d27297a5c419195180201321026
CompactCells139To170.Box143.QInterval	CompactCells139To170.lean:2676-2678	ed30b07baa3c46a50eb8668d465d6fcfc2565f2858556b8178232a4437302ee8
CompactCells139To170.Box143.lambdaiInterval	CompactCells139To170.lean:2679-2681	c123239135618db5be267158194bad92e0fa0b4330b28b7289d435ea3dbca077
CompactCells139To170.Box143.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:3328-3339	397c14585c813b89621b075e4f6449702f5fde9c8b4ee21d6c5b9f8fbb5a29ef
CompactCells139To170.Box144.QInterval	CompactCells139To170.lean:3340-3342	6bfc7a1dda5830f2d341b6a18028e435bfd2bdc7fd44ee371deca812a7586827
CompactCells139To170.Box144.lambdaiInterval	CompactCells139To170.lean:3343-3345	8398f543591fdfe314c223c2970ac04d10d5662b124f568cee9cada28c791f24
CompactCells139To170.Box144.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:3992-4003	9bb352137f9a2a0e4d615c72bf27879518f52ea8936a8205f09fd0b48891607d
CompactCells139To170.Box145.QInterval	CompactCells139To170.lean:4004-4006	1bfae45ac8f410e7fb5a5998768a19534eaddf23621c1858959ddf8e108b33c66
CompactCells139To170.Box145.lambdaiInterval	CompactCells139To170.lean:4007-4009	7ab0695cff91aaaae3318e2145cb5d8187fd047bbbb8beec4d75da505f8e461d8
CompactCells139To170.Box145.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:4656-4667	0715a3792092b3d36751b417a37518cb5f9a4e4b2808b97f82bb9ad9ab408e48
CompactCells139To170.Box146.QInterval	CompactCells139To170.lean:4668-4670	bc6d6bfaaaeaf2ba77285e075c714dbfbb848557c2972750a40c1a650b922e4
CompactCells139To170.Box146.lambdaiInterval	CompactCells139To170.lean:4671-4673	2b0d5f86657dd88e42cf3210cfc8eb2af30b53d13cb7086757dd932139c35a7a

Declaration	Exact source	SHA-256
CompactCells139To170.Box146.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:5320-5331	e45304aff9eaf5c1b73503d91b8eaaa2c874bf1b25097eb65632a0949128c966
CompactCells139To170.Box147.QInterval	CompactCells139To170.lean:5332-5334	7af9fcc8eab40bb8479f9addb2444baaf0fc8805a6c2a91a2942927727ab7085
CompactCells139To170.Box147.lambdaInterval	CompactCells139To170.lean:5335-5337	bbf2b5157b5fff77dab6146d347f70982c30e079046b5059d6cf4550ed4587956
CompactCells139To170.Box147.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:5984-5995	8d809cae1ddf66956fcd857f1f7bbf019fcfa4385e01cc911d36f6975bc5df5d8
CompactCells139To170.Box148.QInterval	CompactCells139To170.lean:5996-5998	b7cd65884e1e7cca0b52f3f125606f997b34baa596a7dc5b36f7c4a798c0eaf5
CompactCells139To170.Box148.lambdaInterval	CompactCells139To170.lean:5999-6001	57bcdff16aa829beb5ec2abfd22f637a7082a265f2f4e10b24d97e9d46d7652f7
CompactCells139To170.Box148.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:6648-6659	a8d52d8a284546a50e6257162f3f2cae8978851788fccde04ad4bc966e80348c
CompactCells139To170.Box149.QInterval	CompactCells139To170.lean:6660-6662	d31dbbb514b07d095ad3954bd7fa4758e03b2464076831c4f52cc36f8c4d714e
CompactCells139To170.Box149.lambdaInterval	CompactCells139To170.lean:6663-6665	7af84fb070cf753864d8bae4454d9255c6512a0c772cf9f48dc989b9fb02ef67
CompactCells139To170.Box149.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:7312-7323	5f60dbd0dcdb257cbdf52414917411d58f8c0288632b67ab8eaecadcee62a404e
CompactCells139To170.Box150.QInterval	CompactCells139To170.lean:7324-7326	428aa4aa9aa1115e0a715dbb34b0063fb9259c38a187b01b48dfd331c208b126
CompactCells139To170.Box150.lambdaInterval	CompactCells139To170.lean:7327-7329	83f044f64ab5966922908a622789f6a34aeeff7f72131578caa88124b9f198f7d
CompactCells139To170.Box150.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:7976-7987	532256cad8d9c73e2ae2d33ea21cd6ca85cac8cac03727a651fb93b329f12364
CompactCells139To170.Box151.QInterval	CompactCells139To170.lean:7988-7990	ebbd1e4c6b42c0c0948eebdf45a3f5b6a761ea9553879deb143a05b821b8a23e
CompactCells139To170.Box151.lambdaInterval	CompactCells139To170.lean:7991-7993	cd1e575732ac08e433cee2324d067fedd96ae45e4d726f28ce4fe3f5f83300b1
CompactCells139To170.Box151.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:8640-8651	f1a1a5c0f41a12bfb2719cb790ca995d2756b2aad3129de7a062b206d3b25c98
CompactCells139To170.Box152.QInterval	CompactCells139To170.lean:8652-8654	84359d97b5bb1fdae316eccc8cae31711b23271fd2d0d1c1602e9e5b3b254e00
CompactCells139To170.Box152.lambdaInterval	CompactCells139To170.lean:8655-8657	db58c358ac30702efe2c2c046bd8a9c7010dfeee648457fc9a5a5e2d3002b319
CompactCells139To170.Box152.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:9304-9315	cf5678e65642fa8747487fed1d548adecab a73ffcab0508a757df72152547dda
CompactCells139To170.Box153.QInterval	CompactCells139To170.lean:9316-9318	3656bb60eece130a026d50f451c2352271044d8934940b252cdc61fbf9cd7ff4
CompactCells139To170.Box153.lambdaInterval	CompactCells139To170.lean:9319-9321	ca8aaa0b32065f870548c88b663ec49a02fc85423c5e3ae44028dc9888cee12d
CompactCells139To170.Box153.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:9968-9979	2a2b6f4104975e168d8955bfdcd7c1a097f6c652d2287a8cb7f1f0c8e45938e8

Declaration	Exact source	SHA-256
CompactCells139To170.Box154.QInterval	CompactCells139To170.lean:9980-9982	b0349e56abcf41d38931f05c574ec2acc9c5083a387ef823c70e11361418f99
CompactCells139To170.Box154.lambdInterval	CompactCells139To170.lean:9983-9985	4d001d151a391e50a1630624c9f7321f2be867107ec93f82540b2e827b1c64a9
CompactCells139To170.Box154.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:10632-10643	6fdbe4e5b60d8b7f14ed40b99a21681af5afd1229292e415caf79ac47db2eee2
CompactCells139To170.Box155.QInterval	CompactCells139To170.lean:10644-10646	786256132228c7e3f6fb3c7c05e051e5db78c211cefbbdb20601ebeee12f2c81f
CompactCells139To170.Box155.lambdInterval	CompactCells139To170.lean:10647-10649	872574aab1510c2fcf24894ec7eb20b7c4a77c062166a1acd330cac3662cd226
CompactCells139To170.Box155.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:11296-11307	d6a7e3a64b3db65d346ad6e30b157b19590f303ea91ea2c8cd5fca7e3ce88edb
CompactCells139To170.Box156.QInterval	CompactCells139To170.lean:11308-11310	b59abd5489edda5d581c52bc0534da07ee8f7640d83e1457a513a1e67f2d03e1
CompactCells139To170.Box156.lambdInterval	CompactCells139To170.lean:11311-11313	ff8a4629b354b48088bd5c135f5a3e4f88ab1b9cc8421776e16d6b52fbc93a7e
CompactCells139To170.Box156.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:11960-11971	c031085e133f05e678f977b24fd3ff0fce8aaf40926e40587a722cc648c01b7e
CompactCells139To170.Box157.QInterval	CompactCells139To170.lean:11972-11974	1072c84956ae5c6e6e63d6316667385915019c039aba30ba0f8c43ea6491c9ef
CompactCells139To170.Box157.lambdInterval	CompactCells139To170.lean:11975-11977	b7441e33b28c7f68937b84e0ed3017f3e06221b66622472d8cd6479c86a9bf60
CompactCells139To170.Box157.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:12624-12635	6c471a7cf6db3acc0e7140a30475fdb2fca6d72045e80c23337d7d7070d79360
CompactCells139To170.Box158.QInterval	CompactCells139To170.lean:12636-12638	4170f0210c59bf23f0af5e75260894d21b1f09d0c40e801f73ae7a172fff7b46
CompactCells139To170.Box158.lambdInterval	CompactCells139To170.lean:12639-12641	675951be7744b306d89104e3ef52bf60077844a56262a6abb37602f640dfba02
CompactCells139To170.Box158.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:13288-13299	81c711b29998fe0b1d32ed77cd4868a9b54d63046b48e114dd13ef6a93879744
CompactCells139To170.Box159.QInterval	CompactCells139To170.lean:13300-13302	b5f83583aa63cd030ee5b66a047d060b340d718241a0dde79893bd75b5f25f18
CompactCells139To170.Box159.lambdInterval	CompactCells139To170.lean:13303-13305	154787958732c67136de84837579632b0ef191eb171754c0b6a607faac20785e
CompactCells139To170.Box159.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:13952-13963	b10e33942d5df013c912349d3e90308b022419002f6a8c3c36fd00083504ab44
CompactCells139To170.Box160.QInterval	CompactCells139To170.lean:13964-13966	a248a5453d1f64a153fd7b3ebbd5cb1c23a0480fa655cf52a26717e8d867e6c5
CompactCells139To170.Box160.lambdInterval	CompactCells139To170.lean:13967-13969	dc3f1b16f82e1fbf5514e5365db01da55cc446670e7100fc79f7e342256ad38e
CompactCells139To170.Box160.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:14616-14627	7faba6ffe2ded59e54de97c54fe50dd900eab2c85eacd2b6ff301777ce53e418
CompactCells139To170.Box161.QInterval	CompactCells139To170.lean:14628-14630	0b868224b4e3cf71f051b5e21d9025f92c162ce3693039523c5f2029114257c3
CompactCells139To170.Box161.lambdInterval	CompactCells139To170.lean:14631-14633	59217c8be0b9d9a40b5ed563ea49c03823b1d0db9f0f5612836cf38af2e80e3b

Declaration	Exact source	SHA-256
CompactCells139To170.Box161.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:15280-15291	13f9281598ef76402a2b07cd4f1a91ffe65ef772bbdfb817dae40df210cf960e
CompactCells139To170.Box162.QInterval	CompactCells139To170.lean:15292-15294	73e9a5199d8acfbcf10a5b10307806c306867bc471d2b402381a921f1fe6384e
CompactCells139To170.Box162.lambdaInterval	CompactCells139To170.lean:15295-15297	50f6d335ee696f81fa7c7decbb88df5159bb975837bf5cdcc1a4ce6a69c4fc11
CompactCells139To170.Box162.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:15944-15955	21cd06207b13f62c155c26b95c9b87a1daf35125c040fe380e9af7ca06aafb0
CompactCells139To170.Box163.QInterval	CompactCells139To170.lean:15956-15958	c597558af7ba4c6d70c741f1a1b87283fd574d77a916a20a92f6239df9ef5d70
CompactCells139To170.Box163.lambdaInterval	CompactCells139To170.lean:15959-15961	1ecf263a553326757585f4d13c5157d8e1572cbb68dc709830a14b29e92b1244
CompactCells139To170.Box163.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:16608-16619	939a5d950f24e668fdc952381fe941594af8907edaba92c7721b37c584bf6fc7
CompactCells139To170.Box164.QInterval	CompactCells139To170.lean:16620-16622	2ef2e086693e73ee7f2a6293992e6157a4ee26edf990811d1fc5d9ecb606635
CompactCells139To170.Box164.lambdaInterval	CompactCells139To170.lean:16623-16625	9bf15763fdce3e736f94eea05e0d4743bcd488379e0a571586d4fa5f4fedfcb1
CompactCells139To170.Box164.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:17272-17283	755ece5bb320913a87f02706f8ce0c23f07fd14b634f8b76349825ccebcb694ef
CompactCells139To170.Box165.QInterval	CompactCells139To170.lean:17284-17286	4d292416cf671e10650928d606bf02fa167581d9de1ae63cf433e6cbb611ec59
CompactCells139To170.Box165.lambdaInterval	CompactCells139To170.lean:17287-17289	0a20fa40b4603e2bbafb385c3b1c871938d88571e846d8ad53a68266c5849d0f
CompactCells139To170.Box165.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:17936-17947	1b395a1a0da198da1c9d29b0d24a0d62406227dcdbb9bcb4619bc19be93e85f2
CompactCells139To170.Box166.QInterval	CompactCells139To170.lean:17948-17950	08adce2617d58429e5f3a677d0499e15f0b038883fc3036f105dc04abfc60be5
CompactCells139To170.Box166.lambdaInterval	CompactCells139To170.lean:17951-17953	99641535aa3b99610b480375b976757909b1cc5e2ea482f463dbed4508255cc8
CompactCells139To170.Box166.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:18600-18611	4d9e348715fe1241552bae5990fbb01a4d15353436d4c6c9bf131a0ae92ff625
CompactCells139To170.Box167.QInterval	CompactCells139To170.lean:18612-18614	b3851bcf7cfbcf71375f4e271840c1cff4e959f7952f07001f8b596b3cdbf1a8
CompactCells139To170.Box167.lambdaInterval	CompactCells139To170.lean:18615-18617	28d1004ea52cb61b37530169ded081ce361696b6041117243313ed8b7ab77150
CompactCells139To170.Box167.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:19264-19275	d37728a35925d3db83e9b72db6c213fb512854171eeda4a6b7626c22d753acff
CompactCells139To170.Box168.QInterval	CompactCells139To170.lean:19276-19278	2fb7b958e0ef583325bb9dd5b5d52a6fb770bcca4d9145c087f96fe029524578
CompactCells139To170.Box168.lambdaInterval	CompactCells139To170.lean:19279-19281	be0b2d905bcea63c1eb3b1e063accbd53b5448acfa54e76be485f09db9218d21
CompactCells139To170.Box168.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:19928-19939	446b691b40a817af6c2dab9347632c21aa28595d1fe46cfc08d2583c50c53f80

Declaration	Exact source	SHA-256
CompactCells139To170.Box169.QInterval	CompactCells139To170.lean:19940-19942	b644880ef549c7ad05fb3d14ef53ba35ad5728d7afe746c6c9293441991fae1e
CompactCells139To170.Box169.lambdaInterval	CompactCells139To170.lean:19943-19945	142086ae5d45cb59637c66c8512a9f7c372d7e126bb005288a8769c3a7b96469
CompactCells139To170.Box169.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:20592-20603	e1e8d60116f3eaed082262a1681c72ae4de73c89fffaefca9e59f9adee7275e0
CompactCells139To170.Box170.QInterval	CompactCells139To170.lean:20604-20606	97af062d9d011be9569336cf1c50d619d9edcf8358fcdbe8f0e1d4f07c562ee8
CompactCells139To170.Box170.lambdaInterval	CompactCells139To170.lean:20607-20609	5eff4036e6437022f43eb05e532f6c328713f15923f2bfe43b944dc1c35976d2
CompactCells139To170.Box170.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:21256-21267	844f7dc793168913a6ee65e0ff62afe936831b8addff2f346442893b219018
CompactCells139To170.bboxes139To170_candidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:21268-21498	94931ac5e5e6c85ad9a8f787727c31546ddf22bfc85fe062061f5b3fc7bdfef4
CompactCells139To170.bboxes0To170_candidate_not_isWeightedPerimeterMinimizer	CompactCells139To170.lean:21499-21512	190d0bd7e48a18fcfc9203cec84f4d67ae2e885027442d1acda7c15f1a7661d4
CompactCells171To202.Box171.QInterval	CompactCells171To202.lean:20-22	abdf6daf4306e8dbe17760c37a0eb592c79f60105b6799e73e35322c41ddc9de
CompactCells171To202.Box171.lambdaInterval	CompactCells171To202.lean:23-25	217966c5de3deb703c0cf6c405487487578727fe7b5e6f82d2d66acd884203f9
CompactCells171To202.Box171.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:672-683	0cbde0d4f3aac1755d5dfdc5be35ab8015544d57b85e32e51fab2afd95c28c5a
CompactCells171To202.Box172.QInterval	CompactCells171To202.lean:684-686	fd4a578d7f1cabd64b05e2d2511a704a25b4e589d5b02e302ea6db56b40ce80f
CompactCells171To202.Box172.lambdaInterval	CompactCells171To202.lean:687-689	c07cc4c548f3016afcb820186c2d764f7c6db303fc882b3f96b83aeb54e1ca9c
CompactCells171To202.Box172.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:1336-1347	03291cdf1e0e6c0f8fce3c3f085d9a23a774eff54e53eb99c25617ea0785004a
CompactCells171To202.Box173.QInterval	CompactCells171To202.lean:1348-1350	ba076c5f73980fb15ae96c30560b3e19316cb306219ff3168bb11787094c2f9d
CompactCells171To202.Box173.lambdaInterval	CompactCells171To202.lean:1351-1353	a82e3dd248e1c3c740ec79964ee31264258e5e600ddc6e6438160a0c20274917
CompactCells171To202.Box173.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:2000-2011	3aa65a93bf48e9632e85c58f9a71273bb42eb562984c52c79d84474d1314235f
CompactCells171To202.Box174.QInterval	CompactCells171To202.lean:2012-2014	b1e3d35036b2087650afa153158c250941a1cc0632ce802cca5f5542c6cf451e
CompactCells171To202.Box174.lambdaInterval	CompactCells171To202.lean:2015-2017	58cc4fa8d0f9a464acd3fbbbb7ee8bc6d972fe0ed3b498d6b55ba3058f2573c5
CompactCells171To202.Box174.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:2664-2675	ede88503652d4c079799da58072a5605dc326c044d5ded584a544500ccb78179
CompactCells171To202.Box175.QInterval	CompactCells171To202.lean:2676-2678	e03bd14d3d3b319927801907ffb498890a5a50db06f169bd5dc91ea4af5bd24f
CompactCells171To202.Box175.lambdaInterval	CompactCells171To202.lean:2679-2681	14e092a2c22c02b90ec28e0d4300c2ef11df1b5957640a26ddcf22c796ca9002

Declaration	Exact source	SHA-256
CompactCells171To202.Box175.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:3328-3339	7cb966fe63090d6f45978e9f94ed8ca8d50d142d5bdb6f36acd32ad07d843360
CompactCells171To202.Box176.QInterval	CompactCells171To202.lean:3340-3342	ed3ef9d714027b0c0a774a91ec5279f88ca491ff850900b84cb7d2497bc33001
CompactCells171To202.Box176.lambdaInterval	CompactCells171To202.lean:3343-3345	bd6cd3e034cc4e3ca9236a4f34b0de056a01d7cc37d1e225449ce4a90c6fd075
CompactCells171To202.Box176.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:3992-4003	b690b3de394280e615037736b79a57cac8da0526c063393798989a0053fe145e
CompactCells171To202.Box177.QInterval	CompactCells171To202.lean:4004-4006	d235f98a00234fa91ab3dfd4f63e39186cf534de600d018fc9ae13ff9179abeb
CompactCells171To202.Box177.lambdaInterval	CompactCells171To202.lean:4007-4009	a5e86465e5677733e626983e271dde038d1bddde39c06794aa142cff5a71f397
CompactCells171To202.Box177.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:4656-4667	a42a1edba64fef88853b4846342f40d75f096fa4d417924903f6088e5bb05b5e
CompactCells171To202.Box178.QInterval	CompactCells171To202.lean:4668-4670	6b6d04c46d571cecc5ad752bc3632f39953a185fcc4b9370633af4b73b7ee1c0
CompactCells171To202.Box178.lambdaInterval	CompactCells171To202.lean:4671-4673	72190f04fd89f72a5433c5ae7f9982e6eed41c565984f97db3a46207e071935a
CompactCells171To202.Box178.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:5320-5331	5e9ebe0bac33326de2d90205f0af66bc72fee9efae56df7c5ec5f74360001a5
CompactCells171To202.Box179.QInterval	CompactCells171To202.lean:5332-5334	264596259ab6bd21a5e62e25445100335686ebf38b1e977e563563dc2632af1a
CompactCells171To202.Box179.lambdaInterval	CompactCells171To202.lean:5335-5337	3eececee7fa457c375192064beebd3317618ab90965428873d56e2023da8d228
CompactCells171To202.Box179.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:5984-5995	8e60636c5dae2c981eb0adf00d63672faa31538d7c0b21d1a9bf1c0ca8a317a
CompactCells171To202.Box180.QInterval	CompactCells171To202.lean:5996-5998	a8fc77d9750168ce6e73b939748540545112d036ee58019b32083de1d783261d
CompactCells171To202.Box180.lambdaInterval	CompactCells171To202.lean:5999-6001	008d0581bce7391c71c36443fee54b14c1be27fd14c23a7162088c511e54e79c
CompactCells171To202.Box180.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:6648-6659	dadc00e991342874c64d9956764ef5a83e46b911764572b5d8d91cfd6859ac2d
CompactCells171To202.Box181.QInterval	CompactCells171To202.lean:6660-6662	6cf7a752aebf4cbf7acba08c6a87979f4008e6f6aa1039949b187c69a2b06820
CompactCells171To202.Box181.lambdaInterval	CompactCells171To202.lean:6663-6665	838ad6c948757b9dd33f2028b94db61f39c50ae1333e1e4bdf24d36cc6e0caf0
CompactCells171To202.Box181.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:7312-7323	aa866a9c9336e83fb2e9d0a90673786bf999436636f2e1d58783956449c1c8d9
CompactCells171To202.Box182.QInterval	CompactCells171To202.lean:7324-7326	1f247d2e39243386bd4599e7b41903c391c0c9301ddc7dbd578232ea94b8e3e7
CompactCells171To202.Box182.lambdaInterval	CompactCells171To202.lean:7327-7329	10208da5f1b245b045d97af7e9859217fa7cf65c9afadfe82b42a6fdca0088ef
CompactCells171To202.Box182.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:7976-7987	69e3721053dfdd26f9512307d7c101213b5db5b5df05cd6f0a85e44a8950664f

Declaration	Exact source	SHA-256
CompactCells171To202.Box183.QInterval	CompactCells171To202.lean:7988-7990	085a310a7d7110fd8331a5f805af4f32e66a6b02a6171e13345a8e38a7736844
CompactCells171To202.Box183.lambdaiInterval	CompactCells171To202.lean:7991-7993	db7fc94f2dcf2c8b0148156ada416f30fd8d036fcb123a866f25df5ac0366a
CompactCells171To202.Box183.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:8640-8651	807af68267ec0f798e12ecbe4b83314af57ecaba020c24843745b2d90311833e
CompactCells171To202.Box184.QInterval	CompactCells171To202.lean:8652-8654	39a4040539e3fe2c3b94557d772d6da609c432459caf54d9f359132e7811aed4
CompactCells171To202.Box184.lambdaiInterval	CompactCells171To202.lean:8655-8657	99e70ae3816174022ba0dc25de6daadce5bc05fba6d0d07189b642aa0cde04b8
CompactCells171To202.Box184.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:9304-9315	cae5da5624fe76f0e1e4ae44f39f60de33e89610b6f5952e85d0f520bdbc1735
CompactCells171To202.Box185.QInterval	CompactCells171To202.lean:9316-9318	c61946891f4031ce35c75bbdf5ed1ac0c0bc6b501db9311949e90527fcbfa25c
CompactCells171To202.Box185.lambdaiInterval	CompactCells171To202.lean:9319-9321	8003b3fd01163d323a525d4c83a23d890f8003e7d20aad6e19c7e801b2a23982
CompactCells171To202.Box185.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:9968-9979	9af3defe89cad69cce53f8c8064ae51b8287ebf983c8de8d805ca5ca3c027ce3
CompactCells171To202.Box186.QInterval	CompactCells171To202.lean:9980-9982	67b9676306452f8939c73cae6ce1149691e85df949bea4bf8309d3f82a035164
CompactCells171To202.Box186.lambdaiInterval	CompactCells171To202.lean:9983-9985	94740ccddceb632c5aca963996705aa178f15b92db2f548bcb1f5927a3c97d27
CompactCells171To202.Box186.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:10632-10643	e5e7e7c6fb7576d8e8b4249eebf4ecb6c3fdfed98b9a201dedcf7712e07d4acc
CompactCells171To202.Box187.QInterval	CompactCells171To202.lean:10644-10646	c25c444edfd7fb4c348e30f65e0b0318b4e93fc62b831d9851469c8981e17116
CompactCells171To202.Box187.lambdaiInterval	CompactCells171To202.lean:10647-10649	93a35e5d14b40364a92b8d7205260f65faaf3cfb8b162a3bf47d1bc24491e305
CompactCells171To202.Box187.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:11296-11307	10e9f96cb85c6096b0bfe39f19fed3f575b61bbcc4f03dd12e24d7431746ed73
CompactCells171To202.Box188.QInterval	CompactCells171To202.lean:11308-11310	f0f9b2fc73402e05a82b39c0bed3fcacf6e924f5c51f715ffa3e2e3c7e5d57eb
CompactCells171To202.Box188.lambdaiInterval	CompactCells171To202.lean:11311-11313	59a9cf81c922fa901edf898650074b708a86f353ead5a8b072ac44f24b461d95
CompactCells171To202.Box188.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:11960-11971	c60e843d1aea3b60c7f59dae03d0dd61b57cd9b5c6b0070eab0e886a0fe8c46f
CompactCells171To202.Box189.QInterval	CompactCells171To202.lean:11972-11974	8b1d62053ea29d11dbe54564d794ef2771892c4f478fee53c580b16d0e8b7082
CompactCells171To202.Box189.lambdaiInterval	CompactCells171To202.lean:11975-11977	2a768b5e287c4213d8e70a6c31a11f2557e3d90171dddb8c883de1663ce1ac98
CompactCells171To202.Box189.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:12624-12635	88b2e442fa1c500edb28cbe210f2f3b69eb2016180f7acd133f22a55c37069d3
CompactCells171To202.Box190.QInterval	CompactCells171To202.lean:12636-12638	c53cda5b8476a866e9f1c64851effae329eada4a79072d33904644a1bad98b2b
CompactCells171To202.Box190.lambdaiInterval	CompactCells171To202.lean:12639-12641	505125c9057a286c782ff34c63fbdb5dbf837db84e76d223fe67381005eef7e5

Declaration	Exact source	SHA-256
CompactCells171To202.Box190.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:13288-13299	023acb5a8d0e61fa3bfaa96a5705e8d112dc49ff8d1c8d0525c47aea424c751d
CompactCells171To202.Box191.QInterval	CompactCells171To202.lean:13300-13302	a2c4988a019be040bda3518cd09c4f4eee46bd2f5967f304dd051c712d860150
CompactCells171To202.Box191.lambdaInterval	CompactCells171To202.lean:13303-13305	04a26479b2ac5fe7025e36908c3280a2e48683f0ccd805bbf5a782d3389f35a7
CompactCells171To202.Box191.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:13952-13963	933b4af793b5aec123f596edffa833f8138288a88e685e0a90d26c3bd5305f39
CompactCells171To202.Box192.QInterval	CompactCells171To202.lean:13964-13966	b8e9b5b74f9ad0249bdca22bc978ac54c350db898e84f784278354e704ace7ef
CompactCells171To202.Box192.lambdaInterval	CompactCells171To202.lean:13967-13969	24b124e12453ed81bec065bef2c7ea90877d33b29ec8e42962e332ef37b0b695
CompactCells171To202.Box192.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:14616-14627	eae4044e5b388bbb3c4a16053ce9e0b0ba6947b0e3d9f9a7f9b908affe2fd625
CompactCells171To202.Box193.QInterval	CompactCells171To202.lean:14628-14630	425c2c5c08b987117fbd0ac774e3d2a1c29faab7d63327821b0b9ce055fc4f14
CompactCells171To202.Box193.lambdaInterval	CompactCells171To202.lean:14631-14633	39cf5cb140fa4919ea57e4f89901e3d5b6035c5adcfaadf194b27d2114125c387
CompactCells171To202.Box193.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:15280-15291	80bb227ae81ae11a2b308cd7593cc43f3db4f21de1a8fc1e2db86e2ccfaad4f7
CompactCells171To202.Box194.QInterval	CompactCells171To202.lean:15292-15294	acf232527ff0bdb53abbafbf06ddb705d4dd00572df8095939fa2c25baf33bff
CompactCells171To202.Box194.lambdaInterval	CompactCells171To202.lean:15295-15297	fb0d24feb1d7e59b54b16955d1f082e7c5329c4b5f09daaa349fccd67192acb9
CompactCells171To202.Box194.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:15944-15955	cdd8ac3f7723566b6c756e74d789e48b7950d80c93593723bc84f5c3ddd4112d
CompactCells171To202.Box195.QInterval	CompactCells171To202.lean:15956-15958	40e90156b9c97501124e8c3fc58a20e47568787db12359259504410d9bf31cbb
CompactCells171To202.Box195.lambdaInterval	CompactCells171To202.lean:15959-15961	25cbbee3a0adbcb79b1319836f3c22ea57de8d1ef4be6a550dcffa9a961f5aa80
CompactCells171To202.Box195.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:16608-16619	1f830e67807d5ffa4844c81d68e23a1ebd2b47b7c49ca7c722e864db3c8223bd
CompactCells171To202.Box196.QInterval	CompactCells171To202.lean:16620-16622	ba4bf33336a50a7c30b17210de4b5aee1d9620df287231db640fbd59fd54b64
CompactCells171To202.Box196.lambdaInterval	CompactCells171To202.lean:16623-16625	c606d462c9254303385a3ffa06819ea2adc8592f5b05869a4f16c2891e4f811a
CompactCells171To202.Box196.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:17272-17283	ff5dffcc41395f2b361c6be59812cde460c539cb371fc91e1d58e968b183f94f
CompactCells171To202.Box197.QInterval	CompactCells171To202.lean:17284-17286	930da11cd7622dcca87fd380d700ce8668b4b945d69840b1f15f647da3a73bb1
CompactCells171To202.Box197.lambdaInterval	CompactCells171To202.lean:17287-17289	295a6aa735928f2bdf225cbc447381aa8e3ea6c30d5b1a11517f84bef960357d
CompactCells171To202.Box197.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:17936-17947	1cd5cc0a831220fc2b04334145bb93037d468e809da3f3dd9d1526859f2d9067

Declaration	Exact source	SHA-256
CompactCells171To202.Box198.QInterval	CompactCells171To202.lean:17948-17950	8127a70294999d405508e22904029eee35624c47fcd37db247095d7670fdbdfc
CompactCells171To202.Box198.lambdaInterval	CompactCells171To202.lean:17951-17953	bb51166cbc50731c0acb59b47a228d8ce9f917c376ce7169b9c850ab4d79e47f
CompactCells171To202.Box198.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:18600-18611	40c98b4decf579087a745315d56fa0d4014fd7e4b5dd20a8b079ee805d8dbf58
CompactCells171To202.Box199.QInterval	CompactCells171To202.lean:18612-18614	418768080a3f40a4b4680a00826e30ebaa291ba82e1bcbdf32a8ca70444394a
CompactCells171To202.Box199.lambdaInterval	CompactCells171To202.lean:18615-18617	5b246aa92fa97705bb9ab39240679ee4edd5d414dba79ab5a33dd5f8ae38c6ef
CompactCells171To202.Box199.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:19264-19275	5945a229d91ec4be28fb1bbb426dd811092327224ef706bc0b14b8459fc2e88
CompactCells171To202.Box200.QInterval	CompactCells171To202.lean:19276-19278	032d490ecbae24a1c0a9e4080d86424295ba990bad9eec638409cdd5f6a245cf
CompactCells171To202.Box200.lambdaInterval	CompactCells171To202.lean:19279-19281	2963cca907ca00b971b902930aea931d10d4a61994286e607a3c5fb5bc01c2ed5
CompactCells171To202.Box200.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:19928-19939	da54748806ba18ae70a68df1e2121cd77bfe390c57c407fc92e3c43dfaabc15
CompactCells171To202.Box201.QInterval	CompactCells171To202.lean:19940-19942	73be23fab32915a0abe2f03364c4acb7d101c163dd704f3274ed37e73078e6a5
CompactCells171To202.Box201.lambdaInterval	CompactCells171To202.lean:19943-19945	6ee2e1933cf132d0879e52e2691a94596a37ee09ea2d72afdc66873297070f74
CompactCells171To202.Box201.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:20592-20603	c731aee8844d9eacd1cee6db7cbb38c479be29c13e440ca4897a4ae09c4ac05c
CompactCells171To202.Box202.QInterval	CompactCells171To202.lean:20604-20606	df6398c5069205afbad5c5fcea06655913a80782ffdd1db230cf29baac93fdd05
CompactCells171To202.Box202.lambdaInterval	CompactCells171To202.lean:20607-20609	2493c97a875d6de07045f8dc19b0d187ba4954da75a93c47d4eae7ad7c1dafb5
CompactCells171To202.Box202.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:21256-21267	936d79892f3aa5d16e12fe2c10a18ff6761742398b379097834c544f4d34382e
CompactCells171To202.bboxes171To202_candidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:21268-21498	ab8d83a63d780118a9f83af1e615d1d98eafd659431145a94e93f24d17943b8e
CompactCells171To202.bboxes0To202_candidate_not_isWeightedPerimeterMinimizer	CompactCells171To202.lean:21499-21512	58b72b25ef150d1ddf276a3bbc1bb8d5402633158bf87eb42c64040df4198456
CompactCells203To234.Box203.QInterval	CompactCells203To234.lean:20-22	ddce544332912a4a72665523dd0e67ec931e451fdbf97948d2e1d6a6d02166cf
CompactCells203To234.Box203.lambdaInterval	CompactCells203To234.lean:23-25	3bb41c54e0da945a0d3deb41a89cc2248907aba7909628914132530a29efb387
CompactCells203To234.Box203.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:672-683	e9c05345c404f829a0a985a4195006df4853fd4a1738a86c442e066a1ea54fb6
CompactCells203To234.Box204.QInterval	CompactCells203To234.lean:684-686	7c578b4282f58f02fd3c8d86633e7ec6ff4aa87368adfc3d2029e183d08e5d3f
CompactCells203To234.Box204.lambdaInterval	CompactCells203To234.lean:687-689	8f89ef3a0c832c3cd32c8c6f212a1bc8cca8371adcad56bbb882697b4f9661

Declaration	Exact source	SHA-256
CompactCells203To234.Box204.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:1336-1347	2f7632a5600fd4fe5cc8545085948b02279c8627e3412d681094af069aebd277
CompactCells203To234.Box205.QInterval	CompactCells203To234.lean:1348-1350	330e501bf852c19b654189cd59d2099952d9d2294d259f444c6879fcc6d2addd
CompactCells203To234.Box205.lambdaInterval	CompactCells203To234.lean:1351-1353	6582c4462b63a2b8cf8b706c04a450c855f3118ddb066fb7372046e8db8d8e8f
CompactCells203To234.Box205.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:2000-2011	2b010fd04ba612070eaa0c04d94abeea23784d1f02230b86a3f663afb6125b89
CompactCells203To234.Box206.QInterval	CompactCells203To234.lean:2012-2014	a709ba842fdd4301c2c00b3c453a726f82c3ecf9382b07f7787bc3d5822120ee
CompactCells203To234.Box206.lambdaInterval	CompactCells203To234.lean:2015-2017	43a3aa70dd09fffd4745add3e59eb85fdbc8108c4308a09de5e9a07cca8d7013b
CompactCells203To234.Box206.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:2664-2675	16a4152b0f1d332699872ed3206fc9cd303732d9292d0233c0c9424e15b48a77
CompactCells203To234.Box207.QInterval	CompactCells203To234.lean:2676-2678	f9b330e9aefb977723b8359cc6e5f09250a2bbe0a3e4239520caed09ae748b6a
CompactCells203To234.Box207.lambdaInterval	CompactCells203To234.lean:2679-2681	cd5bcfd40e16347c4b4ef5872856daec7a36a304a71a1a095bd8fe5869f19214
CompactCells203To234.Box207.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:3328-3339	2a84344e7a3172ab4a8d0783e426b91acd0c072e10e72662409d4de0671e60b2
CompactCells203To234.Box208.QInterval	CompactCells203To234.lean:3340-3342	8047e71a12c464ec50a8282761697742a636f4d2f458d8de9b63e392ebafcab7
CompactCells203To234.Box208.lambdaInterval	CompactCells203To234.lean:3343-3345	867ddccb9ce2952fce22acd8194ee904cb8c0eead3e369c55d2a2858bf872ff1
CompactCells203To234.Box208.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:3992-4003	a432ef3f4d5296fabe7a6bd2f8d5ca0130191fd511e662c11f52997194b62782
CompactCells203To234.Box209.QInterval	CompactCells203To234.lean:4004-4006	758ff204adc5d5a17d3f91143100af3a89c4151e05a96e3d39bcf5b19866b555
CompactCells203To234.Box209.lambdaInterval	CompactCells203To234.lean:4007-4009	b58e9462b1f78b36722069d02b0726e583cf6a002317e04f6c939e5627ed05b5
CompactCells203To234.Box209.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:4656-4667	66869b479e01f6b8662f6801cfef938005f610290aaf51b5afe4770959092686
CompactCells203To234.Box210.QInterval	CompactCells203To234.lean:4668-4670	afe1c02438b93264dec215c8f818fa6a4c653e036ddcae9bd3a067f2dcc92e2b
CompactCells203To234.Box210.lambdaInterval	CompactCells203To234.lean:4671-4673	532864466236d7e99dc003c52da0c107f1d6dacd9cd57c33079788ea890919eb
CompactCells203To234.Box210.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:5320-5331	e1b0fd2b2a36b6ab4ce5f893bcbce7879faf3421c8a81af9c3017bbb2ca2cde65
CompactCells203To234.Box211.QInterval	CompactCells203To234.lean:5332-5334	d05a795e1a514839cba5cf21dbee0cae06308c74 added2cd4d9f8c1439bfba0453
CompactCells203To234.Box211.lambdaInterval	CompactCells203To234.lean:5335-5337	95561fbe2fc6af353ac7a0513a640249d74a8db8d498d4fb9bb6ad313f9130c2
CompactCells203To234.Box211.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:5984-5995	729daede3ff9959fbfc0ff64715f15735c65babbd46b86edfa50f715558cecef

Declaration	Exact source	SHA-256
CompactCells203To234.Box212.QInterval	CompactCells203To234.lean:5996-5998	4a419397fbe6a2ffb0709054c4809d7d6cab14a2d9bdd5185fa8693ccccf8b138
CompactCells203To234.Box212.lambdaiInterval	CompactCells203To234.lean:5999-6001	cd83bd16fd0cc168b95c19b42876973bfda9a5b7f1c888807fd14b50e21ddf0f
CompactCells203To234.Box212.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:6648-6659	f20dcac18ef3274ffdf797801c2b295f0b4691d44accee86b697fc57f4cd6945
CompactCells203To234.Box213.QInterval	CompactCells203To234.lean:6660-6662	baf2b07d7595285e7211185627ca5f18e12a235fa20b36777977dcca7a63b44
CompactCells203To234.Box213.lambdaiInterval	CompactCells203To234.lean:6663-6665	4aa531dc5558fbcdf57c5981e5b742a25fc8f6ba36edaa7a64886b4be034604f
CompactCells203To234.Box213.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:7312-7323	1defb8f58dfdf29425a17e23b2bafbb7617db531a3b53703b3fd2814750a7426
CompactCells203To234.Box214.QInterval	CompactCells203To234.lean:7324-7326	774eec7e8f4a4d81a8f565112278ce172697edf351d9ef30923a13fc1652af55
CompactCells203To234.Box214.lambdaiInterval	CompactCells203To234.lean:7327-7329	9f368b9da6428cab01fb026393917c00d4aec2304dfc9b3c4cb843cb3b3c7e2d
CompactCells203To234.Box214.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:7976-7987	a2796df42d8e34a6dbcc1568e7bbe3d42f9fa30afdd0486ddbbf875f2e21fe2f4
CompactCells203To234.Box215.QInterval	CompactCells203To234.lean:7988-7990	8ead679749f38e7e9144a6be4b7a341a7da4a6f22d9a49122f72b092f29761ea
CompactCells203To234.Box215.lambdaiInterval	CompactCells203To234.lean:7991-7993	22f4bbf56e7a15171cd473ffaba18ef957c6841fdff74691489585429674c524
CompactCells203To234.Box215.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:8640-8651	45b69f2d062467a4ec04ddc5506549eaaa738718b5d754d5582d3146e39af9fa
CompactCells203To234.Box216.QInterval	CompactCells203To234.lean:8652-8654	0aa1742c71029e51b09dae9c320e189130fff49603bb3508c25d1070a5879346
CompactCells203To234.Box216.lambdaiInterval	CompactCells203To234.lean:8655-8657	bdf17f96c203f2512cf543bb7e31e73df574fef8e93e245fc342dba7d1a35cbf
CompactCells203To234.Box216.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:9304-9315	3044021ddbc71cb6459abdea6d9d36a9b9631341d36cc377d9e7d7eb06257473
CompactCells203To234.Box217.QInterval	CompactCells203To234.lean:9316-9318	52b3baeee032135eebdc36e759d221bacde97facfc38c3f39c1ac42f2e1819cc
CompactCells203To234.Box217.lambdaiInterval	CompactCells203To234.lean:9319-9321	803c7d86a1794c3863572efb04b3644f7eb106a389e3843a2f9b64899f515f83
CompactCells203To234.Box217.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:9968-9979	3aec8461e134361fbbd684d60e50b66fba1d334c5795cbcb8bad2648cb026308
CompactCells203To234.Box218.QInterval	CompactCells203To234.lean:9980-9982	78bb2d4525006b10599ce057c2ac0bd54aae981e9be3e3618257afb4187fdb57
CompactCells203To234.Box218.lambdaiInterval	CompactCells203To234.lean:9983-9985	c12b6b66767949fce840dffeb6b1dbdf49832ecdacce92949fe5094337e59e18
CompactCells203To234.Box218.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:10632-10643	d470b7ab5cb97c74358acd8805c3c9bb8f50bb2a362bd57fababb80718041849
CompactCells203To234.Box219.QInterval	CompactCells203To234.lean:10644-10646	952e0fc2495aa973f923822625fc63c265cc35abcfc3aad952709a87ad935478
CompactCells203To234.Box219.lambdaiInterval	CompactCells203To234.lean:10647-10649	3233bcccd9a3336c169e852700990bb34d84e871c473af14b5758c9e7c4f98ea

Declaration	Exact source	SHA-256
CompactCells203To234.Box219.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:11296-11307	814499edcf3567c879b3db2ccfd4a28d08006a01dc8f8e4d37a8f825393009db
CompactCells203To234.Box220.QInterval	CompactCells203To234.lean:11308-11310	af1a2d011ff13c4f83af935286b5bf477d8df6c2fc45d44cc45c4d233c7ba217
CompactCells203To234.Box220.lambdaInterval	CompactCells203To234.lean:11311-11313	4ba8b5e42e46c630f8477322af5d6cbef33733d3c059c7a3ba3da4d1edf2a4f7
CompactCells203To234.Box220.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:11960-11971	647a63ce34f8c68be69be0d0952252813712e5478522bb79c3458d36d850d5ac
CompactCells203To234.Box221.QInterval	CompactCells203To234.lean:11972-11974	f9420771d2c588cd1aeb798644712dcd09839ea67f583994f1b4d6a2da4b7b67
CompactCells203To234.Box221.lambdaInterval	CompactCells203To234.lean:11975-11977	a53daf3a6f19ab85f615e2f31b2a6dd12cb62d71ef18708e16cc7a6379fc2803
CompactCells203To234.Box221.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:12624-12635	67de096a5aaf8726f98d9b094dd39f58fcd38d58701cd2ca8167547534401804
CompactCells203To234.Box222.QInterval	CompactCells203To234.lean:12636-12638	9e8a51dbde1a2a07175d8076d58625d5e81ca15566bec8830f36dfb28d490564
CompactCells203To234.Box222.lambdaInterval	CompactCells203To234.lean:12639-12641	48a16dd5f7a52a03de9c5c498bfae5823a4a95685170e45799e8d2dd28dab224
CompactCells203To234.Box222.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:13288-13299	b48d7d5727430688a9b8008450dde62a15fde577130ddc88d13d1d712dc16a5c
CompactCells203To234.Box223.QInterval	CompactCells203To234.lean:13300-13302	065b5f7998275f05d24c0031e2bf015b87dae985c8bdaf8a135a93a89c6699c6
CompactCells203To234.Box223.lambdaInterval	CompactCells203To234.lean:13303-13305	ec51b82778b9b16f09f86e22f3efd79e444f2716745b5a96c7b9ec6e87ff0d6e
CompactCells203To234.Box223.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:13952-13963	2252fc71783cffe651565bc937e1b17a935396876feb1a72f5d5782202f13b45
CompactCells203To234.Box224.QInterval	CompactCells203To234.lean:13964-13966	b00d3d719c67ac3a3f6aa9940433e19ae2bdd8181c66b57e2e8c6c13c298af1f
CompactCells203To234.Box224.lambdaInterval	CompactCells203To234.lean:13967-13969	65b85a95609f931eaea0b503edaff59c9c45c3ddccf05a526c03740081e86676
CompactCells203To234.Box224.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:14616-14627	1409a792ee6c19892fed61eca6e55121e7dc62d1947b711fb2032efe52c67336
CompactCells203To234.Box225.QInterval	CompactCells203To234.lean:14628-14630	9a43d0e4a5769785965b4d8305373ead2cfd4a2560ac18ae234db00c70f43e5a
CompactCells203To234.Box225.lambdaInterval	CompactCells203To234.lean:14631-14633	9c35eaf6df9287b4c483d0250c6cdfc9a5c9594755ff8f419c72eca351932e75
CompactCells203To234.Box225.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:15280-15291	6d80c7ce235fca76c0d76f822aaf0e30be1d3155e0b96c5eb25cc133c9cb9fdc
CompactCells203To234.Box226.QInterval	CompactCells203To234.lean:15292-15294	0c8f738a62d609f054eb328b000b86ee4808443f29ac03884ebf8291f8c10010
CompactCells203To234.Box226.lambdaInterval	CompactCells203To234.lean:15295-15297	5d507edef3254a082dacd7eeb17ea6129225477881f4996b4f47db1ba9ca1e42
CompactCells203To234.Box226.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:15944-15955	c42343026c45ad2a567ea25ac14e1e6df220a7c369382fc837ffebc9399605aa

Declaration	Exact source	SHA-256
CompactCells203To234.Box227.QInterval	CompactCells203To234.lean:15956-15958	0c1c2b4573c87567c9c423563bdd12f452efa00ad5d588bdb8709590a4327b9f
CompactCells203To234.Box227.lambdaInterval	CompactCells203To234.lean:15959-15961	9615b00f1c44f64167866087f0afc1e4ecc001d1a6e075114619087469ffea38
CompactCells203To234.Box227.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:16608-16619	03c41543a8ca73c3960ca1deeddd103c4ee0459b24703a530c54a997ced5df1
CompactCells203To234.Box228.QInterval	CompactCells203To234.lean:16620-16622	a51646b5b9369a9b250fe401b368ad230e1709860e1cdc817a2dad382b05e1cf
CompactCells203To234.Box228.lambdaInterval	CompactCells203To234.lean:16623-16625	eee27ed63870ce6b45f5424884324a2ef5a71aa2f06d15e86b5753c9a70ca555
CompactCells203To234.Box228.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:17272-17283	f0a45aad90e60e129c9217ccab6910665b0a74bc9004684004981bffc82d19
CompactCells203To234.Box229.QInterval	CompactCells203To234.lean:17284-17286	342b809eb2073412430655910d96550d18193f614309ca807f59d09c0f90f368
CompactCells203To234.Box229.lambdaInterval	CompactCells203To234.lean:17287-17289	8b41d67e5f4e55610f9b1d762cdc60812ff4abe85e5062693305fd3c8b7136f4
CompactCells203To234.Box229.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:17936-17947	8f4c541980c4348bac1ee24a17e2986fdf9368b6d1da9ee5d5dbcfb7249c0b19
CompactCells203To234.Box230.QInterval	CompactCells203To234.lean:17948-17950	440e3be4353fa5e8384a8dda80718922d8a4484e64704e04a14fb19052016956
CompactCells203To234.Box230.lambdaInterval	CompactCells203To234.lean:17951-17953	556389d01ee71412a68b0b88e42c5b12db30619dcda3f1cd2d1dd2eed75bcfc2
CompactCells203To234.Box230.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:18600-18611	e7c1bc78e4858b21032c371020bb3170d78b0272c189881e044c1ee8496a67c0
CompactCells203To234.Box231.QInterval	CompactCells203To234.lean:18612-18614	d461635bbef5cc35adb9da9f4161aa2230301ea118346a9baaad32e9edef3c9
CompactCells203To234.Box231.lambdaInterval	CompactCells203To234.lean:18615-18617	e38d8ede96612b86c7807394cbbf2c88009b36b3f937d1ab34aced40217acb1c
CompactCells203To234.Box231.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:19264-19275	2968fd9f60c188264fae7f2a3ccdf0904948da02d96e048f46ef430c9d026b29
CompactCells203To234.Box232.QInterval	CompactCells203To234.lean:19276-19278	88a23288993f9403d32e5fc8df8a6202aefc72f0913ba20753aecc30a299a77c
CompactCells203To234.Box232.lambdaInterval	CompactCells203To234.lean:19279-19281	a7d4c329474cff6fadd7d147df6f1b90f5ab986c480566d4f1a1bc48b15a2281
CompactCells203To234.Box232.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:19928-19939	1d81d5985986079f6a2ddac5ae9d9897cc74454dd17232a9e084292282c168c5
CompactCells203To234.Box233.QInterval	CompactCells203To234.lean:19940-19942	568dd7ffb575fa3fb046aa27f225f5607083d12ae5d50f2ddb91538088572581
CompactCells203To234.Box233.lambdaInterval	CompactCells203To234.lean:19943-19945	778edc65040c82eb1975b6c874904f9975f71ffebf7ae9e557eabe197631b64f
CompactCells203To234.Box233.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:20592-20603	858d71e58983a3b73200663656c42c53cd55e252b98c7a1bc4c28518980c987d
CompactCells203To234.Box234.QInterval	CompactCells203To234.lean:20604-20606	b24b5922cdbbe0d09d2a24dab8229698c20953c37286492cdbdf4ec395b41eb8
CompactCells203To234.Box234.lambdaInterval	CompactCells203To234.lean:20607-20609	57c93fbbcf786964ac5676abb82c32fb8ea4a744d88fe9bbff1534b827a20cb

Declaration	Exact source	SHA-256
CompactCells203To234.Box234.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:21256-21267	6a942bef8d64c00ef7d6fdacc3f8041980f017fe235a00277c27c3075fa5db8a
CompactCells203To234.bboxes203To234_candidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:21268-21498	b16b14c19b511f5808a209c1d2d8262bf301e9fc16ad48af8408347d2ac82315
CompactCells203To234.bboxes0To234_candidate_not_isWeightedPerimeterMinimizer	CompactCells203To234.lean:21499-21512	2a4333f6da31eb87bd878ee83d151871a4f7be9ec0959079544fc17aac891bf2
CompactCells235To266.Box235.QInterval	CompactCells235To266.lean:20-22	c8f0e458527f0057dada886e1ae5cc848dd39d75ddba5f5ab0072173ff5207367
CompactCells235To266.Box235.lambdaInterval	CompactCells235To266.lean:23-25	cf31069303d2679e0c6d8e942faca147e3047a2fa08c67d40fea6660eb3aa7e1
CompactCells235To266.Box235.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:672-683	963badaabe151307d63e8ca8c6ebabc6cb5c093d0baecd5f0d77640606e53933
CompactCells235To266.Box236.QInterval	CompactCells235To266.lean:684-686	e6acb588ecd6396e1e4fdcd7dedb11ea521e9a93d3850f3c82a193a85e606bba
CompactCells235To266.Box236.lambdaInterval	CompactCells235To266.lean:687-689	205bb40e563fe0358fb904d92abaaf40815288afc44a6ac9551b3850a0621a91
CompactCells235To266.Box236.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:1336-1347	8f98e0eda6ab9f08ef42bc538bfdb7719adfb98c75cd08c14d5b2deb53939486
CompactCells235To266.Box237.QInterval	CompactCells235To266.lean:1348-1350	3ee23102b7c955ccc8b3c99e70ee404b8195b1866381c34a6a3bdbb37defc588
CompactCells235To266.Box237.lambdaInterval	CompactCells235To266.lean:1351-1353	434c9830956f293ec25872f1f5d4faf8e2b113500f81dae14901bc6636ed9861
CompactCells235To266.Box237.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:2000-2011	ca605fa50545608e508438922f76123c8035f2dabd14f45f877fe021f69f718c
CompactCells235To266.Box238.QInterval	CompactCells235To266.lean:2012-2014	fd1e212adc0faa25cff6bc15572602aaf0085f5ad56c129e7657b233479b79e5
CompactCells235To266.Box238.lambdaInterval	CompactCells235To266.lean:2015-2017	ad52069a7bd60df6c642f60bd266b1475b36f329e872821c044ba53d70f04df8
CompactCells235To266.Box238.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:2664-2675	689df6f7ee64ead3a7ebe84c60b627174330ce46381627e55cfdce9783f54653
CompactCells235To266.Box239.QInterval	CompactCells235To266.lean:2676-2678	ab92d8c753b9eb23549c78fc6c51b686aaf261dbb36c558f595af85ce874d593
CompactCells235To266.Box239.lambdaInterval	CompactCells235To266.lean:2679-2681	19aad87e907d1ba348bbf749fb0cfe1bb0cc80ecd97a192a6f10d9ef397830bc
CompactCells235To266.Box239.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:3328-3339	fe2b995c1db709bb3de7d8fc5663151330492a7fec1d0e4e9a8f16c6d5e85622
CompactCells235To266.Box240.QInterval	CompactCells235To266.lean:3340-3342	f2368bbd55af4d84a366cbeea3f9a5e380d1901ae4868c2e715dd84f3333e273
CompactCells235To266.Box240.lambdaInterval	CompactCells235To266.lean:3343-3345	48fea21c6001bd99af6b1653cf2937dc40fadab34af735d2ad7c4e1d4557c5c7
CompactCells235To266.Box240.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:3992-4003	0f2258d05b22ce75c927b4dc4db283478a7c8e6d92df3f8114587ee806641d0b
CompactCells235To266.Box241.QInterval	CompactCells235To266.lean:4004-4006	456847d7f1a3a4a6e4020d8217421fa33b048bf4059e1236217fc068e6d5b18c

Declaration	Exact source	SHA-256
CompactCells235To266.Box241.lambdaiInterval	CompactCells235To266.lean:4007-4009	ec14b4f3ea3ee994c94738fa51dfdcfb079f99783280a1910215756cef5d66d7
CompactCells235To266.Box241.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:4656-4667	a3b0cd79f9611daccf8f34431f9c440b0bec6b73b248db1c9a0be2b6cc1b8b84
CompactCells235To266.Box242.QInterval	CompactCells235To266.lean:4668-4670	f8825c94abdf2e8f81c0681495ba7538e3d3a302faeddae26722318e80eaca8
CompactCells235To266.Box242.lambdaiInterval	CompactCells235To266.lean:4671-4673	39099eb120387dd35a8ebdbadf5a6a396f06a688439181dd0d4533b014011b63
CompactCells235To266.Box242.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:5320-5331	04d3d58ba3475bf629aac4cb7ec187820e8e8c9b921938221e3c758befe909c0
CompactCells235To266.Box243.QInterval	CompactCells235To266.lean:5332-5334	33c9236b02098fd09aaed35e201533ad62642649b167c81c13bf067a1d789d4d
CompactCells235To266.Box243.lambdaiInterval	CompactCells235To266.lean:5335-5337	b25f1384d6a694a2a0396980a46b2241c64f74f0b9cb502f1c1fbb3fa6f576c5d
CompactCells235To266.Box243.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:5984-5995	60d60dc0df599dc3f0fb2f0e3efc988bfd6891159afd0f7e526bcce1521ecd11
CompactCells235To266.Box244.QInterval	CompactCells235To266.lean:5996-5998	050829c65de050eb03a6ec85b0727860e453db26fcbba87fb429f0ae3a63b255
CompactCells235To266.Box244.lambdaiInterval	CompactCells235To266.lean:5999-6001	ab7c379ee21aa96e8b7e9b44c8f22bf64544868d38a7a4d0f68ab83440a209dd
CompactCells235To266.Box244.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:6648-6659	48d2fb8e44802852044cb484fb5c2db8992510af7d0a4488e21106f3812e6dd1
CompactCells235To266.Box245.QInterval	CompactCells235To266.lean:6660-6662	776c87b0c4944a30856969db0c98d151b5fcd0a2a99f2881b791d5d316d9ea18
CompactCells235To266.Box245.lambdaiInterval	CompactCells235To266.lean:6663-6665	ffcd6dc3c70d1b46ce08ba4dfbadbe79e1dbc3ea681495090db617ad9cbce4b5
CompactCells235To266.Box245.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:7312-7323	47918f8bb5cf372084be4be2cad8a43d9596d118f773a3dc91b5870f0d6aaa14
CompactCells235To266.Box246.QInterval	CompactCells235To266.lean:7324-7326	3b3d4946216c8e5c09f7bbaba259994b60caea1aa8e9159ea0e226571860b8094
CompactCells235To266.Box246.lambdaiInterval	CompactCells235To266.lean:7327-7329	2f72ec96053d14e45695902fb84a1ee41758abf94546e1fb53666178c9f5bc25
CompactCells235To266.Box246.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:7976-7987	f64b977dad1505f9fe6518fc923dc9f820166d2db270aeeb8f8f2fe27f28c84e
CompactCells235To266.Box247.QInterval	CompactCells235To266.lean:7988-7990	ab9ea36c2fa411ab85aba040ee04caa95616cace8368b3f2765b9640f073aa96
CompactCells235To266.Box247.lambdaiInterval	CompactCells235To266.lean:7991-7993	d329aa6edf28cf487d556aa684d40be967aed4b74dece854e3e817b9ff8ddcf6
CompactCells235To266.Box247.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:8640-8651	afd4867661b8d6eee4ba937f532ae098d1aa3f3f746e8166add81ac22c9f393a
CompactCells235To266.Box248.QInterval	CompactCells235To266.lean:8652-8654	cf8bd0ea71ffda1ff5f93b072f6c606c78242eece3153cfff70569854eaea3166
CompactCells235To266.Box248.lambdaiInterval	CompactCells235To266.lean:8655-8657	ec57521aed607461112ac74f47694f46bc89437b1e8a6413c42c05c74468db54

Declaration	Exact source	SHA-256
CompactCells235To266.Box248.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:9304-9315	408e4b078ef3f20785b85fa3ca46c4b272e5ac3b341bc042188a573544a97708
CompactCells235To266.Box249.QInterval	CompactCells235To266.lean:9316-9318	36cf637dbaa9c6e4023d31fbd75328f6ec30d6e427528ad0603edeb247d9e75a
CompactCells235To266.Box249.lambdaInterval	CompactCells235To266.lean:9319-9321	b911a07ba969b65a54e42a9b37a02fdd4ba3b34ea5b223c3f37de3d0d13c0a46
CompactCells235To266.Box249.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:9968-9979	f6843cc9d8f056bef0e17488c38b1741d0aea6e5d7bc4998612ca6b7456de043
CompactCells235To266.Box250.QInterval	CompactCells235To266.lean:9980-9982	04eb6c2e2a3d2690b706c4b7e0b531a3f9a48c35ac300cfeff065d149ebf417f
CompactCells235To266.Box250.lambdaInterval	CompactCells235To266.lean:9983-9985	f06ce2ae2820bafb0aeb37d3413a79aa9a8c4502dba8f64e9db1d559040ab16c
CompactCells235To266.Box250.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:10632-10643	29a9225aad35b433fdecdd24261209bab26d5173720ea1cf3369584bc137aae9c
CompactCells235To266.Box251.QInterval	CompactCells235To266.lean:10644-10646	963e4d61f782271b93835c6d68109a19561be3cc4d7dee32344917cb9034cba6
CompactCells235To266.Box251.lambdaInterval	CompactCells235To266.lean:10647-10649	2b8b22db3bc5bdc4cc68852e5d56115a254c5ea54cab16ff8c77cbbc6ae0cc81
CompactCells235To266.Box251.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:11296-11307	937043be68165fc254692d4c54e20a5306bb33d67956d2719755f1db147ecb3
CompactCells235To266.Box252.QInterval	CompactCells235To266.lean:11308-11310	02d911263f16331eb4476da32774a78ca05cecedfded00283d816e3b2514b554
CompactCells235To266.Box252.lambdaInterval	CompactCells235To266.lean:11311-11313	38446238a1362cd9beac1945c7b6efaeaf573c6e12665c6829014df0982a5c72
CompactCells235To266.Box252.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:11960-11971	4690c2c7dea4929c4edfda1feb9c96f47c9fef63a4b5402744c9176c77de515e6
CompactCells235To266.Box253.QInterval	CompactCells235To266.lean:11972-11974	34214e31b0dc2e6a3f16cfc256ec0ddcac535eb0047adfb99545fce5f7c49669
CompactCells235To266.Box253.lambdaInterval	CompactCells235To266.lean:11975-11977	a248cec8784933488982830084cc2de054c4565b80f5682e0e581f334eec7735
CompactCells235To266.Box253.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:12624-12635	066f796b0a6c144db341655e4c695a5064b47b4f2e2a8a37cdc075baa4fe43ba
CompactCells235To266.Box254.QInterval	CompactCells235To266.lean:12636-12638	cafae28de3c1dea42b550d8c2671c5b426c039a1c4407ec971735b03f8c55665
CompactCells235To266.Box254.lambdaInterval	CompactCells235To266.lean:12639-12641	d4405eb07e12f124496d90411280cfd999eddcd23e7ad0d0dc7cac848f357aab5
CompactCells235To266.Box254.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:13288-13299	77b9f1ede1afb3534a73b621daf62c046034e1c2a9dc4245e04db2fe7f1634de
CompactCells235To266.Box255.QInterval	CompactCells235To266.lean:13300-13302	3460f460c896c74317a25ba4ad2c1984620d2f298b905535b5fcb29c27f0b287
CompactCells235To266.Box255.lambdaInterval	CompactCells235To266.lean:13303-13305	246cc866e43dcbcd09bc564bce8d39be246517fb893ff3733fffb51681a16387
CompactCells235To266.Box255.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:13952-13963	8967fbd729cdfefea978370371907edf98ca46ea6ffd49e7091c813004a364e3

Declaration	Exact source	SHA-256
CompactCells235To266.Box256.QInterval	CompactCells235To266.lean:13964-13966	62ad9a4dd419195d0bbeaf8da7a49e7715c5ee2f081d9ddf907270e111ea0378
CompactCells235To266.Box256.lambdInterval	CompactCells235To266.lean:13967-13969	91440e30cc62ad7c7ebd2d2ea8759f97c898c21181afe83c78a55cb1d98c2121
CompactCells235To266.Box256.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:14616-14627	d5a660076dea68a67d88f7757cfdc50ce65fb2824f4d940684229fe9c20057b8
CompactCells235To266.Box257.QInterval	CompactCells235To266.lean:14628-14630	cbfbbd8c209d592ab8bc4947e15572fbf6c21fb76fbb7691eedb3d724df122fa
CompactCells235To266.Box257.lambdInterval	CompactCells235To266.lean:14631-14633	88edf7bb5eaac340b25140ce77fb44e2756e8e36601642b491f54cf0cc556175
CompactCells235To266.Box257.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:15280-15291	984593caa9e9d53328dc8839cf7d987218e6e004d67438bc8ecb08e7d445656a
CompactCells235To266.Box258.QInterval	CompactCells235To266.lean:15292-15294	a14ab39461dca75e08dc10addbc34f306654072ac0d0bf646a8996d62f38918f
CompactCells235To266.Box258.lambdInterval	CompactCells235To266.lean:15295-15297	74f0c94cf035ebf9e75a9db94565239a1392252ce5f4bb778fe1d89951178b89
CompactCells235To266.Box258.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:15944-15955	253c46cdfdee3cef57c3aa120124c5dbb2f068f526c313d59f152c841c602b89
CompactCells235To266.Box259.QInterval	CompactCells235To266.lean:15956-15958	7aaa195550a112f044c9b2aa0cc7416d8f3055a14a3e168143ed0439f8e8303d
CompactCells235To266.Box259.lambdInterval	CompactCells235To266.lean:15959-15961	62524bf0f420b4b77c582ac6704ff3a920f800cc0bfbca50ee2d2212126207bb
CompactCells235To266.Box259.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:16608-16619	3509846fb2893069f376328027b45ca57b1daf7d0db39992e33919d20f86f7ba
CompactCells235To266.Box260.QInterval	CompactCells235To266.lean:16620-16622	07fc1dc1dcc80470f00a28c0a364b61c8a8ef7411e21c0e4fc5dea5e27b763aa
CompactCells235To266.Box260.lambdInterval	CompactCells235To266.lean:16623-16625	6bd8324a222b4f379cc42f4fd2110124ea7434b826d3865a27b13d4423a422c8
CompactCells235To266.Box260.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:17272-17283	e73c48896e63548d3c90b8359a57dc38fff45c95698bdacbf434d63fe627260
CompactCells235To266.Box261.QInterval	CompactCells235To266.lean:17284-17286	1f5b6b0248c2f98f82bcbba326cac840e35c0e6490c9608e3b0cbbb8c5b57e74
CompactCells235To266.Box261.lambdInterval	CompactCells235To266.lean:17287-17289	689c61a9f25e38439267beaf7880cee2a733b9f2fa6d0a912e0d1058a7cebbd6
CompactCells235To266.Box261.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:17936-17947	e1b3b1997d9b52116495fc14a32cd0dc1910ecceb04ac6f2855b62f7242a148a
CompactCells235To266.Box262.QInterval	CompactCells235To266.lean:17948-17950	66f156b99662e37b2a9f26c9ba811d62adaa34e90abd4ca93914e524fa428683
CompactCells235To266.Box262.lambdInterval	CompactCells235To266.lean:17951-17953	96b7920f1aad2f3fc3bfb1370ac80a217d6481b69560d7bd033b6b9d9fb2eacf
CompactCells235To266.Box262.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:18600-18611	b61e86c44888ca43a5d0d6553bcafe4466d2496b1aa836285a4e4384e5eb1375
CompactCells235To266.Box263.QInterval	CompactCells235To266.lean:18612-18614	e10cabb796e60ee5b44cb0f7b94cdc898f35271ef38169675fbbe8976a22789f
CompactCells235To266.Box263.lambdInterval	CompactCells235To266.lean:18615-18617	14ce025f97e0f80fcd024bb7be928972bd74329524a77d03942eb9214d5a7e1a

Declaration	Exact source	SHA-256
CompactCells235To266.Box263.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:19264-19275	30ad3be735194944e9cd3c963c5becf37b560242b339a2fa0d63c395fc3de861
CompactCells235To266.Box264.QInterval	CompactCells235To266.lean:19276-19278	f9a304198d9aaf6b1c472c831fe530552ce5a9256bc516584e4e90ea6fa01e33
CompactCells235To266.Box264.lambdaInterval	CompactCells235To266.lean:19279-19281	b9f6e9c598b8f3931aeacdf11d68ebfc70cc41b5775e3684dea80f9504ec171f
CompactCells235To266.Box264.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:19928-19939	77727887b400c7eedf05e3ab0a83bcc49bf5fee0bc29243b77859b605e14cad0
CompactCells235To266.Box265.QInterval	CompactCells235To266.lean:19940-19942	dc1b442fc58f932d4d6d8cf6f6c3429c545fdbaaeec19493a2b9555e0134298
CompactCells235To266.Box265.lambdaInterval	CompactCells235To266.lean:19943-19945	23d6186a9bc4d0a842bb8565c2fa9399f960dec398a5fe3b491ae7cb0b259452
CompactCells235To266.Box265.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:20592-20603	0a560b24c9d9984d43ce5e09bd7964d7432d96a29c817224149e63d14a8997a6
CompactCells235To266.Box266.QInterval	CompactCells235To266.lean:20604-20606	60d7a0de30313d19dbac697671a5e2aa1be1a7e35bcec2e1b9e0ba2840d88682
CompactCells235To266.Box266.lambdaInterval	CompactCells235To266.lean:20607-20609	ecdda9cf690a2f6be0d71cad3f5c852544f782a0d309ea25aef807fc49a3c0d5
CompactCells235To266.Box266.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:21256-21267	7a595a97aab78f2be70a9b8ca234ab8d29791a7f9fdbeb1b8573b34931ab92e2
CompactCells235To266.bboxes235To266_candidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:21268-21498	72052237e751d3960bf550f49b68f0e6c2bb777b4a746d382ddadc77a213a3ce
CompactCells235To266.bboxes0To266_candidate_not_isWeightedPerimeterMinimizer	CompactCells235To266.lean:21499-21512	fb420fdf20c9ea9177f9b327a5cbfed2623f6dc31d2e170786825355c74000c9
CompactCells267To298.Box267.QInterval	CompactCells267To298.lean:20-22	7f09bbfb20e9be6b754b17153ed084189233e6c9a6f3de2991f09af1ce83cc16
CompactCells267To298.Box267.lambdaInterval	CompactCells267To298.lean:23-25	43a688415160ff09a511472c53951d668c186333bbade5286436ea2de02b36f3
CompactCells267To298.Box267.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:672-683	04e60e6cf641a8b0d15a23606805622954f313bbf7682ebd18f0c62f591c9943
CompactCells267To298.Box268.QInterval	CompactCells267To298.lean:684-686	9712f9dba2f62028d225f83597e10b2b901837ad7a81cf2e4073ad1922b8febd
CompactCells267To298.Box268.lambdaInterval	CompactCells267To298.lean:687-689	3b06d2811cb059204e0e8172759eeec3ed8843c359e088d7513a8d8d431a8f43
CompactCells267To298.Box268.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:1336-1347	b95c6541ff6876c57adc07be9564400aca3ca1fa0c178d74ab2987212f1b227e
CompactCells267To298.Box269.QInterval	CompactCells267To298.lean:1348-1350	5aa8d0df59b24a34e6214599c507f825e2c903b082e5a086661d87e053916e04
CompactCells267To298.Box269.lambdaInterval	CompactCells267To298.lean:1351-1353	143454ddc1f39f8edbf26f6107827f80ad0a86f9326a998a33fbb61b27f90b53
CompactCells267To298.Box269.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:2000-2011	7500e9be8534ffac0b7ab256f10b735f57d59b35f5eab472d7e6c35733ca7d0e
CompactCells267To298.Box270.QInterval	CompactCells267To298.lean:2012-2014	68b8d2704b021f79419c4fd3d0a1f0834e76a18c4c7b9f713dd60a2726347fc0

Declaration	Exact source	SHA-256
CompactCells267To298.Box270.lambdInterval	CompactCells267To298.lean:2015-2017	1b3b0849838b87e6d28ce14a376780b4a04d497d1bc7d8a98af4b4d9231fe293
CompactCells267To298.Box270.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:2664-2675	7244cd3942ecb0a3dd1c71a8b718e6c06b83f12cbd008457764fd1efefac55fa
CompactCells267To298.Box271.QInterval	CompactCells267To298.lean:2676-2678	e83800beaa1fa5d2fba33bceabad995aa7f86dad568c01a1c74fdd868d009bae
CompactCells267To298.Box271.lambdInterval	CompactCells267To298.lean:2679-2681	f8de82ed4b95651b7426846c926d736f7f82bd6624ff58eed98fef8d7eb1b2e0
CompactCells267To298.Box271.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:3328-3339	b7de13f0d85954877a1adee67668d1f4ab0eba7e2815a67e549c6873ef3146da
CompactCells267To298.Box272.QInterval	CompactCells267To298.lean:3340-3342	8c7caf6a72e0283fe4799e698dfc576d9c50b75155c8c0d26ad9d5c9ba32797e
CompactCells267To298.Box272.lambdInterval	CompactCells267To298.lean:3343-3345	cdc1709ca9649731fbd47f887aa4c3e4c2c7ad24da8ef1364af8f72c7c15ec0e
CompactCells267To298.Box272.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:3992-4003	489c872f7481cc88f26970567d03ce77e36c3156d5efa4b6fd060e84b27f4972
CompactCells267To298.Box273.QInterval	CompactCells267To298.lean:4004-4006	72d6721b3c00ee67d4e040a29960494919c126313a3c20e93fe2ce0eb4ca1d41
CompactCells267To298.Box273.lambdInterval	CompactCells267To298.lean:4007-4009	5cd4f5e5d990cfdacc221ca6c90531709e92865bcac2ca8daacfb31799162e4f
CompactCells267To298.Box273.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:4656-4667	556068bdaf6218472995546d5c66db6c0d7b68ae9278bd5b757141403eba5345
CompactCells267To298.Box274.QInterval	CompactCells267To298.lean:4668-4670	efc61faaa2ac46de6147413296b599a8fc74cd09e96f23c062cb8c3e3eb83372
CompactCells267To298.Box274.lambdInterval	CompactCells267To298.lean:4671-4673	8c6d9619151aba6d08818b60e13aad259240105136d329a4188947439e3e7dd0
CompactCells267To298.Box274.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:5320-5331	4b7758f897978a1ec76f7acb1a565cc9087ce7d191c8932e10569394a816544e
CompactCells267To298.Box275.QInterval	CompactCells267To298.lean:5332-5334	6d8283bd2090eff210b68b79122d36e763d03cb0e35905ada41c91fc6f2ce3
CompactCells267To298.Box275.lambdInterval	CompactCells267To298.lean:5335-5337	17038b2e925788c291604774d10173f483d6274e36a3781c0065c2ee83289259
CompactCells267To298.Box275.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:5984-5995	b79bc3e5231577945702eea3fb0402de2daac421f202b8bf8a3bdce963e128d7
CompactCells267To298.Box276.QInterval	CompactCells267To298.lean:5996-5998	55a23a1f8758cd52e35bd80da99e007704fed75d08825c351aa2995c24fa74c4
CompactCells267To298.Box276.lambdInterval	CompactCells267To298.lean:5999-6001	aa4cf09ce24cde5f85274fc920451061231d32756c45e6f46d0c857bbbb38323
CompactCells267To298.Box276.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:6648-6659	47f7434c6e8d120d3d61350806144024c21d4691c8372b69bfebbf2c374eae66
CompactCells267To298.Box277.QInterval	CompactCells267To298.lean:6660-6662	9ca5f152c93a5a63ac17360955c908769ed4ccf90faa1ea2c72966845ee55404
CompactCells267To298.Box277.lambdInterval	CompactCells267To298.lean:6663-6665	aa202b663066773b50275e6016e1c001e0a67c76b7deb77f4b01cc1a21e7b98f

Declaration	Exact source	SHA-256
CompactCells267To298.Box277.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:7312-7323	e332e958ef67ecb873e3cb32480b7927ff53211cfa5feb97c9d55e6436e0976e
CompactCells267To298.Box278.QInterval	CompactCells267To298.lean:7324-7326	19e5cdba9253c8f88abb00e3bf1dcf11427a64caf49a14b3c9ace6b1d8401ac8
CompactCells267To298.Box278.lambdaInterval	CompactCells267To298.lean:7327-7329	2b4df54ca29036e6e248cadb01e5218c87b92ecc42d39303690177e6c3488e82
CompactCells267To298.Box278.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:7976-7987	abb6db2fba90dfdc1fb66939236d19a650fbcc3f88ad1bbd4d8c3dd81169246d
CompactCells267To298.Box279.QInterval	CompactCells267To298.lean:7988-7990	4886d845046d0a72e812fcd21f0d2f9e2bf6bb2251e3c43a45f8367090d84b5
CompactCells267To298.Box279.lambdaInterval	CompactCells267To298.lean:7991-7993	96cd393fe4b2a719255f80900faaf0e988bd50190e9c68c10c8d9a49daca829f
CompactCells267To298.Box279.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:8640-8651	7fed697dbdf7f7a71b7942ec1f9265f06b8990043eb6537b0b3ed2fe4d2f9efd
CompactCells267To298.Box280.QInterval	CompactCells267To298.lean:8652-8654	a7e9a15279f93102cc0da7ef664c9abf526348548476dc869a1dcf2a74c16d58
CompactCells267To298.Box280.lambdaInterval	CompactCells267To298.lean:8655-8657	78d53ad11ec05642026f38b372e2d5a0c8ff4c7bef34995b473c8f00c9393acb
CompactCells267To298.Box280.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:9304-9315	14d57910c3c248f208966b2a510d07bc778aff1fb2b0c196ff27fdc4e478c540
CompactCells267To298.Box281.QInterval	CompactCells267To298.lean:9316-9318	2e87c9fe086e51d7eaab0143097a3a0552bf8395192c535fb730c05c2e6d6c42
CompactCells267To298.Box281.lambdaInterval	CompactCells267To298.lean:9319-9321	11c79bb7c30e1fecbfc6213aa637b95786186050212df6dca0061033914871ae
CompactCells267To298.Box281.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:9968-9979	0ef84a28c2d67637e1421911f4b502d66a9865e2c17a4f9177218d59acf2f5a7
CompactCells267To298.Box282.QInterval	CompactCells267To298.lean:9980-9982	ea7fc6241256c5b3eae060243019a6f68a2eaab268737eeb89194faae3605083
CompactCells267To298.Box282.lambdaInterval	CompactCells267To298.lean:9983-9985	3d44dd72edc6deafeb8c660bfe31c1f2f00c5ce44ecdf2c716fd9e5c7d4e89cb
CompactCells267To298.Box282.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:10632-10643	b3c3fbbdb4c81f0538ea35c6ca504769c17bf6e3499f75578ebb2780d33eb060e
CompactCells267To298.Box283.QInterval	CompactCells267To298.lean:10644-10646	78d71b82a71168b6e7d6ae267787793e310979640c9920b56bb15e0ebd1de4d8
CompactCells267To298.Box283.lambdaInterval	CompactCells267To298.lean:10647-10649	658e5bf66e2fb78fedc3af3c15b44c36c64d9489c4f52d824d80ffaebda399f0
CompactCells267To298.Box283.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:11296-11307	0fb81e1c469e98951ff0965ff01b247c2bfaea9debfdcf821a84bd70da76da2b
CompactCells267To298.Box284.QInterval	CompactCells267To298.lean:11308-11310	c4060d58be7709d6f53a6508c2e83df2c429a02d13516900da43f6558261539e
CompactCells267To298.Box284.lambdaInterval	CompactCells267To298.lean:11311-11313	d44dc2cdf338f6e5febda6e9d7e494d108e5d36bbd993f09ab0498794eded2eb
CompactCells267To298.Box284.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:11960-11971	4299ef6831a32dc6c8e2617631b89724561e5d15740153b3a15eeec6c65e3a7c

Declaration	Exact source	SHA-256
CompactCells267To298.Box285.QInterval	CompactCells267To298.lean:11972-11974	e503386ea6918099fa1dec8921832a16fe6e36ab02162127d76a57787f0f26ca
CompactCells267To298.Box285.lambdInterval	CompactCells267To298.lean:11975-11977	bfcfc848b4f2bd0b7312818301d43792237f9f5d6231f6865ef64b37d36cdf35
CompactCells267To298.Box285.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:12624-12635	35020c603db4bafa3a610fcf958a9bb87850615086c914cd1d6cb47e965f48a0
CompactCells267To298.Box286.QInterval	CompactCells267To298.lean:12636-12638	162cdf5287eef36a2602636e10a00ebce5efd9fca3128076c0cfb37b25209cf5
CompactCells267To298.Box286.lambdInterval	CompactCells267To298.lean:12639-12641	d42be64d0855eb24dc77b7b34eefef1457458e1ea96f0a9f0890c9707e8a182d
CompactCells267To298.Box286.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:13288-13299	b093bf799cbebf6e42850a894cae8f3977a21651864c077d1e8f0c89445a970b
CompactCells267To298.Box287.QInterval	CompactCells267To298.lean:13300-13302	fa65e4c2a04682349e548e83572339931d996d83b98950a9453657cc74d06095
CompactCells267To298.Box287.lambdInterval	CompactCells267To298.lean:13303-13305	561c6b12bff7bff5118cb8879bfb45ac48ca04d031b1cf978456a2a0943ea8bd
CompactCells267To298.Box287.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:13952-13963	22825096914b16d968c337a561d471459684fdeeb1e41b494e6d27e2fa77d070
CompactCells267To298.Box288.QInterval	CompactCells267To298.lean:13964-13966	065ad8fb0b32bad24fa1f52af0071c82635eafb84ec7d0064e16f087e3678b87
CompactCells267To298.Box288.lambdInterval	CompactCells267To298.lean:13967-13969	c1155ae89e020b59351678347ca4405ccf83db807f099e98e5d5a2fc5e87fbd6
CompactCells267To298.Box288.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:14616-14627	e6b724dc8621b8a4698dc09420f9e0cda2433f4d94e44ca3773752e1b9d2abd6
CompactCells267To298.Box289.QInterval	CompactCells267To298.lean:14628-14630	0125107f5901e5134de0fd7513e881e203bc8ac0fbb11350dce87cb4204bdacc
CompactCells267To298.Box289.lambdInterval	CompactCells267To298.lean:14631-14633	0a5721986f028c864d14d9dade4946590c5972a5fcf83e52d3f4173f46cf4f86
CompactCells267To298.Box289.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:15280-15291	ad1c7e5f7e06558740ce22610827a575ff7c78910b2679e5c4be7fa67d5f07f8
CompactCells267To298.Box290.QInterval	CompactCells267To298.lean:15292-15294	c54bd2ea22b166eba7ce64be6a6b375b33d88d5df3f2d67e14fd5d13821583b7
CompactCells267To298.Box290.lambdInterval	CompactCells267To298.lean:15295-15297	03f13342fb2f93a84b02cf50ae1554ec61ae601b6f86843e325898b54c3b8561
CompactCells267To298.Box290.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:15944-15955	5dee263a2a32e7f718519eb202324c7a94ba79f21e71b4e37aa0b15b25ccc70a
CompactCells267To298.Box291.QInterval	CompactCells267To298.lean:15956-15958	7a67cea69bd2c9dc3c60927bd5cc66114df4f7bd2e91a6f4736a9c4cb9641008
CompactCells267To298.Box291.lambdInterval	CompactCells267To298.lean:15959-15961	ab30391d54411b079fa6a479f69ee05ad062f09357c238c7db3c5466c320564c
CompactCells267To298.Box291.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:16608-16619	ff721490c99aa53371530f972348bdffe2314af8cb09e6bde35b761b98f77bc5
CompactCells267To298.Box292.QInterval	CompactCells267To298.lean:16620-16622	d63f45ded7481248acfdadddcdc15472235ea80322586323d59327ca9f6b859e1
CompactCells267To298.Box292.lambdInterval	CompactCells267To298.lean:16623-16625	e5aece0493a49fed0d209c666c5c685db906dfcd8287ffb5827e326f85fceb12

Declaration	Exact source	SHA-256
CompactCells267To298.Box292.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:17272-17283	d5c6beeebf2655cf336ed5b098b04f27b5a4523fbedf22d28ad246a873776f04
CompactCells267To298.Box293.QInterval	CompactCells267To298.lean:17284-17286	bf644b7e43e45ffdb68415cb696a73185b3f8f9e321425be0de538da71046d51
CompactCells267To298.Box293.lambdaInterval	CompactCells267To298.lean:17287-17289	dc2ede481e715ff4320d928fc2724eb43dd2f81fcc5fb055bedd89f253a73bef
CompactCells267To298.Box293.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:17936-17947	e0496e000f31d02b3f6ef84f5c76daf625cd63a8593d6a18a8e86d6cc1d8a70a
CompactCells267To298.Box294.QInterval	CompactCells267To298.lean:17948-17950	f2f131a58b4fdb98b6bd1a997888472e6f910bc82fb06f416abe8cd9ef153d94
CompactCells267To298.Box294.lambdaInterval	CompactCells267To298.lean:17951-17953	069c5577f7bc03beaa3fbcbb1df2e5f7c8baa4789e3fa6c41a720007a0c41e31
CompactCells267To298.Box294.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:18600-18611	22d1dc5bbef7ffb92de02b8c148934a95f13fa3301c2006bd5734c627ba2d324
CompactCells267To298.Box295.QInterval	CompactCells267To298.lean:18612-18614	648ebc4fed0d481e4d1ad58a7b2b0945541a6614a521ad1ac07c22c47a34c21d
CompactCells267To298.Box295.lambdaInterval	CompactCells267To298.lean:18615-18617	54d9be9313c958e95f33b542189ebf8096128a66aba1b1cf8349329b8426db3b
CompactCells267To298.Box295.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:19264-19275	7901d02fa9e602555ecc279141406393d3a656b9e266fc36b27a3a7cda8c6dd8
CompactCells267To298.Box296.QInterval	CompactCells267To298.lean:19276-19278	d3b4d144489054a7fccda4f6386da471cb2e291c185c505a56f45d87e239e57c
CompactCells267To298.Box296.lambdaInterval	CompactCells267To298.lean:19279-19281	be0c2d3c3edd7ff5cdd967090a7714381255496049da93f6653c4aea173330d0
CompactCells267To298.Box296.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:19928-19939	44f53fab7e5ac740cbd4560326a0af4d6354c1c5865811043e61212ef31ad006
CompactCells267To298.Box297.QInterval	CompactCells267To298.lean:19940-19942	e59dca2244e94f97d297b8d9847a9479a3331a62d97fa44cce4f48b05fd4e2c2
CompactCells267To298.Box297.lambdaInterval	CompactCells267To298.lean:19943-19945	b95e5d2a2b492908ff08031a0c4b9f76afb1c40b37d09cdfdb9e5db4175d27b7
CompactCells267To298.Box297.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:20592-20603	bb9c29612aad64bdd631f54ec5a089b0e2e0eb3f51adc84056d56ed00cfe5ef4
CompactCells267To298.Box298.QInterval	CompactCells267To298.lean:20604-20606	835de753ae0ec3df0dfe83525a259b7b75178ae8c21405d4cf1e7bf0429a5fec
CompactCells267To298.Box298.lambdaInterval	CompactCells267To298.lean:20607-20609	328da4f75ff24eec9e5a6c86ce2e37c38ddf7a0406a00a8b261cef13733e19c6
CompactCells267To298.Box298.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:21256-21267	2666e6a74147ea59e15fac3c19f1ed56ccbfb827e6ed4df4d09e4330214471d5
CompactCells267To298.bboxes267To298_candidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:21268-21498	0a72a1f202f6857175a7e61464061e13c05cbabcb155f735a89c1d320184c206
CompactCells267To298.bboxes0To298_candidate_not_isWeightedPerimeterMinimizer	CompactCells267To298.lean:21499-21512	5d8fa6e5a14d1b11a879f314abdba46a8285eac2ed228c0d25f54818fb503228
CompactCells299To330.Box299.QInterval	CompactCells299To330.lean:20-22	60a01de3e0ae1d37543f957590473300f16fab594809397d957e75d797f7e43a

Declaration	Exact source	SHA-256
CompactCells299To330.Box299.lambdInterval	CompactCells299To330.lean:23-25	ce6719ad5916cefba426a14a88aaee8d758ba20c3f34cae9b260de948de3076b
CompactCells299To330.Box299.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:672-683	de6ce82646aee8cb785a56e4e5b4c0001e7c99d1839744bce87ada71dd35a263
CompactCells299To330.Box300.QInterval	CompactCells299To330.lean:684-686	093a068d9624fe1909b6105b425bc12aeb8ad703ab5e0b56a76f1e80d59d5dbe
CompactCells299To330.Box300.lambdInterval	CompactCells299To330.lean:687-689	6016250cdcab69e402878317f98fa6d9c2d3d2b4bbd1be194fb4496f74db84f6
CompactCells299To330.Box300.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:1336-1347	aeb02067e2c65ab20bd78f83c165ce69462d5844fdf0438f188b434787db5f32
CompactCells299To330.Box301.QInterval	CompactCells299To330.lean:1348-1350	e3a0976cefee777beac7f7108561ad04507e129aaf0aecd138262822d770f5d2
CompactCells299To330.Box301.lambdInterval	CompactCells299To330.lean:1351-1353	7634a0f7a2a4952e7a5afce0d6cc110bc62d661f99d0c27cfd63e99b6599ff3
CompactCells299To330.Box301.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:2000-2011	f681616d975d01fc3996c6080d58b4fb03dd68b078f291a104009d3686d2dfdd
CompactCells299To330.Box302.QInterval	CompactCells299To330.lean:2012-2014	5be1a33e76e9d0dd588cd07873570a2397da5a4c131ce435216e65f611aa671a
CompactCells299To330.Box302.lambdInterval	CompactCells299To330.lean:2015-2017	1d739300a1e88a57435f8b9c334b8e4dcdb6124996e1de939a012cabd149a917
CompactCells299To330.Box302.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:2664-2675	0914b1d1f7537f4bb6cf6d980510f8fca535edbacc6d0a5422e229d436ea0aad
CompactCells299To330.Box303.QInterval	CompactCells299To330.lean:2676-2678	1b23266c1ed0a8d11f5522e84aab68b5b1b66de54c39d45f21f88c7033c51c1d
CompactCells299To330.Box303.lambdInterval	CompactCells299To330.lean:2679-2681	0a47825fdda09e2ba866a869dc0a027b5c1850de48e8067bb7e841d66cd64932
CompactCells299To330.Box303.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:3328-3339	a741fac72fb6878ce6f474f7a86aa497e4976ee21c7c702adedeb67ae23185ea
CompactCells299To330.Box304.QInterval	CompactCells299To330.lean:3340-3342	1e73d71ccc4e64479972c1838882731373117c4f66828ceffba26527c5c2d83d
CompactCells299To330.Box304.lambdInterval	CompactCells299To330.lean:3343-3345	e88652e692eee9ffadb359b8d1b6af60598b082288e4beb8742aec8f7568a2fd
CompactCells299To330.Box304.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:3992-4003	4404c8f32f88e149b61341c2b405bbe926404fdd38c3482f995a2e71526a563c
CompactCells299To330.Box305.QInterval	CompactCells299To330.lean:4004-4006	eadd69ca00ec958a870281401075c2eb8913c02b8e0a9e558d8eb5e3b15b53ee
CompactCells299To330.Box305.lambdInterval	CompactCells299To330.lean:4007-4009	662cd8e379f32a7f523e2337d8e7b18e703c8ef3db07961296acf1679c184fea
CompactCells299To330.Box305.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:4656-4667	bd9b4ca27c373a93c29df6abc20d4ee1ef6851e745f1de49ee89437aafffee4c
CompactCells299To330.Box306.QInterval	CompactCells299To330.lean:4668-4670	a5e56432c086dd5dfcf2eb59b2a19ed8ae0427751fc484327d1bc2db5c6d9825
CompactCells299To330.Box306.lambdInterval	CompactCells299To330.lean:4671-4673	47e90d00cd9c51ed50a8e3efced5da4a6b9c54c5cc7d1573fab8734bb57e7d78

Declaration	Exact source	SHA-256
CompactCells299To330.Box306.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:5320-5331	40e641815fd963e4f4f3b6cf1f0a134362110c3da2ee0ee28d9429c7651df81b
CompactCells299To330.Box307.QInterval	CompactCells299To330.lean:5332-5334	c7920a6e19c8ffa099f9009d8b5d19c4d0188c9b5f488b1dfd7d0142cdd01e91
CompactCells299To330.Box307.lambdaInterval	CompactCells299To330.lean:5335-5337	c036a9aacd3d56e021bf7a89779c88cb42162fd9d7b0c83d2a6ee2a3a470e33c
CompactCells299To330.Box307.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:5984-5995	c5070139c2049e07b0604327a3dc991d43df55c758ec1150b95a2987fd2bc907
CompactCells299To330.Box308.QInterval	CompactCells299To330.lean:5996-5998	a2a2ac5b16f739fa099643b377d3d8ddae985e89e39eecd78eb359ec43ae36ce
CompactCells299To330.Box308.lambdaInterval	CompactCells299To330.lean:5999-6001	471ee94af67e6f6c75ab30845457002c78555e345a87682b2d900f20d4b4d2d7
CompactCells299To330.Box308.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:6648-6659	bf785e172028fecccccebd887e3c50b3c6334aee744613768579aa7cce755020f
CompactCells299To330.Box309.QInterval	CompactCells299To330.lean:6660-6662	ff7d87ddf9d6fe28fc04491d9fdf8fa5d95c50851cdaffff156e4a25c728c9c44
CompactCells299To330.Box309.lambdaInterval	CompactCells299To330.lean:6663-6665	1709e7fd764913424c382984d10577ef09b81f63da14bd9be9c44d477cd7a37c
CompactCells299To330.Box309.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:7312-7323	0623b98063c1ae22482541248ee5b3ec748e260d5c9fde476c782934bdae56d1
CompactCells299To330.Box310.QInterval	CompactCells299To330.lean:7324-7326	60e2aaf49f6f157ed5d42b56f947aa85267ac6f3f0daa52eca4f0c9ab98e7d3c
CompactCells299To330.Box310.lambdaInterval	CompactCells299To330.lean:7327-7329	7c090060de5c098c6bbadadae372696ed13cb70b72491fe2e18e0ad7b43a11b9
CompactCells299To330.Box310.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:7976-7987	e4cee0d40ff2dde6076ace420cb833a364bcbbe4709328a30b51602f95168b61
CompactCells299To330.Box311.QInterval	CompactCells299To330.lean:7988-7990	de171df2059489abe14740364d80921e4cb7153ce8c9e403a2c4e18568058fd4
CompactCells299To330.Box311.lambdaInterval	CompactCells299To330.lean:7991-7993	e6c44839c95b06decc4fdb9e28c6c7ad812cab151a417d3d65e94845ea4613df
CompactCells299To330.Box311.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:8640-8651	9a19e6ca24ec32ac93b3b15bd2c271dbc78dc0d6a7a68624f410e8eacee46bc5
CompactCells299To330.Box312.QInterval	CompactCells299To330.lean:8652-8654	79db48d54ce71d6ce1c9dfac286f20cd658a2bd4fe83a5f87c5d1d76689be6e2
CompactCells299To330.Box312.lambdaInterval	CompactCells299To330.lean:8655-8657	da081a0b30999c000ed929680355bc8f8cf229fbdaa70e15748717cf8b9b2ae6
CompactCells299To330.Box312.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:9304-9315	6ca026e9a949237dad099598116bdc2b398faa2b3a5085173e939855f3449e4a
CompactCells299To330.Box313.QInterval	CompactCells299To330.lean:9316-9318	1ea562ff35001054d45edd713f1f1b097c08208776a45ea02d405c52c38ba802
CompactCells299To330.Box313.lambdaInterval	CompactCells299To330.lean:9319-9321	d70d0f838643111559dc0eb5b8c22bc2b792c3e9f548c1bc4dbb7c8ab1e35df6
CompactCells299To330.Box313.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:9968-9979	3454b69eca246862bf6f6d38ade211f343e3005985759f2539c2afbc8b402cd4

Declaration	Exact source	SHA-256
CompactCells299To330.Box314.QInterval	CompactCells299To330.lean:9980-9982	7fbf392fe51dbd784ad2b12de24561412f38540add2d960bedf20dfbf95f7b4
CompactCells299To330.Box314.lambdInterval	CompactCells299To330.lean:9983-9985	f447ce19739b5788ee5973f13cfd72b5d938d2f4a65acbb76a35f44d614b52ee
CompactCells299To330.Box314.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:10632-10643	6771c062d2de745036ea7ca06a86465d861604819965c1b7465a1bfe4617c7e1
CompactCells299To330.Box315.QInterval	CompactCells299To330.lean:10644-10646	fa74df8f12ae343316417c25682d7a77755c11e6a9d8fe5688038f4b1e8f4bf4
CompactCells299To330.Box315.lambdInterval	CompactCells299To330.lean:10647-10649	8b469cba1866a80f2fbb1301244486a5d74cdd05377c520e2de394607c9810f8
CompactCells299To330.Box315.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:11296-11307	86b588a8899d8261831f407b5aa67197b4cf960aca6df3af75355cb73692b95d
CompactCells299To330.Box316.QInterval	CompactCells299To330.lean:11308-11310	fa4a8222124d3b36f78c7ced0bc5de8c0074e6683d5d608a676ee786d8dd8a53
CompactCells299To330.Box316.lambdInterval	CompactCells299To330.lean:11311-11313	f5b7dc739ef938ccf321f001184c634f4cabff6d015194dc046f0e27085dbd73
CompactCells299To330.Box316.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:11960-11971	196f58197df05abc657e3abc676ccac31d880b1eba6ddf4093cfbe5123399c4d
CompactCells299To330.Box317.QInterval	CompactCells299To330.lean:11972-11974	558950596f069c8b28e24c6db7ffab9960ed232148da5dc11996ee9e18e65986
CompactCells299To330.Box317.lambdInterval	CompactCells299To330.lean:11975-11977	f80e41c5ab217355f4f49899ac0074cfcfcf3a38c844b2b0d5125c7fc7c2dc87
CompactCells299To330.Box317.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:12624-12635	832f7d2b62c3c49a328f032bb1a44b1ab458ce58429d7333ecf0764532c8c2d2
CompactCells299To330.Box318.QInterval	CompactCells299To330.lean:12636-12638	b3c715d0db88b36dfe4dc5610a8692a3b529797288600f0901447533346885e3
CompactCells299To330.Box318.lambdInterval	CompactCells299To330.lean:12639-12641	74131d3d2825d63733c68e350cda3ae3db9f3c0988026a47d3e31e684a4b1dba
CompactCells299To330.Box318.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:13288-13299	7c2de8d0bc07f4d46fbadde73ca829140a01b71e5d4e17fc951e1c20e68dbd94
CompactCells299To330.Box319.QInterval	CompactCells299To330.lean:13300-13302	d492a36ef976deeaee55c0731bddae249be931ddc4e9a4affad7aff7432819ff
CompactCells299To330.Box319.lambdInterval	CompactCells299To330.lean:13303-13305	c354f77a640083e809b48578a34d852f97e0f34c2ebd0bbfdde22c10e203be2d
CompactCells299To330.Box319.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:13952-13963	13c94645ec9b78b59fb44e5282ab48e076476549b0b621ddb19d45495619e36a
CompactCells299To330.Box320.QInterval	CompactCells299To330.lean:13964-13966	26ef9d45efd456553b2bb379629583f8bc21e9a9e3d3091826e1f52aa0883aa8
CompactCells299To330.Box320.lambdInterval	CompactCells299To330.lean:13967-13969	5ed2d92228785f2ceaba2bc5ac102235c9ba186999b5f3a74c3dfffb3817682a
CompactCells299To330.Box320.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:14616-14627	22b2036285bc0fdc2b60bf044ac98601c28c4044aa76c8318033419b478b133
CompactCells299To330.Box321.QInterval	CompactCells299To330.lean:14628-14630	d599041e9d6abfd611b6e9358ebc9f0fc9e6392850eab42d49ab87cd4977ebe0
CompactCells299To330.Box321.lambdInterval	CompactCells299To330.lean:14631-14633	110bc6ce0b5eb6e28462aab842c9a08d6ea3b8c75ea24753623559bfc5f59a0e

Declaration	Exact source	SHA-256
CompactCells299To330.Box321.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:15280-15291	2cd41effad71e4e95fb4d1daaa489ff3ba8ce64972cb01af2f3daa7cb3c49ecc
CompactCells299To330.Box322.QInterval	CompactCells299To330.lean:15292-15294	ae3b7b2c25f5a09f966fd4d114b5b5d1f2bf1c2d0d73e75a4969db775757f795
CompactCells299To330.Box322.lambdaInterval	CompactCells299To330.lean:15295-15297	dc148397a42504a2d0dc9aac1a084ca67f525fa7cfb41dcdff8fcee1a396bd57
CompactCells299To330.Box322.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:15944-15955	9f1d6d334392bac53398f217c8fc2e8f88b7cfefb02e5c56390ca1d48c4afe07
CompactCells299To330.Box323.QInterval	CompactCells299To330.lean:15956-15958	8850df4faf4202b2a5d3e80da3071c19f76882839aa609c4d16bff2469967243
CompactCells299To330.Box323.lambdaInterval	CompactCells299To330.lean:15959-15961	600608e413a570cb5af3c9bd6dc353444990bfe375e05e4f64ec713b75723533
CompactCells299To330.Box323.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:16608-16619	4ae9a5f724b6921f83886af154a54b6aaf2cf920629b9057bca5491a9b435ecf
CompactCells299To330.Box324.QInterval	CompactCells299To330.lean:16620-16622	efc0d066ac40002f98a538997b8bc1945db8eb3f0f6746ca3d7071d542ba8e74
CompactCells299To330.Box324.lambdaInterval	CompactCells299To330.lean:16623-16625	ff8302adc f83db37525abe956df193c0b18807594c7978dcf41ed1a59fc6d3eb
CompactCells299To330.Box324.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:17272-17283	cbd7f1a4c2ce3ffa24fdb8c94b67d1e3d1c0193d7a2fd2b900ac019a3d9c5cf9
CompactCells299To330.Box325.QInterval	CompactCells299To330.lean:17284-17286	62ce2e8685161ad607ae852f7e2a09b659e61b5e5a08477d066864f439c70c47
CompactCells299To330.Box325.lambdaInterval	CompactCells299To330.lean:17287-17289	edca0f7d880380a044dd0b727f1350fe38e6a51a6c2e34cad180ffa80dad8949
CompactCells299To330.Box325.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:17936-17947	c744e529f1043c25c7c769b7fe62104c6d413574f2896c59513b467fb2cf3cde
CompactCells299To330.Box326.QInterval	CompactCells299To330.lean:17948-17950	8dca2876bc33abef5629905788350d1a053658625708f7bc0d43fb8a5e7d6140
CompactCells299To330.Box326.lambdaInterval	CompactCells299To330.lean:17951-17953	9ca000d6b4a7de9d5b03a3407b8d86a2b00237014f6fffc69529abd9bbbe0146f
CompactCells299To330.Box326.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:18600-18611	7a538145310daa6db31eac2887c6d68058880fb44dd634095030d2860f666607
CompactCells299To330.Box327.QInterval	CompactCells299To330.lean:18612-18614	547b56150aad2c9288589655d2ae576da44927b941c6d3faea70e2ad7e20fc0
CompactCells299To330.Box327.lambdaInterval	CompactCells299To330.lean:18615-18617	0eefd5a64a4d0b873c576bb308991fffe255f5700013fe5d350a1f14d57cccdc
CompactCells299To330.Box327.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:19264-19275	c29ac17dc70d9e4d0d3d1432ca7b2d8bbb1c0266d8d270bd6dec9614d8f25717
CompactCells299To330.Box328.QInterval	CompactCells299To330.lean:19276-19278	21a4ccf64983cbf05eb343d468041feaa1f0af519dcaa3bda03c9c558e15e8ae
CompactCells299To330.Box328.lambdaInterval	CompactCells299To330.lean:19279-19281	1c16f729f3e7dcce84ee47024d1ab9c9cc0f7f9d58d188c3c6aa3f4b0ddc8c3c
CompactCells299To330.Box328.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:19928-19939	98b5b21d14a372129416132e40f7f7b30e37c2827ad0216bb1836cb73f427e8c

Declaration	Exact source	SHA-256
CompactCells299To330.Box329.QInterval	CompactCells299To330.lean:19940-19942	48305d08f9c70a0501e45c8ca2eafc02f630dfbbdd7cbaafffa5586405d56a40e
CompactCells299To330.Box329.lambdaInterval	CompactCells299To330.lean:19943-19945	5068c37f038a010d14ba985773c05a207e9da75503c98ee6bef18063ab539a28
CompactCells299To330.Box329.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:20592-20603	c1907667be89f76fb37e8ed2d55c1a74a4818cc961944248b40400e85eef3221
CompactCells299To330.Box330.QInterval	CompactCells299To330.lean:20604-20606	02dce17e64469058e8e81936db3fceda66b8ba35c301374b001424fce5307719
CompactCells299To330.Box330.lambdaInterval	CompactCells299To330.lean:20607-20609	879222679bea52ba5a249f4fbd992b99402316247d65dca69f48307069600e7f
CompactCells299To330.Box330.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:21256-21267	1f6d37ce580ae9f79c6dcd10ab55fc703a1ee04a3e1fc576a05f0208f233a36d
CompactCells299To330.bboxes299To330_candidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:21268-21498	7e98912c86e9fba0851957a22b6bc522e999cd679c540db5946ffa8a98e3a15
CompactCells299To330.bboxes0To330_candidate_not_isWeightedPerimeterMinimizer	CompactCells299To330.lean:21499-21512	6a14b6cb363aec1a44ce2c486c92df6d405a30fcee652a23429e375da8809fac
CompactCells2To10.Box2.QInterval	CompactCells2To10.lean:20-22	9f76101b52fe8ff78f20b4093a715ed7960e954f3f42a0ad2bd3ea183779db92
CompactCells2To10.Box2.lambdaInterval	CompactCells2To10.lean:23-25	c53ed8f607b2a172720b438c38c6ed7a906d00946eda7222edbdeb4c79dd09bf
CompactCells2To10.Box2.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells2To10.lean:672-683	6422b8de2bb2f94e12194734c7b60eed8d82e51c6c7414b3c8f42ede8d08dc8a
CompactCells2To10.Box3.QInterval	CompactCells2To10.lean:684-686	cc3a62b8e72cc6cc216aa391e6dcfbcb5d8f8b54a5e4db46639b23eb0d19a79a9
CompactCells2To10.Box3.lambdaInterval	CompactCells2To10.lean:687-689	bf6e718ced211284e9c083d4b4fa9ccfe489a79410f9800ec5f685d4181bc2de
CompactCells2To10.Box3.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells2To10.lean:1336-1347	b0e96674236f38844c8a847b7abab48ab5958f89638af334f84d84ecf2f78611
CompactCells2To10.Box4.QInterval	CompactCells2To10.lean:1348-1350	9409921c87fd845077b6307abf8caabe89f7ac70027b39327283c61d00e8e024
CompactCells2To10.Box4.lambdaInterval	CompactCells2To10.lean:1351-1353	68ee44e074d072d2070d86ee58844e2a241fa9951963546cc437b4abdde96397
CompactCells2To10.Box4.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells2To10.lean:2000-2011	483d91c7efa65943915d0f5863c4cc1458986b888eb425855746571e8848c5ac
CompactCells2To10.Box5.QInterval	CompactCells2To10.lean:2012-2014	7406a9905a7abf7f64e360b837ec5db014e67f7fb717a80794c5ad9f479d6af7
CompactCells2To10.Box5.lambdaInterval	CompactCells2To10.lean:2015-2017	c15f79510699151720ba5274f43dfff6126bd2e2e732134545df3ca44e4d9f3f
CompactCells2To10.Box5.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells2To10.lean:2664-2675	e5fc5f9c9f09c1677693075b9586334123089b2625722f0bdb75e334cb70f80b
CompactCells2To10.Box6.QInterval	CompactCells2To10.lean:2676-2678	96f255a674a295dbe30690be85590e9713fc2b6d81200e6ded70a6abcfc548902
CompactCells2To10.Box6.lambdaInterval	CompactCells2To10.lean:2679-2681	05880725d08583816e515d816f39c12929ae2b3b9192fd9dba9b19a2843c44ff

Declaration	Exact source	SHA-256
CompactCells2To10.Box6.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells2To10.lean:3328-3339	dbb9357698938c492ad278c41a0baaac4bd5758a0404f7d257830c06deb13662
CompactCells2To10.Box7.QInterval	CompactCells2To10.lean:3340-3342	9b68870b5eb188684790db3c614475b9193521118f2776d515db9e9979ee35d1
CompactCells2To10.Box7.lambdaInterval	CompactCells2To10.lean:3343-3345	dfa020bf522e90c29a884cd2bf11e585f3ec6974bfb964b69370d681d7ab4797
CompactCells2To10.Box7.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells2To10.lean:3992-4003	0da3ed013f1ae59a43b34800e1cce47d435862e1e4664b83f3b3a6ad011adb9b
CompactCells2To10.Box8.QInterval	CompactCells2To10.lean:4004-4006	e343cf12296a2d502d68aea19a384fa8309a136d0061d192a6586b152d781d5a
CompactCells2To10.Box8.lambdaInterval	CompactCells2To10.lean:4007-4009	0e243e5f496ad80341e77752cb24bde29a052e28051e6ff54d9e7fbcc541b87d
CompactCells2To10.Box8.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells2To10.lean:4656-4667	213e896af43316fe86ef8076df8f55c23afcab78918b3b72c2e7fe6f8285d642
CompactCells2To10.Box9.QInterval	CompactCells2To10.lean:4668-4670	a51ed0621f47f23b08218cfce9e0f0ec72279010948538fe5cb726dcf5f956bd
CompactCells2To10.Box9.lambdaInterval	CompactCells2To10.lean:4671-4673	c3094e13b21ef653246445000d5faacec3502ab92d8144de1088bb8d9def7523
CompactCells2To10.Box9.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells2To10.lean:5320-5331	f7b7e3311f48346b700c1b7a1c3c9203b359179faa5c7bfc02c862921692d974
CompactCells2To10.Box10.QInterval	CompactCells2To10.lean:5332-5334	001d1502508992bdf3a391f08d9401dd77f79c1ba2b5c1272f89f97e653cbf22
CompactCells2To10.Box10.lambdaInterval	CompactCells2To10.lean:5335-5337	ea9e70a906c23fd72f3094dadfcb44d3165b9410a4f182b3e4cc1d7e7f02d440
CompactCells2To10.Box10.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells2To10.lean:5984-5995	be835c46a58d63f773b69c88aa4f679b954e8e6c91d8738c232d4b9cc8279ba3
CompactCells2To10.boxes0To10_candidate_not_isWeightedPerimeterMinimize r	CompactCells2To10.lean:5996-6068	aa677d90a7f590f6abedc4ef1586d8b735ddda78f9f506acb42fc8461ef9403f
CompactCells331To362.Box331.QInterval	CompactCells331To362.lean:20-22	dcd686951aafd7f107c353aa472ac41feaba4e19098807cc4724fa8f7260d3e7
CompactCells331To362.Box331.lambdaInterval	CompactCells331To362.lean:23-25	2bbecad084bec1acedeec48bc030c09244d6762d4285b692d238b46ee7b1af5b
CompactCells331To362.Box331.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:672-683	e530b00caedef3bb13b8bcc4eb00d4caab969cb9839a82a56db044900fc2c872
CompactCells331To362.Box332.QInterval	CompactCells331To362.lean:684-686	ae4a3d17f5186a742d00b65e3da84773bd070bb0e4ee873f5b903db9bd850ce
CompactCells331To362.Box332.lambdaInterval	CompactCells331To362.lean:687-689	7204df1dfd8c676a5cb380e2a008886ac119cec8f270d772ecde4c4bf68daaeb
CompactCells331To362.Box332.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:1336-1347	f5249aaa64688d03d7c14c064fbfb7211f445e8e85906105954fd94d0c528dc0
CompactCells331To362.Box333.QInterval	CompactCells331To362.lean:1348-1350	bae3f3d03e178ea0b4f7b103777cca4e77361bdb78319315639870f35a291a07
CompactCells331To362.Box333.lambdaInterval	CompactCells331To362.lean:1351-1353	9d078338e8a134870abf83886b526e2af76eb5a47296e17a68f68305d5a8f489

Declaration	Exact source	SHA-256
CompactCells331To362.Box333.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:2000-2011	e32af367b61b1e6127fcfbfdb608695956f957f4211c9072ab02a796120aec13
CompactCells331To362.Box334.QInterval	CompactCells331To362.lean:2012-2014	26fc7fa01162b96e6872b613c309e950e91a85c1117e248740291fc323bbe058
CompactCells331To362.Box334.lambdaInterval	CompactCells331To362.lean:2015-2017	39953dd5dd5166106d89e7149370365f9126ca1f8a0f0c65482d66d516a09d6c
CompactCells331To362.Box334.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:2664-2675	71c82c8ced2a8358b0954e9f7e2cc687da75e36e3137980337037de0c6e49350
CompactCells331To362.Box335.QInterval	CompactCells331To362.lean:2676-2678	341f103d9b48926621331c3f1c937d3ad4e485ca95578d42bcc8d870a13d91de
CompactCells331To362.Box335.lambdaInterval	CompactCells331To362.lean:2679-2681	daa53595d838912d582f1af16931ada55ba56867301b540fba6cce5e77739618
CompactCells331To362.Box335.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:3328-3339	9e0799cee69427e73d87cdcb1126b6ca6e37400c401a45747da059f53ba8dfdf
CompactCells331To362.Box336.QInterval	CompactCells331To362.lean:3340-3342	cd521c797f15e8d746f7fb3cbd109edfb7fee4e9f4024b83bd45102a11716c03
CompactCells331To362.Box336.lambdaInterval	CompactCells331To362.lean:3343-3345	ddc849933829a99c71bf12916b5a4d500ed368b5e36d3a23d7b9a1d47d60db4c
CompactCells331To362.Box336.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:3992-4003	5ab840cae8bc981f4c0eb709c0c7dc680bbe475e4aed9364a8ed7b6ad1925184
CompactCells331To362.Box337.QInterval	CompactCells331To362.lean:4004-4006	74ae386847f3b723bb828ef8f4e78c32cdc700b276b12ce89f5ed38ced829918
CompactCells331To362.Box337.lambdaInterval	CompactCells331To362.lean:4007-4009	58e79ac11d8c2838c22be993fbb5aac291108838aaff2dcf7d73a0884af059a4
CompactCells331To362.Box337.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:4656-4667	930d7718c997d804ff2997e9cfb18eb2a69f2e0965a4c0936f823595305e4230
CompactCells331To362.Box338.QInterval	CompactCells331To362.lean:4668-4670	6c26cfb5c56dbeff53aa1a4d02e74badf28ed4ec7949a6a08b61a7b893412561
CompactCells331To362.Box338.lambdaInterval	CompactCells331To362.lean:4671-4673	84b899c3c03952df968578cbfbf632116046a1d9f738e5486eaca2c184612ead
CompactCells331To362.Box338.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:5320-5331	59c1613a682a6d060eccb3ab5112778e315e0a11914109e8175784d5fe54e833
CompactCells331To362.Box339.QInterval	CompactCells331To362.lean:5332-5334	4204fae7058e5bba978c95299080c9315a6a5910b3d76d94153e7367a4c04395
CompactCells331To362.Box339.lambdaInterval	CompactCells331To362.lean:5335-5337	2470f1a39fdaea9c5728360504cf42a800a26e6bda765aac545a2d542d3b02f4
CompactCells331To362.Box339.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:5984-5995	1cec017be4ea3cb52900eaeafb304d1a328112b5ec5f0f6b2e57a0ed9569a484a
CompactCells331To362.Box340.QInterval	CompactCells331To362.lean:5996-5998	a8102a1b59eedcb7202462fa7a054ec3dee5792592350c40533718c7b3058ce
CompactCells331To362.Box340.lambdaInterval	CompactCells331To362.lean:5999-6001	4ccaad73c302dc522dbc5d93e53a7636209ce87bd85fa65a9b88ff6ee31ea3c8
CompactCells331To362.Box340.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:6648-6659	b0e6d54cbea40150086e961e07943871b6da2d0747f0096476c4b27786f4b58c

Declaration	Exact source	SHA-256
CompactCells331To362.Box341.QInterval	CompactCells331To362.lean:6660-6662	b78164502d0081c13458042687c90f2524f6adc17cf841a7abf64fb99ed4c4c7
CompactCells331To362.Box341.lambdaInterval	CompactCells331To362.lean:6663-6665	0be67ae5fd3c37c28cb14c884262a3fa8eeae0b10f52830c7f995bd1330cea52
CompactCells331To362.Box341.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:7312-7323	cca9207874c8e156af23975810fec9cdcfb e31b4e1ef5af64e73ab5b937e2d79
CompactCells331To362.Box342.QInterval	CompactCells331To362.lean:7324-7326	c935a7a2de43dec37d55b13ebc223735230c9745b3c738173f360b232cc8a91a
CompactCells331To362.Box342.lambdaInterval	CompactCells331To362.lean:7327-7329	c1459832a1ea01d5d8b250b4c8c5e6eb74c82c81526dc31ea9a3275d845ac15d
CompactCells331To362.Box342.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:7976-7987	cff7e150c381be6772b2f3197b84bd8558b1d2119b0f24c0dc3beb36d8e24c63
CompactCells331To362.Box343.QInterval	CompactCells331To362.lean:7988-7990	03325ef9fb26a21660b022c873979616a254f97213c6cf7d51a00d6954748d53
CompactCells331To362.Box343.lambdaInterval	CompactCells331To362.lean:7991-7993	776c752f757d4c2ca13019b0efbdc3009491c27886fb780f42346ca10caa560d
CompactCells331To362.Box343.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:8640-8651	897dbf408688a9a6eb9372396239a08e46fb9c0790e5e926e649daf2e7665bd6
CompactCells331To362.Box344.QInterval	CompactCells331To362.lean:8652-8654	6f8e26032db429958e7961a976a101343912b593b3b6bbe22812708dbee84289
CompactCells331To362.Box344.lambdaInterval	CompactCells331To362.lean:8655-8657	9eea941bf858141fe3329066cdad75962bb b057416eecd9b445e7fb2293c31d4
CompactCells331To362.Box344.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:9304-9315	d3c7d5c933f8f261825bdb58cee9cf0b593df88fd5066db60abdd46f333cb3ef
CompactCells331To362.Box345.QInterval	CompactCells331To362.lean:9316-9318	1552d75085568486f1f335e98786617614beb342d08945949fd872745e096667
CompactCells331To362.Box345.lambdaInterval	CompactCells331To362.lean:9319-9321	01292adcd1009e290eac1f833375ae605880f8372caf7e9739ee4a47532dea55
CompactCells331To362.Box345.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:9968-9979	217f18b976027157a71e427a6a50c650488afa607aff56fd936757b7c1ea9222
CompactCells331To362.Box346.QInterval	CompactCells331To362.lean:9980-9982	ccf57ac538f8ba5d65c8724a657f15b0cf90e8aa7e5848d49c043a047ca31482
CompactCells331To362.Box346.lambdaInterval	CompactCells331To362.lean:9983-9985	ac9b58f5f5c07cfc9939c488ff6b711a4578d1935f0c7ffee083c67069c4e528
CompactCells331To362.Box346.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:10632-10643	1d8e0921a226eef1b854a1f715b86aab82099b75014da98092da92c6fa2d544d
CompactCells331To362.Box347.QInterval	CompactCells331To362.lean:10644-10646	87fe0f8a624495547bb65d0f4bf6e3dc168054a47696c8b2f54c8d8c30004176
CompactCells331To362.Box347.lambdaInterval	CompactCells331To362.lean:10647-10649	39eca9c2d1ae0bfc091084650f32e892e31b3ef9549ea848c2f840cf48f3e9ad
CompactCells331To362.Box347.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:11296-11307	f114d260ea2c104c3a6f5120501f022a99233bb07be9ede0b3f12434f41ce246
CompactCells331To362.Box348.QInterval	CompactCells331To362.lean:11308-11310	6cc795ce3261bba43824830efc8f3069a459fb7aa1ea68de64cd709a05eb907a
CompactCells331To362.Box348.lambdaInterval	CompactCells331To362.lean:11311-11313	c401e595684a31d654999212ca694774836ab32f29cb8a4eeab3bbc97a186ba7

Declaration	Exact source	SHA-256
CompactCells331To362.Box348.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:11960-11971	75af9165cb335f0b247187348f65c805017c9cba92f2d3c3d9ddebc15bf7f3c4
CompactCells331To362.Box349.QInterval	CompactCells331To362.lean:11972-11974	45e63b77e72093938621d292cbaf024d5896da3bf7de8ed4d0315d2dc9099420
CompactCells331To362.Box349.lambdaInterval	CompactCells331To362.lean:11975-11977	cf23c9577a0c59b7c211572024798cb9b72d220b23107ed22700ad8c4261128d
CompactCells331To362.Box349.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:12624-12635	a0b31e5eb3bad0f95af02da925d60073a09865cee7a097d49fadb4d62d28cf3
CompactCells331To362.Box350.QInterval	CompactCells331To362.lean:12636-12638	fe9b02fce21fbd3d27c7114122bbeb05ffd3f2fedb6a6aa28ebd49470d60988f
CompactCells331To362.Box350.lambdaInterval	CompactCells331To362.lean:12639-12641	9ea3149c6b57457c52d11896ff5c3ee079beab4bcd518f3813539c0e1c69419
CompactCells331To362.Box350.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:13288-13299	6c6dafc18fa01991065d64b82360131fbb07d7bd8107864f87889cc491349b7b
CompactCells331To362.Box351.QInterval	CompactCells331To362.lean:13300-13302	0a76023843ee278e15b0e472a4321f8d927aceedbe036cc217e1775a44659f99
CompactCells331To362.Box351.lambdaInterval	CompactCells331To362.lean:13303-13305	e6c7fc040e57c23af77814e96a78878e5461c13f0844f73d1962a594e3efeb1b
CompactCells331To362.Box351.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:13952-13963	efdfe5af19776adb30180860014fd086dd4625eb5396ea4938c3c33819520228
CompactCells331To362.Box352.QInterval	CompactCells331To362.lean:13964-13966	2c2acf2542e45bb425f5a32d78bc85bbdde a042ab1497e1626d4a65873aacbf6
CompactCells331To362.Box352.lambdaInterval	CompactCells331To362.lean:13967-13969	54292194e3ccc5be06e2b9c19392f2abf2d32fcae973851bae31a2543c1b0656
CompactCells331To362.Box352.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:14616-14627	74275ed76a2f53818e650c9f74eddf5ea627c5b0599dc83b361617024d7f6b87
CompactCells331To362.Box353.QInterval	CompactCells331To362.lean:14628-14630	7ce69d63d4c20275084396a1506ec02d1884c635b3bef95a19d29836ae028756
CompactCells331To362.Box353.lambdaInterval	CompactCells331To362.lean:14631-14633	2f38ac9e124e12e39f48791789b4782d7123eb5e421d49d245719c775fd62b50
CompactCells331To362.Box353.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:15280-15291	c5de22dc0c6c0fb7ca10e320cb8dcd7387c45d149f5f0555086bfed11fbc4c35
CompactCells331To362.Box354.QInterval	CompactCells331To362.lean:15292-15294	ac92608c41795599933b5bc92820cf46c961de8fdd3469fb1027b50992b914d9
CompactCells331To362.Box354.lambdaInterval	CompactCells331To362.lean:15295-15297	0b7142d87131d0e7e48dd885a1e97dcef65c4cd69f5067fcfe98b2c3c366f278
CompactCells331To362.Box354.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:15944-15955	be250a765eee04a3ab55ab2e9ccff72cf3fe622bb1af45c153da02c7850d6e1
CompactCells331To362.Box355.QInterval	CompactCells331To362.lean:15956-15958	a2c289155b8b1e26518ccf265962850b668f074edd9b5a744e6eae801c4a3928
CompactCells331To362.Box355.lambdaInterval	CompactCells331To362.lean:15959-15961	86e53a86ea7fa538e9084bdb87af4062a2d42e8eed0fe83c51edf942bf659c1e
CompactCells331To362.Box355.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:16608-16619	42b4b76bf7de4ff44491e69c5002f7b8f73e08fea3253e66e2eb9a5507529d54

Declaration	Exact source	SHA-256
CompactCells331To362.Box356.QInterval	CompactCells331To362.lean:16620-16622	448e92c42fe82d0d81b7fff06ce02cebbabced3d68f157c162f19a2eadc5052f
CompactCells331To362.Box356.lambdaInterval	CompactCells331To362.lean:16623-16625	8c885c8b8d994bbbf93d658e2e6a5692584d4d5d07ded7405ea7d00057037ff4
CompactCells331To362.Box356.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:17272-17283	94031cd835c3f3deef1ba68ef9c854b336f8ca0f4c322047878659c8d422a0c6
CompactCells331To362.Box357.QInterval	CompactCells331To362.lean:17284-17286	b77d60d77c94bc05d2743c12523ffa3a0c9c16cbad8542803934f43fcedc4732
CompactCells331To362.Box357.lambdaInterval	CompactCells331To362.lean:17287-17289	508fa6f63234bb79f815d3755ddbd25f207ab6ae80fd05df66fa6038bc41e628
CompactCells331To362.Box357.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:17936-17947	401fb2690f668e1fe4c158ddca75c7366ad80d497965e5ea5aebc750ad492cb
CompactCells331To362.Box358.QInterval	CompactCells331To362.lean:17948-17950	4744300896da49de1add85ce3ea500a5613511e5a03913f7d04d2b135d1e3d95
CompactCells331To362.Box358.lambdaInterval	CompactCells331To362.lean:17951-17953	fe8d3aa936e634726a777cb03345b4a32f090fac6956e77e3e16965868fa46c6
CompactCells331To362.Box358.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:18600-18611	de2f02fa53b1afa8b59d9314bbf5ec76c918fa988730ea5f834dc751fdb44e53
CompactCells331To362.Box359.QInterval	CompactCells331To362.lean:18612-18614	9924deddfefe1e90c5cd0b256bc14d4dfd5ca4d98b415b7c4e0e9aca6bccd804
CompactCells331To362.Box359.lambdaInterval	CompactCells331To362.lean:18615-18617	61c91dcbbbac201b8d230eed38ce21f992ce8ec2e12d99162b766c78f72ce83
CompactCells331To362.Box359.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:19264-19275	09b66f405ce7bb606f1d8b0a19637bc04b9b03f0c64ca1d273de5cc7f31b2922
CompactCells331To362.Box360.QInterval	CompactCells331To362.lean:19276-19278	7b7aaf5992425332f6100cf04033e08e93e3f802b21cdbe07d13087e24dcc3cc
CompactCells331To362.Box360.lambdaInterval	CompactCells331To362.lean:19279-19281	4a1ca2a9e75d32522ecea6ccb1b56c68e2f88433930e9913f1eb41be4c56060f
CompactCells331To362.Box360.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:19928-19939	7cb1a6d714a60a13cb4c63a8b671c3a919c40747e6285787da9e4261f7e34e43
CompactCells331To362.Box361.QInterval	CompactCells331To362.lean:19940-19942	493687b0c553ac59ff4c7638fd6736d034421d7b61a577cd7ac0be64a83ed6c4
CompactCells331To362.Box361.lambdaInterval	CompactCells331To362.lean:19943-19945	e663afc5d4bdd885992b58d3c3f804e3a2021bac6e3e990a799ce64b7049c8f3
CompactCells331To362.Box361.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:20592-20603	8a377c80535649868a783ffa4aec3a0bd566e4c1ef7651e5a12a12eb7483e869
CompactCells331To362.Box362.QInterval	CompactCells331To362.lean:20604-20606	6fa0d777c135c8d087b1f11528c6ec103564b17a44026a834faf1ac36d663b68
CompactCells331To362.Box362.lambdaInterval	CompactCells331To362.lean:20607-20609	4b74818de65f764b4bf2a7ab320f2c3f1e10f17e347a37d0aa5f2e21274c9a06
CompactCells331To362.Box362.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:21256-21267	dc832c687c7332b25e7c7e3f26ccad38a622cfa43ca26c0027fc926495313c3
CompactCells331To362.bboxes331To362_candidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:21268-21498	a679d85532a5511eaeb40fa13aa30ed8386ca711c43a0d8cba7e12d1fdf1f8a7

Declaration	Exact source	SHA-256
CompactCells331To362.Boxes0To362_candidate_not_isWeightedPerimeterMinimizer	CompactCells331To362.lean:21499-21512	bff72333aa6bc2f95060c0f54daac3bc1e92f6ca45c9b41617ba871071eae0a
CompactCells363To394.Box363.QInterval	CompactCells363To394.lean:20-22	b7226dd73de0d66b264ff14cc6fb35349906e017b5e5afb1d2540500ad357a95
CompactCells363To394.Box363.lambaInterval	CompactCells363To394.lean:23-25	883ab4ad0b008b3cfb7da24f8c27ec97c8ab3aa48de16268d2a81122498a5722
CompactCells363To394.Box363.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:672-683	957b698b8812fe43b27550f0d46c255db47a74717fe9d296880c073aa8a94ad
CompactCells363To394.Box364.QInterval	CompactCells363To394.lean:684-686	4768147bde8447307afa04d90346380e6ae24affbf799a75995b188c9773b9db
CompactCells363To394.Box364.lambaInterval	CompactCells363To394.lean:687-689	70486ccbe374e7cf384ad5ab35b6b4f97a2f7d05d1c73e627504bf519c496942
CompactCells363To394.Box364.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:1336-1347	efab64bbc23a3b3ab49b11e324eb12d77d44294c7d1dc786ef88d34444695d7c
CompactCells363To394.Box365.QInterval	CompactCells363To394.lean:1348-1350	18dc1942d0d076d173f8a47794f075ee80fd8e50a1ce864ec06d001d99a8c761
CompactCells363To394.Box365.lambaInterval	CompactCells363To394.lean:1351-1353	50a1f6663d3102e6aa331056e8d1c959fca95f136d4781dbfc549e23477f600a
CompactCells363To394.Box365.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:2000-2011	51b7be19caeb1536cdb6023014d897f00bf985e657ce568f2c8b0b4a3af84517
CompactCells363To394.Box366.QInterval	CompactCells363To394.lean:2012-2014	0ab31ce4dfd6fc438a2d522f4a581f34ec041b61a31946f5a577adf75c5e4a81
CompactCells363To394.Box366.lambaInterval	CompactCells363To394.lean:2015-2017	98b10a6bec78fa43e18834fa1e967a3868fb3dc68592114d4f1ab94177df036e
CompactCells363To394.Box366.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:2664-2675	f8b718cb37a51ac2d1f782a047bf4ff654ea82da138f6d49d58c79766cd392ba
CompactCells363To394.Box367.QInterval	CompactCells363To394.lean:2676-2678	d251007366e6dab3afd297587099a51b22a8d505aaa331e4bd65bbc0c1510600
CompactCells363To394.Box367.lambaInterval	CompactCells363To394.lean:2679-2681	36f30dc96a63e125c50645f2736989eacb9b5d6bd91033e7432f46d58634e946
CompactCells363To394.Box367.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:3328-3339	30a428b6028f2eecba079280ad1bb0e94457dd4c05fc5e282b115b113b0dde86
CompactCells363To394.Box368.QInterval	CompactCells363To394.lean:3340-3342	b8021a7ec94679a0a256ea1cdbbf87ed0c378c49579677fca02672687b6342ed
CompactCells363To394.Box368.lambaInterval	CompactCells363To394.lean:3343-3345	6fdf39b0edcdc03a142e42f6f0f504d34cdfcfc16d2ce85c083532042e31d836
CompactCells363To394.Box368.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:3992-4003	39e14bc771d597f80685f15ac40fb4b47348146ac73b4e17325d24aad6edd4de
CompactCells363To394.Box369.QInterval	CompactCells363To394.lean:4004-4006	aa4cfa5c2655c6a0414ced55329dcc727e95d2045276f515fc71a2fc7fad48a9
CompactCells363To394.Box369.lambaInterval	CompactCells363To394.lean:4007-4009	9096b15eb07790daadda7dcb44c5b7852ed2433295c4c674f2c2c2fa09c2def
CompactCells363To394.Box369.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:4656-4667	4b1875ba56a4484c9eaab0a4c832cdd85cc98e39b41160a5c2638e125140a84b

Declaration	Exact source	SHA-256
CompactCells363To394.Box370.QInterval	CompactCells363To394.lean:4668-4670	676c3bd28873144d3c9287dc577fdec74a872a806bc03f51951d38d23835afa8
CompactCells363To394.Box370.lambdaiInterval	CompactCells363To394.lean:4671-4673	f5653f9893cfc354c9a675416c8486cd6d895297a4cce0968ef75b8dadf1f36e
CompactCells363To394.Box370.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:5320-5331	e4c6981880fac8807da307d8acd53ad6be709c5df6f548c7b1ed9373571a0e31
CompactCells363To394.Box371.QInterval	CompactCells363To394.lean:5332-5334	c20ebc5709bd0e6b0d2964d132c92ab3cf5fdceae0c17d72447b917a71d6943
CompactCells363To394.Box371.lambdaiInterval	CompactCells363To394.lean:5335-5337	76612ffcad624d71c47831509cf5c38be8d79dcbfcfc996008428344470e6a4b
CompactCells363To394.Box371.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:5984-5995	9425116f1f48722f924983451bd89e55454a5aa6e1144e8d980a5f6636556de0
CompactCells363To394.Box372.QInterval	CompactCells363To394.lean:5996-5998	9558439b18844edf2124ac0f66bb05d840d0c9d5a90cc104a4835b9141a1da15
CompactCells363To394.Box372.lambdaiInterval	CompactCells363To394.lean:5999-6001	47cd732e1afc4ed6a8ed9b268ecb262d139b0fa43730a0ec4ced8fbca2da680c
CompactCells363To394.Box372.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:6648-6659	8bdcaad5d6565a58aaa409aa65d6c6d1ed1e279a512dd56ae84f1c4987307b1e
CompactCells363To394.Box373.QInterval	CompactCells363To394.lean:6660-6662	edbce8d676c332ce9ab2cc44ddeb93c75e0957e67a5e597b37ac27dd14ffbaaa
CompactCells363To394.Box373.lambdaiInterval	CompactCells363To394.lean:6663-6665	656b780aa27f83e0e0b125abbd0bfcde7287a6b6511909ab77a7588b6a6ee8c9
CompactCells363To394.Box373.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:7312-7323	ef7485b3c7b019b5cfaac85001a9c9844e90dc158cb06d1820cc4c88d73359e9
CompactCells363To394.Box374.QInterval	CompactCells363To394.lean:7324-7326	d08198086ed733ce66d1f4d2d8a5b54ba231e1bc24e10e20988dbad4c8ab49a5
CompactCells363To394.Box374.lambdaiInterval	CompactCells363To394.lean:7327-7329	62dac308684fb127ea6ed333d0aab88325342bd13060c03831394c3aa85864a7
CompactCells363To394.Box374.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:7976-7987	75a7646955a202ef1e2cdbc9d5ec9cc107bfc00fd465cba0b0262574d3bf60df
CompactCells363To394.Box375.QInterval	CompactCells363To394.lean:7988-7990	69ed5a7455b8679687efc6f24b9dbb2932ed84a1263f8dec683139e885046102
CompactCells363To394.Box375.lambdaiInterval	CompactCells363To394.lean:7991-7993	d3cd21662c6aed792f6765a5f6a49d2b0b18c164389764291808f34adbdb9aa6
CompactCells363To394.Box375.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:8640-8651	02fa9d3cde03db0f9ad6ba9865451391086eaf600bc05ab563cc19c73031456
CompactCells363To394.Box376.QInterval	CompactCells363To394.lean:8652-8654	c80f54d6c5be78ea30692ec741dad47521ad6e2d711b93d4a2f4953bdc00596e
CompactCells363To394.Box376.lambdaiInterval	CompactCells363To394.lean:8655-8657	b512d7df3359c2be892cbe06c66878d0715f9e74eb62ad59b346386153823e2b
CompactCells363To394.Box376.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:9304-9315	f258a58d595d52ce5c004bf0d88499e0f59c23eba95e708295825feed3529a5f
CompactCells363To394.Box377.QInterval	CompactCells363To394.lean:9316-9318	b5a3b7701121b569e1d5597914ec445fe9745b2ad79e0338d638beb50933367c
CompactCells363To394.Box377.lambdaiInterval	CompactCells363To394.lean:9319-9321	423ad85bb7dfa21aaacff575cac599a10c3a66602e01ee0d0be38f60639555b

Declaration	Exact source	SHA-256
CompactCells363To394.Box377.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:9968-9979	6c89312b2851be71902e7b86489bc697413b086d054ac58a1ba5e914d7637551
CompactCells363To394.Box378.QInterval	CompactCells363To394.lean:9980-9982	fc5d87fd5ed24b68c7c06ee2dd639ac209829c348d55c59529c46ccb8b621177
CompactCells363To394.Box378.lambdaInterval	CompactCells363To394.lean:9983-9985	d6ae335f79d137f0e250020e9ae0a7115b7fc473c27f4de87aa1b75d648943a6
CompactCells363To394.Box378.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:10632-10643	b527dc1ba17dc7fcee8209d9e377ab0cbc3452b7efc331e121d61a3d1ecabb2
CompactCells363To394.Box379.QInterval	CompactCells363To394.lean:10644-10646	6b400d9a3806ebefd9f7e1b2becae17cbc936e034ffd091ba2b62a1ecce1cefb
CompactCells363To394.Box379.lambdaInterval	CompactCells363To394.lean:10647-10649	9c290c816d6f5f48337237607787e3bf3cc8df1e3b320e6c44039111ad657c7f
CompactCells363To394.Box379.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:11296-11307	ae6b2d0c09420e389f2d38b20ee2d7eb73e3bef338c4f3d460aed45442908d52
CompactCells363To394.Box380.QInterval	CompactCells363To394.lean:11308-11310	04d10d6568845d980636151a3f8ca7fc4bfec0ac41538b3680689768f44c02b0
CompactCells363To394.Box380.lambdaInterval	CompactCells363To394.lean:11311-11313	510a2c504f7fbb8b425d71cde5a0b9c97f077fd6352df1c89d6b9cf172312604
CompactCells363To394.Box380.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:11960-11971	a69d5a582fce10d2369c381b5c42d15cb910981c4d02a06da8c1ea2917ac2294
CompactCells363To394.Box381.QInterval	CompactCells363To394.lean:11972-11974	868584aa79dc91ec69a15c6b146cca1983f54a59e1c25a3ca7ac6fe836dfad8e
CompactCells363To394.Box381.lambdaInterval	CompactCells363To394.lean:11975-11977	dd5505a0aa98fa750024b2c4e66a628f49e5e4957f6e941674b39bbad8401428
CompactCells363To394.Box381.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:12624-12635	6afe6ee7e6fa909c4e5a9c119483497a89a96a85142662e4da632e307921461a
CompactCells363To394.Box382.QInterval	CompactCells363To394.lean:12636-12638	40a7084a00c6b6f30bfb026e57dfa3583815ba5d96872618ab276937cb742193
CompactCells363To394.Box382.lambdaInterval	CompactCells363To394.lean:12639-12641	6841dbfc9773fca5df89d5f8b5372380ac3b8fbb38b3c516b2ac1a6ca0c8addc
CompactCells363To394.Box382.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:13288-13299	82755ec7055aa5596c79aa8176604542928ab066cb39a45ec101f67f240acc19
CompactCells363To394.Box383.QInterval	CompactCells363To394.lean:13300-13302	a035de3f2cee2a0298a4cb47dc5222c09a191fc6338fb28713e87fa649ec475b
CompactCells363To394.Box383.lambdaInterval	CompactCells363To394.lean:13303-13305	ab99c902df77524952ce6cd6793e0caf25ec7d1ae80850717296b366e279f224
CompactCells363To394.Box383.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:13952-13963	bbbd8d6eeefc2dbc2bba984a519068881593aad2a085a8bf7266c65b1ba8d4b4
CompactCells363To394.Box384.QInterval	CompactCells363To394.lean:13964-13966	6f673219c7d38ab6ab5403b6578bc706aed46fe0ea778b2354fd9ba223deafc9
CompactCells363To394.Box384.lambdaInterval	CompactCells363To394.lean:13967-13969	500d0d181b8a24f119ce108b08654c172c110ec2bde26beb35412454a45ac88a
CompactCells363To394.Box384.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:14616-14627	e10a16884ba0ae475cf84031982835157afc3691e90efe2cf5722ccfa5472c6d

Declaration	Exact source	SHA-256
CompactCells363To394.Box385.QInterval	CompactCells363To394.lean:14628-14630	2ef21fb1b246e24a3a1daa7d8c41b2e9fc9570c9b63a0bda830ff34a90a89c93
CompactCells363To394.Box385.lambdaInterval	CompactCells363To394.lean:14631-14633	5ec6879d21f7cb9b5e7f8bed4e5447cc54f736aabffa53332f9cf1aed46c8c43
CompactCells363To394.Box385.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:15280-15291	bf0fbcb3e326cf407d727557f1d4ac2c4f06138e4332782560e68f6ca2db708d
CompactCells363To394.Box386.QInterval	CompactCells363To394.lean:15292-15294	07056a578ae196a8db197d56deb93c85623530d2bc984de1f22e3e9f06d43d11
CompactCells363To394.Box386.lambdaInterval	CompactCells363To394.lean:15295-15297	3819d708c70ebb75faacc336c6d14ae07534a179ec560eab15f53af4807032f2
CompactCells363To394.Box386.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:15944-15955	93f8cc09f0731b97486ea6a8de62e1e647845ff6b8735bab4bfb7238c8da2aa0
CompactCells363To394.Box387.QInterval	CompactCells363To394.lean:15956-15958	af65eb5fc4a449a2791fffed3dd0bc18512fc96dbb888978bb2c3142c6da7f3d
CompactCells363To394.Box387.lambdaInterval	CompactCells363To394.lean:15959-15961	af58ebdd4c390315df5f533232016b0b77980da7b3541a1ca72304b690244074
CompactCells363To394.Box387.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:16608-16619	02ee2f181a1032de485b06a3dd2a8fbe597be0612c8cc19542c6df54215407e6
CompactCells363To394.Box388.QInterval	CompactCells363To394.lean:16620-16622	d7e285041e493dcedb2d10e143171a8bf27d55be662e285d6c6299be5e2b0c45
CompactCells363To394.Box388.lambdaInterval	CompactCells363To394.lean:16623-16625	568b72cfa3f099d07215eb5cc665b1adfd8d13477066b598e5f29908fd8151
CompactCells363To394.Box388.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:17272-17283	fac447f25bcd9903ee9e6f18def65345a0709f8606bdc454b7833541baecf4b0
CompactCells363To394.Box389.QInterval	CompactCells363To394.lean:17284-17286	692aae7765fd31cf346d2890372c7ed1b45b781f8c88983e91d72301d473e1ef
CompactCells363To394.Box389.lambdaInterval	CompactCells363To394.lean:17287-17289	5ae4a1a81ae4247892bdc6e0a147aa065d11e78e2140dbc5d2076ca54776c374
CompactCells363To394.Box389.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:17936-17947	14d1c457c9b761afd747f763acacf989e0dab7458d208f346ece8d9b1b4fb7dc
CompactCells363To394.Box390.QInterval	CompactCells363To394.lean:17948-17950	717c95916ef1e64191fe2946ebcb7a49949a86c7af0e0034de0816a4043cdf02
CompactCells363To394.Box390.lambdaInterval	CompactCells363To394.lean:17951-17953	24ec9dcfba68955e9f6da9156c919fa72199d31cb47c40ef2035042720d99801
CompactCells363To394.Box390.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:18600-18611	7a25fa47668a9cded7ea0974dfba5aa424f786b177669ca22efb2a07773f6d37
CompactCells363To394.Box391.QInterval	CompactCells363To394.lean:18612-18614	f030b0d1ab92f800ba7375e937f97d2c9d918864b8d71a67365d264379fc5e33
CompactCells363To394.Box391.lambdaInterval	CompactCells363To394.lean:18615-18617	bf473bce6ac972acc7cb9088d92b491f855072c7a09f7603f53417b171694d27
CompactCells363To394.Box391.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:19264-19275	0ad94d0cf6255c408da1cdfcd38a7444a5827f27443b02e7f9aedbbcd0b748fa
CompactCells363To394.Box392.QInterval	CompactCells363To394.lean:19276-19278	5bda4def7352bffda088d1b357afc87cc66c2c691260bb755cc6380352500515
CompactCells363To394.Box392.lambdaInterval	CompactCells363To394.lean:19279-19281	5c6986486289f9f6b9702a68c7f158715f206a6b07f078eb698148477d1df987

Declaration	Exact source	SHA-256
CompactCells363To394.Box392.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:19928-19939	1c2f0098c42f5b9f2c891997afbfa38170c81e8c32ed53a713cf6738e8a38c12
CompactCells363To394.Box393.QInterval	CompactCells363To394.lean:19940-19942	bb1d7c351986f377e18431b2bfe5eecb64a04ebd86830a3abc3e293bd5a5f23c
CompactCells363To394.Box393.lambdaInterval	CompactCells363To394.lean:19943-19945	6012891d468f94280251df363509c3e620c3b8c65fd4bb55faa0009aeed757e6
CompactCells363To394.Box393.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:20592-20603	cc06db6a8777b5bd56272cd991c372fbbda94dff8364abf1393efecc11dd2c4
CompactCells363To394.Box394.QInterval	CompactCells363To394.lean:20604-20606	9b7248c928ab8e06ae31b0b0d42d1d39e7d32d0e6402134ee14ca02049cce957
CompactCells363To394.Box394.lambdaInterval	CompactCells363To394.lean:20607-20609	0f9a5b6964e35a0bd90ae503d81457a6595d0d810ee0563faa7fb68b74e91df7
CompactCells363To394.Box394.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:21256-21267	abefc3987ea094dda8352a3838aced3c7653e88c831ba9c6292c9540fc82acdd
CompactCells363To394.bboxes363To394_candidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:21268-21498	0fa450abdd7678b5a8610e464a7b15b0e82e6eb86c06d02f425f47fbdde54dc9
CompactCells363To394.bboxes0To394_candidate_not_isWeightedPerimeterMinimizer	CompactCells363To394.lean:21499-21512	294384e6ee34e0c65f14626c29e1b34677a087948e642ad024bf21993373c5ff
CompactCells395To426.Box395.QInterval	CompactCells395To426.lean:20-22	a07269d89ccbc06a8ab4e544b848f43cce5bbf0ad4bcf7800a65737592835bc7
CompactCells395To426.Box395.lambdaInterval	CompactCells395To426.lean:23-25	85d6da50494bfff10b74464750c03d2b51887f0d5eea78aef34e68525845d0
CompactCells395To426.Box395.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:672-683	e4cf85d9af9a72be9abf64846566a6a8c174f19b0b375a773f8f1018c0144f4e
CompactCells395To426.Box396.QInterval	CompactCells395To426.lean:684-686	0c89fd06d9c74aacf54bc4af0fc1ea2e6be6d2dbed8f56e21c7e7cc7a974a49e
CompactCells395To426.Box396.lambdaInterval	CompactCells395To426.lean:687-689	cfcc806918e8e4c6856601eb5b7ae832f31d7dd8b50997fd239773c689664f4f
CompactCells395To426.Box396.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:1336-1347	435f66d3ad22af20a26819b9c5e81ddb221fe4eb365eb651b41c66fdea3eea69
CompactCells395To426.Box397.QInterval	CompactCells395To426.lean:1348-1350	833a432822de64c72459fd85336a6344c48d6ebab024fe2d25042c99a0e68530
CompactCells395To426.Box397.lambdaInterval	CompactCells395To426.lean:1351-1353	74b1a6f9cedf57ad2c305a8c8e590e99877e87f21c118c608c9a1e6578f42ae3
CompactCells395To426.Box397.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:2000-2011	364260882799b1b279ca1aa98a3dff9ec088c0dbd12c80c41d56356332c8dfc7
CompactCells395To426.Box398.QInterval	CompactCells395To426.lean:2012-2014	d8d708bb139753e571f00e71bf9785aff9d1fb64eee4e53d9edd81c0dd04d4e8
CompactCells395To426.Box398.lambdaInterval	CompactCells395To426.lean:2015-2017	fc59c1975add3c13656949f03ebce560df1a2d2b870170e74545ac5dc4238bef
CompactCells395To426.Box398.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:2664-2675	bd33c189dd3087e0e727384510a1b5ffbddbf8467bdf376bc290f7c21ba5651c
CompactCells395To426.Box399.QInterval	CompactCells395To426.lean:2676-2678	b583d1e271c441b743bf104535be43f49bb8806597c2c8475e692fd6410b72b6

Declaration	Exact source	SHA-256
CompactCells395To426.Box399.lambdaiInterval	CompactCells395To426.lean:2679-2681	a5f5708abd11faa79d3f88c2e202c11bfe1db3a6aaa1b343a478349e455c4cc9
CompactCells395To426.Box399.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:3328-3339	bd519cd934131912563b4449a70b481b5cfc5e6072ff8111eb4c9b6c15bb4d5
CompactCells395To426.Box400.QInterval	CompactCells395To426.lean:3340-3342	26b2bdf3b6713685dc70941fa01549da45d8c8ca9e9cbc895d2b3d7f1ed97fc7
CompactCells395To426.Box400.lambdaiInterval	CompactCells395To426.lean:3343-3345	4e1967ef6f737040be102e6c4c50517e77303939da92f1615596ad271818e0e1
CompactCells395To426.Box400.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:3992-4003	a81b23042eff76b0543941dca89ac0970d84c878fb7fbf1f80abea839e4e3c23
CompactCells395To426.Box401.QInterval	CompactCells395To426.lean:4004-4006	49deb1ac50290e3ceef1ce6795cea481f0b6fdd4432a35ff9c1162224f957c4f
CompactCells395To426.Box401.lambdaiInterval	CompactCells395To426.lean:4007-4009	b1c3eab5be677a9a4c6d14d0f673b5141b8826cba24605849ebabbdf8c75cd2e
CompactCells395To426.Box401.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:4656-4667	a82f3ee3816cfed113de94cc7004d9dc5569d379c63538c143d75c0dd121a0b5
CompactCells395To426.Box402.QInterval	CompactCells395To426.lean:4668-4670	09dd10f843ac72962ed73585318eeb5012cc09ea4541dbd137b9d0bbca9bd5f3
CompactCells395To426.Box402.lambdaiInterval	CompactCells395To426.lean:4671-4673	a110988d10fb1fd7958262ed20d5be17ace8b84493cbc5d16089ece25258269
CompactCells395To426.Box402.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:5320-5331	b14ddbe3fd8b5ab22ff30bb01e2dbecfce8d0d08e6280e4257afed6753f144ea
CompactCells395To426.Box403.QInterval	CompactCells395To426.lean:5332-5334	aca4f2ec1bc3784718d25ed4e45a96a7d0765abf49e9d0d509fc62b22f78d3cc
CompactCells395To426.Box403.lambdaiInterval	CompactCells395To426.lean:5335-5337	a3a8fd8e5c171915cabf6004e26e0ecbcce2d328999b2884dd3ce5f284aebbd0
CompactCells395To426.Box403.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:5984-5995	0f546ce5d2b38d9590b13a57deb03237f2a92347e587fefade974da47c0f29a2
CompactCells395To426.Box404.QInterval	CompactCells395To426.lean:5996-5998	bc212241ff36711cf32445e761a8f41da9c8b0017b32972bc0f9aab715637e6e
CompactCells395To426.Box404.lambdaiInterval	CompactCells395To426.lean:5999-6001	3c7c8bca35858f6ac3a4598cf5d059fd709f0815cf128cac2297b6889e0e037a
CompactCells395To426.Box404.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:6648-6659	6b14e125ca4ce1c2d0130e40003318484c10e35423cc9601a71f44406ab8110d
CompactCells395To426.Box405.QInterval	CompactCells395To426.lean:6660-6662	ae70945355d84943dce73a27b803b0bd1fe8312d7ffb72763c7ad6a9d26d4722
CompactCells395To426.Box405.lambdaiInterval	CompactCells395To426.lean:6663-6665	311307310acb2069c3c6006242a662a4ec347baa07619555662a727b8d32b046
CompactCells395To426.Box405.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:7312-7323	8ec0842240b2de90532ab90001e8f707c5d5694e6f00c35592ceb8aa444d70cb
CompactCells395To426.Box406.QInterval	CompactCells395To426.lean:7324-7326	27c26acd944d37400cd794d6e543fca44c851b746f19a261919ac0ca3d288bba
CompactCells395To426.Box406.lambdaiInterval	CompactCells395To426.lean:7327-7329	25cca2a9468ed00ccfdbd9cb27c5fa0fb2ab23a9a5fb5ababc9714e58263a076

Declaration	Exact source	SHA-256
CompactCells395To426.Box406.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:7976-7987	bc20cd598716752f718bd4c5fb3856ca47f6796078d0e5dce8838fef81cd4aa7
CompactCells395To426.Box407.QInterval	CompactCells395To426.lean:7988-7990	790f07753642c72ab2be270f3e546c12d2a972af9fb44783fad5614677014c1a
CompactCells395To426.Box407.lambdaInterval	CompactCells395To426.lean:7991-7993	b8b8ab3318da9679cc5f8d4bc94c73234cf59a13204028fbc8970b197ee5ab1
CompactCells395To426.Box407.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:8640-8651	1be58a004b3a602d5ce7b10871dcb9fa7f5a92d0e41f26b3d88966c1b764584c
CompactCells395To426.Box408.QInterval	CompactCells395To426.lean:8652-8654	1e516c4b9644b8c754c8a43dd2237357701b2fdb04f3b96209f0ae4581c82067
CompactCells395To426.Box408.lambdaInterval	CompactCells395To426.lean:8655-8657	1353ac525d7cc7517a375e03dcd3599326023ed63072617eec29de8d7a944b98
CompactCells395To426.Box408.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:9304-9315	e93e45f866baa3bdb8586b36832f582382bb900508c61b97d2f7151adb69f5b
CompactCells395To426.Box409.QInterval	CompactCells395To426.lean:9316-9318	779e974d0c79f03d6bc9b324c56124251b1513157105eccad80b22a930631f9
CompactCells395To426.Box409.lambdaInterval	CompactCells395To426.lean:9319-9321	190a12f3126dcce561d8b0c740db80801446e18f44c225e5343b133cefeef4e4
CompactCells395To426.Box409.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:9968-9979	9dc958982a4d48eb69adc819dfd49f2c7c3962576ab4136a44fff07b9f5b322e
CompactCells395To426.Box410.QInterval	CompactCells395To426.lean:9980-9982	0c6aad63a19d621dfdfaacc9ce4e345890bc26960e39839356a10b62a46554ad
CompactCells395To426.Box410.lambdaInterval	CompactCells395To426.lean:9983-9985	074bd2bbc46f0525d876f329522b6b7eb0bfe800b75754fba7f5250ca98a3673
CompactCells395To426.Box410.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:10632-10643	2131af0b8dcf50df90bb191d382e45037858254c6471feccd8baf7be7b0af490
CompactCells395To426.Box411.QInterval	CompactCells395To426.lean:10644-10646	348a4fe534af1394f9a14e56434b01609377384f441ca03bd869a7f31e9a019b
CompactCells395To426.Box411.lambdaInterval	CompactCells395To426.lean:10647-10649	de30d8ba556b053cd4b5b1f888b15c4ecac44574c6faae0ac09fb064d6ed4a9b
CompactCells395To426.Box411.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:11296-11307	77f91a70aa00a9c9844edebdd7af09f274291b408877c948c58a325cbbe42dec
CompactCells395To426.Box412.QInterval	CompactCells395To426.lean:11308-11310	97234dd7246d2729d381c0fa61407ba5fb3f14703d78b5975300492f95d80f9a
CompactCells395To426.Box412.lambdaInterval	CompactCells395To426.lean:11311-11313	c3424ee2e15e2295298387941571843f65164133e36d093ed37781e6bec52c60
CompactCells395To426.Box412.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:11960-11971	7ad4e21cb7f5aed0b5a60d7f3df7e6f05cd8e84444d01dbcc4e8cb3516ccdc53
CompactCells395To426.Box413.QInterval	CompactCells395To426.lean:11972-11974	5755336573eb2b9ccf0100e6b85529f69295b6ead54a242749e89b0e131b6264
CompactCells395To426.Box413.lambdaInterval	CompactCells395To426.lean:11975-11977	72f970e95a87fe92d0553d0aded1b5feefd006001992d82e06faba2606d17c4b
CompactCells395To426.Box413.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:12624-12635	9eb576b8114f84a548c1e2c4b6a7247e5d55ff18a7df028a4baf5de3be44a431

Declaration	Exact source	SHA-256
CompactCells395To426.Box414.QInterval	CompactCells395To426.lean:12636-12638	c80fffff751783cdbcc307011169df236e7cb1c6fb27a479edda689b0169737e
CompactCells395To426.Box414.lambdInterval	CompactCells395To426.lean:12639-12641	233ab33b16d00375289c31549ef39afb779acbd20e15e8f60f6cdb2298e0c8db
CompactCells395To426.Box414.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:13288-13299	6269108dcd6cc97be34bce5697f6c7711f34f36a471107b3948a4d49a4a33e4b
CompactCells395To426.Box415.QInterval	CompactCells395To426.lean:13300-13302	8dd14d33f18720f1133f6f176785d78cffc2b5b91b463dc4ca78692ddf3bd829
CompactCells395To426.Box415.lambdInterval	CompactCells395To426.lean:13303-13305	832eb00b35573156e89b9ca0566088a2636d3e760b687e37160226f435161531
CompactCells395To426.Box415.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:13952-13963	8b652a56b6c9eb10aab98bd5e4c8d5cbbb2d7c6341225bf5577b2f591cf13aa4
CompactCells395To426.Box416.QInterval	CompactCells395To426.lean:13964-13966	8ea9275220a858f1d2cba9fec55aeb8e46917adc836f0b4718b9b16a9512d3d8
CompactCells395To426.Box416.lambdInterval	CompactCells395To426.lean:13967-13969	33c38fa9c2725aad2a0db5140a3910e5b30a87006abec133ab648c75ce087e26
CompactCells395To426.Box416.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:14616-14627	d476e4f976d93f2b190410a90e325c2c29b248ab4013b63fe1eb43c2bba53c16
CompactCells395To426.Box417.QInterval	CompactCells395To426.lean:14628-14630	d08e07924cb3fe63c9b626e808eb004fee83e3f00f0cedcc97274d8dc0b81516
CompactCells395To426.Box417.lambdInterval	CompactCells395To426.lean:14631-14633	7c3b65f899ece19da04e1432a7b22c267b0f3bc727b498194b1987fee7c5fca7
CompactCells395To426.Box417.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:15280-15291	11742fd6cff0bf9629ededc6a75c32df9bd694342d1a2d0a354ad034f706c980
CompactCells395To426.Box418.QInterval	CompactCells395To426.lean:15292-15294	91891a9255fbdffc2ad26289c261422781b959fd627fb9d663346f2a857031fcd
CompactCells395To426.Box418.lambdInterval	CompactCells395To426.lean:15295-15297	5f57414ee580781597f582886a31f1e2c792c965ec81b82ad5ad493ea592ff2e
CompactCells395To426.Box418.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:15944-15955	6f72181f3b2fe905015b03a177f8764f2f0c332b4345e8e85db06bc374b7269f
CompactCells395To426.Box419.QInterval	CompactCells395To426.lean:15956-15958	2784f59fa5df2198a021d67d794bfbbcc3f2b516576a66cb1f1a082c991defc0
CompactCells395To426.Box419.lambdInterval	CompactCells395To426.lean:15959-15961	d73a759562b86345f1f1eedb6be6cdfcc189a1c0534c1bdd5df0bd29295d0914
CompactCells395To426.Box419.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:16608-16619	714608c5f87663c5d98006185635fdcd293c225be9f77b998331c82f5b2b61b6
CompactCells395To426.Box420.QInterval	CompactCells395To426.lean:16620-16622	570c0dd9009a2175e4a5c4eb7717b050a5ded628aedd2af64c8b684dfe977676
CompactCells395To426.Box420.lambdInterval	CompactCells395To426.lean:16623-16625	0155516ec33c21d4fca7e8cc09ac5eadeec15ce657921e4f207a2ab5713da0d8
CompactCells395To426.Box420.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:17272-17283	e8b0e9a09596a2ef34e7e2eb59d4e781bbe676b1a9632aa72ce1eac18bf8f345
CompactCells395To426.Box421.QInterval	CompactCells395To426.lean:17284-17286	91958e6b45f6f816e17f7e22247d8ba168717fd629ea5d1a1766bf2e6d3fcbb5
CompactCells395To426.Box421.lambdInterval	CompactCells395To426.lean:17287-17289	6ad91299211379276c2a72a3d7fc63b247f5b0466a2f1b1f4521b0b025af3b60

Declaration	Exact source	SHA-256
CompactCells395To426.Box421.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:17936-17947	3e6ec76c53ff767fc1ff99ca0c8170a86bd312f24697076e02a81f15e9219eee
CompactCells395To426.Box422.QInterval	CompactCells395To426.lean:17948-17950	2e98bc82c95bdd4c03882eac25dc14fdb2422a9985b97eff16d0231d12cebebc
CompactCells395To426.Box422.lambdaInterval	CompactCells395To426.lean:17951-17953	8210296a52c8589510f6c9f6156bbbc12b1629f5dcd68e9df91db58d3cad309b
CompactCells395To426.Box422.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:18600-18611	70f6cc54f9b06cd1cba9c4cd7adadb6f52dcb10c411f08d3142a3edfa18a0615
CompactCells395To426.Box423.QInterval	CompactCells395To426.lean:18612-18614	8560443e78bd86c885f9a5d77987ebd9cc86039c5719fb87d316b53ec4716025
CompactCells395To426.Box423.lambdaInterval	CompactCells395To426.lean:18615-18617	401ed80f1a6066b0543ca8ff1cb3dee4649d9738446aa5d8bf96c37c534423d9
CompactCells395To426.Box423.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:19264-19275	2941bf86af3abdc20552b4c9a8ba95d5aedfaea2de65c8c0cbd65c7cc22ecfc3
CompactCells395To426.Box424.QInterval	CompactCells395To426.lean:19276-19278	1b4109758033e3edb15b7951040486563947c6b068f8d1bd28f7ae70df14e1f5
CompactCells395To426.Box424.lambdaInterval	CompactCells395To426.lean:19279-19281	4616dda428cfbc6c9f0406627c28c65d28b87049b3d8effbfa99e9ee6ad463d
CompactCells395To426.Box424.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:19928-19939	eddc62afadc076da0add5436816e17487a30274239809206263a33aa85fc7f0a
CompactCells395To426.Box425.QInterval	CompactCells395To426.lean:19940-19942	0cdd4c938f7cfff53ae11a92a0c9d8a6c929828b239ce8af8fb692e13d7fcff86
CompactCells395To426.Box425.lambdaInterval	CompactCells395To426.lean:19943-19945	31096686c6961e1bfe04201a95c76518561d70d522b66596b367acda81863370
CompactCells395To426.Box425.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:20592-20603	4e4b541e2cc84962e2c99e46abc086ede99fcbec0357baff7edd3c68e743be58
CompactCells395To426.Box426.QInterval	CompactCells395To426.lean:20604-20606	b865b07abe4f1ec83cbf719933dae6af3f5550d80a604a308f0a684240f60c7e
CompactCells395To426.Box426.lambdaInterval	CompactCells395To426.lean:20607-20609	d8d991dc2b9d79e7df5891fd526db28894f7c60d9c798dfada16a77efc7d8212
CompactCells395To426.Box426.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:21256-21267	6ba7c0eaf9512f92514d313c0a9b16b4cafb9a058b3019d6f7bc68447c416ecc
CompactCells395To426.bboxes395To426_candidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:21268-21498	13bb594ee3c9c8e40e879a62e7361315bc33319360f7b2001d62b02a93f644f3
CompactCells395To426.bboxes0To426_candidate_not_isWeightedPerimeterMinimizer	CompactCells395To426.lean:21499-21512	5de91fccaf4a735282810a47d096ff485f65d3c560f267891c93be715b4f98cf
CompactCells427To458.Box427.QInterval	CompactCells427To458.lean:20-22	0e9df8ef2b7d1a5c5d20b73c2455c1fbdfd5196e664401f42331dd7b95f395b6
CompactCells427To458.Box427.lambdaInterval	CompactCells427To458.lean:23-25	8b4ee7994f7bf1a4e705f6b2b3993d368a2bd844883c85ad9447428ab1aa4739
CompactCells427To458.Box427.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:672-683	bb116500c4da23be7d02c7a61784002be42000db10e11ca53b91790f89c90fc2
CompactCells427To458.Box428.QInterval	CompactCells427To458.lean:684-686	4f716ac1bb8ce46a42520187991bc106ccf23efcaf0eda843ecd6beadcdd7141

Declaration	Exact source	SHA-256
CompactCells427To458.Box428.lambdaInterval	CompactCells427To458.lean:687-689	79bf934830606759fcd44b3451e35bcb5ac3c16a9b8e2a2e49190442fe99c972
CompactCells427To458.Box428.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:1336-1347	7edb4e8f28fa33fd7f72eba789663dd53b137c2c8459b5ced0ad2d89edb68f3b
CompactCells427To458.Box429.QInterval	CompactCells427To458.lean:1348-1350	9448df56ad8c7013fc379034bbc78f04dc5a12b6dd408a7545effd253cd2b43f
CompactCells427To458.Box429.lambdaInterval	CompactCells427To458.lean:1351-1353	caf9f31f57c6cea1a492911badb658c58c02127562828408df42f567c7eb016d
CompactCells427To458.Box429.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:2000-2011	dc9d2ca52756f73c671920eacbf418ef31201995bacdf68cf2d1458834daa7cd
CompactCells427To458.Box430.QInterval	CompactCells427To458.lean:2012-2014	710dc01ca031c594e0ef2b68b94d540b99d0783b69fefefe7357eddbc18a444731
CompactCells427To458.Box430.lambdaInterval	CompactCells427To458.lean:2015-2017	365caf48f0ef4d77d0f9bad7a30e4161145b137ef40c25cf3aff5e6418fac42
CompactCells427To458.Box430.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:2664-2675	60881ba86aaf6391f3bb94b1c22251081417692e3a10770dba9ac471ab8b4fe6
CompactCells427To458.Box431.QInterval	CompactCells427To458.lean:2676-2678	5f772c15ec9c435b92c0bfd8b50025861edb80e46d392fc80859655ea5b86ca
CompactCells427To458.Box431.lambdaInterval	CompactCells427To458.lean:2679-2681	dbc445ca317000f55d89ff1a3b57173166a89a44b7c2e336a85fe12e448e3443
CompactCells427To458.Box431.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:3328-3339	479f3da88b52a6a2aac2ed5915ebe7114957893b5cfd2d95192536f0b5875e55
CompactCells427To458.Box432.QInterval	CompactCells427To458.lean:3340-3342	a26413da62df34429d04aea8f4fd694716748307eadef9e5b051fa5946554c8b
CompactCells427To458.Box432.lambdaInterval	CompactCells427To458.lean:3343-3345	18eda012bbbf2ef7a465bbfbc67c121e5ed6b1d22990cb50ab7b571e14feceb2bc
CompactCells427To458.Box432.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:3992-4003	3a5eea04393f2450013c37daa18cb9c2044638aff9aa9b02f71cab7ad5c9446e
CompactCells427To458.Box433.QInterval	CompactCells427To458.lean:4004-4006	6d6e51d98ac9e9d6bc20fd23c9a9491775f620d6e7c31e567fc5b8fca2cb58ad
CompactCells427To458.Box433.lambdaInterval	CompactCells427To458.lean:4007-4009	a9cc061a078e293969bc9f972b3c31eff2db7194526c25da06126f0acf8855c2
CompactCells427To458.Box433.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:4656-4667	ba3dbf59026ae186e7f81f1408e309dddab71a68cbe379cd1a3ee053f9bce2a7
CompactCells427To458.Box434.QInterval	CompactCells427To458.lean:4668-4670	2d6defe00bfe232a9ff22eb23924d8937a4a049a6ba72fd11df76aa9a1728e4f
CompactCells427To458.Box434.lambdaInterval	CompactCells427To458.lean:4671-4673	d7014dda0955ce04ce4ee9326d0392306002b9fd7352e212473dd4f78eb5e393
CompactCells427To458.Box434.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:5320-5331	94507cb96c4a312676d3e1581649106d953d58fda852773a22c417933aec5157
CompactCells427To458.Box435.QInterval	CompactCells427To458.lean:5332-5334	638464a9bc9421e5b4faf29dd7923736f0691a9d0ff9cd0a9af9fc8cd7af8383
CompactCells427To458.Box435.lambdaInterval	CompactCells427To458.lean:5335-5337	e18267b330abcd7bdeb6323ff401dfc8512b38e3f5ed80d417a774354acfbb27

Declaration	Exact source	SHA-256
CompactCells427To458.Box435.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:5984-5995	4059fc14676b5130128eb3d893b59ce5a040f142883e131794457e6667ec90a9
CompactCells427To458.Box436.QInterval	CompactCells427To458.lean:5996-5998	af0c47461962aa7b44eb2dac337e0fb7dda cca8ea8cc910ebd3f490715354a5f
CompactCells427To458.Box436.lambdaInterval	CompactCells427To458.lean:5999-6001	887f116158fae5abb0dc007641030233d3d54d5e5eef39129aea2dddc18440
CompactCells427To458.Box436.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:6648-6659	92d18db1d15b49d5154953758f849ab8c6d18851be3549980e5adf3b786e27ae
CompactCells427To458.Box437.QInterval	CompactCells427To458.lean:6660-6662	70918866a2dae0c81d6a98f9364c35294b5757381f06b64c3eb3674e330c587e
CompactCells427To458.Box437.lambdaInterval	CompactCells427To458.lean:6663-6665	398ed393093b873fe2f94b3ca51e49dd7e5064993da2bbcd33d1c16af1b11725
CompactCells427To458.Box437.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:7312-7323	280368221eb83d09085f6c01fe4755692a13bf40a6bc7ad0e1b5e5e6e52040c1
CompactCells427To458.Box438.QInterval	CompactCells427To458.lean:7324-7326	b92ef4555e796cf37abe62d0c50ff19825b77c94f25753e0a259d70c00d8d233
CompactCells427To458.Box438.lambdaInterval	CompactCells427To458.lean:7327-7329	8af46ee39be29a55af5cd7d542252e5a2cfe748d3713a7e3e3c4879f4cd7707d
CompactCells427To458.Box438.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:7976-7987	83243af559d2af9a29596a4c fec7b241cb2e6702db4fc11436d25623e6c8ef7f
CompactCells427To458.Box439.QInterval	CompactCells427To458.lean:7988-7990	e356e5b74d51d2f4b55dcffa9dbce138931276cc28bd9f5c67db704146021a20
CompactCells427To458.Box439.lambdaInterval	CompactCells427To458.lean:7991-7993	9052faaa15f39815b7cae066e6352ecd5108496b2fed30435246e8f76563f869
CompactCells427To458.Box439.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:8640-8651	4e492c23481cebedfb776f73496b881e156a2a228eef8f5594448c1c13b2ec89
CompactCells427To458.Box440.QInterval	CompactCells427To458.lean:8652-8654	b2518b467896a9d7ddb4770152c44adc40f748444f18bba607a23ade680111a3
CompactCells427To458.Box440.lambdaInterval	CompactCells427To458.lean:8655-8657	4626770ee2f112c38fb7847bc207dfbeb806c4dadd7c9897773cab9439df83bb
CompactCells427To458.Box440.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:9304-9315	af64ac6996c4ce38d5c8ab69e8b464f9175037bad243a50ff3732ce1f9b304e4
CompactCells427To458.Box441.QInterval	CompactCells427To458.lean:9316-9318	63e70f36efd591250c0be03135bd06a62ed88d3413caa25f5e9e4fce5e529694
CompactCells427To458.Box441.lambdaInterval	CompactCells427To458.lean:9319-9321	eff9aee0b657b0e6a090e7b498a85b0abc86344a0f6d86ec50d275ed00058db
CompactCells427To458.Box441.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:9968-9979	25ad56acab767eee7dbff902f54a0ed6daa96dd6eaaba39b02adc478680ec477
CompactCells427To458.Box442.QInterval	CompactCells427To458.lean:9980-9982	974155c2417053da71a40cb6c2032295d53b6cd41186f3d5b2b6c6c69a452041
CompactCells427To458.Box442.lambdaInterval	CompactCells427To458.lean:9983-9985	b4cc3496f38b9316403de5200e0bd89b6e69f3d23f8b4cb33b6dc10b297764f2
CompactCells427To458.Box442.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:10632-10643	ef4b15cb54c2233aac5f3f0ceb8ee639bb47b829b58578c4d983ec8b93c5b699

Declaration	Exact source	SHA-256
CompactCells427To458.Box443.QInterval	CompactCells427To458.lean:10644-10646	7f3e3ee47b0d28b83778acb9e505d505b8f5b6e1229143dcb05b857b19e5f0dd
CompactCells427To458.Box443.lambdInterval	CompactCells427To458.lean:10647-10649	700f06d46b06461ffc1aa1476a1d41b64248236fd4511a1efd08483d99f9f5a8
CompactCells427To458.Box443.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:11296-11307	5185c9ff7b04a21acabdcdb0441376f43a69d2e89920dc6e1a66b599a6a50255a
CompactCells427To458.Box444.QInterval	CompactCells427To458.lean:11308-11310	a993f3aeca34e3ab69fa239b0ae1b754d5020a3fbe9475c2c8f7540332956a37
CompactCells427To458.Box444.lambdInterval	CompactCells427To458.lean:11311-11313	b9a7961fa8a4b943379dc9338ca8c526c8182c7d175a93b475e08211cbdeda7f
CompactCells427To458.Box444.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:11960-11971	dee48ee6adf2d06f4fdc12a56f0e2dfc8a50797400b4d52dbad319568a807d5f
CompactCells427To458.Box445.QInterval	CompactCells427To458.lean:11972-11974	407c6ba4f34c30cea10ce0955f8266c118df225b65f2c5f3e0a07f1f72a47f29
CompactCells427To458.Box445.lambdInterval	CompactCells427To458.lean:11975-11977	3f968474f948b31812d70b9738a6f26bcfcfe0d52f10f7872f4ead8edc3271e9
CompactCells427To458.Box445.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:12624-12635	a5c944c8f102211211a8b689f35ac9b8f269d050e10efb30df396d7c6244e142
CompactCells427To458.Box446.QInterval	CompactCells427To458.lean:12636-12638	f7055f7c00786c25b8f266391c4582707f6b4177e05dd54d9ea861590fd2c84e
CompactCells427To458.Box446.lambdInterval	CompactCells427To458.lean:12639-12641	dc7b4a560457b8bce3fa3c85785a7d38266c0e6e3e38c3846086d443f3d540b8
CompactCells427To458.Box446.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:13288-13299	d53ec82cd205065363f1f5714cc4b9f481345ef0db2de8fc53d2e5a7efd489bd
CompactCells427To458.Box447.QInterval	CompactCells427To458.lean:13300-13302	3439052cdeb04fdd6f4ef638ac216fdce4ed06aaeca16522511eef9d6b1bb59a
CompactCells427To458.Box447.lambdInterval	CompactCells427To458.lean:13303-13305	3790d0ea9396ae6dcfbfbf94a2a9c0051800af05ececdd6891e8c2e9db61b314d
CompactCells427To458.Box447.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:13952-13963	75e7d98a84fe185312cc60b432b548dfffa67bc1fa5eef3ed87d9edd6ecdeed06
CompactCells427To458.Box448.QInterval	CompactCells427To458.lean:13964-13966	faa7c36ee0514a669ecaa51a7ba73b7d7d4a976b946ebcdb40a3e43a50115bcb
CompactCells427To458.Box448.lambdInterval	CompactCells427To458.lean:13967-13969	2ca1f76d698652c7290f8a652799e7d646927447a4fa17cac15a30093836d3f0
CompactCells427To458.Box448.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:14616-14627	b21d90b5cdf89bd463394fa8de9b42156d8939e3b7086aa50689e757caae361
CompactCells427To458.Box449.QInterval	CompactCells427To458.lean:14628-14630	6ef0145098d69f8e7f3e67c1b3704b3ce338b8e4c96c6da27d77c65120bd2a1d
CompactCells427To458.Box449.lambdInterval	CompactCells427To458.lean:14631-14633	f974c6528b02c0f360388595e44d4811b4b31ed9b9dee7234c7141aca7739b7d
CompactCells427To458.Box449.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:15280-15291	e89c4d6dd1edba920b8ed14fcf7d133bcfd8f71923264e506e4917e306c6d8ea
CompactCells427To458.Box450.QInterval	CompactCells427To458.lean:15292-15294	e41f3dbca8c94bf75c3fcb107fdaa0079167ee4346dd8ef24509c19eca2c4076
CompactCells427To458.Box450.lambdInterval	CompactCells427To458.lean:15295-15297	a5b21d3b9bc34e9434e4607e81c0118b6833afea051cfe2e61ebf0c95b352ec6

Declaration	Exact source	SHA-256
CompactCells427To458.Box450.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:15944-15955	d38c143e3a5715121d0855a71e4f7b410b514df0c27b470b1e79fd0a5c75eedd
CompactCells427To458.Box451.QInterval	CompactCells427To458.lean:15956-15958	d47040773846753f77adff080188dd895b56832bf2b6abf1af85f74a07c2ad0a
CompactCells427To458.Box451.lambdaInterval	CompactCells427To458.lean:15959-15961	3895e213a5a297e39107e4e73e332607a30676b33f5b5f1e0e0424c56f601ff5
CompactCells427To458.Box451.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:16608-16619	d38161b30986b4fab10f454e35cf4d76e289be0e0c04e8775cb7c2d2eae5a34a
CompactCells427To458.Box452.QInterval	CompactCells427To458.lean:16620-16622	8e529d1cac05c0b85171982d10a6f14f3268197274099b6a25a64fecf4708b22
CompactCells427To458.Box452.lambdaInterval	CompactCells427To458.lean:16623-16625	209655edc6d6161a1b6f630e3809f9ff0c4b8c65d44360878771f69a133b6657
CompactCells427To458.Box452.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:17272-17283	f697758635874a79d91a9dac2b269ba4b4d49a99e230a3d097b2c1c507f51dd4
CompactCells427To458.Box453.QInterval	CompactCells427To458.lean:17284-17286	0ec1df2a5f528fc9d5c16e00cda6d357497fbb7006b3a058132277ed28f59778
CompactCells427To458.Box453.lambdaInterval	CompactCells427To458.lean:17287-17289	5c02e38903b9bab519d29e6b148cc11192f388940ca18edbd0d136544177adca
CompactCells427To458.Box453.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:17936-17947	6373249b9bee0a82397c0d428def9b4aadf125929bc8a3cea28c987fae0107f0
CompactCells427To458.Box454.QInterval	CompactCells427To458.lean:17948-17950	b1f96df69e8007da8dd6b2dd4c6dd397adf97f85584b8af1c96e894a3291cf9a
CompactCells427To458.Box454.lambdaInterval	CompactCells427To458.lean:17951-17953	2fd105e8e7afe3cbca7ceff9b1288c68118d68372454c0a0b4b2096abbc84c4b
CompactCells427To458.Box454.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:18600-18611	311448ebbeb847cbcf6c6499d161cc68dafff0f3bf1e36a215a46a92fc56848b8
CompactCells427To458.Box455.QInterval	CompactCells427To458.lean:18612-18614	75a2bbf229dd7b76b42d8154d7a4b7f1711aa7d5894dcb3d22898e5fd7b07176
CompactCells427To458.Box455.lambdaInterval	CompactCells427To458.lean:18615-18617	b061a93caedbb00c5215d0e394abc4af5db5cb018e577124005a915cf1e8b9d6
CompactCells427To458.Box455.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:19264-19275	6ebd77cc03481e09529d7755e2818f28178ec57b726fa0ecafa5b793795640b6
CompactCells427To458.Box456.QInterval	CompactCells427To458.lean:19276-19278	0a4c02709d8a7dff169c634c3e00a4e057c8f141d35c3bc036c7fabcc2663f654
CompactCells427To458.Box456.lambdaInterval	CompactCells427To458.lean:19279-19281	f1ac27a4cebd6de06e3e4c04f1048b9df86965029538f4ea326e8ed53c92a6a4
CompactCells427To458.Box456.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:19928-19939	c5570aa038b42c106008f451f7518e57b7e806b296172ef03dc6f5fb6fdee935
CompactCells427To458.Box457.QInterval	CompactCells427To458.lean:19940-19942	ea109537146e2804958175c67574162e059fa681f422295604c75087e451e7a
CompactCells427To458.Box457.lambdaInterval	CompactCells427To458.lean:19943-19945	e78a898129c07c485ad013ecf5d2820adced5f7d5fad593640be3150adb9f19
CompactCells427To458.Box457.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:20592-20603	a2e785cd3792484caf08dcb3a914c6d6cc069549dd8b52522ea3d3117f88f800

Declaration	Exact source	SHA-256
CompactCells427To458.Box458.QInterval	CompactCells427To458.lean:20604-20606	c8f8c431d7f703224c983d9c3d75f859069e0e556e75a95917b328392e91ab4f
CompactCells427To458.Box458.lambdaInterval	CompactCells427To458.lean:20607-20609	5e307d9a2b43ecb19e46fb387e1d337a789f943be51dab2d58552855ca5b1131
CompactCells427To458.Box458.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:21256-21267	76f8360c6e70c576aab39db72683e0b7adf063e7b8780fdef4ae512f2e6f3c95
CompactCells427To458.boxes427To458_candidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:21268-21498	e0b16fdb42feea7420ec6803dd722182023c63369007c60b5a5af45d6cd63111
CompactCells427To458.boxes0To458_candidate_not_isWeightedPerimeterMinimizer	CompactCells427To458.lean:21499-21512	8b2d1248408db49b206aedc59cf59834bd17891d05545d10c80cf286db38b007
CompactCells43To74.Box43.QInterval	CompactCells43To74.lean:20-22	1261c4cca7331e376975e91d01aa4235a2fc5f9aa2e296a768142f2d201290ad
CompactCells43To74.Box43.lambdaInterval	CompactCells43To74.lean:23-25	8215442a970a3993db087e9cdf8869e8858f3ae02ef1389430f6fc78dd0a5323
CompactCells43To74.Box43.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:672-683	93a8b5dc4819f8e247b51400e398c1829db6f00906524147f8e86d2cf4a78029
CompactCells43To74.Box44.QInterval	CompactCells43To74.lean:684-686	5a37a9ec3887b9b9ee1109d3a0e2b08b378601789d12ec40145776d71de99d5a
CompactCells43To74.Box44.lambdaInterval	CompactCells43To74.lean:687-689	2269a45ad233ec4bc3ecbb0b9df9c2e11a6be80d69bf08bede0f58ccc53c93ac
CompactCells43To74.Box44.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:1336-1347	b11aca50a996c5cbecad6fffb891e25b3bb4abfc930640eb5bcc784e79d7a0dd7
CompactCells43To74.Box45.QInterval	CompactCells43To74.lean:1348-1350	6f530d66d303c5a180a75d56b846027f530212f3ff6e21b0926be2d0116bac2d
CompactCells43To74.Box45.lambdaInterval	CompactCells43To74.lean:1351-1353	ce1e80758a0109d5014d744c9466ee424fa51903ffed8014ac3b8d22dd847f89
CompactCells43To74.Box45.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:2000-2011	68108078406295819cb9167c1a8804b604bb3c24a5aa5157bbbbcb2238273a6f
CompactCells43To74.Box46.QInterval	CompactCells43To74.lean:2012-2014	054926e64b6485910c93cf7780878aa4275da6bb6298cf73f99ca3cf09a9264d
CompactCells43To74.Box46.lambdaInterval	CompactCells43To74.lean:2015-2017	821d6320896fd36d17a00ce870d6cf3ff61425f33d943a197d59fe7b98a8261
CompactCells43To74.Box46.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:2664-2675	b27f88f8b3ced2c42dadf47c62075d01fb654bc72c941b36360728f69fb4d787
CompactCells43To74.Box47.QInterval	CompactCells43To74.lean:2676-2678	db6d2c9a9d1faf028f60601fdc7b8ecdcc7f4ae2f1447400bc49d9ce09da149b
CompactCells43To74.Box47.lambdaInterval	CompactCells43To74.lean:2679-2681	a60a4b435f0f684f763b9ca4942d9b9b265ffd541dd8d51923198b04a6f2964b
CompactCells43To74.Box47.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:3328-3339	84067a2d7c1f1d595eb8e54f5c05784e298f1e5f0e0a4784d1e8bf32c0ae6fea
CompactCells43To74.Box48.QInterval	CompactCells43To74.lean:3340-3342	750f930199f8966fac74c1ea044c007fef7a5c28e025918e4a73ae1b0ef0a589
CompactCells43To74.Box48.lambdaInterval	CompactCells43To74.lean:3343-3345	034fe2df7ecc445f3fdb260977927988fb431cc8073045b74191b6e5e90f3eb3

Declaration	Exact source	SHA-256
CompactCells43To74.Box48.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:3992-4003	b8dd5c7a93b372cac5bcdcf26e3b9b7e7613a6ef1ec4db01fe4c267c99b2803e
CompactCells43To74.Box49.QInterval	CompactCells43To74.lean:4004-4006	7e4cb71a771371b9db2b0cac87ff66c07aa4bac604c4bebf95e4ec91d7765c1
CompactCells43To74.Box49.lambdaInterval	CompactCells43To74.lean:4007-4009	8d283497644b4c96b730d05fa685566b45ec1e82eb42c27791d7a9a45be40102
CompactCells43To74.Box49.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:4656-4667	31e2836bc3784629860194745bea9201cdb6da0362cbb41295c5cbc75dc59e9b
CompactCells43To74.Box50.QInterval	CompactCells43To74.lean:4668-4670	e0a7441cb4db45e013e3d26056e2c043d8cf06cc845f01c01d0d22522dbb21fb
CompactCells43To74.Box50.lambdaInterval	CompactCells43To74.lean:4671-4673	2f7d38def8f3a0b9b3706ba0feeabf17e9d754bf9e2afffaa2c4583de17bf7df
CompactCells43To74.Box50.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:5320-5331	5b209ccdf2c3da828cea2cd1599e92af848180fc6d5ea16dd5d43fbfcfb776ef
CompactCells43To74.Box51.QInterval	CompactCells43To74.lean:5332-5334	b60ba65e206ab0d8865adf83a48f822bde47e408396c732fc1b9a4418aab8b58
CompactCells43To74.Box51.lambdaInterval	CompactCells43To74.lean:5335-5337	190db8017cbab52d839ebe7b8bef8b980e56aea470005e75e8ac709170babbf2
CompactCells43To74.Box51.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:5984-5995	7e04f05f6c2bd72a59443702d0513160b19369b00718e5ab59bfe47b3cf12cb4
CompactCells43To74.Box52.QInterval	CompactCells43To74.lean:5996-5998	ff0748893af35af5c3e407d91cbe510f3a5d497eb2dc00479341f55705d99984
CompactCells43To74.Box52.lambdaInterval	CompactCells43To74.lean:5999-6001	20f9f399cbe1454f15fdffa1c4581b041ebed3ae191909d6902bd456ab475d8c
CompactCells43To74.Box52.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:6648-6659	90bead8888ae6073cea83cffe63439f20339ea64bd0fd6c22a002c8fe4f52f2
CompactCells43To74.Box53.QInterval	CompactCells43To74.lean:6660-6662	2eb7e97b40ec3fdf9e2ee54ae7d3dc4dfa0cdcaab2b975a8b4940383a3555bfa
CompactCells43To74.Box53.lambdaInterval	CompactCells43To74.lean:6663-6665	c69df8e821fce9ba00a17f4d4658f12d6975dccfa9856c03294f9ceb1fe84feb
CompactCells43To74.Box53.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:7312-7323	c2e75679a360817cfe6cd0356678e9c02309c720fbfad93b47f2211427814d1d
CompactCells43To74.Box54.QInterval	CompactCells43To74.lean:7324-7326	3f800bcc30972a8960053c11f89c77c7bd308706a741368c5838896166ef6ccd
CompactCells43To74.Box54.lambdaInterval	CompactCells43To74.lean:7327-7329	17ade2c5bfab035a898562db430d6476c3a70704da3db5e734c9f1ac9feedf4
CompactCells43To74.Box54.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:7976-7987	162f8fd98724fdea4d7bc759d1b6a461187136cba9c8a804d6c9e389389777c9
CompactCells43To74.Box55.QInterval	CompactCells43To74.lean:7988-7990	dbcd8856fd5c4f59b7a1e8c0b5b8b45bd1db55eca4a0a11f05f58b1b65a2dd4b
CompactCells43To74.Box55.lambdaInterval	CompactCells43To74.lean:7991-7993	3798f1499ed03ef25fcd2298e043614655e286ba896f6ec96d7632fb8763a604
CompactCells43To74.Box55.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:8640-8651	b7952daf857d84e9e5a877af0114eec3e214c7b2b42cc316dc564808ad9a88ee

Declaration	Exact source	SHA-256
CompactCells43To74.Box56.QInterval	CompactCells43To74.lean:8652-8654	2e894b3e7bdef46531b7e13f73c0fa353f7269feeb84fdfcebb0a01053cac35f
CompactCells43To74.Box56.lambdaiInterval	CompactCells43To74.lean:8655-8657	c65c9906ade2b4181b7f26de7c8be76c1c225c44e9446ec0d93af3ceacdb101d
CompactCells43To74.Box56.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:9304-9315	b8ed6290280439a9f2bb3cfc34f6560666919637295fb1c047727943f6b00caa
CompactCells43To74.Box57.QInterval	CompactCells43To74.lean:9316-9318	ef44b3ede9464aa6498133463c0924f2cf058102b8c161e749fce481b98e8db0
CompactCells43To74.Box57.lambdaiInterval	CompactCells43To74.lean:9319-9321	0398ef385c6368b3ac260ff973fd7e7967d322abb331c335bc3f47e00ce0017c
CompactCells43To74.Box57.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:9968-9979	fba5dbfe07d5da2defd58110b07a2643f57f7bc873b89880b567d063751e89ec
CompactCells43To74.Box58.QInterval	CompactCells43To74.lean:9980-9982	1d500fce47d738929acf7ef6bca6a2b2526b953821f742cb5282b0aed4050b02
CompactCells43To74.Box58.lambdaiInterval	CompactCells43To74.lean:9983-9985	a2f2dd96505db5f0e9ca7b5e00822710cdfdbec738fb9e8de57d79a62dcb6bc9
CompactCells43To74.Box58.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:10632-10643	06a0dd0b2763606f9f07a0a16dc8f26c54cbd75180a8b16561759a0e15387cfa
CompactCells43To74.Box59.QInterval	CompactCells43To74.lean:10644-10646	017b30b75ffbe3cc96599c327f79a2ae7efea074098612a54ac97a6c53023ced
CompactCells43To74.Box59.lambdaiInterval	CompactCells43To74.lean:10647-10649	554c099a192b11604086e0464ddcdcc9091701383fc905ab704d15d4665dd2ae
CompactCells43To74.Box59.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:11296-11307	2b096ce162c52d6bf33996cc1eaa864ad87a550052eb581d4708de281dfcb239
CompactCells43To74.Box60.QInterval	CompactCells43To74.lean:11308-11310	6e9b27174bc6603e1570675cca1456cd897e3ee6c8d82dac9063b1d8bdb264ac
CompactCells43To74.Box60.lambdaiInterval	CompactCells43To74.lean:11311-11313	d650ab34e2c33e07980bc64c6757250b4c041a8c4a968360bf3859fce1b018d5
CompactCells43To74.Box60.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:11960-11971	f968d849cb0946380b3b679a11b784cbee15e48c6bb6a8fa67aff8260acf0120
CompactCells43To74.Box61.QInterval	CompactCells43To74.lean:11972-11974	bafb7f457f6d60f70f9f3ad225baa2d4e6339d0a58914e26c4d0ed0b123ecf86
CompactCells43To74.Box61.lambdaiInterval	CompactCells43To74.lean:11975-11977	e92146d71092e7e3d020b2e0f71d94e4e70a7006294056ff173d474092711571
CompactCells43To74.Box61.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:12624-12635	9d695cc5ea35c76f56936cd49e29d58365cd9b98547847f4f5ba66b83dae7bdf
CompactCells43To74.Box62.QInterval	CompactCells43To74.lean:12636-12638	939eaecd896f91acd80f040486a1813e406fc34c7f287d7780528431e1fb76a
CompactCells43To74.Box62.lambdaiInterval	CompactCells43To74.lean:12639-12641	81da428a5ea380b8cb68d7ebdfa498dd43e6d848e9ed30ce02473b23f7706dbc
CompactCells43To74.Box62.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:13288-13299	20e395eb241db6752c4ea4c3212f40e253fc74a309b439c65c2be675bf6d826f
CompactCells43To74.Box63.QInterval	CompactCells43To74.lean:13300-13302	28485561aa8d972dae4b5de78fd04f94179170518da30e81773b0de692bcf228
CompactCells43To74.Box63.lambdaiInterval	CompactCells43To74.lean:13303-13305	0118296a2b484821c9a3fb78996e889366e5465bcfb2b9c1e71d373517860221

Declaration	Exact source	SHA-256
CompactCells43To74.Box63.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:13952-13963	8f8b8bbd38f41f2b290508cdb32b3f0c32afb616577423af8dfd1c0aa7ab2dc5
CompactCells43To74.Box64.QInterval	CompactCells43To74.lean:13964-13966	a0cdf8fd4871eeba0c54606199b032b7640b04758209d448f61406f031dc8c74
CompactCells43To74.Box64.lambdaInterval	CompactCells43To74.lean:13967-13969	74a754f5af46cbd0b4f960c96936065242b108c8f13d81a9a266cc9bb3f2e890
CompactCells43To74.Box64.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:14616-14627	aab883fa654eba4f8b669295129dd99a35ec60b994847a9b864fd47b733a7d55
CompactCells43To74.Box65.QInterval	CompactCells43To74.lean:14628-14630	d9456ab76afa02b7c8dded329f9319e796c58e693e24e72460d1d397750e5d25
CompactCells43To74.Box65.lambdaInterval	CompactCells43To74.lean:14631-14633	169f990120e2469839e3b73ecaf0c4cd73a76d62ecd2d5f5cb0a369a70b8b6fce
CompactCells43To74.Box65.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:15280-15291	c83a46ee79e17481a4c90a3506a41dd05412369d3ecc452acad5be105a94c6a5
CompactCells43To74.Box66.QInterval	CompactCells43To74.lean:15292-15294	3d5508ffcf5317a29e2673fb991b1362b8c2a9a5734fc3c47883381fcb10989
CompactCells43To74.Box66.lambdaInterval	CompactCells43To74.lean:15295-15297	d64945d3aa5cf4af984621f496b1e699e8b52588273aad2b310405c43eaa8075
CompactCells43To74.Box66.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:15944-15955	7140181c83713ef747c8f97381d2d6fe776a3a5d145f3af340f88dd248d2f85d
CompactCells43To74.Box67.QInterval	CompactCells43To74.lean:15956-15958	b18225b9c9b927c915ac29c0c02ce45ec712a103eebb9fe7d3abcf6f72551dee
CompactCells43To74.Box67.lambdaInterval	CompactCells43To74.lean:15959-15961	c254c3531067eda5c6f75e7cd8b3fe8901d956596677f380ec2adba6e6099561
CompactCells43To74.Box67.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:16608-16619	f5cedf091e430ad9df75a58be99b500676626e4037dcf6efc9b3ff706027095d
CompactCells43To74.Box68.QInterval	CompactCells43To74.lean:16620-16622	8de528a2707ff93ab2cf1a16b1ad625b37e3a95a41cd55e270afbeffc332957a
CompactCells43To74.Box68.lambdaInterval	CompactCells43To74.lean:16623-16625	f1e87e6e9e893b824fba763ac353cc14232909730664fbd8512f489cd1bc6214
CompactCells43To74.Box68.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:17272-17283	8334d4a03a1198e262496f5c679f5059bfe1166fc8d65659a5878dcffac52838
CompactCells43To74.Box69.QInterval	CompactCells43To74.lean:17284-17286	e8e015800831a7111f2ea90d6d3f211628cf72460ed89029ea0c51fdf811eb20
CompactCells43To74.Box69.lambdaInterval	CompactCells43To74.lean:17287-17289	f7aeba115f34d4969a43003d3b91de4831a958687e6379a49ff3746f018b6398
CompactCells43To74.Box69.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:17936-17947	e3e53e58a9a59b117b157509112a7c12fb1a26cd3452ace0bf9d1981a0b2ee3e
CompactCells43To74.Box70.QInterval	CompactCells43To74.lean:17948-17950	62eed18f2fc14cc2b639f3b76a5eb5b1af60f226485d3b81d86ca6f4f4a4833f
CompactCells43To74.Box70.lambdaInterval	CompactCells43To74.lean:17951-17953	e461d0d5f94dd6022abf0786dee48bde8e7df5f24c794c2c8bd703d08ac28b8a
CompactCells43To74.Box70.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:18600-18611	bd46be24bc96458bc1bc8c7e4860c0bac3914e33ee098eebf0b3d69dce102dc9

Declaration	Exact source	SHA-256
CompactCells43To74.Box71.QInterval	CompactCells43To74.lean:18612-18614	889c78e50c0c964641848be9825bd7bfe4bdf54aabd4ce965f878c18fb4aa80b
CompactCells43To74.Box71.lambdaiInterval	CompactCells43To74.lean:18615-18617	3daef6dcb9626deeb3a9f04a6160787bb8a826c74efbba618807ce3fbbd1ae7d
CompactCells43To74.Box71.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:19264-19275	734006c5743e342f3d4c0926f3f36c4330b6ce847890c386a861978595366a31
CompactCells43To74.Box72.QInterval	CompactCells43To74.lean:19276-19278	1e264414824aced85fb66dc7c5907c2a425516488a3eb0af63d91d3ca64823a6
CompactCells43To74.Box72.lambdaiInterval	CompactCells43To74.lean:19279-19281	b3fb98adbac9b9d32d24a983baeaf83dc50787be8baca21e9d7fcd8c8cdc140
CompactCells43To74.Box72.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:19928-19939	7b0e735575c318af1c7f299548fe38520af7e686b8d7be54ae35cdf3942e2939
CompactCells43To74.Box73.QInterval	CompactCells43To74.lean:19940-19942	849cd971d41b79e7af538b93841f205560ecae5852307f2fdb267873e66add85
CompactCells43To74.Box73.lambdaiInterval	CompactCells43To74.lean:19943-19945	1bfe66e1d2738e27fda346c5133f3152a5925a2f481c5c3941c7e64a6a6a9fc8
CompactCells43To74.Box73.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:20592-20603	4858c8837cd7dd0e5c6966437dff8d9edf2f8da84e2a652f25244411e50df5cd
CompactCells43To74.Box74.QInterval	CompactCells43To74.lean:20604-20606	f8994da3096dfd2bb7f31929a0f8752ba40900cd88353714911bdc2745037210
CompactCells43To74.Box74.lambdaiInterval	CompactCells43To74.lean:20607-20609	9da36d762f052853d2d0b09c51cfdefa3e3f77e8db61eeb89ca36f9777d05220
CompactCells43To74.Box74.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:21256-21267	6c7282268fff20e2126edbf0979123c5285108017283f643bad7c1e4ff505db
CompactCells43To74.boxes43To74_candidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:21268-21498	899c5a82ace73846d50118707c3d3dbbe69807b75da9128da1fca8d919e346cc
CompactCells43To74.boxes0To74_candidate_not_isWeightedPerimeterMinimizer	CompactCells43To74.lean:21499-21512	19a57bf71e6f4e645925e362a438f84e571c445db421bd1aecd567a18e07ae39
CompactCells459To490.Box459.QInterval	CompactCells459To490.lean:20-22	52903f640dde5a600fb0677517b747db2c0a6377f3cd5deb5c7dfe1065235a38
CompactCells459To490.Box459.lambdaiInterval	CompactCells459To490.lean:23-25	3e7a349fad65708b04976ee3d08e361538b9ee6abbffbfdf7f16a7fd5520c71c0
CompactCells459To490.Box459.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:672-683	f1edb2f7e2270167f49bcdd7ee0edf0b17f2709b6d5de4044f967551fcd99775
CompactCells459To490.Box460.QInterval	CompactCells459To490.lean:684-686	0297e1805b602c6474e5867108b72643826fd9f89fd13db0558bbe41e9992b0b
CompactCells459To490.Box460.lambdaiInterval	CompactCells459To490.lean:687-689	9b3e576178fab309d7c6e3b399ec23d8ab39a1553e3dea2e7140f48383b1276f
CompactCells459To490.Box460.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:1336-1347	23dd8ed191c476b8b84d811d62ab2a1670a4c057a0549572415a33c502f0ba74
CompactCells459To490.Box461.QInterval	CompactCells459To490.lean:1348-1350	6646be10f46e75ba667b713b87efaffc23f1cdd1551f9f31bc6a76d167a00127
CompactCells459To490.Box461.lambdaiInterval	CompactCells459To490.lean:1351-1353	c21cfc67f7a38508d9e6655239aefcfa45efc0cb0cf541187cf6059a0a77a618

Declaration	Exact source	SHA-256
CompactCells459To490.Box461.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:2000-2011	5d41577405a6740c2d19b83fbd50ab5d9d5b21d64c25d0f32b8731739c696fe4
CompactCells459To490.Box462.QInterval	CompactCells459To490.lean:2012-2014	9e79ad8649e8c087447088939b02af133b2f2dbd8e0a3788df3a978c114f4ff2
CompactCells459To490.Box462.lambdaInterval	CompactCells459To490.lean:2015-2017	98889d1c37e26e8121e62a3ebe67f4a0327f648458eb6457c88105d81219d7b0
CompactCells459To490.Box462.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:2664-2675	b20308c9b89c64bd576cd17382d29a39abc0de9919e9560e6837916cac719169
CompactCells459To490.Box463.QInterval	CompactCells459To490.lean:2676-2678	2876c9cbacc9f5d4f055c2131741ea53af2dde2d956642124f75d586805480eb
CompactCells459To490.Box463.lambdaInterval	CompactCells459To490.lean:2679-2681	dba89f39dca7f6f3819b3969e69b9f74520b549801381e69b9311ef1d1af1698
CompactCells459To490.Box463.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:3328-3339	f6d7574bb3e557cacf12bf9878c9fae9bb37e3cf3675874566e81b068e683d20
CompactCells459To490.Box464.QInterval	CompactCells459To490.lean:3340-3342	a37abf85657a48490f5b881fba25e4c8363797bf6b07944b4fe311e34ce01253
CompactCells459To490.Box464.lambdaInterval	CompactCells459To490.lean:3343-3345	5130293d01706c05629e59711e503cd41a4c3c87d10a758908bd0a29b3d87672
CompactCells459To490.Box464.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:3992-4003	85d39014566fd29e77bc49ce86443989137d1e0a523082736f00d3e42a734e64
CompactCells459To490.Box465.QInterval	CompactCells459To490.lean:4004-4006	fe64da221eb70144157ae58729138a17492bdf8e08d25fb1452c698c08038dbf
CompactCells459To490.Box465.lambdaInterval	CompactCells459To490.lean:4007-4009	badd90e18c0712c3cefbe868590c403b8d0e818585f8bf35cbb7690bc599d7db
CompactCells459To490.Box465.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:4656-4667	9110d08478142c0552b2d5363a01e94591e4cc178fc9a904ecdd9f4678288c37
CompactCells459To490.Box466.QInterval	CompactCells459To490.lean:4668-4670	27c8f653eb15aa41351fa8fab002f53ede a7ff5865ae89948e29606747c4522
CompactCells459To490.Box466.lambdaInterval	CompactCells459To490.lean:4671-4673	75def372e993534c0f76982872c6156a62e beefaa3ce03804a96dca400ab89a9
CompactCells459To490.Box466.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:5320-5331	6374fb5ca080788862365d896665b687bc950d97a154d4f9e69be5a110c2c8b7
CompactCells459To490.Box467.QInterval	CompactCells459To490.lean:5332-5334	b2de94ccdb890e82c1b7cba8b24c67a0f9969d1f4ebc7e88cd1b6af68d287cb1
CompactCells459To490.Box467.lambdaInterval	CompactCells459To490.lean:5335-5337	44d8db37c655cc1ec36d9d2996b8e537dc283587d51ce9a5a556b8b879abd685
CompactCells459To490.Box467.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:5984-5995	923813d1157aeb9e48b1e74a189496d93d25983749615308fab552c171a7dc28
CompactCells459To490.Box468.QInterval	CompactCells459To490.lean:5996-5998	d257ce2c1a1b0d6efcd2f8281723938acdf77e9b40bce77a3fe1e4ec961da305
CompactCells459To490.Box468.lambdaInterval	CompactCells459To490.lean:5999-6001	ff04b5cea4902cf2851600d208b7049e64972229d41b3be2804d61a7ab9641f4
CompactCells459To490.Box468.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:6648-6659	724e38bdd4b63da6d033480519b6253d45e0f088fdb056cf8d701d96e90cbefe

Declaration	Exact source	SHA-256
CompactCells459To490.Box469.QInterval	CompactCells459To490.lean:6660-6662	ffe9c1286d2b4a30b5c4c18aaed980749809baff619bec3163b2312b0611d076
CompactCells459To490.Box469.lambdaiInterval	CompactCells459To490.lean:6663-6665	133145e7c56bf94f99ece4ff4682fe2412c713e3eeee07259acad141ca47a9cb
CompactCells459To490.Box469.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:7312-7323	bba03dfd0fe777b9e149449e37818195dad5bb36c2c6cd5222727ed9340a7f8
CompactCells459To490.Box470.QInterval	CompactCells459To490.lean:7324-7326	bead64a31bfb243002ad726e1ea9101b0915f37e55a5cb4c0411b806b339af2d
CompactCells459To490.Box470.lambdaiInterval	CompactCells459To490.lean:7327-7329	4a449a5be23af67d0eb92c26bbdc02f2a3c4897478551e652ad517e483a1aa85
CompactCells459To490.Box470.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:7976-7987	c09ce9b4e58ecce21d0dbd1d2cd93a629f3d89f5c5fe2f0d2a680ff54f92737a
CompactCells459To490.Box471.QInterval	CompactCells459To490.lean:7988-7990	af18b86305c66c18d27034ba9e5772ded07397bfc201e36eb2b76f287af8774f
CompactCells459To490.Box471.lambdaiInterval	CompactCells459To490.lean:7991-7993	a1b01d8edf62e7ec36c2019e2b97369960cb56f5720ad828b93e0232e252c324
CompactCells459To490.Box471.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:8640-8651	c09b2d249f76fe0f95be4ee46b68b37ed8177cd3c889c3b860f0f5c6bd03cc7c
CompactCells459To490.Box472.QInterval	CompactCells459To490.lean:8652-8654	f452253de8736cf53da8f41a8e6cf61734fead0150b70be479ecd84ff2cbf2ff
CompactCells459To490.Box472.lambdaiInterval	CompactCells459To490.lean:8655-8657	9a2e53945947682f2d9fa718d4569300c2fb82400451099cdd180493a503a206
CompactCells459To490.Box472.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:9304-9315	2cc18fb92b49cc7cb1acb45eefd34698bb95741142965318091bd1b6bb36eb59
CompactCells459To490.Box473.QInterval	CompactCells459To490.lean:9316-9318	6222d9c0f76b01798e712273c51da4eb1c9905cac716e7e4cfc12505e91146ef
CompactCells459To490.Box473.lambdaiInterval	CompactCells459To490.lean:9319-9321	8a06cc2f2db289505bf8ed7ee403124813536c7d08780ec38a92989d006b1e8
CompactCells459To490.Box473.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:9968-9979	d30738899c0c63b8bbf49b316f6ac8bf979ecb227a551dc6b26d7eda190ca0a5e
CompactCells459To490.Box474.QInterval	CompactCells459To490.lean:9980-9982	ec837aa8822f3c9ffea8bccbb92249bb560f3eea2855fdc246547f62d3277590
CompactCells459To490.Box474.lambdaiInterval	CompactCells459To490.lean:9983-9985	1972f4dc925f75d5742a38807f0a21ff1861e55177c3952ba7eb202e2c11dd14
CompactCells459To490.Box474.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:10632-10643	4660ffe10835d581853c89880b73bf9436eb6f592c5245eab7fe59703e72383c
CompactCells459To490.Box475.QInterval	CompactCells459To490.lean:10644-10646	a58a1c64bc48b12cb70c33f30105dbf302b6aa99aa74d9ddc9ec66d636350404
CompactCells459To490.Box475.lambdaiInterval	CompactCells459To490.lean:10647-10649	c22d1eda5370f7c63975e4f336344a03fd712692202f3af314ef4a13cc36c09c
CompactCells459To490.Box475.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:11296-11307	7f445e8920ddaa125452fdd91df94b3441d072cd077b226d84cc218eb33e2b31
CompactCells459To490.Box476.QInterval	CompactCells459To490.lean:11308-11310	bf797dccdc85d275ad116773da45e10674f5beab5b49f3ac52e8fb2c17b2b242
CompactCells459To490.Box476.lambdaiInterval	CompactCells459To490.lean:11311-11313	d483af74702ed14c5a56ef1293152ea847b58c5ffcd816d16952d022759d002c

Declaration	Exact source	SHA-256
CompactCells459To490.Box476.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:11960-11971	743ef470b1692776f996df7a525a67591abe3171e5cd8d55e89f1b50d9baf688
CompactCells459To490.Box477.QInterval	CompactCells459To490.lean:11972-11974	6c4ed6288553acc83f856fd48c6e4a74d8a5b18ea984853368fb8f133d433f9
CompactCells459To490.Box477.lambdaInterval	CompactCells459To490.lean:11975-11977	8e0174967449fe210117220d6635b122487e9eee2bddeea3b89f60a7535746df
CompactCells459To490.Box477.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:12624-12635	01144bc5f70910e93fd69ca3df771f9e3e4fd00f84312d5403a06afeb1735595
CompactCells459To490.Box478.QInterval	CompactCells459To490.lean:12636-12638	b23efa1efef0d8ca461742cfc7d48805571e1782eda1420c2547bcf35f31ddb4
CompactCells459To490.Box478.lambdaInterval	CompactCells459To490.lean:12639-12641	031b07fa25aea43782c3ba64d54029cac915ccbec328cfde7c825edbdcd1225ac
CompactCells459To490.Box478.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:13288-13299	002fb7722d9e1d136ff9028c3a2e70e756cf93cf57e9383141885d2df39ffc94
CompactCells459To490.Box479.QInterval	CompactCells459To490.lean:13300-13302	20ac7fc0be6aca2e75325d2d44d92406fae6ae59045eb03d5bae2838057e382c
CompactCells459To490.Box479.lambdaInterval	CompactCells459To490.lean:13303-13305	d6bb3f6a64cb8c72173181376ac02411ee4f0b59b8bcb5e011f74e64def4438d
CompactCells459To490.Box479.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:13952-13963	5545964dcd9fe0460cb6f7276cdf7fa7d2a001dd75d9cfcdbdb86209237db414
CompactCells459To490.Box480.QInterval	CompactCells459To490.lean:13964-13966	68e9bfa0c9862156d1546c20097d5cb2b516b7517abbd58e99f290958177c251
CompactCells459To490.Box480.lambdaInterval	CompactCells459To490.lean:13967-13969	bd8939df406a4bc00f7c89d04b707f33fe86e3abf086ec44494377d6ab4f5e73
CompactCells459To490.Box480.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:14616-14627	f50d214f505a1f19bc6f9c8eb1d0d3d2b59b176716ba8c81b987da8927840148
CompactCells459To490.Box481.QInterval	CompactCells459To490.lean:14628-14630	1d96fc44ead5d1ca5097df070986e83549eb3f636e6ba2fa48dc15003c227090
CompactCells459To490.Box481.lambdaInterval	CompactCells459To490.lean:14631-14633	c38b42afd6c97f1448aa2692415438744a5ddd9095972970819fe88f0e34922a
CompactCells459To490.Box481.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:15280-15291	f3ba628c4b5c95f66fe98ae45a75c4e73749aeb9349ff3a5147000d4efc1c047
CompactCells459To490.Box482.QInterval	CompactCells459To490.lean:15292-15294	cb7db6cbf108b65e24ff1fa948e925077c82517b10eddb38ff21de037749d57f
CompactCells459To490.Box482.lambdaInterval	CompactCells459To490.lean:15295-15297	de00879da4524ceee005a17d7ad561ca3b64f41f27e973aa55846a77b7fd54a2
CompactCells459To490.Box482.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:15944-15955	d52a8bd2c44a312891e7434a45448388645c5f7bc425a536790184ad5a076ade
CompactCells459To490.Box483.QInterval	CompactCells459To490.lean:15956-15958	0502ded01a935960b0135f2365ff4d0bc6e6f0adc048b8c605815893a0ec28f4
CompactCells459To490.Box483.lambdaInterval	CompactCells459To490.lean:15959-15961	f02d65b5fbdfb2ad82b459cc8350fe59115ce820c84ce9d774c914ed5aec20f5
CompactCells459To490.Box483.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:16608-16619	63b4f87afacd1ca1a5b7e4f079401f38360588d1f5928514fc53662131ab0dd8

Declaration	Exact source	SHA-256
CompactCells459To490.Box484.QInterval	CompactCells459To490.lean:16620-16622	7defff93e844207663ffa465ecbe176e38ad4d5beb04173b57f32b6ba7239ab1
CompactCells459To490.Box484.lambdInterval	CompactCells459To490.lean:16623-16625	622241801ed83a2ebbe4b93421c9c9c898d01f649a636f3bc83353302f6e51cb
CompactCells459To490.Box484.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:17272-17283	ca014f69ed5aa5cc0484029671d38a86029db475a3cb1964e82709128c1b5fc7
CompactCells459To490.Box485.QInterval	CompactCells459To490.lean:17284-17286	0e00d2a02ad916c9a0e9f13a6f520efeb430d189ad1131243053dd8567b7e361
CompactCells459To490.Box485.lambdInterval	CompactCells459To490.lean:17287-17289	ddba1978970f34fcfac14ed53c5a175d95841b7c3f5a564e0437fd86b52c8486
CompactCells459To490.Box485.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:17936-17947	23101b72fe58adcb92088d4e9176d75da3e4dd43e88218b273d9be5ced2cb42
CompactCells459To490.Box486.QInterval	CompactCells459To490.lean:17948-17950	902ceb24f71125e9e58c0f09aa2fbd9257d4bbecfe20969c0d8e6760912d6533
CompactCells459To490.Box486.lambdInterval	CompactCells459To490.lean:17951-17953	b8592803abdc728953adb5f5dbbf538fe52243ca4818db67eb8fa70e2c1f38cd1
CompactCells459To490.Box486.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:18600-18611	e99daeda5d3ae53e2ba0ac34e4f244ec418442f4fcbba9a8b3f86d8c2e01e0b0
CompactCells459To490.Box487.QInterval	CompactCells459To490.lean:18612-18614	ff616a4569f54097d0db24f54013882c98d6ec79c6acac1991a1c999d2672300
CompactCells459To490.Box487.lambdInterval	CompactCells459To490.lean:18615-18617	7c56859922394c5dd1c552b0f645fe1b00c5d0cf1aac9a948a7d1c9766eff09f
CompactCells459To490.Box487.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:19264-19275	a8f893f0c6e302ebe9630b11f2b12422b3f2e03d5a6a80d944a1ef7ec2db758e
CompactCells459To490.Box488.QInterval	CompactCells459To490.lean:19276-19278	564da7988817fc7de5e6af1888e14699577a994c6112c0d2a307c1d52b226697
CompactCells459To490.Box488.lambdInterval	CompactCells459To490.lean:19279-19281	aeb9d7b61d75c3efb63cd0f99c32300805992e332be749f0bf49d8e3d2bba1c5
CompactCells459To490.Box488.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:19928-19939	fa2f9c1db165d170f2cf5156dc9715399a4c41a0ccf0273f1d3cdc712efb133f
CompactCells459To490.Box489.QInterval	CompactCells459To490.lean:19940-19942	54a91f82fc9b82d557d64bc5d5f7dee91ab08db14580d89f634b4813e8473627
CompactCells459To490.Box489.lambdInterval	CompactCells459To490.lean:19943-19945	5732d0670f554cfead118291b56e1d324e53f52f2d5fcd9b07b6da9f596f3120
CompactCells459To490.Box489.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:20592-20603	d8f4fde36411c4a38f50c1e02d378bb9eed0a43bcc3d391ae20450413682662c
CompactCells459To490.Box490.QInterval	CompactCells459To490.lean:20604-20606	495ac24da6cccc019535c9c9f047e5b7a4ad24a4fa3eba2acefee5bce42d9f21
CompactCells459To490.Box490.lambdInterval	CompactCells459To490.lean:20607-20609	2bb97661d89fa0278602d94f860c965baa7fc88965f5b4bb43d324b8daf4cbfc
CompactCells459To490.Box490.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:21256-21267	559db61d5e7ab4c17f2ac2619d5781d5f6f5915e546d17f87f0f4a1875a528bf
CompactCells459To490.bboxes459To490_candidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:21268-21498	27ad82bf73b6abb8762fc45ee2a255aca2971ddc7f458b820ac0c4925e9acffa

Declaration	Exact source	SHA-256
CompactCells459To490.Boxes0To490_candidate_not_isWeightedPerimeterMinimizer	CompactCells459To490.lean:21499-21512	33fb8ab16ea6825a544fc1a3086d20f4e603f22327858bceaaa87909ad993a65
CompactCells491To522.Box491.QInterval	CompactCells491To522.lean:20-22	2caa802180db4525643828346dee0e4bd65fa2016fa8519f099c6d043356b387
CompactCells491To522.Box491.lambdaInterval	CompactCells491To522.lean:23-25	839fa7b16604f10ac6a21ba1b27dcacbe3033f471d3a31b1509f83f4a5e22635
CompactCells491To522.Box491.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:672-683	ac4202f31f5fa7c6e1ddf753ad18d772a874e5b36be4fd767c18f384ea4145d9
CompactCells491To522.Box492.QInterval	CompactCells491To522.lean:684-686	fd03fd73b6ca656d38ab3b871c9016b7a835468df316f1a74dd5e7aafca6be12
CompactCells491To522.Box492.lambdaInterval	CompactCells491To522.lean:687-689	b2a2ad79309c343d27605a6ef939ea03e56c6a40b52d868af022dd5143539b66
CompactCells491To522.Box492.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:1336-1347	b6a33122f6153fb29fbbc91a56e7677328841412030fcd0a8e9c7342cbc28bc3
CompactCells491To522.Box493.QInterval	CompactCells491To522.lean:1348-1350	046a3c16c611b329d19343dc720e5813a54a5682ff83df97e872b1a420f2bb5d
CompactCells491To522.Box493.lambdaInterval	CompactCells491To522.lean:1351-1353	9c07ab1d79712ad223669c1c203c0e77874ee44327e49b7e1277264ecb1327cc
CompactCells491To522.Box493.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:2000-2011	9f64f6980a0842652eed2985becf145e24eb9001e9e7cece31f418c2d4a79fce
CompactCells491To522.Box494.QInterval	CompactCells491To522.lean:2012-2014	ac7896d932f851bb4c9f790ca3b1a3f33ceb6292709aaf4d269382cb7b1308f
CompactCells491To522.Box494.lambdaInterval	CompactCells491To522.lean:2015-2017	07fcb9048731018f75610de192b83f4e35d3901e8477ab0f79c240c04a4df856
CompactCells491To522.Box494.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:2664-2675	ff10f7236006880e4111940890847cb129ed99329ad6ab35b600271293bf9641
CompactCells491To522.Box495.QInterval	CompactCells491To522.lean:2676-2678	e654407911c7603390e8973119d7b5699896f3a0ec2980a48a0649a9aae9aecd
CompactCells491To522.Box495.lambdaInterval	CompactCells491To522.lean:2679-2681	db23d39a3c491193111f118f2cea54ab8c78a9b8c96a3c0c4d8cfc7d870b2dd7
CompactCells491To522.Box495.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:3328-3339	4532be52cce096892adf1684a0c5fdec75725345cae32354adad0ba2dfe5ede
CompactCells491To522.Box496.QInterval	CompactCells491To522.lean:3340-3342	872a17bc5fd8bd9dc2ff572e77e21019d7de85beb2645b6cee9a00e0675a93fd
CompactCells491To522.Box496.lambdaInterval	CompactCells491To522.lean:3343-3345	34dbfd054b1c8c3b148ed7b3671245f89d35ab035b68440e20c4a9a8f8c82f6f
CompactCells491To522.Box496.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:3992-4003	3835a9a2c1a163793740d7a314c8901a2bc48c35941679f03cf6ad9adab93926
CompactCells491To522.Box497.QInterval	CompactCells491To522.lean:4004-4006	66ff02264e117ac4799aabe8a756cb204b829815e6ed33fe17573f58c4820e52
CompactCells491To522.Box497.lambdaInterval	CompactCells491To522.lean:4007-4009	b59fdde634a83fb3d48d52481fec2f729ee36718e7e97986caa330f82869452
CompactCells491To522.Box497.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:4656-4667	d3de44f0d2185da2f9b86dd4ec07a85835e5cdfdf679ee057ca5258abe6dd925

Declaration	Exact source	SHA-256
CompactCells491To522.Box498.QInterval	CompactCells491To522.lean:4668-4670	4a5d83fd1db3c5d2751333497211b1dea1a58e8dd2e0e349fb404f62af3e7049
CompactCells491To522.Box498.lambdInterval	CompactCells491To522.lean:4671-4673	3591ab8602f4a8c49722864b2cc3898af59fde834033ea3713269fe0cc67740
CompactCells491To522.Box498.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:5320-5331	5629f07b23483ce030d8bfa0a2a2b1477a98d4dfa71d95a6091c279103261979
CompactCells491To522.Box499.QInterval	CompactCells491To522.lean:5332-5334	4abf24c49ba56f2e7e2e1afddd607e06c0e715b8e476680e6c5ea2eb8cc485cf
CompactCells491To522.Box499.lambdInterval	CompactCells491To522.lean:5335-5337	a3585c4e1ca33f9e5d6b3d1a3e6a6e0a3fafcee407c69fc01d60e2d7b5a8f02a
CompactCells491To522.Box499.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:5984-5995	f93133a5091653445c502a8e1eb789628d53a7b60f0bbac907cde35fa490379d
CompactCells491To522.Box500.QInterval	CompactCells491To522.lean:5996-5998	9a389f78e6ec4372d1ea956d424437bcac1e650b2592c3a250dc0f6fc08f0b15
CompactCells491To522.Box500.lambdInterval	CompactCells491To522.lean:5999-6001	ead17e4c7e5aa3c32ac6da502013e470ef5ccb3c1a4acbd146f8cdbc84274da1
CompactCells491To522.Box500.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:6648-6659	63d188611ebc9c4806818b3ff83330ca1c0953557d5240826ffeeba01f93c4b0
CompactCells491To522.Box501.QInterval	CompactCells491To522.lean:6660-6662	58f13e67a08e5054e8f6c9811b9863295c4f90af55185e407c99a0aec5454ce7
CompactCells491To522.Box501.lambdInterval	CompactCells491To522.lean:6663-6665	eb9d9fd84b2efcdd8ff96b1fb744dae5912be00f4bc0f5a9dc97066f1c7072c2
CompactCells491To522.Box501.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:7312-7323	804031c0a8104a9e59a03719e66574e185a6eb4a31272ccd9a77a2c60e436f18
CompactCells491To522.Box502.QInterval	CompactCells491To522.lean:7324-7326	0a35a65427c1ccccf8b5dc6106ce69cb0075f8f72d0c215fe0da1944134299795
CompactCells491To522.Box502.lambdInterval	CompactCells491To522.lean:7327-7329	ad5f0bc82d3de7c5639cb1a7235ebb7b16702c070ec925dcdae2d561651407c0
CompactCells491To522.Box502.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:7976-7987	85ac2a95b34598c0d93ddc41b4ffd0fe21434b061341bdea16e67b7553136ea8
CompactCells491To522.Box503.QInterval	CompactCells491To522.lean:7988-7990	506de77ab469aeb67478ccda50acb2036f513550a5d24de9ff5e4fe4a5dbc334
CompactCells491To522.Box503.lambdInterval	CompactCells491To522.lean:7991-7993	03e2cc212c8857748cdd0b7a90de493f46e1fa39b5b07c45cba31ba15fea959c
CompactCells491To522.Box503.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:8640-8651	309d7eb400d841a7daf4dd3cc65d2594255359b9ce1cdedf8522a3b0a730d9a1
CompactCells491To522.Box504.QInterval	CompactCells491To522.lean:8652-8654	b345dc55c2acc75847ba567c23671e5d6f58c2ba00391c789b9bd469b5e60f9b
CompactCells491To522.Box504.lambdInterval	CompactCells491To522.lean:8655-8657	43749bab63740d72e8ab4c251ba27754f8d7a89197da9d0af4fc28e9cccd971a
CompactCells491To522.Box504.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:9304-9315	0fca3809e36cf565d2c13355bb14677018177be01a9c948dfa14e7a87c517afb
CompactCells491To522.Box505.QInterval	CompactCells491To522.lean:9316-9318	60c29608c31f0149dc5730def007efbc36b4e9c064976f888a7fd3a38c77ef7f
CompactCells491To522.Box505.lambdInterval	CompactCells491To522.lean:9319-9321	3e3f53594ab3ef30dc202df5ea625169e1b1e808bcd08c8f64a6789e5c37c66d

Declaration	Exact source	SHA-256
CompactCells491To522.Box505.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:9968-9979	464e466d8213abbbe65973ea0ce35e7123efc81fc0d78938459b7f0dfe50abc2
CompactCells491To522.Box506.QInterval	CompactCells491To522.lean:9980-9982	5add6483d020c447da6703c4a66d62a6f2aa59da63e30594e55e161fbcabc13e1
CompactCells491To522.Box506.lambdaInterval	CompactCells491To522.lean:9983-9985	52692a38757a2f591b9207a09a3dbd177ebc6e28984c0ff94c91bb1b90beab5b
CompactCells491To522.Box506.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:10632-10643	55ce169726d539fb4ead5893ab2f8cb15d65062f3d1854012716f7d9d74584ea
CompactCells491To522.Box507.QInterval	CompactCells491To522.lean:10644-10646	86c631d49138bbc457999e1515cbec1cf561d34b500c2e467d352fa142fce423
CompactCells491To522.Box507.lambdaInterval	CompactCells491To522.lean:10647-10649	e6e8f6344ddfc8aa732eae634c5ad14c24582147b5cfdb7323d95b5d6c34961
CompactCells491To522.Box507.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:11296-11307	8a790c53416fb4c9a924c87aca35c7ebdc0c5c930584b44b0752df50c16c6ea8
CompactCells491To522.Box508.QInterval	CompactCells491To522.lean:11308-11310	40b1de33df560728a6384315941b3ab0141ff37c94a1b6d7939ad048e5540858
CompactCells491To522.Box508.lambdaInterval	CompactCells491To522.lean:11311-11313	8056fd16a2444a6060c3680a0d8dec83666bac2ed959a8847a85feb1cd8c279d
CompactCells491To522.Box508.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:11960-11971	56c5fb76c1b6565ccab831bd6f6416333b0265a6df73fd5e84e1bea6e58a2486
CompactCells491To522.Box509.QInterval	CompactCells491To522.lean:11972-11974	d2f319bb1468583b7ea0b15fffe78369ec6c88ca3e885957e407b904af1b0e879
CompactCells491To522.Box509.lambdaInterval	CompactCells491To522.lean:11975-11977	0c6c379481d7da337310d93d6f254d516c4b4ef83e230a8cd7ce9502e54c7bb2
CompactCells491To522.Box509.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:12624-12635	30a9da5d1c2110d814072fcac7d66606b9f0508ccd4801b1c794d565ea12c445
CompactCells491To522.Box510.QInterval	CompactCells491To522.lean:12636-12638	a6b9c529b40d22213c9f243f725e5a3aaeba2ee0e093edcecb88c14d8c6678861
CompactCells491To522.Box510.lambdaInterval	CompactCells491To522.lean:12639-12641	fddfe9527f8bc92282b6748126e51ccdbfe15c5db89b39de4b0bb3720445dcdcb
CompactCells491To522.Box510.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:13288-13299	5cf68642652e93ecce889b87aef101ec8bc33fccf0ab6a784fc7d8efce874c74
CompactCells491To522.Box511.QInterval	CompactCells491To522.lean:13300-13302	a12628c5718985f0076bb25e9a86b6a5ac8cc56acf51958dbcf9b66378f1140c
CompactCells491To522.Box511.lambdaInterval	CompactCells491To522.lean:13303-13305	4ffc674af01b0964497fc7069e50a1423349767d0c4d2bd7c01a31981d63330c
CompactCells491To522.Box511.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:13952-13963	7c8c6b86fd6daf54ef14111d6510724fa6d6f63a77638b75d608d0abc4e78a9
CompactCells491To522.Box512.QInterval	CompactCells491To522.lean:13964-13966	26d4a08b21e2a86ff64d6c7d7532144a6b411726330f78fb4d963712642cad0e
CompactCells491To522.Box512.lambdaInterval	CompactCells491To522.lean:13967-13969	151dac92c7917491b58659816d638a024972a97f908224432d2bca219921217
CompactCells491To522.Box512.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:14616-14627	e2948e4453db05175c56a2eb6bf0f43722556d7c990d68942351b917264540d9

Declaration	Exact source	SHA-256
CompactCells491To522.Box513.QInterval	CompactCells491To522.lean:14628-14630	0642c5b8ee59fd81f0fb38c2d0e5d3a2f530ac51c19aaed58449abd18bc108e0
CompactCells491To522.Box513.lambdInterval	CompactCells491To522.lean:14631-14633	3de63310ff71711ef8d46ceed93a52d3127a88d12364984cd3e1e55bc4e0aced
CompactCells491To522.Box513.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:15280-15291	fa536e63ef246cdec10a02d5d2c4b1467b4c86dc37ce065982a54deea0ad4033
CompactCells491To522.Box514.QInterval	CompactCells491To522.lean:15292-15294	d0d17083d32f93c946f9471341f39b1ff5f49c895051289657d57f5dbac8bdec
CompactCells491To522.Box514.lambdInterval	CompactCells491To522.lean:15295-15297	6a282e1b162b68b99618593f8b673838e693a3b89311a1b6dc95e8e4a5ac5e80
CompactCells491To522.Box514.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:15944-15955	c6c52c39666a9c934e92e42001e235e25791f25a35317c36089e6f501b9d810d
CompactCells491To522.Box515.QInterval	CompactCells491To522.lean:15956-15958	2461fb5f08b2041335564dbe4ae8df56c3b17e0fd01f21cdce4af0a8b879b0d1
CompactCells491To522.Box515.lambdInterval	CompactCells491To522.lean:15959-15961	efdee65c029d838f8eb279b5b133e8be2b71792e8c5d30b9b5e6b1edef4df07a
CompactCells491To522.Box515.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:16608-16619	76d9ea707a099440b814084e72ef82dd9a0d577d10048936938cc778ba1bdac9
CompactCells491To522.Box516.QInterval	CompactCells491To522.lean:16620-16622	16094adfc729caecef3cf807861d5795d16821c2aabe7fbe736161d13a311db0
CompactCells491To522.Box516.lambdInterval	CompactCells491To522.lean:16623-16625	2551144c48e07476e57353193c53e0814fe34d98b9954ba59efe19aef677824a
CompactCells491To522.Box516.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:17272-17283	f73f37d2abd4bcd7246ced201dabad7ed8fb56233f0a073bb0de3303ac45244
CompactCells491To522.Box517.QInterval	CompactCells491To522.lean:17284-17286	d4ba99887fe13e0b741d505d092a1c34d304f3195370f9c5ee057edb3775b4cf
CompactCells491To522.Box517.lambdInterval	CompactCells491To522.lean:17287-17289	1e21b162d13f5050cabce5066ca725c62b88b711f2cae8f65f2154238be2a526
CompactCells491To522.Box517.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:17936-17947	e3c41ecaee6e9322cfc6b66d3ffedca9f3d5f818cc985484e8bd9700c72b727b
CompactCells491To522.Box518.QInterval	CompactCells491To522.lean:17948-17950	44bea5e4034d311c54d4c0973d2ff1a491e005ff1342bff3e808709227b44dc9
CompactCells491To522.Box518.lambdInterval	CompactCells491To522.lean:17951-17953	4265787d49a047dda2ea530e7085eb66561d0e6c018cfb1af32da099557b8c1d
CompactCells491To522.Box518.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:18600-18611	7c96c7a4cd3e87e955c1b49363e8f20e172e78e50b5b02c797ef7bed2b44014a
CompactCells491To522.Box519.QInterval	CompactCells491To522.lean:18612-18614	ac096f39d01e31b9de0421fd16f39b69103957c5e7e1539a7becf564af7c840b
CompactCells491To522.Box519.lambdInterval	CompactCells491To522.lean:18615-18617	34f23b7694e2c369f30cc0a0603017b6043a40afaadd6b58e9760c7d6a7c66201
CompactCells491To522.Box519.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:19264-19275	2bc12ed36dae2a32cdfd922f4e4c5ffe8adb77ff3f810609485df5ff8d68272a
CompactCells491To522.Box520.QInterval	CompactCells491To522.lean:19276-19278	d6f6f70cbaedef22c98422eb8f56976c19c6f7edd04ad4d367ff1e2025ae3b87
CompactCells491To522.Box520.lambdInterval	CompactCells491To522.lean:19279-19281	2c64024d16f4c7da10b53b50dee24b3b704176018f4e7d468fd01a9e86f21a5a

Declaration	Exact source	SHA-256
CompactCells491To522.Box520.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:19928-19939	4b6ee823012c1296d201162c2516e7307a794f5554ca9f3a1a7d3344b328a4f7
CompactCells491To522.Box521.QInterval	CompactCells491To522.lean:19940-19942	a513b4fa03e98276206e6da426ee7d860bf3eb3ce96fc8123c187f5d5d9737a
CompactCells491To522.Box521.lambdaInterval	CompactCells491To522.lean:19943-19945	a0cc9c5d2f98e1e1d08f4e7fe774c96692dfc5fb13148ddab2e3ef1e700aee81
CompactCells491To522.Box521.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:20592-20603	d2b4c84a92ac35f8a151b2a3f8daeebb57f662fa945f9f269ec796bef4f10f36
CompactCells491To522.Box522.QInterval	CompactCells491To522.lean:20604-20606	807a5fc228b8e856ed363d33085f118bd22955567a1635933a1b4f26a60af0c5
CompactCells491To522.Box522.lambdaInterval	CompactCells491To522.lean:20607-20609	051dd504923c14faa3f1079396305980eaa8d8eed564cd70f981ed4183f031ca1
CompactCells491To522.Box522.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:21256-21267	26d358f0ed666e8b52112629d6f56e22b4d60cbe6fa08c9ccd706cc6b8076b82
CompactCells491To522.bboxes491To522_candidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:21268-21498	519a5a87e0161528802dcf4bb2eeb37fef001868a1a13132b239753a9bcd9fb
CompactCells491To522.bboxes0To522_candidate_not_isWeightedPerimeterMinimizer	CompactCells491To522.lean:21499-21512	bc47b736499c0e8fe846f5b70579af4c1cb5a61b2f667e3f4ca4ed5ba96ea1be
CompactCells523To554.Box523.QInterval	CompactCells523To554.lean:20-22	491ad0c954a58e1d93f4960aa0caa317b81c572669a7ff39c2b86a9c6e2d782f
CompactCells523To554.Box523.lambdaInterval	CompactCells523To554.lean:23-25	749b37b26a68ac274412454ba05acc4b7a453cb704eeffe1bf2e2fdabab60c6f
CompactCells523To554.Box523.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:672-683	f5543b354075724095ff656effcf47c4b264a111745db07a8a6b065b36d991b3
CompactCells523To554.Box524.QInterval	CompactCells523To554.lean:684-686	ef2154b2be4689316df39112cf03270df7394ca158b922439e842a6622b1ae00
CompactCells523To554.Box524.lambdaInterval	CompactCells523To554.lean:687-689	8c8bd6fe9b029c83bb3b0a1dbe9d6e35af6fa80ab9af501373bf6686e3d29d0d
CompactCells523To554.Box524.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:1336-1347	3ac558b3ff73ca184f84d29b48fd93cb9cc8fc1303dee6feececeda465f3e910
CompactCells523To554.Box525.QInterval	CompactCells523To554.lean:1348-1350	d45aa2d92173761f68df21e4ed2c129deca8a64a6ebb82cc5046128477bb2c95
CompactCells523To554.Box525.lambdaInterval	CompactCells523To554.lean:1351-1353	2ffdd19c5ff2105207cb59676f89af03a58386f1be97817f05063cfc1bcf09ea
CompactCells523To554.Box525.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:2000-2011	e07d39712cf76f673a880ea6a5aca8fd7fdf440b5edf2f256a147bbd909d517f
CompactCells523To554.Box526.QInterval	CompactCells523To554.lean:2012-2014	f554546977bc59de940485a81d3fec6a15cf67ac5b5bafdac7489f7d14fbb67
CompactCells523To554.Box526.lambdaInterval	CompactCells523To554.lean:2015-2017	aa0fbab013b5207891939b9e9a8378ad6d84836ce85d776c0ffb3fd325c5b13
CompactCells523To554.Box526.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:2664-2675	287b9cc5d4d55655c8e98609ec7ba10fedf1fffe4337746002648fa3f722f3dd
CompactCells523To554.Box527.QInterval	CompactCells523To554.lean:2676-2678	2e2a7ee0cfffeda90fd8ece61a419762cc1b8f5741124e9d65029777b29620de5

Declaration	Exact source	SHA-256
CompactCells523To554.Box527.lambdaiInterval	CompactCells523To554.lean:2679-2681	290b39b1acbf766549d534c68f8190ef1df8dae0865a1675ca2ed0fde3980dfd1
CompactCells523To554.Box527.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:3328-3339	8735e25ae951e366240b621c61c38c3015696ed1e8db9304d209c2925bf8ce57
CompactCells523To554.Box528.QInterval	CompactCells523To554.lean:3340-3342	62a3904f9cf7d36d188b3e32d4a5d8012b038395cbeac6054afc2ae1b751298b
CompactCells523To554.Box528.lambdaiInterval	CompactCells523To554.lean:3343-3345	731df43aa4889377175aa20e4e2004d6d482130f10de83fdc45258c6632a5779
CompactCells523To554.Box528.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:3992-4003	03c526a0cd987ef8a1e8ead8fe65d9a8bf9fe324917a12f267ec4c30c4617ca8
CompactCells523To554.Box529.QInterval	CompactCells523To554.lean:4004-4006	191bbe5b5ba16aa54b3f36ced64c4aeebbf26b13102f020c0f91952afd7de2a8
CompactCells523To554.Box529.lambdaiInterval	CompactCells523To554.lean:4007-4009	5839d6a96c2d2ce2e9e53db31ec696bfdae bfc5942929bd4511c65be02cc75af
CompactCells523To554.Box529.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:4656-4667	9c32af118924d4315094bfcf8066649647a34c5c41cdd442afe95f1848ff147f
CompactCells523To554.Box530.QInterval	CompactCells523To554.lean:4668-4670	237df7f577ffa2ea8180caa48f184534c5c792b97bac13a90e1a81501d0bf8b9
CompactCells523To554.Box530.lambdaiInterval	CompactCells523To554.lean:4671-4673	f68f75fc63e507f5310517bef58927743ce49c76e5e37b5aa36c1c2ecb8c89ef
CompactCells523To554.Box530.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:5320-5331	1dd1df67e9406a54d13c9a4199c5bb283d7a8ca330a99ab11f7a36f4f7532607
CompactCells523To554.Box531.QInterval	CompactCells523To554.lean:5332-5334	f7c533ff6cc52e54171b4de760dbd3fd87375ca37b50cdf9a5cdaef0ebc4ae40
CompactCells523To554.Box531.lambdaiInterval	CompactCells523To554.lean:5335-5337	83247397af41217377de1aaa4b9fa58a6d4356244f6fc9c1a5e94eb3b93d3fe5
CompactCells523To554.Box531.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:5984-5995	1d95640c82ae63efbe2e51e6b082cbba0596849ffd4c5e644e1fe5138f122f1c
CompactCells523To554.Box532.QInterval	CompactCells523To554.lean:5996-5998	14732dadbd294741d67fb90c92a9b4ca94836901f84d61b140a9ed0724d90abf9
CompactCells523To554.Box532.lambdaiInterval	CompactCells523To554.lean:5999-6001	9c0b5d51c35ec7a20cf95dbba92c7ff12ee7025d30509b4db494b8c21b1fac52
CompactCells523To554.Box532.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:6648-6659	498941a89413bb72e660c846941f6254156dad2d5bd8d5e6751c0b0185b9e6e8
CompactCells523To554.Box533.QInterval	CompactCells523To554.lean:6660-6662	7c25cc10392b4d45d9f92e84a74b6edcde87fe558473d13137187ae0f8767bde
CompactCells523To554.Box533.lambdaiInterval	CompactCells523To554.lean:6663-6665	a6aa2b6f236a3650e84dd6ef52de0689ebae f78d3d7f33806f277df5a80433a0
CompactCells523To554.Box533.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:7312-7323	a00c3db76568afd4a466eb5de8f028502cbdd7ad980e6f8cef08456889f31500
CompactCells523To554.Box534.QInterval	CompactCells523To554.lean:7324-7326	ab466863750b3f2e669c347e9691b60f6b4d29933db7e2bbb791b2be48c841d3
CompactCells523To554.Box534.lambdaiInterval	CompactCells523To554.lean:7327-7329	0948827390378bde12998918e42b3cfc42d4a7ae9779a66bd2b4e2875fe7b48f

Declaration	Exact source	SHA-256
CompactCells523To554.Box534.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:7976-7987	dccd254836f9daff39e0b1b8f3865425fcff4aaaae069b9d056eeb15f4d40b69
CompactCells523To554.Box535.QInterval	CompactCells523To554.lean:7988-7990	596d377e4cc474702c98b615fe5c42bdbb47ea4d0ecaa35293e36eba005082cc
CompactCells523To554.Box535.lambdaInterval	CompactCells523To554.lean:7991-7993	759fb57df18fa9f245043b3297d3819bdf21f1bb9c811fb2c7767e1be19b0eed
CompactCells523To554.Box535.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:8640-8651	d23281126a737df8ab45b7f3df1b1e1b02125f329f16e9293a8b2a938e7100a8
CompactCells523To554.Box536.QInterval	CompactCells523To554.lean:8652-8654	7f3f161da8d5fd9d0208a604f562c0057f72469df99f58fd83ebbf8070c81061
CompactCells523To554.Box536.lambdaInterval	CompactCells523To554.lean:8655-8657	9839d68bb2cdf9d4b598dfbad3b7e611b7cdb8481473dfa6fc4e976c567e1214
CompactCells523To554.Box536.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:9304-9315	35b8a03bdf1b53205ab28a3cee82d1cc2ef192b6d0bf253f582b99ca7322a13a
CompactCells523To554.Box537.QInterval	CompactCells523To554.lean:9316-9318	c319ac4535f4ba85e879356c25065c1675b08de4930ad9e6985af50cd358e1f7
CompactCells523To554.Box537.lambdaInterval	CompactCells523To554.lean:9319-9321	2ad4bece67db09e997b7a61f77544efb23dd5812574037ff2bb7b7d63c3daae1
CompactCells523To554.Box537.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:9968-9979	b7d735c5ca50de0f9d84c7b7b6a8943ffb157d92424ab81548d88511098e211c
CompactCells523To554.Box538.QInterval	CompactCells523To554.lean:9980-9982	8606b13d6dbf89d59327dea11322a6e2434745be4685282b258777182e9808c
CompactCells523To554.Box538.lambdaInterval	CompactCells523To554.lean:9983-9985	e993d68629b7442f7de22a3b4287d6982f0c828e56cc394e354bca4b2c34409d
CompactCells523To554.Box538.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:10632-10643	e9d8709c6a4ff80da906505d30addf6e480d6aeada78f87e2492f082feb79aaa
CompactCells523To554.Box539.QInterval	CompactCells523To554.lean:10644-10646	fc0b023bf0ee51446cb8b1bd31eb6898da7080a09b6bd42ddd0abc6e80174672
CompactCells523To554.Box539.lambdaInterval	CompactCells523To554.lean:10647-10649	cf017cd56090e1f47c7914654afd524d9f2f97dbabee36fbdcf345b8709f0e7a
CompactCells523To554.Box539.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:11296-11307	92589a07bac32083de8ea61b609bdbadf19dba8f1de4d46c25cfdc06b85d61d2
CompactCells523To554.Box540.QInterval	CompactCells523To554.lean:11308-11310	a87209bb1a157b5350403a06c6d45b9215b5abc4d197a3051efcb01669020fbf
CompactCells523To554.Box540.lambdaInterval	CompactCells523To554.lean:11311-11313	76bc951e10c788f3922695c0d0bafdf3bf5d60a17a166f7bd2be95c831a35e4e7
CompactCells523To554.Box540.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:11960-11971	23df5ead564a4e962654236cd5e5e094b1cd74b4c4ced661ab70f540efa56608
CompactCells523To554.Box541.QInterval	CompactCells523To554.lean:11972-11974	0508a4f3fbc8b856de0d7d57aa1cf639ca3edbac4271ac3cd4ce9fa67883354a
CompactCells523To554.Box541.lambdaInterval	CompactCells523To554.lean:11975-11977	8a4534ad20dcf0c030ccf54ad3597135154f09fda67cefe7f3c4cd4a380fad44
CompactCells523To554.Box541.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:12624-12635	69838fdcf6365f7255ad0bb8890fd640e18110a955286086a368e50d18299d4

Declaration	Exact source	SHA-256
CompactCells523To554.Box542.QInterval	CompactCells523To554.lean:12636-12638	87c52696b14a5c79fd58671fbfbaa531f3e9e1ed2a5044a5772f0bb51182ea7e
CompactCells523To554.Box542.lambdInterval	CompactCells523To554.lean:12639-12641	cc90f290b652ce9e24edfe1a19cc015797cd7a55c8bccaf8323cb3a3af7dfae3
CompactCells523To554.Box542.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:13288-13299	bc35d864bb9ec802fae94f1bdb9376152a3febe834b0bf5c6c75eccc6b89a42
CompactCells523To554.Box543.QInterval	CompactCells523To554.lean:13300-13302	1c2a04be20f1fb98308e0bf565d56f4fe7132c398ebe7c6f4a4342c5fb6fe56c
CompactCells523To554.Box543.lambdInterval	CompactCells523To554.lean:13303-13305	fcf554e9f5cf60dc22be62b3533630204259375a962a479a26ed42e9628b4e9f
CompactCells523To554.Box543.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:13952-13963	940a1ab80bb7cccfb8ec7e357f4ce9fa81daf48f6c065bd3b61fcae0d63fe95f
CompactCells523To554.Box544.QInterval	CompactCells523To554.lean:13964-13966	a93cc435ea21f9759fad8ccf8b2d39c85a385af22c5212294169086a288da456
CompactCells523To554.Box544.lambdInterval	CompactCells523To554.lean:13967-13969	2a40fce765afe1ef4030c15aa6bdc03dd94c6bc1691f64e224de0c1f1890c254
CompactCells523To554.Box544.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:14616-14627	44480c8f4c604ca53ee549faa1f5229be4c33d3fd0dec1a315571c07cfe5e58c
CompactCells523To554.Box545.QInterval	CompactCells523To554.lean:14628-14630	7cadab5643baa4f38116d21a9fed25f66f561c024048e762c26190eac66d86fc
CompactCells523To554.Box545.lambdInterval	CompactCells523To554.lean:14631-14633	c9e1e9a322612696cae72e19df732e4037c052be77d087c8dc53e879cc1d351e
CompactCells523To554.Box545.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:15280-15291	e037839ca75136ebfefe0e5adf2e4420d214cc7662baae90caf8b6d8af9817d2
CompactCells523To554.Box546.QInterval	CompactCells523To554.lean:15292-15294	0ac28d867e21188fb2d84bdd6e90d07c456cb7ae3cf475f335f3b5fffb81eed17
CompactCells523To554.Box546.lambdInterval	CompactCells523To554.lean:15295-15297	a769e3b2913d0dd984f5b2149129539c1708989722896630258439bbb695b759
CompactCells523To554.Box546.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:15944-15955	7007e4f7cc8806aa89aa27fa3b9b2885b5bb5a7a355ba32c4bb57296560867c2
CompactCells523To554.Box547.QInterval	CompactCells523To554.lean:15956-15958	42295d7683e2c8399a0560922aa028dca2891406ff4e53c287aca21afa1708b0
CompactCells523To554.Box547.lambdInterval	CompactCells523To554.lean:15959-15961	0121c63ba2e181370cfc486a6f485653d9da5c7649cabfb12efdcc4e51e143c1
CompactCells523To554.Box547.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:16608-16619	340fe769b94abd0172bd72193685dea6b872143b3f17f933df6bbccd1942b3dd
CompactCells523To554.Box548.QInterval	CompactCells523To554.lean:16620-16622	a3cc49035b0658228f310feb485ce7ab59c97618bcfda9c512ee42c5aefa7c7c
CompactCells523To554.Box548.lambdInterval	CompactCells523To554.lean:16623-16625	637c2feb9a3f1747a4a58cf49c4bbe3f6417c12741869c7892099319b70ef811
CompactCells523To554.Box548.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:17272-17283	96daaddfd520cfd965088ca940986a0153446d078fd008baea4d790f3a9e445c
CompactCells523To554.Box549.QInterval	CompactCells523To554.lean:17284-17286	a3a2da63eb00fb9f464995ef10a17337d19ed1ae002277be02e8d9081c5faf21
CompactCells523To554.Box549.lambdInterval	CompactCells523To554.lean:17287-17289	a52c63c374ac6e6ba12f77900f28474643f0a6bb360c0f65bdfbeb6875067d61

Declaration	Exact source	SHA-256
CompactCells523To554.Box549.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:17936-17947	a91141c652b0bb4e29448dc3de1bada620831dfab2e1b610d658f7d488abc56f
CompactCells523To554.Box550.QInterval	CompactCells523To554.lean:17948-17950	bbc099caa2eac28ef598ae1461a40a56b00d7114c4ae416b59d50a945fcf3873
CompactCells523To554.Box550.lambdaInterval	CompactCells523To554.lean:17951-17953	392331f024cf15271f3321a546849670f7349a37e8aed6061c73185f40a28858
CompactCells523To554.Box550.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:18600-18611	0103dd1564cdf03058cd8ce7f20ec29aae9364e5291815d5c7a8d92cd9049742
CompactCells523To554.Box551.QInterval	CompactCells523To554.lean:18612-18614	551751d8219c0b8aa707cf812a632d489f749e856a6e5bb10fe145913f771c26
CompactCells523To554.Box551.lambdaInterval	CompactCells523To554.lean:18615-18617	20ecffaa8f8c194ffff7a64d3b2c1c59e6d460ecea36d80d41a0f2b479a50285a
CompactCells523To554.Box551.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:19264-19275	56fb7bcdcb839f714b10bce1c62598b9252a553704c46fe18391656ea7a98acf
CompactCells523To554.Box552.QInterval	CompactCells523To554.lean:19276-19278	8479308034df9ec213cafcd19e6090b95ab6aa708ed17df12224e03231eb3a7f
CompactCells523To554.Box552.lambdaInterval	CompactCells523To554.lean:19279-19281	3e7a851bebee38f641e41f787c13675343e4d081a9068e8a344f5b33965b6b6a
CompactCells523To554.Box552.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:19928-19939	3bab700b2f1d9e74f83cb8da301a73a531bc194f73b673a8bd90ac2be5580e2b
CompactCells523To554.Box553.QInterval	CompactCells523To554.lean:19940-19942	d359dca3221598bf9d338112b156142f6fe809bf76b8c0f9acd970e21a5eaf85
CompactCells523To554.Box553.lambdaInterval	CompactCells523To554.lean:19943-19945	f86870775c9e10b86e7af76db986c429a53072650747519fe6d7b082322c5b09
CompactCells523To554.Box553.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:20592-20603	64fde315ead0a76ca76a7b5bb5ae905573b444a4f7bc70f9c4b9ba6783fffd2dd
CompactCells523To554.Box554.QInterval	CompactCells523To554.lean:20604-20606	1de5339ad93fda53e0b32fd45e7466f706a92836be5459ec1bfb1091f0346372
CompactCells523To554.Box554.lambdaInterval	CompactCells523To554.lean:20607-20609	99294e6ee935279fe992ec61dae99d5c5b437af4394b01f9f24c434a581495a9
CompactCells523To554.Box554.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:21256-21267	4258c2f5442d9897495c1f8051e43663c5f9e52f0efa3e508148c4aed2d066cd
CompactCells523To554.boxes523To554_candidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:21268-21498	6f287b6ceb7989f7dc1265f35f2c2c2c4a5d1c0db2babcdc1fa6457c20ab15d8
CompactCells523To554.boxes0To554_candidate_not_isWeightedPerimeterMinimizer	CompactCells523To554.lean:21499-21512	a7689b4b3d8bc3beaa1b72b84f2f8c1147de017e3e7919dbd671c8ff9d0854b5
CompactCells555To586.Box555.QInterval	CompactCells555To586.lean:20-22	d404dac7b9282781354c4a6e85a233b1dc5f66c1e9a1eec0de36a13a8c57bc9c
CompactCells555To586.Box555.lambdaInterval	CompactCells555To586.lean:23-25	5d4fefc977bf40262990560fe33d06adfdcc565cff63c5756af78621ed7b10c
CompactCells555To586.Box555.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:672-683	e4fa023ec42768a04ec999b66236a00f7a799f8cc90bcc68b7de2bca5f32230c
CompactCells555To586.Box556.QInterval	CompactCells555To586.lean:684-686	5cd5db36bac943521617da54b9522ad0a4d579ae0f1e41eba477c5be18d210e5

Declaration	Exact source	SHA-256
CompactCells555To586.Box556.lambdaInterval	CompactCells555To586.lean:687-689	37ab701a70c472504b2871eae b245f8028f3393b1798f4d071e16efc5b2e717b
CompactCells555To586.Box556.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:1336-1347	897e448cc1b48d6f1754fcab48bba189e0091b729d184635af6f7ce74578224c
CompactCells555To586.Box557.QInterval	CompactCells555To586.lean:1348-1350	bacaf8920d281513f3b4c8e024c01187351fbd968d3ec836d62b49fa97dd596d
CompactCells555To586.Box557.lambdaInterval	CompactCells555To586.lean:1351-1353	921e58143f1b7c9d8d14e72b4e89a702a6605f7bc886e238ff6623896ea004d8
CompactCells555To586.Box557.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:2000-2011	e1299e5432c990437588c21477f85e813d9cec98c5d495387ba9cb89989edd83
CompactCells555To586.Box558.QInterval	CompactCells555To586.lean:2012-2014	8358c3c26511cbbd3a3957344459928474e208724d63197085afb378cb605ee6
CompactCells555To586.Box558.lambdaInterval	CompactCells555To586.lean:2015-2017	009293198687a1a3c084947a1586cb91ef84ebda7da546247031a78b96b181bc
CompactCells555To586.Box558.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:2664-2675	2b2a5a3435a90e613c6e88bc394a47d139b4e454287d2d20c2878ea402a69657
CompactCells555To586.Box559.QInterval	CompactCells555To586.lean:2676-2678	aecc6134782b14aeba67e68b35fa79c7c104d7470b305aec99945543d8e3d84d
CompactCells555To586.Box559.lambdaInterval	CompactCells555To586.lean:2679-2681	882a9170aab04fc3dafed0ed0aa8e332813e0bb199ec619153bd16f7de295f2d
CompactCells555To586.Box559.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:3328-3339	377c8e905a6c84790b2b9d330c610dfb52680bfd219e9d5b7cf8c5ee00f9c067
CompactCells555To586.Box560.QInterval	CompactCells555To586.lean:3340-3342	e6c1a090f9506653a8f8a8ff1f4fd9e5cdb1a28f3f6bb4872593f82fe855d3453
CompactCells555To586.Box560.lambdaInterval	CompactCells555To586.lean:3343-3345	040c07287418b3f009533e7d1b448bfff58bbea8132903177863f28dd896c001
CompactCells555To586.Box560.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:3992-4003	5ef6d55630f4fcdcbf4abfec2e64f20aca8e8ffc0a2c840ece4bc251cb2e92b1
CompactCells555To586.Box561.QInterval	CompactCells555To586.lean:4004-4006	72ef4ef789a95d9b4d844d9295a4f5e9efdaf2c4854e8bac23307e9e6cd72089
CompactCells555To586.Box561.lambdaInterval	CompactCells555To586.lean:4007-4009	b7d7e9f7f26dd470d8713cd887546cddc37a9779a65fdb45d64faf71cb1bc774
CompactCells555To586.Box561.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:4656-4667	c99ae54d7b530872505eef19c3fb557b7b38b058fd4d5e35f558585f4a691994
CompactCells555To586.Box562.QInterval	CompactCells555To586.lean:4668-4670	118e9458ad33a5dc4f32950d3b4db71a01f26a82977b03c1e2827c88823c797b
CompactCells555To586.Box562.lambdaInterval	CompactCells555To586.lean:4671-4673	a58ba6874b8c5ec3d77be516293b14e6cc52906903c0114f46dd0a8bff805e0c
CompactCells555To586.Box562.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:5320-5331	2a5625ca8638989a9dcd335a7e24298d27f2b9a266a0596bc6c326cbd1276b81
CompactCells555To586.Box563.QInterval	CompactCells555To586.lean:5332-5334	071a99335484ad08e4897b19646afb5496eb78ab17d2a55fd0e008d799ab9ff
CompactCells555To586.Box563.lambdaInterval	CompactCells555To586.lean:5335-5337	0982089e75d6ae768d188477d7b20c5a9fcbd3da029c0d0410d877282e0d0394

Declaration	Exact source	SHA-256
CompactCells555To586.Box563.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:5984-5995	ec78db004e071147bc18031cd806d08473a3824ed53e5e9d1ac67215577bb6f9
CompactCells555To586.Box564.QInterval	CompactCells555To586.lean:5996-5998	6502df884b4de2bb9ee9c82a8ce4588cf71dc240b4e287513eb942fd6b616ce4
CompactCells555To586.Box564.lambdaInterval	CompactCells555To586.lean:5999-6001	3a94a485742d7e7bd081ab48fd86dcc097d06338988d1589a0bcf43c91033f31
CompactCells555To586.Box564.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:6648-6659	d696c4d3b873cfaa3670a0eb2f4f8c bfb606ac63fdd0f2d461b5843cf8652b6f
CompactCells555To586.Box565.QInterval	CompactCells555To586.lean:6660-6662	a12ec0bc9652d16efa21afd0a5f912e64683797a3d832bdacf7068ba04b976c3
CompactCells555To586.Box565.lambdaInterval	CompactCells555To586.lean:6663-6665	f3c992e473cd2bcc8e4cb8704efa21740fe727ed587ac40fb7530f8547dc607f
CompactCells555To586.Box565.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:7312-7323	842702280c7c307a8dfbcd57e4912152a63a9732cfae8847e471cf3ed7df7a35
CompactCells555To586.Box566.QInterval	CompactCells555To586.lean:7324-7326	bc56cd04e97eccd4e3c7fbfc729ceae706eb5f89f8a475fc2fb0dddf216d5553a
CompactCells555To586.Box566.lambdaInterval	CompactCells555To586.lean:7327-7329	c7d29c38f8e640a9f088a0ea67b35ced4b69b1b2e5ad7d2ca9ba2427c3390c0d
CompactCells555To586.Box566.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:7976-7987	8e079bd29013a5be2e2716b19765e89dc01316c825ce4b38aa50ce0625f86e2f
CompactCells555To586.Box567.QInterval	CompactCells555To586.lean:7988-7990	8359b78af999ec598985229a1444aaaa020529e4a12931e47ae14750ed047f78
CompactCells555To586.Box567.lambdaInterval	CompactCells555To586.lean:7991-7993	918936c6d2b5c5c3c621519bb34b8ce398bf14885ac4f9740904fd97532e61f9
CompactCells555To586.Box567.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:8640-8651	51447a0466681869a4a7b3a65a5843cd25c9c69a8288991d142c1fe137c9974b
CompactCells555To586.Box568.QInterval	CompactCells555To586.lean:8652-8654	8770152fb901fdb8068e4ca66027a172f75e6603444129dda2d7135a719269ba
CompactCells555To586.Box568.lambdaInterval	CompactCells555To586.lean:8655-8657	88e7fc81f0aa9c9bb2e9be0aef4fcc14f355a9e2d894509a36764be5966b29f5
CompactCells555To586.Box568.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:9304-9315	200302c045c4198ae508b0dd077dbd588648a34c9921444f219ed77789b6caa5
CompactCells555To586.Box569.QInterval	CompactCells555To586.lean:9316-9318	a8c241b6b408677a0764b2d33a488b707ef0ce03ca1fa1ff25511968d8874982
CompactCells555To586.Box569.lambdaInterval	CompactCells555To586.lean:9319-9321	ab4dd9fc8ac681d406e156cf6500f057de5bfbecd50fecbf2345469e11b2faae
CompactCells555To586.Box569.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:9968-9979	7686f29da535f523a12f492e019cf090dc884310e9a8ae0042dbe2b1a33f2823
CompactCells555To586.Box570.QInterval	CompactCells555To586.lean:9980-9982	6a134a5ca33b27be64f827c2cb2ad2052244e8c0c43316e4c9a533104ebead98
CompactCells555To586.Box570.lambdaInterval	CompactCells555To586.lean:9983-9985	e1054a9c883cf5569128a49dd7fb294c8209191a3a4e6a1587aab7e8d3c48e5d
CompactCells555To586.Box570.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:10632-10643	55479c5d395cc6959440a1dbe46253957eb7dc32c828363e49945e4af5373d92

Declaration	Exact source	SHA-256
CompactCells555To586.Box571.QInterval	CompactCells555To586.lean:10644-10646	a81ca6412589d60a4d6e01c96574cfe6cad5b9ac2d5772e22ecef01bb7516c60
CompactCells555To586.Box571.lambdaInterval	CompactCells555To586.lean:10647-10649	1de55d384f9a6cc9f8a7095b06a5d3ee301015901d5d73fec13d48eab7bf0077
CompactCells555To586.Box571.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:11296-11307	325d59a769fd79a99c9d1faba68da21ad76d1f9567615cd37d8e726f299af71a
CompactCells555To586.Box572.QInterval	CompactCells555To586.lean:11308-11310	261964c29fa6b14759f8dd70675a4d16e191b718e9c2bf5c99a4a91c56576cf3
CompactCells555To586.Box572.lambdaInterval	CompactCells555To586.lean:11311-11313	d4741683f38a269acce93b96a2d0560496d59a02f677e876445ba68c49093f35
CompactCells555To586.Box572.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:11960-11971	b2c6403fb29f07c85baad4b28d799aac1f3a05d0e4287df47a5aa3aba51e652c
CompactCells555To586.Box573.QInterval	CompactCells555To586.lean:11972-11974	63fb9da3597ba2046d86904c2519973446d6b1d04ff0a3566aa49a79e331ffef
CompactCells555To586.Box573.lambdaInterval	CompactCells555To586.lean:11975-11977	fb418b6f5c09435e8e0ee9b959f0000ceb3658f968c73175421135005d51952d
CompactCells555To586.Box573.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:12624-12635	3cea8a6a7ed6ff5f8e08b29401a96feed77a94781970436c5b324cadaf1e9fde
CompactCells555To586.Box574.QInterval	CompactCells555To586.lean:12636-12638	5110b1f64ccdc7f6c4bcf21964813722d93d3ab5212b39111938f749f863e63f
CompactCells555To586.Box574.lambdaInterval	CompactCells555To586.lean:12639-12641	c13bfe9892bad72848416c23b80104f8a193491011d2c976532e8d74e9b464bd
CompactCells555To586.Box574.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:13288-13299	479715e2f06c9258d2bba5c0fa648f7b7c61686056f220c3ab7d0e1dcf735ceb
CompactCells555To586.Box575.QInterval	CompactCells555To586.lean:13300-13302	8bfdde00716c5f23a33798a2a9f08378e32c4bf533643f29e555b74732f3c0b8
CompactCells555To586.Box575.lambdaInterval	CompactCells555To586.lean:13303-13305	408b01e87343a7cb6f9c12692603402a1f1199cab62d2b114f2972dc3a42b422
CompactCells555To586.Box575.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:13952-13963	01024732efaebf5eeb94e7faa24bb5c1f108e80d21ce7538d665b9872862b057
CompactCells555To586.Box576.QInterval	CompactCells555To586.lean:13964-13966	0c484b45991e080e2e5e205104cb1bd3463a10d76bd4c13087fb03cf1311fcd9
CompactCells555To586.Box576.lambdaInterval	CompactCells555To586.lean:13967-13969	e01439800a478cbd260adfcc6785ab28d1f2b48596c955044100997c705c63c9
CompactCells555To586.Box576.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:14616-14627	052cdeaa21c430f113b500887170fd97f6b4473ef82c8ad0681a1a50e38ff433
CompactCells555To586.Box577.QInterval	CompactCells555To586.lean:14628-14630	323b04d30dab82c63362a0ff357bd4cf40ac85fd04755c3a5de2f6195c936825
CompactCells555To586.Box577.lambdaInterval	CompactCells555To586.lean:14631-14633	7b1ff7b4f51c6c7bf8cc64dd07fb311d86c312458b5594230721af4578d75990
CompactCells555To586.Box577.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:15280-15291	36234d4952f9b2edcd0bffa5332ee59ffa2831298a6a565b32d0f2594ec3c4cd
CompactCells555To586.Box578.QInterval	CompactCells555To586.lean:15292-15294	653b1499c3418f5d04610a8f1ca7cc3216105048c616c8f80007c904a8b385af
CompactCells555To586.Box578.lambdaInterval	CompactCells555To586.lean:15295-15297	b323c5ba6e7e2a9c52b17665047cae64de3cee8a789eac402db4ac4e44c32f55

Declaration	Exact source	SHA-256
CompactCells555To586.Box578.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:15944-15955	93052f94ab80999790975d930e937e996645a0c32ff58c33bc07a50588403f46
CompactCells555To586.Box579.QInterval	CompactCells555To586.lean:15956-15958	5e83bf2b343105736c86c80158d1894f8ef496dc826631751aea71591785606e
CompactCells555To586.Box579.lambdaInterval	CompactCells555To586.lean:15959-15961	8af94576a7a1002fdccc9d9416847ea90a773c62c0c91ba869b5ec2ac2d514f1
CompactCells555To586.Box579.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:16608-16619	526aac93a2f28ff09204d0ec4afcadf07fd5fdf2cd7dbdd0b1b611d4896c8127
CompactCells555To586.Box580.QInterval	CompactCells555To586.lean:16620-16622	fe20773ce2d5b180d00214959ba0db56d3e4452b9ee429425cd3bfff07f49017
CompactCells555To586.Box580.lambdaInterval	CompactCells555To586.lean:16623-16625	7ad7bbf2ba5544230a66005ec11f47ee10a08229741c3dd349c41fb4e3da6623
CompactCells555To586.Box580.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:17272-17283	6eb87e101a3cd54db561529b9b515d870f75a3df5ff34ee64534a93d01ecbd64
CompactCells555To586.Box581.QInterval	CompactCells555To586.lean:17284-17286	121822657782ef0f050d9c4ad0e2fbefb71fd92634fda171e66d19b7bc4503d1
CompactCells555To586.Box581.lambdaInterval	CompactCells555To586.lean:17287-17289	5657852017e5f6a1f995cb73fd6b1fd7c8dd698c5666695ac9e6b408c1f408cd
CompactCells555To586.Box581.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:17936-17947	046b5b35df1eb0d291f70d71f594afe12641cc7c3733a5dde1c32d7564166647
CompactCells555To586.Box582.QInterval	CompactCells555To586.lean:17948-17950	561da3c1bef639dd86d84fa97220b22d1e74128cecb7a0a47b3e879f37653bd0
CompactCells555To586.Box582.lambdaInterval	CompactCells555To586.lean:17951-17953	299dd326fc02fa405796a7b4064fc1a6694ca25455656c7c6b5472aba21ccc4
CompactCells555To586.Box582.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:18600-18611	8a57cac0b5af64cbf08bd7ee28d68763c793e21cadf28715ba134ea0a6ee34cf
CompactCells555To586.Box583.QInterval	CompactCells555To586.lean:18612-18614	c88f2bb90f5fb504f4a042ac1edbc944cddb5ffd2db616afadcbf0326673a320
CompactCells555To586.Box583.lambdaInterval	CompactCells555To586.lean:18615-18617	57c1c7d85838d8e8786d06722b91fb52dedcf2cc4e10f3651fae00fa6b87e80e
CompactCells555To586.Box583.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:19264-19275	c94f13489c1b5c1a51eeceb02477b152cd44d55e69a07f0ea8584c68e7424d9b
CompactCells555To586.Box584.QInterval	CompactCells555To586.lean:19276-19278	06230eae9f07220b5e6a7cab94281b017df054cf464a0625f1eb4ede34450495
CompactCells555To586.Box584.lambdaInterval	CompactCells555To586.lean:19279-19281	e482cd20cd4cb3a46c8fa53a96254c378db78d4d3db9ea056641dbe7a64ee19f
CompactCells555To586.Box584.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:19928-19939	38ece1403b8193262d5d06be2879701d4aa856390ad8ef316664783902346b2f
CompactCells555To586.Box585.QInterval	CompactCells555To586.lean:19940-19942	540fb1107287be34e78a077a950b0b6abcef408f00498d511477d944be9dca4a
CompactCells555To586.Box585.lambdaInterval	CompactCells555To586.lean:19943-19945	46036b8de5452eedb5fd96b6dbddc022901efaaa0e6060ebabf916a31ca9c4a4
CompactCells555To586.Box585.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:20592-20603	b5a177149d26bafbd6934e68017c2d221209a1827d3059e01a73930c29834761

Declaration	Exact source	SHA-256
CompactCells555To586.Box586.QInterval	CompactCells555To586.lean:20604-20606	3281f4362b90534c9081afd43e94435a11e8dc33aa697621a5b412a3e1138117
CompactCells555To586.Box586.lambdaiInterval	CompactCells555To586.lean:20607-20609	90965c1b91578ee3bc5e4fda8e273b9cf9012aceec513ef66c55a3bd41d527d6
CompactCells555To586.Box586.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:21256-21267	b4a13aea8701da4fa1cffbb5a17c692f4398c3dbadcbb26608fbdf8b53a40e30
CompactCells555To586.boxes555To586_candidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:21268-21498	43695f63fc0a7b371c8a41b7e8ded42abf49092fed1430f50caa71594c86021e
CompactCells555To586.boxes0To586_candidate_not_isWeightedPerimeterMinimizer	CompactCells555To586.lean:21499-21512	4e0355d670f6fa8e166f437507c48e7ef4f5bdcaf99f36e6d5c963c0cd59f92d
CompactCells587To618.Box587.QInterval	CompactCells587To618.lean:20-22	51e87baeb57fe905a844fb478576697997eb53f500caa70ec1a7c26dad058b4b
CompactCells587To618.Box587.lambdaiInterval	CompactCells587To618.lean:23-25	6fb0699ad85a6683bf79e474d5d2bb8dc143fdae3d19dfed3935c56329676643
CompactCells587To618.Box587.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:672-683	452728d366df445942690325a3c490e0c3b29d6494e041c7798e45a9cdc5263c
CompactCells587To618.Box588.QInterval	CompactCells587To618.lean:684-686	8b74605a4578fb491980b4133a0b2f0ea3311e505ae4595e9c6fcd2882a63d96
CompactCells587To618.Box588.lambdaiInterval	CompactCells587To618.lean:687-689	accda278b063b66bb630c5eda8db9c67818b01d174df99626ef9d735d2a03cbd
CompactCells587To618.Box588.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:1336-1347	fd0a6cbe448ee48a0b516582f21ee5558d30b405c8e380fec6222d328540115c
CompactCells587To618.Box589.QInterval	CompactCells587To618.lean:1348-1350	4449f1c79d2fa3bb4ccacfca325ae4ea6fed204a3a61049c5d5bd0bc36c9555b
CompactCells587To618.Box589.lambdaiInterval	CompactCells587To618.lean:1351-1353	056d629663e9d02e20cd51a7379f2a588747e02fd1e014fc1b9d33133c6b27df
CompactCells587To618.Box589.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:2000-2011	5e426c81eafcca9561096af2d9c00603d018c8388754e6e70f264fb007f99c7b
CompactCells587To618.Box590.QInterval	CompactCells587To618.lean:2012-2014	d1e4b19f93979c2cdfa7ae1eb205d26719481503f05786dc71fc6b82a7f61848
CompactCells587To618.Box590.lambdaiInterval	CompactCells587To618.lean:2015-2017	b1979d11727affc8992cdf6e7a75cee2d1520c12e352eb19dfb420c7323e2428
CompactCells587To618.Box590.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:2664-2675	43ce0004d829282ecbf1abebfc0f800daa24827a5dfd57c7376c7ea92cd85699
CompactCells587To618.Box591.QInterval	CompactCells587To618.lean:2676-2678	8519ca41ac6f61c091860883cbe2591cb79a1f844afebda190d1ebc4ad329db34
CompactCells587To618.Box591.lambdaiInterval	CompactCells587To618.lean:2679-2681	ee2a712a43c7df31a7657f92e8780de38dec000471017cf8c4ada83205430831
CompactCells587To618.Box591.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:3328-3339	2e258d0a56e6e5c5228dd0552a004dac32a25920f2c178547e5c47913ecfaf01
CompactCells587To618.Box592.QInterval	CompactCells587To618.lean:3340-3342	88c07788c313bc2145b469244280664f223efbe66e5c03b2739775dc8739b0a4
CompactCells587To618.Box592.lambdaiInterval	CompactCells587To618.lean:3343-3345	4ee459fc1f08337ea2c00eafce607854afa7055d51b29162a6db8777b91f552b

Declaration	Exact source	SHA-256
CompactCells587To618.Box592.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:3992-4003	46ec382876cb7c9a51386a5ace3333c294d4fe34091183e8e567fb030df32616
CompactCells587To618.Box593.QInterval	CompactCells587To618.lean:4004-4006	3e9a268b445586e518435e01ef8683e2241bbddfa7c6e9980e22516b587206d8
CompactCells587To618.Box593.lambdaInterval	CompactCells587To618.lean:4007-4009	f5ec1a2570bc8c5f101ec414b9470e99801295a69337650a2a6d01e270a9357c
CompactCells587To618.Box593.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:4656-4667	fb37b8e21312665c94c73e82aed5f6a019f7594444c5cf6a18bb5f54c31cb8eb
CompactCells587To618.Box594.QInterval	CompactCells587To618.lean:4668-4670	eff90a32c96c2192fc22f0cdfc83ce187b9b2c8919697d389b4e28493e3be6e0
CompactCells587To618.Box594.lambdaInterval	CompactCells587To618.lean:4671-4673	e2b08004fed1476d2766912308804e8c566a0ee91234de488c2c6ecfa273b647
CompactCells587To618.Box594.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:5320-5331	73cbfcbba715b4859f10574217bf2eff444b6c80bcb3e2d3bd36399d1cab30c2
CompactCells587To618.Box595.QInterval	CompactCells587To618.lean:5332-5334	19b3cac8932b5041013920f91145ad25189b66aeea998117f3eb13ca0ce16a4e
CompactCells587To618.Box595.lambdaInterval	CompactCells587To618.lean:5335-5337	a87106007054015c8dbbe16ae2e86da72dd638c74766d243e2c19a5a48d8659b
CompactCells587To618.Box595.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:5984-5995	161e2aedcd1ba58010d8f6ebf7d81f044187cde1457eb0ca1e030d7f15fd4a02
CompactCells587To618.Box596.QInterval	CompactCells587To618.lean:5996-5998	472b151a1777192edc29a0349c7bb55186014d51f6f9fb7558b1919f87254921
CompactCells587To618.Box596.lambdaInterval	CompactCells587To618.lean:5999-6001	d3895eb746d004a2d40eb7e1c66e6c49962f6c3bd7b64950ca4e737549ce1a7
CompactCells587To618.Box596.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:6648-6659	8126d0731f0835ed89f0adcee759c58873cac29c21f81a741460d9006c8bf3a1
CompactCells587To618.Box597.QInterval	CompactCells587To618.lean:6660-6662	d16329386b8ddb9829c576d5d46026db6d2621f6a0be0a712d92d015abcb8878
CompactCells587To618.Box597.lambdaInterval	CompactCells587To618.lean:6663-6665	0765f4bae4bfae3463fbef6015cc499de8a4b7ae5d3da97d2bcb8c1ae56c98
CompactCells587To618.Box597.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:7312-7323	19ce9affd1fdf5c1544327e223f0e350cda118bc2600762fcc37faf271ba115
CompactCells587To618.Box598.QInterval	CompactCells587To618.lean:7324-7326	6aa6160f1b60e6a57560cc92216fd7e1de1bed3dbcedde9e6a91110edf47cced
CompactCells587To618.Box598.lambdaInterval	CompactCells587To618.lean:7327-7329	5f7e7a6ac2e0080ce7f97a1c8d6faeb5e663f1f3b35292b2e79b79aec6b73783
CompactCells587To618.Box598.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:7976-7987	46ddc24696327c4acb610d495dbb119dfeb7b43cdc3e093f6456d0be0e5a254
CompactCells587To618.Box599.QInterval	CompactCells587To618.lean:7988-7990	88f6c9b91034f54140f36600dda12f3a089947b08a79e3d393cc10619ebbb84d
CompactCells587To618.Box599.lambdaInterval	CompactCells587To618.lean:7991-7993	663569c7c74b2c665950a099967c5e41a8eb82c73efbf8344f72adf5342d556b
CompactCells587To618.Box599.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:8640-8651	5d747951a3f98ab0007d69f702eeaf0330e69999ae60fe16287a9206de2f073a

Declaration	Exact source	SHA-256
CompactCells587To618.Box600.QInterval	CompactCells587To618.lean:8652-8654	23a641bbaa8905d66b5435bc96fdb79453ab653831a2e5d61eed59887581278d
CompactCells587To618.Box600.lambdInterval	CompactCells587To618.lean:8655-8657	793323a75c0a341948552ad69525b7814996dccb7ba3efe413c21acb9825fd41
CompactCells587To618.Box600.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:9304-9315	7d2f812f0d18a0903f82f737f6b5f30c420f22538a121d3a44d5977760976e6d
CompactCells587To618.Box601.QInterval	CompactCells587To618.lean:9316-9318	887ccb7db444fc77b777ef0558ea29d5aff2ae9ca1a83ddd1839297047aa7a9c
CompactCells587To618.Box601.lambdInterval	CompactCells587To618.lean:9319-9321	19fc83b18bf55be1df1aa1392751d00127f54882e6b4fc8fda03df0d9a000937
CompactCells587To618.Box601.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:9968-9979	a3638ece5064167ae649acdbf88642da6da0ab7c6decbb2b072ac9466673bae8b
CompactCells587To618.Box602.QInterval	CompactCells587To618.lean:9980-9982	8cbb8d1e142b631d3c6de3fd50145ef90ff378141204629a12071e4eecd3b8c
CompactCells587To618.Box602.lambdInterval	CompactCells587To618.lean:9983-9985	f23f3f3fe05afd2eb531332a204eef881554e611b6abde25d3d3f87139c45d23
CompactCells587To618.Box602.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:10632-10643	66807cdbee988c70ab5dd5ab9b9d1b4d9c9a3437a4c552be16a8ae3341784a7c
CompactCells587To618.Box603.QInterval	CompactCells587To618.lean:10644-10646	e1896cc15dd91218f81eb245807195110fb4d76593f9f582cfa810dc64b8d643
CompactCells587To618.Box603.lambdInterval	CompactCells587To618.lean:10647-10649	ded1e4d36e660bf03f6cc3f616c4b179b42e11e364f6348c8e4d05c67470d1bf
CompactCells587To618.Box603.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:11296-11307	b0b337338b129c218ebcd0af7448a6681ec7f9e9d76570d160ce155c475b13c0
CompactCells587To618.Box604.QInterval	CompactCells587To618.lean:11308-11310	d42c9a68406b3079cf80ac139bcdaff807b92272944dc16e3d3e458095c559b7
CompactCells587To618.Box604.lambdInterval	CompactCells587To618.lean:11311-11313	6b9398f1dcb187d275c8eb5a104f8554a081ce17060b456e32f57a3a48824e25
CompactCells587To618.Box604.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:11960-11971	576f9519357783f305077fa8fadbb2212b930aae6ef98bfcdcc535989680eeed70
CompactCells587To618.Box605.QInterval	CompactCells587To618.lean:11972-11974	0ba5b97c62eb4f56c21f3ac93a5db4725219c030c147667e4e9fb7f0c207ceae
CompactCells587To618.Box605.lambdInterval	CompactCells587To618.lean:11975-11977	1bdac8456874faf8b52233a3b1cfaa9628338c16acef609c6b0e2795f3f50e2e
CompactCells587To618.Box605.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:12624-12635	c3f85213b8f8bb77ec70b40ed3efda237c900573164d003b65387005f4ebf18d
CompactCells587To618.Box606.QInterval	CompactCells587To618.lean:12636-12638	d2c4594148d0d6bd3ad01b13037c0ede0199c22db728e792e8bfbed7ac484288
CompactCells587To618.Box606.lambdInterval	CompactCells587To618.lean:12639-12641	63c8a0a304b4c8c79594f91119d20658c0f90e2fb6373ca0d49e0497ada430a0
CompactCells587To618.Box606.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:13288-13299	a4d7513eaf40fc883d922324d60b5a12ea2df0b94c6ba7c067bac3b73601fbaf
CompactCells587To618.Box607.QInterval	CompactCells587To618.lean:13300-13302	bf81d1121305e2326d16fbf17bfbfc65f0b1da6669659fed651461f611d2020b
CompactCells587To618.Box607.lambdInterval	CompactCells587To618.lean:13303-13305	e384cab53e51cd43f0efaf1be2a486a8e2d4de8b76f4acd38ad1ae2358b44c96

Declaration	Exact source	SHA-256
CompactCells587To618.Box607.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:13952-13963	c518add820b462eee03450ffe8874df7af848eac4e0451b3f242b7c263289782
CompactCells587To618.Box608.QInterval	CompactCells587To618.lean:13964-13966	21f189af3608778a5c1807949a28544047f7ce14286d8cbc963156828b56cdce
CompactCells587To618.Box608.lambdaInterval	CompactCells587To618.lean:13967-13969	3f3483f5c08cc6e900c1c0c9d774ec60937ffaca822f2211d19f455790de2570
CompactCells587To618.Box608.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:14616-14627	03db74da999670f34bcfe5d2b1ddb7ce7e13dcc23c55c7daf5e561aa3c75a27c
CompactCells587To618.Box609.QInterval	CompactCells587To618.lean:14628-14630	6131a5867d64f591031e860d1ac8e05c1a0f68d9372bb6f1ae22f83811a99ef5
CompactCells587To618.Box609.lambdaInterval	CompactCells587To618.lean:14631-14633	7593b61b49aa4835ff77e3b153a2cca89964c3cec84b6c3af136712e6c710d7d
CompactCells587To618.Box609.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:15280-15291	ccf3355b718fff534a6b95126283d4e17da9eb7fac75a0e645e311c7703de8a7
CompactCells587To618.Box610.QInterval	CompactCells587To618.lean:15292-15294	b9c07de58bd66263441c7ba83aacd1a2c47b98b9490c82a1159b8958cb7b59f0
CompactCells587To618.Box610.lambdaInterval	CompactCells587To618.lean:15295-15297	33e848075094bd127c96102af96b95c789c40221a726a87a05f775746721ffa1
CompactCells587To618.Box610.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:15944-15955	850c745712988e3e9d1c416b640da134714d049e4d0c9246e893896b1a88f88d
CompactCells587To618.Box611.QInterval	CompactCells587To618.lean:15956-15958	c757ecb867ce4eaba32d95c8b42b8af72d5e748533307e77daf176c136a61ef4
CompactCells587To618.Box611.lambdaInterval	CompactCells587To618.lean:15959-15961	4a5863e7f9eda4b116e709875b74544570fee1086b8f47d3244550dee9dd0472
CompactCells587To618.Box611.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:16608-16619	a01e4ed0dc7390a1e63e0f7685d1df7bc817ebeaa66063c625f0d8cc79c9c28
CompactCells587To618.Box612.QInterval	CompactCells587To618.lean:16620-16622	6b5cba28da9b689840eff1ad275027a78745bda776cdc7bb5854c727512b78f9
CompactCells587To618.Box612.lambdaInterval	CompactCells587To618.lean:16623-16625	fe858b335d3e33db8004d5f61a6639010156149a06fddd199432b72f821c3479
CompactCells587To618.Box612.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:17272-17283	da7a96f5410a52a69dc44765818c54756e8d9da2580a437269a6db794e215611
CompactCells587To618.Box613.QInterval	CompactCells587To618.lean:17284-17286	79acb5153a01fff3299691bc12ead38b4b532a8ba74a841e2ca1fbd03337a2f9
CompactCells587To618.Box613.lambdaInterval	CompactCells587To618.lean:17287-17289	c9f136fcab3c5e61d112384ca1b75c815455daf807a00cc27620b53b7d8b2082
CompactCells587To618.Box613.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:17936-17947	2a4868aaf8bdca2ac2f272fbbd07d318f637802f2c39e91e5d348ec1e5832dc0
CompactCells587To618.Box614.QInterval	CompactCells587To618.lean:17948-17950	47ef075fceb10aa89aab73f129152fe2cb145602672dccc8f648732641c4dd1
CompactCells587To618.Box614.lambdaInterval	CompactCells587To618.lean:17951-17953	09bc0c67255c1cd590290941ffb76d3e4551960267f7d1859b83aacc20e5f576
CompactCells587To618.Box614.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:18600-18611	a4bc9f4f27360a3eb92ef60896346d722392d263dc618027003e41a5efc0770c

Declaration	Exact source	SHA-256
CompactCells587To618.Box615.QInterval	CompactCells587To618.lean:18612-18614	a0940c903cca59865d936fad457f428a0ac0c7f749f79e5510c8ab1da1f1631d
CompactCells587To618.Box615.lambdaiInterval	CompactCells587To618.lean:18615-18617	aa10a192581e788aa992af5a2535fb48c122d23182f08ee7a9d3d4ed14c7449d
CompactCells587To618.Box615.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:19264-19275	77a7fc8088172d51a9fd8d782beefea98feab20fc47b23c2b5f23288632f06d7
CompactCells587To618.Box616.QInterval	CompactCells587To618.lean:19276-19278	819eac5638c9ca46df706f2f4c9a7f1fa7359f6005655699f6efd536b749e2e1
CompactCells587To618.Box616.lambdaiInterval	CompactCells587To618.lean:19279-19281	462b522b0811e26ee54b515bce1d18a04c8ed8ddd4224f7431abdf76863aebda
CompactCells587To618.Box616.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:19928-19939	197d7bc1302ced824df2293aad169fb90ac3bb0a257382acd27839543823193d
CompactCells587To618.Box617.QInterval	CompactCells587To618.lean:19940-19942	480af36011f9605f622e668e6d3389a920cf24d8e9a056599ce7cb775072a7ea
CompactCells587To618.Box617.lambdaiInterval	CompactCells587To618.lean:19943-19945	aa8a25f7bfe2c4c3677192733df60f7f2684ebe2b3def8ec94782819cad06f7a
CompactCells587To618.Box617.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:20592-20603	c7a7eb32973c675cffa0fbdbd6fdc6761dec33e11eae0bbc46ba45b2308233
CompactCells587To618.Box618.QInterval	CompactCells587To618.lean:20604-20606	e8f90712e7ab8985588c5300ba4b7d5a920267da45a8f9877b94413bd240e528
CompactCells587To618.Box618.lambdaiInterval	CompactCells587To618.lean:20607-20609	c710df8e0089f2fcd73638bd11e942bb4af962bd9717cf5bcbffa4af6fb9c130
CompactCells587To618.Box618.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:21256-21267	a152fcf362ae519ffe64e781590e8fdb589a289bf4a7c255c5091509309a710f
CompactCells587To618.bboxes587To618_candidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:21268-21498	e54a337d7aa2a41ffff24ac9f1348db5c1e5a6af3e85a0afeea734d85fa44ad74
CompactCells587To618.bboxes0To618_candidate_not_isWeightedPerimeterMinimizer	CompactCells587To618.lean:21499-21512	1b0ed147674dac6a2c6fd87897d076a463b3e3fa9269908743a14a02ecad21f6
CompactCells619To650.Box619.QInterval	CompactCells619To650.lean:20-22	f4b72ad9f2a29644a981fc5b7e10053fc6e5442b57ea3ff8d52fcdc720e1f4cc
CompactCells619To650.Box619.lambdaiInterval	CompactCells619To650.lean:23-25	5ce9c50a18bf313adeb9592dfb6aa84d657cb7bf93e7a37a8e1c4073e844d75f
CompactCells619To650.Box619.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:672-683	4b8f90918c2dd537317a9c64eaacfa69493ef71bc0c2bde39b6eebb143f2b9fb
CompactCells619To650.Box620.QInterval	CompactCells619To650.lean:684-686	54b0d4189e1c19ca210e1ac5050c69bdd577ad497e813ac3b0097dde5f4a632a
CompactCells619To650.Box620.lambdaiInterval	CompactCells619To650.lean:687-689	04f0d820cf8c74490a3f420c6cc5e9b084d139a99dcbaa89f54b1241c56f4cba
CompactCells619To650.Box620.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:1336-1347	1dfee46cacf261bdb2ed9aeb1f7c86e845e1042a262ff351ac23c0f0afe40b22
CompactCells619To650.Box621.QInterval	CompactCells619To650.lean:1348-1350	2aa42ac8c0aa03e11cb62c3741adf11fcd364864b410ed9b2f10c6fbc84d8ee0
CompactCells619To650.Box621.lambdaiInterval	CompactCells619To650.lean:1351-1353	b3a4eca4f7cbde959cc4bdd027d605f1bcb a5abfafcfb903303562ef159494cc

Declaration	Exact source	SHA-256
CompactCells619To650.Box621.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:2000-2011	0726427ad7668b0b82b8136cf46b26b496e953e29afc3b1b9c29555a6377f6fb
CompactCells619To650.Box622.QInterval	CompactCells619To650.lean:2012-2014	1a84551a04bd6233d8ad8bf9b7571fabf3861e512c2a09cd4a170fa412786aa2
CompactCells619To650.Box622.lambdaInterval	CompactCells619To650.lean:2015-2017	fa6bf3309348df64d1c957ea06d02913bbecc8cd5ad13f35396d80165575e542
CompactCells619To650.Box622.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:2664-2675	706a43fba3752c1db02274a38d4f3ce451f2b60ea6b8f86d8fde5923dcaa041a
CompactCells619To650.Box623.QInterval	CompactCells619To650.lean:2676-2678	cc0ce746ac859306bee99521720588b5e7322accbfd86e91940f9bddaa132ada
CompactCells619To650.Box623.lambdaInterval	CompactCells619To650.lean:2679-2681	e72c41d4edf5739db09cfd245ffd0b067858b799febe3a878636e9d3e1c77864
CompactCells619To650.Box623.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:3328-3339	7d0f8dbfc104d7aa8d9c34829680ce6f82870228c76b09b9589e06a6629879f5
CompactCells619To650.Box624.QInterval	CompactCells619To650.lean:3340-3342	6a8a4be74c11a9dc9ebe81a8b940e7747f0d0fe0f083e2fe5205a1e2fc260d6e
CompactCells619To650.Box624.lambdaInterval	CompactCells619To650.lean:3343-3345	f7745369d2c81af69bde4189dd976357f0814607501e3b80fc0d893d8f11e56a
CompactCells619To650.Box624.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:3992-4003	0a060b05e14daabb4befce5b131e341f1de19fee77457d534ec30c38b06ff0d
CompactCells619To650.Box625.QInterval	CompactCells619To650.lean:4004-4006	5468d7ea6957ffffcf042ce1c7d85f5c5de0a4ae8b4085958c25b19fb8a0b90b2
CompactCells619To650.Box625.lambdaInterval	CompactCells619To650.lean:4007-4009	f10bbf286802e53e2b9e8720a83344f2064b13135b902a35f0e4359d4dc1f089
CompactCells619To650.Box625.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:4656-4667	5d41f76d69ae1573cfeb4e9697e14d63a25616e005de3d685f246a317f3c6ddfe
CompactCells619To650.Box626.QInterval	CompactCells619To650.lean:4668-4670	000a101ee28c0334c74b2c980d6afe4633d36ceabfa689dd0b754d73d48ed6da
CompactCells619To650.Box626.lambdaInterval	CompactCells619To650.lean:4671-4673	dcdff239a70d5069c7186d89c0f6fbc34923e97faea3329a7a70c876cd477b25
CompactCells619To650.Box626.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:5320-5331	6f6a334461ba1af4e800bcb721fb452ff1ee6344a74fe7468ff0297560c34212
CompactCells619To650.Box627.QInterval	CompactCells619To650.lean:5332-5334	b75e4a98087e1b5b9d6291a43415a93ba0674d45741d5cf376dbdadd39f654bc
CompactCells619To650.Box627.lambdaInterval	CompactCells619To650.lean:5335-5337	0affe955d5e3a5f7ae327b0d94e22a62040041e6a3dbc4b03c45769037c0c6ef
CompactCells619To650.Box627.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:5984-5995	fcff20813f5db5f4f3692715517415bf6e32afc39148dcd80feea4e75465c68
CompactCells619To650.Box628.QInterval	CompactCells619To650.lean:5996-5998	5d3350b54121860a0e24726d906cd042b97d1d8cad3fd9bea62be1b7819f2bab
CompactCells619To650.Box628.lambdaInterval	CompactCells619To650.lean:5999-6001	409cae742a59a417ac731a3cfa6ff5edcf1015277446ea28c6b73fe075b92e67
CompactCells619To650.Box628.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:6648-6659	265b957d728bb4a850094909e3bc003d7d66e1ffe9bed0505aa825195e6be86e

Declaration	Exact source	SHA-256
CompactCells619To650.Box629.QInterval	CompactCells619To650.lean:6660-6662	bc5532b78c047c151d4dc01653b737d788d0b5ca0a74cbc790841ef5e684e7f1
CompactCells619To650.Box629.lambdaInterval	CompactCells619To650.lean:6663-6665	36082152be777ea58fbf8acc7c996bfd1cf3d9afa3006369a43a1576630fd98
CompactCells619To650.Box629.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:7312-7323	948b31fcb593713eff83717c35bdcaf1cb16e67b6ffd5573a4550362fb8d9d12
CompactCells619To650.Box630.QInterval	CompactCells619To650.lean:7324-7326	2b82b7fb12a98cb55f64edb97ef79f048496c32b85cd811e78604d231f16c5d2
CompactCells619To650.Box630.lambdaInterval	CompactCells619To650.lean:7327-7329	91117903a50a9d20479c802bc030f89ef13e1173351b924d46d40c868dec5bcb
CompactCells619To650.Box630.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:7976-7987	9508821bc73c98d0e8ba11c48ebdcbc021c47282c2f36ec11ece2366ee050d5b
CompactCells619To650.Box631.QInterval	CompactCells619To650.lean:7988-7990	99278cf344b3c80d6ea155ee3c7132cf9ca84ce3412faf22803bcc26078d2229
CompactCells619To650.Box631.lambdaInterval	CompactCells619To650.lean:7991-7993	6db4ebfd8389c3eca7df0021c8a5499ead880a5d67b42bdbb00bd19c953fcc56
CompactCells619To650.Box631.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:8640-8651	905f026a9956e70f1185413ed9bc72f401b962af11d3d95a2b833f58e36e26ab
CompactCells619To650.Box632.QInterval	CompactCells619To650.lean:8652-8654	864b8903af9763211243fc456a94a5102927a7734ba912aaef26927374491c48
CompactCells619To650.Box632.lambdaInterval	CompactCells619To650.lean:8655-8657	4486add855295c746606ee3f3b96a1dc397c9b0fcf10110567fc36fb9cbcf895
CompactCells619To650.Box632.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:9304-9315	4337600041e7bdf2d98ef369b9b78de557631a707d5eb9e24f2d8d8cbb810360
CompactCells619To650.Box633.QInterval	CompactCells619To650.lean:9316-9318	0f39e6d1a9c59c41ff45f953eaac5afffa96a7068284073abe168155b6f37b7b
CompactCells619To650.Box633.lambdaInterval	CompactCells619To650.lean:9319-9321	9c17e342492b1537761d495279b3eee1cb174463fa3cff7224fdccc74a06fd62
CompactCells619To650.Box633.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:9968-9979	dc304d4bb5920df3192e28bf1d4fa6e92b53d997e7462772539a9652b287379f
CompactCells619To650.Box634.QInterval	CompactCells619To650.lean:9980-9982	9b33867212c87ec0c6259631734a32e3311356816f7f1a8ea5f62936dbd993f0
CompactCells619To650.Box634.lambdaInterval	CompactCells619To650.lean:9983-9985	a3783b296de7383808d22d23dda520fb190e45e968ceafde25ba856ee631960b
CompactCells619To650.Box634.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:10632-10643	42eb79c7892d552bd0e2c1215c07a58656642d110870a50111a38730cfa96c9a
CompactCells619To650.Box635.QInterval	CompactCells619To650.lean:10644-10646	0003096bf18867ed75354ef0109b893f9708875a6ee9309bcb3fdf128a100695
CompactCells619To650.Box635.lambdaInterval	CompactCells619To650.lean:10647-10649	05bf2d62826763cc9e4c95c8256d3a618e7a80fbae380f9ab95379f66f0d2a20
CompactCells619To650.Box635.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:11296-11307	2af542f8a25cfd957c3564653f52604ffc91b21669efd4bdd5c6969014c6615b
CompactCells619To650.Box636.QInterval	CompactCells619To650.lean:11308-11310	07fb2c8c1f6be68df6e0f4ff8f6c3278bee833dc0f892382def11487ac1038a8
CompactCells619To650.Box636.lambdaInterval	CompactCells619To650.lean:11311-11313	d372c6213436801297d0510efe40641e640ae88cd857f7f0a38a797047cc5c41

Declaration	Exact source	SHA-256
CompactCells619To650.Box636.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:11960-11971	0bcb142269fbc19bddb5df6cbcd6c8e6a7ca0e9725a287a24980656423fe0757
CompactCells619To650.Box637.QInterval	CompactCells619To650.lean:11972-11974	94f323d71101240cb4e603a312fa4946b94b9718a1757c9fcd197ba377149833
CompactCells619To650.Box637.lambdaInterval	CompactCells619To650.lean:11975-11977	671ff074503c4b7313fa138591894ec195e8f47ad3e919ed76ed54a3d57b72d5
CompactCells619To650.Box637.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:12624-12635	684404c0730ad9ac014af7635641055e0ec66db3c27439872b12a2f6ef159fb4
CompactCells619To650.Box638.QInterval	CompactCells619To650.lean:12636-12638	80d8a6db18a98004cf5ff82625e82adf9e841c1dff44e8199b8467c46c041cfd
CompactCells619To650.Box638.lambdaInterval	CompactCells619To650.lean:12639-12641	be0608a4e36f6d08236628aa35d2472446608edd1fc2c0b62b691213bccd698a
CompactCells619To650.Box638.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:13288-13299	d8c72b86f8379dea18fb49d66d60fe815bb0c00f27566a2d7702f5bc5772dd92
CompactCells619To650.Box639.QInterval	CompactCells619To650.lean:13300-13302	109288fe5bbb495230d510c0f240183d289900ff2f8ec7086f5c7d7a8fbbda68
CompactCells619To650.Box639.lambdaInterval	CompactCells619To650.lean:13303-13305	7b43e3e7e93740962b5eabe9bfcf3552170e10457dfeed5055eff92abc213e58
CompactCells619To650.Box639.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:13952-13963	84be3434241b218fee142c4da3b0e1039d40cc2846a0e38354f5ec9f14874b44
CompactCells619To650.Box640.QInterval	CompactCells619To650.lean:13964-13966	9ca487ec058f6845d778dcf304278665a54ac3aacbcdcf0c7fa2b881ca3c8104
CompactCells619To650.Box640.lambdaInterval	CompactCells619To650.lean:13967-13969	847b28a283d04441a0332ef38569191f67e11dc87c5f317ebb350a3ce46f53df
CompactCells619To650.Box640.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:14616-14627	e1cb38a51b2cb1b25dac45b8c04ff7db33c62b6b935f08963d97ac0a85cb0b4d
CompactCells619To650.Box641.QInterval	CompactCells619To650.lean:14628-14630	9c83663a09e6ffe83675a36c947f44cf06ea9bbb4ee2ab6846d48612b1ef57ec
CompactCells619To650.Box641.lambdaInterval	CompactCells619To650.lean:14631-14633	9fcf2ed15e31ed9859122abca423de8f353ea4f15c7bdab7d73f69dd0c4c432e
CompactCells619To650.Box641.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:15280-15291	b5cbfc51ea524441bc0bf4c267b6ac0a9777648ee5a415453b318c104f8c77cc
CompactCells619To650.Box642.QInterval	CompactCells619To650.lean:15292-15294	ab4d955af1fb30d93f9b33d44fc6b6d9e2c4ceb7895a9b62b2dd4c25ccbb51ca
CompactCells619To650.Box642.lambdaInterval	CompactCells619To650.lean:15295-15297	f24d3234c4b95f416a645aff529ee807b6a9b007aeb267163435f23e17590b85
CompactCells619To650.Box642.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:15944-15955	6d20144c346de9ab3412af879a6c3f45a9891b9ff721a331bd18894b5fc28eb3
CompactCells619To650.Box643.QInterval	CompactCells619To650.lean:15956-15958	8aa23957d6d03fb4dcbf8d4039ceaa860ad8e8b2bb05c581b6fee0b05cdd336e
CompactCells619To650.Box643.lambdaInterval	CompactCells619To650.lean:15959-15961	0f79c14a71c3645dc51ca42b97b98f3c6e99025f2f502fdb0ddcbf61de65dcda
CompactCells619To650.Box643.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:16608-16619	a3f2c6156d067b58541516cc421f7ac174535b609f4d76541e82ae0b3318a988

Declaration	Exact source	SHA-256
CompactCells619To650.Box644.QInterval	CompactCells619To650.lean:16620-16622	cc46409e3220dbf7ab0b6c5e142bf69b118d30d66d3d828378d021b775e8c98f
CompactCells619To650.Box644.lambdaiInterval	CompactCells619To650.lean:16623-16625	9842e29ebb0c967de995c5ca994ba08047e67329204d6e6dfa48d32daa74afe7
CompactCells619To650.Box644.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:17272-17283	23bacedce984d240aa88dc49d08b6529b9ad8e1fe432e5802c92ef9c761c553c
CompactCells619To650.Box645.QInterval	CompactCells619To650.lean:17284-17286	7e41c2e44bbd7a58480e0b460cdc02d28a5d7933e1d310af37b40ae0989faf07
CompactCells619To650.Box645.lambdaiInterval	CompactCells619To650.lean:17287-17289	92048706d846c0131900e8c48fa1b815054f0a023b92b7890f78380ef67f6e15
CompactCells619To650.Box645.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:17936-17947	a32d5adc4c300e2480db81d36186e3bdf838e72aef7aae766d3a468a5b0018a7
CompactCells619To650.Box646.QInterval	CompactCells619To650.lean:17948-17950	9745deb9de0762556ababd5c3224829ee9b09ac72c5ffc04237d3772ae85abe0
CompactCells619To650.Box646.lambdaiInterval	CompactCells619To650.lean:17951-17953	583be25a6f72ce5e16e811938d1469b93375aa719de0270584ec2cf107b13a6c
CompactCells619To650.Box646.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:18600-18611	8eb64b04b1953a5c0710c1d9a6e0fe0318e32edde0f8d1db12cb6d27176b2289
CompactCells619To650.Box647.QInterval	CompactCells619To650.lean:18612-18614	e92af27724002ba0030911fccd5ba2871f74b9d57d4fbabde267394c0929ec7f
CompactCells619To650.Box647.lambdaiInterval	CompactCells619To650.lean:18615-18617	d209658116134865596dc9fedb0bc10cdb12d4dc43da49ff94ae925acfdcd021
CompactCells619To650.Box647.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:19264-19275	5739bd52b2f20a7d08bf1c6b163631821a8ebbd97547d9f609b2908f34697e56
CompactCells619To650.Box648.QInterval	CompactCells619To650.lean:19276-19278	0c6908d8a6fb682d06cc7f72998576d3089c248afde9b78ab0198dd3bfbf568b
CompactCells619To650.Box648.lambdaiInterval	CompactCells619To650.lean:19279-19281	68132f6d8acfb1c662a4d9b5db5e9b09bd33321649d34cb42213810b128fde5e
CompactCells619To650.Box648.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:19928-19939	2a6b10d5a146088d344446564092e67865a62c3d25440449986570097ec01a33
CompactCells619To650.Box649.QInterval	CompactCells619To650.lean:19940-19942	76477ce06bf23d67d88dfe079ff8906e0e08f288dc1ee73b782d07dd6915acc3
CompactCells619To650.Box649.lambdaiInterval	CompactCells619To650.lean:19943-19945	773f1095fd6e13a478443c91b13344415fb3ce06eaad63c16d0c9f0b07e202bb
CompactCells619To650.Box649.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:20592-20603	ae77bc071662fb386b65001f70b79d5406ddce7086844c84e03cc9974bd70e3b
CompactCells619To650.Box650.QInterval	CompactCells619To650.lean:20604-20606	0d8be187cfd768b6a9c2e0299c212f35763e62520bc5e160d28d5ac0ebca6ab8
CompactCells619To650.Box650.lambdaiInterval	CompactCells619To650.lean:20607-20609	11359763f4d554fd72a30aab25d0305b9c8169b17da0213a3b99032840e5024d
CompactCells619To650.Box650.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:21256-21267	ac7aba840171e73866684958227c5a799a4c977863a5716f659e0e2cd6caf344
CompactCells619To650.bboxes619To650_candidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:21268-21498	5eb50b7567e60b792c061c6c2e09e28ed4625c4b11d54d88867571339d816fc1

Declaration	Exact source	SHA-256
CompactCells619To650.Boxes0To650_candidate_not_isWeightedPerimeterMinimizer	CompactCells619To650.lean:21499-21512	d489bb1a51ec984a7ad9bfab6fc2ab9d52490501a0db592e09cd6d04704be4a6
CompactCells651To682.Box651.QInterval	CompactCells651To682.lean:20-22	accf9d76568ca504be806575938912d00d6686bf81dcf46e42adf68c5b80c666
CompactCells651To682.Box651.lambdaiInterval	CompactCells651To682.lean:23-25	6b6e89cd328ce235bc30c403d835c86deb4bd59fc9ed3c0dee56ec8a2e225b16
CompactCells651To682.Box651.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:672-683	a157be5451024043a7a1844a23a9ca1b2cf3ea192d2b60be94d98b4e6839eaa6
CompactCells651To682.Box652.QInterval	CompactCells651To682.lean:684-686	257cac110f763deabc7c98d10d35b303855b6260a930d4df41d08138b1861a28
CompactCells651To682.Box652.lambdaiInterval	CompactCells651To682.lean:687-689	4696bdb06200077aa0f4d602aca7cb4ad0600c274cc66da39a182c76d2649ecd
CompactCells651To682.Box652.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:1336-1347	9fedaaee8f155d7e775e44ff559e090d4b08e2bdcc22ff51a7ce5b847abe9aac
CompactCells651To682.Box653.QInterval	CompactCells651To682.lean:1348-1350	516030603c34aaa21b481d13b0fc11752369bdcc7df8649c3e5c662a59473436
CompactCells651To682.Box653.lambdaiInterval	CompactCells651To682.lean:1351-1353	9f499a1179edd6f47df50c8c6f9e14e2e570e7d3f685a1d2c9257ffd7667197d
CompactCells651To682.Box653.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:2000-2011	c6908b89688414d04ab3e100a4711458c4b3878a27f3c1ad69dd9188670ce9f0
CompactCells651To682.Box654.QInterval	CompactCells651To682.lean:2012-2014	e7b05fd31bd2b6d180a04ba4b9cb8145ed534f40eb33482b94abb91e3f1b7101
CompactCells651To682.Box654.lambdaiInterval	CompactCells651To682.lean:2015-2017	40e30995733206809441f4f5cc50fa45e4465cc9810103d8bee5b362a17b3f6d
CompactCells651To682.Box654.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:2664-2675	7eb88be9b0eb09d4abc6f4348b57538111c39b8828bce7997d1fa17b010b7914
CompactCells651To682.Box655.QInterval	CompactCells651To682.lean:2676-2678	4ab2c235f6d4d726d6d23f60fa2a397984f205cd1b62e729f1366f38348db564
CompactCells651To682.Box655.lambdaiInterval	CompactCells651To682.lean:2679-2681	09e0b3e465a1bb9dc9fc5df67416382163be6d3ed1745aad660c564415375c5
CompactCells651To682.Box655.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:3328-3339	5e8a405175ae85e3a279faeab232c0b05b47283fd795ae10978a7c03277e07f9
CompactCells651To682.Box656.QInterval	CompactCells651To682.lean:3340-3342	d574bed3c026e53e6d076b6c8da34df5ec9346aa8a24f8013d51717ad3203f99
CompactCells651To682.Box656.lambdaiInterval	CompactCells651To682.lean:3343-3345	cc4e4d33ea7535d36e25cd95933c981e29fce3c7adbf7341501eaf7fe82663b6
CompactCells651To682.Box656.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:3992-4003	d0460bf2be435d608cefd4c8e4717a4ee8bde593e86ae473b345ad69df3b358
CompactCells651To682.Box657.QInterval	CompactCells651To682.lean:4004-4006	a08bb7ee15ff1a42948cef1e33a3ea996447b826f3d91b0cd0037220e8fa0e56
CompactCells651To682.Box657.lambdaiInterval	CompactCells651To682.lean:4007-4009	29c9ee185eec43d6a1c38bfa9a983be776157e79d6bda2b5cbfe4c407cfd65da
CompactCells651To682.Box657.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:4656-4667	a41d1e024250e4649b4b856ac0f9addb332d7611da390461138c628574ffb18a

Declaration	Exact source	SHA-256
CompactCells651To682.Box658.QInterval	CompactCells651To682.lean:4668-4670	5742575876ac1290a48593f21cfd978fb513725bce57873aef4b84434cbcd06a
CompactCells651To682.Box658.lambdaiInterval	CompactCells651To682.lean:4671-4673	c6f86e0b4eec0f9e64ac402d84be1c89faa495163ce53198d9a624d946309c9e
CompactCells651To682.Box658.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:5320-5331	7f827a9aba7655a4c3ddbcaf938b7a11cf93a6236c5b1f62a05e5c9502e6e256
CompactCells651To682.Box659.QInterval	CompactCells651To682.lean:5332-5334	ca0b4145dceda421fd9cdd526f66f309113fdeeb5c3f06059f3cc6eb792a1ab0
CompactCells651To682.Box659.lambdaiInterval	CompactCells651To682.lean:5335-5337	08c472eed1dbc62d4633905c2136dc78b0b28a1969a4515091647016dcedfa0a
CompactCells651To682.Box659.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:5984-5995	6ac090c5b9beb5dfd293c177ad105711100b6ecdfe839450d34137b55fe72d85
CompactCells651To682.Box660.QInterval	CompactCells651To682.lean:5996-5998	0e0236dbdc18ebabf0bbc4efa32838a6ed40aadb7c7a6f5e77ad37f2cfd4674
CompactCells651To682.Box660.lambdaiInterval	CompactCells651To682.lean:5999-6001	adc825d097d624396ea75c2699ea5aac2060d85720121ef0cfb78435829313a5
CompactCells651To682.Box660.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:6648-6659	7d68bf90118abfacc46e00cff0c3ca653b217db2d80bab6ca89d12a93eeab9ac
CompactCells651To682.Box661.QInterval	CompactCells651To682.lean:6660-6662	c56e5d8f946b57465849ab75bff0a499b34a9ec8101e147ba32f506e573366f8
CompactCells651To682.Box661.lambdaiInterval	CompactCells651To682.lean:6663-6665	4709c0f9153b7d6c00c5113cbc5dc284a067bea69c748d4371693766d5f87eab
CompactCells651To682.Box661.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:7312-7323	2993ab1f81c8a111ab48e3bfa8d9b43893d03418ffe9e15256c5106a25ae58af
CompactCells651To682.Box662.QInterval	CompactCells651To682.lean:7324-7326	10f544dbf0e6f79a92108e5980df4d097c30efc153f69ac876da6be675bd67be
CompactCells651To682.Box662.lambdaiInterval	CompactCells651To682.lean:7327-7329	01327020b992ae071b4731566b89afcc265950cb7fdb58c4bc848ef60615c1fb
CompactCells651To682.Box662.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:7976-7987	e044d14b76e06769a2ae34c94b6ff3318d8b63808b2de461cc6ac004b02f4642
CompactCells651To682.Box663.QInterval	CompactCells651To682.lean:7988-7990	13216958b4fea9c2ce957d844ad4546f3d23b3e471e81b4b265139e59c9b13a9
CompactCells651To682.Box663.lambdaiInterval	CompactCells651To682.lean:7991-7993	01a96da1bcd167e5f13ac9cc68cac968b4d987157714a0416d6909ee3ea76dc6
CompactCells651To682.Box663.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:8640-8651	72078e45f1cd3f786ee3d2ac495560fc8d2296d5e42bfffef28f8dc86718c89a3
CompactCells651To682.Box664.QInterval	CompactCells651To682.lean:8652-8654	caf8d8e35bceae73106f0f4958738d32b9e5fda1349e6f2f20b9194c4c43d778
CompactCells651To682.Box664.lambdaiInterval	CompactCells651To682.lean:8655-8657	c35a00dff7d3e25762d2f1c91ed89fc140d164638e3a34d27c6553ebfb6fd802
CompactCells651To682.Box664.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:9304-9315	4403a18937ab22644e7104ff170a7dd6d4cbc199dc7ddc0110c0105170ec82b2
CompactCells651To682.Box665.QInterval	CompactCells651To682.lean:9316-9318	d0809ba99e9d3100af54b350a9f1bbabfbf28b121fba121c8f3ed0e34551a9b02
CompactCells651To682.Box665.lambdaiInterval	CompactCells651To682.lean:9319-9321	ee7e6d470c43626709721168b7e454e2981f0d86a003f5b9071dec2bfd50b8ab

Declaration	Exact source	SHA-256
CompactCells651To682.Box665.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:9968-9979	8e669281de3837d1821722bdf699840a75fdd45c011970e912ad81e29626c222
CompactCells651To682.Box666.QInterval	CompactCells651To682.lean:9980-9982	bc1ab998fb522b839342637402ad8ff103fb5f96f3308f8ed0252c391cb66472
CompactCells651To682.Box666.lambdaInterval	CompactCells651To682.lean:9983-9985	f3d6f1486d30c25065e4c840179e5a897e5db92f66daa39b32e30db60849f697
CompactCells651To682.Box666.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:10632-10643	870e5ae5ecdea3eeca8a80aa37bf8bed12f4f35f5fa46ef64c03303d33f6bf88
CompactCells651To682.Box667.QInterval	CompactCells651To682.lean:10644-10646	50e16366331a554523cac38dcd793375549b915b7996e7b6c8b873281b7bbc05
CompactCells651To682.Box667.lambdaInterval	CompactCells651To682.lean:10647-10649	aa4096ca0e8ef220b91d071eed21af4bfd3a282629ce854b53ff931c2e151063
CompactCells651To682.Box667.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:11296-11307	179c1c16f10b603b4d404b7c05a73d7d76b536de6c24e2e1d6f5791aac6c1426
CompactCells651To682.Box668.QInterval	CompactCells651To682.lean:11308-11310	bad93032d8318319c304d48c17e89838e966fffaee4dda747597bcd68378634f
CompactCells651To682.Box668.lambdaInterval	CompactCells651To682.lean:11311-11313	56cb563492867150ba2000e1a88206f53ac9b2d521043e7a76b9724f7047ee8c
CompactCells651To682.Box668.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:11960-11971	521704d08333735c75b52b854b23509ea535d2ba9a5cb3ebc8a4748aa06ae393
CompactCells651To682.Box669.QInterval	CompactCells651To682.lean:11972-11974	642fc169149df41e62e2bf03c823821cd9bcd692a2a29c2abdbd083b5226cdbb
CompactCells651To682.Box669.lambdaInterval	CompactCells651To682.lean:11975-11977	da7a299a17751d33d2df140d2936d2684c9a135e8e520d785610c4d26cc68954
CompactCells651To682.Box669.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:12624-12635	37df9e6d25902d9f2449c07f4692759e10f3dad8c86ef9633a91731befca315d
CompactCells651To682.Box670.QInterval	CompactCells651To682.lean:12636-12638	e09ce9807966726b755c5deb077b03eadcf8aa49414981221fc71f2b1965eace
CompactCells651To682.Box670.lambdaInterval	CompactCells651To682.lean:12639-12641	b205a4c28ac6a359a447d1c1829417f7fa5764204601f84978d5bd7b4b66f6ce
CompactCells651To682.Box670.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:13288-13299	6bf9d7c76b45f8d8aed34b4459a773dd70816034933fc26f0a0e6a3a7ad3d62e
CompactCells651To682.Box671.QInterval	CompactCells651To682.lean:13300-13302	e4cd3e8b783a8551a2d6316ade3e31f6d8833a433fc3ae42e9f9dcb22918fa09
CompactCells651To682.Box671.lambdaInterval	CompactCells651To682.lean:13303-13305	4a6b0d899a1023e2ade54f4b47c32804fa82227d77cc55ca329066964f223c71
CompactCells651To682.Box671.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:13952-13963	7e0ef7f9adcaa1b6f0b6f644ec2269e09f46a252f660012a7397897d0883ccce
CompactCells651To682.Box672.QInterval	CompactCells651To682.lean:13964-13966	1fe9ea7da9b75cbfbc039163c51e211ae4b7d3e26f76c9f3b46f85d6ab36b55d
CompactCells651To682.Box672.lambdaInterval	CompactCells651To682.lean:13967-13969	b4c4cc9d9efed0e1b23f6950839652e73bb6c2c00e89f13157ffa3570b2c656d
CompactCells651To682.Box672.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:14616-14627	d5e6860c4b318587bdb6691c358fa24465ed670808ea62fbd5ac0072bf6c4e27

Declaration	Exact source	SHA-256
CompactCells651To682.Box673.QInterval	CompactCells651To682.lean:14628-14630	15395cfac007b579cdcacc90bb6e7dedf39cebc8fe15498368a4a0d33865894
CompactCells651To682.Box673.lambdInterval	CompactCells651To682.lean:14631-14633	a0312129995064d554302b80f622932108c5ef37873784bbea30c98544819de1
CompactCells651To682.Box673.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:15280-15291	0b6b652add1ee55db7cc9fba17b3309489bde09ff47f505305d2c65ba3a3ed1d
CompactCells651To682.Box674.QInterval	CompactCells651To682.lean:15292-15294	b16af38b207605f1eb9a78b2d73b32d23dc628b9cdb5fc09474349d50255803
CompactCells651To682.Box674.lambdInterval	CompactCells651To682.lean:15295-15297	338709e6ba21be66a117278fa40809d2cdc467540e0ee4d82010be4fd3136ec2
CompactCells651To682.Box674.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:15944-15955	b7eea96e21da4272d6c7b918d3bc8cd3df3bb1c7ee3e99a19dcfa56b8e0ae4bd
CompactCells651To682.Box675.QInterval	CompactCells651To682.lean:15956-15958	4aa609b89be60aacdcaa0816cca772989aba1c5d3d143bd13c0a02e5c8a17260
CompactCells651To682.Box675.lambdInterval	CompactCells651To682.lean:15959-15961	2df050d0d3ce23a0cca898e2c9995ab4b3cb0a34c2d6fc205b2ecc898787bd0a
CompactCells651To682.Box675.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:16608-16619	6832bce798a26c78a9ddd84a3dc1c46cf31c6f8b1e62a813e8ba5b261cb13245
CompactCells651To682.Box676.QInterval	CompactCells651To682.lean:16620-16622	75b1b1fcbcb004e5eb39210775c64442fe78a6e5c8dd185c3a32f54ccf6b40633
CompactCells651To682.Box676.lambdInterval	CompactCells651To682.lean:16623-16625	ccc2224d6d7a65b83d43b562e3f64b4846d4e71aba5a471ed585fd84c30ec1e6
CompactCells651To682.Box676.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:17272-17283	abc639c3fcb4d7c9f2a454b5980d363e5a4a619c17c35fcc3660f2238a2260b8
CompactCells651To682.Box677.QInterval	CompactCells651To682.lean:17284-17286	094dbb8f83b87fa0f218b0e9e749fa792538e24db5008e2ccbb9612d7cb140aa
CompactCells651To682.Box677.lambdInterval	CompactCells651To682.lean:17287-17289	48d95719761b83c8a675f01dc6065e269e2287aba6f2cc59e29c1dd27f878481
CompactCells651To682.Box677.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:17936-17947	958bfd01bcdcd4d8c666f2bc32008eaa383d440c18decbb99c45aa8931ed21a358
CompactCells651To682.Box678.QInterval	CompactCells651To682.lean:17948-17950	0a04531380f7f0dca80e33d3d7a890f71990d70c2bdd0f88dae90c8d4727017d
CompactCells651To682.Box678.lambdInterval	CompactCells651To682.lean:17951-17953	f346c91b5e98191c33d015e7d3c23eff2ba486859b1c0d916d476c9b4af28701
CompactCells651To682.Box678.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:18600-18611	0d584c7e2cdc2194d34108c0e75e015744c88e71fcfa82b9e691ce3bc6c6146b
CompactCells651To682.Box679.QInterval	CompactCells651To682.lean:18612-18614	636afba493041ccb3560630eea6406a2f5db1e5080157c7d48e5f00419ed72e3
CompactCells651To682.Box679.lambdInterval	CompactCells651To682.lean:18615-18617	25a6a5ce74338b0652b9ea837b1285fe18c06fd934a699781ddd26876bc26ea9
CompactCells651To682.Box679.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:19264-19275	bd3addba362a118bb05e6ccdbcbcf2a10b05761240de58b1d30c026884a953
CompactCells651To682.Box680.QInterval	CompactCells651To682.lean:19276-19278	cd3a25a7cc37c6b9ac145d14b9b121e1fa9e718dc0f3194e26e1410d459fb4ce
CompactCells651To682.Box680.lambdInterval	CompactCells651To682.lean:19279-19281	511c8e9df1c9d928d6bdfdd6a09c3d18507777a5cd7b8c890b3b4465b6b5e67

Declaration	Exact source	SHA-256
CompactCells651To682.Box680.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:19928-19939	6a14910f8de953dca167eea0e014e4a1fbb14465f7f040098a30a69d65c7d99c
CompactCells651To682.Box681.QInterval	CompactCells651To682.lean:19940-19942	92c5e750a6437ff2b1106c0ded9f6ff528231c6960643b54714f79ba0ed92c32
CompactCells651To682.Box681.lambdaInterval	CompactCells651To682.lean:19943-19945	e8d04ea0a262e567d744d5c437070af7c127410270616c0403a75c225f2ff58f
CompactCells651To682.Box681.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:20592-20603	1112e0361a19ac74ea8239ffd8a3456c731e5ef21f978f76051fc54594044f2c
CompactCells651To682.Box682.QInterval	CompactCells651To682.lean:20604-20606	296c8be0ce897e3e380048cd7adcd993461554fc3a0b5b07bd45075f68c044e1
CompactCells651To682.Box682.lambdaInterval	CompactCells651To682.lean:20607-20609	505ab63bb5341d3ee96541f8a2c86bf3b214276b98297bd43abf7fe3fca3bf3d
CompactCells651To682.Box682.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:21256-21267	504c0f23f54843b4e895d299f48f9cd75cc55cd47160d35eaaaffacc32da33dbf
CompactCells651To682.bboxes651To682_candidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:21268-21498	5a366bf166f5e4e29cefed51f25ee61d0bbd6e759f2f065810a08813ff1b3335
CompactCells651To682.bboxes0To682_candidate_not_isWeightedPerimeterMinimizer	CompactCells651To682.lean:21499-21512	9d862510e06e53f6fb69dafaf725e55e6af9a3abecd59353c1a4b54caa23156d
CompactCells683To714.Box683.QInterval	CompactCells683To714.lean:20-22	a39e5b7b02385e8e1329341ed8bebae8278ea95bb2f868bc54593b84e0f5fffd4
CompactCells683To714.Box683.lambdaInterval	CompactCells683To714.lean:23-25	1f414d00e10757abfab91e6a25b9eab0463057600f484a50191beb99ab646737
CompactCells683To714.Box683.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:672-683	688deda50b41e95f884186c4e6aea45b772c21a06d04ee4a6c1ddab1c1d6c9e4
CompactCells683To714.Box684.QInterval	CompactCells683To714.lean:684-686	3f49c67ad08ecbcee30200cd7de1d0c8bba10b6d30189907ee0410304565642c
CompactCells683To714.Box684.lambdaInterval	CompactCells683To714.lean:687-689	edda46a97b8475ba5f97642b277d051328291d590cda49e107749073bf6499ef
CompactCells683To714.Box684.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:1336-1347	26c8cbcd7665c5a7d5e935a6a8daf114bf7b0970be352ca5a949b828d5719179
CompactCells683To714.Box685.QInterval	CompactCells683To714.lean:1348-1350	bf4c29a01ba36f42de11cfb6795feb50e92de7493a79f812fb8e675baafba4f7
CompactCells683To714.Box685.lambdaInterval	CompactCells683To714.lean:1351-1353	c7fafa4944a2eb29bbc074ba6cbdfef165ce1f16b4dc8ab028f62fb68e2fbc0d6
CompactCells683To714.Box685.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:2000-2011	0e42e112d30b24f357c2dd7d4c36794a9e95c8f350b1e8225fd5105c42fb12c7
CompactCells683To714.Box686.QInterval	CompactCells683To714.lean:2012-2014	a7d3572875010a7fe2bf55110db04c854652a69817615be78d8e0ef61c4691b3
CompactCells683To714.Box686.lambdaInterval	CompactCells683To714.lean:2015-2017	d777fc413b90f59852904ebc7bfe7f332463cef8192cb78cb1c24459e8f1d3c3
CompactCells683To714.Box686.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:2664-2675	81d501717df88b960bcd1cc924c8ef01a0985634606c3fae5ab45bb7b8e54745
CompactCells683To714.Box687.QInterval	CompactCells683To714.lean:2676-2678	073760dbe5cc7f5ae82f1629e24939245e2afa40a235dea9dad108b1aa362

Declaration	Exact source	SHA-256
CompactCells683To714.Box687.lambdaiInterval	CompactCells683To714.lean:2679-2681	8ed1178c8627eabe55ad61efe491cfaa76fdccaaa9837898c68ba64799e9e1cc
CompactCells683To714.Box687.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:3328-3339	f5a82843458f196266d667f24af52e7d3a1b8e29053b4265b727f69d06cf374b
CompactCells683To714.Box688.QInterval	CompactCells683To714.lean:3340-3342	ffcd995de249ecee184ecdbc67a2fa66371d1ca0ee39eb456f6fdd50a14ea0e6
CompactCells683To714.Box688.lambdaiInterval	CompactCells683To714.lean:3343-3345	4bbc0c94163ec989d7fc3294c861ab69db9a1dcfa7668e1fb975e235515bcf54
CompactCells683To714.Box688.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:3992-4003	2e7ad6ef40bc82a78a7f2480104c4bd5c6840a3ca02e2b047ce9805121d9bae0
CompactCells683To714.Box689.QInterval	CompactCells683To714.lean:4004-4006	b5368317791f41f9c730bd78d9ba0e408ecbee46d313f071c51131de054897d6
CompactCells683To714.Box689.lambdaiInterval	CompactCells683To714.lean:4007-4009	a35c7aa3ff6e7c6b54b2a762a34b9a333284ca26d45f7dfc7f99c40d8fca43b7
CompactCells683To714.Box689.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:4656-4667	77fb3c759a73d9779003a5a15f1183af37c1aa49e332c6f24148e36cca7daebb
CompactCells683To714.Box690.QInterval	CompactCells683To714.lean:4668-4670	fab9744634a30134f4ad3f74c9c3e96c940fa58424e0615aba3656f01af6ce7d
CompactCells683To714.Box690.lambdaiInterval	CompactCells683To714.lean:4671-4673	acd15d465906f2850237dbd40843f188c93193fbc0305107ea16c012dab78ec1
CompactCells683To714.Box690.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:5320-5331	d1329c392d64a6b1b93ceb5cbb7c8ec344d4f98e996cab84e74ac1ca663500d1
CompactCells683To714.Box691.QInterval	CompactCells683To714.lean:5332-5334	368d18a75424fa6cc662358a732b9a698be90e91e525c196e974b57672c474a2
CompactCells683To714.Box691.lambdaiInterval	CompactCells683To714.lean:5335-5337	3216db827395363dc1a40ea8cb583257cb37e967081fa872816341cbec880af3
CompactCells683To714.Box691.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:5984-5995	932f460f57b064bc1c128be646215470e3e56624d0a1a8061712704a01c1dcbf
CompactCells683To714.Box692.QInterval	CompactCells683To714.lean:5996-5998	0145bdc11a4d86772e748646eb7db4508db1406037a9356812740d1acf91a477
CompactCells683To714.Box692.lambdaiInterval	CompactCells683To714.lean:5999-6001	5cbb89069abfcadacb2add327cca9128dfd7c7d9bb3dc3f8fec1a84f149b2bb9f
CompactCells683To714.Box692.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:6648-6659	55e2346a721dc4d322fc6041ab3bdb7dcd7588c36ca7cb5fc50aca779137a055
CompactCells683To714.Box693.QInterval	CompactCells683To714.lean:6660-6662	f0f9c31817be380f590c4170af353b4ad05aa07367cf7ee9c54de39fa7e816bb
CompactCells683To714.Box693.lambdaiInterval	CompactCells683To714.lean:6663-6665	807f90062102076a864eff7bccec55b4123099a2c718e4416a8408d929030aaa
CompactCells683To714.Box693.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:7312-7323	bd294a8d8811d82d49add1b6d8ac455a2b5d954e3dc99de22b67d91e249f44ec
CompactCells683To714.Box694.QInterval	CompactCells683To714.lean:7324-7326	384f47269e242d5ca092662478e8e50d89895d34f8d1d63823929fa4ca2b047b
CompactCells683To714.Box694.lambdaiInterval	CompactCells683To714.lean:7327-7329	c422253cf1a2e45b2fa6137ac2837bcf0549201f688ef279555a8a99e065fcff

Declaration	Exact source	SHA-256
CompactCells683To714.Box694.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:7976-7987	47939b984acb4aec464475333b7d2a2c56189cd945f384b7c61b2afdc99cc22
CompactCells683To714.Box695.QInterval	CompactCells683To714.lean:7988-7990	72db47c7b475156eda566188d552456dbd4a6729128c63ef832992a1322d7d16
CompactCells683To714.Box695.lambdaInterval	CompactCells683To714.lean:7991-7993	5979751e75d3a5ae7cebe9c60cc702247f69ae3e2a1ae4dc8fa8a924d4ac6f41
CompactCells683To714.Box695.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:8640-8651	9bb1469317d3f8762c628aac7f4289cd2004ee2d89274df12905f93cc1dcb700
CompactCells683To714.Box696.QInterval	CompactCells683To714.lean:8652-8654	c327eb79c86b18d0bd6439dfcd99f886ba78d8ff9217b64ef6f3d413873faf9a
CompactCells683To714.Box696.lambdaInterval	CompactCells683To714.lean:8655-8657	6cc3b2509c5dc1ba9d32df20e2e9af8322d478c8f14a59be1b0b834096943a56
CompactCells683To714.Box696.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:9304-9315	8a39b4408e784d89f75836371ca7e7d3f65e5fb9eb0dd75757d8403b09a8a8e4
CompactCells683To714.Box697.QInterval	CompactCells683To714.lean:9316-9318	cec1dbe17a83b6ee183548c02f1e8a24072c1960c8f019d76dd8d6542967e075
CompactCells683To714.Box697.lambdaInterval	CompactCells683To714.lean:9319-9321	e5e94fd2e53ddb4159c55e0036bece01fa6087bce51cd61ab0d2da878ebaf683
CompactCells683To714.Box697.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:9968-9979	17bafdf1bdd7867cf476abc68aa3a83b884570d404ba07a0494708af5373c9c4
CompactCells683To714.Box698.QInterval	CompactCells683To714.lean:9980-9982	5e2b3b6d8d5976737a9b1551e919d49f11f6e49954dd10e904ffc8f6a2981b04
CompactCells683To714.Box698.lambdaInterval	CompactCells683To714.lean:9983-9985	03498102a7b8f71cf87ad62071d5a7a796a33df67421240c59ed0c04cde815d9
CompactCells683To714.Box698.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:10632-10643	c24e6269abc54b1af0f664b636f5b5e5e8428ba3aabdfaff0a15ccaa383d2eee
CompactCells683To714.Box699.QInterval	CompactCells683To714.lean:10644-10646	7779a49211bbeb51a2010a08f414641246c93fe9a7e5fef510853c574988da33
CompactCells683To714.Box699.lambdaInterval	CompactCells683To714.lean:10647-10649	9ec34c80b863d1a27b1240680fa26ff9cc35249702422b52975329fa65b7d0d0
CompactCells683To714.Box699.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:11296-11307	e01e8a3407e6e3ae506b1b7d9734d7f39bdba4460934adb901442331ba440187
CompactCells683To714.Box700.QInterval	CompactCells683To714.lean:11308-11310	fd96fef4bbed04d0ae3230f969a8e4e5f9eb96e47326924e5d36691ac84ac8fd
CompactCells683To714.Box700.lambdaInterval	CompactCells683To714.lean:11311-11313	057a064370a535d0e62783d51fcc4dbfdb13dc27e548ba6c719975265e3e7318
CompactCells683To714.Box700.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:11960-11971	9258d906eaa0dbd951a2c08c21ec93253885d7fe5847151ba4b955f0a63d1b24
CompactCells683To714.Box701.QInterval	CompactCells683To714.lean:11972-11974	a6a9876d4bc46e5581f51ab9dbd2bea36b10b8059cda6e95bbaedc95a9fce78e
CompactCells683To714.Box701.lambdaInterval	CompactCells683To714.lean:11975-11977	798b9edc6eae6972757c3673a28cbab13fd a29b7f91c2d8ce55d2c34fa9e3c35
CompactCells683To714.Box701.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:12624-12635	50d71f1df995522321235e95ed3a4b7ab5491604e6733c4254f96a1cf98828e0

Declaration	Exact source	SHA-256
CompactCells683To714.Box702.QInterval	CompactCells683To714.lean:12636-12638	6785de17e0aef1c4ed8c61ce4f24e1c415eb57ac4b8770af7067686065aebedb
CompactCells683To714.Box702.lambdInterval	CompactCells683To714.lean:12639-12641	8ce01cc6a678ffa58a62c2a3acdac75da593215c07dd63b1e5d49b3d82bf2597
CompactCells683To714.Box702.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:13288-13299	a37784f78e762a385282ee23c467430b34943fdf11710a916d6f4face25d9ec0
CompactCells683To714.Box703.QInterval	CompactCells683To714.lean:13300-13302	b4d5596f373707b5a09e940e549ed470e6b1ae53b9e8a7c9390a868329d41931
CompactCells683To714.Box703.lambdInterval	CompactCells683To714.lean:13303-13305	7893b6c1cce50e13ea9c4612a29473c2f1628a90583087f7c2c5492d2c73b8ea
CompactCells683To714.Box703.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:13952-13963	27f5d1c8205c3afdd5ebefd4544504ac555a74abe7e6c7a3b7232a1d8812db9b
CompactCells683To714.Box704.QInterval	CompactCells683To714.lean:13964-13966	2603ebe4aa13968a8def83108610f6300567480c43bbdd54eca819fbce3c15a9
CompactCells683To714.Box704.lambdInterval	CompactCells683To714.lean:13967-13969	1815d664ca1423257a66a1f0de647f2ec80c92931e4fa84934fb5e5da57b487b
CompactCells683To714.Box704.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:14616-14627	0cfd8cee58d5e1a80cd794f00668f91dfffb80c0011bfd2368e8f804c940365f1
CompactCells683To714.Box705.QInterval	CompactCells683To714.lean:14628-14630	afbef6a9b077730085ac7c226b367ce74a31f9b380b5dd3c51d1ce7a67030049
CompactCells683To714.Box705.lambdInterval	CompactCells683To714.lean:14631-14633	191c75e90267f82258a29e03f603f9885a514d2bf0d532c34457ea56d643d28f
CompactCells683To714.Box705.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:15280-15291	5a2e7082af711e04e120fda46e7a1065d6bbd14128283ba1234778b7188601db
CompactCells683To714.Box706.QInterval	CompactCells683To714.lean:15292-15294	461165445a08cc8e26c5c946c1e842c3a8d20125fb80201bf74355d57e80accd
CompactCells683To714.Box706.lambdInterval	CompactCells683To714.lean:15295-15297	447baba8ae6f3adc6441035a10cb68518358dff8ecfc33fa70f4dfaa72e1048a
CompactCells683To714.Box706.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:15944-15955	d1c39d73d2716435be585b24942dbbc58eaf0755385b9a1d25f67e3642e7fca5
CompactCells683To714.Box707.QInterval	CompactCells683To714.lean:15956-15958	af0d046579192a897e2450b0c6d30d67addede0117d95968bd9cd57f528ec272b
CompactCells683To714.Box707.lambdInterval	CompactCells683To714.lean:15959-15961	da58cd3f6ed0d713c6c21277a43a6850bf49c7d07422db94fe5be3a09760301b
CompactCells683To714.Box707.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:16608-16619	15c50b37bb32cccfc229e97d38fef5de8bcab379fffb70369fdcf17c5bc03069e
CompactCells683To714.Box708.QInterval	CompactCells683To714.lean:16620-16622	c9a0e561a0b3e8ac45e9fd143f6afd7008e37110f0b94efa45adbb7f39915dab
CompactCells683To714.Box708.lambdInterval	CompactCells683To714.lean:16623-16625	b1ce36c8342bda0210f10048d6bf07b5c83e88a6366abf905617d8e4f79429df
CompactCells683To714.Box708.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:17272-17283	d2e0d22c4046bace97d182f8055c01105fca1a7a2459303c783f52f07d000911
CompactCells683To714.Box709.QInterval	CompactCells683To714.lean:17284-17286	bf51e473fbc3e56aa2987edaf9c8e460028cb58a430fa9b7330605fbedb7b8f3
CompactCells683To714.Box709.lambdInterval	CompactCells683To714.lean:17287-17289	356238f3f5d0dce5c7b71adf1ab4b978486e99e71d5052024e298f9874d040e6

Declaration	Exact source	SHA-256
CompactCells683To714.Box709.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:17936-17947	35dfadd694a90f144ef3a3074eabf4f6917bf4398fced2c512db68fa748f5972
CompactCells683To714.Box710.QInterval	CompactCells683To714.lean:17948-17950	47c9ee4e9f91901bdbc79406327b866aaa0df5ff4db6d7e7acae645290057a31
CompactCells683To714.Box710.lambdaInterval	CompactCells683To714.lean:17951-17953	dc625c52502ed5be9e4c4bde329c2d49ab69205e80e8aa21de9e6ca462b86431
CompactCells683To714.Box710.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:18600-18611	7617718d3a13971f55c908813f579808c65848234a2b914026d621e1a8b0d88f
CompactCells683To714.Box711.QInterval	CompactCells683To714.lean:18612-18614	97df9181f090f53868077e8cc17e5662cb079a1f2994a71b2913687866bc5668
CompactCells683To714.Box711.lambdaInterval	CompactCells683To714.lean:18615-18617	2b5445f273301412a391c9720d5bfa7e6b70b1abe98f770c39019f0bf0918e03
CompactCells683To714.Box711.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:19264-19275	7fb894038b59becd6ae5d97b8109defb025ab95fbd04db1ca3ed27ad55ef77a5
CompactCells683To714.Box712.QInterval	CompactCells683To714.lean:19276-19278	e226150f474f24596ad4525c4cc5cb82e0eb1b35c7c3259bd8c9f13c81a2cf04
CompactCells683To714.Box712.lambdaInterval	CompactCells683To714.lean:19279-19281	28e7477f7dc6f394537d47948b26f7f452b5e7bf8434b29a259f8ed8164676ca
CompactCells683To714.Box712.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:19928-19939	858ad903e49ce5e1d7c7a79a33c0d4eed4a6859ce7b67cdbdba4041e1fb4cb1b
CompactCells683To714.Box713.QInterval	CompactCells683To714.lean:19940-19942	4d2d5081ec943331e2b95de359189b488ce1a4876c8bf48e5dcbf12e356fabbb6
CompactCells683To714.Box713.lambdaInterval	CompactCells683To714.lean:19943-19945	e73a525bead2ca23d13d661140953d986430c1598bf6827ad9dcab7016b7de3
CompactCells683To714.Box713.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:20592-20603	9a01fe7e4a78c9c317260140dbcc8fe14a64582ac159db99c5315ee2de4eb842
CompactCells683To714.Box714.QInterval	CompactCells683To714.lean:20604-20606	11aaf7fe2b77452c5ac84d6cacb663d388dffa08a632509653b51baeb3380cd
CompactCells683To714.Box714.lambdaInterval	CompactCells683To714.lean:20607-20609	102e0d855c5add8cb3c9b0ebf84d94c47bef8ee00aeb0fa64e1c0ed0a2ba9673
CompactCells683To714.Box714.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:21256-21267	824fe73f266d3caba8be23410920479f6ee09e7a715c496a29246388e0eea6c9
CompactCells683To714.bboxes683To714_candidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:21268-21498	80d47d6dfb0a92dabe4b702e8a677ac3856eb3df280bd93c228441c5aee536b5
CompactCells683To714.bboxes0To714_candidate_not_isWeightedPerimeterMinimizer	CompactCells683To714.lean:21499-21512	bf173ebf38135bf1764ec474da71caedfaa9ef90f181f1dbb6694cce522f5324
CompactCells715To746.Box715.QInterval	CompactCells715To746.lean:20-22	35b868ac3d6583f682c05e44499424702fe917489cf506832040ea6681f45fc6
CompactCells715To746.Box715.lambdaInterval	CompactCells715To746.lean:23-25	b68996a4f9417c2da9eebae039f42f8a7f61501be6f08b2ddcfa812474485f4d
CompactCells715To746.Box715.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:672-683	98de2911113af9aa4801219ed82e5a12a8e5155aaf0f8536b5085183de9b8599
CompactCells715To746.Box716.QInterval	CompactCells715To746.lean:684-686	b59674e5a274c39eb588388567b5a90ad3b07749651075c1af5f61e15b0edac7

Declaration	Exact source	SHA-256
CompactCells715To746.Box716.lambdaInterval	CompactCells715To746.lean:687-689	376469b1e7d464f921d233774c86cabe8238d8766b46645ad5730c038cfbcef2
CompactCells715To746.Box716.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:1336-1347	3d4d9ab416d28e68e2c7a848f9778d38275c694dea42e9a665d6db6f5e8f2514
CompactCells715To746.Box717.QInterval	CompactCells715To746.lean:1348-1350	c02396a32d8b5193683fa71198c23947e7cee9377ccf20c0f0904d929d3176cb
CompactCells715To746.Box717.lambdaInterval	CompactCells715To746.lean:1351-1353	635d47f830aab13c1a487f86b1802188479eb3702a6c0cc21033bd181ed75678
CompactCells715To746.Box717.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:2000-2011	6ae7d30a4415fc4d0cbcd87d18cb33bde409750e686a1dccf2022a70922da1e8
CompactCells715To746.Box718.QInterval	CompactCells715To746.lean:2012-2014	dec4cd7c2f2be606231912259040476267ae52ba62765c4f34273ddf706ddc4a
CompactCells715To746.Box718.lambdaInterval	CompactCells715To746.lean:2015-2017	60353a1d9881437f47719f03d31fafd835b6ef7f0d273fcf85d193c97842cf61
CompactCells715To746.Box718.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:2664-2675	b684946b9b6ed214891916a8de3e1be53fde85011305f2d275fffb77479098c44
CompactCells715To746.Box719.QInterval	CompactCells715To746.lean:2676-2678	86facec1f7cc58b15195c4f0074b43d67490cda41d355b116c08bd5bfd4c724d
CompactCells715To746.Box719.lambdaInterval	CompactCells715To746.lean:2679-2681	61aea9ec9adb1fa481f009cababfe548aeeb11c00c159a904392a9723d00141f
CompactCells715To746.Box719.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:3328-3339	b6d8e3c32cb038ce858b4e273753bcad2d81614401852f58b168620d32ec1ff3
CompactCells715To746.Box720.QInterval	CompactCells715To746.lean:3340-3342	9d4a5e6006b0388bc02482ff0ede40971f6f331e8323ccf529c389162c77f81e
CompactCells715To746.Box720.lambdaInterval	CompactCells715To746.lean:3343-3345	5b4e8d21b02e46bacb60d9b557753c28724c6406cdd7834cdffa8c1bc931fac6
CompactCells715To746.Box720.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:3992-4003	3a2f67f6ece5b190609855376c454a121f06ec1dac547e6420b9c01e614f67f8
CompactCells715To746.Box721.QInterval	CompactCells715To746.lean:4004-4006	57df846bde058e8d7884c81613e4097204d2633a90818d0f979c8c79c3208376
CompactCells715To746.Box721.lambdaInterval	CompactCells715To746.lean:4007-4009	e9714680d46119910025871eccc8b34936baef1e44c5c165ddcb3a2d1b2b2c58
CompactCells715To746.Box721.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:4656-4667	52444ebe3bdf8bd72a0991ee861dca3eb5bc204b421bfa376d1abcf0a5ab373
CompactCells715To746.Box722.QInterval	CompactCells715To746.lean:4668-4670	69123a7610f394eeceace2110856a942ea0078005e741d08f4271bed7238d28f
CompactCells715To746.Box722.lambdaInterval	CompactCells715To746.lean:4671-4673	f717d0ad7703eadd74d7c722be16c00e277abff8db04d961418c674dc2eb2880
CompactCells715To746.Box722.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:5320-5331	889650b5eedfc4c1e63b9c78e52a057d409b7b0b27add1f88db0f1490363c8a9
CompactCells715To746.Box723.QInterval	CompactCells715To746.lean:5332-5334	24da470ce3573b3ed4b35f9127e9bb0ac3773b61cbd335c5d69eac9b5891c7d6
CompactCells715To746.Box723.lambdaInterval	CompactCells715To746.lean:5335-5337	731835651c3e7b14fb67016ffbc40344a52f72bca4c44f5eb63e6a06b9a606d2

Declaration	Exact source	SHA-256
CompactCells715To746.Box723.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:5984-5995	e5e8021a6068b2ef396c1b7a3ad21edd4393d4a60be535cecdfec8f19ce6c792
CompactCells715To746.Box724.QInterval	CompactCells715To746.lean:5996-5998	9242880f264139fc1e44bd3c1f5101a41ec352c150c29560bd5b89ca8739a079
CompactCells715To746.Box724.lambdaInterval	CompactCells715To746.lean:5999-6001	ae71bf6598b3508253787fac3733b6371ca7c57bc031cd8d191351b35833e90c
CompactCells715To746.Box724.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:6648-6659	8064799105d7375f662eb17fc157d212ebc8802777eaad75b93dc2934966be76
CompactCells715To746.Box725.QInterval	CompactCells715To746.lean:6660-6662	c831e9d4ccab2e5dee1a484c86ea48ada6537cd5b51dccb8a62e81d485eb5516
CompactCells715To746.Box725.lambdaInterval	CompactCells715To746.lean:6663-6665	71b89b99e82a86b1d8470cd021d6f71e8c3784b5cd8d3d0029c4662dd5dfc537
CompactCells715To746.Box725.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:7312-7323	c6f2e58868336d86476e5198a30649545109f2832f06966746f6658b6ab4732e
CompactCells715To746.Box726.QInterval	CompactCells715To746.lean:7324-7326	c9117af16b0bdf45647477cc40751d00234d812d74fdd877c13e5296e074067e
CompactCells715To746.Box726.lambdaInterval	CompactCells715To746.lean:7327-7329	bcfe654a912270f48a67ea3ab7b0cfef9216da72bf62066a9fbb2314b6b72e9c
CompactCells715To746.Box726.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:7976-7987	89c5dd302237bf70fcb591e0e4b72e40e93eb1a939d22dfe2c4610ea748877fc
CompactCells715To746.Box727.QInterval	CompactCells715To746.lean:7988-7990	dfbacfc63d2aa837069d10878a641ee1e88e728c7681ab859303dc13d926efd8
CompactCells715To746.Box727.lambdaInterval	CompactCells715To746.lean:7991-7993	f76e6a2c6459e8182ce7b994873461146243755f076d24038d0328f0444f69e3
CompactCells715To746.Box727.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:8640-8651	b2a56f096063a2a1d85f3d6778c478dcff6a455170eb4df2331b726294a7b7f4
CompactCells715To746.Box728.QInterval	CompactCells715To746.lean:8652-8654	9add877e71d0ed7374ca0030dbe07a0ef17948e83b8bd74de81a44430b56721a
CompactCells715To746.Box728.lambdaInterval	CompactCells715To746.lean:8655-8657	a0f8872977ad1128d88ba2a0b3018126ce138a3e4ac7f58fb58112ef46b0f924
CompactCells715To746.Box728.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:9304-9315	0de7ea7514a8d7399c68a593fef1c67cf603ac605f464e6673cefc1ff3630142
CompactCells715To746.Box729.QInterval	CompactCells715To746.lean:9316-9318	28a0905df37f39dba6e802808ba69143330cc8ced1871e76452cf3130c2fc72e
CompactCells715To746.Box729.lambdaInterval	CompactCells715To746.lean:9319-9321	5325f4927489b5b4017a7f9b9cb244666b866fd6cde52e5baee80f86bdf1bfaa
CompactCells715To746.Box729.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:9968-9979	b1b0a469ebb2bfd0325d93e6563a04d1f9b0764e23a1ffca49230cb4912c716b
CompactCells715To746.Box730.QInterval	CompactCells715To746.lean:9980-9982	1cc5da0e2d0e5642744a8d18d7a694d9b74e691a1aea499f23b5f9978d39ac71
CompactCells715To746.Box730.lambdaInterval	CompactCells715To746.lean:9983-9985	b3d0242d2c7e1b33d9a97f53f486d3d5993dee453282f350bd8c5965f51ab54a
CompactCells715To746.Box730.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:10632-10643	74ef036574beb04fd5eb055f38d70705b7db7ca6038d8cff2b33b55d30d641ad

Declaration	Exact source	SHA-256
CompactCells715To746.Box731.QInterval	CompactCells715To746.lean:10644-10646	474b6d2dac4d5733a052b43ebd7106ebc03e3e4457376ab00671f49044a86c3d
CompactCells715To746.Box731.lambdaInterval	CompactCells715To746.lean:10647-10649	f19bd0416f0779cb5ddb73f19eb78a67fd55226d5883180e206eae5767badd22
CompactCells715To746.Box731.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:11296-11307	8eec25a1cbe358ca6ea060754117d7eb4483df553f7fb834555eb23c10383262
CompactCells715To746.Box732.QInterval	CompactCells715To746.lean:11308-11310	05f73d22238c9c0bf5a8321ce82a1bb6c41ed14f6b12878fa6e7eb99019f0d61
CompactCells715To746.Box732.lambdaInterval	CompactCells715To746.lean:11311-11313	232cf174751539e905a95ce73a978eca606a496df6a2a7a0b348f8d15348bc5e
CompactCells715To746.Box732.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:11960-11971	7015dd21a7a237d5e14d6730333095873f3670ed5d1f1c040f095855bf275bc2
CompactCells715To746.Box733.QInterval	CompactCells715To746.lean:11972-11974	ca6d1599c27c5c9681841cbf9f35e73eb58b89df7d5f4f9c5abfc5b6aa411185
CompactCells715To746.Box733.lambdaInterval	CompactCells715To746.lean:11975-11977	187c425fd1c4c13196540d1be69804315dc5d78a0b63d1e14203ecf9b3ea4ef9
CompactCells715To746.Box733.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:12624-12635	74ebc3e9dd446769ad0d483670049095383436dcdd997956470999fee0281f21
CompactCells715To746.Box734.QInterval	CompactCells715To746.lean:12636-12638	4f5a3473c894c6e5cb08109c721e9d58a4934dae2674e75cf6949238bb7afd6a
CompactCells715To746.Box734.lambdaInterval	CompactCells715To746.lean:12639-12641	be41fcd15dfdf59e45b438fcb6eb7ab78273b2a2a68202e8c4af4938ec2cf11d
CompactCells715To746.Box734.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:13288-13299	ffd114f3836bac102c9fe893afe94e6798d570154d2f432c63972498a33c3390
CompactCells715To746.Box735.QInterval	CompactCells715To746.lean:13300-13302	e1238f27b9fdcd24a39172374dce375c42049b051db2cd9f2993d5861379bf09
CompactCells715To746.Box735.lambdaInterval	CompactCells715To746.lean:13303-13305	a4f9d0cf1e8374e2dcecc9647c09c00b843f52d456b5e9c83c82fa9555318be
CompactCells715To746.Box735.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:13952-13963	1ae2b9c5e0dc7e9e1ee11046b78b3adf123aac13ad5992894f45fe59a7d0d0a9
CompactCells715To746.Box736.QInterval	CompactCells715To746.lean:13964-13966	f95ad5a6f0c2408da875ac7274d22f22587cc0c36a5e94cc51c3f66d4cdfd4cd
CompactCells715To746.Box736.lambdaInterval	CompactCells715To746.lean:13967-13969	e34ec184dff0ff7622dbce2911d9663ba37957e5cdbe544b57562b3be62c49a9
CompactCells715To746.Box736.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:14616-14627	d98554abc398183588939e59f3c7f6cf4058a5eab1e673a81055a5a2bf249aa9
CompactCells715To746.Box737.QInterval	CompactCells715To746.lean:14628-14630	403b9d86d0918f23f831e9394f9b96b41c5987d968d1ace796658c16bc90879f
CompactCells715To746.Box737.lambdaInterval	CompactCells715To746.lean:14631-14633	89272ed3bff65fe8e9eab34f02044ab5074ff82b27c6357da13a7890031110b1
CompactCells715To746.Box737.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:15280-15291	ffb34c8bb9ef9b17dc9c0260ebbf99cd212ec5bd3f3e3981b59d60c2ddd41022
CompactCells715To746.Box738.QInterval	CompactCells715To746.lean:15292-15294	e6af23c4267f8d91386ef44c63c09055eabf3841ad1157dc176cb6fede4fc805
CompactCells715To746.Box738.lambdaInterval	CompactCells715To746.lean:15295-15297	db3d9aeac2d7a35d325b9b3737f7be73c4b084a2e2ec4a22bc8b2519c01caf8a

Declaration	Exact source	SHA-256
CompactCells715To746.Box738.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:15944-15955	e847735b23992a38afa1513b2e62adbb59cd573690d5b621b598f88cf06c8ec1
CompactCells715To746.Box739.QInterval	CompactCells715To746.lean:15956-15958	8fb70649323fb034765b20f3c5ee58025a02dbe17e1ce778022849eee80c6037
CompactCells715To746.Box739.lambdaInterval	CompactCells715To746.lean:15959-15961	439a5e6f056512f8a7675449b9b2e39924bb3576218fffb2e33223fa628f0b1
CompactCells715To746.Box739.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:16608-16619	653ac4ab44f7e7b96099367721d1aceb9d2fde64b34f67e80a3a85d2258207fc
CompactCells715To746.Box740.QInterval	CompactCells715To746.lean:16620-16622	4303af5caa048328f56782def4c2961f7aa9cb8f86cf54937206fd1c1b6b674
CompactCells715To746.Box740.lambdaInterval	CompactCells715To746.lean:16623-16625	1c289b35606e628dd42e780e2b3fecac8c4f1897232708f78c4e5f38087f4f38
CompactCells715To746.Box740.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:17272-17283	097f19917d139ab3308cda53bdf4f97b9a694997e57965ca6ae3362c675b3e4d
CompactCells715To746.Box741.QInterval	CompactCells715To746.lean:17284-17286	1802cef1ba7cdee3cc9b02ecdcea72b34fa74a970839d257fb5a347463fd8aaf
CompactCells715To746.Box741.lambdaInterval	CompactCells715To746.lean:17287-17289	99c7590a1f6e14a027ae01f0059d68f27e8a7a1b3579055adf3669814a11a0c4
CompactCells715To746.Box741.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:17936-17947	19b3dce51937e45ac7f941bb4c1132857bca15379545bf223260b5cd3836f380
CompactCells715To746.Box742.QInterval	CompactCells715To746.lean:17948-17950	8a31d42262cd8382b4d45b3bf9134cd568ecc2f2415682140681f23a40fc54aa
CompactCells715To746.Box742.lambdaInterval	CompactCells715To746.lean:17951-17953	0bc05d4bd4578915c51d35fecd04cc653fa460d9ab3b675182f4e52067f1b4ab
CompactCells715To746.Box742.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:18600-18611	ce2f8c12b79cd3f586582e96620b87ffa3e39d7f158363ebef116f45c789659d
CompactCells715To746.Box743.QInterval	CompactCells715To746.lean:18612-18614	ce46dc11c51f93bf8968219868c2d70364e61394f1a27021a9e7842c8ae8b0e3
CompactCells715To746.Box743.lambdaInterval	CompactCells715To746.lean:18615-18617	a912dfc7cc7b3276e8bc6366e7b51466cba1f69c674ba95eb9f61e02903b140f
CompactCells715To746.Box743.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:19264-19275	7b55a07ea3ceed3b257fac0a4c3bb96b3ee064800fafd5e2f6f9dfbf0e24d96
CompactCells715To746.Box744.QInterval	CompactCells715To746.lean:19276-19278	ca5446140a9006c4a6191cd0a11e7b23c521a362bc7bfd5b547cb78336de9485
CompactCells715To746.Box744.lambdaInterval	CompactCells715To746.lean:19279-19281	3a4239aa001cfdad039cbdad9e6aa224e5ede466b968788f153e15f415d6ae6d
CompactCells715To746.Box744.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:19928-19939	7e579dad4cf3151263eab21c8b0ca203b7dbc486e88fcbd33e3d15b334d69a6
CompactCells715To746.Box745.QInterval	CompactCells715To746.lean:19940-19942	a5538bfeadcd81695ef2a7c4a6c767386530324ba3fa569742303028401d2fd0
CompactCells715To746.Box745.lambdaInterval	CompactCells715To746.lean:19943-19945	cf4289ef8ef7732de28f97321980de760fdaf3379bc24387d907cc3b8c4606de
CompactCells715To746.Box745.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:20592-20603	4939e6ebbb056d22eb681bfc3b8b13a24c5fd0d7312a6d50e16c6a3d9b10e9f6

Declaration	Exact source	SHA-256
CompactCells715To746.Box746.QInterval	CompactCells715To746.lean:20604-20606	65ceb33be902159a694de8be913481668ded828abc32248d79078b704e81dba4
CompactCells715To746.Box746.lambdaInterval	CompactCells715To746.lean:20607-20609	93520e93fa90026157a0390ed399b78569281bd7c94c2ea3ace9ed9d4f20cae4
CompactCells715To746.Box746.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:21256-21267	963f8153955418456e8fa50bb4a1e8e069474da7c5e4e186c8d3e0ecd6bcde7a
CompactCells715To746.boxes715To746_candidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:21268-21498	1189909df99af468a91429737d960f11737a8b6b672f112f37836588563f27bc
CompactCells715To746.boxes0To746_candidate_not_isWeightedPerimeterMinimizer	CompactCells715To746.lean:21499-21512	10f01d242597312468da9eef2c4697c3396ee4c042720726dd2d194461b7d655
CompactCells747To778.Box747.QInterval	CompactCells747To778.lean:20-22	9528ba8cc68c8940fe51f873ac86edde382a3d97f58915d58baf26e3ea56b68c
CompactCells747To778.Box747.lambdaInterval	CompactCells747To778.lean:23-25	2bae94d2c94f94594c868d120b7dd57aa4bacbbcaeebbcbdb395caea1df66597d
CompactCells747To778.Box747.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:672-683	f2d0bae9a195cfb77c9cf198b7da74a200e8c6bcf45697d00aac137e8c6661ff
CompactCells747To778.Box748.QInterval	CompactCells747To778.lean:684-686	dec7b67a0307bdb432ccb6db2cc7dcfd15fc2bbd867aa105c2d4c454dc990ba
CompactCells747To778.Box748.lambdaInterval	CompactCells747To778.lean:687-689	c6401d51042fd838fd0d88f8898d307ebaa3ae7534a9272f718e3d7e8ad3c74c
CompactCells747To778.Box748.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:1336-1347	70c4b47edbd8096022064da0e94247542c99f3655c307d2330892c04f13055e3
CompactCells747To778.Box749.QInterval	CompactCells747To778.lean:1348-1350	0216a778cc78b29fedfb8a1e541f82036d3a57b8c6d9580c3c0db5ff24d0a807
CompactCells747To778.Box749.lambdaInterval	CompactCells747To778.lean:1351-1353	1d6a98546003802c7e5cce1905e3829c492bed30d70fa6fb79143e7f41fb653e
CompactCells747To778.Box749.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:2000-2011	e1d891e1a4dc858fb2ed74c37886b65a9c6c7f13ed115bdfd7388a7b1c5d6ea9
CompactCells747To778.Box750.QInterval	CompactCells747To778.lean:2012-2014	a5baea9c72db66c9aa9665ee1dfe4d0eedc7943fcdda83efba130e3622d59e34
CompactCells747To778.Box750.lambdaInterval	CompactCells747To778.lean:2015-2017	abf0aa5e7a02f051cbbcb8f96f56d2e066b2240ddc20256b1659e23402d52ad65
CompactCells747To778.Box750.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:2664-2675	76565b629e403ccafb2737c75e9b102386410b169d001aa29801f4421515fa73
CompactCells747To778.Box751.QInterval	CompactCells747To778.lean:2676-2678	573cc79142032f1002bce1320478104d1917d412df4ee0abb90ff4cb2db011ed
CompactCells747To778.Box751.lambdaInterval	CompactCells747To778.lean:2679-2681	d4dd7446bcba418861c192b89c894dca06f4e515f0a8d43f4071cb2e6dd5747a
CompactCells747To778.Box751.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:3328-3339	ee945760de07fb4ffe8055085f4120f59ac5dad18a1f5d42388b8085fe7c3551
CompactCells747To778.Box752.QInterval	CompactCells747To778.lean:3340-3342	bfe7af7d8d1705f0ed35350e396c31cd803b6deafa8d71121bae42f3e4838657
CompactCells747To778.Box752.lambdaInterval	CompactCells747To778.lean:3343-3345	557a72ccbbcf42b10bad1fab706920388cce8d6f02076fdb23cc8111d36ee76

Declaration	Exact source	SHA-256
CompactCells747To778.Box752.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:3992-4003	7f1b494bbb4ef3509cd66027e77bb09347ef8daca446edd70b92748e1c21e077
CompactCells747To778.Box753.QInterval	CompactCells747To778.lean:4004-4006	2d7b28872bef5fcebff8d9e90dece714573cdb32b86da495bf42e35ace43f951
CompactCells747To778.Box753.lambdaInterval	CompactCells747To778.lean:4007-4009	b7a7ce2d6a70e60bd7e5d3f22371432c62c33f27abb20f7812c9c220fe263c17
CompactCells747To778.Box753.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:4656-4667	bb998d2d6727c8b294846f13ce6c60b8fff732008eeb31b0aba965ea8dce9668
CompactCells747To778.Box754.QInterval	CompactCells747To778.lean:4668-4670	4c90d8bd6b8d7ca549792d936e616ae5b0559b3097cec5050e3d4476bd348168
CompactCells747To778.Box754.lambdaInterval	CompactCells747To778.lean:4671-4673	8543621764f67646bead79e1fde6d784eaf10399305f235376db788d3e52853f
CompactCells747To778.Box754.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:5320-5331	663e6477c9d9e4f57dd74511aa70c820fba818190d57f1b60d4279fcf7cafe62
CompactCells747To778.Box755.QInterval	CompactCells747To778.lean:5332-5334	734fca154d1dfc3f1438f48f771ff9adfe6fb691efb9153180177c0b1aa3ea60
CompactCells747To778.Box755.lambdaInterval	CompactCells747To778.lean:5335-5337	fa1841abd7846f13c32a01abdd24a22458c22d4ffa89fafdf16d5c9f170180a
CompactCells747To778.Box755.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:5984-5995	c8e28afab8750ee6c3b0644484453cedef746cd3b4eccfaac14f3bfe55f7663d
CompactCells747To778.Box756.QInterval	CompactCells747To778.lean:5996-5998	207da29d30f9b036467ca6d99cf6f1c7c310874d5254bb38677ec3c6368abbf0
CompactCells747To778.Box756.lambdaInterval	CompactCells747To778.lean:5999-6001	967f28135edca2f7afc6aea8db703151d4679c7e17c4d8b5d81ec2f165bd5532
CompactCells747To778.Box756.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:6648-6659	12ec0e4124be940d3a6ce407bf3695dab89f9a03f94d04bfc0c66be83e5d1a1e
CompactCells747To778.Box757.QInterval	CompactCells747To778.lean:6660-6662	f55af1640956d21de1510a8eb5319381ca394125f6a1e974131ec6c32934a803
CompactCells747To778.Box757.lambdaInterval	CompactCells747To778.lean:6663-6665	bc41a6aed737d0e387b5b98a56dbce7423033bc475576260bd50061ba423c9c2
CompactCells747To778.Box757.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:7312-7323	8a4ff5a0ee720666fc80f86defe13aaaa28828d19f45bd3be5f8d8122f989577
CompactCells747To778.Box758.QInterval	CompactCells747To778.lean:7324-7326	bb695c095cc04d9668869facafe51f1a6fab9a4a116c9f418686aae3007c2df9
CompactCells747To778.Box758.lambdaInterval	CompactCells747To778.lean:7327-7329	1b284190499e93b638d772ea5461b9cd20a5a8768c8e61f1da1fc6faa04d731c
CompactCells747To778.Box758.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:7976-7987	f9eab568ff90a32f04b22de0fc0f2a7bc5bb8c9df5a4939e09df3d3bdf7d9289
CompactCells747To778.Box759.QInterval	CompactCells747To778.lean:7988-7990	0036048e64e61af0f60954b3bfb1d4a1035f50fed3dd73a6ddecbcb62430ff75d
CompactCells747To778.Box759.lambdaInterval	CompactCells747To778.lean:7991-7993	94115b2a7f6f057bb1bbe345c44f44ef41ff0858b0003fb319e0c13762a5440b
CompactCells747To778.Box759.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:8640-8651	8e359b032948e0bd322d17995ce218d783ed7b909a67ccab22c3e93f67b0ef76

Declaration	Exact source	SHA-256
CompactCells747To778.Box760.QInterval	CompactCells747To778.lean:8652-8654	45f051b8a787d05987c9977bd0233e5fe8bd4da943f6f44d46032ec4616142d4
CompactCells747To778.Box760.lambdaInterval	CompactCells747To778.lean:8655-8657	db3b94ef65e5a25dd91f8c5322c64a64913ea75aaa6a2545d14f720682828c3e
CompactCells747To778.Box760.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:9304-9315	ac5272e796c5f2d554487b607159d0247b619e38d01db0606f91fef728fd62bf
CompactCells747To778.Box761.QInterval	CompactCells747To778.lean:9316-9318	b78b67fc5244ff354ed0aaaf8cda4f35a86e6e2bc7e5ff350df724cbb16a4c2f
CompactCells747To778.Box761.lambdaInterval	CompactCells747To778.lean:9319-9321	bbd1609ece8489a707c9090b4d472a907dfa7d8db25700e03ad8f6245ad2ff59
CompactCells747To778.Box761.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:9968-9979	a85aa4fe7070f4c159ff6d60f5742380ffb6ab3ad412a5ff8197215d2f987612
CompactCells747To778.Box762.QInterval	CompactCells747To778.lean:9980-9982	4bbb08d05e33c79cbc59f8ed908a0d2b420170584c71e5c7f7408d58ebd10b23
CompactCells747To778.Box762.lambdaInterval	CompactCells747To778.lean:9983-9985	31805b446adb8ba6141c24ec58434567ece878cb4fe1b31d55866146f3528895
CompactCells747To778.Box762.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:10632-10643	4141dde4222af05bfaa19cd9eeb1eead8e720ff23a89e878c0111bc122b4e22c
CompactCells747To778.Box763.QInterval	CompactCells747To778.lean:10644-10646	c35c69c98198f31b3198bbb52063f1428bc87b011f9fa0cf59dd581d3d34e18a
CompactCells747To778.Box763.lambdaInterval	CompactCells747To778.lean:10647-10649	6ff5f49e466f1e227f55cb144453f753ae498d63305de3b7a2285f1958dcebf0
CompactCells747To778.Box763.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:11296-11307	1029524dc87fd9bbd59b3bbebb5678ebe68675efc8d6c69988d0deadc77455d
CompactCells747To778.Box764.QInterval	CompactCells747To778.lean:11308-11310	bdbe11ac1d6101e96ac42925e8b37a07b5b582abb0f46c41684ddc06d8cc265b
CompactCells747To778.Box764.lambdaInterval	CompactCells747To778.lean:11311-11313	e72d62dbc528e5b37ba04fe78d1f86bd93f4967959e5cc2b211d03ad422992d4
CompactCells747To778.Box764.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:11960-11971	0d34022c4b0a7300944579d49b5fadb64bf790772b1ded821a0e4e56d34b82e8
CompactCells747To778.Box765.QInterval	CompactCells747To778.lean:11972-11974	fa528822efe6678784707e39b8a0488e0297f9313ee1850dd2b9dbb20e2ef5a5
CompactCells747To778.Box765.lambdaInterval	CompactCells747To778.lean:11975-11977	70a6e5ac406f156d425ab77755b42a462b3515b8994d23efa01e843040a166a2
CompactCells747To778.Box765.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:12624-12635	636f6a7328c62e72ac81c2c5f548d85eecfa6a5f2b05766ff8ec977ddfe03d05
CompactCells747To778.Box766.QInterval	CompactCells747To778.lean:12636-12638	261f69c281980d55c82d21fea53eb7478b10b0ce7a54536d09c5ed2e8568e69a
CompactCells747To778.Box766.lambdaInterval	CompactCells747To778.lean:12639-12641	3f985445d8cc8937a71ffaed2233bb38b1de0fd3cc7d843c9e4efd15ef7797ab
CompactCells747To778.Box766.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:13288-13299	6c1cee18d41b3fd5c80390353d9c9b9a93a822a90568304e6e854f10ae37a5e3
CompactCells747To778.Box767.QInterval	CompactCells747To778.lean:13300-13302	3e8426aebe234bce4a965d923ae6402c566331b0839f78265649fd5fb3b513f6
CompactCells747To778.Box767.lambdaInterval	CompactCells747To778.lean:13303-13305	920d686317dfd53d0ea46ace4a4b5cdd7e49e6bc1d4775e18427a5694f953b8a

Declaration	Exact source	SHA-256
CompactCells747To778.Box767.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:13952-13963	c280219e3a883720034547696007830cebf56977f1dba6f6620478d0220378f
CompactCells747To778.Box768.QInterval	CompactCells747To778.lean:13964-13966	ce58f14592ae9294558b2f569b134265f0d104b91c00b17eaaafeed7ad3245aa
CompactCells747To778.Box768.lambdaInterval	CompactCells747To778.lean:13967-13969	90ebba86c3a5cb23fbe1b9d2ef12668870c4fece121b55be6f0a5815cafb4534
CompactCells747To778.Box768.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:14616-14627	38e219ab7240827d65ed9bcce8d0fde5ef277a0c077b616acb2c22969d2b33fc
CompactCells747To778.Box769.QInterval	CompactCells747To778.lean:14628-14630	d80c3dd1d8cc4d739f41a6e67911f62983d59e68f95b5490b63bf22a437ea460
CompactCells747To778.Box769.lambdaInterval	CompactCells747To778.lean:14631-14633	90e70f402f22ff0a78710f1e7e6199c7602cddecc47a88c64c7c80896bb5139b
CompactCells747To778.Box769.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:15280-15291	2bfc107947099f2fc235f5e2f3234fb7d31f35b0c90d3b3d703eed0a7914ca02
CompactCells747To778.Box770.QInterval	CompactCells747To778.lean:15292-15294	ac818adc2c387c119294220da638c01f7f1949608075974486ab8f04ac45c7f3
CompactCells747To778.Box770.lambdaInterval	CompactCells747To778.lean:15295-15297	4432ec3495a8affdfaacc4524877e14f952d88671a99fd9835c5cd82c232b76d8
CompactCells747To778.Box770.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:15944-15955	ff5b3e3efb6681afe10001156408a31ea434c32acead5be2bff82fe07e3c1c2a
CompactCells747To778.Box771.QInterval	CompactCells747To778.lean:15956-15958	5522bd63e78124243bb63f9c79df39887c95f37eb4e4202eafdb3eaa44408f07
CompactCells747To778.Box771.lambdaInterval	CompactCells747To778.lean:15959-15961	578e1e06f9729bc46a2c5de306ce2b3959494fa3c1c79cf3a3abb8b36be01ae8
CompactCells747To778.Box771.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:16608-16619	02907ed66e87165aedce0aef14dd0fd932feb2f2469a4c9cae48b126bc1ecd65
CompactCells747To778.Box772.QInterval	CompactCells747To778.lean:16620-16622	c350a5329e96360d238d41cb31bdcb932be7c083e4b9c4fd154cd694ef199870
CompactCells747To778.Box772.lambdaInterval	CompactCells747To778.lean:16623-16625	aa8e87c5b9a4888143fae7d9b308089dba8e74166d4dc83fb82c068c8d2e1abe
CompactCells747To778.Box772.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:17272-17283	0e540fa3f4dff5fb97e2d36f5a297394a28fddd1c772bafef718e046c82a7f07
CompactCells747To778.Box773.QInterval	CompactCells747To778.lean:17284-17286	921ff95c5f1eba157cec52df4ee34405b8f25258d6551970d7346a2d9977ddaf
CompactCells747To778.Box773.lambdaInterval	CompactCells747To778.lean:17287-17289	c812b29c354d1eba4b0be8671a1274ae2e9e8aa1571d5bbde47e7fadf7740f
CompactCells747To778.Box773.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:17936-17947	3ef389a9f73cb4813b7a3013cf26502a5c56d1c78b557e9f551bd4824e6f09ff
CompactCells747To778.Box774.QInterval	CompactCells747To778.lean:17948-17950	9b96b3ff4376804030e9cf793bcc06d5ef1829e7b0ea477da5dfdc339c14ed06
CompactCells747To778.Box774.lambdaInterval	CompactCells747To778.lean:17951-17953	47bf6b2beb2178512ab860b3414dca4250822b23b0a433316d9dd623b86599d8
CompactCells747To778.Box774.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:18600-18611	89e55a16142eb4ba72e5c4cd68e6b75fd5dec64e2271afe7f9e15a4bec94bc8e

Declaration	Exact source	SHA-256
CompactCells747To778.Box775.QInterval	CompactCells747To778.lean:18612-18614	f21ba99aafd36970ebfa2e0085e4b96ed0ca6e959625bdeb949ade91ba2eb102
CompactCells747To778.Box775.lambdInterval	CompactCells747To778.lean:18615-18617	64ae2e62f551dbbf65a1f1bf3920c6d7ab9c1cf630c57641c4188a7d97f1bb9c
CompactCells747To778.Box775.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:19264-19275	43fbf3edc5cf3505e908e718f1493e03c925b326f0ca8f91bb2637bd1aaac273
CompactCells747To778.Box776.QInterval	CompactCells747To778.lean:19276-19278	0414c453d80a75d2f1abcc44b29bc1a1877d02d3b04da7e70a5cf9d08941f595
CompactCells747To778.Box776.lambdInterval	CompactCells747To778.lean:19279-19281	38a3409dca522b9a0530f78baeb293b95463f3a7e1a04b7753bc3a49fae9099e
CompactCells747To778.Box776.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:19928-19939	cbbfba2fd5b90fecb37a1d3bdee05bd8ea401b0588d890fbc52b81890fb5a2ed
CompactCells747To778.Box777.QInterval	CompactCells747To778.lean:19940-19942	e74000afbb70ef034562837cc94a146c0cd58fc7a7a27c149c82b2cfbd6e3c0f
CompactCells747To778.Box777.lambdInterval	CompactCells747To778.lean:19943-19945	2cae130236c8ef4d10dec41da95aade8598160ca3999295a039d36eb3e9218e3
CompactCells747To778.Box777.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:20592-20603	fbf89579fba31e56540b5005aeba02c175dcbe2ee7e7ce62c028d3ad7df6a8e3
CompactCells747To778.Box778.QInterval	CompactCells747To778.lean:20604-20606	bd001c0079cc452c820d4f30a87590f71cf1a17f464341e7d4cc5f2a801b0bee
CompactCells747To778.Box778.lambdInterval	CompactCells747To778.lean:20607-20609	107b379b7d3f4b73cd5e0c58568c3ec6e08741b17afdd44f6d1836d03750c2c0
CompactCells747To778.Box778.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:21256-21267	a93edd7bb6f265a20d621d2cca0def30103d0f8604b1b7a2cd9a437300dae730
CompactCells747To778.bboxes747To778_candidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:21268-21498	88cd35cedccf4ca857edac090c27fc142ad622d3b277c38a6f5a21601f23d1c0
CompactCells747To778.bboxes0To778_candidate_not_isWeightedPerimeterMinimizer	CompactCells747To778.lean:21499-21512	89260661d6f382750b7663e0fcbb1a1f311d22dc0f16491baffa94d97c0bcc14
CompactCells75To106.Box75.QInterval	CompactCells75To106.lean:20-22	fd26d809bf758506b1df3de0411c2bff2cbf59e533ece162dc328a5871de099a
CompactCells75To106.Box75.lambdInterval	CompactCells75To106.lean:23-25	b7c912126cad4302a9d52ca0ee70753f6c3e6cafe86cd8bc55838300d96850ad
CompactCells75To106.Box75.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:672-683	20b670a377ee5d780649296de359989a7d2a6e97412b86839c55de2fe22902fc
CompactCells75To106.Box76.QInterval	CompactCells75To106.lean:684-686	609c222e12dba255947b462f4b3571ca2b583018a4967f6356b938c3df3ce2af
CompactCells75To106.Box76.lambdInterval	CompactCells75To106.lean:687-689	af9f420f83f052c648407f463ed0fb4b81a070334cd0728be33c5a2b8407c5e0
CompactCells75To106.Box76.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:1336-1347	e541834560f054b1b493e17548dc4d4cf6b250a5ae45326316386d09eaa3c0b7
CompactCells75To106.Box77.QInterval	CompactCells75To106.lean:1348-1350	8d235171185a2d78af604ac7f1763344a0bc9d660a5a69220601c2b3cbe44681
CompactCells75To106.Box77.lambdInterval	CompactCells75To106.lean:1351-1353	e8d845f3a51faf13e9c9f6ab69a1e3ea145d6278695d26539c38d69641a12654

Declaration	Exact source	SHA-256
CompactCells75To106.Box77.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:2000-2011	2d96c5a64510ff3d69acf4a5b755cdcaffb3797d68e3a45529b1c173362abe95
CompactCells75To106.Box78.QInterval	CompactCells75To106.lean:2012-2014	0466f075f5de220f27b8f8e770490852422d14d152a6f3ff1694d1759f6a96cf
CompactCells75To106.Box78.lambdaInterval	CompactCells75To106.lean:2015-2017	90c9a9d45e0b532dfb8322a9936cdb46a22c1ae0ead561823cb6a698e9b59f60
CompactCells75To106.Box78.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:2664-2675	e3bb6fd795f2e896c6059824186dd88e67f293c463e05c46e5fe48df27c964fb
CompactCells75To106.Box79.QInterval	CompactCells75To106.lean:2676-2678	0f6be7a6803f0f35e59b5e280e7dfe8bd217f973093071ddaf1d7502c897f86
CompactCells75To106.Box79.lambdaInterval	CompactCells75To106.lean:2679-2681	9b519bc5014de061ba05c308933a1300bf747457a8377d5ec5667362fdb8e09b
CompactCells75To106.Box79.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:3328-3339	03d179a89f209353e1f48b848f72be4c2db50261f3165a6b9a3e9baed7983fdf
CompactCells75To106.Box80.QInterval	CompactCells75To106.lean:3340-3342	5ff2fc026969991ef9229deadeedbbd4a00dd0c4f5fd495b8a0f3ec4d2bad574
CompactCells75To106.Box80.lambdaInterval	CompactCells75To106.lean:3343-3345	5c6349bb8347e8ab10aab01950cc36605a66a0217a94bb246f2dcc65cd10ddf6
CompactCells75To106.Box80.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:3992-4003	47b6ff5df2f77c27e008d78d04500c62d9073cd2924f1d22c2679a6a2731040b
CompactCells75To106.Box81.QInterval	CompactCells75To106.lean:4004-4006	071a5e8464a2af9c69b1a3ac78dccbe7557298a8cb3ffce534703b15b618389c
CompactCells75To106.Box81.lambdaInterval	CompactCells75To106.lean:4007-4009	6aeb76b92868542091d755b8579cae53b5e0203e2966c8397171ab1266f8bdd8
CompactCells75To106.Box81.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:4656-4667	5ff5fa935cb0be5bd8f1b185839656cd6f41c225cf8793e42b823433ad74cb0c
CompactCells75To106.Box82.QInterval	CompactCells75To106.lean:4668-4670	a5e06805262101292152ac4e3b38cfe1df95d7862e8ecbe6645b7aeeb73e5fee
CompactCells75To106.Box82.lambdaInterval	CompactCells75To106.lean:4671-4673	9792b5499142134f271d94f6b4445f78e605e9cc61cd6252cf16cbf9efd7cb18
CompactCells75To106.Box82.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:5320-5331	1bb1a2acc2fa64eb9ce4eca67c4031367542c31d322423bf5cef1ab386bc2009
CompactCells75To106.Box83.QInterval	CompactCells75To106.lean:5332-5334	52878858ee8493390c442fa97b85f7e14816e17553f4aaf3710125c193c037ef
CompactCells75To106.Box83.lambdaInterval	CompactCells75To106.lean:5335-5337	a8542f2e10a52c1fb4afe08b79cabafbb228eda65d75219c45857760bbfa19ce
CompactCells75To106.Box83.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:5984-5995	158a5e77fc848d58d05f7e2a5b3d7e372cd0542ed6fbd0e4f40c3c6f34c8cac2
CompactCells75To106.Box84.QInterval	CompactCells75To106.lean:5996-5998	01950c27d3074af280aed25f9d2952a775824b4018c4ee09930e174453239e7d
CompactCells75To106.Box84.lambdaInterval	CompactCells75To106.lean:5999-6001	a01bd4a426b8e9a79caa4b173d99cc58bed8358b5201224f6ba2064b80dc6a37
CompactCells75To106.Box84.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:6648-6659	9992241401a05e5965c210099d1b524af8d03a3bb46f05219161387654878086

Declaration	Exact source	SHA-256
CompactCells75To106.Box85.QInterval	CompactCells75To106.lean:6660-6662	f89eaa1876105aaeccc9f0726c399f0f7117559f72b786c9c5e12ffd2b3a7fe4
CompactCells75To106.Box85.lambdaInterval	CompactCells75To106.lean:6663-6665	2394452abd939a9a068ad6756620c9da1a3be0ed855bbb7877cdac6b15d2b103
CompactCells75To106.Box85.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:7312-7323	ae7d7543c0c599b907447f95e6e892ac18402cc54122ec0e9f483581859aed94
CompactCells75To106.Box86.QInterval	CompactCells75To106.lean:7324-7326	f691ec6624e21bd412cc9df91ee8870d00cdadc9d8502728af96acb4da1ce3e7
CompactCells75To106.Box86.lambdaInterval	CompactCells75To106.lean:7327-7329	b9afd4d63e602875d68b8ff398ef4aaf000646ce65eef3b95e85bad6a18fa583
CompactCells75To106.Box86.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:7976-7987	fdb54ce8adeeaf107b58abb7cb49dc7308095e011addaef81c194261368beed5
CompactCells75To106.Box87.QInterval	CompactCells75To106.lean:7988-7990	3bb5ceb75e174e3040db3ba6211dff6b4869eb9f2d5cbc336b40562badf830b1
CompactCells75To106.Box87.lambdaInterval	CompactCells75To106.lean:7991-7993	2d05cb02033f465fd5087b35c06daa724b6982eaa1534b3f1f9edaecd4304f8
CompactCells75To106.Box87.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:8640-8651	e17a83550a69de9efe7c3cdee2a0aedd35e5ab35145cd5e417a1ae4adaadeb7e
CompactCells75To106.Box88.QInterval	CompactCells75To106.lean:8652-8654	e74b7833ee9f66083566c8bca62b4e9ec28962e8570d0fda64e14bfa61c1f4df
CompactCells75To106.Box88.lambdaInterval	CompactCells75To106.lean:8655-8657	eef429fe8b424b9d0dbffbf7770422c8bdd576f4f0cd3f242c3a5d6caf351eb0e
CompactCells75To106.Box88.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:9304-9315	b4954bb0ed5184acddc2e612a6e8c0219759fb2c8fe890e35b41adb9897a35a9
CompactCells75To106.Box89.QInterval	CompactCells75To106.lean:9316-9318	49ea057f6068b923678f0bf637c4e1d7f0959fbd01d97b3e9dcc84d531be9ec8
CompactCells75To106.Box89.lambdaInterval	CompactCells75To106.lean:9319-9321	a18a5794a6710ecd44c727d545fac7789f7fafa43e5cc8189df0049fbc3df3de
CompactCells75To106.Box89.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:9968-9979	ee071cbfba03bfde533f3889d2c589bf61a832e200383fc35c27a9ad85cfa947
CompactCells75To106.Box90.QInterval	CompactCells75To106.lean:9980-9982	c89b26b7cb3f3551ab50077c95d331feb9ab63dfe45180fd43180e14547eaa0d
CompactCells75To106.Box90.lambdaInterval	CompactCells75To106.lean:9983-9985	313e6671225c16db14ba2f53259aa829925e1ebee6048c16d4ad1b1551eca9b6
CompactCells75To106.Box90.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:10632-10643	c03dbd7a7a7fcafe16206c03c6eb022f5a5d1d0e36b9c856e3b3c4f962202b35
CompactCells75To106.Box91.QInterval	CompactCells75To106.lean:10644-10646	aa884b881753d2a7c0d1725d276f2fe3ac9d4d95b469e53876fa69ab31b2ef0d
CompactCells75To106.Box91.lambdaInterval	CompactCells75To106.lean:10647-10649	daf9c4ff13143c1721ef44b1999f110c76d62817105b6da972193b02dc416f39
CompactCells75To106.Box91.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:11296-11307	34306eed505f7811ca7e8bb2910dd4a958eb63d5387adba2ec5607f90aed25f2
CompactCells75To106.Box92.QInterval	CompactCells75To106.lean:11308-11310	3d65ce750c63050a7a39e0f223cdea14381a083dfac5f7ec01d15aeca99cd0b3
CompactCells75To106.Box92.lambdaInterval	CompactCells75To106.lean:11311-11313	82b3195c62b3e6e3e51bef16c8221e251a3a7d3c2ac9ec9b75e95a7577b0f6c2

Declaration	Exact source	SHA-256
CompactCells75To106.Box92.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:11960-11971	94e60313659aae4aedc71e25d9131c58bd043c80a3dd7fb5f617350853c93508
CompactCells75To106.Box93.QInterval	CompactCells75To106.lean:11972-11974	aae3fefad9104d021468131cfbc6be6487cd881748cc95d905d87105bf65050
CompactCells75To106.Box93.lambdaInterval	CompactCells75To106.lean:11975-11977	2d201c7c4be769b354692492e0ec55f2f23993d497f929e52285e049e8bd3175
CompactCells75To106.Box93.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:12624-12635	af43d8267201072c4853c3c6c1aef16446c76258ba5ebb7e7d40b82c72851389
CompactCells75To106.Box94.QInterval	CompactCells75To106.lean:12636-12638	d7dc7480a618e21323665653ad6e92ab6f64dbc1e90f5db5e6da62b0df43c53b
CompactCells75To106.Box94.lambdaInterval	CompactCells75To106.lean:12639-12641	64d10b3cc1e82aa5ce26339647b7e9442729cc5c238f91d564bb1f263228d404
CompactCells75To106.Box94.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:13288-13299	0dfaceac154e90547ab5b58c81503a83803abf7f0d09e61d219e6d7c5e7af75a
CompactCells75To106.Box95.QInterval	CompactCells75To106.lean:13300-13302	49e3d05150325285f8258bb0842c4cb193c5e9fc486c7d0cebcfb52553a37dae
CompactCells75To106.Box95.lambdaInterval	CompactCells75To106.lean:13303-13305	d0ec1eca90265273d63c0dce5bff7233a2d7cf57fe934975ff4c151db94a267f
CompactCells75To106.Box95.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:13952-13963	3593cbd4afa44eef75c984cd7372695d38713004fc0c2bd29564d433f854c2b9
CompactCells75To106.Box96.QInterval	CompactCells75To106.lean:13964-13966	f9f6e5cb5dd0c4c1eabbab1e473ecc81d1d7a41e8aa5c92ebc12575ef3401bef
CompactCells75To106.Box96.lambdaInterval	CompactCells75To106.lean:13967-13969	b61e9ff13f114c977da2691f67bc6ecf0d3d4a12b8c02da61c8239a3e39772c
CompactCells75To106.Box96.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:14616-14627	d71f0f6ad9f1f2e718d27e3db90237a80bf867589a67fc463e37eabc35dad230
CompactCells75To106.Box97.QInterval	CompactCells75To106.lean:14628-14630	ecca3e67fd159ab73d5884635ccc97d43817aabdccacb576d86303dd5aab0bc0
CompactCells75To106.Box97.lambdaInterval	CompactCells75To106.lean:14631-14633	c083bad9b07bb71df65ee2917be7f91f27ff6ea1c22af8f5575d537ec71dca92
CompactCells75To106.Box97.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:15280-15291	f16649cbb3a0634a22c49a59994fbbbedb44cf2b886e311bf9482e00aaac4b1a2
CompactCells75To106.Box98.QInterval	CompactCells75To106.lean:15292-15294	44632bf5d93aa737ad2425bb468b06a7d683368e37dc6a44aaedbd8a8d08271
CompactCells75To106.Box98.lambdaInterval	CompactCells75To106.lean:15295-15297	a54e16b298ddf1e9d45be856a13991bb5a329769b40c49efcdbe1b23397749e0
CompactCells75To106.Box98.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:15944-15955	170e8243a6cf8e54eba99195472d1df971881aff0b33f7e49bff43ebbd293702
CompactCells75To106.Box99.QInterval	CompactCells75To106.lean:15956-15958	820b3a6cf2009b96a7ce8f4e8ea6440e3a5be83039d9bc8e5724bce95c2f7469
CompactCells75To106.Box99.lambdaInterval	CompactCells75To106.lean:15959-15961	babb8eb5a744daf05023119d90fbcf3d88995bc2c0b978076673d1c43d06bfee
CompactCells75To106.Box99.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:16608-16619	b31ec91256744cb24152a309d47a311ab0ad264ad12bb1f4d7926daf944380de

Declaration	Exact source	SHA-256
CompactCells75To106.Box100.QInterval	CompactCells75To106.lean:16620-16622	c0665446c4b09cd4dccd49cebad9e8e9d7718dbc f59d679a2c299d362adf9d3f
CompactCells75To106.Box100.lambaInterval	CompactCells75To106.lean:16623-16625	ae1e10a5c3ecee6ecf90a0eeeb280e45967436caa2217d92c6a785ad3895a80
CompactCells75To106.Box100.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:17272-17283	1635e986627b7096cf280d7051636e1055bc0de1c7a16746a5d984482785d5ee
CompactCells75To106.Box101.QInterval	CompactCells75To106.lean:17284-17286	79e96c7c6fe4ef48eea3530a9c78b6d1213bb5dbf9d255e09c83f33f114fb43b
CompactCells75To106.Box101.lambaInterval	CompactCells75To106.lean:17287-17289	153502c597f3de90d22f3dd008b1575d50ff81c50d0e458977d5ca8bad3a9f8f
CompactCells75To106.Box101.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:17936-17947	f9da92af0d33ca39b60f7274e5ded2063b869a5ebcf54ed22c524f2a3daf424e
CompactCells75To106.Box102.QInterval	CompactCells75To106.lean:17948-17950	c9f4c678097594de1ef28352d3276c31ea555f3c82645e811e8a5a60f1196c0a
CompactCells75To106.Box102.lambaInterval	CompactCells75To106.lean:17951-17953	dd70a687f1cde29eb5167cd60f62d6d76216348365f53d83297a45ba72d71fe4
CompactCells75To106.Box102.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:18600-18611	266fe654a0be25dec662dfdc013091fdb2b4a5ae914bba2859dc1a2058cc8ded
CompactCells75To106.Box103.QInterval	CompactCells75To106.lean:18612-18614	4250225c62f762dceec61bca992f3f35df5a3ea073a13807e3024ad13c18255c
CompactCells75To106.Box103.lambaInterval	CompactCells75To106.lean:18615-18617	c88f4bd77b6be6d3d11687148ca79c4d40017c2f290497f154b40a3d3ab0d16
CompactCells75To106.Box103.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:19264-19275	5c7ad453925d87153ee8edaf204656d23ea2e3cf538a5bd96656158b06eb53f9
CompactCells75To106.Box104.QInterval	CompactCells75To106.lean:19276-19278	5ffe3d992c83a6f351676aa9fc5cd73710f7991dd2b94a6025dcf3d50a0503ed
CompactCells75To106.Box104.lambaInterval	CompactCells75To106.lean:19279-19281	4ace804745fea95dc4db1d12e0ab32cc3528db5ca13112a8fdb2fdbb01fb1676
CompactCells75To106.Box104.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:19928-19939	7ef30d6cbe602d3b58fc250f639493e024df79f8d82aba4cd88a919e421c6518
CompactCells75To106.Box105.QInterval	CompactCells75To106.lean:19940-19942	a4d6303c0ac7f7dd5308aaeb2ebfb75a1c89c879e2e6fb6750395084a507d92b
CompactCells75To106.Box105.lambaInterval	CompactCells75To106.lean:19943-19945	4cf2a3880bd7b50323133abd994a7a75c23b81b62fe83d2065620d6d5bf00b11
CompactCells75To106.Box105.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:20592-20603	e7899334c29d7b874d9946e09ddf779f06960203fb33a89ca21d24860afb6534
CompactCells75To106.Box106.QInterval	CompactCells75To106.lean:20604-20606	2fcbd7355933982c510e0b3b7d3304442e3d57338fc0ea5e5c89edde9166a7ea
CompactCells75To106.Box106.lambaInterval	CompactCells75To106.lean:20607-20609	ef88cb5feba6caf62154ccc6b775cca2abe1f937ecf0918479145e02153b8e5a
CompactCells75To106.Box106.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:21256-21267	c1a4a22e313edd96babb6284930a2cf555a8c9d3632bf1586e5a1253d08d9adf
CompactCells75To106.bboxes75To106_candidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:21268-21498	08b5f9cad78f29935aa4fef58417683c89c5deb79b1d08ac9db0225fdb5545d2

Declaration	Exact source	SHA-256
CompactCells75To106.Boxes0To106_candidate_not_isWeightedPerimeterMinimizer	CompactCells75To106.lean:21499-21512	1f26f7f057597424051b4b5292c617a1c9b7c66674c2de9f226def6336f15a73
CompactCells779To810.Box779.QInterval	CompactCells779To810.lean:20-22	030b21694da625662bd3de78ae9333a25c86224293eb04453bdd6419c7adf4bc
CompactCells779To810.Box779.lambdaInterval	CompactCells779To810.lean:23-25	f6cb4b161539aad39d91b5bed2f30a9742277ec3c2c256a018ce65ea53bf7024
CompactCells779To810.Box779.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:672-683	c60432de58f336849b93b812e304146e3e45fe7c83151324acd47485755e2a58
CompactCells779To810.Box780.QInterval	CompactCells779To810.lean:684-686	f28fa31810c2dbd3e0e58c6554dd46f71e256912a635b0405d53df0a22cdeea1
CompactCells779To810.Box780.lambdaInterval	CompactCells779To810.lean:687-689	1e0207a34224874ea77d0b0ef841421a43f84509f71e7f9df6c2dfa989f3e2fd
CompactCells779To810.Box780.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:1336-1347	08a5a44f382c92ba63ff4cde6a90be7bae243e7cca2020eb4e02c40a396117a1
CompactCells779To810.Box781.QInterval	CompactCells779To810.lean:1348-1350	45c150b78406546fd4c8e78e074f6f09d1f9c7be91822a61b371e15d233baa0c
CompactCells779To810.Box781.lambdaInterval	CompactCells779To810.lean:1351-1353	bb6b3a27440de100afce593dd3d7e8593d8c1bbed50078d168d7adafc20ba5dd
CompactCells779To810.Box781.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:2000-2011	edc17a0a2fedfa3ac14d5d1e7bc96a4ebfb2a3f3413e6547e1475435c38c51c5
CompactCells779To810.Box782.QInterval	CompactCells779To810.lean:2012-2014	4d11cd92b8d9d158383409f92a817f53776a156057d04b3125107c8fede955d3
CompactCells779To810.Box782.lambdaInterval	CompactCells779To810.lean:2015-2017	77aeae6897491b19021abda41e5835738dd8ced75a4c91ef86f786cb32e4838c
CompactCells779To810.Box782.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:2664-2675	df2af7c648dccd8d6bf2bc15652a284358efbbf5e7acebc82a5827ebfd2cceb6
CompactCells779To810.Box783.QInterval	CompactCells779To810.lean:2676-2678	321babcead80f5418646b51daccf0ea69cbcb75b873e0608c433df914d918071
CompactCells779To810.Box783.lambdaInterval	CompactCells779To810.lean:2679-2681	d2108bcfb2969d3c1bb4d01fe3def1b5fcf7e7253201f2d0e6cf372323ae137a
CompactCells779To810.Box783.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:3328-3339	118958e40e77e235fe90e65c2210fe742818e8b0064d2f39252f28d71276613e
CompactCells779To810.Box784.QInterval	CompactCells779To810.lean:3340-3342	4ebcf9ef044c934a57ddebb49918e8596dd2fe52e58ce298c2d1544c73513d5e
CompactCells779To810.Box784.lambdaInterval	CompactCells779To810.lean:3343-3345	1d12fe7b2bac91622a30799394db4bab6a9566a1f92918e9c169a6b2fad5886
CompactCells779To810.Box784.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:3992-4003	01c3b0609e6aaccff5a21fa182f6c63117a726922e3718d13ce157a704c2967ce
CompactCells779To810.Box785.QInterval	CompactCells779To810.lean:4004-4006	52bd634adb3f4834569415b582a285fcd6d17d880bc8e16f3ac29986b7edb72
CompactCells779To810.Box785.lambdaInterval	CompactCells779To810.lean:4007-4009	0c4b81277c9458a9278b70fb4659211053a239c8d998d69b3adb8db93084f4f9
CompactCells779To810.Box785.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:4656-4667	2d190814b9b60540c912b0192bffb0e6eb10a381e2cb0f64288ce56bdc0dfc4

Declaration	Exact source	SHA-256
CompactCells779To810.Box786.QInterval	CompactCells779To810.lean:4668-4670	1b27d7523d0d11459e7ce111eac4f27ea3fb774cf6723c5cd8c4a281ae7bdd5b
CompactCells779To810.Box786.lambdaInterval	CompactCells779To810.lean:4671-4673	6227e0b4f748afc63a79ce9601827e1a3066ffa00a1c284be700d4f69e6cfa6b
CompactCells779To810.Box786.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:5320-5331	8f8c81fc20d841168087edb4567714db44a50ae62cc63d3a66fdd0ffd31ce287
CompactCells779To810.Box787.QInterval	CompactCells779To810.lean:5332-5334	649b5e7087256855a9eb1cfa25dfffea5d43b28eb85159a8d77d34059a5ead6d
CompactCells779To810.Box787.lambdaInterval	CompactCells779To810.lean:5335-5337	45b2c4d6bcfd25d038d0671563ecc99d22801edea4e5c43909fff1e43fda0ee8a
CompactCells779To810.Box787.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:5984-5995	366fb18235f35438954d87149736f7194e786292bb8cd6867d0511bc5a9ea15f
CompactCells779To810.Box788.QInterval	CompactCells779To810.lean:5996-5998	9ce280bc542f3f71bae835dce32a8d1b95adc1553ee9285151f50ea43f2d2d13
CompactCells779To810.Box788.lambdaInterval	CompactCells779To810.lean:5999-6001	365b8af8cd84ccf3789a1b0ad0cc044b05b264a30021148a3852fd7cf64507ee
CompactCells779To810.Box788.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:6648-6659	4c6978db1fe8fff76162dcadab29de31a051f27b5db0955db69afd70b96d5e7a
CompactCells779To810.Box789.QInterval	CompactCells779To810.lean:6660-6662	9506ce56a7cdd60daf365d51c962f151b036cc2fe7f2df7fb618b0cce8213d4b
CompactCells779To810.Box789.lambdaInterval	CompactCells779To810.lean:6663-6665	5e75daf3301a0e9850805d85692b59f6cd3455cb2b075c84d4255b7ccaa6a27f
CompactCells779To810.Box789.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:7312-7323	ff16facbe237e66b719f1e7f53326aec7bd74b46fd51ef527b317ba86868311d
CompactCells779To810.Box790.QInterval	CompactCells779To810.lean:7324-7326	e76f5a28aa7c273abd2e99c4a7cefeb3e4f34b9325a2230a0c3317c26dbd0579
CompactCells779To810.Box790.lambdaInterval	CompactCells779To810.lean:7327-7329	fd13bf7d3a5397a700d5c7fee32bb75695321b27f8be37f209bf46f4c8e95439
CompactCells779To810.Box790.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:7976-7987	a2a1cf94d4809f14b97ce5cab377427609daf01620fc26f1e102c61fac9dfcc9
CompactCells779To810.Box791.QInterval	CompactCells779To810.lean:7988-7990	8236c8ca56f099209eb1f9752c25d213e3e8068b8157c05ec8622e38f0095bac
CompactCells779To810.Box791.lambdaInterval	CompactCells779To810.lean:7991-7993	5462d6e22fe1bfc642820fbe48de5cff1004cddbdc539dfd3555bfa239e431af
CompactCells779To810.Box791.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:8640-8651	44b2c2ad904e55cd55cbe5aaabe03a898ecba13d8762ec314ad9a0b10658f0c7
CompactCells779To810.Box792.QInterval	CompactCells779To810.lean:8652-8654	6e928fb641fb9650c1c50e45053b1b3d3b58655ef16fb1878efb1d6d121332e6
CompactCells779To810.Box792.lambdaInterval	CompactCells779To810.lean:8655-8657	583b03efc90251024f22fe1513c518491b2d092a4d8df7378be6d906864d6b7e
CompactCells779To810.Box792.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:9304-9315	158f0b493585e264639ef46046044c2783273e533f8322849495bf6b39945013
CompactCells779To810.Box793.QInterval	CompactCells779To810.lean:9316-9318	0eb0b5474791c5e8cba77fcee2b40432b2175e16cf27c9e22704f901f1aa08ad
CompactCells779To810.Box793.lambdaInterval	CompactCells779To810.lean:9319-9321	4bcd5d50572f94cdfd5b69b1a7275900df83f9b02fc0a2a5782cda4864637fc

Declaration	Exact source	SHA-256
CompactCells779To810.Box793.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:9968-9979	2c4f864242ef57b0956a44fa0729b3bccdfce41b2fcaae45f396e769f8f17dcf
CompactCells779To810.Box794.QInterval	CompactCells779To810.lean:9980-9982	975b03a71c7e69c77a396f2d4991629aa023371283390b5e6f5cba33ec9c2f3f
CompactCells779To810.Box794.lambdaInterval	CompactCells779To810.lean:9983-9985	678ea445087918ec53cca4b81b822ec833b6546721aac8c1e2c8fc15ce56f752
CompactCells779To810.Box794.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:10632-10643	23ddf15ed9e6fe80f87cc843a9b02c16dbd70204e35a140ffff4d3b864f901b99
CompactCells779To810.Box795.QInterval	CompactCells779To810.lean:10644-10646	d7f9ebae8f13afd97215e3d90643da063aa870a764a6b354acd198063fe8f2c
CompactCells779To810.Box795.lambdaInterval	CompactCells779To810.lean:10647-10649	2ba3b46cdf71cb48e88365324f32b515b2699cd2fb619c6216424b9981bc2b33
CompactCells779To810.Box795.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:11296-11307	6445d0d9c1eceb556604a0f148434607a08c97e5955c812bded442720ff02d0e
CompactCells779To810.Box796.QInterval	CompactCells779To810.lean:11308-11310	6598794ca535fa993a41880fa033db0668d59c0a31b84a2788b70c2de8ad2403
CompactCells779To810.Box796.lambdaInterval	CompactCells779To810.lean:11311-11313	bbef2e243f4aab89aacbcddece73a20f51dd340b0e3dd83dca37e2f56b63c7ac
CompactCells779To810.Box796.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:11960-11971	22a39ca33038c4cf6184a4bcfe1b5805ac5ee6e2f5df0a56261c5f4f65b3d2d1
CompactCells779To810.Box797.QInterval	CompactCells779To810.lean:11972-11974	bb52862b48bc7eaf933899c5a4a531e9531f81157df2a5f118d57a3f9f6d885c
CompactCells779To810.Box797.lambdaInterval	CompactCells779To810.lean:11975-11977	ae582791fdb82c5e53ec7a6ca5172e23cb37b8d6b00744db93df94f84b0d1f01
CompactCells779To810.Box797.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:12624-12635	5694207737dd1aa1f261dcb472b59f918980a0e16ceeee6534ef92e33aabfec5
CompactCells779To810.Box798.QInterval	CompactCells779To810.lean:12636-12638	96528bafd0acd524a5e5c92ef68c50d23595eb7f5ad806b47ed71343fe63d50e
CompactCells779To810.Box798.lambdaInterval	CompactCells779To810.lean:12639-12641	cf652e86fd35d529b6c94c4c3704664741989b6b38c956a5d66327a0d0764883
CompactCells779To810.Box798.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:13288-13299	584a534585e76896b23479d66cc336190426e328223134df5eccb4b22e7a17f6
CompactCells779To810.Box799.QInterval	CompactCells779To810.lean:13300-13302	80c6e6814a7e80d26171efc7e4a612aed349eef6ddf42f4220fa69240411130c
CompactCells779To810.Box799.lambdaInterval	CompactCells779To810.lean:13303-13305	6dfabacd257632744b7c29a40a076b218e0c948ffcd40ee967b68f19c0d2289f3
CompactCells779To810.Box799.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:13952-13963	eae3ea4a96654b0b204566731434de2c2a184c4af09f150ed4560293880fb0b3
CompactCells779To810.Box800.QInterval	CompactCells779To810.lean:13964-13966	0d42d0de563c01eb97334c6987c13d05ef2133f659cf87e514888de9bf5f1f4c
CompactCells779To810.Box800.lambdaInterval	CompactCells779To810.lean:13967-13969	e466e7054c467bc81481035844c665dadf3a4e73fc79de60eab5587315cd925c
CompactCells779To810.Box800.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:14616-14627	fad0a26b065bca720365b2cd574bd53f7ebca032a42f68c19c60de74c71f852e

Declaration	Exact source	SHA-256
CompactCells779To810.Box801.QInterval	CompactCells779To810.lean:14628-14630	8487a4e018e87b80d4d511734bb4140273dda378cc618bc9d87795f4ea4fbf30
CompactCells779To810.Box801.lambdaInterval	CompactCells779To810.lean:14631-14633	a45033974ac1e9a473a9b04f420190b30ffb56976c32443ba4905ded465c8de5
CompactCells779To810.Box801.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:15280-15291	5bcb2377c16938368f0088da76f7a2d0f6be665c50470f272e78807b64c1ca74
CompactCells779To810.Box802.QInterval	CompactCells779To810.lean:15292-15294	02ffbd987af00760491826eed0b31b7806f9bd6d19512aa8839fd45cc06f8232
CompactCells779To810.Box802.lambdaInterval	CompactCells779To810.lean:15295-15297	ae9bd7d7833a95b9ff2b397b21c1e2332d67013b5dd80e69123984f0cf7d6560
CompactCells779To810.Box802.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:15944-15955	b73ae84368799047e227a7c846c53b42280fca926071d086938441ea879ab3cd
CompactCells779To810.Box803.QInterval	CompactCells779To810.lean:15956-15958	af69038a49fc4cbd5305925e0c840b2100affb65b316c5e6064222d3d194c8dd
CompactCells779To810.Box803.lambdaInterval	CompactCells779To810.lean:15959-15961	0bfd81def38a08e142f33e29d74c677d47644d7362f9e44df37af4fa9cb19524
CompactCells779To810.Box803.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:16608-16619	3db68670802354e526e17293c93b61046ee614908bc87e72c4a8b1ba9c76f313
CompactCells779To810.Box804.QInterval	CompactCells779To810.lean:16620-16622	b1211f5d2dd1f9be7339a98853247ec9a79d412a22a1a634e858e0bd8a84e098
CompactCells779To810.Box804.lambdaInterval	CompactCells779To810.lean:16623-16625	904db87f84959da74e7cb5e4a30277d063e364861acc370ad9f5823845f9420a
CompactCells779To810.Box804.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:17272-17283	0a1656fd74a6990cf94958748c985a601548396e603fa710b05ee3dbbb684633
CompactCells779To810.Box805.QInterval	CompactCells779To810.lean:17284-17286	376ef41c67bc8b22f864736d77ab627879e5f9eca3bc8dfe92f97269f9ed8d83
CompactCells779To810.Box805.lambdaInterval	CompactCells779To810.lean:17287-17289	53ac64e602309c7f8f1945e6cd4c51c84892743a90f1a9236b1d9a903aafac51
CompactCells779To810.Box805.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:17936-17947	1ee5987c8fba7d5f0e82d74e8879dc26a2f96b8e7808c8e16d5d03d2a6d28187
CompactCells779To810.Box806.QInterval	CompactCells779To810.lean:17948-17950	c8028b131ab6a6bef98bd400c0ee1624ad69705f6d726f9b40b3783752e46654
CompactCells779To810.Box806.lambdaInterval	CompactCells779To810.lean:17951-17953	47dd7fa76d95fe220134568959915aa4b1c0a0dca295872328b7fc7b7d1cd458
CompactCells779To810.Box806.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:18600-18611	08327b38f0ee173538feadfba9941eb635d99674888037e350306ac5f9bb4c65
CompactCells779To810.Box807.QInterval	CompactCells779To810.lean:18612-18614	30d8bb05e3eb5198f2fe0198b1b7cc0198d73cfd335edadb557009efd0feb5d
CompactCells779To810.Box807.lambdaInterval	CompactCells779To810.lean:18615-18617	73dcf0fb9ed1d9e40dc32dbafb4cd448fad92bf98c986d9f4ea71517b0d937
CompactCells779To810.Box807.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:19264-19275	e1b8714926920eeb32e378a299fa416204dfaf08347a0294a7689a50c30bd06f
CompactCells779To810.Box808.QInterval	CompactCells779To810.lean:19276-19278	41be0ffacc833bcd3b5f4c0b9c42c7aafa371200f6032a0a916e508a77b05f5a
CompactCells779To810.Box808.lambdaInterval	CompactCells779To810.lean:19279-19281	50f1907f9d597dfb1c7ba555c9891b2da1537645a34a701b77bb7699dcfbf7ae

Declaration	Exact source	SHA-256
CompactCells779To810.Box808.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:19928-19939	71ad92769487fd728eb3afa46a2f7fc231d46b690df8b217d2c2177f2ac39cf8
CompactCells779To810.Box809.QInterval	CompactCells779To810.lean:19940-19942	cad7c7249cfe8d710838d8dba57e9a99b5d8aa242a259c1139b33a46fbf97ac2
CompactCells779To810.Box809.lambdaInterval	CompactCells779To810.lean:19943-19945	98a83b79e48e2fc88e501c56223fb495348316c74b11a99246b8e8f9b6ea6285
CompactCells779To810.Box809.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:20592-20603	d871fa88e8954db6c9509a4da7d419d381a2ed46dc2e9906c4c84242b38f0298
CompactCells779To810.Box810.QInterval	CompactCells779To810.lean:20604-20606	3606648055d638330186f2522a1a23d858e262e1cdc4fb5169d9d79c64f2211a
CompactCells779To810.Box810.lambdaInterval	CompactCells779To810.lean:20607-20609	5075d57c76cbd6740a7fe722b72e8316fa932b46f88f5359fc3b42a88ca97069
CompactCells779To810.Box810.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:21256-21267	4462618a4a659a646640a92baec7ff9b626ba3014c183f0e1d5d4cd12b15bf74
CompactCells779To810.bboxes779To810_candidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:21268-21498	6d302b492c1b278b9cfde0f5e9251db498a14fd684b60801c60068e9a9851f24
CompactCells779To810.bboxes0To810_candidate_not_isWeightedPerimeterMinimizer	CompactCells779To810.lean:21499-21512	0efbcc8cdc5dcd419becefa317f2a16040543d75bd08e7962d419d8b3fef4e3e
CompactCells811To842.Box811.QInterval	CompactCells811To842.lean:20-22	3045f38851d63c713d4815f50c5b636b81b911e9c5676ea19c677ad4a3e09b3c
CompactCells811To842.Box811.lambdaInterval	CompactCells811To842.lean:23-25	86a1603a41bd49a0eb9f664be8089282215ad9b8f06ca0054c88e6531369fe2a
CompactCells811To842.Box811.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:672-683	1204fdb4b1cc140bd028982768a985c2a3d9aa30a94e86711f752b3f61008caa
CompactCells811To842.Box812.QInterval	CompactCells811To842.lean:684-686	ac58ad390318b034aac65bf5def56818fec a51e403f0d18a86fd28be86526f32
CompactCells811To842.Box812.lambdaInterval	CompactCells811To842.lean:687-689	1693a45b257e137e324b168c040e7a4eee792e2ced7998b8fcdd5177fb57737a
CompactCells811To842.Box812.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:1336-1347	b1f3d4823fe0dc21dfcecaae664bb42ae980f0c9be0fe1ddc7a250d81018ea4b
CompactCells811To842.Box813.QInterval	CompactCells811To842.lean:1348-1350	91b72e5affef220f7376b10f981bca395d4c a801f29ad0e50be27e84fba063f43
CompactCells811To842.Box813.lambdaInterval	CompactCells811To842.lean:1351-1353	f7469d229151f3fd77892564e718e78a85b9a5752208067b44dd895acf466710
CompactCells811To842.Box813.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:2000-2011	879851b516772f2d4bb38790b76a51148046c4383a2a66b5ae8b44c04d88a792
CompactCells811To842.Box814.QInterval	CompactCells811To842.lean:2012-2014	be9ad95a2960f938c56625ae2555857e734452fd465f62893dd40156ad6ebeec
CompactCells811To842.Box814.lambdaInterval	CompactCells811To842.lean:2015-2017	39a99e22162f1684b95466e7dd0f684859cf8cf9ee6d7ffcf6db9e55ab427c622
CompactCells811To842.Box814.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:2664-2675	5f81d059de8183ae1b23d0999fd0eb9e4d1ea0bc8c761816a0a7e5af5bb7a6a3
CompactCells811To842.Box815.QInterval	CompactCells811To842.lean:2676-2678	e20476a98237b8bb57f90e28b6418f4dcd966c6edd45732cc3c19001c71c5bc1

Declaration	Exact source	SHA-256
CompactCells811To842.Box815.lambdaInterval	CompactCells811To842.lean:2679-2681	1cf9dd5dfc3a1fd085d4cca759d32b4f05b11ef3062844292a106a845ca5f53c
CompactCells811To842.Box815.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:3328-3339	1ceaa6bf36adbb1317673f7c27e45d26a3643edd7ca3d0299abebfe881e69745
CompactCells811To842.Box816.QInterval	CompactCells811To842.lean:3340-3342	1ecc755e3744487ded8a22215a01f0cf8824fe3d3849fe8dad579b1fa0b258b1
CompactCells811To842.Box816.lambdaInterval	CompactCells811To842.lean:3343-3345	4ceacb81a3870ed170c854171731cc517d8a5832130bf4e45a503c2c9a69037c
CompactCells811To842.Box816.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:3992-4003	228be73079307d56f982e7090095120aec8174cf807378500997cd4e03857dfc
CompactCells811To842.Box817.QInterval	CompactCells811To842.lean:4004-4006	75ce7d42084266f7f252af1245e7fe72fea701361c377e76ea09fa89ffe341ba
CompactCells811To842.Box817.lambdaInterval	CompactCells811To842.lean:4007-4009	0f6934ff7631d8d5fcd8aff170836ace831cf442da668540233002dd903f33e
CompactCells811To842.Box817.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:4656-4667	02899abd2d7e4d66c1acc00fc98a4c87b11e897a64f87a79817521dfac86dca3
CompactCells811To842.Box818.QInterval	CompactCells811To842.lean:4668-4670	e7c7c782a545ba449b2fc666512395f9b510db98b628630e013961a1c3ebcb60
CompactCells811To842.Box818.lambdaInterval	CompactCells811To842.lean:4671-4673	cf000ba7569b6b3bb1c081776bcd2f87d15870c470a038139029db4d79d1a0c
CompactCells811To842.Box818.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:5320-5331	73bb5b7a4954a863185b95b93922a19e7f09c8cafa1ea264ec6964c899c4a99b
CompactCells811To842.Box819.QInterval	CompactCells811To842.lean:5332-5334	95b7fbd73fa88a1096f9ba282cce441cb4a1adfd8fe8160bf92ad54344ea64f
CompactCells811To842.Box819.lambdaInterval	CompactCells811To842.lean:5335-5337	56fb635dd19fcc435beb4c81d20c87964f2b7031ba49f549baeba51c28223d2
CompactCells811To842.Box819.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:5984-5995	bada667f223f4aec53bbfc97082e4524b42e8660c86473f0d74a9c8f8f771bbc
CompactCells811To842.Box820.QInterval	CompactCells811To842.lean:5996-5998	f766cf3b10b9ed1a2b98b1232b8e07f497dfb821291b4a575bd46eb84f7b24c
CompactCells811To842.Box820.lambdaInterval	CompactCells811To842.lean:5999-6001	1e98ae73dc5c2813fca9a2873e9ba07d51dbc8981ff5c3ffbf2a95694bd30e3
CompactCells811To842.Box820.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:6648-6659	4f80f488e7eb785815ec70b0145f3a146c952e90f661c481e4b278c21aae29d6
CompactCells811To842.Box821.QInterval	CompactCells811To842.lean:6660-6662	d3c4bb62977607f54b28ed1e1f53903fe0c664e506f22ddf342f68bd64d7ed87
CompactCells811To842.Box821.lambdaInterval	CompactCells811To842.lean:6663-6665	aa4ba61e2c6ccba73eee0951ef76e0fa1ddb5d49ca9814d1205c17dbef96cf0d
CompactCells811To842.Box821.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:7312-7323	bcf52ab2875635c33ba5423bbde03c23cbd a5d4d0f232ce890c4213b7ee6df67
CompactCells811To842.Box822.QInterval	CompactCells811To842.lean:7324-7326	876cb97672270d738d09bf0f3059e55c781165743aea96dd4bd5979f7a22e41e
CompactCells811To842.Box822.lambdaInterval	CompactCells811To842.lean:7327-7329	058e531c37687ef17afc96b43013a171d63d45bca6a59c341ddec2ae3b94be0

Declaration	Exact source	SHA-256
CompactCells811To842.Box822.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:7976-7987	cc4c289cd59b3280d97ec9d75a51992ae5152dd29ad403533dadd083eb62a4ce
CompactCells811To842.Box823.QInterval	CompactCells811To842.lean:7988-7990	009d255eee25777fc590a5528caaecf3c29b26a2512be9f3f0af54268618b1ed
CompactCells811To842.Box823.lambdaInterval	CompactCells811To842.lean:7991-7993	e48d07bd4c222c031ae737f208d6b748e86490859d073286f15ef4e9b02e7154
CompactCells811To842.Box823.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:8640-8651	ff6a14090f797bcf941de04240ceb6493938245945be49b82f690f80f96ce3d8
CompactCells811To842.Box824.QInterval	CompactCells811To842.lean:8652-8654	dad15f53209f0e79d784cc6c0c2005c4169d9e0179eaeaf0660b5258d562ccea
CompactCells811To842.Box824.lambdaInterval	CompactCells811To842.lean:8655-8657	e5c3036430d8dcd4eadb2d3fa96cccea321dce43565792a7fc40cbbcba964f00
CompactCells811To842.Box824.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:9304-9315	e7410193392e0ef22f7e5e993e7bf74f6260d60f585b1d0516a02ab7413bf53d
CompactCells811To842.Box825.QInterval	CompactCells811To842.lean:9316-9318	c2f550309f955a9dd8a724a0fa9d199d3eb182847a8c83f46033dc4d50b0522e
CompactCells811To842.Box825.lambdaInterval	CompactCells811To842.lean:9319-9321	5d62e37df7f16a93e459de262b9ecf8fce13fb41f795dc426d159ba3807ca689
CompactCells811To842.Box825.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:9968-9979	3ce7252d349b347a384e43ba61f4ad538ff2b0231d38b349f71e751a3ef1f29
CompactCells811To842.Box826.QInterval	CompactCells811To842.lean:9980-9982	b2e9ba55462b6a4e9e36f59a6a318afc91ebd4a1477f7874e6cb1cd4695ea168
CompactCells811To842.Box826.lambdaInterval	CompactCells811To842.lean:9983-9985	5328f98531ced615a766f082d9940483b8a47c5d28d6fbda69f5afebfe29d906
CompactCells811To842.Box826.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:10632-10643	bfcdb3b0c975ac1f9cb54416a47a465ceea0218660c3eae7aeea96fc59c004d4
CompactCells811To842.Box827.QInterval	CompactCells811To842.lean:10644-10646	9ae8f02b77b8ca44c93f591fe5d10bea07cfdc3c84c57500024090b916ccbf6
CompactCells811To842.Box827.lambdaInterval	CompactCells811To842.lean:10647-10649	99ed607504b6b1948c139c002c530b21295995ce6d7bdeb792df7eef0a8d2ef1
CompactCells811To842.Box827.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:11296-11307	2b52b0b33176aa57596c4f30ddd0c316a848673b1c7fb524e282ee35da156e2
CompactCells811To842.Box828.QInterval	CompactCells811To842.lean:11308-11310	492054d70d2da0b153d0f661cd94fa3d583a3c13b9649ec88a841171bda2d039
CompactCells811To842.Box828.lambdaInterval	CompactCells811To842.lean:11311-11313	13101fa6fece40393834d5369deb90f0db43d0dc4ca5e12ea1926ed847143f51
CompactCells811To842.Box828.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:11960-11971	b7785630a541599df44e7aaf1ce0629ce4a3283d12f6019bd11005b2efffc8653
CompactCells811To842.Box829.QInterval	CompactCells811To842.lean:11972-11974	dd0cb984911fc12c7c28c1e79a1c66094b5358f7b4b70e1900cc0c8ef1a20833
CompactCells811To842.Box829.lambdaInterval	CompactCells811To842.lean:11975-11977	18bac0d0cb29e8cde8ef0ba48d7b75f29a26e300f9d77e51ef98f423fd9e7942
CompactCells811To842.Box829.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:12624-12635	ed81ebf5279c26fa8ae02681ef5994d1b5f7d05b45e4ab35b3ae82f9970d1279

Declaration	Exact source	SHA-256
CompactCells811To842.Box830.QInterval	CompactCells811To842.lean:12636-12638	39c3002a6adc387e28357a30e5283b70778800f2f7dfa715d146fff630a90cab
CompactCells811To842.Box830.lambdaiInterval	CompactCells811To842.lean:12639-12641	4ed7c8bf4559989b6c9e27584960318fc30650fdcf21e134dcac72898242809d
CompactCells811To842.Box830.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:13288-13299	9ef076b84f2e3386e7033a41f43fcbdfd7ee195807cf1e750a5748325ca37ebe
CompactCells811To842.Box831.QInterval	CompactCells811To842.lean:13300-13302	8cc5f56765a81eb13d42b65fa4d9ed1d1e30b2f5451c92b4f530a39e533f9359
CompactCells811To842.Box831.lambdaiInterval	CompactCells811To842.lean:13303-13305	d4bbb6c4608d87427485047d5785ecf4654bc81097cb0193442b1ccb7d6f1637
CompactCells811To842.Box831.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:13952-13963	14f1c7edc8f39f447695552762245372d493797aa928808b45657cb5a9cba6fe
CompactCells811To842.Box832.QInterval	CompactCells811To842.lean:13964-13966	2bd5b521f7d6cae00a09bbeebd9b1652c0998e34afa4d60f3d633fd32114026d
CompactCells811To842.Box832.lambdaiInterval	CompactCells811To842.lean:13967-13969	611e59ce941ed51c8111b0ba6a997b3254e3f603c5285b594f807103762cd8f8
CompactCells811To842.Box832.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:14616-14627	5973a1fa61b62d6fe2efde32c65848ad7d3cd4010c73a1a3338f4c10c2f11f0f
CompactCells811To842.Box833.QInterval	CompactCells811To842.lean:14628-14630	72deb7628de2aaff2f09c7212c575f2fdf40cbc32829516105223640cb713057
CompactCells811To842.Box833.lambdaiInterval	CompactCells811To842.lean:14631-14633	356d8d00ea928b637056069290f2082d3ee18d9807dc709df39ac796a4ab263b
CompactCells811To842.Box833.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:15280-15291	133b78eee2121f690db2cd99e591dcbccdd76f10c03cd09fdb477fd74d6f9b34
CompactCells811To842.Box834.QInterval	CompactCells811To842.lean:15292-15294	c59d909e0a9794958757d3c5b606eceda6cacadf5ab1ce7ea845b3c32118882b
CompactCells811To842.Box834.lambdaiInterval	CompactCells811To842.lean:15295-15297	f2e8bf28675b6129313e474598bcad7746bc4291425ab8ab0917d81cea362d43
CompactCells811To842.Box834.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:15944-15955	06f97f463551d150187bf3a83bed3068d85dbf517a8a341ef1b8217447532ac9
CompactCells811To842.Box835.QInterval	CompactCells811To842.lean:15956-15958	a7ff3589da78f41c3814866cf53d8bc38f51457dba6abba0d07db8211c124260
CompactCells811To842.Box835.lambdaiInterval	CompactCells811To842.lean:15959-15961	a1d4531407ed025200eed3b7833f172bbbd86ec0a59a7fe23653e0baa8f7bb1b
CompactCells811To842.Box835.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:16608-16619	94eae1e8f4b05575ead3112f1f99a32e12d27c0c4f83867740efbfcc61e39670
CompactCells811To842.Box836.QInterval	CompactCells811To842.lean:16620-16622	bd6daa7ca6076971a47e34491a83b21dcc7eed6d186cc2033150340a209856fb
CompactCells811To842.Box836.lambdaiInterval	CompactCells811To842.lean:16623-16625	0de7e30a4dcbe219dbc471fb72f37bdae30088730a737221744bff4cfd31cb5e
CompactCells811To842.Box836.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:17272-17283	d616ba313701903dc32726a81e3a9c961bd9f2c0ed6142d3cf95125805546ddd
CompactCells811To842.Box837.QInterval	CompactCells811To842.lean:17284-17286	2a6efe476dbccacfe4620c3b6cbdafdb29fae9f5898792bb640a1daf858a5198
CompactCells811To842.Box837.lambdaiInterval	CompactCells811To842.lean:17287-17289	fcf03f266b73eb62da739cdbdca9e014624379594b846f4d72a4f0b2e73f8cb5

Declaration	Exact source	SHA-256
CompactCells811To842.Box837.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:17936-17947	a89dd22a69e3075dc638de4ad751341caa52aca84ed92bde841648dd6d9bc58
CompactCells811To842.Box838.QInterval	CompactCells811To842.lean:17948-17950	aae3fb3dabe9fe7bf40adc65377d57156070f297ef41971a97777e630f0aa041
CompactCells811To842.Box838.lambdaInterval	CompactCells811To842.lean:17951-17953	189ae894f60ddce88f189553fba387fa125e8481f6fea9c5e1e65078ecc93359
CompactCells811To842.Box838.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:18600-18611	7929f6359afe1fe704e86bdd1385def4891a3966161ae5f03371fd821ba84bc
CompactCells811To842.Box839.QInterval	CompactCells811To842.lean:18612-18614	21006b48c2caa5edbccd1fe0cfcfcfc0c1b2f529d9db0826eacfa969eb72fcb7
CompactCells811To842.Box839.lambdaInterval	CompactCells811To842.lean:18615-18617	5977aa0570ba37e5138383ba01cfee2ce51d9e6b7a2a5e2bc72f98c41343098b
CompactCells811To842.Box839.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:19264-19275	81a4efaaec4418ac782b063c0db488aa6ced32b7628bdba95ce086998861f6e0
CompactCells811To842.Box840.QInterval	CompactCells811To842.lean:19276-19278	32498dab0e5c64a2c1b2b807e7dae2359f822eff10eb315a2fd303b66c33e5d4
CompactCells811To842.Box840.lambdaInterval	CompactCells811To842.lean:19279-19281	57fecc89b502aad146708ec65cdfffd51594c91d59e59be89fe5b228a9efa67d
CompactCells811To842.Box840.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:19928-19939	83f096d882d904f48843ae481edd6995d99adc49e0aad4ad694f7f8530f6119a
CompactCells811To842.Box841.QInterval	CompactCells811To842.lean:19940-19942	736292e3941b939d6b6bd47b95d74c1f69bad8350bcc00e193e6577a1e0d3162
CompactCells811To842.Box841.lambdaInterval	CompactCells811To842.lean:19943-19945	1c486245a96e0c98bcbe07a366ef7da4f23e463909b941198574b795f9ae8b02
CompactCells811To842.Box841.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:20592-20603	246fc943e2b9d29a7644e089b649865225dc36039ed6574e04de3be8aedd38af
CompactCells811To842.Box842.QInterval	CompactCells811To842.lean:20604-20606	b3f008f3abaa2bca5f94d9ca797dde89ca0510e98f0d50653d768ec0874c0133
CompactCells811To842.Box842.lambdaInterval	CompactCells811To842.lean:20607-20609	354cb9f8f5f1ba8327133507f991070092c07553755f047d916a0a85775d9be3
CompactCells811To842.Box842.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:21256-21267	4e0a9eb8a79c706773531f73e775910995df0f43936f7863620a8b32e7a327d7
CompactCells811To842.bboxes811To842_candidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:21268-21498	08441dcc5f6d56e615ab70aeb751ac7b1c0f12eca84e0828d45c03eb9e079
CompactCells811To842.bboxes0To842_candidate_not_isWeightedPerimeterMinimizer	CompactCells811To842.lean:21499-21512	afa957a7194efca701a71a44e3242dff3f6dd069fa9f2518e01fc40db9ad7226
CompactCells843To874.Box843.QInterval	CompactCells843To874.lean:20-22	73027f83dfdbeba89e54647c089b5d667d1c23e5b9edb87cf9985c547b1fb1ff
CompactCells843To874.Box843.lambdaInterval	CompactCells843To874.lean:23-25	35301ddde0ccddcbac03086c3656b0e345f78c711fac1d0b1ec0c5c07d819574
CompactCells843To874.Box843.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:672-683	eb0a9f1fa2e7125603cd4dbc5996f65f794c81b2cdfd2de18ffbac84438c9a41
CompactCells843To874.Box844.QInterval	CompactCells843To874.lean:684-686	5d021b54f147b724be79fa3e12fc265839f61c39bb88520acf15bcc1a44bab39

Declaration	Exact source	SHA-256
CompactCells843To874.Box844.lambdaiInterval	CompactCells843To874.lean:687-689	9a0f7a542ac526a6f7b56bcbe41f8610d8062d081e3cf480e2ce56820a26db4f
CompactCells843To874.Box844.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:1336-1347	87da79dc340514139a936865393cb79b73a8a65bb9dde5dcec42a996338fca92
CompactCells843To874.Box845.QInterval	CompactCells843To874.lean:1348-1350	5362f23b96823bcc7e978106159564b360b1c0fafbc6b0e0eb1d7c6fcbb88b6b
CompactCells843To874.Box845.lambdaiInterval	CompactCells843To874.lean:1351-1353	3d6bde973fac6cf9ee97b827cc0f4afbb21862f1bd93f7323cc312b097c9dca7
CompactCells843To874.Box845.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:2000-2011	28751f4f6c5c5920f436235348431b11dc3facfef9dd2f2b1fc8002e9215b3fb
CompactCells843To874.Box846.QInterval	CompactCells843To874.lean:2012-2014	162571ecfea62d96c4d53b3f85f818635b21493bd1f42d4d20ebb0ffd34a80da
CompactCells843To874.Box846.lambdaiInterval	CompactCells843To874.lean:2015-2017	c08126240a209b1be3448b5f9db1de01e03db8521cf7063250bca3c2c19903fb
CompactCells843To874.Box846.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:2664-2675	aeed4e8b856a0ef3640ef8db520a138a537c84a9a8d3e52f5ce53221f3fba4ec
CompactCells843To874.Box847.QInterval	CompactCells843To874.lean:2676-2678	8f9367847d0f03bb851567d13b72253dd04725ef7256c7839cb47b12c246d5b2
CompactCells843To874.Box847.lambdaiInterval	CompactCells843To874.lean:2679-2681	e5f3e483b83dadcf8825ffd71ff130509ff1731af71de6228b6d3b713351e6ec
CompactCells843To874.Box847.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:3328-3339	3033434fe1d969d329e0af343954caf099904244ad590308be9b09062a999252
CompactCells843To874.Box848.QInterval	CompactCells843To874.lean:3340-3342	2c961de4e25aaea5117f45fb6a517198f80d9b985edc96c280af716ee3005632
CompactCells843To874.Box848.lambdaiInterval	CompactCells843To874.lean:3343-3345	8da1edc35d517d81abaf943c14aabbe6cc96630b152038058d1fd55fa967fced
CompactCells843To874.Box848.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:3992-4003	168e7b8aeb7d98fff804e57e886f54cbf00d7cee940de31cffec415c2ff5f6e5
CompactCells843To874.Box849.QInterval	CompactCells843To874.lean:4004-4006	8e2fe3cd98552d6ca4fbd213ef741b4723579d74659bf604daaf9e5314b44168
CompactCells843To874.Box849.lambdaiInterval	CompactCells843To874.lean:4007-4009	c56a5caf760b159b7286689f790ddf32b1256da0d6bbc7ee5dadaa35d78e81d5
CompactCells843To874.Box849.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:4656-4667	048c1a3c51d39fe99fbabc0e069620a1acfbe907f4abd19490d19c00093fc3e
CompactCells843To874.Box850.QInterval	CompactCells843To874.lean:4668-4670	374cad997d7b0732a9e5620c4df7d63566f8626b72621982fd9c8b62bdf4598f
CompactCells843To874.Box850.lambdaiInterval	CompactCells843To874.lean:4671-4673	4652389aa81b5066cbd3ea0942f4d82759a855d67307b44fbad48d938d43b563
CompactCells843To874.Box850.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:5320-5331	98d0da05bb85dab7882409b4ae3d9011a986739d581062b53c36662102963423
CompactCells843To874.Box851.QInterval	CompactCells843To874.lean:5332-5334	8df47ab4a3a975b5782e02d3e803283fe43644027709cd7702f6a45f79ecd472
CompactCells843To874.Box851.lambdaiInterval	CompactCells843To874.lean:5335-5337	d0a3812cf83fd09c2904aa9af7ba1e320ab27887fd34005f81e43e0cbbbe8d87

Declaration	Exact source	SHA-256
CompactCells843To874.Box851.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:5984-5995	7fe42261a33b018160d3b2208f4bd8be773dfedcb73e62a59d75172eb98a6664
CompactCells843To874.Box852.QInterval	CompactCells843To874.lean:5996-5998	e81122202d5390490eee1cb51362e635507ec080391d9eb9ac67088a5d744221
CompactCells843To874.Box852.lambdaInterval	CompactCells843To874.lean:5999-6001	77f9a01f0a028727767d5a321dd4576e686f3c8da2ae1b01b0321b74badb206e
CompactCells843To874.Box852.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:6648-6659	9c4f3d24ee2e4a5eda09130b35790aef1930b52b6f915ec798f103652b453946
CompactCells843To874.Box853.QInterval	CompactCells843To874.lean:6660-6662	ef83bcc84baa40a471ef7717acc7cd0892a89ef1312f6b4a6a317a3ceb68a35
CompactCells843To874.Box853.lambdaInterval	CompactCells843To874.lean:6663-6665	9fbc16b8f8be8023457b231b9341910acb06beb71e3b22ecbf416fb97adb98e4a
CompactCells843To874.Box853.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:7312-7323	38a6f709743f1781b3edf72c2871d4d1cd2eaf7732507c23115408d10bc6a514
CompactCells843To874.Box854.QInterval	CompactCells843To874.lean:7324-7326	f11c3298cb11ad83190186f9277bb5d94ea2e5ebc8db6440d9f07171a007571c
CompactCells843To874.Box854.lambdaInterval	CompactCells843To874.lean:7327-7329	89cf548284f6c2dfb1fb8d8aab5d85de575cd53ad40bb4c2949817c4b68c0d6d
CompactCells843To874.Box854.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:7976-7987	f7af64907806429aae17b5e796e33bd9a44d1f45375a41b008e12500830693
CompactCells843To874.Box855.QInterval	CompactCells843To874.lean:7988-7990	8ec971460f9d3c7f071b81f4fd6ecf607dd6a642933b863463b84963d175af79
CompactCells843To874.Box855.lambdaInterval	CompactCells843To874.lean:7991-7993	3045e81a81385f4f2845480a2bdb391dd063eeb3eda476b6b8f0a03ac6a57a77
CompactCells843To874.Box855.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:8640-8651	27d7dfbacb70439a351e05fbf49f49870a93c4de71c0cdbfde2bcced4e83a32a
CompactCells843To874.Box856.QInterval	CompactCells843To874.lean:8652-8654	a1536090a23ab0baff7c47bc55b270957d9c7f9362d2ec4fdfa64c26a61e1b4e
CompactCells843To874.Box856.lambdaInterval	CompactCells843To874.lean:8655-8657	183edd1867da706986e8432cd7c94779e0737c4d770e90b5312a8fcf2f7c36df
CompactCells843To874.Box856.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:9304-9315	573ee2731053a2211d404713279ab48070739dece15f8543f6f69ff7fe94959b
CompactCells843To874.Box857.QInterval	CompactCells843To874.lean:9316-9318	72d59c7a40723d269bfb48632dadb612a4bba665aa980504358726f7120c570a
CompactCells843To874.Box857.lambdaInterval	CompactCells843To874.lean:9319-9321	042162d72819b2c74d47f71d714d3af171aeba8378c04db75b7d68f8dd60eb86
CompactCells843To874.Box857.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:9968-9979	566bdaae5f028be4613561d29c3fc99f57af2bccd6a63579c39697b5e256e1af
CompactCells843To874.Box858.QInterval	CompactCells843To874.lean:9980-9982	8c66cc3841cd4e23e74fa74bf400cb66be2b497f00b389a5e62ee84dabff3679
CompactCells843To874.Box858.lambdaInterval	CompactCells843To874.lean:9983-9985	d8b7c1213cdf30ec4feb9053098b51c250377bfb6a6dd8e900eb83a4273251ab
CompactCells843To874.Box858.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:10632-10643	d8d9daca079de400378d49ba247d694ab7e792e164359379b6808b9880145012

Declaration	Exact source	SHA-256
CompactCells843To874.Box859.QInterval	CompactCells843To874.lean:10644-10646	613bf234d2a0ddbc4b47bc20f7032639d8383c3722222f1f1e8ca16e25f7fe7
CompactCells843To874.Box859.lambdaInterval	CompactCells843To874.lean:10647-10649	0ccce00c8feee5a1d6eb188a8d01b8283fa580e0c5efbacdfcd753bf830a5056
CompactCells843To874.Box859.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:11296-11307	b88293c7f0e10dd18432778ff6ecb0bc866a37e280812970c24ee56b9e678a75
CompactCells843To874.Box860.QInterval	CompactCells843To874.lean:11308-11310	60265da9e2fffb01c7d4f1679b0a8fc5948ed83ff4c7154c78a0bc05bb8d6f0aa
CompactCells843To874.Box860.lambdaInterval	CompactCells843To874.lean:11311-11313	e62675a63c5177fd91d336f37d138b03acede20397a2baba7f347e5728ba7893
CompactCells843To874.Box860.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:11960-11971	919498d03b684dd355012db32a488547855a517647ffdf60b52bd7ea38a4794c
CompactCells843To874.Box861.QInterval	CompactCells843To874.lean:11972-11974	eac1c80e433f7be3794b009a85dbb39c98ebd7aa230f94cab271ce65ccf9b4a8
CompactCells843To874.Box861.lambdaInterval	CompactCells843To874.lean:11975-11977	3a193425c6a163dbcb7af2c5af47aff80b0ed68edeacc2f4033fab3732e70810
CompactCells843To874.Box861.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:12624-12635	4ad470a33b745362bcc293e070b9590659eb7d0aaaa7a7a30219aceb173fe5ff
CompactCells843To874.Box862.QInterval	CompactCells843To874.lean:12636-12638	ddd64e2a193f5067fabb1e3ef5667ce0204032c81f3628c03e50ecaf7b9d04c4
CompactCells843To874.Box862.lambdaInterval	CompactCells843To874.lean:12639-12641	8ac3a59f2f9ff9cae4a002cdc481ee5c68475bd9593c37f8bed4f55e570095e2
CompactCells843To874.Box862.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:13288-13299	dbf893434247acd0c19cfaff29eff203e0b5bc17842325de70cb344791d4fbfe
CompactCells843To874.Box863.QInterval	CompactCells843To874.lean:13300-13302	414d63878ffdc9fc90c17eb74659f7712577855ddeff2630d8f48b560540a10d
CompactCells843To874.Box863.lambdaInterval	CompactCells843To874.lean:13303-13305	b5008a27e9322c55a1ad66014d4ac770f5e04079be34779cf82cc3630f87d3a6
CompactCells843To874.Box863.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:13952-13963	c550170a34e325f71192a3cdbfdd0f50f48adc48395f3e996f6e9a41ee8c8db4
CompactCells843To874.Box864.QInterval	CompactCells843To874.lean:13964-13966	460604015958e415f0256242270bbeca46b90e9901d295c89ce0ca60643c53ca
CompactCells843To874.Box864.lambdaInterval	CompactCells843To874.lean:13967-13969	8f60f3b1e4f10926c46ad6e6e5b1947c2573ee750a09318ff829d3d7a466dcd9
CompactCells843To874.Box864.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:14616-14627	e6e3e398865daf5b5f5f9991ba23c8ea63ce1a50eee1702b6e1a6654a26dfbc3
CompactCells843To874.Box865.QInterval	CompactCells843To874.lean:14628-14630	b3a8d8355645ebf1acc070afad3436bac15084bfa5eef391c4db3ab525559e86
CompactCells843To874.Box865.lambdaInterval	CompactCells843To874.lean:14631-14633	2e96b1af74fc69a4e100ec0da4cd20c414aa13a0c9472115f21cf261b4fa49c8
CompactCells843To874.Box865.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:15280-15291	984164a38042c515b260f1707c7b80053e158554e516b060aff41602cab0c11
CompactCells843To874.Box866.QInterval	CompactCells843To874.lean:15292-15294	9104a4fe453bceca980db2fbfce3fdff2213ca02126ab61286cc881b37c22ed9
CompactCells843To874.Box866.lambdaInterval	CompactCells843To874.lean:15295-15297	d55c10680d6eb36857440531519f2371bfb3f344cc4d1d5bb55c4162d7be5d53

Declaration	Exact source	SHA-256
CompactCells843To874.Box866.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:15944-15955	d597403ab4530fa1099b85b7044df86413c139626deb9af2096eab7c2792c55d
CompactCells843To874.Box867.QInterval	CompactCells843To874.lean:15956-15958	03d16d557bed08c2294215615f61f6e4183e4407ee47fa460438187f555e078f
CompactCells843To874.Box867.lambdaInterval	CompactCells843To874.lean:15959-15961	7db1c9a474d893e49c14c0b69c71dd61a237ea756db9acb50c77294a503115e9
CompactCells843To874.Box867.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:16608-16619	48e8e85d85e3a220b467fb6f4a150045406fac286cdbeea8ba458edd89fc90b6
CompactCells843To874.Box868.QInterval	CompactCells843To874.lean:16620-16622	da1526d03f6cb5ce84c8e78e222a01af720d3353abe3f3f83bb44d43e467706e
CompactCells843To874.Box868.lambdaInterval	CompactCells843To874.lean:16623-16625	cfbcd12dfff3dc84f5ba29a82f590127309836e4b27a11eace235c27035642a20
CompactCells843To874.Box868.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:17272-17283	2430987e7d793fe196463d8854249a79c704eb56248e7e0296eb62a9f46a0db0
CompactCells843To874.Box869.QInterval	CompactCells843To874.lean:17284-17286	f67af56a7b07097dcc5272e0c8a00769ddd675413d855302dc78dc0e54789582
CompactCells843To874.Box869.lambdaInterval	CompactCells843To874.lean:17287-17289	9210a6d207225116a30267d388886dbc9f92d82ca6e5b094744c71cc545edec0
CompactCells843To874.Box869.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:17936-17947	a8cd50c892da095b330c50030cb7a87afd43444f2e2543bd3a91a30e706b322c
CompactCells843To874.Box870.QInterval	CompactCells843To874.lean:17948-17950	eb18a1b8a7dbe96b858a902e57ecd7747b4999160fd582bc86f2148d4940421a
CompactCells843To874.Box870.lambdaInterval	CompactCells843To874.lean:17951-17953	e6c24f28d59b4b0e5af8c3f5a75a8b60a5773cd6de2d9b9430fccf260c77599b
CompactCells843To874.Box870.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:18600-18611	fd5fbe4784311c8259a8045a24673993f37e81cf5865d28b01dceb1023aee44f
CompactCells843To874.Box871.QInterval	CompactCells843To874.lean:18612-18614	f6a73ca1dbf159a4839434fb241450c2279e3ef11bbcf9faf7950d8660831f17
CompactCells843To874.Box871.lambdaInterval	CompactCells843To874.lean:18615-18617	2fbdeca582129bcfdb848c6b1ffbb202a5a641a932dfcc0a3e8ebf52ae235f5f
CompactCells843To874.Box871.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:19264-19275	8e806ce9ce6affabe24569fcd1cb1313b12108ea41a878ce26e8c24cf83b94b
CompactCells843To874.Box872.QInterval	CompactCells843To874.lean:19276-19278	75667d110c5d939bffa603dc3b3f7e11e5578b4e4229fc639e78413589ccf847
CompactCells843To874.Box872.lambdaInterval	CompactCells843To874.lean:19279-19281	e444069e1afbdbdd2ff1d5935a10900bb80a5caf822339aface7b74d89c27dfe
CompactCells843To874.Box872.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:19928-19939	0829a54a5305f4bd81f81da8f0327e26bd21bcd6e7177cb6efb973efbf9a6029
CompactCells843To874.Box873.QInterval	CompactCells843To874.lean:19940-19942	27097b7e2030bb3ceb590af4c8392a0d995155816303ab07cbc1a5174c0ac136
CompactCells843To874.Box873.lambdaInterval	CompactCells843To874.lean:19943-19945	329e4529c65f2a1e0fff704233295e59823083a6d81753a94028e509cffa9b2d2
CompactCells843To874.Box873.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:20592-20603	cc3c9b416cf1aa2571820be50239bc018413f22e1ddcab6dbdabaa101ff34e2

Declaration	Exact source	SHA-256
CompactCells843To874.Box874.QInterval	CompactCells843To874.lean:20604-20606	a07764ba8bf465713420556c323aa1191b37a8001de195890034df32e2cbacda
CompactCells843To874.Box874.lambdaInterval	CompactCells843To874.lean:20607-20609	a87c53c2cbdbcdb1d0b6d89390a4161d8db7dd0d578485bfbe5f303471762be3a
CompactCells843To874.Box874.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:21256-21267	a289d590a8d90628e83629a1d79f9043764897936f1f0b7294c1d1f7846304f7
CompactCells843To874.bboxes843To874_candidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:21268-21498	643b0dd434b1a9bf8b054c51df5bf687f08c1c45fd5ed0bb6827fa9add52fcd1
CompactCells843To874.bboxes0To874_candidate_not_isWeightedPerimeterMinimizer	CompactCells843To874.lean:21499-21512	4ba78cbde422e099596af8f9c96fc3b3f64fee508a002cd547513696dd1abdef
CompactCells875To906.Box875.QInterval	CompactCells875To906.lean:20-22	61c6ff96c582d732d3c307e6bb0804f4cef92dcde6e9dc731bf3b66ae0e58132
CompactCells875To906.Box875.lambdaInterval	CompactCells875To906.lean:23-25	0cbbcc7cc12681b824679eeddf65500cb82eaae48cad608f3749530e8dca4ab5
CompactCells875To906.Box875.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:672-683	c61266d58af7ddeb408148948ce67df7d9ff633497009837f657946a4404eeef8
CompactCells875To906.Box876.QInterval	CompactCells875To906.lean:684-686	992be4bb521792a19766fe73859fb98876a2ed7664960608720924980f222221
CompactCells875To906.Box876.lambdaInterval	CompactCells875To906.lean:687-689	3cf811c71b714a5e309d35a19fab9e5554ca784ae199b390667f517ea5beb99e
CompactCells875To906.Box876.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:1336-1347	f19e18baa9047357569c0b2c18e747f5b03a49d979328926db12042b5add5f3a
CompactCells875To906.Box877.QInterval	CompactCells875To906.lean:1348-1350	9ee9417d8905a00bf42decdf40964f0d57a1fd16939e920416543e96bf050090a
CompactCells875To906.Box877.lambdaInterval	CompactCells875To906.lean:1351-1353	ca27777ab21a1109d1092e5b916378d9597775383b5a4ff51aee2e46190d2943
CompactCells875To906.Box877.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:2000-2011	aabccdaf77ec98564e5aa9ccd28f754990c43f315362409d8109419a1613a2a2
CompactCells875To906.Box878.QInterval	CompactCells875To906.lean:2012-2014	5c52636d74c7960c7bd28dd7f655016ce66280bebc3b079f89e4b49691e9cde3
CompactCells875To906.Box878.lambdaInterval	CompactCells875To906.lean:2015-2017	a1974f719f4aea7ec2ef7aaea8d30e1469b5074983187c0909ae6a57554525bf
CompactCells875To906.Box878.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:2664-2675	f1cf25c7f1b8740edca5ac6d33946df53da881d9350ba79980b5d4905417aa62
CompactCells875To906.Box879.QInterval	CompactCells875To906.lean:2676-2678	146c6d78201775c262a92939686f776d15e57f92c179f73412e6267f1d303421
CompactCells875To906.Box879.lambdaInterval	CompactCells875To906.lean:2679-2681	bca1e8654ad870f63d2ff179201f9518c5434571d0bfe577afef986d4cd08a5e
CompactCells875To906.Box879.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:3328-3339	a58292b9492ff66463ebba9c2b7e9962ac07cd7442157b98ecb99c254f3cbe65
CompactCells875To906.Box880.QInterval	CompactCells875To906.lean:3340-3342	54cf6c40faa21039d42914366221b9b3afcadb87c8f265a4fd0afbfb2c2744b8e
CompactCells875To906.Box880.lambdaInterval	CompactCells875To906.lean:3343-3345	0990a5279d278fb80c3758913cffce95aca05c07cfd4901a86c7f5c4cb0756c4

Declaration	Exact source	SHA-256
CompactCells875To906.Box880.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:3992-4003	29a0c02fbaffb7e30be8f12c97f36f2faf48535d4ed0b3df7cae36c160fa75b6
CompactCells875To906.Box881.QInterval	CompactCells875To906.lean:4004-4006	4092d89ed4f6ded9c4a36e14dd4af233129040ef9856e98f9d90983d6436d52e
CompactCells875To906.Box881.lambdaInterval	CompactCells875To906.lean:4007-4009	36614dffeff3d157db4ac74669337ad9001e7108038194797926d21e75db79eb
CompactCells875To906.Box881.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:4656-4667	543f2e9b36c92dea612700e8911099736e2e74247478487230d20910188203c8
CompactCells875To906.Box882.QInterval	CompactCells875To906.lean:4668-4670	32bfcbb803c9011fdafce2210d38b6edf539b0f9025b2e6a14b270c49f3df6809
CompactCells875To906.Box882.lambdaInterval	CompactCells875To906.lean:4671-4673	5da3e05732f46ec608d2847e2bef57e00db98bc3a4ff8e814440dd5e35844bd6
CompactCells875To906.Box882.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:5320-5331	42479505985eaa91a09bf12e176763289402802fa987417a2de25314a391f63b
CompactCells875To906.Box883.QInterval	CompactCells875To906.lean:5332-5334	ae6fce74a8f3894c9a8c98d5cc6f8e5330be5eb5384a46af43a0b52f52851816
CompactCells875To906.Box883.lambdaInterval	CompactCells875To906.lean:5335-5337	0626dfb23454c4f473a5884ca6e1c9e44524929ab80e4a6ea63f9cf65f252e1b
CompactCells875To906.Box883.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:5984-5995	cc8668aac5afa9430687063063d42c32938fb5a67524649a30ef40ee4f5647c3
CompactCells875To906.Box884.QInterval	CompactCells875To906.lean:5996-5998	5d1483f34bc38642ade0faea324a90bb8c9c5f9f96ad066234a36b52ca7adf3b
CompactCells875To906.Box884.lambdaInterval	CompactCells875To906.lean:5999-6001	5b7014e1995b175ad9f95a87b915a6d759e173b425ba2b4a050bf207627816ab
CompactCells875To906.Box884.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:6648-6659	b9a015ca20c4206761f960c6e50133d44de5d6d2ab66624b3e972fe7d0c8c72c
CompactCells875To906.Box885.QInterval	CompactCells875To906.lean:6660-6662	221711f1878608dab2fd56546d047c75a562a7bcc08271c4599befd51a006204
CompactCells875To906.Box885.lambdaInterval	CompactCells875To906.lean:6663-6665	a3fe4eb6cfa5ac7494e6ef49df3187d95ef3b8a5cafba35456501e316ed822c2
CompactCells875To906.Box885.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:7312-7323	11c8d05c0d035f7330c580fa7582b87b0345fb2f8c0df49f1e2473757e5bb752
CompactCells875To906.Box886.QInterval	CompactCells875To906.lean:7324-7326	00e19eb192b202ef60b0aa3bf9f554193ad960a2bd8b844e9cd85407a6ba3ea2
CompactCells875To906.Box886.lambdaInterval	CompactCells875To906.lean:7327-7329	566702cf2214da88868f8761e5f0159e8680ed100aa392c52d8fc54cbfa02bfff
CompactCells875To906.Box886.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:7976-7987	f37f150451c5a663ad4c7470c38f2aea1a04c9852cb2dff0ec97ea0eb2186a5a
CompactCells875To906.Box887.QInterval	CompactCells875To906.lean:7988-7990	fcfa9708c0d582fa75ec01994317ba8b9c341e5f5023ed77d8c5d3458c62ab3e
CompactCells875To906.Box887.lambdaInterval	CompactCells875To906.lean:7991-7993	b5473c109a54ff25cb39e3be0d5eee5bb451ab00541d8a4c2b8737bc640330f1
CompactCells875To906.Box887.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:8640-8651	eb56d5fc69859c2c2dcc3df434f1ad8de8a7e8c1b73b236be9ed9ac329e383fc

Declaration	Exact source	SHA-256
CompactCells875To906.Box888.QInterval	CompactCells875To906.lean:8652-8654	8dd8d8eea7c3156e194bac4eaae7cf8abce7df3740ddf614f015407571bc57c4
CompactCells875To906.Box888.lambdaInterval	CompactCells875To906.lean:8655-8657	e02ee195a437903123bbf74c75c74d6b9ade79681e2dac87aff78b7f6b97fca3
CompactCells875To906.Box888.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:9304-9315	86bffb6ded964592310e47f43effe09bc50d1fce22d65c3f861fbb1cf583375c
CompactCells875To906.Box889.QInterval	CompactCells875To906.lean:9316-9318	cd227fd60334a6d6837e470f790e7d0d381fb324ce39cf241158b709e1aa8eef
CompactCells875To906.Box889.lambdaInterval	CompactCells875To906.lean:9319-9321	c2cca6d9c95127c8c09be96eb2fc636a16afcb84846480d37ba9e258dff71221
CompactCells875To906.Box889.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:9968-9979	b1df2cff9bc21ec0443aaff5974d84deb2e6eed390d0623d5c690eae9f772f0c
CompactCells875To906.Box890.QInterval	CompactCells875To906.lean:9980-9982	cc1679b2b9d9c9e4cf3b0eb5f0f0be42974689b1573587c97ba12d3d1912e381
CompactCells875To906.Box890.lambdaInterval	CompactCells875To906.lean:9983-9985	f1f6e900c3c67f334b08f764dcb82d95b1e1ac2413de7897f15c2e86c14ef6ca
CompactCells875To906.Box890.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:10632-10643	d06239b55fc7ce0438f6ac04e67352edda35417311775f505fb2dae170c5191d
CompactCells875To906.Box891.QInterval	CompactCells875To906.lean:10644-10646	e46a34ccc17f8c4a2a41ee3e7279322247df849d2da5730aece78f6704837254
CompactCells875To906.Box891.lambdaInterval	CompactCells875To906.lean:10647-10649	bfb44f25c4e9ec9824c4349397f9c3ae8eb44266cbf556c320760aa9d8b36310
CompactCells875To906.Box891.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:11296-11307	244e2bc05a5c7ec1a19d54d7030079f341c0f6c0c0fcf9fc88fbca3eef02b1c1
CompactCells875To906.Box892.QInterval	CompactCells875To906.lean:11308-11310	9da55871861c7fd4c96ce65d84ceaf1e80b040c6164198955924a6d983adf7b9
CompactCells875To906.Box892.lambdaInterval	CompactCells875To906.lean:11311-11313	0f6a44cea1516075dda95e913c948f778554ba47f61f7acc921a1b811b6cc3c0
CompactCells875To906.Box892.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:11960-11971	60c3d1ccf848e28562540f2285211506be161539e9dad1603e310002d2c3cd6c
CompactCells875To906.Box893.QInterval	CompactCells875To906.lean:11972-11974	bbdc6c608828b513e0bd34aa5e75c60c40e82e29ba28a2cc4e4803e050050911
CompactCells875To906.Box893.lambdaInterval	CompactCells875To906.lean:11975-11977	a9265193b41f429f2e5c197c73bcf8a0e2a11119bff9f074473f52d172551f18
CompactCells875To906.Box893.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:12624-12635	4e3a8c536855e39dc1a936d95fbd44c9d6d1b154ccc4f024973483e3ce9d2abd
CompactCells875To906.Box894.QInterval	CompactCells875To906.lean:12636-12638	184ff302210ce09dcfa2d823ca1310cb9bf48cafe224057534b75964ae00a0ad
CompactCells875To906.Box894.lambdaInterval	CompactCells875To906.lean:12639-12641	e9791fee73eab42a648d2babd68a7ea43e51e077c76c123835b1e4af64a0842d
CompactCells875To906.Box894.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:13288-13299	889602a000a98c2849fbd6a107736dd92e59e77a557bc652dfd9eef760098c1
CompactCells875To906.Box895.QInterval	CompactCells875To906.lean:13300-13302	648545b94d9746b69f3e43028c38347958d202b7a068a082ba0a43e2808891f7
CompactCells875To906.Box895.lambdaInterval	CompactCells875To906.lean:13303-13305	f822dd4823d0a40be845217c78da87117dc7199e50d3f8196c85adbfa49b2796

Declaration	Exact source	SHA-256
CompactCells875To906.Box895.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:13952-13963	6ffd357955028b263fb83733f2ec4b0ca60f8c2ba4aa61f183b5515f40db97c8
CompactCells875To906.Box896.QInterval	CompactCells875To906.lean:13964-13966	c575a50d5167d0ff064184954fc07b969511af1031298c88066db6bc9128fbbd
CompactCells875To906.Box896.lambdaInterval	CompactCells875To906.lean:13967-13969	eb6acb2494c0c964c990daf62cd1a3730e1db733cf691de3a440ca87a92e5b61
CompactCells875To906.Box896.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:14616-14627	58727b123979570e1b4a927fa4a92641e3663c74fdc3104407eaf3c7fc83d596
CompactCells875To906.Box897.QInterval	CompactCells875To906.lean:14628-14630	f9792561e381a9746b66904fa4278924d6028e89481cb0dc663178a6080edd7a
CompactCells875To906.Box897.lambdaInterval	CompactCells875To906.lean:14631-14633	dbaa4dfe9ac49ebaa1c7438532a769083e70b5fced1050a5eebf614a262c2177
CompactCells875To906.Box897.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:15280-15291	0f13619b653532cd58dac68563aac6b0e101eddeed570779b33a7f59cdb4cc27
CompactCells875To906.Box898.QInterval	CompactCells875To906.lean:15292-15294	3d41c2823fc955dc122c61881baf2b0cc40c2d7f7eabe3f279bc2b068d6ec184
CompactCells875To906.Box898.lambdaInterval	CompactCells875To906.lean:15295-15297	cf28d13a936f3aef74cdaf461d06f1746178568c798e52eafc0bb864009caa68
CompactCells875To906.Box898.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:15944-15955	cf1c655d06c5f50fd090abd3dc1ee54d716a08086c2cb7536d7d9a4d53ae2ce0
CompactCells875To906.Box899.QInterval	CompactCells875To906.lean:15956-15958	867f63fb9ec8f07214cdc8cbc31c24d94efb543e7de72bcf683c69ad87084b24
CompactCells875To906.Box899.lambdaInterval	CompactCells875To906.lean:15959-15961	3c2eaec2c353de79d4a8e5c4a7c34621bab803d690d7d66d3cec3d77fe3760e
CompactCells875To906.Box899.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:16608-16619	d8e783b5012de085047b5193d76abd65b35dc2bc0c00ee768077688d635e7f2a
CompactCells875To906.Box900.QInterval	CompactCells875To906.lean:16620-16622	7f1b220ff74eedace58fbbe74eee6f76f65ff3fe42a33b3cd5072e5e6dad731a
CompactCells875To906.Box900.lambdaInterval	CompactCells875To906.lean:16623-16625	a0ac2d9448e3af4e557e326fd7e59a050c88d060b10a55d2878aa473401a4295
CompactCells875To906.Box900.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:17272-17283	a9d3b0d95e34d46b171ce1d36581e196498eca2866eea83f322cba63657c2cc0
CompactCells875To906.Box901.QInterval	CompactCells875To906.lean:17284-17286	d6ea8a7a57d4544b9920394604e23499a84ff3e789dcc6da56d330d6ba52a5cf
CompactCells875To906.Box901.lambdaInterval	CompactCells875To906.lean:17287-17289	ceaf34f791b7ac79827fd72f7309bd1012be6bb7b31631c66d5ec00d29355ab7
CompactCells875To906.Box901.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:17936-17947	11aa4f12e4c08ab11f1cb11af216b5152cab64320be57c755f36b4cf6a2bf161
CompactCells875To906.Box902.QInterval	CompactCells875To906.lean:17948-17950	98ce15654f44c34842c5e828c3a6989bf4a5184d081f6d4993b3477a4555ac5c
CompactCells875To906.Box902.lambdaInterval	CompactCells875To906.lean:17951-17953	be28895e7a4db9014907838c1197f64bf06e54531c06536d2513911ab0a9dc57
CompactCells875To906.Box902.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:18600-18611	9baea41df623769ab9ee0d10a64c823ee689aa9fd361d44b3cf99f54ce2df4ca

Declaration	Exact source	SHA-256
CompactCells875To906.Box903.QInterval	CompactCells875To906.lean:18612-18614	f7395c0b739dfdc8b2e66301664bfff6940e0a2c2a66460631bd15f4bf14cc14e
CompactCells875To906.Box903.lambdaiInterval	CompactCells875To906.lean:18615-18617	2f29a98f0d11f7093cf7176e1c62970ab7fa0bc4ce0b4511449bb3f2f80be72b
CompactCells875To906.Box903.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:19264-19275	e7986187dd8d2ee7299973752ae23d1b8773f3cc25fa602b726be4fd1d627dc1
CompactCells875To906.Box904.QInterval	CompactCells875To906.lean:19276-19278	0bf7784c122a95fe45d5811cdd4692a40bbebfc1f5522193a3a0a713432dfe4
CompactCells875To906.Box904.lambdaiInterval	CompactCells875To906.lean:19279-19281	a86708692342190554c97207b5ac208e5c0e93f253ac3a09c14fcd8aea824b4
CompactCells875To906.Box904.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:19928-19939	3de40ad6c9459fcd8927a9413124db5f10a7f7e31a549a5f920a925ee94d7ae4
CompactCells875To906.Box905.QInterval	CompactCells875To906.lean:19940-19942	ed68c944831f1deb6c75a4992ce58a21a2a30e22e4e4a9f0c1e203a94972737b
CompactCells875To906.Box905.lambdaiInterval	CompactCells875To906.lean:19943-19945	9dd4ce0a5755a2afb5883d5105fc78368b67cb0a89a81af7c58b731271f16981
CompactCells875To906.Box905.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:20592-20603	72f54e5d00550a1e1346d34d868c131c98760c1584d8f76416801deb07f692ab
CompactCells875To906.Box906.QInterval	CompactCells875To906.lean:20604-20606	f68c1a240795ba7ce53d7acc8f756ac8abe06ac9574bc265791dea359aa02e67
CompactCells875To906.Box906.lambdaiInterval	CompactCells875To906.lean:20607-20609	e2941732cf39d1cf47df5f68c113592d7e32525f845511d90852dce5bca50442
CompactCells875To906.Box906.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:21256-21267	af59d0790dca28dda64f3c33ec77f7be507e6ed8439d4ed44c1182b7e1f789e15
CompactCells875To906.bboxes875To906_candidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:21268-21498	3a00d7bf7a2f4a36f1d6e38f3f47398e6f6f6ecbabde4b4f590cd28db1d4ccfb
CompactCells875To906.bboxes0To906_candidate_not_isWeightedPerimeterMinimizer	CompactCells875To906.lean:21499-21512	5753c755fc4c59462748908ef1312df1292b55cb8422639d41c8061a02975611
CompactCells907To938.Box907.QInterval	CompactCells907To938.lean:20-22	3679bb304174f4ed9e94ec630e96457fafdc0e6c0b79501abda961342cfba6dc
CompactCells907To938.Box907.lambdaiInterval	CompactCells907To938.lean:23-25	813925f120d54e95296835166b9f43eec3e25688e62b7dcc81e2878972053c36
CompactCells907To938.Box907.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:672-683	116ff4f601c39fe9c522594bfbe768f7eabc04e671981553e4b27f5ca98cc6d7
CompactCells907To938.Box908.QInterval	CompactCells907To938.lean:684-686	61bd60855127f650786a2e886a492cbc06102fed6b2d0d5decd3e2d893e27d25
CompactCells907To938.Box908.lambdaiInterval	CompactCells907To938.lean:687-689	f7c4847bff21c478b194a0813714fde7dddf965a57c88fba1f00f4b99f119eb12
CompactCells907To938.Box908.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:1336-1347	6e7a1fa2f46ed0d314668de67a8ed77e7c9e4b10d6bfabc883c80ce3a1fa4ebc
CompactCells907To938.Box909.QInterval	CompactCells907To938.lean:1348-1350	82342a459fcc62120c7d52d36f4ac170e770c5eef9aaaf24d62374ef7ecd3655
CompactCells907To938.Box909.lambdaiInterval	CompactCells907To938.lean:1351-1353	dd829ea128471f8eff5be16e6e5830a35f214158bfb6bedb3d6681668015d7a6

Declaration	Exact source	SHA-256
CompactCells907To938.Box909.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:2000-2011	ebbb308bd5bef97b4be5c51b19b7770e21460f0f9876e52b365944a1a79f21d1
CompactCells907To938.Box910.QInterval	CompactCells907To938.lean:2012-2014	b1eb5a94d83e542e8f68623a7ffb67907683ff1c68e2a0456ef3049fc17953d9
CompactCells907To938.Box910.lambdaInterval	CompactCells907To938.lean:2015-2017	bb28fc4fa0ce1dd737cc0daed7fc2b09453abc8753b92f14d0642ee5095759f6
CompactCells907To938.Box910.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:2664-2675	6c627f6b4f522d2d4bb9bb6d25dd04c5490eedaf5ac441a09a5005b8e190cb97
CompactCells907To938.Box911.QInterval	CompactCells907To938.lean:2676-2678	28e1c65bfc018b032776695d6c2bdc084e9b3032b69570dab242b87acd8c93ae
CompactCells907To938.Box911.lambdaInterval	CompactCells907To938.lean:2679-2681	a94b12952234c8c3cafda67063bc4fff014fb33e63e1384820c94caf22af7979
CompactCells907To938.Box911.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:3328-3339	ea0ae78cd9f60b445e1d7681619c86642d3a83a7922d0b394d2eb75d92966113
CompactCells907To938.Box912.QInterval	CompactCells907To938.lean:3340-3342	5eb490200acf3c2ecd5642ec75a573bcddfc4ab1b94374554617bdd48d3ce117
CompactCells907To938.Box912.lambdaInterval	CompactCells907To938.lean:3343-3345	6774ad39cc33f3304ce5115bcdcd53c00e4b2ebaa5ecd0038ad63251d0cf0797
CompactCells907To938.Box912.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:3992-4003	daa50fcd4623176416268bb243e29ecbf2bc0ba929706d4d14276e970299c261
CompactCells907To938.Box913.QInterval	CompactCells907To938.lean:4004-4006	a2210a61e8e50ba36f16725fb97525ca400c3a0ddf4d5db91e3f5dd84a6f4115
CompactCells907To938.Box913.lambdaInterval	CompactCells907To938.lean:4007-4009	989e121361a51dfc5c67b5f558ba1d699bc0d5c5893ec9ccd003d2a2b21279cc
CompactCells907To938.Box913.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:4656-4667	75f24a3e71772b6ef9ea93bbc91b2eab92f5c0ab109be9aae5e3a942040c0a11
CompactCells907To938.Box914.QInterval	CompactCells907To938.lean:4668-4670	4d754fed995a007ff5c6c118e87e271483b780dea48942a60a2ba78e96057e71
CompactCells907To938.Box914.lambdaInterval	CompactCells907To938.lean:4671-4673	2ef5a809210112a1b1985b18a7734a7d3c00d59811b0bf3775170fc47960b8cc
CompactCells907To938.Box914.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:5320-5331	6ce92252dac73e6863661b9e273e3900e07bbd2a96f47912cb8ea421f4551efd
CompactCells907To938.Box915.QInterval	CompactCells907To938.lean:5332-5334	b81c990a1b1b958ec26b0878711986d579a41c30bc0d40893c659d00959917ef
CompactCells907To938.Box915.lambdaInterval	CompactCells907To938.lean:5335-5337	d25d2e8662953d47bfa5efd9959dd0d8673d0c3db71f2bd599bcd7d2d4041a02
CompactCells907To938.Box915.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:5984-5995	780030b52affe6e5c224001465cd448631aab938d24334ae4fce8a3dade057b9
CompactCells907To938.Box916.QInterval	CompactCells907To938.lean:5996-5998	6c0b0d790d1615b88bc6615fc10d2557a76f20db5f1485a8f5115ccc05b8d2ff
CompactCells907To938.Box916.lambdaInterval	CompactCells907To938.lean:5999-6001	a560f0cd8e79514d13e25043a3cc6c90beebe35794db7d8652a2b6b940e469d2
CompactCells907To938.Box916.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:6648-6659	9601b0df5b9250e82190a8b67de6675652ea2b89cf23b1933ed56f670dfb18e5

Declaration	Exact source	SHA-256
CompactCells907To938.Box917.QInterval	CompactCells907To938.lean:6660-6662	130126da4c6b3762c5ec3f67d711b2cca05484fc01e16b5c5f4d7bc4e714a1b1
CompactCells907To938.Box917.lambdaInterval	CompactCells907To938.lean:6663-6665	7d4423cfd7f25d3aa57aa0fbf840b45aa0ea161e335a6432d106bc4975b4daa0
CompactCells907To938.Box917.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:7312-7323	2f545d5c024977287313a45bf633688eb1af4de82ce1f59e599b413529932909
CompactCells907To938.Box918.QInterval	CompactCells907To938.lean:7324-7326	1cb93746b358933a22905cd0ac5c142b482bd3b5d967cc82cddb8cc36e0d414f
CompactCells907To938.Box918.lambdaInterval	CompactCells907To938.lean:7327-7329	1225d67f6e362caa80e35fade2302bab1835f0db9b9084fdfdb30154a44b14c2
CompactCells907To938.Box918.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:7976-7987	e9fa38b292e334aed1dd4a0774c563dc03a40cc711501441d0353542d38cda76
CompactCells907To938.Box919.QInterval	CompactCells907To938.lean:7988-7990	18a342079ef688034a8c36631a484dd1fd0b6e5184e2852c9a841b8a2321e018
CompactCells907To938.Box919.lambdaInterval	CompactCells907To938.lean:7991-7993	9ccb4a9e3aa97096188be9ba4979fc8adc0f308022d6d74e62965425e7032333
CompactCells907To938.Box919.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:8640-8651	78d1be5e6342ba9f327b440a37d60a9edd2e667dd710dd0f482d94161fb30597
CompactCells907To938.Box920.QInterval	CompactCells907To938.lean:8652-8654	9d9c54c20e5787f98290324a8bee30407ee72a30428145f60e4deb5a22083bbd
CompactCells907To938.Box920.lambdaInterval	CompactCells907To938.lean:8655-8657	9ee0e163e77a6c6dc2e67b28cf88b5c3c80336a25d7139ab087a938cfe2043db
CompactCells907To938.Box920.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:9304-9315	b56742e94e342150bf1d3308489c86870c53f5940b2784d9d43f6c34ccd480a4
CompactCells907To938.Box921.QInterval	CompactCells907To938.lean:9316-9318	de5e0cac35d9abd80f76ce63b20930dc03e1517ae547dbf76c4c1e85a74ed471
CompactCells907To938.Box921.lambdaInterval	CompactCells907To938.lean:9319-9321	35e5ff8e01c46158cea644aa63ba3c1eac20d27ab174786c3945ed5ca25646a5
CompactCells907To938.Box921.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:9968-9979	53105efeb550db993f54dc2b717ae27707a020971d673a8d4a1676b47368030d
CompactCells907To938.Box922.QInterval	CompactCells907To938.lean:9980-9982	3f9d40fa88344177d0baf6ab75c4f5641b71327690038c9edee37479289d2aea
CompactCells907To938.Box922.lambdaInterval	CompactCells907To938.lean:9983-9985	1104361ca62d10b43989b8aea491682be83de3ddd313fc03c8f2635ae0120037
CompactCells907To938.Box922.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:10632-10643	752e1a4b435b04352fb7019e543247961454f3f7cc566a8c8a9b8056965da291
CompactCells907To938.Box923.QInterval	CompactCells907To938.lean:10644-10646	3fc8bafb36805b6f6b286a373d959f198335793a74265f0d2b5ff11502795956
CompactCells907To938.Box923.lambdaInterval	CompactCells907To938.lean:10647-10649	affb217374d14336871903575c8cccc0c5d7e11722e45eb605741180f83624c8
CompactCells907To938.Box923.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:11296-11307	8d3ebf12365f75fbf4ad0fd2ecd7b7a8387737dde9cd4994b3c94d1684cc9505c
CompactCells907To938.Box924.QInterval	CompactCells907To938.lean:11308-11310	9ef70b86343856fd12fc542227227051f9c1f8178e5befe37ca2030eba0cfd34
CompactCells907To938.Box924.lambdaInterval	CompactCells907To938.lean:11311-11313	b51eaf2c1315da6f03e9a9902937ff5b64927765c5d165b36daf735213f7a8e1

Declaration	Exact source	SHA-256
CompactCells907To938.Box924.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:11960-11971	d4fac9fd266d4a64737ab6eb5f7f961356afb0f1f86e2f82c6dc1e120777d157
CompactCells907To938.Box925.QInterval	CompactCells907To938.lean:11972-11974	29085f80f90070ba2942dcf7b5de0f1ce1ddc012af80f2b07618f87394d175fb
CompactCells907To938.Box925.lambdaInterval	CompactCells907To938.lean:11975-11977	ce69963b6806baed68f48ee50e8b2e64e5985e4754c010a7aff0a666586cad4e
CompactCells907To938.Box925.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:12624-12635	500c066a82aa1f962d6d4d20b3403ab1aa03c30addf15a07f3e258acf982443f
CompactCells907To938.Box926.QInterval	CompactCells907To938.lean:12636-12638	4d85cf36091f22f023aad8cc5b3b83b5d0db18280a65d329594424d03f583cd8
CompactCells907To938.Box926.lambdaInterval	CompactCells907To938.lean:12639-12641	6d82c399416ab3c010288f5556ef4ca62735e3d0d743bc3848fef334b3a28c22
CompactCells907To938.Box926.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:13288-13299	de6e520dddf4bf9bead76e30871b86d29be2485b65519ce9fd71510b52f67e734
CompactCells907To938.Box927.QInterval	CompactCells907To938.lean:13300-13302	b26ba705e5f90a3074680b2e9212680751c6dcb66eb7d5197e03d3cf55cb1ff2
CompactCells907To938.Box927.lambdaInterval	CompactCells907To938.lean:13303-13305	aee2747400b701d0bd4a0fd990a368b9f6c982c7781f9eaaeff018e23f57ea7a
CompactCells907To938.Box927.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:13952-13963	d1351647ff01fb53fb2422ecb7156fbac55ec0c943391319af371ce4077a43a6
CompactCells907To938.Box928.QInterval	CompactCells907To938.lean:13964-13966	912c22769481d66c0f8bd6aede9153dccb618018474b46cb5028896c934f4b49
CompactCells907To938.Box928.lambdaInterval	CompactCells907To938.lean:13967-13969	39cec87ebc133e92b70b103d22dbde64609f7d06ef3cd229b2f5b665f763968a
CompactCells907To938.Box928.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:14616-14627	245cc0aaeaba300a63251eeb0da76bcfe76f37b495b1d4b7f89712c422c22cec
CompactCells907To938.Box929.QInterval	CompactCells907To938.lean:14628-14630	ba4455f88d0dc3fcfdaa4da2e0087094e8aac2163d4ef5be45110a563ef0d817
CompactCells907To938.Box929.lambdaInterval	CompactCells907To938.lean:14631-14633	c7c3cec3c5b267ea22a39c9d6c56684fb8a22af9c0a70d16e305380f49ae7aa6
CompactCells907To938.Box929.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:15280-15291	898aef66fb7edae58087ac58bd18a7ff854d7dd7d96cc9002113bc3d86d752f0
CompactCells907To938.Box930.QInterval	CompactCells907To938.lean:15292-15294	463ba8c175b2da2c6563cd1481f991fe382ee798149564ed82a9c17b662f1046
CompactCells907To938.Box930.lambdaInterval	CompactCells907To938.lean:15295-15297	6b3f64b201257ba068d5a2c6abc56d7e4c4315493ad501dc02bc9c8fc6094180
CompactCells907To938.Box930.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:15944-15955	ddbfeb3e4c815b85d9a6c212ab56bd536b38c1b4b1e2e0a0c7b77b6b7c90507d
CompactCells907To938.Box931.QInterval	CompactCells907To938.lean:15956-15958	36d9f412cb3f5e4d4230b35a8d31dc65ed6e8aa47d06fced778cb6428012c834
CompactCells907To938.Box931.lambdaInterval	CompactCells907To938.lean:15959-15961	3494b25653522e2707a9707bd703996abc2d17068d5e971540a4c1a68ad9c1b3
CompactCells907To938.Box931.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:16608-16619	50cde685ba6c7db4886992da0859b9a01d72098223db9c5bc51d681ce6a2935e

Declaration	Exact source	SHA-256
CompactCells907To938.Box932.QInterval	CompactCells907To938.lean:16620-16622	5649937582d583466054b8b067239032f8cfee871717ce6c8dcdaa4e33e380e2f3
CompactCells907To938.Box932.lambdInterval	CompactCells907To938.lean:16623-16625	9663dba0fd2954731573235d0d18bd1d27f6749e0f334823bd36cb1f452ca900
CompactCells907To938.Box932.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:17272-17283	3e4a0f7b57c0cba9f294d97bc83355f9dcc46d9d8be9d15002ce4056d17c65ab
CompactCells907To938.Box933.QInterval	CompactCells907To938.lean:17284-17286	101edadc29c3103dd0753e593a2ed9aba5b234609c63d3a82de1d62b9cabcdc4
CompactCells907To938.Box933.lambdInterval	CompactCells907To938.lean:17287-17289	87f45b5f7919bbcdc93a50ff2718e6243cbe9f1832da2b23b75188fa0ac697f9
CompactCells907To938.Box933.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:17936-17947	2c421c456f2fd900cde1ec7c971d5cb1aa835cbd3acdaa600fede4e8f4b1ad79
CompactCells907To938.Box934.QInterval	CompactCells907To938.lean:17948-17950	82df24e42432f7c5ef8f72c858529955803aad13d900f497202e9cb6199183d0
CompactCells907To938.Box934.lambdInterval	CompactCells907To938.lean:17951-17953	6e6c707a2ee92d722eb42f3c3b9b04dc2d8a6e6df06a0de23fabf0ceee4e3676
CompactCells907To938.Box934.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:18600-18611	99c9d59c95e4a52aec06ecf0fa1806f1c5a00b81545521794061402fc7827eba
CompactCells907To938.Box935.QInterval	CompactCells907To938.lean:18612-18614	f52228bf8e2ae1a82f0b9f1d62089dc19f026b9672cc6c6e188d2480e9e2cb33
CompactCells907To938.Box935.lambdInterval	CompactCells907To938.lean:18615-18617	78e37cb3dc49dd087d04107eb9454c595f9293d5c8d365c86a1eb4fe4b19f318
CompactCells907To938.Box935.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:19264-19275	63a689e96ee0cc8f618487117ee5608a1bfaad3897d784cc77e066394a6abf87
CompactCells907To938.Box936.QInterval	CompactCells907To938.lean:19276-19278	f28e4df8f0edfd20afaff0c28d2f3ec1146fbd6794ef68f43db8021a861d928a
CompactCells907To938.Box936.lambdInterval	CompactCells907To938.lean:19279-19281	4805aaa87e6b8579227980d377834839c1890d8ece3408cbd5c68e5ad56b8ccb
CompactCells907To938.Box936.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:19928-19939	7810bd0cab92e55e2cbcfe3ae7516a3f513f26e7d184360331df91aaff7c9269
CompactCells907To938.Box937.QInterval	CompactCells907To938.lean:19940-19942	08b1c916fcfe4e780c1da3c676ad851d8c989bd6d3132362bccb495c7949ddd
CompactCells907To938.Box937.lambdInterval	CompactCells907To938.lean:19943-19945	cd92c9c5aa4265e7a451a6a8c53f4754376e32055f0f99ccfb58faf4bccdf4cf
CompactCells907To938.Box937.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:20592-20603	f5031cc47b0dfb52585494598131a07ced3cdd54862ffee68ade583b273cefc9
CompactCells907To938.Box938.QInterval	CompactCells907To938.lean:20604-20606	3e55fa120b1cac0e45c454f4412fa1e308bcd91cc4814b2d1da0d8fe23ba66ea
CompactCells907To938.Box938.lambdInterval	CompactCells907To938.lean:20607-20609	c951fc0025362e2a6c384696a83918cbbd6ca3b24a07e09b059662769859de8e
CompactCells907To938.Box938.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:21256-21267	0d125c9148ea77632aead845519c5c2c2f80023b7c7fc78597e65900c1322ab2
CompactCells907To938.boxes907To938_candidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:21268-21498	d50e2da16fc128de2b119a6f1801159d987dac732f0bcac88a2e23434b85234f

Declaration	Exact source	SHA-256
CompactCells907To938.bboxes0To938_candidate_not_isWeightedPerimeterMinimizer	CompactCells907To938.lean:21499-21512	6a29df51ff0443617f7318e6c98da0ff006accbf8a78bdc713b91776d7bbb50e
CompactCells939To970.Box939.QInterval	CompactCells939To970.lean:20-22	001551f1d2e67c3334b8d62160b4cce997dfc97574700940ccc67cf23f4893e7
CompactCells939To970.Box939.lambdaInterval	CompactCells939To970.lean:23-25	e1be114d267cd6bb786bc5796d4bbb84c148356ed21f1c72b991ce79d0428935
CompactCells939To970.Box939.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:672-683	90867974fbb7b19ac688f8fef80e3dde0f4d6b3a29ed6ed72185335a850474d
CompactCells939To970.Box940.QInterval	CompactCells939To970.lean:684-686	51be92a5f5739c61d67b6693e5cef5994bdf3e3ca27d4dc01a74f2523f9d14d6
CompactCells939To970.Box940.lambdaInterval	CompactCells939To970.lean:687-689	fba3bcaef36c9fb1cee63d9a893572e2601e210f7b89cdaf7c64e6e983de141a
CompactCells939To970.Box940.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:1336-1347	02e1fcae3263fbd1b489ca9ca95d990c9e204ca1ebc926e54c20d541acb3c205
CompactCells939To970.Box941.QInterval	CompactCells939To970.lean:1348-1350	d2e2faedb79bab318a12e553866998b9bdd44b28d72e7b3db927d135147d965
CompactCells939To970.Box941.lambdaInterval	CompactCells939To970.lean:1351-1353	4ecbf43297148aec23183275ab1a2738b46ffb447e0786507fed9251db5fd850
CompactCells939To970.Box941.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:2000-2011	14fbc987fe46fd4528b17c3df64e1ac26153998d2f6ce9b8c9e6646280c4eaa6
CompactCells939To970.Box942.QInterval	CompactCells939To970.lean:2012-2014	4d2b23448596a4cdf8427776ab77be2f3730700dd4971c0a5aade60555bc6875
CompactCells939To970.Box942.lambdaInterval	CompactCells939To970.lean:2015-2017	ccd13b81dc55012761673e568cd1932c39d070ff621e4c0b37ff867df92d7f9f
CompactCells939To970.Box942.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:2664-2675	b2bf6eb0d0a040313ce04976ffa380f09290e5fb2a3a88381dcf5a4480a4e0c7
CompactCells939To970.Box943.QInterval	CompactCells939To970.lean:2676-2678	647392fb99a8401e3897edd444842a16d6ab8aa08dc5dbeac5d3915da1b73ff1
CompactCells939To970.Box943.lambdaInterval	CompactCells939To970.lean:2679-2681	8e4cee3e55ee12bc199ea3cc862309d93c15f5e757aa3f7f161a959eb2da3696
CompactCells939To970.Box943.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:3328-3339	483219d65941114eb194922ef4b9c2adb2854c4d656bafca4ab60526139f8a18
CompactCells939To970.Box944.QInterval	CompactCells939To970.lean:3340-3342	8d96fe34e5f0edebb34be47fe39a4c34f84671bea17ba1868cf3dcb1cd81619c8
CompactCells939To970.Box944.lambdaInterval	CompactCells939To970.lean:3343-3345	79712f56722b1cfc45e669ea6cc331eb582b2a343deab405991036b7f6029ba6
CompactCells939To970.Box944.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:3992-4003	66c97cef0fea350de7952bf10e1d649041d6b482ff4615e15e64089d20623f37
CompactCells939To970.Box945.QInterval	CompactCells939To970.lean:4004-4006	39c9818c7fc164ff7fa13fedd76bd0cc4e3f8b8affcecd0d5aeb9e092dad22a4f
CompactCells939To970.Box945.lambdaInterval	CompactCells939To970.lean:4007-4009	3aa7ce69c4608f155016a7b90bb209038d19b06416a0e03f6ce5b1dea81137d3
CompactCells939To970.Box945.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:4656-4667	4a31bccd69dabd8e4cdcb0d3f3f949a575cfb7de7d5d67f5139433393162433d

Declaration	Exact source	SHA-256
CompactCells939To970.Box946.QInterval	CompactCells939To970.lean:4668-4670	6f6d2d9bd66787e4336a6524e4b9a50171566483a467ecde5703f41823b2c652
CompactCells939To970.Box946.lambdaInterval	CompactCells939To970.lean:4671-4673	5d2864e736c877144f8b149547dd85d3e7981a650ea796fec1b0e57cd4c20add
CompactCells939To970.Box946.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:5320-5331	11c5727732a96fc3017c6fb1662464d3cfb845ed3bf3bf6f47eb21c961facfc8
CompactCells939To970.Box947.QInterval	CompactCells939To970.lean:5332-5334	6c9740964607ffd645969d8a22d772fbd917ff7523ffff5c54b18fe5688670219
CompactCells939To970.Box947.lambdaInterval	CompactCells939To970.lean:5335-5337	ad2dc0781ae6e5f84e8409e76247e9f715c793b1f2acea4b1c925f1f8147f2d0
CompactCells939To970.Box947.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:5984-5995	8fa0beb51c5062fab7b94f6643ea5383c475c83d24b55c308ef20ae4eb7044b3
CompactCells939To970.Box948.QInterval	CompactCells939To970.lean:5996-5998	72edaf5cb8b3bcd6d6a2a72bdc969b9801e83b5c540272c1f97ca244d5aa11da
CompactCells939To970.Box948.lambdaInterval	CompactCells939To970.lean:5999-6001	7a65539863ba2b023ffe834c397ac8c7c29ff5e05fcdae2bbdb8a16235ec88bc
CompactCells939To970.Box948.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:6648-6659	295288ad8becd39973d5603f626a74190ff8ee1dfb74a6281e6beeb36f9fba64
CompactCells939To970.Box949.QInterval	CompactCells939To970.lean:6660-6662	f4673233b7197d9448fd460ada053c5ed1f5581b6d310bebc50f2ffc092cc4b3
CompactCells939To970.Box949.lambdaInterval	CompactCells939To970.lean:6663-6665	f88eca34c69bf05b4335cf93672005b742c2147a9ffc804acbf3a33b87ffae83
CompactCells939To970.Box949.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:7312-7323	8168a73fae98902d81c5c797931e99dfa131885bd1ab8ca84e95f9a6c1e529ef
CompactCells939To970.Box950.QInterval	CompactCells939To970.lean:7324-7326	59745501dcf4c4ca9f33aabab5c1a76511fb5a3f64641bd3518714c4b3750d3a
CompactCells939To970.Box950.lambdaInterval	CompactCells939To970.lean:7327-7329	3968f4141082814090b5927dc91616ad804df25af27a6f2876536d16c7aaedfe
CompactCells939To970.Box950.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:7976-7987	44a2547e4bf71e7ba2d0ea2379d8cee623d77d6d708f241a967a1ca607ba6342
CompactCells939To970.Box951.QInterval	CompactCells939To970.lean:7988-7990	7d2a7a427c9bb811dd573907e6471a1becc2ee98dbe53a88d989f80ddb27eee2
CompactCells939To970.Box951.lambdaInterval	CompactCells939To970.lean:7991-7993	e3d5f64c507bb83b2c78c968453dd1005bd3c595ac0d0c8e6b98822bd78da7b8
CompactCells939To970.Box951.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:8640-8651	6eb95f929016db11cdafe4774be9af10815c2a5993780bc0542757dd03bacbb7
CompactCells939To970.Box952.QInterval	CompactCells939To970.lean:8652-8654	a74e2dce97be889926d6695f449c258524c037a44f6da70ccf9427946a5872fe
CompactCells939To970.Box952.lambdaInterval	CompactCells939To970.lean:8655-8657	279994e908d36a8a6359df80a5ef644ed076d653c1ce8472513237d2553df38c
CompactCells939To970.Box952.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:9304-9315	ce20aac007d42946918bfc7663a1f5a599092ce5b6e52c5e0fad814362d44124
CompactCells939To970.Box953.QInterval	CompactCells939To970.lean:9316-9318	71c25972d677e6a73dcd41fbe0e1b1439c2314b56207afd7c68877de132a3dcc
CompactCells939To970.Box953.lambdaInterval	CompactCells939To970.lean:9319-9321	dacf49a0951e03ee3874c25a1102b6b9fb2cc4e0a7fc1c4aaf755df685cbef95

Declaration	Exact source	SHA-256
CompactCells939To970.Box953.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:9968-9979	15c7db81e91a78be20cbe8ecf55f5b15aed da0899b4af94d1b19c02348fd0e26
CompactCells939To970.Box954.QInterval	CompactCells939To970.lean:9980-9982	b8a5cd1bb3d8f5054a7d6b694abd1b7c52b 49124b35b130a5f38af4082db7040
CompactCells939To970.Box954.lambdaInterval	CompactCells939To970.lean:9983-9985	cc9e5c2aa198454a3ca76dfeaf302d481b3 497bf30bbc5d77d3549210b2ae481
CompactCells939To970.Box954.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:10632-10643	9512514c37bfd7998c440b8a0f79da719f9 b0facfe19f019c18820b50e650a39
CompactCells939To970.Box955.QInterval	CompactCells939To970.lean:10644-10646	7b71c305315660a119157267661996973e9 77a0a6852fcae4483a6f895f75103
CompactCells939To970.Box955.lambdaInterval	CompactCells939To970.lean:10647-10649	a955e341d4a7e1193229e1f7a1000622d96 d2a1c14ce1a8c588f04511c00f19c
CompactCells939To970.Box955.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:11296-11307	2e86d6b72f642b102270c4e5779b3912a3a 366c6c28c9e676eef479db0e96250
CompactCells939To970.Box956.QInterval	CompactCells939To970.lean:11308-11310	2d37ac41c2bd3036ee0219d3833275e1803 2d262598ff2492666a64c40042281
CompactCells939To970.Box956.lambdaInterval	CompactCells939To970.lean:11311-11313	253ba1721d93b37feb78a0df4cfc73edc90 38aeebb0e18261d251e3881249b21
CompactCells939To970.Box956.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:11960-11971	49cac853b450278ee05551b48f9bf5e9d65 5b59828171594212d2e143b04e024
CompactCells939To970.Box957.QInterval	CompactCells939To970.lean:11972-11974	7f83f9ceb9934114f6e70354a4d66147a7a c3eaccf580a020437b476cc470c74
CompactCells939To970.Box957.lambdaInterval	CompactCells939To970.lean:11975-11977	75c0bd40d4414ba856506993c036034ebf3 916d9ba78660d1241eb3aadd91ee7
CompactCells939To970.Box957.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:12624-12635	59e30a24e9affba8bc160dfe032b19d0aaa 9a924291d7920fc84fb8ef855cfe8
CompactCells939To970.Box958.QInterval	CompactCells939To970.lean:12636-12638	dcb85d0d316736328556747f0d2760a2c5d 90f4209154f7903f0d041679e960c
CompactCells939To970.Box958.lambdaInterval	CompactCells939To970.lean:12639-12641	3ed2a70d0c43bef6ad482156972e2719967 d8266ce497412d3b5c2156e22f175
CompactCells939To970.Box958.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:13288-13299	7a8c071c35c36827cd36b2451b7be8a3425 874afb724a0afcedf44814b1d937f
CompactCells939To970.Box959.QInterval	CompactCells939To970.lean:13300-13302	b5b66440ab498eb7e1657223135b5189a61 d005020a424ede66d79a9081fbba2
CompactCells939To970.Box959.lambdaInterval	CompactCells939To970.lean:13303-13305	5ecdd5a7ca52cfd61fdd5db23888381b10d 9ec68734dce5d882d7fcb5db3a3e0
CompactCells939To970.Box959.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:13952-13963	08f9aaa0d6d88d34ed8018f4311bc616a70 13288a9f71c42fad1942600fd05ff
CompactCells939To970.Box960.QInterval	CompactCells939To970.lean:13964-13966	15cae777e0493978ac4e468d6d13779dbdf ea15a26efa3b284f4c4690c94bc40
CompactCells939To970.Box960.lambdaInterval	CompactCells939To970.lean:13967-13969	9e5465e434e02eb70588462206d3808680b 823b5b54d96afd27d71bb769ae14f
CompactCells939To970.Box960.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:14616-14627	45c1b4724b9012e973fb5dd21ac50e22c16 a21ff78543dc028dab6ef2f48d409

Declaration	Exact source	SHA-256
CompactCells939To970.Box961.QInterval	CompactCells939To970.lean:14628-14630	6ef04336cde86dec53423c9bf64475221fdbad20cd94897572368a43d5f62f0
CompactCells939To970.Box961.lambdaInterval	CompactCells939To970.lean:14631-14633	69a4573e90c9342badd123fba48eec9fd5aebda6bbcca80a298b29fecbbbe72a
CompactCells939To970.Box961.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:15280-15291	b355eceed03a37f85459be84628256e5a0066f644eda66d8578760be947b7749
CompactCells939To970.Box962.QInterval	CompactCells939To970.lean:15292-15294	703fc4d2a3b9e2939e58cff7bba5848f952a04d0a02ff4f5835bf2852112793e
CompactCells939To970.Box962.lambdaInterval	CompactCells939To970.lean:15295-15297	8c0fe57367a47b6f2514a6fd8fe0c6ee04a4d9a6024ebaa01d4d6855d0e84fb9
CompactCells939To970.Box962.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:15944-15955	caedcc85019999ddd2552d7934827b374244fc6626f904e5144af701c2429f57
CompactCells939To970.Box963.QInterval	CompactCells939To970.lean:15956-15958	a7195164dab555cd72c50173b20f21376f89ed2e1e28d088361ecb6cdc6f15d0
CompactCells939To970.Box963.lambdaInterval	CompactCells939To970.lean:15959-15961	363aeca8917eef76a87527444ab015d84dd436036beb9e7796e8989c9faf47bb
CompactCells939To970.Box963.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:16608-16619	97d4cdd356a3d2ec0d9c493ebb085275d7e806ec73a315cf32e997bb0b6ee33e
CompactCells939To970.Box964.QInterval	CompactCells939To970.lean:16620-16622	b67b07cb928b9809995be20a3093e9a385f5308b2341db9c1d7c6dda18ae233d
CompactCells939To970.Box964.lambdaInterval	CompactCells939To970.lean:16623-16625	ac5f9043bebc1a6600a889bd641b4db2c0d29379bfdc9511db543166c2d8e8b5
CompactCells939To970.Box964.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:17272-17283	989e1015c8e128b481f8300fbe25d4a9af44ce90132748d052d3932f4c937773
CompactCells939To970.Box965.QInterval	CompactCells939To970.lean:17284-17286	063763ea1858ac47afac13b73da87351d67493b4069637cfb8d5b5f208ae356b
CompactCells939To970.Box965.lambdaInterval	CompactCells939To970.lean:17287-17289	1809dcce69cf07c9b70d7ba677cfcfc69632d62e44a1a62964c1be0e244749d7
CompactCells939To970.Box965.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:17936-17947	0d3734b75eaf21025161f51207122d73d0a6b0a6430c337a6b08bc14fb15049f
CompactCells939To970.Box966.QInterval	CompactCells939To970.lean:17948-17950	20e243e17fb53ca8724ff39020928921406e821c382becd483e66855e1a0a264
CompactCells939To970.Box966.lambdaInterval	CompactCells939To970.lean:17951-17953	a46a26e46459a45242cbad7ce0d1de44807953c83d4f2897c72df2450a66d15b
CompactCells939To970.Box966.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:18600-18611	212de0748844646b1092eb8edc13fc8e6a1e749d72bb5ebdc5ff933bb00262b3
CompactCells939To970.Box967.QInterval	CompactCells939To970.lean:18612-18614	67f58cd7a5f892b413ba6410c5002c1e4586601bf4d0713217bac46243e5d942
CompactCells939To970.Box967.lambdaInterval	CompactCells939To970.lean:18615-18617	b24dc3937ee8e181ba653869996cb4dcb877dc7734637e3b763d5c05a97a2a60
CompactCells939To970.Box967.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:19264-19275	177b0771ea3793bd0bf9b258bc27a409650aef5ac1769eb8be7079f04dfc42e1
CompactCells939To970.Box968.QInterval	CompactCells939To970.lean:19276-19278	949aee2ce935aa5f442e529bec71b4f5660c14dbc66507add583b9dcd7d52ce8
CompactCells939To970.Box968.lambdaInterval	CompactCells939To970.lean:19279-19281	8ea1acdec8f741bff9117a13183f68f3eb70a1cf6a6c43198e4d6f4c68d1e6eb

Declaration	Exact source	SHA-256
CompactCells939To970.Box968.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:19928-19939	f419262ae3d80161b71c023b895bee7b7c0894aeb99751a9eb043a78666e592d
CompactCells939To970.Box969.QInterval	CompactCells939To970.lean:19940-19942	bd95fac79dc4eb1b18d971e4ac5705ba87c2163bac9c41d1ee24611a02bcbae8
CompactCells939To970.Box969.lambdaInterval	CompactCells939To970.lean:19943-19945	8bcacf4065bfddb103429b8b7ddd46b1a2fcff967e567a83672c412d849a08625
CompactCells939To970.Box969.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:20592-20603	1b22c0ce0b437b5a315bbe663d30e599283bd8bc64f1d110e59c852293cb2651
CompactCells939To970.Box970.QInterval	CompactCells939To970.lean:20604-20606	a649d47ecf96ed1939d927b98bf488202be0baaec7bce6c9431c850d03607c7d
CompactCells939To970.Box970.lambdaInterval	CompactCells939To970.lean:20607-20609	eb33100df28428bd0e24e60fb63a0e7ef3bf65457736ef77cb5af70f5282c357
CompactCells939To970.Box970.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:21256-21267	21c7a75bda7b6785099a1007db7149201d832fa8d44ef5d8610d27fea71e7942
CompactCells939To970.bboxes939To970_candidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:21268-21498	0348ee46b2efa245082e5e1469eaafe26785fa2d504c1d8568302ce8e0b1bcfc
CompactCells939To970.bboxes0To970_candidate_not_isWeightedPerimeterMinimizer	CompactCells939To970.lean:21499-21512	1af9623ff32ed1306032f7c9c33fa085ce585dc883973c02aff47312f4f9cd4e
CompactCells971To1002.Box971.QInterval	CompactCells971To1002.lean:20-22	2aa9b78703ee2ae1ccc4976a059d3c39afa2156794f1073e57f8f48f2f9bde42
CompactCells971To1002.Box971.lambdaInterval	CompactCells971To1002.lean:23-25	84494f706e6d3c3f3971ae1dfce3e717255960b7f5daa6bd0453c9a0f6dc78a9
CompactCells971To1002.Box971.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:672-683	766f7bd7a11e25a20728254db8a8c193e321ce62a3406a610163e72913da85d5
CompactCells971To1002.Box972.QInterval	CompactCells971To1002.lean:684-686	6f5b3e53fde24d06c6c7617be50dd43fb62b547b430fbafcd8d57f50fa1052886
CompactCells971To1002.Box972.lambdaInterval	CompactCells971To1002.lean:687-689	caf3675876523740ec08e8b2b26a943d05ec578089736807c901c7d2a05dd986
CompactCells971To1002.Box972.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:1336-1347	c07fd47b1c05e8f30a1a29a76ba98530b22ca1b963467ca4390945766365df93
CompactCells971To1002.Box973.QInterval	CompactCells971To1002.lean:1348-1350	194370388974df48653b9ff02197caf4841624aa4ed5c084e8d89d7e0f3f13c6
CompactCells971To1002.Box973.lambdaInterval	CompactCells971To1002.lean:1351-1353	445d2554d1af182a751557e40590eb3635f7d9fa37605d6958930ea6d02171a4
CompactCells971To1002.Box973.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:2000-2011	b0138e5bb6f9afc30514e00fe0c44507604b068e1bb02252a21c62c721ff7d92
CompactCells971To1002.Box974.QInterval	CompactCells971To1002.lean:2012-2014	474b15a8343d35edb75dfe7e596dd1794bc09b770840a0af32d8c6f2157806cd
CompactCells971To1002.Box974.lambdaInterval	CompactCells971To1002.lean:2015-2017	aad8cfadc5a8c51297d085ac86e579ba9f8488d91c7ae155ddc501f0ecd449d7
CompactCells971To1002.Box974.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:2664-2675	7cf2403908688fd8ed07d21a26aa086b500b7b846704af44b85d4506cb9b0301
CompactCells971To1002.Box975.QInterval	CompactCells971To1002.lean:2676-2678	e71ee2d00babd46c24f3586670c218f11a306b922a73804196a52b1ab0e58a30

Declaration	Exact source	SHA-256
CompactCells971To1002.Box975.lambd Interval	CompactCells971To1002.lean:2679- 2681	7d6a06a9dff46d17752bad0b090e91142a4 ad511c05d0fc0b6d00b8bcc0d0ac8
CompactCells971To1002.Box975.compac tCandidate_not_isWeightedPerimeterM inimizer	CompactCells971To1002.lean:3328- 3339	2f4b3099efd876e83ba00797cc1f91a1a94 0d838cfbfff45f09806a4f6a3f7c48
CompactCells971To1002.Box976.QInter val	CompactCells971To1002.lean:3340- 3342	179ed42344f21d4da7b992abc9f9ea01377 7af3ea4b061e7b0b5410d829de506
CompactCells971To1002.Box976.lambd Interval	CompactCells971To1002.lean:3343- 3345	543b035279693e9a75818a4485be4cbc886 c1cd6b60eade416bf464f5fe5ea33
CompactCells971To1002.Box976.compac tCandidate_not_isWeightedPerimeterM inimizer	CompactCells971To1002.lean:3992- 4003	f0c10fc1179791f07d367ad69e6305b5504 4951e2169d3dfead69074a0411ae6
CompactCells971To1002.Box977.QInter val	CompactCells971To1002.lean:4004- 4006	fb76097d90597826820f3b5d2cf2893c940 6df74cb7ab8720fb6d561e5ce80b4
CompactCells971To1002.Box977.lambd Interval	CompactCells971To1002.lean:4007- 4009	87ef25f2b89d3c04d391b76ff38b52d3668 0bbcdad46455ac1e3f2ddc6aa2e5c
CompactCells971To1002.Box977.compac tCandidate_not_isWeightedPerimeterM inimizer	CompactCells971To1002.lean:4656- 4667	da6f7c3c3e47b6cb9841c5868eb62bb8a52 2ecc6e66bfa28b9b5238662184127
CompactCells971To1002.Box978.QInter val	CompactCells971To1002.lean:4668- 4670	c4ce4eda1935c6326b97fdf1d2e6bee8bcb 2009e82e4b17c638b21a4612ba1bb
CompactCells971To1002.Box978.lambd Interval	CompactCells971To1002.lean:4671- 4673	ccc4f8d1d7506caae5208a030a5bf27a98b c837e30bc287274c2152f7fbdf968
CompactCells971To1002.Box978.compac tCandidate_not_isWeightedPerimeterM inimizer	CompactCells971To1002.lean:5320- 5331	bb4783b85dd269cf6d98e9661a60784c8a3 16498500575239be77ec976a25925
CompactCells971To1002.Box979.QInter val	CompactCells971To1002.lean:5332- 5334	84391e9fe79be1812519f804ebc2f5099e8 499514c064480be6c9537b3bfa952
CompactCells971To1002.Box979.lambd Interval	CompactCells971To1002.lean:5335- 5337	e6116ca7af8ebca4c54dccbcb909dc92686 0a23820815194844d0c3ec1804ca7
CompactCells971To1002.Box979.compac tCandidate_not_isWeightedPerimeterM inimizer	CompactCells971To1002.lean:5984- 5995	30fde344598469e10fb736aef664c3a3842 78ba3a09ed611e0f0bcf31d5f06c1
CompactCells971To1002.Box980.QInter val	CompactCells971To1002.lean:5996- 5998	f3e61b554add802d8c566398d8a9faf2bdc 59645b37d2a935b2c9cb0e858b6bc
CompactCells971To1002.Box980.lambd Interval	CompactCells971To1002.lean:5999- 6001	c020e89903fc66b60648b39d358f0431beb 6d3afcd5310d896b4c2b008dc46d6
CompactCells971To1002.Box980.compac tCandidate_not_isWeightedPerimeterM inimizer	CompactCells971To1002.lean:6648- 6659	e005d916aab6876f14597d7ffe297a20df3 ddc3e6f425a38c2c20ffaf375a2d6
CompactCells971To1002.Box981.QInter val	CompactCells971To1002.lean:6660- 6662	8561a0a735df59f771163c6df11cbfb5f10 2e79739473435064f6d8867762131
CompactCells971To1002.Box981.lambd Interval	CompactCells971To1002.lean:6663- 6665	337d32c0be5ddf8c52a11705ba1a723fa8a b1047b42b1bf227046f86cd6c900d
CompactCells971To1002.Box981.compac tCandidate_not_isWeightedPerimeterM inimizer	CompactCells971To1002.lean:7312- 7323	90cd1fb3288c49fa202ec5b26e8da7f420a 39bad62b2120f4b0fe50b0134b0b4
CompactCells971To1002.Box982.QInter val	CompactCells971To1002.lean:7324- 7326	050ff192720c314c1be8222551936c557be c8f2767c08ab6d0489e93f13ee655
CompactCells971To1002.Box982.lambd Interval	CompactCells971To1002.lean:7327- 7329	09dec889acbdd86611d5f3ca8843f95d17e fbe871bc7e092195a17bd66fb9fec

Declaration	Exact source	SHA-256
CompactCells971To1002.Box982.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:7976-7987	b37c61eaf742bb194d4c1728946ee36db5a6effda8835adbc5f84c6c98db258a
CompactCells971To1002.Box983.QInterval	CompactCells971To1002.lean:7988-7990	0b4aba11b4674bc44c815b2824e7118b3d0039da57d557131117796086949987
CompactCells971To1002.Box983.lambdaInterval	CompactCells971To1002.lean:7991-7993	a0dfe45d261ac0ed5331595ccd4412bc0bd805ece44fbc02efdfcdcf5591c2e
CompactCells971To1002.Box983.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:8640-8651	da4590fa2bb38fdb9bd5891b43f394d180a024f6ad3974735d6ad7233c7dbdb2
CompactCells971To1002.Box984.QInterval	CompactCells971To1002.lean:8652-8654	f19abe0731c614be929984583bee621338584dab39b930f4acbf2100d9f61000
CompactCells971To1002.Box984.lambdaInterval	CompactCells971To1002.lean:8655-8657	1563449729c9eedd420413dbb6d24599906d9ad197c2eacffe4079faeb459c19
CompactCells971To1002.Box984.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:9304-9315	bef4194ddb910fe0c2be39189c98184c7397fa858b2f31f20b8d17eb39005760
CompactCells971To1002.Box985.QInterval	CompactCells971To1002.lean:9316-9318	a23b8526641ac8330967f5b388429beb559b292f9ddfe4721b2f15dbf86a77c2
CompactCells971To1002.Box985.lambdaInterval	CompactCells971To1002.lean:9319-9321	d06387a55e3afeb1db5c5ac354360c4a68c7e60a0b0fc6a983a2905b0a9fa9a0
CompactCells971To1002.Box985.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:9968-9979	b63c67d2aa8264d40439fa3780a2ddaefd6dda0488ad278051b0f5c41858662b
CompactCells971To1002.Box986.QInterval	CompactCells971To1002.lean:9980-9982	2d4c69eb541f0c1585f74a8ed7fff83349aab99c262481675296e25f4599f783
CompactCells971To1002.Box986.lambdaInterval	CompactCells971To1002.lean:9983-9985	aa4f8ff3405a54e48336d55c7efe862ab34f93512fb8154ee300965aa1f02dbf
CompactCells971To1002.Box986.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:10632-10643	eea06aa43ce2e1bf72a53f425246c1766ab76789e673ba038d53d98dc37c2a47
CompactCells971To1002.Box987.QInterval	CompactCells971To1002.lean:10644-10646	17e62a2489cc568dd63340b430445f18034cb70a6469bef19a3c053ef2d43862
CompactCells971To1002.Box987.lambdaInterval	CompactCells971To1002.lean:10647-10649	c1475c81450568cab343f232bce2903aa8fc5cc21e3d7bb20ee11a01f29b4310
CompactCells971To1002.Box987.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:11296-11307	df1df9accdc2ca426dbc0a855457aa0b9e083ac59e1a8e5dc0e8b1afeb1f45a8
CompactCells971To1002.Box988.QInterval	CompactCells971To1002.lean:11308-11310	9495f2eac2ffbeaffac27f5f14cad04ec9acafb483052e1438bf720e338c2557
CompactCells971To1002.Box988.lambdaInterval	CompactCells971To1002.lean:11311-11313	00b17bb03298c615ef588f9ade7e87482b7df941c54f750db61b5589c54fbeb0
CompactCells971To1002.Box988.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:11960-11971	4cf68ebaec74889993b0163ec9366bbd3b183f420e1fdaf68ecb1505f1afc9c7
CompactCells971To1002.Box989.QInterval	CompactCells971To1002.lean:11972-11974	42fe43ee65a8c6c15677003b90152b84990dfd5e4640f435b2ea7efff8236f11
CompactCells971To1002.Box989.lambdaInterval	CompactCells971To1002.lean:11975-11977	b34e868ecce2dc7eb60239c2681f686c0a55e31ab11a8b3e75939beb2f16a390
CompactCells971To1002.Box989.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:12624-12635	89c072b87d86d260114abc8d3e197fc44aa346388d1e8882dce481b3c255438b

Declaration	Exact source	SHA-256
CompactCells971To1002.Box990.QInterval	CompactCells971To1002.lean:12636-12638	07247b410490db6a057d000f3fe0838725deb12b72c4cd11431746b2df7d8ed3
CompactCells971To1002.Box990.lambdaiInterval	CompactCells971To1002.lean:12639-12641	4b363cf47197213dc212dc1d518958ffe1e0e90e6a65776ea32dc27af5e90bd4
CompactCells971To1002.Box990.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:13288-13299	48e8b62b068e364ac8a7bb3ea454b72caa8e20832afd54019505243b137dac7a
CompactCells971To1002.Box991.QInterval	CompactCells971To1002.lean:13300-13302	2ceec609a6deb07e9e673a27af6f758e534a7b4d8e57e45cce44c7759071473a
CompactCells971To1002.Box991.lambdaiInterval	CompactCells971To1002.lean:13303-13305	d1af12d7bad7c1808aeb632a3d5d95007fe a872d9bb57e929ee381e76e03c1b6
CompactCells971To1002.Box991.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:13952-13963	8fd720216d6774ce21beb8a3a75c6d735645eccc13ea834ef2e20c42816dff81
CompactCells971To1002.Box992.QInterval	CompactCells971To1002.lean:13964-13966	1c01c4dc856d249300c927ceb21c0d8e42cf93185d99529e32cde02448d80a4f
CompactCells971To1002.Box992.lambdaiInterval	CompactCells971To1002.lean:13967-13969	7a3920dcc082fff02504bd72759de5391c3935a45972faf1377ca728a45fbe6f
CompactCells971To1002.Box992.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:14616-14627	aaf2e8cab5e6d918c9956d5d80d07e7bb4aecf98c74a850f2919e89e6d0ec40b
CompactCells971To1002.Box993.QInterval	CompactCells971To1002.lean:14628-14630	9c102773e53ea1fe6d01bfc3158ab130bff084d3df84438eb0f1e6681fe2d49d
CompactCells971To1002.Box993.lambdaiInterval	CompactCells971To1002.lean:14631-14633	9fd4d2f3cdae1bdfd79846ea2ff4bf631cc5653351ccd752cc83c2494fdb8d74f
CompactCells971To1002.Box993.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:15280-15291	5071fbd44b5c6c4f4d0f985ce2d852c2e5f be63b86a6fa3b17686024144987f8
CompactCells971To1002.Box994.QInterval	CompactCells971To1002.lean:15292-15294	0ca634bab30b7c7f93426d175e27ee344b5ef837b478037da3a050d767675846
CompactCells971To1002.Box994.lambdaiInterval	CompactCells971To1002.lean:15295-15297	56111e600904b0700f2141e7440c8a118862aa90e3e09d80118252e64e7f613c
CompactCells971To1002.Box994.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:15944-15955	4b2a3d8171f514bfdeaa30287a7f1631a7e55bccaed20b656cee4eac5448b874
CompactCells971To1002.Box995.QInterval	CompactCells971To1002.lean:15956-15958	4ade3f22e47219884fcbc1c5a167f8fbf7a418edd840b83443de76688d6aa563
CompactCells971To1002.Box995.lambdaiInterval	CompactCells971To1002.lean:15959-15961	84250a50f6b4eb3d42ac07bbc59eded7d44bc84e5564706f6d193dee3a34b275
CompactCells971To1002.Box995.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:16608-16619	966c4d3f28f99be4c2ae68d12bf74948d9d9dc3372b5c1cdf5ebe98037cadc8e
CompactCells971To1002.Box996.QInterval	CompactCells971To1002.lean:16620-16622	be7e2003ee52330c4e448cda32ce8ae97ff64d1cf1dc0509b61583a4f326ffc9
CompactCells971To1002.Box996.lambdaiInterval	CompactCells971To1002.lean:16623-16625	31c12ca87905e0d762b1cabf68df4cdf36afa903fe45bd7ebb4075b852ba5ccf
CompactCells971To1002.Box996.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:17272-17283	f988c508e1ab8702b9a9d4178993f77596350d7f7f3b521eb298d945d340ef59
CompactCells971To1002.Box997.QInterval	CompactCells971To1002.lean:17284-17286	37958bf6eeea5cf1eac561eb9d9c78a1fff26a59a0e75ac2ba1667e704564de38
CompactCells971To1002.Box997.lambdaiInterval	CompactCells971To1002.lean:17287-17289	ce4794cf31e6777aa554c8c6f67a2e2c4681519149a196d1d1874e2fa51e2479

Declaration	Exact source	SHA-256
CompactCells971To1002.Box997.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:17936-17947	779f545d991a9921a4d141fad5af8a27a650f15eb33d3c51b50b179845b34757
CompactCells971To1002.Box998.QInterval	CompactCells971To1002.lean:17948-17950	94b1a31340830ca038838cf39b649eff96f8ea7a77ae9d62ebf0166a362ff811
CompactCells971To1002.Box998.lambdaInterval	CompactCells971To1002.lean:17951-17953	55a2aad66437d8d53723e5f8ac83bd9e479421a5dbda898df33080f87c67e75e
CompactCells971To1002.Box998.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:18600-18611	addc1922bfe832e90b48b972e75a17a7c86e42f33cf901543273d8a857493fcf
CompactCells971To1002.Box999.QInterval	CompactCells971To1002.lean:18612-18614	ee4fd86044a0e109e594df26c339dd7e700297b1bb14d8b2b36e3947d6085263
CompactCells971To1002.Box999.lambdaInterval	CompactCells971To1002.lean:18615-18617	db541f7d1ea4c2043e6e6cf4e146f846bc6ad3acf409205be9956b165f0a16b
CompactCells971To1002.Box999.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:19264-19275	b1b79732342f6f8d7c717d01d545de2505fd333cc424e2bb0e456e1ab7a710d5
CompactCells971To1002.Box1000.QInterval	CompactCells971To1002.lean:19276-19278	5c9b403518856be5fa968df392fb27236807355eafcc642f05ec8d50af0787f1
CompactCells971To1002.Box1000.lambdaInterval	CompactCells971To1002.lean:19279-19281	b0267289acb827f6c3c15f55346cc603b095c3505104719b2da21e03db7ab4b5
CompactCells971To1002.Box1000.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:19928-19939	5f04bf2359addb365b97378c307ed5238a296df2ec6da2652fb7699bbc4f3a68
CompactCells971To1002.Box1001.QInterval	CompactCells971To1002.lean:19940-19942	92a3ddce95258445948d478bde8b43715586d390d3f64cc9d082e83d90cad013
CompactCells971To1002.Box1001.lambdaInterval	CompactCells971To1002.lean:19943-19945	59d3c1caf14f47535685f495fc2436b4b958683976cab889ecb313ce09ab6320
CompactCells971To1002.Box1001.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:20592-20603	838dff7d593286f068954e6a71d40293d304777cc2d30d3e54491b55f5bc2b8b
CompactCells971To1002.Box1002.QInterval	CompactCells971To1002.lean:20604-20606	56f6bf61216e8ed6339ac911af6291a81dc5e6751715de0c2424cc1f2da8f72d
CompactCells971To1002.Box1002.lambdaInterval	CompactCells971To1002.lean:20607-20609	5be8db7c10e9512edb133db0480af0803fb5fd60978b454a9ac3973078f3cd68
CompactCells971To1002.Box1002.compactCandidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:21256-21267	6c6e6244b7f8187eb1689d1866e69fa29d1695051ac7dbcb92becfc1fc7b11a9
CompactCells971To1002.boxes971To1002_candidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:21268-21498	c05223a8d6bbcb47aea3d4e5a213a08f37849f2c2b7fdd88a452aafe00283a2d
CompactCells971To1002.boxes0To1002_candidate_not_isWeightedPerimeterMinimizer	CompactCells971To1002.lean:21499-21512	38783d955697a0e6460ee6c7af025c6d1a3c7dea04ada888cedc4de8a629d94d

Representative full declarations

CompactCells1003To1023.Box1003.lambdaInterval

```
def lambdaInterval : QInterval := ⟨293715 / 229376, 73443 / 57344, by norm_num⟩

/-- Type-(iv) stationary-root slab of box 1003. -/
```

CompactCells1003To1023.Box1003.compactCandidate_not_isWeightedPerimeterMinimizer

```
theorem compactCandidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ} (hlam : lambdaInterval.RealContains lam)
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer :=
  checkedConditions.candidate_not_isWeightedPerimeterMinimizer
  hlam candidate hcandidate

end Box1003

namespace Box1004
```

CompactCells1003To1023.Box1023.compactCandidate_not_isWeightedPerimeterMinimizer

```
theorem compactCandidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ} (hlam : lambdaInterval.RealContains lam)
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer :=
  checkedConditions.candidate_not_isWeightedPerimeterMinimizer
  hlam candidate hcandidate

end Box1023

/-- Candidate-facing exclusion on the closed union of zero-based boxes
1003 through 1023 across every exact seam. -/
```

CompactCells1003To1023.bboxes0To1023_candidate_not_isWeightedPerimeterMinimizer

```
theorem bboxes0To1023_candidate_not_isWeightedPerimeterMinimizer
  {lam : ℝ}
  (hlower : (33 / 32 : ℝ) ≤ lam)
  (hupper : lam ≤ (9 / 7 : ℝ))
  (candidate : _root_.FourArcCandidate lam)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  by_cases hprefix : lam ≤ (293715 / 229376 : ℝ)
  · exact CompactCells971To1002.bboxes0To1002_candidate_not_isWeightedPerimeterMinimizer
    hlower hprefix candidate hcandidate
  · exact boxes1003To1023_candidate_not_isWeightedPerimeterMinimizer
    (not_le.mp hprefix).le hupper candidate hcandidate

end CompactCells1003To1023
```

MiddleFaceCell[0-9]*.lean

Confirmed as the configured middle-face family. The 113 rational bricks, numbered 0 through 112, cover [102001/100000,25781/25000]. Each brick's checked angular branches, opposite-face signs, negative descending-fold numerator, and negative perimeter residual produce an exact stationary pair through the reusable middle-face assembly and hence a candidate-facing exclusion theorem. Nongenerated batch modules establish the interval coverage used between the two prototype cells. Representatives include the first and final brick and their exported candidate boundaries.

This family contains **452** declarations. Every body is retained verbatim under `supporting-materials/lean/`; the table below is exhaustive.

Declaration	Exact source	SHA-256
MiddleFaceCell0.Brick	MiddleFaceCell0.lean:20-22	927e7b2d26353aa0f6b5291a55da180383cdca4c1a118bd1f2c2be6131ea029e
MiddleFaceCell0.brick	MiddleFaceCell0.lean:23-39	6826922bc65ff716ea9f2f5179745288974bf844804e2d773c508e15617304c9

Declaration	Exact source	SHA-256
MiddleFaceCell0.lamInterval	MiddleFaceCell0.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell0.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell0.lean:2633-2653	c186745e2baa4f5a1e70e59dbcdeb62622e 696b9a3b2aa4cb9d30703f720f43c
MiddleFaceCell1.Brick	MiddleFaceCell1.lean:20-22	37064f1906aa6a9d0382f4665218a3044d4 d6522e77c02dc1748a2643f500442
MiddleFaceCell1.brick	MiddleFaceCell1.lean:23-39	74b9198c97ef1af9e41670b35587391ece7 0b5c463f069a6305d2686f97177dd
MiddleFaceCell1.lamInterval	MiddleFaceCell1.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell1.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell1.lean:1888-1907	cbd80e0421530d63f847a7447cca8727b99 a1484929a8487f1581662fac379d1
MiddleFaceCell10.Brick	MiddleFaceCell10.lean:20-22	dfb2d9943dec214e92a4f54ca07b8ea4b31 3c45b3c52a4b1e1a1ed5322ca36ef
MiddleFaceCell10.brick	MiddleFaceCell10.lean:23-39	875797fb1eb5a25b516cc242ad6452f8dc0 d9ffd04c37c93b8ae744dc331eb80
MiddleFaceCell10.lamInterval	MiddleFaceCell10.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell10.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell10.lean:1889-1907	85428bd03e21ea81d78e51732ed52f91725 65ff417edcc184f9cd3e8079033f7
MiddleFaceCell100.Brick	MiddleFaceCell100.lean:20-22	a8171f87dee8e25cbb24a75f6def9d25dbe def73d4381471a3e5f96b83099bdb
MiddleFaceCell100.brick	MiddleFaceCell100.lean:23-39	dd7a2d1a2b25dce173aaf5738da4a042b57 c7f4eefc8988166f7e09b5a45fed5
MiddleFaceCell100.lamInterval	MiddleFaceCell100.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell100.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell100.lean:1892-1910	c939fbdf952b2a5b36c2046e60ce73214cb 624656ddc380aeed15ef70028
MiddleFaceCell101.Brick	MiddleFaceCell101.lean:20-22	a5e1a929882a03e8fa552c7b425f51e2298 13864b71386437fbf0b7091cca9a8
MiddleFaceCell101.brick	MiddleFaceCell101.lean:23-39	3238c37297b6232bc1e903b223e435821fb 12195fb23e61a7382337ff7a5a1f7
MiddleFaceCell101.lamInterval	MiddleFaceCell101.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell101.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell101.lean:1892-1910	3c99b634f26ee22bb60a6f4407b3c5485cf 22924ab3c86b7c66e9592cf66cae6
MiddleFaceCell102.Brick	MiddleFaceCell102.lean:20-22	7bd71285ffdc2dd9c94e6b306d9064d0494 5740418b700069df9e24de8504ed7
MiddleFaceCell102.brick	MiddleFaceCell102.lean:23-39	6421cbee537a9e9d50cc381bd9bf7661a63 1e8b8ed2eeadceb02c698b199f898
MiddleFaceCell102.lamInterval	MiddleFaceCell102.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell102.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell102.lean:1892-1910	bb704f431ef2c10d4c3dc586f8847f50d08 85b82298b08083e9ce862eb573268
MiddleFaceCell103.Brick	MiddleFaceCell103.lean:20-22	92b99d25c250dfae849fda7d9d4545daa3e 043a47bfe4784c239c4e0ac67c341
MiddleFaceCell103.brick	MiddleFaceCell103.lean:23-39	929f857e44917331990f739d186c69005bc b3c5567605331a2fe5e6012c791a3
MiddleFaceCell103.lamInterval	MiddleFaceCell103.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell103.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell103.lean:1892-1910	01f97e8fc75e086938b09d68722fb18acad 663b37d518995b4e4d9fb448b1271

Declaration	Exact source	SHA-256
MiddleFaceCell104.Brick	MiddleFaceCell104.lean:20-22	07b8a51ca0ec9a8d73c6241e43546f23bdbd26da8f3714ae0113574663693611
MiddleFaceCell104.brick	MiddleFaceCell104.lean:23-39	4317f9d345c970d16c5bf7aad1819eeb62321b686692a83ab0eaa63600fba26c
MiddleFaceCell104.lamInterval	MiddleFaceCell104.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell104.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell104.lean:1892-1910	cbb180cb50850c25226cf7c1f9b089a8f2fe005f3b8d5ba28ec09293fda25e1
MiddleFaceCell105.Brick	MiddleFaceCell105.lean:20-22	90bf41a259cdb75de53fb0bebe41f5bdb2538e068e70568bba7e18438ba0a7ff
MiddleFaceCell105.brick	MiddleFaceCell105.lean:23-39	d34ddb8552e287fe2496e9457a6c103e944ffac22b1a2903ccbb910ac0f1eb3d
MiddleFaceCell105.lamInterval	MiddleFaceCell105.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell105.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell105.lean:1892-1910	3855bede4d6c20c5fa449fd1225215d82c66aabf5dc2c518077388835b97275
MiddleFaceCell106.Brick	MiddleFaceCell106.lean:20-22	43657c08113da4763a323445e0a8d23e9cab2995c04ca900b53e38cbb68cf0d
MiddleFaceCell106.brick	MiddleFaceCell106.lean:23-39	dcdce058b9e3fe7f3b9f273bcbe70dbabd4a36d2d60e42959b7eb682d4ba3538
MiddleFaceCell106.lamInterval	MiddleFaceCell106.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell106.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell106.lean:1892-1910	c4323cd7dcb81dd23bbb13e7a6da6db369314dc092c81930c824f1c12bf05f0
MiddleFaceCell107.Brick	MiddleFaceCell107.lean:20-22	fbb97f7a9d17ed29f207a2bc58f493e65b7756f82f628a6504ff0176944fd563
MiddleFaceCell107.brick	MiddleFaceCell107.lean:23-39	d79e9086a17cc0f1e5ba7ce1e0aeca7cfdd288120b9ccd9ac07eedd01db6a1ed
MiddleFaceCell107.lamInterval	MiddleFaceCell107.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell107.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell107.lean:1892-1910	f3c170ddcf05f71486c4a7d455025f384a3d60752b417e48d9a09a1c7501bae
MiddleFaceCell108.Brick	MiddleFaceCell108.lean:20-22	7a12777f272bea6b574c86aca5e0e027b839357478e71accad6ce6ecc655ed66
MiddleFaceCell108.brick	MiddleFaceCell108.lean:23-39	dd865f669b8c5fccaafeb5146a05d4166797f26e8683d15f71fa7e3972bbddf6
MiddleFaceCell108.lamInterval	MiddleFaceCell108.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell108.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell108.lean:1892-1910	e2399880ca42d06b51edebcdcc1162a133006306e9370f7c007b070be55e30f5
MiddleFaceCell109.Brick	MiddleFaceCell109.lean:20-22	eff349c87e09b79fb97fc3a49f54b7478d67e316bf96fbf8c5451dca7fcb1400
MiddleFaceCell109.brick	MiddleFaceCell109.lean:23-39	8e34bdd669070a698fa44b2d021dfebd9bd639a51e391cb53bd55912d658200f
MiddleFaceCell109.lamInterval	MiddleFaceCell109.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell109.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell109.lean:1892-1910	875f3a258995fcc9566c1c6752f674ac21c416e158f0c088b0726d5313d63ef1
MiddleFaceCell111.Brick	MiddleFaceCell111.lean:20-22	422f1140c77856bb80f87f2a668e15a32e1496e36e3c5d80735b28ca996bf036
MiddleFaceCell111.brick	MiddleFaceCell111.lean:23-39	8d4d270ab7e7beff9107c59c176795ebf6b29af4cd78ed650d9e95debee763ce

Declaration	Exact source	SHA-256
MiddleFaceCell11.lamInterval	MiddleFaceCell11.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell11.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell11.lean:1889-1907	bba989a84f2e693a08430344e5d59fcf650 f736d817781053f3bcfba22f6f409
MiddleFaceCell110.Brick	MiddleFaceCell110.lean:20-22	364e5651eb375637678c7401201c01287e6 0f9480217622bab46937282a3a73f
MiddleFaceCell110.brick	MiddleFaceCell110.lean:23-39	2b973783bf5f5189781c989d58a5453d575 ed1c5bf9732f9cf08e5db07628f77
MiddleFaceCell110.lamInterval	MiddleFaceCell110.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell110.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell110.lean:1892-1910	349927598530c27e4d8f9890c5881cf9ee7 e149fa20350a9a24837a42cc3f6cd
MiddleFaceCell111.Brick	MiddleFaceCell111.lean:20-22	a0572ab587465cbd7939ef4bc9a5812128d 7791b2582595b0979307269e8e2f4
MiddleFaceCell111.brick	MiddleFaceCell111.lean:23-39	170ec15586f29cd18b465787aa85613f84c 4f0cca1d8bab0ddeb17ddfb29d250
MiddleFaceCell111.lamInterval	MiddleFaceCell111.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell111.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell111.lean:1892-1910	fc0fdb0e6d11b157a69766d9e95c0dde52d 140ec249b874e8fb65a9f0d6cd579
MiddleFaceCell112.Brick	MiddleFaceCell112.lean:20-22	1cefd019326f37bcd23c4746a1f17260594 ae1bd43fa54e77bbc59f229fb244c
MiddleFaceCell112.brick	MiddleFaceCell112.lean:23-39	13fb7924d17e89d141f66fb85a48c1b9a66 78969b8b75487fa8596207f305a5d
MiddleFaceCell112.lamInterval	MiddleFaceCell112.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell112.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell112.lean:1900-1918	9d838a95d2ecd15a5c3a7b7579ec34eee2f 0903e2a1cf65b4baf655d19534b4
MiddleFaceCell112.Brick	MiddleFaceCell112.lean:20-22	20f5a3ac6954cc2a35235d046bc1afd1741 df16e34eb1f0043b84128fb5fd7be
MiddleFaceCell112.brick	MiddleFaceCell112.lean:23-39	645b0c50d437bc380f3e2db27b2241f8471 29518715cf148baad24cab7075a6
MiddleFaceCell112.lamInterval	MiddleFaceCell112.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell112.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell112.lean:1889-1907	1261c9b50b52efdb6406a4fc5ca1c13bf40 bba1520cfe0b4412b490d6c6698c9
MiddleFaceCell113.Brick	MiddleFaceCell113.lean:20-22	01ecd47a9f84702695d2ab20463f5f6627b 03642a407bd1be1654cc40cec7a27
MiddleFaceCell113.brick	MiddleFaceCell113.lean:23-39	e068d913393aca117e9e68d3e1724ff0085 8fe7a3ae39cbd46590a079b9d6e60
MiddleFaceCell113.lamInterval	MiddleFaceCell113.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell113.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell113.lean:1889-1907	63dcae663f848cca6593ae1ba84ba47242 2d8456046cf8a86d039eee3cbce501
MiddleFaceCell114.Brick	MiddleFaceCell114.lean:20-22	4b59b7712d40357b392a8bc1c9a38f091c5 e88101e52319e23414ab54774979f
MiddleFaceCell114.brick	MiddleFaceCell114.lean:23-39	3de956ca214dfd5cccd0799485e19d7fe9e 92a3a639eb0adf86181891f74ff5d
MiddleFaceCell114.lamInterval	MiddleFaceCell114.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell114.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell114.lean:1889-1907	5d72385bd489b20beb035d250ff87892f0c 53643833a6171d4986d04a38bfde8

Declaration	Exact source	SHA-256
MiddleFaceCell15.Brick	MiddleFaceCell15.lean:20-22	ab0196e6107213171313caebf38aabcc031fe291d6b066d8fff4d4fc5bf05994
MiddleFaceCell15.brick	MiddleFaceCell15.lean:23-39	2034a2b13d7fff9a7fccf162e2c764fcc76926eb4e3793a46fcc5c0d415d65b
MiddleFaceCell15.lamInterval	MiddleFaceCell15.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell15.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell15.lean:1889-1907	1ca64a20a3b67b4ace3477b29aea5b919693419135610bb1e9fbd03ba3faf125
MiddleFaceCell16.Brick	MiddleFaceCell16.lean:20-22	ad4fc4e603a493f6adec4e71203eb84c0cc86d059ad4e496d5b16eb28e307178
MiddleFaceCell16.brick	MiddleFaceCell16.lean:23-39	86e79a68a77907460d5ac610273fe45b85c112f814f81be4b71fb84d721a548
MiddleFaceCell16.lamInterval	MiddleFaceCell16.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell16.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell16.lean:1889-1907	fcaceeb4a8a1b27254a145e54ffe85d4902383a36e2630bf100580f45c2473b6
MiddleFaceCell17.Brick	MiddleFaceCell17.lean:20-22	c0d813432e1fe40bffea0ab691cbd1ae0295160b22e492478dff5757390b33c4
MiddleFaceCell17.brick	MiddleFaceCell17.lean:23-39	ad60b7a243e71d3581c0de53832c27f60174ec98af830acc003d60666d558d99
MiddleFaceCell17.lamInterval	MiddleFaceCell17.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell17.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell17.lean:1889-1907	74f39636ed37da4a1c62c28cd1ae81d47d3e8c392e547791369a33627eafdb6e
MiddleFaceCell18.Brick	MiddleFaceCell18.lean:20-22	d1f79827df04ecff873ff6923fbde15cbe8690196354c628b60f23899c657573
MiddleFaceCell18.brick	MiddleFaceCell18.lean:23-39	9dc58f5b65bee40fc92533a497e8a65e558a6c35b50d3bd4244b2310fe605ae1
MiddleFaceCell18.lamInterval	MiddleFaceCell18.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell18.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell18.lean:1892-1910	310e9dad68db89ce9f7ad60a1883d1e073f172a1f657a4829ff1d35a50509358
MiddleFaceCell19.Brick	MiddleFaceCell19.lean:20-22	79bd62fd038bb44e1184712d2033563246bba03597c4586af6df9b3a5edf5c5d
MiddleFaceCell19.brick	MiddleFaceCell19.lean:23-39	2d1dfe4ac74d7072ae8c067afa29ec8a6f115fcdccc6ad9f53d441fc81f8b986
MiddleFaceCell19.lamInterval	MiddleFaceCell19.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell19.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell19.lean:1892-1910	a8d4c101ed34f7defea68b7756a9f1130822565b72fc50a30faa5428924c899e
MiddleFaceCell12.Brick	MiddleFaceCell12.lean:20-22	011a73b220738cb17074df2b18f45ebe2ab8d59d1e0642e2f61e8ec4bbff55b2
MiddleFaceCell12.brick	MiddleFaceCell12.lean:23-39	a170dc2fe5cef8659c28f7bac53ffa921a3bed2ce822770eda493d548da6b184
MiddleFaceCell12.lamInterval	MiddleFaceCell12.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell12.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell12.lean:1889-1907	2c79e479b97dcf55c49fc1e2183763dc5eccffa0759c249045ed8f55fd8e71f
MiddleFaceCell120.Brick	MiddleFaceCell120.lean:20-22	049a6f985b14968233b76aa3dc3241eb50ddcc4ea443d1d737d573ab3af0bb77
MiddleFaceCell120.brick	MiddleFaceCell120.lean:23-39	47b194ac9325aeb229e86dc2bd50fe3f24c32e4887600506f152c39f177206b4

Declaration	Exact source	SHA-256
MiddleFaceCell20.lamInterval	MiddleFaceCell20.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell20.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell20.lean:1892-1910	4b2bd45881d855e5f66887d56eadc7166d8 461cc0983298a6496b6551cbc60c6
MiddleFaceCell21.Brick	MiddleFaceCell21.lean:20-22	90ebb1927a9afa9794678bd9c0887ab198a c9eeb40af36e3e80e62c809eb2378
MiddleFaceCell21.brick	MiddleFaceCell21.lean:23-39	7add5151efd595536d0e2f9d261dc6c7498 77ed7f719bb8cf07206afca4f78ea
MiddleFaceCell21.lamInterval	MiddleFaceCell21.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell21.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell21.lean:1892-1910	dcbb0c95c3ff640f798d4a4548b38fe2348 1688b19b2d2590794d3903e030c0d
MiddleFaceCell22.Brick	MiddleFaceCell22.lean:20-22	4999b65af88e2a6bb34c6e2e5b81abebb41 e5a58d487e76595642b47be5fef4e
MiddleFaceCell22.brick	MiddleFaceCell22.lean:23-39	b6794f19d2ad69e4db918a47ef4af5a85b1 103a9174a147d14629e2874ef00c5
MiddleFaceCell22.lamInterval	MiddleFaceCell22.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell22.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell22.lean:1892-1910	3f2eac28da23af7125633d0fc0818813a9b a150dc472fb62c664dc4557c9c6cc
MiddleFaceCell23.Brick	MiddleFaceCell23.lean:20-22	f16d4ecaa36b559fa5cc11e83d02bc4e9c0 6bec9df35f1be22d168d58d4a2e16
MiddleFaceCell23.brick	MiddleFaceCell23.lean:23-39	c93b38920cf23da2708aa0b11c82efb6b49 3353fe89ceeaf62b590517851184
MiddleFaceCell23.lamInterval	MiddleFaceCell23.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell23.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell23.lean:1892-1910	2c9ae9591b3f94474f5ce07961806ab51a5 397725e0f4903e1c8b40dd0fcb6b9
MiddleFaceCell24.Brick	MiddleFaceCell24.lean:20-22	5a8c83462766a16f507824d8f0cd14a5ba4 eedbe88c68797cc0694632c17754d
MiddleFaceCell24.brick	MiddleFaceCell24.lean:23-39	72da8f7d24ba964179fc11b84b13e8fffd08 75de498efc2239f897b1e85c97e00
MiddleFaceCell24.lamInterval	MiddleFaceCell24.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell24.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell24.lean:1892-1910	7993f31b20f5091a307bccc814d02ea4a8e f25fa04ff8f6ba5c6fedd5be99f7c
MiddleFaceCell25.Brick	MiddleFaceCell25.lean:20-22	08591daa72f0dfa9205ac13c3b19e046ffa 0dc133c96da376267ef83b9fa56d6
MiddleFaceCell25.brick	MiddleFaceCell25.lean:23-39	df943767dcac7319b08845e3e7174c9d315 d0e4da6e7ed51e929fb0fd2e40e4a
MiddleFaceCell25.lamInterval	MiddleFaceCell25.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell25.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell25.lean:1892-1910	fd0c9e7c06fe3db2a20f03a2f551b97d302 5cf5b074a07647cb9e45d584ca311
MiddleFaceCell26.Brick	MiddleFaceCell26.lean:20-22	cfae21b626c40d28ba07b5dfce0fb6a3d1b d70c2558e86462a71f450a8b3c111
MiddleFaceCell26.brick	MiddleFaceCell26.lean:23-39	25d98e6750e3cb0533116592870fb40841 e9b7905885f36b17788a2312a703b
MiddleFaceCell26.lamInterval	MiddleFaceCell26.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell26.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell26.lean:1892-1910	2189db9af68806069e8a36da892f06b9844 88c240770a6000e024c02e514184a

Declaration	Exact source	SHA-256
MiddleFaceCell27.Brick	MiddleFaceCell27.lean:20-22	045e2e7fcb3bcf6b90169d7b4491e0733e48c078f4db9b95c13492e5f0a49ac7
MiddleFaceCell27.brick	MiddleFaceCell27.lean:23-39	70e0712d29560bc034491be4133c1efb3e4a4e900e1f7d4041e104cc136b0bd6
MiddleFaceCell27.lamInterval	MiddleFaceCell27.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell27.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell27.lean:1892-1910	d103527cd78c17fbd3bb014125faf1ed069668dc574f0ef1713856da0836c2ac
MiddleFaceCell28.Brick	MiddleFaceCell28.lean:20-22	47425b6995b04404bc0f959aaf7a0aa326c6d60e63cb2020fd96dbbd113e3a70
MiddleFaceCell28.brick	MiddleFaceCell28.lean:23-39	8d0fa6a129fb0893c39672d3c88502f1fd433aced985e8b720a6d61860e3896
MiddleFaceCell28.lamInterval	MiddleFaceCell28.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell28.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell28.lean:1892-1910	f6850a423fa365710137e49ede10fec12de1e3e7a507c28a121cc2db107a427d
MiddleFaceCell29.Brick	MiddleFaceCell29.lean:20-22	730baff5b0ee3434e50b9971440701aeb0f604f541f7ff59d7d24dbf151d199c
MiddleFaceCell29.brick	MiddleFaceCell29.lean:23-39	15f28419af91b0a5276012e31d35d7b2257516a8467659235fec0dc68aebcc
MiddleFaceCell29.lamInterval	MiddleFaceCell29.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell29.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell29.lean:1892-1910	4ba81e0957f1e1644ab8c0e9eeeb5577af35fcf820abf1328a44cd5c188baf60
MiddleFaceCell3.Brick	MiddleFaceCell3.lean:20-22	3c982b71c68a0f3d59d23f747d50174f73c4e56a59a9d8d627e7ec9582237f13
MiddleFaceCell3.brick	MiddleFaceCell3.lean:23-39	7e5f321d57734c4d54642b4c9a74e698411a75699495ce0d1322082caabf79b7
MiddleFaceCell3.lamInterval	MiddleFaceCell3.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell3.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell3.lean:1889-1907	bdc1fb8217b716abe1637d56c691debde3c9e0a58ac781fd83f70193653367de
MiddleFaceCell30.Brick	MiddleFaceCell30.lean:20-22	34c60a0a002d690e3908aa790ca9e292c86dbf37022bb7411e97d74b4b37c113
MiddleFaceCell30.brick	MiddleFaceCell30.lean:23-39	fb3e2254ed16a1230d7bc1d93df46b0c315735872f2c3e42130456a2935a7db4
MiddleFaceCell30.lamInterval	MiddleFaceCell30.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell30.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell30.lean:1892-1910	ace3ab9d3255b54ba8f67b856f01dbecd3259b6aed66ab342ea9a852cfa148bb
MiddleFaceCell31.Brick	MiddleFaceCell31.lean:20-22	22ebbfe17f3adc5b7c39368091701a2353cc132bf6094f9cc6873c4da10940bf
MiddleFaceCell31.brick	MiddleFaceCell31.lean:23-39	75e470ca5d19cdc39769ded4aadbd6daa1cb188ad8446171a7135e4e90ba6762
MiddleFaceCell31.lamInterval	MiddleFaceCell31.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell31.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell31.lean:1892-1910	3f63b972acb1f4943f46a4d8cbcf4cfc06cc82862fea5d7c988258da5bd6581
MiddleFaceCell32.Brick	MiddleFaceCell32.lean:20-22	7b184f7e370273cb649a2d23456c9a9efe9d0d676ca64784298e783f48038484
MiddleFaceCell32.brick	MiddleFaceCell32.lean:23-39	3638094e0418ffed437ca8977e42b6ac4eb44158cca8b82278b4fd8d80b1cedc

Declaration	Exact source	SHA-256
MiddleFaceCell32.lamInterval	MiddleFaceCell32.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell32.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell32.lean:1892-1910	83fa3b157c41d8ebe8daee20ed6cf1979b4038133028e8b39a70c4972d1d1522
MiddleFaceCell33.Brick	MiddleFaceCell33.lean:20-22	e4eb49d5987ca3dbb972f96e24fb122e61fa6da1f5a27858e81577e192df6101
MiddleFaceCell33.brick	MiddleFaceCell33.lean:23-39	1715f8a6f5a6faf227ec0b759e1911d3f458ec5b6c06360342a056c30de4b6a8
MiddleFaceCell33.lamInterval	MiddleFaceCell33.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell33.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell33.lean:1892-1910	993c7ef257d7790e832e8a2d443c53218f112b59b6a87b84387ecf9dfc56b960
MiddleFaceCell34.Brick	MiddleFaceCell34.lean:20-22	600111299cf800a5af5507411d009491aebb343e14c0fa7d359596e57fad5296
MiddleFaceCell34.brick	MiddleFaceCell34.lean:23-39	ebc09de22cda073ec8aae961a1c3c74e4bcd61f837f93d9c30387d35fd78d6d1
MiddleFaceCell34.lamInterval	MiddleFaceCell34.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell34.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell34.lean:1892-1910	ba88fe90e268803835e68ed9cf41af33c2048f5ac7fb4830063e2e511db50768
MiddleFaceCell35.Brick	MiddleFaceCell35.lean:20-22	df71c07137cc5af5c4d9a83aa9856d7ba5a101a0dfe0e21681f944e22c416268
MiddleFaceCell35.brick	MiddleFaceCell35.lean:23-39	9675d1896bfc496d644a8c98209a6242faead44fdd2f05e6188c8c43d555620c
MiddleFaceCell35.lamInterval	MiddleFaceCell35.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell35.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell35.lean:1892-1910	b6da973a892aedf8afc9ed65beafe8c719fa5d43e1acb0777bb083b865a38e3
MiddleFaceCell36.Brick	MiddleFaceCell36.lean:20-22	3e4dfd8587f83c8099531bf55c217014637dc59b3cd5849abdb8cbf2dd4e183c
MiddleFaceCell36.brick	MiddleFaceCell36.lean:23-39	ca7bd4588d8afc48231b0c7548c0529eb064abeefccc154d1488a1c982af279d
MiddleFaceCell36.lamInterval	MiddleFaceCell36.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell36.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell36.lean:1892-1910	fbb4a12a79446178749cf0ab49fac9a34d76fd66beedbb9c4e192a9c87960804
MiddleFaceCell37.Brick	MiddleFaceCell37.lean:20-22	efe1921541ef7d27fbcc7a0ce94a60ddd6cc6b27eea924e91cb9a51dea293ee2
MiddleFaceCell37.brick	MiddleFaceCell37.lean:23-39	80b726a309f53af37c3906c8f094aa67cf1cca606c030c9833542bf26fb020c9
MiddleFaceCell37.lamInterval	MiddleFaceCell37.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell37.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell37.lean:1892-1910	ca2f927cc91214d12c9b048fd17f834624ba8974459b45179a3c52d335484c1c
MiddleFaceCell38.Brick	MiddleFaceCell38.lean:20-22	e98d854f1bd927500ee2821c38b9d65184ab91938777a6ba664c0b696dd04b3d
MiddleFaceCell38.brick	MiddleFaceCell38.lean:23-39	695cb02809bd74ad06040f534f715c882a8a65cada5fb66e07d5ba85e096f09a
MiddleFaceCell38.lamInterval	MiddleFaceCell38.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell38.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell38.lean:1892-1910	98f1e7151b39b252d1496c132272cfaa23201496f712c2358fc7bbf1c699b6f9

Declaration	Exact source	SHA-256
MiddleFaceCell39.Brick	MiddleFaceCell39.lean:20-22	6ce801d52ad8adfa03d7efac7d5f517d3a048c3703428dec26f7d3525d55ac2
MiddleFaceCell39.brick	MiddleFaceCell39.lean:23-39	e77f251a6565b4bcf9a5e8987401386d2a526fc1a574a71e95c0e164be6bfd18
MiddleFaceCell39.lamInterval	MiddleFaceCell39.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell39.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell39.lean:1892-1910	11154e3d5d8e2a2adc8664892701753d3a3f49e7538dc6be34ab8ab7f2aed49e
MiddleFaceCell4.Brick	MiddleFaceCell4.lean:20-22	48cd60fb0640964dc38d625c1eb0182c48e15d3595ef7bb0e0c6cebf94ead9f5
MiddleFaceCell4.brick	MiddleFaceCell4.lean:23-39	76cfc2f29f682f43b63aba2f6c4f71dda700f9c2d080e2953d644d5320ba90e2
MiddleFaceCell4.lamInterval	MiddleFaceCell4.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell4.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell4.lean:1889-1907	f580efaf916338aa54ebb6014efe4aaf0309e7918f40d6f23e93cec375dfed58
MiddleFaceCell40.Brick	MiddleFaceCell40.lean:20-22	0d5a5466e6bd582d980c772ee1a718a398bbf3df14c2b8dbb52ee40fbcd0c391
MiddleFaceCell40.brick	MiddleFaceCell40.lean:23-39	dc48ab98a7807fff971491e811b2e69bc49ded91c13867142be584b4d068ac04
MiddleFaceCell40.lamInterval	MiddleFaceCell40.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell40.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell40.lean:1892-1910	abb19e3c8efe82a1547c9dc36773b9af383b97eeefad59e703d13ad54005817e
MiddleFaceCell41.Brick	MiddleFaceCell41.lean:20-22	4bdad4719d39d9a98a7298d2c42560e620e90066535c6b965736c709257d2ddc
MiddleFaceCell41.brick	MiddleFaceCell41.lean:23-39	bb95231040a422364cab97b8c6bad59479e1ff3634f4e116d63725f2b15184fb
MiddleFaceCell41.lamInterval	MiddleFaceCell41.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell41.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell41.lean:1892-1910	5620720074b49e865939ba17ba360a82773ef2dd1bd3828d6ab4ab0c4b7b7e33
MiddleFaceCell42.Brick	MiddleFaceCell42.lean:20-22	29bf5af8895817fcacf29a0a5314d691781eb1ad105d35ab8febee15fa74ca477
MiddleFaceCell42.brick	MiddleFaceCell42.lean:23-39	7a9c56b7db11cad9037c9e7626ba2ff81e5363f7e943efe7b6d744c881bcd2f2
MiddleFaceCell42.lamInterval	MiddleFaceCell42.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell42.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell42.lean:1892-1910	068e43d5aa65631d3e8af92c77aff5f4525b7e9f34c8e11314bf09c062fcb700
MiddleFaceCell43.Brick	MiddleFaceCell43.lean:20-22	c39ab82e10945b205921be5276ec9aa0a8b34c4f46260057c5bda2953be92a50
MiddleFaceCell43.brick	MiddleFaceCell43.lean:23-39	b125d7317a4df21e4cd5c594a4e895e1278257c61ddd0ad91ccd159f49cd6eef
MiddleFaceCell43.lamInterval	MiddleFaceCell43.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell43.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell43.lean:1892-1910	36490d10389560d7d507fe1efe6727b77967ab1e60ade75d23d7302efd157f24
MiddleFaceCell44.Brick	MiddleFaceCell44.lean:20-22	2de2668313ab5c07ad11effca4120357c634750ebf03f33d29261d2c6e798b92
MiddleFaceCell44.brick	MiddleFaceCell44.lean:23-39	5d0ee609c8e4e5264bc03fe7cf6290e3552d0896e304e420ad3ffeea1f6cc80a

Declaration	Exact source	SHA-256
MiddleFaceCell44.lamInterval	MiddleFaceCell44.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell44.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell44.lean:1892-1910	1bdee456e4b24086876480ccc10a47ac35d 9d46ecccc2ccc9478ed3a4a627fc
MiddleFaceCell45.Brick	MiddleFaceCell45.lean:20-22	6a83cd0fd0879451c629b6fc20ff71078cc 27c407b6de8a1fb35fb793a9b775f
MiddleFaceCell45.brick	MiddleFaceCell45.lean:23-39	fb1c811187719487e22860d60a443b4ba65 c5163b553801a2133ec09518995b8
MiddleFaceCell45.lamInterval	MiddleFaceCell45.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell45.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell45.lean:1892-1910	72e61f315f257f14446b71cca19f4a37b00 e18103be742db82982e0c5a0d2ae1
MiddleFaceCell46.Brick	MiddleFaceCell46.lean:20-22	5280a023d19d033c8b19b594c67cc6f9411 9edd398ec4715bd4e513d57e2eb83
MiddleFaceCell46.brick	MiddleFaceCell46.lean:23-39	a8bfc2a85d0c07f49a0b0645539e666572f 88d911bc9baa61ac366332453b0bc
MiddleFaceCell46.lamInterval	MiddleFaceCell46.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell46.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell46.lean:1892-1910	29087dd65e20e2108e1203971e3a6f6ff4f 5df419fce019e29f220d201804173
MiddleFaceCell47.Brick	MiddleFaceCell47.lean:20-22	63f7920068833f41c2dccc5b46afbe3dd7 93a98e71f15b9b00d3d44ea5f6010
MiddleFaceCell47.brick	MiddleFaceCell47.lean:23-39	4ec8e0c7c1133c4007e3b85ea8f90928a9c 621861c061d97139e12a750a731bd
MiddleFaceCell47.lamInterval	MiddleFaceCell47.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell47.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell47.lean:1892-1910	32ffa8c9c70d9af3f8a0781dd43e96d3f1b b8147b1490c7368c8bde57b25bcb7
MiddleFaceCell48.Brick	MiddleFaceCell48.lean:20-22	01366ad92590a79c3eab14cbabd5a8866a 5528e479a608c11d69b0ab46896fb
MiddleFaceCell48.brick	MiddleFaceCell48.lean:23-39	dd8fb665d1f533527d1c42a91b475f6ace0 59911279f1369c02c4db3dae68ad6
MiddleFaceCell48.lamInterval	MiddleFaceCell48.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell48.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell48.lean:1892-1910	0d2f461c89e5827365ea86b81b5d9bcf755 e80325caf1af6a1c73d0eebf2b556
MiddleFaceCell49.Brick	MiddleFaceCell49.lean:20-22	7459c849bdb8249ef9145590e4f25d65016 3ab067eca7b8302e575f09d87fa39
MiddleFaceCell49.brick	MiddleFaceCell49.lean:23-39	3fab3c59bb3e9a2be3ef27fd40976374e5e 8fb78b79ba0c1ee074a333b896be0
MiddleFaceCell49.lamInterval	MiddleFaceCell49.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell49.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell49.lean:1892-1910	76313ad42a6d2185ba5bda9d348e7ecf55 e41db83dbdc3a3c20150467e34045
MiddleFaceCell15.Brick	MiddleFaceCell15.lean:20-22	9ff337826da5e83fda8f5e259812b9a6fb0 d324165fdc97d31c356dde6b37c1b
MiddleFaceCell15.brick	MiddleFaceCell15.lean:23-39	f8f178042e6ea085477d36184ae9edfd4f3 087b36c6818c69791a66ca7604b72
MiddleFaceCell15.lamInterval	MiddleFaceCell15.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell15.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell15.lean:1889-1907	203ecd80ba5622ebae38198c8d5dd3f62dd e91a84f786f85b1d41f673f431b6d

Declaration	Exact source	SHA-256
MiddleFaceCell150.Brick	MiddleFaceCell150.lean:20-22	757fc72813d427c4b786712da7a49ba4e4b2a061f23dedc0f3b4834322b0ae33
MiddleFaceCell150.brick	MiddleFaceCell150.lean:23-39	bb164d35ab272eabda268f960c4ec1d4b3e8d493de5f4104d3a6f423e5037593
MiddleFaceCell150.lamInterval	MiddleFaceCell150.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell150.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell150.lean:1892-1910	a0d1963f0fe12d432ac674dd5ffe6f064ee1ef4e4ae4104e24a5564404f447e5
MiddleFaceCell151.Brick	MiddleFaceCell151.lean:20-22	9384d7973d7f928352271500593bb86d4967fde6523028cc361c872a435d06a1
MiddleFaceCell151.brick	MiddleFaceCell151.lean:23-39	507c6242616e4e0598ef22db33665a8866238274e6be3608d1a3df81da2e2b35
MiddleFaceCell151.lamInterval	MiddleFaceCell151.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell151.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell151.lean:1892-1910	6f2e0bab5951b6f24bf00d086621baabdc85d1be2c07e01023dbb27b6ea7e2a
MiddleFaceCell152.Brick	MiddleFaceCell152.lean:20-22	53cb46e23454e7fdbba2e48835526423f540cee6716663e79c0c5a878ca73c
MiddleFaceCell152.brick	MiddleFaceCell152.lean:23-39	75a90bebb10d7271d32c5557fcf371c93767decdf1f9f8fb5632d22e679f0071
MiddleFaceCell152.lamInterval	MiddleFaceCell152.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell152.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell152.lean:1892-1910	eb36a0c715aa59eaf2add9fbd1c23cc8978af55f1fabccab93dab08257ef5e
MiddleFaceCell153.Brick	MiddleFaceCell153.lean:20-22	1b372d246b1ec8a516e87941fe764d9027c5d8aa8731c9c7466bdf5fa6c99c7e
MiddleFaceCell153.brick	MiddleFaceCell153.lean:23-39	34cca59419dede788ed44d5f18c049ba33c05eb8193f2c1c26309a762802ccac
MiddleFaceCell153.lamInterval	MiddleFaceCell153.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell153.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell153.lean:1892-1910	028aa7a1e8a7043a0a9875776a74c9565e6f8b6c086c51cdf6d8c5f3a02622b8
MiddleFaceCell154.Brick	MiddleFaceCell154.lean:20-22	019b5e384b555dc6fdc2ea9210dda85105cd997c274c66864bdb7f65d97ad590
MiddleFaceCell154.brick	MiddleFaceCell154.lean:23-39	50e037c1361e8b98b2190c4cb74a93ed535c6c4632df3cc34e8a69d849cc6fef
MiddleFaceCell154.lamInterval	MiddleFaceCell154.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell154.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell154.lean:1892-1910	5c3c33c392db59a4dbd25f45ce48afb44aff17496ec33d42719eaa5165740983
MiddleFaceCell155.Brick	MiddleFaceCell155.lean:20-22	5eb8433f72def7696c21715fb4bddcb5436bf6740e36ded4599742bcd105f43
MiddleFaceCell155.brick	MiddleFaceCell155.lean:23-39	b2498e18c442cdedfdb3e1e5a2ce9515d035b5930a3bedb2b84f389be015c4b
MiddleFaceCell155.lamInterval	MiddleFaceCell155.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell155.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell155.lean:1892-1910	96e4d8ca1cc25d7c02936550ba9b0af6050acddbe055da78e6ec622d445090f1
MiddleFaceCell156.Brick	MiddleFaceCell156.lean:20-22	bc7932632cb06e7872a745a8bda2ffe4ff695439775f570f7930977d5f5f2770
MiddleFaceCell156.brick	MiddleFaceCell156.lean:23-39	2256baccd0a5c1c85c9d448ece0597699de49c994e7a1ef0558f1005589a482a

Declaration	Exact source	SHA-256
MiddleFaceCell56.lamInterval	MiddleFaceCell56.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell56.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell56.lean:1892-1910	9b6d7728072f7fdc6d0b4db27b0aca2a2dd c8e5fed3611a09507220660196fbc
MiddleFaceCell57.Brick	MiddleFaceCell57.lean:20-22	fa632f1bf02ed6f70b702ef7e883c71f7b3 090964739360f5665765c9c393075
MiddleFaceCell57.brick	MiddleFaceCell57.lean:23-39	2a522ec8f1a02d5846061b2bd89947208f4 d68af82c71b4dd024c5900c71341c
MiddleFaceCell57.lamInterval	MiddleFaceCell57.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell57.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell57.lean:1892-1910	376f02a02f821a6f6a31dc563c2e0f6fc2b beb4f8d828bd99d138b1b140fe303
MiddleFaceCell58.Brick	MiddleFaceCell58.lean:20-22	e0c6301d9398a187c1684d4ce31c7e2a663 acddf08456275a865b7b3102287c
MiddleFaceCell58.brick	MiddleFaceCell58.lean:23-39	08c3e019272f8327acc0e6150a36491eeb6 d3577997a3b6c5897a91f2d40ec40
MiddleFaceCell58.lamInterval	MiddleFaceCell58.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell58.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell58.lean:1892-1910	6ed293f5a6565943f6e0ddc509395dbc9f6 eadc61aa0d70336488a1457b3cf9f
MiddleFaceCell59.Brick	MiddleFaceCell59.lean:20-22	93366befc77fc4532e58c80c7c684528c95 703ed3969c09c3af97a3dd978fd8c
MiddleFaceCell59.brick	MiddleFaceCell59.lean:23-39	9af0f4fbddb434df63f8b118938422c0378 13b7992cc4efc53c4896ec73bb517
MiddleFaceCell59.lamInterval	MiddleFaceCell59.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell59.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell59.lean:1892-1910	ca09e9c4b920e14d9b0055a64681e496d51 8dc550f2e2ad38ffe3ce991c54c3e
MiddleFaceCell6.Brick	MiddleFaceCell6.lean:20-22	2c83651390c0d47851e2a70832b426de5fd cd21f0486225e842bd3aa3a8f99e4
MiddleFaceCell6.brick	MiddleFaceCell6.lean:23-39	c1935f798b415ad91f8996c1b753f4461b2 f3344ae0b8108a63450aa93488c9a
MiddleFaceCell6.lamInterval	MiddleFaceCell6.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell6.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell6.lean:1889-1907	8dd35623c9fc58a188684e0ffba352cc8f3 72e6f4e6dcb83b707ab38a32e3532
MiddleFaceCell60.Brick	MiddleFaceCell60.lean:20-22	57f78186ade7afa2a689f64476c530065a6 6da6a66a9ebc8ac590c571a57c353
MiddleFaceCell60.brick	MiddleFaceCell60.lean:23-39	49f55084cc0a3ef783c0d928e40479443be e26669f254368f995e998c8f2dcb4
MiddleFaceCell60.lamInterval	MiddleFaceCell60.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell60.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell60.lean:1892-1910	17b0c72d3f748f7ba74579f27875c404292 09b4533fc2be18bd07b085790e06f
MiddleFaceCell61.Brick	MiddleFaceCell61.lean:20-22	a21e3ead542cbf21c18e37ec15a35f5d1f4 312c14f7730d1274c3faa71190557
MiddleFaceCell61.brick	MiddleFaceCell61.lean:23-39	44a41ae2a19bf7d12322f907c746b316213 e2d91d3dbf772087644d10150f696
MiddleFaceCell61.lamInterval	MiddleFaceCell61.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell61.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell61.lean:1892-1910	59e39c5c4c9d75442163963805af636e4bf 2c0eeb8885a92e2856a1e5edcc105

Declaration	Exact source	SHA-256
MiddleFaceCell62.Brick	MiddleFaceCell62.lean:20-22	ae5b229b0d739a07b2a21fc80fbe78bbf956c5673e002614b179e9a787c2bb14
MiddleFaceCell62.brick	MiddleFaceCell62.lean:23-39	893eda77ff3ace51482a9c8ff9fe9ee7d2e2357abd1376f4cc06bf04130fd508
MiddleFaceCell62.lamInterval	MiddleFaceCell62.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell62.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell62.lean:1892-1910	311ecacd555e3dfd79b0cafadb2999b9e3863e61caac9a27d64b0cb18edc4428
MiddleFaceCell63.Brick	MiddleFaceCell63.lean:20-22	acc685650a48f4be5d2fda8f2f236ef5ec5eb2d7d69f84b899750d8bc9108219
MiddleFaceCell63.brick	MiddleFaceCell63.lean:23-39	8848cf2f984c8cb8295c39cca2de587f59c33c72704ae7495304cd6daff63571
MiddleFaceCell63.lamInterval	MiddleFaceCell63.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell63.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell63.lean:1892-1910	0bd6eff31944c4c935815b0ca4c5630cfa3d7968d8a445f43d01ba427a50d721
MiddleFaceCell64.Brick	MiddleFaceCell64.lean:20-22	6b3f307ed54f914c6ce2b1071f94dbf36a74b3ae65bcc82f18f6f8df3d4665bb
MiddleFaceCell64.brick	MiddleFaceCell64.lean:23-39	02964721a0844905e5a998ee8d930ea6a21b40404ef4869cc032803782a9aef8
MiddleFaceCell64.lamInterval	MiddleFaceCell64.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell64.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell64.lean:1892-1910	3a7ff9a8302c41ed4b3ac35692f785b3ba35492d2e906a1c8fc75a74fa4fa126
MiddleFaceCell65.Brick	MiddleFaceCell65.lean:20-22	f0d6c18d179e7246079064b43e887ec300f6e6e19ea96eb2ab89e446383d7d19
MiddleFaceCell65.brick	MiddleFaceCell65.lean:23-39	2c2d9eedbdc9e63cee8c8cbbd3dfbbec3307843aec5ad346c90615aaa0362a9
MiddleFaceCell65.lamInterval	MiddleFaceCell65.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell65.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell65.lean:1892-1910	3fbe8040d7a6f2069799ff79e7aa8e9ed2b76b5f190e6fad37dd0bedc5d3c898
MiddleFaceCell66.Brick	MiddleFaceCell66.lean:20-22	4e929fcb1272110a43d05e51a429753db9e2aa3a7afdde7c61b635f28f68ec70
MiddleFaceCell66.brick	MiddleFaceCell66.lean:23-39	7d0e3aa6d2b535f398d12fe40f99af27f146291a9a8697d042b4cf5aca0711fe
MiddleFaceCell66.lamInterval	MiddleFaceCell66.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell66.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell66.lean:1892-1910	59fef6b88141739652a2fd3810164fbc9627974e5678a392c22a281deee9ae11
MiddleFaceCell67.Brick	MiddleFaceCell67.lean:20-22	5c6c9bf60ced5754e62e13bec820feecf58c055e90cf8be3d49cb661580cc85
MiddleFaceCell67.brick	MiddleFaceCell67.lean:23-39	6dafdff0b9af15ad7508f097d54ca9c462578b548a10482aed6f2c99db3905a0
MiddleFaceCell67.lamInterval	MiddleFaceCell67.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell67.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell67.lean:1892-1910	2432d56a46c435b434411b9b9fc92429eee8a81ca54d36c3a8a74a8e594c52c11
MiddleFaceCell68.Brick	MiddleFaceCell68.lean:20-22	14d630bf52f4319721b460e79158d608626c195698693badec4784cb95033179
MiddleFaceCell68.brick	MiddleFaceCell68.lean:23-39	a156d2a7fa4a1efb1f5890c59ac72adabd452c668c3f04ffe075732812483d14

Declaration	Exact source	SHA-256
MiddleFaceCell68.lamInterval	MiddleFaceCell68.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell68.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell68.lean:1892-1910	1f91a9ddad52e966b8337d2f8e4fb7b264d403d2659d4249c1e2cb1e13b717f7
MiddleFaceCell69.Brick	MiddleFaceCell69.lean:20-22	f3d90eb5d1befd4b2921fdef318326e423d864868283d67cb14a7b9e35d762cf
MiddleFaceCell69.brick	MiddleFaceCell69.lean:23-39	fa73967f9b53a4fcddfcf8c2268107278daa93a7c35046c3233f8a8206f37c81
MiddleFaceCell69.lamInterval	MiddleFaceCell69.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell69.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell69.lean:1892-1910	2cfdbfd5c3a32b738a4e5a0b1496f80d9fc6fe995018441421b741712d35a87a
MiddleFaceCell7.Brick	MiddleFaceCell7.lean:20-22	a21b216b1cd9d16284f53df6bd94b490c6b6caa2a11b8d8b1d39855f250c9531
MiddleFaceCell7.brick	MiddleFaceCell7.lean:23-39	40b0173b10d63d32bd765f8e28397d57eaf473d9fd834431a37251824234526b
MiddleFaceCell7.lamInterval	MiddleFaceCell7.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell7.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell7.lean:1889-1907	4434b695c4fb5f10cd2218845feb07df257c58a0cd30c28943a08e7dbe523dd
MiddleFaceCell70.Brick	MiddleFaceCell70.lean:20-22	9690f836b7dd451b9296f7cfae7d07a4c0d5f470d494ae52aa661ff35a58d38a
MiddleFaceCell70.brick	MiddleFaceCell70.lean:23-39	61f4fd22eb5d96ff733096e5989f28bd4768b5c36e8970e2c3ed8d759f66a035
MiddleFaceCell70.lamInterval	MiddleFaceCell70.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell70.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell70.lean:1892-1910	2263b741b2069b51601b284e3ef952a054671cbb5daf224d60a7047cb3658482
MiddleFaceCell71.Brick	MiddleFaceCell71.lean:20-22	6437a07026d594aab523cf1d51f33441df96b6b9ac5af25132350846468eb767
MiddleFaceCell71.brick	MiddleFaceCell71.lean:23-39	072d9d8d624c2a2d5c698a163db512a8cb1ab5efd669c3258bc47ed5361244f4
MiddleFaceCell71.lamInterval	MiddleFaceCell71.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell71.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell71.lean:1892-1910	bf33b0a72a5fc4f7cc099260d839869a60a4466fd82f4ec39ef8a6d5421c416b
MiddleFaceCell72.Brick	MiddleFaceCell72.lean:20-22	d917658e3e305283f973bef5d7ccd340b32cf03033c32a776511e20bea10be
MiddleFaceCell72.brick	MiddleFaceCell72.lean:23-39	ce2e84ff21f8558f6dde915afaf766a80abf404ac1ae8271443493df29d82f4e
MiddleFaceCell72.lamInterval	MiddleFaceCell72.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell72.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell72.lean:1892-1910	a79d2bac93d9c1e432858003aafcd2a9e4c655dc43d3b89d177b016976e3f987
MiddleFaceCell73.Brick	MiddleFaceCell73.lean:20-22	1ce244173af7f7b3692a20829f115be2c11f338ece5a8cc01179b918837bf4dd
MiddleFaceCell73.brick	MiddleFaceCell73.lean:23-39	1ba7c7ae7d53474a5ca9b4b72790c16197bed85fb1d28b29e12352327f011d06
MiddleFaceCell73.lamInterval	MiddleFaceCell73.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell73.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell73.lean:1892-1910	3531ba96d91b6ff96ed27296ff32fa125c037ca13e6aa8779a4fd865a7f86ced

Declaration	Exact source	SHA-256
MiddleFaceCell174.Brick	MiddleFaceCell174.lean:20-22	d6e02804be6c550f98e48a2b10b3ce2c44b481d6ede506c43aee8937b229160c
MiddleFaceCell174.brick	MiddleFaceCell174.lean:23-39	672d336bf508cef45beea04d784a1adf379f226a79182d5fd8daba1d1583ac7a
MiddleFaceCell174.lamInterval	MiddleFaceCell174.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell174.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell174.lean:1892-1910	74c5223a1d7bd048fbc29e479d022778b1dd9b8a95c03bf8df544264c0dab7b
MiddleFaceCell175.Brick	MiddleFaceCell175.lean:20-22	6d7a8c3bd56af88c3ac137a2ca4a9a31a793c2a3ff02a19268f5b04d01143587
MiddleFaceCell175.brick	MiddleFaceCell175.lean:23-39	c9ea8c315e795c0f620936e446292bf333b5ba7c00231eb80f90376651edaa74
MiddleFaceCell175.lamInterval	MiddleFaceCell175.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell175.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell175.lean:1892-1910	572604102a270e038fbc33cc32bbd988228223b87181039ef8ff923d28a7da66
MiddleFaceCell176.Brick	MiddleFaceCell176.lean:20-22	de78088c38e40ee33ddea20bbdb174a823674170d4cde2dcd76193a582a617f
MiddleFaceCell176.brick	MiddleFaceCell176.lean:23-39	d9df6035cdf8745fe1d6b40482437dee6307251ce63011665e0ffbc90d91950f
MiddleFaceCell176.lamInterval	MiddleFaceCell176.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell176.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell176.lean:1892-1910	85936dbafb78201c9b4435a5a9629ffaab693545c045a4571ad4d22f3c1da36c
MiddleFaceCell177.Brick	MiddleFaceCell177.lean:20-22	ad2cff64744e9cc38316842db67072d50d3475f2ea8e0f85f790cc6310dca1a4
MiddleFaceCell177.brick	MiddleFaceCell177.lean:23-39	7af9c7d378463801e055d50ca1173ddfaabf781ab041d17473fc4151b411a2a7
MiddleFaceCell177.lamInterval	MiddleFaceCell177.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell177.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell177.lean:1892-1910	143e8c199e91c818b6dd41c233f831915991114c5e792db039d2c23e1510dfb7
MiddleFaceCell178.Brick	MiddleFaceCell178.lean:20-22	4a4361ebeb7f331a6d9e656d3b13bdfb8c1689bc47a03ab5d99a6667e66e8ef79
MiddleFaceCell178.brick	MiddleFaceCell178.lean:23-39	ba18ab673976f8dce8f1fa61f0aaae4d0c792585ac357cdfa229d8a026944118
MiddleFaceCell178.lamInterval	MiddleFaceCell178.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell178.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell178.lean:1892-1910	7d32b14f172a072c1f81e09b8198ce9ca054526eb85b7081f58a202d9a55d606
MiddleFaceCell179.Brick	MiddleFaceCell179.lean:20-22	e06715b99fec3ff6fd1569ad58cc754ed7dce78a4cd93407fefdf33c6ff66f694
MiddleFaceCell179.brick	MiddleFaceCell179.lean:23-39	05b3715d4d1e52a3d4b49c73018c3d6844baed464f967adc9b083fba5fe1247c
MiddleFaceCell179.lamInterval	MiddleFaceCell179.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell179.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell179.lean:1892-1910	6cc0c763d029a2411b2b4b4273e670060f1ccb0ccc619a39c0f872d8aea8139b
MiddleFaceCell18.Brick	MiddleFaceCell18.lean:20-22	c1bd829ce56335f7e21ed7dbedd56b9cd7ade514d87668b57269b6c3ff22304a
MiddleFaceCell18.brick	MiddleFaceCell18.lean:23-39	615e5073546f89e29cce77d7b7428fe8372b3b277471bbb077692c616fe40a95

Declaration	Exact source	SHA-256
MiddleFaceCell8.lamInterval	MiddleFaceCell8.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell8.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell8.lean:1889-1907	a46e2f11d26c8841fd9af98cef1778f3289 7543897e1ef81c0ac324e5675c610
MiddleFaceCell80.Brick	MiddleFaceCell80.lean:20-22	71e2309537720e847c9a6c72c4757a0261f 648fff89f6278c7e52fe1c21566aa
MiddleFaceCell80.brick	MiddleFaceCell80.lean:23-39	560385cca95d01232ddb9cd033540d339e4 8b4ca681fc658ed45cc4b5be90739
MiddleFaceCell80.lamInterval	MiddleFaceCell80.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell80.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell80.lean:1892-1910	1570f8f2e91871b795ab0901244f47648fd 0d28572228aa7fdd47f888a483125
MiddleFaceCell81.Brick	MiddleFaceCell81.lean:20-22	70d98ed149a553d35636fc79a3ebeb3cc8 2f6789828acc336eed4e42f995db2
MiddleFaceCell81.brick	MiddleFaceCell81.lean:23-39	2884468804590f9fe03bf7006b82261b45b 96c1a5c896b5a03917dc9a95b566d
MiddleFaceCell81.lamInterval	MiddleFaceCell81.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell81.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell81.lean:1892-1910	f5557f592400d59db127bc685a091e7ef1a 18482a26c8ec31f57726abc106f20
MiddleFaceCell82.Brick	MiddleFaceCell82.lean:20-22	08cc483ee9f34fd43de5983ec3e372ac8fe b4e39aceba7597dc67e47f129ee8e
MiddleFaceCell82.brick	MiddleFaceCell82.lean:23-39	de42acde7caba71c256dc8337d1165502ad 5b6b84439309329820bf7aa04d2e3
MiddleFaceCell82.lamInterval	MiddleFaceCell82.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell82.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell82.lean:1892-1910	10acd939e2fc5525f8337623d0f1bf2460e d4746e5756849c9fd336eabfa6329
MiddleFaceCell83.Brick	MiddleFaceCell83.lean:20-22	9650ea6442abf5e43c8ca08371f29586e25 d18368b53ca610562b57ba2c0f52e
MiddleFaceCell83.brick	MiddleFaceCell83.lean:23-39	62f8bbc3d45be2b13d1d935f7149bdc8380 5f74acf477c07dfb0b5e6838758f2
MiddleFaceCell83.lamInterval	MiddleFaceCell83.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell83.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell83.lean:1892-1910	666d81da2d0064ab8100666129b2ebea74f bd964cfd2fb5f9376f32a1048d99e
MiddleFaceCell84.Brick	MiddleFaceCell84.lean:20-22	d34e8c6620b44cbf04fbcd8b440d3b9ae90 a70cafe2ecbc7721da6e271a6e603
MiddleFaceCell84.brick	MiddleFaceCell84.lean:23-39	05af87b3578cb7840d1bd95184be863c619 a618db671ea9c370ceb2342f147e7
MiddleFaceCell84.lamInterval	MiddleFaceCell84.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell84.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell84.lean:1892-1910	ba359568852f017ef26961a4dd130650067 528029288a7ed1c08b8c3ba9386d4
MiddleFaceCell85.Brick	MiddleFaceCell85.lean:20-22	be024ce60ca137be0e5fdc9363fe54718e7 04c4eff09e8bae759ff882a7a15d1
MiddleFaceCell85.brick	MiddleFaceCell85.lean:23-39	6642369ddc479d24bfdd9974ef5360aa394 3fb1fbfe5891cfe2f20f9b208065eb
MiddleFaceCell85.lamInterval	MiddleFaceCell85.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell85.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell85.lean:1892-1910	a6247b6da305574c9dde7bdd9a1cbc5e48 a20e811c6305962499e42b7521679

Declaration	Exact source	SHA-256
MiddleFaceCell186.Brick	MiddleFaceCell186.lean:20-22	10ab4f94ecbef3c8c3567ffd6557093c16f5787dabaddf2935b5c822187668c2
MiddleFaceCell186.brick	MiddleFaceCell186.lean:23-39	97f3fa8d817bd40b1b0a001113cedda05d1caeee401240447c3cd8951274f496
MiddleFaceCell186.lamInterval	MiddleFaceCell186.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell186.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell186.lean:1892-1910	dcc22c8bd8491f3c0735b3ac99fee72450e415fd312e9c10faf89345a30c7751
MiddleFaceCell187.Brick	MiddleFaceCell187.lean:20-22	cbc129c276b8619e093d753c3d53b663219b8d3900469455b42811bbba02dad3
MiddleFaceCell187.brick	MiddleFaceCell187.lean:23-39	590c31ec2907d440625fd3defdc16af9124c517ca7b2138b01ef1c771042a2d3
MiddleFaceCell187.lamInterval	MiddleFaceCell187.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell187.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell187.lean:1892-1910	1983f6b5d120f11b9df46d4a05bebfef3acf07084eb9b691c349d23f756d82a8
MiddleFaceCell188.Brick	MiddleFaceCell188.lean:20-22	d756c619ba11b2fb174f79fe7c028e274d0503ddd7e8f5e036910e91ab7a074f
MiddleFaceCell188.brick	MiddleFaceCell188.lean:23-39	3bc79fef75c4cc2bfb54caadebfe4d0a6d32c32d710ab85135791650552c6b99
MiddleFaceCell188.lamInterval	MiddleFaceCell188.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell188.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell188.lean:1892-1910	ee0dc2ae864acfa75f53a16c34b57c7809fcf24f923f77009f6e9c6fcdc5a6b5
MiddleFaceCell189.Brick	MiddleFaceCell189.lean:20-22	9995d8f0735e89d4d22371dc5db0e6b45c79d84622112064caf49cdc9c5b9c8
MiddleFaceCell189.brick	MiddleFaceCell189.lean:23-39	7131e68fd89095f47f2f0651a01b47ee86608d86dac8b9478ca7dd379e7ef991
MiddleFaceCell189.lamInterval	MiddleFaceCell189.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell189.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell189.lean:1892-1910	c539115f908ea3c854f13e214af2c379cda c74ec4091c5662325c2a1f6bd89e1
MiddleFaceCell19.Brick	MiddleFaceCell19.lean:20-22	c90ecb97bcc2b0b1ece9bea1a892e190d84007307c237cb7b84d7ec3b0e6024
MiddleFaceCell19.brick	MiddleFaceCell19.lean:23-39	206210c28923a9253ad7d8345344ea0655b058110c31c53b5e5a6a5d8b053598
MiddleFaceCell19.lamInterval	MiddleFaceCell19.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell19.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell19.lean:1889-1907	7213ad2339ef4769866fbe9f0a71501c0da780c17ab8006f5a6c40b4447551d7
MiddleFaceCell190.Brick	MiddleFaceCell190.lean:20-22	2eb37468b09417053141932c23a09dab269faeccd192307cd73e740a7d43ef17
MiddleFaceCell190.brick	MiddleFaceCell190.lean:23-39	bd32ab36c2769524155ac509d94107635d10cba0428d34fecca4b171de849095
MiddleFaceCell190.lamInterval	MiddleFaceCell190.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell190.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell190.lean:1892-1910	dafff59d6235378af6b0880a575280e1069d461a36206afb1d1e26956655cc951
MiddleFaceCell191.Brick	MiddleFaceCell191.lean:20-22	34f1e9ad7cfd4b476da7cae8a83a0da382284c646ba2a2383b0f7fecfda5e4f7
MiddleFaceCell191.brick	MiddleFaceCell191.lean:23-39	380b05899d820354e486b3ad6b1ae2f996e222ba447488be0433f974c5b6fcc6

Declaration	Exact source	SHA-256
MiddleFaceCell191.lamInterval	MiddleFaceCell191.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell191.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell191.lean:1892-1910	325b487ffb7001998ca4bbd5c8bf369dadb e1c7c09bf25a0b629b46aeb7470f7
MiddleFaceCell192.Brick	MiddleFaceCell192.lean:20-22	c8f88e0ab8be959c8cab6ee6276482346f1 0092933a6346711dbadcea88fe1e4
MiddleFaceCell192.brick	MiddleFaceCell192.lean:23-39	14dc866f267ee2076b6a8a01f8d73b289d5 a6d681617580be877e76c2a69cfd5
MiddleFaceCell192.lamInterval	MiddleFaceCell192.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell192.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell192.lean:1892-1910	3ff009b02820a6a83bdccc24bf4132cae80 7fc772620b71528e22367e6b67133
MiddleFaceCell193.Brick	MiddleFaceCell193.lean:20-22	29ae0c7679f7842b43aa4f39871c3d6c0e4 b9568b52f48639e3cd76e786a9012
MiddleFaceCell193.brick	MiddleFaceCell193.lean:23-39	c7a8d47c0018dcbe08fb8f3444931f223d4 a38b493275594a3f606489af28d0d
MiddleFaceCell193.lamInterval	MiddleFaceCell193.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell193.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell193.lean:1892-1910	9dc1a3f1a04048f01e8fa76f7ae1f00a783 f959fac32923af716a43938dd5445
MiddleFaceCell194.Brick	MiddleFaceCell194.lean:20-22	54d0814c2f91e4d1b0530355ac75bf4a367 3e1cc78f850e9f036515e6a1f3d8e
MiddleFaceCell194.brick	MiddleFaceCell194.lean:23-39	7de96974533b4ef5b836ef752ed9cd536c 4ef41caf6b6c4180f38eee4a07e8e
MiddleFaceCell194.lamInterval	MiddleFaceCell194.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell194.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell194.lean:1892-1910	6b0ec5a16b25bf5b9133990825e7851d57d ad283c6c222688a828f74d3543221
MiddleFaceCell195.Brick	MiddleFaceCell195.lean:20-22	4f6254c1e4db993da672c61044c530a142a af24876f6ea438ea84b38faa5cf6e
MiddleFaceCell195.brick	MiddleFaceCell195.lean:23-39	7cb31dfff056fbeaffd924fd8063457668a 9747d3d11d80cbca01889a6677b16
MiddleFaceCell195.lamInterval	MiddleFaceCell195.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell195.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell195.lean:1892-1910	7684dcd90ea13b1172127269b6966efdbab3 65d74371f7e711935242634ffdb66
MiddleFaceCell196.Brick	MiddleFaceCell196.lean:20-22	3b9bdf29101617b1e9bf8633e4fef5397c4 e6d7dcb133f4405048fe4d9d1ee66
MiddleFaceCell196.brick	MiddleFaceCell196.lean:23-39	c45e2ed57bcd6d49ebd45c3d1c752697b3 37f0bfc1f270fbb3ef5f4075b06c3
MiddleFaceCell196.lamInterval	MiddleFaceCell196.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell196.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell196.lean:1892-1910	c1dabc8c1fc8310ad63535b0bfb9632f22b 6b0b2d012ae504038d0ba6ab3f3d0
MiddleFaceCell197.Brick	MiddleFaceCell197.lean:20-22	06ac3b95089ed503287244914cfd52c3298 3846693cf6dad05ea3fb3e37c6c2c
MiddleFaceCell197.brick	MiddleFaceCell197.lean:23-39	88f1cf09b3253d9a97de813dec4a629284b cbc83b60cb6ffe8d5b08399853fc
MiddleFaceCell197.lamInterval	MiddleFaceCell197.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780e dcfd66cb63df195880be1a2d957b6
MiddleFaceCell197.candidate_not_isWe ightedPerimeterMinimizer	MiddleFaceCell197.lean:1892-1910	37f1de9565ef09275e9666cfa7f7f137646 aab315b65b6ccfb9240f14e9e33f0

Declaration	Exact source	SHA-256
MiddleFaceCell198.Brick	MiddleFaceCell198.lean:20-22	1d489afd608a3fac58e5ca1d1e56b02b8581cf9957f1be0d06901a170a224cad
MiddleFaceCell198.brick	MiddleFaceCell198.lean:23-39	99e4e8b1cbde397c76a9f6c8a98f1b474d643ccf0c2e41c084f19b7050116bfa
MiddleFaceCell198.lamInterval	MiddleFaceCell198.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell198.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell198.lean:1892-1910	6ecc93ce4c3ed951951af788cf1a0886858fa768f5dd4cbc5b54051e12d31ad0
MiddleFaceCell199.Brick	MiddleFaceCell199.lean:20-22	69acff0df68b5b3f545e8743f26e335af570d16f7cb007df1bee399f705fa30a
MiddleFaceCell199.brick	MiddleFaceCell199.lean:23-39	42e1b8daf31e6b66339c1badb8b202b679036c5ef2ad3af3b48805d477ca6ed1
MiddleFaceCell199.lamInterval	MiddleFaceCell199.lean:79-80	f66ba63b466bb0baa42078e66a7ef00780edcfd66cb63df195880be1a2d957b6
MiddleFaceCell199.candidate_not_isWeightedPerimeterMinimizer	MiddleFaceCell199.lean:1892-1910	4be83c4c3331b21c491ec69bafa0f988c57b160f2eb2242b52dabb540a89611e

Representative full declarations

MiddleFaceCell10.brick

```
def brick : Brick where
  lamLo := 102001 / 100000
  lamHi := 102011 / 100000
  centers := ![1852465187781619081 / 10000000000000000000,
    2707826193809187847 / 10000000000000000000,
    2095335723171902431 / 5000000000000000000,
    4612314442061199433 / 10000000000000000000]
  slopes := ![438797283131686507 / 2000000000000000000,
    2506253839052346669 / 5000000000000000000,
    1234671831318607643 / 2000000000000000000,
    7505849097064381361 / 1000000000000000000]
  yXiSlope := 67505189 / 100000000
  vRhoSlope := 4481579 / 5000000
  radii := ![1 / 100000, 1 / 100000, 1 / 100000, 1 / 100000]
  lamOrdered := by norm_num
  radiiPos := by intro j; fin_cases j <=> norm_num
```

MiddleFaceCell10.candidate_not_isWeightedPerimeterMinimizer

```
theorem candidate_not_isWeightedPerimeterMinimizer
  {lam0 : ℝ} (hlam : lamInterval.RealContains lam0)
  (candidate : _root_.FourArcCandidate lam0)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  have ht0 : tInterval.RealContains (lam0 - (lamMid : ℝ)) :=
    lambda_iff_centered.mp hlam
  have ht : (MiddleFaceAssembly.tInterval brick).RealContains
    (lam0 - (MiddleFaceAssembly.lamMid brick : ℝ)) := by
    change tInterval.RealContains (lam0 - (lamMid : ℝ))
  exact ht0
  have hcenter :
    (MiddleFaceAssembly.lamMid brick : ℝ) +
    (lam0 - (MiddleFaceAssembly.lamMid brick : ℝ)) = lam0 := by
    ring
  exact
    MiddleFaceAssembly.Conditions.candidate_not_isWeightedPerimeterMinimizer
      checkedConditions ht hcenter candidate hcandidate

end MiddleFaceCell10
```


MiddleFaceCell112.brick

```
def brick : Brick where
  lamLo := 103121 / 100000
  lamHi := 25781 / 25000
  centers := ![2047981298329424359 / 1000000000000000000,
    3193002040019877679 / 1000000000000000000,
    2384408825489172341 / 500000000000000000,
    2662610780221764269 / 500000000000000000]
  slopes := ![1410220379874433567 / 1000000000000000000,
    3819081251769458811 / 1000000000000000000,
    68808292379469873 / 1562500000000000,
    1101262404840662309 / 200000000000000000]
  yXiSlope := 31412901 / 50000000
  vRhoSlope := 1369859 / 1562500
  radii := ![1 / 100000, 1 / 1000000, 1 / 100000, 1 / 1000000]
  lamOrdered := by norm_num
  radiiPos := by intro j; fin_cases j <=> norm_num
```

MiddleFaceCell112.candidate_not_isWeightedPerimeterMinimizer

```
theorem candidate_not_isWeightedPerimeterMinimizer
  {lam0 : ℝ} (hlam : lamInterval.RealContains lam0)
  (candidate : _root_.FourArcCandidate lam0)
  (hcandidate : candidate.SatisfiesCMVTypeIVHypotheses) :
  ¬ candidate.IsWeightedPerimeterMinimizer := by
  have ht0 : tInterval.RealContains (lam0 - (lamMid : ℝ)) :=
    lambda_iff_centered.mp hlam
  have ht : (MiddleFaceAssembly.tInterval brick).RealContains
    (lam0 - (MiddleFaceAssembly.lamMid brick : ℝ)) := by
    change tInterval.RealContains (lam0 - (lamMid : ℝ))
  exact ht0
  have hcenter :
    (MiddleFaceAssembly.lamMid brick : ℝ) +
    (lam0 - (MiddleFaceAssembly.lamMid brick : ℝ)) = lam0 := by
    ring
  exact
    MiddleFaceAssembly.Conditions.candidate_not_isWeightedPerimeterMinimizer
      checkedConditions ht hcenter candidate hcandidate
end MiddleFaceCell112
```

External references

- **CaneteMirandaVittone2010** — Antonio Cañete; Michele Miranda Jr.; Davide Vittone, *Some Isoperimetric Problems in Planes with Density*, Section 3.2; Propositions 3.5, 3.6, and 3.9; Lemma 3.8; Conjecture 3.12 and equations (26)–(27), printed manuscript p. 19; Theorem 3.16 and Remark 3.17, printed manuscript pp. 22–24, arXiv:0906.1256v1. Journal of Geometric Analysis 20 (2010), 243–290. Frozen local reference: Canete2010.pdf.. <https://arxiv.org/abs/0906.1256v1>